

AAX SDK

2.8.1

Generated by Doxygen 1.9.1

1 Main Page	1
1.1 Welcome to AAX	1
1.1.1 The Basics	1
1.1.2 More Topics	2
1.1.3 Test Tools & Utilities	2
1.1.4 Supplemental Information	2
1.2 SDK Folder Hierarchy	2
1.3 Contacting Avid	3
1.4 Licensing	3
2 Todo List	5
3 Host Compatibility Notes	9
4 Legacy Porting Notes	15
5 Deprecated List	19
6 Not Used by AAX Plug-Ins	21
7 Module Index	23
7.1 Manual	23
8 Namespace Index	25
8.1 Namespace List	25
9 Hierarchical Index	27
9.1 Class Hierarchy	27
10 Class Index	31
10.1 Class List	31
11 File Index	39
11.1 File List	39
12 Module Documentation	47
12.1 AAX SDK Manual	47
12.1.1	47
12.1.2 Welcome to AAX	47
12.1.2.1 The Basics	47
12.1.2.2 More Topics	48
12.1.2.3 Test Tools & Utilities	48
12.1.2.4 Supplemental Information	48
12.1.3 SDK Folder Hierarchy	48
12.1.4 Contacting Avid	49
12.1.5 Licensing	49

12.2 Getting Started with AAX	50
12.2.1 Contents	50
12.2.2 Welcome	50
12.2.3 Quick Start	51
12.2.4 AAX Design Overview	52
12.2.4.1 Architecture Philosophy	52
12.2.4.2 Design Attributes	52
12.2.4.3 Component Structure	52
12.2.4.4 Algorithm	53
12.2.4.5 Data Model	53
12.2.4.6 GUI Interface	53
12.2.4.7 Describe	54
12.2.4.8 Controller	54
12.2.5 DemoGain Example	54
12.2.5.1 AAX Plug-In Parameters	54
12.2.5.2 Data Model: Create Your Parameter	55
12.2.5.3 Algorithm: Add coefficients to the algorithm's context structure	55
12.2.5.4 Describe: Connect the parameter throughout the plug-in	57
12.2.5.5 GUI: Add a control	57
12.2.6 Next Steps	58
12.3 Core AAX Interface	58
12.3.1	58
12.4 Description callback	59
12.4.1	59
12.4.2 On this page	59
12.4.3 About the Describe callback and AAX descriptor interfaces	60
12.4.4 Top level: Collection	61
12.4.5 Middle level: Effects	62
12.4.5.1 Registering multiple Effects	62
12.4.6 Lowest level: Algorithm components	63
12.4.6.1 Algorithm callback properties	63
12.4.7 Host feature and environment checks	64
12.4.8 Checking Results	64
12.4.8.1 Summary	64
12.4.8.2 The Problem	64
12.4.8.3 The Solution	65
12.4.8.4 Handling Errors and Managing Control Flow	65
12.4.9 Describe Validation	66
12.4.9.1 Validation with DSH	66
12.4.9.2 Validation with Pro Tools	67
12.4.10 Additional Topics	67
12.4.11 Function Documentation	67

12.4.11.1 AAXRegisterPlugin()	68
12.4.11.2 GetEffectDescriptions()	68
12.5 Real-time algorithm callback	68
12.5.1 On this page	68
12.5.2 Algorithm definition	69
12.5.3 Algorithm memory management	69
12.5.4 Communicating with the algorithm	70
12.5.5 Algorithm initialization	70
12.5.5.1 Private data initialization	71
12.5.5.2 Optional initialization callback	71
12.5.6 Algorithm processing	71
12.5.7 Persistent algorithm memory	72
12.5.7.1 Private memory characteristics	72
12.5.7.2 Private data port registration	72
12.5.7.3 Private data initialization	72
12.5.7.4 Private data communication	72
12.5.8 Example algorithm callback	73
12.5.9 Port Types and Behavior	73
12.5.9.1 Standard message input	73
12.5.9.2 Internal state storage	74
12.5.9.3 Metering output	74
12.5.9.4 Environment variable retrieval	74
12.5.9.5 Other functionality enhancement	74
12.5.10 Additional Information	74
12.6 Data model interface	75
12.6.1	75
12.6.2 Related classes	76
12.7 Data model overrides interface	77
12.7.1	77
12.7.2 Function Documentation	77
12.7.2.1 GetNumberOfChanges()	77
12.8 GUI interface	78
12.8.1	78
12.9 AAX communication protocols	79
12.9.1 Communication with the C++ interface objects	79
12.9.1.1 Direct host communication	79
12.9.1.2 Custom data blocks	79
12.9.1.3 Notifications	80
12.9.1.4 Direct pointer sharing	80
12.9.2 Communication with the real-time algorithm	80
12.9.2.1 Data packets	80
12.9.2.2 Host-managed context fields	80

12.9.2.3 Direct data transfers	81
12.10 AAX Format Specification	81
12.10.1 .aaxplugin Directory Structure	81
12.10.2 Required Symbols	82
12.11 Additional AAX features	82
12.11.1	82
12.12 Auxiliary Output Stems	83
12.12.1 Overview of Auxiliary Output Stems in AAX	83
12.12.2 Implementing Auxiliary Output Stems	84
12.13 Background processing callback	84
12.13.1 On this page	84
12.13.2 Background thread description	84
12.13.3 Restrictions and limitations of background threads	85
12.13.4 Background thread performance characteristics on DSP systems	85
12.13.5 Background thread memory management	85
12.13.6 Additional information	85
12.14 Direct Memory Access	86
12.14.1 On this page	86
12.14.2 DMA facility overview	86
12.14.3 DMA transfer modes	86
12.14.4 Registering for DMA transfers	86
12.14.5 DMA restrictions	87
12.14.6 Additional information	87
12.15 Direct data access interface	87
12.15.1	87
12.15.2 Convenience class	88
12.15.3 Private data access interface	88
12.15.4 Communicating with other modules	88
12.16 EQ and Dynamics Curve Displays	89
12.16.1	89
12.16.2 Enumeration Type Documentation	91
12.16.2.1 AAX_ECureType	91
12.16.3 Function Documentation	92
12.16.3.1 GetCurveData()	92
12.16.3.2 GetCurveDataMeterIds()	93
12.16.3.3 GetCurveDataDisplayRange()	93
12.17 Hybrid Processing architecture	94
12.17.1	94
12.17.2 Overview of Hybrid	94
12.17.3 Implementing Hybrid processing	95
12.17.4 Additional information	95
12.17.4.1 Parameter update timing	95

12.17.4.2 Host support and alternatives	95
12.17.5 Function Documentation	96
12.17.5.1 RenderAudio_Hybrid()	96
12.17.5.2 GetHybridSignalLatency()	96
12.18 Plug-in meters	97
12.18.1 Overview of metering in AAX	97
12.18.2 Adding meters to an Effect	97
12.18.2.1 Customizing meter behavior	97
12.18.3 Reporting meter values	98
12.18.4 Displaying meter values	98
12.18.5 Alternatives	99
12.19 MIDI	99
12.19.1 MIDI Overview	99
12.19.2 MIDI node types	99
12.19.3 Adding MIDI functionality to a plug-in	99
12.19.4 Using MIDI in a plug-in algorithm	100
12.19.5 Accessing MIDI in the plug-in data model	101
12.19.6 Support functions	101
12.20 Offline processing interface	102
12.20.1	102
12.21 Properties File	102
12.22 Sidechain Inputs	103
12.22.1 Overview of Sidechain Inputs	103
12.22.2 Adding a Sidechain Input to an Effect	103
12.23 Task agent interface	104
12.23.1	104
12.23.2 Communicating with other modules	105
12.23.3 Enumeration Type Documentation	105
12.23.3.1 AAX_TaskCompletionStatus	105
12.24 AAX Library features	106
12.24.1	106
12.25 Parameter Manager	106
12.25.1	106
12.25.2 Parameter concepts	107
12.25.2.1 Parameter value domains	107
12.25.2.2 Taper	108
12.25.2.3 Delegates	108
12.25.2.4 Model-View-Controller	108
12.26 Taper delegates	109
12.26.1	109
12.27 Display delegates	109
12.27.1	109

12.27.2 Display delegate decorators	110
12.27.2.1 Display delegate decorator implementation	110
12.27.2.2 Decibel decorator example	110
12.28 Display delegate decorators	111
12.28.1	111
12.29 Monolithic parameters	112
12.29.1	112
12.30 Additional Topics	112
12.30.1	112
12.31 Real-time performance	112
12.31.1 Things NOT To Do In An Audio Plug-In Render Callback	113
12.31.2 Things To Do In An Audio Plug-In Render Callback	113
12.31.3 Good Resources And Examples	114
12.32 Parameter automation	114
12.32.1 On this page	114
12.32.2 Overview	114
12.32.3 Plug-in elements used for automation	115
12.32.3.1 Defining automatable parameters	115
12.32.4 Advanced automation topics	116
12.33 Parameter updates	116
12.33.1	116
12.34 Parameter update timing	116
12.34.1 On this page	116
12.34.2 Timeline Locations	117
12.34.3 Coordinating the data model and algorithm	117
12.34.3.1 A closer look at the AAX packet delivery system	117
12.34.4 Fixing timing issues due to shared data	118
12.34.4.1 Monolithic plug-ins	118
12.34.4.2 How to resolve timing errors	119
12.34.4.3 Additional considerations	120
12.34.5 Determining the absolute timestamp for a parameter update	121
12.34.5.1 Obtaining timeline information	121
12.34.5.2 Determining the timeline position of a parameter update	121
12.35 Token protocol	123
12.35.1 On this page	123
12.35.2 An Introduction to Tokens	123
12.35.2.1 Touch	124
12.35.2.2 Set	124
12.35.2.3 Update	125
12.35.3 Basic Token Operation	126
12.35.3.1 User Editing	126
12.35.3.2 Automation Playback	126

12.35.3.3 Chunk Restoring	127
12.36 Basic parameter update sequences	127
12.36.1 On this page	127
12.36.1.1 Notes on threading for these sequences	127
12.36.2 User-generated update	127
12.36.2.1 High-level interface calls and events	128
12.36.2.2 Detailed sequence for default implementation	128
12.36.2.3 Updates from control surfaces	129
12.36.3 Automation playback	130
12.36.4 Initialization	130
12.37 Linked parameters	131
12.37.1 On this page	131
12.37.2 Basics of Linked Parameters	131
12.37.2.1 Basic considerations for parameter linking	132
12.37.2.2 Defining proper linked parameter behavior	132
12.37.3 Linked Parameter Operation	134
12.37.3.1 User Editing	134
12.37.3.2 Automation Playback	135
12.37.3.3 Chunk Restoring	135
12.37.4 Changing Tapers	135
12.38 Linked parameter update sequences	136
12.38.1 On this page	136
12.38.1.1 Notes on threading for these sequences	136
12.38.2 User-generated update	137
12.38.2.1 High-level interface calls and events	138
12.38.2.2 Detailed interface calls and events	138
12.38.3 Update from automation playback	139
12.39 Plug-in type conversion	140
12.39.1 About this specification	140
12.39.2 Terminology	141
12.39.3 Scope of this specification	141
12.39.4 Topological constraints	142
12.39.5 Implicit conversions	142
12.39.6 Explicit conversions	143
12.39.7 Type deprecation	143
12.40 The Avid Component Framework (ACF)	144
12.40.1	144
12.40.2 More details	144
12.40.3 ACF interfaces in AAX	145
12.40.4 Using ACF interfaces	146
12.40.4.1 Host-provided interfaces	146
12.40.4.2 Plug-in interfaces	146

12.40.5 Interface versioning in AAX	147
12.41 ACF Elements	149
12.41.1	149
12.42 AAX Host Guides	150
12.42.1	150
12.43 Pro Tools Guide	150
12.43.1 Contents	150
12.43.2 About this document	151
12.43.3 Processing modes	151
12.43.3.1 Real-time processing	151
12.43.3.2 Non-real-time processing (AudioSuite)	152
12.43.3.3 Multichannel and Multi-Mono	152
12.43.4 Requirements for AAX plug-in compatibility with Pro Tools	153
12.43.4.1 Install directories	153
12.43.4.2 Plug-in name and file structure	153
12.43.4.3 Digital signature	153
12.43.5 Audio Engine Behavior and Features	155
12.43.5.1 Plug-in loading and AAE initialization	155
12.43.5.2 Plug-in initialization	156
12.43.5.3 Run-time processing behavior	157
12.43.6 Basic plug-in operation	158
12.43.6.1 Configuration management	158
12.43.6.2 Plug-in activation and deactivation	158
12.43.6.3 Plug-in bypass	158
12.43.6.4 Presets and settings management	159
12.43.6.5 Modifier key behavior	161
12.43.7 Optional plug-in features	161
12.43.7.1 Audio management features	161
12.43.7.2 Plug-in categories	164
12.43.7.3 Advanced non-real-time processing	164
12.43.7.4 Clip drag and drop	165
12.43.8 Using the Pro Tools Scripting SDK with AAX	166
12.43.9 Plug-ins with MIDI support	166
12.43.9.1 Instrument tracks	167
12.43.9.2 MIDI effects and MIDI insert chains	167
12.43.10 Debugging AAX plug-ins	167
12.43.10.1 Debugging within Pro Tools	167
12.43.10.2 DigiShell	168
12.43.10.3 DigiTrace	168
12.43.11 Troubleshooting common AAX plug-in failures	168
12.43.12 Using DigiOptions	169
12.43.12.1 Useful DigiOptions	169

12.43.13 Compatibility Notes	171
12.44 Media Composer Guide	171
12.44.1 Contents	171
12.44.2 About this document	172
12.44.3 Processing modes	172
12.44.3.1 Non-real-time processing (AudioSuite)	172
12.44.3.2 Real-time processing	174
12.44.4 Compatibility requirements	176
12.44.4.1 Install directories	176
12.44.4.2 Plug-in name and file structure	176
12.44.5 AAX feature support in Media Composer	176
12.44.5.1 Processing configurations	177
12.44.5.2 Preset management	177
12.44.5.3 Unsupported features	179
12.44.5.4 Additional feature support notes	179
12.44.6 Additional Information	179
12.44.6.1 Audio Engine features and behavior	179
12.44.6.2 Debugging AAX plug-ins in Media Composer	180
12.45 HDX DSP Guide	180
12.45.1 Contents	180
12.45.2 Overview of TI DSP Algorithms in AAX	180
12.45.3 Getting Started with HDX DSP	181
12.45.4 The HDX DSP Platform	181
12.45.4.1 DSP characteristics: instruction processing	181
12.45.4.2 DSP characteristics: audio buffers	181
12.45.4.3 DSP characteristics: memory	182
12.45.4.4 System characteristics: DSP/host data transfers	182
12.45.4.5 TI Shell characteristics: Memory allocation	183
12.45.4.6 TI Shell characteristics: Data packet services	184
12.45.4.7 TI Shell characteristics: Instance allocation	185
12.45.4.8 Additional TI Shell services	186
12.45.5 Requirements for HDX DSP Plug-Ins	186
12.45.5.1 Plug-in description	186
12.45.5.2 Performance measurement and reporting	187
12.45.5.3 Plug-in compilation and packaging	188
12.45.6 TI Development Tools	189
12.45.6.1 Code Composer Studio	189
12.45.6.2 The TMS320C6000 C++ compiler	193
12.45.6.3 DigiShell test tool (DSH)	194
12.45.6.4 Hardware Debugging	195
12.45.6.5 Tracing	197
12.45.6.6 Testing in Pro Tools	198

12.45.7 Common Issues with TI Development	198
12.45.7.1 Data structure compatibility	198
12.45.8 TI Optimization Guide	202
12.45.8.1 Optimization quick start	202
12.45.8.2 Compiler and linker options	203
12.45.8.3 The load-update-store pattern	205
12.45.8.4 Case study: IIR filter implementation on TI 672x DSPs	206
12.45.8.5 Understanding CGTools-generated ASM files	207
12.45.8.6 C keywords	209
12.45.8.7 Data types	211
12.45.8.8 Case study: Efficient parameter smoothing at single and double precision	213
12.45.8.9 Refactoring conditionals and branches	214
12.45.8.10 Case study: pipeline refactoring in Avid's EQ3 and Dyn3 plug-ins	216
12.45.8.11 Case study: Additional optimization lessons from EQ3 and Dyn3	218
12.45.8.12 Optimization on the HDX platform	220
12.45.8.13 Code Composer Studio optimization tools	221
12.45.9 Error Codes	221
12.45.9.1 -138xx: DHM Core DSP errors	221
12.45.9.2 -140xx: AAX Host errors	222
12.45.9.3 -141xx: TI System errors	222
12.45.9.4 -142xx: DIDL errors	223
12.45.9.5 -144xx: HDX hardware errors	224
12.45.9.6 -145xx: DHM isochronous audio engine errors	224
12.45.9.7 -30xxx: Dynamically-generated error codes	225
12.46 Page Table Guide	225
12.46.1 Contents	225
12.46.2 Introduction	226
12.46.2.1 Control Surfaces Overview	226
12.46.2.2 Page Tables Overview	226
12.46.3 Avid Control Surfaces	226
12.46.3.1 EUCON	226
12.46.3.2 VENUE	227
12.46.4 Plug-In Page Table Guidelines	227
12.46.4.1 General Guidelines	228
12.46.5 Avid Center Section Page Tables	228
12.46.5.1 Center Section Page Table Guidelines	229
12.46.5.2 Center Section Parameter Mapping to Single-Column/Row Layouts	233
12.46.5.3 Center Section Parameter Mapping in S6 Expand Mode	244
12.46.5.4 Center Section Parameter Mapping on VENUE S3L-X	244
12.46.6 EUCON Page Tables	245
12.46.6.1 Specification	245
12.46.6.2 Types	245

12.46.6.3 Conventions	246
12.46.6.4 Requirements	246
12.46.7 Implementing Page Tables	246
12.46.7.1 Page table XML specification	246
12.46.7.2 Parameter identifiers	249
12.46.7.3 Creating page tables using the AAX Plug-In Page Table Editor	250
12.46.7.4 Verifying Page Table Layouts: The Hidden Pop-Up Menu	251
12.46.7.5 Control Highlighting Scheme	252
12.46.7.6 Control Numbering Layouts	252
12.46.7.7 Alphanumeric Displays	253
12.46.7.8 ProControl Display	254
12.46.8 Appendix A. Get Parameter Value Info	255
12.46.8.1 Overview	255
12.46.8.2 Implementation	255
12.47 DigiTrace Guide	257
12.47.1 On this page	257
12.47.2 What is DigiTrace?	258
12.47.2.1 What does DigiTrace do?	258
12.47.3 DigiTrace quick start guide	258
12.47.3.1 Find and decrypt DigiTrace log files	259
12.47.3.2 Configure DigiTrace for AAX plug-in logging	259
12.47.3.3 Configure DigiTrace for plain-text output	259
12.47.3.4 Add tracing to a plug-in	260
12.47.4 DigiTrace log files	260
12.47.4.1 Where are DigiTrace log files stored?	260
12.47.4.2 Monitoring DigiTrace logs	260
12.47.4.3 Log file formatting	261
12.47.5 Configuring DigiTrace	261
12.47.5.1 Trace facilities	262
12.47.5.2 Trace priorities	262
12.47.5.3 Useful DigiTrace facilities	262
12.47.6 Bonus features	263
12.47.6.1 Real-time AAE performance logging with DigiTrace	263
12.47.6.2 Adding signposts to the DigiTrace log at run-time	264
12.47.7 Adding traces to an AAX plug-in	264
12.47.7.1 Basic AAX logging	264
12.47.7.2 Advanced DigiTrace logging features	264
12.47.7.3 Security concerns	266
12.47.8 Advanced DigiTrace configuration	266
12.47.8.1 Configuration command format	266
12.47.8.2 Advanced configuration commands	267
12.47.8.3 Dynamically changing the DigiTrace configuration	268

12.47.9 Compatibility	268
12.47.10 Additional Information	268
12.47.10.1 Confidentiality	268
12.48 DSH Guide	268
12.48.1 Contents	269
12.48.2 What is DSH and how it works	269
12.48.3 Basic set of commands of the DAE dish	269
12.48.3.1 Loading plug-ins in DSH	270
12.48.3.2 Working with HDX card from DSH	271
12.48.3.3 DAE dish tips	271
12.48.4 Basic plug-in tests	271
12.48.4.1 Cycle count performance test	271
12.48.4.2 Cancellation test	273
12.48.5 Debugging and tracing in DSH	274
12.48.6 Scripting interface and batch profiling	274
12.49 DTT Guide	274
12.49.1 Contents	275
12.49.2 What is DTT	275
12.49.3 How to run tests and suites in DTT	275
12.49.4 Writing DTT scripts	276
12.49.4.1 Describing and using input arguments of the script	276
12.49.4.2 Writing body of the script	276
12.49.5 Logging in DTT and debugging DTT scripts	277
12.49.5.1 Interactive mode	277
12.49.6 Working with DTT test suites	277
12.49.6.1 Autogeneration of the suites	278
12.50 Extensions	279
12.50.1	279
12.51 GUI Extensions	279
12.51.1 About the SDK's GUI Extensions	279
12.51.2 Notes	279
12.52 Monolithic VIs and Effects	280
12.53 Other Extensions	280
12.53.1	280
12.53.2 Function Documentation	280
12.53.2.1 AsStringMIDIStream_Debug()	281
12.53.2.2 GetPathToPlugInBundle()	281
12.54 Supplemental Information	281
12.54.1	281
12.55 Troubleshooting	282
12.55.1 Contents	282
12.55.2 Plug-In Fails to Load in Shipping Pro Tools	282

12.55.3 Plug-In Causes Audio Streaming Errors	283
12.56 Distributing Your AAX Plug-In	285
12.56.1 Contents	285
12.56.2 The finishing touches	285
12.56.2.1 Check and finalize page tables	286
12.56.2.2 Create factory presets	286
12.56.2.3 Sign your plug-in	287
12.56.3 Building your plug-in installer	287
12.56.3.1 Installing Track Presets	287
12.56.4 Testing your plug-in	288
12.56.5 Selling your plug-in	288
12.56.5.1 Avid Marketplace	288
12.56.5.2 In-App Purchase	289
12.57 AAX Interfaces	289
12.57.0.1 Interfaces Implemented by the AAX Host	289
12.57.0.2 Interfaces Implemented by the AAX Plug-In	290
12.57.0.3 Interfaces internal to the AAX SDK	290
12.58 Host Support	291
12.58.1 Host Support	291
12.58.1.1 Platform Support	291
12.58.1.2 Describe Interfaces	291
12.58.1.3 Run-Time Interfaces	292
12.58.1.4 Features	292
12.58.2 Host Compatibility Notes	293
12.59 Known Issues	297
12.59.1 Contents	297
12.59.2 Known Issues in the AAX SDK	297
12.59.2.1 AAXSDK-1052	297
12.59.2.2 AAXSDK-1036	297
12.59.2.3 AAXSDK-897	298
12.59.2.4 AAXSDK-851	298
12.59.2.5 AAXSDK-832	298
12.59.2.6 AAXSDK-708	298
12.59.2.7 AAXSDK-705	298
12.59.2.8 AAXSDK-663	298
12.59.2.9 AAXSDK-599	299
12.59.2.10 AAX-578 / PTSW-158310	299
12.59.2.11 AAX-585 / PTSW-157451	299
12.59.2.12 AAX-582 / PTSW-157726	299
12.59.2.13 AAXSDK-561 / PT-232159	299
12.59.2.14 AAXSDK-533	299
12.59.2.15 AAXSDK-514	299

12.59.2.16 AAXSDK-321	300
12.59.2.17 AAXSDK-271	300
12.59.2.18 AAXSDK-186	300
12.59.2.19 AAXSDK-162	300
12.59.2.20 AAXSDK-16	300
12.59.2.21 AAXSDK-14	301
12.59.2.22 AAXSDK-13 / AAX-579 / PTSW-158381	301
12.59.2.23 AAXSDK-11 / AAX-581 / PTSW-158348	301
12.59.2.24 AAXSDK-10 / AAX-580 / PTSW-154083	301
12.59.2.25 AAXSDK-6 / AAX-646	301
12.59.2.26 AAXSDK-5	302
12.59.2.27 AAXSDK-2 / AAX-648	302
12.59.3 Known Issues in Pro Tools	302
12.59.3.1 PT-325781	302
12.59.3.2 PT-323936	302
12.59.3.3 PT-322604	302
12.59.3.4 PT-322526	303
12.59.3.5 PT-317648	303
12.59.3.6 PT-316101	303
12.59.3.7 PT-307986	303
12.59.3.8 PT-307746	303
12.59.3.9 PT-307193	303
12.59.3.10 PT-306863	303
12.59.3.11 PT-305352	304
12.59.3.12 PT-303482	304
12.59.3.13 PT-299906	304
12.59.3.14 PT-297802	304
12.59.3.15 PT-290588	304
12.59.3.16 PT-284916	304
12.59.3.17 PT-282946	304
12.59.3.18 PT-278282	305
12.59.3.19 PT-276280	305
12.59.3.20 PT-274717	305
12.59.3.21 PT-271830	305
12.59.3.22 PT-265539	305
12.59.3.23 PT-263909	305
12.59.3.24 PT-263859	305
12.59.3.25 PT-261394	306
12.59.3.26 PT-258560 / PT-256919	306
12.59.3.27 PT-258394	306
12.59.3.28 PT-257213	306
12.59.3.29 PT-256704	306

12.59.3.30 PT-255800	306
12.59.3.31 PT-255408	306
12.59.3.32 PT-254203	307
12.59.3.33 PT-254118 / PT-275441 / PT-279941	307
12.59.3.34 PT-254103	307
12.59.3.35 PT-250751	307
12.59.3.36 PT-249791	307
12.59.3.37 PT-249790	307
12.59.3.38 PT-248000	307
12.59.3.39 PT-245693 / PT-200756	308
12.59.3.40 PT-243211	308
12.59.3.41 PT-237857	308
12.59.3.42 PT-236755	308
12.59.3.43 PT-235831	308
12.59.3.44 PT-235333	308
12.59.3.45 PT-234681	308
12.59.3.46 PT-233726	309
12.59.3.47 PT-233176	309
12.59.3.48 PT-232678 / PT-236755	309
12.59.3.49 PT-232403	309
12.59.3.50 PT-232159	309
12.59.3.51 PT-230327	309
12.59.3.52 PT-230290	309
12.59.3.53 PT-230288	310
12.59.3.54 PT-229026	310
12.59.3.55 PT-227655	310
12.59.3.56 PT-227173	310
12.59.3.57 PT-226959	310
12.59.3.58 PT-226559	310
12.59.3.59 PT-225763	310
12.59.3.60 PT-225637	311
12.59.3.61 PT-223581	311
12.59.3.62 PT-218545	311
12.59.3.63 PT-218486	311
12.59.3.64 PT-210904 / VSW-14216	311
12.59.3.65 PT-206995	311
12.59.3.66 PT-206541	312
12.59.3.67 PT-206161	312
12.59.3.68 PT-205610	312
12.59.3.69 PT-203420	312
12.59.3.70 PT-202345	312
12.59.3.71 PTSW-200437 / PTSW-197598	312

12.59.3.72 PTSW-197651 / PT-218405	312
12.59.3.73 PTSW-197601 / PT-218459	313
12.59.3.74 PTSW-197593 / PT-218480	313
12.59.3.75 PTSW-197540	313
12.59.3.76 PTSW-197472	313
12.59.3.77 PTSW-197471	313
12.59.3.78 PTSW-197468 / PT-218460	313
12.59.3.79 PTSW-197431 / PT-218414	313
12.59.3.80 PTSW-197075	314
12.59.3.81 PTSW-196772 / PT-218423	314
12.59.3.82 PTSW-196604	314
12.59.3.83 PTSW-196428 / PT-218488	314
12.59.3.84 PTSW-195316 / PT-218485	314
12.59.3.85 PTSW-195257	314
12.59.3.86 PTSW-195256 / PT-218429	314
12.59.3.87 PTSW-195209 / PT-218474	315
12.59.3.88 PTSW-195113	315
12.59.3.89 PTSW-194698 / PT-218478	315
12.59.3.90 PTSW-194231 / PT-218434	315
12.59.3.91 PTSW-193646	315
12.59.3.92 PTSW-193400	315
12.59.3.93 PTSW-193345	316
12.59.3.94 PTSW-193339	316
12.59.3.95 PTSW-193051	316
12.59.3.96 PTSW-192863 / PT-218498	316
12.59.3.97 PTSW-192755	316
12.59.3.98 PTSW-192720 / PT-218467	316
12.59.3.99 PTSW-192635	317
12.59.3.100 PTSW-192456 / PT-218490	317
12.59.3.101 PTSW-192251 / PT-218394	317
12.59.3.102 PTSW-192086 / PT-218465	317
12.59.3.103 PTSW-191875	317
12.59.3.104 PTSW-191446 / PT-218600	317
12.59.3.105 PTSW-191317 / PT-218425	317
12.59.3.106 PTSW-191139	318
12.59.3.107 PTSW-190722	318
12.59.3.108 PTSW-190719	318
12.59.3.109 PTSW-190340	318
12.59.3.110 PTSW-189928 / PT-218456	318
12.59.3.111 PTSW-189738 / PT-218494	318
12.59.3.112 PTSW-189725 / PT-218397	318
12.59.3.113 PTSW-189439 / PT-218427	319

12.59.3.114 PTSW-189279	319
12.59.3.115 PTSW-188836 / PT-218428	319
12.59.3.116 PTSW-188830	319
12.59.3.117 PTSW-188653 / PT-218451	319
12.59.3.118 PTSW-188161	319
12.59.3.119 PTSW-187670	319
12.59.3.120 PTSW-187220 / PT-218584	320
12.59.3.121 PTSW-187216 / PT-218491	320
12.59.3.122 PTSW-187159	320
12.59.3.123 PTSW-187066 / PT-218391	320
12.59.3.124 PTSW-186864	320
12.59.3.125 PTSW-186725	320
12.59.3.126 PTSW-186627	320
12.59.3.127 PTSW-186253	321
12.59.3.128 PTSW-186189	321
12.59.3.129 PTSW-186182	321
12.59.3.130 PTSW-185868 / PT-218439	321
12.59.3.131 PTSW-185867 / PT-218470	321
12.59.3.132 PTSW-185866	321
12.59.3.133 PTSW-185825 / PT-218464	321
12.59.3.134 PTSW-185537	322
12.59.3.135 PTSW-185484	322
12.59.3.136 PTSW-185483	322
12.59.3.137 PTSW-185462	322
12.59.3.138 PTSW-185343	322
12.59.3.139 PTSW-185341	322
12.59.3.140 PTSW-184777 / PT-218483	322
12.59.3.141 PTSW-184770	323
12.59.3.142 PTSW-184682	323
12.59.3.143 PTSW-184642 / PT-218627	323
12.59.3.144 PTSW-184619 / PT-218473 / AAX-600	323
12.59.3.145 PTSW-184541	323
12.59.3.146 PTSW-183902 / PT-218479	323
12.59.3.147 PTSW-183848 / PT-218390	323
12.59.3.148 PTSW-183841	324
12.59.3.149 PTSW-183731	324
12.59.3.150 PTSW-183708	324
12.59.3.151 PTSW-168222	324
12.59.3.152 PTSW-165992	324
12.59.3.153 PTSW-163739	324
12.59.3.154 PTSW-161674	325
12.59.3.155 PTSW-160778	325

12.59.3.156 PTSW-160620	325
12.59.3.157 PTSW-159702	325
12.59.3.158 PTSW-159700	325
12.59.3.159 PTSW-159524	325
12.59.3.160 PTSW-158119	325
12.59.3.161 PTSW-157745	326
12.59.3.162 PTSW-157518	326
12.59.3.163 PTSW-157012	326
12.59.3.164 PTSW-156310	326
12.59.3.165 PTSW-156286	326
12.59.3.166 PTSW-156216	326
12.59.3.167 PTSW-156195	326
12.59.3.168 PTSW-156035	327
12.59.3.169 PTSW-155300 / PT-218458	327
12.59.3.170 PTSW-155177	327
12.59.3.171 PTSW-154361	327
12.59.3.172 PTSW-153140	327
12.59.3.173 PTSW-150047	327
12.59.3.174 PTSW-149880	328
12.59.3.175 PTSW-149819	328
12.59.3.176 PTSW-135536 / PT-218412	328
12.59.3.177 PTSW-3020 / PT-218463	328
12.59.3.178 AAX-686	328
12.59.3.179 AAX-583 / PTSW-157743	328
12.59.4 Known Issues in Venue Live Sound Systems	329
12.59.4.1 VSW-13857	329
12.59.4.2 VSW-13292	329
12.59.4.3 Other Known Issues	329
12.59.5 Known Issues in Media Composer	329
12.59.5.1 MCDEV-2904	329
12.59.6 Known Issues in Control Surfaces	330
12.59.6.1 PT-285383	330
12.59.6.2 PT-226228	330
12.59.6.3 PT-226227	330
12.59.6.4 GWSW-16656	330
12.59.6.5 GWSW-8470	330
12.59.6.6 GWSW-6694	330
12.59.7 Known Issues in Other Software	330
12.59.7.1 XPACE-23	330
12.59.8 Known Issues in AAX Tools	331
12.59.8.1 AAXTOOL-1344	331
12.59.8.2 PT-218597	331

12.59.8.3 Additional Information	331
12.60 Change Log	331
12.60.1 Change Log	331
12.60.1.1 AAX SDK 2.8.1	331
12.60.1.2 AAX SDK 2.8.0	332
12.60.1.3 AAX SDK 2.7.0	332
12.60.1.4 AAX SDK 2.6.1	333
12.60.1.5 AAX SDK 2.6.0	334
12.60.1.6 AAX SDK 2.5.1	334
12.60.1.7 AAX SDK 2.5.0	335
12.60.1.8 AAX SDK 2.4.1	335
12.60.1.9 AAX SDK 2.4.0	336
12.60.1.10 AAX SDK 2.3.2	337
12.60.1.11 AAX SDK 2.3.1	338
12.60.1.12 AAX SDK 2.3.0	339
12.60.1.13 AAX SDK 2.2.2	341
12.60.1.14 AAX SDK 2.2.1	342
12.60.1.15 AAX SDK 2.2.0	343
12.60.1.16 AAX SDK 2.1.1	345
12.60.1.17 AAX SDK 2.1.0	346
12.60.1.18 AAX SDK 2.0.1	347
12.60.1.19 AAX SDK 2.0.0	347
12.60.1.20 AAX SDK 1.5.0	348
12.60.1.21 AAX SDK 1.0.6	348
12.60.1.22 AAX SDK 1.0.5	348
12.60.1.23 AAX SDK 1.0.4	349
12.60.1.24 AAX SDK 1.0.3	350
12.60.1.25 AAX SDK 1.0.2	351
12.61 Example Plug-Ins	351
12.61.1 SDK Example plug-ins	351
12.61.1.1 Basic examples	351
12.61.1.2 Feature examples	352
12.61.1.3 Deprecated Examples	354
12.62 VENUE Guide	354
12.62.1 Contents	354
12.62.2 About this document	355
12.62.3 Overview of VENUE	355
12.62.4 VENUE systems	355
12.62.4.1 VENUE S6L	355
12.62.4.2 VENUE S3L-X	356
12.62.5 Host environment	356
12.62.5.1 Audio engine	356

12.62.5.2 Available DSP resources	357
12.62.5.3 Operating system	357
12.62.5.4 Display	357
12.62.5.5 Page tables	358
12.62.5.6 Network communications	358
12.62.5.7 Host environment summary	358
12.62.6 AAX feature support and compatibility	359
12.62.6.1 Processing configurations	359
12.62.6.2 Presets and automation	359
12.62.6.3 Unsupported features	360
12.62.7 VENUE Plug-in installer specification	360
12.62.7.1 Overview	361
12.62.7.2 Directory structure	361
12.62.7.3 Optional installer files	362
12.62.7.4 Using a VENUE plug-in installer	364
12.62.8 Additional plug-in guidelines	364
12.62.8.1 General Reliability and Fault Tolerance	364
12.62.8.2 Plug-In Dialogs	364
12.62.8.3 Online Help	364
12.62.9 System details	365
12.62.9.1 External dependencies	365
12.62.9.2 Environment variables	365
12.62.9.3 Plug-in file locations	366
12.62.9.4 Installation process	367
12.62.10 Additional Information	369
12.62.10.1 Metering	369
13 Namespace Documentation	371
13.1 AAX Namespace Reference	371
13.1.1 Enumeration Type Documentation	375
13.1.1.1 EStatusNibble	375
13.1.1.2 EStatusByte	375
13.1.1.3 EChannelModeData	376
13.1.1.4 ESpecialData	376
13.1.1.5 ESAMPLERates	377
13.1.2 Function Documentation	377
13.1.2.1 AsString() [1/3]	377
13.1.2.2 AsString() [2/3]	377
13.1.2.3 AsString() [3/3]	377
13.1.2.4 IsNoteOn()	378
13.1.2.5 IsNoteOff()	378
13.1.2.6 IsAllNotesOff()	378

13.1.2.7 IsAccentedClick()	378
13.1.2.8 IsUnaccentedClick()	379
13.1.2.9 IsClick()	379
13.1.2.10 PageTableParameterMappingsAreEqual()	379
13.1.2.11 PageTableParameterNameVariationsAreEqual()	379
13.1.2.12 PageTablesAreEqual()	380
13.1.2.13 CopyPageTable()	380
13.1.2.14 FindParameterMappingsInPageTable()	380
13.1.2.15 ClearMappedParameterByID()	381
13.1.2.16 GetCStringOfLength()	381
13.1.2.17 Caseless_strcmp()	381
13.1.2.18 Binary2String()	381
13.1.2.19 String2Binary()	381
13.1.2.20 IsASCII()	382
13.1.2.21 IsFourCharASCII()	382
13.1.2.22 AsStringFourChar()	382
13.1.2.23 AsStringPropertyValue()	382
13.1.2.24 AsStringInt32()	382
13.1.2.25 AsStringUInt32()	383
13.1.2.26 AsStringIDTriad()	383
13.1.2.27 AsStringStemFormat()	383
13.1.2.28 AsStringStemChannel()	383
13.1.2.29 AsStringResult()	384
13.1.2.30 AsStringSupportLevel()	384
13.1.2.31 SafeLog()	384
13.1.2.32 SafeLogf()	385
13.1.2.33 IsParameterIDEqual()	385
13.1.2.34 IsEffectIDEqual()	385
13.1.2.35 IsAvidNotification()	385
13.1.2.36 alignFree()	385
13.1.2.37 alignMalloc()	386
13.1.2.38 DeDenormal() [1/2]	386
13.1.2.39 DeDenormal() [2/2]	386
13.1.2.40 DeDenormalFine()	386
13.1.2.41 FilterDenormals()	386
13.1.2.42 ClampToZero()	387
13.1.2.43 ZeroMemorySW()	387
13.1.2.44 ZeroMemoryDW()	387
13.1.2.45 Fill() [1/3]	387
13.1.2.46 Fill() [2/3]	388
13.1.2.47 Fill() [3/3]	388
13.1.2.48 fabs() [1/2]	388

13.1.2.49 fabs() [2/2]	388
13.1.2.50 fabsf()	388
13.1.2.51 AbsMax()	389
13.1.2.52 MinMax()	389
13.1.2.53 Max()	389
13.1.2.54 Min()	389
13.1.2.55 Sign()	389
13.1.2.56 PolyEval()	390
13.1.2.57 CeilLog2()	390
13.1.2.58 SinCosMix()	390
13.1.2.59 FastRound2Int32() [1/2]	390
13.1.2.60 FastRound2Int32() [2/2]	390
13.1.2.61 FastRndDbl2Int32()	391
13.1.2.62 FastTrunc2Int32() [1/2]	391
13.1.2.63 FastTrunc2Int32() [2/2]	391
13.1.2.64 FastRound2Int64()	392
13.1.2.65 GetInt32RPDF()	392
13.1.2.66 GetFastInt32RPDF()	392
13.1.2.67 GetRPDFWithAmplitudeOneHalf()	393
13.1.2.68 GetRPDFWithAmplitudeOne()	393
13.1.2.69 GetFastRPDFWithAmplitudeOne()	393
13.1.2.70 GetTPDFWithAmplitudeOne()	393
13.1.3 Variable Documentation	393
13.1.3.1 cBigEndian	393
13.1.3.2 cLittleEndian	394
13.1.3.3 cPi	394
13.1.3.4 cTwoPi	394
13.1.3.5 cHalfPi	394
13.1.3.6 cQuarterPi	394
13.1.3.7 cRootTwo	394
13.1.3.8 cOneOverRootTwo	394
13.1.3.9 cPos3dB	395
13.1.3.10 cNeg3dB	395
13.1.3.11 cPos6dB	395
13.1.3.12 cNeg6dB	395
13.1.3.13 cNormalizeLongToAmplitudeOneHalf	395
13.1.3.14 cNormalizeLongToAmplitudeOne	395
13.1.3.15 cMilli	395
13.1.3.16 cMicro	396
13.1.3.17 cNano	396
13.1.3.18 cPico	396
13.1.3.19 cKilo	396

13.1.3.20 cMega	396
13.1.3.21 cGiga	396
13.1.3.22 cDenormalAvoidanceOffset	396
13.1.3.23 cFloatDenormalAvoidanceOffset	397
13.1.3.24 kPowExtent	397
13.1.3.25 kPowTableSize	397
13.1.3.26 cSeedDivisor	397
13.1.3.27 cInitialSeedValue	397
13.2 AAX::Exception Namespace Reference	397
13.2.1 Description	397
13.3 AAX::internal Namespace Reference	398
13.3.1 Function Documentation	398
13.3.1.1 ToHexadecimal()	398
13.4 AAX_ChunkDataParserDefs Namespace Reference	398
13.4.1 Description	398
13.4.2 Variable Documentation	399
13.4.2.1 FLOAT_TYPE	399
13.4.2.2 FLOAT_STRING_IDENTIFIER	399
13.4.2.3 LONG_TYPE	399
13.4.2.4 LONG_STRING_IDENTIFIER	399
13.4.2.5 DOUBLE_TYPE	399
13.4.2.6 DOUBLE_STRING_IDENTIFIER	399
13.4.2.7 DOUBLE_TYPE_SIZE	399
13.4.2.8 DOUBLE_TYPE_INCR	400
13.4.2.9 SHORT_TYPE	400
13.4.2.10 SHORT_STRING_IDENTIFIER	400
13.4.2.11 SHORT_TYPE_SIZE	400
13.4.2.12 SHORT_TYPE_INCR	400
13.4.2.13 STRING_TYPE	400
13.4.2.14 STRING_STRING_IDENTIFIER	400
13.4.2.15 MAX_STRINGDATA_LENGTH	400
13.4.2.16 DEFAULT32BIT_TYPE_SIZE	401
13.4.2.17 DEFAULT32BIT_TYPE_INCR	401
13.4.2.18 STRING_IDENTIFIER_SIZE	401
13.4.2.19 NAME_NOT_FOUND	401
13.4.2.20 MAX_NAME_LENGTH	401
13.4.2.21 BUILD_DATA_FAILED	401
13.4.2.22 HEADER_SIZE	401
13.4.2.23 VERSION_ID_1	401
14 Class Documentation	403
14.1 _acfUID Struct Reference	403

14.1.1 Member Data Documentation	403
14.1.1.1 Data1	403
14.1.1.2 Data2	403
14.1.1.3 Data3	403
14.1.1.4 Data4	404
14.2 AAX_AggregateResult Class Reference	404
14.2.1 Description	404
14.2.2 Constructor & Destructor Documentation	405
14.2.2.1 AAX_AggregateResult()	405
14.2.2.2 ~AAX_AggregateResult()	405
14.2.3 Member Function Documentation	405
14.2.3.1 operator=()	405
14.2.3.2 operator AAX_Result()	405
14.2.3.3 Check()	405
14.2.3.4 Clear()	406
14.2.3.5 LastFailure()	406
14.2.3.6 NumFailed()	406
14.2.3.7 NumSucceeded()	406
14.2.3.8 NumAttempted()	406
14.3 AAX_CArrayDataBuffer< D > Class Template Reference	406
14.3.1 Description	407
14.3.2 Constructor & Destructor Documentation	407
14.3.2.1 AAX_CArrayDataBuffer() [1/4]	407
14.3.2.2 AAX_CArrayDataBuffer() [2/4]	407
14.3.2.3 AAX_CArrayDataBuffer() [3/4]	408
14.3.2.4 AAX_CArrayDataBuffer() [4/4]	408
14.3.2.5 ~AAX_CArrayDataBuffer()	408
14.3.3 Member Function Documentation	408
14.3.3.1 operator=() [1/2]	408
14.3.3.2 operator=() [2/2]	408
14.3.3.3 Type()	409
14.3.3.4 Size()	409
14.3.3.5 Data()	409
14.4 AAX_CArrayDataBufferOfType< T, D > Class Template Reference	409
14.4.1 Description	410
14.4.2 Constructor & Destructor Documentation	410
14.4.2.1 AAX_CArrayDataBufferOfType() [1/4]	410
14.4.2.2 AAX_CArrayDataBufferOfType() [2/4]	410
14.4.2.3 AAX_CArrayDataBufferOfType() [3/4]	411
14.4.2.4 AAX_CArrayDataBufferOfType() [4/4]	411
14.4.2.5 ~AAX_CArrayDataBufferOfType()	411
14.4.3 Member Function Documentation	411

14.4.3.1 operator=() [1/2]	411
14.4.3.2 operator=() [2/2]	411
14.4.3.3 Type()	412
14.4.3.4 Size()	412
14.4.3.5 Data()	412
14.5 AAX_CAtomicQueue< T, S > Class Template Reference	412
14.5.1 Description	413
14.5.2 Member Typedef Documentation	413
14.5.2.1 template_type	414
14.5.2.2 value_type	414
14.5.3 Constructor & Destructor Documentation	414
14.5.3.1 ~AAX_CAtomicQueue()	414
14.5.3.2 AAX_CAtomicQueue()	414
14.5.4 Member Function Documentation	414
14.5.4.1 Clear()	414
14.5.4.2 Push()	415
14.5.4.3 Pop()	415
14.5.4.4 Peek()	415
14.5.5 Member Data Documentation	416
14.5.5.1 template_size	416
14.6 AAX_CAutoreleasePool Class Reference	416
14.6.1 Constructor & Destructor Documentation	416
14.6.1.1 AAX_CAutoreleasePool()	416
14.6.1.2 ~AAX_CAutoreleasePool()	416
14.7 AAX_CBinaryDisplayDelegate< T > Class Template Reference	417
14.7.1 Description	417
14.7.2 Constructor & Destructor Documentation	417
14.7.2.1 AAX_CBinaryDisplayDelegate() [1/2]	417
14.7.2.2 AAX_CBinaryDisplayDelegate() [2/2]	418
14.7.3 Member Function Documentation	418
14.7.3.1 Clone()	418
14.7.3.2 ValueToString() [1/2]	418
14.7.3.3 ValueToString() [2/2]	419
14.7.3.4 StringToValue()	419
14.7.3.5 AddShortenedStrings()	420
14.8 AAX_CBinaryTaperDelegate< T > Class Template Reference	420
14.8.1 Description	420
14.8.2 Constructor & Destructor Documentation	421
14.8.2.1 AAX_CBinaryTaperDelegate()	421
14.8.3 Member Function Documentation	421
14.8.3.1 Clone()	421
14.8.3.2 GetMaximumValue()	422

14.8.3.3 GetMinimumValue()	422
14.8.3.4 ConstrainRealValue()	422
14.8.3.5 NormalizedToReal()	422
14.8.3.6 RealToNormalized()	423
14.9 AAX_CChunkDataParser Class Reference	423
14.9.1 Description	424
14.9.2 Constructor & Destructor Documentation	425
14.9.2.1 AAX_CChunkDataParser()	426
14.9.2.2 ~AAX_CChunkDataParser()	426
14.9.3 Member Function Documentation	426
14.9.3.1 AddFloat()	426
14.9.3.2 AddDouble()	426
14.9.3.3 AddInt32()	426
14.9.3.4 AddInt16()	427
14.9.3.5 AddString()	427
14.9.3.6 FindFloat()	427
14.9.3.7 FindDouble()	427
14.9.3.8 FindInt32()	427
14.9.3.9 FindInt16()	428
14.9.3.10 FindString()	428
14.9.3.11 ReplaceDouble()	428
14.9.3.12 GetChunkData()	428
14.9.3.13 GetChunkDataSize()	428
14.9.3.14 GetChunkVersion()	428
14.9.3.15 IsEmpty()	429
14.9.3.16 Clear()	429
14.9.3.17 LoadChunk()	429
14.9.3.18 WordAlign() [1/2]	429
14.9.3.19 WordAlign() [2/2]	429
14.9.3.20 FindName()	429
14.9.4 Member Data Documentation	430
14.9.4.1 mLastFoundIndex	430
14.9.4.2 mChunkData	430
14.9.4.3 mChunkVersion	430
14.9.4.4 mDataValues	430
14.10 AAX_CDecibelDisplayDelegateDecorator< T > Class Template Reference	430
14.10.1 Description	431
14.10.2 Constructor & Destructor Documentation	431
14.10.2.1 AAX_CDecibelDisplayDelegateDecorator()	431
14.10.3 Member Function Documentation	431
14.10.3.1 Clone()	432
14.10.3.2 ValueToString() [1/2]	432

14.10.3.3 ValueToString() [2/2]	432
14.10.3.4 StringToValue()	433
14.11 AAX_CEffectDirectData Class Reference	434
14.11.1 Description	434
14.11.2 Constructor & Destructor Documentation	435
14.11.2.1 AAX_CEffectDirectData()	435
14.11.2.2 ~AAX_CEffectDirectData()	435
14.11.3 Member Function Documentation	435
14.11.3.1 Initialize()	435
14.11.3.2 Uninitialize()	435
14.11.3.3 TimerWakeup()	436
14.11.3.4 NotificationReceived()	436
14.11.3.5 Controller()	437
14.11.3.6 EffectParameters()	437
14.11.3.7 Initialize_PrivateDataAccess()	437
14.11.3.8 TimerWakeup_PrivateDataAccess()	437
14.12 AAX_CEffectGUI Class Reference	438
14.12.1 Description	438
14.12.2 Constructor & Destructor Documentation	440
14.12.2.1 AAX_CEffectGUI()	440
14.12.2.2 ~AAX_CEffectGUI()	440
14.12.3 Member Function Documentation	440
14.12.3.1 Initialize()	440
14.12.3.2 Uninitialize()	440
14.12.3.3 NotificationReceived()	441
14.12.3.4 SetViewContainer()	441
14.12.3.5 GetViewSize()	442
14.12.3.6 Draw()	442
14.12.3.7 TimerWakeup()	443
14.12.3.8 ParameterUpdated()	443
14.12.3.9 GetCustomLabel()	443
14.12.3.10 SetControlHighlightInfo()	444
14.12.3.11 CreateViewContents()	444
14.12.3.12 CreateViewContainer()	444
14.12.3.13 DeleteViewContainer()	445
14.12.3.14 UpdateAllParameters()	445
14.12.3.15 GetController() [1/2]	445
14.12.3.16 GetController() [2/2]	445
14.12.3.17 GetEffectParameters() [1/2]	445
14.12.3.18 GetEffectParameters() [2/2]	446
14.12.3.19 GetViewContainer() [1/2]	446
14.12.3.20 GetViewContainer() [2/2]	446

14.12.3.21 Transport() [1/2]	446
14.12.3.22 Transport() [2/2]	446
14.12.3.23 GetViewContainerType()	446
14.12.3.24 GetViewContainerPtr()	447
14.13 AAX_CEffectParameters Class Reference	447
14.13.1 Description	447
14.13.2 Related classes	447
14.13.3 Constructor & Destructor Documentation	453
14.13.3.1 AAX_CEffectParameters()	453
14.13.3.2 ~AAX_CEffectParameters()	453
14.13.4 Member Function Documentation	453
14.13.4.1 operator=()	453
14.13.4.2 Initialize()	453
14.13.4.3 Uninitialize()	454
14.13.4.4 NotificationReceived()	454
14.13.4.5 GetNumberOfParameters()	455
14.13.4.6 GetMasterBypassParameter()	455
14.13.4.7 GetParameterIsAutomatable()	455
14.13.4.8 GetParameterNumberOfSteps()	456
14.13.4.9 GetParameterName()	456
14.13.4.10 GetParameterNameOfLength()	456
14.13.4.11 GetParameterDefaultNormalizedValue()	457
14.13.4.12 SetParameterDefaultNormalizedValue()	457
14.13.4.13 GetParameterType()	458
14.13.4.14 GetParameterOrientation()	458
14.13.4.15 GetParameter()	459
14.13.4.16 GetParameterIndex()	459
14.13.4.17 GetParameterIDFromIndex()	461
14.13.4.18 GetParameterValueInfo()	461
14.13.4.19 GetParameterValueFromString()	462
14.13.4.20 GetParameterStringFromValue()	462
14.13.4.21 GetParameterValueString()	463
14.13.4.22 GetParameterNormalizedValue()	463
14.13.4.23 SetParameterNormalizedValue()	463
14.13.4.24 SetParameterNormalizedRelative()	464
14.13.4.25 TouchParameter()	465
14.13.4.26 ReleaseParameter()	465
14.13.4.27 UpdateParameterTouch()	465
14.13.4.28 UpdateParameterNormalizedValue()	466
14.13.4.29 UpdateParameterNormalizedRelative()	466
14.13.4.30 GenerateCoefficients()	467
14.13.4.31 ResetFieldData()	468

14.13.4.32	GetNumberOfChunks()	468
14.13.4.33	GetChunkIDFromIndex()	469
14.13.4.34	GetChunkSize()	469
14.13.4.35	GetChunk()	469
14.13.4.36	SetChunk()	470
14.13.4.37	CompareActiveChunk()	471
14.13.4.38	GetNumberOfChanges()	471
14.13.4.39	TimerWakeup()	471
14.13.4.40	GetCurveData()	472
14.13.4.41	GetCurveDataMeterIds()	473
14.13.4.42	GetCurveDataDisplayRange()	473
14.13.4.43	UpdatePageTable() [1/2]	474
14.13.4.44	GetCustomData()	475
14.13.4.45	SetCustomData()	475
14.13.4.46	DoMIDITransfers()	476
14.13.4.47	UpdateMIDINodes()	476
14.13.4.48	UpdateControlMIDINodes()	477
14.13.4.49	RenderAudio_Hybrid()	477
14.13.4.50	Controller() [1/2]	477
14.13.4.51	Controller() [2/2]	478
14.13.4.52	Transport() [1/2]	478
14.13.4.53	Transport() [2/2]	478
14.13.4.54	AutomationDelegate() [1/2]	478
14.13.4.55	AutomationDelegate() [2/2]	478
14.13.4.56	SetTaperDelegate()	478
14.13.4.57	SetDisplayDelegate()	479
14.13.4.58	IsParameterTouched()	479
14.13.4.59	IsParameterLinkReady()	479
14.13.4.60	EffectInit()	479
14.13.4.61	UpdatePageTable() [2/2]	480
14.13.4.62	FilterParameterIDOnSave()	480
14.13.4.63	BuildChunkData()	480
14.13.5	Member Data Documentation	480
14.13.5.1	mNumPlugInChanges	481
14.13.5.2	mChunkSize	481
14.13.5.3	mChunkParser	481
14.13.5.4	mNumChunkedParameters	481
14.13.5.5	mPacketDispatcher	481
14.13.5.6	mParameterManager	481
14.13.5.7	mFilteredParameters	481
14.14	AAX_CheckedResult Class Reference	482
14.14.1	Description	482

14.14.2 Member Typedef Documentation	484
14.14.2.1 Exception	484
14.14.3 Constructor & Destructor Documentation	484
14.14.3.1 ~AAX_CheckedResult()	484
14.14.3.2 AAX_CheckedResult() [1/2]	484
14.14.3.3 AAX_CheckedResult() [2/2]	484
14.14.4 Member Function Documentation	484
14.14.4.1 AddAcceptedResult()	484
14.14.4.2 ResetAcceptedResults()	485
14.14.4.3 operator=()	485
14.14.4.4 operator" =()	485
14.14.4.5 operator AAX_Result()	485
14.14.4.6 Clear()	485
14.14.4.7 LastError()	486
14.15 AAX_CHostProcessor Class Reference	486
14.15.1 Description	486
14.15.2 Constructor & Destructor Documentation	488
14.15.2.1 AAX_CHostProcessor()	488
14.15.2.2 ~AAX_CHostProcessor()	488
14.15.3 Member Function Documentation	488
14.15.3.1 Initialize()	488
14.15.3.2 Uninitialize()	489
14.15.3.3 InitOutputBounds()	489
14.15.3.4 SetLocation()	490
14.15.3.5 RenderAudio()	490
14.15.3.6 PreRender()	491
14.15.3.7 PostRender()	491
14.15.3.8 AnalyzeAudio()	492
14.15.3.9 PreAnalyze()	492
14.15.3.10 PostAnalyze()	493
14.15.3.11 GetClipNameSuffix()	493
14.15.3.12 GetEffectParameters() [1/2]	493
14.15.3.13 GetEffectParameters() [2/2]	493
14.15.3.14 GetHostProcessorDelegate() [1/2]	494
14.15.3.15 GetHostProcessorDelegate() [2/2]	494
14.15.3.16 GetLocation()	494
14.15.3.17 GetInputRange()	494
14.15.3.18 GetOutputRange()	494
14.15.3.19 GetSrcStart()	494
14.15.3.20 GetSrcEnd()	495
14.15.3.21 GetDstStart()	495
14.15.3.22 GetDstEnd()	495

14.15.3.23 TranslateOutputBounds()	495
14.15.3.24 GetAudio()	496
14.15.3.25 GetSideChainInputNum()	496
14.15.3.26 Controller() [1/2]	496
14.15.3.27 Controller() [2/2]	496
14.15.3.28 HostProcessorDelegate() [1/2]	497
14.15.3.29 HostProcessorDelegate() [2/2]	497
14.15.3.30 EffectParameters() [1/2]	497
14.15.3.31 EffectParameters() [2/2]	497
14.16 AAX_CHostServices Class Reference	497
14.16.1 Description	497
14.16.2 Member Function Documentation	498
14.16.2.1 Set()	498
14.16.2.2 HandleAssertFailure()	498
14.16.2.3 Trace()	498
14.16.2.4 StackTrace()	499
14.17 AAX_CLinearTaperDelegate< T, RealPrecision > Class Template Reference	499
14.17.1 Description	499
14.17.2 Constructor & Destructor Documentation	500
14.17.2.1 AAX_CLinearTaperDelegate()	500
14.17.3 Member Function Documentation	501
14.17.3.1 Clone()	501
14.17.3.2 GetMinimumValue()	501
14.17.3.3 GetMaximumValue()	501
14.17.3.4 ConstrainRealValue()	501
14.17.3.5 NormalizedToReal()	502
14.17.3.6 RealToNormalized()	502
14.17.3.7 Round()	503
14.18 AAX_CLogTaperDelegate< T, RealPrecision > Class Template Reference	503
14.18.1 Description	503
14.18.2 Constructor & Destructor Documentation	504
14.18.2.1 AAX_CLogTaperDelegate()	504
14.18.3 Member Function Documentation	504
14.18.3.1 Clone()	505
14.18.3.2 GetMinimumValue()	505
14.18.3.3 GetMaximumValue()	505
14.18.3.4 ConstrainRealValue()	505
14.18.3.5 NormalizedToReal()	506
14.18.3.6 RealToNormalized()	506
14.18.3.7 Round()	507
14.19 AAX_CMidiPacket Struct Reference	507
14.19.1 Description	507

14.19.2 Member Data Documentation	507
14.19.2.1 mTimestamp	507
14.19.2.2 mLength	508
14.19.2.3 mData	508
14.19.2.4 mIsImmediate	508
14.20 AAX_CMidiStream Struct Reference	508
14.20.1 Description	508
14.20.2 Member Data Documentation	509
14.20.2.1 mBufferSize	509
14.20.2.2 mBuffer	509
14.21 AAX_CMonolithicParameters Class Reference	509
14.21.1 Description	510
14.21.2 Member Typedef Documentation	511
14.21.2.1 TParamValPair	511
14.21.3 Constructor & Destructor Documentation	511
14.21.3.1 AAX_CMonolithicParameters()	511
14.21.3.2 ~AAX_CMonolithicParameters()	511
14.21.4 Member Function Documentation	512
14.21.4.1 RenderAudio()	512
14.21.4.2 AddSynchronizedParameter()	512
14.21.4.3 UpdateParameterNormalizedValue()	513
14.21.4.4 GenerateCoefficients()	513
14.21.4.5 ResetFieldData()	514
14.21.4.6 TimerWakeup()	515
14.21.4.7 StaticDescribe()	515
14.21.4.8 StaticRenderAudio()	516
14.22 AAX_CMutex Class Reference	516
14.22.1 Description	516
14.22.2 Constructor & Destructor Documentation	517
14.22.2.1 AAX_CMutex()	517
14.22.2.2 ~AAX_CMutex()	517
14.22.3 Member Function Documentation	517
14.22.3.1 Lock()	517
14.22.3.2 Unlock()	517
14.22.3.3 Try_Lock()	517
14.23 AAX_CNumberDisplayDelegate< T, Precision, SpaceAfter > Class Template Reference	518
14.23.1 Description	518
14.23.2 Member Function Documentation	518
14.23.2.1 Clone()	518
14.23.2.2 ValueToString() [1/2]	519
14.23.2.3 ValueToString() [2/2]	520
14.23.2.4 StringToValue()	520

14.24 AAX_Component< aContextType > Class Template Reference	521
14.24.1 Description	521
14.24.2 Member Typedef Documentation	522
14.24.2.1 CProcessProc	522
14.24.2.2 CPacketAllocator	522
14.24.2.3 CInstanceInitProc	522
14.24.2.4 CBackgroundProc	522
14.24.2.5 CInitPrivateDataProc	522
14.25 AAX_CPacket Class Reference	523
14.25.1 Description	523
14.25.2 Constructor & Destructor Documentation	523
14.25.2.1 AAX_CPacket()	523
14.25.2.2 ~AAX_CPacket()	523
14.25.3 Member Function Documentation	523
14.25.3.1 GetPtr() [1/2]	524
14.25.3.2 SetDirty()	524
14.25.3.3 IsDirty()	524
14.25.3.4 GetID()	524
14.25.3.5 GetSize()	524
14.25.3.6 GetPtr() [2/2]	524
14.26 AAX_CPacketDispatcher Class Reference	524
14.26.1 Description	525
14.26.2 Constructor & Destructor Documentation	525
14.26.2.1 AAX_CPacketDispatcher()	525
14.26.2.2 ~AAX_CPacketDispatcher()	525
14.26.3 Member Function Documentation	525
14.26.3.1 Initialize()	526
14.26.3.2 RegisterPacket() [1/3]	526
14.26.3.3 RegisterPacket() [2/3]	526
14.26.3.4 RegisterPacket() [3/3]	526
14.26.3.5 SetDirty()	526
14.26.3.6 Dispatch()	527
14.26.3.7 GenerateSingleValuePacket()	527
14.27 AAX_CPacketHandler< TWorker > Class Template Reference	527
14.27.1 Description	527
14.27.2 Constructor & Destructor Documentation	528
14.27.2.1 AAX_CPacketHandler() [1/2]	528
14.27.2.2 AAX_CPacketHandler() [2/2]	528
14.27.3 Member Function Documentation	528
14.27.3.1 Clone()	528
14.27.3.2 Call()	528
14.27.4 Member Data Documentation	529

14.27.4.1 pt2Object	529
14.27.4.2 fpt	529
14.27.4.3 fptEx	529
14.28 AAX_CParameter< T > Class Template Reference	529
14.28.1 Description	530
14.28.2 Member Enumeration Documentation	534
14.28.2.1 Type	534
14.28.2.2 Defaults	535
14.28.3 Constructor & Destructor Documentation	535
14.28.3.1 AAX_CParameter() [1/4]	535
14.28.3.2 AAX_CParameter() [2/4]	536
14.28.3.3 AAX_CParameter() [3/4]	536
14.28.3.4 AAX_CParameter() [4/4]	537
14.28.3.5 ~AAX_CParameter()	537
14.28.4 Member Function Documentation	537
14.28.4.1 AAX_DEFAULT_MOVE_CTOR()	537
14.28.4.2 AAX_DEFAULT_MOVE_OPER()	537
14.28.4.3 AAX_DELETE() [1/3]	538
14.28.4.4 AAX_DELETE() [2/3]	538
14.28.4.5 AAX_DELETE() [3/3]	538
14.28.4.6 CloneValue()	538
14.28.4.7 Identifier()	538
14.28.4.8 SetName()	538
14.28.4.9 Name()	539
14.28.4.10 AddShortenedName()	539
14.28.4.11 ShortenedName()	539
14.28.4.12 ClearShortenedNames()	540
14.28.4.13 SetNormalizedDefaultValue()	540
14.28.4.14 GetNormalizedDefaultValue()	540
14.28.4.15 SetToDefaultValue()	540
14.28.4.16 SetNormalizedValue()	540
14.28.4.17 GetNormalizedValue()	541
14.28.4.18 SetNumberOfSteps()	541
14.28.4.19 GetNumberOfSteps()	541
14.28.4.20 GetStepValue()	542
14.28.4.21 GetNormalizedValueFromStep()	542
14.28.4.22 GetStepValueFromNormalizedValue()	542
14.28.4.23 SetStepValue()	543
14.28.4.24 SetType()	543
14.28.4.25 GetType()	543
14.28.4.26 SetOrientation()	543
14.28.4.27 GetOrientation()	544

14.28.4.28 SetTaperDelegate()	544
14.28.4.29 SetDisplayDelegate()	544
14.28.4.30 GetValueString() [1/2]	545
14.28.4.31 GetValueString() [2/2]	545
14.28.4.32 GetNormalizedValueFromBool() [1/2]	546
14.28.4.33 GetNormalizedValueFromInt32() [1/2]	546
14.28.4.34 GetNormalizedValueFromFloat() [1/2]	547
14.28.4.35 GetNormalizedValueFromDouble() [1/2]	547
14.28.4.36 GetNormalizedValueFromString()	548
14.28.4.37 GetBoolFromNormalizedValue() [1/2]	548
14.28.4.38 GetInt32FromNormalizedValue() [1/2]	549
14.28.4.39 GetFloatFromNormalizedValue() [1/2]	549
14.28.4.40 GetDoubleFromNormalizedValue() [1/2]	549
14.28.4.41 GetStringFromNormalizedValue() [1/2]	550
14.28.4.42 GetStringFromNormalizedValue() [2/2]	550
14.28.4.43 SetValueFromString()	551
14.28.4.44 SetAutomationDelegate()	551
14.28.4.45 Automatable()	552
14.28.4.46 Touch()	552
14.28.4.47 Release()	552
14.28.4.48 GetValueAsBool()	552
14.28.4.49 GetValueAsInt32()	553
14.28.4.50 GetValueAsFloat()	553
14.28.4.51 GetValueAsDouble()	554
14.28.4.52 GetValueAsString() [1/2]	554
14.28.4.53 SetValueWithBool() [1/2]	555
14.28.4.54 SetValueWithInt32() [1/2]	555
14.28.4.55 SetValueWithFloat() [1/2]	555
14.28.4.56 SetValueWithDouble() [1/2]	556
14.28.4.57 SetValueWithString() [1/2]	556
14.28.4.58 UpdateNormalizedValue()	557
14.28.4.59 SetValue()	557
14.28.4.60 GetValue()	557
14.28.4.61 SetDefaultValue()	558
14.28.4.62 GetDefaultValue()	558
14.28.4.63 TaperDelegate()	558
14.28.4.64 DisplayDelegate()	558
14.28.4.65 GetValueAsString() [2/2]	558
14.28.4.66 SetValueWithBool() [2/2]	559
14.28.4.67 SetValueWithInt32() [2/2]	559
14.28.4.68 SetValueWithFloat() [2/2]	560
14.28.4.69 SetValueWithDouble() [2/2]	560

14.28.4.70 SetValueWithString() [2/2]	560
14.28.4.71 GetNormalizedValueFromBool() [2/2]	561
14.28.4.72 GetNormalizedValueFromInt32() [2/2]	561
14.28.4.73 GetNormalizedValueFromFloat() [2/2]	562
14.28.4.74 GetNormalizedValueFromDouble() [2/2]	562
14.28.4.75 GetBoolFromNormalizedValue() [2/2]	563
14.28.4.76 GetInt32FromNormalizedValue() [2/2]	563
14.28.4.77 GetFloatFromNormalizedValue() [2/2]	564
14.28.4.78 GetDoubleFromNormalizedValue() [2/2]	564
14.28.5 Member Data Documentation	564
14.28.5.1 mNames	565
14.28.5.2 mAutomatable	565
14.28.5.3 mNumSteps	565
14.28.5.4 mControlType	565
14.28.5.5 mOrientation	565
14.28.5.6 mTaperDelegate	565
14.28.5.7 mDisplayDelegate	565
14.28.5.8 mAutomationDelegate	566
14.28.5.9 mNeedNotify	566
14.28.5.10 mValue	566
14.28.5.11 mDefaultValue	566
14.29 AAX_CParameterManager Class Reference	566
14.29.1 Description	566
14.29.2 Constructor & Destructor Documentation	567
14.29.2.1 AAX_CParameterManager()	567
14.29.2.2 ~AAX_CParameterManager()	567
14.29.3 Member Function Documentation	568
14.29.3.1 Initialize()	568
14.29.3.2 NumParameters()	568
14.29.3.3 RemoveParameterByID()	568
14.29.3.4 RemoveAllParameters()	568
14.29.3.5 GetParameterByID() [1/2]	569
14.29.3.6 GetParameterByID() [2/2]	569
14.29.3.7 GetParameterByName() [1/2]	569
14.29.3.8 GetParameterByName() [2/2]	570
14.29.3.9 GetParameter() [1/2]	570
14.29.3.10 GetParameter() [2/2]	570
14.29.3.11 GetParameterIndex()	571
14.29.3.12 AddParameter()	571
14.29.3.13 RemoveParameter()	571
14.29.4 Member Data Documentation	572
14.29.4.1 mAutomationDelegate	572

14.29.4.2 mParameters	572
14.29.4.3 mParametersMap	572
14.30 AAX_CParameterValue< T > Class Template Reference	572
14.30.1 Description	572
14.30.2 Member Enumeration Documentation	574
14.30.2.1 Defaults	574
14.30.3 Constructor & Destructor Documentation	574
14.30.3.1 AAX_CParameterValue() [1/3]	574
14.30.3.2 AAX_CParameterValue() [2/3]	574
14.30.3.3 AAX_CParameterValue() [3/3]	575
14.30.4 Member Function Documentation	575
14.30.4.1 AAX_DEFAULT_DTOR_OVERRIDE()	575
14.30.4.2 AAX_DEFAULT_MOVE_CTOR()	575
14.30.4.3 AAX_DEFAULT_MOVE_OPER()	575
14.30.4.4 AAX_DELETE()	576
14.30.4.5 Get()	576
14.30.4.6 Set()	576
14.30.4.7 Clone()	576
14.30.4.8 Identifier()	576
14.30.4.9 GetValueAsBool() [1/2]	576
14.30.4.10 GetValueAsInt32() [1/2]	577
14.30.4.11 GetValueAsFloat() [1/2]	577
14.30.4.12 GetValueAsDouble() [1/2]	578
14.30.4.13 GetValueAsString() [1/2]	578
14.30.4.14 GetValueAsBool() [2/2]	579
14.30.4.15 GetValueAsInt32() [2/2]	579
14.30.4.16 GetValueAsFloat() [2/2]	579
14.30.4.17 GetValueAsDouble() [2/2]	580
14.30.4.18 GetValueAsString() [2/2]	580
14.31 AAX_CPercentDisplayDelegateDecorator< T > Class Template Reference	581
14.31.1 Description	581
14.31.2 Constructor & Destructor Documentation	582
14.31.2.1 AAX_CPercentDisplayDelegateDecorator()	582
14.31.3 Member Function Documentation	582
14.31.3.1 Clone()	582
14.31.3.2 ValueToString() [1/2]	582
14.31.3.3 ValueToString() [2/2]	583
14.31.3.4 StringToValue()	583
14.32 AAX_CPieceWiseLinearTaperDelegate< T, RealPrecision > Class Template Reference	584
14.32.1 Description	584
14.32.2 Constructor & Destructor Documentation	585
14.32.2.1 AAX_CPieceWiseLinearTaperDelegate() [1/2]	585

14.32.2.2 AAX_CPieceWiseLinearTaperDelegate() [2/2]	586
14.32.2.3 ~AAX_CPieceWiseLinearTaperDelegate()	586
14.32.3 Member Function Documentation	586
14.32.3.1 Clone()	586
14.32.3.2 GetMinimumValue()	586
14.32.3.3 GetMaximumValue()	587
14.32.3.4 ConstrainRealValue()	587
14.32.3.5 NormalizedToReal()	587
14.32.3.6 RealToNormalized()	588
14.32.3.7 Round()	588
14.33 AAX_CRangeTaperDelegate< T, RealPrecision > Class Template Reference	588
14.33.1 Description	589
14.33.2 Constructor & Destructor Documentation	590
14.33.2.1 AAX_CRangeTaperDelegate() [1/2]	590
14.33.2.2 AAX_CRangeTaperDelegate() [2/2]	590
14.33.3 Member Function Documentation	590
14.33.3.1 operator=()	591
14.33.3.2 Clone()	591
14.33.3.3 GetMinimumValue()	591
14.33.3.4 GetMaximumValue()	591
14.33.3.5 ConstrainRealValue()	591
14.33.3.6 NormalizedToReal()	592
14.33.3.7 RealToNormalized()	592
14.33.3.8 Round()	593
14.33.3.9 SmartRound()	593
14.34 AAX_CSessionDocumentClient Class Reference	593
14.34.1 Description	593
14.34.2 Constructor & Destructor Documentation	594
14.34.2.1 AAX_CSessionDocumentClient()	594
14.34.2.2 ~AAX_CSessionDocumentClient()	594
14.34.3 Member Function Documentation	594
14.34.3.1 Initialize()	595
14.34.3.2 Uninitialize()	595
14.34.3.3 SetSessionDocument()	595
14.34.3.4 NotificationReceived()	595
14.34.3.5 SessionDocumentWillChange()	596
14.34.3.6 SessionDocumentChanged()	596
14.34.3.7 GetController() [1/2]	597
14.34.3.8 GetController() [2/2]	597
14.34.3.9 GetEffectParameters() [1/2]	597
14.34.3.10 GetEffectParameters() [2/2]	597
14.34.3.11 GetSessionDocument() [1/2]	597

14.34.3.12 GetSessionDocument() [2/2]	598
14.35 AAX_CStateDisplayDelegate< T > Class Template Reference	598
14.35.1 Description	598
14.35.2 Constructor & Destructor Documentation	599
14.35.2.1 AAX_CStateDisplayDelegate() [1/4]	599
14.35.2.2 AAX_CStateDisplayDelegate() [2/4]	599
14.35.2.3 AAX_CStateDisplayDelegate() [3/4]	599
14.35.2.4 AAX_CStateDisplayDelegate() [4/4]	600
14.35.3 Member Function Documentation	600
14.35.3.1 Clone()	600
14.35.3.2 ValueToString() [1/2]	600
14.35.3.3 ValueToString() [2/2]	601
14.35.3.4 StringToValue()	601
14.35.3.5 AddShortenedStrings()	602
14.35.3.6 Compare()	602
14.36 AAX_CStatelessParameter Class Reference	602
14.36.1 Description	602
14.36.2 Constructor & Destructor Documentation	605
14.36.2.1 AAX_CStatelessParameter() [1/2]	605
14.36.2.2 AAX_CStatelessParameter() [2/2]	605
14.36.3 Member Function Documentation	605
14.36.3.1 AAX_DEFAULT_DTOR_OVERRIDE()	605
14.36.3.2 CloneValue()	605
14.36.3.3 Identifier()	606
14.36.3.4 SetName()	606
14.36.3.5 Name()	606
14.36.3.6 AddShortenedName()	607
14.36.3.7 ShortenedName()	607
14.36.3.8 ClearShortenedNames()	607
14.36.3.9 Automatable()	608
14.36.3.10 SetAutomationDelegate()	608
14.36.3.11 Touch()	608
14.36.3.12 Release()	609
14.36.3.13 SetNormalizedValue()	609
14.36.3.14 GetNormalizedValue()	609
14.36.3.15 SetNormalizedDefaultValue()	609
14.36.3.16 GetNormalizedDefaultValue()	610
14.36.3.17 SetToDefaultValue()	610
14.36.3.18 SetNumberOfSteps()	610
14.36.3.19 GetNumberOfSteps()	610
14.36.3.20 GetStepValue()	611
14.36.3.21 GetNormalizedValueFromStep()	611

14.36.3.22	GetStepValueFromNormalizedValue()	611
14.36.3.23	SetStepValue()	611
14.36.3.24	GetValueString() [1/2]	611
14.36.3.25	GetValueString() [2/2]	612
14.36.3.26	GetNormalizedValueFromBool()	612
14.36.3.27	GetNormalizedValueFromInt32()	613
14.36.3.28	GetNormalizedValueFromFloat()	613
14.36.3.29	GetNormalizedValueFromDouble()	614
14.36.3.30	GetNormalizedValueFromString()	614
14.36.3.31	GetBoolFromNormalizedValue()	615
14.36.3.32	GetInt32FromNormalizedValue()	615
14.36.3.33	GetFloatFromNormalizedValue()	616
14.36.3.34	GetDoubleFromNormalizedValue()	616
14.36.3.35	GetStringFromNormalizedValue() [1/2]	616
14.36.3.36	GetStringFromNormalizedValue() [2/2]	617
14.36.3.37	SetValueFromString()	617
14.36.3.38	GetValueAsBool()	618
14.36.3.39	GetValueAsInt32()	618
14.36.3.40	GetValueAsFloat()	619
14.36.3.41	GetValueAsDouble()	619
14.36.3.42	GetValueAsString()	620
14.36.3.43	SetValueWithBool()	620
14.36.3.44	SetValueWithInt32()	620
14.36.3.45	SetValueWithFloat()	621
14.36.3.46	SetValueWithDouble()	621
14.36.3.47	SetValueWithString()	622
14.36.3.48	SetType()	622
14.36.3.49	GetType()	622
14.36.3.50	SetOrientation()	623
14.36.3.51	GetOrientation()	623
14.36.3.52	SetTaperDelegate()	623
14.36.3.53	SetDisplayDelegate()	624
14.36.3.54	UpdateNormalizedValue()	624
14.36.4	Member Data Documentation	624
14.36.4.1	mNames	624
14.36.4.2	mID	625
14.36.4.3	mAutomationDelegate	625
14.36.4.4	mValueString	625
14.37	AAX_CStateTaperDelegate< T > Class Template Reference	625
14.37.1	Description	625
14.37.2	Constructor & Destructor Documentation	626
14.37.2.1	AAX_CStateTaperDelegate()	626

14.37.3 Member Function Documentation	626
14.37.3.1 Clone()	626
14.37.3.2 GetMinimumValue()	627
14.37.3.3 GetMaximumValue()	627
14.37.3.4 ConstrainRealValue()	627
14.37.3.5 NormalizedToReal()	628
14.37.3.6 RealToNormalized()	628
14.38 AAX_CString Class Reference	629
14.38.1 Description	629
14.38.2 Constructor & Destructor Documentation	630
14.38.2.1 AAX_CString() [1/5]	630
14.38.2.2 AAX_CString() [2/5]	631
14.38.2.3 AAX_CString() [3/5]	631
14.38.2.4 AAX_CString() [4/5]	631
14.38.2.5 AAX_CString() [5/5]	631
14.38.3 Member Function Documentation	631
14.38.3.1 Length()	631
14.38.3.2 MaxLength()	632
14.38.3.3 Get()	632
14.38.3.4 Set()	632
14.38.3.5 operator=() [1/5]	632
14.38.3.6 operator=() [2/5]	632
14.38.3.7 AAX_DEFAULT_MOVE_CTOR()	633
14.38.3.8 StdString() [1/2]	633
14.38.3.9 StdString() [2/2]	633
14.38.3.10 operator=() [3/5]	633
14.38.3.11 operator=() [4/5]	633
14.38.3.12 operator=() [5/5]	633
14.38.3.13 Clear()	633
14.38.3.14 Empty()	634
14.38.3.15 Erase()	634
14.38.3.16 Append() [1/2]	634
14.38.3.17 Append() [2/2]	634
14.38.3.18 AppendNumber() [1/2]	634
14.38.3.19 AppendNumber() [2/2]	634
14.38.3.20 AppendHex()	635
14.38.3.21 Insert() [1/2]	635
14.38.3.22 Insert() [2/2]	635
14.38.3.23 InsertNumber() [1/2]	635
14.38.3.24 InsertNumber() [2/2]	635
14.38.3.25 InsertHex()	635
14.38.3.26 Replace() [1/2]	636

14.38.3.27 Replace() [2/2]	636
14.38.3.28 FindFirst() [1/3]	636
14.38.3.29 FindFirst() [2/3]	636
14.38.3.30 FindFirst() [3/3]	636
14.38.3.31 FindLast() [1/3]	636
14.38.3.32 FindLast() [2/3]	637
14.38.3.33 FindLast() [3/3]	637
14.38.3.34 CString()	637
14.38.3.35 ToDouble()	637
14.38.3.36 ToInteger()	637
14.38.3.37 SubString()	637
14.38.3.38 Equals() [1/3]	638
14.38.3.39 Equals() [2/3]	638
14.38.3.40 Equals() [3/3]	638
14.38.3.41 operator==() [1/3]	638
14.38.3.42 operator==() [2/3]	638
14.38.3.43 operator==() [3/3]	638
14.38.3.44 operator!=() [1/3]	639
14.38.3.45 operator!=() [2/3]	639
14.38.3.46 operator!=() [3/3]	639
14.38.3.47 operator<()	639
14.38.3.48 operator>()	639
14.38.3.49 operator[]() [1/2]	639
14.38.3.50 operator[]() [2/2]	639
14.38.3.51 operator+=() [1/3]	640
14.38.3.52 operator+=() [2/3]	640
14.38.3.53 operator+=() [3/3]	640
14.38.4 Friends And Related Function Documentation	640
14.38.4.1 operator<<	640
14.38.4.2 operator>>	640
14.38.5 Member Data Documentation	640
14.38.5.1 kInvalidIndex	641
14.38.5.2 kMaxStringLength	641
14.38.5.3 mString	641
14.39 AAX_CStringAbbreviations Class Reference	641
14.39.1 Description	641
14.39.2 Constructor & Destructor Documentation	641
14.39.2.1 AAX_CStringAbbreviations()	641
14.39.3 Member Function Documentation	642
14.39.3.1 SetPrimary()	642
14.39.3.2 Primary()	642
14.39.3.3 Add()	642

14.39.3.4 Get()	642
14.39.3.5 Clear()	642
14.40 AAX_CStringDataBuffer Class Reference	643
14.40.1 Description	643
14.40.2 Constructor & Destructor Documentation	643
14.40.2.1 AAX_CStringDataBuffer() [1/5]	643
14.40.2.2 AAX_CStringDataBuffer() [2/5]	643
14.40.2.3 AAX_CStringDataBuffer() [3/5]	644
14.40.2.4 AAX_CStringDataBuffer() [4/5]	644
14.40.2.5 AAX_CStringDataBuffer() [5/5]	644
14.40.2.6 ~AAX_CStringDataBuffer()	644
14.40.3 Member Function Documentation	644
14.40.3.1 operator=() [1/2]	644
14.40.3.2 operator=() [2/2]	644
14.40.3.3 Type()	645
14.40.3.4 Size()	645
14.40.3.5 Data()	645
14.41 AAX_CStringDataBufferOfT< T > Class Template Reference	645
14.41.1 Description	646
14.41.2 Constructor & Destructor Documentation	646
14.41.2.1 AAX_CStringDataBufferOfT() [1/5]	646
14.41.2.2 AAX_CStringDataBufferOfT() [2/5]	646
14.41.2.3 AAX_CStringDataBufferOfT() [3/5]	647
14.41.2.4 AAX_CStringDataBufferOfT() [4/5]	647
14.41.2.5 AAX_CStringDataBufferOfT() [5/5]	647
14.41.2.6 ~AAX_CStringDataBufferOfT()	647
14.41.3 Member Function Documentation	647
14.41.3.1 operator=() [1/2]	647
14.41.3.2 operator=() [2/2]	647
14.41.3.3 Type()	648
14.41.3.4 Size()	648
14.41.3.5 Data()	648
14.42 AAX_CStringDisplayDelegate< T > Class Template Reference	648
14.42.1 Description	649
14.42.2 Constructor & Destructor Documentation	649
14.42.2.1 AAX_CStringDisplayDelegate()	649
14.42.3 Member Function Documentation	650
14.42.3.1 Clone()	650
14.42.3.2 ValueToString() [1/2]	650
14.42.3.3 ValueToString() [2/2]	651
14.42.3.4 StringToValue()	651
14.42.4 Member Data Documentation	652

14.42.4.1 mStringMap	652
14.42.4.2 mInverseStringMap	652
14.43 AAX_CTask Class Reference	652
14.43.1 Constructor & Destructor Documentation	653
14.43.1.1 AAX_CTask()	653
14.43.2 Member Function Documentation	653
14.43.2.1 ACF_DECLARE_STANDARD_UNKNOWN()	653
14.43.2.2 AAX_DELETE()	653
14.43.2.3 AAX_DEFAULT_DTOR_OVERRIDE()	653
14.43.2.4 GetType()	653
14.43.2.5 GetArgumentOfType()	654
14.43.2.6 SetProgress()	654
14.43.2.7 GetProgress()	654
14.43.2.8 AddResult()	655
14.43.2.9 SetDone()	655
14.43.2.10 Status()	655
14.43.3 Member Data Documentation	656
14.43.3.1 AAX_OVERRIDE	656
14.44 AAX_CTaskAgent Class Reference	656
14.44.1 Description	656
14.44.2 Constructor & Destructor Documentation	657
14.44.2.1 AAX_CTaskAgent()	657
14.44.2.2 ~AAX_CTaskAgent()	657
14.44.3 Member Function Documentation	657
14.44.3.1 Initialize()	657
14.44.3.2 Uninitialize()	658
14.44.3.3 AddTask() [1/2]	658
14.44.3.4 CancelAllTasks()	658
14.44.3.5 AddTask() [2/2]	658
14.44.3.6 ReceiveTask()	659
14.44.3.7 GetController()	659
14.44.3.8 GetEffectParameters()	659
14.45 AAX_CTempoBreakpoint Struct Reference	659
14.45.1 Member Data Documentation	659
14.45.1.1 mSampleLocation	659
14.45.1.2 mValue	660
14.46 AAX_CUnitDisplayDelegateDecorator< T > Class Template Reference	660
14.46.1 Description	660
14.46.2 Constructor & Destructor Documentation	661
14.46.2.1 AAX_CUnitDisplayDelegateDecorator()	661
14.46.3 Member Function Documentation	661
14.46.3.1 Clone()	661

14.46.3.2 ValueToString() [1/2]	661
14.46.3.3 ValueToString() [2/2]	662
14.46.3.4 StringToValue()	662
14.46.4 Member Data Documentation	663
14.46.4.1 mUnitString	663
14.47 AAX_CUnitPrefixDisplayDelegateDecorator< T > Class Template Reference	663
14.47.1 Description	664
14.47.2 Constructor & Destructor Documentation	664
14.47.2.1 AAX_CUnitPrefixDisplayDelegateDecorator()	665
14.47.3 Member Function Documentation	665
14.47.3.1 Clone()	665
14.47.3.2 ValueToString() [1/2]	665
14.47.3.3 ValueToString() [2/2]	666
14.47.3.4 StringToValue()	666
14.48 AAX_FastInterpolatedTableLookup< TFLOAT, DFLOAT > Class Template Reference	667
14.48.1 Member Function Documentation	667
14.48.1.1 SetParameters()	667
14.48.1.2 DoTableLookupExtraFast() [1/2]	668
14.48.1.3 DoTableLookupExtraFastMulti()	668
14.48.1.4 DoTableLookupExtraFast() [2/2]	668
14.48.1.5 GetMin()	669
14.48.1.6 GetMaxMinusMin()	669
14.49 AAX_IACFAutomationDelegate Class Reference	669
14.49.1 Description	669
14.49.2 Member Function Documentation	669
14.49.2.1 RegisterParameter()	670
14.49.2.2 UnregisterParameter()	670
14.49.2.3 PostSetValueRequest()	670
14.49.2.4 PostCurrentValue()	671
14.49.2.5 PostTouchRequest()	671
14.49.2.6 PostReleaseRequest()	671
14.49.2.7 GetTouchState()	672
14.50 AAX_IACFCollection Class Reference	672
14.50.1 Description	672
14.50.2 Member Function Documentation	672
14.50.2.1 AddEffect()	673
14.50.2.2 SetManufacturerName()	673
14.50.2.3 AddPackageName()	673
14.50.2.4 SetPackageVersion()	674
14.50.2.5 SetProperties()	674
14.51 AAX_IACFComponentDescriptor Class Reference	674
14.51.1 Description	674

14.51.2 Member Function Documentation	675
14.51.2.1 Clear()	676
14.51.2.2 AddReservedField()	676
14.51.2.3 AddAudioIn()	676
14.51.2.4 AddAudioOut()	677
14.51.2.5 AddAudioBufferLength()	677
14.51.2.6 AddSampleRate()	677
14.51.2.7 AddClock()	678
14.51.2.8 AddSideChainIn()	678
14.51.2.9 AddDataInPort()	679
14.51.2.10 AddAuxOutputStem()	679
14.51.2.11 AddPrivateData()	680
14.51.2.12 AddDmaInstance()	680
14.51.2.13 AddMIDINode()	681
14.51.2.14 AddProcessProc_Native()	682
14.51.2.15 AddProcessProc_TI()	682
14.51.2.16 AddMeters()	683
14.52 AAX_IACFComponentDescriptor_V2 Class Reference	683
14.52.1 Description	683
14.52.2 Member Function Documentation	684
14.52.2.1 AddTemporaryData()	684
14.53 AAX_IACFComponentDescriptor_V3 Class Reference	684
14.53.1 Description	684
14.53.2 Member Function Documentation	685
14.53.2.1 AddProcessProc()	685
14.54 AAX_IACFController Class Reference	686
14.54.1 Description	686
14.54.2 Member Function Documentation	687
14.54.2.1 GetEffectID()	687
14.54.2.2 GetSampleRate()	687
14.54.2.3 GetInputStemFormat()	688
14.54.2.4 GetOutputStemFormat()	688
14.54.2.5 GetSignalLatency()	688
14.54.2.6 GetCycleCount()	689
14.54.2.7 GetTODLocation()	689
14.54.2.8 SetSignalLatency()	690
14.54.2.9 SetCycleCount()	690
14.54.2.10 PostPacket()	691
14.54.2.11 GetCurrentMeterValue()	692
14.54.2.12 GetMeterPeakValue()	692
14.54.2.13 ClearMeterPeakValue()	692
14.54.2.14 GetMeterClipped()	693

14.54.2.15 ClearMeterClipped()	693
14.54.2.16 GetMeterCount()	693
14.54.2.17 GetNextMIDIPacket()	694
14.55 AAX_IACFController_V2 Class Reference	694
14.55.1 Description	694
14.55.2 Member Function Documentation	694
14.55.2.1 SendNotification()	695
14.55.2.2 GetHybridSignalLatency()	695
14.55.2.3 GetCurrentAutomationTimestamp()	696
14.55.2.4 GetHostName()	696
14.56 AAX_IACFController_V3 Class Reference	696
14.56.1 Description	697
14.56.2 Member Function Documentation	697
14.56.2.1 GetPlugInTargetPlatform()	697
14.56.2.2 GetIsAudioSuite()	697
14.57 AAX_IACFDataBuffer Class Reference	698
14.57.1 Description	698
14.57.2 Member Function Documentation	698
14.57.2.1 Type()	698
14.57.2.2 Size()	698
14.57.2.3 Data()	699
14.58 AAX_IACFDescriptionHost Class Reference	699
14.58.1 Description	699
14.58.2 Member Function Documentation	699
14.58.2.1 AcquireFeatureProperties()	699
14.59 AAX_IACFEffectDescriptor Class Reference	700
14.59.1 Description	700
14.59.2 Member Function Documentation	700
14.59.2.1 AddComponent()	700
14.59.2.2 AddName()	701
14.59.2.3 AddCategory()	701
14.59.2.4 AddCategoryBypassParameter()	701
14.59.2.5 AddProcPtr()	702
14.59.2.6 SetProperties()	702
14.59.2.7 AddResourceInfo()	702
14.59.2.8 AddMeterDescription()	703
14.60 AAX_IACFEffectDescriptor_V2 Class Reference	703
14.60.1 Description	703
14.60.2 Member Function Documentation	703
14.60.2.1 AddControlMIDINode()	704
14.61 AAX_IACFEffectDirectData Class Reference	704
14.61.1 Description	704

14.61.2 Member Function Documentation	705
14.61.2.1 Initialize()	705
14.61.2.2 Uninitialize()	705
14.61.2.3 TimerWakeup()	706
14.62 AAX_IACEffectDirectData_V2 Class Reference	706
14.62.1 Member Function Documentation	706
14.62.1.1 NotificationReceived()	707
14.63 AAX_IACEffectGUI Class Reference	707
14.63.1 Description	708
14.63.2 Member Function Documentation	709
14.63.2.1 Initialize()	709
14.63.2.2 Uninitialize()	709
14.63.2.3 NotificationReceived()	710
14.63.2.4 SetViewContainer()	710
14.63.2.5 GetViewSize()	711
14.63.2.6 Draw()	711
14.63.2.7 TimerWakeup()	711
14.63.2.8 ParameterUpdated()	712
14.63.2.9 GetCustomLabel()	712
14.63.2.10 SetControlHighlightInfo()	712
14.64 AAX_IACEffectParameters Class Reference	713
14.64.1 Description	713
14.64.2 Member Function Documentation	717
14.64.2.1 Initialize()	717
14.64.2.2 Uninitialize()	717
14.64.2.3 NotificationReceived()	718
14.64.2.4 GetNumberOfParameters()	718
14.64.2.5 GetMasterBypassParameter()	719
14.64.2.6 GetParameterIsAutomatable()	719
14.64.2.7 GetParameterNumberOfSteps()	719
14.64.2.8 GetParameterName()	720
14.64.2.9 GetParameterNameOfLength()	720
14.64.2.10 GetParameterDefaultNormalizedValue()	721
14.64.2.11 SetParameterDefaultNormalizedValue()	721
14.64.2.12 GetParameterType()	721
14.64.2.13 GetParameterOrientation()	722
14.64.2.14 GetParameter()	722
14.64.2.15 GetParameterIndex()	723
14.64.2.16 GetParameterIDFromIndex()	723
14.64.2.17 GetParameterValueInfo()	724
14.64.2.18 GetParameterValueFromString()	724
14.64.2.19 GetParameterStringFromValue()	725

14.64.2.20	GetParameterValueString()	725
14.64.2.21	GetParameterNormalizedValue()	726
14.64.2.22	SetParameterNormalizedValue()	726
14.64.2.23	SetParameterNormalizedRelative()	727
14.64.2.24	TouchParameter()	727
14.64.2.25	ReleaseParameter()	728
14.64.2.26	UpdateParameterTouch()	728
14.64.2.27	UpdateParameterNormalizedValue()	729
14.64.2.28	UpdateParameterNormalizedRelative()	729
14.64.2.29	GenerateCoefficients()	730
14.64.2.30	ResetFieldData()	730
14.64.2.31	GetNumberOfChunks()	731
14.64.2.32	GetChunkIDFromIndex()	731
14.64.2.33	GetChunkSize()	731
14.64.2.34	GetChunk()	732
14.64.2.35	SetChunk()	733
14.64.2.36	CompareActiveChunk()	733
14.64.2.37	TimerWakeup()	734
14.64.2.38	GetCustomData()	734
14.64.2.39	SetCustomData()	734
14.64.2.40	DoMIDITransfers()	735
14.65	AAX_IACFEffEffectParameters_V2 Class Reference	735
14.65.1	Description	735
14.65.2	Member Function Documentation	736
14.65.2.1	UpdateMIDINodes()	736
14.65.2.2	UpdateControlMIDINodes()	736
14.66	AAX_IACFEffEffectParameters_V3 Class Reference	737
14.66.1	Description	737
14.67	AAX_IACFEffEffectParameters_V4 Class Reference	738
14.67.1	Description	738
14.67.2	Member Function Documentation	738
14.67.2.1	UpdatePageTable()	739
14.68	AAX_IACFFeatureInfo Class Reference	739
14.68.1	Description	740
14.68.2	Member Function Documentation	740
14.68.2.1	SupportLevel()	740
14.68.2.2	AcquireProperties()	741
14.69	AAX_IACFHostProcessor Class Reference	741
14.69.1	Description	741
14.69.2	Member Function Documentation	742
14.69.2.1	Initialize()	742
14.69.2.2	Uninitialize()	742

14.69.2.3 InitOutputBounds()	742
14.69.2.4 SetLocation()	743
14.69.2.5 RenderAudio()	743
14.69.2.6 PreRender()	744
14.69.2.7 PostRender()	745
14.69.2.8 AnalyzeAudio()	745
14.69.2.9 PreAnalyze()	745
14.69.2.10 PostAnalyze()	746
14.70 AAX_IACFHostProcessor_V2 Class Reference	746
14.70.1 Description	747
14.70.2 Member Function Documentation	747
14.70.2.1 GetClipNameSuffix()	747
14.71 AAX_IACFHostProcessorDelegate Class Reference	747
14.71.1 Description	748
14.71.2 Member Function Documentation	748
14.71.2.1 GetAudio()	748
14.71.2.2 GetSideChainInputNum()	749
14.72 AAX_IACFHostProcessorDelegate_V2 Class Reference	749
14.72.1 Description	749
14.72.2 Member Function Documentation	749
14.72.2.1 ForceAnalyze()	749
14.73 AAX_IACFHostProcessorDelegate_V3 Class Reference	750
14.73.1 Description	750
14.73.2 Member Function Documentation	750
14.73.2.1 ForceProcess()	750
14.74 AAX_IACFHostServices Class Reference	750
14.74.1 Description	750
14.74.2 Member Function Documentation	751
14.74.2.1 Assert()	751
14.74.2.2 Trace()	751
14.75 AAX_IACFHostServices_V2 Class Reference	752
14.75.1 Description	752
14.75.2 Member Function Documentation	752
14.75.2.1 StackTrace()	752
14.76 AAX_IACFHostServices_V3 Class Reference	753
14.76.1 Description	753
14.76.2 Member Function Documentation	753
14.76.2.1 HandleAssertFailure()	753
14.77 AAX_IACFPageTable Class Reference	754
14.77.1 Description	754
14.77.2 Member Function Documentation	754
14.77.2.1 Clear()	754

14.77.2.2 Empty()	755
14.77.2.3 GetNumPages()	755
14.77.2.4 InsertPage()	755
14.77.2.5 RemovePage()	756
14.77.2.6 GetNumMappedParameterIDs()	756
14.77.2.7 ClearMappedParameter()	756
14.77.2.8 GetMappedParameterID()	757
14.77.2.9 MapParameterID()	757
14.78 AAX_IACFPageTable_V2 Class Reference	758
14.78.1 Description	758
14.78.2 Member Function Documentation	758
14.78.2.1 GetNumParametersWithNameVariations()	759
14.78.2.2 GetNameVariationParameterIDAtIndex()	759
14.78.2.3 GetNumNameVariationsForParameter()	760
14.78.2.4 GetParameterNameVariationAtIndex()	760
14.78.2.5 GetParameterNameVariationOfLength()	761
14.78.2.6 ClearParameterNameVariations()	762
14.78.2.7 ClearNameVariationsForParameter()	762
14.78.2.8 SetParameterNameVariation()	763
14.79 AAX_IACFPageTableController Class Reference	763
14.79.1 Description	764
14.79.2 Member Function Documentation	764
14.79.2.1 CopyTableForEffect()	764
14.79.2.2 CopyTableOfLayoutForEffect()	765
14.80 AAX_IACFPageTableController_V2 Class Reference	766
14.80.1 Description	766
14.80.2 Member Function Documentation	766
14.80.2.1 CopyTableForEffectFromFile()	766
14.80.2.2 CopyTableOfLayoutFromFile()	767
14.81 AAX_IACFPrivateDataAccess Class Reference	768
14.81.1 Description	768
14.81.2 Member Function Documentation	768
14.81.2.1 ReadPortDirect()	768
14.81.2.2 WritePortDirect()	769
14.82 AAX_IACFPropertyMap Class Reference	769
14.82.1 Description	769
14.82.2 Member Function Documentation	770
14.82.2.1 GetProperty()	770
14.82.2.2 AddProperty()	770
14.82.2.3 RemoveProperty()	771
14.83 AAX_IACFPropertyMap_V2 Class Reference	771
14.83.1 Description	771

14.83.2 Member Function Documentation	771
14.83.2.1 AddPropertyWithIDArray()	771
14.83.2.2 GetPropertyWithIDArray()	772
14.84 AAX_IACFPropertyMap_V3 Class Reference	772
14.84.1 Description	772
14.84.2 Member Function Documentation	773
14.84.2.1 GetProperty64()	773
14.84.2.2 AddProperty64()	773
14.85 AAX_IACFSessionDocument Class Reference	773
14.85.1 Description	774
14.85.2 Member Function Documentation	774
14.85.2.1 GetDocumentData()	774
14.86 AAX_IACFSessionDocumentClient Class Reference	775
14.86.1 Description	775
14.86.2 Member Function Documentation	775
14.86.2.1 Initialize()	775
14.86.2.2 Uninitialize()	776
14.86.2.3 SetSessionDocument()	776
14.86.2.4 NotificationReceived()	776
14.87 AAX_IACFTask Class Reference	777
14.87.1 Description	777
14.87.2 Member Function Documentation	777
14.87.2.1 GetType()	777
14.87.2.2 GetArgumentOfType()	778
14.87.2.3 SetProgress()	778
14.87.2.4 GetProgress()	778
14.87.2.5 AddResult()	779
14.87.2.6 SetDone()	779
14.88 AAX_IACFTaskAgent Class Reference	780
14.88.1 Description	780
14.88.2 Member Function Documentation	780
14.88.2.1 Initialize()	780
14.88.2.2 Uninitialize()	781
14.88.2.3 AddTask()	781
14.88.2.4 CancelAllTasks()	781
14.89 AAX_IACFTransport Class Reference	781
14.89.1 Description	782
14.89.2 Member Function Documentation	782
14.89.2.1 GetCurrentTempo()	782
14.89.2.2 GetCurrentMeter()	783
14.89.2.3 IsTransportPlaying()	783
14.89.2.4 GetCurrentTickPosition()	783

14.89.2.5 GetCurrentLoopPosition()	784
14.89.2.6 GetCurrentNativeSampleLocation()	784
14.89.2.7 GetCustomTickPosition()	785
14.89.2.8 GetBarBeatPosition()	785
14.89.2.9 GetTicksPerQuarter()	786
14.89.2.10 GetCurrentTicksPerBeat()	786
14.90 AAX_IACFTransport_V2 Class Reference	786
14.90.1 Description	786
14.90.2 Member Function Documentation	787
14.90.2.1 GetTimelineSelectionStartPosition()	787
14.90.2.2 GetTimeCodeInfo()	787
14.90.2.3 GetFeetFramesInfo()	787
14.90.2.4 IsMetronomeEnabled()	788
14.91 AAX_IACFTransport_V3 Class Reference	788
14.91.1 Description	788
14.91.2 Member Function Documentation	789
14.91.2.1 GetHDTTimeCodeInfo()	789
14.92 AAX_IACFTransport_V4 Class Reference	789
14.92.1 Description	789
14.92.2 Member Function Documentation	789
14.92.2.1 GetTimelineSelectionEndPosition()	790
14.93 AAX_IACFTransport_V5 Class Reference	790
14.93.1 Description	790
14.93.2 Member Function Documentation	790
14.93.2.1 GetKeySignature()	791
14.94 AAX_IACFTransportControl Class Reference	791
14.94.1 Description	791
14.94.2 Member Function Documentation	792
14.94.2.1 RequestTransportStart()	792
14.94.2.2 RequestTransportStop()	792
14.95 AAX_IACFViewContainer Class Reference	792
14.95.1 Description	792
14.95.2 Member Function Documentation	793
14.95.2.1 GetType()	793
14.95.2.2 GetPtr()	793
14.95.2.3 GetModifiers()	793
14.95.2.4 SetViewSize()	794
14.95.2.5 HandleParameterMouseDown()	794
14.95.2.6 HandleParameterMouseDrag()	794
14.95.2.7 HandleParameterMouseUp()	795
14.96 AAX_IACFViewContainer_V2 Class Reference	795
14.96.1 Description	795

14.96.2 Member Function Documentation	796
14.96.2.1 HandleMultipleParametersMouseDown()	796
14.96.2.2 HandleMultipleParametersMouseDownDrag()	796
14.96.2.3 HandleMultipleParametersMouseDownUp()	797
14.97 AAX_IACFViewContainer_V3 Class Reference	797
14.97.1 Description	797
14.97.2 Member Function Documentation	798
14.97.2.1 HandleParameterMouseEnter()	798
14.97.2.2 HandleParameterMouseExit()	798
14.98 AAX_IAutomationDelegate Class Reference	799
14.98.1 Description	799
14.98.2 Constructor & Destructor Documentation	799
14.98.2.1 ~AAX_IAutomationDelegate()	799
14.98.3 Member Function Documentation	799
14.98.3.1 RegisterParameter()	800
14.98.3.2 UnregisterParameter()	800
14.98.3.3 PostSetValueRequest()	801
14.98.3.4 PostCurrentValue()	801
14.98.3.5 PostTouchRequest()	801
14.98.3.6 PostReleaseRequest()	802
14.98.3.7 GetTouchState()	802
14.98.3.8 ParameterNameChanged()	802
14.99 AAX_ICollection Class Reference	803
14.99.1 Description	803
14.99.2 Constructor & Destructor Documentation	804
14.99.2.1 ~AAX_ICollection()	804
14.99.3 Member Function Documentation	804
14.99.3.1 NewDescriptor()	804
14.99.3.2 AddEffect()	804
14.99.3.3 SetManufacturerName()	805
14.99.3.4 AddPackageName()	805
14.99.3.5 SetPackageVersion()	805
14.99.3.6 NewPropertyMap()	806
14.99.3.7 SetProperties()	806
14.99.3.8 GetHostVersion()	806
14.99.3.9 DescriptionHost() [1/2]	807
14.99.3.10 DescriptionHost() [2/2]	807
14.99.3.11 HostDefinition()	808
14.100 AAX_IComponentDescriptor Class Reference	808
14.100.1 Description	808
14.100.2 Constructor & Destructor Documentation	810
14.100.2.1 ~AAX_IComponentDescriptor()	810

14.100.3 Member Function Documentation	810
14.100.3.1 Clear()	810
14.100.3.2 AddAudioIn()	810
14.100.3.3 AddAudioOut()	811
14.100.3.4 AddAudioBufferLength()	811
14.100.3.5 AddSampleRate()	812
14.100.3.6 AddClock()	812
14.100.3.7 AddSideChainIn()	813
14.100.3.8 AddDataInPort()	813
14.100.3.9 AddAuxOutputStem()	814
14.100.3.10 AddPrivateData()	815
14.100.3.11 AddTemporaryData()	815
14.100.3.12 AddDmaInstance()	816
14.100.3.13 AddMeters()	816
14.100.3.14 AddMIDINode()	817
14.100.3.15 AddReservedField()	817
14.100.3.16 NewPropertyMap()	818
14.100.3.17 DuplicatePropertyMap()	818
14.100.3.18 AddProcessProc_Native() [1/2]	818
14.100.3.19 AddProcessProc_TI()	820
14.100.3.20 AddProcessProc()	821
14.100.3.21 AddProcessProc_Native() [2/2]	822
14.101 AAX_IContainer Class Reference	823
14.101.1 Description	823
14.101.2 Member Enumeration Documentation	823
14.101.2.1 EStatus	823
14.101.3 Constructor & Destructor Documentation	823
14.101.3.1 ~AAX_IContainer()	824
14.101.4 Member Function Documentation	824
14.101.4.1 Clear()	824
14.102 AAX_IController Class Reference	824
14.102.1 Description	824
14.102.2 Constructor & Destructor Documentation	826
14.102.2.1 ~AAX_IController()	826
14.102.3 Member Function Documentation	827
14.102.3.1 GetEffectID()	827
14.102.3.2 GetSampleRate()	827
14.102.3.3 GetInputStemFormat()	827
14.102.3.4 GetOutputStemFormat()	827
14.102.3.5 GetSignalLatency()	828
14.102.3.6 GetCycleCount()	828
14.102.3.7 GetTODLocation()	829

14.102.3.8 SetSignalLatency()	829
14.102.3.9 SetCycleCount()	830
14.102.3.10 PostPacket()	831
14.102.3.11 SendNotification() [1/2]	832
14.102.3.12 SendNotification() [2/2]	832
14.102.3.13 GetCurrentMeterValue()	833
14.102.3.14 GetMeterPeakValue()	833
14.102.3.15 ClearMeterPeakValue()	833
14.102.3.16 GetMeterCount()	834
14.102.3.17 GetMeterClipped()	834
14.102.3.18 ClearMeterClipped()	834
14.102.3.19 GetNextMIDIPacket()	835
14.102.3.20 GetCurrentAutomationTimestamp()	835
14.102.3.21 GetHostName()	836
14.102.3.22 GetPlugInTargetPlatform()	836
14.102.3.23 GetIsAudioSuite()	836
14.102.3.24 CreateTableCopyForEffect()	837
14.102.3.25 CreateTableCopyForLayout()	837
14.102.3.26 CreateTableCopyForEffectFromFile()	839
14.102.3.27 CreateTableCopyForLayoutFromFile()	839
14.103 AAX_IDataBuffer Class Reference	840
14.103.1 Description	840
14.103.2 Member Function Documentation	841
14.103.2.1 ACF_DECLARE_STANDARD_UNKNOWN()	841
14.103.2.2 AAX_DELETE()	841
14.103.3 Member Data Documentation	841
14.103.3.1 AAX_OVERRIDE	841
14.104 AAX_IDataBufferWrapper Class Reference	841
14.104.1 Description	842
14.104.2 Constructor & Destructor Documentation	842
14.104.2.1 ~AAX_IDataBufferWrapper()	842
14.104.3 Member Function Documentation	842
14.104.3.1 Type()	842
14.104.3.2 Size()	842
14.104.3.3 Data()	843
14.105 AAX_IDescriptionHost Class Reference	843
14.105.1 Description	843
14.105.2 Constructor & Destructor Documentation	843
14.105.2.1 ~AAX_IDescriptionHost()	843
14.105.3 Member Function Documentation	843
14.105.3.1 AcquireFeatureProperties()	844
14.106 AAX_IDisplayDelegate< T > Class Template Reference	844

14.106.1 Description	845
14.106.2 Display delegate decorators	845
14.106.2.1 Display delegate decorator implementation	845
14.106.2.2 Decibel decorator example	846
14.106.3 Member Function Documentation	846
14.106.3.1 Clone()	846
14.106.3.2 ValueToString() [1/2]	847
14.106.3.3 ValueToString() [2/2]	847
14.106.3.4 StringToValue()	848
14.107 AAX_IDisplayDelegateBase Class Reference	848
14.107.1 Description	848
14.107.2 Constructor & Destructor Documentation	849
14.107.2.1 ~AAX_IDisplayDelegateBase()	849
14.108 AAX_IDisplayDelegateDecorator< T > Class Template Reference	849
14.108.1 Description	849
14.108.2 Constructor & Destructor Documentation	850
14.108.2.1 AAX_IDisplayDelegateDecorator() [1/2]	850
14.108.2.2 AAX_IDisplayDelegateDecorator() [2/2]	850
14.108.2.3 ~AAX_IDisplayDelegateDecorator()	851
14.108.3 Member Function Documentation	851
14.108.3.1 Clone()	851
14.108.3.2 ValueToString() [1/2]	852
14.108.3.3 ValueToString() [2/2]	852
14.108.3.4 StringToValue()	853
14.109 AAX_IDma Class Reference	853
14.109.1 Description	854
14.109.2 Member Enumeration Documentation	855
14.109.2.1 EState	855
14.109.2.2 EMode	856
14.109.3 Constructor & Destructor Documentation	856
14.109.3.1 ~AAX_IDma()	856
14.109.4 Member Function Documentation	856
14.109.4.1 PostRequest()	856
14.109.4.2 IsTransferComplete()	857
14.109.4.3 SetDmaState()	857
14.109.4.4 GetDmaState()	858
14.109.4.5 GetDmaMode()	858
14.109.4.6 SetSrc()	858
14.109.4.7 GetSrc()	858
14.109.4.8 SetDst()	858
14.109.4.9 GetDst()	859
14.109.4.10 SetBurstLength()	859

14.109.4.11 GetBurstLength()	859
14.109.4.12 SetNumBursts()	859
14.109.4.13 GetNumBursts()	860
14.109.4.14 SetTransferSize()	860
14.109.4.15 GetTransferSize()	860
14.109.4.16 SetFifoBuffer()	861
14.109.4.17 GetFifoBuffer()	861
14.109.4.18 SetLinearBuffer()	861
14.109.4.19 GetLinearBuffer()	861
14.109.4.20 SetOffsetTable()	862
14.109.4.21 GetOffsetTable()	862
14.109.4.22 SetNumOffsets()	862
14.109.4.23 GetNumOffsets()	863
14.109.4.24 SetBaseOffset()	863
14.109.4.25 GetBaseOffset()	863
14.109.4.26 SetFifoSize()	863
14.109.4.27 GetFifoSize()	864
14.110 AAX_IEffectDescriptor Class Reference	864
14.110.1 Description	864
14.110.2 Constructor & Destructor Documentation	865
14.110.2.1 ~AAX_IEffectDescriptor()	865
14.110.3 Member Function Documentation	865
14.110.3.1 NewComponentDescriptor()	865
14.110.3.2 AddComponent()	865
14.110.3.3 AddName()	866
14.110.3.4 AddCategory()	866
14.110.3.5 AddCategoryBypassParameter()	866
14.110.3.6 AddProcPtr()	867
14.110.3.7 NewPropertyMap()	867
14.110.3.8 SetProperties()	867
14.110.3.9 AddResourceInfo()	867
14.110.3.10 AddMeterDescription()	868
14.110.3.11 AddControlMIDINode()	868
14.111 AAX_IEffectDirectData Class Reference	869
14.111.1 Description	869
14.111.2 Member Function Documentation	870
14.111.2.1 ACF_DECLARE_STANDARD_UNKNOWN()	870
14.111.2.2 AAX_DELETE()	870
14.111.3 Member Data Documentation	870
14.111.3.1 override	870
14.112 AAX_IEffectGUI Class Reference	870
14.112.1 Description	871

14.112.2 Member Function Documentation	871
14.112.2.1 ACF_DECLARE_STANDARD_UNKNOWN()	871
14.112.2.2 AAX_DELETE()	871
14.112.3 Member Data Documentation	871
14.112.3.1 override	872
14.113 AAX_IEffectParameters Class Reference	872
14.113.1 Description	872
14.113.2 Related classes	873
14.113.3 Member Function Documentation	873
14.113.3.1 ACF_DECLARE_STANDARD_UNKNOWN()	874
14.113.3.2 AAX_DELETE()	874
14.113.4 Member Data Documentation	874
14.113.4.1 override	874
14.114 AAX_IFeatureInfo Class Reference	874
14.114.1 Constructor & Destructor Documentation	874
14.114.1.1 ~AAX_IFeatureInfo()	874
14.114.2 Member Function Documentation	875
14.114.2.1 SupportLevel()	875
14.114.2.2 AcquireProperties()	875
14.114.2.3 ID()	876
14.115 AAX_IHostProcessor Class Reference	876
14.115.1 Description	876
14.115.2 Member Function Documentation	876
14.115.2.1 ACF_DECLARE_STANDARD_UNKNOWN()	877
14.115.2.2 AAX_DELETE()	877
14.115.3 Member Data Documentation	877
14.115.3.1 override	877
14.116 AAX_IHostProcessorDelegate Class Reference	877
14.116.1 Description	877
14.116.2 Constructor & Destructor Documentation	878
14.116.2.1 ~AAX_IHostProcessorDelegate()	878
14.116.3 Member Function Documentation	878
14.116.3.1 GetAudio()	878
14.116.3.2 GetSideChainInputNum()	879
14.116.3.3 ForceAnalyze()	879
14.116.3.4 ForceProcess()	880
14.117 AAX_IHostServices Class Reference	880
14.117.1 Description	880
14.117.2 Constructor & Destructor Documentation	880
14.117.2.1 ~AAX_IHostServices()	881
14.117.3 Member Function Documentation	881
14.117.3.1 HandleAssertFailure()	881

14.117.3.2 Trace()	881
14.117.3.3 StackTrace()	882
14.118 AAX_IHostTaskAgent Class Reference	882
14.118.1 Description	882
14.118.2 Constructor & Destructor Documentation	883
14.118.2.1 ~AAX_IHostTaskAgent()	883
14.118.3 Member Function Documentation	883
14.118.3.1 Initialize()	883
14.118.3.2 Uninitialize()	883
14.118.3.3 AddTask()	883
14.118.3.4 CancelAllTasks()	884
14.119 AAX_IMIDIMessageInfoDelegate Class Reference	884
14.119.1 Constructor & Destructor Documentation	884
14.119.1.1 ~AAX_IMIDIMessageInfoDelegate()	884
14.119.2 Member Function Documentation	884
14.119.2.1 Mask()	885
14.119.2.2 Length()	885
14.119.2.3 ToString()	885
14.119.2.4 Accepts()	885
14.119.2.5 Accepts_ExactStatus()	885
14.119.2.6 ToString_AppendNumber()	886
14.119.2.7 ToString_AppendCStr()	886
14.119.2.8 ToString_AppendByteRange()	886
14.119.2.9 ToString_AppendValid()	886
14.120 AAX_IMIDINode Class Reference	887
14.120.1 Description	887
14.120.2 Constructor & Destructor Documentation	887
14.120.2.1 ~AAX_IMIDINode()	887
14.120.3 Member Function Documentation	887
14.120.3.1 GetNodeBuffer()	887
14.120.3.2 PostMIDIPacket()	888
14.120.3.3 GetTransport()	888
14.121 AAX_IPacketHandler Struct Reference	888
14.121.1 Description	889
14.121.2 Constructor & Destructor Documentation	889
14.121.2.1 ~AAX_IPacketHandler()	889
14.121.3 Member Function Documentation	889
14.121.3.1 Clone()	889
14.121.3.2 Call()	889
14.122 AAX_IPageTable Class Reference	890
14.122.1 Description	890
14.122.2 Constructor & Destructor Documentation	891

14.122.2.1 ~AAX_IPageTable()	891
14.122.3 Member Function Documentation	891
14.122.3.1 Clear()	891
14.122.3.2 Empty()	891
14.122.3.3 GetNumPages()	892
14.122.3.4 InsertPage()	892
14.122.3.5 RemovePage()	892
14.122.3.6 GetNumMappedParameterIDs()	893
14.122.3.7 ClearMappedParameter()	893
14.122.3.8 GetMappedParameterID()	894
14.122.3.9 MapParameterID()	894
14.122.3.10 GetNumParametersWithNameVariations()	895
14.122.3.11 GetNameVariationParameterIDAtIndex()	895
14.122.3.12 GetNumNameVariationsForParameter()	896
14.122.3.13 GetParameterNameVariationAtIndex()	896
14.122.3.14 GetParameterNameVariationOfLength()	897
14.122.3.15 ClearParameterNameVariations()	898
14.122.3.16 ClearNameVariationsForParameter()	898
14.122.3.17 SetParameterNameVariation()	899
14.123 AAX_IParameter Class Reference	900
14.123.1 Description	900
14.123.2 Constructor & Destructor Documentation	902
14.123.2.1 ~AAX_IParameter()	902
14.123.3 Member Function Documentation	903
14.123.3.1 CloneValue()	903
14.123.3.2 Identifier()	903
14.123.3.3 SetName()	903
14.123.3.4 Name()	904
14.123.3.5 AddShortenedName()	904
14.123.3.6 ShortenedName()	904
14.123.3.7 ClearShortenedNames()	905
14.123.3.8 Automatable()	905
14.123.3.9 SetAutomationDelegate()	905
14.123.3.10 Touch()	905
14.123.3.11 Release()	906
14.123.3.12 SetNormalizedValue()	906
14.123.3.13 GetNormalizedValue()	906
14.123.3.14 SetNormalizedDefaultValue()	906
14.123.3.15 GetNormalizedDefaultValue()	907
14.123.3.16 SetToDefaultValue()	907
14.123.3.17 SetNumberOfSteps()	907
14.123.3.18 GetNumberOfSteps()	907

14.123.3.19	GetStepValue()	908
14.123.3.20	GetNormalizedValueFromStep()	908
14.123.3.21	GetStepValueFromNormalizedValue()	908
14.123.3.22	SetStepValue()	908
14.123.3.23	GetValueString() [1/2]	908
14.123.3.24	GetValueString() [2/2]	909
14.123.3.25	GetNormalizedValueFromBool()	909
14.123.3.26	GetNormalizedValueFromInt32()	910
14.123.3.27	GetNormalizedValueFromFloat()	910
14.123.3.28	GetNormalizedValueFromDouble()	911
14.123.3.29	GetNormalizedValueFromString()	911
14.123.3.30	GetBoolFromNormalizedValue()	912
14.123.3.31	GetInt32FromNormalizedValue()	912
14.123.3.32	GetFloatFromNormalizedValue()	912
14.123.3.33	GetDoubleFromNormalizedValue()	913
14.123.3.34	GetStringFromNormalizedValue() [1/2]	913
14.123.3.35	GetStringFromNormalizedValue() [2/2]	914
14.123.3.36	SetValueFromString()	914
14.123.3.37	GetValueAsBool()	915
14.123.3.38	GetValueAsInt32()	915
14.123.3.39	GetValueAsFloat()	916
14.123.3.40	GetValueAsDouble()	916
14.123.3.41	GetValueAsString()	916
14.123.3.42	SetValueWithBool()	917
14.123.3.43	SetValueWithInt32()	917
14.123.3.44	SetValueWithFloat()	918
14.123.3.45	SetValueWithDouble()	918
14.123.3.46	SetValueWithString()	918
14.123.3.47	SetType()	919
14.123.3.48	GetType()	919
14.123.3.49	SetOrientation()	919
14.123.3.50	GetOrientation()	920
14.123.3.51	SetTaperDelegate()	920
14.123.3.52	SetDisplayDelegate()	920
14.123.3.53	UpdateNormalizedValue()	921
14.124	AAX_IParаметerValue Class Reference	921
14.124.1	Description	921
14.124.2	Constructor & Destructor Documentation	922
14.124.2.1	~AAX_IParаметerValue()	922
14.124.3	Member Function Documentation	922
14.124.3.1	Clone()	922
14.124.3.2	Identifier()	923

14.124.3.3 GetValueAsBool()	923
14.124.3.4 GetValueAsInt32()	923
14.124.3.5 GetValueAsFloat()	924
14.124.3.6 GetValueAsDouble()	924
14.124.3.7 GetValueAsString()	924
14.125 AAX_IPointerQueue< T > Class Template Reference	925
14.125.1 Description	925
14.125.2 Member Typedef Documentation	926
14.125.2.1 template_type	926
14.125.2.2 value_type	926
14.125.3 Constructor & Destructor Documentation	926
14.125.3.1 ~AAX_IPointerQueue()	926
14.125.4 Member Function Documentation	926
14.125.4.1 Clear()	926
14.125.4.2 Push()	927
14.125.4.3 Pop()	927
14.125.4.4 Peek()	927
14.126 AAX_IPrivateDataAccess Class Reference	928
14.126.1 Description	928
14.126.2 Constructor & Destructor Documentation	928
14.126.2.1 ~AAX_IPrivateDataAccess()	928
14.126.3 Member Function Documentation	928
14.126.3.1 ReadPortDirect()	928
14.126.3.2 WritePortDirect()	929
14.127 AAX_IPropertyMap Class Reference	929
14.127.1 Description	930
14.127.2 Constructor & Destructor Documentation	930
14.127.2.1 ~AAX_IPropertyMap()	930
14.127.3 Member Function Documentation	931
14.127.3.1 GetProperty()	931
14.127.3.2 GetPointerProperty()	931
14.127.3.3 AddProperty()	931
14.127.3.4 AddPointerProperty() [1/2]	932
14.127.3.5 AddPointerProperty() [2/2]	932
14.127.3.6 RemoveProperty()	933
14.127.3.7 AddPropertyWithIDArray()	933
14.127.3.8 GetPropertyWithIDArray()	934
14.127.3.9 GetUnknown()	934
14.128 AAX_ISessionDocument Class Reference	934
14.128.1 Description	934
14.128.2 Constructor & Destructor Documentation	935
14.128.2.1 ~AAX_ISessionDocument()	935

14.128.3 Member Function Documentation	935
14.128.3.1 Valid()	935
14.128.3.2 GetTempoMap()	935
14.128.3.3 GetDocumentData()	936
14.129 AAX_ISessionDocumentClient Class Reference	936
14.129.1 Description	936
14.129.2 Member Function Documentation	937
14.129.2.1 ACF_DECLARE_STANDARD_UNKNOWN()	937
14.129.2.2 AAX_DELETE()	937
14.129.3 Member Data Documentation	937
14.129.3.1 override	937
14.130 AAX_IString Class Reference	937
14.130.1 Description	937
14.130.2 Constructor & Destructor Documentation	938
14.130.2.1 ~AAX_IString()	938
14.130.3 Member Function Documentation	938
14.130.3.1 Length()	938
14.130.3.2 MaxLength()	938
14.130.3.3 Get()	939
14.130.3.4 Set()	939
14.130.3.5 operator=() [1/2]	939
14.130.3.6 operator=() [2/2]	939
14.131 AAX_ITaperDelegate< T > Class Template Reference	939
14.131.1 Description	940
14.131.2 Member Function Documentation	940
14.131.2.1 Clone()	940
14.131.2.2 GetMaximumValue()	941
14.131.2.3 GetMinimumValue()	941
14.131.2.4 ConstrainRealValue()	941
14.131.2.5 NormalizedToReal()	941
14.131.2.6 RealToNormalized()	942
14.132 AAX_ITaperDelegateBase Class Reference	942
14.132.1 Description	943
14.132.2 Constructor & Destructor Documentation	943
14.132.2.1 ~AAX_ITaperDelegateBase()	944
14.133 AAX_ITask Class Reference	944
14.133.1 Description	944
14.133.2 Constructor & Destructor Documentation	945
14.133.2.1 ~AAX_ITask()	945
14.133.3 Member Function Documentation	945
14.133.3.1 GetType()	945
14.133.3.2 GetArgumentOfType()	945

14.133.3.3 SetProgress()	946
14.133.3.4 GetProgress()	946
14.133.3.5 AddResult()	946
14.133.3.6 SetDone()	946
14.134 AAX_ITaskAgent Class Reference	947
14.134.1 Description	947
14.134.2 Member Function Documentation	948
14.134.2.1 ACF_DECLARE_STANDARD_UNKNOWN()	948
14.134.2.2 AAX_DELETE()	948
14.134.3 Member Data Documentation	948
14.134.3.1 AAX_OVERRIDE	948
14.135 AAX_ITransport Class Reference	948
14.135.1 Description	948
14.135.2 Constructor & Destructor Documentation	949
14.135.2.1 ~AAX_ITransport()	950
14.135.3 Member Function Documentation	950
14.135.3.1 GetCurrentTempo()	950
14.135.3.2 GetCurrentMeter()	950
14.135.3.3 IsTransportPlaying()	951
14.135.3.4 GetCurrentTickPosition()	951
14.135.3.5 GetCurrentLoopPosition()	951
14.135.3.6 GetCurrentNativeSampleLocation()	952
14.135.3.7 GetCustomTickPosition()	952
14.135.3.8 GetBarBeatPosition()	953
14.135.3.9 GetTicksPerQuarter()	953
14.135.3.10 GetCurrentTicksPerBeat()	954
14.135.3.11 GetTimelineSelectionStartPosition()	954
14.135.3.12 GetTimeCodeInfo()	954
14.135.3.13 GetFeetFramesInfo()	955
14.135.3.14 IsMetronomeEnabled()	955
14.135.3.15 GetHDTIMECodeInfo()	956
14.135.3.16 RequestTransportStart()	956
14.135.3.17 RequestTransportStop()	956
14.135.3.18 GetTimelineSelectionEndPosition()	956
14.135.3.19 GetKeySignature()	957
14.136 AAX_IViewContainer Class Reference	957
14.136.1 Description	958
14.136.2 Constructor & Destructor Documentation	959
14.136.2.1 ~AAX_IViewContainer()	959
14.136.3 Member Function Documentation	959
14.136.3.1 GetType()	959
14.136.3.2 GetPtr()	959

14.136.3.3	GetModifiers()	959
14.136.3.4	SetViewSize()	960
14.136.3.5	HandleParameterMouseDown()	960
14.136.3.6	HandleParameterMouseDown()	960
14.136.3.7	HandleParameterMouseUp()	961
14.136.3.8	HandleParameterMouseEnter()	961
14.136.3.9	HandleParameterMouseExit()	962
14.136.3.10	HandleMultipleParametersMouseDown()	962
14.136.3.11	HandleMultipleParametersMouseDown()	963
14.136.3.12	HandleMultipleParametersMouseUp()	963
14.137	AAX_Map Class Reference	964
14.137.1	Constructor & Destructor Documentation	964
14.137.1.1	AAX_Map()	964
14.137.1.2	~AAX_Map()	964
14.137.2	Member Function Documentation	964
14.137.2.1	SetCoefficients()	964
14.137.2.2	GetCoefficient()	965
14.137.2.3	GetUpperBoundIndex()	965
14.137.2.4	GetX()	965
14.137.2.5	GetY()	965
14.137.2.6	GetFirstX()	965
14.137.2.7	GetFirstY()	965
14.137.2.8	GetLastX()	965
14.137.2.9	GetLastY()	966
14.137.2.10	GetSize()	966
14.138	AAX_Point Struct Reference	966
14.138.1	Description	966
14.138.2	Constructor & Destructor Documentation	966
14.138.2.1	AAX_Point() [1/2]	966
14.138.2.2	AAX_Point() [2/2]	967
14.138.3	Member Data Documentation	967
14.138.3.1	vert	967
14.138.3.2	horz	967
14.139	AAX_Rect Struct Reference	967
14.139.1	Description	967
14.139.2	Constructor & Destructor Documentation	968
14.139.2.1	AAX_Rect() [1/2]	968
14.139.2.2	AAX_Rect() [2/2]	968
14.139.3	Member Data Documentation	968
14.139.3.1	top	968
14.139.3.2	left	968
14.139.3.3	width	968

14.139.3.4 height	969
14.140 AAX_SHybridRenderInfo Struct Reference	969
14.140.1 Description	969
14.140.2 Member Data Documentation	969
14.140.2.1 mAudioInputs	969
14.140.2.2 mNumAudioInputs	969
14.140.2.3 mAudioOutputs	970
14.140.2.4 mNumAudioOutputs	970
14.140.2.5 mNumSamples	970
14.140.2.6 mClock	970
14.141 AAX_SInstrumentPrivateData Struct Reference	970
14.141.1 Description	970
14.141.2 Member Data Documentation	971
14.141.2.1 mMonolithicParametersPtr	971
14.142 AAX_SInstrumentRenderInfo Struct Reference	971
14.142.1 Description	971
14.142.2 Member Data Documentation	972
14.142.2.1 mAudioInputs	972
14.142.2.2 mAudioOutputs	972
14.142.2.3 mNumSamples	972
14.142.2.4 mClock	972
14.142.2.5 mInputNode	972
14.142.2.6 mGlobalNode	973
14.142.2.7 mTransportNode	973
14.142.2.8 mAdditionalInputMIDINodes	973
14.142.2.9 mPrivateData	973
14.142.2.10 mMeters	973
14.142.2.11 mCurrentStateNum	973
14.143 AAX_SInstrumentSetupInfo Struct Reference	974
14.143.1 Description	974
14.143.2 Constructor & Destructor Documentation	975
14.143.2.1 AAX_SInstrumentSetupInfo()	975
14.143.3 Member Data Documentation	975
14.143.3.1 mNeedsGlobalMIDI	976
14.143.3.2 mGlobalMIDINodeName	976
14.143.3.3 mGlobalMIDIEventMask	976
14.143.3.4 mNeedsInputMIDI	976
14.143.3.5 mInputMIDINodeName	977
14.143.3.6 mInputMIDIChannelMask	977
14.143.3.7 mNumAdditionalInputMIDINodes	977
14.143.3.8 mNeedsTransport	977
14.143.3.9 mTransportMIDINodeName	978

14.143.3.10 mNumMeters	978
14.143.3.11 mMeterIDs	978
14.143.3.12 mNumAuxOutputStems	978
14.143.3.13 mAuxOutputStemNames	979
14.143.3.14 mAuxOutputStemFormats	979
14.143.3.15 mHybridInputStemFormat	979
14.143.3.16 mHybridOutputStemFormat	980
14.143.3.17 mInputStemFormat	980
14.143.3.18 mOutputStemFormat	980
14.143.3.19 mUseHostGeneratedGUI	980
14.143.3.20 mCanBypass	981
14.143.3.21 mManufacturerID	981
14.143.3.22 mProductID	981
14.143.3.23 mPluginID	981
14.143.3.24 mAudiosuiteID	981
14.143.3.25 mMultiMonoSupport	982
14.144 AAX_SPlugInChunk Struct Reference	982
14.144.1 Description	982
14.144.2 Member Data Documentation	983
14.144.2.1 fSize	983
14.144.2.2 fVersion	983
14.144.2.3 fManufacturerID	983
14.144.2.4 fProductID	983
14.144.2.5 fPluginID	983
14.144.2.6 fChunkID	984
14.144.2.7 fName	984
14.144.2.8 fData	984
14.145 AAX_SPlugInChunkHeader Struct Reference	984
14.145.1 Description	985
14.145.2 Member Data Documentation	985
14.145.2.1 fSize	986
14.145.2.2 fVersion	986
14.145.2.3 fManufacturerID	986
14.145.2.4 fProductID	986
14.145.2.5 fPluginID	986
14.145.2.6 fChunkID	986
14.145.2.7 fName	987
14.146 AAX_SPlugInIdentifierTriad Struct Reference	987
14.146.1 Description	987
14.146.2 Member Data Documentation	987
14.146.2.1 mManufacturerID	987
14.146.2.2 mProductID	988

14.146.2.3 mPlugInID	988
14.147 AAX_StLock_Guard Class Reference	988
14.147.1 Description	988
14.147.2 Constructor & Destructor Documentation	988
14.147.2.1 AAX_StLock_Guard()	988
14.147.2.2 ~AAX_StLock_Guard()	989
14.148 AAX_TransportStateInfo_V1 Struct Reference	989
14.148.1 Description	989
14.148.2 Constructor & Destructor Documentation	989
14.148.2.1 AAX_TransportStateInfo_V1()	989
14.148.3 Member Function Documentation	989
14.148.3.1 ToString()	990
14.148.4 Member Data Documentation	990
14.148.4.1 mTransportState	990
14.148.4.2 mRecordMode	990
14.148.4.3 mIsRecordEnabled	990
14.148.4.4 mIsRecording	990
14.148.4.5 mIsLoopEnabled	990
14.149 AAX_VAutomationDelegate Class Reference	991
14.149.1 Description	991
14.149.2 Constructor & Destructor Documentation	991
14.149.2.1 AAX_VAutomationDelegate()	991
14.149.2.2 ~AAX_VAutomationDelegate()	991
14.149.3 Member Function Documentation	991
14.149.3.1 GetUnknown()	992
14.149.3.2 RegisterParameter()	992
14.149.3.3 UnregisterParameter()	992
14.149.3.4 PostSetValueRequest()	993
14.149.3.5 PostCurrentValue()	993
14.149.3.6 PostTouchRequest()	993
14.149.3.7 PostReleaseRequest()	994
14.149.3.8 GetTouchState()	994
14.149.3.9 ParameterNameChanged()	994
14.150 AAX_VCollection Class Reference	995
14.150.1 Description	995
14.150.2 Constructor & Destructor Documentation	995
14.150.2.1 AAX_VCollection()	996
14.150.2.2 ~AAX_VCollection()	996
14.150.3 Member Function Documentation	996
14.150.3.1 NewDescriptor()	996
14.150.3.2 AddEffect()	996
14.150.3.3 SetManufacturerName()	997

14.150.3.4 AddPackageName()	997
14.150.3.5 SetPackageVersion()	997
14.150.3.6 NewPropertyMap()	998
14.150.3.7 SetProperties()	998
14.150.3.8 GetHostVersion()	998
14.150.3.9 DescriptionHost() [1/2]	999
14.150.3.10 DescriptionHost() [2/2]	999
14.150.3.11 HostDefinition()	1000
14.150.3.12 GetUnknown()	1000
14.151 AAX_VComponentDescriptor Class Reference	1000
14.151.1 Description	1000
14.151.2 Constructor & Destructor Documentation	1002
14.151.2.1 AAX_VComponentDescriptor()	1002
14.151.2.2 ~AAX_VComponentDescriptor()	1002
14.151.3 Member Function Documentation	1002
14.151.3.1 Clear()	1002
14.151.3.2 AddReservedField()	1002
14.151.3.3 AddAudioIn()	1003
14.151.3.4 AddAudioOut()	1003
14.151.3.5 AddAudioBufferLength()	1004
14.151.3.6 AddSampleRate()	1004
14.151.3.7 AddClock()	1004
14.151.3.8 AddSideChainIn()	1005
14.151.3.9 AddDataInPort()	1005
14.151.3.10 AddAuxOutputStem()	1006
14.151.3.11 AddPrivateData()	1007
14.151.3.12 AddTemporaryData()	1007
14.151.3.13 AddDmaInstance()	1008
14.151.3.14 AddMeters()	1008
14.151.3.15 AddMIDINode()	1010
14.151.3.16 NewPropertyMap()	1010
14.151.3.17 DuplicatePropertyMap()	1011
14.151.3.18 AddProcessProc_Native()	1011
14.151.3.19 AddProcessProc_Tl()	1012
14.151.3.20 AddProcessProc()	1012
14.151.3.21 GetUnknown()	1014
14.151.4 Friends And Related Function Documentation	1014
14.151.4.1 AAX_VPropertyMap	1014
14.152 AAX_VController Class Reference	1014
14.152.1 Description	1014
14.152.2 Constructor & Destructor Documentation	1016
14.152.2.1 AAX_VController()	1016

14.152.2.2 ~AAX_VController()	1016
14.152.3 Member Function Documentation	1016
14.152.3.1 GetEffectID()	1017
14.152.3.2 GetSampleRate()	1017
14.152.3.3 GetInputStemFormat()	1017
14.152.3.4 GetOutputStemFormat()	1017
14.152.3.5 GetSignalLatency()	1018
14.152.3.6 GetHybridSignalLatency()	1018
14.152.3.7 GetPlugInTargetPlatform()	1019
14.152.3.8 GetIsAudioSuite()	1019
14.152.3.9 GetCycleCount()	1019
14.152.3.10 GetTODLocation()	1020
14.152.3.11 GetCurrentAutomationTimestamp()	1020
14.152.3.12 GetHostName()	1021
14.152.3.13 SetSignalLatency()	1021
14.152.3.14 SetCycleCount()	1022
14.152.3.15 PostPacket()	1023
14.152.3.16 SendNotification() [1/2]	1023
14.152.3.17 SendNotification() [2/2]	1024
14.152.3.18 GetCurrentMeterValue()	1024
14.152.3.19 GetMeterPeakValue()	1025
14.152.3.20 ClearMeterPeakValue()	1025
14.152.3.21 GetMeterClipped()	1025
14.152.3.22 ClearMeterClipped()	1026
14.152.3.23 GetMeterCount()	1026
14.152.3.24 GetNextMIDIpacket()	1026
14.152.3.25 CreateTableCopyForEffect()	1027
14.152.3.26 CreateTableCopyForLayout()	1027
14.152.3.27 CreateTableCopyForEffectFromFile()	1028
14.152.3.28 CreateTableCopyForLayoutFromFile()	1029
14.153 AAX_VDataBufferWrapper Class Reference	1029
14.153.1 Description	1029
14.153.2 Constructor & Destructor Documentation	1030
14.153.2.1 AAX_VDataBufferWrapper()	1030
14.153.2.2 ~AAX_VDataBufferWrapper()	1030
14.153.3 Member Function Documentation	1030
14.153.3.1 Type()	1030
14.153.3.2 Size()	1030
14.153.3.3 Data()	1031
14.154 AAX_VDescriptionHost Class Reference	1031
14.154.1 Description	1031
14.154.2 Constructor & Destructor Documentation	1031

14.154.2.1 AAX_VDescriptionHost()	1031
14.154.2.2 ~AAX_VDescriptionHost()	1032
14.154.3 Member Function Documentation	1032
14.154.3.1 AcquireFeatureProperties()	1032
14.154.3.2 Supported()	1032
14.154.3.3 DescriptionHost() [1/2]	1033
14.154.3.4 DescriptionHost() [2/2]	1033
14.154.3.5 HostDefinition()	1033
14.155 AAX_VEffectDescriptor Class Reference	1033
14.155.1 Description	1033
14.155.2 Constructor & Destructor Documentation	1034
14.155.2.1 AAX_VEffectDescriptor()	1034
14.155.2.2 ~AAX_VEffectDescriptor()	1034
14.155.3 Member Function Documentation	1034
14.155.3.1 NewComponentDescriptor()	1035
14.155.3.2 AddComponent()	1035
14.155.3.3 AddName()	1035
14.155.3.4 AddCategory()	1036
14.155.3.5 AddCategoryBypassParameter()	1036
14.155.3.6 AddProcPtr()	1036
14.155.3.7 NewPropertyMap()	1037
14.155.3.8 SetProperties()	1037
14.155.3.9 AddResourceInfo()	1037
14.155.3.10 AddMeterDescription()	1038
14.155.3.11 AddControlMIDINode()	1038
14.155.3.12 GetUnknown()	1039
14.156 AAX_VFeatureInfo Class Reference	1039
14.156.1 Description	1039
14.156.2 Constructor & Destructor Documentation	1039
14.156.2.1 AAX_VFeatureInfo()	1039
14.156.2.2 ~AAX_VFeatureInfo()	1039
14.156.3 Member Function Documentation	1040
14.156.3.1 SupportLevel()	1040
14.156.3.2 AcquireProperties()	1040
14.156.3.3 ID()	1041
14.157 AAX_VHostProcessorDelegate Class Reference	1041
14.157.1 Description	1041
14.157.2 Constructor & Destructor Documentation	1041
14.157.2.1 AAX_VHostProcessorDelegate()	1041
14.157.3 Member Function Documentation	1042
14.157.3.1 GetAudio()	1042
14.157.3.2 GetSideChainInputNum()	1042

14.157.3.3 ForceAnalyze()	1043
14.157.3.4 ForceProcess()	1043
14.158 AAX_VHostServices Class Reference	1043
14.158.1 Description	1043
14.158.2 Constructor & Destructor Documentation	1044
14.158.2.1 AAX_VHostServices()	1044
14.158.2.2 ~AAX_VHostServices()	1044
14.158.3 Member Function Documentation	1044
14.158.3.1 HandleAssertFailure()	1044
14.158.3.2 Trace()	1045
14.158.3.3 StackTrace()	1045
14.159 AAX_VHostTaskAgent Class Reference	1045
14.159.1 Constructor & Destructor Documentation	1046
14.159.1.1 AAX_VHostTaskAgent()	1046
14.159.1.2 ~AAX_VHostTaskAgent()	1046
14.159.2 Member Function Documentation	1046
14.159.2.1 Initialize()	1046
14.159.2.2 Uninitialize()	1046
14.159.2.3 AddTask()	1047
14.159.2.4 CancelAllTasks()	1047
14.160 AAX_VPageTable Class Reference	1047
14.160.1 Description	1047
14.160.2 Constructor & Destructor Documentation	1048
14.160.2.1 AAX_VPageTable()	1048
14.160.2.2 ~AAX_VPageTable()	1048
14.160.3 Member Function Documentation	1048
14.160.3.1 Clear()	1049
14.160.3.2 Empty()	1049
14.160.3.3 GetNumPages()	1049
14.160.3.4 InsertPage()	1049
14.160.3.5 RemovePage()	1050
14.160.3.6 GetNumMappedParameterIDs()	1050
14.160.3.7 ClearMappedParameter()	1051
14.160.3.8 GetMappedParameterID()	1051
14.160.3.9 MapParameterID()	1052
14.160.3.10 GetNumParametersWithNameVariations()	1052
14.160.3.11 GetNameVariationParameterIDAtIndex()	1053
14.160.3.12 GetNumNameVariationsForParameter()	1053
14.160.3.13 GetParameterNameVariationAtIndex()	1054
14.160.3.14 GetParameterNameVariationOfLength()	1055
14.160.3.15 ClearParameterNameVariations()	1056
14.160.3.16 ClearNameVariationsForParameter()	1056

14.160.3.17 SetParameterNameVariation()	1057
14.160.3.18 AsUnknown() [1/2]	1057
14.160.3.19 AsUnknown() [2/2]	1058
14.160.3.20 IsSupported()	1058
14.161 AAX_VPrivateDataAccess Class Reference	1058
14.161.1 Description	1058
14.161.2 Constructor & Destructor Documentation	1058
14.161.2.1 AAX_VPrivateDataAccess()	1058
14.161.2.2 ~AAX_VPrivateDataAccess()	1059
14.161.3 Member Function Documentation	1059
14.161.3.1 ReadPortDirect()	1059
14.161.3.2 WritePortDirect()	1059
14.162 AAX_VPropertyMap Class Reference	1060
14.162.1 Description	1060
14.162.2 Constructor & Destructor Documentation	1061
14.162.2.1 ~AAX_VPropertyMap()	1061
14.162.3 Member Function Documentation	1061
14.162.3.1 Create()	1061
14.162.3.2 Acquire()	1061
14.162.3.3 GetProperty()	1061
14.162.3.4 GetPointerProperty()	1062
14.162.3.5 AddProperty()	1062
14.162.3.6 AddPointerProperty() [1/2]	1063
14.162.3.7 AddPointerProperty() [2/2]	1063
14.162.3.8 RemoveProperty()	1064
14.162.3.9 AddPropertyWithIDArray()	1064
14.162.3.10 GetPropertyWithIDArray()	1064
14.162.3.11 GetUnknown()	1065
14.163 AAX_VSessionDocument Class Reference	1065
14.163.1 Constructor & Destructor Documentation	1065
14.163.1.1 AAX_VSessionDocument()	1066
14.163.1.2 ~AAX_VSessionDocument()	1066
14.163.2 Member Function Documentation	1066
14.163.2.1 Clear()	1066
14.163.2.2 Valid()	1066
14.163.2.3 GetTempoMap()	1066
14.163.2.4 GetDocumentData()	1067
14.164 AAX_VTask Class Reference	1068
14.164.1 Description	1068
14.164.2 Constructor & Destructor Documentation	1068
14.164.2.1 AAX_VTask()	1069
14.164.2.2 ~AAX_VTask()	1069

14.164.3 Member Function Documentation	1069
14.164.3.1 GetType()	1069
14.164.3.2 GetArgumentOfType()	1069
14.164.3.3 SetProgress()	1070
14.164.3.4 GetProgress()	1070
14.164.3.5 AddResult()	1070
14.164.3.6 SetDone()	1071
14.165 AAX_VTransport Class Reference	1071
14.165.1 Description	1071
14.165.2 Constructor & Destructor Documentation	1072
14.165.2.1 AAX_VTransport()	1072
14.165.2.2 ~AAX_VTransport()	1072
14.165.3 Member Function Documentation	1073
14.165.3.1 GetCurrentTempo()	1073
14.165.3.2 GetCurrentMeter()	1073
14.165.3.3 IsTransportPlaying()	1073
14.165.3.4 GetCurrentTickPosition()	1075
14.165.3.5 GetCurrentLoopPosition()	1075
14.165.3.6 GetCurrentNativeSampleLocation()	1076
14.165.3.7 GetCustomTickPosition()	1076
14.165.3.8 GetBarBeatPosition()	1077
14.165.3.9 GetTicksPerQuarter()	1077
14.165.3.10 GetCurrentTicksPerBeat()	1077
14.165.3.11 GetTimelineSelectionStartPosition()	1078
14.165.3.12 GetTimeCodeInfo()	1078
14.165.3.13 GetFeetFramesInfo()	1079
14.165.3.14 IsMetronomeEnabled()	1079
14.165.3.15 GetHDDTimeCodeInfo()	1079
14.165.3.16 GetTimelineSelectionEndPosition()	1080
14.165.3.17 GetKeySignature()	1080
14.165.3.18 RequestTransportStart()	1081
14.165.3.19 RequestTransportStop()	1081
14.166 AAX_VViewContainer Class Reference	1081
14.166.1 Description	1082
14.166.2 Constructor & Destructor Documentation	1082
14.166.2.1 AAX_VViewContainer()	1082
14.166.2.2 ~AAX_VViewContainer()	1083
14.166.3 Member Function Documentation	1083
14.166.3.1 GetType()	1083
14.166.3.2 GetPtr()	1083
14.166.3.3 GetModifiers()	1083
14.166.3.4 SetViewSize()	1084

14.166.3.5 HandleParameterMouseDown()	1084
14.166.3.6 HandleParameterMouseDownDrag()	1084
14.166.3.7 HandleParameterMouseUp()	1086
14.166.3.8 HandleParameterMouseEnter()	1086
14.166.3.9 HandleParameterMouseExit()	1087
14.166.3.10 HandleMultipleParametersMouseDown()	1087
14.166.3.11 HandleMultipleParametersMouseDownDrag()	1087
14.166.3.12 HandleMultipleParametersMouseUp()	1088
14.167 AAX::Exception::Any Class Reference	1088
14.167.1 Description	1089
14.167.2 Constructor & Destructor Documentation	1089
14.167.2.1 ~Any()	1089
14.167.2.2 Any() [1/3]	1089
14.167.2.3 Any() [2/3]	1090
14.167.2.4 Any() [3/3]	1090
14.167.3 Member Function Documentation	1090
14.167.3.1 operator=()	1090
14.167.3.2 AAX_DEFAULT_MOVE_CTOR()	1090
14.167.3.3 AAX_DEFAULT_MOVE_OPER()	1090
14.167.3.4 What()	1090
14.167.3.5 Desc()	1091
14.167.3.6 Function()	1091
14.167.3.7 Line()	1091
14.168 AAX_CChunkDataParser::DataValue Struct Reference	1091
14.168.1 Constructor & Destructor Documentation	1091
14.168.1.1 DataValue()	1092
14.168.2 Member Data Documentation	1092
14.168.2.1 mDataType	1092
14.168.2.2 mDataName	1092
14.168.2.3 mIntValue	1092
14.168.2.4 mStringValue	1092
14.169 IACFDefinition Interface Reference	1092
14.169.1 Description	1093
14.169.2 Member Function Documentation	1093
14.169.2.1 DefineAttribute()	1093
14.169.2.2 GetAttributeInfo()	1094
14.169.2.3 CopyAttribute()	1094
14.170 IACFUnknown Interface Reference	1095
14.170.1 Description	1095
14.170.2 Member Function Documentation	1095
14.170.2.1 QueryInterface()	1095
14.170.2.2 AddRef()	1096

14.170.2.3 Release()	1096
14.171 AAX::Exception::ResultError Class Reference	1096
14.171.1 Description	1096
14.171.2 Constructor & Destructor Documentation	1097
14.171.2.1 ResultError() [1/4]	1097
14.171.2.2 ResultError() [2/4]	1097
14.171.2.3 ResultError() [3/4]	1097
14.171.2.4 ResultError() [4/4]	1097
14.171.3 Member Function Documentation	1098
14.171.3.1 FormatResult()	1098
14.171.3.2 Result()	1098
14.172 SAutoArray< T > Struct Template Reference	1098
14.172.1 Constructor & Destructor Documentation	1098
14.172.1.1 SAutoArray()	1098
14.172.1.2 ~SAutoArray()	1099
14.172.2 Member Function Documentation	1099
14.172.2.1 Reset()	1099
14.172.2.2 Get()	1099
14.173 AAX_ISessionDocument::TempoMap Class Reference	1099
14.173.1 Constructor & Destructor Documentation	1099
14.173.1.1 ~TempoMap()	1100
14.173.2 Member Function Documentation	1100
14.173.2.1 Size()	1100
14.173.2.2 Data()	1100
14.174 AAX_VSessionDocument::VTempoMap Class Reference	1100
14.174.1 Constructor & Destructor Documentation	1100
14.174.1.1 ~VTempoMap()	1101
14.174.1.2 VTempoMap()	1101
14.174.2 Member Function Documentation	1101
14.174.2.1 Size()	1101
14.174.2.2 Data()	1101
15 File Documentation	1103
15.1 AAX.h File Reference	1103
15.1.1 Description	1103
15.1.2 Macro Definition Documentation	1106
15.1.2.1 TI_VERSION	1106
15.1.2.2 AAX_CPP11_SUPPORT	1106
15.1.2.3 AAX_OVERRIDE	1107
15.1.2.4 AAX_FINAL	1107
15.1.2.5 AAX_DEFAULT_DTOR	1107
15.1.2.6 AAX_DEFAULT_DTOR_OVERRIDE	1107

15.1.2.7 AAX_DEFAULT_CTOR	1107
15.1.2.8 AAX_DEFAULT_COPY_CTOR	1107
15.1.2.9 AAX_DEFAULT_ASGN_OPER	1108
15.1.2.10 AAX_DELETE	1108
15.1.2.11 AAX_DEFAULT_MOVE_CTOR	1108
15.1.2.12 AAX_DEFAULT_MOVE_OPER	1108
15.1.2.13 AAX_CONSTEXPR	1108
15.1.2.14 AAX_UNIQUE_PTR	1109
15.1.2.15 AAXPointer_32bit	1109
15.1.2.16 AAXPointer_64bit	1109
15.1.2.17 AAX_PointerSize	1109
15.1.2.18 AAX_ALIGN_FILE_HOST	1109
15.1.2.19 AAX_ALIGN_FILE_ALG	1110
15.1.2.20 AAX_ALIGN_FILE_RESET	1110
15.1.2.21 AAX_ALIGN_FILE_BEGIN	1110
15.1.2.22 AAX_ALIGN_FILE_END	1110
15.1.2.23 AAX_CALLBACK	1111
15.1.2.24 AAX_PREPROCESSOR_CONCAT_HELPER	1111
15.1.2.25 AAX_PREPROCESSOR_CONCAT	1111
15.1.2.26 AAX_FIELD_INDEX	1111
15.1.3 Typedef Documentation	1111
15.1.3.1 AAX_CIndex	1111
15.1.3.2 AAX_CCount	1112
15.1.3.3 AAX_CBoolean	1112
15.1.3.4 AAX_CSelector	1112
15.1.3.5 AAX_CTimestamp	1112
15.1.3.6 AAX_CTimeOfDay	1112
15.1.3.7 AAX_CTransportCounter	1113
15.1.3.8 AAX_CSampleRate	1113
15.1.3.9 AAX_CTypeID	1113
15.1.3.10 AAX_Result	1113
15.1.3.11 AAX_CPropertyValue	1113
15.1.3.12 AAX_CPropertyValue64	1113
15.1.3.13 AAX_CPointerPropertyValue	1114
15.1.3.14 AAX_CTargetPlatform	1114
15.1.3.15 AAX_CFieldIndex	1114
15.1.3.16 AAX_CComponentID	1114
15.1.3.17 AAX_CMeterID	1114
15.1.3.18 AAX_CParamID	1115
15.1.3.19 AAX_CPageTableParamID	1115
15.1.3.20 AAX_CEffectID	1115
15.1.3.21 acfUID	1115

15.1.3.22 AAX_Feature_UID	1116
15.1.3.23 AAX_CAudioInPort	1116
15.1.3.24 AAX_CAudioOutPort	1116
15.1.3.25 AAX_CMeterPort	1116
15.1.3.26 AAX_SPlugInChunkHeader	1117
15.1.3.27 AAX_SPlugInChunk	1117
15.1.3.28 AAX_SPlugInChunkPtr	1117
15.1.3.29 AAX_SPlugInIdentifierTriad	1117
15.1.3.30 AAX_SPlugInIdentifierTriadPtr	1117
15.1.4 Variable Documentation	1117
15.1.4.1 kAAX_ParameterIdentifierMaxSize	1117
15.2 AAX_ACFInterface.doxygen File Reference	1117
15.2.1 Typedef Documentation	1118
15.2.1.1 acfUID	1118
15.2.1.2 acfIID	1118
15.3 AAX_AdditionalFeatures_Algorithm.doxygen File Reference	1118
15.4 AAX_AdditionalFeatures_AOSandSidechain.doxygen File Reference	1118
15.5 AAX_AdditionalFeatures_CurveDisplays.doxygen File Reference	1118
15.6 AAX_AdditionalFeatures_Hybrid.doxygen File Reference	1118
15.7 AAX_AdditionalFeatures_Meters.doxygen File Reference	1118
15.8 AAX_AdditionalFeatures_MIDI.doxygen File Reference	1118
15.9 AAX_AdditionalFeatures_PropertiesFile.doxygen File Reference	1118
15.10 AAX_Alignment.h File Reference	1118
15.10.1 Description	1119
15.11 AAX_Assert.h File Reference	1119
15.11.1 Description	1119
15.11.2 Macro Definition Documentation	1120
15.11.2.1 kAAX_Trace_Priority_None	1121
15.11.2.2 kAAX_Trace_Priority_Critical	1121
15.11.2.3 kAAX_Trace_Priority_High	1121
15.11.2.4 kAAX_Trace_Priority_Normal	1121
15.11.2.5 kAAX_Trace_Priority_Low	1121
15.11.2.6 kAAX_Trace_Priority_Lowest	1121
15.11.2.7 AAX_TRACE_RELEASE	1122
15.11.2.8 AAX_STACKTRACE_RELEASE	1122
15.11.2.9 AAX_TRACEORSTACKTRACE_RELEASE	1123
15.11.2.10 AAX_ASSERT	1123
15.11.2.11 AAX_DEBUGASSERT	1124
15.11.2.12 AAX_TRACE	1124
15.11.2.13 AAX_STACKTRACE	1124
15.11.2.14 AAX_TRACEORSTACKTRACE	1124
15.11.3 Typedef Documentation	1125

15.11.3.1 AAX_ETracePriority	1125
15.12 AAX_Atomic.h File Reference	1125
15.12.1 Description	1125
15.12.2 Macro Definition Documentation	1126
15.12.2.1 AAX_ATOMIC_H	1126
15.12.3 Function Documentation	1126
15.12.3.1 AAX_Atomic_IncThenGet_32()	1126
15.12.3.2 AAX_Atomic_DecThenGet_32()	1126
15.12.3.3 AAX_Atomic_Exchange_32()	1127
15.13 AAX_AuxInterface_DirectData.doxygen File Reference	1127
15.14 AAX_AuxInterface_HostProcessor.doxygen File Reference	1127
15.15 AAX_AuxInterface_TaskAgent.doxygen File Reference	1127
15.16 AAX_BugList.doxygen File Reference	1127
15.17 AAX_Callbacks.h File Reference	1127
15.17.1 Description	1127
15.17.2 Typedef Documentation	1128
15.17.2.1 AAXCreateObjectProc	1128
15.17.2.2 AAX_CProcessProc	1128
15.17.2.3 AAX_CPacketAllocator	1128
15.17.2.4 AAX_CInstanceInitProc	1129
15.17.2.5 AAX_CBackgroundProc	1129
15.17.2.6 AAX_CInitPrivateDataProc	1129
15.17.3 Enumeration Type Documentation	1130
15.17.3.1 AAX_CProcPtrID	1130
15.18 AAX_CArrayDataBuffer.h File Reference	1131
15.18.1 Macro Definition Documentation	1131
15.18.1.1 AAX_CArrayDataBuffer_H	1131
15.19 AAX_CAtomicQueue.h File Reference	1131
15.19.1 Description	1132
15.20 AAX_CAutoreleasePool.h File Reference	1132
15.20.1 Description	1132
15.20.2 Macro Definition Documentation	1132
15.20.2.1 _AAX_CAUTORELEASEPOOL_H	1132
15.21 AAX_CBinaryDisplayDelegate.h File Reference	1132
15.21.1 Description	1132
15.22 AAX_CBinaryTaperDelegate.h File Reference	1133
15.22.1 Description	1133
15.23 AAX_CChunkDataParser.h File Reference	1133
15.23.1 Description	1133
15.23.2 Macro Definition Documentation	1134
15.23.2.1 AAX_CHUNKDATAPARSER_H	1134
15.24 AAX_CDecibelDisplayDelegateDecorator.h File Reference	1134

15.24.1 Description	1134
15.25 AAX_CEffectDirectData.h File Reference	1135
15.25.1 Description	1135
15.25.2 Macro Definition Documentation	1135
15.25.2.1 AAX_CEFFECTDIRECTDATA_H	1135
15.26 AAX_CEffectGUI.h File Reference	1135
15.26.1 Description	1135
15.27 AAX_CEffectParameters.h File Reference	1136
15.27.1 Description	1136
15.27.2 Function Documentation	1136
15.27.2.1 NormalizedToInt32()	1136
15.27.2.2 Int32ToNormalized()	1137
15.27.2.3 BoolToNormalized()	1137
15.27.3 Variable Documentation	1137
15.27.3.1 cPreviewID	1137
15.27.3.2 cDefaultMasterBypassID	1137
15.28 AAX_CHostProcessor.h File Reference	1137
15.28.1 Description	1137
15.29 AAX_CHostServices.h File Reference	1138
15.29.1 Description	1138
15.30 AAX_CLinearTaperDelegate.h File Reference	1138
15.30.1 Description	1138
15.31 AAX_CLogTaperDelegate.h File Reference	1138
15.31.1 Description	1138
15.32 AAX_CMonolithicParameters.cpp File Reference	1139
15.33 AAX_CMonolithicParameters.h File Reference	1139
15.33.1 Description	1139
15.33.2 Macro Definition Documentation	1140
15.33.2.1 kMaxAdditionalMIDINodes	1140
15.33.2.2 kMaxAuxOutputStems	1140
15.33.2.3 kSynchronizedParameterQueueSize	1140
15.34 AAX_CMutex.h File Reference	1140
15.34.1 Description	1140
15.35 AAX_CNumberDisplayDelegate.h File Reference	1140
15.35.1 Description	1140
15.36 AAX_CommonConversions.h File Reference	1141
15.36.1 Function Documentation	1142
15.36.1.1 GainToDB()	1142
15.36.1.2 DBToGain()	1142
15.36.1.3 LongToDouble()	1142
15.36.1.4 DoubleToLong()	1142
15.36.1.5 DoubleToDSPCoef()	1143

15.36.1.6 DSPCoefToDouble()	1143
15.36.1.7 ThirtyTwoBitDSPCoefToDouble()	1143
15.36.1.8 DoubleTo32BitDSPCoefRnd()	1143
15.36.1.9 DoubleTo32BitDSPCoef()	1144
15.36.1.10 DoubleToDSPCoefRnd()	1144
15.36.2 Variable Documentation	1144
15.36.2.1 k32BitPosMax	1144
15.36.2.2 k32BitAbsMax	1144
15.36.2.3 k32BitNegMax	1144
15.36.2.4 k56kFracPosMax	1144
15.36.2.5 k56kFracAbsMax	1145
15.36.2.6 k56kFracHalf	1145
15.36.2.7 k56kFracNegOne	1145
15.36.2.8 k56kFracNegMax	1145
15.36.2.9 k56kFracZero	1145
15.36.2.10 kOneOver56kFracAbsMax	1145
15.36.2.11 k56kFloatPosMax	1145
15.36.2.12 k56kFloatNegMax	1146
15.36.2.13 kNeg144DB	1146
15.36.2.14 kNeg144Gain	1146
15.37 AAX_CommonInterface_Algorithm.doxygen File Reference	1146
15.38 AAX_CommonInterface_Communication.doxygen File Reference	1146
15.39 AAX_CommonInterface_DataModel.doxygen File Reference	1146
15.40 AAX_CommonInterface_Describe.doxygen File Reference	1146
15.41 AAX_CommonInterface_FormatSpecification.doxygen File Reference	1146
15.42 AAX_CommonInterface_GUI.doxygen File Reference	1146
15.43 AAX_Constants.h File Reference	1146
15.43.1 Description	1147
15.43.2 Macro Definition Documentation	1147
15.43.2.1 AAX_CONSTANTS_H	1148
15.44 AAX_CPacketDispatcher.h File Reference	1148
15.44.1 Description	1148
15.45 AAX_CParameter.h File Reference	1148
15.45.1 Description	1148
15.46 AAX_CParameterManager.h File Reference	1149
15.46.1 Description	1149
15.47 AAX_CPercentDisplayDelegateDecorator.h File Reference	1149
15.47.1 Description	1149
15.47.2 Macro Definition Documentation	1150
15.47.2.1 AAX_CPERCENTDISPLAYDELEGATEDECORATOR_H	1150
15.48 AAX_CPieceWiseLinearTaperDelegate.h File Reference	1150
15.48.1 Description	1150

15.49 AAX_CRangeTaperDelegate.h File Reference	1150
15.49.1 Description	1150
15.50 AAX_CSessionDocumentClient.h File Reference	1151
15.50.1 Macro Definition Documentation	1151
15.50.1.1 AAX_CSessionDocumentClient_H	1151
15.51 AAX_CStateDisplayDelegate.h File Reference	1151
15.51.1 Description	1151
15.52 AAX_CStateTaperDelegate.h File Reference	1151
15.52.1 Description	1152
15.53 AAX_CString.h File Reference	1152
15.53.1 Description	1152
15.53.2 Macro Definition Documentation	1152
15.53.2.1 AAX_CSTRING_H	1153
15.53.3 Function Documentation	1153
15.53.3.1 operator+() [1/3]	1153
15.53.3.2 operator+() [2/3]	1153
15.53.3.3 operator+() [3/3]	1153
15.54 AAX_CStringDataBuffer.h File Reference	1153
15.54.1 Macro Definition Documentation	1154
15.54.1.1 AAX_CStringDataBuffer_H	1154
15.55 AAX_CStringDisplayDelegate.h File Reference	1154
15.55.1 Description	1154
15.56 AAX_CTask.h File Reference	1154
15.56.1 Description	1154
15.56.2 Macro Definition Documentation	1155
15.56.2.1 AAX_CTask_H	1155
15.57 AAX_CTaskAgent.h File Reference	1155
15.57.1 Description	1155
15.58 AAX_CUnitDisplayDelegateDecorator.h File Reference	1155
15.58.1 Description	1155
15.59 AAX_CUnitPrefixDisplayDelegateDecorator.h File Reference	1156
15.59.1 Description	1156
15.60 AAX_Denormal.h File Reference	1156
15.60.1 Description	1156
15.60.2 Macro Definition Documentation	1157
15.60.2.1 AAX_DENORMAL_H	1157
15.60.2.2 AAX_SCOPE_COMPUTE_DENORMALS	1157
15.60.2.3 AAX_SCOPE_DENORMALS_AS_ZERO	1157
15.61 AAX_DigiTrace_Guide.doxygen File Reference	1157
15.62 AAX_DistributingYourPlugIn.doxygen File Reference	1157
15.63 AAX_DocsDirectory.doxygen File Reference	1157
15.64 AAX_EndianSwap.h File Reference	1157

15.64.1 Description	1158
15.64.2 Macro Definition Documentation	1158
15.64.2.1 ENDIANSWAP_H	1158
15.64.3 Function Documentation	1159
15.64.3.1 AAX_EndianSwapInPlace()	1159
15.64.3.2 AAX_EndianSwap()	1159
15.64.3.3 AAX_BigEndianNativeSwapInPlace()	1159
15.64.3.4 AAX_BigEndianNativeSwap()	1159
15.64.3.5 AAX_LittleEndianNativeSwapInPlace()	1160
15.64.3.6 AAX_LittleEndianNativeSwap()	1160
15.64.3.7 AAX_EndianSwapSequenceInPlace()	1160
15.64.3.8 AAX_BigEndianNativeSwapSequenceInPlace()	1160
15.64.3.9 AAX_LittleEndianNativeSwapSequenceInPlace()	1161
15.65 AAX_Enums.h File Reference	1161
15.65.1 Description	1161
15.65.2 Macro Definition Documentation	1170
15.65.2.1 AAX_INT32_MIN	1170
15.65.2.2 AAX_INT32_MAX	1170
15.65.2.3 AAX_UINT32_MIN	1170
15.65.2.4 AAX_UINT32_MAX	1170
15.65.2.5 AAX_INT16_MIN	1170
15.65.2.6 AAX_INT16_MAX	1170
15.65.2.7 AAX_UINT16_MIN	1170
15.65.2.8 AAX_UINT16_MAX	1171
15.65.2.9 AAX_ENUM_SIZE_CHECK	1171
15.65.2.10 AAX_STEM_FORMAT	1171
15.65.2.11 AAX_STEM_FORMAT_CHANNEL_COUNT	1171
15.65.2.12 AAX_STEM_FORMAT_INDEX	1171
15.65.3 Typedef Documentation	1171
15.65.3.1 AAX_EParameterType	1171
15.65.3.2 AAX_EParameterOrientation	1172
15.65.4 Enumeration Type Documentation	1172
15.65.4.1 AAX_EHighlightColor	1172
15.65.4.2 AAX_ETracePriorityHost	1172
15.65.4.3 AAX_ETracePriorityDSP	1173
15.65.4.4 AAX_EModifiers	1173
15.65.4.5 AAX_EAudioBufferLength	1173
15.65.4.6 AAX_EAudioBufferLengthDSP	1174
15.65.4.7 AAE_EAudioBufferLengthNative	1175
15.65.4.8 AAX_EMaxAudioSuiteTracks	1175
15.65.4.9 AAX_EStemFormat	1175
15.65.4.10 AAX_EPlugInCategory	1177

15.65.4.11 AAX_EPlugInStrings	1178
15.65.4.12 AAX_EMeterOrientation	1179
15.65.4.13 AAX_EMeterBallisticType	1180
15.65.4.14 AAX_EMeterType	1180
15.65.4.15 AAX_EResourceType	1180
15.65.4.16 AAX_ENotificationEvent	1181
15.65.4.17 AAX_EHostModeBits	1186
15.65.4.18 AAX_EHostMode	1187
15.65.4.19 AAX_EPrivateDataOptions	1187
15.65.4.20 AAX_EConstraintLocationMask	1188
15.65.4.21 AAX_EConstraintTopology	1188
15.65.4.22 AAX_EComponentInstanceInitAction	1189
15.65.4.23 AAX_ESampleRateMask	1189
15.65.4.24 AAX_EParameterType	1190
15.65.4.25 AAX_EParameterOrientationBits	1190
15.65.4.26 AAX_EParameterValueInfoSelector	1191
15.65.4.27 AAX_EEQBandTypes	1191
15.65.4.28 AAX_EEQInCircuitPolarity	1192
15.65.4.29 AAX_EUseAlternateControl	1192
15.65.4.30 AAX_EMIDINodeType	1192
15.65.4.31 AAX_EUpdateSource	1194
15.65.4.32 AAX_EDataInPortType	1194
15.65.4.33 AAX_EFrameRate	1195
15.65.4.34 AAX_EFeetFramesRate	1196
15.65.4.35 AAX_EMidiGlobalNodeSelectors	1196
15.65.4.36 AAX_EPreviewState	1197
15.65.4.37 AAX_EProcessingState	1197
15.65.4.38 AAX_ETargetPlatform	1198
15.65.4.39 AAX_ESupportLevel	1198
15.65.4.40 AAX_EHostLevel	1199
15.65.4.41 AAX_ETextEncoding	1199
15.65.4.42 AAX_EAssertFlags	1200
15.65.4.43 AAX_ETransportState	1200
15.65.4.44 AAX_ERecordMode	1200
15.65.5 Function Documentation	1201
15.65.5.1 AAX_ENUM_SIZE_CHECK() [1/45]	1201
15.65.5.2 AAX_ENUM_SIZE_CHECK() [2/45]	1201
15.65.5.3 AAX_ENUM_SIZE_CHECK() [3/45]	1201
15.65.5.4 AAX_ENUM_SIZE_CHECK() [4/45]	1201
15.65.5.5 AAX_ENUM_SIZE_CHECK() [5/45]	1202
15.65.5.6 AAX_ENUM_SIZE_CHECK() [6/45]	1202
15.65.5.7 AAX_ENUM_SIZE_CHECK() [7/45]	1202

15.65.5.8 AAX_ENUM_SIZE_CHECK() [8/45]	1202
15.65.5.9 AAX_ENUM_SIZE_CHECK() [9/45]	1202
15.65.5.10 AAX_ENUM_SIZE_CHECK() [10/45]	1202
15.65.5.11 AAX_ENUM_SIZE_CHECK() [11/45]	1202
15.65.5.12 AAX_ENUM_SIZE_CHECK() [12/45]	1203
15.65.5.13 AAX_ENUM_SIZE_CHECK() [13/45]	1203
15.65.5.14 AAX_ENUM_SIZE_CHECK() [14/45]	1203
15.65.5.15 AAX_ENUM_SIZE_CHECK() [15/45]	1203
15.65.5.16 AAX_ENUM_SIZE_CHECK() [16/45]	1203
15.65.5.17 AAX_ENUM_SIZE_CHECK() [17/45]	1203
15.65.5.18 AAX_ENUM_SIZE_CHECK() [18/45]	1203
15.65.5.19 AAX_ENUM_SIZE_CHECK() [19/45]	1204
15.65.5.20 AAX_ENUM_SIZE_CHECK() [20/45]	1204
15.65.5.21 AAX_ENUM_SIZE_CHECK() [21/45]	1204
15.65.5.22 AAX_ENUM_SIZE_CHECK() [22/45]	1204
15.65.5.23 AAX_ENUM_SIZE_CHECK() [23/45]	1204
15.65.5.24 AAX_ENUM_SIZE_CHECK() [24/45]	1204
15.65.5.25 sampleRateInMask()	1205
15.65.5.26 getLowestSampleRateInMask()	1205
15.65.5.27 getMaskForSampleRate()	1205
15.65.5.28 AAX_ENUM_SIZE_CHECK() [25/45]	1205
15.65.5.29 AAX_ENUM_SIZE_CHECK() [26/45]	1206
15.65.5.30 AAX_ENUM_SIZE_CHECK() [27/45]	1206
15.65.5.31 AAX_ENUM_SIZE_CHECK() [28/45]	1206
15.65.5.32 AAX_ENUM_SIZE_CHECK() [29/45]	1206
15.65.5.33 AAX_ENUM_SIZE_CHECK() [30/45]	1206
15.65.5.34 AAX_ENUM_SIZE_CHECK() [31/45]	1206
15.65.5.35 AAX_ENUM_SIZE_CHECK() [32/45]	1206
15.65.5.36 AAX_ENUM_SIZE_CHECK() [33/45]	1207
15.65.5.37 AAX_ENUM_SIZE_CHECK() [34/45]	1207
15.65.5.38 AAX_ENUM_SIZE_CHECK() [35/45]	1207
15.65.5.39 AAX_ENUM_SIZE_CHECK() [36/45]	1207
15.65.5.40 AAX_ENUM_SIZE_CHECK() [37/45]	1207
15.65.5.41 AAX_ENUM_SIZE_CHECK() [38/45]	1207
15.65.5.42 AAX_ENUM_SIZE_CHECK() [39/45]	1207
15.65.5.43 AAX_ENUM_SIZE_CHECK() [40/45]	1208
15.65.5.44 AAX_ENUM_SIZE_CHECK() [41/45]	1208
15.65.5.45 AAX_ENUM_SIZE_CHECK() [42/45]	1208
15.65.5.46 AAX_ENUM_SIZE_CHECK() [43/45]	1208
15.65.5.47 AAX_ENUM_SIZE_CHECK() [44/45]	1208
15.65.5.48 AAX_ENUM_SIZE_CHECK() [45/45]	1208
15.66 AAX_EnvironmentUtilities.h File Reference	1208

15.66.1 Description	1209
15.67 AAX_Errors.h File Reference	1209
15.67.1 Description	1209
15.67.2 Enumeration Type Documentation	1210
15.67.2.1 AAX_EError	1210
15.67.3 Function Documentation	1212
15.67.3.1 AAX_ENUM_SIZE_CHECK()	1212
15.68 AAX_Exception.h File Reference	1212
15.68.1 Description	1212
15.68.2 Macro Definition Documentation	1213
15.68.2.1 AAX_SWALLOW	1213
15.68.2.2 AAX_SWALLOW_MULT	1214
15.68.2.3 AAX_CAPTURE	1214
15.68.2.4 AAX_CAPTURE_MULT	1215
15.69 AAX_Exports.cpp File Reference	1215
15.69.1 Macro Definition Documentation	1216
15.69.1.1 AAX_EXPORT	1216
15.69.2 Function Documentation	1216
15.69.2.1 ACFRegisterPlugin()	1216
15.69.2.2 ACFRegisterComponent()	1216
15.69.2.3 ACFGetClassFactory()	1217
15.69.2.4 ACFCanUnloadNow()	1217
15.69.2.5 ACFStartup()	1217
15.69.2.6 ACFShutdown()	1217
15.69.2.7 ACFGetSDKVersion()	1218
15.70 AAX_FastInterpolatedTableLookup.h File Reference	1218
15.70.1 Description	1218
15.70.2 Macro Definition Documentation	1218
15.70.2.1 AAX_FASTINTERPOLATEDTABLELOOKUP_H	1218
15.71 AAX_FastInterpolatedTableLookup.hpp File Reference	1218
15.72 AAX_FastPow.h File Reference	1219
15.72.1 Description	1219
15.72.2 Macro Definition Documentation	1219
15.72.2.1 _AAX_FASTPOW_H_	1219
15.73 AAX_Getting_Started_Guide.doxygen File Reference	1219
15.74 AAX_GUITypes.h File Reference	1219
15.74.1 Description	1220
15.74.2 Typedef Documentation	1220
15.74.2.1 AAX_Point	1221
15.74.2.2 AAX_Rect	1221
15.74.2.3 AAX_EViewContainer_Type	1221
15.74.3 Enumeration Type Documentation	1221

15.74.3.1 AAX_EViewContainer_Type	1221
15.74.4 Function Documentation	1222
15.74.4.1 operator==() [1 / 2]	1222
15.74.4.2 operator!=() [1 / 2]	1222
15.74.4.3 operator<()	1222
15.74.4.4 operator<=()	1222
15.74.4.5 operator>()	1222
15.74.4.6 operator>=()	1223
15.74.4.7 operator==() [2 / 2]	1223
15.74.4.8 operator!=() [2 / 2]	1223
15.74.4.9 AAX_ENUM_SIZE_CHECK()	1223
15.75 AAX_HostSupport.doxygen File Reference	1223
15.76 AAX_IACFAutomationDelegate.h File Reference	1223
15.76.1 Description	1223
15.77 AAX_IACFCollection.h File Reference	1224
15.77.1 Description	1224
15.78 AAX_IACFComponentDescriptor.h File Reference	1224
15.78.1 Description	1224
15.79 AAX_IACFController.h File Reference	1224
15.79.1 Description	1225
15.80 AAX_IACFDataBuffer.h File Reference	1225
15.80.1 Macro Definition Documentation	1225
15.80.1.1 AAX_IACFDataBuffer_H	1225
15.81 AAX_IACFDescriptionHost.h File Reference	1225
15.82 AAX_IACFEffectDescriptor.h File Reference	1226
15.82.1 Description	1226
15.83 AAX_IACFEffectDirectData.h File Reference	1226
15.83.1 Description	1226
15.84 AAX_IACFEffectGUI.h File Reference	1226
15.84.1 Description	1227
15.85 AAX_IACFEffectParameters.h File Reference	1227
15.85.1 Description	1227
15.86 AAX_IACFFeatureInfo.h File Reference	1227
15.87 AAX_IACFHostProcessor.h File Reference	1228
15.87.1 Description	1228
15.88 AAX_IACFHostProcessorDelegate.h File Reference	1228
15.89 AAX_IACFHostServices.h File Reference	1228
15.90 AAX_IACFPageTable.h File Reference	1229
15.91 AAX_IACFPageTableController.h File Reference	1229
15.92 AAX_IACFPrivateDataAccess.h File Reference	1229
15.92.1 Description	1229
15.93 AAX_IACFPropertyMap.h File Reference	1230

15.93.1 Description	1230
15.94 AAX_IACFSessionDocument.h File Reference	1230
15.94.1 Macro Definition Documentation	1230
15.94.1.1 AAX_IACFSessionDocument_H	1230
15.95 AAX_IACFSessionDocumentClient.h File Reference	1231
15.95.1 Macro Definition Documentation	1231
15.95.1.1 AAX_IACFSessionDocumentClient_H	1231
15.96 AAX_IACFTask.h File Reference	1231
15.96.1 Description	1231
15.96.2 Macro Definition Documentation	1232
15.96.2.1 AAX_IACFTask_H	1232
15.97 AAX_IACFTaskAgent.h File Reference	1232
15.97.1 Macro Definition Documentation	1232
15.97.1.1 AAX_IACFTaskAgent_H	1232
15.98 AAX_IACFTransport.h File Reference	1232
15.98.1 Description	1233
15.99 AAX_IACFTransportControl.h File Reference	1233
15.99.1 Description	1233
15.100 AAX_IACFViewContainer.h File Reference	1233
15.100.1 Description	1233
15.101 AAX_IAutomationDelegate.h File Reference	1234
15.101.1 Description	1234
15.102 AAX_ICollection.h File Reference	1234
15.102.1 Description	1234
15.103 AAX_IComponentDescriptor.h File Reference	1234
15.103.1 Description	1235
15.104 AAX_IContainer.h File Reference	1235
15.104.1 Description	1235
15.105 AAX_IController.h File Reference	1235
15.105.1 Description	1235
15.106 AAX_IDataBuffer.h File Reference	1235
15.106.1 Macro Definition Documentation	1236
15.106.1.1 AAX_IDataBuffer_H	1236
15.107 AAX_IDataBufferWrapper.h File Reference	1236
15.107.1 Macro Definition Documentation	1236
15.107.1.1 AAX_IDATABUFFERWRAPPER_H	1236
15.108 AAX_IDescriptionHost.h File Reference	1236
15.109 AAX_IDisplayDelegate.h File Reference	1237
15.109.1 Description	1237
15.110 AAX_IDisplayDelegateDecorator.h File Reference	1237
15.110.1 Description	1237
15.111 AAX_IDma.h File Reference	1237

15.111.1 Description	1237
15.111.2 Macro Definition Documentation	1238
15.111.2.1 AAX_IDMA_H	1238
15.111.2.2 AAX_DMA_API	1238
15.112 AAX_IEffectDescriptor.h File Reference	1238
15.112.1 Description	1238
15.113 AAX_IEffectDirectData.h File Reference	1238
15.113.1 Description	1239
15.114 AAX_IEffectGUI.h File Reference	1239
15.114.1 Description	1239
15.115 AAX_IEffectParameters.h File Reference	1239
15.115.1 Description	1239
15.116 AAX_IFeatureInfo.h File Reference	1240
15.117 AAX_IHostProcessor.h File Reference	1240
15.117.1 Description	1240
15.118 AAX_IHostProcessorDelegate.h File Reference	1240
15.118.1 Description	1240
15.119 AAX_IHostServices.h File Reference	1240
15.119.1 Description	1241
15.120 AAX_IHostTaskAgent.h File Reference	1241
15.120.1 Description	1241
15.120.2 Macro Definition Documentation	1241
15.120.2.1 AAX_IHostTaskAgent_H	1241
15.121 AAX_IMIDINode.h File Reference	1241
15.121.1 Description	1242
15.122 AAX_Init.h File Reference	1242
15.122.1 Description	1242
15.122.2 Function Documentation	1242
15.122.2.1 AAXRegisterComponent()	1243
15.122.2.2 AAXGetClassFactory()	1243
15.122.2.3 AAXCanUnloadNow()	1243
15.122.2.4 AAXStartup()	1244
15.122.2.5 AAXShutdown()	1244
15.122.2.6 AAXGetSDKVersion()	1244
15.123 AAX_InstrumentParameters.doxygen File Reference	1245
15.124 AAX_InterfaceList.doxygen File Reference	1245
15.125 AAX_IPageTable.h File Reference	1245
15.126 AAX_IParameter.h File Reference	1245
15.126.1 Description	1245
15.127 AAX_IPointerQueue.h File Reference	1245
15.127.1 Description	1245
15.128 AAX_IPrivateDataAccess.h File Reference	1246

15.128.1 Description	1246
15.129 AAX_IPropertyMap.h File Reference	1246
15.129.1 Description	1246
15.130 AAX_ISessionDocument.h File Reference	1246
15.130.1 Macro Definition Documentation	1247
15.130.1.1 AAX_ISessionDocument_H	1247
15.131 AAX_ISessionDocumentClient.h File Reference	1247
15.131.1 Macro Definition Documentation	1247
15.131.1.1 AAX_ISessionDocumentClient_H	1247
15.132 AAX_IString.h File Reference	1248
15.132.1 Description	1248
15.133 AAX_ITaperDelegate.h File Reference	1248
15.133.1 Description	1248
15.134 AAX_ITask.h File Reference	1248
15.134.1 Macro Definition Documentation	1249
15.134.1.1 AAX_ITask_H	1249
15.135 AAX_ITaskAgent.h File Reference	1249
15.136 AAX_ITransport.h File Reference	1249
15.136.1 Description	1249
15.137 AAX_IViewContainer.h File Reference	1249
15.137.1 Description	1250
15.138 AAX_LinkedParameters.doxygen File Reference	1250
15.139 AAX_Map.h File Reference	1250
15.139.1 Description	1250
15.139.2 Macro Definition Documentation	1250
15.139.2.1 AAX_MAP_H	1250
15.140 AAX_Media_Composer_Guide.doxygen File Reference	1251
15.141 AAX_MIDILogging.cpp File Reference	1251
15.142 AAX_MIDILogging.h File Reference	1251
15.142.1 Description	1251
15.143 AAX_MIDIUtilities.h File Reference	1251
15.143.1 Description	1252
15.144 AAX_MiscUtils.h File Reference	1253
15.144.1 Description	1253
15.144.2 Macro Definition Documentation	1254
15.144.2.1 AAX_MISCUTILS_H	1254
15.144.2.2 AAX_ALIGNMENT_HINT	1254
15.144.2.3 AAX_WORD_ALIGNED_HINT	1255
15.144.2.4 AAX_DWORD_ALIGNED_HINT	1255
15.144.2.5 AAX_LO	1255
15.144.2.6 AAX_HI	1255
15.144.2.7 AAX_INT_LO	1255

15.144.2.8 AAX_INT_HI	1255
15.145 AAX_OtherExtensions.doxygen File Reference	1256
15.146 AAX_Page_Table_Guide.doxygen File Reference	1256
15.147 AAX_PageTableUtilities.h File Reference	1256
15.148 AAX_ParameterAutomation.doxygen File Reference	1256
15.149 AAX_ParameterManager.doxygen File Reference	1256
15.150 AAX_ParameterUpdateProtocol.doxygen File Reference	1256
15.151 AAX_ParameterUpdateTiming.doxygen File Reference	1256
15.152 AAX_PlatformOptimizationConstants.h File Reference	1256
15.152.1 Description	1256
15.152.2 Macro Definition Documentation	1257
15.152.2.1 AAX_PLATFORMOPTIMIZATIONCONSTANTS_H	1257
15.153 AAX_PluginBundleLocation.h File Reference	1257
15.153.1 Description	1257
15.154 AAX_PopStructAlignment.h File Reference	1257
15.154.1 Description	1257
15.155 AAX_PostStructAlignmentHelper.h File Reference	1258
15.155.1 Description	1258
15.156 AAX_PreStructAlignmentHelper.h File Reference	1258
15.156.1 Description	1258
15.157 AAX_Pro_Tools_Guide.doxygen File Reference	1258
15.158 AAX_Properties.h File Reference	1258
15.158.1 Description	1258
15.158.2 Enumeration Type Documentation	1260
15.158.2.1 AAX_EProperty	1260
15.158.3 Function Documentation	1278
15.158.3.1 AAX_ENUM_SIZE_CHECK()	1278
15.159 AAX_Push2ByteStructAlignment.h File Reference	1278
15.159.1 Description	1278
15.159.2 Usage notes	1279
15.160 AAX_Push4ByteStructAlignment.h File Reference	1279
15.160.1 Description	1279
15.160.2 Usage notes	1279
15.161 AAX_Push8ByteStructAlignment.h File Reference	1280
15.161.1 Description	1280
15.161.2 Usage notes	1280
15.162 AAX_Quantize.h File Reference	1280
15.162.1 Description	1281
15.162.2 Macro Definition Documentation	1281
15.162.2.1 AAX_QUANTIZE_H	1281
15.163 AAX_RandomGen.h File Reference	1281
15.163.1 Description	1282

15.163.2 Macro Definition Documentation	1282
15.163.2.1 AAX_RANDOMGEN_H	1282
15.164 AAX_RealTimePerformance.doxygen File Reference	1282
15.165 AAX_RelatedTypes.doxygen File Reference	1282
15.166 AAX_SampleRateUtils.h File Reference	1282
15.166.1 Description	1282
15.166.2 Enumeration Type Documentation	1283
15.166.2.1 ESRUtils	1283
15.166.3 Function Documentation	1283
15.166.3.1 CoarseSampleRate()	1284
15.166.3.2 CoarseSampleRateFactor()	1284
15.166.3.3 CoarseSampleRateIndex()	1284
15.167 AAX_SDK_ChangeLog.doxygen File Reference	1284
15.168 AAX_SDK_ExamplePlugIns.doxygen File Reference	1284
15.169 AAX_SDK_GUIExtensions.doxygen File Reference	1284
15.170 AAX_SessionDocumentTypes.h File Reference	1284
15.170.1 Macro Definition Documentation	1285
15.170.1.1 AAX_SessionDocumentTypes_H	1285
15.170.2 Variable Documentation	1285
15.170.2.1 kAAX_DataBufferType_TempoBreakpointArray	1285
15.171 AAX_SliderConversions.h File Reference	1285
15.171.1 Description	1285
15.171.2 Macro Definition Documentation	1286
15.171.2.1 AAX_SLIDERCONVERSIONS_H	1286
15.171.2.2 AAX_LIMIT	1286
15.171.3 Function Documentation	1286
15.171.3.1 LongControlToNewRange()	1287
15.171.3.2 LongToLongControl()	1287
15.171.3.3 LongControlToDouble()	1287
15.171.3.4 DoubleToLongControl()	1287
15.171.3.5 DoubleToLongControlNonlinear()	1287
15.171.3.6 LongControlToDoubleNonlinear()	1288
15.171.3.7 LongControlToLogDouble()	1288
15.171.3.8 LogDoubleToLongControl()	1288
15.172 AAX_StringUtilities.h File Reference	1288
15.172.1 Description	1289
15.172.2 Macro Definition Documentation	1289
15.172.2.1 AAXLibrary_AAX_StringUtilities_h	1289
15.173 AAX_StringUtilities.hpp File Reference	1289
15.173.1 Macro Definition Documentation	1290
15.173.1.1 DEFINE_AAX_ERROR_STRING	1290
15.174 AAX_TI_Guide.doxygen File Reference	1290

15.175 AAX_TransportTypes.h File Reference	1290
15.175.1 Description	1290
15.175.2 Function Documentation	1291
15.175.2.1 operator==()	1291
15.175.2.2 operator!=()	1291
15.176 AAX_Troubleshooting.doxygen File Reference	1291
15.177 AAX_UIDs.h File Reference	1291
15.177.1 Description	1291
15.177.2 Typedef Documentation	1295
15.177.2.1 AAX_Feature_UID	1295
15.177.2.2 AAX_DocumentData_UID	1295
15.177.3 Variable Documentation	1295
15.177.3.1 AAXCompID_HostServices	1295
15.177.3.2 IID_IAAXHostServicesV1	1295
15.177.3.3 IID_IAAXHostServicesV2	1296
15.177.3.4 IID_IAAXHostServicesV3	1296
15.177.3.5 AAXCompID_AAXCollection	1296
15.177.3.6 IID_IAAXCollectionV1	1296
15.177.3.7 AAXCompID_AAXEffectDescriptor	1296
15.177.3.8 IID_IAAXEffectDescriptorV1	1296
15.177.3.9 IID_IAAXEffectDescriptorV2	1297
15.177.3.10 AAXCompID_AAXComponentDescriptor	1297
15.177.3.11 IID_IAAXComponentDescriptorV1	1297
15.177.3.12 IID_IAAXComponentDescriptorV2	1297
15.177.3.13 IID_IAAXComponentDescriptorV3	1297
15.177.3.14 AAXCompID_AAXPropertyMap	1297
15.177.3.15 IID_IAAXPropertyMapV1	1298
15.177.3.16 IID_IAAXPropertyMapV2	1298
15.177.3.17 IID_IAAXPropertyMapV3	1298
15.177.3.18 AAXCompID_HostProcessorDelegate	1298
15.177.3.19 IID_IAAXHostProcessorDelegateV1	1298
15.177.3.20 IID_IAAXHostProcessorDelegateV2	1298
15.177.3.21 IID_IAAXHostProcessorDelegateV3	1299
15.177.3.22 AAXCompID_AutomationDelegate	1299
15.177.3.23 IID_IAAXAutomationDelegateV1	1299
15.177.3.24 AAXCompID_Controller	1299
15.177.3.25 IID_IAAXControllerV1	1299
15.177.3.26 IID_IAAXControllerV2	1299
15.177.3.27 IID_IAAXControllerV3	1300
15.177.3.28 AAXCompID_PageTableController	1300
15.177.3.29 IID_IAAXPageTableController	1300
15.177.3.30 IID_IAAXPageTableControllerV2	1300

15.177.3.31 AAXCompID_PrivateDataAccess	1300
15.177.3.32 IID_IAAXPrivateDataAccessV1	1300
15.177.3.33 AAXCompID_ViewContainer	1301
15.177.3.34 IID_IAAXViewContainerV1	1301
15.177.3.35 IID_IAAXViewContainerV2	1301
15.177.3.36 IID_IAAXViewContainerV3	1301
15.177.3.37 AAXCompID_Transport	1301
15.177.3.38 IID_IAAXTransportV1	1301
15.177.3.39 IID_IAAXTransportV2	1302
15.177.3.40 IID_IAAXTransportV3	1302
15.177.3.41 IID_IAAXTransportV4	1302
15.177.3.42 IID_IAAXTransportV5	1302
15.177.3.43 AAXCompID_TransportControl	1302
15.177.3.44 IID_IAAXTransportControlV1	1302
15.177.3.45 AAXCompID_PageTable	1303
15.177.3.46 IID_IAAXPageTableV1	1303
15.177.3.47 IID_IAAXPageTableV2	1303
15.177.3.48 AAX_CompID_DescriptionHost	1303
15.177.3.49 IID_IAAXDescriptionHostV1	1303
15.177.3.50 AAX_CompID_FeatureInfo	1303
15.177.3.51 IID_IAAXFeatureInfoV1	1304
15.177.3.52 AAXCompID_Task	1304
15.177.3.53 IID_IAAXTaskV1	1304
15.177.3.54 AAXCompID_SessionDocument	1304
15.177.3.55 IID_IAAXSessionDocumentV1	1304
15.177.3.56 AAXCompID_EffectParameters	1304
15.177.3.57 IID_IAAXEffectParametersV1	1305
15.177.3.58 IID_IAAXEffectParametersV2	1305
15.177.3.59 IID_IAAXEffectParametersV3	1305
15.177.3.60 IID_IAAXEffectParametersV4	1305
15.177.3.61 AAXCompID_HostProcessor	1305
15.177.3.62 IID_IAAXHostProcessorV1	1305
15.177.3.63 IID_IAAXHostProcessorV2	1306
15.177.3.64 AAXCompID_EffectGUI	1306
15.177.3.65 IID_IAAXEffectGUIV1	1306
15.177.3.66 AAXCompID_EffectDirectData	1306
15.177.3.67 IID_IAAXEffectDirectDataV1	1306
15.177.3.68 IID_IAAXEffectDirectDataV2	1306
15.177.3.69 AAXCompID_TaskAgent	1307
15.177.3.70 IID_IAAXTaskAgentV1	1307
15.177.3.71 AAXCompID_SessionDocumentClient	1307
15.177.3.72 IID_IAAXSessionDocumentClientV1	1307

15.177.3.73 AAXCompID_DataBuffer	1307
15.177.3.74 IID_IAAXDataBufferV1	1307
15.177.3.75 AAXATTR_ClientFeature_StemFormat	1308
15.177.3.76 AAXATTR_ClientFeature_AuxOutputStem	1308
15.177.3.77 AAXATTR_ClientFeature_SideChainInput	1308
15.177.3.78 AAXATTR_ClientFeature_MIDI	1308
15.177.3.79 AAXATTR_Client_Level	1308
15.177.3.80 AAXATTR_Client_Version	1309
15.177.3.81 AAX_DocumentDataType_TempoMap	1309
15.178 AAX_UtilsNative.h File Reference	1309
15.178.1 Description	1309
15.178.2 Macro Definition Documentation	1310
15.178.2.1 _AAX_UTILSNATIVE_H_	1310
15.179 AAX_VAutomationDelegate.h File Reference	1310
15.179.1 Description	1310
15.180 AAX_VCollection.h File Reference	1311
15.180.1 Description	1311
15.181 AAX_VComponentDescriptor.h File Reference	1311
15.181.1 Description	1311
15.182 AAX_VController.h File Reference	1311
15.182.1 Description	1312
15.183 AAX_VDataBufferWrapper.h File Reference	1312
15.183.1 Macro Definition Documentation	1312
15.183.1.1 AAX_VDATABUFFERWRAPPER_H_	1312
15.184 AAX_VDescriptionHost.h File Reference	1312
15.185 AAX_VEffectDescriptor.h File Reference	1313
15.185.1 Description	1313
15.186 AAX_VENUE_Guide.doxygen File Reference	1313
15.187 AAX_Version.h File Reference	1313
15.187.1 Description	1313
15.187.2 Macro Definition Documentation	1314
15.187.2.1 _AAX_VERSION_H_	1314
15.187.2.2 AAX_SDK_VERSION	1314
15.187.2.3 AAX_SDK_CURRENT_REVISION	1315
15.187.2.4 AAX_SDK_1p0p1_REVISION	1315
15.187.2.5 AAX_SDK_1p0p2_REVISION	1315
15.187.2.6 AAX_SDK_1p0p3_REVISION	1315
15.187.2.7 AAX_SDK_1p0p4_REVISION	1315
15.187.2.8 AAX_SDK_1p0p5_REVISION	1315
15.187.2.9 AAX_SDK_1p0p6_REVISION	1315
15.187.2.10 AAX_SDK_1p5p0_REVISION	1316
15.187.2.11 AAX_SDK_2p0b1_REVISION	1316

15.187.2.12 AAX_SDK_2p0p0_REVISION	1316
15.187.2.13 AAX_SDK_2p0p1_REVISION	1316
15.187.2.14 AAX_SDK_2p1p0_REVISION	1316
15.187.2.15 AAX_SDK_2p1p1_REVISION	1316
15.187.2.16 AAX_SDK_2p2p0_REVISION	1316
15.187.2.17 AAX_SDK_2p2p1_REVISION	1316
15.187.2.18 AAX_SDK_2p2p2_REVISION	1317
15.187.2.19 AAX_SDK_2p3p0_REVISION	1317
15.187.2.20 AAX_SDK_2p3p1_REVISION	1317
15.187.2.21 AAX_SDK_2p3p2_REVISION	1317
15.187.2.22 AAX_SDK_2p4p0_REVISION	1317
15.187.2.23 AAX_SDK_2p4p1_REVISION	1317
15.187.2.24 AAX_SDK_2p5p0_REVISION	1317
15.187.2.25 AAX_SDK_2p6p0_REVISION	1317
15.187.2.26 AAX_SDK_2p6p1_REVISION	1318
15.187.2.27 AAX_SDK_2p7p0_REVISION	1318
15.187.2.28 AAX_SDK_2p8p0_REVISION	1318
15.187.2.29 AAX_SDK_2p8p1_REVISION	1318
15.188 AAX_VFeatureInfo.h File Reference	1318
15.189 AAX_VHostProcessorDelegate.h File Reference	1318
15.189.1 Description	1318
15.190 AAX_VHostServices.h File Reference	1319
15.190.1 Description	1319
15.191 AAX_VHostTaskAgent.h File Reference	1319
15.191.1 Macro Definition Documentation	1319
15.191.1.1 AAX_VHostTaskAgent_H	1319
15.192 AAX_VPageTable.h File Reference	1320
15.193 AAX_VPrivateDataAccess.h File Reference	1320
15.193.1 Description	1320
15.194 AAX_VPropertyMap.h File Reference	1320
15.194.1 Description	1320
15.195 AAX_VSessionDocument.h File Reference	1321
15.195.1 Macro Definition Documentation	1321
15.195.1.1 AAX_VSessionDocument_H	1321
15.196 AAX_VTask.h File Reference	1321
15.196.1 Macro Definition Documentation	1322
15.196.1.1 AAX_VTask_H	1322
15.197 AAX_VTransport.h File Reference	1322
15.197.1 Description	1322
15.198 AAX_VViewContainer.h File Reference	1322
15.198.1 Description	1322
15.199 DSH_Guide.doxygen File Reference	1322

15.200 DTT_Guide.doxygen File Reference	1322
15.201 ReadMe.doxygen File Reference	1322
Index	1323

Chapter 1

Main Page

[AAX SDK Manual](#)

1.1 Welcome to AAX

Select the "Manual" tab to see a full list of documentation pages, or choose from the topics below.

Note

Looking for something? The search function only includes indexing of code symbols and page titles. To search for specific text strings in the AAX SDK manual it is best to use a text search tool such as grep or FINDSTR on the AAX SDK directory or search for the desired text within the PDF version of the AAX SDK documentation.

1.1.1 The Basics

New to AAX? Read through the documentation pages listed below to get started!

- See [Getting Started with AAX](#) for a general overview of AAX and a walk-through of the DemoGain example plug-in
- Read through the first few sections of the [Pro Tools Guide](#) if you are new to Pro Tools
- Read the [Digital signature](#) section of the [Pro Tools Guide](#) to review the digital signing requirements for compatibility with Pro Tools
- Review the [sample plug-ins](#) for examples of both basic and advanced AAX features
- See [Core AAX Interface](#) to find out more about the basic structure of AAX plug-ins
- See [Real-time algorithm callback](#) to learn more about audio processing in AAX
- See [Data model interface](#) to learn about adding parameters and controls to your plug-in,
- See [Description callback](#) for more information about plug-in configuration and initial set-up
- See the [HDX DSP Guide](#) to find out how to add AAX DSP support to your plug-in for Avid's HDX and Pro Tools | Carbon products
- Ready to ship your new plug-in? See [Distributing Your AAX Plug-In](#) for information about finalizing and distributing your AAX products
- Check out the [Troubleshooting](#) page if you're having problems, or post a question to the [developer forums](#) and an Avid engineer will be happy to assist you.

1.1.2 More Topics

Have a more specific question? Review the pages below or view the full list of documentation pages under the "Manual" tab above.

- See [AAX_IController](#) to see the interface that an AAX plug-in's host-based modules use to interact with the host
- See [AAX communication protocols](#) to find out more about how different modules in an AAX plug-in communicate with one another
- See [Offline processing interface](#) for information about creating advanced non-real-time AAX plug-ins
- See [Taper delegates](#) and [Display delegates](#) for more information about implementing custom parameter and control behavior
- Look in the /TI/SignalProcessing folder for signal processing utility classes and functions available for optimizing on Native and DSP

1.1.3 Test Tools & Utilities

- The [DSH Guide](#) has information about the tool for testing basic functionality of your plug-in
- See [DTT Guide](#) to learn how to automate different test scenarios for DSH

1.1.4 Supplemental Information

- [Example Plug-Ins](#)
- [Change Log](#)
- [Host Support](#)
- [Known Issues](#)

1.2 SDK Folder Hierarchy

Documentation

SDK documentation

ExamplePlugins

Example plug-in projects [More information](#)

Extensions

Demonstrations of how to extend the AAX SDK, for example by incorporating third-party GUI frameworks into AAX plug-ins. [More information](#)

Interfaces

Interface headers and other resources required for use of the AAX SDK library

Libs

Source code for the AAX SDK library, a collection of default implementations and utility classes for use in all AAX plug-ins

TI

Various resources for use with TI's Code Composer Studio IDE and Avid's TI testing toolset

Utilities

Common SDK utilities and resources

1.3 Contacting Avid

Your personal [avid user account](#) is your hub for AAX Toolkit services and developer support.

Log in at [avid.com](#) for access to the full range of tools and services provided to AAX developers, including the AAX [developer forum](#). If you have any questions on the AAX SDK documentation or require support with AAX development, we encourage you to post them to the forum as your first line of inquiry.

If you have time-sensitive or critical support inquiries, contact the AAX development team directly at avid.developer.services@avid.com. Any AAX questions sent to this alias will be promptly addressed by the most appropriate contact here at Avid.

If you require NFR (Not For Resale) licenses to Avid software for AAX development please send an e-mail to devauth@avid.com with "License Request" in the subject.

If you require access to the digital signing toolkit from PACE Anti-Piracy, Inc. for compatibility with Pro Tools then please follow the instructions [here](#).

The following chart describes these and other ways of connecting with Avid to take advantage of the services provided to AAX developers:

1.4 Licensing

Unless you have entered into a commercial agreement with Avid, you are using this SDK under an evaluation agreement. To review this agreement, see the [AAX Toolkit downloads](#) section under your [my.avid.com](#) account.

As an Avid Developer, you are invited to offer your products on [Avid Marketplace](#) and via [Avid Link](#). If you wish to sell them independently or through other commercial outlets, an authorized representative from your organization is required to sign our Commercial License, which you can read and click through [here](#).

Chapter 2

Todo List

Member [AAX_CAudioInPort](#)

Not used directly by AAX plug-ins

Member [AAX_CAudioOutPort](#)

Not used directly by AAX plug-ins

Class [AAX_CChunkDataParser](#)

Update this documentation for [AAX](#)

Member [AAX_CComponentID](#)

Not used by AAX plug-ins

Member [AAX_CCount](#)

Not used by AAX plug-ins

Member [AAX_CEffectDirectData::Controller](#) (void)

Change to GetController to match other AAX_CEffect modules

Member [AAX_CEffectDirectData::EffectParameters](#) (void)

Change to GetController to match other AAX_CEffect modules

Member [AAX_CEffectGUI::UpdateAllParameters](#) (void)

Rename to `UpdateAllParameterViews()` or another name that does not lead to confusion regarding what exactly this method should be doing.

Member [AAX_CIndex](#)

Not used by AAX plug-ins (except as [AAX_CFieldIndex](#))

Member [AAX_CMeterID](#)

Not used by AAX plug-ins

Member [AAX_CMeterPort](#)

Not used directly by AAX plug-ins

Class [AAX_CParameterManager](#)

Should the Parameter Manager return error codes?

Member [AAX_CParameterManager::AddParameter](#) ([AAX_IParameter](#) *param)

Should this method return success/failure code?

Member [AAX_CParameterManager::RemoveAllParameters](#) ()

Should this method return success/failure code?

Member [AAX_CParameterManager::RemoveParameter](#) ([AAX_IParameter](#) *param)

Should this method return success/failure code?

Member **AAX_CParameterManager::RemoveParameterByID** (AAX_CParamID identifier)

Should this method return success/failure code?

Member **AAX_CRangeTaperDelegate< T, RealPrecision >::AAX_CRangeTaperDelegate** (T *range, double *rangesSteps, unsigned long numRanges, bool useSmartRounding=true)

Document useSmartRounding parameter

Member **AAX_CRangeTaperDelegate< T, RealPrecision >::SmartRound** (double value) const

Document

Member **AAX_CSelector**

Clean up usage; currently used for a variety of ID-related values

Member **AAX_EParameterOrientationBits**

FLAGGED FOR REVISION

Member **AAX_EParameterType**

FLAGGED FOR REMOVAL

Member **AAX_IACFEffEffectGUI::SetControlHighlightInfo** (AAX_CParamID iParameterID, AAX_CBoolean iIsHighlighted, AAX_EHighlightColor iColor)=0

Document this method

Class **AAX_IACFEffEffectParameters**

Add documentation for expected error state return values

Member **AAX_IACFEffEffectParameters::GetParameterOrientation** (AAX_CParamID iParameterID, AAX_EParameterOrientation *oParameterOrientation) const =0

update this documentation

Member **AAX_IACFEffEffectParameters::GetParameterType** (AAX_CParamID iParameterID, AAX_EParameterType *oParameterType) const =0

The concept of parameter type needs more documentation

Member **AAX_IACFEffEffectParameters::SetParameterDefaultNormalizedValue** (AAX_CParamID iParameterID, double iValue)=0

THIS IS NOT CALLED FROM HOST. USEFUL FOR INTERNAL USE ONLY?

Member **AAX_IACFEffEffectParameters::SetParameterNormalizedRelative** (AAX_CParamID iParameterID, double iValue)=0

NOT CURRENTLY CALLED FROM THE HOST. USEFUL FOR INTERNAL USE ONLY?

REMOVE THIS METHOD (?)

Member **AAX_IACFEffEffectParameters::Uninitialize** ()=0

Docs: When exactly is **AAX_IACFEffEffectParameters::Uninitialize()** called, and under what conditions?

Member **AAX_IACFEffEffectParameters::UpdateParameterNormalizedValue** (AAX_CParamID iParameterID, double iValue, AAX_EUpdateSource iSource)=0

FLAGGED FOR CONSIDERATION OF REVISION

Class **AAX_IACFEffEffectParameters_V2**

Add documentation for expected error state return values

Class **AAX_IACFEffEffectParameters_V3**

Add documentation for expected error state return values

Class **AAX_IACFEffEffectParameters_V4**

Add documentation for expected error state return values

Member **AAX_IComponentDescriptor::AddDmaInstance** (AAX_CFieldIndex inFieldIndex, AAX_IDma::EMode inDmaMode)=0

Update the DMA system management such that operation priority can be set arbitrarily

Member **AAX_IComponentDescriptor::AddProcessProc** (AAX_IPropertyMap *inProperties, AAX_CSelector *outProcIDs=NULL, int32_t inProcIDsSize=0)=0

document this parameter Returned array will be NULL-terminated

Member [AAX_IComponentDescriptor::AddProcessProc_Native](#) (AAX_CProcessProc inProcessProc, [AAX_IPropertyMap](#) *inProperties=NULL, AAX_CInstanceInitProc inInstanceInitProc=NULL, AAX_CBackgroundProc inBackgroundProc=NULL, AAX_CSelector *outProcID=NULL)=0

document this parameter

Member [AAX_IComponentDescriptor::AddProcessProc_TI](#) (const char inDLLFileNameUTF8[], const char inProcessProcSymbol[], [AAX_IPropertyMap](#) *inProperties, const char inInstanceInitProcSymbol[]=NULL, const char inBackgroundProcSymbol[]=NULL, AAX_CSelector *outProcID=NULL)=0

document this parameter

Member [AAX_IController::GetCycleCount](#) (AAX_EProperty inWhichCycleCount, AAX_CPropertyValue *outNumCycles) const =0

PLACEHOLDER - NOT CURRENTLY IMPLEMENTED IN HOST

Member [AAX_IController::SetCycleCount](#) (AAX_EProperty *inWhichCycleCounts, AAX_CPropertyValue *iValues, int32_t numValues)=0

PLACEHOLDER - NOT CURRENTLY IMPLEMENTED IN HOST

Member [AAX_IDma::IsTransferComplete](#) ()=0

Clarify return value meaning – ambiguity in documentation

Member [AAX_IParameter::GetType](#) () const =0

Document use cases for control type

Member [AAX_IParameter::SetTaperDelegate](#) ([AAX_ITaperDelegateBase](#) &inTaperDelegate, bool inPreserveValue)=0

Document this parameter

Module [additionalFeatures_Sidechain](#)

Is properties->AddProperty (AAX_eProperty_SupportsSideChainInput, true) even necessary?!?! I believe I saw a p.i. that does not declare this...

Module [advancedTopics_parameterUpdates_sequences](#)

Update this section with information about default chunk setting, which is a separate step following the procedure described below.

Member [DBToGain](#) (double dB)

This should be incorporated into parameters' tapers and not called separately

Member [GainToDB](#) (double aGain)

This should be incorporated into parameters' tapers and not called separately

Chapter 3

Host Compatibility Notes

Member [AAX_CMidiPacket::mIsImmediate](#)

This value is not currently set. Use `mTimestamp == 0` to detect immediate packets

Member [AAX_CParameter< T >::AAX_CParameter](#) ([AAX_CParamID](#) identifier, [const AAX_IString &name](#), [T defaultValue](#), [const AAX_ITaperDelegate< T > &taperDelegate](#), [const AAX_IDisplayDelegate< T > &displayDelegate](#), [bool automatable=false](#))

As of Pro Tools 10.2, DAE will check for a matching parameter NAME and not an ID when reading in automation data from a session saved with an AAX plug-ins RTAS/TDM counter part.

As of Pro Tools 11.1, AAE will first try to match ID. If that fails, AAE will fall back to matching by Name.

Module [AAX_DigiTrace_Guide](#)

This feature is available in Pro Tools 12.6 and higher

Member [AAX_eConstraintLocationMask_DLLChipAffinity](#)

This constraint is supported in Pro Tools 10.2 and higher

Member [AAX_eCurveType_Dynamics](#)

Pro Tools requests this curve type for [Dynamics](#) plug-ins only

Member [AAX_eCurveType_EQ](#)

Pro Tools requests this curve type for [EQ](#) plug-ins only

Member [AAX_eCurveType_Reduction](#)

Pro Tools requests this curve type for [Dynamics](#) plug-ins only

Member [AAX_eDataInPortType_Incremental](#)

Supported in Pro Tools 12.5 and higher; when [AAX_eDataInPortType_Incremental](#) is not supported the port will be treated as [AAX_eDataInPortType_Unbuffered](#)

Member [AAX_EHostModeBits](#)

Supported in Venue 5.6 and higher

Member [AAX_eNotificationEvent_ASPreviewState](#)

Supported in Pro Tools 11 and higher

Not supported by Media Composer

Member [AAX_eNotificationEvent_ASProcessingState](#)

Supported in Pro Tools 11 and higher

Not supported by Media Composer

Member [AAX_eNotificationEvent_DelayCompensationState](#)

Supported in Pro Tools 12.6 and higher

Member [AAX_eNotificationEvent_EnteringOfflineMode](#)

Supported in Pro Tools 11 and higher

Member [AAX_eNotificationEvent_ExitingOfflineMode](#)

Supported in Pro Tools 11 and higher

Member [AAX_eNotificationEvent_HostLocale](#)

Supported in Pro Tools 2024.3 and higher

Member [AAX_eNotificationEvent_HostModeChanged](#)

Supported in Venue 5.6 and higher

Member [AAX_eNotificationEvent_LogState](#)

Pro Tools currently only sends this notification to the Direct Data object in the plug-in

Member [AAX_eNotificationEvent_MaxViewSizeChanged](#)

Supported in Pro Tools 11.1 and higher

Member [AAX_eNotificationEvent_ParameterNameChanged](#)

Supported in Pro Tools 2023.3 and higher

Member [AAX_eNotificationEvent_PresetOpened](#)

Supported in Pro Tools 11 and higher

Member [AAX_eNotificationEvent_PriorSettingsInvalid](#)

Supported in Venue 5.6 and higher

Member [AAX_eNotificationEvent_SessionBeingOpened](#)

Supported in Pro Tools 11 and higher

Not supported by Media Composer

Member [AAX_eNotificationEvent_SessionPathChanged](#)

Supported in Pro Tools 11.1 and higher

Member [AAX_eNotificationEvent_SideChainBeingConnected](#)

Supported in Pro Tools 11.1 and higher

Member [AAX_eNotificationEvent_SideChainBeingDisconnected](#)

Supported in Pro Tools 11.1 and higher

Member [AAX_eNotificationEvent_SignalLatencyChanged](#)

Supported in Pro Tools 11.1 and higher

Member [AAX_eNotificationEvent_TrackNameChanged](#)

Supported in Pro Tools 11.2 and higher

Not supported by Media Composer

Member [AAX_eNotificationEvent_TransportStateChanged](#)

Supported in Pro Tools 2021.10 and higher

Member [AAX_ePlugInStrings_Progress](#)

Not currently supported by Pro Tools

Member [AAX_eProcessingState_BeginPassGroup](#)

AudioSuite pass group notifications are supported starting in Pro Tools 12.0

Member [AAX_eProcessingState_EndPassGroup](#)

AudioSuite pass group notifications are supported starting in Pro Tools 12.0

Member [AAX_eProperty_Constraint_NeverUnload](#)

AAX_eProperty_Constraint_NeverUnload is not currently implemented in DAE or AAE

Member [AAX_eProperty_DestinationTrack](#)

This property is not supported on Media Composer

Member [AAX_eProperty_LatencyContribution](#)

Maximum delay compensation limits will vary from host to host. If your plug-in exceeds the delay compensation sample limit for a given AAX host then you should note this limitation in your user documentation. Example limits:

- Pro Tools 9 and higher: 16,383 samples at 44.1/48 kHz, 32,767 samples at 88.2/96 kHz, or 65,534 samples at 176.4/192 kHz
- Media Composer 8.1 and higher: 16,383 samples at 44.1/48 kHz, 32,767 samples at 88.2/96 kHz

Member [AAX_eProperty_OptionalAnalysis](#)

In Media Composer, optional analysis will also be performed automatically before each channel is rendered. See [MCDEV-2904](#)

Member [AAX_eProperty_SideChainStemFormat](#)

Currently Pro Tools supports only [AAX_eStemFormat_Mono](#) side chain inputs

[AAX_eProperty_SideChainStemFormat](#) is not currently implemented in DAE or AAE

Member [AAX_eProperty_UsesClientGUI](#)

Currently supported by Pro Tools only

Member [AAX_IACFEffParameters::CompareActiveChunk](#) (const [AAX_SPlugInChunk](#) *iChunkP, [AAX_CBoolean](#) *olsEqual) const =0

In Pro Tools, this method will only be called if a prior call to [GetNumberOfChanges\(\)](#) has indicated that the plug-in's state has changed. If the plug-in's current settings are different from the settings in [aChunkP](#) then the plug-in's Compare Light will be illuminated in the plug-in header, allowing users to toggle between the plug-in's custom state and its saved state.

Member [AAX_IACFEffParameters::GetCurveData](#) ([AAX_CTypeID](#) iCurveType, const float *iValues, [uint32_t](#) iNumValues, float *oValues) const =0

Versions of S6 software which support the [GetCurveDataDisplayRange\(\)](#) method will not display a plug-in's curve data unless both [GetCurveData\(\)](#) and [GetCurveDataDisplayRange\(\)](#) are supported by the plug-in.

Member [AAX_IACFEffParameters::GetParameterNameOfLength](#) ([AAX_CParamID](#) iParameterID, [AAX_IString](#) *oName, [int32_t](#) iNameLength) const =0

In most cases, the AAX host will call [GetParameterName\(\)](#) or [GetParameterNameOfLength\(\)](#) to retrieve parameter names for display. However, when Pro Tools is retrieving a plug-in name for display on a control surface the XML data stored in the plug-in's page tables will be used in preference to values retrieved from these methods.

Member [AAX_IComponentDescriptor::AddAuxOutputStem](#) ([AAX_CFieldIndex](#) inFieldIndex, [int32_t](#) inStemFormat, const char inNameUTF8[]) =0

There is a hard limit to the number of outputs that Pro Tools supports for a single plug-in instance. This limit is currently set at 256 channels, which includes all of the plug-in's output channels in addition to the sum total of all of its aux output stem channels.

Pro Tools supports only mono and stereo auxiliary output stem formats

Member [AAX_IComponentDescriptor::AddClock](#) ([AAX_CFieldIndex](#) inFieldIndex) =0

As of Pro Tools 11.1, this field may be used in both Native and DSP plug-ins. The DSP clock data is a 16-bit cycling counter. This field was only available for Native plug-ins in previous Pro Tools versions.

Member [AAX_IComponentDescriptor::AddMIDINode](#) ([AAX_CFieldIndex](#) inFieldIndex, [AAX_EMIDINodeType](#) inNodeType, const char inNodeName[], [uint32_t](#) channelMask) =0

Due to current restrictions MIDI data won't be delivered to DSP algorithms, only to AAX Native.

Member [AAX_IController::GetHostName](#) ([AAX_IString](#) *outHostNameString) const =0

Pro Tools versions from Pro Tools 11.0 to Pro Tools 12.3.1 will return a generic version string to this call. This issue is resolved beginning in Pro Tools 12.4.

Member [AAX_IMIDINode::PostMIDIPacket](#) ([AAX_CMidiPacket](#) *packet) =0

Pro Tools supports the following MIDI events from plug-ins:

- NoteOn
- NoteOff

- Pitch bend
- Polyphonic key pressure
- Bank select (controller #0)
- Program change (no bank)
- Channel pressure

Member [AAX_ITransport::GetBarBeatPosition](#) (int32_t *Bars, int32_t *Beats, int64_t *DisplayTicks, int64_t *SampleLocation) const =0

There is a minor performance cost associated with using this API in Pro Tools. It should not be used excessively without need.

Member [AAX_ITransport::GetCurrentLoopPosition](#) (bool *bLooping, int64_t *LoopStartTick, int64_t *LoopEndTick) const =0

This does not indicate anything about the status of the "Loop Record" option. Even when the host is configured to loop playback, looping may not occur if certain conditions are not met (i.e. the length of the selection is too short)

Member [AAX_ITransport::GetCurrentTickPosition](#) (int64_t *TickPosition) const =0

The tick resolution here is different than that of the tick displayed in Pro Tools. "Display ticks" (as they are called) are 1/960 of a quarter note.

Member [AAX_ITransport::GetCustomTickPosition](#) (int64_t *oTickPosition, int64_t iSampleLocation) const =0

There is a minor performance cost associated with using this API in Pro Tools. It should not be used excessively without need.

Member [AAX_IViewContainer::GetModifiers](#) (uint32_t *outModifiers)=0

Although this method allows plug-ins to acquire the current state of the Windows key (normally blocked by Pro Tools), plug-ins should not use key combinations that require this key.

Module [AAX_Media_Composer_Guide](#)

Some early versions of Media Composer 8 do not search the system plug-ins directory recursively. If your plug-ins are installed into a sub-directory beneath this main directory then they will not be loaded by the affected versions of Media Composer.

Module [AAX_Page_Table_Guide](#)

Pro Tools versions prior to Pro Tools 11.1 use plug-ins' ProControl and ICON page tables (Dynamics, EQ, Channel Strip, Custom Fader, etc.) to map plug-in parameters to EUCON-enabled surfaces, so be sure that your plug-ins also implement these page tables correctly so that users with earlier versions of Pro Tools can have the best possible experience when using your plug-ins.

Module [AAX_Pro_Tools_Guide](#)

Pro Tools requires PACE Eden digital signatures for AAX plug-ins.

Supported in Pro Tools 2019.XX and higher. Also supported (and enabled by default) in Pro Tools developer builds beginning with Pro Tools 2019.6.

Module [AAX_TI_Guide](#)

32 and 64-sample quantum is available in Pro Tools 10.2 and higher

Beginning in Pro Tools 11, AAX DSP algorithms also support optional temporary data spaces that can be described in the Describe module and are shared among all instances on a DSP. This is an alternative to declaring large data blocks on the stack for better memory management and to prevent stack overflows. Please refer to [AAX_IComponentDescriptor::AddTemporaryData\(\)](#) for usage instructions.

Module [AdditionalFeatures_CurveDisplays](#)

For S6 control surface displays, see [PT-226228](#) and [PT-226227](#) in the [Known Issues](#) page for more information about the requirements listed in this section.

Module [advancedTopics_relatedTypes](#)

Pro Tools versions prior to Pro Tools 12.3 do not allow explicit type conversion between types with different product ID values. Beginning in Pro Tools 12.3 both the product ID and the plug-in ID may differ between explicitly related types.

Module [AuxInterface_TaskAgent](#)

This interface is not yet used in any AAX hosts

Module [CommonInterface_Algorithm](#)

As of Pro Tools 10.2.1 an algorithm's initialization callback routine will have up to 5 seconds to execute.

Module [CommonInterface_FormatSpecification](#)

*_ACFGetSDKVersion is required for 64-bit AAX plug-ins only

Module [ExamplePlugins](#)

The DemoDelay_DynamicLatencyComp example is compatible with Pro Tools 11.1 and higher.

Chapter 4

Legacy Porting Notes

Class [AAX_CEffectGUI](#)

The default implementations in this class are mostly derived from their equivalent implementations in CProcess and CEffectProcess. For additional CProcess-derived implementations, see [AAX_CEffectParameters](#).

Class [AAX_CEffectParameters](#)

The default implementations in this class are mostly derived from their equivalent implementations in CProcess and CEffectProcess. For additional CProcess-derived implementations, see [AAX_CEffectGUI](#).

Member [AAX_CHostProcessor::AnalyzeAudio](#) (const float *const inAudiolns[], int32_t inAudiolnCount, int32_t *ioWindowSize) **AAX_OVERRIDE**

Ported from AudioSuite's AnalyzeAudio(bool isMasterBypassed) method

Member [AAX_CHostProcessor::InitOutputBounds](#) (int64_t iSrcStart, int64_t iSrcEnd, int64_t *oDstStart, int64_t *oDstEnd) **AAX_OVERRIDE**

DAE no longer makes use of the mStartBound and mEndBounds member variables that existed in the legacy RTAS/TDM SDK. Use oDstStart and oDstEnd instead (preferably by overriding [TranslateOutputBounds\(\)](#).)

Member [AAX_CHostProcessor::RenderAudio](#) (const float *const inAudiolns[], int32_t inAudiolnCount, float *const iAudioOuts[], int32_t iAudioOutCount, int32_t *ioWindowSize) **AAX_OVERRIDE**

This method is a replacement for the AudioSuite ProcessAudio method

Class [AAX_CMidiPacket](#)

Corresponds to DirectMidiPacket in the legacy SDK

Class [AAX_CMidiStream](#)

Corresponds to DirectMidiNode in the legacy SDK

Member [AAX_eMIDINodeType_Global](#)

Corresponds to RTAS Shared Buffer global nodes in the legacy SDK

Member [AAX_eMIDINodeType_LocalInput](#)

Corresponds to RTAS Buffered MIDI input nodes in the legacy SDK

Member [AAX_eMIDINodeType_LocalOutput](#)

Corresponds to RTAS Buffered MIDI output nodes in the legacy SDK

Member [AAX_eNotificationEvent_ASPreviewState](#)

Replacement for SetPreviewState()

Member [AAX_ePageTable_EQ_Band_Type](#)

converted from eDigi_PageTable_EQ_Band_Type in the legacy SDK

Member [AAX_ePageTable_EQ_InCircuitPolarity](#)

converted from eDigi_PageTable_EQ_InCircuitPolarity in the legacy SDK

Member [AAX_ePageTable_UseAlternateControl](#)

converted from `eDigi_PageTable_UseAlternateControl` in the legacy SDK

Member [AAX_EParameterType](#)

Values must match unnamed type enum in `FicTDMControl.h`

Member [AAX_eParameterType_Continuous](#)

Matches `kDAE_ContinuousValues`

Member [AAX_eParameterType_Discrete](#)

Matches `kDAE_DiscreteValues`

Member [AAX_EParameterValueInfoSelector](#)

converted from `EControlValueInfo` in the legacy SDK

Member [AAX_ePluginStrings_AllSelectedRegionsAnalysis](#)

Was `pluginStrings_AllSelectedRegionsAnalysis` in the RTAS/TDM SDK

Member [AAX_ePluginStrings_Analysis](#)

Was `pluginStrings_Analysis` in the RTAS/TDM SDK

Member [AAX_ePluginStrings_Bypass](#)

Was `pluginStrings_Bypass` in the RTAS/TDM SDK

Member [AAX_ePluginStrings_ClipName](#)

Was `pluginStrings_RegionName` in the RTAS/TDM SDK

Member [AAX_ePluginStrings_MonoMode](#)

Was `pluginStrings_MonoMode` in the RTAS/TDM SDK

Member [AAX_ePluginStrings_MultiInputMode](#)

Was `pluginStrings_MultiInputMode` in the RTAS/TDM SDK

Member [AAX_ePluginStrings_Process](#)

Was `pluginStrings_Process` in the RTAS/TDM SDK

Member [AAX_ePluginStrings_Progress](#)

Was `pluginStrings_Progress` in the RTAS/TDM SDK

Member [AAX_ePluginStrings_RegionByRegionAnalysis](#)

Was `pluginStrings_RegionByRegionAnalysis` in the RTAS/TDM SDK

Member [AAX_EProperty](#)

These property IDs are somewhat analogous to the `pluginGestalt` system in the legacy SDK, and several [AAX_EProperty](#) values correlate directly with a corresponding legacy plug-in gestalt.

To ensure session interchange compatibility, make sure the 4 character IDs for [AAX_eProperty_ManufacturerID](#), [AAX_eProperty_ProductID](#), [AAX_eProperty_PluginID_Native](#), and [AAX_eProperty_PluginID_AudioSuite](#) are identical to the legacy SDK's counterpart.

Member [AAX_eProperty_AllowPreviewWithoutAnalysis](#)

Was `pluginGestalt_AnalyzeOnTheFly`

Member [AAX_eProperty_CanBypass](#)

Was `pluginGestalt_CanBypass`.

Member [AAX_eProperty_ContinuousOnly](#)

Was `pluginGestalt_ContinuousOnly`

Member [AAX_eProperty_DestinationTrack](#)

Was `pluginGestalt_DestinationTrack`

Member [AAX_eProperty_DisablePreview](#)

Was `pluginGestalt_DisablePreview`

Member [AAX_eProperty_DoesntIncrOutputSample](#)

Was `pluginGestalt_DoesntIncrOutputSample`

Member [AAX_eProperty_ManufacturerID](#)

For legacy plug-in session compatibility, this ID should match the Manufacturer ID used in the corresponding legacy plug-ins.

Member [AAX_eProperty_MultiInputModeOnly](#)

Was pluginGestalt_MultiInputModeOnly

Member [AAX_eProperty_NeedsOutputDithered](#)

Was pluginGestalt_NeedsOutputDithered

Member [AAX_eProperty_OptionalAnalysis](#)

Was pluginGestalt_OptionalAnalysis

Member [AAX_eProperty_PluginID_AudioSuite](#)

For legacy plug-in session compatibility, this ID should match the Type ID used in the corresponding legacy AudioSuite plug-in Types.

Member [AAX_eProperty_PluginID_Native](#)

For legacy plug-in session compatibility, this ID should match the Type ID used in the corresponding legacy RTAS plug-in Types.

Member [AAX_eProperty_PluginID_Ti](#)

For legacy plug-in session compatibility, this ID should match the Type ID used in the corresponding legacy TDM plug-in Types.

Member [AAX_eProperty_ProductID](#)

For legacy plug-in session compatibility, this ID should match the Product ID used in the corresponding legacy plug-in.

Member [AAX_eProperty_RequestsAllTrackData](#)

Was pluginGestalt_RequestsAllTrackData

Member [AAX_eProperty_RequiresAnalysis](#)

Was pluginGestalt_RequiresAnalysis

Member [AAX_eProperty_SupportsSaveRestore](#)

Was pluginGestalt_SupportsSaveRestore

Member [AAX_eProperty_UsesRandomAccess](#)

Was pluginGestalt_UsesRandomAccess

Class [AAX_IACFEffEffectGUI](#)

In the legacy plug-in SDK, these methods were found in CProcess and CEffectProcess. For additional CProcess methods, see [AAX_IEffectParameters](#).

Member [AAX_IACFEffEffectGUI::SetControlHighlightInfo](#) (AAX_CParamID iParameterID, AAX_CBoolean iIsHighlighted, AAX_EHighlightColor iColor)=0

This method was re-named from `SetControlHighliteInfo()`, its name in the legacy plug-in SDK.

Member [AAX_IACFEffEffectParameters::GetChunkSize](#) (AAX_CTypeID iChunkID, uint32_t *oSize) const =0

In [AAX](#), the value provided by `GetChunkSize()` should *NOT* include the size of the chunk header. The value should *ONLY* reflect the size of the chunk's data.

Member [AAX_IACFEffEffectParameters::GetParameterOrientation](#) (AAX_CParamID iParameterID, AAX_EParameterOrientation *oParameterOrientation) const =0

[AAX_IEffectParameters::GetParameterOrientation\(\)](#) corresponds to the `GetControlOrientation()` method in the legacy RTAS/TDM SDK.

Member [AAX_IACFEffEffectParameters::GetParameterStringFromValue](#) (AAX_CParamID iParameterID, double iValue, AAX_IString *oValueString, int32_t iMaxLength) const =0

This method corresponds to `CProcess::MapControlValToString()` in the RTAS/TDM SDK

Member [AAX_IACFEffEffectParameters::GetParameterValueFromString](#) (AAX_CParamID iParameterID, double *oValue, const AAX_IString &iValueString) const =0

This method corresponds to `CProcess::MapControlStringToVal()` in the RTAS/TDM SDK

Class [AAX_IACFHostProcessor](#)

This interface provides offline processing features analogous to the legacy AudioSuite plug-in architecture

Class [AAX_ICollection](#)

The information in [AAX_ICollection](#) is roughly analogous to the information provided by `CProcessGroup` in the legacy plug-in library

Class [AAX_IEffectParameters](#)

In the legacy plug-in SDK, these methods were found in `CProcess` and `CEffectProcess`. For additional `CProcess` methods, see [AAX_IEffectGUI](#).

File [AAX_SliderConversions.h](#)

These utilities may be required in order to maintain settings chunk compatibility with plug-ins that were ported from the legacy RTAS/TDM format.

Class [AAX_SPlugInChunkHeader](#)

To ensure compatibility with TDM/RTAS plug-ins whose implementation requires `fSize` to be equal to the size of the chunk's header plus its data, AAE performs some behind-the-scenes record keeping.

The following actions are only taken for AAX plug-ins, so, e.g., if a chunk is stored by an RTAS or TDM plug-in that reports data+header size in `fSize` and this chunk is then loaded by the AAX version of the plug-in, the header size will be cached as-is from the legacy plug-in and will be subtracted out before the chunk data is passed to the AAX plug-in. If a chunk is stored by an AAX plug-in and is then loaded by a legacy plug-in, the legacy plug-in will receive the cached plug-in header with `fSize` equal to the data+header size.

These are the special actions that AAE takes to ensure backwards-compatibility when handling AAX chunk data:

- When AAE retrieves the size of a chunk from an AAX plug-in using [GetChunkSize\(\)](#), it adds the chunk header size to the amount of memory that it allocates for the chunk
- When AAE retrieves a chunk from an AAX plug-in using [GetChunk\(\)](#), it adds the chunk header size to `fChunkSize` before caching the chunk
- Before calling [SetChunk\(\)](#) or [CompareActiveChunk\(\)](#), AAE subtracts the chunk header size from the cached chunk's header's `fChunkSize` member

Module [AdditionalFeatures_Meters](#)

The gain-reduction meter handling for AAX plug-ins is different from that for RTAS/TDM plug-ins. AAX plug-ins must invert their gain-reduction meter values manually before reporting these values from the audio processing callback. The AAX host will always thin reported meter data using a "max" operation, and will later invert gain-reduction meter values before they are available to the plug-in GUI or to control surfaces.

Chapter 5

Deprecated List

Member [AAX::FastRndDbl2Int32](#) (double iVal)

Member [AAX_CInitPrivateDataProc](#)

Member [AAX_CPacketAllocator](#)

Member [AAX_CTaskAgent::AddTask](#) (std::unique_ptr< AAX_ITask > iTask)
Use [ReceiveTask\(\)](#) instead

Member [AAX_EHostMode](#)
This enum is deprecated and will be removed in a future release.

Member [AAX_eHostMode_Config](#)
Use [AAX_eHostModeBits_None](#)

Member [AAX_eHostMode_Show](#)
Use [AAX_eHostModeBits_Live](#)

Member [AAX_ePlugInStrings_PluginFileName](#)

Member [AAX_ePlugInStrings_Preview](#)

Member [AAX_ePlugInStrings_RegionName](#)

Member [AAX_eProcessingState_Start](#)

Member [AAX_eProcessingState_Stop](#)

Member [AAX_eProperty_AudioBufferLength](#)
Use [AAX_eProperty_DSP_AudioBufferLength](#)

Member [AAX_eProperty_Deprecated_Plugin_List](#)
Use [AAX_eProperty_Deprecated_Native_Plugin_List](#) and [AAX_eProperty_Deprecated_DSP_Plugin_List](#) See [AAX_eProperty_PluginID_RTAS](#) for an example.

Member [AAX_eProperty_PluginID_RTAS](#)
Use [AAX_eProperty_PluginID_Native](#)

Member [AAX_eProperty_StoreXMLPageTablesByType](#)

Use [AAX_eProperty_StoreXMLPageTablesByEffect](#)

Member [AAX_IACFEfffectParameters::UpdateParameterNormalizedRelative](#) ([AAX_CParamID](#) iParameterID, double iValue)=0

This is not called from the host. It *may* still be useful for internal calls within the plug-in, though it should only ever be used to update non-automatable parameters. Automatable parameters should always be updated through the [AAX_IParameter](#) interface, which will ensure proper coordination with other automation clients.

Member [AAX_IACFHostServices::Assert](#) (const char *iFile, int32_t iLine, const char *iNote)=0

Legacy version of [AAX_IACFHostServices_V3::HandleAssertFailure\(\)](#) implemented by older hosts

Module [ExamplePlugins](#)

The DemoGain_Delay example is deprecated. See DemoDelay_HostProcessor

Chapter 6

Not Used by AAX Plug-Ins

Member [AAX_eUpdateSource_Delay](#)

Chapter 7

Module Index

7.1 Manual

These pages provide further information about various aspects of AAX.

AAX SDK Manual	47
Getting Started with AAX	50
Core AAX Interface	58
Description callback	59
Real-time algorithm callback	68
Data model interface	75
Data model overrides interface	77
GUI interface	78
AAX communication protocols	79
AAX Format Specification	81
Additional AAX features	82
Auxiliary Output Stems	83
Background processing callback	84
Direct Memory Access	86
Direct data access interface	87
EQ and Dynamics Curve Displays	89
Hybrid Processing architecture	94
Plug-in meters	97
MIDI	99
Offline processing interface	102
Properties File	102
Sidechain Inputs	103
Task agent interface	104
AAX Library features	106
Parameter Manager	106
Taper delegates	109
Display delegates	109
Display delegate decorators	111
Monolithic parameters	112
Additional Topics	112
Real-time performance	112
Parameter automation	114
Parameter updates	116
Parameter update timing	116

Token protocol	123
Basic parameter update sequences	127
Linked parameters	131
Linked parameter update sequences	136
Plug-in type conversion	140
The Avid Component Framework (ACF)	144
ACF Elements	149
AAX Host Guides	150
Pro Tools Guide	150
Media Composer Guide	171
HDX DSP Guide	180
Page Table Guide	225
DigiTrace Guide	257
DSH Guide	268
DTT Guide	274
VENUE Guide	354
Extensions	279
GUI Extensions	279
Monolithic VIs and Effects	280
Other Extensions	280
Supplemental Information	281
Troubleshooting	282
Distributing Your AAX Plug-In	285
AAX Interfaces	289
Host Support	291
Known Issues	297
Change Log	331
Example Plug-Ins	351

Chapter 8

Namespace Index

8.1 Namespace List

Here is a list of all namespaces with brief descriptions:

AAX	371
AAX::Exception	
AAX exception classes	397
AAX::internal	398
AAX_ChunkDataParserDefs	
Constants used by ChunkDataParser class	398

Chapter 9

Hierarchical Index

9.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

_acfUID	403
AAX_AggregateResult	404
AAX_CAutoreleasePool	416
AAX_CChunkDataParser	423
AAX_CheckedResult	482
AAX_CHostServices	497
AAX_CMidiPacket	507
AAX_CMidiStream	508
AAX_CMutex	516
AAX_Component< aContextType >	521
AAX_CPacket	523
AAX_CPacketDispatcher	524
AAX_CParameterManager	566
AAX_CStringAbbreviations	641
AAX_CTempoBreakpoint	659
AAX_FastInterpolatedTableLookup< TFLOAT, DFLOAT >	667
AAX_IAutomationDelegate	799
AAX_VAutomationDelegate	991
AAX_ICollection	803
AAX_VCollection	995
AAX_IComponentDescriptor	808
AAX_VComponentDescriptor	1000
AAX_IContainer	823
AAX_IPointerQueue< T >	925
AAX_CAtomicQueue< TNumberedParamStateList, 256 >	412
AAX_CAtomicQueue< const TParamValPair, 16 *kSynchronizedParameterQueueSize >	412
AAX_CAtomicQueue< T, S >	412
AAX_IController	824
AAX_VController	1014
AAX_IDataBufferWrapper	841
AAX_VDataBufferWrapper	1029
AAX_IDescriptionHost	843
AAX_VDescriptionHost	1031

AAX_IDisplayDelegateBase	848
AAX_IDisplayDelegate< T >	844
AAX_CBinaryDisplayDelegate< T >	417
AAX_CNumberDisplayDelegate< T, Precision, SpaceAfter >	518
AAX_CStateDisplayDelegate< T >	598
AAX_CStringDisplayDelegate< T >	648
AAX_IDisplayDelegateDecorator< T >	849
AAX_CDecibelDisplayDelegateDecorator< T >	430
AAX_CPercentDisplayDelegateDecorator< T >	581
AAX_CUnitDisplayDelegateDecorator< T >	660
AAX_CUnitPrefixDisplayDelegateDecorator< T >	663
AAX_IDma	853
AAX_IEffectDescriptor	864
AAX_VEffectDescriptor	1033
AAX_IFeatureInfo	874
AAX_VFeatureInfo	1039
AAX_IHostProcessorDelegate	877
AAX_VHostProcessorDelegate	1041
AAX_IHostServices	880
AAX_VHostServices	1043
AAX_IHostTaskAgent	882
AAX_VHostTaskAgent	1045
AAX_IMIDIMessageInfoDelegate	884
AAX_IMIDINode	887
AAX_IPacketHandler	888
AAX_CPacketHandler< TWorker >	527
AAX_IPageTable	890
AAX_VPageTable	1047
AAX_IParameter	900
AAX_CParameter< T >	529
AAX_CStatelessParameter	602
AAX_IParameterValue	921
AAX_CParameterValue< T >	572
AAX_IPrivateDataAccess	928
AAX_VPrivateDataAccess	1058
AAX_IPropertyMap	929
AAX_VPropertyMap	1060
AAX_ISessionDocument	934
AAX_VSessionDocument	1065
AAX_IString	937
AAX_CString	629
AAX_ITaperDelegateBase	942
AAX_ITaperDelegate< T >	939
AAX_CBinaryTaperDelegate< T >	420
AAX_CLinearTaperDelegate< T, RealPrecision >	499
AAX_CLogTaperDelegate< T, RealPrecision >	503
AAX_CPieceWiseLinearTaperDelegate< T, RealPrecision >	584
AAX_CRangeTaperDelegate< T, RealPrecision >	588
AAX_CStateTaperDelegate< T >	625
AAX_ITask	944
AAX_VTask	1068
AAX_ITransport	948
AAX_VTransport	1071

AAX_IViewContainer	957
AAX_VViewContainer	1081
AAX_Map	964
AAX_Point	966
AAX_Rect	967
AAX_SHybridRenderInfo	969
AAX_SInstrumentPrivateData	970
AAX_SInstrumentRenderInfo	971
AAX_SInstrumentSetupInfo	974
AAX_SPlugInChunk	982
AAX_SPlugInChunkHeader	984
AAX_SPlugInIdentifierTriad	987
AAX_StLock_Guard	988
AAX_TransportStateInfo_V1	989
AAX::Exception::Any	1088
AAX::Exception::ResultError	1096
CACFUnknown	
AAX_CTask	652
AAX_IDataBuffer	840
AAX_CArrayDataBuffer< D >	406
AAX_CArrayDataBufferOfType< T, D >	409
AAX_CStringDataBuffer	643
AAX_CStringDataBufferOfType< T >	645
AAX_IEffectDirectData	869
AAX_CEffectDirectData	434
AAX_IEffectGUI	870
AAX_CEffectGUI	438
AAX_IEffectParameters	872
AAX_CEffectParameters	447
AAX_CMonolithicParameters	509
AAX_IHostProcessor	876
AAX_CHostProcessor	486
AAX_ISessionDocumentClient	936
AAX_CSessionDocumentClient	593
AAX_ITaskAgent	947
AAX_CTaskAgent	656
AAX_CChunkDataParser::DataValue	1091
IACFPluginDefinition	
AAX_IACFCollection	672
IACFUnknown	1095
AAX_IACFAutomationDelegate	669
AAX_IACFComponentDescriptor	674
AAX_IACFComponentDescriptor_V2	683
AAX_IACFComponentDescriptor_V3	684
AAX_IACFController	686
AAX_IACFController_V2	694
AAX_IACFController_V3	696
AAX_IACFDataBuffer	698
AAX_IDataBuffer	840
AAX_IACFDescriptionHost	699
AAX_IACFEffectDescriptor	700
AAX_IACFEffectDescriptor_V2	703
AAX_IACFEffectDirectData	704
AAX_IACFEffectDirectData_V2	706
AAX_IEffectDirectData	869
AAX_IACFEffectGUI	707

AAX_IEffectGUI	870
AAX_IACFEffEffectParameters	713
AAX_IACFEffEffectParameters_V2	735
AAX_IACFEffEffectParameters_V3	737
AAX_IACFEffEffectParameters_V4	738
AAX_IEffectParameters	872
AAX_IACFFeatureInfo	739
AAX_IACFHostProcessor	741
AAX_IACFHostProcessor_V2	746
AAX_IHostProcessor	876
AAX_IACFHostProcessorDelegate	747
AAX_IACFHostProcessorDelegate_V2	749
AAX_IACFHostProcessorDelegate_V3	750
AAX_IACFHostServices	750
AAX_IACFHostServices_V2	752
AAX_IACFHostServices_V3	753
AAX_IACFPageTable	754
AAX_IACFPageTable_V2	758
AAX_IACFPageTableController	763
AAX_IACFPageTableController_V2	766
AAX_IACFPrivateDataAccess	768
AAX_IACFPropertyMap	769
AAX_IACFPropertyMap_V2	771
AAX_IACFPropertyMap_V3	772
AAX_IACFSessionDocument	773
AAX_IACFSessionDocumentClient	775
AAX_ISessionDocumentClient	936
AAX_IACFTask	777
AAX_CTask	652
AAX_IACFTaskAgent	780
AAX_ITaskAgent	947
AAX_IACFTransport	781
AAX_IACFTransport_V2	786
AAX_IACFTransport_V3	788
AAX_IACFTransport_V4	789
AAX_IACFTransport_V5	790
AAX_IACFTransportControl	791
AAX_IACFViewContainer	792
AAX_IACFViewContainer_V2	795
AAX_IACFViewContainer_V3	797
IACFDefinition	1092
SAutoArray< T >	1098
AAX_ISessionDocument::TempoMap	1099
AAX_VSessionDocument::VTempoMap	1100

Chapter 10

Class Index

10.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

_acfUID	403
AAX_AggregateResult	404
AAX_CArrayDataBuffer< D >	
A convenience class for array data buffers.	406
AAX_CArrayDataBufferOfType< T, D >	
A convenience class for array data buffers	409
AAX_CAtomicQueue< T, S >	412
AAX_CAutoreleasePool	416
AAX_CBinaryDisplayDelegate< T >	
A binary display format conforming to AAX_IDisplayDelegate	417
AAX_CBinaryTaperDelegate< T >	
A binary taper conforming to AAX_ITaperDelegate	420
AAX_CChunkDataParser	
Parser utility for plugin chunks	423
AAX_CDecibelDisplayDelegateDecorator< T >	
A percent decorator conforming to AAX_IDisplayDelegateDecorator	430
AAX_CEffectDirectData	
Default implementation of the AAX_IEffectDirectData interface	434
AAX_CEffectGUI	
Default implementation of the AAX_IEffectGUI interface	438
AAX_CEffectParameters	
Default implementation of the AAX_IEffectParameters interface	447
AAX_CheckedResult	482
AAX_CHostProcessor	
Concrete implementation of the AAX_IHostProcessor interface for non-real-time processing	486
AAX_CHostServices	
Method access to a singleton implementation of the AAX_IHostServices interface	497
AAX_CLinearTaperDelegate< T, RealPrecision >	
A linear taper conforming to AAX_ITaperDelegate	499
AAX_CLogTaperDelegate< T, RealPrecision >	
A logarithmic taper conforming to AAX_ITaperDelegate	503
AAX_CMidiPacket	
Packet structure for MIDI data	507
AAX_CMidiStream	
MIDI stream data structure used by AAX_IMIDINode	508

AAX_CMonolithicParameters	
Extension of the AAX_CEffectParameters class for monolithic VIs and effects	509
AAX_CMutex	
Mutex with try lock functionality	516
AAX_CNumberDisplayDelegate< T, Precision, SpaceAfter >	
A numeric display format conforming to AAX_IDisplayDelegate	518
AAX_Component< aContextType >	
Empty class containing type declarations for the AAX algorithm and associated callbacks	521
AAX_CPacket	
Container for packet-related data	523
AAX_CPacketDispatcher	
Helper class for managing AAX packet posting	524
AAX_CPacketHandler< TWorker >	
Callback container used by AAX_CPacketDispatcher	527
AAX_CParameter< T >	
Generic implementation of an AAX_IParameter	529
AAX_CParameterManager	
A container object for plug-in parameters	566
AAX_CParameterValue< T >	
Concrete implementation of AAX_IParameterValue	572
AAX_CPercentDisplayDelegateDecorator< T >	
A percent decorator conforming to AAX_IDisplayDelegateDecorator	581
AAX_CPieceWiseLinearTaperDelegate< T, RealPrecision >	
A piece-wise linear taper conforming to AAX_ITaperDelegate	584
AAX_CRangeTaperDelegate< T, RealPrecision >	
A piecewise-linear taper conforming to AAX_ITaperDelegate	588
AAX_CSessionDocumentClient	
Default implementation of the AAX_ISessionDocumentClient interface	593
AAX_CStateDisplayDelegate< T >	
A generic display format conforming to AAX_IDisplayDelegate	598
AAX_CStatelessParameter	
A stateless parameter implementation	602
AAX_CStateTaperDelegate< T >	
A linear taper conforming to AAX_ITaperDelegate	625
AAX_CString	
A generic AAX string class with similar functionality to <code>std::string</code>	629
AAX_CStringAbbreviations	
Helper class to store a collection of name abbreviations	641
AAX_CStringDataBuffer	
A convenience class for string data buffers.	643
AAX_CStringDataBufferOfType< T >	
A convenience class for string data buffers	645
AAX_CStringDisplayDelegate< T >	
A string, or list, display format conforming to AAX_IDisplayDelegate	648
AAX_CTask	652
AAX_CTaskAgent	
Default implementation of the AAX_ITaskAgent interface	656
AAX_CTempoBreakpoint	659
AAX_CUnitDisplayDelegateDecorator< T >	
A unit type decorator conforming to AAX_IDisplayDelegateDecorator	660
AAX_CUnitPrefixDisplayDelegateDecorator< T >	
A unit prefix decorator conforming to AAX_IDisplayDelegateDecorator	663
AAX_FastInterpolatedTableLookup< TFLOAT, DFLOAT >	667
AAX_IACFAutomationDelegate	
Versioned interface allowing an AAX plug-in to interact with the host's automation system	669
AAX_IACFCollection	
Versioned interface to represent a plug-in binary's static description	672

AAX_IACFComponentDescriptor	
Versioned description interface for an AAX plug-in algorithm callback	674
AAX_IACFComponentDescriptor_V2	
Versioned description interface for an AAX plug-in algorithm callback	683
AAX_IACFComponentDescriptor_V3	
Versioned description interface for an AAX plug-in algorithm callback	684
AAX_IACFController	
Interface for the AAX host's view of a single instance of an effect. Used by both clients of the AAXHost and by effect components	686
AAX_IACFController_V2	
Interface for the AAX host's view of a single instance of an effect. Used by both clients of the AAXHost and by effect components.	694
AAX_IACFController_V3	
Interface for the AAX host's view of a single instance of an effect. Used by both clients of the AAXHost and by effect components.	696
AAX_IACFDataBuffer	
Versioned interface for reference counted data buffers	698
AAX_IACFDescriptionHost	699
AAX_IACFEffectDescriptor	
Versioned interface for an AAX_IEffectDescriptor	700
AAX_IACFEffectDescriptor_V2	
Versioned interface for an AAX_IEffectDescriptor	703
AAX_IACFEffectDirectData	
Optional interface for direct access to a plug-in's alg memory	704
AAX_IACFEffectDirectData_V2	706
AAX_IACFEffectGUI	
The interface for a AAX Plug-in's GUI (graphical user interface)	707
AAX_IACFEffectParameters	
The interface for an AAX Plug-in's data model	713
AAX_IACFEffectParameters_V2	
Supplemental interface for an AAX Plug-in's data model	735
AAX_IACFEffectParameters_V3	
Supplemental interface for an AAX Plug-in's data model	737
AAX_IACFEffectParameters_V4	
Supplemental interface for an AAX Plug-in's data model	738
AAX_IACFFeatureInfo	739
AAX_IACFHostProcessor	
Versioned interface for an AAX host processing component	741
AAX_IACFHostProcessor_V2	
Supplemental interface for an AAX host processing component	746
AAX_IACFHostProcessorDelegate	
Versioned interface for host methods specific to offline processing	747
AAX_IACFHostProcessorDelegate_V2	
Versioned interface for host methods specific to offline processing	749
AAX_IACFHostProcessorDelegate_V3	
Versioned interface for host methods specific to offline processing	750
AAX_IACFHostServices	
Versioned interface to diagnostic and debugging services provided by the AAX host	750
AAX_IACFHostServices_V2	
V2 of versioned interface to diagnostic and debugging services provided by the AAX host . . .	752
AAX_IACFHostServices_V3	
V3 of versioned interface to diagnostic and debugging services provided by the AAX host . . .	753
AAX_IACFPageTable	
Versioned interface to the host's representation of a plug-in instance's page table	754
AAX_IACFPageTable_V2	
Versioned interface to the host's representation of a plug-in instance's page table	758
AAX_IACFPageTableController	
Interface for host operations related to the page tables for this plug-in	763

AAX_IACFPageTableController_V2	
Interface for host operations related to the page tables for this plug-in.	766
AAX_IACFPrivateDataAccess	
Interface for the AAX host's data access functionality	768
AAX_IACFPropertyMap	
Versioned interface for an AAX_IPropertyMap	769
AAX_IACFPropertyMap_V2	
Versioned interface for an AAX_IPropertyMap	771
AAX_IACFPropertyMap_V3	
Versioned interface for an AAX_IPropertyMap	772
AAX_IACFSessionDocument	
Interface representing information in a host session document	773
AAX_IACFSessionDocumentClient	
Interface representing a client of the session document interface	775
AAX_IACFTask	
Versioned interface for an asynchronous task	777
AAX_IACFTaskAgent	
Versioned interface for a component that accepts task requests	780
AAX_IACFTransport	
Versioned interface to get information about the host's transport state	781
AAX_IACFTransport_V2	
Versioned interface to get information about the host's transport state	786
AAX_IACFTransport_V3	
Versioned interface to get information about the host's transport state	788
AAX_IACFTransport_V4	
Versioned interface to get information about the host's transport state	789
AAX_IACFTransport_V5	
Versioned interface to get information about the host's transport state	790
AAX_IACFTransportControl	
Versioned interface to control the host's transport state	791
AAX_IACFViewContainer	
Interface for the AAX host's view of a single instance of an effect. Used by both clients of the host app and by effect components	792
AAX_IACFViewContainer_V2	
Supplemental interface for the AAX host's view of a single instance of an effect. Used by both clients of the host app and by effect components	795
AAX_IACFViewContainer_V3	
Additional methods to track mouse as it moves over controls	797
AAX_IAutomationDelegate	
Interface allowing an AAX plug-in to interact with the host's event system	799
AAX_ICollection	
Interface to represent a plug-in binary's static description	803
AAX_IComponentDescriptor	
Description interface for an AAX plug-in component	808
AAX_IContainer	823
AAX_IController	
Interface for the AAX host's view of a single instance of an effect. Used by both clients of the AAX host and by effect components	824
AAX_IDataBuffer	
Interface for reference counted data buffers	840
AAX_IDataBufferWrapper	
Wrapper for an AAX_IDataBuffer	841
AAX_IDescriptionHost	843
AAX_IDisplayDelegate< T >	
Classes for parameter value string conversion.	844
AAX_IDisplayDelegateBase	
Defines the display behavior for a parameter	848

AAX_IDisplayDelegateDecorator< T >	
The base class for all concrete display delegate decorators	849
AAX_IDma	
Cross-platform interface for access to the host's direct memory access (DMA) facilities	853
AAX_IEffectDescriptor	
Description interface for an effect's (plug-in type's) components	864
AAX_IEffectDirectData	
The interface for a AAX Plug-in's direct data interface	869
AAX_IEffectGUI	
The interface for a AAX Plug-in's user interface	870
AAX_IEffectParameters	
The interface for an AAX Plug-in's data model	872
AAX_IFeatureInfo	874
AAX_IHostProcessor	
Base class for the host processor interface	876
AAX_IHostProcessorDelegate	
Versioned interface for host methods specific to offline processing	877
AAX_IHostServices	
Interface to diagnostic and debugging services provided by the AAX host	880
AAX_IHostTaskAgent	
Interface to access an AAX_IACFTaskAgent object implemented by the host	882
AAX_IMIDIMessageInfoDelegate	884
AAX_IMIDINode	
Interface for accessing information in a MIDI node	887
AAX_IPacketHandler	
Callback container used by AAX_CPacketDispatcher	888
AAX_IPageTable	
Interface to the host's representation of a plug-in instance's page table	890
AAX_IParameter	
The base interface for all normalizable plug-in parameters	900
AAX_IParameterValue	
An abstract interface representing a parameter value of arbitrary type	921
AAX_IPointerQueue< T >	925
AAX_IPrivateDataAccess	
Interface to data access provided by host to plug-in	928
AAX_IPropertyMap	
Generic plug-in description property map	929
AAX_ISessionDocument	
Interface representing information in a host session document	934
AAX_ISessionDocumentClient	
Interface representing a client of the session document interface	936
AAX_IString	
A simple string container that can be passed across a binary boundary. This class, for simplicity, is not versioned and thus can never change	937
AAX_ITaperDelegate< T >	
Classes for conversion to and from normalized parameter values.	939
AAX_ITaperDelegateBase	
Defines the taper conversion behavior for a parameter	942
AAX_ITask	
Interface representing a request to perform a task	944
AAX_ITaskAgent	
Interface for a component that accepts task requests	947
AAX_ITransport	
Interface to information about the host's transport state	948
AAX_IViewContainer	
Interface for the AAX host's view of a single instance of an effect. Used both by clients of the AAX host and by effect components	957
AAX_Map	964

AAX_Point	
Data structure representing a two-dimensional coordinate point	966
AAX_Rect	
Data structure representing a rectangle in a two-dimensional coordinate plane	967
AAX_SHybridRenderInfo	
Hybrid render processing context	969
AAX_SInstrumentPrivateData	
Utility struct for AAX_CMonolithicParameters	970
AAX_SInstrumentRenderInfo	
Information used to parameterize AAX_CMonolithicParameters::RenderAudio()	971
AAX_SInstrumentSetupInfo	
Information used to describe the instrument	974
AAX_SPlugInChunk	
Plug-in chunk header + data	982
AAX_SPlugInChunkHeader	
Plug-in chunk header	984
AAX_SPlugInIdentifierTriad	
Plug-in Identifier Triad	987
AAX_StLock_Guard	
Helper class for working with mutex	988
AAX_TransportStateInfo_V1	989
AAX_VAutomationDelegate	
Version-managed concrete automation delegate class	991
AAX_VCollection	
Version-managed concrete AAX_ICollection class	995
AAX_VComponentDescriptor	
Version-managed concrete AAX_IComponentDescriptor class	1000
AAX_VController	
Version-managed concrete Controller class	1014
AAX_VDataBufferWrapper	
Wrapper for an AAX_IDataBuffer	1029
AAX_VDescriptionHost	1031
AAX_VEffectDescriptor	
Version-managed concrete AAX_IEffectDescriptor class	1033
AAX_VFeatureInfo	1039
AAX_VHostProcessorDelegate	
Version-managed concrete Host Processor delegate class	1041
AAX_VHostServices	
Version-managed concrete AAX_IHostServices class	1043
AAX_VHostTaskAgent	1045
AAX_VPageTable	
Version-managed concrete AAX_IPageTable class	1047
AAX_VPrivateDataAccess	
Version-managed concrete AAX_IPrivateDataAccess class	1058
AAX_VPropertyMap	
Version-managed concrete AAX_IPropertyMap class	1060
AAX_VSessionDocument	1065
AAX_VTask	
Version-managed concrete AAX_ITask	1068
AAX_VTransport	
Version-managed concrete AAX_ITransport class	1071
AAX_VViewContainer	
Version-managed concrete AAX_IViewContainer class	1081
AAX::Exception::Any	1088
AAX_CChunkDataParser::DataValue	1091
IACFDefinition	
Publicly inherits from IACFUnknown . This abstract interface is used to identify all of the plug-in components in the host	1092

IACFUnknown	
COM compatible IUnknown C++ interface	1095
AAX::Exception::ResultError	1096
SAutoArray< T >	1098
AAX_ISessionDocument::TempoMap	1099
AAX_VSessionDocument::VTempoMap	1100

Chapter 11

File Index

11.1 File List

Here is a list of all files with brief descriptions:

AAX.h	Various utility definitions for AAX	1103
AAX_ACFInterface.doxygen		1117
AAX_AdditionalFeatures_Algorithm.doxygen		1118
AAX_AdditionalFeatures_AOSandSidechain.doxygen		1118
AAX_AdditionalFeatures_CurveDisplays.doxygen		1118
AAX_AdditionalFeatures_Hybrid.doxygen		1118
AAX_AdditionalFeatures_Meters.doxygen		1118
AAX_AdditionalFeatures_MIDI.doxygen		1118
AAX_AdditionalFeatures_PropertiesFile.doxygen		1118
AAX_Alignment.h	Alignment malloc and free methods for optimization	1118
AAX_Assert.h	Declarations for cross-platform AAX_ASSERT, AAX_TRACE and related facilities	1119
AAX_Atomic.h	Atomic operation utilities	1125
AAX_AuxInterface_DirectData.doxygen		1127
AAX_AuxInterface_HostProcessor.doxygen		1127
AAX_AuxInterface_TaskAgent.doxygen		1127
AAX_BugList.doxygen		1127
AAX_Callbacks.h	AAX callback prototypes and ProcPtr definitions	1127
AAX_CArrayDataBuffer.h		1131
AAX_CAtomicQueue.h	Atomic, non-blocking queue	1131
AAX_CAutoreleasePool.h	Autorelease pool helper utility	1132
AAX_CBinaryDisplayDelegate.h	A binary display delegate	1132
AAX_CBinaryTaperDelegate.h	A binary taper delegate	1133
AAX_CChunkDataParser.h	Parser utility for plugin chunks	1133
AAX_CDecibelDisplayDelegateDecorator.h	A decibel display delegate	1134

AAX_CEffectDirectData.h	
A default implementation of the AAX_IEffectDirectData interface	1135
AAX_CEffectGUI.h	
A default implementation of the AAX_IEffectGUI interface	1135
AAX_CEffectParameters.h	
A default implementation of the AAX_IeffectParameters interface	1136
AAX_CHostProcessor.h	
Concrete implementation of the AAX_IHostProcessor interface for non-real-time processing	1137
AAX_CHostServices.h	
Concrete implementation of the AAX_IHostServices interface	1138
AAX_CLinearTaperDelegate.h	
A linear taper delegate	1138
AAX_CLogTaperDelegate.h	
A log taper delegate	1138
AAX_CMonolithicParameters.cpp	1139
AAX_CMonolithicParameters.h	
A convenience class extending AAX_CEffectParameters for monolithic instruments	1139
AAX_CMutex.h	
Mutex	1140
AAX_CNumberDisplayDelegate.h	
A number display delegate	1140
AAX_CommonConversions.h	1141
AAX_CommonInterface_Algorithm.doxygen	1146
AAX_CommonInterface_Communication.doxygen	1146
AAX_CommonInterface_DataModel.doxygen	1146
AAX_CommonInterface_Describe.doxygen	1146
AAX_CommonInterface_FormatSpecification.doxygen	1146
AAX_CommonInterface_GUI.doxygen	1146
AAX_Constants.h	
Signal processing constants	1146
AAX_CPacketDispatcher.h	
Helper classes related to posting AAX packets and handling parameter update events	1148
AAX_CParameter.h	
Generic implementation of an AAX_IParameter	1148
AAX_CParameterManager.h	
A container object for plug-in parameters	1149
AAX_CPercentDisplayDelegateDecorator.h	
A percent display delegate decorator	1149
AAX_CPieceWiseLinearTaperDelegate.h	
A piece-wise linear taper delegate	1150
AAX_CRangeTaperDelegate.h	
A range taper delegate decorator	1150
AAX_CSessionDocumentClient.h	1151
AAX_CStateDisplayDelegate.h	
A state display delegate	1151
AAX_CStateTaperDelegate.h	
A state taper delegate (similar to a linear taper delegate.)	1151
AAX_CString.h	
A generic AAX string class with similar functionality to std::string	1152
AAX_CStringDataBuffer.h	1153
AAX_CStringDisplayDelegate.h	
A string display delegate	1154
AAX_CTask.h	
A default implementation of the AAX_IACFTask interface	1154
AAX_CTaskAgent.h	
A default implementation of the AAX_ITaskAgent interface	1155
AAX_CUnitDisplayDelegateDecorator.h	
A unit display delegate decorator	1155

AAX_CUnitPrefixDisplayDelegateDecorator.h	
A unit prefix display delegate decorator	1156
AAX_Denormal.h	
Signal processing utilities for denormal/subnormal floating point numbers	1156
AAX_DigiTrace_Guide.doxygen	1157
AAX_DistributingYourPlugIn.doxygen	1157
AAX_DocsDirectory.doxygen	1157
AAX_EndianSwap.h	
Utility functions for byte-swapping. Used by AAX_CChunkDataParser	1157
AAX_Enums.h	
Utility functions for byte-swapping. Used by AAX_CChunkDataParser	1161
AAX_EnvironmentUtilities.h	
Useful environment definitions for AAX	1208
AAX_Errors.h	
Definitions of error codes used by AAX plug-ins	1209
AAX_Exception.h	
AAX SDK exception classes and utilities	1212
AAX_Exports.cpp	1215
AAX_FastInterpolatedTableLookup.h	
A set of functions that provide lookup table functionality. Not necessarily optimized for TI, but used internally	1218
AAX_FastInterpolatedTableLookup.hpp	1218
AAX_FastPow.h	
Set of functions to optimize pow	1219
AAX_Getting_Started_Guide.doxygen	1219
AAX_GUITypes.h	
Constants and other definitions used by AAX plug-in GUIs	1219
AAX_HostSupport.doxygen	1223
AAX_IACFAutomationDelegate.h	
Versioned interface allowing an AAX plug-in to interact with the host's automation system	1223
AAX_IACFCollection.h	
Versioned interface to represent a plug-in binary's static description	1224
AAX_IACFComponentDescriptor.h	
Versioned description interface for an AAX plug-in algorithm callback	1224
AAX_IACFController.h	
Interface for the AAX host's view of a single instance of an effect. Used by both clients of the AAXHost and by effect components	1224
AAX_IACFDataBuffer.h	1225
AAX_IACFDescriptionHost.h	1225
AAX_IACFEffectDescriptor.h	
Versioned interface for an AAX_IEffectDescriptor	1226
AAX_IACFEffectDirectData.h	
The direct data access interface that gets exposed to the host application	1226
AAX_IACFEffectGUI.h	
The GUI interface that gets exposed to the host application	1226
AAX_IACFEffectParameters.h	
The data model interface that is exposed to the host application	1227
AAX_IACFFeatureInfo.h	1227
AAX_IACFHostProcessor.h	
The host processor interface that is exposed to the host application	1228
AAX_IACFHostProcessorDelegate.h	1228
AAX_IACFHostServices.h	1228
AAX_IACFPageTable.h	1229
AAX_IACFPageTableController.h	1229
AAX_IACFPrivateDataAccess.h	
Interface for the AAX host's data access functionality	1229
AAX_IACFPropertyMap.h	
Versioned interface for an AAX_IPropertyMap	1230

AAX_IACFSessionDocument.h	1230
AAX_IACFSessionDocumentClient.h	1231
AAX_IACFTask.h	
Defines the interface representing an asynchronous task	1231
AAX_IACFTaskAgent.h	1232
AAX_IACFTransport.h	
Interface for accessing the host's transport state	1232
AAX_IACFTransportControl.h	
Interface for control over the host's transport state	1233
AAX_IACFViewContainer.h	
Interface for the AAX host's view of a single instance of an effect. Used by both clients of the AAXHost and by effect components	1233
AAX_IAutomationDelegate.h	
Interface allowing an AAX plug-in to interact with the host's automation system	1234
AAX_ICollection.h	
Interface to represent a plug-in binary's static description	1234
AAX_IComponentDescriptor.h	
Description interface for an AAX plug-in algorithm	1234
AAX_IContainer.h	
Abstract container interface	1235
AAX_IController.h	
Interface for the AAX host's view of a single instance of an effect	1235
AAX_IDataBuffer.h	1235
AAX_IDataBufferWrapper.h	1236
AAX_IDescriptionHost.h	1236
AAX_IDisplayDelegate.h	
Defines the display behavior for a parameter	1237
AAX_IDisplayDelegateDecorator.h	
The base class for all concrete display delegate decorators	1237
AAX_IDma.h	
Cross-platform interface for access to the host's direct memory access (DMA) facilities	1237
AAX_IEffectDescriptor.h	
Description interface for an effect's (plug-in type's) components	1238
AAX_IEffectDirectData.h	
Optional interface for direct access to alg memory	1238
AAX_IEffectGUI.h	
The interface for a AAX Plug-in's user interface	1239
AAX_IEffectParameters.h	
The interface for an AAX Plug-in's data model	1239
AAX_IFeatureInfo.h	1240
AAX_IHostProcessor.h	
Base class for the host processor interface which is extended by plugin code	1240
AAX_IHostProcessorDelegate.h	
Interface allowing plug-in's HostProcessor to interact with the host's side	1240
AAX_IHostServices.h	
Various host services	1240
AAX_IHostTaskAgent.h	
Interface to access an AAX_IACFTaskAgent object implemented by the host	1241
AAX_IMIDINode.h	
Declaration of the base MIDI Node interface	1241
AAX_Init.h	
AAX library implementations of required plug-in initialization, registration, and tear-down methods	1242
AAX_InstrumentParameters.doxygen	1245
AAX_InterfaceList.doxygen	1245
AAX_IPageTable.h	1245
AAX_IParameter.h	
The base interface for all normalizable plug-in parameters	1245

AAX_IPointerQueue.h	
Abstract interface for a basic FIFO queue of pointers-to-objects	1245
AAX_IPrivateDataAccess.h	
Interface to data access provided by host to plug-in	1246
AAX_IPropertyMap.h	
Generic plug-in description property map	1246
AAX_ISessionDocument.h	1246
AAX_ISessionDocumentClient.h	1247
AAX_IString.h	
An AAX string interface	1248
AAX_ITaperDelegate.h	
Defines the taper conversion behavior for a parameter	1248
AAX_ITask.h	1248
AAX_ITaskAgent.h	1249
AAX_ITransport.h	
The interface for query ProTools transport information	1249
AAX_IViewContainer.h	
Interface for the AAX host's view of a single instance of an effect	1249
AAX_LinkedParameters.doxygen	1250
AAX_Map.h	1250
AAX_Media_Composer_Guide.doxygen	1251
AAX_MIDILogging.cpp	1251
AAX_MIDILogging.h	
Utilities for logging MIDI data	1251
AAX_MIDIUtilities.h	
Utilities for managing MIDI data	1251
AAX_MiscUtils.h	
Miscellaneous signal processing utilities	1253
AAX_OtherExtensions.doxygen	1256
AAX_Page_Table_Guide.doxygen	1256
AAX_PageTableUtilities.h	1256
AAX_ParameterAutomation.doxygen	1256
AAX_ParameterManager.doxygen	1256
AAX_ParameterUpdateProtocol.doxygen	1256
AAX_ParameterUpdateTiming.doxygen	1256
AAX_PlatformOptimizationConstants.h	
Constants descriptor..	1256
AAX_PluginBundleLocation.h	
Utilities for interacting with the host OS	1257
AAX_PopStructAlignment.h	
Resets (pops) the struct alignment to its previous value	1257
AAX_PostStructAlignmentHelper.h	
Helper file for data alignment macros	1258
AAX_PreStructAlignmentHelper.h	
Helper file for data alignment macros	1258
AAX_Pro_Tools_Guide.doxygen	1258
AAX_Properties.h	
Contains IDs for properties that can be added to an AAX_IPropertyMap	1258
AAX_Push2ByteStructAlignment.h	
Set the struct alignment to 2-byte. This file will throw an error on platforms that do not support 2-byte alignment (i.e. TI DSPs)	1278
AAX_Push4ByteStructAlignment.h	
Set the struct alignment to 4-byte	1279
AAX_Push8ByteStructAlignment.h	
Set the struct alignment to 8-byte	1280
AAX_Quantize.h	
Quantization utilities	1280

AAX_RandomGen.h	
Functions for calculating pseudo-random numbers	1281
AAX_RealTimePerformance.doxygen	1282
AAX_RelatedTypes.doxygen	1282
AAX_SampleRateUtils.h	
Description	1282
AAX_SDK_ChangeLog.doxygen	1284
AAX_SDK_ExamplePlugIns.doxygen	1284
AAX_SDK_GUIExtensions.doxygen	1284
AAX_SessionDocumentTypes.h	1284
AAX_SliderConversions.h	
Legacy utilities for converting parameter values to and from the normalized full-scale 32-bit fixed domain that was used for RTAS/TDM plug-ins	1285
AAX_StringUtilities.h	
Various string utility definitions for AAX Native	1288
AAX_StringUtilities.hpp	1289
AAX_TI_Guide.doxygen	1290
AAX_TransportTypes.h	
Structures, enums and other definitions used in transport	1290
AAX_Troubleshooting.doxygen	1291
AAX_UIDs.h	
Unique identifiers for AAX/ACF interfaces	1291
AAX_UtilsNative.h	
Various utility definitions for AAX Native	1309
AAX_VAutomationDelegate.h	
Version-managed concrete AutomationDelegate class	1310
AAX_VCollection.h	
Version-managed concrete Collection class	1311
AAX_VComponentDescriptor.h	
Version-managed concrete ComponentDescriptor class	1311
AAX_VController.h	
Version-managed concrete Controller class	1311
AAX_VDataBufferWrapper.h	1312
AAX_VDescriptionHost.h	1312
AAX_VEffectDescriptor.h	
Version-managed concrete EffectDescriptor class	1313
AAX_VENUE_Guide.doxygen	1313
AAX_Version.h	
Version stamp header for the AAX SDK	1313
AAX_VFeatureInfo.h	1318
AAX_VHostProcessorDelegate.h	
Version-managed concrete HostProcessorDelegate class	1318
AAX_VHostServices.h	
Version-managed concrete HostServices class	1319
AAX_VHostTaskAgent.h	1319
AAX_VPageTable.h	1320
AAX_VPrivateDataAccess.h	
Version-managed concrete PrivateDataAccess class	1320
AAX_VPropertyMap.h	
Version-managed concrete PropertyMap class	1320
AAX_VSessionDocument.h	1321
AAX_VTask.h	1321
AAX_VTransport.h	
Version-managed concrete Transport class	1322
AAX_VViewContainer.h	
Version-managed concrete ViewContainer class	1322
DSH_Guide.doxygen	1322
DTT_Guide.doxygen	1322

ReadMe.doxygen	1322
--	------

Chapter 12

Module Documentation

12.1 AAX SDK Manual

12.1.1

12.1.2 Welcome to AAX

Select the "Manual" tab to see a full list of documentation pages, or choose from the topics below.

Note

Looking for something? The search function only includes indexing of code symbols and page titles. To search for specific text strings in the AAX SDK manual it is best to use a text search tool such as grep or FINDSTR on the AAX SDK directory or search for the desired text within the PDF version of the AAX SDK documentation.

12.1.2.1 The Basics

New to AAX? Read through the documentation pages listed below to get started!

- See [Getting Started with AAX](#) for a general overview of AAX and a walk-through of the DemoGain example plug-in
- Read through the first few sections of the [Pro Tools Guide](#) if you are new to Pro Tools
- Read the [Digital signature](#) section of the [Pro Tools Guide](#) to review the digital signing requirements for compatibility with Pro Tools
- Review the [sample plug-ins](#) for examples of both basic and advanced AAX features
- See [Core AAX Interface](#) to find out more about the basic structure of AAX plug-ins
- See [Real-time algorithm callback](#) to learn more about audio processing in AAX
- See [Data model interface](#) to learn about adding parameters and controls to your plug-in,
- See [Description callback](#) for more information about plug-in configuration and initial set-up
- See the [HDX DSP Guide](#) to find out how to add AAX DSP support to your plug-in for Avid's HDX and Pro Tools | Carbon products
- Ready to ship your new plug-in? See [Distributing Your AAX Plug-In](#) for information about finalizing and distributing your AAX products
- Check out the [Troubleshooting](#) page if you're having problems, or post a question to the [developer forums](#) and an Avid engineer will be happy to assist you.

12.1.2.2 More Topics

Have a more specific question? Review the pages below or view the full list of documentation pages under the "Manual" tab above.

- See [AAX_IController](#) to see the interface that an AAX plug-in's host-based modules use to interact with the host
- See [AAX communication protocols](#) to find out more about how different modules in an AAX plug-in communicate with one another
- See [Offline processing interface](#) for information about creating advanced non-real-time AAX plug-ins
- See [Taper delegates](#) and [Display delegates](#) for more information about implementing custom parameter and control behavior
- Look in the /TI/SignalProcessing folder for signal processing utility classes and functions available for optimizing on Native and DSP

12.1.2.3 Test Tools & Utilities

- The [DSH Guide](#) has information about the tool for testing basic functionality of your plug-in
- See [DTT Guide](#) to learn how to automate different test scenarios for DSH

12.1.2.4 Supplemental Information

- [Example Plug-Ins](#)
- [Change Log](#)
- [Host Support](#)
- [Known Issues](#)

12.1.3 SDK Folder Hierarchy

Documentation

SDK documentation

ExamplePlugIns

Example plug-in projects [More information](#)

Extensions

Demonstrations of how to extend the AAX SDK, for example by incorporating third-party GUI frameworks into AAX plug-ins. [More information](#)

Interfaces

Interface headers and other resources required for use of the AAX SDK library

Libs

Source code for the AAX SDK library, a collection of default implementations and utility classes for use in all AAX plug-ins

TI

Various resources for use with TI's Code Composer Studio IDE and Avid's TI testing toolset

Utilities

Common SDK utilities and resources

12.1.4 Contacting Avid

Your personal [avid user account](#) is your hub for AAX Toolkit services and developer support.

Log in at [avid.com](#) for access to the full range of tools and services provided to AAX developers, including the AAX [developer forum](#). If you have any questions on the AAX SDK documentation or require support with AAX development, we encourage you to post them to the forum as your first line of inquiry.

If you have time-sensitive or critical support inquiries, contact the AAX development team directly at avid.developer.services@avid.com. Any AAX questions sent to this alias will be promptly addressed by the most appropriate contact here at Avid.

If you require NFR (Not For Resale) licenses to Avid software for AAX development please send an e-mail to devauth@avid.com with "License Request" in the subject.

If you require access to the digital signing toolkit from PACE Anti-Piracy, Inc. for compatibility with Pro Tools then please follow the instructions [here](#).

The following chart describes these and other ways of connecting with Avid to take advantage of the services provided to AAX developers:

12.1.5 Licensing

Unless you have entered into a commercial agreement with Avid, you are using this SDK under an evaluation agreement. To review this agreement, see the [AAX Toolkit downloads](#) section under your [my.avid.com](#) account.

As an Avid Developer, you are invited to offer your products on [Avid Marketplace](#) and via [Avid Link](#). If you wish to sell them independently or through other commercial outlets, an authorized representative from your organization is required to sign our Commercial License, which you can read and click through [here](#).

Documents

- [Getting Started with AAX](#)
A brief introduction to AAX.
- [Core AAX Interface](#)
Main classes, callbacks, and format specification details for a standard AAX plug-in.
- [Additional AAX features](#)
How to use additional features and functionality supported by AAX.
- [AAX Library features](#)
AAX Library core support for the AAX interface
- [Additional Topics](#)
Additional information about the AAX design.
- [AAX Host Guides](#)
Documentation for specific AAX host environments.
- [Extensions](#)
Extensions to the AAX SDK.
- [Supplemental Information](#)
Supplemental documents beyond the scope of the AAX SDK.

Collaboration diagram for AAX SDK Manual:

12.2 Getting Started with AAX

A brief introduction to AAX.

12.2.1 Contents

- [Welcome](#)
- [Quick Start](#)
- [AAX Design Overview](#)
- [DemoGain Example](#)
- [Next Steps](#)

12.2.2 Welcome

Welcome to AAX! This guide is designed to introduce you to the fundamental concepts of AAX. By the end of this guide you will understand:

- The purpose of AAX
- The basic components of an AAX plug-in
- The structure of the DemoGain example plug-in
- What you need to do to successfully build and run your own AAX plug-ins

12.2.3 Quick Start

Use the steps below to get up and running quickly with an example plug-in from the AAX SDK.

- Build the AAX Library

If this is your first time opening the AAX SDK then your first step should be to build the AAX Library project. The AAX Library is a static library containing default implementations of the AAX interface and convenience classes designed to make AAX development easy. All of the SDK example plug-ins link to this library, and your plug-ins should too.

Open the AAX Library project for your chosen IDE from the Libs/AAXLibrary directory and build the library. Now you are ready to build plug-ins!

- Open and build the DemoGain example plug-in

The DemoGain project is located in ExamplePlugIns/DemoGain. Once you have built the AAX Library project you will be able to successfully build DemoGain. To learn more about the DemoGain example plug-in, see the [DemoGain Example](#) section below.

- Install a developer build of Pro Tools

If you looking at AAX then you are most likely interested in developing plug-ins for [Pro Tools](#). Publicly available builds of Pro Tools require that AAX plug-ins be digitally signed. During development, you can use a "developer build" of Pro Tools to run unsigned test builds of your plug-ins, like the DemoGain plug-in that you just built.

Download and install the latest Pro Tools developer build from the AAX developer downloads in your [my.↔avid.com](#) account.

- Obtain a Pro Tools Developer iLok license (and an iLok)

Pro Tools developer builds require a special license loaded on an iLok USB device. You can request this Pro Tools Developer license, as well as any other NFR (Not For Resale) licenses which you require for your AAX product development and testing, by writing to devauth@avid.com with "License Request" in the subject.

If you don't have an iLok device you will also need to obtain one from [PACE Anti-Piracy Inc.](#) or a reseller, including most music shops which sell audio software.

- Install DemoGain in the AAX Plug-Ins folder

In order to be loaded into Pro Tools, a plug-in must be present in the system's AAX Plug-Ins directory. See [Install directories](#) in the [Pro Tools Guide](#) document for more information about where to install AAX plug-ins for Pro Tools.

- Launch Pro Tools and run DemoGain

You're now ready to run your very first AAX plug-in!

1. Make sure that your iLok USB device is connected to the system and contains the Pro Tools Developer license, then launch the Pro Tools developer build.
2. Once Pro Tools has launched, you will be prompted to create a new Session or Project. Choose Session (Local Storage) to create a local session file.
3. Create a new mono audio track in your session using the "New Track..." menu item
4. If it is not already visible, reveal the "Inserts A-E" view in the Edit window using the View > Edit Window menu.
5. Click on one of the insert slots for your audio track and choose DemoGain from the insert selection menu.

You're now running an instance of the DemoGain AAX plug-in. If you have a debugger attached to Pro Tools you should now be able to break in the plug-in's methods and step through its code.

- Get ready to release your own products

Once you have created your own AAX plug-in you can follow the steps in [Distributing Your AAX Plug-In](#) to prepare your plug-in for public sale and distribution.

In particular, for Pro Tools compatibility or to test your product with a public Pro Tools release you will need to digitally sign your plug-in using a toolkit from PACE Anti-Piracy Inc. See the [digital signature](#) section of the [Pro Tools Guide](#) for more information about this requirement.

12.2.4 AAX Design Overview

12.2.4.1 Architecture Philosophy

The purpose of the AAX architecture is to provide an *easy-to-use* framework for the development of *high-performance audio plug-ins* that can run on a *variety of platforms*, both present and future, with a *high level of code re-use*.

12.2.4.2 Design Attributes

AAX incorporates a flexible, decoupled component hierarchy, including:

1. A relocatable C-style callback that performs the plug-in's real-time audio processing algorithm
2. A collection of supporting C++ objects that manage the plug-in's data, GUI, and any other non-real-time information
3. A "description" method that statically describes the plug-in's layout and compatibility requirements to the host.

This flexible design facilitates optimal performance and portability; AAX is 64-bit ready and is designed to work well in both host-based (Native), accelerated (DSP), and future (e.g. embedded) environments. Importantly, plug-ins running in each of these environments can use the same code base, and porting to new platforms should not require much more than a re-compile. To satisfy these portability requirements, AAX must be decoupled into three parts, the GUI, data model, and algorithm.

12.2.4.3 Component Structure

The core component structure of an AAX plug-in involves data model, GUI, description, and algorithm modules. The design involves a mixture of C++ objects (data model and GUI modules) and C-style callbacks (algorithm and description modules.)

The figure below shows basic object ownership and composition in an AAX plug-in. The purple components are provided as part of the AAX SDK while the other components are written as needed by the plug-in developer. The classes above the dotted line are pure virtual interfaces, while the classes below the dotted line are concrete implementations.

Figure 1: Core AAX interface design.

As you can see, the plug-in's [data model](#) and [GUI](#) are written as C++ objects inherited from SDK interfaces, while its [algorithm](#) and [Description](#) methods are implemented as simple callbacks. This basic model may be expanded by attaching additional modules, such as the [Host Processor](#) module used by advanced non-real-time plug-ins. In this section, however, we will be considering only the four core interfaces and modules.

12.2.4.4 Algorithm

The most fundamental, and most important, component of any audio plug-in is its processing algorithm. Due to the design requirements of an AAX plug-in, this component must meet several constraints in order to be compatible with accelerated platforms:

1. It must be possible to build the algorithm as a compatible component on a variety of platforms
2. It must be possible for the host to re-locate the algorithm in memory without affecting the algorithm's state
3. The algorithm must be separable from the rest of its plug-in, e.g. into a different memory space
4. The algorithm must be as efficient as possible to call.

To satisfy these constraints, AAX uses a decoupled C-style callback function. State information within the function is obtained through the context, a custom data structure that contains everything from the "outside world" that is relevant to the algorithm. The context includes information such as audio buffers and coefficient packets, which are provided according to a static set of data routing definitions that we will describe further in the next chapter. The host is responsible for ensuring that this structure is up-to-date whenever the algorithm callback is entered.

See also

[Real-time algorithm callback](#)

12.2.4.5 Data Model

The [AAX_IEffectParameters](#) interface represents the data model portion of your plug-in. The AAX host interacts with your plug-in's data model via this interface, which includes methods that store and update of your plug-in's internal data.

In your plug-in's data model implementation, you will be responsible for creating the plug-in's parameters and defining how the plug-in will respond when these parameters are changed via control updates or preset loads. In order for information to be routed from the data model to the plug-in's algorithm, the parameters that are created here must be registered with the host in the plug-in's Description callback (see below).

The data model is composed with [AAX_IController](#), an interface that provides access to the host. This interface provides a means of querying information from the host such as stem format or sample rate, and is also responsible for communication between the data model and the (decoupled) algorithm.

You will most likely inherit your custom data model's implementation from [AAX_CEffectParameters](#), a default implementation of the [AAX_IEffectParameters](#) interface. This class provides basic functionality such as adding custom parameters, setting control values, restoring state, generating coefficients, etc., which you can override and customize as needed.

See also

[Data model interface](#)

12.2.4.6 GUI Interface

The [AAX_IEffectGUI](#) interface contains the plug-in's GUI methods. This interface is decoupled from the plug-in's data model, allowing AAX to support distributed architectures that incorporate remote GUIs.

The GUI is also composed with [AAX_IController](#), from which references to the plug-in's other components, such as its data model interface, may be retrieved.

The AAX SDK includes a set of GUI extensions to help you get started implementing your plug-ins' GUIs. These extensions include both native drawing formats and suggested integrations with third-party graphics frameworks. Although the SDK does not include any actual graphics frameworks, these extensions provide examples of how you can incorporate your chosen GUI framework with [AAX](#).

See also

[GUI interface](#)

12.2.4.7 Describe

A plug-in's Describe callback ties all the plug-in's components together and registers the plug-in with the host. In this callback, your plug-in defines a set of algorithm callbacks, data connections, and static plug-in properties

In order to route data to the plug-in's algorithm, Describe includes a description of the algorithm's context structure. This description involves a set of port definitions, which can be "hard-wired" to receive data from the host (such as audio buffers,) from the plug-in's data model (such as packets of coefficients,) or even from past calls to the algorithm (private, persistent algorithm state.)

Once these connections are made, Describe passes the host a populated description interface and returns. In the next section we will demonstrate how all these interfaces interact with one another by examining a sample plug-in.

See also

[Description callback](#)

12.2.4.8 Controller

There is one additional core component to any AAX plug-in, the Controller. The plug-in does not implement this component - rather, the Controller is an interface that provides access to various facilities provided by the host, as well as to the plug-in's other modules.

12.2.5 DemoGain Example

The AAX SDK includes a basic example plug-in named DemoGain. In this section we will examine this plug-in to show how the various core modules in an AAX plug-in interact with one another. We will focus in particular on how the DemoGain's "gain" parameter is defined, routed to the algorithm, and used to apply a gain effect to audio samples. In this way we will "visit" each of the core components in DemoGain.

For a description of all the example plug-ins included in the SDK, see the [Example Plug-Ins](#) page.

Note

To run DemoGain or other example plug-ins in Pro Tools 10 you must change the `AAX_ePlugInCategory_Example` category to `AAX_ePlugInCategory_Dynamics` in the plug-in's Describe function. The "example" category is not supported in Pro Tools 10.

12.2.5.1 AAX Plug-In Parameters

A good starting point for understanding a plug-in is to understand its parameters. In DemoGain, as in most AAX plug-ins, the plug-in's parameters define its data model state and the plug-in's parameters provide the fundamental connection between user interactions and audio processing.

The sections below will guide you through each of the main steps of parameter creation and connection, making use of the default implementation in [AAX_CEffectParameters](#):

1. [Data Model: Create Your Parameter](#)

- (a) Create a new parameter object
- (b) Register the parameter with the Packet Dispatcher
- (c) Create an update callback that converts the raw parameter value into a packet of processed coefficients

2. [Algorithm: Add coefficients to the algorithm's context structure](#)
 - (a) Create a new field for incoming coefficient packets
 - (b) Generate a *field ID* to reference the new member
 - (c) Add the new coefficient to the plug-in's algorithm code
3. [Describe: Connect the parameter throughout the plug-in](#)
 - (a) Add a new Data Input Port to the algorithm via the component descriptor interface
 - (b) Add a connection between the parameter's *packet ID* and its coefficients' *field ID*
4. [GUI: Add a control](#)
 - (a) Create a GUI widget that can update the parameter's state
 - (b) Add logic to notify the data model when the GUI is edited
 - (c) Add logic to update the GUI when the data model state changes

12.2.5.2 Data Model: Create Your Parameter

DemoGain's data model inherits its functionality from [AAX_CEffectParameters](#) (the default implementation of [AAX_IEffectParameters](#)). The `DemoGain_Parameters.h` and `DemoGain_Parameters.cpp` source files comprise the source code for DemoGain's data model.

During plug-in initialization, `DemoGain_Parameters::EffectInit()` creates the plug-in's custom parameters. A look inside of the `.cpp` file shows how this is done via the creation of a new [AAX_CParameter](#) for the gain parameter: the [AAX_CParameter](#) constructor is parametrized with a set of arguments that define the gain parameter's behavior, such as its default value (`1.0f`), name ("Gain"), taper behavior (linear), etc.

After creating the parameter objects, a series of packets are registered with the host via the inherited [Packet Dispatcher](#), `mPacketDispatcher`, which is a helper object used by [AAX_CEffectParameters](#) to assist with the plug-in's packet management tasks.

```
mPacketDispatcher.RegisterPacket(
    DemoGain_GainID,
    eAlgPortID_CoefsGainIn,
    this,
    &DemoGain_Parameters::UpdatePacket_Gain);
```

Listing 1: Registration of DemoGain's "gain" packet handler

The last parameter in [AAX_CPacketDispatcher::RegisterPacket\(\)](#) takes a reference to a packet handler callback. This method will be called by the Packet Dispatcher whenever new parameter values need to be converted into coefficients that can be used by the algorithm. In DemoGain, a reference to `DemoGain_Parameters::UpdatePacket_Gain()` is used for the gain parameter's coefficient packet.

As a developer, you will choose how this portion of your data model gets handled; you can choose to use the default handler method, which simply forwards the raw parameter values to the algorithm, or you can define a custom handler. You can also choose to bypass the Packet Dispatcher completely (see the `DemoDist_GenCoef` plug-in for an example of this.)

Although the handling of DemoGain's gain parameter is trivial, for the sake of demonstration this plug-in uses both Packet Dispatcher approaches in the registration of its bypass and gain parameters.

12.2.5.3 Algorithm: Add coefficients to the algorithm's context structure

Your plug-in's context is nothing more than a data structure of pointers that is registered with the host during Describe. These pointers function as *ports* where host-managed data may be delivered or retrieved.

12.2.5.3.1 Context definition Looking under the "Component context definitions" section within DemoGain_↵ Alg.h, you will find DemoGain's SDemoGain_Alg_Context context. This is DemoGain's context structure, and its sole purpose is to parametrize DemoGain's algorithm with a set of ports. Note the definition of a port for the gain coefficients that are created by DemoGain_Parameters::UpdatePacket_Gain() and another port for the bypass value that is forwarded via the default packet update handler.

```
int32_t          * mCtrlBypassP;
SDemoGain_CoefsGain * mCoefsGainP;
```

Listing 2: Gain and bypass ports in the DemoGain algorithm's context structure

Once ports have been defined for the algorithm's coefficients and other algorithmic data, unique identifiers for each port in the context must be generated. This is accomplished through use of the `AAX_FIELD_INDEX` macro. The basic idea behind this macro is to generate IDs that the host can use to directly address the ports within their context:

```
, eAlgPortID_CoefsGainIn = AAX_FIELD_INDEX ( SDemoGain_Alg_Context , mCoef ),
, eAlgFieldID_AudioIn = AAX_FIELD_INDEX ( SDemoGain_Alg_Context , mInputPP ) )
```

Listing 3: Some port ID definitions for the DemoGain algorithm's context structure

Now the context's definition is complete. So far these are just fields in a struct; the host doesn't yet know how to route packets from the data model to these ports. That information will come later, in the plug-in's [description](#). Once this context is described to the host, the host will know to populate it and pass it to the processing function located in DemoGain_AlgProc.cpp.

12.2.5.3.2 Algorithm processing callback

```
void
AAX_CALLBACK
DemoGain_AlgorithmProcessFunction (
    SDemoGain_Alg_Context * const inInstancesBegin [],
    const void * inInstancesEnd )
```

Listing 4: The DemoGain algorithm's callback prototype

This is where the plug-in's audio buffer processing of the plug-in occurs. Note that this function is passed a pointer to an array of SDemoGain_Alg_Contexts. Each of these represents the state of a particular instance of DemoGain, and contains pointers to the applicable coefficient and audio buffer data.

Using the SDemoGain_Alg_Context array, instance-specific information and audio buffers are easily retrieved for processing. DemoGain accomplishes this by first iterating over each plug-in instance, then over each channel, and finally over each sample, at which point the gain coefficient is applied to each sample in the input audio buffer and the sample data is copied to the output audio buffer:

```
// ----- Iterate over plug-in instances -----//
for (SDemoGain_Alg_Context * const walk = inInstancesBegin ; walk < inInstancesEnd ; ++ walk )
{
    .
    .
    .
    // ----- Run processing loop over each input channel -----//
    //
    for (int ch = 0; ch < kNumChannelsIn ; ch++)
    {
        // ----- Run processing loop over each sample -----//
        //
        for (int t = 0; t < kAudioWindowSize ; t++)
        {
            .
            .
            .
            pdO [t] = gain * pdI [t];
            .
            .
            .
        } // Go to the next sample
    } // Go to next channel
} // End instance-iteration loop
```

Listing 5: Iterative loops in the DemoGain algorithm

12.2.5.4 Describe: Connect the parameter throughout the plug-in

As mentioned before, Describe provides a static description of all communication pathways between a plug-in's algorithm, host, and data model. In addition, various effect properties are defined that help the host determine how to handle the plug-in.

Communication paths between the various plug-in components are described as connections between source and destination ports. In order for these communication paths to be created, the algorithm must first define some destination ports by actually registering its previously defined context fields as communication destinations. DemoGain does this for its gain parameter through the following call to `AAX_IComponentDescriptor::AddDataInPort()` in `DemoGain_Describe.cpp`'s `DescribeAlgorithmComponent()` method:

```
AAX_IComponentDescriptor * compDesc;
compDesc = outDescriptor->NewComponentDescriptor ();
err = compDesc->AddDataInPort (
    eAlgPortID_CoefsGainIn ,
    sizeof (SDemoGain_CoefsGain) );
```

Listing 6: Adding a data port to DemoGain's algorithm component descriptor

This registration process is required for both custom coefficients (Gain) and for data that all plug-ins need such as audio input and output fields.

That completes the connection, and now the plug-in is fully wired to receive parameter updates, convert raw parameter values to algorithmic coefficients, pack these coefficients into a packet, post the packet to the host for routing, receive the updated packet in the list of context structures that the host provides when calling the algorithm callback, and apply the updated coefficient data in the appropriate context structure to the plug-in's audio data. Whew!

12.2.5.5 GUI: Add a control

Although the specific steps for adding a GUI control to edit a plug-in parameter will vary depending on the GUI framework you choose, there are a few basic design principles that should always be followed.

The basic DemoGain plug-in does not include any GUI implementation. For practical GUI implementation examples, open the DemoGain_GUIExtensions sample plug-ins. DemoGain_Cocoa provides an example of a custom plug-in GUI using the native macOS SDK, while DemoGain_Win32 uses native Windows APIs. The other examples in this folder require common third-party libraries.

12.2.5.5.1 Control edits vs. parameter updates The most important principle for AAX GUI design is that an edit in a plug-in's GUI should never directly set the state of the associated parameter or parameters. This is because there may be other controller clients of the plug-in's data model which will need to be notified of the edit, or which may need to override edits from the GUI.

- On control edit

When a control is edited in the plug-in GUI, call `AAX_IEffectParameters::SetParameterNormalizedValue()`. The implementation of this method should post a request to the host in order to trigger the parameter update. The GUI should not update its state until the corresponding parameter update notification is received.

- On parameter update

A parameter update can occur in response to a GUI edit, an edit from another attached controller, an update to a linked parameter, or any other event that affects the state of the data model. When the state of a parameter changes, `AAX_IEffectGUI::ParameterUpdated()` is called. The plug-in's GUI should be updated in this method in order to reflect the new state of the affected parameter.

For detailed information about the sequence of events and GUI responsibilities during a parameter update, see [Parameter updates](#)

12.2.5.5.2 Notifying the host of GUI events Some GUI events must be handled by the host rather than by the plug-in. For example, in Pro Tools a user should be able to display a pop-up menu for controlling automation information by command-option-control-clicking (Mac) or alt-ctl-right clicking (Windows) a control. These events, and other direct communication between the GUI and the "container" in which the host creates the plug-in view, is accomplished via the [AAX_IViewContainer](#) interface. Be sure to always call the handler methods in this interface before handling a mouse event within the plug-in GUI, in order to maintain the expected host behavior.

12.2.6 Next Steps

Now that you have a basic understanding of AAX, head back to the [front page](#) and continue reading through the suggested documentation. Or dig in to the [sample plug-ins](#) to see how specific features are supported in AAX.

Warning

Your AAX plug-ins will not be compatible with shipping versions of Pro Tools until they are digitally signed using tools provided by PACE Anti-Piracy, Inc. As an AAX developer you can receive these tools free of charge. Read the [Digital signature](#) section of the [Pro Tools Guide](#) to learn about the digital signing requirements for compatibility with Pro Tools.

Collaboration diagram for Getting Started with AAX:

12.3 Core AAX Interface

12.3.1

Main classes, callbacks, and format specification details for a standard AAX plug-in.

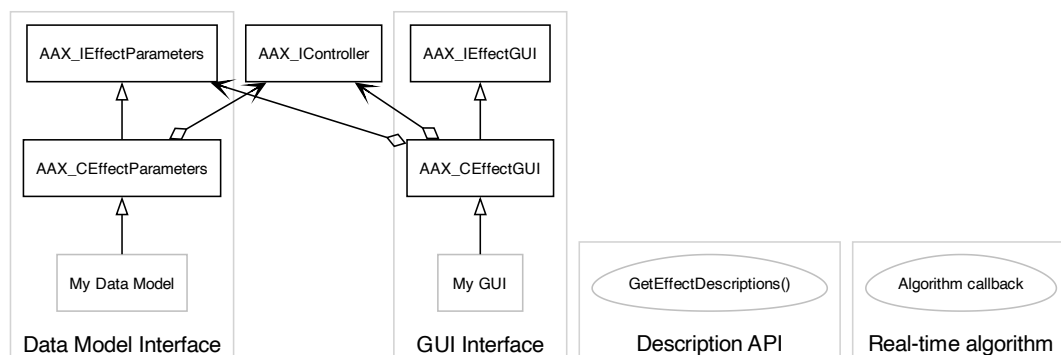


Figure 12.1 Main classes and callbacks for a standard AAX plug-in

These interfaces and components represent the standard interface between an AAX plug-in and the host. Default implementations for each interface are provided in the SDK. See the following modules for more information about each component.

See also

[Component Structure](#) in the [Getting Started Guide](#)

Documents

- [Description callback](#)
Static configuration for an AAX plug-in.
- [Real-time algorithm callback](#)
A plug-in's audio processing core.
- [Data model interface](#)
The interface for an AAX Plug-in's data model.
- [Data model overrides interface](#)
- [GUI interface](#)
The interface for a AAX Plug-in's user interface.
- [AAX communication protocols](#)
How to transfer data between different parts of an AAX plug-in.
- [AAX Format Specification](#)
Additional requirements for AAX plug-ins.

Collaboration diagram for Core AAX Interface:

12.4 Description callback

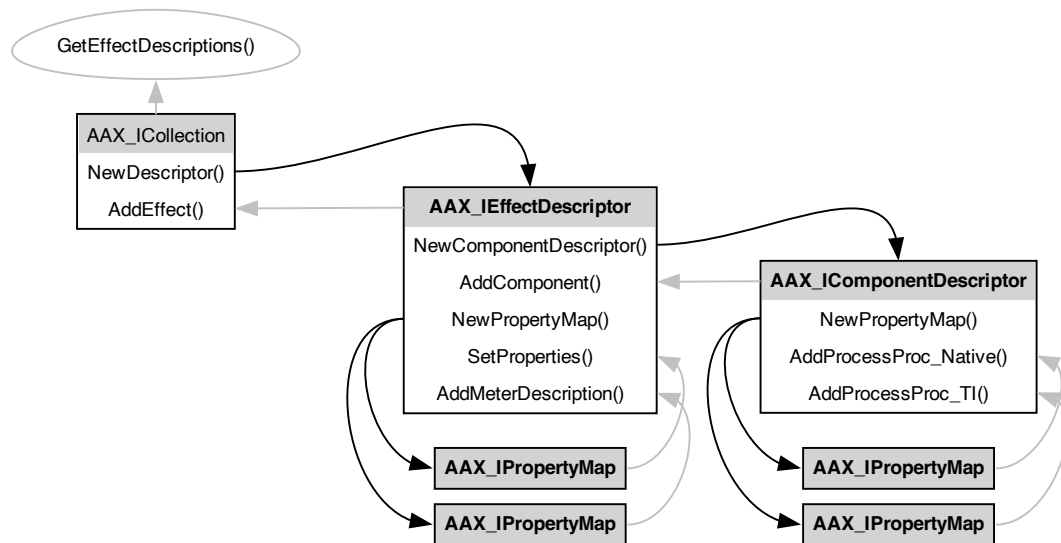
12.4.1

Static configuration for an AAX plug-in.

12.4.2 On this page

- [About the Describe callback and AAX descriptor interfaces](#)
- [Top level: Collection](#)
- [Middle level: Effects](#)
- [Lowest level: Algorithm components](#)
- [Host feature and environment checks](#)
- [Checking Results](#)
- [Describe Validation](#)
- [Additional Topics](#)

12.4.3 About the Describe callback and AAX descriptor interfaces



In Describe, a plug-in declares its static (or default) configuration and any properties that the host will need in order to manage the plug-in.

A plug-in's Describe callback ties the Algorithm, GUI, and Data Model together. In this callback, the plug-in provides a description of its algorithm callbacks, data connections, and other static plug-in properties using a set of host-provided description interfaces. The plug-in uses a tiered hierarchy of these description interfaces to complete its description:

- At the top level, a single Collection interface represents properties that apply to the plug-in binary.
- The Collection interface is populated with one or more Effect descriptors, each of which represents a single kind of effect. For example, a single dynamics plug-in binary may include both single-band and multi-band Effects. Each Effect represents a different plug-in "product" available to users.
- Each Effect is registered with a set of one or more algorithm components that represent the specific processing configurations (e.g. stem formats, sample rates) that the plug-in supports. The set of components in a single Effect provide possible variations of the single plug-in "product", and as such these variations are most transparent to users.

The actual description callback entrypoint is [ACFRegisterPlugin\(\)](#), which is declared in [AAX_Exports.cpp](#). This method is implemented inside of the AAX Library, where it calls the plug-in's custom implementation of the [GetEffectDescriptions\(\)](#) callback.

```

// *****
// ROUTINE: GetEffectDescriptions
// *****
AAX_Result GetEffectDescriptions( AAX_ICollection * outCollection )
{
    AAX_Result result = AAX_SUCCESS;
    AAX_IEffectDescriptor * plugInDescriptor = outCollection->NewDescriptor();
    if ( plugInDescriptor )
    {
        result = DemoGain_GetPlugInDescription( plugInDescriptor );
        if ( result == AAX_SUCCESS )
            outCollection->AddEffect( kEffectID_DemoGain, plugInDescriptor );
    }
    else result = AAX_ERROR_NULL_OBJECT;
}

```



```

    outCollection->SetManufacturerName( "Avid, Inc." );
    outCollection->AddPackageName( "DemoGain Plug-In" );
    outCollection->AddPackageName( "DemoGain" );
    outCollection->AddPackageName( "DmGi" );
    outCollection->SetPackageVersion( 1 );
    return result;
}

```

[GetEffectDescriptions\(\)](#) from the DemoGain example plug-in

In general, the following procedure is used when describing an AAX plug-in:

1. Create an Effect description interface from the Collection interface provided by the host
2. Use the Effect description interface to create references to one or more component description interfaces.
3. Describe each algorithm component by populating the component descriptions.
4. Add the components to the Effect description
5. Add additional modules and properties to the Effect description.
6. Add the completed Effect to the Collection
7. Repeat for any additional Effects included in the plug-in binary.
8. Return the completed Collection interface to the host and exit.

Note

The host owns all memory associated with any descriptors that the plug-in returns via this callback.

12.4.4 Top level: Collection

The [AAX_ICollection](#) interface provides a creation function ([AAX_ICollection::NewDescriptor\(\)](#)) for new plug-in descriptors, which in turn provides access to the various interfaces necessary for describing a plug-in (see the [DemoGain_GetPlugInDescription\(\)](#) and [DescribeAlgorithmComponent\(\)](#) listings below).

When a plug-in description is complete, it is added to the collection via the [AAX_ICollection::AddEffect\(\)](#) method. The [AAX_ICollection](#) interface also provides some additional description methods that are used to describe the overall plug-in package. These methods can be used to describe the plug-in package's name, the name of the plug-in's manufacturer, and the plug-in package version. Once these have been described, the completed description interface is returned to the host and exits.

```

// *****
// ROUTINE: GetEffectDescriptions
// *****
AAX_Result GetEffectDescriptions( AAX_ICollection * outCollection )
{
    .
    .
    .
    AAX_IEffectDescriptor *   plugInDescriptor = outCollection->NewDescriptor();
    .
    .
    .
    result = DemoGain_GetPlugInDescription( plugInDescriptor );
    .
    .
    .
    outCollection->AddEffect( kEffectID_DemoGain, plugInDescriptor );
    .
    .
    .
    outCollection->SetManufacturerName( "Avid, Inc." );
    outCollection->AddPackageName( "DemoGain Plug-In" );
    outCollection->AddPackageName( "DemoGain" );
    outCollection->AddPackageName( "DmGi" );
    outCollection->SetPackageVersion( 1 );
    .
    .
    .
    return result;
}

```

Populating the [AAX_ICollection](#) interface

12.4.5 Middle level: Effects

The [AAX_IEffectDescriptor](#) interface provides description methods that are used to describe the Effect, such as its name, category, associated page table, and, importantly, creation methods for its data model, GUI, and other AAX modules.

```
// *****
// ROUTINE: DemoGain_GetPlugInDescription
// *****
static AAX_Result DemoGain_GetPlugInDescription( AAX_IEffectDescriptor * outDescriptor )
{
    int err;
    AAX_IComponentDescriptor * compDesc = outDescriptor->NewComponentDescriptor ();
    if ( !compDesc )
        return AAX_ERROR_NULL_OBJECT;
    // Add empty component descriptors to the host, register a processing
    // endpoint for each, and populate with description information.
    //
    // Alg component
    DescribeAlgorithmComponent( compDesc );
    err = outDescriptor->AddComponent( compDesc ); AAX_ASSERT (err == 0);
    outDescriptor->AddPlugInName ( "Demo Gain AAX" );
    outDescriptor->AddPlugInName ( "Demo Gain" );
    outDescriptor->AddPlugInName ( "DemoGain" );
    outDescriptor->AddPlugInName ( "DmGain" );
    outDescriptor->AddPlugInName ( "DGpr" );
    outDescriptor->AddPlugInName ( "Dn" );
    outDescriptor->AddPlugInCategory ( AAX_ePlugInCategory_Dynamics );
    outDescriptor->AddProcPtr( (void *) DemoGain_Parameters::Create, kAAX_ProcPtrID_Create_EffectParameters
    );
    outDescriptor->AddResourceInfo ( AAX_eResourceType_PageTable, "DemoGainPages.xml" );
#ifdef PLUGGUI != 0
    outDescriptor->AddProcPtr( (void *) DemoGain_GUI::Create, kAAX_ProcPtrID_Create_EffectGUI );
#endif
    return AAX_SUCCESS;
}
```

Populating an [AAX_IEffectDescriptor](#) interface

All components in an Effect must share the same AAX modules; for example, it is not possible to use one data model definition for one sample rate and another data model definition for a different sample rate. Therefore, a plug-in's AAX modules are defined in its Effect description.

12.4.5.1 Registering multiple Effects

A single plug-in package may include multiple Effects, which are added in turn in the description method. Once these connections are made, Describe passes the host a populated description interface and returns.

For example, consider an EQ plug-in that contains both one-band and four-band variations, each of which the user should see as a distinct plug-in. These Effects would be described and added separately to the collection object and would appear as separate products to the user.

```
AAX_Result GetEffectsDescriptions ( AAX_ICollection * outCollection )
{
    AAX_Result result = AAX_SUCCESS ;
    if ( result == AAX_SUCCESS )
    {
        AAX_IEffectDescriptor * aDesc1 = outCollection -> NewDescriptor ();
        // ...
        // Populate aDesc1 with one - band EQ description
        // ...
        result = outCollection -> AddEffect ( kEffectID_MyOneBandEQ , aDesc1 );
    }
    if ( result == AAX_SUCCESS )
    {
        AAX_IEffectDescriptor * aDesc4 = outCollection -> NewDescriptor ();
        // ...
        // Populate aDesc4 with four - band EQ description
        // ...
        result = outCollection -> AddEffect ( kEffectID_MyFourBandEQ , aDesc4 );
    }
    if ( result == AAX_SUCCESS )
    {
        outCollection -> SetManufacturerName ( "My Plug -Ins , Inc." );
        outCollection -> AddPackageName ( " MyEQ Plug -In" );
        outCollection -> AddPackageName ( " MyQ" ); // Short name
        outCollection -> SetPackageVersion ( 1 );
    }
    return result ;
}
```

Registering multiple Effects in a single Collection

12.4.6 Lowest level: Algorithm components

In order to register an algorithm component, a plug-in must describe the component's external interface. This includes each of the component's ports and any other fields in its context structure, a reference to its processing function entrypoint (its "Process Procedure", or ProcessProc,) and any other special properties that the host should know about.

The description of a context structure involves a set of port definitions, which can be "hard-wired" to receive data from the host (such as audio buffers), from the plug-in's data model (such as packets of coefficients), or even from past calls to the algorithm (private, persistent algorithm state). See [Real-time algorithm callback](#) for more information on an algorithm's context structure.

```
static void DescribeAlgorithmComponent( AAX_IComponentDescriptor * outDesc )
{
    .
    .
    .
    AAX_Result err;
    // Subscribe context fields to host-provided services or information
    err = outDesc->AddField ( eAlgFieldID_AudioIn, kAAX_FieldTypeAudioIn ); AAX_ASSERT(err == 0);
    err = outDesc->AddField ( eAlgFieldID_AudioOut, kAAX_FieldTypeAudioOut ); AAX_ASSERT (err == 0);
    err = outDesc->AddField ( eAlgFieldID_BufferSize, kAAX_FieldTypeAudioBufferLength ); AAX_ASSERT (err == 0);
    // Register context fields as communications destinations
    err = outDesc->AddDataInPort ( eAlgPortID_BypassIn, sizeof (int32_t) ); AAX_ASSERT (err == 0);
    err = outDesc->AddDataInPort ( eAlgPortID_CoefsGainIn, sizeof (SDemoGain_CoefsGain) ); AAX_ASSERT (err == 0);
    .
    .
    .
}
```

Populating a single [AAX_IComponentDescriptor](#) interface

12.4.6.1 Algorithm callback properties

A set of callback properties is required when adding a Process Procedure to an algorithm component. This is done via the [AAX_IPropertyMap](#) interface. Using distinct property maps, a single component may register multiple versions of its callback. For example, an audio processing component might register mono and stereo callbacks, or Native and TI callbacks, assigning each the applicable property mapping. This allows the host to determine the correct callback to use depending on the environment in which the plug-in is instantiated.

```
static void DescribeAlgorithmComponent( AAX_IComponentDescriptor * outDesc )
{
    AAX_IPropertyMap * properties = outDesc->NewPropertyMap();
    .
    .
    .
    properties->Clear ();
    properties->AddProperty ( AAX_eProperty_ManufacturerID, cDemoGain_ManufactureID );
    properties->AddProperty ( AAX_eProperty_ProductID, cDemoGain_ProductID );
    properties->AddProperty ( AAX_eProperty_InputStemFormat, AAX_eStemFormat_Mono );
    properties->AddProperty ( AAX_eProperty_OutputStemFormat, AAX_eStemFormat_Mono );
    properties->AddProperty ( AAX_eProperty_CanBypass, true );
    // Native and AudioSuite versions
    properties->AddProperty ( AAX_eProperty_PlugInID_Native, cDemoGain_PlugInID_Native );
    properties->AddProperty ( AAX_eProperty_PlugInID_AudioSuite, cDemoGain_PlugInID_AudioSuite ); // Since
    // this is a linear plug-in the RTAS version can also be an AudioSuite version.
    properties->AddProperty ( AAX_eProperty_DSP_AudioBufferLength, kAAX_NativeAudioBufferLength_Default );
    err = outDesc->AddProcessProc_Native ( DemoGain_AlgorithmProcessFunction <1, 1,
    1<kAAX_NativeAudioBufferLength_Default>, properties ); AAX_ASSERT (err == 0);
    // TI DSP Version
    properties->AddProperty ( AAX_eProperty_PlugInID_TI, cDemoGain_PlugInID_TI );
    properties->AddProperty ( AAX_eProperty_DSP_AudioBufferLength, AAX_eAudioBufferLengthDSP_Default );
    properties->AddProperty ( AAX_eProperty_TI_InstanceCycleCount, 1055 );
    properties->AddProperty ( AAX_eProperty_TI_SharedCycleCount, 58 );
}
```

Adding properties to a component description

AAX does not require that every value in [AAX_IPropertyMap](#) be assigned by the developer. However, if a specific value is not assigned to one of an element's properties then the element must support any value for that property. For example, if an audio processing component does not provide any stem format properties then the host will assume that the callback will support any stem format.

12.4.7 Host feature and environment checks

AAX includes some mechanisms for detecting feature support in the host during Describe, such as [AAX_IDescriptionHost](#) , [AAX_ICollection::HostDefinition\(\)](#), and [AAX_ICollection::GetHostVersion\(\)](#). A plug-in may also check other aspects of its environment such as the OS version.

Warning

IMPORTANT: A plug-in should never add, remove, or change individual parts of its Description based on the host or environment. It should either provide a full and consistent Description or, if the host is not compatible, it should provide an empty Description.

The information provided by Describe is stored in a cache file. If the configuration information depends on factors outside of the plug-in itself then the information that was cached in one environment may be invalid in another environment.

A plug-in may perform checks during Describe to see if the host supports necessary features, to verify that the current OS is supported, or to check for other kinds of basic compatibility. If the plug-in determines that the host is not compatible then it should return an empty Description, i.e. it should not add any Effects to the Collection. An empty Description is never cached and therefore will not invalidate any existing cache information from a supported environment.

12.4.8 Checking Results

12.4.8.1 Summary

- Use [AAX_CheckedResult](#) to store result values from all method calls in Describe.
- Use the [AAX_SWALLOW](#) and [AAX_SWALLOW_MULT](#) macros to encapsulate independent describe code, such as registration logic for separate Effects or for separate ProcessProc variations within a single Effect.

12.4.8.2 The Problem

With plain [AAX_Result](#) values it can be challenging to properly detect and handle error states. Each description method call returns an [AAX_Result](#) to indicate success or failure, and often problems in a plug-in's configuration can be addressed by properly detecting and resolving errors that occur here. However, adding a return value check after every method and providing conditional logic in the case of a failure is onerous, ugly, and difficult to maintain.

```
AAX_Result result = AAX_SUCCESS;
result = descriptor->SomeMethod();
result = descriptor->AnotherMethod(); // oops! might ignore an error
// -----
// Safer, but not a good solution:
// Information about the errors is lost, the
// merged error code is meaningless, and
// debugging to find the location of the
// failure is hard.
//
result |= descriptor->SomeMethod();
result |= descriptor->AnotherMethod();
// ...
if (AAX_SUCCESS != result)
{
    // handle the merged error code
}
// -----
// This is also not a good solution:
// There is no actual handling of errors
// (from the SDK example plug-ins)
//
result = descriptor->SomeMethod();
AAX_ASSERT(AAX_SUCCESS == result);
```

```

result = descriptor->AnotherMethod();
AAX_ASSERT(AAX_SUCCESS == result);
// -----
// This is correct but is too hard
//
AAX_Result result = AAX_SUCCESS;
result = descriptor->SomeMethod();
if (AAX_SUCCESS != result) {
    // logic to handle this error:
    // assert and/or log the failure?
    // return or continue execution?
}
result = descriptor->AnotherMethod();
if (AAX_SUCCESS != result) {
    // ditto
}

```

[AAX_Result](#) based error checking is awkward

12.4.8.3 The Solution

The [AAX_CheckedResult](#) class is designed to solve this problem. [AAX_CheckedResult](#) can be used just like a plain-old-data [AAX_Result](#) :

```

AAX_CheckedResult result = AAX_SUCCESS;
result = descriptor->SomeMethod();
result = descriptor->AnotherMethod();

```

Simpler result checking with [AAX_CheckedResult](#)

When a failure is encountered, [AAX_CheckedResult](#) will:

- Store the error value
- Log the error using `AAX_TRACE_RELEASE`
- Throw an exception of type [AAX_CheckedResult::Exception](#)

To make this safe to use in the Describe routine, the [AAX](#) Library includes a try/catch block around the call to the plug-in's `GetEffectDescriptions()` routine.

Warning

Do not use [AAX_CheckedResult](#) anywhere where an exception could escape to the host (`GetEffectDescriptions()` is OK)

12.4.8.4 Handling Errors and Managing Control Flow

With the basic approach shown above, any error which is encountered will throw an exception which will cancel the plug-in's registration and prevent the plug-in from being shown in the host. However, most errors can be safely handled without canceling the entire plug-in registration.

```

AAX_CheckedResult result = AAX_SUCCESS;
// effect 1 registration
result = DescribeMyEffect1( effect1Descriptor );
result = outCollection->AddEffect( myEffect1ID, effect1Descriptor );
// effect 2 registration
result = DescribeMyEffect2( effect2Descriptor );
result = outCollection->AddEffect( myEffect2ID, effect2Descriptor );

```

Example with no error handling

In this example, a failure when describing either individual effect will prevent the other effect from being registered. Registration of individual ProcessProcs within a single effect, e.g. for different stem formats, is similar.

To allow the registration of other effects to proceed in the event of a failure, any exceptions thrown during the registration of one effect should be caught and should only prevent the registration of that individual effect.

```
AAX_CheckedResult result = AAX_SUCCESS;
// effect 1 registration
try {
    result = DescribeMyEffect1( effect1Descriptor );
    result = outCollection->AddEffect( myEffect1ID, effect1Descriptor );
}
catch (const AAX_CheckedResult::Exception& ex) {
    // log the error using ex.What()
    // swallow the exception and proceed
}
// effect 2 registration
try {
    result = DescribeMyEffect2( effect2Descriptor );
    result = outCollection->AddEffect( myEffect2ID, effect2Descriptor );
}
catch (const AAX_CheckedResult::Exception& ex) {
    // ditto
}
```

Example of error handling with try/catch

This solves the problem fully, but it is still cumbersome - especially when registering a long list of separate Process↵ Proc variants!

The [AAX_SWALLOW_MULT](#) macro makes it easier to handle errors which are thrown by [AAX_CheckedResult](#) :

```
AAX_CheckedResult result = AAX_SUCCESS;
// effect 1 registration
AAX_SWALLOW_MULT (
    result = DescribeMyEffect1( effect1Descriptor );
    result = outCollection->AddEffect( myEffect1ID, effect1Descriptor );
);
// effect 2 registration
AAX_SWALLOW_MULT (
    result = DescribeMyEffect2( effect2Descriptor );
    result = outCollection->AddEffect( myEffect2ID, effect2Descriptor );
);
```

Example of error handling with [AAX_SWALLOW_MULT](#)

Variations

- For single-line try/catch there is also [AAX_SWALLOW](#).
- If you need to reference the error value after the exception is caught, use [AAX_CAPTURE_MULT](#) (multi-line) or [AAX_CAPTURE](#) (single-line)
- If you know that a certain error code is OK and should not throw in a given situation then you can add it as an exception to the [AAX_CheckedResult](#) object with [AAX_CheckedResult::AddAcceptedResult\(\)](#).

For examples of [AAX_CheckedResult](#) in use, see the [DemoGain_Multichannel](#) and [DemoGain_UpMixer](#) plug-ins

12.4.9 Describe Validation

12.4.9.1 Validation with DSH

You can validate your plug-in's Describe routine using the [DigiShell](#) command-line tool. The validation command is available directly in the aaxh dish and is also available through an AAX Validator test module:

aaxh dish

```
dsh> load_dish aaxh
dsh> loadpi "/quoted/path/without escape chars/MyPlugIn.aaxplugin"
dsh> getdescriptionvalidationinfo 0
```

AAX Validator

```
dsh> load_dish aaxval
dsh> runtest [test.describe_validation, "/quoted/path/without escape chars/MyPlugIn.aaxplugin"]
```

12.4.9.2 Validation with Pro Tools

Beginning in Pro Tools 12.8.2, developer builds of Pro Tools will also check plug-in describe routines and will present an alert dialog when the plug-in is scanned if any aspect of the plug-in's describe code has failed the validation step.

Describe validation warning in a Pro Tools developer build

The specific kinds of errors which were encountered will be printed to the [DigiTrace](#) log file:

```
13033.502646,00307,0073: ERROR: Unknown target host for the plug-in.
13033.502662,00307,0073: ERROR: PlugInID property is missing for a ProcessProc (process, initialization, or ba
13033.502734,00307,0e0f: CMN_TRACEASSERT Sandbox.aaxplugin configuration contains 2 errors. See the DigiTrace
```

This check may be suppressed using the following [DigiOption](#):

```
TestPlugInDescriptions 0
```

12.4.10 Additional Topics

See also

[Plug-in meters](#)

[Plug-in loading and AAE initialization in Pro Tools](#)

Classes

- class [AAX_ICollection](#)
Interface to represent a plug-in binary's static description.
- class [AAX_IComponentDescriptor](#)
Description interface for an AAX plug-in component.
- class [AAX_IEffectDescriptor](#)
Description interface for an effect's (plug-in type's) components.
- class [AAX_IPropertyMap](#)
Generic plug-in description property map.

Functions

- [AAX_Result AAXRegisterPlugin](#) ([IACFUnknown](#) *pUnkHost, [IACFPluginDefinition](#) **ppPluginDefinition)
The main plug-in registration method.
- [AAX_Result GetEffectDescriptions](#) ([AAX_ICollection](#) *inCollection)
The plug-in's static Description entrypt.

12.4.11 Function Documentation

12.4.11.1 AAXRegisterPlugin()

```
AAX_Result AAXRegisterPlugin (
    IACFUnknown * pUnkHost,
    IACFPluginDefinition ** ppPluginDefinition )
```

The main plug-in registration method.

This method determines the number of components defined in the dll. The implementation of this method in the AAX library calls the following function, which must be implemented somewhere in your plug-in:

```
extern AAX_Result GetEffectDescriptions( AAX_ICollection * outCollection );
```

Wrapped by [ACFRegisterPlugin\(\)](#)

Referenced by [ACFRegisterPlugin\(\)](#).

Here is the caller graph for this function:

12.4.11.2 GetEffectDescriptions()

```
AAX_Result GetEffectDescriptions (
    AAX_ICollection * inCollection )
```

The plug-in's static Description entrypoint.

This function is responsible for describing an AAX plug-in to the host. It does this by populating an [AAX_ICollection](#) interface.

This function must be included in every plug-in that links to the AAX library. It is called when the host first loads the plug-in.

Parameters

out	inCollection	
-----	--------------	--

Collaboration diagram for Description callback:

12.5 Real-time algorithm callback

A plug-in's audio processing core.

12.5.1 On this page

- [Algorithm definition](#)
- [Algorithm memory management](#)
- [Communicating with the algorithm](#)
- [Algorithm initialization](#)

- [Algorithm processing](#)
- [Persistent algorithm memory](#)
- [Example algorithm callback](#)
- [Port Types and Behavior](#)
- [Additional Information](#)

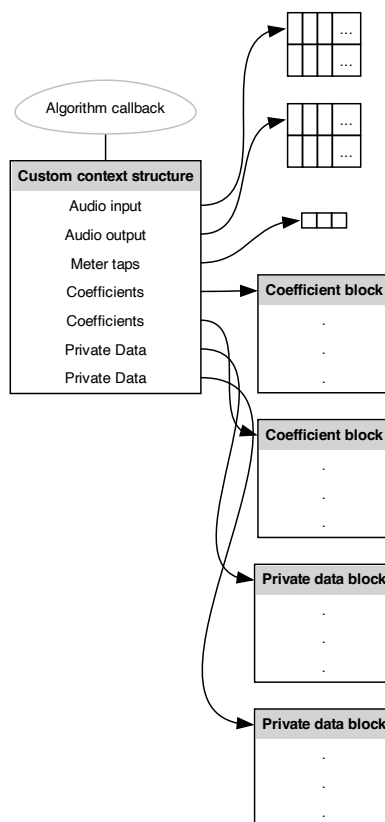
12.5.2 Algorithm definition

Algorithm processing in AAX plug-ins is handled via a C-style algorithm processing callback (see code below.) Each Effect variation in a plug-in must register an algorithm entypoint in the plug-in [description](#), and the host will call this entypoint to render a buffer of audio samples.

```
void AAX_CALLBACK MyPlugIn_AlgorithmProcessFunction(
    SMyPlugIn_Alg_Context * const inInstancesBegin [],
    const void * inInstancesEnd)
{
    //
    // Processing code...
    //
}
```

12.5.3 Algorithm memory management

This callback pattern is designed such that plug-in algorithms may be easily loaded in remote memory spaces on a variety of devices and quickly re-compiled for different operating environments without significant changes to the code, and this design goal informs the algorithm's memory management techniques.



When the AAX host calls a plug-in's algorithm callback, it provides a block of memory describing the state of the plug-in. This block of memory, known as the algorithm's *context*, can be thought of as the algorithm's interface to the outside world: when another part of the plug-in interacts with the algorithm, it does so by posting information to the algorithm's context.

```
//=====
// Component context definitions
//=====
// Context structure
struct SDemoDist_Algorithm_Context
{
    int32_t          * mCtrlBypassP;           // Coefficient message destination
    float            * mCtrlMixP;              // Coefficient message destination
    float            * mCtrlInpGP;             // Coefficient message destination
    float            * mCtrlOutGP;             // Coefficient message destination
    SDemoDist_DistCoefs * mCoefsDistP;          // Coefficient message destination
    SDemoDist_FiltCoefs * mCoefsFiltP;          // Coefficient message destination
    CSimpleBiquad     * mBiquads;              // Private data
    float*            * mInputPP;              // Audio signal input
    float*            * mOutputPP;             // Audio signal output
    float*            * mMeterTapsPP;          // Meter signal output
};
```

It is important to note that, in most circumstances, algorithm callbacks do not own their own memory. The algorithm and its memory is managed entirely by the host or shell environment, and relies on the host-provided context structure for all state information.

If persistent memory is required, algorithms can register for block(s) of persistent state data via the [AAX_IComponentDescriptor::AddPrivateData\(\)](#) API (as in `SDemoDist_Algorithm_Context::mBiquads` above.) A plug-in may store state data in the resulting "private data" context fields and this data will be restored by the host when the algorithm is next called. See the [Persistent algorithm memory](#) section below for more information.

12.5.4 Communicating with the algorithm

Plug-ins communicate with their algorithms via a buffered, host-managed message system. The host guarantees that messages posted to this system will be delivered to the applicable context field and that the algorithm's context is up to date every time the component is entered.

This system utilizes a static data routing scheme that is defined in the plug-in's `Describe` method. Once the routing scheme has been defined, the plug-in may post packets of data to its algorithm using [AAX_IController::PostPacket\(\)](#).

In order to reference the fields in its algorithm's context, the plug-in's host-side code uses unique identifiers generated with the [AAX_FIELD_INDEX](#) macro:

```
enum EDemoDist_Algorithm_PortID
{
    eAlgPortID_BypassIn          = AAX_FIELD_INDEX (SDemoDist_Algorithm_Context, mCtrlBypassP)
    ,eAlgPortID_MixIn            = AAX_FIELD_INDEX (SDemoDist_Algorithm_Context, mCtrlMixP)
    ,eAlgPortID_InpGIn           = AAX_FIELD_INDEX (SDemoDist_Algorithm_Context, mCtrlInpGP)
    ,eAlgPortID_OutGIn           = AAX_FIELD_INDEX (SDemoDist_Algorithm_Context, mCtrlOutGP)
    ,eAlgPortID_CoefsDistIn      = AAX_FIELD_INDEX (SDemoDist_Algorithm_Context, mCoefsDistP)
    ,eAlgPortID_CoefsFiltIn      = AAX_FIELD_INDEX (SDemoDist_Algorithm_Context, mCoefsFiltP)
    ,eAlgFieldID_Biquads         = AAX_FIELD_INDEX (SDemoDist_Algorithm_Context, mBiquads)
    ,eDemoDist_AlgorithmFieldID_AudioIn = AAX_FIELD_INDEX (SDemoDist_Algorithm_Context, mInputPP)
    ,eDemoDist_AlgorithmFieldID_AudioOut = AAX_FIELD_INDEX (SDemoDist_Algorithm_Context, mOutputPP)
    ,eAlgFieldID_MeterTaps       = AAX_FIELD_INDEX (SDemoDist_Algorithm_Context, mMeterTapsPP)
};
```

See [Description callback](#) for more information about registering context fields and defining a plug-in's message routing scheme.

12.5.5 Algorithm initialization

The following events occur before the AAX host begins calling a plug-in's algorithm:

- The Effect's [data model](#) is initialized

- An initial call to [AAX_IEffectParameters::ResetFieldData\(\)](#) is made for each private data block in the algorithm.
- An initial call to [AAX_IEffectParameters::GenerateCoefficients\(\)](#) is made and coefficient packets are dispatched to each of the algorithm's data ports based on the default model state.
- All packets are delivered and initial algorithm context state is set
- If one has been registered, the algorithm's optional initialization callback is called with the default context
- (Algorithmic processing begins)

12.5.5.1 Private data initialization

To initialize an algorithm's private data blocks, [AAX_IEffectParameters::ResetFieldData\(\)](#) is called on the host for each block in the algorithm. The host uses this method to acquire a default initialized memory block for each private data port, which is then copied into the algorithm's memory pool and provided to its context.

The default implementation of this method in [AAX_CEffectParameters](#) will initialize the data to zero.

See also

[Persistent algorithm memory](#)

12.5.5.2 Optional initialization callback

If any additional initialization or de-initialization steps are required for proper operation of the algorithm, an optional initialization routine may be registered and associated with the algorithm's processing callback. This initialization routine will be called in the same device / memory space as the algorithm's processing context. The initialization callback is provided with the algorithm's default context and is called both before every new instance of the Effect begins its algorithm render callbacks and before every instance is destroyed.

This initialization routine is provided in [Describe](#) as an argument to the platform's `AddProcessProc` registration method:

- [AAX_IComponentDescriptor::AddProcessProc_Native\(\)](#)
- [AAX_IComponentDescriptor::AddProcessProc_TI\(\)](#)

Host Compatibility Notes As of Pro Tools 10.2.1 an algorithm's initialization callback routine will have up to 5 seconds to execute.

See also

[AAX_CInstanceInitProc](#)

12.5.6 Algorithm processing

Once the algorithm has been initialized and processing begins, the algorithm function is called regularly by the host audio engine. The algorithm may read the context data provided by the host and is responsible for writing data to all of the samples in its output buffers each time it is executed.

Note

The data in an algorithm's output buffers is not initialized before the algorithm is called, thus the algorithm must always write data into all output samples. This is to ensure equivalent behavior between all platforms, some of which do not have the resource budget to pre-initialize output data buffers.

12.5.7 Persistent algorithm memory

An AAX plug-in algorithm may contain one or more *private data* ports in its context. These are the only context fields in which an algorithm may store persistent state data.

12.5.7.1 Private memory characteristics

Each private data port is a pointer to a preallocated block of memory. The size of each port is defined during Describe when the port is registered. On DSP systems, the plug-in may request that the data block be placed in the chip's external memory.

Once private data is allocated by the plug-in host or DSP shell, it will not be relocated or re-allocated until the algorithm is destroyed (see [Optional initialization callback](#))

12.5.7.2 Private data port registration

Private data ports are registered during Describe via [AAX_IComponentDescriptor::AddPrivateData\(\)](#). This method defines the size of the data block that will be allocated as well as an initialization callback with format [AAX_CInitPrivateDataProc](#).

12.5.7.3 Private data initialization

[AAX_IEffectParameters::ResetFieldData\(\)](#) is called on the host for both Native and DSP plug-ins. For DSP plug-ins, the initialized data block is copied to the DSP by the AAX host following the initialization callback. The initialization callbacks for a plug-in's private data blocks are called after all host modules have been initialized and before the algorithm's optional initialization callback.

See also

[Algorithm initialization](#)

12.5.7.4 Private data communication

It is possible to transfer data to and from the algorithm's private data blocks using the [AAX_IPrivateDataAccess](#) interface, which is available in a TimerWakeup context through the [AAX_IEffectDirectData](#) interface. For more information about this API, see [auxinterface_directdata_privatedataaccess](#).

12.5.8 Example algorithm callback

As a final example, the code below describes a simple audio processing component. The component's context contains one message pointer to receive incoming "gain" parameter values, as well as one audio data input, "pdI", and one audio data output, "pdO". Additionally there is a message pointer to receive "bypass" on/off values. The host calls the component each time a new input sample buffer must be processed, and each time the component is called the host ensures that all context fields are up-to-date.

```
void AAX_CALLBACK MyPlugIn_AlgorithmProcessFunction(
    SMyPlugInAlg_Context* const inInstancesBegin [],
    const void* inInstancesEnd)
{
    // Get a pointer to the beginning of the memory block table
    SMyPlugInAlg_Context* AAX_RESTRICT instance = inInstancesBegin [0];
    //----- Iterate over plug-in instances -----//
    for (SMyPlugInAlg_Context* const* walk = inInstancesBegin; walk < inInstancesEnd; ++walk)
    {
        instance = *walk;
        //----- Retrieve instance-specific information -----//
        //
        const SMyPlugIn_CoefsGain* const AAX_RESTRICT coefsGainP = instance->mCoefsGainP; // Input
        (const)
        const int32_t bypass = *instance->mCtrlBypassP;
        const float gain = coefsGainP->mGain;
        //----- Run processing loop over each input channel -----//
        //
        for (int ch = 0; ch < kNumChannelsIn; ch++) // Iterate over all input channels
        {
            //----- Run processing loop over each sample -----//
            //
            for (int t = 0; t < kAudioWindowSize; t++)
            {
                float* const AAX_RESTRICT pdI = instance->mInputPP [ch];
                float* const AAX_RESTRICT pdO = instance->mOutputPP [ch];
                if ( pdI && pdO )
                {
                    pdO [t] = gain * pdI [t];
                    if (bypass) { pdO [t] = pdI [t]; }
                }
            } // Go to the next sample
        } // Go to next channel
    } // End instance-iteration loop
}
```

12.5.9 Port Types and Behavior

In this section, we will examine the various kinds of ports that can be used by the algorithm component in an AAX plug-in:

1. Standard message input
2. Internal state storage
3. Metering output
4. Environment variable retrieval
5. Other functionality enhancement

12.5.9.1 Standard message input

Most ports will function as pointers to incoming data. This data can have any type. For example, an algorithm's context may include a port of type `float*` to receive incoming float data and another port of type `SMyCustomStructure*` to receive incoming `SMyCustomStructure` data.

Like all registered context fields, input ports are managed by the hosting environment such that they always point to the most recently received data at the time that the algorithm callback is entered. The algorithm may not store or alter data in a standard message input port: this data is available as read-only input. If data is stored in the space allocated for the port's data then the result will be undefined behavior.

To define a standard message input port, a plug-in should call `AAX_IComponentDescriptor::AddDataInPort()`.

12.5.9.2 Internal state storage

Most plug-ins require local data to be accessible to their algorithms. These may be static data, such as lookup tables, or dynamic data, such as coefficient smoothing history or delay lines. In the DemoDist sample plug-in, `SDemoDist_Alg_Context::mBiquads` is an example of this type of port: it is not modified by any other component and `DemoDist_AlgorithmProcessFunction()` relies on the `mBiquads` data persisting between processing calls.

A component that has registered a private data field is given access to a block of private data. Although the memory in this block will be allocated by the host, its data is fully owned by the component. Because this data is considered private to its parent component, other components cannot overwrite or target this data. Plug-ins that need to transmit data directly between their algorithms' private data ports and their other modules may use the [AAX_IEffectDirectData](#) interface, which provides an API for reading from or writing to this data from outside of the algorithm callback.

The plug-in's data model includes an initialization function that is called by the AAX host at the time of plug-in instantiation and "reset" events. This initialization method is called on the host for both Native and DSP plug-ins. Since this method is part of the plug-in's data model, it has direct access to plug-in state information.

12.5.9.3 Metering output

Plug-in metering ports are populated with an array of float values, or 'taps'. One tap is provided per plug-in meter. The algorithm writes per-buffer peak values to this port and the AAX host applies standardized ballistics to these values. Both raw and processed meter values are available to the plug-in's GUI.

12.5.9.4 Environment variable retrieval

Another use of ports is to receive data from the AAX host describing the execution environment. For example, an algorithm may include a port to receive the number of samples in its processing window or the sample rate. These services are provided automatically by the host once the component registers ports for them.

12.5.9.5 Other functionality enhancement

An algorithm component may use ports to gain additional functionality that is provided by the host. For example, an algorithm that will be compiled for accelerated environments may take advantage of the TI chip's Direct Memory Access functionality by registering a DMA port. The host will then allow this port to access memory directly using [AAX's DMA APIs](#).

12.5.10 Additional Information

For information about optional features for the algorithm processing callback, see the following [Additional AAX features](#) documentation:

- [Direct Memory Access](#)
- [Background processing callback](#)

Collaboration diagram for Real-time algorithm callback:

12.6 Data model interface

12.6.1

The interface for an AAX Plug-in's data model.

:Implemented by the Plug-In

The interface for an instance of a plug-in's data model. A plug-in's implementation of this interface is responsible for creating the plug-in's set of parameters and for defining how the plug-in will respond when these parameters are changed via control updates or preset loads. In order for information to be routed from the plug-in's data model to its algorithm, the parameters that are created here must be registered with the host in the plug-in's [Description callback](#).

At [initialization](#), the host provides this interface with a reference to [AAX_IController](#), which provides access from the data model back to the host. This reference provides a means of querying information from the host such as stem format or sample rate, and is also responsible for communication between the data model and the plug-in's (decoupled) algorithm. See [Real-time algorithm callback](#).

You will most likely inherit your implementation of this interface from [AAX_CEffectParameters](#), a default implementation that provides basic data model functionality such as adding custom parameters, setting control values, restoring state, generating coefficients, etc., which you can override and customize as needed.

The following tags appear in the descriptions for methods of this class and its derived classes:

- **CALL**: Components in the plug-in should call this method to get / set data in the data model.

Note

- This class always inherits from the latest version of the interface and thus requires any subclass to implement all the methods in the latest version of the interface. The current version of [AAX_CEffectParameters](#) provides a convenient default implementation for all methods in the latest interface.
- Except where noted otherwise, the parameter values referenced by the methods in this interface are normalized values. See [Parameter Manager](#) for more information.

Legacy Porting Notes In the legacy plug-in SDK, these methods were found in `CProcess` and `CEffectProcess`. For additional `CProcess` methods, see [AAX_IEffectGUI](#).

12.6.2 Related classes

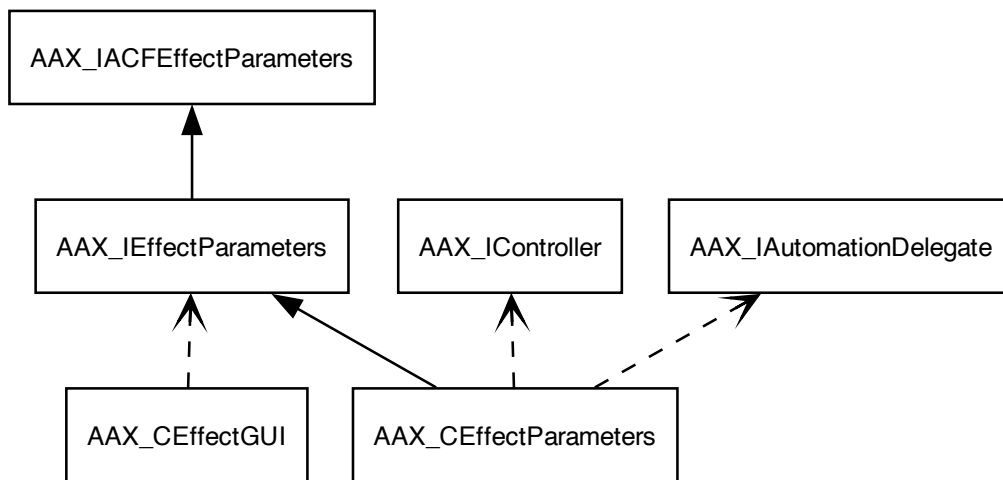


Figure 12.2 Classes related to **AAX_IEffectParameters** by inheritance or composition

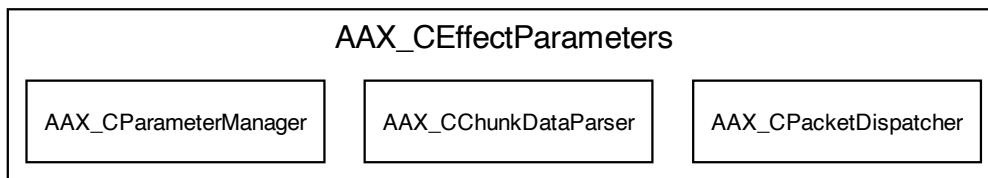


Figure 12.3 Classes owned as member objects of **AAX_CEffectParameters**

Classes

- class [AAX_CEffectParameters](#)
Default implementation of the [AAX_IEffectParameters](#) interface.
- class [AAX_IACFEffParameters](#)
The interface for an AAX Plug-in's data model.
- class [AAX_IACFEffParameters_V2](#)
Supplemental interface for an AAX Plug-in's data model.
- class [AAX_IACFEffParameters_V3](#)
Supplemental interface for an AAX Plug-in's data model.
- class [AAX_IACFEffParameters_V4](#)
Supplemental interface for an AAX Plug-in's data model.
- class [AAX_IEffectParameters](#)
The interface for an AAX Plug-in's data model.

Collaboration diagram for Data model interface:

12.7 Data model overrides interface

12.7.1

Chunk methods

These methods are used to save and restore collections of plug-in state information, known as chunks. Chunks are used by the host when saving or restoring presets and session settings and when providing "compare" functionality for plug-ins.

The default implementation of these methods in [AAX_CEffectParameters](#) supports a single chunk that includes state information for all of the plug-in's registered parameters. Override all of these methods to add support for additional chunks in your plug-in, for example if your plug-in contains any persistent state that is not encapsulated by its set of registered parameters.

Warning

Remember that plug-in chunk data may be loaded on a different platform from the one where it is saved. All data structures in the chunk must be properly data-aligned for compatibility across all platforms that the plug-in supports. See [AAX_ALIGN_FILE_ALG](#) for notes about common cross-platform pitfalls for data structure alignment.

For reference, see also:

- [AAX_CChunkDataParser](#)
- [AAX_SPlugInChunk](#)
- virtual [AAX_Result AAX_IACFEffectParameters::GetNumberOfChanges](#) (int32_t *oNumChanges) const =0
Retrieves the number of parameter changes made since the plug-in's creation.

12.7.2 Function Documentation

12.7.2.1 GetNumberOfChanges()

```
virtual AAX\_Result AAX_IACFEffectParameters::GetNumberOfChanges (
    int32_t * oNumChanges ) const [pure virtual]
```

Retrieves the number of parameter changes made since the plug-in's creation.

This method is polled regularly by the host, and can additionally be triggered by some events such as mouse clicks. When the number provided by this method changes, the host subsequently calls [CompareActiveChunk\(\)](#) to determine if the plug-in's Compare light should be activated.

The value provided by this method should increment with each call to [UpdateParameterNormalizedValue\(\)](#)

Parameters

out	<i>oNumChanges</i>	Must be set to indicate the number of parameter changes that have occurred since plug-in initialization.
-----	--------------------	--

Implemented in [AAX_CEffectParameters](#).

Collaboration diagram for Data model overrides interface:

12.8 GUI interface

12.8.1

The interface for a AAX Plug-in's user interface.

The [GUI interface](#) includes methods for handling the plug-in's GUI window and events.

Accessing the window

In AAX, the plug-in's window is provided as a native window pointer through the [AAX_IViewContainer](#) interface. The plug-in may also use this interface to forward events in its window back to the host for handling.

Default implementation

A default implementation of the GUI interface, [AAX_CEffectGUI](#), is compiled in to the AAX library. This class includes a few helper methods and other extensions to the base interface. Of particular note are several additional pure virtual methods that are used by this class to extend the GUI API, and which must be overridden by any inheriting class.

Extensions

The AAX SDK includes several examples of how the basic GUI interface may be extended to support native or third-party GUI frameworks. These examples are not a core part of the SDK, but are provided to developers as a convenience when incorporating their own chosen GUI framework.

Classes

- class [AAX_CEffectGUI](#)
Default implementation of the [AAX_IEffectGUI](#) interface.
- class [AAX_IACFEffctGUI](#)
The interface for a AAX Plug-in's GUI (graphical user interface).
- class [AAX_IEffectGUI](#)
The interface for a AAX Plug-in's user interface.
- class [AAX_IViewContainer](#)
Interface for the AAX host's view of a single instance of an effect. Used both by clients of the AAX host and by effect components.

Collaboration diagram for GUI interface:

12.9 AAX communication protocols

How to transfer data between different parts of an AAX plug-in.

AAX is a highly modular architecture. This section describes the various means by which AAX plug-in modules may communicate with one another and with the host.

There are two fundamental categories of communication in [AAX](#):

1. [Communication with the C++ interface objects](#)
2. [Communication with the real-time algorithm](#)

12.9.1 Communication with the C++ interface objects

12.9.1.1 Direct host communication

Most communication between the AAX host and the plug-in is accomplished via the [AAX_IController](#) interface. This interface contains methods for such things as:

- Retrieving environment information such as the current [sample rate](#)
- Getting and setting Effect parameters such as the Effect's [algorithmic delay](#)
- Accessing host-managed information such as [Plug-in meters](#) and MIDI
- Accessing other host-managed communications protocols like [Data packets](#) and MIDI

In addition, the GUI uses a separate interface for managing view and event details with the host. This interface, [AAX_IViewContainer](#), includes methods for:

- Retrieving information like the raw view and the currently held modifier keys
- Requesting changes to view parameters (e.g. size)
- Passing GUI events on to the host.
 - This is an important function because the host may require its own specific behavior for certain events. For example, a command-control-option click in Pro Tools should bring up the parameter's automation menu.

12.9.1.2 Custom data blocks

Often it is necessary to transmit arbitrary blocks of custom plug-in data between different plug-in modules. In AAX, this is accomplished by "pushing" data to and "pulling" it from the plug-in's [data model](#).

The abstract data model interface includes two custom data methods for this:

- [AAX_IEffectParameters::GetCustomData\(\)](#)
- [AAX_IEffectParameters::SetCustomData\(\)](#)

It is the data model's job to act as a go-between when custom data must be transmitted between a plug-in's other modules.

For example, a plug-in may wish to send analysis data from its [direct data module](#) to its [GUI](#). In this situation, the Direct Data object would call [SetCustomData\(\)](#) to update the data model whenever new data was available, while the GUI would "pull" the most up-to-date data via [GetCustomData\(\)](#) whenever an update was required.

Note that the default implementations of these methods are empty and thus all implementation details, including thread safety guards, are left to the plug-in.

12.9.1.3 Notifications

The [data model](#) and [GUI](#) interfaces include [notification hook](#) methods. These methods used for [host-to-Effect notifications](#) by default, but may also be called with custom notification IDs in order to create custom notifications within a plug-in.

12.9.1.4 Direct pointer sharing

If co-location is guaranteed, plug-in modules may directly share data pointers. For example, a non-real-time plug-in's [Host Processor](#) object may share a `this` pointer with its [data model](#) object.

To guarantee co-location between modules that could normally be placed into different memory spaces by the host, use "constraint" properties:

- [AAX_eProperty_Constraint_Location](#)
- [AAX_eProperty_Constraint_Topology](#)

To help avoid forwards-compatibility issues with future devices that support AAX, these constraints should be set whenever a plug-in requires co-location of its components. Note, however, that using a design that relies on co-location will prevent the plug-in from running in distributed environments and should therefore be avoided when possible.

12.9.2 Communication with the real-time algorithm

An AAX plug-in's algorithm is essentially a stateless callback and, therefore, all of its state data must at some level be managed by the host. This model is fundamentally different from the other plug-in modules, which are each objects with their own memory and state.

Most algorithmic data management is performed via the algorithm's context structure. More information about memory management in AAX real-time algorithms can be found [here](#).

12.9.2.1 Data packets

The most common form of communication with a plug-in's real-time algorithm callback is the transmission of read-only data from the data model to the context structure.

AAX includes a dedicated API for this task that provides buffered, optimized delivery of read-only data packets to the algorithm. For more information, see [Communicating with the algorithm](#).

12.9.2.2 Host-managed context fields

Algorithms can also send data to the host and receive environment information through dedicated context fields. For example, the host can provide access to DMA facilities through an object accessed via a DMA field, and a plug-in can report meter values to the host via a dedicated meter field. For more information, see [Communicating with the algorithm](#).

12.9.2.3 Direct data transfers

When other modules in the plug-in must interact directly with the algorithm's state information this is accomplished via the [Direct Data](#) interface. This interface provides an idle-time context in which the plug-in may read from or write to the algorithm's private data memory. These transfers are unbuffered and therefore the plug-in must handle any appropriate thread-safety considerations. Collaboration diagram for AAX communication protocols:

12.10 AAX Format Specification

Additional requirements for AAX plug-ins.

This document describes aspects of the AAX plug-in format specification that are beyond the scope of the [common interface classes and callbacks](#) that the plug-in must implement.

12.10.1 .aaxplugin Directory Structure

AAX uses a bundle packaging format. On macOS, AAX plug-ins are built as standard OS bundles, while on Windows they are simple directories. All AAX plug-in bindles must use the .aaxplugin extension and the following directory structure:

- /Contents
 - /Resources
 - * *This directory contains all of the additional resource files that will be needed by the plug-in at run time such as DSP algorithm DLLs, XML page tables, and image files for the plug-in's GUI*
 - * AAXProperties.xml (optional)
 - [AAX Properties file](#) for the bundle
 - /MacOS
 - * *Contains the plug-in's macOS binary (Mach-O)**
 - /Win32
 - * *Contains the plug-in's Windows x86 binary**
 - /x64
 - * *Contains the plug-in's Windows x64 binary**
 - /Factory Presets (optional)
 - * *This directory includes built-in plug-in presets. For more information, see [Presets and settings management](#) in the [Pro Tools Guide](#) documentation*
 - PkgInfo (macOS only)
 - * *This file must include the concatenation of the plug-in's CFBundlePackageType (TDMw or BNDL) and CFBundleSignature (PTul)*
 - Info.plist (macOS only)
 - * *The plug-in's property list*
- desktop.ini (Windows only)
 - *The .aaxplugin directory's view resource file, used to set its custom icon in Windows Explorer*
- PlugIn.ico (Windows only)

- *Custom plug-in icon file*

*See the following compatibility notes

Host Compatibility Notes

- The plug-in's binary filename must be the same as the outer .aaxplugin bundle name

Host Compatibility Notes

- On Windows, the plug-in binary (DLL) must use the ".aaxplugin" suffix; i.e. the DLL must use exactly the same name as the outer .aaxplugin folder. On macOS, the plug-in binary does not require a specific suffix.

Host Compatibility Notes

- On Windows, the plug-in's binary filename (and therefore also the outer .aaxplugin file name) must not contain any spaces. There is a bug in AAE that will prevent binaries with spaces from being loaded properly. This is logged as PTSW-189928.

Note

This directory structure is also used for plug-in installer directories in the VENUE plug-in installer system. See [VENUE Plug-in installer specification](#) for more information.

12.10.2 Required Symbols

The following symbols are required in any AAX plug-in and must not be stripped from the binary:

- `_ACFRegisterPlugin`
- `_ACFRegisterComponent`
- `_ACFGetClassFactory`
- `_ACFCanUnloadNow`
- `_ACFStartup`
- `_ACFShutdown`
- `_ACFGetSDKVersion` *

Host Compatibility Notes * `_ACFGetSDKVersion` is required for 64-bit AAX plug-ins only

Collaboration diagram for AAX Format Specification:

12.11 Additional AAX features

12.11.1

How to use additional features and functionality supported by AAX.

Documents

- [Auxiliary Output Stems](#)
Routing custom audio streams from a plug-in.
- [Background processing callback](#)
Background processing support for AAX DSP and Native plug-in algorithms.
- [Direct Memory Access](#)
DMA support for AAX DSP plug-ins, with emulation for AAX Native.
- [Direct data access interface](#)
A host interface providing direct access to a plug-in's algorithm memory.
- [EQ and Dynamics Curve Displays](#)
Displaying EQ and Dynamics curves in Pro Tools, control surfaces, and other auxiliary graphical interfaces.
- [Hybrid Processing architecture](#)
An architecture combining low-latency and high-latency audio processing.
- [Plug-in meters](#)
How to manage metering data for AAX plug-ins.
- [MIDI](#)
How to route and process MIDI in AAX plug-ins.
- [Offline processing interface](#)
Advanced offline processing features.
- [Properties File](#)
Properties for an AAX plugin bundle.
- [Sidechain Inputs](#)
Routing custom audio streams to a plug-in.
- [Task agent interface](#)
A mechanism for hosts to request that plug-ins perform tasks.

Collaboration diagram for Additional AAX features:

12.12 Auxiliary Output Stems

Routing custom audio streams from a plug-in.

12.12.1 Overview of Auxiliary Output Stems in AAX

Pro Tools has the capability to show and route multiple "auxiliary" outputs from a plug-in to other tracks. These are known as Auxiliary Output Stems (AOS), a stem referring to one set of outputs. A stereo stem contains two outputs, left and right, and a mono stem contains one output. The outputs will appear in the input assignment pop-up menu of each track under the category "plug-in".

Your plug-in is responsible for the definition of valid aux output paths. This definition includes the total number of outputs and the desired order of stereo and mono paths. Pro Tools will query each plug-in for available valid paths and populate its track input selector popup menus accordingly.

Plug-ins must define the lowest available aux output number. In other words, the port number of an aux output needs to be the lowest available port number after the main outputs of the track the plug-in is instantiated on. For example, the first available aux output for the plug-in residing on a 5.1 surround track would have a port number of 7, since there are 6 main outputs for the track.

Additionally, port numbers must be declared sequentially and in the order aux output stems are added. For example, a stem cannot be added with the port number 10 if it precedes a stem with the port number 4.

12.12.2 Implementing Auxiliary Output Stems

The Auxiliary Output Stems API has a specific descriptor associated with it that needs to be added in `Describe`↵: `AAX_IComponentDescriptor::AddAuxOutputStem()`. Make sure this method is called for each component that supports a different stem format. For example, a mono aux output would be defined as follows:

```
// *****
// ROUTINE: DescribeAlgorithmComponent
// Algorithm component description
// *****
static void DescribeAlgorithmComponent( AAX_IComponentDescriptor * outDesc )
{
    AAX_Result err = AAX_SUCCESS;

    [...]
    err = outDesc->AddAuxOutputStem(0 /* first parameter is not used */,
                                    AAX_eStemFormat_Mono,
                                    "My Auxiliary Output Channel");
    AAX_ASSERT (err == AAX_SUCCESS);
    [...]
}
```

The auxiliary output buffers for the plug-in will be appended to the normal output buffer array in the plug-in algorithm.

Warning

Some hosts, such as Media Composer, do not support Auxiliary Output Stems. You must clearly document that your plug-ins are not supported on these hosts; attempts by the plug-in to write data beyond the end of the audio output buffer may cause crashes and other bugs in these hosts. See [Host Support](#) for more information.

In your plug-in's algorithm, you will simply need to account for the extra outputs when it processes the audio. Pro Tools will not automatically route your processed audio to all the extra outputs. As with main outputs, make sure the processed audio samples are placed in the auxiliary outputs' buffers as well. Collaboration diagram for Auxiliary Output Stems:

12.13 Background processing callback

Background processing support for AAX DSP and Native plug-in algorithms.

12.13.1 On this page

- [Background thread description](#)
- [Restrictions and limitations of background threads](#)
- [Background thread performance characteristics on DSP systems](#)
- [Background thread memory management](#)
- [Additional information](#)

12.13.2 Background thread description

Each algorithm render callback may optionally be associated with a background processing callback. This background callback will be triggered regularly in an idle context on a separate thread, and can be used to perform any background task required by the algorithm.

Background thread processing is supported for both AAX DSP and AAX Native plug-ins.

12.13.3 Restrictions and limitations of background threads

- An AAX DSP Effect that registers for background processing will not share a DSP with any other Effect type. It may share a DSP with multiple instances of its own type, but only if its resource requirements allow for this.
- The frequency of background thread executions relative to render thread executions will vary depending on the processing situation. For example, a host may pre-process a series of audio buffers as quickly as possible during an offline render. In this case there would be many executions of the render thread callback for each execution of the background thread callback. Be sure to consider this when using the background thread feature in plug-ins that support an AudioSuite processing type.

12.13.4 Background thread performance characteristics on DSP systems

The background processing callback is called from a true idle thread context. On DSP accelerated platforms, this means that the callback will be triggered continuously whenever the chip is not executing an interrupt, i.e. the algorithm render callback. Since the render callback's resource requirements are well-defined (or at least strictly bounded,) the background thread's available cycles are also deterministically bounded.

However, the background thread itself has a lower priority than the DSP shell. While the background callback's execution will not be interrupted by shell operations, it will be blocked in the event of a contention for memory resources with the shell. As a result, the number of memory operations that may be performed in this callback will be less well-defined when the host is consuming memory resources, e.g. when delivering a very large coefficient block to the DSP.

If your TDM plug-in does not perform any resource-intensive memory operations then you can assume a guaranteed performance level for its DSP background thread. Development tools are available that will test a plug-in by refreshing its entire context memory at every interrupt, and the background thread performance characteristics measured by these tools, plus an additional buffer to account for any pathological cases that may be missed by the performance check, should provide a guaranteed performance baseline for the background thread that will be completely safe for any Pro Tools operation scenario.

12.13.5 Background thread memory management

The background processing callback is not provided with any data pointers and does not have access to any facilities for managed communication with the rest of the plug-in. Therefore, the background process must use shared global data structures to interact with the render callback. Your plug-in will need to manually synchronize access to this data.

Usually the background callback will want to interact with the render callback via the algorithm's private data blocks. Therefore, private data blocks that are provided to an algorithm's context will not be relocated by the host between calls to the render callback, and background processes can reliably access this data once provided with a pointer. The same is not true for audio buffers, meters, coefficient ports, etc. - this data can all be relocated by the host when the render callback is not executing.

12.13.6 Additional information

HDX DSP Guide

- [Background processing](#)
- [DMA and background thread performance reporting](#)

Collaboration diagram for Background processing callback:

12.14 Direct Memory Access

DMA support for AAX DSP plug-ins, with emulation for AAX Native.

12.14.1 On this page

- [DMA facility overview](#)
- [DMA transfer modes](#)
- [Registering for DMA transfers](#)
- [DMA restrictions](#)
- [Additional information](#)

12.14.2 DMA facility overview

AAX provides an [abstract interface](#) for accessing the host environment's DMA or other memory-transfer facilities. All platform-specific details are handled by the AAX host environment, allowing plug-ins that use this interface to be re-targeted to Native or DSP environments without changing their memory transfer implementation.

12.14.3 DMA transfer modes

AAX hosts may support the following DMA modes, as listed in [AAX_IDma::EMode](#) :

- In [Scatter](#) mode, data is transferred from a linear buffer to a series of offset segments in a circular buffer. This mode is most often used to transfer data from linear internal memory to a large external memory buffer.
- In [Gather](#) mode, data is collected from a series of offset segments in a circular buffer and concatenated in a linear buffer. This mode is most often used to transfer data from an external memory buffer to an internal memory buffer.
- In [Burst](#) mode, data is written linearly from one location to another. Burst mode transfers may be used for linear transfers of data to or from external memory. During the transfer, the source data is broken into a series of individual bursts. This mode is included for completeness, though the Scatter/Gather modes are expected to be more appropriate for the vast majority of real-world DMA use cases.

12.14.4 Registering for DMA transfers

Algorithm Components register for DMA transfers by adding one or more DMA fields to their context via [AAX_IComponentDescriptor::AddDmaInstance\(\)](#). At runtime, each field will be populated with a valid [DMA interface](#) for the specified DMA mode.

12.14.5 DMA restrictions

The following restrictions apply to DMA transfers on all AAX platforms:

- The maximum burst size for any DMA transfer is 64B. The minimum burst size is 1B.
- Only one DMA transfer request may be posted per [AAX_IDma](#) object per processing callback.
- Scatter and Gather requests each require that the circular memory buffer be padded by at least the size of one burst

12.14.6 Additional information

HDX DSP Guide

- [DMA support](#)
- [DMA and background thread performance reporting](#)

Collaboration diagram for Direct Memory Access:

12.15 Direct data access interface

12.15.1

A host interface providing direct access to a plug-in's algorithm memory.

This interface represents an optional component that you can add to your plug-in in order to support extended features of the AAX SDK.

Some plug-ins require the host to retrieve non-meter data from the decoupled algorithm module to display on a GUI or perform additional computation. For example, the result of computing the audio spectrum or pitch data in the algorithm can be delivered to the host to display on-screen. This is the purpose of the [AAX_IEffectDirectData](#) interface.

The [Direct Data interface](#) provides facilities for directly accessing a plug-in's algorithm memory. This interface may be used to transfer private data from the algorithm to other plug-in components, such as the [GUI](#). It may also be used as an alternative to [PostPacket\(\)](#) to perform direct writes to the algorithm's private data memory.

To set up Direct Data, the module must be registered with the host in the plug-in's Description callback like other process pointers. To add this interface to your plug-in at describe time, call [AAX_IEffectDescriptor::AddProcPtr\(\)](#) using the [kAAX_ProcPtrID_Create_EffectDirectData](#) selector.

The DirectData module works for all plug-in types, including AAX Native, AAX DSP, and AAX AudioSuite.

12.15.2 Convenience class

[AAX_CEffectDirectData](#), the concrete implementation of [AAX_IEffectDirectData](#), consists of a [TimerWakeup_PrivateDataAccess\(\)](#) function that you subclass in order to access an algorithm's private state data. This timer wakes up at a periodic interval. In this function you can read the algorithm's private data port to pull the state of an algorithm. Note that the wakeup period is variable depending on the plug-in's buffer size and running context (real time processing, AudioSuite, offline bounce, etc.) Care must be taken to ensure that any data retrieved from the algorithm is either buffered to handle the thread callback periods for the various running contexts or that the plug-in does not depend on the Direct Data timer catching every state update.

[AAX_CEffectDirectData](#) also includes convenience accessors to the Controller and Data Model in order to help facilitate common access scenarios. Using these, you can do any computation necessary to handle the incoming algorithm state data and send results on to the Data Model and/or the GUI interface.

12.15.3 Private data access interface

The Direct Data API provides a [TimerWakeup](#) callback with access to [AAX_IPrivateDataAccess](#). This reference is only valid within the context of the wakeup callback and cannot be stored to provide private data access in other contexts.

The Private Data Access interface can be used to directly read from and write to an algorithm's private data. These operations are not synchronized with the algorithm's processing callback, which may asynchronously pre-empt the read or write operations. Plug-ins that use this interface should buffer all access to their private data to ensure data integrity.

12.15.4 Communicating with other modules

The Direct Data API does not include any facilities for inter-module communication. In order to transfer data between a plug-in's [AAX_IEffectDirectData](#) object and its other objects, dedicated custom data methods in those objects' interfaces should be used. For example, to communicate with the plug-in's data model, use [AAX_IEffectParameters::GetCustomData\(\)](#) and [AAX_IEffectParameters::SetCustomData\(\)](#)

See also

[Hybrid Processing architecture](#) for another approach to transferring large amounts of (audio) data between the algorithm callback and the plug-in's data model.

Classes

- class [AAX_CEffectDirectData](#)
Default implementation of the [AAX_IEffectDirectData](#) interface.
- class [AAX_IACFEffectDirectData](#)
Optional interface for direct access to a plug-in's alg memory.
- class [AAX_IACFPrivateDataAccess](#)
Interface for the AAX host's data access functionality.
- class [AAX_IEffectDirectData](#)
The interface for a AAX Plug-in's direct data interface.
- class [AAX_IPrivateDataAccess](#)
Interface to data access provided by host to plug-in.

Collaboration diagram for Direct data access interface:

12.16 EQ and Dynamics Curve Displays

12.16.1

Displaying EQ and Dynamics curves in Pro Tools, control surfaces, and other auxiliary graphical interfaces.

About Pro Tools, control surfaces, and other auxiliary displays connected to the AAX host may provide a curve data display to enhance the graphical representation of the plug-in's state.

A "bouncing ball" meter may also be overlaid within the curve data presentation for Dynamics plug-ins.

Pro Tools Mix Window displaying EQ plug-in instances

Pro Tools | S6 MTM display showing a Dynamics plug-in instance with bouncing-ball metering

Requirements

Host Compatibility Notes For S6 control surface displays, see [PT-226228](#) and [PT-226227](#) in the [Known Issues](#) page for more information about the requirements listed in this section.

These are the requirements for supporting the AAX curve data display features:

- To support EQ curve data displays, a plug-in must support [AAX_IEffectParameters::GetCurveData\(\)](#)
- To support Dynamics curve data displays, a plug-in must also support [AAX_IEffectParameters::GetCurveDataDisplayRange\(\)](#) for the Dynamics curve data types.

The AAX host will only query and display curve data for plug-ins of the applicable [Category](#), as specified in the [AAX_ECurveType](#) documentation.

In order to present a bouncing-ball metering display, Dynamics plug-ins must also support [AAX_IEffectParameters::GetCurveDataMetering\(\)](#) in addition to the two other curve data methods. This feature is always optional: a Dynamics plug-in may present a curve only without support for a bouncing-ball meter overlay.

Pro Tools Implementation There are three different kinds of calls that Pro Tools will make when querying EQ plug-ins for curve data:

- An initial query with a small set of points. This query is used only to determine whether the plug-in supports the EQ Curve display feature. The result data is not used. If the plug-in does not support the feature it must return an error value from [GetCurveData\(\)](#)
- A normal full-curve query to get the base curve data across the full display range
- One or more targeted queries around any detected inflection points. These queries are used to increase display resolution for plug-ins with very narrow Q settings.

Pro Tools will call [GetCurveData\(\)](#) from a thread in a low-priority thread pool. Most other Pro Tools operations will not be blocked by the execution of this method, though note that if a control surface is also issuing queries then the method may be called concurrently from multiple host threads.

In Pro Tools, curve updates are triggered by incrementing the plug-in's change counter. This is the counter value returned from the [GetNumberOfChanges\(\)](#) method in the plug-in. This counter is updated automatically when any plug-in parameter changes.


```

-60 | ****
-----
| -60                                     0

supported: true
AXeq:
  values:
supported: false
  instanceID: 0
  pluginID: 0
message_type: cmd_result
...

```

Enumerations

- enum [AAX_ECurveType](#) {
[AAX_eCurveType_None](#) = 0 ,
[AAX_eCurveType_EQ](#) = 'AXeq' ,
[AAX_eCurveType_Dynamics](#) = 'AXdy' ,
[AAX_eCurveType_Reduction](#) = 'AXdr' }

Different Curve Types that can be queried from the Host.

Auxiliary UI methods

- virtual [AAX_Result](#) [AAX_IACFEffEffectParameters::GetCurveData](#) ([AAX_CTypeID](#) iCurveType, const float *iValues, uint32_t iNumValues, float *oValues) const =0
Generate a set of output values based on a set of given input values.

Auxiliary UI methods

- virtual [AAX_Result](#) [AAX_IACFEffEffectParameters_V3::GetCurveDataMeterIds](#) ([AAX_CTypeID](#) iCurveType, uint32_t *oXMeterId, uint32_t *oYMeterId) const =0
Indicates which meters correspond to the X and Y axes of the EQ or Dynamics graph.
- virtual [AAX_Result](#) [AAX_IACFEffEffectParameters_V3::GetCurveDataDisplayRange](#) ([AAX_CTypeID](#) iCurveType, float *oXMin, float *oXMax, float *oYMin, float *oYMax) const =0
Determines the range of the graph shown by the plug-in.

12.16.2 Enumeration Type Documentation

12.16.2.1 AAX_ECurveType

enum [AAX_ECurveType](#)

Different Curve Types that can be queried from the Host.

Note

All 'AX__' IDs are reserved for host messages

See also

[AAX_IEffectParameters::GetCurveData\(\)](#)
[AAX_IEffectParameters::GetCurveDataMeterIds\(\)](#)
[AAX_IEffectParameters::GetCurveDataDisplayRange\(\)](#)

Enumerator

AAX_eCurveType_None	
AAX_eCurveType_EQ	EQ Curve, input values are in Hz, output values are in dB. Host Compatibility Notes Pro Tools requests this curve type for EQ plug-ins only
AAX_eCurveType_Dynamics	Dynamics Curve showing input vs. output, input and output values are in dB. Host Compatibility Notes Pro Tools requests this curve type for Dynamics plug-ins only
AAX_eCurveType_Reduction	Gain-reduction curve showing input vs. gain reduction, input and output values are in dB. Host Compatibility Notes Pro Tools requests this curve type for Dynamics plug-ins only

12.16.3 Function Documentation

12.16.3.1 GetCurveData()

```
virtual AAX_Result AAX_IACFEffectParameters::GetCurveData (
    AAX_CTypeID iCurveType,
    const float * iValues,
    uint32_t iNumValues,
    float * oValues ) const [pure virtual]
```

Generate a set of output values based on a set of given input values.

This method is used by the host to generate graphical curves. Given a set of input values, e.g. frequencies in Hz, this method should generate a corresponding set of output values, e.g. dB gain at each frequency. The semantics of these input and output values are dictated by `iCurveType`. See [AAX_ECurveType](#).

Plug-ins may also define custom curve type IDs to use this method internally. For example, the plug-in's GUI could use this method to request curve data in an arbitrary format.

Note

- This method may be called by the host simultaneously from multiple threads with different `iValues`.

Note

- `oValues` must be allocated by caller with the same size as `iValues` (`iNumValues`).

Host Compatibility Notes Versions of S6 software which support the [GetCurveDataDisplayRange\(\)](#) method will not display a plug-in's curve data unless both [GetCurveData\(\)](#) and [GetCurveDataDisplayRange\(\)](#) are supported by the plug-in.

Warning

S6 currently polls this method to update a plug-in's EQ or dynamics curves based on changes to the parameters mapped to the plug-in's EQ or dynamics center section page tables. Parameters that are not included in these page tables will not trigger updates to the curves displayed on S6. (GWSW-7314, [PTSW-195316 / PT-218485](#))

Parameters

in	<i>iCurveType</i>	One of AAX_ECurveType
in	<i>iValues</i>	An array of input values
in	<i>iNumValues</i>	The size of <i>iValues</i>
out	<i>oValues</i>	An array of output values

Returns

This method must return [AAX_ERROR_UNIMPLEMENTED](#) if the plug-in does not support curve data for the requested *iCurveType*

Implemented in [AAX_CEffectParameters](#).

12.16.3.2 GetCurveDataMeterIds()

```
virtual AAX\_Result AAX_IACFEffectParameters_V3::GetCurveDataMeterIds (
    AAX\_CTypeID iCurveType,
    uint32_t * oXMeterId,
    uint32_t * oYMeterId ) const [pure virtual]
```

Indicates which meters correspond to the X and Y axes of the EQ or Dynamics graph.

These meters can be used by attached control surfaces to present an indicator in the same X/Y coordinate plane as the plug-in's curve data.

Parameters

in	<i>iCurveType</i>	One of AAX_ECurveType
out	<i>oXMeterId</i>	Id of the X-axis meter
out	<i>oYMeterId</i>	Id of the Y-axis meter

Returns

This method should return [AAX_ERROR_UNIMPLEMENTED](#) if the plug-in does not implement it.

Implemented in [AAX_CEffectParameters](#).

12.16.3.3 GetCurveDataDisplayRange()

```
virtual AAX\_Result AAX_IACFEffectParameters_V3::GetCurveDataDisplayRange (
    AAX\_CTypeID iCurveType,
    float * oXMin,
    float * oXMax,
    float * oYMin,
    float * oYMax ) const [pure virtual]
```

Determines the range of the graph shown by the plug-in.

Min/max arguments define the range of the axes of the graph.

Parameters

in	<i>iCurveType</i>	One of AAX_ECurveType
out	<i>oXMin</i>	Min value of X-axis range
out	<i>oXMax</i>	Max value of X-axis range
out	<i>oYMin</i>	Min value of Y-axis range
out	<i>oYMax</i>	Max value of Y-axis range

Returns

This method should return [AAX_ERROR_UNIMPLEMENTED](#) if the plug-in does not implement it.

Implemented in [AAX_CEffectParameters](#).

Collaboration diagram for EQ and Dynamics Curve Displays:

12.17 Hybrid Processing architecture

12.17.1

An architecture combining low-latency and high-latency audio processing.

12.17.2 Overview of Hybrid

Hybrid processing is an optional feature that allows a single plug-in to simultaneously render data on the host's low- and high-latency signal networks. In many large plug-ins this can be very useful. For example, consider a reverb algorithm with both early reflection and tail processing. With AAX Hybrid, this plug-in can process the early reflections at low latency while allowing the tail algorithm to be handled at higher latency (and thus higher efficiency.) Other kinds of algorithms that could benefit from Hybrid processing are noise reductions, analyzers, multi-effect suites, and instruments.

Because the low-latency AAX signal network may be run on DSP hardware, AAX DSP plug-ins that incorporate Hybrid processing can split audio processing between the DSP and the host. This provides the benefits of low latency, highly deterministic DSP-based processing while also allowing the plug-in to leverage the high-latency power of the Intel core where appropriate.

AAX Hybrid is an internal feature and is not exposed to users, except in terms of better plug-in performance and more efficient DSP usage.

Note

AAX Hybrid may be protected by one or more U.S. and non-U.S. patents. Details are available at www.avid.com/patents.

12.17.3 Implementing Hybrid processing

For an example of Hybrid processing, see the [DemoDelay_Hybrid](#) example plug-in

To register for Hybrid processing, a plug-in should add values for [AAX_eProperty_HybridInputStemFormat](#) and [AAX_eProperty_HybridOutputStemFormat](#) to the associated ProcessProc property map. Once these values have been registered, both the ProcessProc callback and the Hybrid render function in the plug-in's data model will be invoked during processing.

Hybrid processing context information is provided via a dedicated [Hybrid processing context structure](#). It is not possible to register additional fields on this context. However, unlike a normal algorithm ProcessProc, the Hybrid render method is implemented directly within the plug-in's effect parameters object and has direct access to the data model memory. This is possible since the render method will always run on the host, and makes it easier to implement algorithms that require access to the data model, e.g. for direct access to impulse responses, etc.

The AAX host provides dedicated audio buffers in both the ProcessProc context and the Hybrid processing context. These buffers can be used to pass audio data between the low-latency ProcessProc and the Hybrid render contexts.

- The plug-in may pass output from the low-latency ProcessProc to the Hybrid render method using additional audio buffers that are added at the end of the ProcessProc context's normal output buffer array. The ProcessProc may perform any pre-processing that is desired before passing audio to the Hybrid render context via these buffers. The [AAX_eProperty_HybridOutputStemFormat](#) property defines how many buffers will be sent from the ProcessProc to the Hybrid render method.
- Similarly, the plug-in may pass samples from the Hybrid processing callback to the low-latency ProcessProc using additional audio buffers that are added at the end of the ProcessProc context's normal input buffer array. The [AAX_eProperty_HybridInputStemFormat](#) property defines how many buffers will be sent from the Hybrid render method to the ProcessProc.

Samples which are sent from the ProcessProc to the Hybrid processing callback and back to the ProcessProc are delayed by a fixed amount relative to the normal input samples that are processed directly by the ProcessProc to its output buffers. The number of samples of delay that are added in this round-trip is available to the plug-in via [AAX_IController::GetHybridSignalLatency\(\)](#).

12.17.4 Additional information

12.17.4.1 Parameter update timing

Because updates are not passed to the [Hybrid processing context](#) using the normal AAX port infrastructure, any parameter updates from automation will be reflected in this context a little bit ahead of time (~21 ms at 44.1 kHz.) See the [Parameter update timing](#) page for a discussion of parameter timing accuracy and some suggestions of how you can maintain accurate parameter update timing.

12.17.4.2 Host support and alternatives

Not all [AAX](#) hosts support [AAX](#) Hybrid processing. See the [Host Support](#) page for additional information.

See also

[Direct data access interface](#) for another approach for transferring non-audio data between the algorithm callback and the plug-in's data model.

Classes

- struct [AAX_SHybridRenderInfo](#)
Hybrid render processing context.

Hybrid audio methods

- virtual [AAX_Result](#) [AAX_IACFEffParameters_V2::RenderAudio_Hybrid](#) ([AAX_SHybridRenderInfo](#) *ioRenderInfo)=0
Hybrid audio render function.

MIDI methods

Methods to access the plug-in's host-managed MIDI information.

- virtual [AAX_Result](#) [AAX_IController::GetHybridSignalLatency](#) (int32_t *outSamples) const =0
CALL: Returns the latency between the algorithm normal input samples and the inputs returning from the hybrid component.

12.17.5 Function Documentation

12.17.5.1 RenderAudio_Hybrid()

```
virtual AAX\_Result AAX_IACFEffParameters_V2::RenderAudio_Hybrid (
    AAX\_SHybridRenderInfo * ioRenderInfo ) [pure virtual]
```

Hybrid audio render function.

This method is called from the host to render audio for the hybrid piece of the algorithm.

Note

To use this method plug-in should register some hybrid inputs and outputs in "Describe"

Implemented in [AAX_CEffectParameters](#).

12.17.5.2 GetHybridSignalLatency()

```
virtual AAX\_Result AAX_IController::GetHybridSignalLatency (
    int32_t * outSamples ) const [pure virtual]
```

CALL: Returns the latency between the algorithm normal input samples and the inputs returning from the hybrid component.

This method provides the number of samples that the AAX host expects the plug-in to delay a signal. The host will use this value when accounting for latency across the system.

Note

This value will generally scale up with sample rate, although it's not a simple multiple due to some fixed overhead. This value will be fixed for any given sample rate regardless of other buffer size settings in the host app.

Parameters

out	<i>outSamples</i>	The number of samples of hybrid signal delay
-----	-------------------	--

Implemented in [AAX_VController](#).

Collaboration diagram for Hybrid Processing architecture:

12.18 Plug-in meters

How to manage metering data for AAX plug-ins.

12.18.1 Overview of metering in AAX

AAX provides a host-managed metering system for plug-ins. The host buffers, thins, and applies ballistics to each of the plug-in's meters. When the plug-in GUI retrieves this processed data, it receives the exact same information that is displayed on control surfaces and other metering devices.

12.18.2 Adding meters to an Effect

Meters are added to an algorithm Component in [Describe](#) using [AAX_IComponentDescriptor::AddMeters\(\)](#). The resulting meter context field will be populated with an array of meter "tap" values, one for each of the Component's meters.

12.18.2.1 Customizing meter behavior

Using the [Effect Descriptor](#), each meter in the Effect may optionally be associated with a [property map](#) that applies a particular set of display properties to the meter. These are the properties that may be set on a meter:

- [AAX_EMeterOrientation](#)
- [AAX_EMeterBallisticType](#)
- [AAX_EMeterType](#)

Note that, because meter properties are added at the Effect level, it is not possible to describe different meter property configurations for different algorithms in the same Effect.

12.18.3 Reporting meter values

Meter values are reported by the algorithm using one "tap" per channel per buffer. For each tap, the algorithm must report the maximum metered sample value for each processing buffer.

Meter tap values can be interpreted as the maximum value of the meter per buffer, on a scale of [0.0 1.0]. In all cases the plug-in's meter position should be normalized between 0 and 1, where 0 is no gain reduction. For example:

- An input meter should report the maximum absolute sample value that is present in the input audio buffer for the appropriate channel
- An output meter should report the maximum absolute sample value that is present in the output audio buffer for the appropriate channel
- A gain-reduction meter (CL or EG types) should report the largest amount of gain reduction in the current buffer for the appropriate channel. If no gain reduction occurred for a buffer then a value of 0.0 should be reported. If a full-scale signal was reduced to silence then a value of 1.0 should be reported.

Gain-reduction meter values should report peak gain reduction, not RMS or other algorithms, and may use any normalization mapping (e.g. linear, exponential) which is desired. Ideally the gain-reduction metering UI in the host and on attached control surfaces will match the Peak gain reduction metering in the plug-in's GUI.

Legacy Porting Notes The gain-reduction meter handling for AAX plug-ins is different from that for RTAS/TDM plug-ins. AAX plug-ins must invert their gain-reduction meter values manually before reporting these values from the audio processing callback. The AAX host will always thin reported meter data using a "max" operation, and will later invert gain-reduction meter values before they are available to the plug-in GUI or to control surfaces.

12.18.4 Displaying meter values

The meter values that are reported to the system from the algorithm are available, in buffered and (optionally) ballistics-smoothed form, from [AAX_IController](#). The meter values returned from methods such as [GetCurrentMeterValue\(\)](#) and [GetMeterPeakValue\(\)](#) are the same values used by the system when displaying plug-in meters on control surfaces, and when a plug-in clears the peak value using [ClearMeterPeakValue\(\)](#) this change will likewise be reflected throughout the system.

The literal values provided by these methods can be interpreted as the distance from "rest" that the meter must travel to represent the current value, again on a scale of [0.0 1.0]. Note that this is not necessarily equivalent to the semantics of the meter's reported values in the algorithm:

- For "standard" meters such as input meters, this corresponds to the value provided by the algorithm, since a maximum metered sample value (1.0) corresponds to a meter that should be drawn "furthest from rest" (1.0), i.e. at the top of a standard bottom-to-top meter graphic, or at the far right of a standard left-to-right graphic.
- For "inverted" meters, such as gain-reduction meters, these semantics are reversed: a maximum metered sample value (1.0) corresponds to a meter drawn "at rest" (0.0), i.e. at the bottom of a bottom-to-top meter graphic or at the far left of a left-to-right graphic.

These values are independent of [meter orientation](#): an input or output meter that is oriented with [AAX_eMeterOrientation_TopRight](#) will still use 0.0 as its "at rest" position, and likewise a gain-reduction meter that is oriented with [AAX_eMeterOrientation_BottomLeft](#) will still use 1.0.

12.18.5 Alternatives

For advanced metering applications a single tap value may not be sufficient. To transmit more detailed information from the algorithm to its other components, a plug-in must use the [Direct Data](#) interface. Collaboration diagram for Plug-in meters:

12.19 MIDI

How to route and process MIDI in AAX plug-ins.

12.19.1 MIDI Overview

AAX plugins create MIDI nodes. MIDI nodes define points where the plugin can send or receive MIDI data. The data at these nodes is represented as a sequence of AAX MIDI packet data structures. The AAX host manages connections between these nodes and other points in the system. The host also performs conversion between raw MIDI data and AAX MIDI packets.

12.19.2 MIDI node types

There are four kinds of nodes an AAX plug-in can create. See [AAX_eMIDINodeType](#) for additional details about these node types:

- [AAX_eMIDINodeType_LocalInput](#)
- [AAX_eMIDINodeType_LocalOutput](#)
- [AAX_eMIDINodeType_Global](#)
- [AAX_eMIDINodeType_Transport](#)

12.19.3 Adding MIDI functionality to a plug-in

Plug-in may access MIDI data in its algorithm or data model. If plug-in needs MIDI in both places or just in the algorithm, it should add a MIDI node to the algorithm context, i.e. call [AAX_IComponentDescriptor::AddMIDINode\(\)](#) with the appropriate node type.

```
//=====
// Algorithm context definitions
//=====
// Context structure
struct SMy_Algorithm_Context
{
    [...]
    AAX_eMIDINode * mMIDIInNodeP;           // Local input MIDI node pointer
    AAX_eMIDINode * mMIDIOutNodeP;         // Local output MIDI node pointer
    AAX_eMIDINode * mMIDITransportP;       // Transport node
    [...]
};
enum EDemoMIDI_Algorithm_PortID
{
    [...]
    //
    // Add the MIDI node as a physical address within the context field
    ,eAlgPortID_MIDIInNodeP           = AAX_FIELD_INDEX (SDemoMIDI_Algorithm_Context, mMIDIInNodeP)
    ,eAlgPortID_MIDIOutNodeP         = AAX_FIELD_INDEX (SDemoMIDI_Algorithm_Context, mMIDIOutNodeP)
    ,eAlgPortID_MIDITransportP       = AAX_FIELD_INDEX (SDemoMIDI_Algorithm_Context, mMIDITransportP)
    [...]
};
```

```
// *****
// ROUTINE: DescribeAlgorithmComponent
// Algorithm component description
// *****
static void DescribeAlgorithmComponent( AAX_IComponentDescriptor * outDesc )
{
    AAX_Result err;

    [...]
    // Register MIDI nodes
    err = outDesc->AddMIDINode(eAlgPortID_MIDINodeA, AAX_eMIDINodeType_LocalInput, "DemoMIDI", 0xffff);
    AAX_ASSERT (err == 0);
    err = outDesc->AddMIDINode(eAlgPortID_MIDINodeOut, AAX_eMIDINodeType_LocalOutput, "DemoMIDIOut",
    0xffff); AAX_ASSERT (err == 0);
    err = outDesc->AddMIDINode(eAlgPortID_MIDINodeTransport, AAX_eMIDINodeType_Transport, "DemoMIDITrnsprt",
    0xffff); AAX_ASSERT (err == 0);
    [...]
}
```

If MIDI data is needed in the plug-in's data model only, plug-in should describe MIDI node with [AAX_IEffectDescriptor::AddControlMIDI](#)

```
// *****
// ROUTINE: GetPlugInDescription
// *****
static AAX_Result GetPlugInDescription( AAX_IEffectDescriptor * outDescriptor )
{
    AAX_Result err;

    [...]
    // Register MIDI nodes
    err = outDesc->AddControlMIDINode('linp', AAX_eMIDINodeType_LocalInput, "DemoMIDI", 0xffff);
    AAX_ASSERT (err == 0);
    err = outDesc->AddControlMIDINode('lout', AAX_eMIDINodeType_LocalOutput, "DemoMIDIOut", 0xffff);
    AAX_ASSERT (err == 0);
    err = outDesc->AddControlMIDINode('tran', AAX_eMIDINodeType_Transport, "DemoMIDITrnsprt", 0xffff);
    AAX_ASSERT (err == 0);
    [...]
    return err;
}
```

Note

These two types of MIDI nodes can't be used together in the same plug-in's effect.

12.19.4 Using MIDI in a plug-in algorithm

Like with other algorithm context ports, data in MIDI nodes is directly available in the plug-in's algorithm process function. Here is an example from the DemoMIDI_NoteOn sample plug-in:

```
template<int kNumChannelsIn, int kNumChannelsOut>
void
AAX_CALLBACK
DemoMIDI_AlgorithmProcessFunction (
    SDemoMIDI_Alg_Context * const inInstancesBegin [],
    const void * inInstancesEnd)
{
    [...]
    // Setup MIDI In node pointers
    AAX_IMIDINode* midiNodeIn = instance->mMIDINodeP;
    AAX_CMidiStream* midiBufferIn = midiNodeIn->GetNodeBuffer();
    AAX_CMidiPacket* midiBufferInPtr = midiBufferIn->mBuffer;
    uint32_t packets_count_in = midiBufferIn->mBufferSize;

    // Setup MIDI Out node pointers
    AAX_IMIDINode* midiNodeOut = instance->mMIDINodeOutP;
    AAX_CMidiStream* midiBufferOut = midiNodeOut->GetNodeBuffer();
    AAX_CMidiPacket* midiBufferOutPtr = midiBufferOut->mBuffer;
    uint32_t packets_count_out = midiBufferOut->mBufferSize;

    // Setup MIDI Transport node pointers
    // NOTE: See warning at AAX_IMIDINode::GetTransport() regarding use of this interface
    AAX_IMIDINode* midiTransport = instance->mMIDINodeTransportP;
    AAX_ITransport * transport = midiTransport->GetTransport();
    bool transport_is_playing = false;
    if (transport) {
        transport->IsTransportPlaying(&transport_is_playing);
    }

    if(transport_is_playing) {
        //
        // While there are packets in the node
    }
}
```



```

while (packets_count_in > 0) {
    midiBufferOutPtr = midiBufferInPtr;           // Copy the packet from the input MIDI node
                                                // to the output MIDI node
    midiBufferOutPtr->mTimestamp = timeStamp;      // Set the MIDI time stamp
    midiNodeOut->PostMIDIPacket(midiBufferOutPtr); // Post the MIDI packet
    midiBufferOut->mBufferSize = packets_count_in;

    midiBufferInPtr++;
    packets_count_in--;
}
}
[...]
```

Also data from the MIDI nodes that were described with [AAX_IComponentDescriptor::AddMIDINode\(\)](#) can be accessed via [AAX_CEffectParameters::UpdateMIDINodes\(\)](#) method. This method provides an [AAX_CMidiPacket](#). Because the MIDI packet structure does not identify the associated MIDI stream's type (input, output, global, or transport) this method also provides an index into the plug-in's algorithm context structure which can be used to identify the semantics of the MIDI packet.

12.19.5 Accessing MIDI in the plug-in data model

A plug-in may access MIDI data in its data model via the [AAX_CEffectParameters::UpdateMIDINodes\(\)](#) or [AAX_CEffectParameters::UpdateControlMIDINodes\(\)](#) methods. Both of these methods provide an [AAX_CMidiPacket](#). Because the MIDI packet structure does not identify the associated MIDI stream's type (input, output, global, or transport) [UpdateMIDINodes](#) method also provides an index into the plug-in's algorithm context structure which can be used to identify the semantics of the MIDI packet, while [UpdateControlMIDINodes](#) provides MIDI node ID for the same reason.

```

AAX_Result DemoMIDI_Parameters::UpdateMIDINodes ( AAX_CFieldIndex inFieldIndex, AAX_CMidiPacket& inPacket
)
{
    if (eAlgPortID_MIDINodeIn == inFieldIndex)
    {
        if ( (inPacket.mData[0] & 0xF0) == 0x90 )
        {
            if ( inPacket.mData[2] == 0x00 )
            {
                // Note Off
            }
            else
            {
                // Note On
            }
        }
    }

    return AAX_SUCCESS;
}
```

Note

Only one of the [UpdateMIDINodes](#) and [UpdateControlMIDINodes](#) can be used in the single plug-in's effect at a time. If plug-in uses MIDI nodes described with [AddMIDINode](#) function, then only [UpdateMIDINodes](#) method can be used to receive MIDI messages. Otherwise [UpdateControlMIDINodes](#) should be used.

12.19.6 Support functions

Additional definitions and support functions for MIDI in AAX are available in [AAX_MIDIUtilities.h](#) Collaboration diagram for MIDI:

12.20 Offline processing interface

12.20.1

Advanced offline processing features.

This interface represents an optional component that you can add to your plug-in in order to support extended features of the AAX SDK.

The HostProcessor interface provides offline plug-ins with useful offline processing features such as random-access facilities and a non-processing analysis callback. For documentation, see the following classes:

- [Host processor module](#)
- [Host processor delegate](#)

To add this interface to your plug-in at describe time, register a [ProcPtr](#) using the [kAAX_ProcPtrID_Create_HostProcessor](#) selector.

Note

If your plug-in does not require the specific offline processing features provided by this interface then it should not register a host processor. Instead, register an offline version of the plug-in's real-time algorithm using the [AAX_eProperty_PluginID_AudioSuite](#) property.

Classes

- class [AAX_CHostProcessor](#)
Concrete implementation of the [AAX_IHostProcessor](#) interface for non-real-time processing.
- class [AAX_IACFHostProcessor](#)
Versioned interface for an AAX host processing component.
- class [AAX_IHostProcessor](#)
Base class for the host processor interface.
- class [AAX_IHostProcessorDelegate](#)
Versioned interface for host methods specific to offline processing.
- class [AAX_VHostProcessorDelegate](#)
Version-managed concrete [Host Processor delegate](#) class.

Collaboration diagram for Offline processing interface:

12.21 Properties File

Properties for an AAX plugin bundle.

About An XML properties file can optionally be used in an AAX plugin bundle to provide information to the host prior to loading the plugin binary.

The file should be named AAXProperties.xml and should be placed in the bundle's Resources folder.

Collection Properties AuthDialog (Optional)

- Required attribute: `type_="bool"`
- Value `true`: Declares that the plugin may show an authorization dialog when loaded.
- Value `false`: Declares that the plugin will never show an authorization dialog when loaded.

Schema

```
<?xml version="1.0"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
<xs:element name="AAXPropertiesFile">
  <xs:complexType>
    <xs:element name="Collection">
      <xs:complexType>
        <xs:element name="AuthDialog" minOccurs="0">
          <xs:complexType>
            <xs:simpleContent>
              <xs:extension base="xs:boolean">
                <xs:attribute name="type_" type="xs:string" minOccurs="1" maxOccurs="1"
fixed="bool"/>
              </xs:extension>
            </xs:simpleContent>
          </xs:complexType>
        </xs:element>
      </xs:complexType>
    </xs:element>
  </xs:complexType>
</xs:element>
</xs:schema>
```

Example

```
<?xml version='1.0' encoding='UTF-8' standalone='yes'?>
<AAXPropertiesFile>
  <Collection>
    <AuthDialog type_="bool">false</AuthDialog>
  </Collection>
</AAXPropertiesFile>
```

Collaboration diagram for Properties File:

12.22 Sidechain Inputs

Routing custom audio streams to a plug-in.

12.22.1 Overview of Sidechain Inputs

If applicable, plug-ins may choose to enable sidechain inputs. If a sidechain is enabled, a menu is added to the plug-in's header that allows the user to choose an interface or bus as the sidechain, or "key input". Once enabled, the plug-in will be able to access sidechain input just like any other input signal. Currently, DAE is limited to mono sidechain inputs.

12.22.2 Adding a Sidechain Input to an Effect

Setting up a sidechain input is fairly straight forward. You will want to add a physical address within your context structure, and then "describe" the sidechain in Describe.

Context Structure:

```
//=====
// Component context definitions
//=====
// Context structure
struct SMyPlugIn_Alg_Context
{
  [...]
```

```

    int32_t * mSideChainP;
    [...]
};
// Physical addresses within the context
enum EDemoDistAlgPortID
{
    [...]
    ,MyPlugInAlgFieldID_SideChain = AAX_FIELD_INDEX (SDemoDistAlgContext, mSideChainP)
    [...]
};

```

Describe:

```

// *****
// ROUTINE: DescribeAlgorithmComponent
// Algorithm component description
// *****
static void DescribeAlgorithmComponent( AAX_IComponentDescriptor * outDesc )
{
    AAX_Result err = AAX_SUCCESS;
    [...]
    err = outDesc.AddSideChainIn(eDemoDistAlgFieldID_SideChain);
    [...]
    properties->AddProperty ( AAX_eProperty_SupportsSideChainInput, true );
    [...]
}

```

Todo Is properties->AddProperty (AAX_eProperty_SupportsSideChainInput, true) even necessary?!? I believe I saw a p.i. that does not declare this...

In order to tell whether there is sidechain information available to your plug-in, check for a null pointer within your algorithm's process function. The sidechain channel will show up as an additional stem from the original stem format you declare. That is to say, for a stereo plug-in, the sidechain channel will be the third channel passed in.

```

//=====
// Processing function definition
//=====
void
AAX_CALLBACK
MyPlugIn_AlgorithmProcessFunction (
    SMyPlugInAlgContext * const inInstancesBegin [],
    const void * inInstancesEnd)
{
    [...]
    int32_t sideChainChannel = *instance->mSideChainP;
    float * AAX_RESTRICT sideChainInput = 0;
    if ( sideChainChannel )
        sideChainInput = instance->mInputPP [sideChain]Channel;
    [...]
}

```

Collaboration diagram for Sidechain Inputs:

12.23 Task agent interface

12.23.1

A mechanism for hosts to request that plug-ins perform tasks.

This interface represents an optional component that you can add to your plug-in in order to support extended features of the AAX SDK.

Host Compatibility Notes This interface is not yet used in any AAX hosts

The plug-in implements an [AAX_ITaskAgent](#), which is used by the host to add or cancel tasks.

The host implements [AAX_IACFTask](#) for task objects representing each task that it wants the plug-in to perform.

To request a task, the host adds a task object to the plug-in's task agent interface.

The type of each task is identified with a four-char ID. If additional arbitrary data is necessary to describe the task it is provided via [data buffers](#).

The plug-in checks this data to understand what work needs to be done, then performs the task. The task may be, and usually is, executed asynchronously. The plug-in optionally updates the task progress as it proceeds, then calls [AAX_ITask::SetDone\(\)](#) when the work is completed. If the task involves returning data back to the host, the plug-in first calls [AAX_ITask::AddResult\(\)](#) one or more times to provide the data via data buffers.

To be available as a task agent, the plug-in's task agent implementation must be registered with the host in the plug-in's Description callback like other process pointers. To add this interface to your plug-in at describe time, call [AAX_IEffectDescriptor::AddProcPtr\(\)](#) using the [kAAX_ProcPtrID_Create_TaskAgent](#) selector.

12.23.2 Communicating with other modules

Like other modules, the task agent interface is provided with a reference to the plug-in's [AAX_IEffectDirectData](#) object at initialization. In order to transfer data between a plug-in's [AAX_IEffectDirectData](#) object and its other objects, dedicated custom data methods in those objects' interfaces should be used. For example, to communicate with the plug-in's data model, use [AAX_IEffectParameters::GetCustomData\(\)](#) and [AAX_IEffectParameters::SetCustomData\(\)](#).

Classes

- class [AAX_CTaskAgent](#)
Default implementation of the [AAX_ITaskAgent](#) interface.
- class [AAX_IACFTask](#)
Versioned interface for an asynchronous task.
- class [AAX_IACFTaskAgent](#)
Versioned interface for a component that accepts task requests.
- class [AAX_ITask](#)
Interface representing a request to perform a task.
- class [AAX_ITaskAgent](#)
Interface for a component that accepts task requests.

Enumerations

- enum class [AAX_TaskCompletionStatus](#) : int32_t {
[None](#) = 0 ,
[Done](#) = 1 ,
[Canceled](#) = 2 ,
[Error](#) = 3 }

12.23.3 Enumeration Type Documentation

12.23.3.1 AAX_TaskCompletionStatus

```
enum AAX\_TaskCompletionStatus : int32_t [strong]
```

Completion status for use with [AAX_ITask::SetDone\(\)](#)

Enumerator

None	
Done	
Canceled	
Error	

Collaboration diagram for Task agent interface:

12.24 AAX Library features

12.24.1

AAX Library core support for the AAX interface

The AAX Library includes several built-in features that are designed to facilitate plug-in development and to make it easy to create plug-ins with correct and consistent behavior. Although these features are not a part of the AAX API, they are a core part of the SDK.

Documents

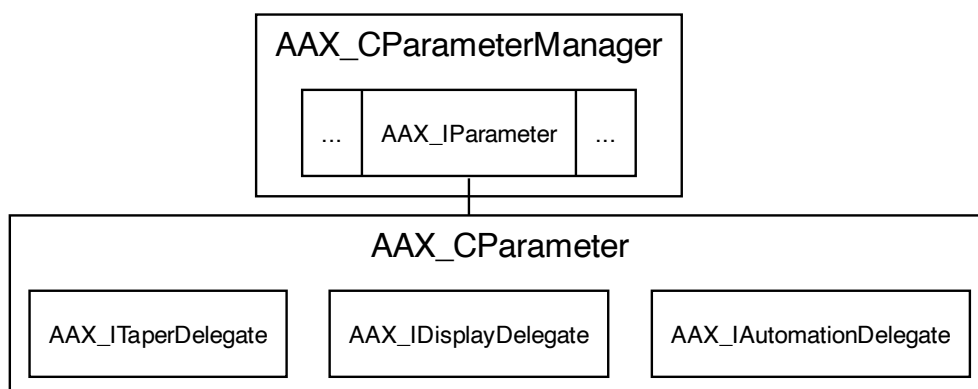
- [Parameter Manager](#)
Optional (but recommended) system for managing AAX plug-in parameters.
- [Monolithic parameters](#)

Collaboration diagram for AAX Library features:

12.25 Parameter Manager

12.25.1

Optional (but recommended) system for managing AAX plug-in parameters.



The Parameter Manager is a generic container for a plug-in's parameters, which constitute the complete externally-facing state of a plug-in's data model. Additional internal state data may be stored via settings chunks. The Parameter Manager is owned and operated by the plug-in's [Data model interface](#).

The Parameter Manager provides a convenient and consistent interface by which a plug-in's data model implementation may access its parameters. Other plug-in components that require access to the data model may also use this interface, or a proxy of it, to view the current state of the plug-in.

In the Parameter Manager, implementation-specific parameter behaviors such as taper and display formatting are modular and are applied through delegation. Because of this model, it is possible to easily create a wide variety of behavior combinations without additional subclassing; any display behavior may be combined with any taper behavior, and a newly written behavior can be quickly "mixed in" to many parameters.

12.25.2 Parameter concepts

- [Parameter value domains](#)
- [Taper](#)
- [Delegates](#)
- [Model-View-Controller](#)

12.25.2.1 Parameter value domains

In AAX, parameter values can be represented in one of two "domains". Developers work with parameters in the *real* domain, while the host handles parameters in a scaled, *normalized* format.

Real (or "logical") domain

AAX plug-ins and parameter controllers work with typed parameter values that represent the *real* (logical) state of the parameter. The type, form, and meaning of this value is dependent on the parameter's implementation and is unknown to the host.

Normalized domain

The AAX host works with parameter values that have been scaled (*normalized*) to a type-agnostic format. Although normalized values make little logical sense, they provide the host with a consistent means of handling, storing, and communicating parameters' values without having to worry about the actual implementation or meaning of the parameters. Normalized parameter values are 64-bit floating point and are scaled to a range of [0, 1].

For more information about conversion between parameter domains, see [AAX_IParameter](#).

Note

The [AAX_IEffectParameters](#) interface currently utilizes a secondary normalization to full-scale int32_t values. In the future, this will be unified with the double precision floating point normalization documented above.

12.25.2.2 Taper

A *taper* is the conversion function that translates a parameter's value between its real and normalized forms.

For example, a taper could be created that converts between a normalized value ([0, 1]) and a real frequency value ranging from [20 2000]. The conversion between these two ranges could be linear or logarithmic, or could use any other desired mapping. This mapping, as well as the specific range of the possible logical values, is defined by the taper.

For more information about tapers in AAX, see [AAX_ITaperDelegate](#).

12.25.2.3 Delegates

In AAX, individual parameters achieve their own unique behavior by being associated with behavioral delegates.

For example, when [AAX_CParameter::SetNormalizedValue\(\)](#) is called on a particular parameter through its [AAX_IParameter](#) interface, the [AAX_CParameter](#) calls into a [AAX_ITaperDelegate](#) that it owns in order to convert the normalized value to its real equivalent. This real value is then set as the parameter's new state.

For more information about how delegates are used to create a parameter's behavior see [AAX_CParameter](#)

12.25.2.4 Model-View-Controller

AAX adheres roughly to a Model-View-Controller pattern. The Parameter Manager functions within the context of [AAX_IEffectParameters](#), which in turn acts as an AAX plug-in's Data Model in an MVC sense. Views, such as the plug-in's GUI, attached control surfaces, or the automation facilities in the AAX host, are given access to the Data Model via a central Controller, which is represented by the [AAX_IController](#) interface.

For more information about how MVC applies to AAX, see the [Data model interface](#) documentation page.

Classes

- class [AAX_CParameter< T >](#)
Generic implementation of an [AAX_IParameter](#).
- class [AAX_CParameterManager](#)
A container object for plug-in parameters.
- class [AAX_IParameter](#)
The base interface for all normalizable plug-in parameters.

Documents

- [Taper delegates](#)
Classes for conversion to and from normalized parameter values.
- [Display delegates](#)
Classes for parameter value string conversion.

Collaboration diagram for Parameter Manager:

12.26 Taper delegates

12.26.1

Classes for conversion to and from normalized parameter values.

Taper delegates are used to convert real parameter values to and from their normalized representations. All taper delegates implement the [AAX_ITaperDelegate<T>](#) interface template, which contains two conversion functions:

```
virtual T      NormalizedToReal(double normalizedValue) const = 0;
virtual double RealToNormalized(T realValue) const = 0;
```

In addition, tapers may incorporate logical value constraints via the following interface methods:

```
virtual T      GetMaximumValue() const = 0;
virtual T      GetMinimumValue() const = 0;
virtual T      ConstrainRealValue(T value) const = 0;
```

For more information, see the [AAX_ITaperDelegate](#) class documentation.

Classes

- class [AAX_CBinaryTaperDelegate< T >](#)
A binary taper conforming to [AAX_ITaperDelegate](#).
- class [AAX_CLinearTaperDelegate< T, RealPrecision >](#)
A linear taper conforming to [AAX_ITaperDelegate](#).
- class [AAX_CLogTaperDelegate< T, RealPrecision >](#)
A logarithmic taper conforming to [AAX_ITaperDelegate](#).
- class [AAX_CPieceWiseLinearTaperDelegate< T, RealPrecision >](#)
A piece-wise linear taper conforming to [AAX_ITaperDelegate](#).
- class [AAX_CRangeTaperDelegate< T, RealPrecision >](#)
A piecewise-linear taper conforming to [AAX_ITaperDelegate](#).
- class [AAX_CStateTaperDelegate< T >](#)
A linear taper conforming to [AAX_ITaperDelegate](#).
- class [AAX_ITaperDelegateBase](#)
Defines the taper conversion behavior for a parameter.
- class [AAX_ITaperDelegate< T >](#)
Classes for conversion to and from normalized parameter values.

Collaboration diagram for Taper delegates:

12.27 Display delegates

12.27.1

Classes for parameter value string conversion.

Display delegates are used to convert real parameter values to and from their formatted string representations. All display delegates implement the [AAX_IDisplayDelegate](#) interface, which contains two conversion functions:

```
virtual bool   ValueToString(T value, std::string& valueString) const = 0;
virtual bool   StringToValue(const std::string& valueString, T& value) const = 0;
```

12.27.2 Display delegate decorators

The AAX SDK utilizes a decorator pattern in order to provide code re-use while accounting for a wide variety of possible parameter display formats. The SDK includes a number of sample display delegate decorator classes.

Each concrete display delegate decorator implements [AAX_IDisplayDelegateDecorator](#) and adheres to the decorator pattern. The decorator pattern allows multiple display behaviors to be composited or wrapped together at run time. For instance it is possible to implement a dBV (dB Volts) decorator, by wrapping an [AAX_CDecibelDisplayDelegateDecorator](#) with an [AAX_CUnitDisplayDelegateDecorator](#).

12.27.2.1 Display delegate decorator implementation

By implementing [AAX_IDisplayDelegateDecorator](#), each concrete display delegate decorator class implements the full [AAX_IDisplayDelegate](#) interface. In addition, it retains a pointer to the [AAX_IDisplayDelegateDecorator](#) that it wraps. When the decorator performs a conversion, it calls into its wrapped class so that the wrapped decorator may apply its own conversion formatting. By repeating this pattern in each decorator, all of the decorator subclasses call into their "wrapper" in turn, resulting in a final string to which all of the decorators' conversions have been applied in sequence.

Here is the relevant implementation from [AAX_IDisplayDelegateDecorator](#) :

```
template <typename T>
AAX_IDisplayDelegateDecorator<T>::AAX_IDisplayDelegateDecorator(const AAX_IDisplayDelegate<T>&
    displayDelegate) :
    AAX_IDisplayDelegate<T>(),
    mWrappedDisplayDelegate(displayDelegate.Clone())
{
}
template <typename T>
bool AAX_IDisplayDelegateDecorator<T>::ValueToString(T value, AAX_CString* valueString) const
{
    return mWrappedDisplayDelegate->ValueToString(value, valueString);
}
template <typename T>
bool AAX_IDisplayDelegateDecorator<T>::StringToValue(const AAX_CString& valueString, T* value) const
{
    return mWrappedDisplayDelegate->StringToValue(valueString, value);
}
```

12.27.2.2 Decibel decorator example

Here is a concrete example of how a decibel decorator might be implemented

```
template <typename T>
bool AAX_CDecibelDisplayDelegateDecorator<T>::ValueToString(T value, AAX_CString* valueString) const
{
    if (value <= 0)
    {
        *valueString = AAX_CString("--- dB");
        return true;
    }

    value = 20*log10(value);
    bool succeeded = AAX_IDisplayDelegateDecorator<T>::ValueToString(value, valueString);
    *valueString += AAX_CString("dB");
    return succeeded;
}
```

Notice in this example that the [ValueToString\(\)](#) method is called in the parent class, [AAX_IDisplayDelegateDecorator](#). This results in a call into the wrapped class' implementation of [ValueToString\(\)](#), which converts the decorated value to a redecorated string, and so forth for additional decorators.

Classes

- class [AAX_CBinaryDisplayDelegate< T >](#)
A binary display format conforming to [AAX_IDisplayDelegate](#).
- class [AAX_CNumberDisplayDelegate< T, Precision, SpaceAfter >](#)
A numeric display format conforming to [AAX_IDisplayDelegate](#).
- class [AAX_CStateDisplayDelegate< T >](#)
A generic display format conforming to [AAX_IDisplayDelegate](#).
- class [AAX_CStringDisplayDelegate< T >](#)
A string, or list, display format conforming to [AAX_IDisplayDelegate](#).
- class [AAX_IDisplayDelegateBase](#)
Defines the display behavior for a parameter.
- class [AAX_IDisplayDelegate< T >](#)
Classes for parameter value string conversion.

Documents

- [Display delegate decorators](#)
Classes for adapting parameter value strings.

Collaboration diagram for Display delegates:

12.28 Display delegate decorators

12.28.1

Classes for adapting parameter value strings.

The AAX parameter display strategy uses a decorator pattern for parameter value formatting. This approach allows developers to maximize code re-use across display delegates with many different kinds of varying formatting, all without creating interdependencies between the different display delegates themselves.

For more information, see [Display delegate decorators](#). For even more information, about the Decorator design pattern, please consult the GOF design patterns book.

Classes

- class [AAX_CDecibelDisplayDelegateDecorator< T >](#)
A percent decorator conforming to [AAX_IDisplayDelegateDecorator](#).
- class [AAX_CPercentDisplayDelegateDecorator< T >](#)
A percent decorator conforming to [AAX_IDisplayDelegateDecorator](#).
- class [AAX_CUnitDisplayDelegateDecorator< T >](#)
A unit type decorator conforming to [AAX_IDisplayDelegateDecorator](#).
- class [AAX_CUnitPrefixDisplayDelegateDecorator< T >](#)
A unit prefix decorator conforming to [AAX_IDisplayDelegateDecorator](#).
- class [AAX_IDisplayDelegateDecorator< T >](#)
The base class for all concrete display delegate decorators.

Collaboration diagram for Display delegate decorators:

12.29 Monolithic parameters

12.29.1

Classes

- class [AAX_CMonolithicParameters](#)
Extension of the [AAX_CEffectParameters](#) class for monolithic VIs and effects.

Collaboration diagram for Monolithic parameters:

12.30 Additional Topics

12.30.1

Additional information about the AAX design.

Documents

- [Real-time performance](#)
Guidelines for avoiding audio streaming errors.
- [Parameter automation](#)
Information about parameter automation.
- [Parameter updates](#)
The anatomy of a parameter update.
- [Plug-in type conversion](#)
Specification for valid conversions between plug-in types.
- [The Avid Component Framework \(ACF\)](#)
How the AAX C++ interfaces work.

Collaboration diagram for Additional Topics:

12.31 Real-time performance

Guidelines for avoiding audio streaming errors.

This page provides an overview of best practices for avoiding streaming errors and achieving good performance for audio processing on real-time threads.

These recommendations are based on observations we have made when reviewing common Pro Tools streaming errors, especially those caused by plug-ins, as well as on information we have gathered from a number of partners and other experts in the field.

See also

[Plug-In Causes Audio Streaming Errors](#)

12.31.1 Things NOT To Do In An Audio Plug-In Render Callback

- No unbounded calls/loops
- No access to paged memory or files
- No system calls
- No memory allocations or deallocations
- No exceptions
- No locks (priority inversions)
- No data races
- Avoid context switches
- No Objective-C or Swift code. These can incur system calls and take locks - see this article for more information.
- Do not use the JUCE callAsync() function - it is not real-time safe. As an alternative, you can use a separate dedicated thread that wakes on a timer or the [AAX TimerWakeup\(\)](#) method to handle non-real time work.
- Never perform PACE license checks in audio processing thread; always add code annotations to prevent license checks in any code which will be executed from the audio processing callback.

12.31.2 Things To Do In An Audio Plug-In Render Callback

- If passing data, always use lock-free FIFOs
 - When making data from other parts of the plug-in available to its real-time callback you should always use the [AAX](#) packet system; this will ensure thread safety, proper timing of the data delivery with respect to the audio being processed, and optimal real-time thread performance.
 - There is a nice reference implementation for a general-purpose FIFO in the [farbot](#) project
 - Lock-free FIFOs are also good for passing data from the real-time thread to a low-priority thread in order to do heavy lifting like writes to disk
- If sharing small amounts of data (≤ 8 bytes), use atomics
 - Make a local copy/cache of any atomic values that need to be read multiple times from your render function
 - When using atomics, always make the compiler prove that its implementation is lock-free e.g. using a static_assert that `std::atomic<T>::is_lock_free`
- When sharing larger data, if it is acceptable if the data sometimes cannot be accessed, use a try_lock() in the real-time thread and a lock() in any non-real-time threads.
 - When using this strategy you should use a spin lock, not a std::mutex; std::mutex::unlock() can block in a system call to wake the waiting thread
- When sharing larger data, if it is acceptable to access a stale copy of the data, then use a compare-and-exchange loop
 - Be careful about memory leaks with this strategy
 - See the NonRealtimeMutable template in the [farbot](#) project

12.31.3 Good Resources And Examples

- [Real-time audio programming 101: time waits for nothing](#) by Ross Bencina
- [Four common mistakes in audio development](#) by Michael Tyson
- [farbot: FAbian's Realtime Box o' Tricks](#) project on GitHub

Collaboration diagram for Real-time performance:

12.32 Parameter automation

Information about parameter automation.

12.32.1 On this page

12.32.2 Overview

The term "automation" can mean two things in AAX:

1. A host feature allowing users to record and play back plug-in parameter changes. In this documentation, this data is referred to as **automation data**, and it is stored in **automation lanes** in the host.
2. A system for arbitrating between changes from different parameter editors such as the plug-in GUI, control surfaces, and pre-recorded automation values. In this documentation, this is referred to as the **event system** for parameters.

Here are some examples of how these two different meanings are used in AAX:

- The [AAX_IAutomationDelegate](#) provides methods for interacting with the host's parameter event system.
- [AAX_IACFEffectParameters::GetParameterIsAutomatable\(\)](#) and the `automatable` parameter in the [AAX_CParameter](#) constructor reflect whether a parameter can have automation written and read by the host.
- [AAX_IController::GetCurrentAutomationTimestamp\(\)](#) gets the timestamp for pre-recorded automation data when it is received by the plug-in during playback

For more information about the parameter event system, see the [Parameter updates](#) pages, and particularly the information on the [Token protocol](#)

12.32.3 Plug-in elements used for automation

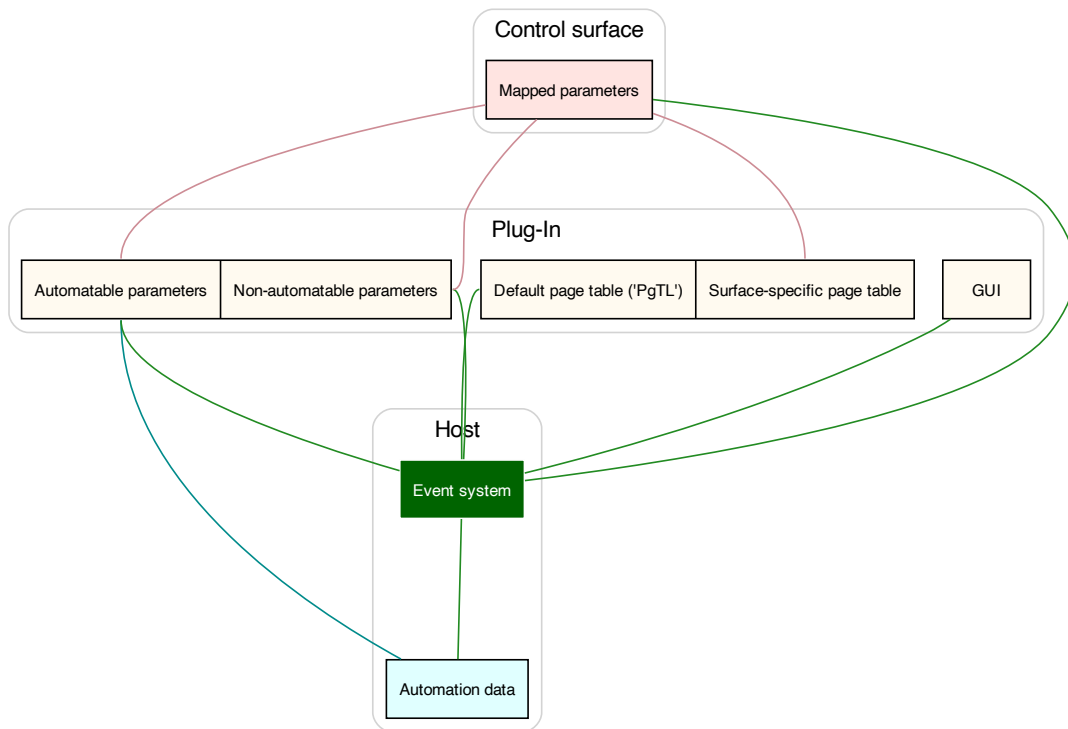


Figure 12.4 Plug-in elements used for events and automation

12.32.3.1 Defining automatable parameters

In order for a parameter to be available for automation recording, editing, and playback, the plug-in must meet the following criteria:

- It must provide `true` when the host calls `GetParameterIsAutomatable()` for the parameter. In nearly all plug-ins, this means providing `true` to the `automatable` parameter in the parameter's `AAX_CParameter` constructor.
- It must expose the parameter to the parameter event system (see below.)

In order for a parameter to be exposed to the event system, the plug-in must meet the following criteria:

- It must respond to all parameter methods in the `AAX_IEffectParameters` interface, particularly `GetNumberOfParameters()` and `GetParameterIDFromIndex()`. Generally this is accomplished by adding an `AAX_CParameter` object for each parameter to the plug-in's `Parameter Manager`.
- It must include the parameter in its one-parameter-per-page 'PgTL' (default) page tables. See [Implementing Page Tables](#) in the [Page Table Guide](#) for more information about defining this page table type.

All plug-in parameters must be registered with the host's event system in order for editors, including the plug-in's GUI, to work properly. Therefore a plug-in should always define a complete 'PgTL' (default) page table including all of its parameters, even the parameters that are not "automatable".

12.32.4 Advanced automation topics

- [Linked parameters](#)

Collaboration diagram for Parameter automation:

12.33 Parameter updates

12.33.1

The anatomy of a parameter update.

Documents

- [Parameter update timing](#)
Details about parameter timing and how to keep parameter updates in sync.
- [Token protocol](#)
Communicating parameter state with the host.
- [Basic parameter update sequences](#)
Sequence diagrams for some common parameter update scenarios.
- [Linked parameters](#)
How to link parameters.
- [Linked parameter update sequences](#)
Sequence diagrams for some common linked parameter update scenarios.

Collaboration diagram for Parameter updates:

12.34 Parameter update timing

Details about parameter timing and how to keep parameter updates in sync.

12.34.1 On this page

- [Timeline Locations](#)
- [Coordinating the data model and algorithm](#)
- [Fixing timing issues due to shared data](#)
- [Determining the absolute timestamp for a parameter update](#)

12.34.2 Timeline Locations

At any given moment, a plug-in may be asked to handle events from multiple locations on the timeline. Each module in an AAX plug-in may be updated using a different timeline position. For example:

- During automation playback the host may choose to send parameter updates in advance, while the algorithm is still processing audio from earlier in the timeline.
- When a processing chain involves a significant amount of latency, the host may delay the metering data which is available to the plug-in's GUI until the point in time when the corresponding processed audio is actually being played back to the user.

In this article, we will refer to the following timeline locations:

- *Automation time*: The location that corresponds to the state of the plug-in's data model
- *Playhead*: The location where the audio engine is currently gathering samples for processing
- *Render time*: The location of the audio samples currently being processed by the plug-in's algorithm
- *Presentation time*: The location that corresponds to the playback presentation to the user (i.e. the sound coming out of the speakers)

Figure 1: Timeline locations

12.34.3 Coordinating the data model and algorithm

As an AAX plug-in developer, you don't usually need to worry about the fact that your plug-in's data model and algorithm may each represent a different point in the timeline; the [AAX packet system](#) handles all of the necessary synchronization between these two locations.

This works seamlessly in a normal AAX plug-in because the real-time algorithm is fully decoupled from the plug-in's data model. Since all of the state information for the algorithm is delivered through its [context structure](#), the host can simply swap in the correct context data for each call to the processing callback. The plug-in does not require any special handling code to synchronize between the two timeline locations, and, as a bonus, AAX plug-ins can achieve deterministic, accurate automation playback without doing any extra work to handle time-stamped parameter update queues or other overhead.

Figure 2: Synchronization through the AAX packet system

12.34.3.1 A closer look at the AAX packet delivery system

Adding new packets for automation events

When playing back automation, the AAX host calls [UpdateParameterNormalizedValue\(\)](#) to update the data model state, then calls [GenerateCoefficients\(\)](#) to trigger the generation of new packets. See [Basic parameter update sequences](#) for a full description of this sequence.

Before the host calls [GenerateCoefficients\(\)](#) to generate packets for an automation breakpoint, it records the timeline position of the breakpoint ([AAX_IController::GetCurrentAutomationTimestamp\(\)](#) provides this value as a sample offset from the beginning of playback.) Every packet that is posted during execution of [GenerateCoefficients\(\)](#) is tagged with this timestamp when it is queued for delivery.

Packet delivery for AAX Native plug-ins

As the playhead advances and sample buffers are queued for processing, the host tracks the location of the next time-stamped packet in the packet queue. As the render time location for a Native plug-in processing chain approaches the next packet time-stamp for a plug-in in the chain, the host divides the plug-in's processing buffers into smaller buffers. When the render time location is as close as possible to the packet's time-stamp, the host delivers the packet. The packet data is available to the algorithm in its context the next time it is executed.

Because the host may divide native processing buffers down to a minimum size of [AAX_eAudioBufferLengthNative_Min](#) - 32 samples - the host can guarantee that all automation playback will be effected within 32 samples of the actual automation breakpoint location. In addition, with the help of some extra internal bookkeeping, AAX hosts also guarantee that the exact sample where an automation breakpoint is applied will be deterministic and will not change between different playback passes.

Packet delivery for AAX DSP plug-ins

The packet delivery system for AAX DSP plug-ins works similarly to the system for AAX Native plug-ins. AAX DSP plug-ins use a fixed buffer size, so the host is not able to divide their playback buffers into smaller units: the plug-in will receive each data packet in the fixed-size playback buffer which most closely corresponds to the location of the automation event which triggered the packet.

An AAX DSP plug-in which declares an [AAX_eProperty_DSP_AudioBufferLength](#) value of N will be guaranteed to receive data packets within N/2 samples of the actual automation event position on the timeline. Since the default buffer size for an AAX DSP plug-in is 4 samples, this yields extremely accurate automation playback with no extra work required in the plug-in algorithm.

12.34.4 Fixing timing issues due to shared data

The packet system works perfectly to synchronize the states of the plug-in data model and algorithm, *but only when the plug-in algorithm is fully decoupled from the data model*. If the algorithm directly shares data with the data model then the algorithm will immediately start using any new data model state without waiting for the corresponding coefficient delivery.

Figure 3 shows one kind of problem that can arise when a plug-in uses the same state for both its data model and its algorithm. In this case, the plug-in applied a volume trim (shown in the automation lane at the top of the image) to its algorithm as soon as the parameter update was applied to its data model, even though the algorithm was not yet processing the audio at the Automation time location. As a result, the audio trim was applied several hundred samples too early.

Figure 3: Offset automation playback due to lack of timeline location synchronization in a monolithic plug-in

12.34.4.1 Monolithic plug-ins

Plug-ins that share data directly between their data model and algorithm are referred to as *monolithic*. All plug-ins that inherit from the [AAX_CMonolithicParameters](#) helper class are monolithic.

Note

Monolithic plug-ins must always set the [AAX_eProperty_Constraint_Location](#) property to include [AAX_eConstraintLocationMask_DataModel](#) in order to avoid being loaded into incompatible AAX hosts.

All monolithic plug-ins must include special handling code to reconcile the plug-in's automation time state with its render time state.

12.34.4.2 How to resolve timing errors

There are many possible solutions for the timing errors that arise when a plug-in combines data from different time locations. Ultimately, the plug-in must separate the state that is represented at different time locations.

In most cases, this requires deferring data model state changes from being applied to the algorithm until the relevant samples are being processed in the render callback. One easy way to accomplish this separation is to take advantage of the synchronization provided by the AAX packet delivery system. This approach benefits from the fact that it emulates the design of a normal, decoupled AAX plug-in.

After a packet is queued with a call to `PostPacket()`, the packet delivery system will wait to update the algorithm's context structure with the packet's data until the Render time location is very close to the automation event (see [above](#).) This provides an appropriate mechanism for deferring state changes in the plug-in's data model until the Render time location has "caught up" to the correct sample.

Figure 4 shows the same scenario as Figure 3, but now the plug-in has been updated to defer data model updates from the automation time location so that they are applied as coefficients in the algorithm when the render time location has reached the correct point on the timeline.

Figure 4: Deferring a data model update in a monolithic plug-in using the packet queue

Here is one way to use the packet delivery system to defer changes to the data model state:

```
AAX_Result
MyEffectParameters::UpdateParameterNormalizedValue(
    AAX_CParamID iParamID,
    double aValue,
    AAX_EUpdateSource inSource)
{
    // Call inherited
    AAX_Result result = AAX_CMonolithicParameters::UpdateParameterNormalizedValue(
        iParamID,
        aValue,
        inSource);
    if (AAX_SUCCESS != result) { return result; }

    // Do whatever additional work is required to note that the
    // parameter has been updated - for example, set a "dirty"
    // flag for the parameter.

    return result;
}

AAX_Result
MyEffectParameters::GenerateCoefficients()
{
    // Call inherited
    AAX_Result result = AAX_CMonolithicParameters::GenerateCoefficients();
    if (AAX_SUCCESS != result) { return result; }

    const uint32_t stateNum = mMyStateCounter++; // member uint32_t

    // Do whatever additional work is required to capture the current
    // parameter state and associate it with stateNum, for example
    // check for "dirty" parameters and create a list of these
    // parameters with their values, add this list to a map using
    // stateNum as a key, and clear the "dirty" flags.

    result = Controller()->PostPacket(
        kCurrentStateFieldIndex,
        &stateNum,
        sizeof(uint32_t));

    return result;
}

struct MyContextStructure
{
    int32_t * mCurrentStateNum; // Private data
    // ...
};

void
MyAudioRenderCallback(
    MyContextStructure* const inInstancesBegin [],
    const void* inInstancesEnd)
{
    /* For each instance... */
    const uint32_t stateNum = instance->mCurrentStateNum;
```

```

// Update the custom plug-in object state based on stateNum
// and the additional data that was cached during
// GenerateCoefficients().
}

```

Figure 5: One specific solution for deferring a data model update in a monolithic plug-in using the packet queue

This approach is incorporated directly into the design of [AAX_CMonolithicParameters](#). If your plug-in data model is a subclass of [AAX_CMonolithicParameters](#) then you can follow these steps to ensure accurate parameter update timing in your plug-in:

1. After creating an automatable parameter, call [AAX_CMonolithicParameters::AddSynchronizedParameter\(\)](#) to add the parameter to an internal list of parameters to synchronize using the deferred-update system
2. In the plug-in's [RenderAudio\(\)](#) implementation, iterate through the incoming queue of deferred parameter values
3. Update the coefficients used by the plug-in's algorithm or other processing components

NOTES

- Remember to use the deferred parameter values, not values of the plug-in's [AAX_IParameter](#) objects, when setting the state of the plug-in's coefficients
- The deferred parameter values are delivered in the real-time thread, so all synchronized updates should follow the basic principles of real-time operation such as avoiding memory allocation/free, thread synchronization, access to shared resources, or any other actions which could block the real-time thread

For reference, see [DemoMIDI_Synth](#) and the other example instrument plug-ins. All of the instrument examples in the AAX SDK use these facilities to achieve deterministic, accurate playback for automated parameters.

One benefit of this approach is that it provides a compatible interface with monolithic plug-in objects which are designed to work across multiple plug-in formats. For example, the set of parameter updates provided to [AAX_CMonolithicParameters::RenderAudio\(\)](#) "RenderAudio" can be provided to plug-in objects which require a queue of time-stamped parameter updates for each audio render callback.

12.34.4.3 Additional considerations

Of course, the approach described in this section is just one possible solution. The [timestamp](#) section below provides some alternatives to using the packet queue system for synchronization. Ultimately, the best design for your plug-in will depend on the facilities that are available in the plug-in's monolithic state object, the size of this object, its interface, the number of parameters representing its state, and other internal details.

Here are some additional factors to consider when using the packet queue system for time location synchronization of parameter updates:

- The algorithm callback / [RenderAudio\(\)](#) method is called from a real-time thread, and may be called concurrently with data model methods. You should use a synchronization strategy that is optimized for high performance in this thread.
- If a parameter is not automatable then you should probably ignore these additional steps and directly update the plug-in's monolithic state object from within [UpdateParameterNormalizedValue\(\)](#) when that parameter is changed. Updates for non-automatable parameters can always be applied to the algorithm "as soon as possible".
- Depending on your plug-in's design you may not need or want to apply this solution to some automatable parameters either. For example, parameters that are unlikely to be automated or which require CPU-intensive changes in your instrument object should probably be updated on the object directly from within [UpdateParameterNormalizedValue\(\)](#), and not from within the real-time thread

12.34.5 Determining the absolute timestamp for a parameter update

The AAX packet queue provides a host-managed system for applying parameter updates at the correct location without requiring any special knowledge about the timeline. However, In some situations a plug-in may need to know the absolute sample position of a parameter change.

For example, a plug-in that synchronizes parameter changes to some external system, and which wants to forward these changes over to the external system as early as possible, would want to know the sample position for a coefficient update when the update is first triggered by a call to [GenerateCoefficients](#).

In these situations it is not suitable to simply use a method like [AAX_ITransport::GetCurrentNativeSampleLocation\(\)](#) which returns the current position of the audio render thread. The parameter update may be occurring at a different location on the timeline from the current render position, so using the current render position for the update would result in timeline offset problems similar to those described above.

12.34.5.1 Obtaining timeline information

AAX provides a variety of information that can be used for timeline synchronization. This information is provided through a combination of [AAX_ITransport](#), [AAX_IController](#), and MIDI beat clock data. Here is a summary of the relevant ways that a plug-in can get information about the timeline and timing synchronization data:

- [AAX_ITransport::GetCurrentNativeSampleLocation\(\)](#) Provides the absolute sample position of the first sample in the audio buffer that is currently being processed by the plug-in's worker chain
- [AAX_IController::GetTODLocation\(\)](#) Provides the current "time of day" value, which is a counter within the audio engine that counts the number of samples that the playhead has traversed since playback start
- [AAX_IController::GetCurrentAutomationTimestamp\(\)](#) Must be called from within [GenerateCoefficients](#). Provides the timestamp for the beginning of the hardware audio buffer during which the generated coefficients will be applied to the algorithm. This timestamp is provided in terms of the "time of day" counter, i.e. the number of samples since playback started.
- MIDI Beat Clock Sends transport start/continue/stop events to plug-ins that register global MIDI nodes

12.34.5.2 Determining the timeline position of a parameter update

Each of the available methods for getting information about the timeline position has a particular purpose. No single interface method can be used to directly determine the sample location for a parameter update, but it is possible to determine this value by combining information from a few of the available methods.

Here are some possible approaches for determining the timeline position of a parameter update

Note

Remember that these are not strict recipes; the specific requirements for what kinds of timeline information are needed will vary from plug-in to plug-in. You may be able to refine these approach to better match the needs of your specific plug-in.

12.34.5.2.1 1. Defer the update to the real-time thread

1. Queue state updates using a plug-in design similar to the one described [above](#)
2. When a state update is received on the real-time thread, call [GetCurrentNativeSampleLocation](#) to get the sample location for the start of the current render buffer
3. Perform all necessary update handling using this value as the sample location

NOTES

- This approach yields a sample location value which is accurate within 32 samples
- Event handling must be performed on the real-time render thread, which may not be viable depending on the types of operations that the plug-in must perform
- Event handling cannot be performed in advance to reduce overall system latency

12.34.5.2.2 2. Compute the timestamp as a TOD offset

1. Add a queue for update events which will be used internally within the plug-in's [AAX_IEffectParameters](#) object
2. In [UpdateParameterNormalizedValue](#), enqueue an update event
3. In [GenerateCoefficients](#), call [AAX_IController::GetTODLocation\(\)](#) and [AAX_IController::GetCurrentAutomationTimestamp\(\)](#)
4. Subtract the current TOD value from the automation timestamp to find the number of samples currently lie between the data model location and the render audio location on the timeline
5. Call [AAX_ITransport::GetCurrentNativeSampleLocation\(\)](#) and add the resulting value to the sample offset that was determined in the last step. The sum of these two values is the approximate absolute sample location for the coefficient update.
6. Once this sample location has been calculated, dequeue all pending update events and handle them using the calculated timestamp

The reason that this approach yields an approximate value is that the TOD location and current playback location are both given in terms of the real-time audio workers, and these values continue to progress simultaneously with execution of methods on the automation update thread. As a result, this approach will yield an absolute timestamp that is "late" by between zero and one hardware buffer.

NOTES

- Using this approach it is possible to handle parameter updates in advance to reduce overall system latency
- This approach yields a sample location value which is accurate within one hardware buffer
- This approach uses AAX interface methods that are not supported in older AAX hosts such as Pro Tools 10

12.34.5.2.3 3. Compute the timestamp with improved accuracy using MIDI Beat Clock You can refine the approach described above by using MBC events to detect the location of playback start.

1. Register a global MIDI node in your plug-in using [AAX_IEffectDescriptor::AddControlMIDINode\(\)](#) with [AAX_eMIDINodeType_Global](#) and the appropriate event mask bitfield for MBC events
2. Override [AAX_IEffectParameters::UpdateControlMIDINodes\(\)](#) to receive MBC data
3. When an MBC Start or Continue event is received, call [AAX_ITransport::GetCurrentNativeSampleLocation\(\)](#) to get the current render location. Cache this value. This value should represent the absolute playback start sample since audio render will not have started before the MBC event dispatch.
4. As in the previous solution, queue relevant update events in [UpdateParameterNormalizedValue](#)
5. In [GenerateCoefficients](#), call [GetCurrentAutomationTimestamp](#) and add the resulting value to the cached playback start sample location
6. Dequeue all pending update events and handle them using the calculated absolute sample timestamp

NOTES

- Using this approach it is possible to handle parameter updates in advance to reduce overall system latency
- This approach will yield timestamps within a few samples of the actual automation event location on the Pro Tools timeline
- This approach uses AAX interface methods that are not supported in older AAX hosts such as Pro Tools 10

Collaboration diagram for Parameter update timing:

12.35 Token protocol

Communicating parameter state with the host.

12.35.1 On this page

- [An Introduction to Tokens](#)
- [Basic Token Operation](#)

12.35.2 An Introduction to Tokens

One way in which a plug-in can communicate with the "outside world" is through Shared Data Services, also known as the Token System. This is a mechanism that allows Pro Tools to share parameter information with external hardware and software modules. While the AAX SDK only uses the Token System indirectly, knowing how it works will provide a good understanding of how linked parameters should operate.

12.35.2.1 Touch

Touch tokens inform the system of user interaction with a parameter. When a parameter is being touched the system knows to stop sending automation data to the plug-in and just use the SET value of the parameter. It is also used to tell the system when to start/stop recording new automation data.

In AAX, the touch message is sent to the host by `AAX_IAutomationDelegate::PostTouchRequest()`. The most common way to call this method is via the following methods:

```
class AAX_IEffectParameters
{
    virtual AAX_Result TouchParameter ( AAX_CParamID inParameterID );
    virtual AAX_Result ReleaseParameter ( AAX_CParamID inParameterID );
};

class AAX_IParameter
{
    virtual void Touch ();
    virtual void Release ();
};
```

However, AAX plug-ins will rarely need to call these methods directly since the `AAX_CParameter` and `AAX_CEffectParameters` implementations will automatically handle parameter touch and release tokens whenever a new value is set on the parameter by the plug-in.

Other clients besides the plug-in may touch a parameter. Since the TOUCH token can come from a control surface the touch state will actually come back to the plug-in via:

```
class AAX_IEffectParameters
{
    virtual AAX_Result UpdateParameterTouch ( AAX_CParamID iParameterID, AAX_CBoolean iTouchState );
};
```

This method is mainly important for [linked parameters](#).

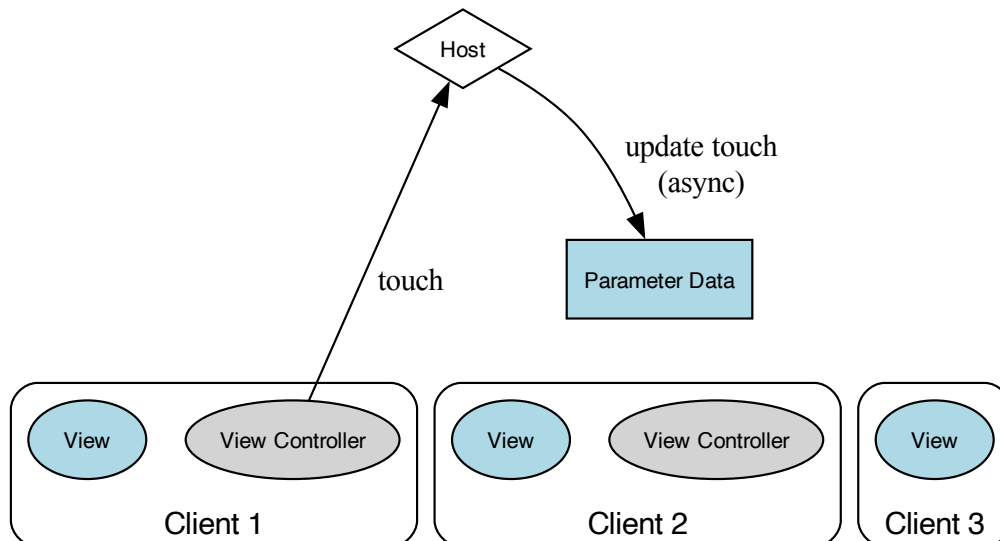


Figure 12.5 Touch request from a view controller, with resulting async touch update

12.35.2.2 Set

SET tokens can come from many different locations: the plug-in GUI, a control surface, loading a chunk or automation playback. Eventually the value of a SET token comes into the plug-in and that's when the internal value of the parameter gets updated. In AAX the SET token will be sent as a result of calling the following method:


```
class AAX_CParameter<T>
{
    void SetValue ( T newValue );
};
```

which will be called from many other supporting methods:

```
class AAX_CParameter<T>
{
    bool SetValueWithBool ( bool value );
    bool SetValueWithInt32 ( int32_t value );
    bool SetValueWithFloat ( float value );
    bool SetValueWithDouble ( double value );
    void SetToDefaultValue ();
    void SetNormalizedValue ( double normalizedNewValue );
    bool SetValueFromString ( const AAX_CString & newValueString );
};
```

When a SET token enters the system from the GUI, control surface or automation the value comes bak to the plug-in via the following method:

```
class AAX_CEffectParameters
{
    AAX_Result UpdateParameterNormalizedValue ( AAX_CParamID iParameterID, double aValue, AAX_EUpdateSource inSource);
};
```

At this point the internal contents of the plug-in are set.

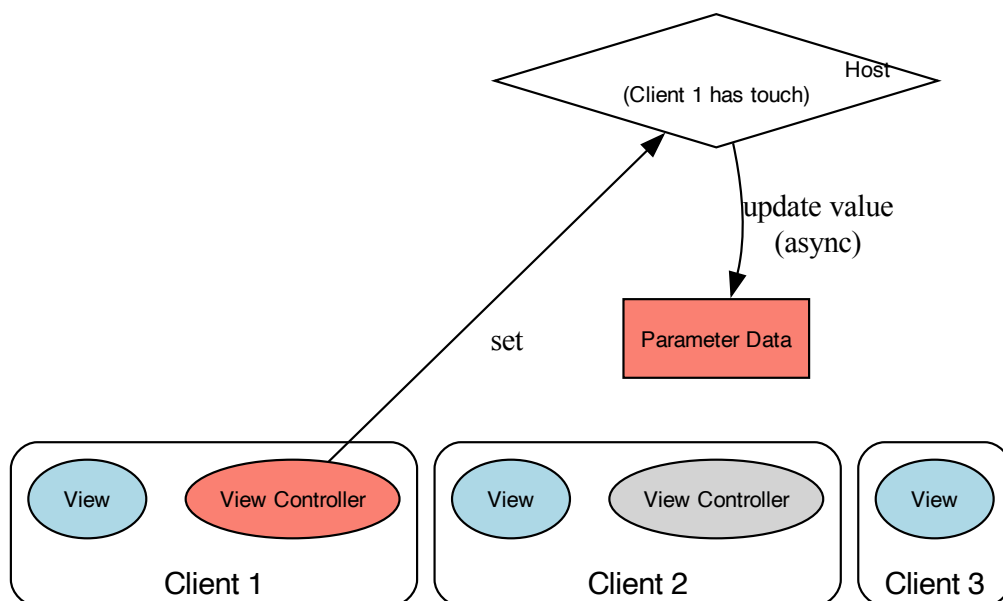


Figure 12.6 Set token asynchronously changes state of the parameter data

12.35.2.3 Update

An update token is generated when the internal value of a parameter has been set. GUIs and control surfaces listen for UPDATE tokens to update the displayed values. In AAX the UPDATE token is sent by calling the following method:

```
class AAX_CParameter<T>
{
    void UpdateNormalizedValue ( double newNormalizedValue );
};
```

All views of the parameter are then asynchronously notified that the value has changed. The plug-in GUI is notified via a call to `AAX_IEffectGUI::ParameterUpdated()`.

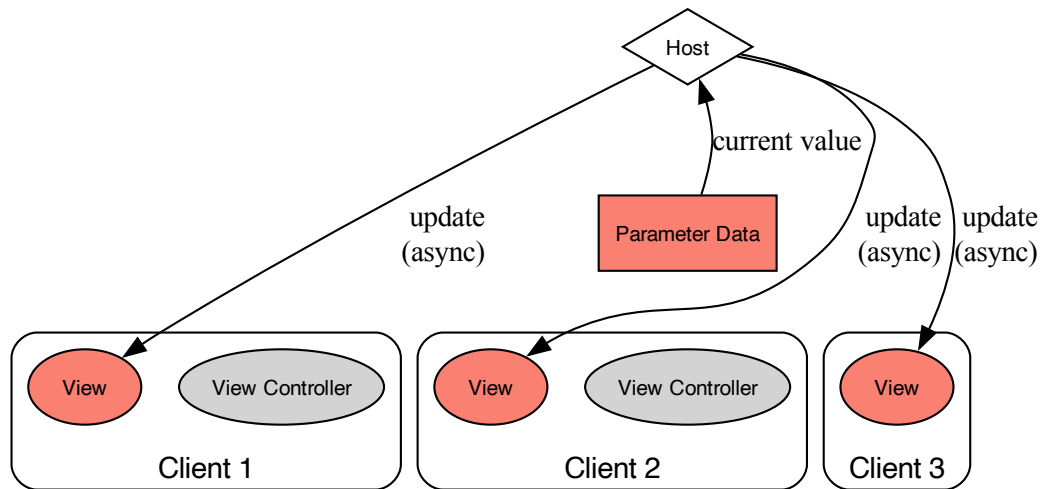


Figure 12.7 Update token triggers async updates to all views

12.35.3 Basic Token Operation

The lists below indicate how the system works in a few different standard update scenarios. To enable logging for these events set `DTF_AUTOMATION=file@DTP_LOW` in the [DigiTrace](#) configuration file. For more detailed information about the sequence of calls used to update parameters in different situations, see [Basic parameter update sequences](#).

12.35.3.1 User Editing

1. User clicks on a parameter in the GUI or grabs a parameter on the controls surface. A TOUCH token should be sent at this point.
2. The user changes the parameter from the GUI or controls surface. A SET token should be sent at this point.
3. The SET token goes into the system and comes back to the plugin via `UpdateParameterNormalizedValue()`.
4. The plug-in updates it's internal state and sends an UPDATE token.
5. Repeat steps 2-4 while changing the parameter.
6. The user lets go of the GUI or controls surface. A TOUCH token with the released state should be sent.

12.35.3.2 Automation Playback

1. The SET token comes from the automation system and enters the plugin via `UpdateParameterNormalizedValue()`.
2. The plug-in updates it's internal state and sends an UPDATE token.
3. Repeat steps 1-2 while playing back automation.

12.35.3.3 Chunk Restoring

1. Plug-in loads the chunk.
2. The plug-in sets every parameters value. Another thing to note is that the
3. SetValue() method also contains Touch() and Release() calls. So, while setting every parameter there is a combination of TOUCH and SET tokens sent to the system.
4. The SET tokens comes back to the plugin via UpdateParameterNormalizedValue().
5. The plug-in updates it's internal state and sends out UPDATE tokens.

Collaboration diagram for Token protocol:

12.36 Basic parameter update sequences

Sequence diagrams for some common parameter update scenarios.

12.36.1 On this page

- [User-generated update](#)
- [Automation playback](#)
- [Initialization](#)

Note

To enable logging for these events at run time set `DTF_AUTOMATION=file@DTP_LOW` in the [DigiTrace](#) configuration file.

12.36.1.1 Notes on threading for these sequences

- Calls from the host into [AAX_IEffectParameters](#) may occur on any thread. In general, the only synchronization that is guaranteed for data model calls in these diagrams is that the call will follow whatever event is indicated as its trigger.
- Calls from the host into [AAX_IEffectGUI](#) will occur on the main application thread unless indicated otherwise in the [AAX_IEffectGUI](#) documentation.
- Host-driven updates to the algorithm context are always synchronized with the real-time processing thread

12.36.2 User-generated update

This is the sequence of calls for a basic, unlinked parameter update triggered by the user. For this sequence, we assume that the edit was triggered by a GUI event.

12.36.2.1 High-level interface calls and events



Figure 12.8 High-level sequence of interface calls and events for a parameter update following a user-generated edit

12.36.2.2 Detailed sequence for default implementation

Note that this diagram assumes a GUI implementation that uses [SetParameterNormalizedValue\(\)](#). The implementation could also use other parameter set methods, either in [AAX_IEffectParameters](#) or directly on an [AAX_IParameter](#). The overall sequence would remain the same.

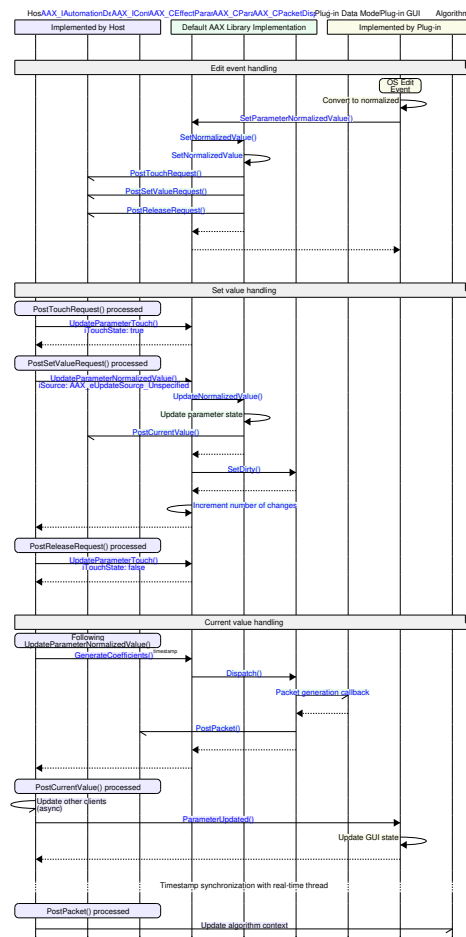
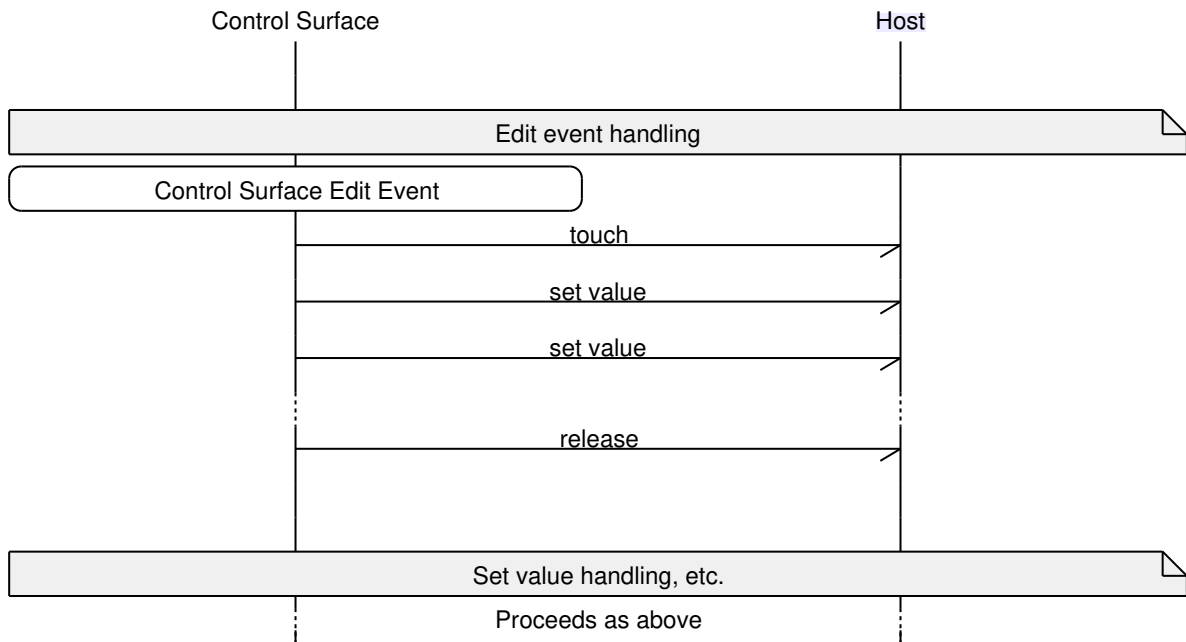


Figure 12.9 Detailed sequence of method calls and events for a parameter update following a user-generated edit on the plug-in GUI

12.36.2.3 Updates from control surfaces

Updates from control surfaces are handled in exactly the same way. In this case, though, the parameter touch, set value, and release tokens are generated by the control surface.



12.36.3 Automation playback

Automation playback handling is similar to the handling for user-generated parameter updates. However, parameters are never touched/released during automation playback. This allows touches from other clients, such as the GUI or control surfaces, to override the automation playback.

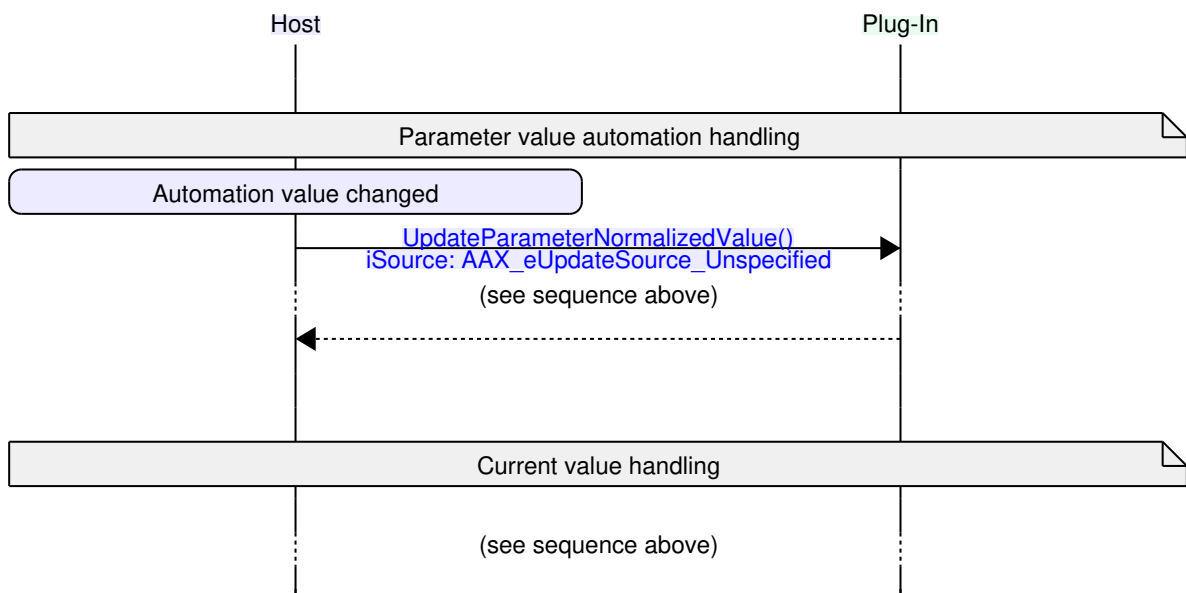


Figure 12.10 Sequence of method calls and events for playback of parameter automation

12.36.4 Initialization

This is the sequence of calls for the initial parameter updates made during data model initialization. Steps that are redundant with sections of the [standard user-generated update sequence](#) are elided.

Todo Update this section with information about default chunk setting, which is a separate step following the procedure described below.

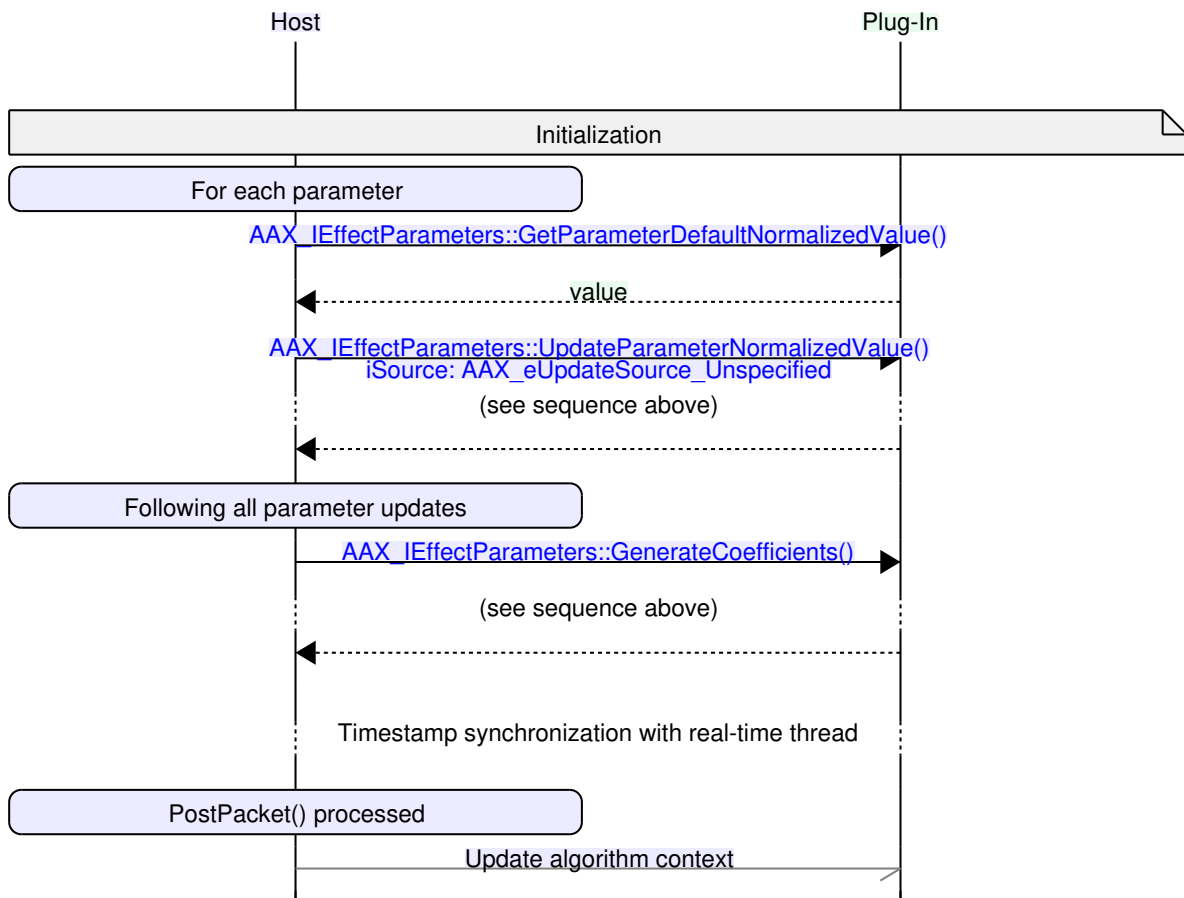


Figure 12.11 Sequence of method calls and events for parameter updates at plug-in initialization

Collaboration diagram for Basic parameter update sequences:

12.37 Linked parameters

How to link parameters.

12.37.1 On this page

- [Basics of Linked Parameters](#)
- [Linked Parameter Operation](#)
- [Changing Tapers](#)

12.37.2 Basics of Linked Parameters

A "linked" parameter can be defined as any parameter whose state is somehow dependent on another parameter. Within this general definition, there are various different kinds of parameter linking:

- Linking behavior can operate one-way or parameters can be reciprocally linked
- Linking between parameters can be one-to-one or one-to-many

12.37.2.1 Basic considerations for parameter linking

Although the concept of parameter linking is simple, implementing linked parameter behavior that is intuitive and consistent can require careful design.

- Parameter interdependencies and constraints can become complex, especially when handling multiple sets of linked parameters.
- Linked parameters that update other parameters during playback can result in subtle timing inconsistencies
- Automated parameters may contain arbitrary and conflicting automation data
- A user may attempt to edit multiple linked parameters simultaneously during playback, e.g. using multiple encoders on a control surface
- A plug-in may contain dependency cycles between interdependent parameters. These cycles can cause undesired behavior that is difficult to debug, especially if it only occurs in certain circumstances such as when loading particular presets.

In all of these cases, a plug-in should provide consistent linked parameter behavior: every automated playback pass should be identical, parameters should never "fight" one another or trigger rapid and unexpected changes in other parameters, parameters should not become "stuck" in a particular state, etc.

Here comes trouble

12.37.2.2 Defining proper linked parameter behavior

A good way to approach parameter linking is to start with an understanding of exactly what behavior you desire.

Here are some behaviors that you probably *don't* want in your plug-in:

- BAD: Parameters are only linked when edited from the plug-in GUI Users may attempt to edit linked parameters from attached control surfaces or using the host's automation features. The parameters should behave the same way regardless of which method is used to edit them.
- BAD: Parameters try to match all automation data Automation data can be written arbitrarily: Pro Tools doesn't have any restrictions that a user with a pencil tool must draw inside the lines, or a user may attempt to edit multiple parameters on an attached control surface simultaneously. Any parameter that attempts to match both its own automation data and the automation data of another parameter, or any parameter that attempts to set another automatable parameter's state based on its own automation data, will lead to "fighting" during playback of non-conformant automation.
- BAD: Automation data is only written to one lane at a time One approach to parameter linking may be to only write automation data to a single parameter at a time. This could be the parameter that is currently being touched and edited, or it could be a dedicated "master" parameter within the linked group. While this approach can be used to solve some types of conflicts, it can still lead to unnecessarily complex or inconsistent behavior in certain situations: for example, arbitrary automation data can still be written to multiple parameters' automation lanes, or a user can choose to record automation for only one parameter in a set but can skip the "master" parameter. Furthermore, it is difficult if not impossible to properly handle parameters that can be dynamically linked or un-linked using this approach.

With those potential problems in mind, here is a description of how parameter linking *should* behave.

12.37.2.2.1 Correct behavior for linked parameters *Notes*

- In this proposal (and throughout the rest of this page) the term "linker" will refer to a parameter that initiates a change and the term "linked" will refer to a dependent parameter that receives the change.
- The following discussion will focus on *automatable* parameters that are *reciprocally linked*. This case tends to be the most complex, with the greatest need for consistency of implementation.

During user-generated real-time edits (from the plug-in GUI or a control surface) both the linker and the linked parameters should be updated. Without this requirement, there would be no parameter linking. In order for this requirement to be enforced consistently the following behaviors must be maintained:

- The linked parameter should not jump to a new value if the user attempts to edit both parameters simultaneously using a control surface.
- To ensure proper automation playback, automation should be written to both the linker and the linked parameters.

When playing back automation, parameters should operate independently and should not attempt to force dependent parameters to a new state. This prevents fighting in the presence of incompatible automation and ensures deterministic automation playback with every playback pass. As above, there are some more subtle behaviors that must be maintained for this to work properly:

- If the user begins a real-time edit during automation playback then the parameter linking behavior should resume as described above
- If the plug-in's algorithm cannot support certain parameter configurations then its automatable parameters should be decoupled from the algorithm using a set of coefficients that is aware of the algorithm's constraints. In this way every combination of parameter states can map to a particular coefficient state, maintaining determinism, and incompatible parameter combinations can simply resolve to the "closest" match in the possible coefficient space during playback of edited parameter automation data.
- Another, simpler approach for plug-ins that do not support arbitrary parameter configurations is to ensure that the problematic parameters are not automatable. Handling non-automatable parameter linking is much easier in general, so consider this approach if automation is not a requirement for some of your plug-in's parameters.

When handling preset changes and plug-in initialization, a similar approach should be taken as with plug-in automation playback. In these cases it is very unlikely that the plug-in's parameters will be left in an incompatible state and attempts at linking may result in unwanted update cycles between inter-dependent parameters or unnecessary coefficient churn. This latter concern can be a real problem for AAX DSP plug-ins that initialize internal algorithmic state based on initial coefficient data.

12.37.2.2.2 Compatibility caveat This behavior was not possible under the RTAS/TDM format, and many RTAS and TDM plug-ins reverted to workarounds such as writing automation to only one parameter at a time and linking the parameters during playback. Therefore, plug-ins that previously supported linked automatable parameters under the RTAS/TDM format may not be able to both implement this recommended parameter linking behavior and maintain compatibility with automation in saved sessions.

Most of Avid's plug-ins that were available in the RTAS and TDM formats fall into this category and should not be used as examples of proper parameter linking behavior. Instead, use the SDK's DemoGain_LinkedParameters example plug-in as an example of proper linked parameter operation.

12.37.3 Linked Parameter Operation

As described above, the key rule for linked parameters is to link during real-time user edits *only*, and should operate the parameters independently (without linked behavior) during automation playback and preset restore. This rule will simplify many issues: it will prevent conflicts with automation data, avoid potentially strange behaviors when restoring presets, and more.

Here is how the system works WITH linked parameters, using code snippets from the DemoGain_LinkedParameters example plug-in:

12.37.3.1 User Editing

1. User clicks on a parameter in the GUI or grabs a parameter on the controls surface. A TOUCH token should be sent at this point.

- The touched parameter status comes back to the plug-in. If the parameters are linked the other linked parameter should have a TOUCH token sent. This really should only be done for linked continuous parameters. This is done by overriding the `AAX_CEffectParameters::UpdateParameterTouch()` method.

2. The user changes the parameter from the GUI or controls surface. A SET token should be sent at this point.

```
// *****
// METHOD: UpdateParameterTouch
// *****
AAX_Result DemoGain_Parameters::UpdateParameterTouch ( AAX_CParamID inParameterID, AAX_CBoolean
inTouchState )
{
    if ( inTouchState )
    {
        AAX_CParamID linkedControl = this->GetLinkedControl ( inParameterID );
        if ( linkedControl )
        {
            this->TouchParameter ( linkedControl );
            mLinkTouchMap.insert ( std::pair<std::string, std::string>( inParameterID, linkedControl )
);
        }
    }
    [...]
}
```

3. The SET token goes into the system and comes back to the plugin via `AAX_CEffectParameters::UpdateParameterNormalizedValue()`

- If the parameter is linked then the other linked parameter should have its value set for its linked behaviour. The system knows this is a linked parameter so when the value comes back to the plug-in via `UpdateParameterNormalizedValue()` it will know not to perform linked behaviors on that value change. To determine if a parameter should set a linked parameter you check it with the `AAX_CEffectParameters::IsParameterTouched()` method.

4. The plug-in updates its internal state and sends an UPDATE tokens for both parameters.

```
// *****
// METHOD: UpdateParameterNormalizedValue
// *****
AAX_Result DemoGain_Parameters::UpdateParameterNormalizedValue ( AAX_CParamID inParameterID, double
inValue, AAX_EUpdateSource inSource )
{
    AAX_Result result = AAX_CEffectParameters::UpdateParameterNormalizedValue ( inParameterID,
inValue, inSource );
    bool touched = this->IsParameterTouched ( inParameterID );
    [...]
    if ( touched && inSource == AAX_eUpdateSource_Unspecified )
    {
        if ( type == eType_Pan )
            this->SetParameterNormalizedValue( linkedControl, (1.0 - inValue) );
        else if ( type == eType_Gain )
            this->SetParameterNormalizedValue( linkedControl, inValue );
    }
    [...]
}
```

5. Repeat steps 2-4 while changing the parameter.

6. The user lets go of the GUI or controls surface. A TOUCH token with the released state should be sent.

- The touched parameter status comes back to the plug-in. If the parameters were linked the other linked parameter should have a TOUCH token with the release status sent. This again is done by overriding the `AAX_CEffectParameters::UpdateParameterTouch()` method.

```
// *****
// METHOD: UpdateParameterTouch
// *****
AAX_Result DemoGain_Parameters::UpdateParameterTouch ( AAX_CParamID inParameterID, AAX_CBoolean
inTouchState )
{
    if ( inTouchState )
    {
        [...]
    }
    else
    {
        [...]
        this->ReleaseParameter ( iter->second.c_str () );
        [...]
    }
    return AAX_SUCCESS;
}
```

12.37.3.2 Automation Playback

1. The SET token comes from the automation system and enters the plugin via `UpdateParameterNormalizedValue()`.
 - The plug-in will know this is not from the user editing therefore it will NOT set the other linked parameter. Remember ONLY LINK USER EDITING. That way there's no conflicts if the user edited the automation or if the order in which automation arrives at the plug-in changes.
2. The plug-in updates its internal state and sends an UPDATE token.
3. Repeat steps 1-2 while playing back automation.

12.37.3.3 Chunk Restoring

1. Plug-in loads the chunk.
2. The plug-in sets every parameters value.
3. The SET tokens comes back to the plugin via `UpdateParameterNormalizedValue()`.
 - The plug-in will know this is not from the user editing therefore it will NOT set the other linked parameter. Remember ONLY LINK USER EDITING. Hopefully the result of this is that the contents of the chunk will be restored to its exact state.
4. The plug-in updates its internal state and sends out UPDATE tokens.

12.37.4 Changing Tapers

One common use of linked parameters is to change the taper associated with a parameter. For changing tapers there are basically only a two rules you need to follow:

1. When you're loading a new chunk you need to set the taper values first. If a parameter is what updates the taper then set that value first. That way when the value of a parameter is set from a chunk it wont change because of a taper change.
2. Update the taper from the `UpdateParameterNormalizedValue()` method. If the new taper needs to change the value of the parameter you only do so if the user is editing the linked parameter. This still follows the ONLY LINK USER EDITING rule.

```

AAX_Result Simple_Parameters::UpdateParameterNormalizedValue ( AAX_CParamID inParameterID, double inValue,
    AAX_EUpdateSource inSource )
{
    // GetLinkedControl() is a user defined method which determines the linked control ID.
    AAX_CParamID linkedControl = this->GetLinkedControl ( inParameterID );
    if ( linkedControl )
    {
        // IsParameterLinkReady()* is a built in method of AAX_CEffectParameters which determines if the
        // parameter should perform linked behaviors based on the touch state of the parameter and the
        // source of the UpdateParameterNormalizedValue() call.
        if ( this->IsParameterLinkReady ( inParameterID, inSource ) )
            this->SetParameterNormalizedValue( linkedControl, inValue );
    }
    // Call the inherited method for the original parameter
    AAX_Result result = AAX_CEffectParameters::UpdateParameterNormalizedValue ( inParameterID, inValue,
        inSource );
    return result;
}

```

Collaboration diagram for Linked parameters:

12.38 Linked parameter update sequences

Sequence diagrams for some common linked parameter update scenarios.

12.38.1 On this page

- [User-generated update](#)
- [Update from automation playback](#)

Note

To enable logging for these events at run time set `DTF_AUTOMATION=file@DTP_LOW` in the [DigiTrace](#) configuration file.

12.38.1.1 Notes on threading for these sequences

- Calls from the host into [AAX_IEffectParameters](#) may occur on any thread. In general, the only synchronization that is guaranteed for data model calls in these diagrams is that the call will follow whatever event is indicated as its trigger.
- Calls from the host into [AAX_IEffectGUI](#) will occur on the main application thread unless indicated otherwise in the [AAX_IEffectGUI](#) documentation.
- Host-driven updates to the algorithm context are always synchronized with the real-time processing thread

See also

[Basic parameter update sequences](#)

12.38.2 User-generated update

This is the sequence of calls for a parameter update triggered by the user. For this sequence, we assume that the edit was triggered by a GUI event. Updates from control surfaces are handled in exactly the same way, except that the parameter touch, set value, and release tokens are generated by the control surface.

In this example the updated parameter is reciprocally linked to one other parameter. These are the "linker" and "linked" parameters, respectively.

This procedure is very similar to the non-linked case described [here](#). In the diagrams below, red arcs and pink section headings are used to indicate events that are specific to the linked parameter case.

Notes:

1. This sequence shows the linked parameter reciprocally issuing a touch on the linker parameter. The touch fails since the linker parameter is already touched at this time. If the roles were reversed (if an edit occurred on the linked parameter) then this touch would succeed.
2. The host flags all set value tokens that are triggered by a plug-in within the scope of [AAX_IEffectParameters::UpdateParameterNormalizedValue\(\)](#). When those set value tokens are processed they result in additional calls to [AAX_IEffectParameters::UpdateParameterNormalizedValue\(\)](#). The host sets `iSource` to [AAX_eUpdateSource_Parameter](#) for each of these subsequent calls to indicate that the update originated from within a parameter update event.
3. [IsParameterLinkReady\(\)](#) returns `true` during the linker parameter update because the update source is unknown and the parameter is touched. Both conditions must be true in order for the linking logic to proceed with setting linked parameters' values.
4. [IsParameterLinkReady\(\)](#) returns `false` during the linked parameter update because the source is [AAX_eUpdateSource_Parameter](#). This prevents update cycles for reciprocally linked parameters, as demonstrated here.

12.38.2.1 High-level interface calls and events

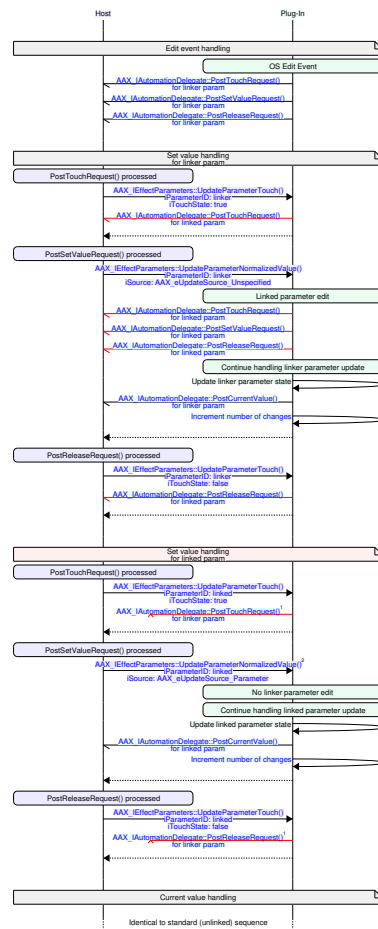


Figure 12.12 High-level sequence of interface calls and events for a reciprocally linked parameter update following a user-generated edit

12.38.2.2 Detailed interface calls and events

Note that this diagram assumes a GUI implementation that uses [SetParameterNormalizedValue\(\)](#). The implementation could also use other parameter set methods, either in [AAX_IEffectParameters](#) or directly on an [AAX_IParameter](#). The overall sequence would remain the same.

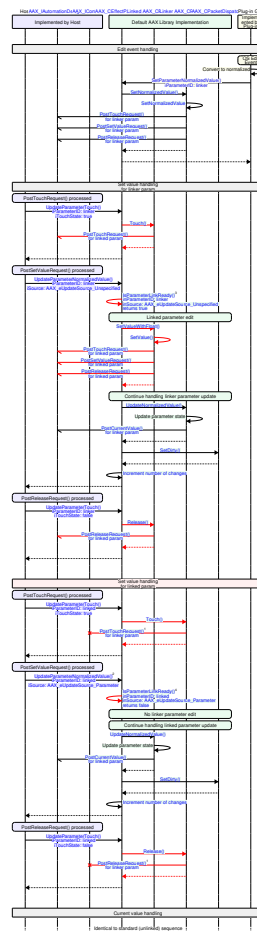


Figure 12.13 Detailed sequence of method calls and events for a reciprocally linked parameter update following a user-generated edit on the plug-in GUI

12.38.3 Update from automation playback

Since all parameter linking occurs while recording automation, automation playback is very simple. The automation lanes may contain any arbitrary values, so, in order to avoid fighting between incompatible values, the plug-in should respect all automation values during playback.

Notes:

1. `IsParameterLinkReady()` returns `false` during automation playback because the updated parameter is not touched. This ensures that automation playback will proceed with the written values and also guarantees that the user will always be able to override the automation using a control surface encoder or GUI editor.

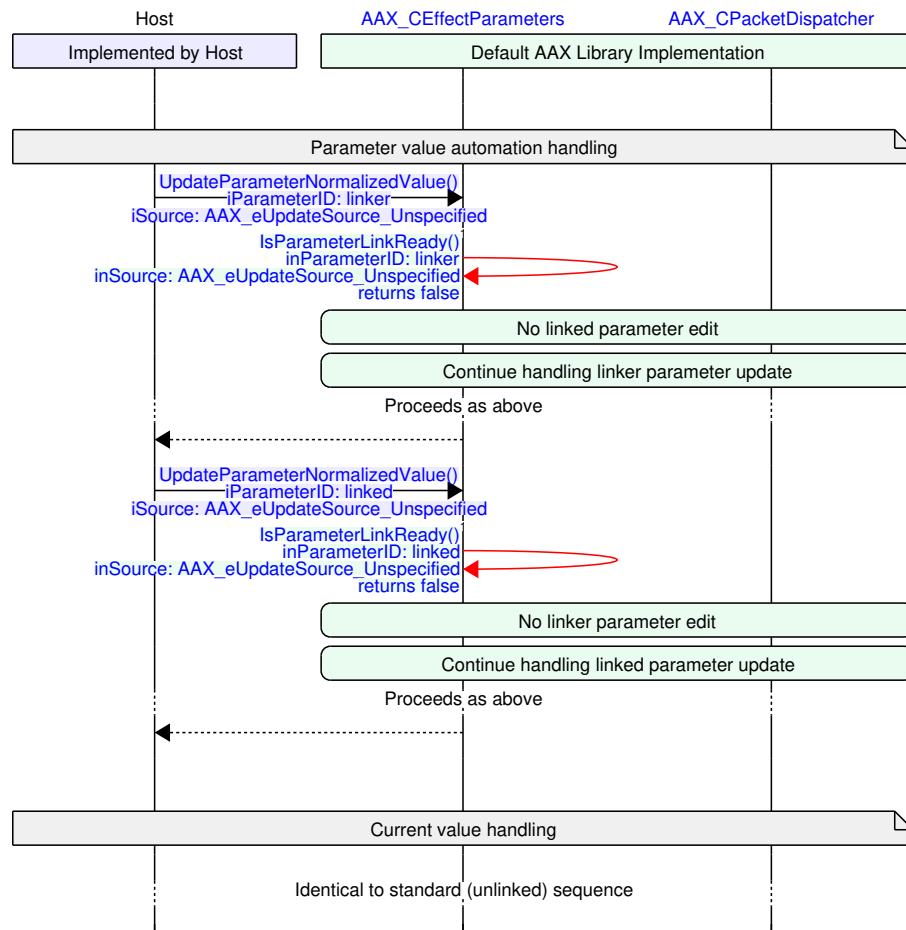


Figure 12.14 Sequence of method calls and events during automation playback with linked parameters

Collaboration diagram for Linked parameter update sequences:

12.39 Plug-in type conversion

Specification for valid conversions between plug-in types.

12.39.1 About this specification

The specification on this page defines the valid AAX plug-in type conversions. An AAX host may use this specification to perform automatic type conversions. For example:

- When a session that was saved with a DSP plug-in Type is opened on a Native system the saved DSP Type should be converted to its Native counterpart.
- When a session saved with the free version of a plug-in is opened on a system that has the full version installed then the saved free Type may be converted to the full version.

12.39.2 Terminology

In this specification the term "Type" refers to a specific configuration of an AAX plug-in.

Each Type is uniquely identified by a combination of five values:

- Manufacturer ID
- Product ID
- Plug-In ID
- Sample rate bit mask
- Architecture

Each Type is defined by a particular call to [AAX_IComponentDescriptor::AddProcessProc_Native](#) or [AAX_IComponentDescriptor::AddProcessProc_TI](#) in the plug-in's description method.

The Plug-In ID is defined as one of [AAX_eProperty_PluginID_Native](#) or [AAX_eProperty_PluginID_TI](#).

In this specification, the format for describing an ID is:

[ID triad + sample rate + architecture]

Where the ID triad may be expanded to [[Manufacturer ID] [Product ID] [Plug-In ID]]

- Explicit values are given in plain face
- Arbitrary constant values are given in *bold face*
- Wildcard values are given in *italics*

For example:

- [*ID triad* + *any sample rate* + Native]
 - Defines all Type identifiers with the same ID triad and the Native architecture, regardless of sample rate.
- [[*Manufacturer ID*] [*Product ID*] [*any ID*] + *sample rate* + Native]
 - Defines all Type identifiers with the same Manufacturer and Product IDs, the same sample rate, and the Native architecture, but with any plug-in ID.

12.39.3 Scope of this specification

For the purposes of this specification we are not concerned with AudioSuite Types. Currently these Types are never type-converted.

In theory, an AAX host may apply a "partial" type conversion by swapping between different ProcessProcs without destroying and re-building any of the plug-in's objects (data model, GUI). For the purposes of this specification we are not concerned about whether a given conversion is "partial" or "total"; all conversions are treated the same.

Plug-in conversions are also required between Types of different stem formats. In fact, the supported stem format is an integral part of a Type's unique identifier. This specification ignores the question of stem formats entirely; we assume that each Type supports all necessary stem formats for legal conversion with other Types.

In general, type swapping rules for deprecated types and related types are equivalent. See [Type deprecation](#) for more information about the differences between related and deprecated types.

12.39.4 Topological constraints

- All Types included in a single [AAX_ICollection](#) must have the same Manufacturer ID
- All Types included in a single [AAX_IEffectDescriptor](#) must have the same Manufacturer ID and Product ID
- The Sample rate bit masks for two Types that share all other identifiers must be non-overlapping. I.e. $0 \times 0 == sr_mask1 \ \& \ sr_mask2$
- Two Types may not be registered that differ only in their Architecture
- Type relationships may only be defined between mutually exclusive Types. Types are mutually exclusive if:
 - Their sample rate support is non-overlapping
 - The "before" Type's ID triad is not associated with any Type in the plug-in

12.39.5 Implicit conversions

The AAX host should automatically convert between Types within the following IDs:

- [*ID triad* + *any sample rate* + *Architecture*]
- [[*Manufacturer ID*] [*Product ID*] [*any ID*] + *sample rate* + *any architecture*]

These conversions occur only if both Types are included in the plug-in when the conversion is made. Consider the following scenario:

1. A session is saved including an plug-in instance with the following Type identifier: [My ID triad + 48 kHz + TI]
2. The plug-in is updated to a version that does not include this Type identifier, but that does include [My ID triad + 48 kHz + Native]
3. The session is opened on a Native system

In this scenario, if the plug-in had not been updated then an automatic conversion would occur. However, since the plug-in no longer includes the saved Type identifier, no automatic conversion occurs.

A plug-in can work around this situation by including the "old" Type identifier as a Related (or Deprecated) Type (see [Explicit conversions](#).)

When a plug-in instance is saved after making an implicit conversion, plug-ins may be saved with the session using their new Type identifier. This is not required. For example, Pro Tools will prefer to save plug-in instances that were converted from DSP types as DSP, even if they were converted to corresponding Native types when the session was loaded onto and saved from a native Pro Tools system.

12.39.6 Explicit conversions

AAX includes properties that allow a plug-in to define explicit relationships between different Types. These properties only operate on ID triads. Each property can be associated with an array of ID triads to define a one-to-one or a many-to-one association between the given ID triads and the specific Type to which the property is attached.

- [AAX_eProperty_Related_DSP_Plugin_List](#) / [AAX_eProperty_Deprecated_DSP_Plugin_List](#)
 - All of the given ID triads specify TI Types
- [AAX_eProperty_Related_Native_Plugin_List](#) / [AAX_eProperty_Deprecated_Native_Plugin_List](#)
 - All of the given ID triads specify Native Types

The AAX host should convert between related Types within the following constraints:

- [[[Manufacturer ID] [Related Product ID] [Related TI Plug-In ID]] + *sample rate* + TI]
 - -> [[[Manufacturer ID] [New Product ID | Related Product ID] [New Plug-In ID]] + *sample rate* + *any architecture*]
- [[[Manufacturer ID] [Related Product ID] [Related Native Plug-In ID]] + *sample rate* + Native]
 - -> [[[Manufacturer ID] [New Product ID | Related Product ID] [New Plug-In ID]] + *sample rate* + *any architecture*]

These conversions will occur regardless of whether the related ID triad is used for any of the Types in the plug-in.

When a plug-in instance is saved after making an explicit conversion, all plug-ins are saved with the session using their new Type identifier.

Host Compatibility Notes Pro Tools versions prior to Pro Tools 12.3 do not allow explicit type conversion between types with different product ID values. Beginning in Pro Tools 12.3 both the product ID and the plug-in ID may differ between explicitly related types.

12.39.7 Type deprecation

There are two varieties of explicit plug-in type association: related types and deprecated types.

Properties that create a related type association:

- [AAX_eProperty_Related_Native_Plugin_List](#)
- [AAX_eProperty_Related_DSP_Plugin_List](#)

Properties that create a deprecated type association:

- [AAX_eProperty_Deprecated_Native_Plugin_List](#)
- [AAX_eProperty_Deprecated_DSP_Plugin_List](#)
- [AAX_eProperty_PluginID_Deprecated](#)

With a few exceptions, these two types of explicit association are treated identically. These are the ways in which deprecated plug-in type association differs from related plug-in type association:

- If plug-in types A and B are both installed, and if type A deprecates type B, then type B will be excluded from all insert lists in Pro Tools and will be made invisible to the user.
- If plug-in types A and B are both installed, and if type A deprecates type B, then instances of type B will be swapped to type A when a session containing saved instances of type B is opened.
- Upon the next session save following a swap due to a deprecated type association, the plug-in instance will be saved into the session using the ID of the new plug-in type. Therefore deprecated type swaps do not round-trip when moving between multiple systems if some of the systems have the deprecated types, but not the deprecating types, installed.

Type deprecation should only be used when a new version of an effect completely replaces an old version of the effect. For all other situations, type relationship should be used.

Host Compatibility Notes

- Pro Tools versions before Pro Tools 12.3 treat deprecated and related type associations identically and do not support type deprecation features
- Media Composer does not support type deprecation features
- VENUE does not support type deprecation features

Collaboration diagram for Plug-in type conversion:

12.40 The Avid Component Framework (ACF)

12.40.1

How the AAX C++ interfaces work.

The objects and interfaces in AAX are based on the Avid Component Framework (ACF). The ACF is Avid's implementation of COM, and is the framework that AAX, as well as AVX (Avid Video Extensions) plug-ins are built on.

ACF can be considered an implementation detail of the AAX SDK; the SDK is written to protect plug-in developers from the intricacies of ACF, and it is not necessary to understand ACF or COM in order to use the SDK.

12.40.2 More details

As in COM, ACF draws a distinction between the concept of an object and the concept of an interface. An object is treated as a "black box" of code, whereas an interface is a class of pure virtual methods that allows one to access the functionality inside the object. An object in ACF is represented by the [IACFUnknown](#) interface, which is binary compatible with the COM class IUnknown. (Likewise, [IACFUnknown](#) follows the same reference counting rules as IUnknown objects.) This interface allows a client to get pointers to other interfaces on a given object using the [QueryInterface\(\)](#) method.

Reference counting is an important aspect of both COM and ACF. Simply put, reference counting is the practice of tracking all references to an object, so that a program can determine when the object can safely be deleted. The AAX SDK library handles this reference counting behind the scenes, so plug-ins that call into the SDK library to manage their component interfaces will not leak references.

Many additional resources can be found both online and print that cover COM and reference counting in greater detail.

12.40.3 ACF interfaces in AAX

The binary interface between an AAX plug-in and host is defined by a series of ACF interfaces. Each of these interfaces inherits from `IACFUnknown`. The implementation of each ACF interface typically uses `CACFUnknown`, a utility class that provides basic reference counting and additional fundamental ACF details to satisfy `IACFUnknown`.

These ACF interfaces may be implemented by either the AAX plug-in or the host. The host retains a reference to each interface that is implemented by the plug-in in order to call methods on the plug-in's implementation. Correspondingly, the plug-in retains references to various interfaces that are implemented by the host, and may call host methods via these interfaces.

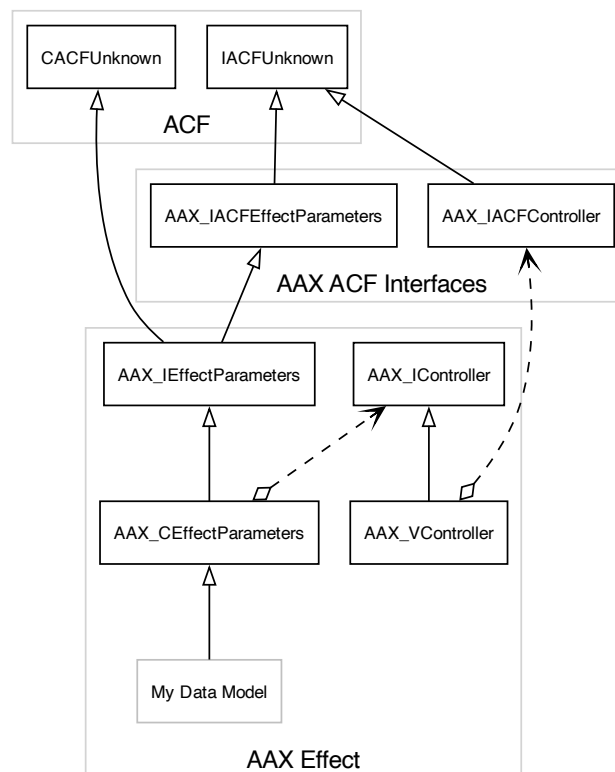


Figure 12.15 ACF interfaces: `AAX_IACFEffectParameters` and `AAX_IACFController`

The figure above demonstrates this design: the plug-in implements `AAX_IACFEffectParameters` directly, and retains a reference to an `AAX_IACFController` that is implemented by the host.

In order to implement `AAX_IACFEffectParameters`, `AAX_IEffectParameters` inherits from `CACFUnknown` and implements `QueryInterface()` to ensure that the `IACFUnknown` interface is implemented. The rest of the implementation of `AAX_IACFEffectParameters` is contained in `AAX_CEffectParameters` and the plug-in's custom data model class.

The reference to `AAX_IACFController` is managed by a versioned implementation class. For more information about this design, see below.

12.40.4 Using ACF interfaces

Depending on where an interface is implemented, there are two specific ways to acquire a reference to the underlying AAX object from an [IACFUnknown](#) pointer:

- [Host-provided interfaces](#)
- [Plug-in interfaces](#)

12.40.4.1 Host-provided interfaces

Interfaces that are managed by the host must be carefully version-controlled in order to maintain compatibility with many different host versions. The AAX SDK includes "AAX_V" classes to handle this versioning. AAX_V classes are concrete classes that query the host for the correct version of the requested interface. These classes can also handle re-routing deprecated calls and other complicated versioning logic.

To create an AAX_V object, pass an [IACFUnknown](#) pointer to the underlying host-managed interface in to the AAX_V class' constructor. ACF reference counting is handled automatically by the object's construction and destruction routines, so no additional calls are necessary to acquire and release the reference.

```
#include "AAX_VController.h"
void SomeFunction (IACFUnknown * inController)
{
    // When object is created, a reference is acquired
    AAX_VController theController (inController);

    //
    // ...
    //

    // When object goes out of scope, the reference is released
}
```

12.40.4.2 Plug-in interfaces

Interfaces to objects that are owned by the plug-in always have a known version and therefore do not require AAX_V object management. Instead, these interfaces must be acquired and released directly using ACF.

```
#include "AAX_UIDs.h"
#include "AAX_Assert.h"
#include "AAX_IEffectParameters.h"
#include "acfunknown.h"
void SomeFunction (IACFUnknown * inController)
{
    // When interface is queried, a reference is acquired
    if ( inController )
    {
        AAX_IEffectParameters* myEffectParameters = NULL;
        ACFRESULT acfErr = ACF_OK;
        acfErr = inController->QueryInterface(
            IID_IAAXEffectParametersV1,
            (void **)&myEffectParameters);

        AAX_ASSERT(ACFSUCCEEDED(acfErr));
    }

    //
    // ...
    //

    // The reference must be explicitly released when finished
    if (myEffectParameters)
    {
        myEffectParameters->Release();
        myEffectParameters = NULL;
    }
}
```

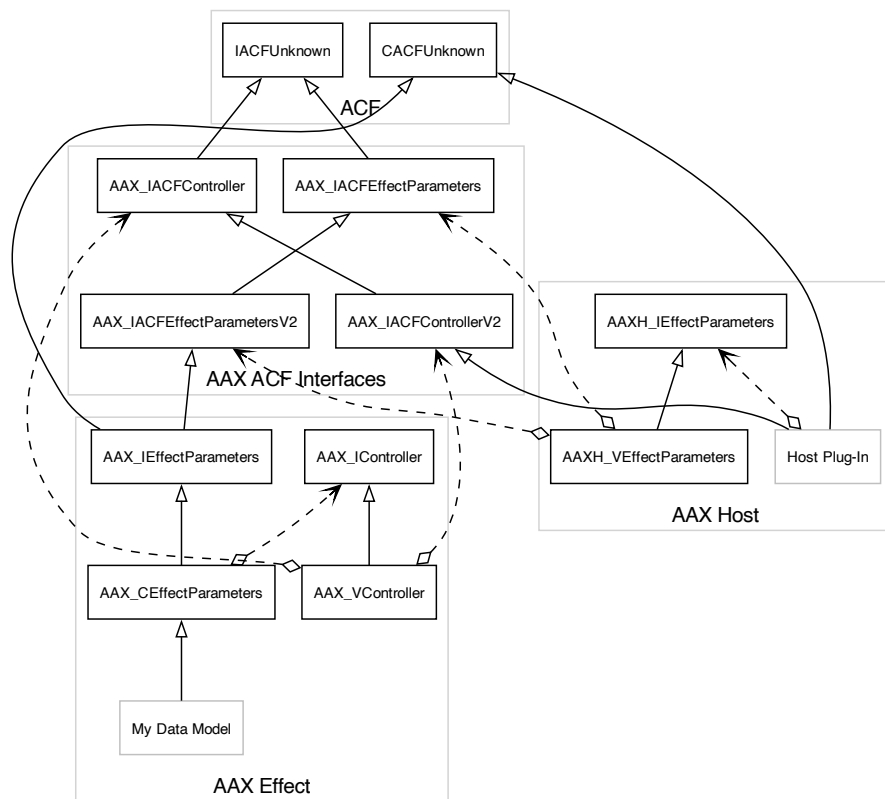



Figure 12.18 Complete design example with versioned ACF interfaces

Documents

- [ACF Elements](#)

ACF classes that are used by common AAX interfaces.

Collaboration diagram for The Avid Component Framework (ACF):

12.41 ACF Elements

12.41.1

ACF classes that are used by common AAX interfaces.

Classes

- interface [IACFUnknown](#)

COM compatible IUnknown C++ interface.

Collaboration diagram for ACF Elements:

12.42 AAX Host Guides

12.42.1

Documentation for specific AAX host environments.

Documents

- [Pro Tools Guide](#)
Details about using AAX plug-ins in Pro Tools.
- [Media Composer Guide](#)
Details about using AAX plug-ins in Media Composer.
- [HDX DSP Guide](#)
How to write AAX plug-ins for Avid's TI DSP-based platforms.
- [Page Table Guide](#)
How to map a plug-in's parameters to control surfaces.
- [DigiTrace Guide](#)
How to add tracing to your plug-ins and view logging from the plug-in host.
- [DSH Guide](#)
How to test basic functionality of AAX plug-ins using DSH test tool.
- [DTT Guide](#)
How to automate different test scenarios for DSH.
- [VENUE Guide](#)
Details about using AAX plug-ins in VENUE live sound systems.

Collaboration diagram for AAX Host Guides:

12.43 Pro Tools Guide

Details about using AAX plug-ins in Pro Tools.

12.43.1 Contents

- [About this document](#)
- [Processing modes](#)
- [Requirements for AAX plug-in compatibility with Pro Tools](#)
- [Audio Engine Behavior and Features](#)
- [Basic plug-in operation](#)
- [Optional plug-in features](#)
- [Using the Pro Tools Scripting SDK with AAX](#)
- [Plug-ins with MIDI support](#)
- [Debugging AAX plug-ins](#)
- [Troubleshooting common AAX plug-in failures](#)
- [Using DigiOptions](#)
- [Compatibility Notes](#)

12.43.2 About this document

This guide discusses specific details related to using AAX plug-ins with Pro Tools, such as loading and initialization procedures, GUI hosting, and other application-specific features. This guide is not intended to provide complete documentation for the Pro Tools application. For more information about the features, functionality, and use of Pro Tools see the Pro Tools Reference Guide, available for download from the Avid web site.

This guide is a work in progress, and is extended as needed to describe different aspects of and caveats to the Pro Tools implementation of the AAX host spec.

12.43.3 Processing modes

Pro Tools supports three AAX processing modes: AudioSuite, AAX Native, and AAX DSP.

- AudioSuite plug-ins perform non-real-time, random-access, file-based processing entirely on the host CPU.
- AAX Native plug-ins perform real-time, linear, non-destructive processing entirely on the host CPU. Native plug-ins are also used by Pro Tools to perform offline rendering.
- DSP plug-ins perform real-time, linear, non-destructive processing on DSP-accelerated hardware, with non-real-time tasks performed on the host CPU.

Each of these processing modes offers specific advantages and trade-offs in functionality, power, and development effort, and plug-in developers may choose to develop only for specific processing modes if the features provided by those modes are required by the plug-in.

12.43.3.1 Real-time processing

Real-time processing allows users to operate plug-ins in live signal paths or in complicated audio routing schemes when the future input data is not known.

Plug-ins operating in real-time are clients of the Pro Tools automation system, meaning that control movements can be dynamically recorded and played back with the audio track, written by hand onto the Pro Tools timeline for future playback, and/or edited by and broadcast to attached control surfaces.

To instantiate a plug-in for real-time processing in Pro Tools, click on an insert slot in the desired track and select the plug-in from the menu that appears

12.43.3.1.1 Native real-time processing When an AAX plug-in is run natively, all of its components and processing elements are loaded into the host environment. The host CPU handles the plug-in's real-time audio processing as well as its data model, GUI, and other tasks.

12.43.3.1.2 Virtual Instruments and MIDI effects Virtual Instruments and MIDI effects are special categories of real-time processing effects. Instead of processing audio data, these types of effects convert incoming MIDI data into output audio or MIDI data. Although a user interacts with these kinds of effects differently than with pure audio processing effects, the AAX API does not generally distinguish between them. See [Plug-ins with MIDI support](#) for more information.

12.43.3.1.3 DSP real-time processing When an AAX plug-in is run with Avid's DSP-enabled hardware, the plug-in's real-time processing code is loaded onto the external DSP device, while the remaining plug-in modules continue to be run by the host CPU. Each DSP in this system provides dedicated processing capacity that is not shared with an OS or other processes, and therefore this architecture allows users to achieve highly reliable and deterministic low-latency processing even when many DSP plug-ins are instantiated.

Figure 1: Real-time insert slots in the Pro Tools Edit and Mix windows.

12.43.3.1.4 CPU reporting To guarantee absolute reliability, AAX DSP plug-ins are required to report their worst-case performance metrics to Pro Tools. Pro Tools uses this information to ensure that each DSP in the system will be loaded with only the number of plug-ins that it can support given a worst-case processing load.

12.43.3.2 Non-real-time processing (AudioSuite)

The non-real-time AudioSuite processing mode is file-based, meaning that the results of AudioSuite processing are applied destructively to audio files (generally to new, empty files provided by Pro Tools.) AudioSuite processing can only be performed on preexisting blocks of audio.

There are two primary ways to apply AudioSuite processing in Pro Tools. The first way is to selectively apply the processing algorithm to the audio tracks and clips that are selected on the Pro Tools timeline. This is known as "destructive" processing, because the original audio track is replaced by the new processed audio track. There are no limitations governing the amount of time required to process a track in this manner.

Audio Suite plug-ins also have a second optional mode in which they can run. This is referred to as Preview mode. The Preview feature allows you to monitor the audio processing applied to a track in semi-real-time. Because this is a real-time process, it is not applicable to all types of file based processing. You may elect not to support this mode in your plug-in if its algorithm does not lend itself to real-time, linear processing. Preview mode is implemented in a non-destructive manner, as Preview mode exists for auditioning only with no actual replacement of audio data on the Pro Tools timeline.

To instantiate an AudioSuite plug-in in Pro Tools, select the plug-in from the "AudioSuite" menu in the Pro Tools application menu bar

12.43.3.3 Multichannel and Multi-Mono

Pro Tools supports various surround stem formats throughout the entire signal chain, including multi-channel processing through AAX plug-ins.

Pro Tools also allows a plug-in to function in multi-mono mode if the plug-in does not explicitly support certain channel formats. In multi-mono mode, Pro Tools instantiates a separate instance of a plug-in for every channel in the track. In this mode, plug-in controls across all channel-instances in a multi-mono collection are linked by default, though channels can be unlinked by toggling the blue link button in the plug-in header and selecting the channel whose controls you wish to modify.

For more information about multi-mono mode, please refer to the Pro Tools Reference Guide.

12.43.4 Requirements for AAX plug-in compatibility with Pro Tools

In addition to implementing the client-side AAX API, all Pro Tools plug-ins must:

1. Be installed to the AAX plug-ins directory
2. Use a valid file name
3. Be signed with a valid digital signature

Note that digital signatures for plug-ins are not required in Pro Tools Developer builds. However, you will need a Pro Tools Developer Build licence in order to run Pro Tools Developer. To obtain your licence, contact devauth@avid.com.

12.43.4.1 Install directories

AAX plug-ins must be installed in the system's AAX Plug-Ins directory. See [Building your plug-in installer](#) for more information about creating a plug-in installer.

Plug-ins that are uninstalled but still present on the system are placed into the "Plug-Ins (Unused)" directory, which is located next to the Plug-Ins directory.

Pro Tools will also search for a Plug-Ins directory next to the actual Pro Tools application, and this directory will be used if present. This debug feature can be useful for testing specific plug-ins.

12.43.4.2 Plug-in name and file structure

In order to be recognized by AAE, all AAX plug-in bundles must use the ".aaxplugin" file name suffix. On macOS, the plug-in bundle must use this suffix while the binary itself does not require a suffix. On Windows, the plug-in binary (DLL) must use this suffix.

The directory structure of an AAX plug-in bundle is also important. See [.aaxplugin Directory Structure](#) in the [AAX Format Specification](#) document for more information.

12.43.4.3 Digital signature

As an added security measure against digital piracy, all AAX plug-in binaries must be digitally signed in order to run in Pro Tools. This signature step does not interfere with your existing copy protection and licensing solutions - it is simply a build step that you incorporate into your plug-in before releasing the binary.

Digital signatures are generated based on the plug-in binary and act as a guarantee against binary modification. Therefore, any build steps that modify the binary, such as symbol stripping, must be performed prior to signature generation. Digital signatures apply to the full .aaxplugin bundle, so any operation that modifies the contents of the bundle will invalidate its digital signature even if the operation does not affect the plug-in binary itself. Therefore, the generation and application of a digital signature should be the last step in any release plug-in build process. The digital signature requirement applies to Beta and Release software. This requirement does not apply to Development builds of Pro Tools or to other developer tools which can load unsigned binaries.

Note

You will need a Pro Tools Developer Build licence in order to run the Pro Tools developer builds. To obtain your licence, see the Obtain a Pro Tools Developer iLok license (and an iLok) section in the Getting Started with AAX guide.

If you are having problems with digitally signing your plug-ins see the [Plug-In Fails to Load in Shipping Pro Tools](#) section in the [Troubleshooting](#) guide.

Requesting the digital signing toolkit

The AAX digital signatures required by Pro Tools are generated using digital signing tools licensed from PACE Anti-Piracy, Inc., which acts as the certificate authority for all AAX digital signatures. To request access to these tools, send an e-mail to audiosdk@avid.com with "Pace Tools Request" in the subject. Include the following information in your request:

- Company name
- Admin full name
- Email
- Telephone number
- iLok username

Once your request has been approved you will be contacted by PACE with further instructions for acquiring and using the digital signature toolkit.

What you will need

The digital signing toolkit which you will receive as an AAX developer will require a physical iLok USB key. You will also need a registered iLok user account which will be used when applying the digital signature. If your build toolchain requires hardware-free signing then you can contact PACE regarding their current offerings.

In order to successfully use the signing tools you should be familiar with the latest Gatekeeper and `codesign` (for Mac) and Authenticode (for Windows) digital signature schemes.

Although it is possible to use self-signed certificates for AAX digital signatures, before making your AAX plug-ins commercially available it is recommended that you acquire an Apple-issued Application Developer ID for Gatekeeper and an "Extended Verification" (EV) Authenticode certificate from a Microsoft approved certificate authority.

See the Getting Started Guide in the PACE digital signing toolkit for more information about using these tools.

Signature requirements in Pro Tools

Host Compatibility Notes Pro Tools requires PACE Eden digital signatures for AAX plug-ins.

Pro Tools and higher use the Eden toolset. This toolset integrates fully with platform-specific signatures, so you only need to do one post-build step using the Eden digital signing tools to sign your plug-in with both the Eden signature and the relevant Apple GateKeeper or Microsoft Authenticode signature. For more information, see the Eden digital signature toolkit documentation.

Pro Tools and higher will only accept the Eden signature; AAX plug-ins signed by earlier generations of PACE digital signing tools will not load in Pro Tools.

Optional Signature for Pro Tools AAX DSP binaries

Binary-level encryption can be added to AAX DSP algorithms. Please note that this is *NOT* a requirement of AAX DSP plug-ins, and serves only as an additional security measure to protect an algorithm's DLL.

For more information about signing AAX plug-ins for use with Pro Tools, please contact PACE.

Automatic signature application by PACE tools

If you already protect your plug-ins using one of the anti-piracy technologies available from PACE then you may not need to perform any additional action:

- you are wrapping using PACE InterLok MasterMaker, your binaries will be automatically signed.
- If you are using Fusion Hybrid without wrapping with MasterMaker, please carefully review the "Adding digital signature checks" section of the Fusion Hybrid manual.
- If you are using PACE APIs (like PACE Interface or CDRM) without InterLok wrapping, please see either the latest PACESigTool read me or the Fusion Hybrid manual for additional details regarding digital signing.

12.43.5 Audio Engine Behavior and Features

Pro Tools hosts AAX plug-ins using the *Avid Audio Engine* (AAE). AAE implements all host-side AAX interfaces such as [AAX_IController](#) and the [AAX_ICollection](#).

12.43.5.1 Plug-in loading and AAE initialization

When Pro Tools launches, it immediately begins loading AAE. AAE searches the system for valid AAX plug-ins, checks each plug-in's digital signature, and loads, initializes and catalogs any valid plug-in modules that it happens to find.

This initialization is performed via the plug-in's Describe implementation; once AAE loads a plug-in binary, it calls the plug-in's Describe method to retrieve (and cache) the basic configuration of the plug-in. AAE then hands this information back to Pro Tools so that Pro Tools knows what plug-ins are available and what their basic properties are. Once a complete list of plug-in descriptions has been generated, Pro Tools can show lists of available plug-ins and can construct any plug-in's individual modules.

12.43.5.1.1 Plug-in configuration cacheing AAE uses a cache to avoid loading every plug-in into memory on every launch. During initialization, AAE checks whether each plug-in has changed since the last time the plug-in's configuration information was cached. If nothing has changed, AAE relies on plug-in descriptions that it cached during the previous launch to speed through the plug-in loading process. If, however, any plug-ins have been added, removed, or updated since the last launch, AAE loads each changed plug-in and re-caches its description information.

The plug-in cache files are located at:

- Pro Tools 2023.12 and later
 - Mac: /Users/Shared/Pro Tools/AAXPlugInCache
 - Windows: C:\Users\Public\Pro Tools\AAXPlugInCache
- Pro Tools 2023.9 and earlier, and fallback for Pro Tools 2023.12
 - Mac: ~/Library/Preferences/Avid/Pro Tools/InstalledAAXPlugIns
 - Windows: USERPROFILE%\AppData\Roaming\Avid\Pro Tools\InstalledAAXPlugIns

While troubleshooting plug-in caching issues, you can delete, rename, or move the cache file for a plug-in to force Pro Tools to re-load and re-cache that plug-in on next launch.

12.43.5.1.2 Out-of-proc loading To speed up the launch process and to avoid problems that can occur when hundreds or thousands of individual plug-in binaries are loaded, AAE uses an out-of-process loading mechanism. During launch, if a plug-in's cached information needs to be updated then AAE will run a subprocess that loads the plug-in, calls its Describe routine, and builds the plug-in's cache file. If this subprocess fails then AAE will load the plug-in to generate the cache as normal.

Because a plug-in's Describe information is cached, it is very important that the configuration details for a particular plug-in binary are always consistent. A plug-in should never include conditional logic in its Describe routine that, for example, causes a stem format to be supported or not supported depending on the host version.

Similarly, a plug-in's Describe information must be consistent whether the plug-in is loaded directly into the Pro Tools application or into a subprocess. Otherwise, the plug-in's cached information may not match the actual plug-in configuration when it is loaded in Pro Tools, resulting in some unusual problems that can be difficult to debug.

If you suspect that your plug-in may contain a bug related to out-of-process loading, it can be useful to force Pro Tools to load an AAX plug-in either directly or via a subprocess to examine possible differences. This can be accomplished using DigiOptions. See [Using DigiOptions](#) for more information about available options.

It can also be useful to run the subprocess directly in order to debug the plug-in's Describe routine execution. The subprocess program is located at:

- Mac: Pro Tools.app/Contents/CommandLineTools/pluginrunner
- Windows: C:\Program Files\Avid\Pro Tools\DAEPlugInRunner.exe

The program accepts paths to plug-ins as command line arguments. It will load each plug-in in turn, if that plug-in does not already have a valid cache file.

12.43.5.2 Plug-in initialization

When a new plug-in instance is created in Pro Tools, AAE performs the following steps:

1. The plug-in's data model component is loaded
2. The default state of the plug-in is set (see [Default plug-in settings](#))
3. The plug-in's GUI and other host modules are loaded
4. The plug-in's algorithm private data state is initialized
5. The plug-in's algorithm is loaded and initialized
6. The plug-in's algorithm processing is initiated

12.43.5.3 Run-time processing behavior

The audio engine in Pro Tools includes some advanced real-time processing features that are not present in earlier versions of Pro Tools:

- When certain tracks with plug-ins have been silent for a period of time or Pro Tools is not in playback, those plug-in instances are automatically deactivated to reduce processing load on the host processor
- In certain situations such as playback or offline bounce where low latency is not required, Pro Tools may call AAX Native plug-ins with a larger buffer than normal.

This latter behavior is possible due to the fact that AAE uses two latency domains for plug-ins: a high-latency domain that operates over large block sizes and a low-latency domain that operates over small block sizes. Since processing at higher block sizes is generally more efficient, plug-in instances that are running in the high-latency domain generally consume less CPU cycles for their processing than instances that are running in the low-latency domain.

Pro Tools may swap plug-in instances back and forth between these two domains at run-time and uses a set of rules designed to optimize the system's CPU resources while at the same time providing the best and most responsive user experience in every situation. These rules are different depending on whether the system is using an HDX card as its playback engine.

Here are some of the specific rules that are followed by the current versions of Pro Tools at the time of this writing. These rules are subject to change from release to release:

- *(HDX classic and Native)* If there is any live audio or MIDI feeding into the plug-in's track and if the track is sending audio to an active output then all plug-in instances on the track will be run at low latency.
- *(HDX classic only)* If any AAX DSP plug-in instances are present in the signal path that feeds an AAX Native plug-in instance then the AAX Native plug-in will be run at low latency.
- *(HDX classic only)* Any AAX Native plug-in instance on an AUX track will be run at low latency.

For a full list of compatibility and feature differences between different AAX plug-in hosts, see [Host Support](#).

12.43.5.3.1 Deterministic Plug-in Automation Native and DSP plug-ins will receive automation changes in a deterministic manner. Each time the transport is played, automation events will be delivered to the plug-in for processing at the same moment on the timeline. Note that this does not mean automation is sample-accurate with respect to where the automation breakpoints are placed in the timeline, but rather that the timing will be the same between transport runs.

12.43.5.3.2 Offline Bounce Pro Tools supports faster-than-real-time offline bounce for all sessions. All plug-ins with AAX Native types are supported. For AAX DSP plug-ins, the offline bounce process will temporarily convert those to their corresponding AAX Native types to complete the bounce. Because offline bounce is faster-than-real-time, audio processing callbacks will occur as fast as the algorithm will allow for. For this and other reasons, your algorithms should never depend on wall-clock time for features such as LFO, delay time, etc. Instead, all algorithms should always base time calculations on sample time so that the output will still be correct even if the algorithm is being called from an offline bounce.

12.43.5.3.3 The Hybrid Engine and AAX DSP Pro Tools HDX and Pro Tools Carbon systems support the Hybrid Engine, which optimizes system latency by splitting the Pro Tools mix topology between native and DSP processors. When the Hybrid Engine is in use, Pro Tools tracks are configured as either DSP Mode or Native Mode. All AAX effect instances on a DSP Mode track are switched to their AAX DSP type. Plug-ins that do not support AAX DSP are de-activated while the track is in DSP Mode. Pro Tools HDX also supports a Classic mode. In this mode, the user chooses whether each plug-in instance will be AAX Native or AAX DSP.

12.43.5.3.4 AAX Hybrid Plug-ins Pro Tools also supports [AAX Hybrid](#) plug-ins, which can have both a Native and a DSP algorithm processing component. Audio-rate data can also be shared between the two processing components, which allows you to split your algorithm up into low-latency and high-latency contexts for better efficiency and to enable plug-ins such as convolution reverbs, spectrum analyzers, and other similar architectures.

Note

AAX Hybrid is only supported by Pro Tools HDX when running in Classic mode. See [PT-257213](#) for more information.

12.43.6 Basic plug-in operation

12.43.6.1 Configuration management

Each Effect in an AAX plug-in may contain multiple configurations. Pro Tools automatically determines the appropriate plug-in configuration for each Effect insert at run time based on the insert's required sample rate, channel width, and processing mode (Native or DSP.) Under some circumstances, the configuration requirements for an Effect insert may change at run-time. Here are some examples:

- When a width-changing plug-in (e.g. mono-to-stereo) is instantiated on a track then all of the following inserts must be converted to the new stem format
- When a user imports session data between sessions at different sample rates then all of the imported plug-ins must be converted from the old sample rate to the new sample rate
- When a user opens a session that contains deprecated effects, they must be replaced by the corresponding installed effects

Whenever a new configuration is required, Pro Tools automatically determines whether one is available that meets the new requirements and, if it is, swaps in a new plug-in instance using a copy of the previous configuration's settings.

In order for Pro Tools to deterministically select the appropriate Effect configuration to load in any given scenario, each of the configurations that are registered in the Effect must be described with distinct and mutually exclusive compatibility requirements.

12.43.6.2 Plug-in activation and deactivation

In Pro Tools, real-time plug-in inserts can be either active or inactive. Inactive plug-ins are not instantiated and are entirely removed from the processing chain, though they are still saved with the session and maintain a placeholder in their track's insert list for easy activation at a later point.

Active plug-ins may be de-activated manually by the user or automatically by Pro Tools. Plug-ins may be loaded as inactive when a plug-in that has been saved in a session has been uninstalled and is no longer available, when a required plug-in configuration is not available, or at any other time when a particular plug-in instance cannot be loaded.

12.43.6.3 Plug-in bypass

AAX plug-ins must implement a Master Bypass parameter, which is controlled via the "Bypass" button in the Pro Tools plug-in window header. While bypassed, the plug-in must not apply any processing to the audio that is passed to it (except delay, see below.) The plug-in may choose to smoothly transition into and out of bypass however it chooses.

Any algorithmic delay that a plug-in incurs during normal operation must be maintained by the plug-in during bypass. For more information about this requirement, see [Automatic Delay Compensation](#).

12.43.6.4 Presets and settings management

Pro Tools includes a plug-in preset management system that can be accessed from the plug-in window header. With this system, users can save plug-in settings to disk and load the settings later to restore the plug-in's configuration.

Preset files can be bundled with an AAX plug-in to demonstrate a variety of uses for the plug-in or as recommended settings for different situations, and, as a plug-in developer, you are encouraged to provide a large selection of pre-configured presets along with your AAX plug-ins. See [Create factory presets](#) for more information about bundling presets with your plug-in.

Aside from user preset management, there are many cases when the state of a plug-in must be captured or restored by AAE. For example, AAE must restore plug-in settings when a session is loaded and when converting a plug-in between different configurations.

12.43.6.4.1 The plug-in preset menu Plug-in presets are available to the user via the Plug-In Settings menu in the Pro Tools plug-in window header. This menu supports nesting presets into sub-folders, which provides a convenient way to categorize and organize large sets of presets. In addition, users may save custom presets and add these custom presets to the menu.

Figure 2: The Plug-In Settings menu in the Pro Tools plug-in window header

The preset menu in the Pro Tools plug-in header is built from the following two directories:

- *Session file*../Plug-In Settings
- *User Library root*/Plug-In Settings

The default setting for the User Library root directory is ~/Documents/Pro Tools on macOS, but the user can change this setting in the Pro Tools preferences.

12.43.6.4.2 Factory presets Pro Tools supports automatic installation of plug-in presets. AAX plug-ins should include a set of presets in the following directory within the .aaxplugin:

- *MyPlugIn.aaxplugin/Contents/Factory Presets/MyPlugInPackage/*

Where *MyPlugInPackage* is the plug-in's longest [Package Name](#) with 16 characters or fewer.

On Pro Tools launch, all installed AAX plug-in settings are copied from the .aaxplugin bundles' "Factory Presets" folders into the User Library directory (see [above](#).)

Note

Since the User Library root directory is a customizable setting, you should never install presets directly onto a user's system. If you require a central repository of settings on the system that is under control of your installer then you should handle these settings as external resources. You can use custom settings chunks in the plug-in's "Factory Presets" .tfx files to redirect your plug-in to read the appropriate installed resources.

12.43.6.4.3 Default plug-in settings When the first instance of an effect is made active in a session, Pro Tools queries the instance's state and stores this data as the effect's "factory default" preset. This preset is cached by Pro Tools and will be set on each subsequent instance of the plug-in with the same configuration

The plug-in's factory default settings are stored on disk in a temporary file location that is specific to the user. Pro Tools looks for the factory default settings file for a plug-in each time an instance of the plug-in goes from an inactive state to an active state, including when the instance is first created. If there is no factory default settings file on disk then Pro Tools will create it using the plug-in's current settings.

All factory default settings files are deleted during the Pro Tools shutdown procedure. Therefore, under normal operation, these files will be refreshed with each launch of Pro Tools.

Note

If the Pro Tools shutdown procedure is not completing, for example if you regularly terminate Pro Tools from a debugger, then the plug-in factory default settings files will not be deleted automatically.

When a session is loaded Pro Tools will perform the following steps on each plug-in instance:

1. Instantiate the plug-in and create a [AAX_IEffectParameters](#) object
2. Look for the cached factory default settings file in the file system
3. If the factory default settings file is not found, query the plug-in for its current settings and create the factory default settings file using these settings
4. Set the instance's default settings based on the settings stored in the cached factory default file
5. Send the instance a [AAX_eNotificationEvent_SessionBeingOpened](#) notification
6. Set the saved settings from the session

12.43.6.4.4 The Compare Light The plug-in window header in Pro Tools includes a "Compare" button, the Compare Light control. This control allows the user to compare the current state of the plug-in with the last preset that was loaded, or the plug-in's default settings if no other preset has yet been loaded.

Pro Tools polls each displayed plug-in periodically to determine whether or not its state matches the currently loaded preset. While the state matches, the Compare Light is inactive and unlit. As soon as the plug-in's state differs from the preset, the Compare Light becomes active and is highlighted.

When the Compare Light is active, the user may click on it to cache the current plug-in settings and temporarily swap in the last preset that was loaded. Clicking on the Compare Light a second time will restore the cached plug-in settings.

The specific operation of the Compare Light is determined by the plug-in's implementation of [AAX_IEffectParameters](#). To determine the correct state for a plug-in's compare light, Pro Tools makes regular calls to [AAX_IEffectParameters::GetNumberOfChannels](#) from a callback timer. If this method's `aValueP` parameter has changed since the last time the plug-in was queried then Pro Tools proceeds to call [CompareActiveChunk\(\)](#). If [CompareActiveChunk\(\)](#) returns with `isEqual==false` then the Compare Light will be lit, otherwise the light will be dimmed.

12.43.6.4.5 Basic chunk handling All of these situations use the same basic settings management infrastructure in Pro Tools, which uses the "chunk" API of [AAX_IEffectParameters](#) to retrieve arbitrary blocks of data from the plug-in (to retrieve a preset) and send the same block back to the plug-in (to set a preset.)

When retrieving a preset from a plug-in, Pro Tools first asks for the size of the plug-in's settings chunk(s). Pro Tools then provides the plug-in with a pre-allocated buffer of memory into which the plug-in may store its settings information using any format that it chooses.

When Pro Tools needs to restore the plug-in to this preset state, it sends a copy of this data back to the plug-in. The plug-in must interpret this data and set its internal state to match the preset.

12.43.6.5 Modifier key behavior

In order for users to have a consistent experience, all AAX plug-ins should provide standard modifier key behaviors as described in this section. These operations are demonstrated by all Avid plug-ins in Pro Tools, and you can experiment with Avid's AAX plug-ins to demonstrate the correct plug-in modifier key behavior.

The following modifier key combinations must be handled explicitly by the plug-in:

macOS Keys	Windows Keys	Expected Behavior
Command-click	Control-click	Adjust the parameter's value with fine control, for continuous controller widgets
Option-click	Alt-click	Return the parameter's value to default*
Shift-click	Shift-click	Link parameters across all channels, if applicable
*Set-to-default may also be handled directly by the host, depending on the host version (see below).		

In addition to these events, there are also specific behaviors which Pro Tools and other AAX hosts provide for certain key combinations in plug-in GUIs. For example, Pro Tools provides the following modifier key behavior overrides:

macOS Keys	Windows Keys	Expected Behavior
Command-Control-click Command-Right click	Control-Start-click Control-Right click	Show parameter automation lane in the Pro Tools Edit Window, if automation is enabled for the parameter
Command-Option-Control-click Command-Option-Right click	Control-Alt-Start-click Control-Alt-Right click	Activate pop-up menu for automation

Other AAX plug-in hosts implement different host-managed behavior for modifier key combinations, and additional host-managed key combinations may be added to any AAX host in the future. For example, Pro Tools adds host-managed support for setting plug-in parameters to their default values.

In order to allow the AAX host to handle these operations, a plug-in must always call the handler methods in [AAX_IViewContainer](#) before handling any mouse events in its own GUI. It is important to call these methods for *all* mouse events, in case additional handlers are added to future versions of the host or the plug-in is run in a new AAX host with a different set of handled modifier key combinations.

See the [AAX_IViewContainer](#) class documentation for more information about passing mouse events to the AAX host.

12.43.7 Optional plug-in features

Pro Tools plug-ins offer users a rich set of integrated features. To make sure your plug-ins integrate into users' expected Pro Tools workflows, where applicable you should implement all of the features presented in this chapter.

For more information about any of these features in Pro Tools, see the latest Pro Tools Reference Guide.

12.43.7.1 Audio management features

12.43.7.1.1 Sidechain input If applicable, plug-ins may choose to enable [sidechain inputs](#). If a sidechain is enabled, a menu is added to the plug-in's header that allows the user to choose an interface or bus as the sidechain, or "key input". For AudioSuite, the user can only use an existing audio track as the sidechain input. Once enabled, the plug-in will be able to access sidechain input just like any other input signal.

12.43.7.1.2 Auxiliary Output Stems Pro Tools has the capability to show and route multiple "auxiliary" outputs from a plug-in to other tracks. These are known as [Auxiliary Output Stems](#) (AOS), a stem referring to one set of outputs. A stereo stem contains two outputs, left and right, and a mono stem contains one output. The outputs will appear in the input assignment pop-up menu of each track under the category "plug-in".

Some notes regarding this feature:

- Only mono and stereo stems are available as auxiliary outputs.
- The aux outputs cannot be added and removed from the system dynamically though they can be made inactive by the user. The total number of aux outputs, stem types, names, paths, and ordering are defined only once by the plug-in.
- Plug-in aux outputs are not available from the sidechain input popup menu in other plug-ins. Users will not see the "plug-in" submenu when clicking on a plug-in sidechain popup.
- There cannot be any multi-mono multi-output plug-ins. If a mono plug-in instance offers multiple outputs it cannot support multi-mono.

If a plug-in is going to utilize the AOS feature, it will be responsible for a few details that are summarized below:

- **Aux Output Paths**

The plug-in is responsible for the definition of valid aux output paths. This definition includes the total number of outputs, the desired order of stereo and mono paths. Pro Tools will query each plug-in for available valid paths and populate its track input selector popup menus accordingly.

- **Aux Output Path Order**

The plug-in is responsible for specifying the type and name of each of its aux output paths. A plug-in decides whether the aux outputs are all stereo, all mono, "X" stereo outputs followed by "Y" mono outputs, or some other combination. Pro Tools lists each output in the order given by the plug-in. If mono and stereo paths are interleaved the input popup menu of the mono tracks keeps that order and breaks the stereo paths into their respective left and right sides using ".L" and ".R" suffixes.

- **Aux Output Names**

A plug-in is responsible for giving meaningful names to aux outputs. Names are only defined once, so they will stick. At the very least, individual outputs should be labeled "Output xx", where "xx" is the aux output number as it is defined in the plug-in. The output name should also include the words "mono" and "stereo" to support when users are looking for an output with a specific stem format.

- **Aux Output Numbering**

The plug-in is responsible for defining the lowest available aux output number. Plug-ins should base this number on the width of the plug-in's main outputs. For example, when using a stereo instance of a sampler the first aux output should be #3, when using a 5.1 instance of the sampler the first aux output should be #7, etc. This is to keep the numbering scheme inside of the plug-in and in Pro Tools consistent. From the perspective of Pro Tools, plug-ins typically enumerate all available outputs and do not differentiate between main and aux outputs. The first "N" outputs are used for the main outputs, and all the remaining outputs are available for aux output paths.

- **Separate Multi-Output Plug-in Process Type**

Plug-in developers are encouraged to offer both "regular" and "multi-output" versions/types of any multi-output capable plug-in. We strongly suggest this to conserve resources and to keep the user's workspace as uncluttered as possible. Users can choose to use the regular version/type for plug-ins they don't need aux outputs for. Multi-output versions can be created as separate process types so that there need not be separate binaries. Such additional process types will be listed in the plug-in menu next to their regular version siblings. They should be nominally distinguished by appending phrases like "multi-output" to the plug-in name, for example.

Note

When moving sessions between different PT systems, multi-output process types will NOT be automatically converted to regular process types if multi-output types are not available.

- No Multi-Mono Implementations

A plug-in is responsible for not having multi-mono enabled if it utilizes auxiliary outputs stems. Auxiliary output stems will not work in multi-mono enabled plug-ins. Multi-mono is automatically disabled for AOS in the Effect Layer.

12.43.7.1.3 External metering and internal clip Pro Tools may use the meter values reported by a plug-in for display on attached control surfaces and other external plug-in views. In general, the behavior of a plug-in meter on these devices will depend on the meter's properties as registered in Describe. The meter behavior may also depend on the plug-in's registered category. See [Plug-in meters](#) for more information about how to register your plug-in's meters.

- Gain reduction metering

Pro Tools supports gain reduction metering and will display an inverted gain reduction meter next to each plug-in insert and also next to the track's main meter in the Pro Tools Mix and Edit windows.

All registered plug-in gain reduction meters are used by the Pro Tools gain-reduction metering UI. The plug-in gain reduction meters in the Pro Tools Mix and Edit windows will combine metering data for all gain-reduction meters of the same type ([Compressor/Limiter](#) or [Expander/Gate](#)) in the plug-in:

- For plug-ins with multiple gain-reduction meters of the same type, the minimum (most gain-reduced) meter value for the current buffer will be used
- For multi-mono plug-ins, the minimum meter value across all of the per-channel mono instances will be used

Pro Tools can be set up to display [Compressor/Limiter](#), [Expander/Gate](#), or both types of metering data in these displays via Preferences > Metering > Display > Gain Reduction Meter Type. If both types are used, the displayed meter level is simply the sum of the selected values for each type.

The track gain-reduction meter displays the sum of all the track's inserts' gain reductions, using the same rules as above.

- Plug-in internal clipping

The plug-in header has a clip light that indicates that the plug-in has reported that it has clipped somewhere internally. It is up to the plug-in itself to set and clear its clip indicators as needed. Additionally, plug-ins that have clipped internally will appear in red on the insert, even if the plug-in window is not open. This allows users to see at a glance where clipping has occurred in their mix.

12.43.7.1.4 Automatic Delay Compensation Automatic Delay Compensation maintains time-alignment between tracks that have plug-ins with differing algorithmic delays, tracks with different mixing paths, tracks that are split off and recombined within the mixer, and tracks with hardware inserts. To maintain time alignment, Pro Tools adds the exact amount of delay to each track necessary to make that particular track's delay equal to the delay of the track that has the longest delay.

In order to be compensated correctly, AAX plug-ins must report any algorithmic delay that they incur. This delay may be reported in the plug-in's description, and may also be changed at run-time.

Automatic Delay Compensation Notes

- Currently, Pro Tools will not update its delay compensation settings while Pro Tools is in playback, so a plug-in that dynamically changes its delay settings at run-time should either prevent any algorithmic delay updates during playback or give a visual indication to the user when the delay that it applies and the delay that Pro Tools is compensating for different delay settings.

- Pro Tools does not update delay compensation settings when plug-ins go into and out of bypass, and does not automatically maintain bypass audio buffers for delayed plug-ins. It is therefore required that all plug-ins incur the same amount of delay when bypassed as during normal operation.
- Automatic Delay Compensation is not applied during offline (AudioSuite) processing for plug-ins which use the [Host Processor](#) interface. If your Host Processor plug-in incurs algorithmic delay then you must incorporate audio lookahead via the Host Processor interface's random timeline access API.
- Given the many routing possibilities in Pro Tools, the Automatic Delay Compensation feature involves some subtleties that may not be immediately apparent or intuitive. For more information about this feature, we strongly recommend that you review the relevant chapters in the Pro Tools Reference Guide.

12.43.7.2 Plug-in categories

The plug-in menus in Pro Tools are hierarchical and by default are organized by category. These general categories represent common plug-in functions like EQ, dynamics, reverb, etc. Plug-ins may report one or more of these categories in order to be placed into the proper menu. For a complete list of plug-in categories available, refer to the [AAX_EPlugInCategory](#) enum.

Some features, such as control surface center-section mappings, are only available to plug-ins that report a particular category, so it is important for plug-ins to report the correct set of categories.

12.43.7.3 Advanced non-real-time processing

AudioSuite processing allows AAX plug-in to operate on audio in a non-real-time manner. AudioSuite plug-ins will appear in the AudioSuite menu in Pro Tools. By default, any AAX-Native plug-in will appear in the menu as long as an [AAX_eProperty_PluginID_AudioSuite](#) property is defined alongside the corresponding [AAX_eProperty_PluginID_Native](#) ID. However, to make use of extended AudioSuite features such as non-real-time sample access, the Analysis pass, a separate entry method subclassed from the [AAX_CHostProcessor](#) implementation in the SDK should be used.

12.43.7.3.1 AudioSuite processing modes Pro Tools includes a number of different AudioSuite processing modes, each of which changes the precise behavior of an AudioSuite processing event.

Output modes

- *Overwrite files* Output audio destructively overwrites the selected audio files on disk
- *Create individual files* Individual new files are created for each processed clip
- *Create continuous file* A single new file is created with data from the full processing pass

Input modes

- *Clip by clip*
- *Entire selection*

The plug-in may optionally disable the "clip by clip" processing mode if continuous input data is required, by using the property [AAX_eProperty_ContinuousOnly](#).

Channel modes

- *Mono mode* Each selected channel is processed as an individual mono audio stream
- *Multi-input mode* Selected channels are sent to the plug-in in multi-channel streams

Multi-input mode is only valid with the "entire selection" input mode, since the "clip by clip" input mode requires that each clip be processed individually as a standalone audio channel.

The plug-in may optionally disable "mono mode" processing if its algorithm is only valid for multi-channel input, by defining the [AAX_eProperty_MultiInputModeOnly](#) property.

12.43.7.3.2 Analysis AudioSuite plug-ins support an optional Analysis pass, which allows a plug-in to access the incoming audio before the actual Render pass starts. When Analysis is defined with either [AAX_eProperty_OptionalAnalysis](#) or [AAX_eProperty_RequiresAnalysis](#), an Analyze button will appear in the plug-in footer in the GUI.

An analysis pass is useful for collecting pitch, spectrum, loudness, noise threshold, or other data that will help the user set up parameters based on the audio content being processed.

12.43.7.3.3 Reverse A "reverse" feature is available for [Reverb](#) and [Delay](#) AudioSuite plug-ins. This effect will reverse the source audio, apply the AudioSuite plug-in processing, and re-reverse the source audio back to its original orientation, thereby applying the AudioSuite effect in reverse.

Reverse replaces the Analysis pass in the Pro Tools UI, so AudioSuite plug-ins in these categories do not receive a user-triggered analysis pass.

12.43.7.3.4 Random-access and non-linear processing The generation of output samples by an AudioSuite Process must occur linearly and incrementally; however, the source of input samples may optionally be randomly accessed from the entire selected track. This enables many advanced processing capabilities such as whole-file analysis, audio reverse effects, and timeline-level modifications such as expanding, contracting, or shifting the processed region.

In order to prevent invalid data from being randomly accessed, the "overwrite files" processing mode is disabled for plug-ins that use random-access processing.

12.43.7.3.5 AudioSuite Handles By default, when processing audio segments with an AudioSuite plug-in, Pro Tools will also process an extra region before and after the selected audio. These extra regions will be trimmed out of the selected.

The reason for this feature is so that the user has some room to expand the resulting audio clip (for fades or other reasons). However, certain AudioSuite plug-ins will operate more intuitively if these handles are not processed (such as delay, reverb, loudness normalization, and other plug-in types). To disable extended handle processing, set the [AAX_eProperty_DisableHandles](#) property to true.

12.43.7.3.6 Extended features AAX-AudioSuite plug-ins also have several other optional features including custom progress dialogs, reverse mode, and side-chains. For a complete reference of supported AudioSuite-related properties, refer to the properties between [AAX_eProperty_AudiosuitePropsBase](#) and [AAX_eProperty_MaxASProp](#) found in `Interfaces\AAX_Properties.h`.

12.43.7.4 Clip drag and drop

Pro Tools supports drag and drop of audio clips from the Edit Window into AAX plug-ins. When a clip is dragged and dropped into an AAX plug-in window, it will be committed with any Clip Gain, Clip Effects, Elastic Audio, ARA, and track insert processing applied, and a new file will be created in the Rendered Files folder within the session folder.

The drop operation is performed via OS APIs and the drag and drop operation will be received by the plug-in as a normal audio file drag and drop event using the newly rendered file as its target.

Clip drag and drop is supported starting in Pro Tools 2024.10, with support for drag and drop of clips from mono and stereo audio tracks

12.43.7.4.1 Implementation notes for Mac On Mac, Pro Tools generates an audio file drag and drop event after the mouse button is released. This allows Pro Tools to render the audio clip only when the drop event is actually executed, not before. As a side effect of this approach, there is no indication in the plug-in GUI itself (e.g. highlight) that a clip can be dropped. In fact, your plug-in GUI code will not observe the drag and drop event until the mouse button is released. Once the mouse button is released, the plugin GUI views will receive a sequence of messages representing the drag and drop of the rendered audio file.

12.43.7.4.2 Implementation notes for Windows On Windows, the plug-in will receive notifications of the drag and drop event through the `IDropTarget` interface before the mouse button is released. The target file name will be provided, although the file does not yet exist. If your plug-in attempts to read the file on disk during a drag and drop operation before the actual drop occurs then you should include error handling in the event that the file cannot be read. The plug-in should allow the operation to proceed until the drop action.

We recommend that plug-ins call `IDataObject::QueryGetData()` during the drag operation, not `GetData()`, until the actual contents of the data object payload are required. This is usually not until the actual drop (`IDropTarget::Drop()`). Regardless, the AAX drag and drop support is implemented to handle cases when plug-ins use `GetData()` prior to the actual drop event to determine drop compatibility, although in this case the data that is provided references an audio file that does not yet exist.

To create the target file, the plugin must call `IDataObject::GetData()` from within, or after, `IDropTarget::Drop()` in order to signal to Pro Tools that the file data will be used as part of the drop event. This will trigger the file render.

12.43.8 Using the Pro Tools Scripting SDK with AAX

The Pro Tools Scripting SDK provides a way to control various aspects of Pro Tools. AAX plug-ins may incorporate the Pro Tools Scripting SDK in order to control Pro Tools in ways that are not possible through AAX alone.

The Pro Tools Scripting SDK requires a PTSL connection to be established with Pro Tools. When establishing a PTSL connection from within an AAX plug-in, the connection request must be made using a non-main application thread. Spawn a new thread from within your AAX plug-in to perform all PTSL connection requests and Pro Tools Scripting SDK commands.

12.43.9 Plug-ins with MIDI support

In most cases, users are able to route MIDI arbitrarily in Pro Tools. Plug-in MIDI outputs and inputs are available as MIDI sources and destinations and are available at other routing points such as MIDI track input selectors. AAX plug-ins with MIDI inputs and outputs can be instantiated just like other real-time effects. AAX MIDI processing and audio processing are not mutually exclusive.

For more information about MIDI in AAX, see [MIDI](#)

Pro Tools includes special handling for two common categories of plug-ins that process MIDI data: Virtual Instruments and MIDI effects

- Virtual Instruments convert MIDI data into audio data. To define a Virtual Instrument, include [AAX_ePlugInCategory_SWGenerators](#) in the effect's category property.
- MIDI effects manipulate MIDI data in real time. To define an AAX MIDI effect, include [AAX_EPlugInCategory_MIDIEffect](#) in the effect's category property. See below for more information about MIDI effects.

Pro Tools supports MIDI processing in real time effects only. Virtual Instruments and MIDI effects should not register as AudioSuite plug-ins.

12.43.9.1 Instrument tracks

Pro Tools Instrument tracks apply some special rules for MIDI connections. An Instrument track's MIDI input and MIDI playlist are connected to the first MIDI input node of the first Virtual Instrument insert on the track. Instrument tracks also connect chains of MIDI effects together, and include special features for managing inputs and outputs of these chains.

Because inserts on Instrument tracks process both audio and MIDI, the audio format of Virtual Instrument and other effects on Instrument tracks will determine whether the effect is shown in the track's insert lists. Virtual Instruments should register component descriptions for all supported multichannel formats. This will ensure that the instrument shows in the MIDI plug-ins menu on all Instrument tracks regardless of the track's audio format.

For more information about Pro Tools Instrument tracks, Virtual Instruments, and MIDI effect chaining, see the Pro Tools Reference Guide.

12.43.9.2 MIDI effects and MIDI insert chains

By manipulating MIDI data in real time as it flows through a processing chain, MIDI Effects allow users to apply insert effects to MIDI data just like adding effects to audio data.

To function as a MIDI effect, an effect should define both a MIDI input port and a MIDI output port. When chaining MIDI effects on an Instrument track, Pro Tools will connect incoming MIDI to the first channel of the plug-in's first MIDI input node and will connect the first channel of the plug-in's first MIDI output node to the next effect in the chain.

MIDI effects that use the [AAX_EPlugInCategory_MIDIEffect](#) category will appear in a MIDI plug-ins menu that is separate from the other real-time insert lists on Instrument tracks. This category can also trigger other special handling of the effect in Pro Tools, such as disabling dynamic plug-in processing.

Because MIDI effects are also standard AAX inserts, it is possible to process both audio and MIDI. Multi-mono processing is not allowed for MIDI effects since there is only one MIDI channel to process even on multi-channel Instrument tracks. Therefore, even if a MIDI effect's audio processing is only a pass-through, the effect should register component descriptions for all multichannel formats. This will ensure that the product shows in the MIDI plug-ins menu on all Instrument tracks regardless of the track's audio format.

12.43.10 Debugging AAX plug-ins

12.43.10.1 Debugging within Pro Tools

Shipping versions of Pro Tools do not support attaching a debugger. This is to prevent malicious users from compromising Pro Tools security as well as the security of third-party plug-ins.

As an AAX plug-in developer, you are granted access to debuggable "developer build" versions of Pro Tools to help your development efforts. Some Pro Tools developer builds are feature-limited; for example, developer builds of Pro Tools do not allow saving or exporting sessions.

The easiest way to debug plug-ins within Pro Tools is to start up Pro Tools, open a session, attach your debugger, and then instantiate your plug-in. This order seems to work the best for most users. If you need to debug the initial host query of your plug-in at Pro Tools start, it is possible to launch Pro Tools from within your debugger. However, this method is sometimes known to cause problems with certain debuggers.

AAX plug-in developers are also able to download pre-release and beta versions of upcoming Pro Tools releases. For now, these pre-release versions are not debuggable, but that is expected to change in the future as we work to make a unified debuggable pre-release installer available for upcoming Pro Tools versions.

Both debuggable and pre-release versions are available for download as part of the AAX SDK Toolkit on the [My Toolkits and Downloads](#) page at [avid.com](#). In order to use developer and pre-release builds, you will need special licences which you must request from devauth@avid.com.

12.43.10.2 DigiShell

DigiShell is a software tool that provides a general framework for running tests on Avid audio hardware. As a command-line application, DigiShell may be driven as part of a standard, automated test suite for maximum test coverage. The latest DigiShell tools may be downloaded as part of the AAX SDK Toolkit on the [My Toolkits and Downloads](#) page at [avid.com](#).

More information about DSH in general and about loading and testing plug-ins in DSH can be found in [DSH Guide](#).

12.43.10.3 DigiTrace

All Avid AAX hosts provide tracing functionality based on Avid's DigiTrace tool. DigiTrace is a library that provides high-performance logging and tracing capabilities to Pro Tools and its components, including plug-ins. More information about DigiTrace can be found on Avid's audio developer website.

To enable trace logging for AAX plug-ins, use the `AAX_TRACE` macro defined in `AAX_Assert.h`. A separate macro, `AAX_ASSERT`, is also available to signal run-time errors. These macros are both cross-platform and will function whether the algorithm is running on the TI or on the host.

For more information about DigiTrace, see [DigiTrace Guide](#).

12.43.10.3.1 Tracing requirements The `AAX_ASSERT` and `AAX_TRACE` macros are debug-only and will not provide tracing output from release builds of your plug-in. `AAX_TRACE_RELEASE` may be used for tracing in both debug and release configurations. These macros require that the `DTF_AAXPLUGINS` facility to your DigiTrace configuration file. You can toggle this facility to enable or disable AAX algorithm-level tracing. For details on setting up tracing on AAX TI plug-ins, please refer to the [HDX DSP Guide](#).

12.43.11 Troubleshooting common AAX plug-in failures

Pro Tools presents a "Move Unauthorized Plug-ins" dialog after attempting to launch with my plug-ins installed, and the plug-ins do not appear in the Pro Tools insert menus

- This error indicates that Pro Tools was not able to load the plug-in binary for some reason. The error indicates a copy protection failure since that is by far the most common reason for users to encounter this kind of error in released plug-ins, but any other error that prevents the plug-in DLL from loading in Pro Tools may also cause this error message.

This error does *not* indicate a failure when checking the plug-in's digital signature. A digital signature failure would generate a different error message that would specifically mention the plug-in's signature.

The `DTF_AAXHOST` [DigiTrace](#) facility provides additional information about AAX plug-in load errors.

One common cause of DLL loading failures is a failure to dynamically link to other required libraries. In this case, the `DTF_AAXHOST` tracing will indicate something like the following:

```
- AAXH_CEffectFactory::ParseDLL - exception thrown(The specified module could not be found. (126) while
```

This exception indicates that some DLL upon which the plug-in depends is not present in the system. This is most commonly due to dynamic linking to CRT libs, but it could also be caused by any other DLL dependency.

12.43.12 Using DigiOptions

DigiOptions provide a way to override the standard Pro Tools configuration. These options are designed to assist with testing and development of Pro Tools, and they are often useful during plug-in development as well.

To configure DigiOptions, place a plain-text file named DigiOptionsFile.txt next to the Pro Tools application. On macOS, place this file next to the Pro Tools.app bundle and on Windows place it next to the Pro Tools executable.

A red notice will appear in the Pro Tools splash screen and in the application About Box indicating when DigiOptions are enabled in the build.

Figure 3: DigiOption activation notice in the Pro Tools splash screen.

Note

If suffixes are hidden on your OS then be careful that you do not accidentally give the file an incorrect name such as DigiOptionsFile.txt.txt.

12.43.12.1 Useful DigiOptions

Note

Support for these options may vary from release to release

- `AlwaysRebuildCache 1`

This option forces Pro Tools to re-load all installed plug-ins each time the application is launched. This can be very useful during development, since some plug-in updates during development will not result in an updated plug-in binary or an updated modification date for the top-level .aaxplugin bundle.

Note

If you have changed your plug-in's ID values during development and if you encounter AAE error -20038 when your plug-in is loaded then Pro Tools may be using a cache of the outdated plug-in ID. Use the AlwaysRebuildCache DigiOption or launch Pro Tools once without your plug-in installed to clear this state.

- `OutOfProcPlugInCache 0`

When combined with a reset of the [plug-in cache](#) for a particular plug-in, this option will ensure that the plug-in is loaded by the main Pro Tools process, not by a subprocess.

- `OutOfProcPlugInCache 1`
`SecondsToWaitForOutOfProcPlugInCache 30`

When combined with a reset of the [plug-in cache](#) for a particular plug-in, these options will ensure that the plug-in is loaded by the subprocess, not by the main Pro Tools process.

- `NeverUnloadPlugInBundles <int>`

Enable plug-in bundle unloading. The default value for this option is 0. In order to test your plug-ins and make sure that they operate correctly this option must be set to 0.

- `LogInterruptDataEveryNSeconds <int>`
`LogInterruptDataEveryNSeconds_HL <int>`

These options enable regular DigiTrace performance logging from the low-latency and high-latency real-time audio render threads in the Avid Audio Engine. For example, `LogInterruptDataEveryNSeconds↵_HL 2` would trigger a performance log for the high-latency render thread every two seconds. For more information about performance logging in AAE see [Real-time AAE performance logging with DigiTrace](#).

- `PauseDuringLaunchToAttachDebugger 1`

- `DisplayHostPlugInLatency 1`

Display information about the plug-in's processing domain and dynamic processing status in the Pro Tools plug-in window header.

Figure 4: `DisplayHostPlugInLatency` info in plug-in header.

- `TestGetCurveData <0, 1, 2>`

Display the plug-in's curve data as a plot in the Pro Tools plug-in window header. Possible values are:

1. Off
2. [EQ curve](#)
3. [Gain reduction curve](#)

Warning

The implementations of this test curve and the actual curve data shown on Avid control surfaces are different. In particular, the display in Pro Tools is updated regularly at idle time, whereas the curve on a control surface is only updated when certain parameter changes occur (see [PTSW-195316 / PT-218485](#)). The graph point interpolation, range, and sample points are also not exactly equivalent to the values used on an actual control surface, so you should not expect the curve shown in the debug view in Pro Tools to exactly match what would appear to users. After performing early prototyping of your curve data using the `TestGetCurveData` DigiOption you are strongly encouraged to use either the S6 Sur-fulator software or the [Pro Tools | Control](#) app to test and refine the plug-in's curve data in a real-world environment.

For more information about graph curves see [EQ and Dynamics Curve Displays](#).

Figure 5: `TestGetCurveData` EQ graph in plug-in header.

- `RenderMissingFilesAsBlank 1`

This option may be used to automatically render test tones into audio clips with missing source media. The test tones are rendered with different frequencies and magnitudes. This option can be useful when troubleshooting using a session file provided by an end user, since session-specific issues are rarely dependent on the source media, but may depend on there being some signal present in the session. This option requires that the Avid Signal Generator plug-in is installed.

- `44100_Rate <int>`
`48000_Rate <int>`

Set the new base sample rate for each set of sample rate multiples. Can be useful for simulating sample rate pull-up by up to +5% (e.g. `44100_Rate 45000` in `DigiOptions.txt`.) The `44100_Rate` option affects 44100, 88200, and 176400 Hz rates, while the `48000_Rate` option affects 48000, 96000, and 192000 Hz rates.

- `EnablePlugInHotSwap 1`

Note

This option is currently non-functional for AAX plug-ins Pro Tools. See [PTSW-188653 / PT-218451](#)

This option will allow Pro Tools to recognize changes to your plug-in during run-time. This allows you to re-compile and load your updated plug-in without re-launching Pro Tools. The following conditions must be true in order to enable hot-swapping between versions of your plug-in:

- There cannot be any instances of the plug-in currently in Pro Tools.
- Both `EnablePlugInHotSwap 1` and `AlwaysRebuildCache 1` must be set.

- `WinDLLErrorMode <int>`

Set the Windows error mode during DLL loading and unloading. The value of this option will be set as the `uMode` for a call to the `SetErrorMode` Windows API during DLL loading and unloading.

This option can be useful when debugging plug-in load errors on Windows, for example errors that cause a "The following Plug-Ins failed to load because no valid authorization could be found" dialog to appear during Pro Tools launch.

- `DisableCMNAssert 1`

Disable assert dialogs in Pro Tools. Use this option if your Pro Tools debugging sessions are being interrupted or terminated due to assert failures in the app.

Note that Pro Tools asserts may be triggered by your plug-ins; you should always investigate any assert that you see to determine whether it is being caused by a plug-in. If you need information about any Pro Tools asserts, post a question here on the AAX developer forums or write to us at [avid.developer.↵services@avid.com](mailto:avid.developer.services@avid.com) and we will be happy to help.

- `TestPlugInDescriptions 0`

Use this DigiOption to toggle the plug-in description validation check in Pro Tools developer builds. Developer builds will check plug-in description information when the plug-in is loaded and will present a warning dialog if the check fails. See [Describe Validation](#) section in the [Description callback](#) page for more information.

- `RealTimeDenormalsAreZero <int>`

Use this DigiOption to toggle the default denormal handling policy of the AAE real-time processing threads. A value of 1 means that DAZ+FZ will be enabled for all AAX real-time processing threads unless explicitly changed using thread-specific primitives, while a value of 0 means that DAZ+FZ will be disabled unless explicitly changed.

The default state of the DAZ+FZ flags for AAE real-time processing threads is turned on by default.

- `DigiTraceWindow 1`

Enable the Console window in Pro Tools which displays the application's [DigiTrace](#) output in a live stream.

Host Compatibility Notes Supported in Pro Tools 2019.XX and higher. Also supported (and enabled by default) in Pro Tools developer builds beginning with Pro Tools 2019.6.

12.43.13 Compatibility Notes

See [Host Support](#) and [Known Issues in Pro Tools](#) for additional details regarding Pro Tools compatibility

Collaboration diagram for Pro Tools Guide:

12.44 Media Composer Guide

Details about using AAX plug-ins in Media Composer.

12.44.1 Contents

- [About this document](#)
- [Processing modes](#)
- [Compatibility requirements](#)
- [AAX feature support in Media Composer](#)
- [Additional Information](#)

12.44.2 About this document

This guide discusses specific details related to using AAX plug-ins with Media Composer, such as loading and initialization procedures, GUI hosting, and other application-specific features.

For more information about the features, functionality, and use of Media Composer see the Media Composer user documentation.

12.44.3 Processing modes

Media Composer supports AAX plug-ins in two processing modes: AudioSuite and AAX Native

- AudioSuite plug-ins perform non-real-time, random-access, file-based processing entirely on the host CPU.
- AAX Native plug-ins perform real-time, linear, non-destructive processing entirely on the host CPU.

AAX plug-in processing in Media Composer is managed by specific Tools. Each of these Tools can be accessed using the "Tools" menu in the Media Composer application.

12.44.3.1 Non-real-time processing (AudioSuite)

Use the AudioSuite Tool to perform AudioSuite processing in Media Composer. The AudioSuite Tool applies an effect to a clip in the timeline of the record monitor.

Specific AudioSuite plug-ins appear in the Plug-In Selection menu in the AudioSuite window.

Note

Unlike Pro Tools, the effect to clip relationship is remembered along with the effect parameters used. Parameters to the effects can be changed at a later time, and at any time the effect can be re-rendered with the saved effect parameters. Therefore it is very important for AudioSuite plug-ins to maintain compatibility between instances, versions, and systems in order to function properly in Media Composer workflows. See [Preset management](#) for more information.

Media Composer supports two AudioSuite processing modes:

- Apply a plug-in to a clip in the Timeline. This method creates a rendered effect.
- Use the controls in the AudioSuite window to create a new master clip. This method lets you process more than one channel at a time and to create new media with a duration longer or shorter than the source media.

By default, the AudioSuite window displays the controls for applying a plug-in to a clip in the Timeline. When you drag a master clip into the window, the window expands to display additional parameters for working with master clips.

12.44.3.1.1 Applying an AudioSuite Plug-in to a Clip in the Timeline The following illustration shows the default layout of the AudioSuite window:

Figure 1: The AudioSuite window

To apply an AudioSuite plug-in to a clip in the Timeline:

1. Open the AudioSuite window by doing one of the following:
 - Select Tools > AudioSuite
 - If an audio tool is already open, click the Effect Mode Selector menu and select AudioSuite
2. Use the Track Selection Menu button to select the tracks that you want to modify.
 - When you select an item from this menu, the system selects or deselects the corresponding track in the Timeline
 - To select multiple tracks, press the Shift key while you select additional tracks from the Track Selection menu. Plus signs (+) mark the additional tracks and indicate that the effect is applied to more than one track.
3. Click the Plug-In Selection menu, and select a plug-in
4. Click the Activate Current Plug-In button. This opens a dialog box associated with the plug-in.

From the AudioSuite dialog box, you may make any necessary adjustments to the plug-in and Preview the effect in real-time.

- To save the effect, click OK
- To close the dialog box without saving the effect, click Cancel
- To save the effect as a template, drag the effect icon to a bin

12.44.3.1.2 AudioSuite Master Clip Mode Drag a Master Clip into the AudioSuite Tool to engage AudioSuite Master Clip Mode. This mode supports all AudioSuite effects, including those that change the width or length of the effected clip. A new Master Clip is generated for each AudioSuite processing pass applied in this mode.

In Master Clip Mode, the AudioSuite window will be expanded to display additional controls. You can also click the Display/Hide Master Clip Controls button to display or hide the additional parameters.

The following operations are available in Master Clip Mode:

- Apply AudioSuite plug-ins to more than one track at the same time. For example, a plug-in might let you process two separate tracks as a stereo pair. This enables you to use plug-ins that perform linked compression, reverb, and other effects that allow multichannel input.
- Create new media with a longer or shorter duration than the source media. This lets you use effects that perform time compression and expansion. For example, you can use a Time Compression Expansion plug-in to change the length of the audio file, or you can lengthen the file in order to add a reverb trail.
- Apply one mono AudioSuite effect to multiple inputs of a master clip in a multiple-mono fashion.

For more information about processing in Master Clip Mode, see the Media Composer user documentation.

12.44.3.1.3 Restrictions on AudioSuite processing

- Media Composer does not support width-changing AudioSuite effects except in [Master Clip Mode](#). See [Processing configurations](#) for more information about supported stem formats in Media Composer.
- AudioSuite effects that change the clip length should only be used in [Master Clip Mode](#), because consolidated sequences will not consolidate the correct media length.

12.44.3.2 Real-time processing

Use the Audio Track Effect Tool to perform real-time processing in Media Composer. Audio Track Effects appear in the Audio tab of the Effect Palette, as well as in the menus of the Audio Track inserts in the Audio Mixer Window and the Timeline Track Control Panel.

Real-time AAX processing in Media Composer is analogous to the track inserts feature in Pro Tools. For more information about track inserts in Pro Tools, see the [Real-time processing](#) section in the [Pro Tools Guide](#).

12.44.3.2.1 Creating and accessing real-time plug-in instances To insert a plug-in effect on a track in Media Composer, select the track where you want to apply the effect, which insert location you want to use on the track, and the specific effect you want to add to your sequence.

You can also insert a plug-in track effect by dragging an Audio Track Effect template from a bin to your sequence.

To insert an Audio Track Effect plug-in from the Timeline Right-click the Record Track button or the Track Control panel for the track where you want to apply the insert and select AAX Effects *[track number]* > Insert *[a-e]* > *[insert]*.

To insert an Audio Track Effect plug-in using the insert button

1. Click an Audio Effect insert button in the Track Control panel for the track where you want to apply the insert. This opens the Audio Track Effect tool.
2. Click the Select Effect button, and select an Audio Track Effect plug-in effect. Figure 1: Select an insert in the Audio Track Effect Tool

To insert an plug-in using the Effect Palette

1. In the Project window, click the Effects tab. This opens the Effect Palette. Figure 2: The Effect Palette
2. Click the Audio tab.
3. Click an effect category, select the effect you want, and drag it to the segment or to the Audio Track Effect insert button where you want to apply the insert. This opens the Select Insert dialog box. Figure 3: The Select Insert dialog box

Note

You can only insert mono effects on a mono track, stereo effects on a stereo track, and surround sound effects on a surround sound track.

4. Do one of the following:

- If you want to add a new insert, click an [Empty] insert button.
- If you want to replace an existing insert, click the appropriate insert button.

The plug-in effect is inserted in the track to which you dragged the effect icon.

To edit an existing Audio Track Effect Plug-In After you insert an Audio Track Effect plug-in on an audio track, you can access the plug-in controls by using the Track Control panel or the Audio Track Effect tool.

Figure 4: Audio Track Effect plug-in inserts in the Track Control panel Figure 5: Audio Track Effect tool: Select Track, Select Insert, and Select Effect buttons (left), Bypass button (center), and Save Effect button (right)

When you select an insert button in the Track Control panel or an effect in the Audio Track Effect tool, the controls for the plug-in appear in the Audio Track Effect tool window.

Figure 6: The Compressor/Limiter Dyn 3 plug-in window displayed in the Audio Track Effect tool dialog box

You can also open the tool by selecting Tools > Audio Track Effect Tool or right-clicking the Record Track button for the track where you want to edit an insert and selecting Audio Track Effect tool. You can use the buttons in the tool to select a specific insert to edit.

To save changes to a plug-in's settings, do one of the following:

- Click the Save Effect icon in the Audio Track Effect tool
- Close the Audio Track Effect tool

12.44.3.2.2 Using Audio Track Effect Templates If you apply an Audio Track effect and make a set of adjustments to it, you can quickly recreate the same sound on other tracks in your sequence or project. You can save an Audio Track effect with its parameter settings to a bin as an effect template. You can then apply the template to other audio tracks at any time.

You can apply an Audio Track effect template with all its parameters directly to an Audio Track Effect insert button in the Track Selection panel or to clips in the Timeline.

To save an Audio Track Effect as a template Do one of the following:

- Click the Save Effect button in the Audio Track Effect tool and drag it to a bin
- Click an Audio Track Effect button and drag it to a bin

A new track effect template appears in the bin, containing the parameter setting information for the effect. The new effect template is identified in the bin by an effect icon. By default, your Avid editing application names the template by the plug-in name.

To apply an Audio Track Effect template to an audio track Do one of the following:

- Drag the Audio Track Effect template from the bin to an insert button in the Track Selection panel
- Drag the Audio Track Effect template from the bin to a segment on the track where you want to apply the effect. The Select Insert dialog box opens so you can select the insert where you want to apply the effect.

This applies the effect to the track.

12.44.4 Compatibility requirements

Media Composer supports 64-bit AAX Native plug-ins beginning in Media Composer 8.1. There are no Media Composer versions that support 32-bit AAX plug-ins, and Media Composer does not currently support AAX DSP plug-ins.

In addition to implementing the client-side AAX API for a supported platform, Media Composer AAX plug-ins must:

1. Be installed to the AAX plug-ins directory
2. Use a valid file name

12.44.4.1 Install directories

AAX plug-ins must be installed in the system's AAX Plug-Ins directory. See [Building your plug-in installer](#) for more information about creating a plug-in installer.

Host Compatibility Notes Some early versions of Media Composer 8 do not search the system plug-ins directory recursively. If your plug-ins are installed into a sub-directory beneath this main directory then they will not be loaded by the affected versions of Media Composer.

Plug-ins that are uninstalled but still present on the system are placed into the "Plug-Ins (Unused)" directory, which is located next to the Plug-Ins directory.

Media Composer will also search for a Plug-Ins directory next to the actual Media Composer application, and this directory will be used if present. This debug feature can be useful for testing specific plug-ins.

12.44.4.2 Plug-in name and file structure

In order to be recognized by AAE, all AAX plug-in bundles must use the ".aaxplugin" file name suffix. On macOS, the plug-in bundle must use this suffix while the binary itself does not require a suffix. On Windows, the plug-in binary (DLL) must use this suffix.

The directory structure of an AAX plug-in bundle is also important. See [.aaxplugin Directory Structure](#) in the [AAX Format Specification](#) document for more information.

12.44.5 AAX feature support in Media Composer

Media Composer supports many of the same AAX features as Pro Tools. However, some features are not available in Media Composer, and other features are managed differently between the two applications. This section describes how Media Composer handles various optional AAX features.

12.44.5.1 Processing configurations

Sample rates Media Composer operates at sample rates of 32000, 44100, 48000, 88200, 96000 Hz, as well as each rate's film pulldown version scaled by a ratio of 1000/1001: approximately 31968, 40959, 47952, 88111, 95904 Hz.

Note

The AAX API does not currently provide a selector for 32 kHz sample rate support

Track formats Media Composer supports only four track formats:

- Mono
- Stereo (interleaved L/R)
- 5.1 surround in Pro Tools order (L, C, R, Ls, Rs, Lfe)
- 7.1 surround in Pro Tools order (L, C, R, Lss, Rss, Lsr, Lsr, Lfe)

Effects will only see these track formats on input.

Note

Plug-ins that support width-changing configurations between supported and unsupported track formats are not compatible with Media Composer

Channel ordering for plug-ins in Media Composer is identical to the channel ordering in Pro Tools. The channel ordering presentation to users may vary from the channel ordering that is used when sending audio buffers to Pro Tools; Media Composer re-orders channels to Pro Tools order prior to presenting the audio to the effect.

12.44.5.2 Preset management

Media Composer stores plug-in presets in several locations within the app. Presets may be stored and accessed through the following workflows:

- Presets can be stored in Media Composer bins by dragging the pink effect icon from the top of the effect editor window into a bin window.
- Track effect presets are stored with their tracks in the sequence
- AudioSuite presets are stored with their audio clips in the sequence

The storage of AudioSuite presets with clips in Media Composer is very different from Pro Tools. To ensure compatibility with Media Composer, it is very important that any AudioSuite effect can be re-rendered from the source media at any time.

12.44.5.2.1 Plug-in preset compatibility and persistence It is always important to design AAX plug-in preset data in a way that will be compatible across different systems and at different points in time. This is particularly true when designing an AAX plug-in to be compatible with Media Composer.

Media Composer sequences carrying presets can be exported as AAF, and these sequences may be moved freely between Media Composer systems on different operating systems and platforms. Therefore, it is important that plug-in preset data is not platform specific. A plug-in loaded in any given Media Composer system must be able to successfully read, parse, and apply preset information that was created on a different system.

Presets also persist for a long time in sequences, so preset information should be formatted in a way that newer plug-ins can read older version's data, and older versions can read newer version's data.

In addition, Media Composer 8.4 and higher can access factory presets and user-created presets interoperably with Pro Tools. A user can save a preset in one application, and access it in the other.

These preset compatibility considerations also apply to plug-ins carried over from legacy plug-in formats such as TDM/RTAS. Media Composer 8.1 and higher (with 64-bit AAX support) will match plug-in IDs when loading sequence data saved with Media Composer 7 and below, which use older plug-in formats. The same system is used for matching plug-in IDs when moving presets between different versions of Pro Tools, and between Pro Tools and Media Composer: in all cases, a preset saved for a particular plug-in ID must be compatible with all other plug-ins that use that ID, regardless of the plug-in format.

12.44.5.2.2 Plug-in preset data comparison Media Composer's rendered AudioSuite effect feature relies on a comparison of plug-in settings chunk data. Unlike in Pro Tools, this operation uses direct data comparison rather than [AAX_IEffectParameters::CompareActiveChunk\(\)](#). Therefore, Media Composer compatibility and proper operation of AudioSuite rendering in Media Composer depends on the plug-in having fully consistent AAX preset contents from one run to the next.

Two specific areas where problems can occur are:

- Uninitialized memory in the preset chunk data
- Floating point values in the preset chunk data

Both of these can result in differences between settings chunks representing the same plug-in state, which causes Media Composer to perpetually re-render the plug-in.

The problem of uninitialized memory is obvious. Given a particular plug-in state, Media Composer expects that any retrieved settings chunk will contain matching data regardless of when the chunk is retrieved. When the chunk contains uninitialized data this data does not match between different retrieved chunks. The fix, of course, is to make sure the entire chunk is initialized, for example by setting the entire chunk to zeroes before filling in the data.

The problem of floating point values is more subtle. Depending on the plug-in's parameter implementation, floating point values may be slightly different in the lowest-order bits when set onto the plug-in as part of an incoming chunk and when subsequently read out. When this occurs, Media Composer sees a mismatch in the chunk data, which causes the AudioSuite plug-in to unexpectedly be seen as requiring a new render.

AAX plug-in developers will need to avoid both of these conditions in order to maintain compatibility with Media Composer's AudioSuite effect rendering model.

12.44.5.3 Unsupported features

The following AAX features are not supported by Media Composer. Plug-ins that require these features will not be compatible with Media Composer. If your plug-ins use these features for advanced functionality but not for basic operation then you should document this restriction for Media Composer users.

- Advanced audio routing Media Composer has a simplified audio topology with only tracks and a single master fader. There are no side chains, no busses, and no submasters. As a result, Media Composer does not support extended routing options such as [Sidechain Inputs](#) or [Auxiliary Output Stems](#)
- Transport interface Media Composer does not fully support the [AAX_ITransport](#) interface. In addition, early versions of Media Composer 8 that do not support this interface at all may incorrectly return [AAX_SUCCESS](#) to method calls on this interface. More recent versions of Media Composer will either provide valid information or return [AAX_ERROR_UNIMPLEMENTED](#).
- MIDI Media Composer does not support MIDI routing to and from plug-in instances, and no [AAX MIDI features](#) are supported by Media Composer.
- External control surfaces Although Media Composer does support external control surfaces for some editing functions, it is not currently possible to control plug-in parameters using external control surface hardware in Media Composer.

12.44.5.4 Additional feature support notes

- [AAX](#) plug-in notification support varies between Media Composer and Pro Tools. Media Composer does not support the following notifications, and may not support additional notifications as well:
 - [AAX_eNotificationEvent_ASProcessingState](#)
 - [AAX_eNotificationEvent_ASPreviewState](#).
 - [AAX_eNotificationEvent_SessionBeingOpened](#)
- Media Composer does not support AudioSuite rendering to a separate track (see [AAX_eProperty_DestinationTrack](#))

12.44.6 Additional Information

12.44.6.1 Audio Engine features and behavior

Media Composer shares the same audio engine as Pro Tools (AAE) and both applications share the same advanced audio processing features. However, some aspects of plug-in operation are different between the two apps.

Here are some important notes regarding how Media Composer handles plug-in instances within the audio engine:

- Media Composer only runs plug-ins when Media Composer is playing. Unlike Pro Tools, Plug-ins stop processing when Media Composer stops playing.
- Media Composer buffer sizes are always 1024 samples, and execution is not strictly linked to real-time. Processing is generally between four and eight frames ahead of when the audio is heard.
- Media Composer will render, mix down, and export real-time effects as fast as the processor will allow, typically much faster than real-time, so be careful of introducing real-time dependencies.
- Media Composer has a background render capability, so you cannot expect the GUI to be available, or even be possible on the system performing the render.
- Plug-ins are frequently disposed and re-created on their preset data. This happens with every edit that changes the number, length, or position of playable clips in the timeline.

For more detailed information about how AAE handles plug-in loading and processing, see [Audio Engine Behavior and Features](#) in the [Pro Tools Guide](#).

12.44.6.2 Debugging AAX plug-ins in Media Composer

Media Composer does not support attaching a debugger in order to debug plug-ins while they are loaded within the app. In addition, Avid does not currently provide debuggable "developer build" versions of Media Composer. You must therefore rely on logging information for debugging your plug-ins in Media Composer, or debug your plug-ins using other AAX hosts such as a Pro Tools development build or the DigiShell command-line environment.

For more information about debugging in Pro Tools and DigiShell, see [Debugging AAX plug-ins](#) in the [Pro Tools Guide](#).

Collaboration diagram for Media Composer Guide:

12.45 HDX DSP Guide

How to write AAX plug-ins for Avid's TI DSP-based platforms.

12.45.1 Contents

- [Overview of TI DSP Algorithms in AAX](#)
- [Getting Started with HDX DSP](#)
- [The HDX DSP Platform](#)
- [Requirements for HDX DSP Plug-Ins](#)
- [TI Development Tools](#)
- [Common Issues with TI Development](#)
- [TI Optimization Guide](#)
- [Error Codes](#)

12.45.2 Overview of TI DSP Algorithms in AAX

Avid's hardware-accelerated audio systems allow AAX plug-ins to offload their real-time processing tasks to a dedicated processor, guaranteeing reliable performance at ultra-low latency. Avid's TI DSP-based products utilize Texas Instruments DSP chips to host plug-ins in a managed shell environment.

The AAX host handles all system-level communications and resources on the DSP and provides a consistent API to manage communication between the plug-in's real-time algorithm and its other components. This design allows AAX plug-ins to use the same communication methods whether they are running natively, on a TI-based accelerated system, or in some other distributed environment.

Each AAX plug-in contains a real-time algorithm callback. For TI DSP-based platforms, this callback is compiled into a relocatable ELF DLL. This library is loaded onto the appropriate DSP by the host, and may share the DSP with other plug-ins if the host determines that the required system resources are available. A real-time execution environment called the TI Shell is also loaded onto each DSP. The TI Shell manages the DSP's memory and interrupts and guarantees reliable real-time performance even at single sample operation.

12.45.3 Getting Started with HDX DSP

This section provides a quick overview of what you will need for creating an AAX DSP plug-in to run on Avid's TI-based HDX DSP platforms.

- Plug-in structure

The algorithm component for an AAX DSP plug-in is compiled into a binary that runs on the TI DSP. Because this algorithm callback runs on a separate device than the rest of the plug-in, the algorithm must be separated from the plug-in's other components and no pointers may be shared between the two. All memory used by the algorithm must be set up via fields in the algorithm's context structure, and the AAX packet system must be used for transmitting coefficients from the plug-in data model to the algorithm.

For more information about the structure of an AAX plug-in algorithm and features for communicating with the algorithm, see [Real-time algorithm callback](#).

- Development Environment

To compile your plug-in's AAX DSP binary you will need to run TI's free Code Composer Studio (CCS) IDE on a Windows system or VM.

The latest Code Composer Studio versions no longer come bundled with compilation tools compatible with C6727 DSPs, so you must use download and install version 7.x from <https://www.ti.com/tool/download/C6000-CGT/7.4.24>.

To get Code Composer Studio version 12, visit <https://www.ti.com/tool/ccstudio> and navigate to Downloads > View all versions

For additional steps to set up Code Composer Studio see [TI Development Tools](#)

- Language support

The C6727 compiler for AAX DSP plug-ins supports C and C++ up to C++98. In particular, note that no C++11 or later language support is available. For more specific details see [C++ standard support](#).

12.45.4 The HDX DSP Platform

HDX DSP is Avid's core mixer and plug-in accelerator platform. Avid's HDX and Pro Tools | Carbon systems both use the HDX DSP platform, with multiple TI C6727 DSPs each clocked at 350 MHz. These DSPs utilize a 32-bit floating-point architecture, with the option to perform 64-bit double-precision operations at some performance cost. Each HDX card includes 18 DSPs and is connected to the host system over a high bandwidth PCIe connection, while each Pro Tools | Carbon system includes 8 DSPs and is connected to the host system over a Gigabit Ethernet connection.

12.45.4.1 DSP characteristics: instruction processing

The C6727 DSP utilizes a VLIW architecture and contains dual data paths. Each data path includes four independent functional units, so the DSP can accommodate up to 8 parallel instructions per cycle. To take advantage of this architecture, the TI compiler relies heavily on instruction pipelining for optimization.

12.45.4.2 DSP characteristics: audio buffers

In order to realize the maximum possible performance benefit from this architecture, the algorithm routine for a single HDX DSP plug-in is always called with the same buffer size. By guaranteeing that each algorithm will be called with a consistent buffer size, the TI compiler is able to properly account for any possible iterative instruction pipelining, resulting in large performance gains.

HDX DSP uses a four-sample processing quantum by default for plug-in instances. Plug-ins that require additional processing time per callback, e.g. to mitigate the overhead cost of the chip's DMA facilities, may optionally request a 16, 32, or 64-sample quantum. Note that at higher block sizes, the number of potential I/O channels available to plug-ins on a chip will be reduced.

Host Compatibility Notes 32 and 64-sample quantum is available in Pro Tools 10.2 and higher

12.45.4.3 DSP characteristics: memory

Each DSP on the HDX DSP platform includes 16 MB of external RAM and 256 kB of internal RAM. The DSP has the ability to execute code from either internal or external RAM, though the real-time performance cost of external RAM accesses is significant. The chip's internal RAM is addressable at the core clock rate.

Each DSP also has a program cache of 32 kB. Plug-in code is loaded into this cache from internal memory, so for best performance your plug-in should not use more than 32 kB for its program code. You can look at the CCS-generated .map file to find your plug-in's program code size.

12.45.4.3.1 SDRAM performance Asynchronous access to data in the C6727's SDRAM is very slow, requiring 50 cycles/word to read and 15 cycles/word to write. This is primarily due to clock domain bridging, lack of data caching, and the fact that data from the core is given a low priority in order to avoid stalling real-time DMA transfers.

12.45.4.3.2 Executing program code from external memory The TI C6727 supports executing program code from external memory. When executing from uncached external memory, expect cycle counts to increase by a factor of 4x to 5x compared with the equivalent internal-memory code. Assuming that no cache thrashing occurs, subsequent calls will be cached and thus the program's location in either external or internal memory will produce similar cycle counts.

Note

The CCSv4 Profiler contains a bug that produces incorrect cycle counts for cached external-memory program code. Therefore, when gathering cycle count data for a plug-in that stores its program data in external memory, an RTI-based timing method should be used.

12.45.4.4 System characteristics: DSP/host data transfers

Plug-ins loaded onto the HDX DSP platform may transfer arbitrarily large data blocks between the DSP and the host, within the limits of available DSP memory and system bandwidth.

12.45.4.4.1 DSP/host bandwidth Neither AAX nor the HDX DSP platform include any explicit plug-in bandwidth limiting constraints. If a plug-in's data transfer requests bump up against the physical bandwidth limit for the system then this will delay the blocking data transfer request on the host, as the transfer will be held off for higher-priority operations on the DSP, and may also delay automation data from reaching other plug-ins on the affected DSPs in the same group.

The recommended upper limit for DSP/host data transfer requests in an individual plug-in when running on an HDX PCIe card is 10 MB/s, divided by the maximum number of plug-in instances that will run on a single chip. On the HDX card, DSPs are wired to the FPGA crossbar in groups of three, with a data bandwidth of approximately 67 MB/s for each group. The overall system bandwidth for each DSP is therefore approximately 20 MB/s. This bandwidth is shared by all data reads and writes, including custom data transfer requests as well as plug-in and mixer automation and metering data.

This limit is significantly lower on Pro Tools | Carbon. Carbon uses a single group for all eight DSPs, so the overall system bandwidth for each DSP is approximately 8 MB/s. In addition, data transfers between Carbon and the host system must be executed over a Gigabit Ethernet connection with up to 75% of its bandwidth already reserved for AVB audio data. This leaves 250 Mb/s for all other command traffic. If your plug-in utilizes frequent or large DSP/host data transfers then be sure to test it on Pro Tools | Carbon to verify whether it is compatible.

12.45.4.4.2 DSP/host data transfer characteristics The minimum data transfer size for all host-to-DSP communications for HDX PCIe cards is 128 bytes. This limit applies to all host-to-DSP data transfers, including data sent to buffered ports, unbuffered ports, and private data blocks (via the AAX Direct Data interface.)

Since each transfer has a minimum size of 128 bytes, the use of many small packets does not increase transfer efficiency or save system bandwidth. Quite the opposite: updating a single 64-byte packet would require less bandwidth than updating two 4-byte packets in an HDX PCIe system, since the former would require only one 128-byte transfer while the latter will require two.

On Pro Tools | Carbon there is no minimum data transfer size. That said, for best performance on Carbon it is still recommended to minimize the number of data packets that are sent.

12.45.4.5 TI Shell characteristics: Memory allocation

12.45.4.5.1 Memory resource availability The TI Shell code that is loaded onto each DSP uses approximately 56 kB of internal memory, leaving 200 kB of internal memory per DSP. This memory is shared between the plug-ins on the chip and holds the plug-ins' code and data, per-instance blocks declared in `Describe()`, and instance overhead.

As a general guideline, plug-in instances should not use more than $200 / n$ kB of internal memory, where n is the number of instances of your plug-in that will run on a single chip based on its cycle count requirements. If each plug-in instance on the chip requires more internal memory than this then the plug-in may need to declare an explicit number of instances that can run per chip based on this memory usage rather than declaring its cycle count utilization.

12.45.4.5.2 Shared and per-instance memory allocation When a plug-in instance is created on a DSP, its program code is loaded onto that DSP. This copy of the program code is then re-used for all subsequent instances of the effect that are loaded onto the DSP. Static and global data are also shared between all instances of an effect on the DSP. Other allocations, such as coefficient and private data blocks, are per-instance.

Host Compatibility Notes Beginning in Pro Tools 11, AAX DSP algorithms also support optional temporary data spaces that can be described in the `Describe` module and are shared among all instances on a DSP. This is an alternative to declaring large data blocks on the stack for better memory management and to prevent stack overflows. Please refer to [AAX_IComponentDescriptor::AddTemporaryData\(\)](#) for usage instructions.

12.45.4.5.3 Placing data into external memory An AAX plug-in may optionally request that its private data or program code be placed into external memory. Because standard access calls to the DSP's SDRAM are very slow, it is strongly recommended that all of a plug-in's real-time data be placed in internal RAM, and the TI Shell will load a plug-in's program code and all private plug-in data blocks into internal memory by default.

Requesting more than 256 kB of data in internal memory for plug-in data plus the memory required by the TI Shell will lead to undefined behavior, so it is important to explicitly request external memory for plug-in data when appropriate.

For private data blocks that should be loaded into external memory, use the [AAX_ePrivateDataOptions_External](#) flag when calling [AAX_IComponentDescriptor::AddPrivateData\(\)](#). This flag will be ignored by the host, so Native AAX plug-ins will have the same functionality with or without this property.

To load program code, static data, or global variables into external memory, use the `TI_SECTION` pragmas. For example, `#pragma CODE_SECTION_( ".extmem" )` can be used before function definitions that are either initialization code, or infrequently used background code. For static variables, use `#pragma DATA_SECTION_( ".extmemdata" )` before each variable definition.

12.45.4.5.4 DMA support Because of the slower access time of external RAM, you should consider using a [DMA transfer](#) for recurring transfers, and possibly even for larger one-time transfers. This is of particular relevance for data reads, which must traverse the various clock domains and priority switches twice (address send, and then data return.)

The TI Shell supports three DMA modes: Scatter (for transfers from internal to external memory), Gather (for transfers from external to internal memory), and Burst (contiguous block copies). The Scatter mode can accomplish transfer speeds of up to 2.1 DSP cycles/byte transferred, while the Gather mode can accomplish 2.7 cycles/byte transferred.

The Scatter and Gather DMA facilities use a linear buffer for internal memory and a FIFO for external memory. It is possible to transfer to or from multiple offsets within the external memory FIFO using an offset table, which can contain up to 65,536 (2^{16}) entries. The offset (burst) length may be 4, 8, 16, 32, or 64 bytes long.

The TI Shell also supports a Burst DMA mode which implements linear data reads or writes.

For more information on DMA support and for example code, see `\ExamplePlugIns\DemoGain_DMA` in the SDK.

12.45.4.6 TI Shell characteristics: Data packet services

In addition to supporting direct transfers of arbitrary data via DMA, the TI Shell also supports a packetized data delivery mechanism for host-to-DSP data transfers. Packet delivery ports may be either unbuffered or buffered, and are described using the [AAX_EDataInPortType](#) parameter in [AAX_VComponentDescriptor::AddDataInPort\(\)](#).

12.45.4.6.1 Unbuffered ports Unbuffered ports use a straightforward implementation that delivers posted packets to the algorithm as soon as possible. In an unbuffered port, newer packets will always override older packets. Therefore, an algorithm may not receive every packet that was posted to an unbuffered port, but it will always receive the most up-to-date information possible.

Unbuffered ports deliver their data without blocking or synchronizing with the algorithm's execution. Although bus arbitration guarantees that a read from the algorithm callback will not occur in the middle of a write from the host, it is important to note that the data in an unbuffered port may change during algorithm execution.

12.45.4.6.2 Buffered ports Buffered data ports store incoming packets in a host-managed queue. This queue acts as a buffer and provides the host with more flexibility in how it delivers packets. A key feature of buffered data ports is that new data will never be delivered to these ports during algorithm execution.

The behavior of buffered data ports varies depending on the host platform. In HDX DSP plug-ins, Buffered data ports use a FIFO to queue data packets as they are posted. New packets are dequeued and delivered to the algorithm individually, with the next packet arriving before each algorithm render callback.

12.45.4.6.3 Data port overhead and restrictions Each HDX DSP supports a maximum of 164 buffered data ports, which matches the maximum I/O limit for each DSP. System overhead costs associated with using the on-chip packet services are as follows:

Memory Overhead

- The memory overhead for an unbuffered data port is simply the size of the data packet.
- This DSP memory overhead for a buffered data port is two times the size of the data packet. A large (>100-element) packet queue is also allocated on the host.

CPU overhead Unbuffered ports do not incur any additional CPU overhead.

Individual buffered ports incur non-trivial CPU overhead. For example, in Pro Tools 10.2 each buffered port requires 5 cycles of overhead per render callback. This overhead can quickly add up in "small" plug-ins that contain many buffered data ports. Therefore, we strongly recommend that plug-ins use consolidated coefficient packets when possible in order to minimize this overhead. This optimization can result in large performance gains for callbacks that require 1000 or fewer cycles to operate.

The trade-off of this optimization is that more work ends up being done on the host and more data must be transmitted to the algorithm, since the entire coefficient packet must be re-calculated and re-sent every time any of its input parameters change. This is usually beneficial trade-off to make, especially given the 128-Byte per-transfer minimum for HDX PCIe cards discussed above. However, care must be taken in extreme cases such as when packet delivery threatens to bump up against the maximum recommended bandwidth for host/DSP data transfers, especially on Pro Tools | Carbon.

12.45.4.7 TI Shell characteristics: Instance allocation

12.45.4.7.1 Multi-shell packing With a few exceptions, AAX DSP plug-ins will share DSPs with other plug-ins. This occurs transparently to the plug-in due to the fact that all system resource management is handled by the TI Shell.

When a new plug-in instance is created, the TI Shell and AAX host will attempt to intelligently allocate it to a DSP based on both memory and CPU resource requirements. If one plug-in on the chip requires a large amount of memory and very few processing cycles, it may be packed with another plug-in that does not require much memory but that is very CPU intensive.

Each DSP chip runs audio callbacks at a single buffer size, so plug-ins that run at different buffer sizes will not be loaded onto the same DSP chip. For example, if a plug-in processes at 16 sample buffers then it will only share a chip with other plug-ins that process at 16 sample buffers. An AAX DSP plug-in's buffer size is defined by its [AAX_eProperty_DSP_AudioBufferLength](#) property. Most plug-ins use the [default](#) of 4 sample buffers, and that is the value that will maximize chip sharing.

Certain plug-ins cannot share a DSP with other plug-ins:

- Plug-ins that use [DMA](#)
- Plug-ins that register for a [background processing](#) callback
- Plug-ins that register a maximum number of instances per chip using [AAX_eProperty_TI_MaxInstancesPerChip](#)

These plug-ins will receive dedicated DSPs to which only additional instances of the same plug-in type will be added.

The TI shell also supports a [processor affinity](#) property, which indicates that a DSP ProcessProc should be preferentially loaded onto the same DSP as other instances from the same DLL binary. This is a requirement for some designs that must share global data between different processing configurations.

12.45.4.7.2 DSP Shuffles A DSP shuffle will occur in Pro Tools when the engine must re-allocate DSP resources in order to make more processing power available. A shuffle will force the re-instantiation of the plug-in's DSP algorithm component, potentially on a new chip, while leaving the plug-in's host objects intact. During a shuffle, the engine will perform the following steps:

1. Disconnect audio from an effect
2. Call instance initialization with the removing instance flag on the old location
3. Repeat for all instances of all DSP Effects in the system
4. Load the effect in the new location
5. Re-send the last packets to all data-in ports
6. Call private data init for any private data
7. Call instance init with the 'adding instance' flag, in the new location
8. Begin audio processing
9. Reconnect audio
10. Repeat the instantiation and connection process for all instances of all DSP Effects in the system

Note that the system may perform some audio processing with each new instance before all of the Effect instances in the system have been re-instantiated.

12.45.4.8 Additional TI Shell services

12.45.4.8.1 Background processing AAX plug-ins may request idle time from the main TI Shell thread. This results in a true idle context callback which can be used for non-critical [background processing](#) tasks on the DSP. This facility restricts the DSP to only allocate plug-in instances of the same type.

A plug-in's background processing callback is not provided with a reference to the plug-in's data structures and must therefore access plug-in data via global variables. The background process will be interrupted by system events and the audio render callback. For more information and an example on how to create a plug-in that relies on background processing, see `\ExamplePlugins\DemoGain_Background` in the SDK.

12.45.5 Requirements for HDX DSP Plug-Ins

12.45.5.1 Plug-in description

To support HDX DSP platforms, a plug-in must add a TI ProcessProc (real-time processing entrypoint) for each of its algorithms. This is done via a call to [AAX_IComponentDescriptor::AddProcessProc_TI\(\)](#), which is parametrized with the names of both the algorithm's TI DLL and of its exported entrypoint.

At minimum, the TI ProcessProc requires the following AAX Properties:

- A TI plug-in ID: [AAX_eProperty_PluginID_TI](#)
- The audio buffer size that will be used by the ProcessProc: [AAX_eProperty_DSP_AudioBufferLength](#), set with a value from [AAX_EAudioBufferLengthDSP](#)

12.45.5.2 Performance measurement and reporting

In order to determine each algorithm's resource requirements, the host collects cycle count information from the plug-in via the plug-in's Describe callback. Each plug-in Effect is responsible for correctly reporting its algorithms' cycle counts for each accelerated platform that it supports. For plug-ins that use DMA or background threads, a maximum per-chip instance count is also required.

Note

All reported values must represent the algorithm's worst case performance.

Each of these values are reported as properties of a given algorithm ProcessProc and are provided by the plug-in via [AAX_IComponentDescriptor::AddProcessProc_Tl\(\)](#). If an effect does not report its cycle count usage then it will be limited to a single instance per TI chip. This can be useful during development, but is not a supported mode for general use; all shipped plug-ins must correctly report their cycle requirements.

The DigiShell utility can be used to accurately measure plug-in cycle count requirements. For more information about DigiShell, see [DSH Guide](#).

12.45.5.2.1 Shared vs. per-instance cycles Because a single call into a plug-in is used to process multiple instances of that effect on that chip, two cycle count properties must be reported for each TI algorithm:

1. [AAX_eProperty_Tl_SharedCycleCount](#) This property describes the algorithm's one-time processing overhead that doesn't change as instances are added to a chip.
2. [AAX_eProperty_Tl_InstanceCycleCount](#) This property describes the additional cycle counts that each instance adds to the base shared overhead.

Many plug-ins exhibit different performance characteristics for both of these metrics depending on the plug-in's state. When reporting a plug-in's shared and per-instance cycle count requirements it is important to ensure that the reported values are the *maximum possible requirements* of the algorithm.

Often a plug-in will experience its worst-case per-instance processing load in one configuration and its worst-case shared processing load in another configuration. In this situation, the plug-in's reported cycle count requirements should reflect the state in which the *sum* of the two metrics is highest.

It's a common practice to not describe [AAX_eProperty_Tl_InstanceCycleCount](#) and [AAX_eProperty_Tl_SharedCycleCount](#) for the plug-ins during development and debugging process of the DSP plug-ins. This is acceptable, although in this case the one instance of such a plug-in will require the whole chip. In AAX SDK example plug-ins this is implemented using `AAX_Tl_BINARY_IN_DEVELOPMENT` macros. If defined, it turns off the cycle count properties for the plug-in.

12.45.5.2.2 Measuring shared cycles Measuring shared cycle counts requires instantiating multiple instances of an effect and observing how the processing time changes as instances are added. The shared and instance cycle counts are then calculated by performing a linear regression on the number of uncached cycle counts as the number of plug-in instances on the chip increases.

Note that these values will differ between debug and release builds of an algorithm, so a plug-in's describe function should report the correct cycle count values based on the relevant build configuration.

DigiShell includes the ability to measure shared cycle counts using the `DAE.cyclesshared` command. For more information about performance profiling using DigiShell, see [Cycle count performance test](#).

Note

HDX DSP requires reporting of an algorithm's *worst-case* cycle counts.

Because HDX PCIe and Pro Tools | Carbon use the same HDX DSP platform, either product may be used to take plug-in cycle count measurements.

12.45.5.2.3 DMA and background thread performance reporting For algorithms that use [DMA](#) or [background thread](#) facilities, the maximum number of algorithm instances that will fit on a chip is difficult to predict from cycle counts alone. Due to the asynchronous behavior and limited capacity of the DMA system, the DMA system may begin to miss its deadlines before the CPU is fully loaded. In addition, due to differences in background processing requirements between algorithms, an effect's background process may begin to miss its deadlines and be starved before the interrupt-time audio processing is at capacity. Plug-ins that use these facilities must therefore report the maximum number of instances that will run reliably at a given sample rate, in addition to reporting their shared and per-instance cycle counts as above.

Some other plug-ins may also wish to report the maximum number of instances that will run reliably at each sample rate. For example, plug-ins that use a lot of host/DSP data bandwidth may need to limit the number of instances per DSP chip in order to run successfully on Pro Tools | Carbon.

Maximum reliable instance counts are reported using an additional property, [AAX_eProperty_TI_MaxInstancesPerChip](#). A plug-in should register separate components for the following three sample rate ranges in order to register distinct values for this property:

1. Sample rates from 42kHz to 50kHz
2. Sample rates from 84kHz to 100kHz
3. Sample rates from 168kHz to 200kHz

Notes regarding DMA and background thread performance reporting:

- Because the number of instances will decrease as sample rate increases, the plug-in must be tested at the highest available pulled-up sample rate (i.e. 50kHz instead of 48kHz) in each of these three ranges.
- On the HDX platform, effects that use DMA or background threads will not be mixed with effects of other types on a given chip.
- The maximum number of instances per DSP cannot be measured via DSH in these cases, so careful listening tests must be manually performed in order to determine whether a certain number of instances of a DMA or background-enabled plug-in actually operate correctly on a DSP.

12.45.5.2.4 Dynamic resource usage All resources used by an AAX DSP plug-in algorithm are considered static. Plug-ins may not dynamically change the amount of memory or DSP cycles that are allocated to them after these metrics are provided in `Describe`.

The ability to dynamically change DSP cycle count requirements at run time is provided in the AAX SDK but is not currently supported by any host.

12.45.5.3 Plug-in compilation and packaging

12.45.5.3.1 Exported symbols Each HDX DSP algorithm (ELF DLL) may contain multiple entrypoints. A single DLL may be used for all of your plug-in's entrypoints and program code, or you may divide your plug-in's entrypoints and program code between multiple DLLs.

Your plug-in must export one "C"-style callback for each algorithm `ProcessProc` that your plug-in registers. This entrypoint must conform to the standard AAX real-time algorithm callback prototype:

```
# include "elf_linkage_aax_ccsv5.h" // Includes required TI_EXPORT definition
extern "C"
TI_EXPORT
void
MyEffect_AlgorithmProcessFunction(
    SMyEffectAlg_Context * const inInstancesBegin [],
    const void * inInstancesEnd)
```

Listing 1.1: The standard AAX real-time algorithm callback prototype

See [Settings for exported symbols](#) if you are running into problems linking your AAX DSP binary.

12.45.5.3.2 Packaging The ELF DLLs for an AAX DSP plug-in must be placed in the ./Content/Resources directory within the plug-in bundle.

12.45.6 TI Development Tools

Development for TI algorithms is primarily performed in TI's Code Composer Studio. Code Composer Studio (CCS) is a full-featured, Eclipse-based IDE providing JTAG hardware debugger support, a hardware simulator, and a suite of profiling tools. Most importantly, CCS includes an excellent C compiler that is capable of providing highly optimized DSP instructions without too much tuning.

Note

As of this writing, Code Composer Studio for Mac does not support the C6000 series processor. CCS for Windows is required for AAX DSP plug-in development. See [MacOS Host Support for CCS](#) on the Texas Instruments wiki for current compatibility information.

12.45.6.1 Code Composer Studio

The AAX SDK supports Code Composer Studio versions 4 ("CCSv4") and higher ("CCSv5", etc.), with hardware debugging support beginning in version 4.2. As of the writing of this documentation, CCS versions 4, 5, 7 and 12 have been tested by Avid.

Note

This documentation was originally written for CCSv4 and was later updated with instructions for updating from CCSv4 to CCSv5. Versions 5 and higher use a different project file format from version 4; when this documentation describes changes required for version 5 then these changes will also be required by other later versions which use this new project format.

12.45.6.1.1 Installation

1. Download and install the latest Code Composer Studio from TI's website.

Note

Windows 10 requires Code Composer Studio version 6.1.3 or higher

As of Code Composer Studio version 7 TI does not charge for licenses. You can simply download the tool and start using it. Along with this the end user license agreement has changed to a simple TSPA compatible license. For more information see the TI web site.

2. The default installation will work fine, but a custom install will be smaller. You only need support for the C6000 chipset and, if you have an HDX board with JTAG pins, the Spectrum Digital JTAG drivers, so you can deselect all the other chipsets and JTAG drivers.
3. Download and install the C6000 Code Generation Tools v7.x:
 - (a) Launch CCS and go to Help > Install New Software...
 - (b) In the opened dialog select "Code Generation Tools Updates" in the "Work with:" drop-down list.
 - (c) Select "TI Compiler Updates" > "C6000 Compiler Tools [version 7.x]".
 - (d) Press Next and continue installation using the "typical" installation settings.

As of the publishing of this version of the AAX SDK Avid is internally using v7.4.24. Avid has tested 7.4.4, 7.4.6 and 7.4.24, all later v7 revisions should work as well. CGTools versions higher than v7 do not support the C6727 DSP.

12.45.6.1.2 Workspace setup Each time you launch CCS you will be prompted to select a workspace directory. A CCS workspace is similar to a Visual Studio solution file. Note that workspaces tend to store absolute paths and developer-specific info, so you may wish to avoid checking them in to your source control server.

Setting up workspace-global macros *To set up workspace global macros:*

1. When you open CCS for the first time, select a directory for your "workspace". As mentioned above, we recommend that this be outside of your source tree.

Note

If you are updating from a previous CCS version you may not be able to reuse your workspace. For example, we have found the CCSv4 workspaces are incompatible with CCSv5. After updating your system to a later Code Composer Studio version you should create a new workspace and import your existing projects into this new workspace.

2. Go to File > Import... and select Code Composer Studio > Build Variables (CCS > Managed Build Macros in CCSv4.) Click Next.
3. Browse to TI/Common/macros.ini in your AAX SDK directory and click Finish.
4. This will define an "SDK_SOURCE_ROOT" Linked Resource path variable and Managed Build macro, which associates the CCS workspace with a single AAX SDK installation.

Note

A side effect of this is that you cannot use projects from multiple distinct AAX SDK installations in the same CCS workspace.

5. To verify that the correct path has been set, go to Window > Preferences... and look in General > Workspace > Linked Resources, and C/C++ > Build > Build Variables (C/C++ > Managed Build > Macros for CCSv4.)

Importing projects into your workspace *To import projects into your workspace:*

1. In the IDE, go to Project > Import CCS Projects...
2. In Select search-directory, select the root of your AAX SDK installation.
3. The CCS projects in the AAX SDK will be added to the Discovered projects field. Select All or choose the specific projects you want to import.
4. Click Finish, and then wait while the projects are imported.

In order to import CCSv4 projects into later versions of Code Composer Studio it is necessary to add a .cdtproject file to the project. If you don't have this file in your project, then you can copy it from any other existing project which was created using CCSv5 or later. Otherwise you will most likely see something similar to this error:

"Error: Import failed for project 'xxxx' because its meta-data cannot be interpreted."

If you try to build this newly imported CCSv4 project in a later version of Code Composer Studio then you will get the warning:

"This project was created using a version of compiler that is not currently installed: 7.0.5 [C6000]. Another version of the compiler will be used during build: 7.4.24. Please install the compiler of the required version, or migrate the project to one of the available compiler versions by adjusting project properties."

This warning may be cleared by changing Properties > General > Compiler Version from TI v7.0.x to the current version (e.g. TI v7.4.x). After that the "Output format" field, which is next one to the "Compiler version" field and is typically grayed out, will become active. You should choose "eabi (ELF)" there. Otherwise Code Composer the build will fail with errors:

- "--dynamic=lib not supported when producing TI-COFF output files"
- "--export=_auto_init_elf not supported when producing TI-COFF output"

Note

After successful conversion of the project and successful build, the remeasurement of cycle count should be done, because it may change. Most likely it will decrease, as compared to the version which was built with CCSv4, but that is not guaranteed. Also the size of the DLL may increase, which may require reducing code size in order to properly instantiate the plug-in.

12.45.6.1.3 Creating new projects

New project setup Use the following settings in the "New Project..." wizard. Defaults are in *italics*.

- Project Type: C6000
- Output type: Executable
- Device Variant: Generic C67x+ Device
- Device Endianness: little
- Code Generation Tools: 7.4.24 or later (7.0.5 for CCSv4)
- Output format: eabi (ELF) (in CCSv4 this field will be grayed out.)
- Linker Command File: CommonPlugIn_LinkersCmd.cmd (see note below)
- Runtime Support Library: <automatic>

Note

You can edit the Linker Command File setting to use the `SDK_SOURCE_ROOT` macro by manually editing the project's .project XML file or by adding the file to your project using a relative path. See the SDK sample plug-in projects for an example.

12.45.6.1.4 Recommended settings for AAX plug-in projects Tool Settings C6000 Compiler Include Options

```
-include_path "${SDK_SOURCE_ROOT}/Interfaces" -include_path "${SDK_SOURCE_ROOT}/[Plug-in directory]"
```

The `SDK_SOURCE_ROOT` macro is defined via the macros.ini file, located in the SDK's /TI/CCSv4 directory. If you encounter errors using this macro, import the file using File > Import... > CCS > Managed Build Macros.

Tool Settings C6000 Compiler Command Files -cmd_file "\${SDK_SOURCE_ROOT}\\TI\\CCSv4\\CommonPlugIn_CompilerCmd.cmd"

This file contains additional compiler commands that should be common to all AAX plug-in projects

Tool Settings C6000 Linker Basic Options `-o "${ConfigDir}/${PackageName}/Contents/Resources/${ProjName}.dll"`

This path will ensure that your compiled TI DLL is placed in the appropriate location inside your AAX plug-in bundle.

Tool Settings C6000 Linker Runtime Environment (No "Initialization model" options set)

Build Settings Artifact name `${ConfigDir}/${PackageName}/Contents/Resources/${ProjName}`

This path will ensure that your compiled TI DLL is placed in the appropriate location inside your AAX plug-in bundle.

Build Settings Artifact extension `dll`

AAX TI libraries should use the `.dll` extension

Binary Parser Elf Parser

AAX TI libraries should use the Elf binary parser only

Macros Project User Macros `ConfigDir = ${OutDir}/${ConfigName} IntDir = ${ConfigDir}/int/${PackageName}/TI/${ProjName} OutDir = ${ProjDirPath}/../../WinBuild PackageName = [Plug-in name]`

These macros are used by the other settings here to ensure proper path set-up and artifact naming. Don't worry that `ConfigName` shows up as undefined - it will be defined as `Debug/Release` at compilation.

12.45.6.1.5 Recommended Release configuration settings Tool Settings C6000 Compiler Basic Options `-symdebug:none -O3`

Tool Settings C6000 Compiler Predefined Symbols `-define=NDEBUG`

Tool Settings C6000 Compiler Optimizations `-os -on2 -op3`

Tool Settings C6000 Compiler Assembler Options `-keep_asm`

12.45.6.1.6 Other useful project settings Tool Settings C6000 Compiler Predefined Symbols `-define _DEBUG`

This option is useful for differentiating cycle count reporting for Debug vs. Release builds.

Tool Settings C6000 Compiler Directory Specifier `-ft "${IntDir}" -fr "${IntDir}" -fs "${IntDir}"`

Useful for collecting intermediate files

Tool Settings C6000 Linker Basic Options `-m "${IntDir}/${ProjName}.map"`

Useful for placing the map file alongside all other intermediates

Tool Settings C6000 Linker File Search Path `-l (nothing)`

You can exclude `libc.a`, which is included by default, from this option unless you require C library features.

12.45.6.1.7 Adding files and folders In CCS, dragging files into the project, using "Add Files to Project...", or using "Link Files to Project..." will either copy the file into the project directory or create an absolute path to the file. This is usually not the desired behavior. Use the following steps to add a file using a relative path:

1. Right click on the project you'd like to add files to, and select New > File (NOT "Source File" or "Header File").
2. Click "Advanced >>".
3. Check the box that says "Link to the file in the system". Click "Variables..."
4. Select the appropriate variable (usually either `SDK_SOURCE_ROOT` or `SOURCE_ROOT`) and click "Extend..."
5. Find the file you want to add. Click OK. Click Finish.

Note that, when adding folders, *everything* in the folder will be built by default. You can exclude files to work around this behavior.

12.45.6.1.8 Settings for exported symbols

- There is a compiler option in Code Composer Studio that will add an underscore to the exported entrypoint's name. We recommend keeping this option disabled in order to avoid ambiguity between the exported symbol name and the function name as it appears in your source code.
- If you encounter undefined symbol errors when linking to a DSP library that uses a C-style interface then add the extern "C" keyword before the lib function prototypes. This should resolve the majority of such linker errors.

12.45.6.2 The TMS320C6000 C++ compiler

One of the primary goals of AAX is to provide a platform-agnostic development architecture in which products can easily be developed and re-used across a wide variety of platforms. However, it is still occasionally necessary to write platform-specific code. This section will document methods for producing code that is specific to the TI C6727 platform using the TMS320C6000 C++ compiler.

12.45.6.2.1 C++ standard support The TMS320C6000 compiler supports C++ as defined in the ISO/IEC 14882:1998 standard. The exceptions to the standard are as follows:

- Complete C++ standard library support is not included. C subset and basic language support is included.
- These C++ headers for C library facilities are not included:
 - `<clocale>`
 - `<csignal>`
 - `<cwchar>`
 - `<cwctype>`
 - `<ciso646>`
- These C++ headers are the only C++ standard library header files included:
 - `<new>`
 - `<typeinfo>`
- No support for `bad_cast` or `bad_type_id` is included in the `typeinfo` header.

- Run-time type information (RTTI) is disabled by default. RTTI can be enabled with the `-rtti` compiler option.
- The `reinterpret_cast` type does not allow casting a pointer to member of one class to a pointer to member of another class if the classes are unrelated.
- Two-phase name binding in templates, as described in `tesp.res` and `temp.dep` of the standard, is not implemented.
- The export keyword for templates is not implemented.
- A typedef of a function type cannot include member function cv-qualifiers.
- A partial specialization of a class member template cannot be added outside of the class definition.

12.45.6.2.2 Predefined environment symbols The following symbols are predefined by the compiler on the TI architecture, and should be used in code concerned with cross-platform support:

- `_TMS320C6X` Identifies that the chip is a C6000 variant. This is the symbol that we commonly use to distinguish whether code is being compiled for AAX-Native (Mac/Windows) or AAX-TI.
- `_TMS320C6700_PLUS` Identifies that the chip is a C6700-plus variant

Although you should not require them for AAX development, equivalent assembly predefines are as follows:

- `.TMS320C6X` Identifies that the chip is a C6000 variant
- `.TMS320C6700_PLUS` Identifies that the chip is a C6700-plus variant

12.45.6.2.3 Loop controls The TI compiler supports several pragmas that can be used to give the compiler additional information about loops.

- `#pragma MUST_ITERATE( min, max, multiple )` This pragma helps the compiler optimize loops. `min` is the minimum number of times the loop will execute, `max` is the maximum number of times the loop will execute, and `modulo` is used if the loop will only execute a certain multiple of some number.
- `#pragma PROB_ITERATE( min , max )` If extreme cases prevent the use of `MUST_ITERATE`, `PROB_ITERATE` allows you to specify the usual number of times a loop executes. For example, `PROB_ITERATE( 8 )` could be applied to a loop that executes for eight iterations in the majority of cases but that sometimes may execute more or less than eight iterations.
- `pragma UNROLL( n )` Helps the compiler use SIMD instructions, where `n` is the unrolling factor. By specifying `UNROLL( 1 )` you can prevent the compiler from automatically unrolling a loop. In general, we recommend using `MUST_ITERATE` instead unless you have specifically identified a situation where manually unrolling a loop improves performance.

12.45.6.3 DigiShell test tool (DSH)

DigiShell is a software tool that provides a general framework for running tests on Avid audio hardware. As a command-line application, DigiShell may be driven as part of a standard, automated test suite for maximum test coverage. DSH supports loading all types of AAX plug-ins including Native and DSP, and is especially useful when running performance and cancellation tests of AAX-TI types. DigiShell is included in Pro Tools Development Builds as `dsh.exe` (Windows) or as `dsh` in the `CommandLineTools` directory (Mac).

More information on DSH test tool can be found in [DSH Guide](#).

12.45.6.4 Hardware Debugging

12.45.6.4.1 Requirements Relocatable ELF DLLs (TI algorithms) can be debugged with some help from the DIDL loader, the TI Shell Manager, and a script called `DLLView_Elf_Avid.js`.

These are the minimum requirements for hardware debugging for TI plug-ins:

- Code Composer Studio version 4.2 or later
- XDS510 hardware debugger
- JTAG-enabled HDX card

We recommend using Spectrum Digital's XDS510 USB Plus JTAG Emulator, as it is the only one our internal developers have used and tested in-house. Both Spectrum Digital and TI have useful technical reference/installation guides, both of which can be found on the AAX Developer Forum under the 'Development Tools' discussion.

12.45.6.4.2 How it works The `ridl` ELF loader inside DIDL stores a module and segment list containing the paths of all loaded modules and where their segments are loaded. The TI Shell Manager gets a serialized version of this table and loads it to a block of external memory on the chip at a known location. The `DLLView_Elf_Avid.js` script queries this memory via the debugger and extracts the paths of the modules and the ELF segment load locations, which it then passes on to the `GEL_SymbolAddELFRel` scripting console command (new to CCSv4.2). You can also use that command directly at the console.

12.45.6.4.3 Connecting a JTAG Emulator A JTAG-enabled HDX development card includes a "riser" PCB section extending about a centimeter above the production card PCB. This riser includes two JTAG connectors. The two connectors correspond to the two banks of 9 DSPs on the HDX card. Assuming that you are instantiating your plug-in for debugging on the first available DSP, you will want to connect your JTAG emulator to the connector that is closest to the card's user-visible ports. This connector corresponds to the first 9 DSPs on the card.

12.45.6.4.4 Linking to TIShell.out Hardware debugging, as well as several other debugging facilities, requires that the DSP plug-in project is linked to `TIShell.out` in Code Composer Studio.

To link a plug-in project to TIShell.out, follow these steps:

1. Open the plug-in project's properties window and navigate to the *C/C++ Build > Tool Settings > C6000 Linker > File Search Path* properties pane.
2. Add "TIShell.out" to the "Include library file" (-l) property list.
3. Under "Add <dir> to library search path" (-i), add the file path of the Pro Tools build you will be using to test the plug-in. This directory should already include the build's `TIShell.out` file.
4. Repeat this process for each Configuration of the plug-in project that you will be testing.
5. Add "[path to AAX SDK root]\\TI" to the project's list of source file include directories

12.45.6.4.5 Adding the HDX Target Descriptor File *To add the HDX Target Descriptor File:*

1. In the IDE, go to Window > Preferences, CCS > Debug. Point the "Shared target configuration directory" to `/TI/Common` in your AAX SDK source tree
2. In the IDE, go to Window > Show View > Target Configurations.
3. Click refresh if you don't see the configuration file
4. Right click `Raven_C672x_XDS510_USB.ccxml`, and click "Set as Default".

12.45.6.4.6 Setting up the DLLView script Once you have successfully installed the XDS510, you will have to do a little bit of setup with CCS. Before starting this process, verify that you are running CCSv4.2 or later and the C6000 code generation tools v7.4 or later (or 7.0.5 for CCSv4). CCS should recognize the installed emulator and prompt you to download the necessary drivers. Once completed, you will then want to setup your DLLView script.

To set up the DLLView script:

1. In the IDE, open the Scripting Console under View > Scripting Console
2. At the Scripting console, type one of the following to load the DLLView script (insert your own source tree path, and make sure to load the version that corresponds to your installed CCS version): Code Composer Studio 4: `loadJSFile "[PATH TO AAX SDK]/TI/CCSv4/dllView_Elf_Avid.js" true` Code Composer Studio 5 and later: `loadJSFile "[PATH TO AAX SDK]/TI/CCSv5/dllView_Elf_Avid.js" true`

You should now see a new menu item under the Scripts menu: "DLLView -Load Pro Tools Plug-In Symbols" This should load every time CCS starts.

12.45.6.4.7 Loading Symbols for Debugging You will need to get your code loaded and running on the TI before you load symbols. You can do this directly through Pro Tools, or by using our DigiShell test tool. If using the DigiShell test tool, load the DAE dish and then a plug-in via the following commands: `load_dish` DAE Loads the DAE dish `run` Lists available plug-ins with their index and `spec run<index>` Instantiates the <index> plug-in

Use the DLLView script to load symbols for ELF DLLs. After setting up the DLLView script and connecting to the desired chip in the Debug pane, run the "DLLView -Load Pro Tools Plug-In Symbols" script from the Scripts menu in Code Composer Studio.

Note

The chip will need to be Suspended in the debugger in order to load symbols.

To load symbols for debugging:

1. In CCS, Launch the TI Debugger (Target > Launch TI Debugger)
2. Connect the debug target to the appropriate chip
3. Suspend the chip
4. Run Scripts > DLLView -Load Pro Tools Plug-In Symbols.

Note

This script can take a moment to load; look at the Scripting Console to view its progress if you like
This script may print a warning about TIShell.out not existing. This warning is benign for plug-in debugging since the TIShell symbols are not required in this case.

This will load symbols for all symbol-rich modules running on the chip(s) connected to the debugger. If you load or unload plug-ins after this, you can simply repeat the "DLLView -Load Pro Tools Plug-In Symbols" command, which will synchronize the debugger with the current configuration.

Note

When running a plug-in in Pro Tools, the first DSP chip is reserved for the HDX mixer. Therefore the first available DSP chip for plug-in instantiation is `C672x_1`. Under DSH, the first available DSP chip is `C672x_0`.

12.45.6.4.8 Breaking on first entry into algorithm To break on the first entry into the plug-in's processing routine, use the manual single-buffer processing mode in DSH: `piproctrigger manual run<index>` Attach debugger, suspend the chip, load symbols, set breakpoint, resume `piproctrigger auto`

12.45.6.4.9 Breaking in the on-chip algorithm initialization callback It is not currently possible to hit a breakpoint in the optional on-chip algorithm initialization callback for a plug-in. If you need to troubleshoot this callback then you should use tracing to print debug information to a log file.

12.45.6.5 Tracing

Avid's AAX DSP platforms provide tracing functionality based on Avid's [DigiTrace](#) tool.

To enable trace logging for TI plug-ins, use the [AAX_TRACE](#) or [AAX_TRACE_RELEASE](#) macros defined in [AAX_Assert.h](#). A separate macro, [AAX_ASSERT](#), is also available for conditional tracing. These macros are cross-platform and will function whether the algorithm is running on the TI or on the host.

12.45.6.5.1 Tracing requirements

- The [AAX_ASSERT](#) and [AAX_TRACE](#) macros are debug-only and will not provide tracing output from release builds of your plug-in. [AAX_TRACE_RELEASE](#) may be used for tracing in both debug and release configurations.
- These macros require that the `DTF__AAXPLUGINS` facility is enabled in the DigiTrace configuration file. You can toggle this facility to enable or disable AAX algorithm-level tracing.
- In order for tracing to be successful on TI platforms, your plug-in's ELF DLL must dynamically link against `TIShell.out`, a component that is installed alongside the Pro Tools application. This file includes the 'glue' that is required in order for the linker to resolve the DigiTrace entrypoint symbol in the DLL.

To link your plug-in project to `TIShell.out` in Code Composer Studio, follow the steps listed in [Linking to TIShell.out](#).

12.45.6.5.2 Tracing example

```
int32_t
AAX_CALLBACK
MyExamplePlugIn_AlgorithmInit ( SExample_Alg_Context const *
    inInstance , AAX_EComponentInstanceInitAction inAction )
{
    AAX_TRACE_RELEASE (
        kAAX_Trace_Priority_Normal ,
        "MyExamplePlugIn_AlgorithmInit called for action : %d",
        inAction );
    return 0;
}
```

Listing 2: Adding trace code on TI

12.45.6.5.3 Usage notes

- When running on the DSP, the actual handling of each tracing call occurs in a separate thread. This can lead to incorrect data reporting if volatile data, such as a pointer to an audio sample, is passed in to the tracing statement as a parameter.
- DSP tracing is most reliable when using debug TI builds and when all TI compiler optimizations have been disabled
- Known and resolved issues with DSP tracing are logged on the [Known Issues](#) page

12.45.6.6 Testing in Pro Tools

12.45.6.6.1 The System Usage window The System Usage window in Pro Tools includes some features specifically targeted at testing DSP plug-ins, and particularly for testing shuffle events. Starting in Pro Tools 10, the System Usage window includes the following test features:

- Shift + Drag DSP Meter - This shuffles everything on the chosen chip to another chip, which allows you to quickly test shuffle for a given chip.
- Hover mouse over DSP - Presents a tooltip to show the running plug-ins on a chip
- Cmd+Option+Shift Hover - Detailed debugging tooltip info
- Cmd+Option+Shift Click - Forces a full shuffle of all chips / cards
- Click on empty chip - Reserves a DSP to prevent allocation on that chip

12.45.6.6.2 DSP information tooltip Pro Tools can display additional information for DSP plug-ins using some debug tooltips that are hidden in the plug-in window header and the System Usage window.

The tooltip in the plug-in window header displays information about the particular plug-in instance that is currently shown in the window. To display this tooltip, hold Command-Option-Shift (Mac) or Control-Alt-Shift (Windows) and hover the mouse cursor over the DSP > Native button in the plug-in header.

The tooltip in the System Usage window displays usage information for each DSP chip in the system. You can reveal this tooltip for a particular chip by mousing over the chip's usage meter while holding Command-Option-Shift (Mac) or Control-Alt-Shift (Windows). This tooltip shows the chip's total allocated cycles, internal, and external memory.

The information in these tooltips is generally targeted at systems-level debugging, but can prove useful for some plug-in troubleshooting as well.

Figure 1: DSP tooltip in the Pro Tools plug-in window header.

Figure 2: DSP tooltip in the Pro Tools System Usage window.

12.45.7 Common Issues with TI Development

12.45.7.1 Data structure compatibility

AAX DSP plug-ins use a set of custom data structures to exchange information with host. In order to preserve a consistent binary interface between the plug-in's host and algorithm, the layout of these structures must be identical on both platforms. Each structure must have the same size when compiled by both the host platform compiler and the TI DSP compiler, and any members that are referenced by both the host code and the DSP code must reside at the same offset within the struct on both platforms.

In order to satisfy this requirement, it is essential that an AAX plug-in's algorithm context structure and any other data structures that are passed between the host and the DSP use appropriate alignment. Data structures are usually aligned to 32-bit boundaries, and both Intel and TI compilers use identical struct alignment and packing for most cases. However, this behavior is not explicitly defined in the C standard.

Furthermore, different compilers may use different sizes for some built-in data types. It is therefore very important to use explicitly-sized types such as `int32_t` and `float` rather than ambiguous types such as `bool` or `int`. One particularly tricky data type is pointers, which may be compiled as 64-bit values on a 64-bit Intel system but as 32-bit values on the TI DSP.

Here are some specific scenarios when an unexpected difference in alignment or data type size may occur and cause an ABI incompatibility between a plug-in's host and DSP components:

- [Nested structures](#)
- [Usage of pragma pack](#)
- [Dynamic allocation of memory in structures and algorithm](#)
- [Incorrect use of pointer data](#)
- [Pointer data size incompatibility](#)

12.45.7.1.1 Nested structures It can be particularly difficult to debug alignment issues in nested data structures. One reason is that nested structs do not necessarily have the same alignment as the parent struct. A nested structure will have the alignment that is set preceding its declaration, not the alignment of the structure in which it is contained.

Aside from avoiding nested structs entirely, one way to avoid potential issues is to make sure that nested structs always contain a double. This will guarantee that the structure is double-word aligned. We have also found that placing nested structs near the beginning of the parent struct results in more consistent alignment between Intel and TI compilers, even in cases where the actual alignment of each member is strictly ambiguous according to the standard.

Another important rule of thumb with nested structs is to define them inline in the enclosing structure. We have found that including one data structure as a member in another data structure will only be reliably aligned between Visual Studio and the TI compiler tools if the member structure's type is defined in-line. This does not appear to be an issue between clang and the TI compiler - the data structure alignment for the nested structure is consistent between those two compilers regardless of the location of the internal structure's definition.

```
#include AAX_ALIGN_FILE_ALG
struct SomeStruct
{
    float a;
    float b;
};
#include AAX_ALIGN_FILE_RESET
// Somewhere else...
#include AAX_ALIGN_FILE_ALG
class SomeClass
{
public:
    SomeStruct s; // Don't do this! Inconsistent between Visual Studio and TI
    // other stuff...
};
#include AAX_ALIGN_FILE_RESET
```

Listing 3: Problematic code: nested struct not defined in-line

```
#include AAX_ALIGN_FILE_ALG
class SomeClass
{
public:
    struct SomeStruct
    {
        float a;
        float b;
    } s; // This is fine - consistent between Visual Studio, clang, and TI
    // other stuff...
};
#include AAX_ALIGN_FILE_RESET
```

Listing 4: Fixed code: nested struct defined in-line

12.45.7.1.2 Usage of pragma pack If you use pragmas to align your structs, then you should know that in most cases it will only decrease the natural struct alignment of a compiler. That means that if you have

```
#pragma pack(8)
struct x
{
    char a;
    float b;
};
```

Listing 5: Example of usage of #pragma pack where it has no effect

then struct x most likely won't be aligned to the 8 byte boundary. Therefore the pack pragma is not really useful for addressing alignment issues. Instead of using pack, one way to guarantee that a structure is double-word aligned, is to include at least one double member.

```
#pragma pack(8)
struct x
{
    float a;
    double b;
};
```

Listing 6: Example of usage of #pragma pack where it actually affects the alignment of the structure

In this case data will be double-word aligned.

12.45.7.1.3 Dynamic allocation of memory in structures and algorithm The problem with dynamic allocation is that it's difficult to enforce specific alignment of the resulting block beyond the natural alignment of the structure. Newly allocated blocks are not double-word aligned by default. This prevents double-word memory access optimizations (see [Additional data type optimizations](#)) from working.

```
// blocks are not aligned to 8-byte boundaries by default. This prevents double-word
// memory access optimizations from working
float* floatBlock = new float[100];
delete[] floatBlock;
// Though AAX_Alignment.h does include some aligned memory allocators to counteract the alignment
// problem, their use is still strongly discouraged.
float* floatBlock2 = alignMalloc<float>(100, 8);
alignFree(floatBlock2);
```

Listing 7: Problems which may arise when using dynamic allocation of memory in algorithm

12.45.7.1.4 Incorrect use of pointer data In general, you should avoid storing pointers to anything in any data structures that are passed between the host and the DSP. There are many possible problems and bugs that can be caused by this, for example:

- Often the memory map of packets can change out from under the plug-in
- It is easy to accidentally reference data in the wrong memory space when setting pointer values
- Pointer data types are not explicitly sized (see [below](#).)

One alternative to using raw data pointers is to store data offsets into a coefficient array rather than using direct pointers to other structure elements. A solution such as this that does not involve pointer data types will almost always end up being easier to implement, easier to troubleshoot, and easier to maintain than a solution that uses pointer data.

That said, if you must use pointer data types in any data structures that are passed between the AAX host and DSP components then you should be very careful to avoid the problems listed above.

12.45.7.1.5 Pointer data size incompatibility Problems due to pointer data size incompatibility can be particularly difficult to debug. Pointer data types are not explicitly sized in C, and, starting with the 64-bit Pro Tools 11 release, pointers will have different lengths for host and TI binaries. This can cause subtle portability problems in certain circumstances, if proper care is not taken.

Consider the following state block:

```
struct SMyPlugInStateBlock
{
    float mInGain_Smoothed;
    some_t* mPointerP;
    float mOutGain_Smoothed;
};
```

Notice the pointer `mPointerP` (the type that it points to is irrelevant for this discussion). Perhaps it is a pointer that can reference different sets of coefficients, or perhaps it points to some sort of global variable. In any case, this pointer is 64-bits long on the host, and 32-bits long on TI.

In most cases, this won't cause a problem because the host simply allocates a bit more space for the state block than the TI needs and fills the allocated memory with 0s. But consider the case where we overload [ResetFieldData\(\)](#) to set `mOutGain_Smoothed` to something other than 0:

```
AAX_Result MyPlugIn_Parameters::ResetFieldData (AAX_CFieldIndex inFieldIndex, void * inData, uint32_t
    inDataSize) const
{
    AAX_Result result;
    switch (inFieldIndex)
    {
        case (eMyAlgFieldIndex_State):
        {
            memset(inData, 0, inDataSize);
            SMyPlugInStateBlock* stateP = static_cast<SMyPlugInStateBlock*>(inData);
            stateP->mOutGain_Smoothed = mOutGain_Target;
```

```

        result = AAX_SUCCESS;
        break;
    }
    default:
    {
        result = AAX_CEffectParameters::ResetFieldData(inFieldIndex, inData, inDataSize);
        break;
    }
}
return result;
}

```

We might be doing this if `mOutGain_Smoothed` was a smoothing parameter and we want to start it at the target gain value (rather than having it smooth from 0.0 at instantiation). But if the Host and TI can't agree on where in the state block `mOutGain_Smooth` is located, then the result will be unexpected behavior that is difficult to debug.

The most direct way to avoid this problem is to use an explicitly-sized 32-bit type for any pointers in your state block:

```

struct SMyPlugInStateBlock
{
    float mInGain_Smoothed;
    uint32_t mPointerP;
    float mOutGain_Smoothed;
};

```

It will be necessary to use `reinterpret_cast<float*>(stateP->mPointerP)` to recast the pointer to a pointer data type on the TI, but that should not result in any extra processing cycles.

12.45.7.1.6 Alignment Reference These are the data type sizes and default alignments for some common compilers when compiling for 64-bit binary formats:

	TI		MS Visual C++		C++ Builder		GCC	
char	1 byte	1-byte aligned	1 byte	1-byte aligned	1 byte	1-byte aligned	1 byte	1-byte aligned
short	2 bytes	2-byte aligned	2 bytes	2-byte aligned	2 bytes	2-byte aligned	2 bytes	2-byte aligned
int	4 bytes	4-byte aligned	4 bytes	4-byte aligned	4 bytes	4-byte aligned	4 bytes	4-byte aligned
long	4 bytes	4-byte aligned	8 bytes	8-byte aligned	8 bytes	8-byte aligned	8 bytes	8-byte aligned
long long	8 bytes	8-byte aligned	8 bytes	8-byte aligned	8 bytes	8-byte aligned	8 bytes	8-byte aligned
bool	1 byte	1-byte aligned	1 byte	1-byte aligned	1 byte	1-byte aligned	1 byte	1-byte aligned
float	4 bytes	4-byte aligned	4 bytes	4-byte aligned	4 bytes	4-byte aligned	4 bytes	4-byte aligned
double	8 bytes	8-byte aligned	8 bytes	8-byte aligned	8 bytes	8-byte aligned	8 bytes	8-byte aligned
long double	8 bytes	8-byte aligned	8 bytes	8-byte aligned	8 bytes	8-byte aligned	16 bytes	16-byte aligned
pointer	4 bytes	4-byte aligned	8 bytes	8-byte aligned	8 bytes	8-byte aligned	8 bytes	8-byte aligned

Also here are some useful links to web resources on the topic:

- A good resource for the TI DSP is <http://www.ti.com/lit/an/sprab89/sprab89.pdf> (Section 2 especially). This document includes some graphs of simple alignment examples.
- Another good reference regarding general struct alignment issues is available from [publib.boulder.ibm.com](http://publib.boulder.ibm.com/infocenter/macxhelp/v6v81/index.jsp?topic=/com.ibm.vacpp6m.doc/compiler/ref/rnpgpack.htm): <http://publib.boulder.ibm.com/infocenter/macxhelp/v6v81/index.jsp?topic=/com.ibm.vacpp6m.doc/compiler/ref/rnpgpack.htm>

12.45.8 TI Optimization Guide

Optimizing AAX real-time algorithms for Avid's TI-based platforms is very similar to optimizing real-time algorithms for any architecture. When developers think about optimization, they often think "I want to make my code run faster". In reality, however, optimization is about making the processor do less. After all, the processor's clock rate is fixed and can only perform a limited number of instructions in a set amount of time. Therefore, our focus in this section will be on helping the compiler produce code with shorter execution paths and make full use of the TI chip's architecture.

Modern compilers have become extremely powerful at being able to optimize code, which is fortunate given the complicated architectures of today's DSP products. In this section we will not focus on instruction-level "optimizations" like the one below, which will automatically be done by the compiler. Instead of making our code faster, which it won't, little "tricks" like this really just make code harder to read:

```
int y = x;
y = y >> 1; // y = y / 2;
```

Listing 8: The kind of optimization that you won't be seeing in this section

Rather, we will focus on refactoring audio processing algorithms to be more efficient and on giving the TI compiler better information about the code, pointers, and data it is working with so it can perform more effective compile-time optimizations.

Finally, our optimization efforts will focus on the worst-case code path. For example, developers often try to optimize algorithms by conditionally bypassing portions of code that may be disabled by particular parameter states. This is counter-productive, because the system has to assume a plug-in's worst-case execution performance regardless of how much time the plug-in is actually using. Therefore, in the context of real-time algorithms running on AAX DSP platforms, it is best to only worry about worst-case execution time.

For more information about using TI's toolset to profile your code's performance, see [Cycle count performance test](#).

Note

The optimizations described in this section assume that you are using version 7 or higher of TI's C6000 Code Generation Tools (CGTools). We strongly recommend using v7.0.5 or later as earlier versions throw linking errors.

12.45.8.1 Optimization quick start

Here is a quick outline of the general optimization steps for an AAX DSP algorithm:

1. Before beginning your DSP optimizations, make sure that your Native algorithm has basic optimizations in place. In our experience, beginning the TI optimization process with a slow or needlessly precise Native algorithm will result in a long porting process. Here are some suggestions for common Native optimizations:
 - Identify unnecessary double precision
 - Identify tables that have too high of granularity
2. Make sure your compiler Release settings enable the compiler to optimize fully and give full optimization comments: `-k -s -pm -op3 -os -o3 -mo -mw -consultant -verbose -mv67p`
3. Use the load/update/store design pattern to reduce memory accesses in inner loops
4. Move any processing that does not directly depend on the audio signal out of the real-time algorithm
5. Declare non-changing variables and pointers (both local and in parameter lists) as `const`
6. Declare non-aliased pointers (both local variables and function parameters) as `AAX_RESTRICT`
7. Change any `long` variables to `int`, and change `double` variables to `float` if the reduced precision does not affect signal integrity (usually defined as cancellation with the plug-in's Native algorithm.)
8. Restructure inner processing loops so that they do not contain large conditional statements or other branches
9. Declare any functions that are called within the innermost processing loop as `inline` in order to allow the inner loops to pipeline
10. Add loop count information when known, using `#pragma MUST_ITERATE(min,max,quant)`

12.45.8.2 Compiler and linker options

As with any complex environment, many performance gains on the TI rely on the appropriate compiler and linker options. The options documented here will allow CGTools to apply its optimization logic to your algorithm.

When tweaking compiler options on the TI, keep in mind that, like on any CPU, it is useless to optimize Debug code or to profile its performance. This is especially true on TI processors because of the fact that generated Debug and Release assembly is almost completely different, assuming that heavy optimization options were chosen for the Release configuration.

In general, all recommended compiler options should be set correctly in the AAX SDK's example plug-in projects, and these settings may be used as a guide for your own plug-in projects. See the SDK files `CommonPlugIn_↔CompilerCmd.cmd` and `CommonPlugIn_LinkCmd.cmd` for the latest recommended settings.

12.45.8.2.1 Overview of optimization-related compiler options

- `-g` Full symbolic debug. This setting should be used in debug configurations to make stepping through code easier. It should not be defined in release configurations, as it will prevent the compiler from being able to fully optimize code.
- `-k` Keep generated .asm files. This should be turned on in release configurations so that you can use the ASM output as feedback when making optimization decisions and performance improvements.
- `-d "_DEBUG"` Defines the `_DEBUG` preprocessor macro that alters how certain code is generated (asserts, stdlib, etc). This should be turned on in debug configurations only. Note that TI does not require `NDEBUG` to be defined in release configurations.

Note

This will eventually be deprecated in favor of the pre-defined `"_TMS320C6X"` macro.

- `-mv67p` Specifies that the compiler should build code for the C67x+ chip variant we are using, which has some improvements beyond the original C67x. This option should be enabled in all build configurations that target the HDX platform.
- `-s` Specifies Opt-C/ASM interlisting. This interweaves modified C-code and ASM in the .ASM file produced by the `-k` option. You should use `-s` in release configurations so that the ASM file can be read more easily.

Note

Do NOT use the `-ss` option in release configurations. This option will negatively affect optimization

- `-pm` Program mode compilation. Instructs the C compiler to compile all files in the same compilation unit, so that it can optimize code further using information from all files being compiled. See [Program Mode optimization \(-pm\)](#) for more information.
- `-op3` A modifier for the `-pm` option, this specifies that there are no external variable references in the project. This option is appropriate for TI algorithms, which do have an external function reference (the process entry point) but do not have external variable references. This option allows the compiler to further optimize global variables without worrying whether they will be accessed outside of the compilation unit. See [Program Mode optimization \(-pm\)](#) for more information
- `-o3` File-level optimization. This flag gives the compiler full ability to optimize C-code by reordering instructions, inlining functions, and performing other optimizations. Note that the resulting ASM code will be very difficult to parse back into the original C and will make debugging very difficult, so this flag should only be used for Release code. See [Optimization flags \(-o\)](#) for more information.
- `-mo` Use Function Subsections. This instructs the compiler to place all functions into their own separate subsection in the linker map. This allows the linker to remove unused functions in order to reduce memory usage.

- `-mw` Generate a single iteration view of SP loops. This flag adds important information to the ASM output file that is useful when optimizing your code for pipelined loops.
- `-verbose` Output verbose status messages when compiling files. Though not very useful for humans, verbose output will produce some key information that text parsers can use, such as compiler versions and other details.

12.45.8.2.2 Overview of optimization-related linker options

- `-relocatable` Generate a relocatable non-executable.
- `-m"file.map"` Generate a map file. This file contains useful information about the memory footprint of your plug-in, which is useful for fixing large plug-ins that may not have fit into available program memory.
- `-w` Warn about output sections. This flag generates very useful information that tells you if there might be a problem with memory output sections you are trying to generate.
- `-x` Exhaustively read libraries. This is a useful flag if you do not want to worry about the order in which you specify required libraries.

12.45.8.2.3 Optimization flags (-o)

- Register (`-o0`) This option allows for some performance gains over non-optimized code by allocating variables to registers, inlining functions declared inline, etc.
- Local (`-o1`) This option enables local optimizations, with very similar results to the register-level optimizations of `-o0`.
- Function (`-o2`) This is the standard optimization level, and provides large gains over unoptimized code. This optimization level allows function-level optimizations such as software pipelining, loop optimization/unrolling, etc.
- File (`-o3`) This option can provide some speedup beyond function-level optimizations, but also mutilates assembly code beyond recognition. At this optimization level the compiler will remove unused functions, simplify code in the case of unused return values, auto-inline small functions, etc.

Like the corresponding Visual Studio options, `-o0` and `-o1` allow you to step through code line-by-line for debugging, at the cost of reduced performance. `-o2` and `-o3` sacrifice the ability to step through code and watch memory in favor of optimized code.

12.45.8.2.4 Program Mode optimization (-pm) Program mode optimization gives the compiler further optimization information by compiling all files at once rather than individually. Thus global constants, function implementations, etc. can be made known to the entire program at compilation. This allows the compiler to inline functions more effectively and to determine loop unrolling based on constant loop iterators.

There are a few `-pm` options:

- `-pm -op0` Contains functions and variables that are called or modified from outside the source code provided to the compiler.
- `-pm -op1` Contains variables modified from outside the source code provided to the compiler but does not use functions called from outside the source code.

This option is not appropriate for AAX plug-in algorithms, because the algorithm component will be exported and called from outside the compiled source code.

- `-pm -op2` Contains no functions or variables that are called or modified from outside the source code provided to the compiler.

This option is not appropriate for AAX plug-in algorithms, because the algorithm component will be exported and called from outside the compiled source code.

- `-pm -op3` Contains functions that are called from outside the source code provided to the compiler but does not use variables modified from outside the source code.

This is the recommended Program Mode optimization level for TI plug-ins. This optimization level requires that no global variables are used outside of the algorithm callback. In general, any such variables should be passed in to a TI algorithm via the algorithm's context structure.

12.45.8.2.5 Compiler options to avoid The following information was taken from the TMS320C6000 Programmer's Guide:

- `-g/-s/-ss` These options limit the amount of optimization across C statements, leading to larger code size and slower program execution.
- `-mu` This option disables software pipelining for debugging. If a reduction in code size is necessary, use the `-ms2/-ms3` options. These options will disable software pipelining among their other code size optimizations.
- `-mz` This option is obsolete. When using 3.00+ compilers, this option will decrease performance and increase code size.

12.45.8.3 The load-update-store pattern

The load-update-store pattern is one of the cornerstones of a fast iterative algorithm. This pattern specifies that locally accessed data should be loaded into memory at the start of processing, accessed during processing, and stored or saved after processing has completed. By using this pattern you will move memory reads and writes outside of your plug-in's innermost processing loop, which reduces data dependencies and shortens the critical inner loop.

As an example, consider the following unoptimized filter code:

```
inline void
ProcessDirectFormII(float* input, float* output, float* state, float*
    coefs, int nsamp)
{
    // eB0 .. eB2 and eA0, eA1 are just integer enums to partition
    // the filter coefficients into A and B
    for(int i = 0; i < nsamp; ++i)
    {
        output[i] = input[i]*coefs[eB0] + state[0];
        state[0] = input[i]*coefs[eB1] + state[1] - output[i]*coefs[eA0];
        state[1] = input[i]*coefs[eB2] - output[i]*coefs[eA1];
    }
}
```

Listing 9: Unoptimized filter algorithm

Notice that in this code there are at least 15 memory accesses per loop iteration! This algorithm will be very inefficient as the value of `nsamp` increases.

The compiler should be able to optimize this algorithm to some extent by pulling certain memory accesses outside of the loop. However, the compiler cannot completely optimize the loop because it must assume that the input/output/state/coefs pointers are aliased in memory. We will discuss the `const` and `restrict` keywords later, which are ways to give the compiler additional information it can use to optimize this loop. However, for now let's focus back on the basic design of this code.

Using load-update-store, we can refactor this loop to pull the memory accesses outside of the loop:

```
void
ProcessDirectFormII (float* input, float* output, float* state, float *
    coefs, int nsamp)
```

```

{
    // eB0 .. eB2 and eA0, eA1 are just integer enums to partition
    // the filter coefficients into A and B
    // ---- LOAD ----
    float coefA0 = coefs [eA0];
    float coefA1 = coefs [eA1];
    float coefB0 = coefs [eB0];
    float coefB1 = coefs [eB1];
    float coefB2 = coefs [eB2];
    float state0 = state [0];
    float state1 = state [1];
    float output;
    // ---- UPDATE ----
    for (int i = 0; i < nsamp; ++i)
    {
        output = input [i]* coefB0 + state0;
        state0 = input [i]* coefB1 + state1 - output * coefA0;
        state1 = input [i]* coefB2 - output * coefA1;
        output [i] = output;
    }
    // ---- STORE ----
    state [0] = state0;
    state [1] = state1;
}

```

Listing 10: Refactored filter algorithm with load-update-store pattern applied. Not fully optimized.

Though the code initially appears longer, you will notice that we have reduced the loop to only 4 memory accesses! Though we have an additional 9 memory accesses outside the loop, they will only occur once per function call, resulting in significant savings at higher values of `nsamp`.

Note

we are not finished with this loop yet, because we can make some very significant gains by using the `restrict` and `const` keywords, as discussed in the section on [C keywords](#).

Before moving on from load-update-store, let's consider how this pattern should be applied to different categories of data that may be provided in an AAX DSP processing context:

- **Coefficients and parameters** Coefficients and parameters are read-only by definition. As such, they should be loaded into a local variable at the beginning of the algorithm callback and should not be modified further.
- **Private state** State parameters are writable and may be changed by the algorithm. Therefore, private state data should be loaded into a local variable copy, then stored back into memory after the local copy is updated.
- **Output** Output is write-only, so all calculations may be performed on a local variable and then stored into memory once per loop.

12.45.8.4 Case study: IIR filter implementation on TI 672x DSPs

In this section we will examine various IIR filter implementations as a specific example of the considerations that must be made when optimizing DSP code for the 672x.

The TI 67xx family of DSPs is notably different from some other typical DSP processors, such as the 56k and the Intel FPU, in that the TI DSP does not have an implicit higher-precision multiply-accumulate. It is of course capable of double precision accumulation, but this must be coded explicitly. In some ways, this is similar to the Intel SSE processing unit, which jettisoned the 80-bit floating point stack used in the Intel FPU. The lack of higher precision accumulation in TI (and SSE) can sometimes result in unacceptable quantization noise performance for single precision filter implementations. Luckily, with the right choice of filter structure or coding for explicit double precision accumulation, excellent results can be achieved.

On fixed-point DSPs such as 56k, Direct Form I (DF1) implementation is the standard due to moderately good fixed point scaling properties, decent noise performance, and simple implementation. However, on a 672x DSP a single precision DF1 filter can have terrible noise performance (depending on the filter coefficients and the audio

material being processed.) A degenerate case is a DF1 highpass filter processing low frequency material; in DF1, the feedforward coefficients subtract the previous sample from the current sample, and for low frequency material this produces very small numbers with low precision. Single precision DF2 structures also produce similarly poor results in this respect.

One option to improve upon these results is to use double precision throughout the 672x filter implementation. However, this results in a heavy cycle performance penalty due to the high cost of double operations on the TI DSP.

Another, often better, option is to use single precision coefficients and state, with double precision accumulation:

```
float in, b0, b1, a1, state1;
double accum;
accum = double (b0) * double (in) +
        double (b1) * double (state1) +
        double (a1) * double (accum);
state1 = in;
```

Listing 11: Mixed-precision DF1 filter implementation

The TI compiler will implement this using the mpysp2dp instruction, since it knows that the operands started out as single precision and end up as double precision. This is considerably faster than going to a full double precision implementation, but it is still relatively slow compared to straight single precision. Making the state double precision will improve noise performance further, with some increase in cycle usage.

Another option that generally gets good results is the single precision DF2 Transpose (DF2T) filter. On TI the DF2T implementation is fast and generally has good noise performance. If you are looking for a simple recommendation that should work well enough for most applications, DF2T is a good choice.

The optimized C filter library available from TI uses the DF2 structure in its implementation. Even though DF2 has some limitations, this is a good starting point for seeing how to optimize filter code on TI; peak performance on TI is 2.25 cycles per biquad, so it's pretty amazing what can be done (to achieve that level of performance multiple series or parallel biquads need be put in a tight loop.) We have adapted some of this filter code to DF2T, and still achieved fairly similar cycle performance.

If the single precision DF2T noise performance is not good enough for your application, then either double precision or one of the myriad other filter structures, such as State Space, Gold-Rader, Lattice or Zolzer, should do the job. In fact, there is one relatively new filter structure which we think stands out, called the Direct Wave Form (DWF) filter. Details about this filter structure can be found in *Direct Wave Form Digital Filter Structure: an Easy Alternative for the Direct Form* by Jean H.F. Ritzterfel. According to the author the noise performance is 3dB within optimal, it's relatively efficient (5 multiplies per biquad), free of limit cycles, has simple coefficient generation and low coefficient quantization sensitivity. It might just be the perfect filter structure, but we'll let you be the judge of that; keep in mind that all filter structures have some tradeoffs, and the recommendations made here might not be the best for your particular application.

12.45.8.5 Understanding CGTools-generated ASM files

The ability to read the ASM files that are generated by CGTools is essential when optimizing a TI algorithm. Specifically, the information in these files will allow you to determine if anything is preventing software pipelining from occurring, which is the single most effective form of optimization on the C6727.

To view your project's ASM file, turn on the `-k` compiler option ("Keep Generated .asm Files", found under Build Options > Compiler > Assembly in the Code Composer Studio IDE.) By default, ASM files will be placed in the same directory as the corresponding source file.

Note

You should only examine ASM listings of Release code that has been optimized by the compiler. Debug code should not be optimized.

Each ASM file for a TI algorithm callback should contain text that marks the start of the assembly listing for the processing loop. For example:

```
*****
; * FUNCTION NAME: // [Your algorithm's ProcessProc symbol] _____ *
; * _____ *
; * Regs Modified: A0,A1,A2,A3,A4,A5,A6,A7,A8,A9,A10,A11,A12,A13,A14, _ *
; * _____ *
; * _____ *
; * _____ *
; * _____ *
; * _____ *
; * Regs Used: A0,A1,A2,A3,A4,A5,A6,A7,A8,A9,A10,A11,A12,A13,A14, _ *
; * _____ *
; * _____ *
; * _____ *
; * _____ *
; * _____ *
; * _____ *
; * Local Frame Size: 0 Args + 148 Auto + 44 Save = 192 byte _____ *
*****
```

Listing 12: CGTools-generated header for a processing loop assembly listing

Within this listing, you are looking for several things:

1. Function calls
2. Branches or control code
3. Software pipelining notes

```
12.45.8.5.1 Function calls      [!B0]  CALL  .S1  __divd      ; [213]
|| [!B0] MVKH  .S2  0x40080000 ,B5 ; [213]
|| [ B0] MV   .L1X B10 ,A4      ; [213]
;$SRL9 : ; CALL OCCURS {__divd}   ; [213]
```

Listing 13: Function call in a CGTools-generated assembly listing

Function calls, such as the call in the listing above, cannot be effectively pipelined. If you find a function call figure out what C instruction it is caused by. Sometimes a function call will be made implicitly, such as when casting from float to int or when doing division. All function calls should be removed from the processing loop or inlined in order for the compiler to optimize effectively.

```
12.45.8.5.2 Branches           NOP      1
B      .S1  $C$L5      ; [213]
NOP      4
MPYDP  .M1X  A5:A4 ,B5:B4 ,A11:A10 ; [213]
|| LDW   .D2T2  ** SP (124) ,B5      ; [218]
; BRANCH OCCURS { $C$L5 }           ; [213]
```

Listing 14: Branch in a CGTools-generated assembly listing

Branches can also prevent loop pipelining. If you find a branch in your algorithm's assembly, determine whether it is preventing the compiler from pipelining a loop. If it is preventing pipelining, you must figure out how to rewrite the conditional in your C code so that it will not be compiled into a branch.

12.45.8.5.3 Software pipelining notes For each loop the compiler finds and is able to pipeline, the .ASM file should contain a section similar to the one below:

```

/*-----*
/* SOFTWARE PIPELINE INFORMATION
/*
/* Loop source line : 68
/* Loop opening brace source line : 69
/* Loop closing brace source line : 124
/* Loop Unroll Multiple : 2x
/* Known Minimum Trip Count : 1
/* Known Max Trip Count Factor : 1
/* Loop Carried Dependency Bound (^) : 15
/* Unpartitioned Resource Bound : 20
/* Partitioned Resource Bound (*) : 20
/* Resource Partition :
/* A- side B- side
/* .L units 0 0
/* .S units 0 1
/* .D units 20* 20*
/* .M units 7 5
/* .X cross paths 5 6
/* .T address paths 20* 20*
/* Long read paths 5 1
/* Long write paths 0 0
/* Logical ops (. LS) 5 4 (.L or .S unit )
/* Addition ops (. LSD) 0 1 (.L or .S or .D unit )
/* Bound (.L .S .LS) 3 3
/* Bound (.L .S .D .LS .LSD) 9 9
/*
/* Searching for software pipeline schedule at ...
/* ii = 20 Schedule found with 3 iterations in parallel

```

Listing 15: Pipelined loop header in a CGTools-generated assembly listing

These are the important items to note in this listing:

- **Loop Carried Dependency Bound and Partitioned Resource Bound** The maximum of these numbers is the minimum number of clock cycles one instance of the loop will require in its current form. You can reduce these numbers by performing some of the optimizations listed in this guide.
- **Loop Unroll Multiple** This line will appear if the compiler is partially unrolling the loop to improve performance.

If a loop section instead displays `Disqualified loop`: then some of the conditions required to enable software pipelining have not been met:

- `-o2` or `-o3` optimizations must be enabled
- The loop cannot contain a function call. Make all called functions inline.
- The loop cannot contain any branches or jumps, often caused by large conditional statements
- Software pipelining will not work with nested loops; only the innermost loop will be pipelined. You should completely unroll the inner loop or refactor the algorithm so that the loop can be pipelined

For more information about pipelining and loop/branch optimization, see [Refactoring conditionals and branches](#).

12.45.8.6 C keywords

There are a few keywords in C that give the compiler additional information about the variables you declare and parameters you pass into functions. This allows the compiler to further optimize the code it is compiling, which can result in significant performance gains.

12.45.8.6.1 const Effective use of `const` lets the compiler know whether pointers, scalars, or objects will remain constant in memory.

Let's add the `const` keyword to the filter function from our example of [The load-update-store pattern](#).

```
void
ProcessDirectFormII (
    const float * const input, // read - only
    float * const output, // read - write
    float * const state, // read - write
    const float * const coeffs, // read - only
    int nsamp )
{
    // eB0 .. eB2 and eA0, eA1 are just integer enums to partition
    // the filter coefficients into A and B
    // ---- LOAD ----
    const float coefA0 = coeffs [ eA0 ];
    const float coefA1 = coeffs [ eA1 ];
    const float coefB0 = coeffs [ eB0 ];
    const float coefB1 = coeffs [ eB1 ];
    const float coefB2 = coeffs [ eB2 ];
    float state0 = state [0];
    float state1 = state [1];
    // ---- UPDATE ----
    for (int i =0; i<nsamp; ++i)
    {
        const float output = input [i]* coefB0 + state0 ;
        state0 = input [i]* coefB1 + state1 - output * coefA0 ;
        state1 = input [i]* coefB2 - output * coefA1;
        output [i] = output;
    }
    // ---- STORE ----
    state [0] = state0;
    state [1] = state1;
}
```

Listing 16: Refactored filter algorithm with load-update-store pattern and `const` keyword applied.

It is especially important to note that the declaration of `const float output` was moved inside the loop. Why did we do this? Because we see that `output` is constant over an iteration of the loop, but it does change between iterations. By declaring it `const` inside the loop body we remove the data dependency that existed in `output` and allow the loop to optimize more effectively.

As demonstrated by this change to `const float output`, `const` is useful for manually breaking dependencies in DSP code. Variable re-use introduces unnecessary data dependencies in code, which can be avoided by using individual local `const` variables.

12.45.8.6.2 restrict The `restrict` keyword tells the compiler that a specific pointer is not aliased, meaning that none of the memory locations accessed by the pointer are read or written to by any other variable within its local scope. This keyword is very important when optimizing TI code that involves pointers, as all AAX algorithms do due to the nature of the algorithm context structure.

`restrict` was introduced with the C99 standard. AAX plug-ins use the `AAX_RESTRICT` keyword, which is a cross-platform macro for the C99 standard `restrict`.

Note

Now that MSVC has added C99 support to its compiler, `AAX_RESTRICT` will eventually be deprecated in favor of the `restrict` keyword.

The following example demonstrates the use of `restrict` in our filter code.

```
void
ProcessDirectFormII (
    const float * const AAX_RESTRICT input,
    float * const AAX_RESTRICT output,
    float * const AAX_RESTRICT state,
    const float * const AAX_RESTRICT coeffs,
    int nsamp )
{
    // eB0 .. eB2 and eA0, eA1 are just integer enums to partition
    // the filter coefficients into A and B
```

```

// ---- LOAD ----
const float coefA0 = coefs [ eA0 ];
const float coefA1 = coefs [ eA1 ];
const float coefB0 = coefs [ eB0 ];
const float coefB1 = coefs [ eB1 ];
const float coefB2 = coefs [ eB2 ];
float state0 = state [0];
float state1 = state [1];
// ---- UPDATE ----
for (int i =0; i< nsamp; ++i)
{
    const float output = input [i]* coefB0 + state0;
    state0 = input [i]* coefB1 + state1 - output * coefA0;
    state1 = input [i]* coefB2 - output * coefA1;
    output [i] = output;
}
// ---- STORE ----
state [0] = state0;
state [1] = state1;
}

```

Listing 17: Refactored filter algorithm with load-update-store pattern and `const` and `restrict` keywords applied.

Note

- This example applies `restrict` to the algorithm's input and output audio buffer pointers. These pointers do not alias each other in most algorithms, but this may not be the case for all algorithms and should be verified by the developer before applying `restrict`.
- The `restrict` keyword is somewhat redundant when used with the load-update-store pattern. This is because by asserting to the compiler that the pointers are not aliased, it should be able to partially do the load-update-store refactoring automatically. However, because some compilers have limited or no support for the `restrict` keyword, using the load-update-store pattern is still recommended.

12.45.8.6.3 Keywords to avoid There are some keywords which do more harm than good, but are still being used either due to legacy code or developer superstitions. These keywords should not be used in AAX plug-ins.

- `register` The `register` keyword is a suggestion to the compiler that a certain variable will be accessed frequently and should be stored in a register rather than a memory location. Use this keyword only when you are sure that the compiler is placing a frequently-used variable in memory when it would be advantageous to keep it in a register. Note that the `register` keyword has no effect if the CGTools optimizations are enabled.
- `static` In C, the `static` keyword tells the compiler to initialize the variable at compilation time and retain the value between calls. Though there are some valid situations to use the `static` keyword, its use in AAX plug-ins on all platforms is extremely limited. One of its most "popular" uses, declaring local variables inside a function as `static` in order to achieve a type of global counter, should never be used in AAX algorithm code. If you are using `static` to make a local variable hold its variable across calls to a function, it is always preferable to either pass it in to the function as a modifiable parameter or declare it as a member variable of the method (if C++).

12.45.8.7 Data types

The TI C672x+ is a 32-bit floating point DSP platform, and has a few peculiarities that you should be aware of.

- Use `int` instead of `long` Integers of type `long int` are 40 bits wide on TI, and are very inefficient. Always use the `int` data type (or, even better, the C99-standard `int32_t`) instead.
- Use `float` instead of `double` Double-precision floating-point data types have a significant performance penalty on TI processors. Use `float` instead of `double` wherever possible, as long as this substitution does not affect signal integrity or cancellation.
- Use unsigned values when referencing memory In general, explicitly typed pointers should always be used to reference memory. If you do have need of a generic memory representation, use an unsigned integer to avoid implicit conversion costs.

12.45.8.7.1 Unintended data type conversions When developing for the TI platform it is important to keep an eye out for unintended type conversions, and especially for implicit double-precision instructions. The following points are helpful for both program efficiency and for future maintenance of the code, since they clarify the developer's understanding of how the code should operate, e.g. by specifying that a cast is occurring, and make it obvious that steps such as data type conversions are an intentional part of the algorithm.

- Explicitly declare constants as single-precision. For example, use `0.0f` instead of `0.0`. Often a compiler will be able to do this automatically at compile time, but it is better to be explicit with your intended precision.
- If any casts are required in your code, make them explicit. For example, `float output = (float)doubleVar` as opposed to `float output = doubleVar`.
- Use single-precision `math.h` functions (such as `fabsf()`) instead of the double-precision equivalents (`fabs()`).
- Do not directly reference memory addresses using integer data types; instead, use a pointer data type. If an integer data type is required, use an unsigned 32-bit type.

To help ensure that you are not violating these principles, always be aware of any warnings generated by the compiler. In particular, do not ignore warnings related to "implicit conversion from 'double' to 'float'" or "implicit conversion from 'double' to 'int'"; these warnings may indicate that you are declaring a double when a float would be just as good.

In the final stages of optimization, examine the generated assembly code to make sure there are no unintended double-precision instructions or memory accesses.

12.45.8.7.2 Additional data type optimizations The AAX SDK includes cross-platform macros that can be used to convert two single-precision float loads to one double-precision load. The coefficient smoothing case study below includes an example use case for these macros.

```
const float * pTable = &SmoothCoefTable[address];
AAX_ALIGNMENT_HINT(pTable,8);
float firstCoef = AAX_LO(*pTable);
float secondCoef = AAX_HI(*pTable);
```

Listing 18: Example of using AAX macros for converting two `float` loads to one `double` load.

In this example the `AAX_ALIGNMENT_HINT` macro checks whether data is aligned on a 8-byte boundary, then the double word is loaded, and finally the `AAX_LO` and `AAX_HI` macros get the double word's first and second (`float`) parts.

If `SmoothCoefTable` consists of floats and is 8-byte aligned, then this scenario will work fine for loads when `address` is even. This raises the question about how to load double word from `&SmoothCoefTable[address]`, when `address` is odd. Since this kind of optimization is most useful for loading data from external memory, where the CPU savings of a single double word load vs two 32-bit loads is greatest, then one trick which can help is to trade off memory (as external memory is plentiful) for performance. Specifically, `SmoothCoefTable` can be organized in a such way that for every member of this table, except the first and the last ones, there will be two consequent entries.

```
const int32_t size = 4;
// instead of this classic variant...
const float SmoothCoefTable[size] = {
    -0.1, -0.2, -0.3, -0.4
}
// ...table can be organized this way
const float SmoothCoefTable[size*2 - 2] = {
    -0.1, -0.2,
    -0.2, -0.3,
    -0.3, -0.4,
    -0.4, 0.0 /* last member is dummy */
}
```

Listing 19: Example of restructuring the table so that it can be easily used in the optimization scenario given above.

In this case the number of loads will be halved at the cost of doubling the size of the table. If the table is located in external memory then the additional memory requirement can be an excellent trade-off for the performance gained.

12.45.8.8 Case study: Efficient parameter smoothing at single and double precision

Coefficient smoothing ("de-zippering") can often be one of the most difficult parts of a plug-in to optimize for real-time operation. This is especially true in cases when full double-precision smoothing filters have been used in a plug-in's Native code, with the possibility of very small coefficients. In these cases it can be difficult to optimize the smoothing code while also satisfying requirements for audio data parity between the plug-in's Native and DSP configurations.

```
double * const AAX_RESTRICT deZipper = dzCoefsP->mDeZip [ch ][0];
const double * AAX_RESTRICT coefs = myCoefsP->mBiqCoefsBuf [0];
// Double - precision
for (int i = 0; i < eNumBiquads * eNumCoefs ; ++i)
{
    double dz = deZipper [i];
    dz += zeroCoef * ( coefs [i] - deZipper [i]);
}
```

Listing 20: Example of double-precision smoothing.

In this section we will describe three specific approaches that may be taken to perform optimized real-time smoothing without compromising sound quality.

12.45.8.8.1 Method 1: Clamped single-precision smoothing The simplest approach for optimization of a double-precision smoothing filter is to replace it with modified single-precision smoothing. Unfortunately, we have found that this approach can lead to glitches and instability at higher sample rates when adjusting controls due to transient inaccuracies in the smoothing.

```
double * const AAX_RESTRICT deZipper = dzCoefsP->mDeZip [ch ][0];
const double * AAX_RESTRICT coefs = myCoefsP->mBiqCoefsBuf [0];
// Method 1 - single - precision
for (int i = 0; i < eNumBiquads * eNumCoefs ; ++i)
{
    float dz = deZipper [i];
    dz += zeroCoef * ( coefs [i] - deZipper [i]);
    // If the de -zip step is so small that the coefficient doesn't change then clamp
    // the value to the target to ensure we are using exactly the desired value .
    deZipper [i] = (dz == deZipper [i]) ? coefs [i] : dz;
}
```

Listing 21: Example of clamped single-precision smoothing.

12.45.8.8.2 Method 2: Mixed-precision smoothing To resolve the stability issues at high sample rates, the state may be accumulated at double-precision. This results in mixed-precision operations that are much faster on TI DSPs than full double-precision calculations, though still slower than single-precision.

```
float * const AAX_RESTRICT deZipper = dzCoefsP->mDeZip [ch ][0];
double * const AAX_RESTRICT deZipState = dzCoefsP->mDZState [ch ][0];
const float * AAX_RESTRICT coefs = myCoefsP->mBiqCoefsBuf [0];
// Method 2 - partial double precision
# pragma UNROLL ( CBiquad::eNumCoefs )
for(int i = 0; i < eNumBiquads * eNumCoefs ; i ++ )
{
    double dz = deZipState [i];
    dz += zeroCoef * ((coefs [i]) - ( deZipper [i]));
    deZipState [i] = dz;
    deZipper [i] = float (dz);
}
```

Listing 22: Example of mixed-precision smoothing.

12.45.8.8.3 Method 3: Loop unrolling and double-word memory accesses Further performance gains can be made by unrolling the loop and using double word memory accesses. This code is faster, but is still not as fast as full single-precision.

```
float * const AAX_RESTRICT deZipper = dzCoefsP->mDeZip [ch][0];
double * const AAX_RESTRICT deZipState = dzCoefsP->mDZState [ch][0];
const float * AAX_RESTRICT coefs = myCoefsP->mBigCoefsBuf [0];
// Method 3 - partial double precision - unrolled with double-precision memory accesses for(int i = 0; i <
    (eNumBiquads * eNumCoefs); i +=2 )
{
    double dz0 = deZipState [i];
    double dz1 = deZipState [i+1];
    dz0 += zeroCoef * (AAX_LO ( coefs [i]) - AAX_LO ( deZipper [i]));
    dz1 += zeroCoef * ( AAX_HI ( coefs [i]) - AAX_HI ( deZipper [i]));
    deZipState [i] = dz0;
    deZipper [i] = float (dz0);
    deZipState [i+1] = dz1;
    deZipper [i+1] = float (dz1);
}
```

Listing 23: Example of loop unrolling and double-precision memory accesses for smoothing optimization.

12.45.8.8.4 Coefficient smoothing example summary

- Full single-precision smoothing (method 1) is an excellent and simple solution for gain coefficients and other scalar values which are not extremely sensitive to coefficient quantization at small values. This method does not always reach the target value, so clamping should be used to ensure signal integrity.
- Mixed-precision smoothing (method 2) uses slightly more CPU, but gives full double precision accuracy. This approach should generally be used for EQs and other sensitive coefficients.
- Further low-level optimizations are also possible via manual loop unrolling and double-precision memory access (method 3).

12.45.8.9 Refactoring conditionals and branches

Note

For more detailed information on how to reduce or eliminate the use of branches in algorithms, see section 5.2 of the **Hand-Tuning Loops and Control Code on the TMS320C6000** guide provided by TI.

An important technique in refactoring algorithms to enhance loop performance is to reduce or eliminate conditionals and branches in code. The TI compiler focuses a lot of its optimization energy on keeping its pipeline full of inside loops. However, it cannot pipeline a loop if the one of the following is true:

- The loop contains a branch
- The loop contains a function call
- The loop is too long

To demonstrate this, we will again begin with an unoptimized example:

```
for ( int i = 0; i < numSamples ; ++i)
{
    if (! bypass )
    {
        const float filtOutput1 = input [i] * coef0 + state0 * coef1 ;
        const float filtOutput2 = filtOutput1 * coef2 + state1 * coef3 ;
        output [i] = filtOutput2 ;
    }
    else
    {
        output [i] = input [i];
    }
}
```

Listing 24: Another unoptimized filter algorithm.

Though trivial, this example illustrates the problem with conditionals inside of loops. In TI assembly, conditional code usually translates into code branches, which prevents loops from pipelining effectively see [Understanding CGTools-generated ASM files](#). Let's refactor the loop in our example to reduce the size of its conditional branch:

```
for (int i = 0; i < numSamples ; ++i)
{
    const float filtOutput1 = input [i] * coef0 + state0 * coef1 ;
    const float filtOutput2 = filtOutput1 * coef2 + state1 * coef3 ;
    output [i] = filtOutput2 ;
    if ( bypass )
    {
        output [i] = input [i];
    }
}
```

Listing 25: Filter algorithm with a refactored conditional branch.

At first, it may seem wasteful to perform the filter calculation if `bypass` will simply throw away the result. In reality, however, the opposite is true: as a real-time algorithm, this code is constrained by its maximum, worst-case cycle count. It is important to understand this point: essentially, the cycle count of the plug-in is always its worst-case performance.

By reducing the algorithm's maximum cycle count we are therefore reducing waste, even though we are increasing the plug-in's cycle count when it is bypassed. In fact, the ideal scenario for most algorithms is to use only one code path (and, consequentially, a single deterministic cycle count) despite the fact that this can result in worse performance for some specific states. To state this fundamental principle in a different way:

The performance of specific states in an AAX DSP algorithm is not relevant if there is another possible state with worse performance.

Going back to our optimized example, you may also notice that the conditional still exists. Doesn't this create a branch in the assembly code as well and prevent pipelining?

In the case of very brief conditionals such as this, the answer is usually no. On TI processors, most instructions can be executed conditionally, depending on the value of a control register. Thus, the single assignment (`output = input`) inside this conditional will reduce to a few conditional instructions without having to execute a branch. As a result, the TI compiler will be able to efficiently pipeline this loop.

That said, it is occasionally necessary to eliminate conditionals entirely. One effective solution for these situations is to execute the branched logic algorithmically rather than conditionally. To demonstrate this approach, here is our filter example again, this time with the conditional completely eliminated from the loop:

```
for (int i = 0; i < numSamples ; ++i)
{
    const float filtOutput1 = input [i] * coef0 + state0 * coef1 ;
    const float filtOutput2 = filtOutput1 * coef2 + state1 * coef3 ;
    output [i] = (! bypass ) * filtOutput2 + bypass * input [i];
}
```

Listing 26: Filter algorithm with branching logic executed algorithmically.

This code is shorter and completely eliminates the conditional from inside the loop body. However, there is an associated cost in readability, in that it is not initially obvious how exactly `bypass` affects the output. This is of course a tradeoff that you will need to consider on a case-by-case basis. In general, we encourage you to consider this technique only when you have verified in the assembly code that simply reducing the size of the conditional is not enough to achieve effective instruction pipelining.

Another useful technique for optimizing loops is to use `pragma MUST_ITERATE` and `pragma PROBABLY_ITERATE` (see more about these pragmas in [Loop controls](#)), which help the compiler guess the number of iterations for the loop. It is extremely useful when you know the exact number of the iterations, and this number never changes during plug-in processing. For example, this is applicable for the loops which iterate through the audio samples in the input and output buffers. The number of input samples is always constant for an AAX DSP plug-in

algorithm; the buffer length must be described with the option `AAX_eProperty_DSP_AudioBufferLength` for each DSP component in the plug-in's description.

The following code example shows an algorithm processing function template. For convenience, this function template takes the audio buffer length as a template parameter:

```
template<int kAudioWindowSize>
void AAX_CALLBACK
Example_AlgorithmProcessFunction( SExample_AlgorithmContext * const inInstancesBegin [], const void *
                               inInstancesEnd)
{
    for (SExample_AlgorithmContext * const * walk = inInstancesBegin; walk != inInstancesEnd; ++walk)
    {
        SExample_AlgorithmContext* const AAX_RESTRICT contextP = *walk;
        const float * const AAX_RESTRICT inputP = contextP->mInputPP;
        float * const AAX_RESTRICT outputP = contextP->mOutputPP;
        #pragma MUST_ITERATE( kAudioWindowSize, kAudioWindowSize, kAudioWindowSize )
        for (int32_t i = 0; i < kAudioWindowSize; ++i)
        {
            outputP[i] = inputP[i];
        }
    }
}
```

Listing 27: Optimizing loop using pragma `MUST_ITERATE`.

Note that the audio buffer length property takes a `AAX_EAudioBufferLengthDSP` value. The values of this enum are set to the power-of-two for each buffer length, so in this case the `kAudioWindowSize` value would be set to match `2 << AAX_eProperty_DSP_AudioBufferLength` when compiling this algorithm callback into the TI DLL.

The same optimization can be used for the loops that iterate through input/output channels, as demonstrated by the DemoDist example plug-in.

12.45.8.10 Case study: pipeline refactoring in Avid's EQ3 and Dyn3 plug-ins

While optimizing the "stock" Pro Tools equalization and dynamics processors we came across many real-world optimization scenarios that will be applicable to a broad variety of plug-ins. In this section we will consider specific techniques that we used to enable software pipelining of these algorithms by the TI compiler, including an in-depth look at the pseudo-speculative execution approach used in our Dyn3 plug-in's polynomial gain calculation loop.

12.45.8.10.1 Move individual processing operations into separate loops Oftentimes a sample-by-sample iterative loop that is not software pipelining can be broken up into individual loops that incrementally apply changes to the audio buffer. These smaller loops have a much better chance of being successfully pipelined by the compiler. In EQ3, moving our biquad audio processing stages to dedicated loops that do not include coefficient smoothing or other tasks resulted in large performance gains.

12.45.8.10.2 Avoid pipeline dependencies The goal of the above optimization is to allow the compiler to successfully pipeline each iterative loop. However, even a pipelined loop may be optimized further. One of the best ways of optimizing loops is to keep the processor busy while pipeline dependencies are cleared.

For example, in EQ3 we found that it was better to perform the plug-in's input and output meter calculations in the same loop rather than separating them out into individual loops. This is because each meter calculation has a dependency on its previous value, which puts a dependency in the pipeline. Doing both at the same time gives the process more to do while waiting for the next value. In Dyn3 we had similar results merging table lookup, attack, and release loops into a single iterative loop. As long as the loop is still successfully pipelined by the compiler, these "larger" loops tended to have much better performance due to the reduction in blocking dependencies.

12.45.8.10.3 Detailed example of loop optimization in Dyn3 At this point it will be helpful to go into greater detail about our optimizations for Dyn3's polynomial gain calculation loop, because the increase in performance was quite large and is fairly representative of other algorithms. The unoptimized code took 43 cycles to execute one iteration of the loop. After rearranging the code it now takes 6 cycles. The basic problem was numerous pipeline dependencies: the *Loop Carried Dependency Bound* was 42 cycles, yet the *Partitioned Resource Bound* was 4 cycles. In other words, if all of these dependencies were removed the loop could potentially execute in 4 cycles.

```

2760 ;* SOFTWARE PIPELINE INFORMATION
2761 ;*
2762 ;* Loop source line : 199
2763 ;* Loop opening brace source line : 200
2764 ;* Loop closing brace source line : 213
2765 ;* Known Minimum Trip Count : 4
2766 ;* Loop Carried Dependency Bound (^) : 42
2767 ;* Unpartitioned Resource Bound : 4
2768 ;* Partitioned Resource Bound (*) : 4
2769 ;*
2770 ;* Searching for software pipeline schedule at ...
2771 ;* ii = 42 Did not find schedule
2772 ;* ii = 43 Schedule found with 1 iterations in parallel
2773 ;* Done
for (int i =0; i< kAudioWindowSize ; i++) // cSmoothingBlockSize
{
    const float * smoothCoeffs = stateP -> mSmoothedPoly ;
    float logEnv = logEnvArray [i]; // logEnvArray [ fIdx +i];
    logEnv -= smoothThrLow ;
    if( logEnv >= 0.0 f) // In the knee
        smoothCoeffs += eCpdPolyOrder ;
    if( logEnv >= 0.0 f) // In the knee
        logEnv -= smoothThrLowDelta ;
    if( logEnv >= 0.0 f) // In the linear GR stage
        smoothCoeffs += eCpdPolyOrder ;
    const float filteredLogEnv = smoothCoeffs [ eCpdPolyCoeffsC ] +
        logEnv * ( smoothCoeffs [ eCpdPolyCoeffsB ] +
            smoothCoeffs [ eCpdPolyCoeffsA ] * logEnv );
    filtLogEnvArray [i] = filteredLogEnv + smoothedMakeupGain ;
}

```

Listing 28: Dyn3's unoptimized polynomial gain calculation loop and asm listing.

- `logEnv -= smoothThrLow` *depends on the result of logEnvArray[i]*
- `if(logEnv >= 0.0f)` *depends on the result of logEnv -= smoothThrLow*
- `logEnv -= smoothThrLowDelta` *depends on the result of logEnv -= smoothThrLow*
- `Thrid if(logEnv >= 0.0f)` *depends on the result of logEnv -= smoothThrLowDelta*
- `Second smoothCoeffs += eCpdPolyOrder` *depends on the result of the first smoothCoeffs += eCpdPolyOrder*
- `logEnv*smoothCoeffs[eCpdPolyCoeffsB]` *depends on the result of logEnv -= smoothThrLowDelta*
- `smoothCoeffs[eCpdPolyCoeffs], etc.` *depend on the result of the second smoothCoeffs += eCpdPolyOrder*
- `filteredLogEnv+smoothedMakeupGain` *depends on the result of filteredLogEnv = smoothCoeffs[eCpdPolyCoeffsC]*
- `filtLogEnvArray[i]` *depends on the result of filteredLogEnv + smoothedMakeupGain*

And I don't think that even covers every case, but you get the idea. The bottom line is there is no way this loop can pipeline well. In contrast, here is the optimized code and listing file output once these dependencies have been removed:

```

2476 ;* Loop opening brace source line : 167
2477 ;* Loop closing brace source line : 179
2478 ;* Known Minimum Trip Count : 4
2479 ;* Loop Carried Dependency Bound (^) : 1
2480 ;* Unpartitioned Resource Bound : 4
2481 ;* Partitioned Resource Bound (*) : 4
2482 ;*
2483 ;* ii = 6 Schedule found with 5 iterations in parallel
for (int i =0; i< cProcessingBlockSize ; i++)
{

```

```

float logEnv = logEnvArray [i];
float logEnvThrHi = logEnv - smoothThrHigh ;
const float gainSlope = smoothThrSlope +
    logEnv * smoothSlope ;
const float gainKnee = smoothKneeC +
    logEnvThrHi * ( smoothKneeB +
        smoothKneeA * logEnvThrHi );
const bool bKnee = ( logEnv >= smoothThrLow );
const bool bSlope = ( logEnv >= smoothThrHigh );
float filteredLogEnv = bKnee ? gainKnee : 0.0f;
filteredLogEnv = bSlope ? gainSlope : filteredLogEnv ;
filtLogEnvArray [i] = filteredLogEnv ;
}

```

Listing 29: Dyn3's optimized polynomial gain calculation loop and asm listing

In this case `gainSlope` is only dependent on the loading of `logEnv`, so that can begin almost immediately. `gainKnee` must wait for `logEnvThrHi`, but `gainSlope` can be calculated during that time. `bKnee` and `bSlope` are also only dependent on `logEnv`, and start right away. The main dependency is `filteredLogEnv` which is dependent on `bKnee` and `gainKnee` and then `bSlope` and `gainSlope`. Anyhow, this is far fewer dependencies. Here is another version which runs in exactly the same number of cycles. (In fact, under the hood it may be creating the same asm code; we have not compared instruction-by-instruction.)

```

for (int i =0; i< kAudioWindowSize ; i++)
{
    float logEnv = logEnvArray [i];
    float logEnvThrHi = logEnv - smoothThrHigh ;
    const bool bKnee = ( logEnv >= thrLow );
    const bool bSlope = ( logEnv >= thrHigh );
    float filteredLogEnv = bKnee ?
        kneeC + logEnvThrHi * ( kneeB + kneeA * logEnvThrHi ) :
        0.0 f;
    filteredLogEnv = bSlope ?
        thrSlope + logEnv * slope :
        filteredLogEnv ;
    filtLogEnvArray [i] = filteredLogEnv ;
}

```

Listing 30: An alternative optimization for Dyn3's polynomial gain calculation loop.

12.45.8.10.4 But what about Native? You might expect this altered code to execute well on a TI DSP but poorly on x86. However, keep in mind that a large degree of speculative execution is used on Intel's processors. This means that pipeline dependencies due to conditionals can be broken because multiple paths are executed. In these cases, only one of the results is used and the others are thrown away. In other words, if you saw pseudo code showing the literal execution of the unoptimized code above on Intel then it would probably look a lot like the optimized code. The lesson? For TI it is important to rearrange your code so that essentially it implements speculative execution as much as possible, and if applied correctly this optimization should not negatively impact your plug-in's native performance.

12.45.8.11 Case study: Additional optimization lessons from EQ3 and Dyn3

The pipeline optimization example above is just one example, and the following techniques also helped us achieve many-fold increases in performance. Note that many of these techniques are discussed in greater detail in the sections above.

12.45.8.11.1 Watch the assembly listing In the process of optimizing these plug-ins we found their asm listing files very helpful, especially the *Loop Carried Dependency Bound* and the *Partitioned Resource Bound* information. The listing file shows how many cycles the code is taking to execute, and we could make an estimate of how far away we were from the optimal implementation by seeing how well the pipeline is being utilized.

12.45.8.11.2 Divide processing tasks over multiple calls In the old RTAS version of EQ3 the coefficients were updated (smoothed) every 8 samples. Initially, this was changed to every 4 samples in the AAX version in order to easily work with 4-sample blocks on HDX. However, we were able to achieve better results by adding "ping pong" logic that alternates between smoothing the first and second half of the coefficients on each pass. To make this work in our odd-banded EQ we had to pad the smoothing coefficients by one biquad's worth to make an even number of biquads, but regardless of this inefficiency we still achieved performance gains.

12.45.8.11.3 Eliminate branches that block pipelining Eliminating large conditional branches is critical to optimal performance on TI. This can be an especially tempting pitfall for developers who are used to coding only for x86 processors.

Consider the "ping pong" optimization described above. This logic does not break pipelining because the conditional logic that checks the state of the flag does not result in a large branch; once the ping pong value is set, the exact same logic operates in every processing callback. If instead we used an if statement to determine which "side" should execute, this would prevent pipelining optimizations and would seriously impact performance.

12.45.8.11.4 Remove double-precision operations where they are not required Here is some coefficient smoothing code from our pre-optimization EQ3 algorithm. This code was embedded in the inner biquad processing loop:

```
# pragma UNROLL ( CBiquad::eNumCoefs )
for (int k = 0; k < CBiquad::eNumCoefs; ++k)
{
    double &dz = deZipper[k];
    AAX::DeDenormal (dz);
    step[k] = zeroCoef * ( coefs[k] - dz);
}
# pragma UNROLL ( CBiquad::eNumCoefs )
for(int k = 0; k < CBiquad::eNumCoefs; ++k)
{
    double nml_dz = deZipper[k]; // read state
    nml_dz += step[k];
    biquadCoefs[k] = static_cast< float > ( nml_dz );
    deZipper[k] = nml_dz ; // write state
}
```

Listing 31: Unoptimized coefficient smoothing in EQ3

To optimize this code, we converted the logic to use single-precision de-zipper values. However, this resulted in a sonic difference due to the fact that the smoothed coefficients would not necessarily ramp all the way to the correct target value. To solve that we added a conditional "clamp" that halts the smoothing once there is no difference between the 32-bit smoothed value and the target value. On examination of the assembler output, we found that this conditional pipelines very well.

```
# pragma UNROLL ( CBiquad::eNumCoefs )
for(int i = 0; i < (cMaxNumBiquadsWithPad / 2) * CBiquad::eNumCoefs; ++i)
{
    float dz = deZipper[i];
    dz += zeroCoef * ( coefs[i] - deZipper[i]);
    deZipper[i] = (dz == deZipper[i]) ? coefs[i] : dz; // clamp
}
```

Listing 32: Optimized coefficient smoothing in EQ3

12.45.8.11.5 Make coefficients contiguous We were able to achieve significant performance gains in iterative loops like the smoothing code shown above by ensuring that all of the coefficients that would be accessed by the loop are contiguous in memory. In addition, note that in the optimized code there is only one loop, which iterates NumBiquads*NumCoefs times. This optimization is possible due to the fact that each filter's coefficients are contiguous in the coefs array.

12.45.8.11.6 Use AAX_RESTRICT wherever applicable We have found that the restrict keyword is vital for optimal performance on TI DSPs. For example, the parameter smoothing logic in our Dyn3 plug-in was reduced from 18 cycles to 3 cycles per loop iteration simply by the addition of this keyword to the applicable pointer variables.

For more information about the restrict keyword, see [restrict](#).

12.45.8.11.7 Be aware of shell overhead In the TI Shell there is code that loops through every buffered coefficient FIFO before every sample buffer in order to swap the algorithm's context field pointers to a new set of coefficients if one is available. This uses a nominal number of cycles per buffered port, which can add up very quickly in small plug-ins.

For example, before our optimizations EQ3 used eight individual buffered coefficient blocks. On investigation, we found that the shell overhead from managing these buffers added up to be roughly equivalent to the algorithm's total processing cycles! To work around this we merged the 8 coefficient blocks into one large block. The trade-off of this optimization is that more work must be done on the host to re-generate and copy the whole coefficient state every time any parameter changes, so this is an optimization that should be applied only when appropriate for the individual plug-in. For example, before our optimizations EQ3 used eight individual buffered coefficient blocks. On investigation, we found that the shell overhead from managing these buffers added up to be roughly equivalent to the algorithm's total processing cycles! To work around this we merged the 8 coefficient blocks into one large block. The trade-off of this optimization is that more work must be done on the host to re-generate and copy the whole coefficient state every time any parameter changes, so this is an optimization that should be applied only when appropriate for the individual plug-in.

12.45.8.11.8 Watch for opportunities to merge or eliminate operations Keep an eye out for unnecessary processing stages performed by your algorithm. Gain stages, phase toggles, and "dummy" coefficients are particularly good candidates for this kind of optimization. For example:

- In our EQ3 plug-in, we found that we could achieve significant performance improvement by merging the plug-in's input and output gain stages with the overall gain of the first and last biquads. As a side benefit, this reduced the total quantization noise in the algorithm.
- In our Dyn3 plug-in, we found that we were applying smoothing logic to filter coefficients that would always be zero.
- When we looked more closely at Dyn3 we found that we were also computing and discarding sidechain filter information for the LFE, which is not part of the sidechain

12.45.8.11.9 Read the TI documentation There are many helpful optimization resources available from Texas Instruments. Out of all of the TI optimization documents we encountered, we found the *Hand-Tuning Loops and Control Code on the TMS320C6000* guide to be the most helpful and complete.

12.45.8.12 Optimization on the HDX platform

12.45.8.12.1 Interrupt latency Besides the large latency due to context switching (lots of data file registers to store) and the pipeline (many stages), interrupts can be disabled around pipelined loops, which cannot be interrupted. This can be controlled with the `-mi=X` compiler option, which will disallow unsafe pipelining for loops that are longer than X cycles. See TI's documentation (SPRU187O Section 2.12) for more details and references regarding this behavior.

12.45.8.12.2 External memory access A loop which performs many reads and writes may require access to external memory. In this scenario, the loop may take 10's or even 100's of times longer to execute than the compiler expects it to!

There are two options for dealing with this:

1. Search and destroy these loops individually
 - Move all the data used by the loop to internal RAM.

- Use HDX's DMA facilities for external memory accesses.
 - `#pragma FUNC_INTERRUPT_THRESHOLD` can be used to disable pipelining on a case by case basis.
2. For modules that are known to have these loops but are not worth hand optimizing, then turn off pipelined loop optimization altogether. (`-mu` aka `-disable_software_pipelining`).

Note

This is only a problem in the C67(0-2)x ISAx used on the HDX platform. In The C64xx and C674x ISA, there is an SPLOOP command which can buffer the branches within pipelined loops to allow them to be interruptable.

12.45.8.13 Code Composer Studio optimization tools

12.45.8.13.1 Compiler Consultant The Compiler Consultant tool can be used to suggest additional optimizations.

To enable the Compiler Consultant in Code Composer Studio, do the following:

1. Set an optimization level of `-o2` or `-o3` (Found in CCSv4 under Build Options > Compiler > Basic)
2. Set the `-consultant: Generate Compiler Consultant Advise` switch (Found in CCSv4 under Build Options > Compiler > Feedback)

12.45.8.13.2 Optimization information file Optimization information files can be generated in Code Composer Studio by selecting the option Build Options > Compiler > Feedback > Opt Info File. Optimization information files have an .info extension and are placed into the project's intermediate build products directory. In general, these files list function call-graph information and describe whether or not individual functions can be inlined.

12.45.9 Error Codes

The following appendices document error codes that are specific to plug-in hosting in Pro Tools HDX and other AAX platforms based on the TI DSP environment.

12.45.9.1 -138xx: DHM Core DSP errors

These errors relate to routing and assignment problems on Pro Tools HDX hardware. Plug-ins should never be able to trigger these error codes, which indicate low-level problems in the system.

Table 1: DHM Core DSP error codes	
Value	Definition
-13801	ePSError_CTIDSP_WrongSampleRate
-13802	ePSError_CTIDSP_NoFreeStreams
-13803	ePSError_CTIDSP_StreamCreationTimeout
-13804	ePSError_CTIDSP_StreamDestruction
-13805	ePSError_CTIDSP_InactiveStream
-13806	ePSError_CTIDSP_StreamCorrupted
-13807	ePSError_CTIDSP_QueueFull
-13808	ePSError_CTIDSP_NullPointer

-13809	ePSError_CTIDSP_WrongStreamID
-13810	ePSError_CTIDSP_ImageError
-13811	ePSError_CTIDSP_ResetError
-13812	ePSError_CTIDSP_ImageVerify
-13813	ePSError_CTIDSP_DSPAlreadyInBootOrReset
-13814	ePSError_CTIDSP_TriggerInterrupt
-13815	ePSError_CTIDSP_BufferSizeNotAligned
-13816	ePSError_CTIDSP_TimeoutWaitingForHPIC
-13817	ePSError_CTIDSP_SetUHPIError
-13818	ePSError_CTIDSP_UHPINotReady

12.45.9.2 -140xx: AAX Host errors

These errors relate to logic failures in the AAX host software. These errors can be due to plug-in bugs or system configuration problems.

Table 2: AAX Host Software error codes	
Value	Definition
-14001	kAAXH_Result_Warning
-14003	kAAXH_Result_UnsupportedPlatform
-14004	kAAXH_Result_EffectNotRegistered
-14005	kAAXH_Result_IncompleteInstantiationRequest
-14006	kAAXH_Result_NoShellMgrLoaded
-14007	kAAXH_Result_UnknownExceptionLoadingTIPlugIn
-14008	kAAXH_Result_EffectComponentsMissing
-14009	kAAXH_Result_BadLegacyPlugInIDIndex
-14010	kAAXH_Result_EffectFactoryInitiatedTooManyTimes
-14011	kAAXH_Result_InstanceNotFoundWhenDeinstantiating
-14012	kAAXH_Result_FailedToRegisterEffectPackage
-14013	kAAXH_Result_PlugInSignatureNotValid
-14014	kAAXH_Result_ExceptionDuringInstantiation
-14015	kAAXH_Result_ShuffleCancelled
-14016	kAAXH_Result_NoPacketTargetRegistered
-14017	kAAXH_Result_ExceptionReconnectingAfterShuffle
-14018	kAAXH_Result_EffectModuleCreationFailed
-14019	kAAXH_Result_AccessingUninitializedComponent
-14020	kAAXH_Result_TIComponentInstantiationPostponed
-14021	kAAXH_Result_FailedToRegisterEffectPackageNotAuthorized
-14022	kAAXH_Result_FailedToRegisterEffectPackageWrongArchitecture
-14023	kAAXH_Result_PluginBuiltAgainstIncompatibleSDKVersion
-14023	kAAXH_Result_PluginBuiltAgainstIncompatibleSDKVersion
-14100*	kAAXH_Result_InvalidArgumentValue
-14101*	kAAXH_Result_NameNotFoundInPageTable

*Overlaps with -141xx: [TI System errors](#) definitions

12.45.9.3 -141xx: TI System errors

These errors relate to logic failures in the TI management software and generally indicate a failure in the HDX system services such as buffered message queues, context management, and callback timing.

Table 3: TI system error codes	
Value	Definition
-14101	eTISysErrorNotImpl
-14102	eTISysErrorMemory
-14103	eTISysErrorParam
-14104	eTISysErrorNull
-14105	eTISysErrorCommunication
-14106	eTISysErrorIllegalAccess
-14107	eTISysErrorDirectAccessOfFifoBlocksUnsupported
-14108	eTISysErrorPortIdOutOfBounds
-14109	eTISysErrorPortTypeDoesNotSupportDirectAccess
-14110	eTISysErrorFIFOFull
-14111	eTISysErrorRPCTimeOutOnDSP
-14112	eTISysErrorShellMgrChip_SegsDontMatchAddrs
-14113	eTISysErrorOnChipRPCNotRegistered
-14114	eTISysErrorUnexpectedBufferLength
-14115	eTISysErrorUnexpectedEntryPointName
-14116	eTISysErrorPortIDTooLargeForContextBlock
-14117	eTISysErrorMixerDelayNotSupportedForPlugIns
-14118	eTISysErrorShellFailedToStartUp
-14119	eTISysErrorUnexpectedCondition
-14120	eTISysErrorShellNotRunningWhenExpected
-14121	eTISysErrorFailedToCreateNewPIInstance
-14122	eTISysErrorUnknownPIInstance
-14123	eTISysErrorTooManyInstancesForSingleBufferProcessing
-14124	eTISysErrorNoDSPs
-14125	eTISysBadDSPID
-14126	eTISysBadPIContextWriteBlockSize
-14128	eTISysInstanceInitFailed
-14129	eTISysSameModuleLoadedTwiceOnSameChip
-14130	eTISysCouldNotOpenPlugInModule
-14130	eTISysCouldNotOpenPlugInModule
-14131	eTISysPlugInModuleMissingDependencies
-14132	eTISysPlugInModuleLoadableSegmentCountMismatch
-14133	eTISysPlugInModuleLoadFailure
-14134	eTISysOutOfOnChipDebuggingSpace
-14135	eTISysMissingAlgEntryPoint
-14136	eTISysInvalidRunningStatus
-14137	eTISysExceptionRunningInstantiation
-14138	eTISysTIShellBinaryNotFound
-14139	eTISysTimeoutWaitingForTIShell
-14140	eTISysSwapScriptTimeout
-14141	eTISysTIDSPModuleNotFound
-14142	eTISysTIDSPReadError

12.45.9.4 -142xx: DIDL errors

These errors all relate to the dynamic library loading system that manages ELF DLL binaries on Pro Tools HDX hardware. For example, a `eDIDL_FileNotFound` error will be raised if the ELF DLL name specified by an Effect's Describe code does not match any DLL that is present in the plug-in's bundle.

Table 4: DIDL error codes

Value	Definition
-14201	eDIDL_FileNotFound
-14202	eDIDL_FileNotOpen
-14203	eDIDL_FileAlreadyOpen
-14204	eDIDL_InvalidElfFile
-14205	eDIDL_ImageNotFound
-14206	eDIDL_SymbolNotFound
-14207	eDIDL_DependencyNotLoaded
-14208	eDIDL_BadAlignment
-14209	eDIDL_NotImplemented

12.45.9.5 -144xx: HDX hardware errors

These errors relate to failures on the HDX hardware itself. Plug-ins should never be able to trigger these error codes, which indicate low-level problems in the system.

Table 5: HDX hardware error codes	
Value	Definition
-14401	eBerlinImageError
-14402	eBerlinImageWriteError
-14403	eBerlinInvalidArgs
-14404	eBerlinCantGetTMSChannel
-14405	eBerlinChunkWriteError
-14406	eBerlinChunkReadError
-14407	eBerlinInvalidReqID
-14408	eBerlinDSPInResetError
-14409	eBerlinDSPTimeOut
-14410	eBerlinIncorrectTdmCableWiring
-14411	eBerlinInvalidClock

12.45.9.6 -145xx: DHM isochronous audio engine errors

These errors relate to failures within the HDX audio engine software. Plug-ins should never be able to trigger these error codes, which indicate low-level problems in the system.

Table 6: DHM isochronous audio engine error codes	
Value	Definition
-14500	eDsiIsochEngineGenericError
-14501	eDsiIsochEngineWrongChannelNumber
-14502	eDsiIsochEngineTxRingFull
-14503	eDsiIsochEngineRxRingNotReady
-14504	eDsiIsochEngineWrongNumberOfSamplesRequest
-14505	eDsiIsochEngineUnrecognizedSampleRate
-14506	eDsiIsochEngineUnsupportedSampleSizeBytes
-14507	eDsiIsochEngineUnsupportedNumberOfChannels
-14508	eDsiIsochEngineUnsupportedSampleRate
-14509	eDsiIsochEngineDMAAlreadyEnabled
-14510	eDsiIsochEngineDMAAlreadyDisabled
-14511	eDsiIsochEngineInterruptHandlerAlreadyInstalled
-14512	eDsiIsochEngineBadCardRecord

-14513	eDsiIsochEngineCantSetValueDuringStreaming
-14514	eDsiIsochEngineStreamingAlreadyStarted
-14515	eDsiIsochEngineStreamingAlreadyStopped
-14516	eDsiIsochEngineStreamingCantBeStarted
-14517	eDsiIsochEngineUnsupportedSamplesPerInterrupt
-14518	eDsiIsochEngineCantSetSamplesPerInterrupt
-14519	eDsiIsochEngineInterruptLoopAlreadyExists
-14520	eDsiIsochEngineGlobalDMADisabled
-14521	eDsiIsochEngineActiveInterruptMaskAlreadyEnabled
-14522	eDsiIsochEngineSDIOErrors

12.45.9.7 -30xxx: Dynamically-generated error codes

Errors in the -30xxx range are dynamically generated codes, and thus the same failure point could generate a different error code depending on the order in which errors occurred. These kinds of error codes are used heavily by the TI Shell Manager, the host component that interacts with the on-DSP shell environment.

If one of these error codes is being generated by the TI Shell Manager (the most common case) then you should be able to get more information about the failure by enabling the following [DigiTrace](#) logging facility:

```
DTF_TISHELLMGR=file@DTP_NORMAL
```

or, within the DSH tool:

```
enable_trace_facility [DTF_TISHELLMGR, DTP_NORMAL]
```

This should result in a log with more information such as the name of the failing plug-in, the dynamically generated error code, and a string description of its meaning. Depending on the failure case, the DAE dish command `getlastdsploaderror` can also sometimes be used to retrieve the description string for a dynamically-generated error if it was the last error generated during the DSP loading operation.

Collaboration diagram for HDX DSP Guide:

12.46 Page Table Guide

How to map a plug-in's parameters to control surfaces.

12.46.1 Contents

- [Introduction](#)
- [Avid Control Surfaces](#)
- [Plug-In Page Table Guidelines](#)
- [Avid Center Section Page Tables](#)
- [EUCON Page Tables](#)
- [Implementing Page Tables](#)
- [Appendix A. Get Parameter Value Info](#)

12.46.2 Introduction

12.46.2.1 Control Surfaces Overview

A tactile, external hardware control surface can be used to control different aspects of an application such as Pro Tools or Media Composer. Users prefer purpose-built control surfaces for DAW manipulation due to the control surface's superior accessibility, tactile feel, ergonomics, and user feedback.

Avid provides several different control surface products designed to accommodate a wide variety of user needs and workflows. Avid control surfaces implement EUCON (Extended User Control), a high-speed open control protocol featuring high-resolution, responsive control over almost all software functions. Pro Tools includes support for Mackie's HUI protocol and can interface with third-party control surfaces that implement this protocol.

12.46.2.2 Page Tables Overview

When a control surface is attached to a DAW system running AAX plug-ins the surface can be used to manipulate plug-ins' parameters. Plug-ins define the mapping between their parameters and control surface encoders using *page tables*.

Abstractly, a page table is a static mapping of a plug-in's controls to the interface of the control surface. Since a plug-in may have many more controls than the control surface can accommodate at a given time, the controls may be split across several "pages" that the user can freely switch between.

More concretely, a set of page tables is simply a set of single dimensional arrays. Each slot of the array corresponds to a particular rotary encoder or push-button of the control surface. By inserting control indices into the elements of the array, a plug-in's controls are mapped to particular tangible controls of the CS. Page tables are stored as XML data created by the [Page Table Editor](#) application available as part of the [AAX SDK Toolkit](#) on the [My Toolkits and Downloads](#) page at [avid.com](#). The XML is referenced by the plug-in as a resource using a call to `AAX_IEffectDescriptor::AddResourceInfo()`.

The following sections describe the various interfaces that the supported control surfaces provide for modifying plug-in parameters. Later, the specifics of implementation of page tables is described.

12.46.3 Avid Control Surfaces

12.46.3.1 EUCON

12.46.3.1.1 Avid Dock, Avid S1 and Avid Control app The free Avid Control app provides a EUCON-enabled control surface for iPad or Android tablet. The app offers a host of touch controls and visual feedback. Combining Avid Control with Avid Dock, or Artist S1 hardware adds additional touch workflows and custom control. When laying out plug-in parameters on its display, Avid Control uses the same page table layouts as Avid S1.

Avid Control app display controlling an EQ plug-in instance

The Avid S1 can run as either a standalone device or connected to additional units to form a larger system for audio mixing applications. The plug-in editing section for the S1 consists of 8 touch and velocity sensitive rotary encoders, and 8 switches. The rotary knobs are physically laid out horizontally along the top of the control surface as a group of 8, with the switches located directly below the encoders (the "ON" buttons). The alpha-numeric scribble strip and plug-in editing display are 8 characters wide. The S1 is mapped using the Av18 page table, with the recommendation that the most important parameter is listed first. Both rotary encoders and switches are numbered right to left, as 0 through 7, and 8 through 15 respectively.

Avid Dock provides dedicated transport, focus fader and automation and navigation controls. It is designed with a dock to mount an iPad or Android Tablet running the free Avid Control app, providing control of Pro Tools and plugin parameters. The Dock provides eight push-top, touch-sensitive knobs that interact with whatever knobset has been chosen in the Avid Control app. Select an EQ, plug-in, send, pan, or other item, and all parameters instantly map to the knobs for tweaking. The knob section is mapped using the Av81 page table, divided into 2 series of 4 knobs.

Avid Dock with iPad and Avid Control app

12.46.3.1.2 Avid S6 Avid S6 is a modular control surface solution for the most demanding audio mixing and production environments. Built on the same proven technology that is core to the industry-leading ICON and System 5 product families, S6 enables mixers to quickly turn around complex projects while swiftly handling last-minute changes. With its unparalleled ability to simultaneously control multiple Pro Tools and other EUCON-enabled DAWs over simple Ethernet, S6 also speeds workflows and enables network collaboration on a single integrated platform. Avid S6

The main touch screen display on S6 can be configured to show graphs representing a plug-in's frequency or dynamics response curves. To support this feature, a plug-in must implement [AAX_IEffectParameters::GetCurveData\(\)](#) for the applicable [AAX_ECurveType](#) selector.

12.46.3.2 VENUE

VENUE is a revolutionary line of digital live sound systems that deliver amazing sound quality, reliability, flexibility, and ease-of-use. These live sound systems come equipped with plug-in editing sections, and work together to deliver studio-grade sound and powerful performance. For more information about using AAX plug-ins with VENUE systems see the [VENUE Guide](#).

12.46.3.2.1 VENUE | S6L VENUE | S6L is a modular system designed to take on the world's most demanding tours and events with ease. Offering unprecedented processing capabilities - with over 300 processing channels - S6L delivers unrelenting performance and reliability through its advanced engine design and backs it up with modern touchscreen workflows and scalability to meet any challenge. VENUE | S6L console

S6L uses the same modular components as [Avid S6](#), but uses a different set of encoder layouts on these components in order to best support mixing workflows in a live sound setting. When displaying plug-in parameters, S6L maps the parameters onto its CKM. The leftmost two columns on the CKM are reserved (one column for constant operations, one as a "spacer" row with not assignments), leaving six columns of knob cells available for plug-in parameters. S6L uses the 'Av46' 4x6 page table type to map plug-in parameters to the CKM knob cells in this mode.

If a 'Av46' page table is not available from the plug-in, S6L uses the plug-in's C|24 'FrTL' layout in order to map one plug-in parameter to each of the 24 available knob cells. This generally leads to a very sub-optimal layout of parameters on the surface, both because the C|24 page table is designed for a linear layout and because it results in only one parameter assigned per knob cell, leaving two of the available encoders unassigned. Therefore, Avid strongly recommends that all AAX DSP plug-ins which are compatible with S6L are updated to include the new 4x6 page table layout.

S6L also supports dedicated EQ and dynamics plug-ins: a user can select a particular plug-in as his EQ or dynamics processor and use the surface's EQ, Compressor/Limiter, and Expander/Gate selectors to display the plug-in's parameters using a fixed layout for the given plug-in type. This mode uses the plug-in's D-Control Center Section EQ or Dynamics page tables when mapping plug-in controls. In order to be selectable as the system's EQ or dynamics processor a plug-in must support the [Avid center section page tables](#).

12.46.4 Plug-In Page Table Guidelines

This section is intended as a guide in setting up defined 'pages' using some general rules. However, due to the sheer number of variables, it is simply not possible to account for all scenarios. But by following these suggestions, an AAX plug-in developer should find these guidelines useful in setting up their own plug-in pages. Moreover, it is hoped that a consistent and somewhat standard mapping topology will be realized across the broad range of plug-ins and control surfaces.

Here, we are primarily concerned with the number of controls on a control surface (CS) available for plug-in editing; and secondarily with the layout of controls provided for plug-ins. Accordingly, we need a method of mapping a plug-in's control parameters to a CS, and we need to take into account the varying numbers of controls available on

a CS. Using 'page tables', from a software point of view, a plugin's control parameters can be mapped to a CS. Each page of the page table describes which of the plugin's parameters will be accessible from the CS's controls that are used for plug-in editing. Multiple pages are needed in the case where a CS has fewer controls available than the actual number of controls on a plug-in. We begin by stating guidelines that should be followed when mapping a plug-in's control parameters to a CS. At the end of this chapter, you will find the technical details of creating page tables for your plug-in.

The following guidelines are for simple, generic control surfaces. More advanced CSs, such as the MackieHUI and ProControl, have some guidelines of their own which are listed after these.

12.46.4.1 General Guidelines

Map a plug-in's controls from left-top to right-bottom sequentially onto each page. Follow the layout of the plug-in GUI as closely as possible, allowing the controls to sequentially map to the Control Surface in the order specified above. In so doing, the CS controls will match the plug-in GUI; in the sense that by counting the location of a given control on the PI GUI, one should be able to grasp the corresponding slider or pot on the CS.

Note that Master Bypass, located in the plug-in's floating inserts window, should nearly always be placed as the first control on the first page. The only time this guideline might not be followed is if the plug-in has a particularly favorable layout for the control surface, and where this placement of Master Bypass would disrupt it. Also, on some control surfaces a dedicated bypass is already provided, in which case the Master Bypass should not be mapped into the page table.

Related control parameters should be grouped together on the same page. Controls that are often 'adjusted' with other similar or related controls should be mapped to the same page. This enhances the users ability to tweak related parameters and alleviates unnecessary paging.

Related control parameters should not be split across pages. This follows directly from the bullet above. If some closely related controls cannot all fit on the same page, it is better to leave some blanks (i.e., unused pots, sliders, or switches) and move onto the next page where they can be adjusted together.

As a hypothetical example, let's say a control surface has 5 sliders, and we are mapping an EQ PI with 6 parameters - a low, mid, and high frequency band which has gain for each band. It would be best to map them to the CS as follows on page 1, from left to right on the CS: low freq, low gain, mid freq, mid gain, blank. Then map the remaining two parameters onto page 2: high freq, high gain, blank, blank, blank.

Equivalent left and right stereo parameters should remain on the same page. Since adjusting the left or right parameter of a stereo PI has considerable impact on the sound field, it is important that equivalent left and right stereo controls remain on the same page. Contrast this to placing the left parameters on one page, and the right parameters on another which is not desirable. This rule also changes according to the controller's layout. As an example, the CS-10 has 6 pots for PI editing arranged in a matrix of 3 rows x 2 columns. From left to right, the pots in row 1 are numbered 1 and 4. In row 2 the pots are numbered 2 and 5. Finally, the pots in row 3 are numbered 3 and 6. A layout for L/R controls should be mapped param1L = control 1, param1R = control 4, and so on.

Repeat control parameters on pages where it makes sense to do so. In some situations, it is desirable to have access to the same control on many pages. For example, this might mean having an output and/or input gain control available on each page of an EQ PI - since EQs change the overall gain. This is especially desirable if there would otherwise be blanks (i.e., unused pots, sliders, or switches).

12.46.5 Avid Center Section Page Tables

"Center Section" page tables provide a mapping of plug-in parameters to dedicated functions. These page tables are used by D-Control/D-Command (ICON), D-Show (VENUE), and EUCON-enabled consoles to provide a consistent user experience when interacting with EQ and Dynamics plug-ins.

There are three Center Section page table types:

Table type	Plug-in category
'DgEQ '	EQ
'DgCP '	Compressor/Limiter (Dynamics)
'DgGT '	Expander/Gate (Dynamics)

Dynamics plug-ins that include both Compressor/Limiter and Expander/Gate processing should support both DgCP and DgGT page tables.

The control surfaces which use these page tables each use a different physical layout of parameter functions onto the surface. These layouts have been designed to provide an intuitive and consistent way to control EQ and Dynamics plug-ins in a way that is appropriate for the encoder layout on each surface. By adding these page tables to your EQ and Dynamics plug-ins, your plug-ins will map correctly to all products which use these tables.

12.46.5.1 Center Section Page Table Guidelines

It is important to note that Center Section page table types are different from all other page table types in a fundamental way:

Each slot in the page table is *pre-defined* for a specific type of control. Therefore, your plug-in must conform to this pre-defined layout.

The purpose of these tables is to give the user a standard interface for EQ and Dynamics plug-ins - no matter what particular plug-in they are using. It allows the user to quickly access the most common controls in their favorite EQ or Dynamics plug-ins. This is different from all other page table types because the only restriction on other page table types is that - for types that have dedicated discrete controls - you cannot place continuous controls in a dedicated switch. However, the user will also know that if there are controls they would like to access that are missing from the Center Section, they can access them through one of the other layouts available on the control surface.

You'll notice looking at the pictures of these sections in D-Control (below) that the only scribble strips are in the center of the section. Unlike the Channel Strip section, there is not a scribble strip to label each control. The control's purpose is physically printed on the control surface. This model of hard-coding "center section" plug-in functions to specific encoders is followed on VENUE systems and on EUCON-enabled control surfaces which use these table types as well. That is why it is imperative that your plug-in conform to the pre-defined layout.

Because of the strict definitions of the layouts, it may mean that 1) not all of the controls for your plug-in can fit in these sections, and that 2) there may be controls your plug-in does not have and therefore are blank in this view. For example, let's say your plug-in is a 10-band EQ that does not have individual Q controls on any of the bands. Such a plug-in will be forced to leave off some of its bands, even though all Q controls specific to the bands on the page table are empty. That is fine as there will be another way for the user to display the plug-in on the control surface that will include all of the controls. For example, on D-Control, a user can also view an EQ or Dynamics plug-in in both the Channel Strip and Custom Fader modes which will display all controls. The important point is this:

Do not assign a parameter to a Center Section page table slot that does not match the parameter's function.

If you do, the parameter's function will be mislabeled and will cause confusion for the user.

You should only implement one page for these Center Section layouts. This is different from the other page table types, where it is expected to implement as many pages as necessary to give access to all controls in the plug-in. In the case of the Center Section layouts, you should only define one page, except in the case when the plug-in has separate controls for each channel (Left, Right, Center, etc.).

More than one page in Center Section layouts is allowed *only* if the plug-in has separate controls for each channel.

For example, if your EQ plug-in allows the user to change the EQ differently for the left and right channels, then you would implement two pages for the DgEQ page table. You'll notice in the pictures below for the EQ and Dynamics sections of D-Control, there are buttons labeled for channel selections (L, LC, C, RC, R, etc.). The control surface will automatically map the pages to the buttons, according to the standard order for surround channels. For example, in a stereo EQ, Left controls should be in the first page, and Right controls in the second. D-Control will automatically map page 1 to the L button and page 2 to the R button.

Note

We cannot emphasize strongly enough that trying to fill in all of your controls into these layouts, whether it is by creating extra pages, or by filling in empty slots, will only serve to confuse the user. You must adhere strictly to the guidelines given for these sections.

12.46.5.1.1 EQ Center Section Page Table Guidelines To demonstrate the guidelines for EQ Center Section tables, we will use the D-Control Center Section encoders. Since all control surfaces which use Center Section page tables use a similar approach with hard-coded layouts for these types, the guidelines in this section are equally applicable to all control surfaces.

Take a look at D-Control's EQ section more closely in the image below.

D-Control EQ Center Section.

It supports a maximum of seven bands of EQ, each a vertical column of controls and labeled from left to right as follows:

- HPF (High-Pass Filter, nominally a high-pass filter or low frequency notch filter)
- LF (Low Frequency, nominally a parametric EQ or low frequency shelf filter)
- LMF (Low-Mid Frequency, nominally a parametric EQ)
- MF (Mid Frequency, nominally a parametric EQ)
- HMF (High-Mid Frequency, nominally a parametric EQ)
- HF (High Frequency, nominally a parametric EQ or high frequency shelf filter)
- LPF (Low-Pass Filter, nominally a low-pass filter or high frequency notch filter)

Each of these bands has a Q/Slope control, Frequency control, and an In Circuit/ Out of Circuit button. Five of the bands have an additional Gain control. Four of the bands have an additional EQ type selector switch, each surrounded by a pair of EQ type LED's. Input and Output Level controls are also available, as is a multi-channel Link button in the middle section.

Note

If an EQ plug-in implements a band that does not have an In/Out Circuit control or a Type control, but wants the related LED's to light properly, please see the discussion of the `GetParameterValueInfo()` API below.

The topmost rotary encoders in the HPF, LF, HF, and LPF bands are not labeled but they are indeed Q / Slope controls. The EQ type selector switches located between the Q/Slope and Frequency knobs control the type of filter on that band. Thus they define the behavior of all controls in that band (and not just the unlabeled Q/Slope control).

The EQ page table indices map to the dedicated EQ Center Section hardware encoders as follows:

- Equalization 'DgEQ'
 1. High Pass - In Circuit switch
 2. High Pass - Type switch
 3. High Pass - Frequency
 4. High Pass - Q/Slope
 5. Low Filter - In Circuit switch
 6. Low Filter - Type switch

7. Low Filter - Gain
8. Low Filter - Frequency
9. Low Filter - Q/Slope
10. Low-Mid Filter - In Circuit switch
11. Low-Mid Filter - Type switch
12. Low-Mid Filter - Gain
13. Low-Mid Filter - Frequency
14. Low-Mid Filter - Q/Slope
15. Mid Filter - In Circuit switch
16. Mid Filter - Type switch
17. Mid Filter - Gain
18. Mid Filter - Frequency
19. Mid Filter - Q/Slope
20. High-Mid Filter - In Circuit switch
21. High-Mid Filter - Type switch
22. High-Mid Filter - Gain
23. High-Mid Filter - Frequency
24. High-Mid Filter - Q/Slope
25. High Filter - In Circuit switch
26. High Filter - Type switch
27. High Filter - Gain
28. High Filter - Frequency
29. High Filter - Q/Slope
30. Low Pass - In Circuit switch
31. Low Pass - Type switch
32. Low Pass - Frequency
33. Low Pass - Q/Slope
34. Input Gain
35. Output Gain
36. Multi-channel Link switch
37. High Pass - Q/Slope alternate parameter
38. Low Filter - Q/Slope alternate parameter
39. Low-Mid Filter - Q/Slope alternate parameter
40. Mid Filter - Q/Slope alternate parameter
41. High-Mid Filter - Q/Slope alternate parameter
42. High Filter - Q/Slope alternate parameter
43. Low Pass - Q/Slope alternate parameter

Note

In order to use the "Q/Slope alternate parameter" functions in the EQ page table, a plug-in must respond to the [AAX_ePageTable_UseAlternateControl](#) selector in the [GetParameterValueInfo\(\)](#) method. When the band type is changed, the control surface will call into the plug-in with this selector to determine if the control in the "Alt" position should be used. Please see the discussion of the [GetParameterValueInfo\(\)](#) below.

If your plug-in supports fewer than seven simultaneous bands of EQ, you have some options for where to place them. We recommend the following placement guidelines so users have consistency with various EQ plug-ins.

- If you only have one band of EQ, use the LF band.
- If you have one to four fully parametric bands, use LF, LMF, HMF, and HF (starting from left to right). (Skip the MF band.)
- If you have up to two shelving filters and two parametric bands, use LF (LF shelf), LMF (para), HMF (para), and HF (HF shelf). (Skip the MF band.)

Note that this layout includes a few additional functions not in D-Control's EQ section. These extra functions are the "Low-Mid Filter - Type switch", "Mid Filter - Type switch", and "High-Mid Filter - Type switch" slots.

12.46.5.1.2 Dynamics Center Section Page Table Guidelines To demonstrate the guidelines for Dynamics Center Section tables, we will use the D-Control Center Section encoders. As with the EQ Center Section tables above, all control surfaces which use Center Section page tables use a similar approach with hard-coded layouts for these types, the guidelines in this section are equally applicable to all control surfaces.

The D-Control Dynamics section is shown below. Keep in mind that the D-Control's Dynamics section displays page tables for both the Compressor/Limiter page table type and the Expander/Gate type. Therefore, the function of certain rotaries and switches differs depending on which page table type has been loaded. In these cases, there is a LED to indicate the current function of a rotary or switch.

D-Control Dynamics Center Section.

Several rotary encoders have alternate uses while others are always dedicated to one function. The two page tables' indices map to the dedicated Dynamics Center Section hardware encoders as follows:

- Compressor/Limiter 'DgCP'
 1. Threshold
 2. Ratio
 3. Attack Time
 4. Release Time
 5. Knee
 6. Make-up Gain
 7. High Pass - In Circuit / Out of Circuit switch
 8. High Pass - EQ Type switch (with notch and high-pass filter LEDs)
 9. High Pass - Frequency
 10. High Pass - Q/Slope
 11. Low Pass - In Circuit / Out of Circuit switch
 12. Low Pass - EQ Type switch (with notch and low-pass filter LEDs)
 13. Low Pass - Frequency
 14. Low Pass - Q/Slope
 15. External Key switch (middle section)
 16. Key Listen switch (middle section)
 17. Input Gain
 18. Output Gain
 19. Multi-channel Link switch (middle section)
 20. Depth (not included on ICON)

- Expander/Gate 'DgGT'
 1. Threshold
 2. Ratio
 3. Range
 4. Attack Time
 5. Release Time
 6. Hysteresis
 7. Hold
 8. High Pass - In Circuit / Out of Circuit switch
 9. High Pass - EQ Type switch (with notch and high-pass filter LEDs)
 10. High Pass - Frequency
 11. High Pass - Q/Slope
 12. Low Pass - In Circuit / Out of Circuit switch
 13. Low Pass - EQ Type switch (with notch and low-pass filter LEDs)
 14. Low Pass - Frequency
 15. Low Pass - Q/Slope
 16. External Key switch (middle section)
 17. Key Listen switch (middle section)
 18. Input Gain
 19. Output Gain
 20. Multi-channel Link switch (middle section)
 21. Knee (not included on ICON)

The compressor/limiter page table, DgCP, supports all the controls above except Range, Hysteresis, and Hold. (As you create the page table in the [Page Table Editor](#), this is clear.)

The expander/gate page table, DgGT, supports all the controls above except Knee and Makeup Gain. It does support both Ratio and Range. The user presses the Page button to select between them.

A plug-in can support both DgCP and DgGT page tables. Again, the user presses the Page button to select between them. If a plug-in supports both of these page tables and the DgGT page table includes support for both Ratio and Range controls, pressing the Page button will switch between all available controls as follows:

DgCP -> DgGT with Ratio -> DgGT with Range (and all other controls unchanged)-> DgCP -> ...

12.46.5.2 Center Section Parameter Mapping to Single-Column/Row Layouts

The following tables show the layout mapping and hard-coded names for center section page tables on EUCON surfaces which use single-column or single-row layouts for Eq and Dyn plug-in modes. The tables show the assignment of specific center section table indices to cells on the EUCON control surface. Each EUCON control surface cell includes three encoders: a rotary knob encoder and two push-button encoders. These tables use the following codes to indicate encoders in each control surface cell:

- Knob = Knob encoder
- Knob(Sel I) = Knob encoder with Sel active
- Knob(Sel O) = Knob encoder with Sel inactive
- In = In switch

Note

For center section page table layouts the "Sel" push-button encoder is only used to toggle the rotary encoder between two possible parameters. It is never mapped to a single discrete parameter in these layouts.

With some versions of EUCON, the cell mapping on the first page is "reversed" relative to the second page when the surface is laid out horizontally; the first page moves left to right through increasing cell indices, while later pages move right to left.

	Function	Page 1								Page 2							
		Far from user / Left				Close to user / Right				Far from user / Left				Close to user / Right			
		8	7	6	5	4	3	2	1	16	15	14	13	12	11	10	9
1	HPF In/↔ Out														Knob/In		
2	HPF Type													In			
3	HPF Freq													Knob (Sel O)			
4	HPF Q													Knob (Sel I)			
5	Lo In/↔ Out							In									
6	Lo Type								In								
7	Lo Gain							Knob									
8	Lo Freq								Knob (Sel O)								
9	Lo Q								Knob (Sel I)								
10	Lo Mid In/↔ Out					In											
11	Lo Mid Type						In										
12	Lo Mid Gain					Knob											
13	Lo Mid Freq						Knob (Sel O)										
14	Lo Mid Q						Knob (Sel I)										

		Page 1								Page 2							
15	Mid In/↔ Out											In					
16	Mid Type										In						
17	Mid Gain											Knob					
18	Mid Freq											Knob (Sel O)					
19	Mid Q											Knob (Sel I)					
20	Hi Mid In/↔ Out			In													
21	Hi Mid Type				In												
22	Hi Mid Gain			Knob													
23	Hi Mid Freq				Knob (Sel O)												
24	Hi Mid Q				Knob (Sel I)												
25	Hi In/↔ Out	In															
26	Hi Type		In														
27	Hi Gain	Knob															
28	Hi Freq		Knob (Sel O)														
29	Hi Q		Knob (Sel I)														
30	LPF In/↔ Out										Knob/In						
31	LPF Type									In							
32	LPF Freq										Knob (Sel O)						

		Page 1								Page 2							
33	LPF Q									Knob (Sel I)							
34	In-put Level															Knob	
35	Out-put Level																Knob
36	Link																
37	HPF Q Alt												Knob (Sel I)*				
38	Lo Q Alt							Knob (Sel I)*									
39	Lo Mid Q Alt						Knob (Sel I)*										
40	Mid Q Alt									Knob (Sel I)*							
41	Hi Mid Q Alt			Knob (Sel I)*													
42	Hi Q Alt	Knob (Sel I)*															
43	LPF Q Alt									Knob (Sel I)*							

12.46.5.2.1 'DgEQ' PageTable - Equalizer *Notes*

- The multi-channel link switch (index 36) is not mapped to any encoder in this layout
- The knob assignments marked with an asterisk will be assigned depending on the plug-in's response to [AAX_IEffectParameters::GetParameterValueInfo\(\)](#) with the [AAX_ePageTable_UseAlternateControl](#) selector. If the plug-in provides [AAX_eUseAlternateControl_Yes](#) then the assignment marked with an asterisk will be used.

12.46.5.2.2 Both 'DgCP' and 'DgGT' PageTables - Multi-dynamics If both Dynamics center section page table types are defined for the plug-in then EUCON control surfaces will use the following mapping.

Note

Some control surfaces may only partially follow this mapping. For example, the mapping of the Artist Mix control surface in Dyn mode uses two pages: it follows this mapping for encoder cells 1-8 on the first page and follows the ['DgCP'-only mapping](#) for encoder cells 9-16 on the second page.

		Page 1								Page 2							
		Far from user / Left				Close to user / Right				Close to user / Left				Far from user / Right			
		8	7	6	5	4	3	2	1	16	15	14	13	12	11	10	9
Dg← CP 1	Function C Threshold						Knob				Knob						
Dg← CP 2	Function C Ratio					Knob				Knob							
Dg← CP 3	Function C Attack Time							Knob					Knob				
Dg← CP 4	Function C Release Time								Knob					Knob			
Dg← CP 5	Function C Knee											Knob					
Dg← CP 6	Function C Gain Makeup															Knob	
Dg← CP 7	HPF Enabled																
Dg← CP 8	HPF Type																
Dg← CP 9	HPF Freq																
Dg← CP 10	HPF Q																
Dg← CP 11	LPF Enabled																
Dg← CP 12	LPF Type																

		Page 1								Page 2							
Dg← CP 13	LPF Freq																
Dg← CP 14	LPF Q																
Dg← CP 15	Ext Key																
Dg← CP 16	Key Lis- ten																
Dg← CP 17	In- put Gain																
Dg← CP 18	Out- put Gain																
Dg← CP 19	Link													Knob			
Dg← CP 20	Depth																
Dg← GT 1	X Thresh- old		Knob														
Dg← GT 2	X Ra- tio		Knob														
Dg← GT 3	X Range																
Dg← GT 4	X At- tack Time			Knob													
Dg← GT 5	X Re- lease Time				Knob												
Dg← GT 6	X Hys- tere- sis																

Dg GT 7	X Hold	Page 1								Page 2							

	Function	Page 3								Page 4							
		Close to user / Left				Far from user / Right				Close to user / Left				Far from user / Right			
		24	23	22	21	20	19	18	17	32	31	30	29	28	27	26	25
Dg CP 1	Threshold																
Dg CP 2	Ratio																
Dg CP 3	Attack Time																
Dg CP 4	Release Time																
Dg CP 5	Knee																
Dg CP 6	Gain Makeup																
Dg CP 7	HPF Enabled														Knob/In		
Dg CP 8	HPF Type													In			
Dg CP 9	HPF Freq													Knob (Sel O)			
Dg CP 10	HPF Q													Knob (Sel I)			
Dg CP 11	LPF Enabled												Knob/In				

		Page 3								Page 4							
Dg← CP 12	LPF Type										In						
Dg← CP 13	LPF Freq										Knob (Sel O)						
Dg← CP 14	LPF Q										Knob (Sel I)						
Dg← CP 15	Ext Key									Knob/In							
Dg← CP 16	Key Lis- ten									Knob/In							
Dg← CP 17	In- put Gain													Knob			
Dg← CP 18	Out- put Gain														Knob		
Dg← CP 19	Link																
Dg← CP 20	Depth																
Dg← GT 1	X Thresh- old		Knob														
Dg← GT 2	X Ra- tio		Knob														
Dg← GT 3	X Range						Knob										
Dg← GT 4	X At- tack Time				Knob												
Dg← GT 5	X Re- lease Time					Knob											

		Page 3								Page 4							
Dg← GT 6	X Hys- tere- sis								Knob								
Dg← GT 7	X Hold								Knob								

Notes

- Neither table's multi-channel link switch ('DgCP ' index 19 and 'DgGT ' index 20) is mapped to an encoder in this layout
- The 'DgGT ' filter, external key, and gain parameters (indices 8 through 19) are not mapped to any encoders in this layout

12.46.5.2.3 'DgCP' PageTable - Compressor/Limiter If the plug-in defines a 'DgCP ' page table but does not define a 'DgGT ' page table then EUCON control surfaces will use the following mapping.

		Page 1*								Page 2							
		Far from user / Left				Close to user / Right				Far from user / Left				Close to user / Right			
		8	7	6	5	4	3	2	1	16	15	14	13	12	11	10	9
1	C Thresh- old						Knob										
2	C Ra- tio					Knob											
3	C At- tack Time							Knob									
4	C Re- lease Time								Knob								
5	C Knee	Knob															
6	C Gain Makeup		Knob*			Knob*											
7	HPF En- abled											Knob//In					
8	HPF Type												In				
9	HPF Freq												Knob (Sel O)				

		Page 1*								Page 2							
10	HPF Q											Knob (Sel I)					
11	LPF Enabled												Knob/In				
12	LPF Type													In			
13	LPF Freq													Knob (Sel O)			
14	LPF Q													Knob (Sel I)			
15	Ext Key														Knob/In		
16	Key Listen															Knob/In	
17	In-put Gain				Knob												
18	Out-put Gain					Knob											
19	Link																
20	Depth									Knob							

Notes

- If no parameter is defined at the Ratio parameter index then the Makeup Gain parameter will be mapped to the Ratio parameter's normal spot in order to increase usefulness of the first-page mapping.
- Pro Tools versions prior to Pro Tools 11.1 use a different layout for the first page of this table type

12.46.5.2.4 'DgGT' PageTable - Expander/Gate If the plug-in defines a 'DgGT' page table but does not define a 'DgCP' page table then EUCON control surfaces will use the following mapping.

		Page 1								Page 2							
		Far from user / Left				Close to user / Right				Far from user / Left				Close to user / Right			
		8	7	6	5	4	3	2	1	16	15	14	13	12	11	10	9
1	Function X Threshold							Knob									

		Page 1							Page 2						
2	X Ratio							Knob							
3	X Range			Knob											
4	X Attack Time				Knob										
5	X Release Time			Knob											
6	X Hysteresis	Knob													
7	X Hold		Knob												
8	HPF Enabled									Knob/In					
9	HPF Type										In				
10	HPF Freq										Knob (Sel O)				
11	HPF Q										Knob (Sel I)				
12	LPF Enabled											Knob/In			
13	LPF Type												In		
14	LPF Freq												Knob (Sel O)		
15	LPF Q												Knob (Sel I)		
16	Ext Key													Knob/In	
17	Key Listen														Knob/In
18	Input Gain								Knob						

19	Out-put Gain	Page 1								Page 2							
										Knob							
20	Link																

12.46.5.3 Center Section Parameter Mapping in S6 Expand Mode

In addition to supporting single-row and single-column EQ and Dynamics layouts as described above, S6 also includes an Expand Mode for EQ and Dynamics plug-ins which allows the targeted plug-in's EQ or Dynamics center section mapping to be displayed across an entire CKM module.

12.46.5.3.1 'DgEQ' PageTable - Equalizer EQ layout in S6 Expand Mode

12.46.5.3.2 'DgCP' and 'DgGT' PageTables - Compressor/Limiter and Expander/Gate Dynamics layout in S6 Expand Mode

12.46.5.4 Center Section Parameter Mapping on VENUE | S3L-X

Built-in processing parameters are mapped to the Channel Control encoders on VENUE | S3L. For more information about Channel Control mode in S3L-X see [Using Channel Control](#) in the [VENUE Guide](#).

Channel Control Encoder	Knob	Sel Switch	In Switch
1	Low Gain		Low EQ band in/out
2	Low Freq/Q	Toggles Freq/Q	Low EQ band type
3	LoMid Gain		LoMid EQ band in/out
4	LoMid Freq/Q	Toggles Freq/Q	
5	HiMid Gain		HiMid EQ band in/out
6	HiMid Freq/Q	Toggles Freq/Q	
7	High Gain		High EQ band in/out
8	High Freq/Q	Toggles Freq/Q	High EQ band type

12.46.5.4.1 'DgEQ' PageTable - Equalizer

Channel Control Encoder	Knob	Sel Switch	In Switch
1	Threshold level		Comp/Lim or Exp/Gate in/out
2	Ratio		
3	Attack		
4	Knee		
5	Release		
6	Gain level		

Channel Control Encoder	Knob	Sel Switch	In Switch
7	Key HF	Key Listen	Filter in/out
8	Key LF	Key In	Filter in/out

12.46.5.4.2 'DgCP' and 'DgGT' PageTables - Compressor/Limiter and Expander/Gate

12.46.6 EUCON Page Tables

Plug-ins should implement specific EUCON page tables to take advantage of EUCON-specific features and layouts. By writing EUCON-specific page tables, your plug-in is able to re-define both the in/out button as well as the select button per EUCON control cell on compatible surfaces.

Host Compatibility Notes Pro Tools versions prior to Pro Tools 11.1 use plug-ins' ProControl and ICON page tables (Dynamics, EQ, Channel Strip, Custom Fader, etc.) to map plug-in parameters to EUCON-enabled surfaces, so be sure that your plug-ins also implement these page tables correctly so that users with earlier versions of Pro Tools can have the best possible experience when using your plug-ins.

Note

The legacy PeTE editor will remove all EUCON sections from any page table XML file that it saves. Developers should no longer use PeTE for [AAX](#) page table editing. If you do use Pete, it is important to *always back up your page table file* before editing the file in PeTE.

12.46.6.1 Specification

EUCON page tables use modern formatting, making them more intuitive to implement than non-EUCON tables. Here are some example lines from a EUCON page table:

```
<PageTable type='Av18' pgsz='24' >
  <Page num='1'\>
    <Cell row='1' col='1' knobID="Knob1" inOutButtonID="Button1A" selectButtonID="Button1B" />
    <Cell row='1' col='2' knobID="Knob2" inOutButtonID="Button2A" selectButtonID="Button2B" />
    <Cell row='1' col='3' knobID="Knob3" inOutButtonID="Button3A" selectButtonID="Button3B" />
    ...
  </Page\>
  ...
</PageTable >
```

The EUCON PageTable element includes a series of Cell sub-elements. The attributes of each Cell sub-element are as follows:

- `row` - the cell's row position, ordered furthest to nearest
- `col` - the cell's column position, ordered left to right
- `knobID` - the parameter ID associated with the knob in question
- `inOutButtonID` - the parameter ID associated with the "In" button next to the knob in question. This must be a discrete parameter.
- `selectButtonID` - the parameter ID associated with the "Sel" button next to the knob in question. This can be a discrete or continuous parameter. If it is a continuous parameter then pushing "Sel" will toggle the knob associated with the button between the `selectButtonID` and `knobID` parameters.

12.46.6.2 Types

Av81	Av18	Av48	Av46
type='Av81'	type='Av18'	type='Av48'	type='Av46'
8 rows	1 row	4 rows	4 rows
1 column	8 columns	8 columns	6 columns
pgsz='24' (3 elements per cell)	pgsz='24' (3 elements per cell)	pgsz='96' (3 elements per cell)	pgsz='72' (3 elements per cell)
Used for vertical sets of knob cells	Used for horizontal sets of knob cells	Used for 4x8 knob cell arrays	Used for 6x4 knob cell arrays, e.g. plug-in layout
Most important parameters should be placed in this set	Most important parameters should be placed in this set	Most important parameters should be placed in this set	Most important parameters should be placed in this set

12.46.6.3 Conventions

- To map a single discrete parameter to an encoder cell, assign the parameter to both the knob and the In switch. Assigning the parameter to the cell's knob will ensure that the parameter name and value is always displayed on the surface. The user will be able to edit the parameter using either the rotary encoder or the In switch.
- When assigning discrete parameters, always prefer to use the In switch over the Sel switch. For cells with one continuous and one discrete parameter, users will always expect the discrete parameter to be assigned to the In switch rather than the Sel switch. In other EUCON modes (besides plug-in editing) the In switch is always used to enable/disable parameters, while the Sel switch is usually used to toggle between functions for the cell's rotary encoder.

12.46.6.4 Requirements

- All parameter IDs used in the EUCON page tables must be defined with a `Ctrl ID` element within the `ControlNameVariations` element
- Every `Cell` with at least one parameter assignment must include a `knobID` assignment. It is not valid to assign either `inOutButtonID` or `selectButtonID` without also assigning the cell's `knobID`.
- For a given `knobID`, the same parameters must be assigned to the `selectButtonID` and `inOutButtonID` switches across all EUCON page tables
- Every knob cell assignment set (Rotary+Sel+In assignment) used in the 'Av48' table must be exactly replicated somewhere in the 'Av81' table
- 'Av48' tables may contain no more than two pages

12.46.7 Implementing Page Tables

12.46.7.1 Page table XML specification

This section includes a rough specification for plug-in page table XML. Whenever possible, we encourage developers to use the [Page Table Editor](#) tool to generate plug-in page tables rather than writing or editing the page table XML by hand.

The page table XML format contains three main tags:

- [PageTableLayouts](#) tag
- [ControlNameVariations](#) tag
- [Editor](#) tag

12.46.7.1.1 PageTableLayouts tag This section provides a static mapping of control elements to page table layouts. Multiple layouts may be provided in this section, e.g. in cases where different control sets are used by different plug-in Types.

Each layout includes a complete set of page table descriptions. There are multiple kinds of page tables, each of which may have multiple pages. At minimum, each layout must include a `PageTable` with `type='PgTL'` and `pgsz='1'`. This is the default page table, and the order of the control elements that it describes must match the order in which the corresponding parameters are added to the plug-in itself.

Here is an excerpt from the `PageTableLayouts` section in Avid's Eleven plug-in page tables demonstrating non-EUCON `PageTable` elements. For the EUCON `PageTable` element specification, see [Eucon page table specification](#).

```
<PageTableLayouts>
  <Plugin manID='Digi' prodID='ElvF' plugID='ELFr'>
    <Desc>Eleven Free 1 -&gt; 1 Avid Technology, Inc.</Desc>
    <Layout>PageTable Free</Layout>
  </Plugin><!--manID='Digi' prodID='ElvF' plugID='ELFr'-->
  <Plugin manID='Digi' prodID='Elvn' plugID='ELVr'>
    <Desc>Eleven 1 -&gt; 1 Avid Technology, Inc.</Desc>
    <Layout>PageTable 1</Layout>
  </Plugin><!--manID='Digi' prodID='Elvn' plugID='ELVr'-->
  <!-- ... -->
  <PTLayout name='PageTable 1'>
    <PageTable type='BkCS' pgpsz='12'>
      <Page num='1'>
        <ID>Mic Type</ID>
        <ID>Cab Type</ID>
        <ID>Amp Type</ID>
        <ID>Master</ID>
        <ID>Gain 2</ID>
        <ID>Gain 1</ID>
        <ID>Mic Axis</ID>
        <ID>Cab Type</ID>
        <ID>Amp Type</ID>
        <ID> </ID>
        <ID> </ID>
        <ID>Bright Switch</ID>
      </Page><!--num='1'-->
      <Page num='2'>
        <!-- ... -->
      </Page><!--num='2'-->
      <Page num='3'>
        <!-- ... -->
      </Page><!--num='3'-->
    </PageTable><!--type='BkCS' pgpsz='12'-->
    <!-- ... -->
    <PageTable type='PgTL' pgpsz='1'>
      <Page num='1'>
        <ID>Master Bypass</ID>
      </Page><!--num='1'-->
      <Page num='2'>
        <ID>Input Level</ID>
      </Page><!--num='2'-->
      <Page num='3'>
        <ID>Output Level</ID>
      </Page><!--num='3'-->
      <Page num='4'>
        <ID>Gate Threshold</ID>
      </Page><!--num='4'-->
      <!-- ... -->
    </PageTable><!--type='PgTL' pgpsz='1'-->
  </PTLayout><!--name='PageTable 1'-->
  <PTLayout name='PageTable Free'>
    <!-- ... -->
  </PTLayout><!--name='PageTable Free'-->
</PageTableLayouts>
```

The sub-tags for this section are as follows:

- `Plugin` element

A high-level description of a single plug-in Type.

The `manID`, `prodID`, and `plugID` attributes must match the corresponding Manufacturer, Product, and Type IDs for each plug-in Type that is described in the XML file. Multiple `Plugin` elements may be included in a single XML file.

The `Plugin` element has two sub-elements:

1. The `Desc` sub-element provides a brief description of the plug-in Type. This information is used only by the [Page Table Editor](#) application and does not affect operation of the plug-in in Pro Tools.
2. One `Layout` sub-element is used to bind the plug-in Type to a particular `PTLayout` (see below.)

- **PTLayout element**

A complete control mapping, including a full set of `PageTable` descriptions.

- `PageTable` sub-element - A single page table mapping, with controls specified across multiple `Page` elements as in the example above.

`PageTable` elements have the following attributes:

1. `type` defines the particular device that will use the `PageTable`. `type` may be one of:
 - * `PgTL` - Generic page tables (any size)
 - * `PcTL` - ProControl, VENUE, and fall-back EUCON page table (size 16)
 - * `MkTL` - Makie HUI page table (size 8)
 - * `HgTL` - 002/003 and Command|8 page table (size 8)
 - * `FrTL` - Control 24, C|24, and fall-back S6L page table (size 24)
 - * `BkCS` - ICON Channel Strip (size 12)
 - * `BkSF` - ICON Custom Fader (size 16)
 - * `DgGT` - ICON dynamics section (Gate/Expander) (size 20)
 - * `DgCP` - ICON dynamics section (Compressor/Limiter) (size 19)
 - * `DgEQ` - ICON EQ section (size 43)
 - * `Av81` - EUCON 8x1 section (size 24)
 - * `Av18` - EUCON 1x8 section (size 24)
 - * `Av48` - EUCON 4x8 section (size 96)
 - * `Av46` - EUCON 4x6 section (size 72)
2. `pgsz` defines the number of controls per page in the page table. Most page table types require a specific size, as noted above. The generic `PgTL` page tables may be of any size, and multiple `PgTL` page tables may be provided. However, each plug-in must provide a `PgTL` page table of size 1 that includes all of the plug-in's automatable parameters, in the order in which they are added to the plug-in.

12.46.7.1.2 ControlNameVariations tag This section includes information about the names of each control in the plug-in. Here is an excerpt from our Eleven plug-in page tables which demonstrates the basic format of this section:

```
<Ctrl ID='Amp Bypass'>
  <name typ='PgTL' sz='1'>AB</name>
  <name typ='PgTL' sz='4'>AByp</name>
  <name typ='PgTL' sz='5'>A Byp</name>
  <name typ='PgTL' sz='6'>AmpByp</name>
  <name typ='PgTL' sz='7'>Amp Byp</name>
</Ctrl><!--ID='Amp Bypass'-->
<Ctrl ID='Amp Type'>
  <name typ='PgTL' sz='1'>AT</name>
  <name typ='PgTL' sz='3'>Amp </name>
  <name typ='PgTL' sz='5'>AmpTp</name>
  <name typ='PgTL' sz='6'>AmpTyp</name>
  <name typ='PgTL' sz='7'>Amp Typ</name>
</Ctrl><!--ID='Amp Type'-->
<Ctrl ID='Bright Switch'>
  <name typ='PgTL' sz='1'>Brt </name>
  <name typ='PgTL' sz='6'>Bright</name>
</Ctrl><!--ID='Bright Switch'-->
```

The sub-tags used are as follows:

`Ctrl` element with `ID` attribute - The identifier of the control that is being identified.

The identifiers for all parameters in the page table must and should match the IDs used for the control both in the `<PageTableLayouts>` layouts and in the `Editor` tag's `DiscCtrls` sub-tag (see below). See [Parameter identifiers](#) for more information about parameter identifiers.

name element - The desired abbreviated name of the control for display on control surface UIs, which may have limited display space available.

The **typ** parameter allows you to choose which page table type the given abbreviation will be specific to. For example, a name provided with **typ**= 'HgTL' would only appear on Command|8, 002, and 003 hardware. As with all other tags, the name associated with **typ** PgTL (the generic page table identifier) will be used if no other name is given for the specific page table that is being loaded.

The **sz** parameter defines the size of the control surface display for which the given name is appropriate. The control surface will use the name associated with the largest size that will fit on its display.

To reduce the number of names that must be specified, abbreviated names can be given that are longer than their specified size. These names will be truncated when necessary. For example, a control surface that could accommodate two characters on its display would use the size-1 name for the "Bright Switch" control above, since 1 is the largest number less than or equal to 2 from the provided set of name sizes (in this case, 1 and 6.) The size-1 name, 'BrT', has four characters in it. Therefore, it would be truncated down to 'Br' to fit onto the control surface's display.

12.46.7.1.3 Editor tag The last of the three main sections in an XML plug-in page table is the **Editor** section. This section is used by the [Page Table Editor](#) application and you should not need to modify its contents. However, if you are encountering problems modifying your plug-in in the Page Table Editor then you may wish to verify that all plug-ins and controls are properly identified within the **PluginList** and **DiscCtrls** sub-tabs, respectively. Here is the **Editor** section from the Eleven page table XML:

```
<Editor vers='1.3.7.1'>
  <PluginList>
    <TDM>
      <PluginID manID='Digi' prodID='Elvn' plugID='ELVt'>
        <MenuStr>TDM: Eleven, 1 in X 1 out</MenuStr>
      </PluginID><!--manID='Digi' prodID='Elvn' plugID='ELVt'-->
    </TDM>
    <RTAS>
      <PluginID manID='Digi' prodID='Elvn' plugID='ELVr'>
        <MenuStr>RTAS: Eleven, 1 in X 1 out</MenuStr>
      </PluginID><!--manID='Digi' prodID='Elvn' plugID='ELVr'-->
      <PluginID manID='Digi' prodID='ElvF' plugID='ELFr'>
        <MenuStr>RTAS: Eleven Free, 1 in X 1 out</MenuStr>
      </PluginID><!--manID='Digi' prodID='ElvF' plugID='ELFr'-->
    </RTAS>
  </PluginList>
  <DiscCtrls>
    <CtrlID>Amp Bypass</CtrlID>
    <CtrlID>Amp Type</CtrlID>
    <CtrlID>Bright Switch</CtrlID>
    <CtrlID>Cab Bypass</CtrlID>
    <CtrlID>Cab Type</CtrlID>
    <CtrlID>Master Bypass</CtrlID>
    <CtrlID>Mic Axis</CtrlID>
    <CtrlID>Mic Type</CtrlID>
  </DiscCtrls>
</Editor><!--vers='1.3.7.1'-->
```

12.46.7.2 Parameter identifiers

The **ID** tags/arguments in a page table must reference parameters which are exposed by the plug-in's [AAX_IEffectParameters](#) implementation via methods such as [GetParameterIndex\(\)](#).

There are two supported ways to identify parameters in a page table:

- By the parameter's ID (preferred)
- By the parameter's 31-character name (legacy)

A single page table may only reference the plug-in's parameters using one of these two approaches; a page table file may not reference one parameter by its name and another by its ID, and it may not reference one parameter by its name in one location but by its ID in another location.

If a plug-in will change its parameters' names at run time then the parameter identifiers used in the page tables must reference parameters by ID.

12.46.7.3 Creating page tables using the AAX Plug-In Page Table Editor

Page tables can be created and edited using the AAX Plug-In Page Table Editor application available on the Developer website. The Page Table Editor generates an XML file that can be used in both Windows and Macintosh plug-in projects.

The Page Table Editor can also open page table files in existing .aaxplugin bundles on your system. If you're looking for examples of how to lay out the common parameters in your plug-ins try opening up the page tables for one of the standard Avid plug-ins like Avid Channel Strip or D-Verb.

Note

The Page Table Editor only supports opening a single file at a time. It can be useful to open multiple files in order to compare their layouts. To open multiple page tables at the same time you can launch multiple instances of the Page Table Editor application by running the application from a terminal shell.

The Page Table Editor will make use of the Parameter IDs that are defined in a plug-in, and will associate them with the 'Plugin' tags in the XML file. Using the DemoDist sample plug-in included in the AAX SDK, you'll see that there is one Plugin entry for each plug-in type in the binary.

```
// Type, product, and relation IDs
const AAX_CTypeID cDemoDist_ManufactureID = 'AVID ';
const AAX_CTypeID cDemoDist_ProductID = 'DmDE ';
const AAX_CTypeID cDemoDist_TypeID_AS = 'DmAS ';
const AAX_CTypeID cDemoDist_TypeID_MonoNative = 'DmRT ';
const AAX_CTypeID cDemoDist_TypeID_StereoNative = 'DsRT ';
const AAX_CTypeID cDemoDist_TypeID_MonoTI = 'DDT1 ';
const AAX_CTypeID cDemoDist_TypeID_StereoTI = 'DDT2 ';
```

Listing 1: DemoDist Plug-In IDs

```
.
.
.
<Plugin manID = 'AVID ' prodID = 'DmDE ' plugID = 'DmRT '>
<Desc > DemoDist 1 -&gt; 1 by Avid Technology , Inc .</ Desc >
<Layout > PageTable 1</ Layout >
</ Plugin ><!-- manID = 'AVID ' prodID = 'DmDE ' plugID = 'DmRT '-->
<Plugin manID = 'AVID ' prodID = 'DmDE ' plugID = 'DsRT '>
<Desc > DemoDist 2 -&gt; 2 by Avid Technology , Inc .</ Desc >
<Layout > PageTable 1</ Layout >
</ Plugin ><!-- manID = 'AVID ' prodID = 'DmDE ' plugID = 'DsRT '-->
<Plugin manID = 'AVID ' prodID = 'DmDE ' plugID = 'DDT1 '>
<Desc > DemoDist 1 -&gt; 1 by Avid Technology , Inc .</ Desc >
<Layout > PageTable 1</ Layout >
</ Plugin ><!-- manID = 'AVID ' prodID = 'DmDE ' plugID = 'DDT1 '-->
<Plugin manID = 'AVID ' prodID = 'DmDE ' plugID = 'DDT2 '>
<Desc > DemoDist 2 -&gt; 2 by Avid Technology , Inc .</ Desc >
<Layout > PageTable 1</ Layout >
</ Plugin ><!-- manID = 'AVID ' prodID = 'DmDE ' plugID = 'DDT2 '-->
.
.
.
```

Listing 2: DemoDist XML

Note

For compatibility between your AAX and corresponding RTAS or TDM plug-ins, make sure the 4 character IDs for [AAX_eProperty_ManufacturerID](#), [AAX_eProperty_ProductID](#), [AAX_eProperty_PluginID_Native](#), and [AAX_eProperty_PluginID_AudioSuite](#) are identical to the legacy SDK's counterpart.

For more information about the AAX Plug-In Page Table Editor tool, see the ReadMe file which accompanies the application.

12.46.7.4 Verifying Page Table Layouts: The Hidden Pop-Up Menu

You can verify the page tables created in your plug-in in Pro Tools with a "hidden" developer debug Page Tables popup menu. To verify the page tables, first include the `YourPageTables.r` file in your project and compile the plug-in. Then, after launching Pro Tools, instantiate the plug-in, hold down the Commandkey (Ctrl-key in Windows) and mouse-click the Automation button in the plug-in window to display the menu.

- Category

The Category menu item has a submenu listing the names of possible categories. Any category that the plug-in belongs to will have a check mark next to it. In addition, appended to the name of the category is an indication of whether that category can be bypassed, and if so, the control number (#) and control name of the associated bypass control. Also appended is the number of the first page on which a control associated with the category can be found. This page number is based on whatever the current page table type is selected in the "TableType" menu item. For example: Delay (can bypass, control #9, Master Bypass) (first page #1)

- Table Type

Sets the type of control surface page table.

- Page Size

Sets the number of controls per page. The identifier "custom" is shown next to page sizes that have been specifically implemented (which at minimum, should appear next to page sizes of 5, 6, 8, and 16!). The identifier "default" is shown next to page sizes that do not have specific support in your page table file. Note that the Mackie and ProControl table type will automatically set this to 8 and 16, respectively.

- Control Name Length

Sets the number of characters to be displayed in the control's name (shown in the Page menu below). The identifier "expected length" appears next to lengths that should be specifically addressed (3, 4, 5, 6, 7, 8, and 31). If you use the XML page table system, the names can be specified in the [Page Table Editor](#) application. Otherwise, the function `GetControlNameOfLength()` is responsible for providing names with these lengths.

- Control Value Length

Sets the number of characters to be displayed in the control's value (shown in the Page menu below). The identifier "expected length" appears next to lengths that should be specifically addressed (4, 5, 6, 7, 8, and 31; also, 3 is used for ProControl switch states). The plug-in Library call `GetValueString()` is responsible for providing values with these lengths.

- Highlighted Page

Highlights the selected page in the plug-in window in Pro Tools.

- Highlight Color

Sets the highlight color. The highlight color can be: red, green, blue or yellow. Note that at minimum, plug-in's should support these four colors of highlighting!

- Page X

The actual page table layouts are shown here. The following information can be seen in this menu item.

- The control's name, as returned from the XML page tables. If the table type is Mackie, then the length will be 4 (unless overridden by changing the "Control Name Length" menu item). If the table type is ProControl, the length will be 3 (again, unless overridden, but usually there is no point in doing so). Also, with ProControl set as the table type, the special ProControl symbols will appear here if they are part of the name (however, note that they are small and can be difficult to see). Finally, if the table type is default, the name will be shown with the number of characters as specified in the "Control Name Length" menu item.
- The control's value is shown next. Both the Mackie and ProControl table types will automatically set the control's value length to 4 and 3, respectively. Otherwise, this can be set with the "Control Value Length" menu item. `GetValueString()` is responsible for providing this 'value' information.

- The number of control steps is shown next for continuous and discrete controls. For example, a discrete control will appear as: "(discrete: N steps)", where N is the # of steps of the control.
- If "(NoL)" is displayed, this simply means that it is a new plug-in, and supports either the XML page tables or the `GetParameterNameOfLength()` function call. If "(NoL)" does not appear, then the plug-in is older and you should not expect the control names or values to be optimized - since they are created by just truncating the longer values that the older plug-ins return.
- If "(highlight)" is displayed, this means that the string will be reverse highlighted when displayed on hardware controllers that support it. Not all hardware controllers utilize reverse highlighting.
- Next, orientation flags for each plug-in control will be displayed. The format is: "(Value: xxx yyy)," where xxx is either "BMin" (`kDAE_BottomMinTopMax`) or "TMin" (`kDAE_TopMinBottomMax`); and yyy is either "LMin" (`kDAE_LeftMinRightMax`) or "RMin" (`kDAE_RightMinLeftMax`). This text identifies the value of the control's orientation flags, as returned by `GetParameterOrientation()`.
- Also shown is the radial LED encoder-display mode (as returned by `GetControlOrientation()`) assigned to each control (this radial LED surrounds each encoder). The possible modes are: "spread" `kDAE_RotarySpreadMode`, "wrap" `kDAE_RotaryWrapMode`, "boost" `kDAE_RotaryBoostCutMode`, or "dot" `kDAE_RotarySingleDotMode`, along with either "RMin" `kDAE_RotaryRightMinLeftMax`, or "LMin" `kDAE_RotaryLeftMinRightMax` - indicating which side the minimum AAE value is being mapped to.

12.46.7.5 Control Highlighting Scheme

Note that plug-in controls that are currently controllable on any page of a plug-in will be highlighted in blue when they are the active page on a control surface. Therefore, it is very important to implement the highlighting of controls in your plug-in. Plug-in highlighting is also used with automation. In general, four common colors should be implemented:

- Red: Write automated
- Green: Read automated
- Blue: Accessible on control surface (stays blue if control is also read automated)
- Yellow: Accessible on control surface and write automated

You can use the "hidden" popup menu above to test that your color schemes are working properly.

12.46.7.6 Control Numbering Layouts

Most of the advanced control surfaces have both rotary encoders (knobs) and switches. The knobs and switches are handled differently by different surfaces. C|24 has a set of 24 rotary encoders and 24 switches for plug-in editing, and 003 has 8 encoders and 8 switches, all set up in pairs. However, both of these control surfaces automatically assign a control with only two possible values (e.g., "on" or "off") to a switch, while a control with three or more possible values, whether it is discrete or continuous, is automatically assigned to the rotary encoder. Therefore, no distinction is made between control numbers for switches and control numbers for encoders.

The following tables show how the control numbers are arranged for control surfaces which have distinctions between their encoders and switches.

Table 12.27 Table 2: D-Control Channel Strip - Numbering Layout

Encoders	Switches
1	7
2	8
3	9
4	10
5	11
6	12

Table 12.28 Table 3: D-Control/Pro-Control Custom Fader Mode - Numbering Layout

Encoders	Switches
1	9
2	10
3	11
4	12
5	13
6	14
7	15
8	16

12.46.7.7 Alphanumeric Displays

With the advent of the newer advanced control surfaces (CS), plug-ins now have the opportunity to provide information to the user via alphanumeric displays on the CS. Unfortunately, the displays are limited in the number of characters that can be shown. For instance, on the Mackie HUI, nine characters are provided for each plug-in control, allowing only four characters for the control name, and four characters for the control value, and one for a space between them. On ProControl, eight characters are provided for each plug-in control, with three characters allocated to displaying a control name, and four characters for its control value. On C|24 and 003, four characters are provided for the plug-in name, control name, and the control value. For the control value, these four characters include a +/- and/or any necessary unit abbreviations (ex: K, s, dB, etc.). D-Control and Command|8 provide six characters for the plug-in name, control name, and control value.

In order to display meaningful information in these short character strings, plug-ins will have to optimize the strings that they return to AAE. The dispatcher calls `MapControlValToString()`, which in turn calls `GetValueString()`, is used to obtain these control value strings. Typically in the past, the requested length argument of both these member functions, i.e., `maxChars` in `MapControlValToString()` or `maxLength` in `GetValueString()` has been ignored; but, from here on out, they should be carefully examined and used to create an optimum and meaningful string for any requested length.

To prevent the code from becoming unwieldy, as in the case of trying to provide strings for all requested lengths, a minimum number of expected lengths should be specifically addressed. The expected value lengths are: 4, 5, 6, 7, 8, and 31. In addition, ProControl switch states are a maximum length of 3 (i.e., when dealing with the state of a switch for ProControl, provide this information in 3 characters or less). Therefore, whenever possible, a plug-in should return the most meaningful string that will fit in any particular requested length, and at minimum, handle the expected lengths. If a plug-in does not have custom code to handle a particular requested length, it can round the length down to the next smaller expected length and use the code that it has for it. For example, a request of 9 characters should be converted to an expected request of 8 characters.

Please note: Since truncating a long string to fit within the requested length will not provide meaningful results in most cases, plug-ins must specifically provide code for deriving useful strings for the expected lengths where applicable.

Since the smaller lengths, especially 4 and 5 characters, are usually too short to display a plug-in's true full value including units, some decisions will have to be made about how to suitably shorten them. The following provides some general guidelines.

If needed, and in order of precedence, try to:

- Remove spaces: 13 Hz becomes 13Hz
- Use common abbreviations for units: 16 seconds to 16sec, or 16 s, 156 Hertz to 156Hz
- Drop the units entirely: 1832 Hz to 1832
- Round the value: 173.3 ms to 173

Here is a table depicting a typical example in more detail. The expected lengths are shown in the left most vertical column.

Alphanumeric Characters

While creating the above strings, the requested length argument passed in (`maxChars` in `MapControlValToString()`, and `maxLength` in `GetValueString()`) should be strictly adhered to! The string returned should be no greater than the requested number of characters. Furthermore, the developer should assume that the buffer passed into this function is only as large as the requested length. Any intermediate string processing should be done in temporary local buffers and only when you have the final string should you copy back to the buffer that was passed in, making sure you copy no more than the requested number of characters, plus the Null character or Length byte, as appropriate; since in general, `MapControlValToString()` uses C strings and `GetValueString()` uses Pascal strings.

Also, in an effort to further help prevent buffer overruns, two new functions have been added to the PI library file `SliderConversions.cp`: `SmartAppendNum()` and `SmartAppendXNum()`, which includes a maximum length argument and should be used in place of `FicAppendNum()` and `FicAppendXNum()` from `FicBasics.cpp`.

Finally, note that `ProControl` and `HUI` will sometimes utilize 5 characters to display its value when a sign is involved. For instance, if the number is -100, the negative sign will appear in the space separating the control name from the control value. `Pro Tools` will take care of this conversion as long as all of the expected lengths are properly provided for. For `C|24` and `003`, this is not the case - only four characters are allowed, so the +/- must be a part of those four characters.

`HUI`, `ProControl`, `C|24`, and `003` all provide alphanumeric displays for visual feedback, with the primary purpose being plug-in parameter editing. Functions in the plug-in library are provided that allow customized parameter strings to be created for use on the display. Specifically, these functions are: `GetControlNameOfLength()` and `GetValueString()`. Fortunately, the details of writing to the display are taken care of by the application. Therefore, the plug-in developer only needs to be concerned with providing meaningful display strings for all plug-in parameters that are controllable. Whether using the XML or legacy page table system, `GetControlNameOfLength()` should return the long version (31 characters maximum) of the plug-in's control names. Short versions of plug-in control names are stored in the XML file, edited with the [Page Table Editor](#) application. If you are also using legacy page tables to support versions of `Pro Tools` prior to 6.4, the short names should also be coded in the `GetControlNameOfLength()` function.

AAE clients like `Pro Tools` call `GetControlNameOfLength()`, but if AAE finds XML data stored in the plug-in, it gets the information from there rather than calling into the plug-in. `GetControlNameOfLength()` is responsible for providing the parameter "names" used on the display. As with parameter "value" strings, it is important to carefully create parameter names that are meaningful in the limited space allocated to displaying parameter names. On `ProControl`, string lengths of three characters will be used for the parameter name strings, where four characters will be used on the `HUI`, `C|24`, and `003`. `D-Control` and `Command|8` use six characters for control names. In general, lengths of 3, 4, 5, 6, 7, 8, and 31 should be specifically addressed in the `Page Table Editor` or `GetControlNameOfLength()`. We begin next, by looking at some concrete examples.

12.46.7.8 ProControl Display

`ProControl` also provides an alphanumeric display; however, there are some significant differences that need to be addressed. Shown is a generic representation for the `ProControl` display. The image below shows what users will usually be viewing on `ProControl`'s displays, which are the current Encoder/Value settings. Figure 13 shows the current switch settings when the user toggles to the Switch/State display mode. `ProControl` will only display one of three views at any given time.

Pro-Control Encoder Display

ProControl Switch Display

As with the `HUI`, meaningful strings will have to be provided in the limited available lengths. `ProControl`, `C|24` or `003` value strings require no special treatment other than what has already been stated above for `HUI`, since all

of these control surfaces display their values in 4 digits/characters, as returned by `GetValueString()`. The biggest difference between the HUI display and the Avid control surfaces' displays is that the Avid control surfaces can display symbols.

Let's take a moment to explain how the displays of C|24 and 003 work. Both of these surfaces have a four character LED display located above each encoder/switch pair. When in Channel mode, these scribble strips show the plug-in name (if any) on that channel. When that plug-in is selected, the display switches to show the controls for the plug-in. Now each display shows the name of the control. The display automatically switches to the current value of the control when the encoder or switch is moved or pushed.

12.46.8 Appendix A. Get Parameter Value Info

12.46.8.1 Overview

```
AAX_Result GetParameterValueInfo ( AAX_CParamID iParameterID, int32_t i↔
Selector, int32_t* oValue)
```

`GetParameterValueInfo()` is implemented at the Data Model's [AAX_CEffectParameters](#) level, and will allow the app to query a plug-in for the "meaning" of its parameter values. It was designed as a general purpose mechanism that will find additional uses with new selectors in the future. It is used:

- to ensure the EQ and Dynamics sections' EQ type selector LEDs light appropriately for a given band's filter type. We want the appropriate EQ type LED to light for each band's filter type -whether or not your band can switch between filter types.
- to ensure the state of the EQ section's In buttons matches the values of the associated plug-in controls. When each band's In button is lit, it must mean that the EQ is active/on/enabled. When it is unlit, it must mean that the EQ is off/disabled/bypassed. (Some plug-ins have EQbypass controls (On = bypass); others have EQ In buttons (On = On). We can derive "meaning" from the control regardless of control value.)
- similarly, to ensure the state of the Dynamics section's Filt In buttons matches the values of the associated plug-in controls. When each band's Filt In button is lit, it must mean that the EQ is active/on/enabled. When it is unlit, it must mean that the EQ is off/disabled/bypassed.

12.46.8.2 Implementation

`GetParameterValueInfo()` will be of type [AAX_EParameterValueInfoSelector](#) ([AAX_Enums.h](#)) and will allow for future queries on parameter value "meaning."

```
enum AAX_EParameterValueInfoSelector
{
    AAX_ePageTable_EQ_Band_Type = 0,
    AAX_ePageTable_EQ_InCircuitPolarity = 1,
    AAX_ePageTable_UseAlternateControl = 2
};
```

Listing 3: Parameter Selector Enums

Results (not return values) passed back by `GetParameterValueInfo()` will be of one of these two types:

```
enum AAX_EEQBandTypes
{
    AAX_eEQBandType_HighPass = 0,
    AAX_eEQBandType_LowShelf = 1,
    AAX_eEQBandType_Parametric = 2,
    AAX_eEQBandType_HighShelf = 3,
    AAX_eEQBandType_LowPass = 4,
    AAX_eEQBandType_Notch = 5
};
```

Listing 4: EQ Band Type Enums

```
enum AAX_EEQInCircuitPolarity
```

```
{
    AAX_eEQInCircuitPolarity_Enabled = 0,
    AAX_eEQInCircuitPolarity_Bypassed = 1,
    AAX_eEQInCircuitPolarity_Disabled = 2
};
```

Listing 5: EQ Circuit Polarity Enums

```
enum AAX_EUseAlternateControl
{
    AAX_eUseAlternateControl_No = 0,
    AAX_eUseAlternateControl_Yes = 1
};
```

Listing 6: Alternate Control Enum

Please see [AAX_Enums.h](#) for more information.

To add support for this method, you must to override [AAX_CEffectParameters::GetParameterValueInfo\(\)](#) from within your plug-ins Data Model. For a given [AAX_CParamID](#), selector, and parameter value, you must pass back a result (not a return value) denoting the "meaning" of that parameter value. If a parameter has no meaning in the context of the given selector, you should return [AAX_ERROR_UNIMPLEMENTED](#).

One point to note, is that an EQ or Dynamics plug-in may have a band of EQ that does not include an EQ type selector control. The band is just always, say, a HPF. There's no control to map to the EQ type selector button in the page table and therefore no obvious control for which you would pass back [AAX_eEQBandType_HighPass](#) in [GetParameterValueInfo\(\)](#). What is the solution? Well, the other controls on that band (Frequency, Q/Slope, Gain, In) know what type of band they're on. Thus the plug-in should pass back the relevant [EQ_Band_Types](#) enum in [GetParameterValueInfo\(\)](#) for all controls on a given band of EQ. This also applies to key filter bands in your Dynamics plug-ins. In this way, the EQ type selector LEDs in both the EQ and Dynamics sections will always light appropriately.

The second exception applies to EQ or Dynamics plug-ins that have a band of EQ that does not include an In Circuit / Out of Circuit control for that band. In this case, the band is always In Circuit (or On) and the other controls for this band should return [AAX_eEQInCircuitPolarity_Enabled](#) as the result for [GetParameterValueInfo\(\)](#) when the [AAX_ePageTable_EQ_InCircuitPolarity](#) selector is passed in. Code snippets for [GetParameterValueInfo\(\)](#) from the EQ III 7-Band AAX plug-in is provided below:

Note

The logic for the In Circuit / Out of Circuit LED is available in Pro Tools 6.7 and higher.

```
// *****
// METHOD: GetParameterValueInfo
// *****
AAX_Result EQIII_7_Parameters::GetParameterValueInfo ( AAX_CParamID iParameterID, int32_t iSelector,
int32_t* oValue) const
{
    const AAX_IParameter * parameter = mParameterManager.GetParameterByID( iParameterID );
    if ( !parameter )
        return AAX_ERROR_INVALID_PARAMETER_ID;
    if ( iSelector == AAX_ePageTable_EQ_Band_Type )
    {
        if ( parameter->Name() == EQIII_HPF_Type )
        {
            switch( parameter->GetStepValue() )
            {
                case 0 /*Notch*/: *oValue = AAX_eEQBandType_Notch; break;
                case 1 /*HiPass*/: *oValue = AAX_eEQBandType_HighPass; break;
                default: *oValue = -1; return AAX_ERROR_UNIMPLEMENTED; break;
            }
            return AAX_SUCCESS;
        }
        else if ( parameter->Name() == EQIII_LF_Type )
        {
            switch( parameter->GetStepValue() )
            {
                case 0 /*Peak*/: *oValue = AAX_eEQBandType_Parametric; break;
                case 1 /*Shelf*/: *oValue = AAX_eEQBandType_LowShelf; break;
                default: *oValue = -1; return AAX_ERROR_UNIMPLEMENTED; break;
            }
            return AAX_SUCCESS;
        }
    }
}
```

```

else if (parameter->Name() == EQIII_HF_Type )
{
    switch (parameter->GetStepValue() )
    {
        case 0 /*Peak*/: *oValue = AAX_eEQBandType_Parametric; break;
        case 1 /*Shelf*/: *oValue = AAX_eEQBandType_HighShelf; break;
        default: *oValue = -1; return AAX_ERROR_UNIMPLEMENTED; break;
    }
    return AAX_SUCCESS;
}
else if ( parameter->Name() == EQIII_LPF_Type )
{
    switch ( parameter->GetStepValue() )
    {
        case 0 /*Notch*/: *oValue = AAX_eEQBandType_Notch; break;
        case 1 /*LoPass*/: *oValue = AAX_eEQBandType_LowPass; break;
        default: *oValue = -1; return AAX_ERROR_UNIMPLEMENTED; break;
    }
    return AAX_SUCCESS;
}
}
else if (iSelector == AAX_ePageTable_UseAlternateControl)
{
    if ( (parameter->Name() == EQIII_HPF_Type ) ||
        (parameter->Name() == EQIII_HPF_Q) ||
        (parameter->Name() == EQIII_HPF_Slope) )
    {
        const AAX_IParameter* typeParam = mParameterManager.GetParameterByID(EQIII_HPF_Type_Ch);
        *oValue = (typeParam->GetStepValue() == 0 /*Notch*/) ? AAX_eUseAlternateControl_No :
AAX_eUseAlternateControl_Yes;
        return AAX_SUCCESS;
    }
    else if ( (parameter->Name() == EQIII_LPF_Type) ||
        (parameter->Name() == EQIII_LPF_Q) ||
        (parameter->Name() == EQIII_LPF_Slope) )
    {
        const AAX_IParameter* typeParam = mParameterManager.GetParameterByID(EQIII_LPF_Type_Ch);
        *oValue = (typeParam->GetStepValue() == 0 /*Notch*/) ? AAX_eUseAlternateControl_No :
AAX_eUseAlternateControl_Yes;
        return AAX_SUCCESS;
    }
}
return AAX_ERROR_UNIMPLEMENTED;
};

```

Listing 7: EQIII GetParameterValueInfo()

As you'll notice, the EQ III plug-in has separate controls for Q and Slope in its HPF and LPF bands, depending on whether the band is set to notch or band pass. When the Band Type control is set to notch, the continuous Q control is used. When the Band Type is set to Hi/Lo Pass, the discrete Slope control is used. In the page table, the HPF / LPF Q controls are set to the "Q or Slope" in the [Page Table Editor](#) application, and the HPF / LPF Slope controls are set to the "Q or Slope Alt". Then, with the [GetParameterValueInfo\(\)](#) implementation above, the control surface properly swaps the controls when the band type control is changed. In other words, when the function is called with the [AAX_ePageTable_UseAlternateControl](#) selector, if the band type control is set to notch, then [AAX_eUseAlternateControl_No](#) is returned in the result and the control surface puts the Q control at that position. If the band type is set to pass, then [AAX_eUseAlternateControl_Yes](#) is returned and the Slope is placed at that position. Collaboration diagram for Page Table Guide:

12.47 DigiTrace Guide

How to add tracing to your plug-ins and view logging from the plug-in host.

12.47.1 On this page

- [What is DigiTrace?](#)
- [DigiTrace quick start guide](#)
- [DigiTrace log files](#)

- [Configuring DigiTrace](#)
- [Bonus features](#)
- [Adding traces to an AAX plug-in](#)
- [Advanced DigiTrace configuration](#)
- [Compatibility](#)
- [Additional Information](#)

12.47.2 What is DigiTrace?

DigiTrace is a logging tool used by many Avid audio applications. DigiTrace provides high-performance, real-time tracing capabilities and can help you debug hard-to-isolate problems in real-time code. Pro Tools and other Avid audio products are instrumented with DigiTrace, and it is easy to add DigiTrace logging to your AAX plug-ins.

This document outlines how to use DigiTrace, both as a developer to add trace instrumentation to your code and as an end user to view or record trace instrumentation for an instrumented application.

12.47.2.1 What does DigiTrace do?

DigiTrace generates encrypted logs on users' systems. These log files can be decrypted via the DigiTraceDecryptor application that is included in the DigiTrace Tools package.

By default, DigiTrace logs basic information including details about the system, software, component versions, and any errors that are encountered. By using a simple configuration text file, DigiTrace can be easily configured to provide additional logging information such as plug-in loading details. Here are some examples of how you can use DigiTrace:

- You can use DigiTrace in your plug-ins when you need a convenient, high-performance logging solution.
- You can use the default DigiTrace logs that Pro Tools generates to help you understand problems that your plug-ins encounter when running on Pro Tools.
- You can add DigiTrace statements and stack traces to your released plug-ins in order to help you troubleshoot end-user issues more quickly.
- You can (and should!) submit DigiTrace logs when reporting bugs and other Pro Tools issues to Avid.

12.47.3 DigiTrace quick start guide

This section provides quick steps for the following common tasks:

- [Find and decrypt DigiTrace log files](#)
- [Configure DigiTrace for AAX plug-in logging](#)
- [Configure DigiTrace for plain-text output](#)
- [Add tracing to a plug-in](#)

12.47.3.1 Find and decrypt DigiTrace log files

DigiTrace log files are placed into a common logs directory. The specific directory that is used depends on the version of DigiTrace - see [Where are DigiTrace log files stored?](#)

By default, the version of DigiTrace that is installed with Avid audio products generates logs in an encrypted format with the extension ".dlog". Developer builds of Pro Tools and other applications are configured to generate plain-text logs.

You can convert .dlog files to plain-text using the DigiTraceDecryptor tool that is included in the DigiTrace Tools package available for download from the Avid developer portal. To decrypt a log using this tool, simply drag-and-drop the .dlog file onto the tool. You can also set this tool as the default application for opening .dlog files in your OS, which will allow you to decrypt and open .dlog files directly.

12.47.3.2 Configure DigiTrace for AAX plug-in logging

You must customize the DigiTrace configuration to enable extra logging, such as debug logging for [AAX](#) plug-ins.

DigiTrace uses a plain-text configuration file to enable custom logging. This file uses the suffix ".digitrace" and is located within or beside the application. For example, the configuration files for Pro Tools are located at:

- macOS: Pro Tools.app/Contents/Resources/config.digitrace
- Windows: C:\Program Files\Avid\Pro Tools\ProTools.digitrace

To configure DigiTrace to print logs from [AAX_TRACE](#) or [AAX_TRACE_RELEASE](#) macros in [AAX](#) plug-ins, add the following line to the .digitrace configuration file for the application:

```
DTF_AAXPLUGINS=file@DTP_LOWEST
```

If a config.digitrace file does not already exist for a DigiTrace-enabled application then you can create it to enable DigiTrace. For more information about customizing the DigiTrace configuration and enabling different levels of debug logging, see [Configuring DigiTrace](#).

12.47.3.3 Configure DigiTrace for plain-text output

In order to be able to view streaming log output in real time, DigiTrace must be configured for plain-text output. This is the default configuration for developer builds of Pro Tools and other Avid audio applications.

To configure shipping applications for plain-text log output, you must replace the application's installed DigiTrace library with a development version of the DigiTrace library. Development builds of DigiTrace are included in the DigiTrace Tools package. Search for "Digitrace.framework" on macOS or "DigiTrace.dll" on Windows and replace the installed shipping version of the library with the developer version from the DigiTrace Tools package to configure the application for plain-text output.

Note that the developer version of DigiTrace may output logs to a different directory than the shipping version. In general, developer builds of DigiTrace will place log files in a directory next to the instrumented application. For example, developer builds of Pro Tools will output logs to a logs directory placed adjacent to the Pro Tools application bundle rather than in the user's Library/Logs/Avid folder.

12.47.3.4 Add tracing to a plug-in

To easily add tracing to an [AAX](#) plug-in, use the [AAX_TRACE](#) or [AAX_TRACE_RELEASE](#) macros. Logging from the "release" macro will be enabled for all builds of the plug-in, whereas logging from the "standard" macro will only be enabled in Debug builds of the plug-in.

12.47.4 DigiTrace log files

The default logging in Avid audio applications includes data that can be useful in many different troubleshooting situations. For example:

- Information about the user's system configuration
- A complete list of loaded components and libraries. (If the user has an old or incompatible version of your plug-ins installed on his system, you will know about it!)
- Crash logs in the event of a system failure

In addition, you can add DigiTrace logging code to your plug-ins, helping you examine potential issues in the way your plug-in is running on a user's computer even when you cannot reproduce the issue locally.

12.47.4.1 Where are DigiTrace log files stored?

12.47.4.1.1 Log directory DigiTrace logs are stored in a log files directory on the user's system:

~/Library/Logs/Avid/ (macOS) %userprofile%\AppData\Local\Avid\Logs or C:\Program Files\Avid\Pro Tools\Logs (Windows)

This default log directory can be overridden in the DigiTrace config file. See [Advanced DigiTrace configuration](#) for more information.

12.47.4.1.2 Log file names By default the log file will be given a time-stamped name in the format <App↵ Name>_YYYY_MM_DD_HH_MM_SS.dlog. This timestamp represents the system time when the log was created. Like the log directory, this log file name can be changed using the DigiTrace configuration. See [Advanced DigiTrace configuration](#) for more information.

12.47.4.2 Monitoring DigiTrace logs

12.47.4.2.1 Log files You can of course view a log file by opening it periodically. In addition, assuming that DigiTrace is [configured for plain text output](#), you can also constantly monitor a log file in a "streaming" manner. This is possible using standard Unix tools included with macOS or with Cygwin on Windows. In fact, this approach usually works better than telling DigiTrace to use console output due to buffering of the console output.

- For basic real-time monitoring of a single file, use `tail: tail -f /path/to/digitrace/logs/the↵ _logfile.txt`
- For real-time monitoring of the most recent file in the log file directory, use a combination of `tail` and `ls`:

12.47.4.2.2 Console

Console behavior is quite different between macOS and Windows

Windows On Windows, traces sent to the console go to the system debugging console. The only way to view the console output is to be running with an attached debugger.

macOS On the Mac, console traces are sent to stdout console, which shows up in a few places:

- If you're running in a debugger, the debug console will display stdout output, including DigiTrace messages
- If you're not in the debugger, you can view the output in the Console app (/Applications/Utilities/Console). For Pro Tools, look under ~/Library/Logs/Avid/Pro Tools.X.log in the log list. Note that these messages are not displayed in the "All Messages" log.
- Alternately, you can manually look at the log output, again using the `tail` command, e.g. `tail -f "~/Library/Logs/Avid/Pro Tools.0.log"`

12.47.4.3 Log file formatting

Here is the beginning of an example DigiTrace log:

```
*** Digidesign Session Trace for: /Applications/Pro Tools 11.0.2 3PDev.app (pid=0x5aff, version=11.0.2d626)
*** Starting Timestamp: Tuesday, January 7, 2014 4:10:57 PM Eastern Standard Time (89706938666 uS)
*** System Details: OS Version: 10.8.5, CPU Speed: 2.7 GHz, Architecture: Intel 64 bit, Num Processors: 8
*** DigiTrace Config File: /Applications/Pro Tools 11.0.2 3PDev.app/Contents/Resources/config.digitrace
*** Facilities to trace:

DTF_INSTALLED_COMPONENTS@DTP_NORMAL(0e0d)

Time(us),Tid,Facility,Name : Debug Message
-----
89707181683,00c07,0e0d: Pace eden lib version: 2.0.0, r22343 (2.0.0.22343), [...]
89707220338,00c07,0e0d: ShoeTool_Init - shoe tool installed version is 6.000 [...]
89707220374,00c07,0e0d: ShoeTool_IncreaseAIOLimits - var=46, newVal=512, cur [...]
89707220380,00c07,0e0d: ShoeTool_IncreaseAIOLimits - var=47, newVal=512, cur [...]
```

The log file consists of a header followed by a series of log statements. Each log statement includes the following information:

- Time(us) - The time the message was logged, in microseconds since the machine was started.
- Tid - The thread ID of the thread that logged the message.
- Facility - The Facility ID of the facility that's logging the message.
- Name - This is the config name added to all facilities included by this config file. This can be used to group all facilities related to a feature set, for instance. If not set, this is not included.
- Debug Message - This is the actual string passed to the trace facility.

12.47.5 Configuring DigiTrace

You can configure DigiTrace to include or exclude specific traces using the `config.digitrace` configuration file. This file is plain text and includes a single configuration command on each line.

This is the basic format for a command used to enable tracing for a single [facility](#):

facility=[console@minimum console logging priority],[file@minimum file logging priority]

Here are some examples:

- `DTF_APP_VERSION=file@DTP_LOW`
- `DTF_PLUGINS_3P=file@DTP_LOW,console@DTP_URGENT`
- `DTF_ASSERTHANDLER=console@DTP_URGENT`
- `DTF_DAE_MEM=console@DTP_URGENT,file@DTP_LOWEST`

For more information about special configuration commands, see [Advanced DigiTrace configuration](#).

12.47.5.1 Trace facilities

Trace facilities are used by DigiTrace to determine whether or not the given trace statement should be displayed. Trace facilities allow the user to filter trace statements at the component level.

12.47.5.2 Trace priorities

Trace priorities are used by DigiTrace to determine whether or not the given trace statement should be displayed. DigiTrace specifies five trace priorities:

- `DTP_LOWEST`
- `DTP_LOW`
- `DTP_NORMAL`
- `DTP_HIGH`
- `DTP_URGENT`

The DigiTrace configuration file specifies minimum trace priorities. For example, if a trace statement uses `DTP_LOW` and DigiTrace is configured to use `DTP_NORMAL` as the minimum trace priority, then the trace statement will not be sent to the output target. In general, the `DTP_LOWEST` priority setting will populate the trace output with the most verbose information while the `DTP_URGENT` setting will output only the most high level details.

12.47.5.3 Useful DigiTrace facilities

This section includes descriptions of several facilities that are used in Pro Tools and other Avid audio products. The logging provided by these facilities may be helpful when diagnosing plug-in issues.

- `DTF_AAXPLUGINS` This is the standard facility for [AAX](#) plug-ins. This facility will only log traces that are present in [AAX](#) plug-ins themselves, not traces in any hosting code. Plug-ins may use the [AAX_TRACE](#) or [AAX_TRACE_RELEASE](#) macros to log to this facility.

Note

Disabling the `DTF_AAXPLUGINS` facility will slightly reduce the overhead of trace statements and chip communication on HDX systems.

- `DTF_AAXHOST` at `DTP_NORMAL` Logging from the main [AAX](#) host component. Use a lower priority for additional [AAX](#) Host tracing.
- `DTF_PLUGINS` at `DTP_LOW` Miscellaneous plug-in operations, including page table logging, preset directory errors, and DLL loading and unloading
- `DTF_TIPLUGINS` at `DTP_NORMAL` Logging for HDX plug-in algorithm handling details such as packet management and private data field state reset. Use `DTP_LOW` for deeper tracing.
- `DTF_TISHELLMGR` at `DTP_HIGH` Logging from the HDX RTOS
- `DTF_DAE_HOSTDEVICE` at `DTP_URGENT` Performance logging from the real-time audio render thread. See [Real-time AAE performance logging with DigiTrace](#)
- `DTF_DAE_ERRORS` at `DTP_NORMAL` or `DTP_LOW` Information about any errors that occur in AAE. Use `DTP_LOW` to enable stack traces.

- `DTF_ASSERTHANDLER` at `DTP_NORMAL` or `DTP_LOW` Similar to `DTF_DAE_ERRORS`: Information about any asserts that fail. Use `DTP_LOW` to enable stack traces.
- `DTF_THREAD_NAMES_AND_PRIORITIES` at `DTP_NORMAL` or `DTP_LOW` Allows you to look up a thread's debug name from its ID on the standard trace line. Thread names and IDs will be traced as they are created, and you can then use that ID to resolve the thread name of later trace statements. Use `DTP_↔NORMAL` for just names and IDs, and `DTP_LOW` to include priorities.
- `DTF_PACESUPPORT` at `DTP_NORMAL` Plug-in digital signature logging, with some diagnostics for digital signature verification failures.
- `DTF_ADC` at `DTP_NORMAL` Delay compensation logging, including host accounting for plug-in latency.
- `DTF_AUTOMATION` at `DTP_LOW` Parameter touch and release logging.
- `DTF_AUDIOSUITE` at Logging of events specific to AudioSuite plug-in instances.

12.47.6 Bonus features

12.47.6.1 Real-time AAE performance logging with DigiTrace

Pro Tools 11 and higher includes logging for audio render callback performance. To enable this logging, enabled the `DTF_DAE_HOSTDEVICE` facility at `DTP_URGENT`. This facility will enable logging of real-time audio render thread metrics around any render errors that occur.

Here is an example of a performance log:

```
Int(LL): hstEr=0, ioEr=0, dif=2665(2891,23129), tot=2517(1648,2317), in=51(58,83),
clbk=2158(1508,2158), out=108(99,164), offset=[12], mxW=1101(com.avid.aax.↔
eleven.free)
```

The different values included in this log are:

- `hstEr`
- `ioEr`
- `dif`
- `tot`
- `in`
- `clbk`
- `out`
- `offset`

A log of 'x=a (b,c)' means that the (x) value (e.g. `tot`) for the interrupt was (a) us, the running average was (b) us, and the maximum value encountered was (c) us. Therefore, in the example above:

- 1648 us average total time was spent in each interrupt
- 2517 us was spent in this interrupt
- Eleven Free was the longest worker in this interrupt

In practice, it is difficult to precisely log this information during an error. This is due to changes in the interrupt pattern and scheduling when the audio engine is halted. In order to account for this, the performance logging will print out logs for several interrupts around when any error occurs. The actual audio engine error (`hstEr`) may be reported for a "junk" interrupt cycle that is spuriously logged during this halt process.

12.47.6.2 Adding signposts to the DigiTrace log at run-time

Use the shift-` key combination to add a "Trace Flag" line into the DigiTrace log. This allows you to add a "signpost" line to the log right when an important event happens, or before/after an important operation, so that it is easier to find the important details when inspecting the log later.

```
176603608088,2b603,0016: DSK_PrePrimeDiskTask::PrePrimeDiskTask - finish
176603961348,00307,0000: Trace Flag 3 (diff prev: 2.40s, diff start: 7.18s)
176605252296,00307,0000: Trace Flag 4 (diff prev: 1.29s, diff start: 8.47s)
176605252779,00307,0f09: 2016-07-19 23:36:32.499 PTC_Mgr::Idle() -- performing task (Websocket Base)
176606039830,00307,0000: Trace Flag 5 (diff prev: 0.79s, diff start: 9.26s)
```

Each trace flag signpost includes the text "Trace Flag" and the diff (in seconds) from both the previous trace flag and the first trace flag which was triggered during the current run of the app.

These lines will be printed regardless of the current DigiTrace configuration settings.

Host Compatibility Notes This feature is available in Pro Tools 12.6 and higher

12.47.7 Adding traces to an AAX plug-in

12.47.7.1 Basic AAX logging

Standard `printf`-style logging from [AAX](#) plug-ins is very easy. This feature is built into the [AAX](#) specification and is exposed to plug-ins via the [AAX_TRACE](#) and [AAX_TRACE_RELEASE](#) macros. For more information about basic logging, see the documentation for those macros.

Note

To enable basic [AAX](#) logging via these macros, the `DTF_AAXPLUGINS` trace facility must be enabled.

12.47.7.1.1 Tracing for AAX DSP The [AAX_TRACE](#) and [AAX_TRACE_RELEASE](#) macros, as well as [AAX_ASSERT](#), are cross-platform and are supported for use in [AAX](#) DSP algorithms. For more information about tracing from [AAX](#) DSP algorithms, see the [Tracing](#) section in the [HDX DSP Guide](#).

12.47.7.2 Advanced DigiTrace logging features

As a developer, you can use several advanced macros to extend the functionality of DigiTrace logging in your plug-in beyond the simple `printf`-style features provided by [AAX_TRACE](#). The full DigiTrace macro suite includes macros for stack traces, very long traces, or even the ability to dump a block of memory to the log.

Note that these advanced features are only available on the host system. They are not currently available from algorithms running on embedded hardware.

12.47.7.2.1 What files do I include in my project? To add advanced DigiTrace instrumentation to your source code you must:

1. Include `DigiTrace.h` in the file where you are going to put your trace statements.
2. Compile `CDigitraceAccess.cpp` into your project

If you have problems including the DigiTrace header file, try moving it to the top of the file that you're including it into. DigiTrace has no other dependencies and should be safe to include into any component.

12.47.7.2.2 What do I do if I encounter problems compiling or linking? Should you run into linker errors or other problems after adding the DigiTrace header file, please go through the following items and verify that each is included in your project:

- The CDigitraceAccess.cpp file automatically searches for and loads the DigiTrace.dll (Windows) or DigiTrace.framework (Mac) component and ensures that the appropriate function pointers are initialized. If you receive linker errors, the missing symbols are likely in this file. Note that your project will need the path to CDigitraceAccess.h in order to compile this file.
- If you receive an include file error for DigiPragmas.h then you will need to add the header's path to your project's search paths. This file is included in the most recent DigiTrace Tools packages but may not be included in some older packages.

12.47.7.2.3 Macros DigiTrace provides five core macros for trace output, which are:

- `TRACE_R` - for general printf style tracing. Subject to a total line limit of 256 chars.
- `TRACE_PUTS_R` - prints an arbitrary length buffer, splitting it up into clean lines based on line breaks. No formatting.
- `STACKTRACE_R` - for stack trace printing. See below.
- `MEMTRACE_R` - for memory buffer hex tracing
- `FXTRACE_R` - for automatic function entry and exit tracing

12.47.7.2.4 Debug vs. release macros All of the macros listed above are general-use macros, which generate output in both Debug and Release builds. They each have a debug-only variant which excludes the trailing "`<TT>`". Like [AAX_TRACE](#), these debug-only versions compile to a noop in release builds.

The use of debug-only macros is not usually necessary due to the fact that release traces are encrypted and hidden from end users (but not from other developers.) As a best practice, we recommend using the "_R" version of a macro whenever possible. The debug versions of the macros should only be necessary in special circumstances where you specifically do not want to compile the code into release builds.

For more information about tracing in release builds see [Security concerns](#)

12.47.7.2.5 Syntax Adding a DigiTrace statement to your code is as easy as making a single function call thanks to DigiTrace's predefined macros. The basic macro syntax is:

```
MACRO_NAME( TRACE_FACILITY_NAME [ | TRACE_PRIORITY_LEVEL ], MESSAGE_STRING )
```

Here is a code sample:

```
bool my_function(char* data_buffer, int data_buffer_len)
{
    FXTRACE_R( DTF_PLUGINS_3P, "my_function" ); // Automatically trace function entry and exit.
    // You need only to specify a trace facility.
    OS.Err err = FrobnicateBuffer(data_buffer, data_buffer_len);
    if( noErr != err )
    {
        TRACE_R( DTF_PLUGINS_3P | DTP_HIGH, "Couldn't frobnicate the buffer: %s", OS.ErrToString(err) );
    }
    else
    {
        int cBytesToTrace = 64;
        MEMTRACE( DTF_PLUGINS_3P | DTP_LOW, "Data after frobnication", data_buffer, cBytesToTrace );
    }
}
```

12.47.7.2.6 Generating stack traces The `STACKTRACE_R` macro is very useful for getting stack traces of important events in the code like throwing errors (which can be thrown from many locations). One particularly useful feature of this macro is that it allows you to specify a facility and priority for the `printf` part of the stacktrace, e.g. `DTP_NORMAL`, and another one for the stacktrace step, e.g. `DTP_LOW`. See `DigiTrace.h` for a full list of macros and their documentation.

12.47.7.2.7 Turning off tracing in a specific file You can explicitly disable tracing in an instrumented file in the build by defining the `MTurnDbgTraceOff` symbol at the top of the file.

12.47.7.3 Security concerns

Unless you provide your own logging encryption, DigiTrace logs are not secure and should not be used to store any sensitive information.

Logs generated by Pro Tools release builds on users' systems are encrypted. This is primarily for the sake of avoiding confusion in our user community, since DigiTrace logs can be cryptic and potentially misleading for users who are not familiar with our code.

Avid and other third-party developers will see your plug-ins' release trace statements if they load your plug-ins with the appropriate trace facilities enabled. We highly recommend that you keep this in mind when developing your trace statements, both in order to prevent confusion (see the formatting guidelines in the [AAX_TRACE_RELEASE](#) documentation) and in order to maintain the security of your code.

12.47.8 Advanced DigiTrace configuration

The basic configuration command to enable tracing for a facility is described above in [Configuring DigiTrace](#).

There are also additional commands that can be added to the DigiTrace configuration file for more advanced configurations.

12.47.8.1 Configuration command format

- All DigiTrace configuration commands are listed in the configuration file with the form `<token>=<value>`
- Any blank line or line beginning with a '#' character is ignored
- Tokens are not case sensitive
- If there are repeated tokens in a file, the last token wins

12.47.8.2 Advanced configuration commands

- `FileTracingDir = { DIRPATH }`
 - Default: `USE_RELATIVE_PATH`
 - Custom log file directory. e.g. "C:\MyTraceDir" on Windows or "~/MyTraceDir" on macOS.
 - If `DIRPATH == USE_RELATIVE_PATH` then the output trace file directory will be created next to the target application.
- `Append = { true | false }`
 - Default: `false`
 - Append to file. If `true`, the output of this trace will be appended to any existing log file with the same name. Otherwise, this trace will overwrite an existing log file with the same name.
- `LogFileLimit = { LIMIT }`
 - Default: no limit
 - Limits the number of log files kept around for this config file to the specified number.
 - If set to an integer value, DigiTrace will delete the oldest log file(s) until there are only N most recent log files in the output folder.
 - If you rename an output file so it does not have the standard prefix, it is never deleted by this option.
 - Does not work with the "append" option
- `TraceQueueSize = { small | medium | large }`
 - Default: `small`
 - This controls the amount of memory allocated to the trace queue. You probably won't need to change it.
- `BeQuiet = { true | false }`
 - Default: `false`
 - "Quiet" mode. If set to `true`, this configuration option makes all trace output occur without any decoration (i.e. no timestamps, no thread id, no process id, etc.).
 - This mode may be useful for some types of real-time vector tracing or for configuring formatted logs for post-processing with a text editor.
- `FileId = { FILEID }`
 - Default: `none`
 - If set, this string is included in the filename created by DigiTrace for trace output files.
 - Does not work with the "append" option
- `Name = { NAME }`
 - Default: `none`
 - If set, this string is included in every trace for all the facilities that are enabled in this config file.
 - You can have one of these per config file.

12.47.8.3 Dynamically changing the DigiTrace configuration

DigiTrace config files can be loaded dynamically, which means that you can add new configs while the instrumented application is running. Below are the details surrounding dynamic loading:

- In debug builds, this will happen each time the app comes to the foreground.
- The API only does anything if something has changed in your config files that will result in different tracing of some sort. If nothing has changed, the overhead to make the call is $< 1\text{ms}$, and current tracing is not affected.
- If something has changed, the changes are merged in to the existing config objects in memory. Active log files are not interrupted when possible. This takes around 200ms (mostly because of a sleep command that lets threads finish tracing things). Traces that happen in threads during this changeover will be dropped.
- No code in the actual tracing commands was changed.
- You can change any of the attributes of the trace facilities (priority level, file or console, etc).
- You can add new config files on the fly, or if you start with no config file, you can add one on the fly.

12.47.9 Compatibility

DigiTrace is an internal testing and troubleshooting tool. Although we will try to provide up-to-date documentation so that third-party developers can use this tool, we may need to change the way that DigiTrace works at some point and so we cannot make any promises regarding forwards-compatibility.

At the time of this writing:

- DigiTrace is fully compatible with Pro Tools 8.0.3 and later. DigiTrace is installed by default with compatible Pro Tools shipping versions and with some Pro Tools Development Builds.
- DigiTrace is compatible with all versions of the DAE dish in the DSH environment. Tracing must be explicitly enabled in the dsh executable by placing a config.digitrace config file next to the executable.

If you notice significant bugs or other problems with DigiTrace in any Pro Tools release then we encourage you to report the issues to us on the developer forum. We may not be able to address issues immediately, but your feedback is appreciated.

12.47.10 Additional Information

12.47.10.1 Confidentiality

As with all information provided in the [AAX](#) SDK, the information provided in this documentation is confidential and is bound by the terms of your NDA with Avid. You may provide customized DigiTrace configuration files to end users in order to generate useful debugging information on their systems. However, you may not provide users with decrypted DigiTrace logs or with other details provided in this DigiTrace documentation.

Collaboration diagram for DigiTrace Guide:

12.48 DSH Guide

How to test basic functionality of AAX plug-ins using DSH test tool.

12.48.1 Contents

- [What is DSH and how it works](#)
- [Basic set of commands of the DAE dish](#)
- [Basic plug-in tests](#)
- [Debugging and tracing in DSH](#)
- [Scripting interface and batch profiling](#)

12.48.2 What is DSH and how it works

DigiShell is a software tool that provides a general framework for running tests on Avid audio hardware. As a command-line application, DigiShell may be driven as part of a standard, automated test suite for maximum test coverage. DSH supports loading all types of AAX plug-ins except AS, and is especially useful when running performance and cancellation tests of AAX-TI types. DigiShell is included in Pro Tools Development Builds as dsh.exe (Windows) or as dsh in the CommandLineTools directory (Mac).

After it is launched, DigiShell waits for a command name and parameters to be entered via stdin; command results are output via stdout. DigiShell parses its input as command name, followed by a single space, and then command parameters. The command parameters are expected to be a yaml-encoded string. Here are two examples of strings in compact (single-line) yaml format:

- A hash containing lists in compact yaml syntax `{ key1: [val1, val2], key2: [val3, val4] }`
- A list of two lists `[[PIO, 0, 1], [DSP, 1, 1]]`

12.48.3 Basic set of commands of the DAE dish

DigiShell has built-in commands for getting help, creating a DigiTrace configuration and loading DigiShell modules known as "dishes". One can see the command list by running the help command without any parameters. Passing a command name as a single string parameter to the help command will give a more detailed command description.

The default installation of DigiShell includes a few dishes, including the DAE dish. This dish loads the AAE audio engine and can be used for loading and testing basic functionality of plug-ins. The DAE dish can also be used to load a plug-in for basic debugging purposes, and provides a lighter-weight debugging environment than the full Pro Tools application.

Another dish supplied with DSH, the aaxh dish, provides a lower-level hosting environment for [AAX](#) plug-ins. This dish loads a dedicated [AAX](#) host component without audio routing logic or other audio engine responsibilities. In this guide we will focus on loading and running plug-ins using the DAE dish, but we encourage you to also explore the commands available in the aaxh dish and to learn how to exercise your plug-ins in that environment.

12.48.3.1 Loading plug-ins in DSH

The following commands can be used to load and configure the DAE dish:

- `load_dish DAE` Loads the DAE dish
- `init_dae 48000` This command is optional. It configures AAE to work at a specific sample rate. By default it will work at 44100 Hz.

Loading the DAE dish into the DigiShell environment with the built-in `load_dish` command will extend the set of available commands. Among them there is a `run` command. The `run` command can be used in two ways:

- Execute with no arguments: List all plug-in configurations which are available to AAE
- Execute with arguments specifying a particular plug-in configuration: Load a plug-in instance using the specified configuration

You can also search for the id and spec of the specific plug-in with the `findpi` command, which takes a plug-in's name or part of it as an argument, and then searches through the whole list of available plug-ins using this pattern.

Figure 1: DSH command for loading plug-ins.

If the plug-ins was instantiated successfully, then DSH will list all its parameters, just like on the screenshot. If instantiation fails, then DSH in most cases will output the error code, although it is not always obvious what this error code means. Here is the list of possible reasons of some failures:

- -9060 failed to load DSP Hybrid plug-in
- -14140 IO interface is not connected
- -7050 not enough resources for instantiating plugin
- -14378 plug-in exceeded memory limits
- -14003 something is wrong with your HDX card

-30xxx errors are dynamically-generated and can indicate different failures. Failures due to plug-ins exceeding the cycle limit of the DSP CPU will often appear as -30xxx errors. See [-30xxx: Dynamically-generated error codes](#) in the [HDX DSP Guide](#) for more information.

Note

`run` command works for Native and DSP plug-ins, but not for the Audio Suite ones. Also it will fail for DSP Hybrid plug-ins. To be able to instantiate them, one should run `acquiredeck` command.

There are several DAE dish commands for operating with plug-ins' instances:

- `getcurrentinstance` Returns the index of the current instance. The counting starts from 0 for the first instance that has been instantiated, and increments by one for every next instance.
- `getinstanceproperties` Returns the effect name for the Native plug-ins, and much more detailed info for the DSP instances.
- `setcurrentinstance` Sets the instance with the given index as the current instance.

12.48.3.2 Working with HDX card from DSH

One of the benefits of the DAE dish in DigiShell is that it has direct access to the shell environment that loads DSP plug-ins. The included facilities for retrieving load error information from the DSP manager can be very helpful for debugging DSP plug-in load failures. For example, you can use the following DAE dish commands to determine what resource requirement is preventing additional instances from loading onto a single DSP:

1. `reservetidsp all` Reserves all unused DSPs in the system
 2. `unreservetidsp 0` Frees the first DSP for plug-in allocation
 3. `getlastdsploadererror` Retrieve the text of the error that was generated when the final Effect instance attempted to load
 4. `getdspinfo 0` Returns detailed info about the DSP chip with the given index. By executing this command you can figure out whether particular chip is in use currently, which plug-ins are instantiated there, how many resources they consume and how many resources are still available.
- Figure 2: Info about the DSP chip with the given index.

12.48.3.3 DAE dish tips

- With the standard configuration, the system's **AAX** plug-ins folder will be used by DSH and DTT. To override this, create a folder named "Plug-Ins" next to the DTT and CommandLineTools directories. While that directory exists, AAE will only scan the plug-ins in the new Plug-Ins folder.

12.48.4 Basic plug-in tests

There are some basics tests that can be performed for AAX plug-ins in DSH. Among them are instantiation test, measuring of amount of processor cycles that DSP plug-in may consume on different settings, cancellation test and so on.

12.48.4.1 Cycle count performance test

Use the `DAE.cyclessshared` command in the DAE dish to profile a DSP algorithm's cycle count performance. This command measures both the shared and the per-instance cycles used by a plug-in, both of which must be reported to the host. This command also includes the option to load a custom plug-in preset so that various algorithm code paths may be exercised. It is important to report the maximum possible number of cycles that the plug-in may need, so that it had enough resources, even in the worst case. Otherwise one can obtain noise and clicks in the output audio on the extreme plug-in's settings.

The full syntax of this command is as follows: `cyclessshared <index -- spec -- {key:value, key:value, etc.}>` `index` - Index of the plug-in as listed by the `DAE.run` command `spec` - Plug-in ID triplet in array, e.g. ['AVID', 'DmGn', 'DGDT'] `key:value` hash options: `idx` - Index of the plug-in `spec` - Plug-in ID triplet array `run_cached:` `<true -- false>` - Whether to use cached code when measuring. Defaults to false. `load_preset:` `<filename>` - Load the specified preset for each instance before measuring performance `adjust_controls:` `<true - false>` - Randomly change the plug-in's parameter state before running the test

Examples:

- `cyclessshared 21`
- `cyclessshared ['AVID', 'DmGn', 'DGDT']`

- `cyclesshared {spec: ['AVID', 'DmGn', 'DGDt'], load_preset: "mySettings.tfx"}`
- `cyclesshared {spec: ['AVID', 'DmGn', 'DGDt'], adjust_controls: true}`
- `cyclesshared {idx: 21, run_cached: true}` *Do not use cached measurements for reported cycle counts!*

Normal output of this command should look like this:

Figure 3: Normal cyclesshared command output.

Sometimes during the development process it may happen that this test fails, and the reason of such a failure can be different:

1. Plug-in exceeded the DSP chip's memory limit

Figure 4: Plug-in exceeded the memory limit.

2. Plug-in exceeded the processors cycles budget

- The number of instance and shared cycles looks acceptable, but expected number of plug-in's instances that can be instantiated on the chip/card at different sample rates is zero. Resultant cycle count can be used for calculating how much plug-in has exceeded the limit, and how much it should be optimized.

Figure 5: Plug-in exceeded the processor cycles budget.

- If plug-in exceeds the processor's cycles budget too much, then cyclesshared test will most likely output the warning that is highlighted with orange color on the screenshot below. Also the number of both instance and shared cycles will be shown as zero or one.

Figure 6: Plug-in exceeded the processor cycles budget very much.

3. Plug-in processing is not balanced.

If some big parts of code, which do not really depend on the specific plug-in settings, are located under condition structures, and if they make plug-in to do more calculations in one case and less in another case, then that means that plug-in's processing is not balanced. This may cause some problems, because it is hard to predict when this or that condition may become true and how much the amount of processor's cycles that plug-in needs will increase. So it is better to remove such conditional blocks and to perform those calculations every time, even if their result is not really needed in particular cases.

To indicate such situations the correlation coefficient can be used. If its value close to zero, then plug-in has the described problems.

Figure 7: Plug-in processing is not balanced.

12.48.4.1.1 Performance profiling and test signals Some algorithms' performance characteristics are program-dependent, and in such cases use of the the cycles command alone may not be sufficient. To route a test signal to your plug-in while measuring cycles, use of the cycles command along with the `load_wav_file` command in the DAE dish. The basic approach is as follows:

- Use single-buffer manual processing, rather than continuous
- Split your test signal into several pieces, with each piece to be processed using different settings
- Loop on:
 - Load or adjust the PI's settings,
 - process the next piece of audio while measuring cycles

Example:

1. `piproctrigger manual set to single-buffer processing`
2. `load_wav_file "testaudio_pt1.wav"`
3. `load_wav_file "testaudio_pt2.wav"`
4. `load_wav_file "testaudio_pt3.wav"` load audio buffers; take note of return ...
5. `run <mypluginIndex>`
6. `load_settings "mySavedSettings.tfx"` load the settings OR `control [1,24] #set controls directly`
7. `cycles b1` measure cycles while processing first file
8. `load_settings "mySavedSettings2.tfx"` load the next settings
9. `cycles b2` measure cycles while processing second file ... etc.

12.48.4.2 Cancellation test

When porting plug-ins from RTAS to AAX platform, or from 32-bit to 64-bit architecture, or from Native to DSP, it may be useful to compare output of two plug-in's versions to make sure that it is still the same and nothing has been broken. For this purpose DSH cancellation test can be used.

In the simplest case, when both plug-ins are present in the same version of Pro Tools (Native and DSP version of the same plug-in for example), then `diff` command can be used to perform the test: `diff [ <spec1>, <spec2>, <frames> ]` which reports the peak difference in the output amplitude of plug-ins `<spec1>` and `<spec2>` after processing `<frames>` frames of a 1 kHz full-scale sine wave. The maximum difference will be provided in dB.

Another way to perform the cancellation test is to process audio with each plug-in separately manually and to compare the result after that. This scenario allows you to load custom input audio file and special plug-in settings:

1. `piproctrigger manual` This command should be run for DSP plug-ins before loading them. When this option is set, DSP plug-in will start process audio only after `piproc` command is called. Otherwise it will start processing right after the instantiation process.
2. `run 81` Loading plug-in
3. `load_settings "/Users/settings/pitch_settings.tfx"` Loading settings file
4. `load_wav_file "/Users/audio_files/mono_file.wav"` Wav file will be loaded in a buffer, or in several buffer if it has more than one channel. Command will output references to the buffers, like `b1`, `b2` ...

Note

It is not recommended to choose very long audio files for the DSP processing, since the test is very slow, and processing of 10 sec audio file may take up to 1 min depending on the complexity of the plug-in's algorithm.

5. `bclone b1` The easiest way to create the output buffer of the same size is to copy the input buffer. Command will also output references to the resultant buffers.

6. `piproc [b1, b2]` Command which actually is doing passes the input audio through the current plug-in, and is recording the result to the output buffer (which is b2 here). For stereo plug-in command will look like `piproc [[b1,b2],[b3,b4]]`
7. `bfsave [b2, "/Users/saved_buffer"]` Output buffer can be stored to the disk. This may be needed for the cases, when one wants to compare the output of plug-ins, which can not be loaded in the same instance of the DSH. So the output of one plug-in can be saved to disk and then loaded later in the another instance of DSH. Also output buffer can be saved as .wav file with `save_wav_file` command.
8. `bflload "/Users/saved_buffer"` Saved buffer can be loaded again by this command. It will output the reference to the newly created buffer.
9. `bacmp [b1,b2]` This command will compare the contents of the buffers b1 and b2. So it can compare the output buffers of two plug-ins and thus make the cancellation test.

12.48.5 Debugging and tracing in DSH

DSH provides a lighter-weight debugging environment than the full Pro Tools application. So it should be easier to step through the description code of the plug-in there, rather than in PT, because DSH does significantly less initialization work than PT during the loading process.

Also DSH is very useful in situations, when one wants to debug the plug-in's algorithm on a specific audio buffer. The only way to follow plug-in's algorithm work step-by-step on the certain piece of audio is to debug the `piproc` command.

DSH supports tracing, which is based on Avid's DigiTrace. To enable trace logs in DSH, one should create a `dsh.digitrace` config file for it and put it next to `dsh` executable file. It can be the same as `.digitrace` file for the Pro Tools. DSH has built-in commands to generate a DigiTrace config file. The `clear_trace_config` command creates (or clears if it already exists) a DigiTrace config file. The `enable_trace_facility` command enables logging of a specified facility/priority pair.

Note

On the Mac, DigiShell must be relaunched before a new DigiTrace configuration will take effect.

12.48.6 Scripting interface and batch profiling

DigiShell can be scripted using DishTestTool, a Ruby-based command line tool. More details can be found in [DTT Guide](#).

Collaboration diagram for DSH Guide:

12.49 DTT Guide

How to automate different test scenarios for DSH.

12.49.1 Contents

- [What is DTT](#)
- [How to run tests and suites in DTT](#)
- [Writing DTT scripts](#)
- [Logging in DTT and debugging DTT scripts](#)
- [Working with DTT test suites](#)

12.49.2 What is DTT

DishTestTool (DTT) is a Ruby-based command line tool, which provides the ability to script and thus automate DSH test scenarios. It is included in the DigiShell Tools package in the /DTT directory.

Note

Ruby is installed by default on macOS. On Windows, you will need to install Ruby and add it to your PATH variable manually. For information regarding Ruby version compatibility with a specific build of DTT, see the ReadMe.txt file in /DTT/sources.

The DTT folder consists of:

- *Sources*
 - DTT core
 - *scripts* folder - place all DTT script files here
By default, this folder includes a few example scripts demonstrating basic DTT operations and plug-in testing steps:
 - * *DSH_SigCancellation.rb* - script for the cancellation test
 - * *DSH_TI_CycleCounts* - script for performing cycle count test
 - * *SuiteGenerator.rb* - generates suites for the DSH_SigCancellation and DSH_TI_CycleCounts tests
 - *suites* folder - place your DTT suite files here
- *run_test.command* (on Mac) or *run_test.bat* (on Windows) - command file for running tests
- *run_irb.command* (on Mac) or *run_irb.bat* (on Windows) - command-line interpreter

12.49.3 How to run tests and suites in DTT

`run_test` is the main DTT execution program. `run_test` is able to execute Ruby scripts which have been placed in the `scripts` folder within the DTT directory.

- `run_test.command -l` - lists all the available scripts and suites
- `run_test.command 1` - runs test by number
- `run_test.command DSH_SigCancellation` - runs a test by name. Pay attentions that the name of test should be without (!) extension.
- `run_test.command -script DSH_SigCancellation -a sample_rate=48000 -a threshold=-80` - runs a test with test script arguments, which are specified using the `-a` option

Figure 1: running DTT tests.

For more information about script arguments, see [Describing and using input arguments of the script](#)

12.49.4 Writing DTT scripts

Most of the DTT scripts require `DigiShell`, which allows them to run `dsh` and execute different `dsh` commands. Each script should be represented in the form of class, which inherits `Script` class, and also each script must have at least two elements: `self.inputs` section, where all the input arguments of the test should be described, and `run` method, which is the main body of your script.

```
require 'DigiShell'
class ScriptSample < Script
  def self.inputs
    return {}
  end
  def run
    return pass("Well, it didn't explode. So that's something.");
  end
end
```

Listing 1: Skeleton of the script

12.49.4.1 Describing and using input arguments of the script

The available parameters and their values for a script are listed in the static `self.inputs` routine. Input arguments must be organized in the form of a hash map which is returned from this routine.

```
def self.inputs
  return {
    :sample_rate    => [44100, [44100, 48000, 88200]],
    :path_to_tfx    => ['none'],
    :threshold      => [-96],
  }
end
@dsh.init_dae(sample_rate)
```

Listing 2: Describing input arguments for the script and using them

Hash entries should be in the following format: `:arg_name => [default_value, [range of allowed values]]`

These arguments can be used then by just calling them by name, like in the example above with `sample_rate` argument.

12.49.4.2 Writing body of the script

The body of the script must be enclosed in the body of the `run` method of the script class. As far as most DTT tests need DSH, in the example below it's shows how to create a DSH instance in the script and how to use it then. DSH instance can be created with `DigiDShell.new` method, which requires `DigiShell` module, as has already been said. Then all the DSH commands become available as methods of the DSH instance, and input arguments can be passed to these command as input arguments for the methods, i.e. in parentheses `dsh.load_dish("DAE")`. Also it's recommended to handle possible exceptions that may occur during the execution of the code, and to make sure that DSH has been closed, if it was instantiated on the moment of the failure.

```
def run
  begin
    dsh = DigiShell.new(target)
    dsh.load_dish("DAE")
    dsh.init_dae(sample_rate)
    dsh.close
    return pass("Well, it didn't explode. So that's something.")
  rescue Exception => e
    # make sure to close down dsh before returning
    if (dsh)
      dsh.close
    end

    return fail(e)
  end
end
```

Listing 3: Example of the body of the script.

12.49.5 Logging in DTT and debugging DTT scripts

DTT tool has logging, and all the logs collected in the Logs folder, which is located in root directory of the DTT. DTT creates a separate folder for each test and names these folders with the corresponding names + the time when the particular test has been executed. For example:

```
DSH_SigCancellation_20131225_185146_0001
```

Inside each folder there are several log files:

- *xxx.html* – contains info about input & output of the test in a fancy form (tables)
- *xxx_c.txt* – contains the list of the DSH commands that have been executed
- *xxx_d.txt* – DSH output
- *xxx_i.txt* – info about your system
- *xxx_l.txt* – standard output
- *xxx_v.txt* – verbose output

12.49.5.1 Interactive mode

There is an option to run DTT in interactive mode using interactive ruby shell (irb). When running in this mode, DTT creates a shell which is an extended version of the standard Ruby interpreter. Besides the standard functionality, it knows how to work with DTT classes and can give hints on their methods. In particular, the DTT interactive mode shell knows how to work with DigiShell.

To run DTT in interactive mode, go to the DTT folder and launch the `run_irb` program. At this point you will send ruby commands to dsh through the pipe in YAML format:

```
t = Target.new # creates an instance of Target. In this case target is a local machine, though we have a
               # possibility to run test on remote machine.
dsh = DigiShell.new(t) # creates an instance of DigiShell(aka launching dsh binary)
dsh.load_dish('DAE') # Loads 'DAE' dish
dsh.help('init_dae') # Requests help from dsh for 'init_dae' command
dsh.init_dae
plugins = dsh.run #returns an array of plug-ins and writes them to plugins var.
plug-ins[0] #reaching first plug-in from the list
#... whatever you want to do
```

Listing 4: Running DTT in interactive mode

12.49.6 Working with DTT test suites

Suites are files which contain the list of DTT scripts that should be run, and parameteres for these tests. These files should be created in YAML format. The list of the tests should be preceeded by `tests:` line. Then tests to be run should be described as a map with the following members:

- `name:` - name of the test
- `enabled:` - determines whether test will be run or skipped
- `args:` - input parameters of the test

The input parameters of the test should be orginized as a hash map. That means that all keys should start with `":` and look like `":plugin_spec: "`.

Also suite may contain a section with the general parameters like:

- `verbose:` `false` which determines whether the output of the test in the console will be verbose or not.
- `timeoutFactor:` `"16.0"` which defines the time period, after which test will exit in case it stuck on the execution of the certain peice of code.

Example of the suite:

```
suitesettings:
  verbose: false
tests:
#
# Cycle counting test
#
- name: DSH_TI_CycleCounts
  enabled: true
  args:
    :plugin_spec: 'Digi,Pich,Psmm'
    :sample_rate: 48000
```

Listing 5: Example of the DTT test suite

Note

All the suites files should have an extension `.gss`

12.49.6.1 Autogeneration of the suites

Sometimes it is necessary to generate the suites for the particular script for all the plug-ins from the bundle and/or for different sample rates. In this case instead of the copy-paste, which may lead to some mistakes and erratums, suitegenerator can be used. This is a special script, which takes as arguments the name of the script(s), for which the suites should be generated, and the list of their input parameters. Strange as it may sound, this data should be formed as a suite. Script itself is available as SuiteGenerator.rb along with other scripts in the DTT.

Note

SuiteGenerator.rb can generate suites only for the two tests: DSH_SigCancellation test and DSH_TI_CycleCounts test

Here is the example of how to use this script to generate the suites for all the plug-ins from the 'D-Verb' bundle for all the sample rates the cancellation test AAX Native vs AAX DSP, and for the cycle counts test:

```
tests:
# Generate suite for Cancellation test: AAX Native vs AAX DSP
- name: SuiteGenerator
  args:
    :plugin_name: 'D-Verb'
    :path_to_audio_files: /Volumes/G_Audio/GS_Test_Resources/audio/
    :path_to_presets: /Volumes/G_Audio/GS_Test_Resources/PL_Settings/
    :test_script: DSH_SigCancellation
  enabled: true
# Generate suite for Instance Count test
- name: SuiteGenerator
  args:
    :plugin_name: 'D-Verb'
    :path_to_presets: /Volumes/G_Audio/GS_Test_Resources/PL_Settings/
    :test_script: DSH_TI_CycleCounts_se
  enabled: true
suitesettings:
  verbose: false
  timeoutFactor: "16.0"
```

Listing 6: Example of how to use SuiteGenerator script for generating suites for all the plug-ins from the 'D-Verb' bundle for the all sample rates for the cancellation and for the cycle counts test.

To generate the suites, one should run this suite for the SuiteGenerator as an ordinary suite by executing the `run_test.command` <the name of the suite for the SuiteGenerator> command. All the suites will be generated into the single file, which will be located inside the suites folder, and will be called like:

`dspVSnative(optional)_<the name of the plug-in>_<the name of the test>.gss`

Examples:

- `TRIM_DSH_TI_CycleCounts.gss`
- `dspVSnative_TRIM_DSH_SigCancellation.gss`

Collaboration diagram for DTT Guide:

12.50 Extensions

12.50.1

Extensions to the AAX SDK.

Documents

- [GUI Extensions](#)
GUI Extensions for the AAX SDK.
- [Monolithic VIs and Effects](#)
Extension of the [AAX_CEffectParameters](#) class for monolithic VIs and effects.
- [Other Extensions](#)

Collaboration diagram for Extensions:

12.51 GUI Extensions

GUI Extensions for the AAX SDK.

12.51.1 About the SDK's GUI Extensions

The code and projects in the SDK's Extensions/GUI/ directory demonstrate how to extend the AAX SDK's GUI programming interface using a variety of popular GUI frameworks, including:

- Native Cocoa
- Native Win32
- VSTGUI
- JUCE

These projects do not represent core functionality of the AAX SDK, but rather they serve as examples of how plug-in GUIs can be written to the AAX specification using a variety of different approaches.

12.51.2 Notes

- The VST and JUCE GUI Extension library projects use a macro value to resolve file paths to the installed framework directory. This macro is defined in a Visual Studio property sheet on Windows and as a custom project variable on Mac. Because this macro will not be resolved on Mac until compilation, the Xcode GUI will not be able to find the included files. However, the projects should build successfully once the macros are updated to point to the correct directory.
- The JUCE GUI Extension code in this SDK was written using version 1.51 of the JUCE framework
- Several VSTGUI-based headers with an "_ext" filename are provided along with the SDK's GUI Extensions code. These headers are slightly modified versions of the corresponding headers that are distributed with VSTGUI. The headers have been modified for use with our example plug-ins because we had several problems when using the VSTGUI SDK for 64 bit. For example, we encountered conflicting typedefs/redefinitions like `int32_t` etc., which are preprocessed out of the `vstguibase_ext.h` file. These headers should not be required to build a 32-bit plug-in and they were only added in our transitional AAX SDK, version 1.5.

Collaboration diagram for GUI Extensions:

12.52 Monolithic VIs and Effects

Extension of the [AAX_CEffectParameters](#) class for monolithic VIs and effects.

This extension to [AAX_CEffectParameters](#) adds some conveniences for Virtual Instrument (VI) plug-ins and for other plug-ins that use a monolithic processing object, i.e. an object that combines state data with the audio render routine in a single object.

- The [RenderAudio](#) method provides a direct audio processing callback within the data model object. Perform all audio processing in this method.
- The [StaticDescribe](#) method establishes a generic MIDI processing context for the Effect. Call this method from the plug-in's [Description callback](#) implementation.
- The [AddSynchronizedParameter](#) method provides a mechanism for synchronizing parameter updates with the real-time thread, allowing deterministic, accurate automation playback. For more information about this feature, see [Fixing timing issues due to shared data](#)

Note

This convenience class assumes a monolithic processing environment (i.e. [AAX_eConstraintLocationMask_DataModel](#).) This precludes the use of [AAX_CMonolithicParameters](#) -derived Effects in distributed-processing formats such as AAX DSP.

[AAX_CMonolithicParameters](#) Collaboration diagram for Monolithic VIs and Effects:

12.53 Other Extensions

12.53.1

MIDI logging utilities

- void [AAX::AsStringMIDIStream_Debug](#) (const [AAX_CMidiStream](#) &inStream, char *outBuffer, int32_t in↔ BufferSize)

Filesystem utilities

- bool [AAX::GetPathToPlugInBundle](#) (const char *iBundleName, int iMaxLength, char *oModuleName)
Retrieve the file path of the .aaxplugin bundle.

12.53.2 Function Documentation

12.53.2.1 AsStringMIDIStream_Debug()

```
void AAX::AsStringMIDIStream_Debug (
    const AAX_CMidiStream & inStream,
    char * outBuffer,
    int32_t inBufferSize )
```

Print a MIDI stream as a C-string

Sets an empty string in release builds

12.53.2.2 GetPathToPlugInBundle()

```
bool AAX::GetPathToPlugInBundle (
    const char * iBundleName,
    int iMaxLength,
    char * oModuleName )
```

Retrieve the file path of the .aaxplugin bundle.

Parameters

in	<i>iBundleName</i>	• macOS: The <code>CFBundleIdentifier</code> value set in the plug-in's .plist file • Other: This parameter is ignored
in	<i>iMaxLength</i>	
out	<i>oModuleName</i>	A preallocated buffer of size <code>iMaxLength</code>

Collaboration diagram for Other Extensions:

12.54 Supplemental Information**12.54.1**

Supplemental documents beyond the scope of the AAX SDK.

Documents

- [Troubleshooting](#)
How to solve common issues.
- [Distributing Your AAX Plug-In](#)
Details about packaging and distributing your AAX plug-ins.
- [AAX Interfaces](#)
Full list of AAX interfaces.
- [Host Support](#)
Supported features in each AAX host.

- [Known Issues](#)

A list of known bugs affecting AAX plug-ins.

- [Change Log](#)

Changes between AAX SDK versions.

- [Example Plug-Ins](#)

Descriptions of the SDK's example plug-ins.

Collaboration diagram for Supplemental Information:

12.55 Troubleshooting

How to solve common issues.

12.55.1 Contents

- [Plug-In Fails to Load in Shipping Pro Tools](#)
- [Plug-In Causes Audio Streaming Errors](#)

12.55.2 Plug-In Fails to Load in Shipping Pro Tools

If your plug-in fails to load in shipping Pro Tools with the message "The following plug-ins failed to load because they are not valid 64 bit AAX plug-ins" then the most likely reason is that the plug-in does not have a valid digital signature.

Your AAX plug-ins will not be compatible with shipping versions of Pro Tools until they are digitally signed using tools provided by PACE Anti-Piracy, Inc. As an AAX developer you can receive these tools free of charge. Read the [Digital signature](#) section of the [Pro Tools Guide](#) to learn about the digital signing requirements for compatibility with Pro Tools.

To verify whether this failure is due to an invalid digital signature vs. some other library loading failure, check the Pro Tools [log file](#). A failure caused by an invalid digital signature will result in log lines like the following:

```

Sys_PACE::GetDigitalSignature - looking for Eden dsig for path "/Applications/ProTools/Plug-Ins/DemoGain_example.aaxplugin/ 0
Sys_PACE::GetDigitalSignature - dsig error name /Applications/ProTools/Plug-Ins/DemoGain_example.aaxplugin/ 0
legacy Dsig check disabled??
Sys_PACE::GetDigitalSignature - did NOT get valid dsig /Applications/ProTools/Plug-Ins/DemoGain_example.aaxplugin/ 0
Plug-In Binary "DemoGain_example.aaxplugin" failed to load with err = -7054.
Plug-In Binary "DemoGain_example.aaxplugin" 1.0 : Failed to load.

```

Another way to check whether a plug-in's digital signature is invalid is to test the plug-in in a Pro Tools developer build or with the [DigiShell](#) utility. If the plug-in successfully loads and runs in these tools but not in a shipping build of Pro Tools then it is very likely that the problem is in the plug-in's digital signature.

If you are having an issue running the signing tools then please check this list of the most common failure points:

1. Bad command line arguments for `wraptool`
2. An invalid developer certificate

3. An expired developer certificate
4. The Eden Tools license is not activated to your iLok USB key
5. Your code signing certificate is not installed on your iLok USB key
6. For Mac, the Xcode command line tools are not installed on your signing system
7. The plug-in bundle itself is malformed and will not load
8. The plug-in bundle is being modified at some point after being signed, thereby invalidating its digital signature

If a digital signature was successfully applied to an AAX plug-in at one point in the build process but now the plug-in fails to load due to a bad signature then the most likely reason is that someone or something has altered the signature or the contents of the .aaxplugin bundle thereby invalidating the signature. The most common reason for a digital signature to become invalidated is that something is changed within the .aaxplugin bundle when moving between different systems or when archiving/unarchiving.

Several things can cause this kind of signature invalidation. Here are some examples:

- If symlink are not preserved when copying the plug-in
- If there was some actual tampering of the plug-in after the build
- If there is corruption of the plug-in binary itself
- If the .aaxplugin bundle contains one or more file names with exotic characters which change representations when moved between filesystems with different character encoding schemes

Note that the AAX digital signature covers the entire .aaxplugin bundle so any actions which affect the contents of this bundle in any way after signing will invalidate the bundle's digital signature.

If the failure is occurring on an isolated system then replacing the .aaxplugin which has an invalidated signature with an original, untampered copy (e.g. via a reinstall) should resolve this issue.

If the failure is occurring only on certain systems then try archiving and copying the failing plug-ins back to a system where the plug-in loads successfully then comparing the archived copy to a known successful copy to see if there are any differences to the file names or binary contents of the files within the bundle.

12.55.3 Plug-In Causes Audio Streaming Errors

See also

[Real-time performance](#)

The algorithm callback in audio plug-ins is executed within a complex real-time environment, often with tight deadlines for not only the plug-in itself but for other plug-ins participating in the same processing chain. The real-time threading model is managed outside of the plug-in and may be different across different plug-in hosts and formats. Sometimes, things go wrong and a deadline is missed.

Figure 1: Processing thread utilization during a sporadic audio streaming error

This can happen naturally when the system is loaded to capacity and the CPU simply does not have time to complete all of the work required by the plug-in algorithm routine before the processing deadline. Most often, however, audio processing errors occur in situations when there ought to be more than enough time to do all of the work required by

the plug-in. As shown in the image above, the real-time threads can appear to have low CPU utilization and plenty of overhead, then suddenly a deadline is missed. What happened?

In most cases, audio engine errors occur when a single plug-in instance significantly overruns the processing deadline. The instance usually processes quickly in prior executions and does not give any indication of impending doom.

Figure 2: Call execution times for three plug-in instances in a chain

There are many reasons why this can happen. One excellent tool for evaluating these kinds of failures on macOS is the `ktrace` utility. This utility collects system calls, thread interactions, and backtraces similar to Instruments data in a simple command line tool. This can provide a detailed view of the state of the system leading up to an audio streaming error, and can be used to capture logs on any Mac system, even those without special developer tools installed. Once the tool is running there is a minimal performance impact. Avid provides a `ktrace` capture utility for use with Pro Tools that can trigger captures based on Pro Tools audio engine errors. You can download this tool from the AAX SDK downloads area.

Figure 3: Beginning in Pro Tools 2021.6, this dialog is presented when a plug-in significantly overruns its deadline

Here are some of the culprits that Avid has found when investigating these kinds of performance issues using `ktraces` and similar utilities. Use these examples as a guide for the kinds of things to watch out for in your plug-ins, especially when you hear reports that your plug-ins may be triggering sporadic audio engine errors.

- System calls, C++ library calls, and other language features' call synchronization

When tracking down intermittent performance issues, watch for any calls into the standard C++ library or any use of higher-level language features that are not explicitly designed for real-time use.

You should never trust that STL and C++ library implementations are safe to use in a real time context. Of course STL containers are not thread safe, but in real-time code you should avoid all use of C++ library functions, not just containers, unless you are certain that the library implementation you are using will have no performance related side effects.

For example, in some macOS versions `std::clock` will take a kernel mutex, causing an unexpected priority inversion with any lower-priority thread using `std::clock` at the same time.

Furthermore, the runtime features of higher-level languages are often not designed for running in real time. Objective-C and Swift messaging calls can take locks and should never be used in a real time thread and other languages' features are similarly out of your control as a developer. Avoid them in your algorithm logic.

- Other components and libraries

Similarly, any third-party component or library should not be used from the real-time thread unless it is explicitly designed for use in this context. It can sometimes be difficult to track calls into library code, especially if the library use is not isolated to a particular function in the plug-in such as its graphics or signal processing. Be particularly careful to isolate the plug-in's algorithm logic when using general-purpose libraries that serve multiple functions in a plug-in or when using objects that combine multiple functions.

- Synchronization within the plug-in

Any synchronization of data access within a plug-in must be performed in a way such that the real-time algorithm can never block on a resource held by a lower priority thread.

In an AAX plug-in, parameter changes are applied on a different thread than audio processing. AAX includes a system for synchronizing delivery of parameter updates to the algorithm at the correct time without requiring any synchronization by the plug-in, and you are strongly encouraged to understand the details of how [parameter updates](#) work, in particular how the [parameter update timing](#) is achieved and how to implement proper timing and synchronization even in a plug-in that does not use a standard decoupled algorithm callback.

Be particularly careful if you do not use this system for decoupling and if instead your plug-in shares access to the same data between its algorithm and other logic, even if you are using a third party framework. If your plug-in triggers sporadic deadline misses in the host engine then this can be a productive area to inspect

- File access

File access can accidentally creep into the logic executed by a plug-in algorithm, causing random but severe timing failures. Check to make sure that your plug-in cannot possibly trigger any logging to a file from the audio processing thread. Even having a file handle open on the processing thread can cause performance problems: the OS may trigger a flush on such an open file during a filesystem synchronization event, causing the thread to block despite there being no explicit file I/O operations performed.

- Virtual memory faults

If your plug-in frequently accesses large amounts of data then be sure to check for possible virtual memory faults in any logs concerning audio buffer overruns. Avoid any access to paged memory or files from within the plug-in's real-time code.

- Memory allocation

Memory allocation is the classic example of what not to do in a real-time thread, yet it can be quite difficult to track down. There are no guarantees for the time that a `malloc` call may execute; the call may even need to page memory in from disk in order to complete. It can be very difficult to determine whether any particular call can result in memory allocation, and this underscores the total control that you must take over your plug-in's algorithm logic. Every operation performed within the audio processing thread must be guaranteed to be free of memory allocation, which requires a deep understanding and control over the implementation of all methods called from the algorithm.

- User authorization and copy protection

Some copy protection schemes will helpfully scatter authorization checks throughout your code, relieving you of that chore. Be sure to exclude your plug-in's real-time logic from this process. Authorization checks are complex and can take multiple milliseconds to complete, or more if they involve external licensing hardware or contact with an external server.

Collaboration diagram for Troubleshooting:

12.56 Distributing Your AAX Plug-In

Details about packaging and distributing your AAX plug-ins.

12.56.1 Contents

- [The finishing touches](#)
- [Building your plug-in installer](#)
- [Testing your plug-in](#)
- [Selling your plug-in](#)

12.56.2 The finishing touches

You've completed your main development work and your new AAX plug-in is nearly ready to ship! Now it's time to put the polish on your release.

12.56.2.1 Check and finalize page tables

After development has completed on your plug-in, we recommend that you check and finalize the plug-in's page tables using the [Page Table Editor](#) tool. It can be easy to forget to update the plug-in's page tables after making changes to the plug-in's list of parameters or to other aspects of the plug-in during development. To check for problems, open and view the plug-in's page tables for every layout in the editor app. Verify that the plug-in parameters are arranged properly for each control surface and that the list of available parameters in each layout is correct.

Correct and complete page tables are an important part of the user experience for many AAX plug-in users, and your users will appreciate your attention to detail here!

12.56.2.2 Create factory presets

Each AAX plug-in may be bundled with a set of factory presets. These presets will be made available to users through the host application's plug-in preset management UI.

Plug-in factory presets are stored as .tfx settings files. These files can be generated from any AAX host application which supports plug-in preset management. For example, in Pro Tools it is possible to create a new .tfx settings file by following these steps:

1. Create an instance of your plug-in in a Pro Tools session
2. Manually apply the desired preset settings
3. Choose "Save Settings As..." from the Presets drop-down menu in the plug-in window header

Once you have saved your desired factory presets as .tfx files onto your system you can package them with your plug-in bundle in *.aaxplugin/Contents/Factory Presets. Any presets found in this directory will be copied to the plug-in settings location for the running instance of Pro Tools when Pro Tools scans the plug-in on launch. See [.aaxplugin Directory Structure](#) for more information about supported sub-directories within the .aaxplugin bundle.

The feature for automatically copying factory presets from the .aaxplugin bundle to the plug-in settings directory on the user's system is supported by Pro Tools 11 and later and by all versions of Media Composer with AAX plug-in support.

Plug-in installers for 32-bit plug-ins supporting Pro Tools 10.3.5 and earlier must copy the settings to the plug-in settings folder when the plug-in is installed.

These are the paths for plug-in settings used by Pro Tools and Media Composer versions which support 32-bit AAX plug-ins:

- Mac: /Library/Application Support/Digidesign/Plug-In Settings
- Win: C:\Program Files(x86)\Common Files\Digidesign\DAE\Plug-In Settings

The default paths for plug-in settings used by Pro Tools and Media Composer versions which support 64-bit AAX plug-ins are provided below. However, you should **not** use these paths in your installers since they may be customized using the host application's preferences (for example, the "User Media and Settings Location" preference in Pro Tools.) Instead, use the Factory Preset bundling system described above for installing presets for 64-bit plug-ins.

Default plug-in settings locations for 64-bit [AAX](#) plug-in hosts:

- Mac: ~/Documents/Pro Tools/Plug-In Settings
- Win: C:\[user folder path]\Documents\Pro Tools\Plug-In Settings

For more information about using plug-in presets in the various AAX hosts, see the following pages in the documentation for each host:

- [Pro Tools](#)
- [Media Composer](#)
- [VENUE](#)

12.56.2.3 Sign your plug-in

Pro Tools requires that all AAX plug-ins be signed with a digital signature. The certificate authority for this signature is PACE Anti-Piracy, Inc. and all AAX plug-ins for Pro Tools must be signed with the digital signing tools from PACE. See the [Digital signature](#) section in the [Pro Tools Guide](#) for more information about this requirement.

12.56.3 Building your plug-in installer

Your plug-in installer should place all .aaxplugin bundles into the system's AAX Plug-Ins directory:

- macOS: /Library/Application Support/Avid/Audio/Plug-Ins
- Windows (32-bit plug-ins): C:\Program Files (x86)\Common Files\Avid\Audio\Plug-Ins
- Windows (64-bit plug-ins): C:\Program Files\Common Files\Avid\Audio\Plug-Ins

This directory is searched recursively, so AAX plug-ins may be installed into sub-directories. For example, you may install all AAX plug-ins into a new sub-directory labelled with your manufacturer name.

12.56.3.1 Installing Track Presets

The Track Presets feature in Pro Tools allows users to recall entire tracks, or entire sets of tracks, and to add specific track data such as insert chains, sends, and routing. For example, if a user doesn't know in advance what vocal chain they may want to use, they can begin tracking, and then instantiate a whole set of inserts with stored settings from an existing track preset by clicking on an insert selector and finding that preset.

You are encouraged to create your own track presets and provide them to users in your installers. For example, if you sell plug-in bundles then you may wish to provide users with Track Presets demonstrating useful combinations of multiple plug-ins from the bundle, or if your plug-ins involve some "boilerplate" routing configuration then you can provide a multi-track Track Preset with this routing already established.

Installation Location

Track Presets are stored in the Pro Tools documents folder. Use these locations for default installation

- Mac ~/Documents/Pro Tools/Track Presets
- PC: C:\Users\[username]\Documents\Pro Tools\Track Presets

This location is indexed automatically by Pro Tools.

All of the Track Preset files which you install should be added to a folder with the name of your company. This will ensure that your Track Presets appear as expected in the preset menus in Pro Tools:

- *Pro Tools Documents Folder*
 - /Track Presets
 - * · *Name of your company*

Tagging

A default tags dictionary is available from the [My Toolkits and Downloads](#) page at avid.com. These are not the only tags you can use, but any of these that you do use will be increasing the value and usability of the default set included with Avid products. Using this shared dictionary will ensure that your users can quickly find your Track Presets. Workflow Considerations

- Audibility
 - If you want a track to be heard automatically then route that track to the Monitor Path. If a user is using a Monitor Path the track preset will be instantiated and audible immediately.
- Track Data to Recall
 - In most cases a Track Preset will be created exactly as a user wants to recall it. The available Track Data to Recall from a preset is quite broad though, so you should consider what default import settings make the most sense for each of your presets.
Here are some ideal default settings for a generic single track plug-in focused preset:
- Plug-in Format Conversion
 - Format conversion for plug-ins is designed to work if formats are enumerated correctly and available. This would take place for instance when recalling inserts from a stereo track preset to a 5.1 track preset - most often this should just work if your plug-in is available in all/most formats.
- Including Avid Stock Plug-ins
 - If you wish to include any stock Avid plug-ins in your presets for any reason, stick to these plug-ins that are automatically installed by Pro Tools to be as sure as possible that your end user will be able to fully recall the preset:
 - * *AutoPan; BF-76; Channel Strip; Click II; Dither; Down Mixer; D-Verb; Dynamics III; Eleven Lite; EQ III; InTune; Invert/Duplicate; LoFi; Master Meter; Maxim; ModDelay III; Normalize-Gain; Pitch Shift Legacy; Pitch II; RectiFi; Reverse/DC Removal; SansAmp PSA-1; SciFi; Signal Generator; Time Shift; Time Adjuster; Trim; VariFi*

The following Virtual Instruments are installed separately but come for free with paid Pro Tools versions:

 - * *Boom; DB-33; Mini Grand; Structure Free; Vacuum; Xpand!2*

12.56.4 Testing your plug-in

The AAX Plug-In Burnthrough Grid document describes a number of test cases and workflows for multiple AAX plug-in hosts. This document is available for download as part of the AAX SDK Toolkit on the [My Toolkits and Downloads](#) page at avid.com.

12.56.5 Selling your plug-in

12.56.5.1 Avid Marketplace

Avid may offer to sell your compatible products through our online store. We offer test tools and support services that will help you get your products to market with the highest quality whether you decide to offer them through our online store or independently. Registered developers can further register as Sellers, then work with Avid to add their solutions to the online store. Please visit your My Avid account and go to "My Developer Account" then to "Access Seller Portal" to explore this program or write to partners@avid.com for more information.

Get your AAX Plug-In ready for sale on Avid Marketplace by following these steps:

- *Explore the Avid Webstore* - Review the [Avid Webstore description](#) and learn about this valuable and expanding offering. E-mail us at partners@avid.com with your questions.
- *Sign up* - Register as a Seller (sometimes referred to as a "vendor") by following the link from the "My Developer Account" page and selecting "Access the Seller Portal."
- *Prepare your submission* - Gather the plug-in and other information required to onboard as described in the Onboarding FAQ. Your experience will be easier if you collect these items in advance.
- *Send us your Product* - Submit your products and other required information for testing and publication on the Avid Store!

12.56.5.2 In-App Purchase

In-App Purchase provides a direct path to purchase your products from directly within the AAX host application. For example, when a user opens a session which contains unavailable plug-ins, In-App Purchase can be used to prompt the user to purchase the plug-ins immediately.

See [this article](#) for more information about how to add support for In-App Purchase to your on-boarded AAX plug-ins. Additional documentation regarding In-App Purchase is available under the "In-App Purchase Tools" section of the [AAX SDK Toolkit](#) downloads page in your avid.com account.

Collaboration diagram for Distributing Your AAX Plug-In:

12.57 AAX Interfaces

Full list of AAX interfaces.

12.57.0.1 Interfaces Implemented by the AAX Host

These interfaces are implemented by the AAX Host. References to the host-managed objects are provided to the plug-in through accessor methods, most commonly [IACFUnknown::QueryInterface\(\)](#).

Class [AAX_IAutomationDelegate](#)

Class [AAX_ICollection](#)

Class [AAX_IComponentDescriptor](#)

Class [AAX_IController](#)

Class [AAX_IDma](#)

Class [AAX_IEffectDescriptor](#)

Class [AAX_IHostProcessorDelegate](#)

Class [AAX_IHostServices](#)

Class [AAX_IMIDINode](#)

Class [AAX_IPrivateDataAccess](#)

Class [AAX_IPropertyMap](#)

Class [AAX_ITask](#)

Class [AAX_ITransport](#)

Class [AAX_IViewContainer](#)

12.57.0.2 Interfaces Implemented by the AAX Plug-In

These interfaces must be implemented by the AAX plug-in. Default implementations are provided in the AAX Library via the `AAX_C` classes. Plug-in classes may inherit from the `AAX_C` classes to override the default behavior.

Class [AAX_IACFTaskAgent](#)

Class [AAX_IEffectDirectData](#)

Class [AAX_IEffectGUI](#)

Class [AAX_IEffectParameters](#)

Class [AAX_IHostProcessor](#)

12.57.0.3 Interfaces internal to the AAX SDK

These classes and interfaces are used internally within the AAX Library. References to objects implementing these classes are never passed between the plug-in and the AAX Host, and the AAX Host has no knowledge of these classes.

Class [AAX_IParameter](#)

Class [AAX_IParameterValue](#)

Class [AAX_ITaperDelegateBase](#)

Collaboration diagram for AAX Interfaces:

12.58 Host Support

Supported features in each AAX host.

12.58.1 Host Support

These tables list AAX host support for various AAX interfaces, as well as support for general features. The tables include the version number for the earliest version of each Avid host software which supports the given interface or feature.

The earliest version of each host to support AAX plug-ins is:

- [Pro Tools](#) 10.0
- [Media Composer](#) 8.1
- [VENUE](#) 4.1

For more information about versioning in AAX, including how to check for host support of a particular interface, see [The Avid Component Framework \(ACF\)](#).

12.58.1.1 Platform Support

	Pro Tools	Media Composer	VENUE	
AAX Native	10.0	8.1	<i>none</i>	
AAX DSP	10.0	<i>none</i>	4.1	
AAX Hybrid	11.0*	<i>none</i>	<i>none</i>	
Offline processing (AudioSuite)	10.0	8.1**	<i>none</i>	
ProcessProc / data model co-location	10.0	8.1	<i>none</i>	
Monolithic topology	10.0	8.1	<i>none</i>	
Native processor architecture	10: x86/i386 11+: x86_64	8.1+: x86_64	4.1: x86/i386 4.5+: x86_64	
Compatibility with arm64/x86_64 FAT binaries on macOS	2021.10		n/a	
Properties File	2024.3	<i>none</i>	<i>none</i>	

Note

Pro Tools 11.0 supports AAX Hybrid processing for real-time plug-ins only. Support for AudioSuite processing for AAX Hybrid is supported starting in Pro Tools 11.1.

Media Composer 8.5 and higher support both multichannel and mono AudioSuite processing. Earlier versions of Media Composer support mono only.

12.58.1.2 Describe Interfaces

	Pro Tools	Media Composer	VENUE	
AAX_IACFCollection	10.0	8.1	4.1	
AAX_IACFComponentDescriptor	10.0	8.1	4.1	
V2	11.0	8.1	4.5?	

V3	12.8	<i>none</i>	5.6	
AAX_IACFEffectorDescriptor	10.0	8.1	4.1	
V2	11.0	8.1	4.5?	
AAX_IACFPropertyMap	10.0	8.1	4.1	
V2	11.0	8.1	4.5?	
V3	12.9	<i>none</i>	5.6	
AAX_IACFDescriptionHost	12.8	<i>none</i>	<i>none</i>	
AAX_IACFFeatureInfo	12.8	<i>none</i>	<i>none</i>	

12.58.1.3 Run-Time Interfaces

	Pro Tools	Media Composer	VENUE	
AAX_IACFAutomationDelegate	10.0	8.1	4.1	
AAX_IACFController	10.0	8.1	4.1	
V2	11.0	8.1	<i>none</i>	
V3	12.4	8.6	<i>none</i>	
AAX_IACFEffectorDirectData	10.0	8.1	4.1	
V2				
AAX_IACFEffectorGUI	10.0	8.1	4.1	
AAX_IACFEffectorParameters	10.0	8.1	4.1	
V2	11.0	8.1	4.5?	
V3	11.2	8.1	<i>none</i>	
V4	<i>none</i>	<i>none</i>	5.6	
AAX_IACFHostProcessor	10.0	8.1	<i>none</i>	
V2	12.0	<i>none</i>	<i>none</i>	
AAX_IACFHostProcessorDelegate	10.0	<i>none</i>	<i>none</i>	
V2	11.0	<i>none</i>	<i>none</i>	
V3	12.0	<i>none</i>	<i>none</i>	
AAX_IACFHostServices	10.0	8.1	4.1?	
V2	12.0	8.6	<i>none</i>	
V3	12.8.3	<i>none</i>	<i>none</i>	
AAX_IACFPageTable	12.8	<i>none</i>	5.7	
V2	12.8.2	<i>none</i>	5.7	
AAX_IACFPageTableController	<i>none</i>	<i>none</i>	5.7	
AAX_IACFPrivateDataAccess	10.0	8.1	4.1	
AAX_IACFSessionDocument	2023.6.0	<i>none</i>	<i>none</i>	
AAX_IACFSessionDocumentClient	2023.6.0	<i>none</i>	<i>none</i>	
AAX_IACFTaskAgent	<i>none</i>	<i>none</i>	<i>none</i>	
AAX_IACFTransport	10.0	8.5 (partial)	<i>none</i>	
V2	10.3.7	8.5 (partial)	<i>none</i>	
V3	2021.3		<i>none</i>	
V4	2023.9		<i>none</i>	
V5	2024.3		<i>none</i>	
AAX_IACFTransportControl	2023.9	<i>none</i>	<i>none</i>	
AAX_IACFViewContainer	10.0	8.1	4.1	
V2	12.0.1	<i>none</i>	<i>none</i>	
V3	2022.4	<i>none</i>	<i>none</i>	

12.58.1.4 Features

	Pro Tools	Media Composer	VENUE	
Basic Stem Formats (mono through 7.1)	10.0	8.1	none	
7.x.2 Stem Formats	12.8	none	none	
Surround Stem Formats (5.0.2 to 9.1.6)	none	none	none	
Ambisonics Stem Formats (first through third order)	12.8.2	none	none	
Ambisonics Stem Formats (fourth through seventh order)	none	none	none	
Plug-in type conversion	10.3.8, 11.1	11.0*, none	none	
Auxiliary Output Stems	10.0	none	none	
Sidechain Inputs	10.0	8.5	none	
MIDI	10.0	none	none	
Automation recording and playback	10.0	none	none	
Plug-in presets	10.0	8.4	4.1	
External control surfaces	10.0	8.1	none	

12.58.2 Host Compatibility Notes

See also

[Compatibility Notes](#) in the [Pro Tools Guide](#) document

Member [AAX_CMidiPacket::mIsImmediate](#)

This value is not currently set. Use `mTimeStamp == 0` to detect immediate packets

Member [AAX_CParameter< T >::AAX_CParameter](#) ([AAX_CParamID](#) identifier, `const AAX\_IString &name`, `T defaultValue`, `const AAX\_ITaperDelegate< T > &taperDelegate`, `const AAX\_IDisplayDelegate< T > &displayDelegate`, `bool automatable=false`)

As of Pro Tools 10.2, DAE will check for a matching parameter NAME and not an ID when reading in automation data from a session saved with an AAX plug-ins RTAS/TDM counter part.

As of Pro Tools 11.1, AAE will first try to match ID. If that fails, AAE will fall back to matching by Name.

Module [AAX_DigiTrace_Guide](#)

This feature is available in Pro Tools 12.6 and higher

Member [AAX_eConstraintLocationMask_DLLChipAffinity](#)

This constraint is supported in Pro Tools 10.2 and higher

Member [AAX_eCurveType_Dynamics](#)

Pro Tools requests this curve type for [Dynamics](#) plug-ins only

Member [AAX_eCurveType_EQ](#)

Pro Tools requests this curve type for [EQ](#) plug-ins only

Member [AAX_eCurveType_Reduction](#)

Pro Tools requests this curve type for [Dynamics](#) plug-ins only

Member [AAX_eDataInPortType_Incremental](#)

Supported in Pro Tools 12.5 and higher; when [AAX_eDataInPortType_Incremental](#) is not supported the port will be treated as [AAX_eDataInPortType_Unbuffered](#)

Member [AAX_EHostModeBits](#)

Supported in Venue 5.6 and higher

Member [AAX_eNotificationEvent_ASPreviewState](#)

Supported in Pro Tools 11 and higher

Not supported by Media Composer

Member [AAX_eNotificationEvent_ASProcessingState](#)

Supported in Pro Tools 11 and higher

Not supported by Media Composer

Member [AAX_eNotificationEvent_DelayCompensationState](#)

Supported in Pro Tools 12.6 and higher

Member [AAX_eNotificationEvent_EnteringOfflineMode](#)

Supported in Pro Tools 11 and higher

Member [AAX_eNotificationEvent_ExitingOfflineMode](#)

Supported in Pro Tools 11 and higher

Member [AAX_eNotificationEvent_HostLocale](#)

Supported in Pro Tools 2024.3 and higher

Member [AAX_eNotificationEvent_HostModeChanged](#)

Supported in Venue 5.6 and higher

Member [AAX_eNotificationEvent_LogState](#)

Pro Tools currently only sends this notification to the Direct Data object in the plug-in

Member [AAX_eNotificationEvent_MaxViewSizeChanged](#)

Supported in Pro Tools 11.1 and higher

Member [AAX_eNotificationEvent_ParameterNameChanged](#)

Supported in Pro Tools 2023.3 and higher

Member [AAX_eNotificationEvent_PresetOpened](#)

Supported in Pro Tools 11 and higher

Member [AAX_eNotificationEvent_PriorSettingsInvalid](#)

Supported in Venue 5.6 and higher

Member [AAX_eNotificationEvent_SessionBeingOpened](#)

Supported in Pro Tools 11 and higher

Not supported by Media Composer

Member [AAX_eNotificationEvent_SessionPathChanged](#)

Supported in Pro Tools 11.1 and higher

Member [AAX_eNotificationEvent_SideChainBeingConnected](#)

Supported in Pro Tools 11.1 and higher

Member [AAX_eNotificationEvent_SideChainBeingDisconnected](#)

Supported in Pro Tools 11.1 and higher

Member [AAX_eNotificationEvent_SignalLatencyChanged](#)

Supported in Pro Tools 11.1 and higher

Member [AAX_eNotificationEvent_TrackNameChanged](#)

Supported in Pro Tools 11.2 and higher

Not supported by Media Composer

Member [AAX_eNotificationEvent_TransportStateChanged](#)

Supported in Pro Tools 2021.10 and higher

Member [AAX_ePlugInStrings_Progress](#)

Not currently supported by Pro Tools

Member [AAX_eProcessingState_BeginPassGroup](#)

AudioSuite pass group notifications are supported starting in Pro Tools 12.0

Member [AAX_eProcessingState_EndPassGroup](#)

AudioSuite pass group notifications are supported starting in Pro Tools 12.0

Member [AAX_eProperty_Constraint_NeverUnload](#)

[AAX_eProperty_Constraint_NeverUnload](#) is not currently implemented in DAE or AAE

Member [AAX_eProperty_DestinationTrack](#)

This property is not supported on Media Composer

Member [AAX_eProperty_LatencyContribution](#)

Maximum delay compensation limits will vary from host to host. If your plug-in exceeds the delay compensation sample limit for a given AAX host then you should note this limitation in your user documentation. Example limits:

- Pro Tools 9 and higher: 16,383 samples at 44.1/48 kHz, 32,767 samples at 88.2/96 kHz, or 65,534 samples at 176.4/192 kHz
- Media Composer 8.1 and higher: 16,383 samples at 44.1/48 kHz, 32,767 samples at 88.2/96 kHz

Member [AAX_eProperty_OptionalAnalysis](#)

In Media Composer, optional analysis will also be performed automatically before each channel is rendered. See [MCDEV-2904](#)

Member [AAX_eProperty_SideChainStemFormat](#)

Currently Pro Tools supports only [AAX_eStemFormat_Mono](#) side chain inputs

[AAX_eProperty_SideChainStemFormat](#) is not currently implemented in DAE or AAE

Member [AAX_eProperty_UsesClientGUI](#)

Currently supported by Pro Tools only

Member [AAX_IACFEffParameters::CompareActiveChunk](#) (const [AAX_SPlugInChunk](#) *iChunkP, [AAX_Boolean](#) *olsEqual) const =0

In Pro Tools, this method will only be called if a prior call to [GetNumberOfChanges\(\)](#) has indicated that the plug-in's state has changed. If the plug-in's current settings are different from the settings in `aChunkP` then the plug-in's Compare Light will be illuminated in the plug-in header, allowing users to toggle between the plug-in's custom state and its saved state.

Member [AAX_IACFEffParameters::GetCurveData](#) ([AAX_CTypeID](#) iCurveType, const float *iValues, [uint32_t](#) iNumValues, float *oValues) const =0

Versions of S6 software which support the [GetCurveDataDisplayRange\(\)](#) method will not display a plug-in's curve data unless both [GetCurveData\(\)](#) and [GetCurveDataDisplayRange\(\)](#) are supported by the plug-in.

Member [AAX_IACFEffParameters::GetParameterNameOfLength](#) ([AAX_CParamID](#) iParameterID, [AAX_IString](#) *oName, [int32_t](#) iNameLength) const =0

In most cases, the AAX host will call [GetParameterName\(\)](#) or [GetParameterNameOfLength\(\)](#) to retrieve parameter names for display. However, when Pro Tools is retrieving a plug-in name for display on a control surface the XML data stored in the plug-in's page tables will be used in preference to values retrieved from these methods.

Member [AAX_IComponentDescriptor::AddAuxOutputStem](#) ([AAX_CFieldIndex](#) inFieldIndex, [int32_t](#) inStemFormat, const char inNameUTF8[]) =0

There is a hard limit to the number of outputs that Pro Tools supports for a single plug-in instance. This limit is currently set at 256 channels, which includes all of the plug-in's output channels in addition to the sum total of all of its aux output stem channels.

Pro Tools supports only mono and stereo auxiliary output stem formats

Member [AAX_IComponentDescriptor::AddClock](#) ([AAX_CFieldIndex](#) inFieldIndex) =0

As of Pro Tools 11.1, this field may be used in both Native and DSP plug-ins. The DSP clock data is a 16-bit cycling counter. This field was only available for Native plug-ins in previous Pro Tools versions.

Member [AAX_IComponentDescriptor::AddMIDINode](#) ([AAX_CFieldIndex](#) inFieldIndex, [AAX_EMIDINodeType](#) inNodeType, const char inNodeName[], [uint32_t](#) channelMask) =0

Due to current restrictions MIDI data won't be delivered to DSP algorithms, only to AAX Native.

Member [AAX_IController::GetHostName](#) ([AAX_IString](#) *outHostNameString) const =0

Pro Tools versions from Pro Tools 11.0 to Pro Tools 12.3.1 will return a generic version string to this call. This issue is resolved beginning in Pro Tools 12.4.

Member [AAX_IMIDINode::PostMIDIPacket](#) ([AAX_CMidiPacket](#) *packet)=0

Pro Tools supports the following MIDI events from plug-ins:

- NoteOn
- NoteOff
- Pitch bend
- Polyphonic key pressure
- Bank select (controller #0)
- Program change (no bank)
- Channel pressure

Member [AAX_ITransport::GetBarBeatPosition](#) (int32_t *Bars, int32_t *Beats, int64_t *DisplayTicks, int64_t *SampleLocation) const =0

There is a minor performance cost associated with using this API in Pro Tools. It should not be used excessively without need.

Member [AAX_ITransport::GetCurrentLoopPosition](#) (bool *bLooping, int64_t *LoopStartTick, int64_t *LoopEndTick) const =0

This does not indicate anything about the status of the "Loop Record" option. Even when the host is configured to loop playback, looping may not occur if certain conditions are not met (i.e. the length of the selection is too short)

Member [AAX_ITransport::GetCurrentTickPosition](#) (int64_t *TickPosition) const =0

The tick resolution here is different than that of the tick displayed in Pro Tools. "Display ticks" (as they are called) are 1/960 of a quarter note.

Member [AAX_ITransport::GetCustomTickPosition](#) (int64_t *oTickPosition, int64_t iSampleLocation) const =0

There is a minor performance cost associated with using this API in Pro Tools. It should not be used excessively without need.

Member [AAX_IViewContainer::GetModifiers](#) (uint32_t *outModifiers)=0

Although this method allows plug-ins to acquire the current state of the Windows key (normally blocked by Pro Tools), plug-ins should not use key combinations that require this key.

Module [AAX_Media_Composer_Guide](#)

Some early versions of Media Composer 8 do not search the system plug-ins directory recursively. If your plug-ins are installed into a sub-directory beneath this main directory then they will not be loaded by the affected versions of Media Composer.

Module [AAX_Page_Table_Guide](#)

Pro Tools versions prior to Pro Tools 11.1 use plug-ins' ProControl and ICON page tables (Dynamics, EQ, Channel Strip, Custom Fader, etc.) to map plug-in parameters to EUCON-enabled surfaces, so be sure that your plug-ins also implement these page tables correctly so that users with earlier versions of Pro Tools can have the best possible experience when using your plug-ins.

Module [AAX_Pro_Tools_Guide](#)

Pro Tools requires PACE Eden digital signatures for AAX plug-ins.

Supported in Pro Tools 2019.XX and higher. Also supported (and enabled by default) in Pro Tools developer builds beginning with Pro Tools 2019.6.

Module [AAX_TI_Guide](#)

32 and 64-sample quantum is available in Pro Tools 10.2 and higher

Beginning in Pro Tools 11, AAX DSP algorithms also support optional temporary data spaces that can be described in the Describe module and are shared among all instances on a DSP. This is an alternative to declaring large data blocks on the stack for better memory management and to prevent stack overflows. Please refer to [AAX_IComponentDescriptor::AddTemporaryData\(\)](#) for usage instructions.

Module [AdditionalFeatures_CurveDisplays](#)

For S6 control surface displays, see [PT-226228](#) and [PT-226227](#) in the [Known Issues](#) page for more information about the requirements listed in this section.

Module [advancedTopics_relatedTypes](#)

Pro Tools versions prior to Pro Tools 12.3 do not allow explicit type conversion between types with different product ID values. Beginning in Pro Tools 12.3 both the product ID and the plug-in ID may differ between explicitly related types.

Module [AuxInterface_TaskAgent](#)

This interface is not yet used in any AAX hosts

Module [CommonInterface_Algorithm](#)

As of Pro Tools 10.2.1 an algorithm's initialization callback routine will have up to 5 seconds to execute.

Module [CommonInterface_FormatSpecification](#)

*_ACFGetSDKVersion is required for 64-bit AAX plug-ins only

Module [ExamplePlugins](#)

The DemoDelay_DynamicLatencyComp example is compatible with Pro Tools 11.1 and higher.

Collaboration diagram for Host Support:

12.59 Known Issues

A list of known bugs affecting AAX plug-ins.

12.59.1 Contents

- [Known Issues in the AAX SDK](#)
- [Known Issues in Pro Tools](#)
- [Known Issues in Venue Live Sound Systems](#)
- [Known Issues in Media Composer](#)
- [Known Issues in Control Surfaces](#)
- [Known Issues in Other Software](#)
- [Known Issues in AAX Tools](#)

12.59.2 Known Issues in the AAX SDK

12.59.2.1 AAXSDK-1052

DemoGain_Multichannel does not appear in Pro Tools inserts and AudioSuite listings

Details: The DemoGain_Multichannel example plugin's describe routine fails to register any effects due to an exception.

Resolution: This bug is unresolved

12.59.2.2 AAXSDK-1036

[AAX_ASSERT](#) exposes third party build machine file paths, even in public facing protected and/or stripped binaries

Resolution: This bug is unresolved

12.59.2.3 AAXSDK-897

Calling [AAX_CParameter](#) `SetValue()` methods during [AAX_CEffectParameters::EffectInit\(\)](#) does not set the parameter value

As a workaround, use [AAX_CParameter::SetNormalizedDefaultValue\(\)](#) to control the initial value of a parameter.

Resolution: This bug is unresolved

12.59.2.4 AAXSDK-851

The `register` keyword is incorrectly used in [AAX_Atomic.h](#)

This causes compilation failures in clang on Windows

Resolution: This bug is unresolved

12.59.2.5 AAXSDK-832

Rectifi example does not output any signal in DSP mode

Details: The AAX SDK example version of Rectifi initiates but does not output any sound when in DSP mode

Resolution: This bug will not be fixed

12.59.2.6 AAXSDK-708

The AAX SDK library will not compile using macOS SDK 10.13

Details: Compilation results in an error in NSUUID.h. This error does not occur when using SDK 10.14 or later

Resolution: This bug has been fixed in the macOS SDK

12.59.2.7 AAXSDK-705

[AAX_VHostProcessorDelegate](#) does not detect hosts with [V2](#) support

Resolution: This bug is fixed as of AAX SDK 2.4.0

12.59.2.8 AAXSDK-663

AAX SDK `#pragma pack` errors with XCode 10 and later

Resolution: This bug is fixed as of AAX SDK 2.3.2

12.59.2.9 AAXSDK-599

In the Win32 GUI example plug-in, the mouse cursor disappears when text is entered in text box and only re-appears when the mouse is moved out of the plug-in window bounds

Resolution: This bug is unresolved

12.59.2.10 AAX-578 / PTSW-158310

AAX SDK JUCE example plug-in does not "snap to default" on option-click

Resolution: This bug is fixed as of AAX SDK 1.0.4

12.59.2.11 AAX-585 / PTSW-157451

AAX DemoGain GUI example plug-ins do not correctly handle alt/opt-click for resetting controls to their default state

Resolution: This bug is partially resolved as of AAX SDK 1.0.4. See also PTSW-158348 and PTSW-158381.

12.59.2.12 AAX-582 / PTSW-157726

AAX SDK example plug-ins' controls do not write automation properly when in 'touch' mode (frequently revert to default value while writing)

Resolution: This bug is fixed as of the 1.0.4 SDK

12.59.2.13 AAXSDK-561 / PT-232159

An AAX Hybrid DSP plug-in with 7.1.2 input and output stem formats and a 16-sample processing buffer size will throw an AAE -14382 error upon instantiation at 192kHz

Resolution: This bug is unresolved

12.59.2.14 AAXSDK-533

AAXLibrary compiles with warnings in Visual Studio 2015

Resolution: This bug is fixed as of AAX SDK 2.3.0

12.59.2.15 AAXSDK-514

Using collection-level properties leads to a leaked ACF object

Resolution: This bug is fixed as of AAX SDK 2.3.0

12.59.2.16 AAXSDK-321

Demo Delay (mono) DSP / Demo Gain (Cocoa UI) (mono) DSP can't be instantiated

Resolution: This bug is fixed as of AAX SDK 2.2.0

12.59.2.17 AAXSDK-271

DemoMIDI_Sampler: No audio on right channel (multi-mono)

Resolution: This bug is fixed as of AAX SDK 2.2.0

Discussion: DemoMIDI_Sampler and DemoMIDI_Synth are now restricted to not use multi-mono, via the [AAX_eProperty_Constraint_MultiMonoSupport](#) property.

In Pro Tools, when a track's MIDI destination is set to "none" and a new plug-in that includes a MIDI input node (e.g. any Instrument plug-in) is instantiated, the track's MIDI destination is set to the first newly created MIDI input node.

When the track in question is greater than mono, and the Instrument plug-in is multi-mono, the default behavior is for the track's MIDI destination to be set to the first of the newly created input nodes, not to all of them simultaneously. As a result, the track's MIDI destination is set to the MIDI input node of the first (left) multi-mono instance of the plug-in.

To route MIDI to all channels of a multi-mono plug-in in Pro Tools, ctrl-select the MIDI destination and choose the additional MIDI nodes.

12.59.2.18 AAXSDK-186

C99Compatibility constructs are incorrect when used with VS2012

Resolution: This bug is fixed as of AAX SDK 2.1.1

12.59.2.19 AAXSDK-162

Misleading warning message when attempting hardware debugging using a non-local TIShell.out

The "Load ProTools Plug-in Symbols" script gives the following warning: `D:/Code_7/dev.ws.↵
backup-win7-concert/AAX/Internal/SystemSoftware/TIShell/CCS_Project/TIShell/../../../../WinBag/x64/Release/bin/TIShell.out does not exist! Please question everything.`

This error message includes a hard-coded path that is not relevant to the running system. This error is benign but it can be confusing.

Resolution: This bug will not be fixed

12.59.2.20 AAXSDK-16

AAX SDK: Win32 example plug-in GUI does not appear in Windows 8

Resolution: This bug is fixed as of AAX SDK 2.2.0

12.59.2.21 AAXSDK-14

AAX DemoGain_VST: Text box entry is not acknowledged upon click outside of window

Resolution: This bug is unresolved

12.59.2.22 AAXSDK-13 / AAX-579 / PTSW-158381

AAX SDK Win32 example plug-in does not "snap to default" on option-click

Resolution: This bug is fixed as of AAX SDK 2.2.0

12.59.2.23 AAXSDK-11 / AAX-581 / PTSW-158348

AAX SDK VSTGUI example plug-in does not respond to 'alt' or 'win' modifier keys (Windows)

Resolution: This bug is fixed as of AAX SDK 2.2.0

12.59.2.24 AAXSDK-10 / AAX-580 / PTSW-154083

AAX DemoGain_VST and DemoGain_Cocoa require initial click on GUI to take focus before editing (macOS)

Resolution: This bug is not yet resolved. For macOS, one workaround is to modify VSTGUI's cocoasupport.mm file in order to add a handler for the `acceptsFirstMouse` selector:

```
static BOOL VSTGUI_NSView_acceptsFirstMouse(  
    id self,  
    SEL _cmd)  
{  
    return YES;  
}  
// In VSTGUI_NSView_isOpaque()  
res = class_addMethod(  
    viewClass,  
    @selector(acceptsFirstMouse:),  
    IMP (VSTGUI_NSView_acceptsFirstMouse),  
    "B@:@:");
```

Special thanks to Nick Protokowicz for suggesting this workaround.

12.59.2.25 AAXSDK-6 / AAX-646

AAX SDK: Incorrect output from scatter/gather DMA example plug-in when increasing playback buffer size while audio is present (Native decks)

Resolution: This bug is unresolved

Workaround: The workaround for this issue is to not run audio through this plug-in while increasing the playback buffer size.

12.59.2.26 AAXSDK-5

[AAX_CChunkDataParser::LoadChunk](#) doesn't handle unknown chunk items well

Resolution: This bug will not be fixed

Discussion: [AAX_CChunkDataParser](#) does not store the size of each chunk item in the data stream. Therefore there is no way to determine the correct size for each data element when reading a chunk that was generated by this parser.

Workaround: If you know the correct size of each data element in a chunk when it is read by the plug-in, you can override the [AAX_CChunkDataParser](#) methods to ensure that each data element is correctly sized.

12.59.2.27 AAXSDK-2 / AAX-648

AAX SDK: Output from DMA example plug-in is one buffer early

Resolution: This bug is unresolved

12.59.3 Known Issues in Pro Tools

12.59.3.1 PT-325781

Pro Tools crashes when loading an AAX plugin compiled with Objective C++ in the project languages

Details: When trying to load an [AAX](#) plugin in Pro Tools that was compiled with OBJC and OBJCXX included in LANGUAGES, Pro Tools crashes due to memory failure

Resolution: This bug is unresolved

12.59.3.2 PT-323936

Plugin window may be clipped on Windows when using a system display scaling value other than 100%

Resolution: This bug is unresolved

12.59.3.3 PT-322604

AudioSuite plugins that use [AAX_eProperty_UsesRandomAccess](#) prevent certain processing modes

Details: When using an AudioSuite plugin that sets [AAX_eProperty_UsesRandomAccess](#) to true, changing the AudioSuite processing mode to "Create Continuous File" forces "Entire Selection" mode, and changing the processing mode to "Create Individual Files" forces "Clip by Clip" mode.

Resolution: This bug is unresolved

12.59.3.4 PT-322526

MIDI recorded via chain out from a MIDI Effect plugin with latency is not aligned correctly

Resolution: This bug is fixed as of Pro Tools 2024.6

12.59.3.5 PT-317648

Pro Tools does not support plugins that use Auxiliary Output Stems and have more standard input channels than standard output channels

Resolution: This bug will not be fixed

12.59.3.6 PT-316101

Pro Tools doesn't throw the "Move Failed Plugins" dialog when launching with unsigned plugins

Resolution: This bug is unresolved

12.59.3.7 PT-307986

Certain Avid plugins, such as SynthCELL and GrooveCELL, prevent [AAX](#) plugins from forking subprocesses

Resolution: This bug is unresolved

12.59.3.8 PT-307746

When the transport is started at the beginning of the timeline, plugins that use tick position and/or sample location from [AAX_ITransport](#) may start early and play out of time if the plugin follows other inserts that incur their own latency

Resolution: This bug is fixed as of Pro Tools PT 2024.10

Discussion: Synchronization using [AAX_ITransport::GetBarBeatPosition\(\)](#) or MIDI Beat Clock may resolve this issue.

12.59.3.9 PT-307193

Committing creates a consolidated clip when plug-in sets [AAX_eProperty_Constraint_AlwaysProcess](#) to `true`

Resolution: This bug will not be fixed

12.59.3.10 PT-306863

The [AAX_eProperty_Constraint_DoNotApplyDefaultSettings](#) property is applied when "Set Plug-in Default To" is set to "User Setting" and a preset has been "Set As User Default", but the property should be ignored in this case

Resolution: This bug is unresolved

12.59.3.11 PT-305352

Automation values change unpredictably when changing a plug-in parameter from continuous to discrete or discrete to continuous

Resolution: This bug is unresolved

12.59.3.12 PT-303482

(Important Escalation) Contextual menus and child windows miss mouse clicks in some 3P plug-ins, new to PT 2023.3

Resolution: This bug is fixed in Pro Tools 2023.6

12.59.3.13 PT-299906

AudioSuite plug-ins with sidechains crash PT (arm64 only)

Resolution: This bug is Closed as Fixed in PT 2023.R1.0

12.59.3.14 PT-297802

Integer overflow in [AAX_ITransport::GetTimelineSelectionStartPosition](#) on Windows

[AAX_ITransport::GetTimelineSelectionStartPosition\(\)](#) supplies an incorrect sample position when the value is greater than about 12.42 hours at 48kHz.

Resolution: This bug is fixed in Pro Tools 2023.9

12.59.3.15 PT-290588

AudioSuite offline processing option cannot be used after performing certain actions (it simply becomes greyed out)

Resolution: This bug is fixed as of Pro Tools 2022.12

12.59.3.16 PT-284916

Modifications to non-automatable plug-in parameters do not set the Pro Tools session dirty flag for AAX plug-ins that disable default settings via [AAX_eProperty_Constraint_DoNotApplyDefaultSettings](#)

Resolution: This bug will not be fixed

12.59.3.17 PT-282946

AAX timestamps for live plug-ins are two buffers out of sync with timestamps for non-live plug-ins

This relates to timestamps derived from [AAX_IComponentDescriptor::AddClock\(\)](#)

Resolution: This bug will not be fixed

12.59.3.18 PT-278282

Crash when running certain Accelerate.framework operations

Resolution: This bug is fixed as of Pro Tools 2021.12

12.59.3.19 PT-276280

The VoiceOver accessibility tree for JUCE based plug-in GUIs is not connected to the Pro Tools plug-in window

Resolution: This bug is unresolved

12.59.3.20 PT-274717

AudioSuite settings are saved with the system rather than with the session

Resolution: This bug is unresolved

12.59.3.21 PT-271830

Crash when re-sizing a window for a plug-in that uses JUCE with OpenGL rendering enabled

Details: When Pro Tools resizes an AAX plug-in window it can cause the the `GL rendercontext` used by JUCE to become invalidated. Plug-ins must take care not to use this object within the scope of a concurrent window re-size request via `AAX_IViewContainer::SetViewSize()` .

Resolution: Plug-in developers must synchronize access to any OpenGL objects that are not thread-safe.

12.59.3.22 PT-265539

Side chain data is not provided to AudioSuite plug-ins in handle regions

Details: AudioSuite does not process the audio in the handles outside of the clip selection

Resolution: This bug will not be fixed

12.59.3.23 PT-263909

Clipboard is pasted twice when pasting text into a JUCE plug-in text box

Resolution: This bug is fixed as of Pro Tools 2022.12

12.59.3.24 PT-263859

Committing up to an insert that is followed by a width-converting insert also commits the width-converting insert

Resolution: This bug is unresolved

12.59.3.25 PT-261394

Frame rate offsets are calculated incorrectly for plug-ins when the session is at a higher frame rate

Resolution: This bug is fixed as of Pro Tools 2020.5

12.59.3.26 PT-258560 / PT-256919

Multi-input only AudioSuite plug-ins are processed as multi-mono

Details: Plug-ins that define [AAX_eProperty_MultiInputModeOnly](#) actually get mono mode only

Resolution: This bug will not be fixed

12.59.3.27 PT-258394

JUCE [AAX](#) plug-ins which use images from `BinaryData` crash on Catalina

Resolution: This bug is unresolved

12.59.3.28 PT-257213

AAX Hybrid plug-ins produce distorted signal on tracks in DSP Mode when using the HDX Hybrid Engine or Pro Tools Carbon

Details: Because of this bug, we have disabled DSP Mode for certain AAX Hybrid plug-ins.

Resolution: The fix is pending bug review

12.59.3.29 PT-256704

Pro Tools Plug-In Folder permissions (macOS) are set to allow write access by anyone

Resolution: This bug is fixed as of Pro Tools 2019.12

12.59.3.30 PT-255800

AAX Hybrid plug-in output on HDX contains a gap which varies with host buffer size

Resolution: This bug is unresolved

12.59.3.31 PT-255408

Changing one plug-in insert results in a redundant audio buffer on a later insert in the chain

Resolution: This bug from an external report could not be verified by Avid

12.59.3.32 PT-254203

Reverb and Delay AudioSuite plug-ins cannot provide an "Analysis" button

Resolution: This bug is fixed as of Pro Tools 2024.6; the [AAX_eProperty_DisableAudioSuiteReverse](#) property is now implemented.

12.59.3.33 PT-254118 / PT-275441 / PT-279941

Dynamic Plug-In Processing doesn't work during playback

Details: Plug-ins are processed continuously during playback even when Dynamic Plug-In Processing is engaged. This issue has been fixed for certain track and plug-in types in various recent releases, but there are still common configurations for which dynamic plug-in processing does not work.

Resolution: This bug is fixed as of Pro Tools 2023.3

12.59.3.34 PT-254103

Some AAX plug-ins are displayed with incorrect colors

Details: This issue affects plug-ins that use OpenGL in Pro Tools 2019.5 and higher. It relates to a Pro Tools optimization in which color space conversion is skipped for certain parts of the application UI.

Resolution: This bug is unresolved

12.59.3.35 PT-250751

AudioSuite plug-ins do not set a [custom suffix](#) on a clip in case the selection reference is "Clips list"

Resolution: This bug is will not be fixed

12.59.3.36 PT-249791

AudioSuite: [Custom clip name](#) is not applied to generated audio file on disk

Resolution: This bug will not be fixed

12.59.3.37 PT-249790

AudioSuite plug-ins cannot set a [custom clip name suffix](#)

Resolution: This bug is fixed as of Pro Tools 2019.10

12.59.3.38 PT-248000

Plug-in partial bypass should support more than just EQ and Dynamics categories

Resolution: This enhancement is not yet supported

12.59.3.39 PT-245693 / PT-200756

Dynamic plug-in processing on HDX cuts off reverb tails

Resolution: This bug is fixed as of Pro Tools 2024.6

12.59.3.40 PT-243211

Crash when closing a plug-in window unless the plug-in leaks its [AAX_IViewContainer](#) reference counts

Resolution: This bug is fixed as of Pro Tools 2019.5

12.59.3.41 PT-237857

Custom EQ curve display ranges are not supported

Resolution: This enhancement is not yet supported

12.59.3.42 PT-236755

Up-mixing plug-ins with AOS drop output channels on HDX

Resolution: This bug is fixed as of Pro Tools 2018.7

12.59.3.43 PT-235831

The plug-in frame overlaps the plug-in window header if the plug-in GUI is taller than the screen height less the plug-in window header height.

Resolution: This bug is unresolved

Workaround: Avoid resizing the plug-in GUI to a height greater than the display height.

12.59.3.44 PT-235333

Dynamic plug-in processing incorrectly shuts off processing in mixed multi-mono/multichannel chains that should force processing

This bug can occur if a multi-mono plug-in preceeds a multichannel plug-in which sets [AAX_eProperty_Constraint_AlwaysProcess](#)

Resolution: This bug will not be fixed

12.59.3.45 PT-234681

AAX plug-in parameter handling may cause audio glitches on Windows for plug-ins with very long [GenerateCoefficients\(\)](#) execution time

Resolution: This bug will not be fixed

12.59.3.46 PT-233726

Unprintable characters in four-char parameter IDs may result in -9105 errors

Resolution: This bug is fixed as of Pro Tools 2018.1

12.59.3.47 PT-233176

AAX digital signature check fails on pre-Sierra systems for plug-ins signed on Sierra

Resolution: This bug has been reported to Avid but is not yet confirmed. Contact PACE Anti-Piracy, Inc. if you encounter this behavior.

12.59.3.48 PT-232678 / PT-236755

Plug-in aux outputs are silent for upmix plug-ins when using AAX Native plug-ins with the HDX playback engine

Resolution: This bug is fixed as of Pro Tools 2018.1

12.59.3.49 PT-232403

ProTools shows error if the plug-in's multi-chunk preset file contain incorrect chunk listed in the end

For example, if a new chunk type has been added to a later version of a plug-in and a preset from that version is opened in an earlier version of the plug-in.

Resolution: This bug is fixed as of Pro Tools 12.8.2

12.59.3.50 PT-232159

AAX Hybrid plug-ins with more than 16 total input channels (direct input and hybrid input) raise AAE -14382 error upon instantiation at 192 kHz sample rate

Resolution: This bug is unresolved

12.59.3.51 PT-230327

The AAX related types feature is broken ([AAX_IACFPropertyMap_V3](#) inheritance is incorrect)

Resolution: This bug was introduced in Pro Tools 12.8 and is fixed as of Pro Tools 12.8.1

12.59.3.52 PT-230290

The plug-in preset menu takes a long time to build for plug-ins with a very large number of preset .tfx files

Resolution: This bug is fixed as of Pro Tools 2018.7

12.59.3.53 PT-230288

The plug-in preset menu contains empty folders for other Effects in the same .aaxplugin bundle

Resolution: This bug is fixed as of Pro Tools 12.8.2

12.59.3.54 PT-229026

Pro Tools may crash after plug-in parameter tweaks on Windows

Resolution: This crash has not been reproduced starting with Pro Tools 2018.1

12.59.3.55 PT-227655

[AAX_ITransport::GetTimelineSelectionStartPosition\(\)](#) provides incorrect values for real-time plug-ins - value depends on transport time display selection in Pro Tools

Resolution: This bug is fixed as of Pro Tools 12.8

12.59.3.56 PT-227173

Incorrect timecode is sent to plug-ins when delay is present before the plug-in

Resolution: This bug will not be fixed

Workaround: Attach a debugger after opening a session in Pro Tools. After the first session open, subsequent session open actions will not result in a crash.

12.59.3.57 PT-226959

A crash may occur when a multi-mono plug-in uses an incrementally buffered [AAX](#) port

Resolution: This bug will not be fixed

12.59.3.58 PT-226559

Transport location provided to plug-ins is incorrect during half-speed playback

Resolution: This bug will not be fixed

12.59.3.59 PT-225763

Incorrect AudioSuite processing modes are available for multichannel random access plug-ins

Resolution: This bug will not be fixed

12.59.3.60 PT-225637

Plug-in automation can receive incorrect values when playback is initiated with tracks in auto write mode immediately after a session is loaded

Resolution: This bug is unresolved

12.59.3.61 PT-223581

Pro Tools removes plug-ins from the insert menu if unsupported stem formats are detected

Resolution: This bug is fixed as of Pro Tools 12.8

This bug is also now fixed in earlier versions of Pro Tools via a workaround which is now built into the AAX SDK during Describe. See [AAX_VPropertyMap::AddProperty\(\)](#)

12.59.3.62 PT-218545

There is no way for AAX plug-ins to opt out of the default settings chunk sequence during plug-in instantiation

Resolution: This enhancement will be supported in a Pro Tools 2019 release; see [AAX_eProperty_Constraint_DoNotApplyDefaultSettings](#)

12.59.3.63 PT-218486

Removing all automation parameters doesn't stop control from jumping back on first playback

Resolution: This bug will not be fixed

12.59.3.64 PT-210904 / VSW-14216

HDX errors can occur due to over-allocation of certain plug-ins

This bug applies specifically to AAX DSP plug-ins which register the same DLL and algorithm entry point name for multiple modules with different [AAX_eProperty_TI_MaxInstancesPerChip](#) requirements.

In VENUE, this bug causes a 'No Information Available' error dialog on a single DSP chip

Resolution: This bug is fixed as of Pro Tools 12.5 and VENUE 5.1

12.59.3.65 PT-206995

AOS is not cleaned up in [AAX_IComponentDescriptor::Clear](#)

Resolution: This bug will not be fixed

12.59.3.66 PT-206541

AAX automation playback is late and non-deterministic

Resolution: This bug is fixed as of Pro Tools 2024.10

12.59.3.67 PT-206161

HDX: AAX packets are not delivered to ports 16 or 24 (zero-indexed) when `PostPacket()` is called outside of `GenerateCoefficients()`

Workaround: Make all calls to `AAX_IController::PostPacket()` within the scope of `AAX_IEffectParameters::GenerateCoefficients()`

Resolution: This bug will not be fixed

12.59.3.68 PT-205610

AAX Hybrid: transport location and clock methods do not provide correct values when called from the Hybrid render callback

Resolution: This bug is fixed as of Pro Tools 12.4

12.59.3.69 PT-203420

`TestGetCurveData` DigiOption results in incorrect plug-in view offset for plug-ins with MIDI

Resolution: This bug will not be fixed

Workaround: Temporarily disable MIDI in your plug-in while developing or debugging the plug-in's curve data, then re-enable MIDI once you have finished using the `TestGetCurveData` DigiOption.

12.59.3.70 PT-202345

Pro Tools may incorrectly identify plug-ins when a single plug-in uses identical plug-in IDs across different Effects

Resolution: This bug is fixed as of Pro Tools 12.2

12.59.3.71 PTSW-200437 / PTSW-197598

Plug-Ins that use the "Related Types" feature cannot relate to plug-ins with a different ProductID

Resolution: This bug is fixed as of Pro Tools 11.3.2 and Pro Tools 10.3.11

12.59.3.72 PTSW-197651 / PT-218405

In some cases AAX plug-ins do not show the correct Control Name Variation and just read out the automation name

Resolution: This bug is will not be fixed

12.59.3.73 PTSW-197601 / PT-218459

Control surfaces can send illegal parameter values to plug-ins

Resolution: This bug will not be fixed

12.59.3.74 PTSW-197593 / PT-218480

HDX: A chip may be full with just a small percent of the System Usage meter filled

Resolution: This bug will not be fixed

12.59.3.75 PTSW-197540

AudioSuite: Analyze mode is not working properly when processing method is set to "clip-by-clip"

Resolution: This bug is fixed as of Pro Tools 11.3.1

12.59.3.76 PTSW-197472

Dynamic Plug-In Processing is unnecessarily disabled for plug-ins in the "Other" category

Resolution: This bug is fixed as of Pro Tools 12

12.59.3.77 PTSW-197471

AudioSuite only analyzes the first clip in "clip list" mode

Resolution: This bug is fixed as of Pro Tools 12

12.59.3.78 PTSW-197468 / PT-218460

Pro Tools may incorrectly change a plug-in instance when another instance is edited

Resolution: This bug is fixed in Pro Tools 2018.1

Discussion: This issue only happens when the two plug-in types use the same Manufacturer and Plug-In IDs and use Product IDs with unprintable chars when interpreted as four-char values.

We strongly recommend that you select Product IDs in the printable ASCII four-char range.

12.59.3.79 PTSW-197431 / PT-218414

Pro Tools 10: Plug-in Side-chain input is silent when the plug-in supports Aux Outputs

Resolution: This bug will not be fixed

12.59.3.80 PTSW-197075

Plug-in preset files for some plug-ins are not cross-compatible between Mac and Windows

Resolution: This bug is fixed as of Pro Tools 11.2 and Pro Tools 10.3.10

12.59.3.81 PTSW-196772 / PT-218423

A [View Size Changed](#) notification is not sent when connecting/disconnecting a display

Resolution: This bug will not be fixed

12.59.3.82 PTSW-196604

Cannot adjust plug-in parameters with large numbers of steps using control surface (Artist Series)

Resolution: This bug is fixed as of Pro Tools 12.4

12.59.3.83 PTSW-196428 / PT-218488

AudioSuite preview allows processing with incorrect number of channels

Resolution: This bug will not be fixed

12.59.3.84 PTSW-195316 / PT-218485

EQ/Dyn graphs on EUCON surfaces are not always updated for plug-ins with many EQ/Dyn parameters

Resolution: This bug will not be fixed

Discussion: Currently, EUCON surfaces only update a plug-in's EQ/Dyn curve plots when an update occurs to one of the parameters which is mapped to the plug-in's "center section" EQ/Dyn page tables. Other parameters will not trigger an update to the plug-in's EQ/Dyn curve plot.

12.59.3.85 PTSW-195257

Calling [AAX_ICollection::AddEffect\(\)](#) multiple times using the same `iEffectID` only returns an error if called with the same [effect descriptor](#), but this is always illegal

Resolution: This bug is fixed as of Pro Tools 12

Discussion: Calling [AAX_ICollection::AddEffect\(\)](#) using the same ID will now return an error in all cases

12.59.3.86 PTSW-195256 / PT-218429

Plug-in is not notified of preset load when loading factory default presets

Resolution: This bug will not be fixed

12.59.3.87 PTSW-195209 / PT-218474

[AAX_IViewContainer::HandleParameterMouseUp\(\)](#) returns [AAX_ERROR_UNIMPLEMENTED](#) when using control-command-option-click on a plug-in GUI control

Resolution: This bug will not be fixed

Discussion: See the discussion of this bug in the [AAX_IViewContainer](#) documentation.

12.59.3.88 PTSW-195113

Automation problems with plug-in parameter names > 31 characters

Resolution: This bug is fixed as of Pro Tools 11.2.1

Discussion: This bug was introduced in Pro Tools 11.1

12.59.3.89 PTSW-194698 / PT-218478

Very hard to edit plug-in parameters with many steps using a control surface rotary encoder

Resolution: This bug will not be fixed

12.59.3.90 PTSW-194231 / PT-218434

When the output on a track is set to "no output" then no audio is sent to Auxiliary Outputs of the plug-ins on the track

Resolution: This bug is unresolved

Workaround: Users can work around this issue by ensuring that a track output is always assigned for tracks with plug-ins that generate Auxiliary Output channels. For example, the user may pull down the track's output gain fader, enable MUTE, or select an unused output channel or bus.

12.59.3.91 PTSW-193646

AudioSuite plug-ins are not able to partially re-name clips

Resolution: This bug is fixed as of Pro Tools 12

12.59.3.92 PTSW-193400

AOS plug-ins become active when moving an inactive plug-in to another insert

Resolution: This bug is fixed as of Pro Tools 11.2

12.59.3.93 PTSW-193345

AudioSuite processing notifications are not sent at the start and end of a processing event

Resolution: This bug is fixed as of Pro Tools 12

Discussion: Two new notifications were added to provide this behavior. The existing AudioSuite notifications retain their behavior: they are sent before and after each processing pass, i.e. at the beginning and end of each audio channel that is processed, even if the current selection includes multiple channels. For more information, see [AAX_EProcessingState](#)

12.59.3.94 PTSW-193339

Pro Tools does not update plug-in settings when a new setting's name matches an old setting and the modification date is later

Resolution: This bug will not be fixed

Workaround: To update the settings that are bundled with a plug-in, the plug-in's installer should search for and remove any deprecated settings files on the system.

12.59.3.95 PTSW-193051

Using Aux Output Stems on DSP plug-ins causes them to crash

Resolution: This bug is fixed as of Pro Tools 11.2

Discussion: This bug was introduced in Pro Tools 11.1

12.59.3.96 PTSW-192863 / PT-218498

Plug-in side chain input is not properly delay compensated: aligned with output instead of input, no individual tap per insert

Resolution: This bug is fixed as of Pro Tools 2021.6

12.59.3.97 PTSW-192755

Key focus is not returned to a plug-in after it launches a dialog

Resolution: This bug will not be fixed (design limitation)

12.59.3.98 PTSW-192720 / PT-218467

External source (SideChain) key input is not reported to DSP Dynamics plug-ins after HDX re-shuffle, hence no Gain Reduction occurs.

Resolution: This bug is unresolved

12.59.3.99 PTSW-192635

Implement Manufacturer ID byteswap

Resolution: This bug is fixed as of Pro Tools 11.2 and Pro Tools 10.3.10

12.59.3.100 PTSW-192456 / PT-218490

Plug-in settings chunks with incorrect fSize result in junk data

Resolution: This bug will not be fixed

12.59.3.101 PTSW-192251 / PT-218394

EUCON surface cells sometimes behave inconsistently when discrete plug-in parameters are mapped to rotary encoders

Resolution: This bug will not be fixed

12.59.3.102 PTSW-192086 / PT-218465

AudioSuite AAX: Pro Tools performs multiple unnecessary render passes when rendering in multi-input mode

Resolution: This bug will not be fixed

12.59.3.103 PTSW-191875

Pro Tools uses a hard-coded version string when publishing its version to AAX plug-ins ([AAX_IController::GetHostName](#))

Resolution: This bug is fixed as of Pro Tools 12.4

12.59.3.104 PTSW-191446 / PT-218600

Global symbols due to statically linked boost libs in Pro Tools components may conflict with plug-in components that use boost

Resolution: This bug is unresolved

12.59.3.105 PTSW-191317 / PT-218425

The Pro Tools meter decay setting is not applied to plug-in meters

Resolution: This bug will not be fixed

12.59.3.106 PTSW-191139

Plug-ins do not receive parameter touch state when automation-enabled in Pro Tools 11.1

Resolution: This bug is fixed as of Pro Tools 11.1.2

12.59.3.107 PTSW-190722

Some plug-in state changes do not trigger the Pro Tools session "dirty" flag

Resolution: This bug is fixed as of Pro Tool 11.1.2 and and Pro Tools 10.3.9

12.59.3.108 PTSW-190719

Unexpected behavior for plug-in auxiliary output channels > 128

Resolution: This bug will be fixed as of Pro Tools 11.1.3

12.59.3.109 PTSW-190340

In some cases AAX plug-ins do not show the correct Control Name Variation on control surfaces

This bug can occur when a page table references parameters by ID and some parameters' ID strings are exactly as long as the control surface's display. In this scenario, the control surface will display the parameter's ID string rather than a Control Name Variations (abbreviation) string of equivalent length.

Resolution: This bug is fixed as of Pro Tools 12

12.59.3.110 PTSW-189928 / PT-218456

Failure to load AAX plug-ins with spaces in DLL filename

Resolution: This bug will not be fixed

12.59.3.111 PTSW-189738 / PT-218494

AAX: [AAX_IController::PostPacket\(\)](#) doesn't return any error if you attempt to post to a private data field

Resolution: This bug will not be fixed

12.59.3.112 PTSW-189725 / PT-218397

Auto-generated AudioSuite plug-in GUIs are non-functional for the first plug-in loaded into the window

This bug applies to AudioSuite plug-ins wich use the [AAX_eProperty_UsesClientGUI](#) property. This bug is present in all Pro Tools versions which support AAX.

Workaround: This bug only applies when an AudioSuite window is first created. To resolve the issue, toggle an open AudioSuite window between different plug-ins. After toggling to another plug-in and back to the original plug-in, the auto-generated GUI will again be functional.

Resolution: This bug will not be fixed

12.59.3.113 PTSW-189439 / PT-218427

Attempts to set signal latency by non-linear AudioSuite plug-ins should fail, but do not

Resolution: This bug will not be fixed

12.59.3.114 PTSW-189279

An AudioSuite plug-in ID may be incorrectly used as a related type, preventing type-swapping

Resolution: This bug is fixed as of Pro Tools 11.2 and Pro Tools 10.3.10.

12.59.3.115 PTSW-188836 / PT-218428

[AAX_CMidiPacket::mIsImmediate](#) field is not getting set for real-time MIDI messages

Resolution: This bug will not be fixed

12.59.3.116 PTSW-188830

AudioSuite: [PreRender\(\)](#) is not called before each preview pass

Resolution: This bug is fixed as of Pro Tools 11.1

12.59.3.117 PTSW-188653 / PT-218451

Plug-ins are not unloaded and cannot be swapped when EnablePlugInHotSwap option is enabled

Resolution: This bug will not be fixed

The workaround for this issue is to re-launch Pro Tools after installing a new build of the plug-in

12.59.3.118 PTSW-188161

It is not possible to launch some Pro Tools 11 and later development builds from a debugger

Resolution: This bug is unresolved. It applies to all Pro Tools 11 and higher development builds on Windows and to some development builds on Mac.

Workaround Use the PauseDuringLaunchToAttachDebugger [DigiOption](#) to attach the debugger at a safe point in the Pro Tools launch process

12.59.3.119 PTSW-187670

Plug-in preset menu takes a long time to load with many presets

Resolution: This bug is fixed as of Pro Tools 11.1

12.59.3.120 PTSW-187220 / PT-218584

[AAX_ePrivateDataOptions_KeepOnReset](#) is not implemented

Resolution: This bug is unresolved

12.59.3.121 PTSW-187216 / PT-218491

Pro Tools has a problem with [AAX_IController::PostPacket\(\)](#) being called during [AAX_IEffectParameters::TimerWakeup\(\)](#)

Resolution: This bug will not be fixed

Workaround: This bug occurs only with unbuffered plug-ins' ports for coefficients, so the workaround for this issue is to use buffered ports instead.

12.59.3.122 PTSW-187159

Plug-in parameters can get stuck in touched state; touch/release tokens do not always match

One race condition that could result in this bug behavior has been addressed in Pro Tools 11.1.0. However, the bug can still occur when using EUCON control surfaces due to a mismatch in touch/release tokens sent from those surfaces.

Resolution: This bug is resolved as of Pro Tools 11.1.3. It is unresolved in Pro Tools 10

12.59.3.123 PTSW-187066 / PT-218391

[AAX_ITransport::GetCurrentNativeSampleLocation\(\)](#) returns invalid value on the start of playback

Resolution: This bug will not be fixed

12.59.3.124 PTSW-186864

Automatable parameter values may change between [Set](#) and [Update](#)

Resolution: This bug is unresolved

12.59.3.125 PTSW-186725

Related types do not work when used with [AAX_eProperty_SampleRate](#)

Resolution: This bug is fixed as of Pro Tools 11.1

12.59.3.126 PTSW-186627

AAX plug-ins whose context field IDs are not defined in Describe cause a crash in Pro Tools

Resolution: This bug is closed (unable to reproduce)

12.59.3.127 PTSW-186253

AudioSuite GUI work causes audio playback glitches and stutters

Resolution: This bug is fixed as of Pro Tools 11.2.

12.59.3.128 PTSW-186189

If an AAX plug-in does not declare all the fields in its context block, undefined behavior may occur (possibly a crash)

Resolution: This bug is fixed as of Pro Tools 10.3.8 and Pro Tools 11.0.2

12.59.3.129 PTSW-186182

On Windows, VSTGUIv4 plug-in GUIs do not receive key events (PT11 only)

Resolution: This bug is addressed with a patch to the AAX SDK's VSTGUI extension implementation as of AAX SDK 2.1.0

12.59.3.130 PTSW-185868 / PT-218439

AAX: Calls to [SetValue\(\)](#) early in plug-in life may not propagate to [UpdateParameterNormalizedValue\(\)](#)

Resolution: This bug will not be fixed

The workaround for this issue is to call SetValue redundantly until the desired value is updated.

12.59.3.131 PTSW-185867 / PT-218470

Session tempo should be available during [EffectInit\(\)](#)

Resolution: This bug will not be fixed

Workaround: The workaround for this issue is to poll the transport interface in [TimerWakeup\(\)](#) or otherwise call it after [EffectInit\(\)](#) completes.

12.59.3.132 PTSW-185866

Pro Tools does not respond to [SetParameterNormalizedValue\(\)](#) while offline bouncing

Resolution: This bug is fixed as of Pro Tools 11.1. However, note that we do not recommend implementing linked parameters using direct calls to [SetParameterNormalizedValue\(\)](#). For an explanation of the correct approach to parameter linking, see [Linked parameters](#), with examples provided in the SDK example plug-ins.

12.59.3.133 PTSW-185825 / PT-218464

Undo key events do not reach plug-ins (Windows)

Resolution: This bug will not be fixed

12.59.3.134 PTSW-185537

Use of DigiTrace results in `eTISysSwapScriptTimeout`

Resolution: This bug fixed as of Pro Tools 11.1

12.59.3.135 PTSW-185484

[AAX_TRACE_RELEASE](#) crashes at highest optimization setting in AAX DSP plug-ins

Resolution: This bug is unresolved

12.59.3.136 PTSW-185483

DigiTrace: Only one parameter can be sent per trace on HDX

Resolution: This bug is fixed as of Pro Tools 11.1

12.59.3.137 PTSW-185462

AudioSuite: Error 1224 on AudioSuite render when significantly changing the length of a clip (Windows 8)

Resolution: This bug is fixed as of Pro Tools 11.1

12.59.3.138 PTSW-185343

[AAX_ITransport::GetTimeCodeInfo](#) returns invalid values for AAX Instruments

Resolution: This bug is fixed as of Pro Tools 10.3.7 and Pro Tools 11.0.2

12.59.3.139 PTSW-185341

Related types come up as inactive when going from HDX > Native

Resolution: This bug is fixed as of Pro Tools 11.1

12.59.3.140 PTSW-184777 / PT-218483

AAX plug-in meters are not cleared during silence

This bug is new to Pro Tools 11. It does not occur in Pro Tools 10.

Resolution: This bug will not be fixed

12.59.3.141 PTSW-184770

AAX Hybrid plug-ins cannot be opened as AudioSuite (AAE -7103 error)

Resolution: This bug is fixed as of Pro Tools 11.1

12.59.3.142 PTSW-184682

Incorrect audio buffer length provided when a native plug-in (erroneously) registers [AAX_eProperty_AudioBufferLength](#)

Resolution: Since this is an unsupported plug-in configuration this bug will not be fixed

12.59.3.143 PTSW-184642 / PT-218627

AudioSuite "progress" dialog re-naming is not supported by AAX (it was supported in Pro Tools 9 and earlier)

Resolution: This feature is not implemented

12.59.3.144 PTSW-184619 / PT-218473 / AAX-600

AAX MIDI plug-ins' MIDI channels are not uniquely labeled

Resolution: This bug will not be fixed

12.59.3.145 PTSW-184541

Native engine strides by 2048 samples at 96kHz (expect ≤ 1024)

Resolution: This bug fixed as of Pro Tools 11.0.1

12.59.3.146 PTSW-183902 / PT-218479

[AAX_IHostProcessorDelegate::GetAudio\(\)](#) responds to invalid iLocation as if everything succeeded

Resolution: This bug will not be fixed

12.59.3.147 PTSW-183848 / PT-218390

[AAX_IHostProcessorDelegate::GetAudio\(\)](#) ignores input audio buffer parameter

Resolution: This bug will not be fixed

The workaround for this issue is to make sure that HostProcessor plug-ins only request valid audio - do the boundary-condition checking inside the plug-in.

12.59.3.148 PTSW-183841

Plug-ins defining [AAX_eProperty_RequestsAllTrackData](#) quit when processing a timeline region with no audio

Resolution: This bug is fixed as of Pro Tools 11.0.2

12.59.3.149 PTSW-183731

Failures returned by 3P AAX-AS PIs in Pre- Analyze/Render are not used by the host

Resolution: This bug is fixed as of Pro Tools 11.1

12.59.3.150 PTSW-183708

AudioSuite: plug-in parameters are not changed upon 1st click after you click Bypass. [Win]

Resolution: This was found to be an issue in certain plug-ins with JUCE-based GUI implementations. In JUCE, the real-time variants of the of the modifiers key getter method can cause seemingly unrelated problems with the responsiveness of the GUI. In this instance, the symptom was that plug-in parameters would not be changed on the first click inside the GUI window.

The workaround for this issue, and for other unusual GUI behavior in these plug-ins, is to always use `juce::ModifierKeys::getCurrentModifiers()`; do not use `juce::ModifierKeys::getCurrentModifiersRealtime()`.

12.59.3.151 PTSW-168222

Sample rate specific plug-ins cause Pro Tools to throw a misleading error message when opened in non-supported sample rate sessions

Resolution: This bug is fixed as of Pro Tools 11.1

12.59.3.152 PTSW-165992

Make automation link by Parameter ID instead of Parameter Name. Fall-back to Parameter Name if no match

Resolution: This behavior is supported starting in Pro Tools 11.1

12.59.3.153 PTSW-163739

AudioSuite works incorrectly in Clip List mode.

Resolution: This bug is fixed as of Pro Tools 12

12.59.3.154 PTSW-161674

Stereo instrument plug-ins: "MIDI Node" field in plug-in window header disappears when insert is dragged to a new slot

Resolution: This bug is unresolved in Pro Tools and will not be fixed for the foreseeable future.

12.59.3.155 PTSW-160778

After making a Preview pass, AudioSuite plug-ins no longer make calls to InitOutputBounds()

Resolution: This bug is fixed as of Pro Tools 10.2.1

12.59.3.156 PTSW-160620

AAX plug-ins receive meaningless Clock data on Native decks, and less-than-ideal data on DSP decks

Resolution: This bug is fixed as of Pro Tools 10.2

12.59.3.157 PTSW-159702

AAX VI Issue - All AAX VIs do not have MIDI Nodes

Resolution: This bug is fixed as of Pro Tools 10.2

12.59.3.158 PTSW-159700

AAX VI Issue - Instrument Tracks do not automatically map to the AAX VI that is instantiated on them

Resolution: This bug is fixed as of Pro Tools 10.2

12.59.3.159 PTSW-159524

Incorrect error message when power is not connected to HDX card (EDIT: occurs with pre-A1 HDX prototypes only)

Resolution: This bug will not be fixed

12.59.3.160 PTSW-158119

Some plug-ins' DSP Instance counts are much lower in Pro Tools 10.2 than in Pro Tools 10.1

Resolution: This issue affects plug-ins that employ more than one buffered data port and that support many instances per DSP chip on HDX. As of Pro Tools 10.2, there is a limit of 164 buffered data ports per DSP (this is equal to the total I/O limit per DSP.)

To work around this issue, use as few data ports in your plug-in's algorithm context as possible. Note that DMA transfers on HDX occur in 128-byte chunks, so packet sizes below 128 bytes do not increase transfer efficiency on HDX.

12.59.3.161 PTSW-157745

Plug-ins write automation with pairs of updates, causing undesired "stepping" in recorded automation

Resolution: This bug is fixed as of Pro Tools 10.2 and 10.1.1

12.59.3.162 PTSW-157518

Poor plug-in performance with multiple processors selected; plug-ins are not consistently assigned to the same worker/thread by DAE, leading to cache thrashing.

Resolution: This bug is fixed as of the audio engine changes in Pro Tools 11

12.59.3.163 PTSW-157012

AAX DSP plug-ins with same DLL name are not properly labeled in the System Usage window

Resolution: This bug is fixed as of Pro Tools 10.2

12.59.3.164 PTSW-156310

Mouse cursor does not reliably update when positioned over plug-ins. Instead the mouse cursor shows the current Edit Tool.

Resolution: This bug is fixed as of Pro Tools 10.2

12.59.3.165 PTSW-156286

GUI elements fill window in some 3P AAX plug-ins GUIs on Windows

Resolution: This bug is fixed as of Pro Tools 10.2

12.59.3.166 PTSW-156216

`pluginGestalt_SupportsControlChangesInThread` is not properly implemented for AAX plug-ins

Resolution: Parameter updates are handled by a non-main thread for all AAX plug-ins as of Pro Tools 10.1

12.59.3.167 PTSW-156195

Silent failure when plug-ins attempt to register components with different platform support

Resolution: This bug is not yet resolved. This is an expected constraint, but the silent failure is unexpected

12.59.3.168 PTSW-156035

`GetCurrentTDMSampleLocation()` returns the wrong value.

Resolution: This bug is fixed as of Pro Tools 10.2

12.59.3.169 PTSW-155300 / PT-218458

When an AudioSuite plug-in modifies the output audio length, the audio is not positioned at the correct location

This bug is due to AudioSuite handles processing. A plug-in that modifies the output audio length may move audio from the handle region into the visible clip region, which is unexpected behavior from the user's perspective.

Resolution: This bug will not be fixed

The workaround is for plug-ins that experience this issue to disable AudioSuite handles, thereby only processing the audio that the user sees on the timeline.

12.59.3.170 PTSW-155177

`eFicGestalt_GetASPreHandleLength` and `eFicGestalt_GetASPostHandleLength` return the wrong handle length values upon a call to `AnalyzeAudio` with 'WHOLE FILE' mode selected

Resolution: This bug is fixed as of Pro Tools 10.2

12.59.3.171 PTSW-154361

Highlight info sent to plug-ins before GUI is created.

Resolution: This bug is fixed as of Pro Tools 10.0

12.59.3.172 PTSW-153140

Crash on Pro Tools quit when plug-in GUI is open (macOS)

Resolution: This bug is unresolved in Pro Tools and will not be fixed for the foreseeable future. Plug-in workarounds are demonstrated in the `DemoGain_GUIExtensions` example plug-ins:

a) Separating all Obj-C elements into a separate bundle that is loaded manually by the main plug-in bundle (see `DemoGain_Cocoa`) b) Applying an `NSAutoreleasePool` to the AAX GUI object destructors (see `DemoGain_VST` and `DemoGain_JUCE`)

12.59.3.173 PTSW-150047

AAX MIDI plug-ins do not get correct MIDI routing on Instrument tracks

Resolution: This bug is fixed as of Pro Tools 10.2

12.59.3.174 PTSW-149880

Configurations with duplicate PlugInID properties are silently hidden with no error

Resolution: As of Pro Tools 10.2, duplicate PlugInID properties will trigger the following DigiTrace log:

DTF_AAXHOST DTP_NORMAL

"AAXH ERROR: Attempted to add new configuration with duplicate ID: %x" existingID

12.59.3.175 PTSW-149819

MIDI packet alignment is not identical between DAE and AAX

This is a known bug in Pro Tools 10.0. This bug results in corrupted MIDI stream data to AAX plug-ins.

Resolution: This bug is fixed as of Pro Tools 10.0.1

12.59.3.176 PTSW-135536 / PT-218412

Erroneous transport location information provided to plug-ins after playback (new to PT9)

Resolution: This bug will not be fixed

12.59.3.177 PTSW-3020 / PT-218463

Groups do not follow changes to "Inserts" Globals group settings

Resolution: This bug will not be fixed

The workaround for this issue is to modify the group's settings to de-select "follow globals", then re-modify the group's settings to select "follow globals". This will apply the current Globals settings as well as any future changes to the Globals without need for additional workarounds.

12.59.3.178 AAX-686

Re-add support for AudioSuite "progress" dialog re-naming (was supported in PT 9 and earlier) (see PTSW-159768)

Resolution: This bug is unresolved

12.59.3.179 AAX-583 / PTSW-157743

AAX SDK Win32 GUI example plug-in does not draw correctly

Resolution: Duplicate of PTSW-156286 (see above.) Resolved as of Pro Tools 10.2

12.59.4 Known Issues in Venue Live Sound Systems

12.59.4.1 VSW-13857

Plug-in installers cannot associate a single thumbnail image with multiple variants

Resolution: This capability is supported starting in Venue 6.3

Prior to this change a plug-in thumbnail filename always contained 24 hexadecimal digits:

1. The first 8 digits refer to the plug-in's Manufacturer ID
2. The middle 8 digits refer to the plug-in's Product ID
3. The last 8 digits refer to the plug-in's "plug-in ID", which is usually a plug-in variant (Mono, Stereo, etc.).

With this change, Venue supports using a generic thumbnail file for all variants, thus having only the first 16 identifying digits.

12.59.4.2 VSW-13292

Plug-in parameters are not mapped to S6L if custom page tables are not provided

Details: S6L does not fall back to using the 'PgTL' page table type if a plug-in does not provide any parameter mapping for the primary 'Av46' or secondary 'FrTL' page tables. If a plug-in does not provide any of these page tables then its parameters will not display on the console.

Resolution: This bug will not be fixed

12.59.4.3 Other Known Issues

- [PT-210904 / VSW-14216](#)

12.59.5 Known Issues in Media Composer

12.59.5.1 MCDEV-2904

Optional analysis is not applied to every channel in a multi-channel selection

Resolution: This bug is fixed as of Media Composer 8.4

Discussion: When an optional analysis pass is triggered in Media Composer, only the channel that is currently represented in the AudioSuite Dialog will be analyzed. Other channels in a multi-channel selection will not be analyzed.

This issue is fixed in Media Composer 8.4; now the following behavior will occur for AudioSuite plug-ins:

- If a plug-in defines only [AAX_eProperty_RequiresAnalysis](#) then an Analyze pass will be performed before Render/Preview and the "Analyze" button will be disabled
- If a plug-in defines only [AAX_eProperty_OptionalAnalysis](#) then an Analyze pass will be performed before Render/Preview as well as when the "Analyze" button is clicked, and the "Analyze" button will be enabled
- If a plug-in defines both [AAX_eProperty_RequiresAnalysis](#) and [AAX_eProperty_OptionalAnalysis](#) then an Analyze pass will be performed before Render/Preview as well as when the "Analyze" button is clicked, and the "Analyze" button will be enabled

12.59.6 Known Issues in Control Surfaces

12.59.6.1 PT-285383

EUCON surfaces display plug-in parameters incorrectly if the plug-in page table contains an Av18 layout

Resolution: This bug is fixed in 2023.3

12.59.6.2 PT-226228

On EUCON control surfaces, Dynamics curves are not displayed if a plug-in does not provide a custom curve display range

Workaround: In order for a plug-in's Dynamics curve to be displayed, the plug-in must implement [AAX_IEffectParameters::GetCurveDataDisplayRange\(\)](#) for whichever Dynamics [curve types](#) it supports

12.59.6.3 PT-226227

EUCON control surfaces do not support custom EQ curve display ranges

Resolution: This feature is not yet implemented

12.59.6.4 GWSW-16656

Avid Control does not map plug-in controls to cells that do not have an encoder assignment

Resolution: This bug is unresolved

12.59.6.5 GWSW-8470

S6: Knob velocity changes are too sensitive for plug-in parameters with a large number of steps

Resolution: Resolved as of S6 Software 2.0

12.59.6.6 GWSW-6694

S6: Plug-in parameter order is inverted when using ProControl page tables

Resolution: Resolved as of S6 Software 1.3

12.59.7 Known Issues in Other Software

12.59.7.1 XPACE-23

Performance issues on Azure VMs with some copy protected binaries

Pro Tools and Media Composer support operation in an Azure VM environment. Some early versions of Eden copy protection by PACE Anti-Piracy, Inc. does not perform well in this environment.

Resolution: This issue is resolved in PACE Eden versions 5 and later. To avoid this issue be sure to update to the latest version of your copy protection.

12.59.8 Known Issues in AAX Tools

12.59.8.1 AAXTOOL-1344

Page Table Editor recognises 7.0.6 and 7.1.6 as "unknown stem format"

Resolution: This bug is unresolved

12.59.8.2 PT-218597

The cyclesshared dish command provides an average, but should provide the worst observed case

Resolution: This bug is unresolved

12.59.8.3 Additional Information

For a list of additional known issues in AAX Tools such as Pro Tools Developer Builds, [DigiShell](#) or the [AAX Plug-In Page Table Editor](#), see the dedicated ReadMe file that is distributed with each tool. Collaboration diagram for Known Issues:

12.60 Change Log

Changes between AAX SDK versions.

Note

See [Host Support](#) for information about which host versions support various AAX features.

12.60.1 Change Log

12.60.1.1 AAX SDK 2.8.1

12.60.1.1.1 Documentation

- Updated documentation on [plugin caching](#) in the Pro Tools Guide.
- Added documentation for [Clip drag and drop](#) to the Pro Tools Guide.
- Added warnings against setting cached plugin information based on run-time checks. See [Host feature and environment checks](#) for more information.

12.60.1.1.2 Build

- Changed use of nonstandard WIN32 to `_WIN32`

12.60.1.1.3 AAX Library

- Moved functions `AAX_GetStemFormatChannelCount` , `sampleRateInMask` , `getLowestSampleRateInMask` , and `getMaskForSampleRate` from `AAX.h` to `AAX_Enums.h`.

12.60.1.2 AAX SDK 2.8.0

12.60.1.2.1 AAX Library

- Added task-related support files for `AAX_CTask` and `AAX_IHostTaskAgent`

12.60.1.2.2 License

- Added an open source option to the AAX SDK license

12.60.1.2.3 Directory Changes

- Renamed all `msvc` and `vs2017` directories to `vs`

12.60.1.2.4 Documentation

- Converted the SDK's readme file to markdown
- Documented that `AAX_eProperty_DisableAudioSuiteReverse` is now supported (starting in Pro Tools 2024.6)

12.60.1.2.5 Example plug-ins

- Added new example plug-in `DemoMIDI_Transpose`

12.60.1.2.6 Extensions

- Converted use of `juce::PlatformUtilities` to `juce::Process` when setting the module instance handle in the JUCE GUI extension to enable support for JUCE version 7 and higher

12.60.1.2.7 Resolved bugs

- Added explicit copy constructors to classes in `AAX_Exception.h` to resolve an MSVC warning

12.60.1.3 AAX SDK 2.7.0

12.60.1.3.1 AAX Library

- Enhanced `AAX_VCollection` to provide information about the host version when required during Describe. Access via `AAX_ICollection::GetHostVersion()`

12.60.1.3.2 Definitions

- Added a notification [AAX_eNotificationEvent_HostLocale](#) to allow plugins to follow the host's localization setting.
- Added new feature identifier [AAXATTR_Client_Version](#) to support [AAX_ICollection::GetHostVersion\(\)](#)

12.60.1.3.3 Documentation

- Added documentation for MIDI Effects to the [Pro Tools Guide](#)

12.60.1.3.4 Interface

- Added [AAX_IACFTransport_V5](#) that provides the ability to query key signature, typically accessed through [AAX_ITransport::GetKeySignature\(\)](#)

12.60.1.3.5 Resolved bugs

- Fixed compiler errors in [AAX_CRangeTaperDelegate](#) when instantiated for certain integer types
- Fixed a compiler error in [AAX_CParameter](#) constructors affecting certain compilers

12.60.1.3.6 specification

- Added the [Properties File](#) as an optional file in the [plugin bundle specification](#)

12.60.1.4 AAX SDK 2.6.1

12.60.1.4.1 Build

- Reduced default warning Visual Studio level to `Level4` to eliminate warnings from standard library headers

12.60.1.4.2 Definitions

- Added [AAX_eProperty_ShowInMenus](#) for plugins that should not be shown in the host's effect menus
- Added new plugin category [AAX_EPluginCategory_MIDIEffect](#) for MIDI Effect plugins

12.60.1.4.3 Example plug-ins

- The [DemoMIDI_Synth_AuxOutput](#) plugin now demonstrates stereo auxiliary outputs

12.60.1.4.4 Extensions

- Updated the JUCE GUI extension to support JUCE version 7

12.60.1.5 AAX SDK 2.6.0

12.60.1.5.1 AAX Library

- Added a implementation [AAX_CSessionDocumentClient](#) for the session document client interface
- Changed the behavior of [AAX_AggregateResult](#) to avoid crash on an exception thrown from the class destructor
- Added additional reference implementations of [AAX_IDataBuffer](#) - [AAX_CArrayDataBuffer](#) and [AAX_CArrayDataBufferOfType](#)

12.60.1.5.2 Definitions

- Added a new tracing priority, [kAAX_Trace_Priority_Critical](#)

12.60.1.5.3 Documentation

- Re-named TI Guide to [HDX DSP Guide](#)
- Clarified documentation for HDX DSP [Multi-shell packing](#)
- Added documentation regarding [Using the Pro Tools Scripting SDK with AAX](#)
- Added BNDL as a valid bundle ID in [AAX Format Specification](#)

12.60.1.5.4 Interface

- – Added [AAX_IACFTransportControl](#) and [AAX_IACFTransport_V4](#), with methods typically accessed through [AAX_ITransport](#)
- – Added [AAX_IACFSessionDocument](#), with methods typically accessed through [AAX_ISessionDocument](#), and [AAX_IACFSessionDocumentClient](#), with implementation usually performed via [AAX_CSessionDocumentClient](#)
- – Added [AAX_IACFDataBuffer](#), with methods typically accessed through [AAX_IDataBuffer](#) when the data buffer is implemented on the host

12.60.1.5.5 Resolved bugs

- Added return value checks for internal calls to [AAX_IEffectParameters::GetNumberOfParameters](#) and [AAX_IEffectParameters::GetParameterIDFromIndex](#)

12.60.1.6 AAX SDK 2.5.1

12.60.1.6.1 AAX Library

- Updated [AAX_CParameter](#) so that the host is now automatically notified whenever [AAX_CParameter::SetName\(\)](#) is used.

12.60.1.6.2 Definitions

- Added [kAAX_ParameterIdentifierMaxSize](#) to define the maximum size of a [AAX_CParamID](#) c-string.

12.60.1.6.3 Interface

- Added [Task agent interface](#) including the [AAX_ITaskAgent](#) module and the generic [AAX_IDataBuffer](#) reference counted data container.
- Added [AAX_eNotificationEvent_ParameterNameChanged](#) notification type. Users of [AAX_CParameter](#) do not need to post this notification directly.

12.60.1.6.4 Utilities

- Replaced `snprintf` and fixed size buffers with C++ idioms in the [AAX_StringUtilities.h](#) utility functions

12.60.1.7 AAX SDK 2.5.0

12.60.1.7.1 Definitions

- Added [AAX_EStemFormat](#) definitions for surround stem formats from 5.0.2 to 9.1.6 and for Ambisonics formats from fourth to seventh order

12.60.1.7.2 Example plug-ins

- DemoGain_UpMixer has been updated with all combinations of the new stem formats

12.60.1.7.3 Interface

- New interfaces:
 - [AAX_IACFViewContainer_V3](#), with methods to track mouse movement over controls, accessed through [AAX_IViewContainer](#)

See [Host Support](#) for host support information

12.60.1.8 AAX SDK 2.4.1

12.60.1.8.1 Build

- Treat Warnings As Errors is now disabled for the AAX Library Xcode project
- The AAX Library Xcode project is no longer configured to use the Legacy Build System, which is deprecated in current Xcode

12.60.1.9 AAX SDK 2.4.0

12.60.1.9.1 Build

- Compilation for arm64 is now supported
- Explicitly set macOS project architectures to `x86_64` and `arm64`
- Updated Visual Studio project format to VS2017 and resolved newly detected warnings
- Reduced Visual Studio warning level from `EnableAllWarnigs` to `Level4` for the AAXLibrary project

12.60.1.9.2 Definitions

- Added [AAX_eProperty_AlwaysBypass](#) for plug-ins that always pass the audio signal through unaltered
- Added [AAX_eProperty_ObservesTransportState](#) , [AAX_eNotificationEvent_TransportStateChanged](#) , [AAX_ETransportState](#) , [AAX_ERecordMode](#) , and [AAX_TransportStateInfo_V1](#) to provide information about the current state of the host transport
- Added [AAX_eNotificationEvent_LogState](#) as an optional logging convenience mechanism for certain plug-ins that use the Direct Data feature
- Extended [AAX_EFrameRate](#) to include additional frame rates. These additional rates can be queried using [AAX_ITransport::GetHDTIMECodeInfo\(\)](#)
- Added [AAX_ERROR_PRINT_FAILURE](#) for printing library method failures

12.60.1.9.3 Documentation

- Added the [Real-time performance](#) page and the [Plug-In Causes Audio Streaming Errors](#) troubleshooting section with overview of best practices for avoiding streaming errors and achieving good performance for audio processing on real-time threads
- Updated the [HDX DSP Guide](#) page with a "Getting Started" section and with information about Pro Tools | Carbon
- Corrected the documented range of acceptable values between [AAX_ERROR_PLUGIN_BEGIN](#) and [AAX_ERROR_PLUGIN_END](#)
- Improved Doxygen dot image resolution, now using SVG

12.60.1.9.4 Example plug-ins

- `DemoGain_UpMixer` now registers non-converting combinations

12.60.1.9.5 Interface

- New interfaces:
 - [AAX_IACFEfffectDirectData_V2](#), with methods accessed through [AAX_IEffectDirectData](#)
 - [AAX_IACFTTransport_V3](#), with methods typically accessed through [AAX_ITransport](#)

See [Host Support](#) for host support information

12.60.1.9.6 Resolved bugs

- Fixed [AAXSDK-705](#)

12.60.1.9.7 Utilities

- CreatePackage.bat now removes the read-only attribute from the .aaxplugin folder on Windows

12.60.1.10 AAX SDK 2.3.2

12.60.1.10.1 AAX Library

- Removed unnecessary `virtual` keyword usage for method overrides
- Removed unused `mClipped` member of [AAX_CEffectParameters](#)
- Convert `mViewContainer` member of [AAX_CEffectGUI](#) to a smart pointer

12.60.1.10.2 Build

- Xcode 10 and Visual Studio 2017 are now supported
- Added Xcode workspace and Visual Studio solution containing all projects in the AAX SDK for convenience
- Removed 32-bit architecture targets from all project configurations. 32-bit architectures are still supported by AAX if you choose to explicitly add them to your build project configurations.
- Updated all Xcode projects to recommended `CFBundleIdentifier` usage and build settings

12.60.1.10.3 Definitions

- Added [AAX_eProperty_Constraint_DoNotApplyDefaultSettings](#) for plug-ins which need to disable the normal default settings application procedure used by Pro Tools
- Added [AAX_eNotificationEvent_PriorSettingsInvalid](#) which may be useful for certain plug-ins when running in Venue systems
- Added [AAX_eProperty_PluginID_NoProcessing](#) for Effect types that do not process audio
- Removed [AAX_eProperty_SupportsProgressDialog](#) which is not supported in any AAX host

12.60.1.10.4 Documentation

- Fixes and improvements in the "Plug-In spec properties" section of [AAX_Properties.h](#)
- Added [Quick Start](#) and [Troubleshooting](#) documentation sections
- Added additional detail to the [Digital signature](#) section of the [Pro Tools Guide](#)

12.60.1.10.5 Example plug-ins

- Changed ID generation algorithm for [DemoGain_UpMixer](#). Older copies of this example plug-in will not be recovered in saved sessions.

12.60.1.10.6 Resolved bugs

- Fixed [AAXSDK-663](#) AAX SDK `#pragma pack` errors with XCode 10 and later

12.60.1.11 AAX SDK 2.3.1

12.60.1.11.1 AAX Library

- Enhanced support for [AAX_CheckedResult](#) - added [AAX_CAPTURE](#), [AAX_CAPTURE_MULT](#), and [AAX_AggregateResult](#) to assist with common Describe error handling scenarios
- Updated [AAX_CMonolithicParameters::StaticDescribe\(\)](#) to use [AAX_CheckedResult](#) for error checking
- Added [AAX_CStatelessParameter](#) for "momentary" parameters which do not require state, such as tap tempo buttons which can be mapped to a control surface
- Improved tolerance for unknown parameters when building or parsing plug-in settings chunk data
- Added [AAX_DEBUGASSERT](#), [AAX_STACKTRACE](#), and [AAX_TRACEORSTACKTRACE](#) to the library of tracing and assertion macros in [AAX_Assert.h](#)
- Fixed warnings which would prevent compilation in Visual Studio 2015 and Visual Studio 2017 when Treat Warnings As Errors is enabled
- Removed Visual Studio 2008, Visual Studio 2010, and Xcode 3 projects

12.60.1.11.2 Definitions

- Removed guard preventing [AAX_CPP11_SUPPORT](#) from being set for PACE Fusion compiler builds
- Deprecated [AAX_EHostMode](#) - replaced by [AAX_EHostModeBits](#)

12.60.1.11.3 Documentation

- Documentation added to [EQ and Dynamics Curve Displays](#) for the EQ Curves feature in Pro Tools 2018.1
- Added [Checking Results](#) and [Describe Validation](#) sections to [Description callback](#)
- Added [Building your plug-in installer](#) section to [Distributing Your AAX Plug-In](#), including information about bundling Track Presets with the plug-in installer

12.60.1.11.4 Example plug-ins

- Updated all example plug-ins' Describe routines to use [AAX_CheckedResult](#) for error checking
- Updated some example plug-ins' parameter registration code in [EffectInit\(\)](#) with a safer parameter creation and release style using `std::unique_ptr`
- Updated the [RectiFi](#) example plug-in to match the current shipping version of Avid's Recti-Fi plug-in
- [DemoGain_UpMixer](#) now converts arbitrarily between all stem formats, both wider and narrower
- Removed Visual Studio 2008, Visual Studio 2010, and Xcode 3 projects
- Common Xcode settings updated with "macosx10.11" base SDK and "10.9" deployment target
- Added AAX DSP for higher stem formats in [DemoGain_Multichannel](#) and [DemoGain_UpMixer](#)

12.60.1.11.5 Extensions

- Updated the VSTGUI extension and example plug-in to use VSTGUI 4.3

12.60.1.11.6 Interface

- New interfaces:
 - [AAX_IACFPageTable_V2](#), with methods accessed through [AAX_IPageTable](#)
 - [AAX_IACFHostServices_V3](#), with methods typically accessed through the macros in [AAX_Assert.h](#)

See [Host Support](#) for host support information

12.60.1.11.7 Utilities

- Added reference count tracing logic to [AAX_CACFUnknown.cpp](#), which can be toggled on using the `AAX_↔
DEBUG_ACF_REFCOUNT` macro
- Added some convenience functions to [AAX_PageTableUtilities.h](#)
- Added [getLowestSampleRateInMask\(\)](#) and [getMaskForSampleRate\(\)](#) convenience functions

12.60.1.12 AAX SDK 2.3.0

12.60.1.12.1 AAX Library

- Added [AAX_Exception.h](#) with the [AAX::Exception](#) namespace for AAX-specific exception objects and the [AAX_CheckedResult](#) class which can be used for throwing AAX exceptions when an error is encountered.
- Added a try/catch block in the library implementation of [AAXRegisterPlugin](#) such that exceptions may safely be thrown during Describe
- [AAX_ICollection](#) now provides convenience methods to access an [AAX_IDescriptionHost](#) and [IACFDefinition](#), if these interfaces are supported by the host during Describe
- [AAX_IComponentDescriptor](#) now provides the generic [AddProcessProc\(\)](#) method for specifying multiple ProcessProcs at once using a property map
- [AAX_IController](#) now provides methods for copying page table data from other effect variants or from arbitrary page table files on disk
- [AAX_IPropertyMap](#) now supports pointer-sized properties
- [AAX_IPropertyMap](#) objects can now be generated from other property map objects without requiring access to a component factory interface

12.60.1.12.2 Definitions

- Added a new stem format definition for the [7.0.2](#) format
- Removed the previous FuMa Ambisonics formats and added definitions for [second-order](#), and [third-order](#) ACN Ambisonics stems
- Added new notification types:
 - [AAX_eNotificationEvent_ParameterMappingChanged](#) (plug-in to host)
 - [AAX_eNotificationEvent_HostModeChanged](#) (host to plug-in)
- C++11 keyword compatibility macros added to [AAX.h](#)
- Removed the `AAX_AlignedDouble` definition, which was unused

12.60.1.12.3 Documentation

- New documentation:
 - [Distributing Your AAX Plug-In](#)
 - [EQ and Dynamics Curve Displays](#)
 - [Adding signposts to the DigiTrace log at run-time](#)
 - [Plug-in preset data comparison](#) for Media Composer
 - [Interactive mode](#) for DTT
 - Descriptions of [Avid Dock](#), [Avid S1](#) and [Avid Control app](#) in the [Page Table Guide](#)
- There is a new process for [requesting the digital signing toolkit](#) for digitally signing AAX plug-ins
- Added a PDF print-out of this Doxygen documentation to assist with text-based searches
- Updated the [Contacting Avid](#) section of the main page to clarify the various processes for communicating with Avid
- Updated [AAX_Errors.h](#) with a list of current internal AAX host error values, which are useful for reference when troubleshooting host errors.
- Updated the [HDX DSP Guide](#) with information about using the latest version of Code Composer Studio with this AAX SDK

12.60.1.12.4 Example plug-ins

- Base Mac OS SDK setting in the common .xcconfig files is now macosx10.9
- DemoGain_Multichannel now includes an example of gain reduction metering
- DemoGain_Multichannel now supports [7.0.2](#) and [First-order](#), [second-order](#), and [third-order](#) Ambisonics stem formats
- DemoGain_UpMixer example plug-in added to demonstrate a width-changing effect
- The DemoMIDI_NoteOn example plug-in algorithm now supports note hold

12.60.1.12.5 Interface

- New interfaces:
 - [AAX_IACFComponentDescriptor_V3](#), with methods accessed through [AAX_IComponentDescriptor](#)
 - [AAX_IACFDescriptionHost](#), with methods accessed through [AAX_IDescriptionHost](#)
 - [AAX_IACFEffEffectParameters_V4](#), with methods accessed through [AAX_IEffectParameters](#)
 - [AAX_IACFFeatureInfo](#), with methods accessed through [AAX_IFeatureInfo](#)
 - [AAX_IACFPageTable](#), with methods accessed through [AAX_IPageTable](#)
 - [AAX_IACFPageTableController](#), with methods accessed through [AAX_IController](#)
 - [AAX_IACFPropertyMap_V3](#), with methods accessed through [AAX_IPropertyMap](#)

See [Host Support](#) for host support information

- Added the concept of a host "feature" which can be queried during Describe execution using [AAX_IDescriptionHost](#) and [AAX_IFeatureInfo](#)

12.60.1.12.6 Resolved bugs

- Resolved [AAXSDK-533](#): AAXLibrary compiles with warnings in VS2015 / VS2017
- Resolved [AAXSDK-514](#): Using collection-level properties leads to a leaked ACF object
- Fixed bugs with taper delegates when the minimum and maximum values are equal
- Some unnecessary headers removed or converted to forward declarations

12.60.1.13 AAX SDK 2.2.2

12.60.1.13.1 AAX Library

- Added new methods to [AAX_IParameter](#) for easier conversion between logical and normalized parameter values
- Re-named `AAX_CParameterManager::ControlIndexFromID()` to [AAX_CParameterManager::GetParameterIndex](#)
- Added AAX Library project for Visual Studio 2013
- Added warning exclusion for C4738 to 32-bit Release configuration of the AAX Library project on Windows to fix a treat-warnings-as-errors build failure that can occur in this configuration when linking statically to the MSVC run-time libraries

12.60.1.13.2 Definitions

- Added new stem format selectors for the following stem formats:
 - The [7.1.2](#) speaker configuration
 - [First-order](#), [second-order](#), and [third-order](#) Ambisonics
- Added a new notification type for information regarding the host's delay compensation state↔: [AAX_eNotificationEvent_DelayCompensationState](#)
- Added a new [input data port type](#) property for ports which request [incrementally-buffered](#) packet delivery
- Added a property to allow different AAX DSP plug-in types to share the same DSP chip even if [AAX_eProperty_TI_MaxInstancesPerChip](#) is declared: [AAX_eProperty_TI_ForceAllowChipSharing](#)

12.60.1.13.3 Documentation

- Added specific details about display hardware to the [VENUE Guide](#)

12.60.1.13.4 Example plug-ins

- Added the [DemoGain_Multichannel](#) example plug-in
- Updated page tables of all example plug-ins
- Example plug-in Xcode projects now use C++11 and libc++ by default
- Updated [DemoDelay_Hybrid](#) to fix problems with instantiation in [DSH](#) and other test hosts
- Removed multi-mono support from [DemoMIDI_Synth](#) to provide a better example of a standard VI configuration
- Updated [Recti-Fi](#) example plug-in IDs so that they will not collide with the shipping version of Recti-Fi

12.60.1.13.5 Extensions

- Updated `AAX_JuceContentView::mouseMove()` for compatibility with Juce version 4 and higher
- Updated `AAX_CEffectGUI_VST` for compatibility with 32-bit plug-ins when used with VSTGUI 4.2

12.60.1.13.6 Interface

- ACF interface files updated to a more recent version of the ACF SDK

12.60.1.13.7 Resolved bugs

- [AAX_CEffectParameters::UpdateParameterNormalizedValue\(\)](#) now increments the effect change counter only when the parameter's value actually changes

12.60.1.13.8 Utilities

- New utility functions: [AAX::AsStringStemFormat\(\)](#), [AAX::AsStringStemChannel\(\)](#)
- Added [AAX_SCOPE_COMPUTE_DENORMALS\(\)](#) for forcing denormal float values to be calculated within a scope, rather than being treated as zero (currently implemented for Mac only)

12.60.1.14 AAX SDK 2.2.1

12.60.1.14.1 Interface

- New interfaces:
 - [AAX_IACFController_V3](#)

12.60.1.14.2 Documentation

- Added the [VENUE Guide](#) page
- Updated the [Page Table Guide](#)
 - Updated VENUE information: Added information about [VENUE | S6L](#) and S3L and removed information about VENUE systems which do not support AAX plug-ins
 - Added information for S6, including details about the 'Av46' page table type and a new section on [Center Section Parameter Mapping in S6 Expand Mode](#)
- Updated the documentation for [Plug-in type conversion](#), including a new section describing [Type deprecation](#)
- Fixed image display problems on the [DSH Guide](#) page
- Added pre-built HDX DLL files to the SDK for all example plug-ins which support AAX DSP

Note

The example plug-ins' Visual Studio projects now include a `PostBuildEvent` command which will copy the plug-in's HDX DLL from the project's `TI/bin/Release` folder to the built .aaxplugin's `Resources` folder.

- Additional minor example plug-in fixes
 - Removed unnecessary build phases and framework dependencies from the plug-ins' Xcode projects
 - Removed "%AAX" from the example plug-ins' display names
 - Changed the guard for AAX DSP cycle count declarations to check for the definition of the `AAX_↔ TI_BINARY_IN_DEVELOPMENT` preprocessor symbol before adding cycle counts to the plug-in's description
 - Added "example" to the names of all example plug-ins

12.60.1.14.3 AAX Library

- Extended [AAX_CParameter::GetValueAsString\(\)](#) and [AAX_CParameter::SetValueWithString\(\)](#) with support for all value types
- Fixed the specialization of [AAX_CPacket::GetPtr\(\)](#) for `void*` so that it is called when the `void*` version of the function template is requested

12.60.1.14.4 Definitions

- Added [AAX_ePlugInStrings_ClipNameSuffix](#)
- Added a definition of the `TI_VERSION` preprocessor macro for the TI DSP compiler in [AAX.h](#)

12.60.1.15 AAX SDK 2.2.0

12.60.1.15.1 Interface

- New interfaces:
 - [AAX_IACFEfffectParameters_V3](#)
 - [AAX_IACFHostProcessor_V2](#)
 - [AAX_IACFHostProcessorDelegate_V3](#)
 - [AAX_IACFHostServices_V2](#)
 - [AAX_IACFViewContainer_V2](#)

12.60.1.15.2 Directory changes

- Moved common processing classes for the SDK example plug-ins to `ExamplePlugIns/Common/ProcessingClasses`
- Moved MIDI logging utilities to the Extensions folder
- Moved [AAX_CMonolithicParameters](#) to the Extensions folder and removed it from the AAX Library

12.60.1.15.3 Extensions

- Changed VST project to use the newest version of VSTGUI sources - VSTGUI 4.2
- Created Visual Studio 2012 projects for GUI Extensions
- Fixed [AAX_CMonolithicParameters](#) so that it correctly supports [AAX_eConstraintLocationMask_DataModel](#)

Note

This value is **required** for all plug-ins that share memory between their data model and algorithm call-back

- Updated [AAX_CMonolithicParameters](#) to include parameter value synchronization
- Updated [AAX_CMonolithicParameters](#) to support Hybrid and include a state counter field

12.60.1.15.4 Definitions

- Changed name of `AAX_eProperty_StoreXMLPageTablesByType` to [AAX_eProperty_StoreXMLPageTablesByEffect](#) to best reflect the actual behavior of this property
- Replaced [AAX_EPlugInCategory_Effect](#) category (erroneously removed in AAX SDK 2.1)

12.60.1.15.5 Utilities

- Added utilities for atomic operations and a thread-safe FIFO queue class: [AAX_CAtomicQueue](#)
- Added AAX stacktrace logging support to make plug-in debugging easier: see [AAX_STACKTRACE](#) and [AAX_TRACEORSTACKTRACE](#)
- Added a utility for locating the .aaxplugin bundle to provide an ability to access resources in the bundle

12.60.1.15.6 AAX Library

- Created an AAX Library project for Visual Studio 2012
- Created a libc++ target in the AAX Library Xcode project
- Resolved "incompatible ms_struct" warning in Xcode 6; removed [AAX_ALIGN_FILE_ALG](#) from inappropriate locations such as virtual classes that do not cross library boundaries
- Added [AAX_IParameterValue](#), an abstract value class for parameter data, and refactored [AAX_CParameter](#) to use this interface
- Re-named `AAX_CInstrumentParameters` to [AAX_CMonolithicParameters](#) (see the [Extensions](#) section for more information)
- Added an [AAX_CStateDisplayDelegate](#) constructor taking `std::vector<AAX_IString*>`
- Added an [AAX_CParameter](#) constructor taking [AAX_IString](#) as an identifier
- Added hex conversion methods to [AAX_CString](#)
- Fixed chunk size error handling in [AAX_CChunkDataParser](#)

12.60.1.15.7 Example plug-ins

- Added [DemoMIDI_Synth](#) and [DemoMIDI_Synth_AuxOutput](#) plug-ins
- Created Visual Studio 2012 projects for all example plug-ins
- Added EUCON page tables for all example plug-ins
- Various fixes for modifier-click event handling in example plug-ins
- Updated the example plug-in projects so that all built plug-in bundle filenames include "_Example"
- Corrected input/output property usage in HostProcessor example plug-ins
- Fixed multi-channel processing in [DemoDelay_HostProcessor](#)
- Fixed a bug with dynamic processing in [DemoMIDI_NoteOn](#) example plug-in
- Fixed [DemoGain_GUIExtensions](#) Win32 example plug-in GUI so that it is correctly displayed in Windows 8

12.60.1.15.8 Documentation

- Added [Media Composer Guide](#)
- Updated [Host Support](#) documentation for latest AAX host versions
- Updated the [Page Table Guide](#)
 - Updated [EUCON Page Tables](#) documentation
 - Updated [Avid Center Section Page Tables](#) documentation with tables mapping the EQ, Comp/Lim, and Exp/Gate table indices to their respective functions
- Updated [MIDI node](#) documentation
- Added new documentation pages for [Parameter update timing](#) and [Parameter automation](#)
- Improved [Presets and settings management](#) documentation
- Documented the [plug-in caching](#) behavior in Pro Tools
- Added documentation for optimizing an AAX DSP plug-in by using a hard-coded buffer size in the algorithm callback/ See the [Refactoring conditionals and branches](#) section of the [HDX DSP Guide](#)

12.60.1.16 AAX SDK 2.1.1

12.60.1.16.1 Definitions

- Explicitly removed support for the SDK's C99Compatibility headers in Microsoft Visual C++ 10.0 and later

12.60.1.16.2 DSP

- Added support and documentation for compiling AAX DSP plug-ins using Code Composer Studio 5
- Updated all example plug-in projects for use with Code Composer Studio 5

12.60.1.16.3 Documentation

- Extended the [parameter update documentation pages](#) with sequence diagrams and further information about linked parameter behavior
- Added guides for [DigiTrace](#) and [DSH](#)
- Added a reference list of [AAX interfaces](#)

12.60.1.17 AAX SDK 2.1.0

12.60.1.17.1 Interface

- New method added to [AAX_IACFTransport_V2](#) : [IsMetronomeEnabled\(\)](#)

12.60.1.17.2 AAX Library

- New methods in [AAX_CString](#) for direct copy from, assignment to, and comparison with `std::string`
- Fixed many implicit sign conversions
- Added `const` qualification to some [AAX_C...](#) methods
- Updated [AAX_IParameter::GetValueAsString\(\)](#) to take a pointer-to [AAX_IString](#) (was lvalue ref)
- Fixed a bug in [AAX_CEffectParameters::GetParameterNameOfLength\(\)](#); the method now correctly truncates a parameter name if the requested length is shorter than the shortest available abbreviated name
- Treat Warnings As Errors enabled in AAX Library projects
- clang pragmas added to avoid warnings for non-virtual destructors in ACF interface classes (cf. Microsoft COM)
- Xcode 3 project added for the AAX Library

12.60.1.17.3 Definitions

- Alignment of [AAX_CMidiPacket](#) and [AAX_CMidiStream](#) on 32-bit macOS is now explicitly set using `#pragma options align=power` to maintain backwards-compatibility with earlier versions of Pro Tools
- New property added: [AAX_eProperty_RequiresChunkCallsOnMainThread](#)
- New property added: [AAX_eProperty_Constraint_AlwaysProcess](#)
- Converted [AAX_eProperty_Related_Plugin_List](#) (property #22) to dedicated [DSP](#) and [Native](#) versions
- Re-named [AAX_eProperty_AudioBufferLength](#) to [AAX_eProperty_DSP_AudioBufferLength](#)
- Added new [AAX_ECurveType](#) selector: [AAX_eCurveType_Reduction](#)
- Added various new selectors to [AAX_ENotificationEvent](#)
- Updated [AAX_STEM_FORMAT](#) macros to allow negative index values
- Added new error codes to [AAX_EError](#)

12.60.1.17.4 Utilities

- New utility functions: [AAX::IsAvidNotification\(\)](#), [AAX_IsASCII\(\)](#), [AAX_AsStringFourChar\(\)](#)
- [AAX_ASSERT](#) and [AAX_TRACE](#) now require a trailing semicolon
- Re-named `LIMIT` to [AAX_LIMIT](#)
- Removed unused extended-80 conversion utilities

12.60.1.17.5 Extensions

- Resolved issue in which VSTGUI v4 key events were not received on Windows
- Xcode 3 projects added for the Juce and VSTGUI extension libraries

12.60.1.17.6 Documentation

- .pdf documentation moved to Doxygen
- Added several new sample plug-ins
- Expanded documentation for Host Processor and AAX Hybrid

12.60.1.18 AAX SDK 2.0.1

12.60.1.19 AAX SDK 2.0.0

12.60.1.19.1 AAX Library

- Added support for the AAX Hybrid processing architecture
- Added methods for better access to global MIDI data from [AAX_IEffectParameters](#)
- Extended the [AAX_ITransport](#) interface with several new methods
- Host Processor plug-ins can now trigger an analysis pass programmatically

12.60.1.19.2 Definitions

- Added new selectors to [AAX_ENotificationEvent](#) for state information during AudioSuite, bounce, and restore events
- AudioSuite reverb and delay plug-ins may opt out of the "Reverse" processing mode

12.60.1.19.3 Algorithm

- Support for temporary algorithm data blocks

12.60.1.20 AAX SDK 1.5.0

12.60.1.20.1 AAX Library

- Plug-ins now receive a different notification when receiving chunks from session and preset loads
- Aux output stems now support up to 256 output channels
- Added alpha versions of V2 interfaces
- Added projects for Visual Studio 2005 and 2008

12.60.1.21 AAX SDK 1.0.6

12.60.1.21.1 Documentation

- 64-bit targets enabled for the AAX Library and sample plug-ins

12.60.1.21.2 AAX Library

- Changed scope of some methods in [AAX_CEffectParameters](#) and [AAX_CEffectGUI](#)
- New 8 byte structure alignment added to [AAX.h](#)
- Changed the scope of some chunk parser items
- Clock context field is set to be synchronized across multiple plug-in instances
- Support for multiple input MIDI nodes
- Support for multiple named Aux Outputs ([AAX_CInstrumentParameters](#))
- Instrument parameters no longer uses host generated GUI by default

12.60.1.21.3 DSP

- Algorithm initialization routine now has 5 seconds to execute

12.60.1.22 AAX SDK 1.0.5

12.60.1.22.1 Directory Changes

- Removed 3 files in /ExamplePlugIns/Common
- Added [AAX_UtilsNative.h](#) and [AAX_Version.h](#)
- Moved [AAXLog\(\)](#), [AAXLogf\(\)](#), and [isParameterIDEqual\(\)](#) to [AAX_UtilsNative.h](#)

12.60.1.22.2 Documentation

- Fixed instance tracking bugs in DemoGain_BackGround
- Added a time-stamp parameter to DemoMIDI_NoteOn
- Added MIDI-through to DemoMIDI_NoteOn
- Added DemoGain_DMA sample plug-in

12.60.1.22.3 AAX Library

- Changed scope of some methods in [AAX_CEffectParameters](#)
- Set default number of steps in [AAX_CParameter.h](#) to non-zero
- Renamed enum AAX_EConstraintLocation to AAX_EConstraintLocationMask

12.60.1.22.4 DSP

- Larger buffer size allowed on TI
- Support for DLL chip affinity in Pro Tools 10.2 and higher
- New AAX_INT_LO and AAX_INT_HI utilities defined

12.60.1.23 AAX SDK 1.0.4

12.60.1.23.1 Describe

- Multi-mono support constraint property added
 - Will be supported in DAE versions 10.2 and higher

12.60.1.23.2 AAX Library

- AAX_CInstrumentParameters class added as helper for monolithic instruments
- AAX_CTimestamp type changed to signed 64-bit integer
- Maximum string length support added to binary display delegate

12.60.1.23.3 Documentation

- Resolved several DemoGain_GUIExtensions example plug-in bugs and improved parity with expected Pro Tools plug-in GUI features
- Added DemoMIDI_Sampler example plug-in
- Added /TI/SignalProcessing directory with example signal processing utilities
- Added new "AAX for Pro Tools" document (still in progress)

12.60.1.24 AAX SDK 1.0.3

12.60.1.24.1 Describe

- Added "deprecated type" feature for swapping in new Effect types
- Removed AAX_eProperty_TI_UncachedCycleCount
- Removed AAX_eProperty_UseSmallPreviewBuffer, as this property is now mandatory

12.60.1.24.2 Algorithm

- Established 1024 as the maximum expected audio buffer length for any AAX plug-in
- Created new instance initialization action flag for instance reset events

12.60.1.24.3 AAX Library

- Fixed reference-counting bug in [AAXRegisterPlugin\(\)](#)

12.60.1.24.4 DSP

- Extra software pipeline information added to CCS asm output by default
- External memory support added to default CommonPlugIn_LinkerCmd.cmd file
 - ExtendedPlugIn_LinkerCmd.cmd is now deprecated

12.60.1.24.5 Utilities

- DigiTrace facility for AAX_Assert changed from DTF_TIPLUGINS to DTF_AAXPLUGINS
- Added example DTT script for signal cancellation testing to Development builds
- Added DSP information tooltip feature to plug-in window header in Pro Tools

12.60.1.24.6 Documentation

- Win32 GUI example plug-in added to the SDK
- Basic coefficient smoothing example plug-in added to SDK
- Side Chain and Auxiliary Output Stem information page added to Doxygen
- Resolved SetControlHighlightInfo() naming inconsistency in sample plug-ins
- Expanded GUI information in AAX Manual

12.60.1.25 AAX SDK 1.0.2

12.60.1.25.1 AAX Library

- Moved AAX Library source to /Libs directory
- Added complete library source code and project files
- Removed pre-compiled AAX library binaries

12.60.1.25.2 Documentation

- Added correct mouse event handling logic to DemoGain_GUIExtensions plug-ins
- Added meters to DemoGain_Cocoa
- New TI optimization case studies added to the TI Guide document

12.60.1.25.3 Resolved bugs

- PTSW-149745
 - Loading code into external DSP memory is functional as of TI Shell build 10.1x828

Collaboration diagram for Change Log:

12.61 Example Plug-Ins

Descriptions of the SDK's example plug-ins.

12.61.1 SDK Example plug-ins

This SDK includes the following example plug-ins. These plug-ins are designed to demonstrate good AAX plug-in design with varying levels of complexity.

In general, the SDK includes one basic version of each example plug-in, as well as multiple variations on this basic version. Each of these variations demonstrates a particular feature or design approach. To see the specific changes that were made to implement a feature, compare the example plug-in variant that demonstrates the feature to the basic version of the plug-in.

Aside from the GUI Extension examples, which are designed to work with third-party GUI frameworks, each sample plug-in should successfully compile "out of the box". However, you may receive compilation errors during the plug-ins' post-build copy step due to the fact that compiled TI DLLs are not included with this SDK.

12.61.1.1 Basic examples

These plug-ins provide complete working examples of AAX plug-ins without a lot of extra features. Use these plug-ins as a starting point for understanding [AAX](#).

12.61.1.1.1 DemoGain DemoGain is the simplest example plug-in, incorporating a mono algorithm with gain and bypass parameters.

12.61.1.1.2 DemoDist DemoDist demonstrates some more sophisticated techniques such as coefficient calculation and packaging, private data allocation, and multiple stem format support. DemoDist also demonstrates some basic optimization strategies for improving real-time algorithmic performance.

12.61.1.1.3 DemoDelay DemoDelay implements a basic delay algorithm. The variants of this example demonstrate a variety of alternative processing features provided by [AAX](#).

12.61.1.1.4 DemoMIDI_NoteOn DemoMIDI_NoteOn implements basic MIDI input and output functionality. The example demonstrates a simple MIDI processing loop that either copies MIDI input to MIDI output or holds Note On events depending on a Hold parameter. The example also detects Note On events. To visually indicate these events, the plugin will output DC offset while a note is held. This offset is not audible but will appear in the Pro Tools track and can be used to verify the timing of the Note On and Note Off events. The example also shows how to handle MIDI packages in the Data Model by overriding the [AAX_CEffectParameters::UpdateMIDINodes\(\)](#) method.

12.61.1.1.5 RectiFi This is a fully ported version of the Recti-Fi plug-in from Avid's D-Fi suite. For more information about Recti-Fi, see <http://www.avid.com/plugins/d-fi>

Note

The SDK's Recti-Fi example plug-in is currently out of date and does not accurately represent Avid's shipping Recti-Fi plug-in.

12.61.1.2 Feature examples

Each of these plug-ins is a slight variation on one of the [Basic examples](#). Each feature example plug-in demonstrates a specific feature or a possible alternative design approach for the plug-in. Compare these plug-ins with the corresponding basic example plug-in when you want to understand how a feature or design should be applied to your own AAX plug-ins.

12.61.1.2.1 DemoGain_GUIExtensions These examples demonstrate the use of various native and third-party GUI frameworks with [AAX](#). The examples that use third-party frameworks are configured to link to static libraries that combine the SDK's [GUI Extensions](#) (located in /Extensions/GUI) and the applicable third-party GUI framework. These libraries are not included in the SDK, and you will need to install the applicable framework SDK before it will be possible to compile these example plug-ins.

Note

See bug [AAXSDK-599](#)

12.61.1.2.2 DemoGain_LinkedParameters This example demonstrates parameter linking. The plug-in is a stereo version of DemoGain, with options to link the left and right channels in two different modes.

12.61.1.2.3 DemoGain_Smoothed This example demonstrates efficient algorithmic coefficient smoothing using a slight variation on the basic DemoGain plug-in algorithm.

12.61.1.2.4 DemoGain_Background This example demonstrates a background routine for algorithm processing. This example also uses the AAX [direct data interface](#) for communicating algorithmic delay to the plug-in's controller.

12.61.1.2.5 DemoGain_DMA This example includes two Effects that demonstrate use of the Scatter/Gather and Burst DMA facilities in [AAX](#).

12.61.1.2.6 DemoGain_Multichannel This example demonstrates a multichannel plug-in configuration supporting all available point source stem formats.

This plug-in also includes a simple example of gain-reduction metering, which can be used to test host features which use this data such as the [gain reduction meters](#) in Pro Tools.

12.61.1.2.7 DemoGain_UpMixer This example demonstrates conversion between different stem formats

12.61.1.2.8 DemoGain_ParamValueInfo This example demonstrates an implementation of the [GetParameterValueInfo\(\)](#) method, which is used to properly display certain parameter details on attached control surfaces. See [Avid Center Section Page Tables](#) in the [Page Table Guide](#).

12.61.1.2.9 DemoDist_GenCoef This example demonstrates an alternative approach to parameter update handling. It bypasses the packet dispatcher helper class and directly overrides [UpdateParameterNormalizedValue\(\)](#) and [GenerateCoefficients\(\)](#). This approach may be appropriate for plug-ins that involve complex mapping between parameter updates, coefficient generation algorithms, and coefficient data packets.

12.61.1.2.10 DemoDelay_HostProcessor This example includes two Effects that demonstrate the optional [Offline processing interface](#) for advanced offline processing features. One Effect implements a simple offline delay line, while the other Effect implements the same delay line but compensates for its delay when rendering to the timeline. This demonstrates how to manually compensate for inherent algorithmic delay in an offline processor.

Note

The output of offline plug-ins that do not use the [Offline processing interface](#) will be automatically adjusted by the host to account for any declared latency. The manual compensation technique demonstrated by `DemoDelay_HostProcessor` is **only** necessary in plug-ins that implement the [Offline processing interface](#), e.g. plug-ins that require nonlinear offline processing features.

12.61.1.2.11 DemoDelay_Hybrid This example demonstrates the optional [Hybrid Processing architecture](#) architecture for AAX plug-ins. This plug-in implements a short delay line that is rendered in the high-latency hybrid context. It can be built and run for either AAX Native or AAX DSP.

12.61.1.2.12 DemoDelay_DynamicLatencyComp This example demonstrates how to properly handle algorithmic latency changes at run-time. It uses a delay line to emulate a latency-inducing algorithm with varying latency based on the delay parameter setting. When the plug-in's latency compensation feature is enabled it declares this latency to the host.

Host Compatibility Notes The `DemoDelay_DynamicLatencyComp` example is compatible with Pro Tools 11.1 and higher.

12.61.1.2.13 DemoMIDI_Synth A basic synthesizer plug-in demonstrating use of an external object to manage the plug-in's state. AAX Native plug-ins that are designed to work with a cross-format framework may use a similar design. This plug-in uses [AAX_CMonolithicParameters](#) and therefore is AAX Native only.

12.61.1.2.14 DemoMIDI_Synth_AuxOutput A variation on [DemoMIDI_Synth](#) demonstrating the [Auxiliary Output Stems](#) feature. This instrument plug-in supports four independently-routable synthesizer objects.

12.61.1.2.15 DemoMIDI_Sampler This simple "drum machine" style sampler plug-in demonstrates sample-accurate global and local MIDI input and the MIDI Transport interface. This plug-in uses [AAX_CMonolithicParameters](#) and therefore is AAX Native only.

12.61.1.2.16 DemoMIDI_Transpose A basic MIDI effect that transposes incoming notes. The example implements audio passthrough, switchable MIDI passthrough and MIDI bypass. Moreover, the plugin overrides [GenerateCoefficients\(\)](#) to generate coefficients as a single data structure and it also overrides [ResetFieldData\(\)](#) to customise the initial values of private data fields.

12.61.1.3 Deprecated Examples

12.61.1.3.1 DemoGain_Delay

Deprecated The DemoGain_Delay example is deprecated. See DemoDelay_HostProcessor

Collaboration diagram for Example Plug-Ins:

12.62 VENUE Guide

Details about using AAX plug-ins in VENUE live sound systems.

12.62.1 Contents

- [About this document](#)
- [Overview of VENUE](#)
- [VENUE systems](#)
- [Host environment](#)
- [AAX feature support and compatibility](#)
- [VENUE Plug-in installer specification](#)
- [Additional plug-in guidelines](#)
- [System details](#)
- [Additional Information](#)

12.62.2 About this document

This guide discusses specific details related to creating AAX plug-ins which are compatible with Avid VENUE systems.

This guide includes a general overview of the new VENUE architecture as it pertains to plug-ins, a set of guidelines for developing compatible plug-ins, and details for creating full-featured plug-in installers for VENUE.

Note

Any reference in this document to "VENUE" refers specifically to VENUE | S6L, and VENUE | S3L systems. Older VENUE systems such as VENUE Profile, D-Show, and SC48 are not compatible with AAX plug-ins and are not considered in this document.

12.62.3 Overview of VENUE

VENUE is Avid's product line aimed at live sound users. VENUE systems are modular, with audio engine, control surface, console, I/O, and external GUI units.

VENUE offers plug-in racks to utilize the power of AAX DSP plug-ins. As virtual outboard racks inside the VENUE system, the plug-in racks allow users to take their AAX DSP plug-ins out of the studio and into a live performance.

Figure 1: The main VENUE software interface

Figure 2: VENUE plug-in rack

Using the VENUE GUI, an operator is able to see thumbnails for each of the plug-ins in a plug-in rack. An operator can choose to zoom in on a plug-in from this rack view and, from there, graphically control the plug-in with the mouse, keyboard, or touch-screen. Only one plug-in interface can be displayed at a time in this mode.

Figure 3: Plug-in zoom view

12.62.4 VENUE systems

This section will provide a brief overview of Avid's AAX-compatible VENUE systems. For more information about the features, functionality, and use of these systems see the VENUE user documentation.

12.62.4.1 VENUE | S6L

VENUE | S6L is a modular system designed to take on the world's most demanding tours and events with ease. Offering unprecedented processing capabilities - with over 300 processing channels - S6L delivers unrelenting performance and reliability through its advanced engine design and backs it up with modern touchscreen workflows and scalability to meet any challenge.

The S6L engine contains dedicated HDX-powered DSPs handling all plug-in processing and supports 64-bit AAX DSP plug-ins.

12.62.4.2 VENUE | S3L-X

The VENUE | S3L-X System is a modular live sound solution including an HDX-powered processing engine, scalable remote I/O, and a EUCON-enabled control surface.

At the heart of the S3L system lies the E3 engine. The E3 runs Windows Embedded and a version of VENUE software that can load AAX DSP plug-ins onto a built-in HDX platform. Accompanying the E3 engine is the S3 control surface and one or more Stage 16 remote I/O boxes.

Most system parameters, including plug-ins, can be controlled using either the on-screen VENUE software or directly via encoders on the S3 control surface. When being used as part of a VENUE | S3L-X system, the S3 control surface is divided into three main sections: A - Channel Section The Channel section provides control of Input Channels, FX Returns, Output Channels, some channel parameters (such as Input Channel Gain and Aux Send levels), and channel banking. Channels are selected using the channel Select switches next to each fader.

B - Channel Control The eight Channel Control encoders provide control of processing functions for the currently selected Input or Output Channel. Inserted Dynamics and EQ plug-ins can be selected and adjusted in Channel Control.

C - Global Control The eight Global Control encoders provide control of system-wide parameters, including control of plug-ins. The Global Control encoders can be placed into Insert Mode, and can then be used to select and adjust any plug-ins.

Figure 4: Main control sections on the S3 control surface

12.62.4.2.1 Using Channel Control If a channel has an EQ, Comp/Lim, or Expander/Gate plug-in inserted on it, it can be controlled using the eight Channel Control encoders. The user can toggle between controlling the built-in Dynamics or EQ processors and the plug-in versions.

Each Input and Output Channel has built-in EQ and Comp/Lim processors. Each Input Channel also has a built-in Expander/Gate. To adjust the built-in processors, the user selects a channel, assigns a processing function to Channel Control by pressing the corresponding encoder from the Channel Control main menu, then adjusts the available parameters.

Figure 5: The Channel Control main menu

See [Center Section Parameter Mapping on VENUE | S3L-X](#) for a description of how plug-in parameters are mapped to the S3L Channel Control encoders for EQ, Compressor/Limiter, and Expander/Gate plug-ins.

12.62.5 Host environment

12.62.5.1 Audio engine

The audio engine in VENUE is based around Avid's HDX technology. Each VENUE S6L and S3L System contains a specialized HDX core card. For more information about HDX, see the [HDX DSP Guide](#). Because the VENUE architecture is so similar to HDX, AAX DSP plug-ins are cross-compatible with VENUE and most plug-ins will run seamlessly on VENUE with little or no modification.

Each VENUE system operates at a single native sample rate. VENUE supports multiple processing block sizes at this sample rate. Like in Pro Tools | HDX systems, each DSP chip in the system will only be able to load plug-ins using a single block size; plug-ins which process using different block sizes cannot be allocated to the same DSP.

12.62.5.2 Available DSP resources

The following information reflects plug-in processing abilities of VENUE systems:

- S3L-X
 - 4 TI C6727 DSP chips are available for plug-in processing
 - 40 plug-in rack slots are available
- S6L
 - All HDX DSP cards are dedicated to plug-in processing; each HDX DSP card has 18 TI C6727 DSP Chips
 - Depending on E6L engine type, 125 (E6L-144) or 200 (E6L-192) plug-in rack slots are available

12.62.5.3 Operating system

The core host software in a VENUE system is built upon Windows Embedded 8. The installation used on VENUE systems is a customized version of Windows 8 that includes only what is necessary for the VENUE software.

Core services from Windows 8 are available, such as the Win32 API, but some advanced services may not be available. Such services include MIDI, printing, video codec, .NET, etc. If your code relies on advanced Win32 APIs, or you are in doubt about specific APIs, please contact Avid for more information.

Using unavailable services may cause a plug-in to not load (e.g. if it attempts to link against DLLs that aren't included in the Windows Embedded 8 image) or to fail during run-time. Whenever possible, before using any advanced Win32 API, you should verify the availability of the service and/or handle the fact that the service might not be functional.

See the [VENUE Plug-in installer specification](#) section for more information about ensuring that all required run-time components are available to your plug-in.

12.62.5.4 Display

VENUE S6L requires that plug-in windows be restricted to a certain size. If a plug-in exceeds this size, it will overlap VENUE's GUI and may possibly be truncated. The specifications are as follows:

- S3L-X
 - Total GUI size: 1024 W x 768 H
 - Max plug-in window size (w/o sidechain support): 749 W x 617 H
 - Max plug-in window size (with sidechain support): 749 W x 565 H
- S6L
 - Total GUI size: 1920 W x 1080 H
 - Max plug-in window size (w/o sidechain support): 1436 W x 855 H
 - Max plug-in window size (with sidechain support): 1436 W x 796 H

VENUE will dispatch a [AAX_eNotificationEvent_MaxViewSizeChanged](#) notification indicating the maximum size for a plug-in's GUI. Calls to [AAX_IViewContainer::SetViewSize\(\)](#) will fail with an error if the plug-in attempts to set its view size to a larger value than the system supports, though the plug-in's initial GUI will be displayed (and possibly truncated) at its normal size before any resize requests are made.

The actual hardware and graphical acceleration available in VENUE systems is as follows:

	S3L-X	S6L
CPU model	Celeron P4500	Core i5-2510E
GPU model	Intel HD Graphics	Intel HD Graphics 3000
DirectX support	10.1	10.1
OpenGL support	2.1	3.1
OpenCL support	None	None
Shader model	4	4.1

12.62.5.5 Page tables

- VENUE S3L-X uses 'PcTL' (ProControl) page tables
- VENUE S6L uses 'Av46', a EUCON-style page table with a 4x6 knob cell configuration.

Note

In S6L, the 'FrTL' (C|24) page table is used as a fallback when 4x6 is not available. This is only a temporary solution to support legacy plug-ins. All plug-ins targeting VENUE S6L support must support the 4x6 knob cell layout and should not rely on this C|24 fallback behavior.

Page table design guidelines Primary plug-in parameters should be located on the first page in the page tables for a surface. This is especially true for the 4x6 knob cell layout used by S6L. Users should not be required to navigate between pages for the majority of common operations.

For more information about page tables, including additional guidelines for good page table design, see the [Page Table Guide](#).

12.62.5.6 Network communications

Some plug-ins may require interaction with other devices in a network. VENUE systems have two Gigabit Ethernet ports available:

1. **ECx port** Intended for connection of VNC Viewer to control the VENUE system remotely. The IP address and network mask for this port are user-configurable in the VENUE UI.
2. **AVB port** Intended for connection of all other VENUE system components, as well as a computer running Pro Tools software. This port always uses link-local addressing. Because of AVB traffic, the effective bandwidth of this port is limited to 100 Mb/s.

Note

Plug-ins must not use a significant portion of the available bandwidth on the AVB port, since it will affect mission-critical control connections of a VENUE system.

Both S3L-X and S6L systems include the Apple Bonjour service. Plug-ins may use Bonjour for interfacing with other software in the network. Plug-ins must not install their own version of Bonjour or attempt to modify the Bonjour installation on the system.

12.62.5.7 Host environment summary

	S3L-X	S6L
Operating System	Windows Embedded 8	Windows Embedded 8
Sample Rate	48 kHz	96 kHz
Max GUI size	749 W x 617 H (no sidechain) 749 W x 565 H (sidechain)	1436 W x 855 H (no sidechain) 1436 W x 796 H (sidechain)
Page table	'PcTL'	'Av46'

12.62.6 AAX feature support and compatibility

VENUE supports many of the same AAX features as Pro Tools. However, some features are not available in VENUE, and other features are managed differently between the two applications. This section describes how VENUE handles various optional AAX features.

12.62.6.1 Processing configurations

Architectures VENUE supports 64-bit AAX DSP plug-ins only. AAX Native and AAX Hybrid plug-ins are not supported. Plug-ins compiled for 32-bit processors are not supported, though they may be included in a VENUE-compatible .aaxplugin bundle alongside the plug-in's 64-bit binary.

Stem formats

- Mono plug-ins may be inserted as channel inserts on mono input strips and output busses.
- Stereo plug-ins can be inserted as channel inserts on stereo input strips and output busses.
- Greater-than-stereo formats are not supported by VENUE
- Multi-mono processing is not supported; an operator must use the stereo version of a plug-in in stereo processing locations.

Width-changing plug-ins Width Changing plug-ins are not allowed as inserts except in mix busses. Unlike in Pro Tools, a plug-in cannot change the output stem format of a strip or bus by using a mono-to-stereo plug-in on a mono track.

However, width-changing plug-ins are supported on output busses. For these, the outputs of the plug-in can either be routed back to an FX return or routed out to physical outputs. This functionality does not require any additional implementation specific to VENUE.

12.62.6.2 Presets and automation

Plug-In settings are persisted (saved & restored) the same way for Show files, snapshots & settings files, using a single method to extract settings and a single method to apply setting. The code uses the "chunk" APIs. That's similar to what Pro Tools does, except that for automation, VENUE uses exclusively snapshots (that is, VENUE does not record & playback individual control changes).

Many VENUE snapshots users are known to store settings for every Plug-In in every snapshot (or almost). This causes performance issues because settings are often fairly slow to load, making a snapshot recall last too long (sometimes 30 seconds or more!) whereas users expect a snapshot recall to take instantly. However, extremely frequently, settings are not actually changing from a snapshot to the next one (that is, from one snapshot to the next one, the vast majority of Plug-Ins contain the same settings). To mitigate this, VENUE calls [AAX_IEffectParameters::CompareActiveChunk\(\)](#) to determine whether a chunk from an incoming snapshot would result in any change to the plug-in's current settings. If not, the new chunk will not be loaded onto the plug-in. This

optimization is extremely effective, but requires that Plug-Ins implement the [CompareActiveChunk\(\)](#) method properly at any time (in particular regardless of whether the plug-in is visible or not). With VENUE, you must implement this API very well or the Plug-In may not be controllable. This optimization will affect settings application when a show is loaded, when presets are loaded and when snapshots are recalled.

All chunks are first compared one by one to the active chunks, until one is different or an error is returned. If any chunk compare fails (not equal or error returned), then all chunks are sent in sequence.

As it is the basic method for plug-in settings manipulation, it is critical that plug-ins process chunks as accurately and efficiently as possible.

Plug-In Chunks The size of a plug-in chunk cannot exceed 64KB in VENUE. If a Plug-In requires more than 64KB of chunk data total (all chunk sizes added), settings for this plug-in won't be persisted, snapshots won't work for this plug-in and users won't be able to load or save settings. If you can not meet this requirement, you should detect that you are running on VENUE and not declared the process type as it won't be usable.

12.62.6.3 Unsupported features

The following AAX features are not supported by VENUE. Plug-ins that require these features will not be compatible with VENUE systems. If your plug-ins use these features for advanced functionality but not for basic operation then you should document this restriction for VENUE users.

- Advanced audio routing VENUE does not support [Auxiliary Output Stems](#) from plug-ins.

Warning

[Description callback](#) calls to register auxiliary output stems will return an error code on VENUE systems, indicating that the host will not provide audio buffers for auxiliary output stems during processing. A plug-in must not attempt to write data into auxiliary output stem buffers which have not been provided by the host!

- Transport interface VENUE operates entirely in real-time and does not contain a timeline of pre-recorded audio. Therefore VENUE does not support the [AAX_ITransport](#) interface. VENUE will return [AAX_ERROR_UNIMPLEMENTED](#) to unsupported transport interface method calls.
- MIDI VENUE does not support MIDI routing to and from plug-in instances, and no [AAX MIDI features](#) are supported by VENUE.

12.62.7 VENUE Plug-in installer specification

To install plug-ins, the VENUE software includes a simple installation interface. To install a plug-in, the operator simply plugs in a USB Flash Drive with an installer for the plug-in and the plug-in will show up in an installer menu on the VENUE interface (shown below).

This menu will automatically list all the installable plug-ins on the drive. With the click of a button, the user can install the plug-ins onto his VENUE system.

Figure 6: The VENUE plug-in installer tab

For this custom installation to work properly, the plug-in installation USB Key must follow a certain layout. This layout is designed to be as flexible and as expandable as possible, giving the developer many options while retaining the simplicity that makes VENUE's plug-in installation appealing to the user. This layout is also designed to coexist on a drive with a Pro Tools plug-in install. The following is a detailed description of how the file hierarchy should be laid out.

12.62.7.1 Overview

The VENUE Plug-In installer specification is an extension of an AAX plug-in bundle, i.e. of the *MyPlugIn.aaxplugin* folder.

A standard .aaxplugin directory forms a basic, compatible plug-in installer for VENUE. See [.aaxplugin Directory Structure](#) for more information about this folder.

The following optional items can be added to the .aaxplugin folder to extend its functionality when used as a VENUE plug-in installer:

- License file that will need to be accepted by end user
- Pre-install action (either .bat script or executable or both)
- Post-install action (either .bat script or executable or both)
- Pre-uninstall action (either .bat script or executable or both)
- Post-uninstall action (either .bat script or executable or both)
- PACE Eden installer to update the version pre-installed on the VENUE system
- Factory presets
- Registry entries in a form of .reg files
- Program files to be placed in the system's C:\Program Files folder
- Plug-in thumbnails to be shown in the rack in VENUE Software UI

12.62.7.2 Directory structure

Here is a layout of the optional elements in the .aaxplugin plug-in installer directory:

- /Contents
 - *standard AAX plug-in contents*
- /Pace Eden
 - Setup.exe
 - Setup.bat
 - Version.txt
- /Program Files
 - ...
- /Thumbnails
 - *id1.bmp* (example: 424644204C41324131314C41.bmp)
 - *id2.bmp*
- /License.rtf or License.txt
- /Install_before.bat
- /Install_before.exe
- /Install_after.bat

- /Install_after.exe
- /SomeSettings.reg
- /Uninstall_before.bat
- /Uninstall_before.exe
- /Uninstall_after.bat
- /Uninstall_after.exe

12.62.7.3 Optional installer files

12.62.7.3.1 License terms A license stored as a file of either RTF or ASCII plain text format. The file must be located in the root folder of the installer. Depending on the text file format, the file name must be either *License.rtf* or *License.txt*.

12.62.7.3.2 Registry entries Registry settings to be applied during installation need to be stored in .reg files in the root folder of installer. All such files must be of the Windows Registry format. Particular file names does not matter.

Registry files are applied during plug-in installation.

It is important to not alter any system settings or settings of other software installed.

Note

Changes in the registry are not reverted during the plugin uninstallation.

12.62.7.3.3 Program files Files under the *Program Files* subfolder in the plug-in installer will be copied to the system's *C:\Program Files* folder. All contents of the *Program Files* subfolder are copied as-is into *C:\Program Files*, retaining the internal folder structure of nested directories.

These changes are undone when the plug-in is uninstalled.

12.62.7.3.4 Plug-in thumbnails VENUE uses thumbnail images to display plug-in GUIs in the plug-in rack while the full-size plug-in GUI is hidden.

If thumbnail images are not provided in the plug-in installer then VENUE will display a generic thumbnail image for the plug-in until it has been focused in Zoom Mode in the VENUE interface. In this case VENUE will create and cache a thumbnail image for the plug-in GUI the first time that it is focused.

The user may regenerate a thumbnail by right-clicking a rack with a plug-in and choosing "Recreate Thumbnail".

Including thumbnails in plug-in installers

A separate thumbnail should be provided in the plug-in installer for each variant supported by the plug-in, i.e. each unique AAX DSP ID triad registered by the plug-in. Use the "Recreate Thumbnail" feature to create the initial versions of your plug-in thumbnail images. Package these thumbnail images into your plug-in installer in order to guarantee that thumbnail images will be available to users immediately upon installing the plug-in.

Each thumbnail bitmap file is named after the following plug-in parameters:

1. [AAX_eProperty_ManufacturerID](#)
2. [AAX_eProperty_ProductID](#)
3. [AAX_eProperty_PluginID_Ti](#)

All of three are converted to hexadecimal representation and concatenated to form a file name that uniquely identifies a plug-in variant. For example, the Avid Channel Strip plug-in has a thumbnail file named 41564944 43685374 434D5469.bmp (no spaces).

The "Recreate Thumbnail" feature in VENUE will ensure that the generated thumbnail images use the correct file names, resolution, and image format.

12.62.7.3.5 Actions A plug-in installer may define custom actions for the following cases:

1. Pre-install action - executed when plug-in installation starts
2. Post-install action - executed when plug-in installation finishes
3. Pre-uninstall action - executed when plug-in uninstallation starts
4. Post-uninstall action - executed when plug-in uninstallation finishes

Each action is defined by either a .bat file or an Win32/64 executable file (.exe). Action files must be placed in the root folder of installer and use these file names:

1. Pre-install action - *Install_before.bat*, *Install_before.exe*
2. Post-install action - *Install_after.bat*, *Install_after.exe*
3. Pre-uninstall action - *Uninstall_before.bat*, *Uninstall_before.exe*
4. Post-uninstall action - *Uninstall_after.bat*, *Uninstall_after.exe*

If both .exe and .bat files are present for a certain action, then both are executed, with the .bat file being run before the .exe.

When executing an action, no exit code is tested. In order to report errors, the following needs to be done:

1. .bat files:

The following line should be used to report an error message from a script: `reg add HKCU\Software\Digidesign\tmp /f /v InstallResult /d "Error description"`

2. .exe files:

The error message needs to be added to Windows registry as a REG_SZ value in *HKEY_CURRENT_USER\Software\Digidesign\tmp* and named *InstallResult*.

The presence of the error message will abort a plug-in installation. Any error strings will be written to the VENUE logs so that Avid support will be able to see them. Error strings from these actions are not shown to the user.

12.62.7.3.6 PACE software installer

Warning

This functionality must not be used without prior approval from Avid. Before releasing **any** VENUE plug-in update with a bundled PACE installer you must contact Avid to confirm that the bundled installer will not cause issues for deployed VENUE systems.

VENUE allows plug-ins to install updated version of PACE iLok software immediately after the plug-in installation. In general, Avid tries to provide the latest Pace software with each VENUE software release and update. Therefore this step should not be necessary in most cases.

The PACE installer files must be located in *Pace Eden* subfolder of the installer. This folder must contain the following files:

- *Version.txt* containing a version of the *LDSvc.exe* PACE executable being installed. The version information must be in the form of *1.2.3.4*
- *Setup.exe* - the PACE installer itself.
- (optional) *Setup.bat* containing an installation script. Usually used to run PACE installer in a silent (no UI, no interaction) mode.

During installation, *Setup.bat*, if present, is run. Otherwise *Setup.exe* is executed with the following command line arguments:

```
Setup.exe /s /v"REINSTALLMODE=vamus REBOOT=ReallySuppress /qn"
```

If the version of installer is not higher than the version installed in system, the installation will not be performed.

An OS reboot is prompted in VENUE UI after PACE was installed.

12.62.7.4 Using a VENUE plug-in installer

In order to install a plug-in to the VENUE system, end user is expected to perform the following steps:

1. Download a VENUE Plug-in Installer(s) in a form of archive (Zip is suggested). Is it ok to have multiple plug-ins in one archive as soon as each plug-in is in own VENUE Plug-in Installer (i.e. in own .aaxplugin folder).
2. Unpack archive and copy installers to USB drive in the following way:
 - (a) "AAX Plug-Ins" folder must be placed in the root of USB drive.
 - (b) Each installer needs to be copied directly the "AAX Plug-Ins" folder. In the end, resulting folder structure will look like this:
3. Install plug-ins in a way described in documentation of a particular VENUE Software version.

12.62.8 Additional plug-in guidelines

12.62.8.1 General Reliability and Fault Tolerance

Since VENUE is a more "mission critical" type of application where there is no room for error during a live show, additional precautions have to be taken with respect to reliability of its various components. We have built provisions in VENUE to protect the system from catastrophic failure due to a plug-in crashing and bringing down the entire system. On top of this, extra care should be taken in developing stable software when targeting VENUE as a platform.

If a plug-in crashes, the user will be warned through a dialog. A crash brings down all plug-in processes, but audio keeps flowing through the console and through the DSPs, including the plug-ins' DSPs. All the effects continue to be effective, but their parameters can't be accessed or modified anymore (the show goes on...).

At this point, audio should be totally unaffected, even for the effect that caused the crash (assuming the crash took place in the host code, not the DSP code, of course). At the user's discretion, all plug-ins will be bypassed or muted (depending on where they are used in the system), any dependencies on the plug-ins' DSPs will be removed, the plug-ins' DSPs will be reset, and all the plug-ins will start again. When the rebuilding operation is complete, the user will be prompted to decide when he wishes the new plug-ins to be connected.

12.62.8.2 Plug-In Dialogs

Plug-ins should avoid invoking dialog windows in VENUE. We strongly suggest that any unnecessary dialog window your plug-in creates, whether at installation or instantiation, be removed. For VENUE-only plug-ins, we strongly suggest to not make use of any dialog windows.

Should you nevertheless need to make use of additional windows or dialogs, you need to make sure that they are front-most, so that they will not be hidden behind VENUE's GUI. The VENUE software will try to force your windows to be front-most, but it is safer if your plug-in enforces this in the first place.

12.62.8.3 Online Help

VENUE currently doesn't include any standardized help menu for plug-ins. We recommend that you use tooltips and other "live" help techniques similar to what plug-ins like ReVibe II, Reverb One, and Smack! use to help the user. For instance, when a user clicks on the "Side-Chain EQ" label of the Smack! Plug-In, here's what they see:

Figure 7: Tooltip help in Avid's Smack! plug-in

One of the major benefits of this technique is that it is supported across platforms and will work the same in all [AAX](#) hosts.

12.62.9 System details

12.62.9.1 External dependencies

AAX plug-ins may rely on the presence of the following items in VENUE systems:

- All VENUE systems
 - Bonjour service and library

Note

Plug-in installers are forbidden from installing over or modifying the pre-installed version of Bonjour on the VENUE system.

- VC 2005 x64 runtime
 - VC 2008 x64 runtime
 - VC 2010 x64 runtime
 - VC 2012 x64 runtime
 - VC 2013 x64 runtime
- S6L versions 5.7 and higher
 - VC 2015 x64 runtime
 - VC 2017 x64 runtime
- S6L versions 7.0 and higher
 - VC 2015-2019 x64 runtime
- S3L-X version 4.6.1 with "S3L-X Touch Support Patch" installed
 - VC 2015-2019 x64 runtime

Because VENUE does not execute standard software installers for plug-ins, Avid tries to keep VC runtime versions up to date relative to the moment of release of a particular VENUE Software version.

As of the time of this writing, Venue S3L-X systems are no longer receiving software updates and thus the S3L software will not be updated to include any additional system components beyond VC 2019. The last runtime update done for S3L-X was provided by the optional "S3L-X Touch Support Patch".

If you would like to provide compatibility with Venue host software which does not include your plug-in's required runtime libraries then we recommend statically linking your plug-in to these runtime libraries.

12.62.9.2 Environment variables

Both plug-in installers and actual plug-ins may rely on a presence of the following environment variables in a VENUE system:

- **DAEPLUGINSFOLDER** - is always set to the Installed Plug-ins location. Currently this is *C:\Program Files\Common Files\Digidesign\DAE\Plug-Ins*. Final backslash is absent.
- **JEX_HOST_TYPE** - equals "venue". If required, this may be used to provide a custom behavior of the plug-in when it's run on VENUE system.

12.62.9.3 Plug-in file locations

Installed Plug-Ins Located at *C:\Program Files\Common Files\Digidesign\DAE\Plug-Ins*

This folder is the only location used by VENUE software to instantiate a plug-in.

This location is different from the one used by Pro Tools and Media Composer for 64-bit [AAX](#) plug-ins. The only way for a plug-in to appear at that location is to be installed from VENUE Software's "Options">"Plug-Ins" page; standard plug-in installers will place the plug-in into a different directory.

Note

This location may change in future VENUE software releases. Plug-ins should not make any assumptions about the install directory and should rely on the VENUE plug-in installer to place them in the correct location.

Plug-ins available for installation

- Local: Located at *C:\Program Files\Common Files\Avid\Audio\Plug-Ins*
- On USB drive: Located at *(USB drive letter):\AAX Plug-Ins*

These locations can be chosen as sources for plug-in installation on VENUE Software's "Options">"Plug-Ins" page.

Cached plug-in installers Located at *D:\D-Show\Plug-In Installers*

Contains copies of plug-in installers installed via VENUE Software's "Options">"Plug-Ins" page.

This location can be chosen as source for plug-in installation on VENUE Software's "Options">"Plug-Ins" page under the name "Previous Installs".

Factory presets Located at *D:\D-Show\User Data\Effect Presets\Factory Presets*

Contains preset files for plug-ins, as well as for certain VENUE parameters. Presets are organized in folders.

Each subfolder corresponds to a particular preset type. Plug-in presets are named after the plug-in's name and the plug-in's `AAX_SPlugInChunkHeader::fProductID` value. For example, for an Avid Channel Strip plug-in the subfolder name is *Channel Strip [31313736]*, where "31313736" is an unsigned integer of the Channel Strip product ID.

Contents of subfolders are .tfx files of plug-in presets. Each file name will be visible to end user as a preset name.

Presets are copied into file location during a plug-in installation.

Plug-in thumbnails Located at *C:\Program Files\Digidesign\Plug-In Icons*

Contains .bmp files of plug-in thumbnails generated by VENUE Software as a result of saving current plug-in graphics into a bitmap. See [Plug-in thumbnails](#).

12.62.9.4 Installation process

12.62.9.4.1 Plug-in installation These are the steps followed by VENUE when installing a plug-in:

1. First, a VENUE plug-in installer is cached. This is done by copying a plug-in from installation source to the Cached VENUE Plug-In Installers location. All files are copied with an exception of the "Documentation" and "Pro Tools" folders.

All of the following steps are executed from the cached installer location, not from the original source location.

2. The pre-install batch script ("Install_before.bat"), if present, is executed. Execution assumes running the script without a console window.

Note

The pre-install script must not contain any installation steps, since it's executed before the plug-in license is accepted. In general, you should only use the pre-install script for pre-install checks.

3. The pre-install executable ("Install_before.exe"), if present, is executed. Execution assumes running the executable without a console window.

Note

The pre-install executable must not perform any installation steps, since it's executed before the plug-in license is accepted. In general, you should only use the pre-install script for pre-install checks.

4. A license ("License.rtf" or "License.txt"), if present, is shown to the user. If "License.rtf" is not found, "License.txt" is used. A license, if present, must be accepted by user; otherwise installation will be aborted.
5. All files of the VENUE plug-in installer are copied to the system Installed Plug-Ins location, keeping the .aaxplugin folder structure. Failure to copy any of the items results in installation being aborted.
6. If the plug-in installer contains a subfolder named "Program Files", its contents are copied into "C:\Program Files". Failure to copy any of items results in installation being aborted.
7. If the plug-in installer contains a subfolder named "Contents\Factory Presets", its contents are imported as plug-in presets. The "Factory Presets" folder must contain only valid plug-in .tfx preset files in an arbitrary folder structure. All preset files are read and copied into the system's Factory Presets location.

Note

It is important for a plug-in installer to contain only plug-in presets corresponding to plug-in being installed.

8. Plug-in thumbnails, if present, are copied from the "Thumbnails" subfolder of the installer to the system Plug-in Thumbnails location.
9. Registry files, if any, are imported. Every file with .reg extension in the root of plug-in installer is treated as a Windows Registry file and gets imported by calling

```
regedit /s "<file.reg>"
```

No error checking is performed.
10. The post-install batch script ("Install_after.bat"), if present, is executed. Execution assumes running the script without a console window.
11. The post-install executable ("Install_after.exe"), if present, is executed. Execution assumes running the executable without a console window.

12. The PACE software installer, if present, is run. If the version of the installer is not higher than the version installed in system, the installation is not performed.

When installing multiple plug-ins at once, PACE installation happens only after installing the final plug-in. VENUE will use the PACE installer with the highest available version among the installed plug-ins.

13. Plug-in installation is considered successful.

If errors occur during installation, the following happens:

1. Plug-in files are removed from the disk (see "File removal" section for details).
2. Cached plug-in installer is removed from the Cached VENUE Plug-in Installers location.

12.62.9.4.2 File removal File removal happens either in case of plug-in uninstallation or in case of a failed installation cleanup.

The following happens:

1. Plug-in files, as present in Cached VENUE Plug-in Installers location, are removed from Installed Plug-Ins location.
2. Plug-in Program Files files, as present in Cached VENUE Plug-in Installers location, are removed from Installed Plug-Ins location.

12.62.9.4.3 Plug-in uninstallation Plug-in installation process is done by VENUE Software. It removes a plug-in from the Installed Plugins location. Here's a step by step process of uninstalling plug-in:

1. The pre-uninstall batch script ("Uninstall_before.bat"), if present, is executed. Execution assumes running the script without a console window. NO return code is examined. Instead, a script is allowed to report an error string that will be visible to Avid support when examining VENUE log files. The following line should be used to write an error message from a script:

```
reg add HKCU\Software\Digidesign\tmp /f /v InstallResult /d "Error description"
```

The presence of this string means an error has occurred and a plug-in uninstallation will abort.

2. The pre-uninstall executable ("Uninstall_before.exe"), if present, is executed. Execution assumes running the executable without a console window. NO return code is examined. Instead, an executable is allowed to report an error string that will be visible to Avid support when examining VENUE log files. The error needs to be added to Windows registry as a REG_SZ value in *HKEY_CURRENT_USER\Software\Digidesign\tmp* and named *InstallResult*. The presence of this string means an error has occurred and a plug-in uninstallation will abort.
3. Plug-in files are removed. See [File removal](#) for details.

4. The post-uninstall batch script ("Install_after.bat"), if present, is executed. Execution assumes running the script without a console window. NO return code is examined. Instead, a script is allowed to report an error string that will be visible to Avid support when examining VENUE log files. The following line should be used to write an error message from a script:

```
reg add HKCU\Software\Digidesign\tmp /f /v InstallResult /d "Error description"
```

The presence of this string means an error has occurred and a plug-in uninstallation will abort.

5. The post-uninstall executable ("Install_after.exe"), if present, is executed. Execution assumes running the executable without a console window. NO return code is examined. Instead, an executable is allowed to report an error string that will be visible to Avid support when examining VENUE log files. The error needs to be added to Windows registry as a REG_SZ value in *HKEY_CURRENT_USER\Software\Digidesign\tmp* and named *InstallResult*. The presence of this string means an error has occurred and a plug-in uninstallation will abort.
6. Plug-in removal is complete.

Please note that plug-in being uninstalled is not being removed from the cache. Removal from the Cached VENUE Plug-in Installers is possible for plug-ins being not installed. In order to accomplish this, end user needs to go to VENUE Software's "Options">"Plug-Ins" page, right click on cached installer, and choose "Delete plug-in name".

12.62.10 Additional Information

12.62.10.1 Metering

For metering displays, VENUE uses dB units referenced to VENUE's nominal operating level of +4dBu. A signal at the nominal level in VENUE (i.e. registers 0dB on the VENUE meters) will, at unity gain, generate a +4dBu analog output signal (-20dBFS digital output signal).

As a result, a signal that registers +20dB on the VENUE meters will register 0dBFS on the plug-in meters. A signal at 0 dB in VENUE will be -20dBFS in the plug-in.

To map between dBFS units used in plug-ins and dB units used in VENUE the operator simply needs to add 20 to any plug-in dBFS value.

Collaboration diagram for VENUE Guide:

Chapter 13

Namespace Documentation

13.1 AAX Namespace Reference

Namespaces

- [Exception](#)
AAX exception classes
- [internal](#)

Enumerations

- enum [EStatusNibble](#) {
 [eStatusNibble_NoteOff](#) = 0x80 ,
 [eStatusNibble_NoteOn](#) = 0x90 ,
 [eStatusNibble_KeyPressure](#) = 0xA0 ,
 [eStatusNibble_ControlChange](#) = 0xB0 ,
 [eStatusNibble_ChannelMode](#) = 0xB0 ,
 [eStatusNibble_ProgramChange](#) = 0xC0 ,
 [eStatusNibble_ChannelPressure](#) = 0xD0 ,
 [eStatusNibble_PitchBend](#) = 0xE0 ,
 [eStatusNibble_SystemCommon](#) = 0xF0 ,
 [eStatusNibble_SystemRealTime](#) = 0xF0 }
Values for the status nibble in a MIDI packet.
- enum [EStatusByte](#) {
 [eStatusByte_SysExBegin](#) = 0xF0 ,
 [eStatusByte_MTCQuarterFrame](#) = 0xF1 ,
 [eStatusByte_SongPosition](#) = 0xF2 ,
 [eStatusByte_SongSelect](#) = 0xF3 ,
 [eStatusByte_TuneRequest](#) = 0xF6 ,
 [eStatusByte_SysExEnd](#) = 0xF7 ,
 [eStatusByte_TimingClock](#) = 0xF8 ,
 [eStatusByte_Start](#) = 0xFA ,
 [eStatusByte_Continue](#) = 0xFB ,
 [eStatusByte_Stop](#) = 0xFC ,
 [eStatusByte_ActiveSensing](#) = 0xFE ,
 [eStatusByte_Reset](#) = 0xFF }
Values for the status byte in a MIDI packet.

- enum [EChannelModeData](#) {
[eChannelModeData_AllSoundOff](#) = 120 ,
[eChannelModeData_ResetControllers](#) = 121 ,
[eChannelModeData_LocalControl](#) = 122 ,
[eChannelModeData_AllNotesOff](#) = 123 ,
[eChannelModeData_OmniOff](#) = 124 ,
[eChannelModeData_OmniOn](#) = 125 ,
[eChannelModeData_PolyOff](#) = 126 ,
[eChannelModeData_PolyOn](#) = 127 }
Values for the first data byte in a Channel Mode Message MIDI packet.
- enum [ESpecialData](#) {
[eSpecialData_AccentedClick](#) = 0x00 ,
[eSpecialData_UnaccentedClick](#) = 0x01 }
Special message data for the first data byte in a message.
- enum [ESampleRates](#) {
[e44100SampleRate](#) = 44100 ,
[e48000SampleRate](#) = 48000 ,
[e88200SampleRate](#) = 88200 ,
[e96000SampleRate](#) = 96000 ,
[e176400SampleRate](#) = 176400 ,
[e192000SampleRate](#) = 192000 }

Functions

- [std::string AsString](#) (const char *inStr)
- [const std::string & AsString](#) (const std::string &inStr)
- [const std::string & AsString](#) (const [Exception::Any](#) &inStr)
- [bool IsNoteOn](#) (const [AAX_CMidiPacket](#) *inPacket)
Returns true if inPacket is a Note On message.
- [bool IsNoteOff](#) (const [AAX_CMidiPacket](#) *inPacket)
Returns true if inPacket is a Note Off message, or a Note On message with velocity zero.
- [bool IsAllNotesOff](#) (const [AAX_CMidiPacket](#) *inPacket)
Returns true if inPacket is an All Sound Off or All Notes Off message.
- [bool IsAccentedClick](#) (const [AAX_CMidiPacket](#) *inPacket)
Returns true if inPacket is a special Pro Tools accented click message.
- [bool IsUnaccentedClick](#) (const [AAX_CMidiPacket](#) *inPacket)
Returns true if inPacket is a special Pro Tools unaccented click message.
- [bool IsClick](#) (const [AAX_CMidiPacket](#) *inPacket)
Returns true if inPacket is a special Pro Tools click message.
- [template<class T1 , class T2 >](#)
[bool PageTableParameterMappingsAreEqual](#) (const T1 &inL, const T2 &inR)
- [template<class T1 , class T2 >](#)
[bool PageTableParameterNameVariationsAreEqual](#) (const T1 &inL, const T2 &inR)
- [template<class T1 , class T2 >](#)
[bool PageTablesAreEqual](#) (const T1 &inL, const T2 &inR)
- [template<class T >](#)
[void CopyPageTable](#) (T &to, const T &from)
- [template<class T >](#)
[std::vector< std::pair< int32_t, int32_t > >](#) [FindParameterMappingsInPageTable](#) (const T &inTable, [AAX_CParamID](#) inParameterID)
- [template<class T >](#)
[void ClearMappedParameterByID](#) (T &ioTable, [AAX_CParamID](#) inParameterID)
- [void GetCStringOfLength](#) (char *stringOut, const char *stringIn, int32_t aMaxChars)

=====

- `int32_t Caseless_strcmp` (const char *cs, const char *ct)
- `std::string Binary2String` (uint32_t binaryValue, int32_t numBits)
- `uint32_t String2Binary` (const `AAX_IString` &s)
- `bool IsASCII` (char inChar)
- `bool IsFourCharASCII` (uint32_t inFourChar)
- `std::string AsStringFourChar` (uint32_t inFourChar)
- `std::string AsStringPropertyValue` (`AAX_EProperty` inProperty, `AAX_CPropertyValue` inPropertyValue)
- `std::string AsStringInt32` (int32_t inInt32)
- `std::string AsStringUInt32` (uint32_t inUInt32)
- `std::string AsStringIDTriad` (const `AAX_SPlugInIdentifierTriad` &inIDTriad)
- `std::string AsStringStemFormat` (`AAX_EStemFormat` inStemFormat, bool inAbbreviate=false)
- `std::string AsStringStemChannel` (`AAX_EStemFormat` inStemFormat, uint32_t inChannelIndex, bool inAbbreviate)
- `std::string AsStringResult` (`AAX_Result` inResult)
- `std::string AsStringSupportLevel` (`AAX_ESupportLevel` inSupportLevel)
- `double SafeLog` (double aValue)

Double-precision safe log function. Returns zero for input values that are <= 0.0.
- `float SafeLogf` (float aValue)

Single-precision safe log function. Returns zero for input values that are <= 0.0.
- `AAX_CBoolean IsParameterIDEqual` (`AAX_CParamID` iParam1, `AAX_CParamID` iParam2)

Helper function to check if two parameter IDs are equivalent.
- `AAX_CBoolean IsEffectIDEqual` (const `AAX_IString` *iEffectID1, const `AAX_IString` *iEffectID2)

Helper function to check if two Effect IDs are equivalent.
- `AAX_CBoolean IsAvidNotification` (`AAX_CTypeID` inNotificationID)

Helper function to check if a notification ID is reserved for host notifications.
- `void alignFree` (void *p)
- `template<class T >`
`T * alignMalloc` (int iArraySize, int iAlignment)
- `void DeDenormal` (double &iValue)

Clamps very small floating point values to zero.
- `void DeDenormal` (float &iValue)

Clamps very small floating point values to zero.
- `void DeDenormalFine` (float &iValue)
- `void FilterDenormals` (float *inSamples, int32_t inLength)

Round all denormal/subnormal samples in a buffer to zero.
- `template<class GFLOAT >`
`GFLOAT ClampToZero` (GFLOAT iValue, GFLOAT iClampThreshold)
- `void ZeroMemorySW` (void *iPointer, int iNumBytes)
- `void ZeroMemoryDW` (void *iPointer, int iNumBytes)
- `template<typename T, int N>`
`void Fill` (T *iArray, const T *iVal)
- `template<typename T, int M, int N>`
`void Fill` (T *iArray, const T *iVal)
- `template<typename T, int L, int M, int N>`
`void Fill` (T *iArray, const T *iVal)
- `double fabs` (double iVal)
- `float fabs` (float iVal)
- `float fabsf` (float iVal)
- `template<class T >`
`T AbsMax` (const T &iValue, const T &iMax)
- `template<class T >`
`T MinMax` (const T &iValue, const T &iMin, const T &iMax)
- `template<class T >`
`T Max` (const T &iValue1, const T &iValue2)

- `template<class T >`
`T Min (const T &iValue1, const T &iValue2)`
- `template<class T >`
`T Sign (const T &iValue)`
- `double PolyEval (double x, const double *coefs, int numCoefs)`
- `double CeilLog2 (double iValue)`
- `void SinCosMix (float aLinearMix, float &aSinMix, float &aCosMix)`
- `int32_t FastRound2Int32 (double iVal)`
Round to Int32.
- `int32_t FastRound2Int32 (float iVal)`
Round to Int32.
- `int32_t FastRndDbf2Int32 (double iVal)`
- `int32_t FastTrunc2Int32 (double iVal)`
Float to Int conversion with truncation.
- `int32_t FastTrunc2Int32 (float iVal)`
Float to Int conversion with truncation.
- `int64_t FastRound2Int64 (double iVal)`
Round to Int64.
- `int32_t GetInt32RPDF (int32_t *iSeed)`
- `int32_t GetFastInt32RPDF (int32_t *iSeed)`
CALL: Calculate pseudo-random 32 bit number based on linear congruential method.
- `float GetRPDFWithAmplitudeOneHalf (int32_t *iSeed)`
- `float GetRPDFWithAmplitudeOne (int32_t *iSeed)`
- `float GetFastRPDFWithAmplitudeOne (int32_t *iSeed)`
- `float GetTPDFWithAmplitudeOne (int32_t *iSeed)`

MIDI logging utilities

- `void AsStringMIDIStream_Debug (const AAX_CMidiStream &inStream, char *outBuffer, int32_t inBuffer↵
Size)`

Filesystem utilities

- `bool GetPathToPlugInBundle (const char *iBundleName, int iMaxLength, char *oModuleName)`
Retrieve the file path of the .aaxplugin bundle.

Variables

- `const int cBigEndian =0`
- `const int cLittleEndian =1`
- `const double cPi = 3.1415926535897932384626433832795`
- `const double cTwoPi = 6.2831853071795862319959269370884`
- `const double cHalfPi = 1.5707963267948965579989817342721`
- `const double cQuarterPi = 0.78539816339744827899949086713605`
- `const double cRootTwo = 1.4142135623730950488016887242097`
- `const double cOneOverRootTwo = 0.70710678118654752440084436210485`
- `const double cPos3dB =1.4142135623730950488016887242097`
- `const double cNeg3dB =0.70710678118654752440084436210485`
- `const double cPos6dB =2.0`
- `const double cNeg6dB =0.5`
- `const double cNormalizeLongToAmplitudeOneHalf = 0.00000000023283064365386962890625`
- `const double cNormalizeLongToAmplitudeOne = 1.0/double(1<<31)`

- const double `cMilli` =0.001
- const double `cMicro` =0.001*0.001
- const double `cNano` =0.001*0.001*0.001
- const double `cPico` =0.001*0.001*0.001*0.001
- const double `cKilo` =1000.0
- const double `cMega` =1000.0*1000.0
- const double `cGiga` =1000.0*1000.0*1000.0
- const double `cDenormalAvoidanceOffset` =3.0e-34
- const float `cFloatDenormalAvoidanceOffset` =3.0e-20f
- const unsigned int `kPowExtent` = 9
- const unsigned int `kPowTableSize` = 1 << `kPowExtent`
- const float `cSeedDivisor` = 1/127773.0f
- const int32_t `clInitialSeedValue` =0x00F54321

13.1.1 Enumeration Type Documentation

13.1.1.1 EStatusNibble

enum `AAX::EStatusNibble`

Values for the status nibble in a MIDI packet.

Enumerator

<code>eStatusNibble_NoteOff</code>	
<code>eStatusNibble_NoteOn</code>	
<code>eStatusNibble_KeyPressure</code>	
<code>eStatusNibble_ControlChange</code>	
<code>eStatusNibble_ChannelMode</code>	
<code>eStatusNibble_ProgramChange</code>	
<code>eStatusNibble_ChannelPressure</code>	
<code>eStatusNibble_PitchBend</code>	
<code>eStatusNibble_SystemCommon</code>	
<code>eStatusNibble_SystemRealTime</code>	

13.1.1.2 EStatusByte

enum `AAX::EStatusByte`

Values for the status byte in a MIDI packet.

Enumerator

<code>eStatusByte_SysExBegin</code>	
-------------------------------------	--

Enumerator

eStatusByte_MTCQuarterFrame	
eStatusByte_SongPosition	
eStatusByte_SongSelect	
eStatusByte_TuneRequest	
eStatusByte_SysExEnd	
eStatusByte_TimingClock	
eStatusByte_Start	
eStatusByte_Continue	
eStatusByte_Stop	
eStatusByte_ActiveSensing	
eStatusByte_Reset	

13.1.1.3 EChannelModeData

enum [AAX::EChannelModeData](#)

Values for the first data byte in a Channel Mode Message MIDI packet.

Enumerator

eChannelModeData_AllSoundOff	
eChannelModeData_ResetControllers	
eChannelModeData_LocalControl	
eChannelModeData_AllNotesOff	
eChannelModeData_OmniOff	
eChannelModeData_OmniOn	
eChannelModeData_PolyOff	
eChannelModeData_PolyOn	

13.1.1.4 ESpecialData

enum [AAX::ESpecialData](#)

Special message data for the first data byte in a message.

Enumerator

eSpecialData_AccentedClick	For use when the high status nibble is eStatusNibble_NoteOn and the low status nibble is zero.
eSpecialData_UnaccentedClick	For use when the high status nibble is eStatusNibble_NoteOn and the low status nibble is zero.

13.1.1.5 ESsampleRates

```
enum AAX::ESampleRates
```

Enumerator

e44100SampleRate	
e48000SampleRate	
e88200SampleRate	
e96000SampleRate	
e176400SampleRate	
e192000SampleRate	

13.1.2 Function Documentation

13.1.2.1 AsString() [1/3]

```
std::string AAX::AsString (
    const char * inStr ) [inline]
```

Generic conversion of a string-like object to a std::string

13.1.2.2 AsString() [2/3]

```
const std::string & AAX::AsString (
    const std::string & inStr ) [inline]
```

Generic conversion of a string-like object to a std::string

13.1.2.3 AsString() [3/3]

```
const std::string & AAX::AsString (
    const Exception::Any & inStr ) [inline]
```

Generic conversion of a string-like object to a std::string

References AAX::Exception::Any::What().

Here is the call graph for this function:

13.1.2.4 IsNoteOn()

```
bool AAX::IsNoteOn (
    const AAX_CMidiPacket * inPacket ) [inline]
```

Returns true if `inPacket` is a Note On message.

References `eStatusNibble_NoteOn`, and `AAX_CMidiPacket::mData`.

13.1.2.5 IsNoteOff()

```
bool AAX::IsNoteOff (
    const AAX_CMidiPacket * inPacket ) [inline]
```

Returns true if `inPacket` is a Note Off message, or a Note On message with velocity zero.

References `eStatusNibble_NoteOff`, `eStatusNibble_NoteOn`, and `AAX_CMidiPacket::mData`.

13.1.2.6 IsAllNotesOff()

```
bool AAX::IsAllNotesOff (
    const AAX_CMidiPacket * inPacket ) [inline]
```

Returns true if `inPacket` is an All Sound Off or All Notes Off message.

References `eChannelModeData_AllNotesOff`, `eChannelModeData_AllSoundOff`, `eChannelModeData_OmniOff`, `eChannelModeData_OmniOn`, `eChannelModeData_PolyOff`, `eChannelModeData_PolyOn`, `eStatusNibble_ChannelMode`, and `AAX_CMidiPacket::mData`.

13.1.2.7 IsAccentedClick()

```
bool AAX::IsAccentedClick (
    const AAX_CMidiPacket * inPacket ) [inline]
```

Returns true if `inPacket` is a special Pro Tools accented click message.

References `eSpecialData_AccentedClick`, `eStatusNibble_NoteOn`, and `AAX_CMidiPacket::mData`.

Referenced by `IsClick()`.

Here is the caller graph for this function:

13.1.2.8 IsUnaccentedClick()

```
bool AAX::IsUnaccentedClick (
    const AAX_CMidiPacket * inPacket ) [inline]
```

Returns true if `inPacket` is a special Pro Tools unaccented click message.

References `eSpecialData_UnaccentedClick`, `eStatusNibble_NoteOn`, and `AAX_CMidiPacket::mData`.

Referenced by `IsClick()`.

Here is the caller graph for this function:

13.1.2.9 IsClick()

```
bool AAX::IsClick (
    const AAX_CMidiPacket * inPacket ) [inline]
```

Returns true if `inPacket` is a special Pro Tools click message.

References `IsAccentedClick()`, and `IsUnaccentedClick()`.

Here is the call graph for this function:

13.1.2.10 PageTableParameterMappingsAreEqual()

```
template<class T1 , class T2 >
bool AAX::PageTableParameterMappingsAreEqual (
    const T1 & inL,
    const T2 & inR ) [inline]
```

Compare the parameter mappings in two page tables

T1 and T2 : Page table class types (e.g. [AAX_IACFPageTable](#), [AAX_IPageTable](#))

References `AAX_SUCCESS`.

Referenced by `PageTablesAreEqual()`.

Here is the caller graph for this function:

13.1.2.11 PageTableParameterNameVariationsAreEqual()

```
template<class T1 , class T2 >
bool AAX::PageTableParameterNameVariationsAreEqual (
    const T1 & inL,
    const T2 & inR ) [inline]
```

References `AAX_SUCCESS`, and `AAX_CString::Get()`.

Referenced by `PageTablesAreEqual()`.

Here is the call graph for this function: Here is the caller graph for this function:

13.1.2.12 PageTablesAreEqual()

```
template<class T1 , class T2 >
bool AAX::PageTablesAreEqual (
    const T1 & inL,
    const T2 & inR ) [inline]
```

References [PageTableParameterMappingsAreEqual\(\)](#), and [PageTableParameterNameVariationsAreEqual\(\)](#).

Here is the call graph for this function:

13.1.2.13 CopyPageTable()

```
template<class T >
void AAX::CopyPageTable (
    T & to,
    const T & from ) [inline]
```

Copy a page table

T: A page table class type (e.g. [AAX_IACFPageTable](#), [AAX_IPageTable](#))

References [AAX_SUCCESS](#), [AAX_CString::CString\(\)](#), and [AAX_CString::Get\(\)](#).

Here is the call graph for this function:

13.1.2.14 FindParameterMappingsInPageTable()

```
template<class T >
std::vector<std::pair<int32_t, int32_t> > AAX::FindParameterMappingsInPageTable (
    const T & inTable,
    AAX\_CParamID inParameterID ) [inline]
```

Find all slots where a particular parameter is mapped

T: A page table class type (e.g. [AAX_IACFPageTable](#), [AAX_IPageTable](#))

Returns

A vector of pairs of [page index, slot index] each representing a single mapping of the parameter

References [AAX_SUCCESS](#).

Referenced by [ClearMappedParameterByID\(\)](#).

Here is the caller graph for this function:

13.1.2.15 ClearMappedParameterByID()

```
template<class T >
void AAX::ClearMappedParameterByID (
    T & ioTable,
    AAX_CParamID inParameterID ) [inline]
```

Remove all mappings of a particular from a page table

T: A page table class type (e.g. [AAX_IACFPageTable](#), [AAX_IPageTable](#))

References [FindParameterMappingsInPageTable\(\)](#).

Here is the call graph for this function:

13.1.2.16 GetCStringOfLength()

```
void AAX::GetCStringOfLength (
    char * stringOut,
    const char * stringIn,
    int32_t aMaxChars ) [inline]
```

=====

References [AAX_ASSERT](#).

13.1.2.17 Caseless_strcmp()

```
int32_t AAX::Caseless_strcmp (
    const char * cs,
    const char * ct ) [inline]
```

13.1.2.18 Binary2String()

```
std::string AAX::Binary2String (
    uint32_t binaryValue,
    int32_t numBits ) [inline]
```

Referenced by [AsStringPropertyValue\(\)](#).

Here is the caller graph for this function:

13.1.2.19 String2Binary()

```
uint32_t AAX::String2Binary (
    const AAX\_IString & s ) [inline]
```

References [AAX_ASSERT](#), [AAX_IString::Get\(\)](#), and [AAX_IString::Length\(\)](#).

Here is the call graph for this function:

13.1.2.20 IsASCII()

```
bool AAX::IsASCII (
    char inChar ) [inline]
```

Referenced by AsStringFourChar(), and IsFourCharASCII().

Here is the caller graph for this function:

13.1.2.21 IsFourCharASCII()

```
bool AAX::IsFourCharASCII (
    uint32_t inFourChar ) [inline]
```

References IsASCII().

Referenced by AsStringPropertyValue().

Here is the call graph for this function: Here is the caller graph for this function:

13.1.2.22 AsStringFourChar()

```
std::string AAX::AsStringFourChar (
    uint32_t inFourChar ) [inline]
```

References AAX_CONSTEXPR, and IsASCII().

Referenced by AsStringIDTriad(), and AsStringPropertyValue().

Here is the call graph for this function: Here is the caller graph for this function:

13.1.2.23 AsStringPropertyValue()

```
std::string AAX::AsStringPropertyValue (
    AAX_EProperty inProperty,
    AAX_CPropertyValue inPropertyValue ) [inline]
```

References AAX_CONSTEXPR, AAX_eProperty_Constraint_Location, AAX_eProperty_SampleRate, AsStringFourChar(), Binary2String(), IsFourCharASCII(), and AAX::internal::ToHexadecimal().

Here is the call graph for this function:

13.1.2.24 AsStringInt32()

```
std::string AAX::AsStringInt32 (
    int32_t inInt32 ) [inline]
```

Referenced by AAX::Exception::ResultError::FormatResult().

Here is the caller graph for this function:

13.1.2.25 AsStringUInt32()

```
std::string AAX::AsStringUInt32 (
    uint32_t inUInt32 ) [inline]
```

13.1.2.26 AsStringIDTriad()

```
std::string AAX::AsStringIDTriad (
    const AAX_SPlugInIdentifierTriad & inIDTriad ) [inline]
```

References [AsStringFourChar\(\)](#), [AAX_SPlugInIdentifierTriad::mManufacturerID](#), [AAX_SPlugInIdentifierTriad::mPlugInID](#), and [AAX_SPlugInIdentifierTriad::mProductID](#).

Here is the call graph for this function:

13.1.2.27 AsStringStemFormat()

```
std::string AAX::AsStringStemFormat (
    AAX_EStemFormat inStemFormat,
    bool inAbbreviate = false ) [inline]
```

References [AAX_eStemFormat_5_0](#), [AAX_eStemFormat_5_0_2](#), [AAX_eStemFormat_5_0_4](#), [AAX_eStemFormat_5_1](#), [AAX_eStemFormat_5_1_2](#), [AAX_eStemFormat_5_1_4](#), [AAX_eStemFormat_6_0](#), [AAX_eStemFormat_6_1](#), [AAX_eStemFormat_7_0_2](#), [AAX_eStemFormat_7_0_4](#), [AAX_eStemFormat_7_0_6](#), [AAX_eStemFormat_7_0_DTS](#), [AAX_eStemFormat_7_0_SDDS](#), [AAX_eStemFormat_7_1_2](#), [AAX_eStemFormat_7_1_4](#), [AAX_eStemFormat_7_1_6](#), [AAX_eStemFormat_7_1_DTS](#), [AAX_eStemFormat_7_1_SDDS](#), [AAX_eStemFormat_9_0_4](#), [AAX_eStemFormat_9_0_6](#), [AAX_eStemFormat_9_1_4](#), [AAX_eStemFormat_9_1_6](#), [AAX_eStemFormat_Ambi_1_ACN](#), [AAX_eStemFormat_Ambi_2_ACN](#), [AAX_eStemFormat_Ambi_3_ACN](#), [AAX_eStemFormat_Ambi_4_ACN](#), [AAX_eStemFormat_Ambi_5_ACN](#), [AAX_eStemFormat_Ambi_6_ACN](#), [AAX_eStemFormat_Ambi_7_ACN](#), [AAX_eStemFormat_Any](#), [AAX_eStemFormat_INT32_MAX](#), [AAX_eStemFormat_LCR](#), [AAX_eStemFormat_LCRS](#), [AAX_eStemFormat_Mono](#), [AAX_eStemFormat_None](#), [AAX_eStemFormat_Quad](#), [AAX_eStemFormat_Stereo](#), and [AAX_eStemFormatNum](#).

13.1.2.28 AsStringStemChannel()

```
std::string AAX::AsStringStemChannel (
    AAX_EStemFormat inStemFormat,
    uint32_t inChannelIndex,
    bool inAbbreviate ) [inline]
```

References [AAX_eStemFormat_5_0](#), [AAX_eStemFormat_5_0_2](#), [AAX_eStemFormat_5_0_4](#), [AAX_eStemFormat_5_1](#), [AAX_eStemFormat_5_1_2](#), [AAX_eStemFormat_5_1_4](#), [AAX_eStemFormat_6_0](#), [AAX_eStemFormat_6_1](#), [AAX_eStemFormat_7_0_2](#), [AAX_eStemFormat_7_0_4](#), [AAX_eStemFormat_7_0_6](#), [AAX_eStemFormat_7_0_DTS](#), [AAX_eStemFormat_7_0_SDDS](#), [AAX_eStemFormat_7_1_2](#), [AAX_eStemFormat_7_1_4](#), [AAX_eStemFormat_7_1_6](#), [AAX_eStemFormat_7_1_DTS](#), [AAX_eStemFormat_7_1_SDDS](#), [AAX_eStemFormat_9_0_4](#), [AAX_eStemFormat_9_0_6](#), [AAX_eStemFormat_9_1_4](#), [AAX_eStemFormat_9_1_6](#), [AAX_eStemFormat_Ambi_1_ACN](#), [AAX_eStemFormat_Ambi_2_ACN](#), [AAX_eStemFormat_Ambi_3_ACN](#), [AAX_eStemFormat_Ambi_4_ACN](#), [AAX_eStemFormat_Ambi_5_ACN](#), [AAX_eStemFormat_Ambi_6_ACN](#), [AAX_eStemFormat_Ambi_7_ACN](#), [AAX_eStemFormat_Any](#), [AAX_eStemFormat_INT32_MAX](#), [AAX_eStemFormat_LCR](#), [AAX_eStemFormat_LCRS](#), [AAX_eStemFormat_Mono](#), [AAX_eStemFormat_None](#), [AAX_eStemFormat_Quad](#), [AAX_eStemFormat_Stereo](#), and [AAX_eStemFormatNum](#).

13.1.2.29 AsStringResult()

```
std::string AAX::AsStringResult (
    AAX_Result inResult ) [inline]
```

References AAX_ERROR_ACF_ERROR, AAX_ERROR_ARGUMENT_BUFFER_OVERFLOW, AAX_ERROR_↵
CONTEXT_ALREADY_HAS_METERS, AAX_ERROR_DIRECT_ACCESS_OUT_OF_BOUNDS, AAX_ERROR_↵
_DUPLICATE_EFFECT_ID, AAX_ERROR_DUPLICATE_TYPE_ID, AAX_ERROR_EMPTY_EFFECT_NAME,↵
AAX_ERROR_FIELD_TYPE_DOES_NOT_SUPPORT_DIRECT_ACCESS, AAX_ERROR_FIFO_FULL, AAX_↵
_ERROR_INCORRECT_CHUNK_SIZE, AAX_ERROR_INITIALIZING_PACKET_STREAM_THREAD, AAX_↵
_ERROR_INVALID_ARGUMENT, AAX_ERROR_INVALID_CHUNK_ID, AAX_ERROR_INVALID_CHUNK_↵
INDEX, AAX_ERROR_INVALID_FIELD_INDEX, AAX_ERROR_INVALID_INTERNAL_DATA, AAX_ERROR_↵
INVALID_METER_INDEX, AAX_ERROR_INVALID_METER_TYPE, AAX_ERROR_INVALID_PARAMETER_↵
ID, AAX_ERROR_INVALID_PARAMETER_INDEX, AAX_ERROR_INVALID_PATH, AAX_ERROR_INVALID_↵
STRING_CONVERSION, AAX_ERROR_INVALID_VIEW_SIZE, AAX_ERROR_MALFORMED_CHUNK, AAX_↵
_ERROR_MIXER_THREAD_FALLING_BEHIND, AAX_ERROR_NO_COMPONENTS, AAX_ERROR_NOT_↵
_INITIALIZED, AAX_ERROR_NOTIFICATION_FAILED, AAX_ERROR_NULL_ARGUMENT, AAX_ERROR_↵
_NULL_COMPONENT, AAX_ERROR_NULL_OBJECT, AAX_ERROR_OLDER_VERSION, AAX_ERROR_↵
PLUGIN_BEGIN, AAX_ERROR_PLUGIN_END, AAX_ERROR_PLUGIN_NOT_AUTHORIZED, AAX_ERROR_↵
_PLUGIN_NULL_PARAMETER, AAX_ERROR_PORT_ID_OUT_OF_RANGE, AAX_ERROR_POST_PACKET_↵
_FAILED, AAX_ERROR_PROPERTY_UNDEFINED, AAX_ERROR_SIGNED_INT_OVERFLOW, AAX_ERROR_↵
_TOD_BEHIND, AAX_ERROR_UNIMPLEMENTED, AAX_ERROR_UNKNOWN_EXCEPTION, AAX_ERROR_↵
_UNKNOWN_ID, AAX_ERROR_UNKNOWN_PLUGIN, AAX_RESULT_ADD_FIELD_UNSUPPORTED_FIELD_↵
_TYPE, AAX_RESULT_NEW_PACKET_POSTED, AAX_RESULT_PACKET_STREAM_NOT_EMPTY, AAX_↵
SUCCESS, and DEFINE_AAX_ERROR_STRING.

Referenced by AAX::Exception::ResultError::FormatResult().

Here is the caller graph for this function:

13.1.2.30 AsStringSupportLevel()

```
std::string AAX::AsStringSupportLevel (
    AAX_ESupportLevel inSupportLevel ) [inline]
```

References AAX_eSupportLevel_ByProperty, AAX_eSupportLevel_Disabled, AAX_eSupportLevel_Supported, AAX_eSupportLevel_Uninitialized, and AAX_eSupportLevel_Unsupported.

13.1.2.31 SafeLog()

```
double AAX::SafeLog (
    double aValue ) [inline]
```

Double-precision safe log function. Returns zero for input values that are ≤ 0.0 .

Referenced by AAX_CLogTaperDelegate< T, RealPrecision >::NormalizedToReal(), and AAX_CLogTaper_↵
Delegate< T, RealPrecision >::RealToNormalized().

Here is the caller graph for this function:

13.1.2.32 SafeLogf()

```
float AAX::SafeLogf (
    float aValue ) [inline]
```

Single-precision safe log function. Returns zero for input values that are ≤ 0.0 .

13.1.2.33 IsParameterIDEqual()

```
AAX_CBoolean AAX::IsParameterIDEqual (
    AAX_CParamID iParam1,
    AAX_CParamID iParam2 ) [inline]
```

Helper function to check if two parameter IDs are equivalent.

13.1.2.34 IsEffectIDEqual()

```
AAX_CBoolean AAX::IsEffectIDEqual (
    const AAX_IString * iEffectID1,
    const AAX_IString * iEffectID2 ) [inline]
```

Helper function to check if two Effect IDs are equivalent.

References AAX_IString::Get().

Here is the call graph for this function:

13.1.2.35 IsAvidNotification()

```
AAX_CBoolean AAX::IsAvidNotification (
    AAX_CTypeID inNotificationID ) [inline]
```

Helper function to check if a notification ID is reserved for host notifications.

13.1.2.36 alignFree()

```
void AAX::alignFree (
    void * p ) [inline]
```

13.1.2.37 alignMalloc()

```
template<class T >
T* AAX::alignMalloc (
    int iArraySize,
    int iAlignment )
```

13.1.2.38 DeDenormal() [1/2]

```
void AAX::DeDenormal (
    double & iValue ) [inline]
```

Clamps very small floating point values to zero.

On Pentiums and Pentium IIs the generation of denormal floats causes enormous performance losses. This routine removes denormals by clamping very small values to zero. The clamping threshold is very small, but is not the absolute minimum. If absolute minimum clamping is desired, use [AAX::DeDenormalFine\(\)](#)

References cDenormalAvoidanceOffset.

13.1.2.39 DeDenormal() [2/2]

```
void AAX::DeDenormal (
    float & iValue ) [inline]
```

Clamps very small floating point values to zero.

On Pentiums and Pentium IIs the generation of denormal floats causes enormous performance losses. This routine removes denormals by clamping very small values to zero. The clamping threshold is very small, but is not the absolute minimum. If absolute minimum clamping is desired, use [AAX::DeDenormalFine\(\)](#)

References cFloatDenormalAvoidanceOffset.

13.1.2.40 DeDenormalFine()

```
void AAX::DeDenormalFine (
    float & iValue ) [inline]
```

Similar to [AAX::DeDenormal\(\)](#), but uses the minimum possible normal float value as the clamping threshold

13.1.2.41 FilterDenormals()

```
void AAX::FilterDenormals (
    float * inSamples,
    int32_t inLength ) [inline]
```

Round all denormal/subnormal samples in a buffer to zero.

Parameters

in	<i>inSamples</i>	Samples to convert
in	<i>inLength</i>	Number of samples in inSamples

References fabsf().

Here is the call graph for this function:

13.1.2.42 ClampToZero()

```
template<class GFLOAT >
GFLOAT AAX::ClampToZero (
    GFLOAT iValue,
    GFLOAT iClampThreshold ) [inline]
```

13.1.2.43 ZeroMemorySW()

```
void AAX::ZeroMemorySW (
    void * iPointer,
    int iNumBytes ) [inline]
```

13.1.2.44 ZeroMemoryDW()

```
void AAX::ZeroMemoryDW (
    void * iPointer,
    int iNumBytes ) [inline]
```

13.1.2.45 Fill() [1/3]

```
template<typename T , int N>
void AAX::Fill (
    T * iArray,
    const T * iVal )
```

Referenced by Fill().

Here is the caller graph for this function:

13.1.2.46 Fill() [2/3]

```
template<typename T , int M, int N>
void AAX::Fill (
    T * iArray,
    const T * iVal ) [inline]
```

References Fill().

Here is the call graph for this function:

13.1.2.47 Fill() [3/3]

```
template<typename T , int L, int M, int N>
void AAX::Fill (
    T * iArray,
    const T * iVal ) [inline]
```

References Fill().

Here is the call graph for this function:

13.1.2.48 fabs() [1/2]

```
double AAX::fabs (
    double iVal ) [inline]
```

References fabs().

Referenced by fabsf().

Here is the call graph for this function: Here is the caller graph for this function:

13.1.2.49 fabs() [2/2]

```
float AAX::fabs (
    float iVal ) [inline]
```

Referenced by AbsMax(), and fabs().

Here is the caller graph for this function:

13.1.2.50 fabsf()

```
float AAX::fabsf (
    float iVal ) [inline]
```

References fabs().

Referenced by FilterDenormals(), and AAX_CUnitPrefixDisplayDelegateDecorator< T >::ValueToString().

Here is the call graph for this function: Here is the caller graph for this function:

13.1.2.51 AbsMax()

```
template<class T >
T AAX::AbsMax (
    const T & iValue,
    const T & iMax ) [inline]
```

References fabs().

Here is the call graph for this function:

13.1.2.52 MinMax()

```
template<class T >
T AAX::MinMax (
    const T & iValue,
    const T & iMin,
    const T & iMax ) [inline]
```

13.1.2.53 Max()

```
template<class T >
T AAX::Max (
    const T & iValue1,
    const T & iValue2 ) [inline]
```

13.1.2.54 Min()

```
template<class T >
T AAX::Min (
    const T & iValue1,
    const T & iValue2 ) [inline]
```

13.1.2.55 Sign()

```
template<class T >
T AAX::Sign (
    const T & iValue ) [inline]
```

13.1.2.56 PolyEval()

```
double AAX::PolyEval (
    double x,
    const double * coefs,
    int numCoefs ) [inline]
```

13.1.2.57 CeilLog2()

```
double AAX::CeilLog2 (
    double iValue ) [inline]
```

13.1.2.58 SinCosMix()

```
void AAX::SinCosMix (
    float aLinearMix,
    float & aSinMix,
    float & aCosMix ) [inline]
```

References cHalfPi.

13.1.2.59 FastRound2Int32() [1/2]

```
int32_t AAX::FastRound2Int32 (
    double iVal ) [inline]
```

Round to Int32.

Parameters

in	<i>iVal</i>	Value to convert
----	-------------	------------------

Referenced by FastRndDbl2Int32(), FastRound2Int32(), and FastTrunc2Int32().

Here is the caller graph for this function:

13.1.2.60 FastRound2Int32() [2/2]

```
int32_t AAX::FastRound2Int32 (
    float iVal ) [inline]
```

Round to Int32.

Parameters

in	<i>iVal</i>	Value to convert
----	-------------	------------------

References `FastRound2Int32()`.

Here is the call graph for this function:

13.1.2.61 FastRndDb12Int32()

```
int32_t AAX::FastRndDb12Int32 (
    double iVal ) [inline]
```

Deprecated

References `FastRound2Int32()`.

Here is the call graph for this function:

13.1.2.62 FastTrunc2Int32() [1/2]

```
int32_t AAX::FastTrunc2Int32 (
    double iVal ) [inline]
```

Float to Int conversion with truncation.

Parameters

in	<i>iVal</i>	Value to convert
----	-------------	------------------

Note

This truncation is NOT identical to C style casting. Because the Intel (and I would assume PowerPC) processors use convergent rounding by default, exactly whole odd numbers will truncate down by 1.0 (e.g. 0.0->0, 1.0->0, 2.0->2, 3.0->2). Surprisingly, even with these limitations this fast float to int conversion is often very useful in practice, as long as one is aware of these issues.

References `FastRound2Int32()`.

Referenced by `AAX_FastInterpolatedTableLookup< TFLOAT, DFLOAT >::DoTableLookupExtraFast()`, and `AAX_FastInterpolatedTableLookup< TFLOAT, DFLOAT >::DoTableLookupExtraFastMulti()`.

Here is the call graph for this function: Here is the caller graph for this function:

13.1.2.63 FastTrunc2Int32() [2/2]

```
int32_t AAX::FastTrunc2Int32 (
    float iVal ) [inline]
```

Float to Int conversion with truncation.

Parameters

in	<i>iVal</i>	Value to convert
----	-------------	------------------

13.1.2.64 FastRound2Int64()

```
int64_t AAX::FastRound2Int64 (
    double iVal ) [inline]
```

Round to Int64.

Taken from Paul V's implementation in Sys_VecUtils. This only works on values smaller than 2^{52} .

Parameters

in	<i>iVal</i>	Value to convert
----	-------------	------------------

13.1.2.65 GetInt32RPDF()

```
int32_t AAX::GetInt32RPDF (
    int32_t * iSeed ) [inline]
```

References cSeedDivisor.

Referenced by GetRPDFWithAmplitudeOne(), and GetRPDFWithAmplitudeOneHalf().

Here is the caller graph for this function:

13.1.2.66 GetFastInt32RPDF()

```
int32_t AAX::GetFastInt32RPDF (
    int32_t * iSeed ) [inline]
```

CALL: Calculate pseudo-random 32 bit number based on linear congruential method.

This is required if you want our master bypass functionality in the host to hook up to your bypass parameters.

Parameters

in	<i>iSeed</i>	Seed for random generator
----	--------------	---------------------------

Note

This method produces lower quality random numbers (i.e. less random) than plain old GetInt32RPDF, but in many cases it should be plenty good.

Referenced by `GetFastRPDFWithAmplitudeOne()`, and `GetTPDFWithAmplitudeOne()`.

Here is the caller graph for this function:

13.1.2.67 `GetRPDFWithAmplitudeOneHalf()`

```
float AAX::GetRPDFWithAmplitudeOneHalf (
    int32_t * iSeed ) [inline]
```

References `cNormalizeLongToAmplitudeOneHalf`, and `GetInt32RPDF()`.

Here is the call graph for this function:

13.1.2.68 `GetRPDFWithAmplitudeOne()`

```
float AAX::GetRPDFWithAmplitudeOne (
    int32_t * iSeed ) [inline]
```

References `cNormalizeLongToAmplitudeOne`, and `GetInt32RPDF()`.

Here is the call graph for this function:

13.1.2.69 `GetFastRPDFWithAmplitudeOne()`

```
float AAX::GetFastRPDFWithAmplitudeOne (
    int32_t * iSeed ) [inline]
```

References `cNormalizeLongToAmplitudeOne`, and `GetFastInt32RPDF()`.

Here is the call graph for this function:

13.1.2.70 `GetTPDFWithAmplitudeOne()`

```
float AAX::GetTPDFWithAmplitudeOne (
    int32_t * iSeed ) [inline]
```

References `cNormalizeLongToAmplitudeOne`, and `GetFastInt32RPDF()`.

Here is the call graph for this function:

13.1.3 Variable Documentation

13.1.3.1 `cBigEndian`

```
const int AAX::cBigEndian =0
```

13.1.3.2 cLittleEndian

```
const int AAX::cLittleEndian =1
```

13.1.3.3 cPi

```
const double AAX::cPi = 3.1415926535897932384626433832795
```

13.1.3.4 cTwoPi

```
const double AAX::cTwoPi = 6.2831853071795862319959269370884
```

13.1.3.5 cHalfPi

```
const double AAX::cHalfPi = 1.5707963267948965579989817342721
```

Referenced by SinCosMix().

13.1.3.6 cQuarterPi

```
const double AAX::cQuarterPi = 0.78539816339744827899949086713605
```

13.1.3.7 cRootTwo

```
const double AAX::cRootTwo = 1.4142135623730950488016887242097
```

13.1.3.8 cOneOverRootTwo

```
const double AAX::cOneOverRootTwo = 0.70710678118654752440084436210485
```

13.1.3.9 cPos3dB

```
const double AAX::cPos3dB =1.4142135623730950488016887242097
```

13.1.3.10 cNeg3dB

```
const double AAX::cNeg3dB =0.70710678118654752440084436210485
```

13.1.3.11 cPos6dB

```
const double AAX::cPos6dB =2.0
```

13.1.3.12 cNeg6dB

```
const double AAX::cNeg6dB =0.5
```

13.1.3.13 cNormalizeLongToAmplitudeOneHalf

```
const double AAX::cNormalizeLongToAmplitudeOneHalf = 0.00000000023283064365386962890625
```

Referenced by `GetRPDFWithAmplitudeOneHalf()`.

13.1.3.14 cNormalizeLongToAmplitudeOne

```
const double AAX::cNormalizeLongToAmplitudeOne = 1.0/double(1<<31)
```

Referenced by `GetFastRPDFWithAmplitudeOne()`, `GetRPDFWithAmplitudeOne()`, and `GetTPDFWithAmplitudeOne()`.

13.1.3.15 cMilli

```
const double AAX::cMilli =0.001
```

13.1.3.16 cMicro

```
const double AAX::cMicro =0.001*0.001
```

13.1.3.17 cNano

```
const double AAX::cNano =0.001*0.001*0.001
```

13.1.3.18 cPico

```
const double AAX::cPico =0.001*0.001*0.001*0.001
```

13.1.3.19 cKilo

```
const double AAX::cKilo =1000.0
```

13.1.3.20 cMega

```
const double AAX::cMega =1000.0*1000.0
```

13.1.3.21 cGiga

```
const double AAX::cGiga =1000.0*1000.0*1000.0
```

13.1.3.22 cDenormalAvoidanceOffset

```
const double AAX::cDenormalAvoidanceOffset =3.0e-34
```

Referenced by `DeDenormal()`.

13.1.3.23 cFloatDenormalAvoidanceOffset

```
const float AAX::cFloatDenormalAvoidanceOffset = 3.0e-20f
```

Referenced by DeDenormal().

13.1.3.24 kPowExtent

```
const unsigned int AAX::kPowExtent = 9
```

13.1.3.25 kPowTableSize

```
const unsigned int AAX::kPowTableSize = 1 << kPowExtent
```

13.1.3.26 cSeedDivisor

```
const float AAX::cSeedDivisor = 1/127773.0f
```

Referenced by GetInt32RPDF().

13.1.3.27 cInitialSeedValue

```
const int32_t AAX::cInitialSeedValue = 0x00F54321
```

13.2 AAX::Exception Namespace Reference

13.2.1 Description

AAX exception classes

All AAX exception classes inherit from [AAX::Exception::Any](#)

Classes

- class [Any](#)
- class [ResultError](#)

13.3 AAX::internal Namespace Reference

Functions

- `template<typename T>`
`std::string ToHexadecimal (T inValue, bool inLeadingZeros=false)`

13.3.1 Function Documentation

13.3.1.1 ToHexadecimal()

```
template<typename T >
std::string AAX::internal::ToHexadecimal (
    T inValue,
    bool inLeadingZeros = false )
```

References AAX_CONSTEXPR.

Referenced by AAX::AsStringPropertyValue().

Here is the caller graph for this function:

13.4 AAX_ChunkDataParserDefs Namespace Reference

13.4.1 Description

Constants used by ChunkDataParser class.

Variables

- `const int32_t FLOAT_TYPE = 1`
- `const char FLOAT_STRING_IDENTIFIER [] = "f_"`
- `const int32_t LONG_TYPE = 2`
- `const char LONG_STRING_IDENTIFIER [] = "l_"`
- `const int32_t DOUBLE_TYPE = 3`
- `const char DOUBLE_STRING_IDENTIFIER [] = "d_"`
- `const size_t DOUBLE_TYPE_SIZE = 8`
- `const size_t DOUBLE_TYPE_INCR = 8`
- `const int32_t SHORT_TYPE = 4`
- `const char SHORT_STRING_IDENTIFIER [] = "s_"`
- `const size_t SHORT_TYPE_SIZE = 2`
- `const size_t SHORT_TYPE_INCR = 4`
- `const int32_t STRING_TYPE = 5`
- `const char STRING_STRING_IDENTIFIER [] = "r_"`
- `const size_t MAX_STRINGDATA_LENGTH = 255`
- `const size_t DEFAULT32BIT_TYPE_SIZE = 4`
- `const size_t DEFAULT32BIT_TYPE_INCR = 4`
- `const size_t STRING_IDENTIFIER_SIZE = 2`
- `const int32_t NAME_NOT_FOUND = -1`
- `const size_t MAX_NAME_LENGTH = 255`
- `const int32_t BUILD_DATA_FAILED = -333`
- `const int32_t HEADER_SIZE = 4`
- `const int32_t VERSION_ID_1 = 0x01010101`

13.4.2 Variable Documentation

13.4.2.1 FLOAT_TYPE

```
const int32_t AAX_ChunkDataParserDefs::FLOAT_TYPE = 1
```

13.4.2.2 FLOAT_STRING_IDENTIFIER

```
const char AAX_ChunkDataParserDefs::FLOAT_STRING_IDENTIFIER[] = "f_"
```

13.4.2.3 LONG_TYPE

```
const int32_t AAX_ChunkDataParserDefs::LONG_TYPE = 2
```

13.4.2.4 LONG_STRING_IDENTIFIER

```
const char AAX_ChunkDataParserDefs::LONG_STRING_IDENTIFIER[] = "l_"
```

13.4.2.5 DOUBLE_TYPE

```
const int32_t AAX_ChunkDataParserDefs::DOUBLE_TYPE = 3
```

13.4.2.6 DOUBLE_STRING_IDENTIFIER

```
const char AAX_ChunkDataParserDefs::DOUBLE_STRING_IDENTIFIER[] = "d_"
```

13.4.2.7 DOUBLE_TYPE_SIZE

```
const size_t AAX_ChunkDataParserDefs::DOUBLE_TYPE_SIZE = 8
```

13.4.2.8 DOUBLE_TYPE_INCR

```
const size_t AAX_ChunkDataParserDefs::DOUBLE_TYPE_INCR = 8
```

13.4.2.9 SHORT_TYPE

```
const int32_t AAX_ChunkDataParserDefs::SHORT_TYPE = 4
```

13.4.2.10 SHORT_STRING_IDENTIFIER

```
const char AAX_ChunkDataParserDefs::SHORT_STRING_IDENTIFIER[ ] = "s_"
```

13.4.2.11 SHORT_TYPE_SIZE

```
const size_t AAX_ChunkDataParserDefs::SHORT_TYPE_SIZE = 2
```

13.4.2.12 SHORT_TYPE_INCR

```
const size_t AAX_ChunkDataParserDefs::SHORT_TYPE_INCR = 4
```

13.4.2.13 STRING_TYPE

```
const int32_t AAX_ChunkDataParserDefs::STRING_TYPE = 5
```

13.4.2.14 STRING_STRING_IDENTIFIER

```
const char AAX_ChunkDataParserDefs::STRING_STRING_IDENTIFIER[ ] = "r_"
```

13.4.2.15 MAX_STRINGDATA_LENGTH

```
const size_t AAX_ChunkDataParserDefs::MAX_STRINGDATA_LENGTH = 255
```

13.4.2.16 DEFAULT32BIT_TYPE_SIZE

```
const size_t AAX_ChunkDataParserDefs::DEFAULT32BIT_TYPE_SIZE = 4
```

13.4.2.17 DEFAULT32BIT_TYPE_INCR

```
const size_t AAX_ChunkDataParserDefs::DEFAULT32BIT_TYPE_INCR = 4
```

13.4.2.18 STRING_IDENTIFIER_SIZE

```
const size_t AAX_ChunkDataParserDefs::STRING_IDENTIFIER_SIZE = 2
```

13.4.2.19 NAME_NOT_FOUND

```
const int32_t AAX_ChunkDataParserDefs::NAME_NOT_FOUND = -1
```

13.4.2.20 MAX_NAME_LENGTH

```
const size_t AAX_ChunkDataParserDefs::MAX_NAME_LENGTH = 255
```

13.4.2.21 BUILD_DATA_FAILED

```
const int32_t AAX_ChunkDataParserDefs::BUILD_DATA_FAILED = -333
```

13.4.2.22 HEADER_SIZE

```
const int32_t AAX_ChunkDataParserDefs::HEADER_SIZE = 4
```

13.4.2.23 VERSION_ID_1

```
const int32_t AAX_ChunkDataParserDefs::VERSION_ID_1 = 0x01010101
```


Chapter 14

Class Documentation

14.1 `_acfUID` Struct Reference

Public Attributes

- `uint32_t` [Data1](#)
- `uint16_t` [Data2](#)
- `uint16_t` [Data3](#)
- `uint8_t` [Data4](#) [8]

14.1.1 Member Data Documentation

14.1.1.1 `Data1`

```
uint32_t _acfUID::Data1
```

14.1.1.2 `Data2`

```
uint16_t _acfUID::Data2
```

14.1.1.3 `Data3`

```
uint16_t _acfUID::Data3
```

14.1.1.4 Data4

```
uint8_t _acfUID::Data4[8]
```

The documentation for this struct was generated from the following file:

- [AAX_ACFInterface.doxygen](#)

14.2 AAX_AggregateResult Class Reference

```
#include <AAX_Exception.h>
```

14.2.1 Description

RAII failure count convenience class for use with [AAX_CAPTURE\(\)](#) or [AAX_CAPTURE_MULT\(\)](#)

Pass this object as the first argument in a series of [AAX_CAPTURE\(\)](#) calls to count the number of failures that occur and to re-throw the last error if zero of the attempted calls succeed.

```
// example A: throw if all operations fail
AAX_AggregateResult agg;
AAX_CAPTURE( agg, RegisterThingA(); );
AAX_CAPTURE( agg, RegisterThingB(); );
AAX_CAPTURE( agg, RegisterThingC(); );
```

In this example, when `agg` goes out of scope it checks whether any of A, B, or C succeeded. If none succeeded then the last error that was encountered is raised via an [AAX_CheckedResult::Exception](#). If at least one of the calls succeeded then any failures are swallowed and execution continues as normal. This approach can be useful in cases where you want to run every operation in a group and you only want a failure to be returned if all of the operations failed.

```
// example B: throw if any operation fails
AAX_AggregateResult agg;
AAX_CAPTURE( agg, ImportantOperationW(); );
AAX_CAPTURE( agg, ImportantOperationX(); );
AAX_CAPTURE( agg, ImportantOperationY(); );
AAX_CheckedResult err = agg;
```

In this example, the last error encountered by `agg` is converted to an [AAX_CheckedResult](#). This will result in an [AAX_CheckedResult::Exception](#) even if at least one of the attempted operations succeeded. This approach can be useful in cases where you want all operations in a group to be executed before an error is raised for any failure within the group.

Public Member Functions

- [AAX_AggregateResult](#) ()=default
- [~AAX_AggregateResult](#) ()
- [AAX_AggregateResult & operator= \(AAX_Result inResult\)](#)
Overloaded [operator=\(\)](#) for conversion from [AAX_Result](#).
- [operator AAX_Result](#) ()
Implicit conversion to [AAX_Result](#) clears the state.
- void [Check](#) () const
- void [Clear](#) ()
- [AAX_Result LastFailure](#) () const
- int [NumFailed](#) () const
- int [NumSucceeded](#) () const
- int [NumAttempted](#) () const

14.2.2 Constructor & Destructor Documentation

14.2.2.1 AAX_AggregateResult()

```
AAX_AggregateResult::AAX_AggregateResult ( ) [default]
```

14.2.2.2 ~AAX_AggregateResult()

```
AAX_AggregateResult::~~AAX_AggregateResult ( ) [inline]
```

References [Check\(\)](#).

Here is the call graph for this function:

14.2.3 Member Function Documentation

14.2.3.1 operator=()

```
AAX_AggregateResult& AAX_AggregateResult::operator= (
    AAX_Result inResult ) [inline]
```

Overloaded [operator=\(\)](#) for conversion from [AAX_Result](#).

References [AAX_SUCCESS](#).

14.2.3.2 operator AAX_Result()

```
AAX_AggregateResult::operator AAX_Result ( ) [inline]
```

Implicit conversion to [AAX_Result](#) clears the state.

References [Clear\(\)](#), and [LastFailure\(\)](#).

Here is the call graph for this function:

14.2.3.3 Check()

```
void AAX_AggregateResult::Check ( ) const [inline]
```

Referenced by [~AAX_AggregateResult\(\)](#).

Here is the caller graph for this function:

14.2.3.4 Clear()

```
void AAX_AggregateResult::Clear ( ) [inline]
```

References AAX_SUCCESS.

Referenced by operator AAX_Result().

Here is the caller graph for this function:

14.2.3.5 LastFailure()

```
AAX_Result AAX_AggregateResult::LastFailure ( ) const [inline]
```

Referenced by operator AAX_Result().

Here is the caller graph for this function:

14.2.3.6 NumFailed()

```
int AAX_AggregateResult::NumFailed ( ) const [inline]
```

14.2.3.7 NumSucceeded()

```
int AAX_AggregateResult::NumSucceeded ( ) const [inline]
```

14.2.3.8 NumAttempted()

```
int AAX_AggregateResult::NumAttempted ( ) const [inline]
```

The documentation for this class was generated from the following file:

- [AAX_Exception.h](#)

14.3 AAX_CArrayDataBuffer< D > Class Template Reference

```
#include <AAX_CArrayDataBuffer.h>
```

Inheritance diagram for AAX_CArrayDataBuffer< D >:

Collaboration diagram for AAX_CArrayDataBuffer< D >:

14.3.1 Description

```
template<class D>
class AAX_CArrayDataBuffer< D >
```

A convenience class for array data buffers.

The data payload is an array of D

Public Member Functions

- [AAX_CArrayDataBuffer](#) (AAX_CTypeID inType, std::vector< D > const &inData)
- [AAX_CArrayDataBuffer](#) (AAX_CTypeID inType, std::vector< D > &&inData)
- [AAX_CArrayDataBuffer](#) (AAX_CArrayDataBuffer const &)=default
- [AAX_CArrayDataBuffer](#) (AAX_CArrayDataBuffer &&)=default
- [~AAX_CArrayDataBuffer](#) (void) [AAX_OVERRIDE](#)=default
- [AAX_CArrayDataBuffer](#) & operator= (AAX_CArrayDataBuffer const &other)=default
- [AAX_CArrayDataBuffer](#) & operator= (AAX_CArrayDataBuffer &&other)=default
- [AAX_Result Type](#) (AAX_CTypeID *oType) const [AAX_OVERRIDE](#)
- [AAX_Result Size](#) (int32_t *oSize) const [AAX_OVERRIDE](#)
- [AAX_Result Data](#) (void const **oBuffer) const [AAX_OVERRIDE](#)

Additional Inherited Members

14.3.2 Constructor & Destructor Documentation

14.3.2.1 AAX_CArrayDataBuffer() [1/4]

```
template<class D >
AAX_CArrayDataBuffer< D >::AAX_CArrayDataBuffer (
    AAX_CTypeID inType,
    std::vector< D > const & inData ) [inline]
```

14.3.2.2 AAX_CArrayDataBuffer() [2/4]

```
template<class D >
AAX_CArrayDataBuffer< D >::AAX_CArrayDataBuffer (
    AAX_CTypeID inType,
    std::vector< D > && inData ) [inline]
```

14.3.2.3 AAX_CArrayDataBuffer() [3/4]

```
template<class D >
AAX_CArrayDataBuffer< D >::AAX_CArrayDataBuffer (
    AAX_CArrayDataBuffer< D > const & ) [default]
```

14.3.2.4 AAX_CArrayDataBuffer() [4/4]

```
template<class D >
AAX_CArrayDataBuffer< D >::AAX_CArrayDataBuffer (
    AAX_CArrayDataBuffer< D > && ) [default]
```

14.3.2.5 ~AAX_CArrayDataBuffer()

```
template<class D >
AAX_CArrayDataBuffer< D >::~~AAX_CArrayDataBuffer (
    void ) [default]
```

14.3.3 Member Function Documentation

14.3.3.1 operator=() [1/2]

```
template<class D >
AAX_CArrayDataBuffer& AAX_CArrayDataBuffer< D >::operator= (
    AAX_CArrayDataBuffer< D > const & other ) [default]
```

14.3.3.2 operator=() [2/2]

```
template<class D >
AAX_CArrayDataBuffer& AAX_CArrayDataBuffer< D >::operator= (
    AAX_CArrayDataBuffer< D > && other ) [default]
```

14.3.3.3 Type()

```
template<class D >
AAX_Result AAX_CArrayDataBuffer< D >::Type (
    AAX_CTypeID * oType ) const [inline], [virtual]
```

The type of data contained in this buffer

This identifier must be sufficient for a client that knows the type to correctly interpret and use the data.

Implements [AAX_IACFDataBuffer](#).

References AAX_ERROR_NULL_ARGUMENT, and AAX_SUCCESS.

14.3.3.4 Size()

```
template<class D >
AAX_Result AAX_CArrayDataBuffer< D >::Size (
    int32_t * oSize ) const [inline], [virtual]
```

The number of bytes of data in this buffer

Implements [AAX_IACFDataBuffer](#).

References AAX_ERROR_NULL_ARGUMENT, AAX_ERROR_SIGNED_INT_OVERFLOW, and AAX_SUCCESS.

14.3.3.5 Data()

```
template<class D >
AAX_Result AAX_CArrayDataBuffer< D >::Data (
    void const ** oBuffer ) const [inline], [virtual]
```

The buffer of data

Implements [AAX_IACFDataBuffer](#).

References AAX_ERROR_NULL_ARGUMENT, and AAX_SUCCESS.

The documentation for this class was generated from the following file:

- [AAX_CArrayDataBuffer.h](#)

14.4 AAX_CArrayDataBufferOfType< T, D > Class Template Reference

```
#include <AAX_CArrayDataBuffer.h>
```

Inheritance diagram for AAX_CArrayDataBufferOfType< T, D >:

Collaboration diagram for AAX_CArrayDataBufferOfType< T, D >:

14.4.1 Description

```
template<AAX_CTypeID T, class D>
class AAX_CArrayDataBufferOfType< T, D >
```

A convenience class for array data buffers.

The data payload is an array of D

Public Member Functions

- [AAX_CArrayDataBufferOfType](#) (std::vector< D > const &inData)
- [AAX_CArrayDataBufferOfType](#) (std::vector< D > &&inData)
- [AAX_CArrayDataBufferOfType](#) (AAX_CArrayDataBufferOfType const &)=default
- [AAX_CArrayDataBufferOfType](#) (AAX_CArrayDataBufferOfType &&)=default
- [~AAX_CArrayDataBufferOfType](#) (void) AAX_OVERRIDE=default
- [AAX_CArrayDataBufferOfType](#) & operator= (AAX_CArrayDataBufferOfType const &other)=default
- [AAX_CArrayDataBufferOfType](#) & operator= (AAX_CArrayDataBufferOfType &&other)=default
- [AAX_Result Type](#) (AAX_CTypeID *oType) const AAX_OVERRIDE
- [AAX_Result Size](#) (int32_t *oSize) const AAX_OVERRIDE
- [AAX_Result Data](#) (void const **oBuffer) const AAX_OVERRIDE

Additional Inherited Members

14.4.2 Constructor & Destructor Documentation

14.4.2.1 AAX_CArrayDataBufferOfType() [1/4]

```
template<AAX_CTypeID T, class D >
AAX_CArrayDataBufferOfType< T, D >::AAX_CArrayDataBufferOfType (
    std::vector< D > const & inData ) [inline], [explicit]
```

14.4.2.2 AAX_CArrayDataBufferOfType() [2/4]

```
template<AAX_CTypeID T, class D >
AAX_CArrayDataBufferOfType< T, D >::AAX_CArrayDataBufferOfType (
    std::vector< D > && inData ) [inline], [explicit]
```

14.4.2.3 AAX_CArrayDataBufferOfType() [3/4]

```
template<AAX_CTypeID T, class D >
AAX_CArrayDataBufferOfType< T, D >::AAX_CArrayDataBufferOfType (
    AAX_CArrayDataBufferOfType< T, D > const & ) [default]
```

14.4.2.4 AAX_CArrayDataBufferOfType() [4/4]

```
template<AAX_CTypeID T, class D >
AAX_CArrayDataBufferOfType< T, D >::AAX_CArrayDataBufferOfType (
    AAX_CArrayDataBufferOfType< T, D > && ) [default]
```

14.4.2.5 ~AAX_CArrayDataBufferOfType()

```
template<AAX_CTypeID T, class D >
AAX_CArrayDataBufferOfType< T, D >::~~AAX_CArrayDataBufferOfType (
    void ) [default]
```

14.4.3 Member Function Documentation

14.4.3.1 operator=() [1/2]

```
template<AAX_CTypeID T, class D >
AAX_CArrayDataBufferOfType& AAX_CArrayDataBufferOfType< T, D >::operator= (
    AAX_CArrayDataBufferOfType< T, D > const & other ) [default]
```

14.4.3.2 operator=() [2/2]

```
template<AAX_CTypeID T, class D >
AAX_CArrayDataBufferOfType& AAX_CArrayDataBufferOfType< T, D >::operator= (
    AAX_CArrayDataBufferOfType< T, D > && other ) [default]
```

14.4.3.3 Type()

```
template<AAX_CTypeID T, class D >
AAX_Result AAX_CArrayDataBufferOfType< T, D >::Type (
    AAX_CTypeID * oType ) const [inline], [virtual]
```

The type of data contained in this buffer

This identifier must be sufficient for a client that knows the type to correctly interpret and use the data.

Implements [AAX_IACFDataBuffer](#).

References AAX_ERROR_NULL_ARGUMENT, and AAX_SUCCESS.

14.4.3.4 Size()

```
template<AAX_CTypeID T, class D >
AAX_Result AAX_CArrayDataBufferOfType< T, D >::Size (
    int32_t * oSize ) const [inline], [virtual]
```

The number of bytes of data in this buffer

Implements [AAX_IACFDataBuffer](#).

References AAX_ERROR_NULL_ARGUMENT, AAX_ERROR_SIGNED_INT_OVERFLOW, and AAX_SUCCESS.

14.4.3.5 Data()

```
template<AAX_CTypeID T, class D >
AAX_Result AAX_CArrayDataBufferOfType< T, D >::Data (
    void const ** oBuffer ) const [inline], [virtual]
```

The buffer of data

Implements [AAX_IACFDataBuffer](#).

References AAX_ERROR_NULL_ARGUMENT, and AAX_SUCCESS.

The documentation for this class was generated from the following file:

- [AAX_CArrayDataBuffer.h](#)

14.5 AAX_CAtomicQueue< T, S > Class Template Reference

```
#include <AAX_CAtomicQueue.h>
```

Inheritance diagram for AAX_CAtomicQueue< T, S >:

Collaboration diagram for AAX_CAtomicQueue< T, S >:

14.5.1 Description

```
template<typename T, size_t S>
class AAX_CAtomicQueue< T, S >
```

Multi-writer, single-reader implementation of [AAX_IPointerQueue](#)

Template parameters:

- **T** : Type of the objects pointed to by this queue
- **S** : Size of the queue's ring buffer. Should be a power of two less than `UINT_32_MAX`

Properties:

- Read operations are non-blocking
- Write operations are synchronized, but very fast
- Supports only one read thread - do not call [Pop\(\)](#) or [Peek\(\)](#) concurrently
- Supports any number of write threads
- Does not support placing `NULL` values onto the queue. [Push](#) will return `eStatus_Unsupported` if a `NULL` value is pushed onto the queue, and the value will be ignored.

Public Types

- typedef [AAX_IPointerQueue< T >::template_type](#) `template_type`
The type used for this template instance.
- typedef [AAX_IPointerQueue< T >::value_type](#) `value_type`
The type of values stored in this queue.

Public Member Functions

- virtual [~AAX_CAtomicQueue](#) ()
- [AAX_CAtomicQueue](#) ()
- virtual void [Clear](#) ()
- virtual [AAX_IContainer::EStatus](#) [Push](#) (`value_type` inElem)
- virtual `value_type` [Pop](#) ()
- virtual `value_type` [Peek](#) () const

Static Public Attributes

- static const size_t [template_size](#) = S
The size used for this template instance.

14.5.2 Member Typedef Documentation

14.5.2.1 `template_type`

```
template<typename T , size_t S>
typedef AAX\_IPointerQueue<T>::template_type AAX\_CAtomicQueue< T, S >::template_type
```

The type used for this template instance.

14.5.2.2 `value_type`

```
template<typename T , size_t S>
typedef AAX\_IPointerQueue<T>::value_type AAX\_CAtomicQueue< T, S >::value_type
```

The type of values stored in this queue.

14.5.3 Constructor & Destructor Documentation

14.5.3.1 `~AAX_CAtomicQueue()`

```
template<typename T , size_t S>
virtual AAX\_CAtomicQueue< T, S >::~~AAX\_CAtomicQueue ( ) [inline], [virtual]
```

14.5.3.2 `AAX_CAtomicQueue()`

```
template<typename T , size_t S>
AAX\_CAtomicQueue< T, S >::AAX_CAtomicQueue ( )
```

14.5.4 Member Function Documentation

14.5.4.1 `Clear()`

```
template<typename T , size_t S>
virtual void AAX\_CAtomicQueue< T, S >::Clear ( ) [virtual]
```

Note

This operation is NOT atomic

This does NOT call the destructor for any pointed-to elements; it only clears the pointer values in the queue

Implements [AAX_IPointerQueue](#)< T >.

14.5.4.2 Push()

```
template<typename T , size_t S>
virtual AAX_IContainer::EStatus AAX_CAtomicQueue< T, S >::Push (
    value_type inElem ) [virtual]
```

Push an element onto the queue

Call from: Write thread

Returns

[AAX_IContainer::EStatus_Success](#) if the push succeeded

Implements [AAX_IPointerQueue< T >](#).

Referenced by [AAX_CMonolithicParameters::GenerateCoefficients\(\)](#), and [AAX_CMonolithicParameters::StaticRenderAudio\(\)](#).

Here is the caller graph for this function:

14.5.4.3 Pop()

```
template<typename T , size_t S>
virtual value_type AAX_CAtomicQueue< T, S >::Pop ( ) [virtual]
```

Pop the front element from the queue

Call from: Read thread

Returns

NULL if no element is available

Implements [AAX_IPointerQueue< T >](#).

14.5.4.4 Peek()

```
template<typename T , size_t S>
virtual value_type AAX_CAtomicQueue< T, S >::Peek ( ) const [virtual]
```

Get the current top element without popping it off of the queue

Call from: Read thread

Note

This value will change if another thread calls [Pop\(\)](#)

Implements [AAX_IPointerQueue< T >](#).

14.5.5 Member Data Documentation

14.5.5.1 template_size

```
template<typename T , size_t S>  
const size_t AAX_CAtomicQueue< T, S >::template_size = S [static]
```

The size used for this template instance.

The documentation for this class was generated from the following file:

- [AAX_CAtomicQueue.h](#)

14.6 AAX_CAutoreleasePool Class Reference

```
#include <AAX_CAutoreleasePool.h>
```

Public Member Functions

- [AAX_CAutoreleasePool\(\)](#)
- [~AAX_CAutoreleasePool\(\)](#)

14.6.1 Constructor & Destructor Documentation

14.6.1.1 AAX_CAutoreleasePool()

```
AAX_CAutoreleasePool::AAX_CAutoreleasePool ( )
```

14.6.1.2 ~AAX_CAutoreleasePool()

```
AAX_CAutoreleasePool::~~AAX_CAutoreleasePool ( )
```

The documentation for this class was generated from the following file:

- [AAX_CAutoreleasePool.h](#)

14.7 AAX_CBinaryDisplayDelegate< T > Class Template Reference

```
#include <AAX_CBinaryDisplayDelegate.h>
```

Inheritance diagram for AAX_CBinaryDisplayDelegate< T >:

Collaboration diagram for AAX_CBinaryDisplayDelegate< T >:

14.7.1 Description

```
template<typename T>
class AAX_CBinaryDisplayDelegate< T >
```

A binary display format conforming to [AAX_IDisplayDelegate](#).

This display delegate converts a parameter value to one of two provided strings (e.g. "True" and "False").

Public Member Functions

- [AAX_CBinaryDisplayDelegate](#) (const char *falseString, const char *trueString)
Constructor.
- [AAX_CBinaryDisplayDelegate](#) (const [AAX_CBinaryDisplayDelegate](#) &other)
- [AAX_IDisplayDelegate](#)< T > * [Clone](#) () const [AAX_OVERRIDE](#)
Constructs and returns a copy of the display delegate.
- bool [ValueToString](#) (T value, [AAX_CString](#) *valueString) const [AAX_OVERRIDE](#)
Converts a real parameter value to a string representation.
- bool [ValueToString](#) (T value, int32_t maxNumChars, [AAX_CString](#) *valueString) const [AAX_OVERRIDE](#)
Converts a real parameter value to a string representation using a size hint, useful for control surfaces and other character limited displays.
- bool [StringToValue](#) (const [AAX_CString](#) &valueString, T *value) const [AAX_OVERRIDE](#)
Converts a string to a real parameter value.
- virtual void [AddShortenedStrings](#) (const char *falseString, const char *trueString, int iStrLength)

14.7.2 Constructor & Destructor Documentation

14.7.2.1 AAX_CBinaryDisplayDelegate() [1/2]

```
template<typename T >
AAX_CBinaryDisplayDelegate< T >::AAX_CBinaryDisplayDelegate (
    const char * falseString,
    const char * trueString )
```

Constructor.

Parameters

in	<i>falseString</i>	The string that will be associated with false parameter values
in	<i>trueString</i>	The string that will be associated with true parameter values

References `AAX_CString::Length()`.

Here is the call graph for this function:

14.7.2.2 AAX_CBinaryDisplayDelegate() [2/2]

```
template<typename T >
AAX_CBinaryDisplayDelegate< T >::AAX_CBinaryDisplayDelegate (
    const AAX_CBinaryDisplayDelegate< T > & other )
```

14.7.3 Member Function Documentation

14.7.3.1 Clone()

```
template<typename T >
AAX_IDisplayDelegate< T > * AAX_CBinaryDisplayDelegate< T >::Clone ( ) const [virtual]
```

Constructs and returns a copy of the display delegate.

In general, this method's implementation can use a simple copy constructor:

```
template <typename T>
AAX_CSubclassDisplayDelegate<T>*    AAX_CSubclassDisplayDelegate<T>::Clone() const
{
    return new AAX_CSubclassDisplayDelegate(*this);
}
```

Implements `AAX_IDisplayDelegate< T >`.

14.7.3.2 ValueToString() [1/2]

```
template<typename T >
bool AAX_CBinaryDisplayDelegate< T >::ValueToString (
    T value,
    AAX_CString * valueString ) const [virtual]
```

Converts a real parameter value to a string representation.

Parameters

in	<i>value</i>	The real parameter value that will be converted
out	<i>valueString</i>	A string corresponding to value

Return values

<i>true</i>	The string conversion was successful
<i>false</i>	The string conversion was unsuccessful

Implements [AAX_IDisplayDelegate< T >](#).

14.7.3.3 ValueToString() [2/2]

```
template<typename T >
bool AAX_CBinaryDisplayDelegate< T >::ValueToString (
    T value,
    int32_t maxNumChars,
    AAX_CString * valueString ) const [virtual]
```

Converts a real parameter value to a string representation using a size hint, useful for control surfaces and other character limited displays.

Parameters

in	<i>value</i>	The real parameter value that will be converted
in	<i>maxNumChars</i>	Size hint for the desired maximum number of characters in the string (not including null termination)
out	<i>valueString</i>	A string corresponding to value

Return values

<i>true</i>	The string conversion was successful
<i>false</i>	The string conversion was unsuccessful

Implements [AAX_IDisplayDelegate< T >](#).

14.7.3.4 StringToValue()

```
template<typename T >
bool AAX_CBinaryDisplayDelegate< T >::StringToValue (
    const AAX_CString & valueString,
    T * value ) const [virtual]
```

Converts a string to a real parameter value.

Parameters

in	<i>valueString</i>	The string that will be converted
out	<i>value</i>	The real parameter value corresponding to valueString

Return values

<i>true</i>	The string conversion was successful
<i>false</i>	The string conversion was unsuccessful

Implements [AAX_IDisplayDelegate< T >](#).

14.7.3.5 AddShortenedStrings()

```
template<typename T >
void AAX_CBinaryDisplayDelegate< T >::AddShortenedStrings (
    const char * falseString,
    const char * trueString,
    int iStrLength ) [virtual]
```

The documentation for this class was generated from the following file:

- [AAX_CBinaryDisplayDelegate.h](#)

14.8 AAX_CBinaryTaperDelegate< T > Class Template Reference

```
#include <AAX_CBinaryTaperDelegate.h>
```

Inheritance diagram for AAX_CBinaryTaperDelegate< T >:

Collaboration diagram for AAX_CBinaryTaperDelegate< T >:

14.8.1 Description

```
template<typename T>
class AAX_CBinaryTaperDelegate< T >
```

A binary taper conforming to [AAX_ITaperDelegate](#).

This taper maps positive real values to 1 and negative or zero real values to 0. This is the standard taper used on all bool parameters.

When this taper is constructed with a bool template type, its normalized values are automatically typecast to the proper boolean value.

Public Member Functions

- [AAX_CBinaryTaperDelegate](#) ()
Constructs a Binary Taper.
- [AAX_ITaperDelegate](#)< T > * [Clone](#) () const [AAX_OVERRIDE](#)
Constructs and returns a copy of the taper delegate.
- T [GetMaximumValue](#) () const [AAX_OVERRIDE](#)
Returns the taper's maximum real value.
- T [GetMinimumValue](#) () const [AAX_OVERRIDE](#)
Returns the taper's minimum real value.
- T [ConstrainRealValue](#) (T value) const [AAX_OVERRIDE](#)
Applies a constraint to the value and returns the constrained value.
- T [NormalizedToReal](#) (double normalizedValue) const [AAX_OVERRIDE](#)
Converts a normalized value to a real value.
- double [RealToNormalized](#) (T realValue) const [AAX_OVERRIDE](#)
Normalizes a real parameter value.

14.8.2 Constructor & Destructor Documentation

14.8.2.1 AAX_CBinaryTaperDelegate()

```
template<typename T >
AAX_CBinaryTaperDelegate< T >::AAX_CBinaryTaperDelegate
```

Constructs a Binary Taper.

14.8.3 Member Function Documentation

14.8.3.1 Clone()

```
template<typename T >
AAX_ITaperDelegate< T > * AAX_CBinaryTaperDelegate< T >::Clone ( ) const [virtual]
```

Constructs and returns a copy of the taper delegate.

In general, this method's implementation can use a simple copy constructor:

```
template <typename T>
AAX_CSubclassTaperDelegate<T>* AAX_CSubclassTaperDelegate<T>::Clone() const
{
    return new AAX_CSubclassTaperDelegate(*this);
}
```

Implements [AAX_ITaperDelegate](#)< T >.

14.8.3.2 GetMaximumValue()

```
template<typename T >
T AAX_CBinaryTaperDelegate< T >::GetMaximumValue ( ) const [virtual]
```

Returns the taper's maximum real value.

Implements [AAX_ITaperDelegate< T >](#).

14.8.3.3 GetMinimumValue()

```
template<typename T >
T AAX_CBinaryTaperDelegate< T >::GetMinimumValue ( ) const [virtual]
```

Returns the taper's minimum real value.

Implements [AAX_ITaperDelegate< T >](#).

14.8.3.4 ConstrainRealValue()

```
template<typename T >
T AAX_CBinaryTaperDelegate< T >::ConstrainRealValue (
    T value ) const [virtual]
```

Applies a constraint to the value and returns the constrained value.

This method is useful if the taper requires a constraint beyond simple minimum and maximum real value limits.

Note

This is the function that should actually enforces the constraints in [NormalizeToReal\(\)](#) and [RealToNormalized\(\)](#).

Parameters

in	<i>value</i>	The unconstrained value
----	--------------	-------------------------

Implements [AAX_ITaperDelegate< T >](#).

14.8.3.5 NormalizedToReal()

```
template<typename T >
T AAX_CBinaryTaperDelegate< T >::NormalizedToReal (
    double normalizedValue ) const [virtual]
```

Converts a normalized value to a real value.

This is where the actual taper algorithm is implemented.

This function should perform the exact inverse of [RealToNormalized\(\)](#), to within the roundoff precision of the individual taper implementation.

Parameters

in	<i>normalizedValue</i>	The normalized value that will be converted
----	------------------------	---

Implements [AAX_ITaperDelegate< T >](#).

14.8.3.6 RealToNormalized()

```
template<typename T >
double AAX_CBinaryTaperDelegate< T >::RealToNormalized (
    T realValue ) const [virtual]
```

Normalizes a real parameter value.

This is where the actual taper algorithm is implemented.

This function should perform the exact inverse of [NormalizedToReal\(\)](#), to within the roundoff precision of the individual taper implementation.

Parameters

in	<i>realValue</i>	The real parameter value that will be normalized
----	------------------	--

Implements [AAX_ITaperDelegate< T >](#).

The documentation for this class was generated from the following file:

- [AAX_CBinaryTaperDelegate.h](#)

14.9 AAX_CChunkDataParser Class Reference

```
#include <AAX_CChunkDataParser.h>
```

Collaboration diagram for AAX_CChunkDataParser:

14.9.1 Description

Parser utility for plugin chunks.

Todo Update this documentation for [AAX](#)

This class acts as generic repository for data that is stuffed into or extracted from a `SFicPlugInChunk`. It has an abstracted Add/Find interface to add or retrieve data values, each uniquely referenced by a c-string. In conjunction with the Effect Layer and the "ControlManager" aspects of the `CProcess` class, this provides a more transparent & resilient system for performing save-and-restore on settings that won't break so easily from endian issues or from the hard-coded structs that have typically been used to build chunk data.

Format of the Chunk Data

The first 4 bytes of the data are the version number (repeated 4 times to be immune to byte swapping). Data follows next.

Example: "f_bypa %\$\$@d_gain #!#@%\$\$_oms_i # \$"

type	name	value
float	bypa	%\$#@
double	gain	#!#@%\$\$_
int16_t	oms_i	#\$

- The first character denotes the data type:

```
'f' = float
'd' = double
'l' = int32
's' = int16
```

- "_" is an empty placekeeper that could be used to addition future information. Currently, it's ignored when a chunk is parsed.
- The string name identifier follows next, and can up to 255 characters `int32_t`. The Effect Layer builds chunks it always converts the `AAX_FourCharCode` of the control to a string. So, this will always be 4 characters `int32_t`. The string is null terminated to indicate the start of the data value.
- The data value follows next, but is possible shifted to aligned word aligned. The size of is determined, of course, by the data type.

Classes

- struct [DataValue](#)

Public Member Functions

- [AAX_CChunkDataParser](#) ()
- virtual [~AAX_CChunkDataParser](#) ()
- void [AddFloat](#) (const char *name, float value)
CALL: Adds some data of type float with name and value to the current chunk.
- void [AddDouble](#) (const char *name, double value)
CALL: See [AddFloat\(\)](#)
- void [AddInt32](#) (const char *name, int32_t value)

- CALL: See [AddFloat\(\)](#)
- void [AddInt16](#) (const char *name, int16_t value)
 - CALL: See [AddFloat\(\)](#)
- void [AddString](#) (const char *name, [AAX_CString](#) value)
- bool [FindFloat](#) (const char *name, float *value)
 - CALL: Finds some data of type float with name and value in the current chunk.
- bool [FindDouble](#) (const char *name, double *value)
 - CALL: See [FindFloat\(\)](#)
- bool [FindInt32](#) (const char *name, int32_t *value)
 - CALL: See [FindFloat\(\)](#)
- bool [FindInt16](#) (const char *name, int16_t *value)
 - CALL: See [FindFloat\(\)](#)
- bool [FindString](#) (const char *name, [AAX_CString](#) *value)
- bool [ReplaceDouble](#) (const char *name, double value)
- int32_t [GetChunkData](#) ([AAX_SPluginChunk](#) *chunk)
 - CALL: Fills passed in chunk with data from current chunk; returns 0 if successful.
- int32_t [GetChunkDataSize](#) ()
 - CALL: Returns size of current chunk.
- int32_t [GetChunkVersion](#) ()
 - CALL: Lists fVersion in chunk header for convenience.
- bool [IsEmpty](#) ()
 - CALL: Returns true if no data is in the chunk.
- void [Clear](#) ()
 - Resets chunk.

Internal Methods

An Effect Layer plugin can ignore these methods. They are handled by or used internally by the Effect Layer.

- int32_t [mLastFoundIndex](#)
 - The last index found in the chunk.
- char * [mChunkData](#)
- int32_t [mChunkVersion](#)
 - Equal to fVersion from the chunk header. Equal to -1 if no chunk is loaded.
- std::vector< [DataValue](#) > [mDataValues](#)
- void [LoadChunk](#) (const [AAX_SPluginChunk](#) *chunk)
 - Sets current chunk to data in chunk parameter.
- void [WordAlign](#) (uint32_t &index)
 - sets index to 4-byte boundary
- void [WordAlign](#) (int32_t &index)
 - sets index to 4-byte boundary
- int32_t [FindName](#) (const [AAX_CString](#) &Name)
 - used by public Find methods

14.9.2 Constructor & Destructor Documentation

14.9.2.1 AAX_CChunkDataParser()

```
AAX_CChunkDataParser::AAX_CChunkDataParser ( )
```

14.9.2.2 ~AAX_CChunkDataParser()

```
virtual AAX_CChunkDataParser::~~AAX_CChunkDataParser ( ) [virtual]
```

14.9.3 Member Function Documentation

14.9.3.1 AddFloat()

```
void AAX_CChunkDataParser::AddFloat (
    const char * name,
    float value )
```

CALL: Adds some data of type float with *name* and *value* to the current chunk.

See also

[AddDouble\(\)](#), [AddInt32\(\)](#), and [AddInt16\(\)](#) are the same but with different data types.

14.9.3.2 AddDouble()

```
void AAX_CChunkDataParser::AddDouble (
    const char * name,
    double value )
```

CALL: See [AddFloat\(\)](#)

14.9.3.3 AddInt32()

```
void AAX_CChunkDataParser::AddInt32 (
    const char * name,
    int32_t value )
```

CALL: See [AddFloat\(\)](#)

14.9.3.4 AddInt16()

```
void AAX_CChunkDataParser::AddInt16 (
    const char * name,
    int16_t value )
```

CALL: See [AddFloat\(\)](#)

14.9.3.5 AddString()

```
void AAX_CChunkDataParser::AddString (
    const char * name,
    AAX_CString value )
```

14.9.3.6 FindFloat()

```
bool AAX_CChunkDataParser::FindFloat (
    const char * name,
    float * value )
```

CALL: Finds some data of type float with *name* and *value* in the current chunk.

See also

[FindDouble\(\)](#), [FindInt32\(\)](#), and [FindInt16\(\)](#) are the same but with different data types.

14.9.3.7 FindDouble()

```
bool AAX_CChunkDataParser::FindDouble (
    const char * name,
    double * value )
```

CALL: See [FindFloat\(\)](#)

14.9.3.8 FindInt32()

```
bool AAX_CChunkDataParser::FindInt32 (
    const char * name,
    int32_t * value )
```

CALL: See [FindFloat\(\)](#)

14.9.3.9 FindInt16()

```
bool AAX_CChunkDataParser::FindInt16 (
    const char * name,
    int16_t * value )
```

CALL: See [FindFloat\(\)](#)

14.9.3.10 FindString()

```
bool AAX_CChunkDataParser::FindString (
    const char * name,
    AAX_CString * value )
```

14.9.3.11 ReplaceDouble()

```
bool AAX_CChunkDataParser::ReplaceDouble (
    const char * name,
    double value )
```

14.9.3.12 GetChunkData()

```
int32_t AAX_CChunkDataParser::GetChunkData (
    AAX_SPlugInChunk * chunk )
```

CALL: Fills passed in *chunk* with data from current chunk; returns 0 if successful.

14.9.3.13 GetChunkDataSize()

```
int32_t AAX_CChunkDataParser::GetChunkDataSize ( )
```

CALL: Returns size of current chunk.

14.9.3.14 GetChunkVersion()

```
int32_t AAX_CChunkDataParser::GetChunkVersion ( ) [inline]
```

CALL: Lists fVersion in chunk header for convenience.

References mChunkVersion.

14.9.3.15 IsEmpty()

```
bool AAX_CChunkDataParser::IsEmpty ( )
```

CALL: Returns true if no data is in the chunk.

14.9.3.16 Clear()

```
void AAX_CChunkDataParser::Clear ( )
```

Resets chunk.

14.9.3.17 LoadChunk()

```
void AAX_CChunkDataParser::LoadChunk (
    const AAX_SPlugInChunk * chunk )
```

Sets current chunk to data in *chunk* parameter.

14.9.3.18 WordAlign() [1/2]

```
void AAX_CChunkDataParser::WordAlign (
    uint32_t & index ) [protected]
```

sets *index* to 4-byte boundary

14.9.3.19 WordAlign() [2/2]

```
void AAX_CChunkDataParser::WordAlign (
    int32_t & index ) [protected]
```

sets *index* to 4-byte boundary

14.9.3.20 FindName()

```
int32_t AAX_CChunkDataParser::FindName (
    const AAX_CString & Name ) [protected]
```

used by public Find methods

14.9.4 Member Data Documentation

14.9.4.1 mLastFoundIndex

```
int32_t AAX_CChunkDataParser::mLastFoundIndex [protected]
```

The last index found in the chunk.

Since control values in chunks should tend to stay in order and in sync with the way they're checked with controls within the plug-in, we'll keep track of the value index to speed up searching.

14.9.4.2 mChunkData

```
char* AAX_CChunkDataParser::mChunkData [protected]
```

14.9.4.3 mChunkVersion

```
int32_t AAX_CChunkDataParser::mChunkVersion [protected]
```

Equal to fVersion from the chunk header. Equal to -1 if no chunk is loaded.

Referenced by GetChunkVersion().

14.9.4.4 mDataValues

```
std::vector<DataValue> AAX_CChunkDataParser::mDataValues
```

The documentation for this class was generated from the following file:

- [AAX_CChunkDataParser.h](#)

14.10 AAX_CDecibelDisplayDelegateDecorator< T > Class Template Reference

```
#include <AAX_CDecibelDisplayDelegateDecorator.h>
```

Inheritance diagram for AAX_CDecibelDisplayDelegateDecorator< T >:

Collaboration diagram for AAX_CDecibelDisplayDelegateDecorator< T >:

14.10.1 Description

```
template<typename T>
class AAX_CDecibelDisplayDelegateDecorator< T >
```

A percent decorator conforming to [AAX_IDisplayDelegateDecorator](#).

This class is an [AAX_IDisplayDelegateDecorator](#), meaning that it acts as a wrapper for other display delegates or concrete display types. For more information about display delegate decorators in [AAX](#), see [Display delegate decorators](#)

The behavior of this class is to provide conversion to and from dB values. It performs a decibel conversion on the square of the provided value (i.e. 20 log) before passing the value on to a concrete display delegate to get a value string. This class then appends the "dB" suffix to signify that the value was converted. This allows something like a gain value to remain internally linear at all times even though its display is converted to decibels.

The inverse is also supported; this class can convert a decibel-formatted string into its associated real value. The string will first be converted to a number, then that number will have the inverse dB calculation applied to it to retrieve the parameter's real value.

Public Member Functions

- [AAX_CDecibelDisplayDelegateDecorator](#) (const [AAX_IDisplayDelegate](#)< T > &displayDelegate)
- [AAX_CDecibelDisplayDelegateDecorator](#)< T > * [Clone](#) () const [AAX_OVERRIDE](#)
Constructs and returns a copy of the display delegate.
- bool [ValueToString](#) (T value, [AAX_CString](#) *valueString) const [AAX_OVERRIDE](#)
Converts a real parameter value to a string representation.
- bool [ValueToString](#) (T value, int32_t maxNumChars, [AAX_CString](#) *valueString) const [AAX_OVERRIDE](#)
Converts a real parameter value to a string representation using a size hint, useful for control surfaces and other character limited displays.
- bool [StringToValue](#) (const [AAX_CString](#) &valueString, T *value) const [AAX_OVERRIDE](#)
Converts a string to a real parameter value.

14.10.2 Constructor & Destructor Documentation

14.10.2.1 AAX_CDecibelDisplayDelegateDecorator()

```
template<typename T >
AAX_CDecibelDisplayDelegateDecorator< T >::AAX_CDecibelDisplayDelegateDecorator (
    const AAX\_IDisplayDelegate< T > & displayDelegate )
```

14.10.3 Member Function Documentation

14.10.3.1 Clone()

```
template<typename T >
AAX_CDecibelDisplayDelegateDecorator< T > * AAX_CDecibelDisplayDelegateDecorator< T >::Clone
( ) const [virtual]
```

Constructs and returns a copy of the display delegate.

In general, this method's implementation can use a simple copy constructor:

```
template <typename T>
AAX_CSubclassDisplayDelegate<T>*    AAX_CSubclassDisplayDelegate<T>::Clone() const
{
    return new AAX_CSubclassDisplayDelegate(*this);
}
```

Implements [AAX_IDisplayDelegate< T >](#).

14.10.3.2 ValueToString() [1/2]

```
template<typename T >
bool AAX_CDecibelDisplayDelegateDecorator< T >::ValueToString (
    T value,
    AAX_CString * valueString ) const [virtual]
```

Converts a real parameter value to a string representation.

Parameters

in	<i>value</i>	The real parameter value that will be converted
out	<i>valueString</i>	A string corresponding to value

Return values

<i>true</i>	The string conversion was successful
<i>false</i>	The string conversion was unsuccessful

Implements [AAX_IDisplayDelegate< T >](#).

References [AAX_IDisplayDelegateDecorator< T >::ValueToString\(\)](#).

Here is the call graph for this function:

14.10.3.3 ValueToString() [2/2]

```
template<typename T >
bool AAX_CDecibelDisplayDelegateDecorator< T >::ValueToString (
    T value,
    int32_t maxNumChars,
    AAX_CString * valueString ) const [virtual]
```

Converts a real parameter value to a string representation using a size hint, useful for control surfaces and other character limited displays.

Parameters

in	<i>value</i>	The real parameter value that will be converted
in	<i>maxNumChars</i>	Size hint for the desired maximum number of characters in the string (not including null termination)
out	<i>valueString</i>	A string corresponding to value

Return values

<i>true</i>	The string conversion was successful
<i>false</i>	The string conversion was unsuccessful

Implements [AAX_IDisplayDelegate< T >](#).

References [AAX_CString::Append\(\)](#), [AAX_CString::Length\(\)](#), and [AAX_IDisplayDelegateDecorator< T >::Value↔ToString\(\)](#).

Here is the call graph for this function:

14.10.3.4 StringToValue()

```
template<typename T >
bool AAX_CDecibelDisplayDelegateDecorator< T >::StringToValue (
    const AAX_CString & valueString,
    T * value ) const [virtual]
```

Converts a string to a real parameter value.

Parameters

in	<i>valueString</i>	The string that will be converted
out	<i>value</i>	The real parameter value corresponding to valueString

Return values

<i>true</i>	The string conversion was successful
<i>false</i>	The string conversion was unsuccessful

Implements [AAX_IDisplayDelegate< T >](#).

References [AAX_CString::Length\(\)](#), [AAX_IDisplayDelegateDecorator< T >::StringToValue\(\)](#), and [AAX_CString↔::SubString\(\)](#).

Here is the call graph for this function:

The documentation for this class was generated from the following file:

- [AAX_CDecibelDisplayDelegateDecorator.h](#)

14.11 AAX_CEffectDirectData Class Reference

```
#include <AAX_CEffectDirectData.h>
```

Inheritance diagram for AAX_CEffectDirectData:

Collaboration diagram for AAX_CEffectDirectData:

14.11.1 Description

Default implementation of the [AAX_IEffectDirectData](#) interface.

This class provides a default implementation of the [AAX_IEffectDirectData](#) interface.

Public Member Functions

- [AAX_CEffectDirectData](#) (void)
- virtual [~AAX_CEffectDirectData](#) (void)

Initialization and uninitialization

- [AAX_Result Initialize](#) (IACFUnknown *iController) [AAX_OVERRIDE](#) [AAX_FINAL](#)
Non-virtual implementation of AAX_IEfectDirectData::Initialize()
- [AAX_Result Uninitialize](#) (void) [AAX_OVERRIDE](#)
Main uninitialization.

Data update callbacks

- [AAX_Result TimerWakeup](#) (IACFUnknown *iDataAccessInterface) [AAX_OVERRIDE](#)
Non-virtual implementation of AAX_IEfectDirectData::TimerWakeup()

AAX host and plug-in event notification

- [AAX_Result NotificationReceived](#) (AAX_CTypeID inNotificationType, const void *inNotificationData, uint32_t inNotificationDataSize) [AAX_OVERRIDE](#)
Notification Hook.

Private member accessors

- [AAX_IController](#) * [Controller](#) (void)
Returns a pointer to the plug-in's controller interface.
- [AAX_IEffectParameters](#) * [EffectParameters](#) (void)
Returns a pointer to the plug-in's data model interface.

AAX_CEffectDirectData virtual interface

- virtual [AAX_Result Initialize_PrivateDataAccess](#) ()
Initialization routine for classes that inherit from [AAX_CEffectDirectData](#). This method is called by the default [Initialize\(\)](#) implementation after all internal members have been initialized, and provides a safe location in which to perform any additional initialization tasks.
- virtual [AAX_Result TimerWakeup_PrivateDataAccess](#) (AAX_IPrivateDataAccess *iPrivateDataAccess)
Callback provided with an [AAX_IPrivateDataAccess](#). Override this method to access the algorithm's private data using the [AAX_IPrivateDataAccess](#) interface.

Additional Inherited Members

14.11.2 Constructor & Destructor Documentation

14.11.2.1 AAX_CEffectDirectData()

```
AAX_CEffectDirectData::AAX_CEffectDirectData (
    void )
```

14.11.2.2 ~AAX_CEffectDirectData()

```
virtual AAX_CEffectDirectData::~~AAX_CEffectDirectData (
    void ) [virtual]
```

14.11.3 Member Function Documentation

14.11.3.1 Initialize()

```
AAX_Result AAX_CEffectDirectData::Initialize (
    IACFUnknown * iController ) [virtual]
```

Non-virtual implementation of AAX_IEffectDirectData::Initialize()

This implementation initializes all private [AAX_CEffectDirectData](#) members and calls [Initialize_PrivateDataAccess\(\)](#). For custom initialization, inherited classes should override [Initialize_PrivateDataAccess\(\)](#).

Parameters

in	<i>iController</i>	Unknown pointer that resolves to an AAX_IController .
----	--------------------	---

Implements [AAX_IACFEffectDirectData](#).

14.11.3.2 Uninitialize()

```
AAX_Result AAX_CEffectDirectData::Uninitialize (
    void ) [virtual]
```

Main uninitialization.

Called when the interface is destroyed.

Implements [AAX_IACFEfffectDirectData](#).

14.11.3.3 TimerWakeup()

```
AAX_Result AAX_CEffectDirectData::TimerWakeup (
    IACFUnknown * iDataAccessInterface ) [virtual]
```

Non-virtual implementation of AAX_IEffectDirectData::TimerWakeup()

This implementation interprets the [IACFUnknown](#) and forwards the resulting [AAX_IPrivateDataAccess](#) to [TimerWakeup_PrivateDataAccess\(\)](#)

Parameters

in	<i>iDataAccessInterface</i>	Unknown pointer that resolves to an AAX_IPrivateDataAccess . This interface is only valid for the duration of this method's execution and is discarded when the method returns.
----	-----------------------------	---

Implements [AAX_IACFEfffectDirectData](#).

14.11.3.4 NotificationReceived()

```
AAX_Result AAX_CEffectDirectData::NotificationReceived (
    AAX_CTypeID inNotificationType,
    const void * inNotificationData,
    uint32_t inNotificationDataSize ) [virtual]
```

Notification Hook.

Called from the host to deliver notifications to this object.

Look at the [AAX_ENotificationEvent](#) enumeration to see a description of events you can listen for and the data they come with.

Note

- some notifications are sent only to the plug-in GUI or plug-in data model while other notifications are sent only to the EffectDirectData. If you are not seeing an expected notification, try checking the other plug-in objects' [NotificationReceived\(\)](#) methods.

Note

- the host may dispatch notifications synchronously or asynchronously, and calls to this method may occur concurrently on multiple threads.

Parameters

in	<i>inNotificationType</i>	Type of notification being received. Notifications from the host are one of AAX_ENotificationEvent
in	<i>inNotificationData</i>	Block of incoming notification data
in	<i>inNotificationDataSize</i>	Size of inNotificationData, in bytes

Implements [AAX_IACFEffDirectData_V2](#).

14.11.3.5 Controller()

```
AAX_IController* AAX_CEffectDirectData::Controller (
    void )
```

Returns a pointer to the plug-in's controller interface.

Todo Change to GetController to match other AAX_CEffect modules

14.11.3.6 EffectParameters()

```
AAX_IEffectParameters* AAX_CEffectDirectData::EffectParameters (
    void )
```

Returns a pointer to the plug-in's data model interface.

Todo Change to GetController to match other AAX_CEffect modules

14.11.3.7 Initialize_PrivateDataAccess()

```
virtual AAX_Result AAX_CEffectDirectData::Initialize_PrivateDataAccess ( ) [protected], [virtual]
```

Initialization routine for classes that inherit from [AAX_CEffectDirectData](#). This method is called by the default [Initialize\(\)](#) implementation after all internal members have been initialized, and provides a safe location in which to perform any additional initialization tasks.

14.11.3.8 TimerWakeup_PrivateDataAccess()

```
virtual AAX_Result AAX_CEffectDirectData::TimerWakeup_PrivateDataAccess (
    AAX_IPrivateDataAccess * iPrivateDataAccess ) [protected], [virtual]
```

Callback provided with an [AAX_IPrivateDataAccess](#). Override this method to access the algorithm's private data using the [AAX_IPrivateDataAccess](#) interface.

Parameters

in	<i>iPrivateDataAccess</i>	Pointer to an AAX_IPrivateDataAccess interface. This interface is only valid for the duration of this method.
----	---------------------------	---

The documentation for this class was generated from the following file:

- [AAX_CEffectDirectData.h](#)

14.12 AAX_CEffectGUI Class Reference

```
#include <AAX_CEffectGUI.h>
```

Inheritance diagram for AAX_CEffectGUI:

Collaboration diagram for AAX_CEffectGUI:

14.12.1 Description

Default implementation of the [AAX_IEffectGUI](#) interface.

This class provides a default implementation of the [AAX_IEffectGUI](#) interface.

Legacy Porting Notes The default implementations in this class are mostly derived from their equivalent implementations in CProcess and CEffectProcess. For additional CProcess-derived implementations, see [AAX_CEffectParameters](#).

Note

See [AAX_IACFEffectGUI](#) for further information.

Public Member Functions

- [AAX_CEffectGUI](#) (void)
- [~AAX_CEffectGUI](#) (void) [AAX_OVERRIDE](#)

Initialization and uninitialization

- [AAX_Result Initialize](#) ([IACFUnknown](#) *iController) [AAX_OVERRIDE](#)
Main GUI initialization.
- [AAX_Result Uninitialize](#) (void) [AAX_OVERRIDE](#)
Main GUI uninitialization.

AAX host and plug-in event notification

- [AAX_Result NotificationReceived](#) ([AAX_CTypeID](#) inNotificationType, const void *inNotificationData, uint32_t inNotificationDataSize) [AAX_OVERRIDE](#)
Notification Hook.

View accessors

- [AAX_Result SetViewContainer](#) ([IACFUnknown](#) *iViewContainer) [AAX_OVERRIDE](#)
Provides a handle to the main plug-in window.
- [AAX_Result GetViewSize](#) ([AAX_Point](#) *) const [AAX_OVERRIDE](#)
Retrieves the size of the plug-in window.

GUI update methods

- [AAX_Result Draw](#) ([AAX_Rect](#) *) [AAX_OVERRIDE](#)
DEPRECATED, Not called from host any longer. Your chosen graphics framework should be directly handling draw events from the OS.
- [AAX_Result TimerWakeup](#) (void) [AAX_OVERRIDE](#)
Periodic wakeup callback for idle-time operations.
- [AAX_Result ParameterUpdated](#) ([AAX_CParamID](#) paramID) [AAX_OVERRIDE](#)
Notifies the GUI that a parameter value has changed.

Host interface methods

Miscellaneous methods to provide host-specific functionality

- [AAX_Result GetCustomLabel](#) ([AAX_EPlugInStrings](#) iSelector, [AAX_IString](#) *oString) const [AAX_OVERRIDE](#)
Called by host application to retrieve a custom plug-in string.
- [AAX_Result SetControlHighlightInfo](#) ([AAX_CParamID](#), [AAX_CBoolean](#), [AAX_EHighlightColor](#)) [AAX_OVERRIDE](#)
Called by host application. Indicates that a control widget should be updated with a highlight color.

Protected Member Functions**AAX_CEffectGUI pure virtual interface**

The implementations of these methods will be specific to the particular GUI framework that is being incorporated with [AAX_CEffectGUI](#). Classes that inherit from [AAX_CEffectGUI](#) must override these methods with their own framework-specific implementations.

- virtual void [CreateViewContents](#) (void)=0
Creates any required top-level GUI components.
- virtual void [CreateViewContainer](#) (void)=0
Initializes the plug-in window and creates the main GUI view or frame.
- virtual void [DeleteViewContainer](#) (void)=0
Uninitializes the plug-in window and deletes the main GUI view or frame.

Helper methods

- virtual void [UpdateAllParameters](#) (void)
Requests an update to the GUI for every parameter view.

Private member accessors

- [AAX_IController](#) * [GetController](#) (void)
Retrieves a reference to the plug-in's controller interface.
- const [AAX_IController](#) * [GetController](#) (void) const
- [AAX_IEffectParameters](#) * [GetEffectParameters](#) (void)
Retrieves a reference to the plug-in's data model interface.
- const [AAX_IEffectParameters](#) * [GetEffectParameters](#) (void) const
- [AAX_IViewContainer](#) * [GetViewContainer](#) (void)
Retrieves a reference to the plug-in's view container interface.
- const [AAX_IViewContainer](#) * [GetViewContainer](#) (void) const
- [AAX_ITransport](#) * [Transport](#) ()
Retrieves a reference to the plug-in's Transport interface.
- const [AAX_ITransport](#) * [Transport](#) () const
- [AAX_EViewContainer_Type](#) [GetViewContainerType](#) ()
Retrieves the Container and it's type.
- void * [GetViewContainerPtr](#) ()

Additional Inherited Members

14.12.2 Constructor & Destructor Documentation

14.12.2.1 AAX_CEffectGUI()

```
AAX_CEffectGUI::AAX_CEffectGUI (
    void )
```

14.12.2.2 ~AAX_CEffectGUI()

```
AAX_CEffectGUI::~~AAX_CEffectGUI (
    void )
```

14.12.3 Member Function Documentation

14.12.3.1 Initialize()

```
AAX_Result AAX_CEffectGUI::Initialize (
    IACFUnknown * iController ) [virtual]
```

Main GUI initialization.

Called when the GUI is created

Parameters

in	<i>iController</i>	A versioned reference that resolves to an AAX_IController interface
----	--------------------	---

Implements [AAX_IACFEffectGUI](#).

14.12.3.2 Uninitialize()

```
AAX_Result AAX_CEffectGUI::Uninitialize (
    void ) [virtual]
```

Main GUI uninitialization.

Called when the GUI is destroyed. Frees the GUI.

Implements [AAX_IACFEffectGUI](#).

14.12.3.3 NotificationReceived()

```
AAX_Result AAX_CEffectGUI::NotificationReceived (
    AAX_CTypeID inNotificationType,
    const void * inNotificationData,
    uint32_t inNotificationDataSize ) [virtual]
```

Notification Hook.

Called from the host to deliver notifications to this object.

Look at the [AAX_ENotificationEvent](#) enumeration to see a description of events you can listen for and the data they come with.

Note

- some notifications are sent only to the plug-in GUI while other notifications are sent only to the plug-in data model. If you are not seeing an expected notification, try checking the other plug-in objects' [NotificationReceived\(\)](#) methods.

Note

- the host may dispatch notifications synchronously or asynchronously, and calls to this method may occur concurrently on multiple threads.

A plug-in may also dispatch custom notifications using [AAX_IController::SendNotification\(\)](#). Custom notifications will be posted back to the plug-in's other objects which support a [NotificationReceived\(\)](#) method (e.g. the data model).

Parameters

in	<i>inNotificationType</i>	Type of notification being received. Notifications from the host are one of AAX_ENotificationEvent
in	<i>inNotificationData</i>	Block of incoming notification data
in	<i>inNotificationDataSize</i>	Size of <i>inNotificationData</i> , in bytes

Note

The default implementation doesn't do anything at this point, but it is probably still a good idea to call into the base class [AAX_CEffectGUI::NotificationReceived\(\)](#) function in case we want to implement some default behaviors in the future.

Implements [AAX_IACFEEffectGUI](#).

14.12.3.4 SetViewContainer()

```
AAX_Result AAX_CEffectGUI::SetViewContainer (
    IACFUnknown * iViewContainer ) [virtual]
```

Provides a handle to the main plug-in window.

Parameters

in	<i>iViewController</i>	An AAX_IViewController providing a native handle to the plug-in's window
----	------------------------	--

Implements [AAX_IACFEffctGUI](#).

14.12.3.5 GetViewSize()

```
AAX_Result AAX_CEffectGUI::GetViewSize (
    AAX_Point * oViewSize ) const [inline], [virtual]
```

Retrieves the size of the plug-in window.

See also

[AAX_IViewController::SetViewSize\(\)](#)

Parameters

out	<i>oViewSize</i>	The size of the plug-in window as a point (width, height)
-----	------------------	---

Implements [AAX_IACFEffctGUI](#).

References [AAX_SUCCESS](#).

14.12.3.6 Draw()

```
AAX_Result AAX_CEffectGUI::Draw (
    AAX_Rect * iDrawRect ) [inline], [virtual]
```

DEPRECATED, Not called from host any longer. Your chosen graphics framework should be directly handling draw events from the OS.

Implements [AAX_IACFEffctGUI](#).

References [AAX_SUCCESS](#).

14.12.3.7 TimerWakeup()

```
AAX_Result AAX_CEffectGUI::TimerWakeup (
    void ) [inline], [virtual]
```

Periodic wakeup callback for idle-time operations.

GUI animation events such as meter updates should be triggered from this method.

This method is called from the host's main thread. In general, it should be driven at approximately one call per 30 ms. However, the wakeup is not guaranteed to be called at any regular interval - for example, it could be held off by a high real-time processing load - and there is no host contract regarding maximum latency between wakeup calls.

This wakeup runs continuously and cannot be armed/disarmed by the plug-in.

Implements [AAX_IACFEffectGUI](#).

References [AAX_SUCCESS](#).

14.12.3.8 ParameterUpdated()

```
AAX_Result AAX_CEffectGUI::ParameterUpdated (
    AAX_CParamID inParamID ) [virtual]
```

Notifies the GUI that a parameter value has changed.

This method is called by the host whenever a parameter value has been modified

This method may be called on a non-main thread

Implements [AAX_IACFEffectGUI](#).

14.12.3.9 GetCustomLabel()

```
AAX_Result AAX_CEffectGUI::GetCustomLabel (
    AAX_EPlugInStrings iSelector,
    AAX_IString * oString ) const [virtual]
```

Called by host application to retrieve a custom plug-in string.

If no string is provided then the host's default will be used.

Parameters

in	<i>iSelector</i>	The requested strong. One of AAX_EPlugInStrings
out	<i>oString</i>	The plug-in's custom value for the requested string

Implements [AAX_IACFEffectGUI](#).

14.12.3.10 SetControlHighlightInfo()

```
AAX_Result AAX_CEffectGUI::SetControlHighlightInfo (
    AAX_CParamID iParameterID,
    AAX_CBoolean iIsHighlighted,
    AAX_EHighlightColor iColor ) [inline], [virtual]
```

Called by host application. Indicates that a control widget should be updated with a highlight color.

Todo Document this method

Legacy Porting Notes This method was re-named from `SetControlHighliteInfo()`, its name in the legacy plug-in SDK.

Parameters

in	<i>iParameterID</i>	ID of parameter whose widget(s) must be highlighted
in	<i>iIsHighlighted</i>	True if turning highlight on, false if turning it off
in	<i>iColor</i>	Desired highlight color. One of AAX_EHighlightColor

Implements [AAX_IACFEEffectGUI](#).

References [AAX_SUCCESS](#).

14.12.3.11 CreateViewContents()

```
virtual void AAX_CEffectGUI::CreateViewContents (
    void ) [protected], [pure virtual]
```

Creates any required top-level GUI components.

This method is called by default from [AAX_CEffectGUI::Initialize\(\)](#)

14.12.3.12 CreateViewContainer()

```
virtual void AAX_CEffectGUI::CreateViewContainer (
    void ) [protected], [pure virtual]
```

Initializes the plug-in window and creates the main GUI view or frame.

This method is called by default from [AAX_CEffectGUI::SetViewContainer\(\)](#) when a valid window is present

14.12.3.13 DeleteViewContainer()

```
virtual void AAX_CEffectGUI::DeleteViewContainer (
    void ) [protected], [pure virtual]
```

Uninitializes the plug-in window and deletes the main GUI view or frame.

This method is called by default from [AAX_CEffectGUI::SetViewContainer\(\)](#) when no valid window is present. It may also be appropriate for inheriting classes to call this method from their destructors, depending on their own internal implementation.

14.12.3.14 UpdateAllParameters()

```
virtual void AAX_CEffectGUI::UpdateAllParameters (
    void ) [protected], [virtual]
```

Requests an update to the GUI for every parameter view.

By default, calls [AAX_CEffectGUI::ParameterUpdated\(\)](#) on every registered parameter.

By default, called from [AAX_CEffectGUI::SetViewContainer\(\)](#) after a new view container has been created.

Todo Rename to `UpdateAllParameterViews()` or another name that does not lead to confusion regarding what exactly this method should be doing.

14.12.3.15 GetController() [1/2]

```
AAX_IController* AAX_CEffectGUI::GetController (
    void )
```

Retrieves a reference to the plug-in's controller interface.

14.12.3.16 GetController() [2/2]

```
const AAX_IController* AAX_CEffectGUI::GetController (
    void ) const
```

14.12.3.17 GetEffectParameters() [1/2]

```
AAX_IEffectParameters* AAX_CEffectGUI::GetEffectParameters (
    void )
```

Retrieves a reference to the plug-in's data model interface.

14.12.3.18 GetEffectParameters() [2/2]

```
const AAX_IEffectParameters* AAX_CEffectGUI::GetEffectParameters (
    void ) const
```

14.12.3.19 GetViewContainer() [1/2]

```
AAX_IViewContainer* AAX_CEffectGUI::GetViewContainer (
    void )
```

Retrieves a reference to the plug-in's view container interface.

14.12.3.20 GetViewContainer() [2/2]

```
const AAX_IViewContainer* AAX_CEffectGUI::GetViewContainer (
    void ) const
```

14.12.3.21 Transport() [1/2]

```
AAX_ITransport* AAX_CEffectGUI::Transport ( )
```

Retrieves a reference to the plug-in's Transport interface.

14.12.3.22 Transport() [2/2]

```
const AAX_ITransport* AAX_CEffectGUI::Transport ( ) const
```

14.12.3.23 GetViewContainerType()

```
AAX_EViewContainer_Type AAX_CEffectGUI::GetViewContainerType ( )
```

Retrieves the Container and it's type.

14.12.3.24 GetViewContainerPtr()

```
void* AAX_CEffectGUI::GetViewContainerPtr ( )
```

The documentation for this class was generated from the following file:

- [AAX_CEffectGUI.h](#)

14.13 AAX_CEffectParameters Class Reference

```
#include <AAX_CEffectParameters.h>
```

Inheritance diagram for AAX_CEffectParameters:

Collaboration diagram for AAX_CEffectParameters:

14.13.1 Description

Default implementation of the [AAX_IEffectParameters](#) interface.

This class provides a default implementation of the [AAX_IEffectParameters](#) interface. In nearly all cases, your plug-in's data model should inherit from this class and override only those functions that you wish to explicitly customize.

Legacy Porting Notes The default implementations in this class are mostly derived from their equivalent implementations in CProcess and CEffectProcess. For additional CProcess-derived implementations, see [AAX_CEffectGUI](#).

14.13.2 Related classes

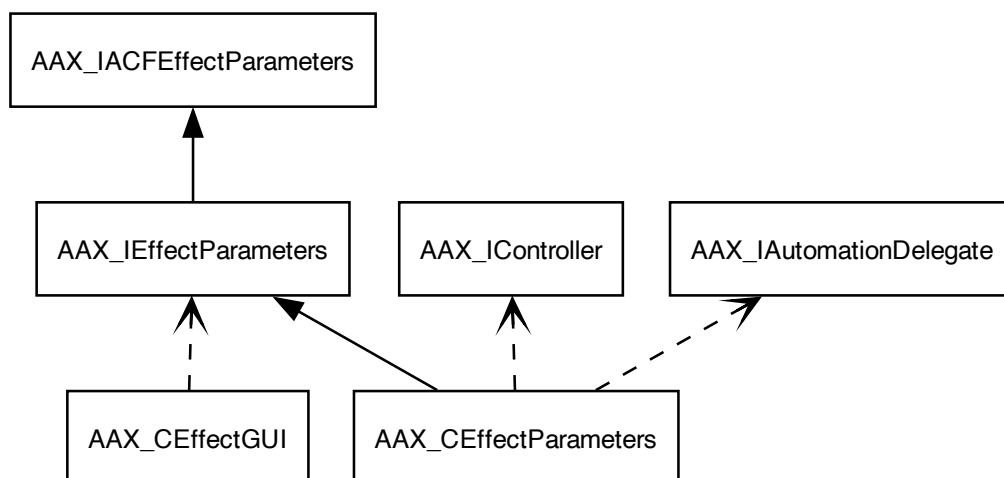


Figure 14.1 Classes related to AAX_IEffectParameters by inheritance or composition

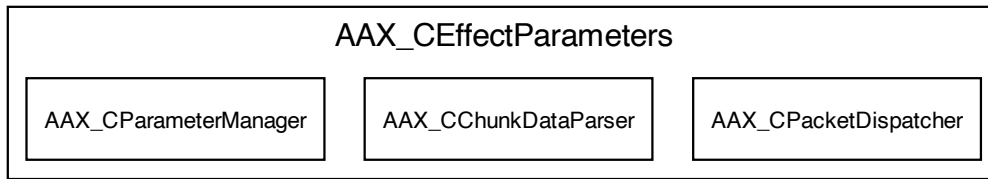


Figure 14.2 Classes owned as member objects of AAX_CEffectParameters

Public Member Functions

- [AAX_CEffectParameters](#) (void)
- [~AAX_CEffectParameters](#) (void) [AAX_OVERRIDE](#)
- [AAX_CEffectParameters](#) & [operator=](#) (const [AAX_CEffectParameters](#) &other)

Initialization and uninitialization

- [AAX_Result Initialize](#) ([IAcfUnknown](#) *iController) [AAX_OVERRIDE](#)
Main data model initialization. Called when plug-in instance is first instantiated.
- [AAX_Result Uninitialize](#) (void) [AAX_OVERRIDE](#)
Main data model uninitialization.

AAX host and plug-in event notification

- [AAX_Result NotificationReceived](#) ([AAX_CTypeID](#) inNotificationType, const void *inNotificationData, [uint32_t](#) inNotificationDataSize) [AAX_OVERRIDE](#)
Notification Hook.

Parameter information

These methods are used by the AAX host to retrieve information about the plug-in's data model. For information about adding parameters to the plug-in and otherwise modifying the plug-in's data model, see [AAX_CParameterManager](#). For information about parameters, see [AAX_IParameter](#).

- [AAX_Result GetNumberOfParameters](#) ([int32_t](#) *oNumControls) const [AAX_OVERRIDE](#)
CALL: Retrieves the total number of plug-in parameters.
- [AAX_Result GetMasterBypassParameter](#) ([AAX_IString](#) *oIDString) const [AAX_OVERRIDE](#)
CALL: Retrieves the ID of the plug-in's Master Bypass parameter.
- [AAX_Result GetParameterIsAutomatable](#) ([AAX_CParamID](#) iParameterID, [AAX_CBoolean](#) *oAutomatable) const [AAX_OVERRIDE](#)
CALL: Retrieves information about a parameter's automatable status.
- [AAX_Result GetParameterNumberOfSteps](#) ([AAX_CParamID](#) iParameterID, [int32_t](#) *oNumSteps) const [AAX_OVERRIDE](#)
CALL: Retrieves the number of discrete steps for a parameter.
- [AAX_Result GetParameterName](#) ([AAX_CParamID](#) iParameterID, [AAX_IString](#) *oName) const [AAX_OVERRIDE](#)
CALL: Retrieves the full name for a parameter.
- [AAX_Result GetParameterNameOfLength](#) ([AAX_CParamID](#) iParameterID, [AAX_IString](#) *oName, [int32_t](#) iNameLength) const [AAX_OVERRIDE](#)
CALL: Retrieves an abbreviated name for a parameter.

- [AAX_Result GetParameterDefaultNormalizedValue](#) ([AAX_CParamID](#) iParameterID, double *oValue) const [AAX_OVERRIDE](#)
CALL: Retrieves default value of a parameter.
- [AAX_Result SetParameterDefaultNormalizedValue](#) ([AAX_CParamID](#) iParameterID, double iValue) [AAX_OVERRIDE](#)
CALL: Sets the default value of a parameter.
- [AAX_Result GetParameterType](#) ([AAX_CParamID](#) iParameterID, [AAX_EParameterType](#) *oParameterType) const [AAX_OVERRIDE](#)
CALL: Retrieves the type of a parameter.
- [AAX_Result GetParameterOrientation](#) ([AAX_CParamID](#) iParameterID, [AAX_EParameterOrientation](#) *oParameterOrientation) const [AAX_OVERRIDE](#)
CALL: Retrieves the orientation that should be applied to a parameter's controls.
- [AAX_Result GetParameter](#) ([AAX_CParamID](#) iParameterID, [AAX_IParameter](#) **oParameter) [AAX_OVERRIDE](#)
CALL: Retrieves an arbitrary setting within a parameter.
- [AAX_Result GetParameterIndex](#) ([AAX_CParamID](#) iParameterID, int32_t *oControllIndex) const [AAX_OVERRIDE](#)
CALL: Retrieves the index of a parameter.
- [AAX_Result GetParameterIDFromIndex](#) (int32_t iControllIndex, [AAX_IString](#) *oParameterIDString) const [AAX_OVERRIDE](#)
CALL: Retrieves the ID of a parameter.
- [AAX_Result GetParameterValueInfo](#) ([AAX_CParamID](#) iParameterID, int32_t iSelector, int32_t *oValue) const [AAX_OVERRIDE](#)
CALL: Retrieves a property of a parameter.

Parameter setters and getters

These methods are used by the AAX host and by the plug-in's UI to retrieve and modify the values of the plug-in's parameters.

Note

The parameter setters in this section may generate asynchronous requests.

- [AAX_Result GetParameterValueFromString](#) ([AAX_CParamID](#) iParameterID, double *oValue, const [AAX_IString](#) &iValueString) const [AAX_OVERRIDE](#)
CALL: Converts a value string to a value.
- [AAX_Result GetParameterStringFromValue](#) ([AAX_CParamID](#) iParameterID, double iValue, [AAX_IString](#) *oValueString, int32_t iMaxLength) const [AAX_OVERRIDE](#)
CALL: Converts a normalized parameter value into a string representing its corresponding real value.
- [AAX_Result GetParameterValueString](#) ([AAX_CParamID](#) iParameterID, [AAX_IString](#) *oValueString, int32_t iMaxLength) const [AAX_OVERRIDE](#)
CALL: Retrieves the value string associated with a parameter's current value.
- [AAX_Result GetParameterNormalizedValue](#) ([AAX_CParamID](#) iParameterID, double *oValuePtr) const [AAX_OVERRIDE](#)
CALL: Retrieves a parameter's current value.
- [AAX_Result SetParameterNormalizedValue](#) ([AAX_CParamID](#) iParameterID, double iValue) [AAX_OVERRIDE](#)
CALL: Sets the specified parameter to a new value.
- [AAX_Result SetParameterNormalizedRelative](#) ([AAX_CParamID](#) iParameterID, double iValue) [AAX_OVERRIDE](#)
CALL: Sets the specified parameter to a new value relative to its current value.

Automated parameter helpers

These methods are used to lock and unlock automation system 'resources' when updating automatable parameters.

Note

You should never need to override these methods to extend their behavior beyond what is provided in [AAX_CEffectParameters](#) and [AAX_IParameter](#)

- [AAX_Result TouchParameter](#) ([AAX_CParamID](#) iParameterID) [AAX_OVERRIDE](#)
"Touches" (locks) a parameter in the automation system to a particular control in preparation for updates
- [AAX_Result ReleaseParameter](#) ([AAX_CParamID](#) iParameterID) [AAX_OVERRIDE](#)
Releases a parameter from a "touched" state.
- [AAX_Result UpdateParameterTouch](#) ([AAX_CParamID](#) iParameterID, [AAX_CBoolean](#) iTouchState) [AAX_OVERRIDE](#)
Sets a "touched" state on a parameter.

Asynchronous parameter update methods

These methods are called by the AAX host when parameter values have been updated. They are called by the host and can be triggered by other plug-in modules via calls to [AAX_IParameter](#)'s `SetValue` methods, e.g. [SetValueWithFloat\(\)](#)

These methods are responsible for updating parameter values.

Do not call these methods directly! To ensure proper synchronization and to avoid problematic dependency chains, other methods (e.g. [SetParameterNormalizedValue\(\)](#)) and components (e.g. [AAX_IEffectGUI](#)) should always call a `SetValue` method on [AAX_IParameter](#) to update parameter values. The `SetValue` method will properly manage automation locks and other system resources.

- [AAX_Result UpdateParameterNormalizedValue](#) ([AAX_CParamID](#) iParameterID, double iValue, [AAX_EUpdateSource](#) iSource) [AAX_OVERRIDE](#)
Updates a single parameter's state to its current value.
- [AAX_Result UpdateParameterNormalizedRelative](#) ([AAX_CParamID](#) iParameterID, double iValue) [AAX_OVERRIDE](#)
Updates a single parameter's state to its current value, as a difference with the parameter's previous value.
- [AAX_Result GenerateCoefficients](#) (void) [AAX_OVERRIDE](#)
Generates and dispatches new coefficient packets.

State reset handlers

- [AAX_Result ResetFieldData](#) ([AAX_CFieldIndex](#) inFieldIndex, void *oData, uint32_t inDataSize) const [AAX_OVERRIDE](#)
Called by the host to reset a private data field in the plug-in's algorithm.

Chunk methods

These methods are used to save and restore collections of plug-in state information, known as chunks. Chunks are used by the host when saving or restoring presets and session settings and when providing "compare" functionality for plug-ins.

The default implementation of these methods in [AAX_CEffectParameters](#) supports a single chunk that includes state information for all of the plug-in's registered parameters. Override all of these methods to add support for additional chunks in your plug-in, for example if your plug-in contains any persistent state that is not encapsulated by its set of registered parameters.

For reference, see also:

- [AAX_CChunkDataParser](#)
- [AAX_SPlugInChunk](#)
- [AAX_Result GetNumberOfChunks](#) (int32_t *oNumChunks) const [AAX_OVERRIDE](#)
Retrieves the number of chunks used by this plug-in.
- [AAX_Result GetChunkIDFromIndex](#) (int32_t iIndex, [AAX_CTypeID](#) *oChunkID) const [AAX_OVERRIDE](#)
Retrieves the ID associated with a chunk index.
- [AAX_Result GetChunkSize](#) ([AAX_CTypeID](#) iChunkID, uint32_t *oSize) const [AAX_OVERRIDE](#)

- Get the size of the data structure that can hold all of a chunk's information.
- [AAX_Result GetChunk](#) (AAX_CTypeID iChunkID, AAX_SPlugInChunk *oChunk) const [AAX_OVERRIDE](#)
Fills a block of data with chunk information representing the plug-in's current state.
- [AAX_Result SetChunk](#) (AAX_CTypeID iChunkID, const AAX_SPlugInChunk *iChunk) [AAX_OVERRIDE](#)
Restores a set of plug-in parameters based on chunk information.
- [AAX_Result CompareActiveChunk](#) (const AAX_SPlugInChunk *iChunkP, AAX_CBoolean *oIsEqual) const [AAX_OVERRIDE](#)
Determine if a chunk represents settings that are equivalent to the plug-in's current state.
- [AAX_Result GetNumberOfChanges](#) (int32_t *oNumChanges) const [AAX_OVERRIDE](#)
Retrieves the number of parameter changes made since the plug-in's creation.

Threads

Threading functions

- [AAX_Result TimerWakeup](#) () [AAX_OVERRIDE](#)
Periodic wakeup callback for idle-time operations.

Auxiliary UI methods

Methods defining the presentation of the plug-in on auxiliary UIs such as control surfaces

- [AAX_Result GetCurveData](#) (AAX_CTypeID iCurveType, const float *iValues, uint32_t iNumValues, float *oValues) const [AAX_OVERRIDE](#)
Generate a set of output values based on a set of given input values.
- [AAX_Result GetCurveDataMeterIds](#) (AAX_CTypeID iCurveType, uint32_t *oXMeterId, uint32_t *oYMeterId) const [AAX_OVERRIDE](#)
Indicates which meters correspond to the X and Y axes of the EQ or Dynamics graph.
- [AAX_Result GetCurveDataDisplayRange](#) (AAX_CTypeID iCurveType, float *oXMin, float *oXMax, float *oYMin, float *oYMax) const [AAX_OVERRIDE](#)
Determines the range of the graph shown by the plug-in.
- [AAX_Result UpdatePageTable](#) (uint32_t inTableType, int32_t inTablePageSize, IACFUnknown *iHostUnknown, IACFUnknown *ioPageTableUnknown) const [AAX_OVERRIDE](#) [AAX_FINAL](#)
Allow the plug-in to update its page tables.

Custom Data Methods

These functions exist as a proxiable way to move data between different modules (e.g. [AAX_IEffectParameters](#) and [AAX_IEffectGUI](#).) Using these, the GUI can query any data through [GetCustomData\(\)](#) with a plug-in defined *typeID*, *void** and *size*. This has an advantage over just sharing memory in that this function can work as a remote proxy as we enable those sorts of features later in the platform. Likewise, the GUI can also set arbitrary data on the data model by using the [SetCustomData\(\)](#) function with the same idea.

Note

These are plug-in internal only. They are not called from the host right now, or likely ever.

- [AAX_Result GetCustomData](#) (AAX_CTypeID iDataBlockID, uint32_t inDataSize, void *oData, uint32_t *oDataWritten) const [AAX_OVERRIDE](#)
An optional interface hook for getting custom data from another module.
- [AAX_Result SetCustomData](#) (AAX_CTypeID iDataBlockID, uint32_t inDataSize, const void *iData) [AAX_OVERRIDE](#)
An optional interface hook for setting custom data for use by another module.

MIDI methods

- [AAX_Result DoMIDITransfers](#) () [AAX_OVERRIDE](#)
MIDI update callback.
- [AAX_Result UpdateMIDINodes](#) (AAX_CFieldIndex inFieldIndex, AAX_CMidiPacket &iPacket) [AAX_OVERRIDE](#)

- *MIDI update callback.*
 • [AAX_Result UpdateControlMIDINodes](#) ([AAX_CTypeID](#) nodeId, [AAX_CMidiPacket](#) &iPacket) [AAX_OVERRIDE](#)
MIDI update callback for control MIDI nodes.

Hybrid audio methods

- [AAX_Result RenderAudio_Hybrid](#) ([AAX_SHybridRenderInfo](#) *ioRenderInfo) [AAX_OVERRIDE](#)
Hybrid audio render function.

Private data accessors

- [AAX_IController](#) * [Controller](#) ()
Access to the Effect controller.
- const [AAX_IController](#) * [Controller](#) () const
const access to the Effect controller
- [AAX_ITransport](#) * [Transport](#) ()
Access to the Transport object.
- const [AAX_ITransport](#) * [Transport](#) () const
const access to the Transport object
- [AAX_IAutomationDelegate](#) * [AutomationDelegate](#) ()
Access to the Effect's automation delegate.
- const [AAX_IAutomationDelegate](#) * [AutomationDelegate](#) () const
const access to the Effect's automation delegate

Protected Member Functions

Parameter management methods

- [AAX_Result SetTaperDelegate](#) ([AAX_CParamID](#) iParameterID, [AAX_ITaperDelegateBase](#) &iTaper↔
 Delegate, bool iPreserveValue)
- [AAX_Result SetDisplayDelegate](#) ([AAX_CParamID](#) iParameterID, [AAX_IDisplayDelegateBase](#) &iDisplay↔
 Delegate)
- bool [IsParameterTouched](#) ([AAX_CParamID](#) iParameterID) const
- bool [IsParameterLinkReady](#) ([AAX_CParamID](#) inParameterID, [AAX_EUpdateSource](#) inSource) const

Convenience functions

These convenience functions provide quick access to various aspects of the default [AAX_CEffectParameters](#) implementation.

- int32_t [mNumPlugInChanges](#)
- int32_t [mChunkSize](#)
- [AAX_CChunkDataParser](#) [mChunkParser](#)
- int32_t [mNumChunkedParameters](#)
- [AAX_CPacketDispatcher](#) [mPacketDispatcher](#)
- [AAX_CParameterManager](#) [mParameterManager](#)
- std::set< std::string > [mFilteredParameters](#)
- virtual [AAX_Result EffectInit](#) (void)
Initialization helper routine. Called from [AAX_CEffectParameters::Initialize](#).
- virtual [AAX_Result UpdatePageTable](#) (uint32_t, int32_t, [AAX_IPageTable](#) &) const
- void [FilterParameterIDOnSave](#) ([AAX_CParamID](#) controlId)
CALL: Indicates the indices of parameters that should not be saved in the default [AAX_CEffectParameters](#) chunk.
- void [BuildChunkData](#) (void) const
Clears out the current chunk in Chunk Parser and adds all of the new values. Used by default implementations of [GetChunk\(\)](#) and [GetChunkSize\(\)](#).

Additional Inherited Members

14.13.3 Constructor & Destructor Documentation

14.13.3.1 AAX_CEffectParameters()

```
AAX_CEffectParameters::AAX_CEffectParameters (
    void )
```

14.13.3.2 ~AAX_CEffectParameters()

```
AAX_CEffectParameters::~~AAX_CEffectParameters (
    void )
```

14.13.4 Member Function Documentation

14.13.4.1 operator=()

```
AAX_CEffectParameters& AAX_CEffectParameters::operator= (
    const AAX_CEffectParameters & other )
```

14.13.4.2 Initialize()

```
AAX_Result AAX_CEffectParameters::Initialize (
    IACFUnknown * iController ) [virtual]
```

Main data model initialization. Called when plug-in instance is first instantiated.

Note

Most plug-ins should override [AAX_CEffectParameters::EffectInit\(\)](#) rather than directly overriding this method

Parameters

in	<i>iController</i>	A versioned reference that resolves to an AAX_IController interface
----	--------------------	---

This default implementation calls [EffectInit\(\)](#). Only override [Initialize\(\)](#) when additional initialization steps must be

performed prior to [EffectInit\(\)](#).

Implements [AAX_IACFEffParameters](#).

14.13.4.3 Uninitialize()

```
AAX_Result AAX_CEffectParameters::Uninitialize (
    void ) [virtual]
```

Main data model uninitialization.

Todo Docs: When exactly is [AAX_IACFEffParameters::Uninitialize\(\)](#) called, and under what conditions?

Implements [AAX_IACFEffParameters](#).

14.13.4.4 NotificationReceived()

```
AAX_Result AAX_CEffectParameters::NotificationReceived (
    AAX_CTypeID inNotificationType,
    const void * inNotificationData,
    uint32_t inNotificationDataSize ) [virtual]
```

Notification Hook.

Called from the host to deliver notifications to this object.

Look at the [AAX_ENotificationEvent](#) enumeration to see a description of events you can listen for and the data they come with.

Note

- some notifications are sent only to the plug-in GUI while other notifications are sent only to the plug-in data model. If you are not seeing an expected notification, try checking the other plug-in objects' [NotificationReceived\(\)](#) methods.

Note

- the host may dispatch notifications synchronously or asynchronously, and calls to this method may occur concurrently on multiple threads.

A plug-in may also dispatch custom notifications using [AAX_IController::SendNotification\(\)](#). Custom notifications will be posted back to the plug-in's other objects which support a [NotificationReceived\(\)](#) method (e.g. the GUI).

Parameters

in	<i>inNotificationType</i>	Type of notification being received. Notifications from the host are one of AAX_ENotificationEvent
in	<i>inNotificationData</i>	Block of incoming notification data
in	<i>inNotificationDataSize</i>	Size of <i>inNotificationData</i> , in bytes

Implements [AAX_IACFEffectParameters](#).

14.13.4.5 GetNumberOfParameters()

```
AAX_Result AAX_CEffectParameters::GetNumberOfParameters (
    int32_t * oNumControls ) const [virtual]
```

CALL: Retrieves the total number of plug-in parameters.

Parameters

out	<i>oNumControls</i>	The number of parameters in the plug-in's Parameter Manager
-----	---------------------	---

Implements [AAX_IACFEffectParameters](#).

14.13.4.6 GetMasterBypassParameter()

```
AAX_Result AAX_CEffectParameters::GetMasterBypassParameter (
    AAX_IString * oIDString ) const [virtual]
```

CALL: Retrieves the ID of the plug-in's Master Bypass parameter.

This is required if you want our master bypass functionality in the host to hook up to your bypass parameters.

Parameters

out	<i>oIDString</i>	The ID of the plug-in's Master Bypass control
-----	------------------	---

Implements [AAX_IACFEffectParameters](#).

14.13.4.7 GetParameterIsAutomatable()

```
AAX_Result AAX_CEffectParameters::GetParameterIsAutomatable (
    AAX_CParamID iParameterID,
    AAX_CBoolean * oAutomatable ) const [virtual]
```

CALL: Retrieves information about a parameter's automatable status.

Parameters

in	<i>iParameterID</i>	The ID of the parameter that is being queried
out	<i>oAutomatable</i>	True if the queried parameter is automatable, false if it is not

Implements [AAX_IACFEffParameters](#).

14.13.4.8 GetParameterNumberOfSteps()

```
AAX_Result AAX_CEffectParameters::GetParameterNumberOfSteps (
    AAX_CParamID iParameterID,
    int32_t * oNumSteps ) const [virtual]
```

CALL: Retrieves the number of discrete steps for a parameter.

Note

The value returned for `oNumSteps` MUST be greater than zero. All other values will be considered an error by the host.

Parameters

in	<i>iParameterID</i>	The ID of the parameter that is being queried
out	<i>oNumSteps</i>	The number of steps for this parameter

Implements [AAX_IACFEffParameters](#).

14.13.4.9 GetParameterName()

```
AAX_Result AAX_CEffectParameters::GetParameterName (
    AAX_CParamID iParameterID,
    AAX_IString * oName ) const [virtual]
```

CALL: Retrieves the full name for a parameter.

Parameters

in	<i>iParameterID</i>	The ID of the parameter that is being queried
out	<i>oName</i>	Reference to an AAX_IString owned by the host. The plug-in must set this string equal to the parameter's full name.

Implements [AAX_IACFEffParameters](#).

14.13.4.10 GetParameterNameOfLength()

```
AAX_Result AAX_CEffectParameters::GetParameterNameOfLength (
    AAX_CParamID iParameterID,
```

```
AAX_IString * oName,
int32_t iNameLength ) const [virtual]
```

CALL: Retrieves an abbreviated name for a parameter.

In general, lengths of 3 through 8 and 31 should be specifically addressed.

Host Compatibility Notes In most cases, the AAX host will call [GetParameterName\(\)](#) or [GetParameterNameOfLength\(\)](#) to retrieve parameter names for display. However, when Pro Tools is retrieving a plug-in name for display on a control surface the XML data stored in the plug-in's page tables will be used in preference to values retrieved from these methods.

Parameters

in	<i>iParameterID</i>	The ID of the parameter that is being queried
out	<i>oName</i>	Reference to an AAX_IString owned by the host. The plug-in must set this string equal to an abbreviated name for the parameter, using <i>iNameLength</i> characters or fewer.
in	<i>iNameLength</i>	The maximum number of characters in <i>oName</i>

Implements [AAX_IACFEffectParameters](#).

14.13.4.11 GetParameterDefaultNormalizedValue()

```
AAX_Result AAX_CEffectParameters::GetParameterDefaultNormalizedValue (
    AAX_CParamID iParameterID,
    double * oValue ) const [virtual]
```

CALL: Retrieves default value of a parameter.

Parameters

in	<i>iParameterID</i>	The ID of the parameter that is being queried
out	<i>oValue</i>	The parameter's default value

Implements [AAX_IACFEffectParameters](#).

14.13.4.12 SetParameterDefaultNormalizedValue()

```
AAX_Result AAX_CEffectParameters::SetParameterDefaultNormalizedValue (
    AAX_CParamID iParameterID,
    double iValue ) [virtual]
```

CALL: Sets the default value of a parameter.

Parameters

in	<i>iParameterID</i>	The ID of the parameter that is being updated
out	<i>iValue</i>	The parameter's new default value

Todo THIS IS NOT CALLED FROM HOST. USEFUL FOR INTERNAL USE ONLY?

Implements [AAX_IACFEffParameters](#).

14.13.4.13 GetParameterType()

```
AAX_Result AAX_CEffectParameters::GetParameterType (
    AAX_CParamID iParameterID,
    AAX_EParameterType * oParameterType ) const [virtual]
```

CALL: Retrieves the type of a parameter.

Todo The concept of parameter type needs more documentation

Parameters

in	<i>iParameterID</i>	The ID of the parameter that is being queried
out	<i>oParameterType</i>	The parameter's type

Implements [AAX_IACFEffParameters](#).

14.13.4.14 GetParameterOrientation()

```
AAX_Result AAX_CEffectParameters::GetParameterOrientation (
    AAX_CParamID iParameterID,
    AAX_EParameterOrientation * oParameterOrientation ) const [virtual]
```

CALL: Retrieves the orientation that should be applied to a parameter's controls.

Todo update this documentation

This method allows you to specify the orientation of knob controls that are managed by the host (e.g. knobs on an attached control surface.)

Here is an example override of this method that reverses the orientation of a control for a parameter:

```
// AAX_IParameter* myBackwardsParameter
if (iParameterID == myBackwardsParameter->Identifier())
{
    *oParameterType =
```

```

    AAX_eParameterOrientation_BottomMinTopMax |
    AAX_eParameterOrientation_LeftMinRightMax |
    AAX_eParameterOrientation_RotaryWrapMode |
    AAX_eParameterOrientation_RotaryLeftMinRightMax;
}

```

The orientation options are set according to [AAX_EParameterOrientationBits](#)

Legacy Porting Notes [AAX_IEffectParameters::GetParameterOrientation\(\)](#) corresponds to the `GetControlOrientation()` method in the legacy RTAS/TDM SDK.

Parameters

in	<i>iParameterID</i>	The ID of the parameter that is being queried
out	<i>oParameterOrientation</i>	The orientation of the parameter

Implements [AAX_IACFEffectParameters](#).

14.13.4.15 GetParameter()

```

AAX_Result AAX_CEffectParameters::GetParameter (
    AAX_CParamID iParameterID,
    AAX_IParameter ** oParameter ) [virtual]

```

CALL: Retrieves an arbitrary setting within a parameter.

This is a convenience function for accessing the richer parameter interface from the plug-in's other modules.

Note

This function must not be called by the host; [AAX_IParameter](#) is not safe for passing across the binary boundary with the host!

Parameters

in	<i>iParameterID</i>	The ID of the parameter that is being queried
out	<i>oParameter</i>	A pointer to the returned parameter

Implements [AAX_IACFEffectParameters](#).

14.13.4.16 GetParameterIndex()

```

AAX_Result AAX_CEffectParameters::GetParameterIndex (
    AAX_CParamID iParameterID,
    int32_t * oControlIndex ) const [virtual]

```

CALL: Retrieves the index of a parameter.

Although parameters are normally referenced by their AAX_CParamID, each parameter is also associated with a unique numeric index.

Parameters

in	<i>iParameterID</i>	The ID of the parameter that is being queried
out	<i>oControlIndex</i>	The parameter's numeric index

Implements [AAX_IACFEffectParameters](#).

14.13.4.17 GetParameterIDFromIndex()

```
AAX_Result AAX_CEffectParameters::GetParameterIDFromIndex (
    int32_t iControlIndex,
    AAX_IString * oParameterIDString ) const [virtual]
```

CALL: Retrieves the ID of a parameter.

This method can be used to convert a parameter's unique numeric index to its AAX_CParamID

Parameters

in	<i>iControlIndex</i>	The numeric index of the parameter that is being queried
out	<i>oParameterIDString</i>	Reference to an AAX_IString owned by the host. The plug-in must set this string equal to the parameter's ID.

Implements [AAX_IACFEffectParameters](#).

14.13.4.18 GetParameterValueInfo()

```
AAX_Result AAX_CEffectParameters::GetParameterValueInfo (
    AAX_CParamID iParameterID,
    int32_t iSelector,
    int32_t * oValue ) const [virtual]
```

CALL: Retrieves a property of a parameter.

This is a general purpose query that is specialized based on the value of *iSelector*. The currently supported selector values are described by [AAX_EParameterValueInfoSelector](#). The meaning of *oValue* is dependent upon *iSelector*.

Parameters

in	<i>iParameterID</i>	The ID of the parameter that is being queried
in	<i>iSelector</i>	The selector of the parameter value to retrieve. See AAX_EParameterValueInfoSelector
out	<i>oValue</i>	The value of the specified parameter

Implements [AAX_IACFEffectParameters](#).

14.13.4.19 GetParameterValueFromString()

```
AAX_Result AAX_CEffectParameters::GetParameterValueFromString (
    AAX_CParamID iParameterID,
    double * oValue,
    const AAX_IString & iValueString ) const [virtual]
```

CALL: Converts a value string to a value.

This method uses the queried parameter's display delegate and taper to convert a `char*` string into its corresponding value. The formatting of valueString must be supported by the parameter's display delegate in order for this call to succeed.

Legacy Porting Notes This method corresponds to `CProcess::MapControlStringToVal()` in the RTAS/TDM SDK

Parameters

in	<i>iParameterID</i>	The ID of the parameter that is being queried
out	<i>oValue</i>	The value associated with valueString
in	<i>iValueString</i>	The formatted value string that will be converted into a value

Implements [AAX_IACFEffEffectParameters](#).

14.13.4.20 GetParameterStringFromValue()

```
AAX_Result AAX_CEffectParameters::GetParameterStringFromValue (
    AAX_CParamID iParameterID,
    double iValue,
    AAX_IString * oValueString,
    int32_t iMaxLength ) const [virtual]
```

CALL: Converts a normalized parameter value into a string representing its corresponding real value.

This method uses the queried parameter's display delegate and taper to convert a normalized value into the corresponding `char*` value string for its real value.

Legacy Porting Notes This method corresponds to `CProcess::MapControlValToString()` in the RTAS/TDM SDK

Parameters

in	<i>iParameterID</i>	The ID of the parameter that is being queried
in	<i>iValue</i>	The normalized value that will be converted to a formatted valueString
out	<i>oValueString</i>	The formatted value string associated with value
in	<i>iMaxLength</i>	The maximum length of valueString

Implements [AAX_IACFEffectParameters](#).

14.13.4.21 GetParameterValueString()

```
AAX_Result AAX_CEffectParameters::GetParameterValueString (
    AAX_CParamID iParameterID,
    AAX_IString * oValueString,
    int32_t iMaxLength ) const [virtual]
```

CALL: Retrieves the value string associated with a parameter's current value.

This method uses the queried parameter's display delegate and taper to convert its current value into a corresponding `char*` value string.

Parameters

in	<i>iParameterID</i>	The ID of the parameter that is being queried
out	<i>oValueString</i>	The formatted value string associated with the parameter's current value
in	<i>iMaxLength</i>	The maximum length of valueString

Implements [AAX_IACFEffectParameters](#).

14.13.4.22 GetParameterNormalizedValue()

```
AAX_Result AAX_CEffectParameters::GetParameterNormalizedValue (
    AAX_CParamID iParameterID,
    double * oValuePtr ) const [virtual]
```

CALL: Retrieves a parameter's current value.

Parameters

in	<i>iParameterID</i>	The ID of the parameter that is being queried
out	<i>oValuePtr</i>	The parameter's current value

Implements [AAX_IACFEffectParameters](#).

14.13.4.23 SetParameterNormalizedValue()

```
AAX_Result AAX_CEffectParameters::SetParameterNormalizedValue (
    AAX_CParamID iParameterID,
    double iValue ) [virtual]
```

CALL: Sets the specified parameter to a new value.

[SetParameterNormalizedValue\(\)](#) is responsible for initiating any process that is required in order to update all of the parameter's controls (e.g. in the plug-in's GUI, on control surfaces, in automation lanes, etc.) In most cases, the parameter manager will handle this initiation step.

Parameters

in	<i>iParameterID</i>	The ID of the parameter that is being set
in	<i>iValue</i>	The value to which the parameter should be set

Implements [AAX_IACFEffParameters](#).

14.13.4.24 SetParameterNormalizedRelative()

```
AAX_Result AAX_CEffectParameters::SetParameterNormalizedRelative (
    AAX_CParamID iParameterID,
    double iValue ) [virtual]
```

CALL: Sets the specified parameter to a new value relative to its current value.

This method is used in cases when a relative control value is more convenient, for example when updating a GUI control using a mouse wheel or the arrow keys. Note that the host may apply the parameter's step size prior to calling [SetParameterNormalizedRelative\(\)](#) in order to determine the correct value for aValue.

[SetParameterNormalizedRelative\(\)](#) can be used to incorporate "wrapping" behavior in a parameter's controls, if desired. If this behavior is not desired, then this method must properly account for overflow of the parameter's normalized value.

[SetParameterNormalizedRelative\(\)](#) is responsible for initiating any process that is required in order to update all of the parameter's controls (e.g. in the plug-in's GUI, on control surfaces, in automation lanes, etc.) In most cases, the parameter manager will handle this initiation step.

See also [UpdateParameterNormalizedRelative\(\)](#).

Todo REMOVE THIS METHOD (?)

Parameters

in	<i>iParameterID</i>	The ID of the parameter that is being queried
in	<i>iValue</i>	The change in value that should be applied to the parameter

Todo NOT CURRENTLY CALLED FROM THE HOST. USEFUL FOR INTERNAL USE ONLY?

Implements [AAX_IACFEffParameters](#).

14.13.4.25 TouchParameter()

```
AAX_Result AAX_CEffectParameters::TouchParameter (
    AAX_CParamID iParameterID ) [virtual]
```

"Touches" (locks) a parameter in the automation system to a particular control in preparation for updates

This method is called by the Parameter Manager to prime a parameter for receiving new automation data. When an automatable parameter is touched by a control, it will reject input from other controls until it is released.

Note

You should never need to override this method when using [AAX_CEffectParameters](#).

Parameters

in	<i>iParameterID</i>	The parameter that is being touched
----	---------------------	-------------------------------------

Implements [AAX_IACFEffParameters](#).

14.13.4.26 ReleaseParameter()

```
AAX_Result AAX_CEffectParameters::ReleaseParameter (
    AAX_CParamID iParameterID ) [virtual]
```

Releases a parameter from a "touched" state.

This method is called by the Parameter Manager to release a parameter so that any control may send updates to the parameter.

Note

You should never need to override this method when using [AAX_CEffectParameters](#).

Parameters

in	<i>iParameterID</i>	The parameter that is being released
----	---------------------	--------------------------------------

Implements [AAX_IACFEffParameters](#).

14.13.4.27 UpdateParameterTouch()

```
AAX_Result AAX_CEffectParameters::UpdateParameterTouch (
    AAX_CParamID iParameterID,
    AAX_CBoolean iTouchState ) [virtual]
```

Sets a "touched" state on a parameter.

Note

This method should be overridden when dealing with linked parameters. Do NOT use this method to keep track of touch states. Use the [automation delegate](#) for that.

Parameters

in	<i>iParameterID</i>	The parameter that is changing touch states.
in	<i>iTouchState</i>	The touch state of the parameter.

Implements [AAX_IACFEffParameters](#).

14.13.4.28 UpdateParameterNormalizedValue()

```
AAX_Result AAX_CEffectParameters::UpdateParameterNormalizedValue (
    AAX_CParamID iParameterID,
    double iValue,
    AAX_EUpdateSource iSource ) [virtual]
```

Updates a single parameter's state to its current value.

Note

Do *not* call this method from the plug-in. This method should be called by the host only. To set parameter values from within the plug-in, use the [AAX_IParameter](#) interface.

Todo FLAGGED FOR CONSIDERATION OF REVISION

Parameters

in	<i>iParameterID</i>	The ID of the parameter that is being updated
in	<i>iValue</i>	The parameter's current value, to which its internal state must be updated
in	<i>iSource</i>	The source of the update

Implements [AAX_IACFEffParameters](#).

Reimplemented in [AAX_CMonolithicParameters](#).

Referenced by `AAX_CMonolithicParameters::UpdateParameterNormalizedValue()`.

Here is the caller graph for this function:

14.13.4.29 UpdateParameterNormalizedRelative()

```
AAX_Result AAX_CEffectParameters::UpdateParameterNormalizedRelative (
    AAX_CParamID iParameterID,
    double iValue ) [virtual]
```

Updates a single parameter's state to its current value, as a difference with the parameter's previous value.

Deprecated This is not called from the host. It *may* still be useful for internal calls within the plug-in, though it should only ever be used to update non-automatable parameters. Automatable parameters should always be updated through the [AAX_IParameter](#) interface, which will ensure proper coordination with other automation clients.

[UpdateParameterNormalizedRelative\(\)](#) can be used to incorporate "wraparound" behavior in a parameter's controls, if desired. If this behavior is not desired, then this method must properly account for overflow of the parameter's normalized value.

See also

[SetParameterNormalizedRelative\(\)](#)

Parameters

in	<i>iParameterID</i>	The ID of the parameter that is being updated
in	<i>iValue</i>	The difference between the parameter's current value and its previous value (normalized). The parameter's state must be updated to reflect this difference.

Implements [AAX_IACFEffectParameters](#).

14.13.4.30 GenerateCoefficients()

```
AAX_Result AAX_CEffectParameters::GenerateCoefficients (
    void ) [virtual]
```

Generates and dispatches new coefficient packets.

This method is responsible for updating the coefficient packets associated with all parameters whose states have changed since the last call to [GenerateCoefficients\(\)](#). The host may call this method once for every parameter update, or it may "batch" parameter updates such that changes for several parameters are all handled by a single call to [GenerateCoefficients\(\)](#).

For more information on tracking parameters' statuses using the [AAX_CPacketDispatcher](#), helper class, see [AAX_CPacketDispatcher::SetDirty\(\)](#).

Note

Do *not* call this method from the plug-in. This method should be called by the host only. To set parameter values from within the plug-in, use the [AAX_IParameter](#) interface.

Implements [AAX_IACFEffectParameters](#).

Reimplemented in [AAX_CMonolithicParameters](#).

Referenced by [AAX_CMonolithicParameters::GenerateCoefficients\(\)](#).

Here is the caller graph for this function:

14.13.4.31 ResetFieldData()

```
AAX_Result AAX_CEffectParameters::ResetFieldData (
    AAX_CFieldIndex inFieldIndex,
    void * oData,
    uint32_t inDataSize ) const [virtual]
```

Called by the host to reset a private data field in the plug-in's algorithm.

This method is called sequentially for all private data fields on Effect initialization and during any "reset" event, such as priming for a non-real-time render. This method is called before the algorithm's optional initialization callback, and the initialized private data will be available within that callback via its context block.

See also

[Algorithm initialization.](#)

Warning

Any data structures that will be passed between platforms (for example, sent to a TI DSP in an AAX DSP plug-in) must be properly data-aligned for compatibility across both platforms. See [AAX_ALIGN_FILE_ALG](#) for more information about guaranteeing cross-platform compatibility of data structures used for algorithm processing.

Parameters

in	<i>inFieldIndex</i>	The index of the field that is being initialized
out	<i>oData</i>	The pre-allocated block of data that should be initialized
in	<i>inDataSize</i>	The size of the data block, in bytes

Implements [AAX_IACFEffEffectParameters](#).

Reimplemented in [AAX_CMonolithicParameters](#).

Referenced by [AAX_CMonolithicParameters::ResetFieldData\(\)](#).

Here is the caller graph for this function:

14.13.4.32 GetNumberOfChunks()

```
AAX_Result AAX_CEffectParameters::GetNumberOfChunks (
    int32_t * oNumChunks ) const [virtual]
```

Retrieves the number of chunks used by this plug-in.

Parameters

out	<i>oNumChunks</i>	The number of distinct chunks used by this plug-in
-----	-------------------	--

Implements [AAX_IACFEffEffectParameters](#).

14.13.4.33 GetChunkIDFromIndex()

```
AAX_Result AAX_CEffectParameters::GetChunkIDFromIndex (
    int32_t iIndex,
    AAX_CTypeID * oChunkID ) const [virtual]
```

Retrieves the ID associated with a chunk index.

Parameters

in	<i>iIndex</i>	Index of the queried chunk
out	<i>oChunkID</i>	ID of the queried chunk

Implements [AAX_IACFEffParameters](#).

14.13.4.34 GetChunkSize()

```
AAX_Result AAX_CEffectParameters::GetChunkSize (
    AAX_CTypeID iChunkID,
    uint32_t * oSize ) const [virtual]
```

Get the size of the data structure that can hold all of a chunk's information.

If *chunkID* is one of the plug-in's custom chunks, initialize **size* to the size of the chunk's data in bytes.

This method is invoked every time a chunk is saved, therefore it is possible to have dynamically sized chunks. However, note that each call to [GetChunkSize\(\)](#) will correspond to a following call to [GetChunk\(\)](#). The chunk provided in [GetChunk\(\)](#) *must* have the same size as the *size* provided by [GetChunkSize\(\)](#).

Legacy Porting Notes In *AAX*, the value provided by [GetChunkSize\(\)](#) should *NOT* include the size of the chunk header. The value should *ONLY* reflect the size of the chunk's data.

Parameters

in	<i>iChunkID</i>	ID of the queried chunk
out	<i>oSize</i>	The chunk's size in bytes

Implements [AAX_IACFEffParameters](#).

14.13.4.35 GetChunk()

```
AAX_Result AAX_CEffectParameters::GetChunk (
    AAX_CTypeID iChunkID,
    AAX_SPlugInChunk * oChunk ) const [virtual]
```

Fills a block of data with chunk information representing the plug-in's current state.

By calling this method, the host is requesting information about the current state of the plug-in. The following chunk fields should be explicitly populated in this method. Other fields will be populated by the host.

- [AAX_SPlugInChunk::fData](#)
- [AAX_SPlugInChunk::fVersion](#)
- [AAX_SPlugInChunk::fName](#) (Optional)
- [AAX_SPlugInChunk::fSize](#) (Data size only)

Warning

Remember that this chunk data may be loaded on a different platform from the one where it is saved. All data structures in the chunk must be properly data-aligned for compatibility across all platforms that the plug-in supports. See [AAX_ALIGN_FILE_ALG](#) for notes about common cross-platform pitfalls for data structure alignment.

Parameters

in	<i>iChunkID</i>	ID of the chunk that should be provided
out	<i>oChunk</i>	A preallocated block of memory that should be populated with the chunk's data.

Implements [AAX_IACFEffEffectParameters](#).

14.13.4.36 SetChunk()

```
AAX_Result AAX_CEffectParameters::SetChunk (
    AAX_CTypeID iChunkID,
    const AAX_SPlugInChunk * iChunk ) [virtual]
```

Restores a set of plug-in parameters based on chunk information.

By calling this method, the host is attempting to update the plug-in's current state to match the data stored in a chunk. The plug-in should initialize itself to this new state by calling [SetParameterNormalizedValue\(\)](#) for each of the relevant parameters.

Parameters

in	<i>iChunkID</i>	ID of the chunk that is being set
in	<i>iChunk</i>	The chunk

Implements [AAX_IACFEffEffectParameters](#).

14.13.4.37 CompareActiveChunk()

```
AAX_Result AAX_CEffectParameters::CompareActiveChunk (
    const AAX_SPlugInChunk * iChunkP,
    AAX_CBoolean * oIsEqual ) const [virtual]
```

Determine if a chunk represents settings that are equivalent to the plug-in's current state.

Host Compatibility Notes In Pro Tools, this method will only be called if a prior call to [GetNumberOfChanges\(\)](#) has indicated that the plug-in's state has changed. If the plug-in's current settings are different from the settings in `aChunkP` then the plug-in's Compare Light will be illuminated in the plug-in header, allowing users to toggle between the plug-in's custom state and its saved state.

Parameters

in	<i>iChunkP</i>	The chunk that is to be tested
out	<i>oIsEqual</i>	True if the chunk represents equivalent settings when compared with the plug-in's current state. False if the chunk represents non-equivalent settings

Implements [AAX_IACFEffParameters](#).

14.13.4.38 GetNumberOfChanges()

```
AAX_Result AAX_CEffectParameters::GetNumberOfChanges (
    int32_t * oNumChanges ) const [virtual]
```

Retrieves the number of parameter changes made since the plug-in's creation.

This method is polled regularly by the host, and can additionally be triggered by some events such as mouse clicks. When the number provided by this method changes, the host subsequently calls [CompareActiveChunk\(\)](#) to determine if the plug-in's Compare light should be activated.

The value provided by this method should increment with each call to [UpdateParameterNormalizedValue\(\)](#)

Parameters

out	<i>oNumChanges</i>	Must be set to indicate the number of parameter changes that have occurred since plug-in initialization.
-----	--------------------	--

Implements [AAX_IACFEffParameters](#).

14.13.4.39 TimerWakeup()

```
AAX_Result AAX_CEffectParameters::TimerWakeup ( ) [virtual]
```

Periodic wakeup callback for idle-time operations.

This method is called from the host using a non-main thread. In general, it should be driven at approximately one call per 30 ms. However, the wakeup is not guaranteed to be called at any regular interval - for example, it could be held off by a high real-time processing load - and there is no host contract regarding maximum latency between wakeup calls.

This wakeup thread runs continuously and cannot be armed/disarmed or by the plug-in.

Implements [AAX_IACFEffParameters](#).

Reimplemented in [AAX_CMonolithicParameters](#).

Referenced by `AAX_CMonolithicParameters::TimerWakeup()`.

Here is the caller graph for this function:

14.13.4.40 GetCurveData()

```
AAX_Result AAX_CEffectParameters::GetCurveData (
    AAX_CTypeID iCurveType,
    const float * iValues,
    uint32_t iNumValues,
    float * oValues ) const [virtual]
```

Generate a set of output values based on a set of given input values.

This method is used by the host to generate graphical curves. Given a set of input values, e.g. frequencies in Hz, this method should generate a corresponding set of output values, e.g. dB gain at each frequency. The semantics of these input and output values are dictated by `iCurveType`. See [AAX_ECurveType](#).

Plug-ins may also define custom curve type IDs to use this method internally. For example, the plug-in's GUI could use this method to request curve data in an arbitrary format.

Note

- This method may be called by the host simultaneously from multiple threads with different `iValues`.

Note

- `oValues` must be allocated by caller with the same size as `iValues` (`iNumValues`).

Host Compatibility Notes Versions of S6 software which support the [GetCurveDataDisplayRange\(\)](#) method will not display a plug-in's curve data unless both [GetCurveData\(\)](#) and [GetCurveDataDisplayRange\(\)](#) are supported by the plug-in.

Warning

S6 currently polls this method to update a plug-in's EQ or dynamics curves based on changes to the parameters mapped to the plug-in's EQ or dynamics center section page tables. Parameters that are not included in these page tables will not trigger updates to the curves displayed on S6. (GWSW-7314, [PTSW-195316 / PT-218485](#))

Parameters

in	<i>iCurveType</i>	One of AAX_ECurveType
in	<i>iValues</i>	An array of input values
in	<i>iNumValues</i>	The size of <i>iValues</i>
out	<i>oValues</i>	An array of output values

Returns

This method must return [AAX_ERROR_UNIMPLEMENTED](#) if the plug-in does not support curve data for the requested *iCurveType*

Implements [AAX_IACFEffectParameters](#).

14.13.4.41 GetCurveDataMeterIds()

```
AAX_Result AAX_CEffectParameters::GetCurveDataMeterIds (
    AAX_CTypeID iCurveType,
    uint32_t * oXMeterId,
    uint32_t * oYMeterId ) const [virtual]
```

Indicates which meters correspond to the X and Y axes of the EQ or Dynamics graph.

These meters can be used by attached control surfaces to present an indicator in the same X/Y coordinate plane as the plug-in's curve data.

Parameters

in	<i>iCurveType</i>	One of AAX_ECurveType
out	<i>oXMeterId</i>	Id of the X-axis meter
out	<i>oYMeterId</i>	Id of the Y-axis meter

Returns

This method should return [AAX_ERROR_UNIMPLEMENTED](#) if the plug-in does not implement it.

Implements [AAX_IACFEffectParameters_V3](#).

14.13.4.42 GetCurveDataDisplayRange()

```
AAX_Result AAX_CEffectParameters::GetCurveDataDisplayRange (
    AAX_CTypeID iCurveType,
    float * oXMin,
    float * oXMax,
    float * oYMin,
    float * oYMax ) const [virtual]
```

Determines the range of the graph shown by the plug-in.

Min/max arguments define the range of the axes of the graph.

Parameters

in	<i>iCurveType</i>	One of AAX_ECurveType
out	<i>oXMin</i>	Min value of X-axis range
out	<i>oXMax</i>	Max value of X-axis range
out	<i>oYMin</i>	Min value of Y-axis range
out	<i>oYMax</i>	Max value of Y-axis range

Returns

This method should return [AAX_ERROR_UNIMPLEMENTED](#) if the plug-in does not implement it.

Implements [AAX_IACFEffParameters_V3](#).

14.13.4.43 UpdatePageTable() [1/2]

```
AAX_Result AAX_CEffectParameters::UpdatePageTable (
    uint32_t inTableType,
    int32_t inTablePageSize,
    IACFUnknown * iHostUnknown,
    IACFUnknown * ioPageTableUnknown ) const [virtual]
```

Allow the plug-in to update its page tables.

Called by the plug-in host, usually in response to a [AAX_eNotificationEvent_ParameterMappingChanged](#) notification sent from the plug-in.

Use this method to change the page table mapping for the plug-in instance or to apply other changes to auxiliary UIs which use the plug-in page tables, such as setting focus to a new page.

See [Page Table Guide](#) for more information about page tables.

Parameters

in	<i>inTableType</i>	Four-char type identifier for the table type (e.g. 'PgTL', 'Av81', etc.)
in	<i>inTablePageSize</i>	Page size for the table
in	<i>iHostUnknown</i>	Unknown interface from the host which may support interfaces providing additional features or information. All interfaces queried from this unknown will be valid only within the scope of this UpdatePageTable() execution and will be relevant for only the current plug-in instance.
in, out	<i>ioPageTableUnknown</i>	Unknown interface which supports AAX_IPageTable . This object represents the page table data which is currently stored by the host for this plug-in instance for the given table type and page size. This data and may be edited within the scope of UpdatePageTable() to change the page table mapping for this plug-in instance.

Returns

This method should return [AAX_ERROR_UNIMPLEMENTED](#) if the plug-in does not implement it or when no change is requested by the plug-in. This allows optimizations to be used in the host when no UI update is required following this call.

See also

[AAX_eNotificationEvent_ParameterMappingChanged](#)

Note

For convenience, do not override this method. Instead, override the [protected overload](#) which provides a prepared copy of the relevant [AAX_IPageTable](#) host interface.

Implements [AAX_IACFEffectParameters_V4](#).

14.13.4.44 GetCustomData()

```
AAX_Result AAX_CEffectParameters::GetCustomData (
    AAX_CTypeID iDataBlockID,
    uint32_t inDataSize,
    void * oData,
    uint32_t * oDataWritten ) const [virtual]
```

An optional interface hook for getting custom data from another module.

Parameters

in	<i>iDataBlockID</i>	Identifier for the requested block of custom data
in	<i>inDataSize</i>	Size of provided buffer, in bytes
out	<i>oData</i>	Pointer to an allocated buffer. Data will be written here.
out	<i>oDataWritten</i>	The number of bytes actually written

Implements [AAX_IACFEffectParameters](#).

14.13.4.45 SetCustomData()

```
AAX_Result AAX_CEffectParameters::SetCustomData (
    AAX_CTypeID iDataBlockID,
    uint32_t inDataSize,
    const void * iData ) [virtual]
```

An optional interface hook for setting custom data for use by another module.

Parameters

in	<i>iDataBlockID</i>	Identifier for the provided block of custom data
in	<i>inDataSize</i>	Size of provided buffer, in bytes
in	<i>iData</i>	Pointer to the data buffer

Implements [AAX_IACFEffEffectParameters](#).

14.13.4.46 DoMIDITransfers()

```
AAX_Result AAX_CEffectParameters::DoMIDITransfers ( ) [inline], [virtual]
```

MIDI update callback.

Call [AAX_IController::GetNextMIDIPacket\(\)](#) from within this method to retrieve and process MIDI packets directly within the Effect's data model. MIDI data will also be delivered to the Effect algorithm.

This method is called regularly by the host, similarly to [AAX_IEffectParameters::TimerWakeup\(\)](#)

Implements [AAX_IACFEffEffectParameters](#).

References AAX_SUCCESS.

14.13.4.47 UpdateMIDINodes()

```
AAX_Result AAX_CEffectParameters::UpdateMIDINodes (
    AAX_CFieldIndex inFieldIndex,
    AAX_CMidiPacket & iPacket ) [virtual]
```

MIDI update callback.

This method is called by the host for each pending MIDI packet for MIDI nodes in algorithm context structure. Overwrite this method in Plug-In's EffectParameter class if you want to receive MIDI data packets directly in the data model. MIDI data will also be delivered to the Effect algorithm.

The host calls this method in Effects that register one or more MIDI nodes using [AAX_IComponentDescriptor::AddMIDINode\(\)](#). Effects that do not require MIDI data to be sent to the plug-in algorithm should override [UpdateControlMIDINodes\(\)](#).

Parameters

in	<i>inFieldIndex</i>	MIDI node field index in algorithm context structure
in	<i>iPacket</i>	The incoming MIDI packet for the node

Implements [AAX_IACFEffEffectParameters_V2](#).

14.13.4.48 UpdateControlMIDINodes()

```
AAX_Result AAX_CEffectParameters::UpdateControlMIDINodes (
    AAX_CTypeID nodeID,
    AAX_CMidiPacket & iPacket ) [virtual]
```

MIDI update callback for control MIDI nodes.

This method is called by the host for each pending MIDI packet for Control MIDI nodes. Overwrite this method in Plug-In's EffectParameter class if you want to receive MIDI data packets directly in the data model.

The host calls this method in Effects that register one or more Control MIDI nodes using [AAX_IEffectDescriptor::AddControlMIDINode\(\)](#). Effects with algorithms that use MIDI data nodes should override [UpdateMIDINodes\(\)](#).

Note

This method will not be called if an Effect includes any MIDI nodes in its algorithm context structure.

Parameters

in	<i>nodeID</i>	Identifier for the MIDI node
in	<i>iPacket</i>	The incoming MIDI packet for the node

Implements [AAX_IACFEffectParameters_V2](#).

14.13.4.49 RenderAudio_Hybrid()

```
AAX_Result AAX_CEffectParameters::RenderAudio_Hybrid (
    AAX_SHybridRenderInfo * ioRenderInfo ) [virtual]
```

Hybrid audio render function.

This method is called from the host to render audio for the hybrid piece of the algorithm.

Note

To use this method plug-in should register some hybrid inputs and outputs in "Describe"

Implements [AAX_IACFEffectParameters_V2](#).

14.13.4.50 Controller() [1/2]

```
AAX_IController* AAX_CEffectParameters::Controller ( )
```

Access to the Effect controller.

Referenced by [AAX_CMonolithicParameters::GenerateCoefficients\(\)](#).

Here is the caller graph for this function:

14.13.4.51 Controller() [2/2]

```
const AAX_IController* AAX_CEffectParameters::Controller ( ) const
```

const access to the Effect controller

14.13.4.52 Transport() [1/2]

```
AAX_ITransport* AAX_CEffectParameters::Transport ( )
```

Access to the Transport object.

14.13.4.53 Transport() [2/2]

```
const AAX_ITransport* AAX_CEffectParameters::Transport ( ) const
```

const access to the Transport object

14.13.4.54 AutomationDelegate() [1/2]

```
AAX_IAutomationDelegate* AAX_CEffectParameters::AutomationDelegate ( )
```

Access to the Effect's automation delegate.

14.13.4.55 AutomationDelegate() [2/2]

```
const AAX_IAutomationDelegate* AAX_CEffectParameters::AutomationDelegate ( ) const
```

const access to the Effect's automation delegate

14.13.4.56 SetTaperDelegate()

```
AAX_Result AAX_CEffectParameters::SetTaperDelegate (
    AAX_CParamID iParameterID,
    AAX_ITaperDelegateBase & iTaperDelegate,
    bool iPreserveValue ) [protected]
```

14.13.4.57 SetDisplayDelegate()

```
AAX_Result AAX_CEffectParameters::SetDisplayDelegate (
    AAX_CParamID iParameterID,
    AAX_IDisplayDelegateBase & iDisplayDelegate ) [protected]
```

14.13.4.58 IsParameterTouched()

```
bool AAX_CEffectParameters::IsParameterTouched (
    AAX_CParamID iParameterID ) const [protected]
```

14.13.4.59 IsParameterLinkReady()

```
bool AAX_CEffectParameters::IsParameterLinkReady (
    AAX_CParamID inParameterID,
    AAX_EUpdateSource inSource ) const [protected]
```

14.13.4.60 EffectInit()

```
virtual AAX_Result AAX_CEffectParameters::EffectInit (
    void ) [inline], [protected], [virtual]
```

Initialization helper routine. Called from [AAX_CEffectParameters::Initialize](#).

Override to add parameters, packets, meters, and to do any other custom initialization.

Add custom parameters:

- Create an [AAX_CParameter](#) for each parameter in the plug-in
- Call [AAX_CParameterManager::AddParameter\(\)](#) using mParameterManager to add parameters to the Parameter Manager

Note

See bug [AAXSDK-897](#)

Register packets:

- Call [AAX_CPacketDispatcher::RegisterPacket\(\)](#) using mPacketDispatcher to register a packet and handling callback.

References [AAX_SUCCESS](#).

14.13.4.61 UpdatePageTable() [2/2]

```
virtual AAX_Result AAX_CEffectParameters::UpdatePageTable (
    uint32_t ,
    int32_t ,
    AAX_IPageTable & ) const [inline], [protected], [virtual]
```

Protected overload of [UpdatePageTable\(\)](#)

Override this version of the method for convenience. This allows the default [UpdatePageTable\(\)](#) implementation to handle the interface conversion from [IACFUnknown](#) to [AAX_IPageTable](#).

Returns

This method should return [AAX_ERROR_UNIMPLEMENTED](#) if the plug-in does not implement it or when no change is made by the plug-in. This allows optimizations to be used in the host when no UI update is required following this call.

References [AAX_ERROR_UNIMPLEMENTED](#).

14.13.4.62 FilterParameterIDOnSave()

```
void AAX_CEffectParameters::FilterParameterIDOnSave (
    AAX_CParamID controlId ) [protected]
```

CALL: Indicates the indices of parameters that should not be saved in the default [AAX_CEffectParameters](#) chunk.

Allows specific parameters to be filtered out of the default [AAX_CEffectParameters](#) "Save Settings" functionality. This call is automatically invoked on the Master Bypass control when specified by the [DefineMasterBypassControlIndex\(\)](#) call.

Parameters

in	<i>controlID</i>	The ID of the parameter that should be removed from the default chunk
----	------------------	---

14.13.4.63 BuildChunkData()

```
void AAX_CEffectParameters::BuildChunkData (
    void ) const [protected]
```

Clears out the current chunk in Chunk Parser and adds all of the new values. Used by default implementations of [GetChunk\(\)](#) and [GetChunkSize\(\)](#).

14.13.5 Member Data Documentation

14.13.5.1 mNumPlugInChanges

`int32_t AAX_CEffectParameters::mNumPlugInChanges [protected]`

14.13.5.2 mChunkSize

`int32_t AAX_CEffectParameters::mChunkSize [mutable], [protected]`

14.13.5.3 mChunkParser

`AAX\_CChunkDataParser AAX_CEffectParameters::mChunkParser [mutable], [protected]`

14.13.5.4 mNumChunkedParameters

`int32_t AAX_CEffectParameters::mNumChunkedParameters [protected]`

14.13.5.5 mPacketDispatcher

`AAX\_CPacketDispatcher AAX_CEffectParameters::mPacketDispatcher [protected]`

14.13.5.6 mParameterManager

`AAX\_CParameterManager AAX_CEffectParameters::mParameterManager [protected]`

Referenced by `AAX_CMonolithicParameters::UpdateParameterNormalizedValue()`.

14.13.5.7 mFilteredParameters

`std::set<std::string> AAX_CEffectParameters::mFilteredParameters [protected]`

The documentation for this class was generated from the following file:

- [AAX_CEffectParameters.h](#)

14.14 AAX_CheckedResult Class Reference

```
#include <AAX_Exception.h>
```

14.14.1 Description

Error checker convenience class for [AAX_Result](#)

Implicitly convertible to an [AAX_Result](#).

Provides an overloaded `operator=()` which will throw an [AAX::Exception::ResultError](#) if assigned a non-success result.

Warning

Never use this class outside of an exception catch scope

If the host supports [AAX_TRACE](#) tracing, a log is emitted when the exception is thrown. A stacktrace is added if the host's trace priority filter level is set to [kAAX_Trace_Priority_Lowest](#)

When an error is encountered, [AAX_CheckedResult](#) throws an [AAX_CheckedResult::Exception](#) exception and clears its internal result value.

```
#include "AAX_Exception.h"
AAX_Result SomeCheckedMethod()
{
    AAX_Result result = AAX_SUCCESS;
    try {
        AAX_CheckedResult cr;
        cr = ResultFunc1();
        cr = ResultFunc2();
    }
    catch (const AAX_CheckedResult::Exception& ex)
    {
        // handle exception; do not rethrow
        result = ex.Result();
    }
    catch (...)
    {
        result = AAX_ERROR_UNKNOWN_EXCEPTION;
    }
    return result;
}
```

Note

The [AAX](#) Library method which calls [GetEffectDescriptions\(\)](#) on the plug-in includes an appropriate exception handler, so [AAX_CheckedResult](#) objects may be used within a plug-in's describe code without additional catch scopes.

```
#include "AAX_Exception.h"
AAX_Result GetEffectDescriptions( AAX_ICollection * outCollection )
{
    AAX_CheckedResult cr;
    cr = MyDescriptionSubroutine1();
    cr = outCollection->AddEffect(...);
    // etc.
    return cr;
}
```

It is assumed that the exception handler will resolve any error state and that the [AAX_CheckedResult](#) may therefore continue to be used from a clean state following the exception catch block.

If the previous error value is required then it can be retrieved using [AAX_CheckedResult::LastError\(\)](#).

```
// in this example, the exception is handled and
// success is returned from MyFunc1()
AAX_Result MyFunc1()
```

```

{
    AAX_CheckedResult cr;
    try {
        cr = MethodThatReturnsError();
    } catch (const AAX::Exception::ResultError& ex) {
        // exception is fully handled here
    }

    // cr now holds a success value
    return cr;
}

// in this example, MyFunc2() returns the first
// non-successful value which was encountered
AAX_Result MyFunc2()
{
    AAX_CheckedResult cr;
    try {
        AAX_SWALLOW(cr = MethodThatMayReturnError1());
        AAX_SWALLOW(cr = MethodThatMayReturnError2());
        cr = MethodThatMayReturnError3();
    } catch (const AAX::Exception::ResultError& ex) {
        // exception might not be fully handled
    }

    // pass the last error on to the caller
    return cr.LastError();
}

```

It is possible to add one or more accepted non-success values to an [AAX_CheckedResult](#) so that these values will not trigger exceptions:

```

AAX_CheckedResult cr;
try {
    cr.AddAcceptedResult(AcceptableErrCode);
    cr = MethodThatReturnsAcceptedError();
    cr = MethodThatReturnsAnotherError();
} catch (const AAX::Exception::ResultError& ex) {
    // handle the exception
}

```

Public Types

- typedef [AAX::Exception::ResultError](#) Exception

Public Member Functions

- [~AAX_CheckedResult](#) ()
- [AAX_CheckedResult](#) ()
Construct an [AAX_CheckedResult](#) in a success state.
- [AAX_CheckedResult](#) ([AAX_Result](#) inResult)
Implicit conversion constructor from [AAX_Result](#).
- void [AddAcceptedResult](#) ([AAX_Result](#) inResult)
Add an expected result which will not result in a throw.
- void [ResetAcceptedResults](#) ()
- [AAX_CheckedResult](#) & [operator=](#) ([AAX_Result](#) inResult)
Assignment to [AAX_Result](#).
- [AAX_CheckedResult](#) & [operator|=](#) ([AAX_Result](#) inResult)
bitwise-or assignment to [AAX_Result](#)
- [operator AAX_Result](#) () const
Conversion to [AAX_Result](#).
- void [Clear](#) ()
Clears the current result state.
- [AAX_Result](#) [LastError](#) () const
Get the last non-success result which was stored in this object, or [AAX_SUCCESS](#) if no non-success result was ever stored in this object.

14.14.2 Member Typedef Documentation

14.14.2.1 Exception

```
typedef AAX::Exception::ResultError AAX\_CheckedResult::Exception
```

14.14.3 Constructor & Destructor Documentation

14.14.3.1 ~AAX_CheckedResult()

```
AAX\_CheckedResult::~AAX\_CheckedResult ( ) [inline]
```

14.14.3.2 AAX_CheckedResult() [1/2]

```
AAX\_CheckedResult::AAX\_CheckedResult ( ) [inline]
```

Construct an [AAX_CheckedResult](#) in a success state.

14.14.3.3 AAX_CheckedResult() [2/2]

```
AAX\_CheckedResult::AAX\_CheckedResult (   
    AAX\_Result inResult ) [inline]
```

Implicit conversion constructor from [AAX_Result](#).

Implicit conversion is OK in order to support [AAX_CheckedResult](#) cr = SomeFunc()

14.14.4 Member Function Documentation

14.14.4.1 AddAcceptedResult()

```
void AAX\_CheckedResult::AddAcceptedResult (   
    AAX\_Result inResult ) [inline]
```

Add an expected result which will not result in a throw.

It is acceptable for some methods to return certain non-success values such as [AAX_RESULT_PACKET_STREAM_NOT_EMPTY](#) or [AAX_RESULT_NEW_PACKET_POSTED](#)

14.14.4.2 ResetAcceptedResults()

```
void AAX_CheckedResult::ResetAcceptedResults ( ) [inline]
```

References [AAX_SUCCESS](#).

14.14.4.3 operator=()

```
AAX_CheckedResult& AAX_CheckedResult::operator= (
    AAX_Result inResult ) [inline]
```

Assignment to [AAX_Result](#).

Referenced by `operator|=( )`.

Here is the caller graph for this function:

14.14.4.4 operator" |="()

```
AAX_CheckedResult& AAX_CheckedResult::operator|= (
    AAX_Result inResult ) [inline]
```

bitwise-or assignment to [AAX_Result](#)

Sometimes used in legacy code to aggregate results into a single [AAX_Result](#) value

References `operator=( )`.

Here is the call graph for this function:

14.14.4.5 operator AAX_Result()

```
AAX_CheckedResult::operator AAX_Result ( ) const [inline]
```

Conversion to [AAX_Result](#).

14.14.4.6 Clear()

```
void AAX_CheckedResult::Clear ( ) [inline]
```

Clears the current result state.

Does not affect the set of accepted results

References [AAX_SUCCESS](#).

14.14.4.7 LastError()

```
AAX_Result AAX_CheckedResult::LastError ( ) const [inline]
```

Get the last non-success result which was stored in this object, or AAX_SUCCESS if no non-success result was ever stored in this object.

The documentation for this class was generated from the following file:

- [AAX_Exception.h](#)

14.15 AAX_CHostProcessor Class Reference

```
#include <AAX_CHostProcessor.h>
```

Inheritance diagram for AAX_CHostProcessor:

Collaboration diagram for AAX_CHostProcessor:

14.15.1 Description

Concrete implementation of the [AAX_IHostProcessor](#) interface for non-real-time processing.

Host processor objects are used to process regions of audio data in a non-real-time context.

- Host processors must generate output samples linearly and incrementally, but may randomly access samples from the processing region on the timeline for input. See [GetAudio\(\)](#) for more information.
- Host processors may re-define the processing region using [AAX_CHostProcessor::TranslateOutputBounds\(\)](#).

See also

[AAX_IHostProcessorDelegate](#)

Public Member Functions

- [AAX_CHostProcessor](#) (void)
- virtual [~AAX_CHostProcessor](#) ()

Initialization and uninitialization

- [AAX_Result Initialize](#) (IACFUnknown *iController) [AAX_OVERRIDE](#)
Host Processor initialization.
- [AAX_Result Uninitialize](#) () [AAX_OVERRIDE](#)
Host Processor teardown.

Host processor interface

- [AAX_Result InitOutputBounds](#) (int64_t iSrcStart, int64_t iSrcEnd, int64_t *oDstStart, int64_t *oDstEnd) [AAX_OVERRIDE](#)
Sets the processing region.
- [AAX_Result SetLocation](#) (int64_t iSample) [AAX_OVERRIDE](#)
Updates the relative sample location of the current processing frame.
- [AAX_Result RenderAudio](#) (const float *const inAudioIns[], int32_t inAudioInCount, float *const iAudioOuts[], int32_t iAudioOutCount, int32_t *ioWindowSize) [AAX_OVERRIDE](#)
Perform the signal processing.
- [AAX_Result PreRender](#) (int32_t inAudioInCount, int32_t iAudioOutCount, int32_t iWindowSize) [AAX_OVERRIDE](#)
Invoked right before the start of a Preview or Render pass.
- [AAX_Result PostRender](#) () [AAX_OVERRIDE](#)
Invoked at the end of a Render pass.
- [AAX_Result AnalyzeAudio](#) (const float *const inAudioIns[], int32_t inAudioInCount, int32_t *ioWindowSize) [AAX_OVERRIDE](#)
Override this method if the plug-in needs to analyze the audio prior to a Render pass.
- [AAX_Result PreAnalyze](#) (int32_t inAudioInCount, int32_t iWindowSize) [AAX_OVERRIDE](#)
Invoked right before the start of an Analysis pass.
- [AAX_Result PostAnalyze](#) () [AAX_OVERRIDE](#)
Invoked at the end of an Analysis pass.
- [AAX_Result GetClipNameSuffix](#) (int32_t inMaxLength, [AAX_IString](#) *outString) const [AAX_OVERRIDE](#)
Called by host application to retrieve a custom string to be appended to the clip name.

Convenience methods

- [AAX_IEffectParameters](#) * [GetEffectParameters](#) ()
- const [AAX_IEffectParameters](#) * [GetEffectParameters](#) () const
- [AAX_IHostProcessorDelegate](#) * [GetHostProcessorDelegate](#) ()
- const [AAX_IHostProcessorDelegate](#) * [GetHostProcessorDelegate](#) () const
- int64_t [GetLocation](#) () const
The relative sample location of the current processing frame.
- int64_t [GetInputRange](#) () const
The length (in samples) of the current timeline selection.
- int64_t [GetOutputRange](#) () const
The length (in samples) of the clip that will be rendered to the timeline.
- int64_t [GetSrcStart](#) () const
The sample position of the beginning of the current timeline selection relative to the beginning of the current input selection, i.e. 0.
- int64_t [GetSrcEnd](#) () const
The sample position of the end of the current timeline selection relative to the beginning of the current input selection.
- int64_t [GetDstStart](#) () const
The sample position of the beginning of the of the clip that will be rendered to the timeline relative to the beginning of the current input selection.
- int64_t [GetDstEnd](#) () const
The sample position of the end of the of the clip that will be rendered to the timeline relative to the beginning of the current input selection.
- virtual [AAX_Result TranslateOutputBounds](#) (int64_t iSrcStart, int64_t iSrcEnd, int64_t &oDstStart, int64_t &oDstEnd)
Define the boundaries of the clip that will be rendered to the timeline.
- virtual [AAX_Result GetAudio](#) (const float *const inAudioIns[], int32_t inAudioInCount, int64_t inLocation, int32_t *ioNumSamples)
Randomly access audio from the timeline.
- virtual int32_t [GetSideChainInputNum](#) ()
CALL: Returns the index of the side chain input buffer.

- [AAX_IController](#) * [Controller](#) ()
- const [AAX_IController](#) * [Controller](#) () const
- [AAX_IHostProcessorDelegate](#) * [HostProcessorDelegate](#) ()
- const [AAX_IHostProcessorDelegate](#) * [HostProcessorDelegate](#) () const
- [AAX_IEffectParameters](#) * [EffectParameters](#) ()
- const [AAX_IEffectParameters](#) * [EffectParameters](#) () const

Additional Inherited Members

14.15.2 Constructor & Destructor Documentation

14.15.2.1 AAX_CHostProcessor()

```
AAX_CHostProcessor::AAX_CHostProcessor (
    void )
```

14.15.2.2 ~AAX_CHostProcessor()

```
virtual AAX_CHostProcessor::~~AAX_CHostProcessor ( ) [virtual]
```

14.15.3 Member Function Documentation

14.15.3.1 Initialize()

```
AAX_Result AAX_CHostProcessor::Initialize (
    IACFUnknown * iController ) [virtual]
```

Host Processor initialization.

Parameters

in	<i>iController</i>	A versioned reference that can be resolved to both an AAX_IController interface and an AAX_IHostProcessorDelegate
----	--------------------	---

Implements [AAX_IACFHostProcessor](#).

14.15.3.2 Uninitialize()

```
AAX_Result AAX_CHostProcessor::Uninitialize ( ) [virtual]
```

Host Processor teardown.

Implements [AAX_IACFHostProcessor](#).

14.15.3.3 InitOutputBounds()

```
AAX_Result AAX_CHostProcessor::InitOutputBounds (
    int64_t iSrcStart,
    int64_t iSrcEnd,
    int64_t * oDstStart,
    int64_t * oDstEnd ) [virtual]
```

Sets the processing region.

This method allows offline processing plug-ins to vary the length and/or start/end points of the audio processing region.

This method is called in a few different scenarios:

- Before an analyze, process or preview of data begins.
- At the end of every preview loop.
- After the user makes a new data selection on the timeline.

Plug-ins that inherit from [AAX_CHostProcessor](#) should not override this method. Instead, use the following convenience functions:

- To retrieve the length or boundaries of the processing region, use [GetInputRange\(\)](#), [GetSrcStart\(\)](#), etc.
- To change the boundaries of the processing region before processing begins, use [AAX_CHostProcessor::TranslateOutputBounds\(\)](#).

Note

Currently, a host processor may not randomly access samples outside of the boundary defined by `oDstStart` and `oDstEnd`.

Legacy Porting Notes DAE no longer makes use of the `mStartBound` and `mEndBounds` member variables that existed in the legacy RTAS/TDM SDK. Use `oDstStart` and `oDstEnd` instead (preferably by overriding [TranslateOutputBounds\(\)](#).)

Parameters

in	<i>iSrcStart</i>	The selection start of the user selected region. This will always return 0 for a given selection on the timeline.
in	<i>iSrcEnd</i>	The selection end of the user selected region. This will always return the value of the selection length on the timeline.
Generated by Doxygen	in <i>oDstStart</i>	The starting sample location in the output audio region. By default, this is the same as <i>iSrcStart</i> .
	in <i>oDstEnd</i>	The ending sample location in the output audio region. By default, this is the same as <i>iSrcEnd</i> .

Implements [AAX_IACFHostProcessor](#).

14.15.3.4 SetLocation()

```
AAX_Result AAX_CHostProcessor::SetLocation (
    int64_t iSample ) [virtual]
```

Updates the relative sample location of the current processing frame.

This method is called by the host to update the relative sample location of the current processing frame.

Note

Plug-ins should not override this method; instead, use [AAX_CHostProcessor::GetLocation\(\)](#) to retrieve the current relative sample location.

Parameters

in	<i>iSample</i>	The sample location of the first sample in the current processing frame relative to the beginning of the full processing buffer
----	----------------	---

Implements [AAX_IACFHostProcessor](#).

14.15.3.5 RenderAudio()

```
AAX_Result AAX_CHostProcessor::RenderAudio (
    const float *const inAudioIns[],
    int32_t inAudioInCount,
    float *const iAudioOuts[],
    int32_t iAudioOutCount,
    int32_t * ioWindowSize ) [virtual]
```

Perform the signal processing.

This method is called by the host to invoke the plug-in's signal processing.

Legacy Porting Notes This method is a replacement for the AudioSuite `ProcessAudio` method

Parameters

in	<i>inAudioIns</i>	Input audio buffer
in	<i>inAudioInCount</i>	The number of input channels
in	<i>iAudioOuts</i>	The number of output channels
in	<i>iAudioOutCount</i>	A user defined destination end of the ingested audio
in	<i>ioWindowSize</i>	Window buffer length of the received audio

Implements [AAX_IACFHostProcessor](#).

14.15.3.6 PreRender()

```
AAX_Result AAX_CHostProcessor::PreRender (
    int32_t inAudioInCount,
    int32_t iAudioOutCount,
    int32_t iWindowSize ) [virtual]
```

Invoked right before the start of a Preview or Render pass.

This method is called by the host to allow a plug-in to make any initializations before processing actually begins. Upon a Preview pass, PreRender will also be called at the beginning of every "loop".

See also

[AAX_eProcessingState_StartPass](#), [AAX_eProcessingState_BeginPassGroup](#)

Parameters

in	<i>inAudioInCount</i>	The number if input channels
in	<i>iAudioOutCount</i>	The number of output channels
in	<i>iWindowSize</i>	Window buffer length of the ingested audio

Implements [AAX_IACFHostProcessor](#).

14.15.3.7 PostRender()

```
AAX_Result AAX_CHostProcessor::PostRender ( ) [virtual]
```

Invoked at the end of a Render pass.

Note

Upon a Preview pass, PostRender will not be called until Preview has stopped.

See also

[AAX_eProcessingState_StopPass](#), [AAX_eProcessingState_EndPassGroup](#)

Implements [AAX_IACFHostProcessor](#).

14.15.3.8 AnalyzeAudio()

```
AAX_Result AAX_CHostProcessor::AnalyzeAudio (
    const float *const inAudioIns[],
    int32_t inAudioInCount,
    int32_t * ioWindowSize ) [virtual]
```

Override this method if the plug-in needs to analyze the audio prior to a Render pass.

Use this after declaring the appropriate properties in Describe. See [AAX_eProperty_RequiresAnalysis](#) and [AAX_eProperty_OptionalAnalysis](#)

To request an analysis pass from within a plug-in, use [AAX_IHostProcessorDelegate::ForceAnalyze\(\)](#)

Legacy Porting Notes Ported from AudioSuite's `AnalyzeAudio(bool isMasterBypassed)` method

Parameters

in	<i>inAudioIns</i>	Input audio buffer
in	<i>inAudioInCount</i>	The number of input channels
in	<i>ioWindowSize</i>	Window buffer length of the ingested audio

Implements [AAX_IACFHostProcessor](#).

14.15.3.9 PreAnalyze()

```
AAX_Result AAX_CHostProcessor::PreAnalyze (
    int32_t inAudioInCount,
    int32_t iWindowSize ) [virtual]
```

Invoked right before the start of an Analysis pass.

This method is called by the host to allow a plug-in to make any initializations before an Analysis pass actually begins.

See also

[AAX_eProcessingState_StartPass](#), [AAX_eProcessingState_BeginPassGroup](#)

Parameters

in	<i>inAudioInCount</i>	The number if input channels
in	<i>iWindowSize</i>	Window buffer length of the ingested audio

Implements [AAX_IACFHostProcessor](#).

14.15.3.10 PostAnalyze()

```
AAX_Result AAX_CHostProcessor::PostAnalyze ( ) [virtual]
```

Invoked at the end of an Analysis pass.

Note

In Pro Tools, a long execution time for this method will hold off the main application thread and cause a visible hang. If the plug-in must perform any long running tasks before initiating processing then it is best to perform these tasks in [AAX_IHostProcessor::PreRender\(\)](#)

See also

[AAX_eProcessingState_StopPass](#), [AAX_eProcessingState_EndPassGroup](#)

Implements [AAX_IACFHostProcessor](#).

14.15.3.11 GetClipNameSuffix()

```
AAX_Result AAX_CHostProcessor::GetClipNameSuffix (
    int32_t inMaxLength,
    AAX_IString * outString ) const [virtual]
```

Called by host application to retrieve a custom string to be appended to the clip name.

If no string is provided then the host's default will be used.

Parameters

in	<i>inMaxLength</i>	The maximum allowed string length, not including the NULL terminating char
out	<i>outString</i>	Add a value to this string to provide a custom clip suffix

Implements [AAX_IACFHostProcessor_V2](#).

14.15.3.12 GetEffectParameters() [1/2]

```
AAX_IEffectParameters* AAX_CHostProcessor::GetEffectParameters (
    void ) [inline]
```

14.15.3.13 GetEffectParameters() [2/2]

```
const AAX_IEffectParameters* AAX_CHostProcessor::GetEffectParameters (
    void ) const [inline]
```

14.15.3.14 GetHostProcessorDelegate() [1/2]

```
AAX_IHostProcessorDelegate* AAX_CHostProcessor::GetHostProcessorDelegate ( ) [inline]
```

14.15.3.15 GetHostProcessorDelegate() [2/2]

```
const AAX_IHostProcessorDelegate* AAX_CHostProcessor::GetHostProcessorDelegate ( ) const [inline]
```

14.15.3.16 GetLocation()

```
int64_t AAX_CHostProcessor::GetLocation ( ) const [inline]
```

The relative sample location of the current processing frame.

This method returns the relative sample location for the current [RenderAudio\(\)](#) processing frame. For example, if a value of 10 is provided for the [RenderAudio\(\)](#) `ioWindow` parameter, then calls to this method from within each execution of [RenderAudio\(\)](#) will return 0, 10, 20,...

14.15.3.17 GetInputRange()

```
int64_t AAX_CHostProcessor::GetInputRange ( ) const [inline]
```

The length (in samples) of the current timeline selection.

14.15.3.18 GetOutputRange()

```
int64_t AAX_CHostProcessor::GetOutputRange ( ) const [inline]
```

The length (in samples) of the clip that will be rendered to the timeline.

14.15.3.19 GetSrcStart()

```
int64_t AAX_CHostProcessor::GetSrcStart ( ) const [inline]
```

The sample position of the beginning of the current timeline selection relative to the beginning of the current input selection, i.e. 0.

14.15.3.20 GetSrcEnd()

```
int64_t AAX_CHostProcessor::GetSrcEnd ( ) const [inline]
```

The sample position of the end of the current timeline selection relative to the beginning of the current input selection.

14.15.3.21 GetDstStart()

```
int64_t AAX_CHostProcessor::GetDstStart ( ) const [inline]
```

The sample position of the beginning of the of the clip that will be rendered to the timeline relative to the beginning of the current input selection.

This value will be equal to the value returned by [GetSrcStart\(\)](#) unless the selection boundaries have been modified by overriding [TranslateOutputBounds\(\)](#)

14.15.3.22 GetDstEnd()

```
int64_t AAX_CHostProcessor::GetDstEnd ( ) const [inline]
```

The sample position of the end of the of the clip that will be rendered to the timeline relative to the beginning of the current input selection.

This value will be equal to the value returned by [GetSrcStart\(\)](#) unless the selection boundaries have been modified by overriding [TranslateOutputBounds\(\)](#)

14.15.3.23 TranslateOutputBounds()

```
virtual AAX_Result AAX_CHostProcessor::TranslateOutputBounds (
    int64_t iSrcStart,
    int64_t iSrcEnd,
    int64_t & oDstStart,
    int64_t & oDstEnd ) [protected], [virtual]
```

Define the boundaries of the clip that will be rendered to the timeline.

This method is called from [AAX_CHostProcessor::InitOutputBounds\(\)](#), providing a convenient hook for re-defining the processing region boundaries. See [InitOutputBounds\(\)](#) for more information.

Parameters

in	<i>iSrcStart</i>	The selection start of the user selected region. This is will always return 0 for a given selection on the timeline.
in	<i>iSrcEnd</i>	The selection end of the user selected region. This will always return the value of the selection length on the timeline.
in	<i>oDstStart</i>	The starting sample location in the output audio region. By default, this is the same as <i>iSrcStart</i> .
in	<i>oDstEnd</i>	The ending sample location in the output audio region. By default, this is the same as <i>iSrcEnd</i> .

14.15.3.24 GetAudio()

```
virtual AAX_Result AAX_CHostProcessor::GetAudio (
    const float *const inAudioIns[],
    int32_t inAudioInCount,
    int64_t inLocation,
    int32_t * ioNumSamples ) [protected], [virtual]
```

Randomly access audio from the timeline.

This is a convenience wrapper around [AAX_IHostProcessorDelegate::GetAudio\(\)](#).

Parameters

in	<i>inAudioIns</i>	Timeline audio buffer(s). This must be set to <i>inAudioIns</i> from AAX_IHostProcessor::RenderAudio()
in	<i>inAudioInCount</i>	Number of buffers in <i>inAudioIns</i> . This must be set to <i>inAudioInCount</i> from AAX_IHostProcessor::RenderAudio()
in	<i>inLocation</i>	A sample location relative to the beginning of the currently processed region, e.g. a value of 0 corresponds to the timeline location returned by AAX_CHostProcessor::GetSrcStart()
in, out	<i>ioNumSamples</i>	- Input: The maximum number of samples to read. - Output: The actual number of samples that were read from the timeline

14.15.3.25 GetSideChainInputNum()

```
virtual int32_t AAX_CHostProcessor::GetSideChainInputNum ( ) [protected], [virtual]
```

CALL: Returns the index of the side chain input buffer.

This is a convenience wrapper around [AAX_IHostProcessorDelegate::GetSideChainInputNum\(\)](#)

14.15.3.26 Controller() [1/2]

```
AAX_IController* AAX_CHostProcessor::Controller (
    void ) [inline], [protected]
```

14.15.3.27 Controller() [2/2]

```
const AAX_IController* AAX_CHostProcessor::Controller (
    void ) const [inline], [protected]
```

14.15.3.28 HostProcessorDelegate() [1/2]

```
AAX_IHostProcessorDelegate* AAX_CHostProcessor::HostProcessorDelegate ( ) [inline], [protected]
```

14.15.3.29 HostProcessorDelegate() [2/2]

```
const AAX_IHostProcessorDelegate* AAX_CHostProcessor::HostProcessorDelegate ( ) const [inline], [protected]
```

14.15.3.30 EffectParameters() [1/2]

```
AAX_IEffectParameters* AAX_CHostProcessor::EffectParameters ( ) [inline], [protected]
```

14.15.3.31 EffectParameters() [2/2]

```
const AAX_IEffectParameters* AAX_CHostProcessor::EffectParameters ( ) const [inline], [protected]
```

The documentation for this class was generated from the following file:

- [AAX_CHostProcessor.h](#)

14.16 AAX_CHostServices Class Reference

```
#include <AAX_CHostServices.h>
```

14.16.1 Description

Method access to a singleton implementation of the [AAX_IHostServices](#) interface.

Static Public Member Functions

- static void [Set](#) ([IACFUnknown](#) *pUnkHost)
- static [AAX_Result](#) [HandleAssertFailure](#) (const char *iFile, int32_t iLine, const char *iNote, int32_t iFlags=[AAX_eAssertFlags_Default](#))
Handle an assertion failure.
- static [AAX_Result](#) [Trace](#) ([AAX_ETracePriorityHost](#) iPriority, const char *iMessage,...)
Log a trace message.
- static [AAX_Result](#) [StackTrace](#) ([AAX_ETracePriorityHost](#) iTracePriority, [AAX_ETracePriorityHost](#) iStack←
TracePriority, const char *iMessage,...)
Log a trace message or a stack trace.

14.16.2 Member Function Documentation

14.16.2.1 Set()

```
static void AAX_CHostServices::Set (
    IACFUnknown * pUnkHost ) [static]
```

14.16.2.2 HandleAssertFailure()

```
static AAX_Result AAX_CHostServices::HandleAssertFailure (
    const char * iFile,
    int32_t iLine,
    const char * iNote,
    int32_t iFlags = AAX_eAssertFlags_Default ) [static]
```

Handle an assertion failure.

Use this method to delegate assertion failure handling to the host

Use `iFlags` to request that specific behavior be included when handling the failure. This request may not be fulfilled by the host, and absence of a flag does not preclude the host from using that behavior when handling the failure.

Parameters

in	<i>iFile</i>	The name of the file containing the assert check. Usually <code>__FILE__</code>
in	<i>iLine</i>	The line number of the assert check. Usually <code>__LINE__</code>
in	<i>iNote</i>	Text to display related to the assert. Usually the condition which failed
in	<i>iFlags</i>	Bitfield of AAX_EAssertFlags to request specific handling behavior

14.16.2.3 Trace()

```
static AAX_Result AAX_CHostServices::Trace (
    AAX_ETracePriorityHost iPriority,
    const char * iMessage,
    ... ) [static]
```

Log a trace message.

Parameters

in	<i>iPriority</i>	Priority of the trace, used for log filtering. One of kAAX_Trace_Priority_Low , kAAX_Trace_Priority_Normal , kAAX_Trace_Priority_High
in	<i>iMessage</i>	Message string to log

14.16.2.4 StackTrace()

```
static AAX_Result AAX_CHostServices::StackTrace (
    AAX_ETracePriorityHost iTracePriority,
    AAX_ETracePriorityHost iStackTracePriority,
    const char * iMessage,
    ... ) [static]
```

Log a trace message or a stack trace.

If the logging output filtering is set to include logs with `iStackTracePriority` then both the logging message and a stack trace will be emitted, regardless of `iTracePriority`.

If the logging output filtering is set to include logs with `iTracePriority` but to exclude logs with `iStackTracePriority` then this will emit a normal log with no stack trace.

Parameters

in	<i>iTracePriority</i>	Priority of the trace, used for log filtering. One of kAAX_Trace_Priority_Low , kAAX_Trace_Priority_Normal , kAAX_Trace_Priority_High
in	<i>iStackTracePriority</i>	Priority of the stack trace, used for log filtering. One of kAAX_Trace_Priority_Low , kAAX_Trace_Priority_Normal , kAAX_Trace_Priority_High
in	<i>iMessage</i>	Message string to log

The documentation for this class was generated from the following file:

- [AAX_CHostServices.h](#)

14.17 AAX_CLinearTaperDelegate< T, RealPrecision > Class Template Reference

```
#include <AAX_CLinearTaperDelegate.h>
```

Inheritance diagram for AAX_CLinearTaperDelegate< T, RealPrecision >:

Collaboration diagram for AAX_CLinearTaperDelegate< T, RealPrecision >:

14.17.1 Description

```
template<typename T, int32_t RealPrecision = 0>
class AAX_CLinearTaperDelegate< T, RealPrecision >
```

A linear taper conforming to [AAX_ITaperDelegate](#).

This taper spaces a parameter's real values evenly between its minimum and maximum, with a linear mapping between the parameter's real and normalized values.

RealPrecision

In addition to its type templization, this taper includes a precision template parameter. RealPrecision is a multiplier that works in conjunction with the round() function to limit the precision of the real values provided by this taper. For example, if RealPrecision is 1000, it will round to the closest 0.001 when doing any sort of value conversion. If RealPrecision is 1, it will round to the nearest integer. If RealPrecision is 1000000, it will round to the nearest 0.000001. This is particularly useful for preventing things like 1.9999999 truncating down to 1 instead of rounding up to 2.

To accomplish this behavior, the taper multiplies its unrounded parameter values by RealPrecision, rounds the result to the nearest valid value, then divides RealPrecision back out.

Rounding will be disabled if RealPrecision is set to a value less than 1. This is the default.

Public Member Functions

- [AAX_CLinearTaperDelegate](#) (T minValue=0, T maxValue=1)
Constructs a Linear Taper with specified minimum and maximum values.
- [AAX_CLinearTaperDelegate](#)< T, RealPrecision > * [Clone](#) () const [AAX_OVERRIDE](#)
Constructs and returns a copy of the taper delegate.
- T [GetMinimumValue](#) () const [AAX_OVERRIDE](#)
Returns the taper's minimum real value.
- T [GetMaximumValue](#) () const [AAX_OVERRIDE](#)
Returns the taper's maximum real value.
- T [ConstrainRealValue](#) (T value) const [AAX_OVERRIDE](#)
Applies a constraint to the value and returns the constrained value.
- T [NormalizedToReal](#) (double normalizedValue) const [AAX_OVERRIDE](#)
Converts a normalized value to a real value.
- double [RealToNormalized](#) (T realValue) const [AAX_OVERRIDE](#)
Normalizes a real parameter value.

Protected Member Functions

- T [Round](#) (double iValue) const

14.17.2 Constructor & Destructor Documentation

14.17.2.1 AAX_CLinearTaperDelegate()

```
template<typename T , int32_t RealPrecision>
AAX_CLinearTaperDelegate< T, RealPrecision >::AAX_CLinearTaperDelegate (
    T minValue = 0,
    T maxValue = 1 )
```

Constructs a Linear Taper with specified minimum and maximum values.

Note

The parameter's default value should lie within the min to max range.

Parameters

in	<i>minValue</i>	
in	<i>maxValue</i>	

14.17.3 Member Function Documentation

14.17.3.1 Clone()

```
template<typename T , int32_t RealPrecision>
AAX_CLinearTaperDelegate< T, RealPrecision > * AAX_CLinearTaperDelegate< T, RealPrecision >↵
::Clone ( ) const [virtual]
```

Constructs and returns a copy of the taper delegate.

In general, this method's implementation can use a simple copy constructor:

```
template <typename T>
AAX_CSubclassTaperDelegate<T>* AAX_CSubclassTaperDelegate<T>::Clone() const
{
    return new AAX_CSubclassTaperDelegate(*this);
}
```

Implements [AAX_ITaperDelegate< T >](#).

14.17.3.2 GetMinimumValue()

```
template<typename T , int32_t RealPrecision = 0>
T AAX_CLinearTaperDelegate< T, RealPrecision >::GetMinimumValue ( ) const [inline], [virtual]
```

Returns the taper's minimum real value.

Implements [AAX_ITaperDelegate< T >](#).

14.17.3.3 GetMaximumValue()

```
template<typename T , int32_t RealPrecision = 0>
T AAX_CLinearTaperDelegate< T, RealPrecision >::GetMaximumValue ( ) const [inline], [virtual]
```

Returns the taper's maximum real value.

Implements [AAX_ITaperDelegate< T >](#).

14.17.3.4 ConstrainRealValue()

```
template<typename T , int32_t RealPrecision>
T AAX_CLinearTaperDelegate< T, RealPrecision >::ConstrainRealValue (
    T value ) const [virtual]
```

Applies a constraint to the value and returns the constrained value.

This method is useful if the taper requires a constraint beyond simple minimum and maximum real value limits.

Note

This is the function that should actually enforces the constraints in [NormalizeToReal\(\)](#) and [RealToNormalized\(\)](#).

Parameters

in	<i>value</i>	The unconstrained value
----	--------------	-------------------------

Implements [AAX_ITaperDelegate< T >](#).

14.17.3.5 NormalizedToReal()

```
template<typename T , int32_t RealPrecision>
T AAX\_CLinearTaperDelegate< T, RealPrecision >::NormalizedToReal (
    double normalizedValue ) const [virtual]
```

Converts a normalized value to a real value.

This is where the actual taper algorithm is implemented.

This function should perform the exact inverse of [RealToNormalized\(\)](#), to within the roundoff precision of the individual taper implementation.

Parameters

in	<i>normalizedValue</i>	The normalized value that will be converted
----	------------------------	---

Implements [AAX_ITaperDelegate< T >](#).

14.17.3.6 RealToNormalized()

```
template<typename T , int32_t RealPrecision>
double AAX\_CLinearTaperDelegate< T, RealPrecision >::RealToNormalized (
    T realValue ) const [virtual]
```

Normalizes a real parameter value.

This is where the actual taper algorithm is implemented.

This function should perform the exact inverse of [NormalizedToReal\(\)](#), to within the roundoff precision of the individual taper implementation.

Parameters

in	<i>realValue</i>	The real parameter value that will be normalized
----	------------------	--

Implements [AAX_ITaperDelegate< T >](#).

14.17.3.7 Round()

```
template<typename T , int32_t RealPrecision>
T AAX_CLinearTaperDelegate< T, RealPrecision >::Round (
    double iValue ) const [protected]
```

The documentation for this class was generated from the following file:

- [AAX_CLinearTaperDelegate.h](#)

14.18 AAX_CLogTaperDelegate< T, RealPrecision > Class Template Reference

```
#include <AAX_CLogTaperDelegate.h>
```

Inheritance diagram for AAX_CLogTaperDelegate< T, RealPrecision >:

Collaboration diagram for AAX_CLogTaperDelegate< T, RealPrecision >:

14.18.1 Description

```
template<typename T, int32_t RealPrecision = 1000>
class AAX_CLogTaperDelegate< T, RealPrecision >
```

A logarithmic taper conforming to [AAX_ITaperDelegate](#).

This taper spaces a parameter's real values between its minimum and maximum bounds, with a natural logarithmic mapping between the parameter's real and normalized values.

RealPrecision

In addition to its type templatization, this taper includes a precision template parameter. RealPrecision is a multiplier that works in conjunction with the round() function to limit the precision of the real values provided by this taper. For example, if RealPrecision is 1000, it will round to the closest 0.001 when doing any sort of value conversion. If RealPrecision is 1, it will round to the nearest integer. If RealPrecision is 1000000, it will round to the nearest 0.000001. This is particularly useful for preventing things like 1.9999999 truncating down to 1 instead of rounding up to 2.

To accomplish this behavior, the taper multiplies its unrounded parameter values by RealPrecision, rounds the result to the nearest valid value, then divides RealPrecision back out.

Rounding will be disabled if RealPrecision is set to a value less than 1

Public Member Functions

- [AAX_CLogTaperDelegate](#) (T minValue=0, T maxValue=1)
Constructs a Log Taper with specified minimum and maximum values.
- [AAX_CLogTaperDelegate](#)< T, RealPrecision > * [Clone](#) () const [AAX_OVERRIDE](#)
Constructs and returns a copy of the taper delegate.
- T [GetMinimumValue](#) () const [AAX_OVERRIDE](#)
Returns the taper's minimum real value.
- T [GetMaximumValue](#) () const [AAX_OVERRIDE](#)
Returns the taper's maximum real value.
- T [ConstrainRealValue](#) (T value) const [AAX_OVERRIDE](#)
Applies a constraint to the value and returns the constrained value.
- T [NormalizedToReal](#) (double normalizedValue) const [AAX_OVERRIDE](#)
Converts a normalized value to a real value.
- double [RealToNormalized](#) (T realValue) const [AAX_OVERRIDE](#)
Normalizes a real parameter value.

Protected Member Functions

- T [Round](#) (double iValue) const

14.18.2 Constructor & Destructor Documentation

14.18.2.1 AAX_CLogTaperDelegate()

```
template<typename T , int32_t RealPrecision>
AAX_CLogTaperDelegate< T, RealPrecision >::AAX_CLogTaperDelegate (
    T minValue = 0,
    T maxValue = 1 )
```

Constructs a Log Taper with specified minimum and maximum values.

Note

The parameter's default value should lie within the min to max range.

Parameters

in	<i>minValue</i>	
in	<i>maxValue</i>	

14.18.3 Member Function Documentation

14.18.3.1 Clone()

```
template<typename T , int32_t RealPrecision>
AAX_CLogTaperDelegate< T, RealPrecision > * AAX_CLogTaperDelegate< T, RealPrecision >::Clone
( ) const [virtual]
```

Constructs and returns a copy of the taper delegate.

In general, this method's implementation can use a simple copy constructor:

```
template <typename T>
AAX_CSubclassTaperDelegate<T>* AAX_CSubclassTaperDelegate<T>::Clone() const
{
    return new AAX_CSubclassTaperDelegate(*this);
}
```

Implements [AAX_ITaperDelegate< T >](#).

14.18.3.2 GetMinimumValue()

```
template<typename T , int32_t RealPrecision = 1000>
T AAX_CLogTaperDelegate< T, RealPrecision >::GetMinimumValue ( ) const [inline], [virtual]
```

Returns the taper's minimum real value.

Implements [AAX_ITaperDelegate< T >](#).

14.18.3.3 GetMaximumValue()

```
template<typename T , int32_t RealPrecision = 1000>
T AAX_CLogTaperDelegate< T, RealPrecision >::GetMaximumValue ( ) const [inline], [virtual]
```

Returns the taper's maximum real value.

Implements [AAX_ITaperDelegate< T >](#).

14.18.3.4 ConstrainRealValue()

```
template<typename T , int32_t RealPrecision>
T AAX_CLogTaperDelegate< T, RealPrecision >::ConstrainRealValue (
    T value ) const [virtual]
```

Applies a constraint to the value and returns the constrained value.

This method is useful if the taper requires a constraint beyond simple minimum and maximum real value limits.

Note

This is the function that should actually enforces the constraints in [NormalizeToReal\(\)](#) and [RealToNormalized\(\)](#).

Parameters

in	<i>value</i>	The unconstrained value
----	--------------	-------------------------

Implements [AAX_ITaperDelegate< T >](#).

14.18.3.5 NormalizedToReal()

```
template<typename T , int32_t RealPrecision>
T AAX\_CLogTaperDelegate< T, RealPrecision >::NormalizedToReal (
    double normalizedValue ) const [virtual]
```

Converts a normalized value to a real value.

This is where the actual taper algorithm is implemented.

This function should perform the exact inverse of [RealToNormalized\(\)](#), to within the roundoff precision of the individual taper implementation.

Parameters

in	<i>normalizedValue</i>	The normalized value that will be converted
----	------------------------	---

Implements [AAX_ITaperDelegate< T >](#).

References `AAX::SafeLog()`.

Here is the call graph for this function:

14.18.3.6 RealToNormalized()

```
template<typename T , int32_t RealPrecision>
double AAX\_CLogTaperDelegate< T, RealPrecision >::RealToNormalized (
    T realValue ) const [virtual]
```

Normalizes a real parameter value.

This is where the actual taper algorithm is implemented.

This function should perform the exact inverse of [NormalizedToReal\(\)](#), to within the roundoff precision of the individual taper implementation.

Parameters

in	<i>realValue</i>	The real parameter value that will be normalized
----	------------------	--

Implements [AAX_ITaperDelegate< T >](#).

References `AAX::SafeLog()`.

Here is the call graph for this function:

14.18.3.7 Round()

```
template<typename T , int32_t RealPrecision>
T AAX_CLogTaperDelegate< T, RealPrecision >::Round (
    double iValue ) const [protected]
```

The documentation for this class was generated from the following file:

- [AAX_CLogTaperDelegate.h](#)

14.19 AAX_CMidiPacket Struct Reference

```
#include <AAX.h>
```

14.19.1 Description

Packet structure for MIDI data.

See also

[AAX_CMidiStream](#)

Legacy Porting Notes Corresponds to DirectMidiPacket in the legacy SDK

Public Attributes

- uint32_t [mTimestamp](#)
This is the playback time at which the MIDI event should occur, relative to the beginning of the current audio buffer.
- uint32_t [mLength](#)
The length of MIDI message, in terms of bytes.
- unsigned char [mData](#) [4]
The MIDI message itself. Each array element is one byte of the message, with the 0th element being the first byte.
- [AAX_CBoolean mIsImmediate](#)
Indicates that the message is to be sent as soon as possible.

14.19.2 Member Data Documentation

14.19.2.1 mTimestamp

```
uint32_t AAX_CMidiPacket::mTimestamp
```

This is the playback time at which the MIDI event should occur, relative to the beginning of the current audio buffer.

14.19.2.2 mLength

```
uint32_t AAX_CMidiPacket::mLength
```

The length of MIDI message, in terms of bytes.

14.19.2.3 mData

```
unsigned char AAX_CMidiPacket::mData[4]
```

The MIDI message itself. Each array element is one byte of the message, with the 0th element being the first byte.

Referenced by `AAX::IsAccentedClick()`, `AAX::IsAllNotesOff()`, `AAX::IsNoteOff()`, `AAX::IsNoteOn()`, and `AAX::IsUnaccentedClick()`.

14.19.2.4 mIsImmediate

```
AAX_CBoolean AAX_CMidiPacket::mIsImmediate
```

Indicates that the message is to be sent as soon as possible.

Host Compatibility Notes This value is not currently set. Use `mTimestamp == 0` to detect immediate packets

The documentation for this struct was generated from the following file:

- [AAX.h](#)

14.20 AAX_CMidiStream Struct Reference

```
#include <AAX.h>
```

Collaboration diagram for AAX_CMidiStream:

14.20.1 Description

MIDI stream data structure used by [AAX_IMIDINode](#).

For **MIDI input**, `mBufferSize` is set by the AAX host when the buffer is filled.

For **MIDI output**, the plug-in sets `mBufferSize` with the number of [AAX_CMidiPacket](#) objects it has filled `mBuffer` with. The AAX host will reset `mBufferSize` to 0 after it has received the buffer of MIDI.

System Exclusive (SysEx) messages that are greater than 4 bytes in length can be transmitted via a series of concurrent [AAX_CMidiPacket](#) objects in `mBuffer`. In accordance with the MIDI Specification, `0xF0` indicates the beginning of a SysEx message and `0xF7` indicates its end.

Legacy Porting Notes Corresponds to `DirectMidiNode` in the legacy SDK

Public Attributes

- `uint32_t mBufferSize`
The number of [AAX_CMidiPacket](#) objects contained in the node's buffer.
- `AAX_CMidiPacket * mBuffer`
Pointer to the first element of the node's buffer.

14.20.2 Member Data Documentation

14.20.2.1 mBufferSize

```
uint32_t AAX_CMidiStream::mBufferSize
```

The number of [AAX_CMidiPacket](#) objects contained in the node's buffer.

14.20.2.2 mBuffer

```
AAX_CMidiPacket* AAX_CMidiStream::mBuffer
```

Pointer to the first element of the node's buffer.

The documentation for this struct was generated from the following file:

- [AAX.h](#)

14.21 AAX_CMonolithicParameters Class Reference

```
#include <AAX_CMonolithicParameters.h>
```

Inheritance diagram for AAX_CMonolithicParameters:

Collaboration diagram for AAX_CMonolithicParameters:

14.21.1 Description

Extension of the [AAX_CEffectParameters](#) class for monolithic VIs and effects.

This extension to [AAX_CEffectParameters](#) adds some conveniences for Virtual Instrument (VI) plug-ins and for other plug-ins that use a monolithic processing object, i.e. an object that combines state data with the audio render routine in a single object.

- The [RenderAudio](#) method provides a direct audio processing callback within the data model object. Perform all audio processing in this method.
- The [StaticDescribe](#) method establishes a generic MIDI processing context for the Effect. Call this method from the plug-in's [Description callback](#) implementation.
- The [AddSynchronizedParameter](#) method provides a mechanism for synchronizing parameter updates with the real-time thread, allowing deterministic, accurate automation playback. For more information about this feature, see [Fixing timing issues due to shared data](#)

Note

This convenience class assumes a monolithic processing environment (i.e. [AAX_eConstraintLocationMask_DataModel](#).) This precludes the use of [AAX_CMonolithicParameters](#) -derived Effects in distributed-processing formats such as AAX DSP.

Public Member Functions

- [AAX_CMonolithicParameters](#) (void)
- [~AAX_CMonolithicParameters](#) (void) [AAX_OVERRIDE](#)

Protected Types

- typedef std::pair< [AAX_CParamID](#) const, const [AAX_IParameterValue](#) * > [TParamValPair](#)

Protected Member Functions

Real-time functions

Virtual functions called on the real-time thread

- virtual void [RenderAudio](#) ([AAX_SInstrumentRenderInfo](#) *ioRenderInfo, const [TParamValPair](#) *in← SynchronizedParamValues[], int32_t inNumSynchronizedParamValues)

Configuration methods

- void [AddSynchronizedParameter](#) (const [AAX_IParameter](#) &inParameter)

Convenience Layer Methods

Note

You should not need to override these methods, but if you do, make sure to call into the base class.

- [AAX_Result UpdateParameterNormalizedValue](#) ([AAX_CParamID](#) iParamID, double aValue, [AAX_EUpdateSource](#) inSource) [AAX_OVERRIDE](#)
Updates a single parameter's state to its current value.
- [AAX_Result GenerateCoefficients](#) () [AAX_OVERRIDE](#)
Generates and dispatches new coefficient packets.
- [AAX_Result ResetFieldData](#) ([AAX_CFieldIndex](#) iFieldIndex, void *oData, uint32_t iDataSize) const [AAX_OVERRIDE](#)
Called by the host to reset a private data field in the plug-in's algorithm.
- [AAX_Result TimerWakeup](#) () [AAX_OVERRIDE](#)
Periodic wakeup callback for idle-time operations.
- static [AAX_Result StaticDescribe](#) ([AAX_IEffectDescriptor](#) *ioDescriptor, const [AAX_SInstrumentSetupInfo](#) &setupInfo)
- static void [AAX_CALLBACK StaticRenderAudio](#) ([AAX_SInstrumentRenderInfo](#) *const inInstancesBegin[], const void *inInstancesEnd)

Additional Inherited Members

14.21.2 Member Typedef Documentation

14.21.2.1 TParamValPair

```
typedef std::pair<AAX\_CParamID const, const AAX\_IParameterValue*> AAX\_CMonolithicParameters::TParamValPair
[protected]
```

14.21.3 Constructor & Destructor Documentation

14.21.3.1 AAX_CMonolithicParameters()

```
AAX\_CMonolithicParameters::AAX\_CMonolithicParameters (
    void )
```

14.21.3.2 ~AAX_CMonolithicParameters()

```
AAX\_CMonolithicParameters::~~AAX\_CMonolithicParameters (
    void )
```

14.21.4 Member Function Documentation

14.21.4.1 RenderAudio()

```
virtual void AAX_CMonolithicParameters::RenderAudio (
    AAX_SInstrumentRenderInfo * ioRenderInfo,
    const TParamValPair * inSynchronizedParamValues[],
    int32_t inNumSynchronizedParamValues ) [inline], [protected], [virtual]
```

Perform audio render

Parameters

in, out	<i>ioRenderInfo</i>	State data for the current render buffer
in	<i>inSynchronizedParamValues</i>	The parameter values which should be applied for the current render buffer

See also

[AddSynchronizedParameter](#)

Parameters

in	<i>inNumSynchronizedParamValues</i>	The number of parameter values provided in <i>inSynchronizedParamValues</i>
----	-------------------------------------	---

Referenced by `StaticRenderAudio()`.

Here is the caller graph for this function:

14.21.4.2 AddSynchronizedParameter()

```
void AAX_CMonolithicParameters::AddSynchronizedParameter (
    const AAX_IParameter & inParameter ) [protected]
```

Add a parameter for state synchronization

A parameter should be added for synchronization if:

- It is important for the parameter's automation to be applied at the correct point on the timeline
- It is possible to quickly update the plug-in's state to reflect a parameter change

See [Parameter update timing](#) for more information

Parameters

in	<i>inParameter</i>	The parameter to be synchronized. This string will be copied internally and is not required to persist
----	--------------------	--

References [AAX_ASSERT](#), [AAX_IParameter::Automatable\(\)](#), [AAX_IParameter::Identifier\(\)](#), and [kSynchronizedParameterQueueSize](#).

Here is the call graph for this function:

14.21.4.3 UpdateParameterNormalizedValue()

```
AAX_Result AAX_CMonolithicParameters::UpdateParameterNormalizedValue (
    AAX_CParamID iParameterID,
    double iValue,
    AAX_EUpdateSource iSource ) [virtual]
```

Updates a single parameter's state to its current value.

Note

Do *not* call this method from the plug-in. This method should be called by the host only. To set parameter values from within the plug-in, use the [AAX_IParameter](#) interface.

Todo FLAGGED FOR CONSIDERATION OF REVISION

Parameters

in	<i>iParameterID</i>	The ID of the parameter that is being updated
in	<i>iValue</i>	The parameter's current value, to which its internal state must be updated
in	<i>iSource</i>	The source of the update

Reimplemented from [AAX_CEffectParameters](#).

References [AAX_SUCCESS](#), [AAX_CParameterManager::GetParameterByID\(\)](#), [AAX_CEffectParameters::mParameterManager](#), and [AAX_CEffectParameters::UpdateParameterNormalizedValue\(\)](#).

Here is the call graph for this function:

14.21.4.4 GenerateCoefficients()

```
AAX_Result AAX_CMonolithicParameters::GenerateCoefficients ( ) [virtual]
```

Generates and dispatches new coefficient packets.

This method is responsible for updating the coefficient packets associated with all parameters whose states have changed since the last call to [GenerateCoefficients\(\)](#). The host may call this method once for every parameter update, or it may "batch" parameter updates such that changes for several parameters are all handled by a single call to [GenerateCoefficients\(\)](#).

For more information on tracking parameters' statuses using the [AAX_CPacketDispatcher](#), helper class, see [AAX_CPacketDispatcher::SetDirty\(\)](#).

Note

Do *not* call this method from the plug-in. This method should be called by the host only. To set parameter values from within the plug-in, use the [AAX_IParameter](#) interface.

Reimplemented from [AAX_CEffectParameters](#).

References [AAX_ASSERT](#), [AAX_FIELD_INDEX](#), [AAX_SUCCESS](#), [AAX_IParameter::CloneValue\(\)](#), [AAX_CEffectParameters::Controller\(\)](#), [AAX_IContainer::eStatus_Success](#), [AAX_CEffectParameters::GenerateCoefficients\(\)](#), [AAX_IParameter::Identifier\(\)](#), [AAX_IController::PostPacket\(\)](#), and [AAX_CAtomicQueue< T, S >::Push\(\)](#).

Here is the call graph for this function:

14.21.4.5 ResetFieldData()

```
AAX_Result AAX_CMonolithicParameters::ResetFieldData (
    AAX_CFieldIndex inFieldIndex,
    void * oData,
    uint32_t inDataSize ) const [virtual]
```

Called by the host to reset a private data field in the plug-in's algorithm.

This method is called sequentially for all private data fields on Effect initialization and during any "reset" event, such as priming for a non-real-time render. This method is called before the algorithm's optional initialization callback, and the initialized private data will be available within that callback via its context block.

See also

[Algorithm initialization](#).

Warning

Any data structures that will be passed between platforms (for example, sent to a TI DSP in an AAX DSP plug-in) must be properly data-aligned for compatibility across both platforms. See [AAX_ALIGN_FILE_ALG](#) for more information about guaranteeing cross-platform compatibility of data structures used for algorithm processing.

Parameters

in	<i>inFieldIndex</i>	The index of the field that is being initialized
out	<i>oData</i>	The pre-allocated block of data that should be initialized
in	<i>inDataSize</i>	The size of the data block, in bytes

Reimplemented from [AAX_CEffectParameters](#).

References [AAX_ASSERT](#), [AAX_FIELD_INDEX](#), [AAX_SUCCESS](#), [AAX_SInstrumentPrivateData::mMonolithicParametersPtr](#), and [AAX_CEffectParameters::ResetFieldData\(\)](#).

Here is the call graph for this function:

14.21.4.6 TimerWakeup()

```
AAX_Result AAX_CMonolithicParameters::TimerWakeup ( ) [virtual]
```

Periodic wakeup callback for idle-time operations.

This method is called from the host using a non-main thread. In general, it should be driven at approximately one call per 30 ms. However, the wakeup is not guaranteed to be called at any regular interval - for example, it could be held off by a high real-time processing load - and there is no host contract regarding maximum latency between wakeup calls.

This wakeup thread runs continuously and cannot be armed/disarmed or by the plug-in.

Reimplemented from [AAX_CEffectParameters](#).

References [AAX_CEffectParameters::TimerWakeup\(\)](#).

Here is the call graph for this function:

14.21.4.7 StaticDescribe()

```
AAX_Result AAX_CMonolithicParameters::StaticDescribe (
    AAX_IEffectDescriptor * ioDescriptor,
    const AAX_SInstrumentSetupInfo & setupInfo ) [static]
```

Static description callback

This method performs all of the basic context setup and pointer passing work

Parameters

in, out	<i>ioDescriptor</i>	
in	<i>setupInfo</i>	

References [AAX_ASSERT](#), [AAX_eConstraintLocationMask_DataModel](#), [AAX_eMIDINodeType_Global](#), [AAX_eMIDINodeType_LocalInput](#), [AAX_eMIDINodeType_Transport](#), [AAX_ePrivateDataOptions_DefaultOptions](#), [AAX_eProperty_CanBypass](#), [AAX_eProperty_Constraint_Location](#), [AAX_eProperty_Constraint_MultiMonoSupport](#), [AAX_eProperty_HybridInputStemFormat](#), [AAX_eProperty_HybridOutputStemFormat](#), [AAX_eProperty_InputStemFormat](#), [AAX_eProperty_ManufacturerID](#), [AAX_eProperty_OutputStemFormat](#), [AAX_eProperty_PluginID_AudioSuite](#), [AAX_eProperty_PluginID_RTAS](#), [AAX_eProperty_ProductID](#), [AAX_eProperty_UsesClientGUI](#), [AAX_eProperty_UsesTransport](#), [AAX_ERROR_NULL_OBJECT](#), [AAX_eStemFormat_None](#), [AAX_FIELD_INDEX](#), [AAX_IComponentDescriptor::AddAudioBufferLength\(\)](#), [AAX_IComponentDescriptor::AddAudioIn\(\)](#), [AAX_IComponentDescriptor::AddAudioOut\(\)](#), [AAX_IComponentDescriptor::AddAuxOutputStem\(\)](#), [AAX_IComponentDescriptor::AddClock\(\)](#), [AAX_IEffectDescriptor::AddComponent\(\)](#), [AAX_IComponentDescriptor::AddDataInPort\(\)](#), [AAX_IComponentDescriptor::AddMeters\(\)](#), [AAX_IComponentDescriptor::AddMIDINode\(\)](#), [AAX_IComponentDescriptor::AddPrivateData\(\)](#), [AAX_IComponentDescriptor::AddProcessProc_Native\(\)](#), [AAX_IPropertyMap::AddProperty\(\)](#), [kMaxAdditionalMIDINodes](#), [kMaxAuxOutputStems](#), [AAX_SInstrumentSetupInfo::mAudiosuiteID](#), [AAX_SInstrumentSetupInfo::mAuxOutputStemFormats](#), [AAX_SInstrumentSetupInfo::mAuxOutputStemNames](#), [AAX_SInstrumentSetupInfo::mCanBypass](#), [AAX_SInstrumentSetupInfo::mGlobalMIDIEventMask](#), [AAX_SInstrumentSetupInfo::mGlobalMIDINodeName](#), [AAX_SInstrumentSetupInfo::mHybridInputStemFormat](#), [AAX_SInstrumentSetupInfo::mHybridOutputStemFormat](#), [AAX_SInstrumentSetupInfo::mInputMIDIChannelMask](#), [AAX_SInstrumentSetupInfo::mInputMIDINodeName](#), [AAX_SInstrumentSetupInfo::mInputStemFormat](#), [AAX_SInstrumentSetupInfo::mManufacturerID](#), [AAX_SInstrumentSetupInfo::mMeterIDs](#), [AAX_SInstrumentSetupInfo::mMultiMonoSupport](#), [AAX_SInstrumentSetupInfo::mNeedsGlobalMIDI](#), [AAX_SInstrumentSetupInfo::mNeeds](#)

InputMIDI, AAX_SInstrumentSetupInfo::mNeedsTransport, AAX_SInstrumentSetupInfo::mNumAdditionalInput↵
 MIDINodes, AAX_SInstrumentSetupInfo::mNumAuxOutputStems, AAX_SInstrumentSetupInfo::mNumMeters,
 AAX_SInstrumentSetupInfo::mOutputStemFormat, AAX_SInstrumentSetupInfo::mPluginID, AAX_SInstrument↵
 SetupInfo::mProductID, AAX_SInstrumentSetupInfo::mUseHostGeneratedGUI, AAX_IEffectDescriptor::New↵
 ComponentDescriptor(), AAX_IComponentDescriptor::NewPropertyMap(), and StaticRenderAudio().

Here is the call graph for this function:

14.21.4.8 StaticRenderAudio()

```
void AAX_CALLBACK AAX_CMonolithicParameters::StaticRenderAudio (
    AAX_SInstrumentRenderInfo *const inInstancesBegin[],
    const void * inInstancesEnd ) [static]
```

Static RenderAudio (Called by the host)

Plug-ins should override [AAX_CMonolithicParameters::RenderAudio\(\)](#)

Parameters

in	<i>inInstancesBegin</i>	
in	<i>inInstancesEnd</i>	

References [AAX_ASSERT](#), [AAX_IContainer::eStatus_Success](#), [AAX_SInstrumentPrivateData::mMonolithic↵
 ParametersPtr](#), [AAX_CAtomicQueue< T, S >::Push\(\)](#), and [RenderAudio\(\)](#).

Referenced by [StaticDescribe\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

The documentation for this class was generated from the following files:

- [AAX_CMonolithicParameters.h](#)
- [AAX_CMonolithicParameters.cpp](#)

14.22 AAX_CMutex Class Reference

```
#include <AAX_CMutex.h>
```

14.22.1 Description

Mutex with try lock functionality.

Public Member Functions

- [AAX_CMutex \(\)](#)
- [~AAX_CMutex \(\)](#)
- bool [Lock \(\)](#)
- void [Unlock \(\)](#)
- bool [Try_Lock \(\)](#)

14.22.2 Constructor & Destructor Documentation

14.22.2.1 AAX_CMutex()

```
AAX_CMutex::AAX_CMutex ( )
```

14.22.2.2 ~AAX_CMutex()

```
AAX_CMutex::~~AAX_CMutex ( )
```

14.22.3 Member Function Documentation

14.22.3.1 Lock()

```
bool AAX_CMutex::Lock ( )
```

Referenced by AAX_StLock_Guard::AAX_StLock_Guard().

Here is the caller graph for this function:

14.22.3.2 Unlock()

```
void AAX_CMutex::Unlock ( )
```

Referenced by AAX_StLock_Guard::~~AAX_StLock_Guard().

Here is the caller graph for this function:

14.22.3.3 Try_Lock()

```
bool AAX_CMutex::Try_Lock ( )
```

The documentation for this class was generated from the following file:

- [AAX_CMutex.h](#)

14.23 AAX_CNumberDisplayDelegate< T, Precision, SpaceAfter > Class Template Reference

```
#include <AAX_CNumberDisplayDelegate.h>
```

Inheritance diagram for AAX_CNumberDisplayDelegate< T, Precision, SpaceAfter >:

Collaboration diagram for AAX_CNumberDisplayDelegate< T, Precision, SpaceAfter >:

14.23.1 Description

```
template<typename T, uint32_t Precision = 2, uint32_t SpaceAfter = 0>
class AAX_CNumberDisplayDelegate< T, Precision, SpaceAfter >
```

A numeric display format conforming to [AAX_IDisplayDelegate](#).

This display delegate converts a parameter value to a numeric string using a specified precision.

Public Member Functions

- [AAX_CNumberDisplayDelegate](#) * [Clone](#) () const [AAX_OVERRIDE](#)
Constructs and returns a copy of the display delegate.
- bool [ValueToString](#) (T value, [AAX_CString](#) *valueString) const [AAX_OVERRIDE](#)
Converts a real parameter value to a string representation.
- bool [ValueToString](#) (T value, int32_t maxNumChars, [AAX_CString](#) *valueString) const [AAX_OVERRIDE](#)
Converts a real parameter value to a string representation using a size hint, useful for control surfaces and other character limited displays.
- bool [StringToValue](#) (const [AAX_CString](#) &valueString, T *value) const [AAX_OVERRIDE](#)
Converts a string to a real parameter value.

14.23.2 Member Function Documentation

14.23.2.1 Clone()

```
template<typename T , uint32_t Precision, uint32_t SpaceAfter>
AAX_CNumberDisplayDelegate< T, Precision, SpaceAfter > * AAX\_CNumberDisplayDelegate< T, Precision,
SpaceAfter >::Clone ( ) const [virtual]
```

Constructs and returns a copy of the display delegate.

In general, this method's implementation can use a simple copy constructor:

```
template <typename T>
AAX_CSubclassDisplayDelegate<T>* AAX_CSubclassDisplayDelegate<T>::Clone() const
{
    return new AAX_CSubclassDisplayDelegate(*this);
}
```

Implements [AAX_IDisplayDelegate< T >](#).

14.23.2.2 ValueToString() [1/2]

```
template<typename T , uint32_t Precision, uint32_t SpaceAfter>
bool AAX_CNumberDisplayDelegate< T, Precision, SpaceAfter >::ValueToString (
    T value,
    AAX_CString * valueString ) const [virtual]
```

Converts a real parameter value to a string representation.

Parameters

in	<i>value</i>	The real parameter value that will be converted
out	<i>valueString</i>	A string corresponding to value

Return values

<i>true</i>	The string conversion was successful
<i>false</i>	The string conversion was unsuccessful

Implements [AAX_IDisplayDelegate< T >](#).

References [AAX_CString::Append\(\)](#), [AAX_CString::AppendNumber\(\)](#), and [AAX_CString::Clear\(\)](#).

Here is the call graph for this function:

14.23.2.3 ValueToString() [2/2]

```
template<typename T , uint32_t Precision, uint32_t SpaceAfter>
bool AAX_CNumberDisplayDelegate< T, Precision, SpaceAfter >::ValueToString (
    T value,
    int32_t maxNumChars,
    AAX_CString * valueString ) const [virtual]
```

Converts a real parameter value to a string representation using a size hint, useful for control surfaces and other character limited displays.

Parameters

in	<i>value</i>	The real parameter value that will be converted
in	<i>maxNumChars</i>	Size hint for the desired maximum number of characters in the string (not including null termination)
out	<i>valueString</i>	A string corresponding to value

Return values

<i>true</i>	The string conversion was successful
<i>false</i>	The string conversion was unsuccessful

Implements [AAX_IDisplayDelegate< T >](#).

References [AAX_CString::Append\(\)](#), [AAX_CString::AppendNumber\(\)](#), [AAX_CString::Clear\(\)](#), [AAX_CString::Erase\(\)](#), and [AAX_CString::Length\(\)](#).

Here is the call graph for this function:

14.23.2.4 StringToValue()

```
template<typename T , uint32_t Precision, uint32_t SpaceAfter>
bool AAX_CNumberDisplayDelegate< T, Precision, SpaceAfter >::StringToValue (
```

```
const AAX_CString & valueString,
T * value ) const [virtual]
```

Converts a string to a real parameter value.

Parameters

in	<i>valueString</i>	The string that will be converted
out	<i>value</i>	The real parameter value corresponding to valueString

Return values

<i>true</i>	The string conversion was successful
<i>false</i>	The string conversion was unsuccessful

Implements [AAX_IDisplayDelegate< T >](#).

References [AAX_CString::ToDouble\(\)](#).

Here is the call graph for this function:

The documentation for this class was generated from the following file:

- [AAX_CNumberDisplayDelegate.h](#)

14.24 AAX_Component< aContextType > Class Template Reference

```
#include <AAX_Callbacks.h>
```

14.24.1 Description

```
template<typename aContextType>
class AAX_Component< aContextType >
```

Empty class containing type declarations for the AAX algorithm and associated callbacks.

Public Types

- typedef void([AAX_CALLBACK](#) * [CProcessProc](#)) (aContextType *const inContextPtrsBegin[], const void *inContextPtrsEnd)
- typedef void *([AAX_CALLBACK](#) * [CPacketAllocator](#)) (const aContextType *inContextPtr, [AAX_CFieldIndex](#) inOutputPort, [AAX_CTimestamp](#) inTimestamp)
- typedef int32_t([AAX_CALLBACK](#) * [CInstanceInitProc](#)) (const aContextType *inInstanceContextPtr, [AAX_EComponentInstanceInitAction](#) iAction)
- typedef int32_t([AAX_CALLBACK](#) * [CBackgroundProc](#)) (void)
- typedef void([AAX_CALLBACK](#) * [CInitPrivateDataProc](#)) ([AAX_CFieldIndex](#) inFieldIndex, void *inNewBlock, int32_t inSize, [IACFUnknown](#) *const inController)

14.24.2 Member Typedef Documentation

14.24.2.1 CProcessProc

```
template<typename aContextType >
typedef void(AAX_CALLBACK * AAX_Component< aContextType >::CProcessProc) (aContextType *const
inContextPtrsBegin[], const void *inContextPtrsEnd)
```

14.24.2.2 CPacketAllocator

```
template<typename aContextType >
typedef void*(AAX_CALLBACK * AAX_Component< aContextType >::CPacketAllocator) (const aContextType *inContextPtr, AAX_CFieldIndex inOutputPort, AAX_CTimestamp inTimestamp)
```

14.24.2.3 CInstanceInitProc

```
template<typename aContextType >
typedef int32_t(AAX_CALLBACK * AAX_Component< aContextType >::CInstanceInitProc) (const aContextType *inInstanceContextPtr, AAX_EComponentInstanceInitAction iAction)
```

14.24.2.4 CBackgroundProc

```
template<typename aContextType >
typedef int32_t(AAX_CALLBACK * AAX_Component< aContextType >::CBackgroundProc) (void)
```

14.24.2.5 CInitPrivateDataProc

```
template<typename aContextType >
typedef void(AAX_CALLBACK * AAX_Component< aContextType >::CInitPrivateDataProc) (AAX_CFieldIndex inFieldIndex, void *inNewBlock, int32_t inSize, IACFUnknown *const inController)
```

The documentation for this class was generated from the following file:

- [AAX_Callbacks.h](#)

14.25 AAX_CPacket Class Reference

```
#include <AAX_CPacketDispatcher.h>
```

14.25.1 Description

Container for packet-related data.

This class collects a number of packet-related data into the same object and provides a facility for tracking when the parameter is "dirty", i.e. after its value has been updated and before an associated packet has not been posted.

Public Member Functions

- [AAX_CPacket](#) ([AAX_CFieldIndex](#) inFieldIndex)
- [~AAX_CPacket](#) ()
- `template<typename DataType >`
`DataType * GetPtr ()`
- `void SetDirty (bool iDirty)`
- `bool IsDirty () const`
- `AAX\_CFieldIndex GetID () const`
- `uint32_t GetSize () const`
- `template<> const void * GetPtr ()`

14.25.2 Constructor & Destructor Documentation

14.25.2.1 AAX_CPacket()

```
AAX_CPacket::AAX_CPacket (
    AAX\_CFieldIndex inFieldIndex ) [inline]
```

14.25.2.2 ~AAX_CPacket()

```
AAX_CPacket::~~AAX_CPacket ( ) [inline]
```

14.25.3 Member Function Documentation

14.25.3.1 GetPtr() [1/2]

```
template<typename DataType >
DataType* AAX_CPacket::GetPtr ( ) [inline]
```

14.25.3.2 SetDirty()

```
void AAX_CPacket::SetDirty (
    bool iDirty ) [inline]
```

14.25.3.3 IsDirty()

```
bool AAX_CPacket::IsDirty ( ) const [inline]
```

14.25.3.4 GetID()

```
AAX_CFieldIndex AAX_CPacket::GetID ( ) const [inline]
```

14.25.3.5 GetSize()

```
uint32_t AAX_CPacket::GetSize ( ) const [inline]
```

14.25.3.6 GetPtr() [2/2]

```
template<>
const void* AAX_CPacket::GetPtr ( ) [inline]
```

The documentation for this class was generated from the following file:

- [AAX_CPacketDispatcher.h](#)

14.26 AAX_CPacketDispatcher Class Reference

```
#include <AAX_CPacketDispatcher.h>
```


14.26.1 Description

Helper class for managing AAX packet posting.

This optional class can be used to associate individual parameters with custom update callbacks. The update callbacks for all "dirty" parameters are triggered whenever [AAX_CPacketDispatcher::Dispatch\(\)](#) is called. The resulting coefficient data is then posted to the [AAX_IController](#) automatically by the packet dispatcher.

The packet dispatcher supports many-to-one relationships between parameters and handler callbacks, so a single callback may be registered for several related parameters.

See also

[AAX_CEffectParameters::EffectInit\(\)](#)

Public Member Functions

- [AAX_CPacketDispatcher](#) ()
- [~AAX_CPacketDispatcher](#) ()
- void [Initialize](#) ([AAX_IController](#) *iPlugIn, [AAX_IEffectParameters](#) *iEffectParameters)
- [AAX_Result](#) [RegisterPacket](#) ([AAX_CParamID](#) paramID, [AAX_CFieldIndex](#) portID, const [AAX_IPacketHandler](#) *iHandler)
- template<class TWorker , typename Func >
[AAX_Result](#) [RegisterPacket](#) ([AAX_CParamID](#) paramID, [AAX_CFieldIndex](#) portID, TWorker *iPt2Object, Func infPt)
- [AAX_Result](#) [RegisterPacket](#) ([AAX_CParamID](#) paramID, [AAX_CFieldIndex](#) portID)
- [AAX_Result](#) [SetDirty](#) ([AAX_CParamID](#) paramID, bool iDirty=true)
- [AAX_Result](#) [Dispatch](#) ()
- [AAX_Result](#) [GenerateSingleValuePacket](#) ([AAX_CParamID](#) iParam, [AAX_CPacket](#) &iPacket)

14.26.2 Constructor & Destructor Documentation

14.26.2.1 AAX_CPacketDispatcher()

```
AAX_CPacketDispatcher::AAX_CPacketDispatcher ( )
```

14.26.2.2 ~AAX_CPacketDispatcher()

```
AAX_CPacketDispatcher::~~AAX_CPacketDispatcher ( )
```

14.26.3 Member Function Documentation

14.26.3.1 Initialize()

```
void AAX_CPacketDispatcher::Initialize (
    AAX_IController * iPlugin,
    AAX_IEffectParameters * iEffectParameters )
```

14.26.3.2 RegisterPacket() [1/3]

```
AAX_Result AAX_CPacketDispatcher::RegisterPacket (
    AAX_CParamID paramID,
    AAX_CFieldIndex portID,
    const AAX_IPacketHandler * iHandler )
```

Referenced by RegisterPacket().

Here is the caller graph for this function:

14.26.3.3 RegisterPacket() [2/3]

```
template<class TWorker , typename Func >
AAX_Result AAX_CPacketDispatcher::RegisterPacket (
    AAX_CParamID paramID,
    AAX_CFieldIndex portID,
    TWorker * iPt2Object,
    Func infPt ) [inline]
```

References RegisterPacket().

Here is the call graph for this function:

14.26.3.4 RegisterPacket() [3/3]

```
AAX_Result AAX_CPacketDispatcher::RegisterPacket (
    AAX_CParamID paramID,
    AAX_CFieldIndex portID ) [inline]
```

References GenerateSingleValuePacket(), and RegisterPacket().

Here is the call graph for this function:

14.26.3.5 SetDirty()

```
AAX_Result AAX_CPacketDispatcher::SetDirty (
    AAX_CParamID paramID,
    bool iDirty = true )
```

14.26.3.6 Dispatch()

```
AAX_Result AAX_CPacketDispatcher::Dispatch ( )
```

14.26.3.7 GenerateSingleValuePacket()

```
AAX_Result AAX_CPacketDispatcher::GenerateSingleValuePacket (
    AAX_CParamID iParam,
    AAX_CPacket & ioPacket )
```

Referenced by RegisterPacket().

Here is the caller graph for this function:

The documentation for this class was generated from the following file:

- [AAX_CPacketDispatcher.h](#)

14.27 AAX_CPacketHandler< TWorker > Class Template Reference

```
#include <AAX_CPacketDispatcher.h>
```

Inheritance diagram for AAX_CPacketHandler< TWorker >:

Collaboration diagram for AAX_CPacketHandler< TWorker >:

14.27.1 Description

```
template<class TWorker>
class AAX_CPacketHandler< TWorker >
```

Callback container used by [AAX_CPacketDispatcher](#).

Public Member Functions

- [AAX_CPacketHandler](#) (TWorker *iPt2Object, fPt2Fn infPt)
- [AAX_CPacketHandler](#) (TWorker *iPt2Object, fPt2FnEx infPt)
- [AAX_IPacketHandler](#) * Clone () const
- [AAX_Result](#) Call (AAX_CParamID iParamID, [AAX_CPacket](#) &ioPacket) const

Protected Attributes

- TWorker * [pt2Object](#)
- fPt2Fn [fpt](#)
- fPt2FnEx [fptEx](#)

14.27.2 Constructor & Destructor Documentation

14.27.2.1 AAX_CPacketHandler() [1/2]

```
template<class TWorker >
AAX_CPacketHandler< TWorker >::AAX_CPacketHandler (
    TWorker * iPt2Object,
    fPt2Fn infPt ) [inline]
```

Referenced by `AAX_CPacketHandler< TWorker >::Clone()`.

Here is the caller graph for this function:

14.27.2.2 AAX_CPacketHandler() [2/2]

```
template<class TWorker >
AAX_CPacketHandler< TWorker >::AAX_CPacketHandler (
    TWorker * iPt2Object,
    fPt2FnEx infPt ) [inline]
```

14.27.3 Member Function Documentation

14.27.3.1 Clone()

```
template<class TWorker >
AAX_IPacketHandler* AAX_CPacketHandler< TWorker >::Clone ( ) const [inline], [virtual]
```

Implements [AAX_IPacketHandler](#).

References `AAX_CPacketHandler< TWorker >::AAX_CPacketHandler()`.

Here is the call graph for this function:

14.27.3.2 Call()

```
template<class TWorker >
AAX_Result AAX_CPacketHandler< TWorker >::Call (
    AAX_CParamID inParamID,
    AAX_CPacket & ioPacket ) const [inline], [virtual]
```

Implements [AAX_IPacketHandler](#).

References `AAX_ERROR_NULL_OBJECT`, `AAX_CPacketHandler< TWorker >::fpt`, `AAX_CPacketHandler< TWorker >::fptEx`, and `AAX_CPacketHandler< TWorker >::pt2Object`.

14.27.4 Member Data Documentation

14.27.4.1 pt2Object

```
template<class TWorker >  
TWorker* AAX_CPacketHandler< TWorker >::pt2Object [protected]
```

Referenced by AAX_CPacketHandler< TWorker >::Call().

14.27.4.2 fpt

```
template<class TWorker >  
fPt2Fn AAX_CPacketHandler< TWorker >::fpt [protected]
```

Referenced by AAX_CPacketHandler< TWorker >::Call().

14.27.4.3 fptEx

```
template<class TWorker >  
fPt2FnEx AAX_CPacketHandler< TWorker >::fptEx [protected]
```

Referenced by AAX_CPacketHandler< TWorker >::Call().

The documentation for this class was generated from the following file:

- [AAX_CPacketDispatcher.h](#)

14.28 AAX_CParameter< T > Class Template Reference

```
#include <AAX_CParameter.h>
```

Inheritance diagram for AAX_CParameter< T >:

Collaboration diagram for AAX_CParameter< T >:

14.28.1 Description

```
template<typename T>
class AAX_CParameter< T >
```

Generic implementation of an [AAX_IParameter](#).

This is a concrete, templated implementation of [AAX_IParameter](#) for parameters with standard types such as `float`, `uint32`, `bool`, etc.

Many different behaviors can be composited into this class as delegates. [AAX_ITaperDelegate](#) and [AAX_IDisplayDelegate](#) are two examples of delegates that this class uses in order to apply custom behaviors to the [AAX_IParameter](#) interface.

Plug-in developers can subclass these delegates to create adaptable, reusable parameter behaviors, which can then be "mixed in" to individual [AAX_CParameter](#) objects without the need to modify the objects themselves.

Note

Because [AAX_CParameter](#) is a C++ template, each [AAX_CParameter](#) template parameter that is used creates a new subclass that adheres to the [AAX_IParameter](#) interface.

Public Types

- enum [Type](#) {
[eParameterTypeUndefined](#) = 0 ,
[eParameterTypeBool](#) = 1 ,
[eParameterTypeInt32](#) = 2 ,
[eParameterTypeFloat](#) = 3 ,
[eParameterTypeCustom](#) = 4 }
- enum [Defaults](#) {
[eParameterDefaultNumStepsDiscrete](#) = 2 ,
[eParameterDefaultNumStepsContinuous](#) = 128 }

Public Member Functions

- [AAX_CParameter](#) ([AAX_CParamID](#) identifier, const [AAX_IString](#) &name, T defaultValue, const [AAX_ITaperDelegate](#)< T > &taperDelegate, const [AAX_IDisplayDelegate](#)< T > &displayDelegate, bool automatable=false)
Constructs an [AAX_CParameter](#) object using the specified taper and display delegates.
- [AAX_CParameter](#) (const [AAX_IString](#) &identifier, const [AAX_IString](#) &name, T defaultValue, const [AAX_ITaperDelegate](#)< T > &taperDelegate, const [AAX_IDisplayDelegate](#)< T > &displayDelegate, bool automatable=false)
Constructs an [AAX_CParameter](#) object using the specified taper and display delegates.
- [AAX_CParameter](#) (const [AAX_IString](#) &identifier, const [AAX_IString](#) &name, T defaultValue, bool automatable=false)
Constructs an [AAX_CParameter](#) object with no delegates.
- [AAX_CParameter](#) (const [AAX_IString](#) &identifier, const [AAX_IString](#) &name, bool automatable=false)
Constructs an [AAX_CParameter](#) object with no delegates or default value.
- [AAX_DEFAULT_MOVE_CTOR](#) ([AAX_CParameter](#))
- [AAX_DEFAULT_MOVE_OPER](#) ([AAX_CParameter](#))
- [AAX_DELETE](#) ([AAX_CParameter](#)())
- [AAX_DELETE](#) ([AAX_CParameter](#)(const [AAX_CParameter](#) &other))
- [AAX_DELETE](#) ([AAX_CParameter](#) &operator=(const [AAX_CParameter](#) &other))

- [~AAX_CParameter \(\)](#) [AAX_OVERRIDE](#)
Virtual destructor used to delete all locally allocated pointers.
- [AAX_IParameterValue * CloneValue \(\)](#) const [AAX_OVERRIDE](#)
Clone the parameter's value to a new [AAX_IParameterValue](#) object.
- [bool GetValueAsString \(AAX_IString *\)](#) const
Retrieves the parameter's value as a string.
- [bool SetValueWithBool \(bool value\)](#)
Sets the parameter's value as a bool.
- [bool SetValueWithInt32 \(int32_t value\)](#)
Sets the parameter's value as an int32_t.
- [bool SetValueWithFloat \(float value\)](#)
Sets the parameter's value as a float.
- [bool SetValueWithDouble \(double value\)](#)
Sets the parameter's value as a double.
- [bool SetValueWithString \(const AAX_IString &value\)](#)
Sets the parameter's value as a string.
- [bool GetNormalizedValueFromBool \(bool value, double *normalizedValue\)](#) const
Converts a bool to a normalized parameter value.
- [bool GetNormalizedValueFromInt32 \(int32_t value, double *normalizedValue\)](#) const
Converts an integer to a normalized parameter value.
- [bool GetNormalizedValueFromFloat \(float value, double *normalizedValue\)](#) const
Converts a float to a normalized parameter value.
- [bool GetNormalizedValueFromDouble \(double value, double *normalizedValue\)](#) const
Converts a double to a normalized parameter value.
- [bool GetBoolFromNormalizedValue \(double inNormalizedValue, bool *value\)](#) const
Converts a normalized parameter value to a bool representing the corresponding real value.
- [bool GetInt32FromNormalizedValue \(double inNormalizedValue, int32_t *value\)](#) const
Converts a normalized parameter value to an integer representing the corresponding real value.
- [bool GetFloatFromNormalizedValue \(double inNormalizedValue, float *value\)](#) const
Converts a normalized parameter value to a float representing the corresponding real value.
- [bool GetDoubleFromNormalizedValue \(double inNormalizedValue, double *value\)](#) const
Converts a normalized parameter value to a double representing the corresponding real value.

Identification methods

- [AAX_CParamID Identifier \(\)](#) const [AAX_OVERRIDE](#)
Returns the parameter's unique identifier.
- [void SetName \(const AAX_CString &name\)](#) [AAX_OVERRIDE](#)
Sets the parameter's display name.
- [const AAX_CString & Name \(\)](#) const [AAX_OVERRIDE](#)
Returns the parameter's display name.
- [void AddShortenedName \(const AAX_CString &name\)](#) [AAX_OVERRIDE](#)
Sets the parameter's shortened display name.
- [const AAX_CString & ShortenedName \(int32_t iNumCharacters\)](#) const [AAX_OVERRIDE](#)
Returns the parameter's shortened display name.
- [void ClearShortenedNames \(\)](#) [AAX_OVERRIDE](#)
Clears the internal list of shortened display names.

Taper methods

- [void SetNormalizedDefaultValue \(double normalizedDefault\)](#) [AAX_OVERRIDE](#)
Sets the parameter's default value using its normalized representation.
- [double GetNormalizedDefaultValue \(\)](#) const [AAX_OVERRIDE](#)

- Returns the normalized representation of the parameter's real default value.*
- void [SetToDefaultValue](#) () [AAX_OVERRIDE](#)
Restores the state of this parameter to its default value.
- void [SetNormalizedValue](#) (double newNormalizedValue) [AAX_OVERRIDE](#)
Sets a parameter value using it's normalized representation.
- double [GetNormalizedValue](#) () const [AAX_OVERRIDE](#)
Returns the normalized representation of the parameter's current real value.
- void [SetNumberOfSteps](#) (uint32_t numSteps) [AAX_OVERRIDE](#)
Sets the number of discrete steps for this parameter.
- uint32_t [GetNumberOfSteps](#) () const [AAX_OVERRIDE](#)
Returns the number of discrete steps used by the parameter.
- uint32_t [GetStepValue](#) () const [AAX_OVERRIDE](#)
Returns the current step for the current value of the parameter.
- double [GetNormalizedValueFromStep](#) (uint32_t iStep) const [AAX_OVERRIDE](#)
Returns the normalized value for a given step.
- uint32_t [GetStepValueFromNormalizedValue](#) (double normalizedValue) const [AAX_OVERRIDE](#)
Returns the step value for a normalized value of the parameter.
- void [SetStepValue](#) (uint32_t iStep) [AAX_OVERRIDE](#)
Returns the current step for the current value of the parameter.
- void [SetType](#) ([AAX_EParameterType](#) iControlType) [AAX_OVERRIDE](#)
Sets the type of this parameter.
- [AAX_EParameterType](#) [GetType](#) () const [AAX_OVERRIDE](#)
Returns the type of this parameter as an [AAX_EParameterType](#).
- void [SetOrientation](#) ([AAX_EParameterOrientation](#) iOrientation) [AAX_OVERRIDE](#)
Sets the orientation of this parameter.
- [AAX_EParameterOrientation](#) [GetOrientation](#) () const [AAX_OVERRIDE](#)
Returns the orientation of this parameter.
- void [SetTaperDelegate](#) ([AAX_ITaperDelegateBase](#) &inTaperDelegate, bool inPreserveValue=true) [AAX_OVERRIDE](#)
Sets the parameter's taper delegate.

Display methods

- void [SetDisplayDelegate](#) ([AAX_IDisplayDelegateBase](#) &inDisplayDelegate) [AAX_OVERRIDE](#)
Sets the parameter's display delegate.
- bool [GetValueString](#) ([AAX_CString](#) *valueString) const [AAX_OVERRIDE](#)
Serializes the parameter value into a string.
- bool [GetValueString](#) (int32_t iMaxNumChars, [AAX_CString](#) *valueString) const [AAX_OVERRIDE](#)
Serializes the parameter value into a string, size hint included.
- bool [GetNormalizedValueFromBool](#) (bool value, double *normalizedValue) const [AAX_OVERRIDE](#)
Converts a bool to a normalized parameter value.
- bool [GetNormalizedValueFromInt32](#) (int32_t value, double *normalizedValue) const [AAX_OVERRIDE](#)
Converts an integer to a normalized parameter value.
- bool [GetNormalizedValueFromFloat](#) (float value, double *normalizedValue) const [AAX_OVERRIDE](#)
Converts a float to a normalized parameter value.
- bool [GetNormalizedValueFromDouble](#) (double value, double *normalizedValue) const [AAX_OVERRIDE](#)
Converts a double to a normalized parameter value.
- bool [GetNormalizedValueFromString](#) (const [AAX_CString](#) &valueString, double *normalizedValue) const [AAX_OVERRIDE](#)
Converts a given string to a normalized parameter value.
- bool [GetBoolFromNormalizedValue](#) (double normalizedValue, bool *value) const [AAX_OVERRIDE](#)
Converts a normalized parameter value to a bool representing the corresponding real value.
- bool [GetInt32FromNormalizedValue](#) (double normalizedValue, int32_t *value) const [AAX_OVERRIDE](#)
Converts a normalized parameter value to an integer representing the corresponding real value.
- bool [GetFloatFromNormalizedValue](#) (double normalizedValue, float *value) const [AAX_OVERRIDE](#)
Converts a normalized parameter value to a float representing the corresponding real value.
- bool [GetDoubleFromNormalizedValue](#) (double normalizedValue, double *value) const [AAX_OVERRIDE](#)
Converts a normalized parameter value to a double representing the corresponding real value.

- bool [GetStringFromNormalizedValue](#) (double normalizedValue, [AAX_CString](#) &valueString) const [AAX_OVERRIDE](#)
Converts a normalized parameter value to a string representing the corresponding real value.
- bool [GetStringFromNormalizedValue](#) (double normalizedValue, int32_t iMaxNumChars, [AAX_CString](#) &valueString) const [AAX_OVERRIDE](#)
Converts a normalized parameter value to a string representing the corresponding real, size hint included. value.
- bool [SetValueFromString](#) (const [AAX_CString](#) &newValueString) [AAX_OVERRIDE](#)
Converts a string to a real parameter value and sets the parameter to this value.

Automation methods

- void [SetAutomationDelegate](#) ([AAX_IAutomationDelegate](#) *iAutomationDelegate) [AAX_OVERRIDE](#)
Sets the automation delegate (if one is required)
- bool [Automatable](#) () const [AAX_OVERRIDE](#)
Returns true if the parameter is automatable, false if it is not.
- void [Touch](#) () [AAX_OVERRIDE](#)
Signals the automation system that a control has been touched.
- void [Release](#) () [AAX_OVERRIDE](#)
Signals the automation system that a control has been released.

Typed accessors

- bool [GetValueAsBool](#) (bool *value) const [AAX_OVERRIDE](#)
Retrieves the parameter's value as a bool.
- bool [GetValueAsInt32](#) (int32_t *value) const [AAX_OVERRIDE](#)
Retrieves the parameter's value as an int32_t.
- bool [GetValueAsFloat](#) (float *value) const [AAX_OVERRIDE](#)
Retrieves the parameter's value as a float.
- bool [GetValueAsDouble](#) (double *value) const [AAX_OVERRIDE](#)
Retrieves the parameter's value as a double.
- bool [GetValueAsString](#) ([AAX_IString](#) *value) const [AAX_OVERRIDE](#)
Retrieves the parameter's value as a string.
- bool [SetValueWithBool](#) (bool value) [AAX_OVERRIDE](#)
Sets the parameter's value as a bool.
- bool [SetValueWithInt32](#) (int32_t value) [AAX_OVERRIDE](#)
Sets the parameter's value as an int32_t.
- bool [SetValueWithFloat](#) (float value) [AAX_OVERRIDE](#)
Sets the parameter's value as a float.
- bool [SetValueWithDouble](#) (double value) [AAX_OVERRIDE](#)
Sets the parameter's value as a double.
- bool [SetValueWithString](#) (const [AAX_IString](#) &value) [AAX_OVERRIDE](#)
Sets the parameter's value as a string.

Host interface methods

- void [UpdateNormalizedValue](#) (double newNormalizedValue) [AAX_OVERRIDE](#)
Sets the parameter's state given a normalized value.

Direct methods on AAX_CParameter

These methods can be used to access the parameter's state and properties. These methods are specific to the concrete [AAX_CParameter](#) class and are not part of the [AAX_IParameter](#) interface.

- [AAX_CStringAbbreviations](#) mNames
- bool [mAutomatable](#)
- uint32_t [mNumSteps](#)
- [AAX_EParameterType](#) mControlType
- [AAX_EParameterOrientation](#) mOrientation
- [AAX_ITaperDelegate](#)< T > * [mTaperDelegate](#)
- [AAX_IDisplayDelegate](#)< T > * [mDisplayDelegate](#)
- [AAX_IAutomationDelegate](#) * [mAutomationDelegate](#)
- bool [mNeedNotify](#)
- [AAX_CParameterValue](#)< T > [mValue](#)
- T [mDefaultValue](#)
- void [SetValue](#) (T newValue)
Initiates a host request to set the parameter's value.
- T [GetValue](#) () const
Returns the parameter's value.
- void [SetDefaultValue](#) (T newDefaultValue)
Set the parameter's default value.
- T [GetDefaultValue](#) () const
Returns the parameter's default value.
- const [AAX_ITaperDelegate](#)< T > * [TaperDelegate](#) () const
Returns a reference to the parameter's taper delegate.
- const [AAX_IDisplayDelegate](#)< T > * [DisplayDelegate](#) () const
Returns a reference to the parameter's display delegate.

14.28.2 Member Enumeration Documentation

14.28.2.1 Type

```
template<typename T >
enum AAX\_CParameter::Type
```

Enumerator

eParameterTypeUndefined	
eParameterTypeBool	
eParameterTypeInt32	
eParameterTypeFloat	
eParameterTypeCustom	

14.28.2.2 Defaults

```
template<typename T >
enum AAX_CParameter::Defaults
```

Enumerator

eParameterDefaultNumStepsDiscrete	
eParameterDefaultNumStepsContinuous	

14.28.3 Constructor & Destructor Documentation

14.28.3.1 AAX_CParameter() [1/4]

```
template<typename T >
AAX_CParameter< T >::AAX_CParameter (
    AAX_CParamID identifier,
    const AAX_IString & name,
    T defaultValue,
    const AAX_ITaperDelegate< T > & taperDelegate,
    const AAX_IDisplayDelegate< T > & displayDelegate,
    bool automatable = false )
```

Constructs an [AAX_CParameter](#) object using the specified taper and display delegates.

The delegates are passed in by reference to prevent ambiguities of object ownership. For more information about *identifier* and *name*, please consult the base [AAX_IParameter](#) interface.

Parameters

in	<i>identifier</i>	Unique ID for the parameter, these can only be 31 characters long at most. (the fixed length is a requirement for some optimizations in the host)
in	<i>name</i>	The parameter's unabbreviated display name
in	<i>defaultValue</i>	The parameter's default value
in	<i>taperDelegate</i>	A delegate representing the parameter's taper behavior
in	<i>displayDelegate</i>	A delegate representing the parameter's display conversion behavior
in	<i>automatable</i>	A flag to set whether the parameter will be visible to the host's automation system

Note

Upon construction, the state (value) of the parameter will be the default value, as established by the provided *taperDelegate*.

Host Compatibility Notes As of Pro Tools 10.2, DAE will check for a matching parameter NAME and not an ID when reading in automation data from a session saved with an AAX plug-ins RTAS/↔ TDM counter part.

As of Pro Tools 11.1, AAE will first try to match ID. If that fails, AAE will fall back to matching by Name.

References `AAX_CParameter< T >::SetToDefaultValue()`.

Here is the call graph for this function:

14.28.3.2 `AAX_CParameter()` [2/4]

```
template<typename T >
AAX_CParameter< T >::AAX_CParameter (
    const AAX_IString & identifier,
    const AAX_IString & name,
    T defaultValue,
    const AAX_ITaperDelegate< T > & taperDelegate,
    const AAX_IDisplayDelegate< T > & displayDelegate,
    bool automatable = false )
```

Constructs an `AAX_CParameter` object using the specified taper and display delegates.

This constructor uses an `AAX_IString` for the parameter identifier, which can be a more flexible solution for some plug-ins.

References `AAX_CParameter< T >::SetToDefaultValue()`.

Here is the call graph for this function:

14.28.3.3 `AAX_CParameter()` [3/4]

```
template<typename T >
AAX_CParameter< T >::AAX_CParameter (
    const AAX_IString & identifier,
    const AAX_IString & name,
    T defaultValue,
    bool automatable = false )
```

Constructs an `AAX_CParameter` object with no delegates.

Delegates may be set on this object after construction. Most parameter operations will not work until after delegates have been set.

See also

- `AAX_CParameter::SetTaperDelegate()`

See also

- `AAX_CParameter::SetDisplayDelegate()`

References `AAX_CParameter< T >::SetToDefaultValue()`.

Here is the call graph for this function:

14.28.3.4 AAX_CParameter() [4/4]

```
template<typename T >
AAX_CParameter< T >::AAX_CParameter (
    const AAX_IString & identifier,
    const AAX_IString & name,
    bool automatable = false )
```

Constructs an [AAX_CParameter](#) object with no delegates or default value.

Delegates and default value may be set on this object after construction. Most parameter operations will not work until after delegates have been set.

See also

- [AAX_CParameter::SetDefaultValue\(\)](#)

See also

- [AAX_CParameter::SetTaperDelegate\(\)](#)

See also

- [AAX_CParameter::SetDisplayDelegate\(\)](#)

References [AAX_CParameter< T >::SetToDefaultValue\(\)](#).

Here is the call graph for this function:

14.28.3.5 ~AAX_CParameter()

```
template<typename T >
AAX_CParameter< T >::~~AAX_CParameter
```

Virtual destructor used to delete all locally allocated pointers.

14.28.4 Member Function Documentation**14.28.4.1 AAX_DEFAULT_MOVE_CTOR()**

```
template<typename T >
AAX_CParameter< T >::AAX_DEFAULT_MOVE_CTOR (
    AAX_CParameter< T > )
```

Move constructor and move assignment operator are allowed

14.28.4.2 AAX_DEFAULT_MOVE_OPER()

```
template<typename T >
AAX_CParameter< T >::AAX_DEFAULT_MOVE_OPER (
    AAX_CParameter< T > )
```

14.28.4.3 AAX_DELETE() [1/3]

```
template<typename T >
AAX_CParameter< T >::AAX_DELETE (
    AAX_CParameter< T >() )
```

Default constructor not allowed, except by possible wrapper classes.

14.28.4.4 AAX_DELETE() [2/3]

```
template<typename T >
AAX_CParameter< T >::AAX_DELETE (
    AAX_CParameter< T >(const AAX_CParameter< T > &other) )
```

14.28.4.5 AAX_DELETE() [3/3]

```
template<typename T >
AAX_CParameter< T >::AAX_DELETE (
    AAX_CParameter< T > & operator = (const AAX_CParameter< T > &other) )
```

14.28.4.6 CloneValue()

```
template<typename T >
AAX_IParameterValue * AAX_CParameter< T >::CloneValue [virtual]
```

Clone the parameter's value to a new [AAX_IParameterValue](#) object.

The returned object is independent from the [AAX_IParameter](#). For example, changing the state of the returned object will not result in a change to the original [AAX_IParameter](#).

Implements [AAX_IParameter](#).

14.28.4.7 Identifier()

```
template<typename T >
AAX_CParamID AAX_CParameter< T >::Identifier [virtual]
```

Returns the parameter's unique identifier.

This unique ID is used by the [Parameter Manager](#) and by outside applications to uniquely identify and target control messages. This value may not be changed after the parameter has been constructed.

Implements [AAX_IParameter](#).

14.28.4.8 SetName()

```
template<typename T >
void AAX_CParameter< T >::SetName (
    const AAX_CString & name ) [virtual]
```

Sets the parameter's display name.

This name is used for display only, it is not used for indexing or identifying the parameter. This name may be changed after the parameter has been created, but display name changes may not be recognized by all AAX hosts.

Parameters

in	<i>name</i>	Display name that will be assigned to the parameter
----	-------------	---

Implements [AAX_IParameter](#).

14.28.4.9 Name()

```
template<typename T >
const AAX_CString & AAX_CParameter< T >::Name [virtual]
```

Returns the parameter's display name.

Note

This method returns a const reference in order to prevent a string copy. Do not cast away the const to change this value.

Implements [AAX_IParameter](#).

14.28.4.10 AddShortenedName()

```
template<typename T >
void AAX_CParameter< T >::AddShortenedName (
    const AAX_CString & name ) [virtual]
```

Sets the parameter's shortened display name.

This name is used for display only, it is not used for indexing or identifying the parameter. These names show up when the host asks for shorter length parameter names for display on Control Surfaces or other string length constrained situations.

Parameters

in	<i>name</i>	Shortened display names that will be assigned to the parameter
----	-------------	--

Implements [AAX_IParameter](#).

14.28.4.11 ShortenedName()

```
template<typename T >
const AAX_CString & AAX_CParameter< T >::ShortenedName (
    int32_t iNumCharacters ) const [virtual]
```

Returns the parameter's shortened display name.

Note

This method returns a const reference in order to prevent a string copy. Do not cast away the const to change this value.

Implements [AAX_IParameter](#).

References [AAX_CString::Get\(\)](#).

Here is the call graph for this function:

14.28.4.12 ClearShortenedNames()

```
template<typename T >
void AAX\_CParameter< T >::ClearShortenedNames [virtual]
```

Clears the internal list of shortened display names.

Implements [AAX_IParameter](#).

14.28.4.13 SetNormalizedDefaultValue()

```
template<typename T >
void AAX\_CParameter< T >::SetNormalizedDefaultValue (
    double normalizedDefault ) [virtual]
```

Sets the parameter's default value using its normalized representation.

Implements [AAX_IParameter](#).

14.28.4.14 GetNormalizedDefaultValue()

```
template<typename T >
double AAX\_CParameter< T >::GetNormalizedDefaultValue [virtual]
```

Returns the normalized representation of the parameter's real default value.

Implements [AAX_IParameter](#).

14.28.4.15 SetToDefaultValue()

```
template<typename T >
void AAX\_CParameter< T >::SetToDefaultValue [virtual]
```

Restores the state of this parameter to its default value.

Implements [AAX_IParameter](#).

Referenced by [AAX_CParameter< T >::AAX_CParameter\(\)](#).

Here is the caller graph for this function:

14.28.4.16 SetNormalizedValue()

```
template<typename T >
void AAX\_CParameter< T >::SetNormalizedValue (
    double newNormalizedValue ) [virtual]
```

Sets a parameter value using it's normalized representation.

For more information regarding normalized values, see [Parameter Manager](#)

Parameters

in	<i>newNormalizedValue</i>	New value (normalized) to which the parameter will be set
----	---------------------------	---

Implements [AAX_IParameter](#).

14.28.4.17 GetNormalizedValue()

```
template<typename T >
double AAX_CParameter< T >::GetNormalizedValue [virtual]
```

Returns the normalized representation of the parameter's current real value.

Implements [AAX_IParameter](#).

14.28.4.18 SetNumberOfSteps()

```
template<typename T >
void AAX_CParameter< T >::SetNumberOfSteps (
    uint32_t numSteps ) [virtual]
```

Sets the number of discrete steps for this parameter.

Stepped parameter values are useful for discrete parameters and for "jumping" events such as mouse wheels, page up/down, etc. The parameter's step size is used to specify the coarseness of those changes.

Note

numSteps MUST be greater than zero. All other values may be considered an error by the host.

Parameters

in	<i>numSteps</i>	The number of steps that the parameter will use
----	-----------------	---

Implements [AAX_IParameter](#).

References [AAX_ASSERT](#).

14.28.4.19 GetNumberOfSteps()

```
template<typename T >
uint32_t AAX_CParameter< T >::GetNumberOfSteps [virtual]
```

Returns the number of discrete steps used by the parameter.

See [SetNumberOfSteps\(\)](#) for more information about parameter steps.

Implements [AAX_IParameter](#).

14.28.4.20 GetStepValue()

```
template<typename T >
uint32_t AAX_CParameter< T >::GetStepValue [virtual]
```

Returns the current step for the current value of the parameter.

See [SetNumberOfSteps\(\)](#) for more information about parameter steps.

Implements [AAX_IParameter](#).

14.28.4.21 GetNormalizedValueFromStep()

```
template<typename T >
double AAX_CParameter< T >::GetNormalizedValueFromStep (
    uint32_t iStep ) const [virtual]
```

Returns the normalized value for a given step.

See [SetNumberOfSteps\(\)](#) for more information about parameter steps.

Implements [AAX_IParameter](#).

14.28.4.22 GetStepValueFromNormalizedValue()

```
template<typename T >
uint32_t AAX_CParameter< T >::GetStepValueFromNormalizedValue (
    double normalizedValue ) const [virtual]
```

Returns the step value for a normalized value of the parameter.

See [SetNumberOfSteps\(\)](#) for more information about parameter steps.

Implements [AAX_IParameter](#).

14.28.4.23 SetStepValue()

```
template<typename T >
void AAX_CParameter< T >::SetStepValue (
    uint32_t iStep ) [virtual]
```

Returns the current step for the current value of the parameter.

See [SetNumberOfSteps\(\)](#) for more information about parameter steps.

Implements [AAX_IParameter](#).

14.28.4.24 SetType()

```
template<typename T >
void AAX_CParameter< T >::SetType (
    AAX_EParameterType iControlType ) [virtual]
```

Sets the type of this parameter.

See [GetType](#) for use cases

Parameters

in	<i>iControlType</i>	The parameter's new type as an AAX_EParameterType
----	---------------------	---

Implements [AAX_IParameter](#).

14.28.4.25 GetType()

```
template<typename T >
AAX_EParameterType AAX_CParameter< T >::GetType [virtual]
```

Returns the type of this parameter as an [AAX_EParameterType](#).

Todo Document use cases for control type

Implements [AAX_IParameter](#).

14.28.4.26 SetOrientation()

```
template<typename T >
void AAX_CParameter< T >::SetOrientation (
    AAX_EParameterOrientation iOrientation ) [virtual]
```

Sets the orientation of this parameter.

Parameters

in	<i>iOrientation</i>	The parameter's new orientation
----	---------------------	---------------------------------

Implements [AAX_IParameter](#).

14.28.4.27 GetOrientation()

```
template<typename T >
AAX_EParameterOrientation AAX_CParameter< T >::GetOrientation [virtual]
```

Returns the orientation of this parameter.

Implements [AAX_IParameter](#).

14.28.4.28 SetTaperDelegate()

```
template<typename T >
void AAX_CParameter< T >::SetTaperDelegate (
    AAX_ITaperDelegateBase & inTaperDelegate,
    bool inPreserveValue = true ) [virtual]
```

Sets the parameter's taper delegate.

Parameters

in	<i>inTaperDelegate</i>	A reference to the parameter's new taper delegate
in	<i>inPreserveValue</i>	

Todo Document this parameter

Implements [AAX_IParameter](#).

References [AAX_ITaperDelegate< T >::Clone\(\)](#).

Here is the call graph for this function:

14.28.4.29 SetDisplayDelegate()

```
template<typename T >
void AAX_CParameter< T >::SetDisplayDelegate (
    AAX_IDisplayDelegateBase & inDisplayDelegate ) [virtual]
```

Sets the parameter's display delegate.

Parameters

in	<i>inDisplayDelegate</i>	A reference to the parameter's new display delegate
----	--------------------------	---

Implements [AAX_IParameter](#).

References [AAX_IDisplayDelegate< T >::Clone\(\)](#).

Here is the call graph for this function:

14.28.4.30 GetValueString() [1/2]

```
template<typename T >
bool AAX_CParameter< T >::GetValueString (
    AAX_CString * valueString ) const [virtual]
```

Serializes the parameter value into a string.

Parameters

out	<i>valueString</i>	A string representing the parameter's real value
-----	--------------------	--

Return values

<i>true</i>	The string conversion was successful
<i>false</i>	The string conversion was unsuccessful

Implements [AAX_IParameter](#).

14.28.4.31 GetValueString() [2/2]

```
template<typename T >
bool AAX_CParameter< T >::GetValueString (
    int32_t iMaxNumChars,
    AAX_CString * valueString ) const [virtual]
```

Serializes the parameter value into a string, size hint included.

Parameters

in	<i>iMaxNumChars</i>	A size hint for the size of the string being requested. Useful for control surfaces and other limited area text fields. (make sure that size of desired string also has room for null termination)
out	<i>valueString</i>	A string representing the parameter's real value

Return values

<i>true</i>	The string conversion was successful
<i>false</i>	The string conversion was unsuccessful

Implements [AAX_IParameter](#).

14.28.4.32 GetNormalizedValueFromBool() [1/2]

```
template<typename T >
bool AAX_CParameter< T >::GetNormalizedValueFromBool (
    bool value,
    double * normalizedValue ) const [virtual]
```

Converts a bool to a normalized parameter value.

Parameters

in	<i>value</i>	A value for the parameter
out	<i>normalizedValue</i>	The normalized parameter value associated with value

Return values

<i>true</i>	The value conversion was successful
<i>false</i>	The value conversion was unsuccessful

Implements [AAX_IParameter](#).

14.28.4.33 GetNormalizedValueFromInt32() [1/2]

```
template<typename T >
bool AAX_CParameter< T >::GetNormalizedValueFromInt32 (
    int32_t value,
    double * normalizedValue ) const [virtual]
```

Converts an integer to a normalized parameter value.

Parameters

in	<i>value</i>	A value for the parameter
out	<i>normalizedValue</i>	The normalized parameter value associated with value

Return values

<i>true</i>	The value conversion was successful
-------------	-------------------------------------

Return values

<i>false</i>	The value conversion was unsuccessful
--------------	---------------------------------------

Implements [AAX_IParameter](#).

14.28.4.34 GetNormalizedValueFromFloat() [1/2]

```
template<typename T >
bool AAX_CParameter< T >::GetNormalizedValueFromFloat (
    float value,
    double * normalizedValue ) const [virtual]
```

Converts a float to a normalized parameter value.

Parameters

in	<i>value</i>	A value for the parameter
out	<i>normalizedValue</i>	The normalized parameter value associated with value

Return values

<i>true</i>	The value conversion was successful
<i>false</i>	The value conversion was unsuccessful

Implements [AAX_IParameter](#).

14.28.4.35 GetNormalizedValueFromDouble() [1/2]

```
template<typename T >
bool AAX_CParameter< T >::GetNormalizedValueFromDouble (
    double value,
    double * normalizedValue ) const [virtual]
```

Converts a double to a normalized parameter value.

Parameters

in	<i>value</i>	A value for the parameter
out	<i>normalizedValue</i>	The normalized parameter value associated with value

Return values

<i>true</i>	The value conversion was successful
<i>false</i>	The value conversion was unsuccessful

Implements [AAX_IParameter](#).

14.28.4.36 GetNormalizedValueFromString()

```
template<typename T >
bool AAX_CParameter< T >::GetNormalizedValueFromString (
    const AAX_CString & valueString,
    double * normalizedValue ) const [virtual]
```

Converts a given string to a normalized parameter value.

Parameters

in	<i>valueString</i>	A string representing a possible real value for the parameter
out	<i>normalizedValue</i>	The normalized parameter value associated with valueString

Return values

<i>true</i>	The string conversion was successful
<i>false</i>	The string conversion was unsuccessful

Implements [AAX_IParameter](#).

14.28.4.37 GetBoolFromNormalizedValue() [1/2]

```
template<typename T >
bool AAX_CParameter< T >::GetBoolFromNormalizedValue (
    double normalizedValue,
    bool * value ) const [virtual]
```

Converts a normalized parameter value to a bool representing the corresponding real value.

Parameters

in	<i>normalizedValue</i>	The normalized value to convert
out	<i>value</i>	The converted value. Set only if conversion is successful.

Return values

<i>true</i>	The conversion to bool was successful
<i>false</i>	The conversion to bool was unsuccessful

Implements [AAX_IParameter](#).

14.28.4.38 GetInt32FromNormalizedValue() [1/2]

```
template<typename T >
bool AAX_CParameter< T >::GetInt32FromNormalizedValue (
    double normalizedValue,
    int32_t * value ) const [virtual]
```

Converts a normalized parameter value to an integer representing the corresponding real value.

Parameters

in	<i>normalizedValue</i>	The normalized value to convert
out	<i>value</i>	The converted value. Set only if conversion is successful.

Return values

<i>true</i>	The conversion to int32_t was successful
<i>false</i>	The conversion to int32_t was unsuccessful

Implements [AAX_IParameter](#).

14.28.4.39 GetFloatFromNormalizedValue() [1/2]

```
template<typename T >
bool AAX_CParameter< T >::GetFloatFromNormalizedValue (
    double normalizedValue,
    float * value ) const [virtual]
```

Converts a normalized parameter value to a float representing the corresponding real value.

Parameters

in	<i>normalizedValue</i>	The normalized value to convert
out	<i>value</i>	The converted value. Set only if conversion is successful.

Return values

<i>true</i>	The conversion to float was successful
<i>false</i>	The conversion to float was unsuccessful

Implements [AAX_IParameter](#).

14.28.4.40 GetDoubleFromNormalizedValue() [1/2]

```
template<typename T >
bool AAX_CParameter< T >::GetDoubleFromNormalizedValue (
```

```
double normalizedValue,
double * value ) const [virtual]
```

Converts a normalized parameter value to a double representing the corresponding real value.

Parameters

in	<i>normalizedValue</i>	The normalized value to convert
out	<i>value</i>	The converted value. Set only if conversion is successful.

Return values

<i>true</i>	The conversion to double was successful
<i>false</i>	The conversion to double was unsuccessful

Implements [AAX_IParameter](#).

14.28.4.41 GetStringFromNormalizedValue() [1/2]

```
template<typename T >
bool AAX_CParameter< T >::GetStringFromNormalizedValue (
    double normalizedValue,
    AAX_CString & valueString ) const [virtual]
```

Converts a normalized parameter value to a string representing the corresponding real value.

Parameters

in	<i>normalizedValue</i>	A normalized parameter value
out	<i>valueString</i>	A string representing the parameter value associated with <i>normalizedValue</i>

Return values

<i>true</i>	The string conversion was successful
<i>false</i>	The string conversion was unsuccessful

Implements [AAX_IParameter](#).

14.28.4.42 GetStringFromNormalizedValue() [2/2]

```
template<typename T >
bool AAX_CParameter< T >::GetStringFromNormalizedValue (
    double normalizedValue,
    int32_t iMaxNumChars,
    AAX_CString & valueString ) const [virtual]
```

Converts a normalized parameter value to a string representing the corresponding real, size hint included. value.

Parameters

in	<i>normalizedValue</i>	A normalized parameter value
in	<i>iMaxNumChars</i>	A size hint for the size of the string being requested. Useful for control surfaces and other limited area text fields. (make sure that size of desired string also has room for null termination)
out	<i>valueString</i>	A string representing the parameter value associated with <i>normalizedValue</i>

Return values

<i>true</i>	The string conversion was successful
<i>false</i>	The string conversion was unsuccessful

Implements [AAX_IParameter](#).

14.28.4.43 SetValueFromString()

```
template<typename T >
bool AAX_CParameter< T >::SetValueFromString (
    const AAX_CString & newValueString ) [virtual]
```

Converts a string to a real parameter value and sets the parameter to this value.

Parameters

in	<i>newValueString</i>	A string representing the parameter's new real value
----	-----------------------	--

Return values

<i>true</i>	The string conversion was successful
<i>false</i>	The string conversion was unsuccessful

Implements [AAX_IParameter](#).

14.28.4.44 SetAutomationDelegate()

```
template<typename T >
void AAX_CParameter< T >::SetAutomationDelegate (
    AAX_IAutomationDelegate * iAutomationDelegate ) [virtual]
```

Sets the automation delegate (if one is required)

Parameters

in	<i>iAutomationDelegate</i>	A reference to the parameter manager's automation delegate interface
----	----------------------------	--

Implements [AAX_IParameter](#).

References [AAX_IAutomationDelegate::RegisterParameter\(\)](#).

Here is the call graph for this function:

14.28.4.45 Automatable()

```
template<typename T >
bool AAX_CParameter< T >::Automatable [virtual]
```

Returns true if the parameter is automatable, false if it is not.

Note

Subclasses that return true in this method must support host-based automation.

Implements [AAX_IParameter](#).

14.28.4.46 Touch()

```
template<typename T >
void AAX_CParameter< T >::Touch [virtual]
```

Signals the automation system that a control has been touched.

Call this method in response to GUI events that begin editing, such as a mouse down. After this method has been called you are free to call [SetNormalizedValue\(\)](#) as much as you need, e.g. in order to respond to subsequent mouse moved events. Call [Release\(\)](#) to free the parameter for updates from other controls.

Implements [AAX_IParameter](#).

14.28.4.47 Release()

```
template<typename T >
void AAX_CParameter< T >::Release (
    void ) [virtual]
```

Signals the automation system that a control has been released.

Call this method in response to GUI events that complete editing, such as a mouse up. Once this method has been called you should not call [SetNormalizedValue\(\)](#) again until after the next call to [Touch\(\)](#).

Implements [AAX_IParameter](#).

14.28.4.48 GetValueAsBool()

```
template<typename T >
bool AAX_CParameter< T >::GetValueAsBool (
    bool * value ) const [virtual]
```

Retrieves the parameter's value as a bool.

Parameters

out	value	The parameter's real value. Set only if conversion is successful.
-----	-------	---

Return values

true	The conversion to bool was successful
false	The conversion to bool was unsuccessful

Implements [AAX_IParameter](#).

14.28.4.49 GetValueAsInt32()

```
template<typename T >
bool AAX_CParameter< T >::GetValueAsInt32 (
    int32_t * value ) const [virtual]
```

Retrieves the parameter's value as an int32_t.

Parameters

out	value	The parameter's real value. Set only if conversion is successful.
-----	-------	---

Return values

true	The conversion to int32_t was successful
false	The conversion to int32_t was unsuccessful

Implements [AAX_IParameter](#).

14.28.4.50 GetValueAsFloat()

```
template<typename T >
bool AAX_CParameter< T >::GetValueAsFloat (
    float * value ) const [virtual]
```

Retrieves the parameter's value as a float.

Parameters

out	value	The parameter's real value. Set only if conversion is successful.
-----	-------	---

Return values

<i>true</i>	The conversion to float was successful
<i>false</i>	The conversion to float was unsuccessful

Implements [AAX_IParameter](#).

14.28.4.51 GetValueAsDouble()

```
template<typename T >
bool AAX_CParameter< T >::GetValueAsDouble (
    double * value ) const [virtual]
```

Retrieves the parameter's value as a double.

Parameters

out	<i>value</i>	The parameter's real value. Set only if conversion is successful.
-----	--------------	---

Return values

<i>true</i>	The conversion to double was successful
<i>false</i>	The conversion to double was unsuccessful

Implements [AAX_IParameter](#).

14.28.4.52 GetValueAsString() [1/2]

```
template<typename T >
bool AAX_CParameter< T >::GetValueAsString (
    AAX_IString * value ) const [virtual]
```

Retrieves the parameter's value as a string.

Parameters

out	<i>value</i>	The parameter's real value. Set only if conversion is successful.
-----	--------------	---

Return values

<i>true</i>	The conversion to string was successful
<i>false</i>	The conversion to string was unsuccessful

Implements [AAX_IParameter](#).

14.28.4.53 SetValueWithBool() [1/2]

```
template<typename T >
bool AAX_CParameter< T >::SetValueWithBool (
    bool value ) [virtual]
```

Sets the parameter's value as a bool.

Parameters

out	<i>value</i>	The parameter's real value. Set only if conversion is successful.
-----	--------------	---

Return values

<i>true</i>	The conversion from bool was successful
<i>false</i>	The conversion from bool was unsuccessful

Implements [AAX_IParameter](#).

14.28.4.54 SetValueWithInt32() [1/2]

```
template<typename T >
bool AAX_CParameter< T >::SetValueWithInt32 (
    int32_t value ) [virtual]
```

Sets the parameter's value as an int32_t.

Parameters

out	<i>value</i>	The parameter's real value. Set only if conversion is successful.
-----	--------------	---

Return values

<i>true</i>	The conversion from int32_t was successful
<i>false</i>	The conversion from int32_t was unsuccessful

Implements [AAX_IParameter](#).

14.28.4.55 SetValueWithFloat() [1/2]

```
template<typename T >
bool AAX_CParameter< T >::SetValueWithFloat (
    float value ) [virtual]
```

Sets the parameter's value as a float.

Parameters

<i>out</i>	<i>value</i>	The parameter's real value. Set only if conversion is successful.
------------	--------------	---

Return values

<i>true</i>	The conversion from float was successful
<i>false</i>	The conversion from float was unsuccessful

Implements [AAX_IParameter](#).

14.28.4.56 SetValueWithDouble() [1/2]

```
template<typename T >
bool AAX_CParameter< T >::SetValueWithDouble (
    double value ) [virtual]
```

Sets the parameter's value as a double.

Parameters

<i>out</i>	<i>value</i>	The parameter's real value. Set only if conversion is successful.
------------	--------------	---

Return values

<i>true</i>	The conversion from double was successful
<i>false</i>	The conversion from double was unsuccessful

Implements [AAX_IParameter](#).

14.28.4.57 SetValueWithString() [1/2]

```
template<typename T >
bool AAX_CParameter< T >::SetValueWithString (
    const AAX_IString & value ) [virtual]
```

Sets the parameter's value as a string.

Parameters

<i>out</i>	<i>value</i>	The parameter's real value. Set only if conversion is successful.
------------	--------------	---

Return values

<i>true</i>	The conversion from string was successful
<i>false</i>	The conversion from string was unsuccessful

Implements [AAX_IParameter](#).

14.28.4.58 UpdateNormalizedValue()

```
template<typename T >
void AAX_CParameter< T >::UpdateNormalizedValue (
    double newNormalizedValue ) [virtual]
```

Sets the parameter's state given a normalized value.

This is the second half of the parameter setting operation that is initiated with a call to [SetValue\(\)](#). Parameters should not be set directly using this method; instead, use [SetValue\(\)](#).

Parameters

in	<i>newNormalizedValue</i>	Normalized value that will be used to set the parameter's new state
----	---------------------------	---

Implements [AAX_IParameter](#).

14.28.4.59 SetValue()

```
template<typename T >
void AAX_CParameter< T >::SetValue (
    T newValue )
```

Initiates a host request to set the parameter's value.

This method normalizes the provided value and sends a request for the value change to the AAX host. The host responds with a call to [AAX_IParameter::UpdateNormalizedValue\(\)](#) to complete the set operation.

Parameters

in	<i>newValue</i>	The parameter's new value
----	-----------------	---------------------------

14.28.4.60 GetValue()

```
template<typename T >
T AAX_CParameter< T >::GetValue
```

Returns the parameter's value.

This is the parameter's real, logical value and should not be normalized

14.28.4.61 SetDefaultValue()

```
template<typename T >
void AAX_CParameter< T >::SetDefaultValue (
    T newDefaultValue )
```

Set the parameter's default value.

This is the parameter's real, logical value and should not be normalized

Parameters

in	<i>newDefaultValue</i>	The parameter's new default value
----	------------------------	-----------------------------------

14.28.4.62 GetDefaultValue()

```
template<typename T >
T AAX_CParameter< T >::GetDefaultValue
```

Returns the parameter's default value.

This is the parameter's real, logical value and should not be normalized

14.28.4.63 TaperDelegate()

```
template<typename T >
const AAX_ITaperDelegate< T > * AAX_CParameter< T >::TaperDelegate
```

Returns a reference to the parameter's taper delegate.

14.28.4.64 DisplayDelegate()

```
template<typename T >
const AAX_IDisplayDelegate< T > * AAX_CParameter< T >::DisplayDelegate
```

Returns a reference to the parameter's display delegate.

14.28.4.65 GetValueAsString() [2/2]

```
bool AAX_CParameter< AAX_CString >::GetValueAsString (
    AAX_IString * value ) const [virtual]
```

Retrieves the parameter's value as a string.

Parameters

out	value	The parameter's real value. Set only if conversion is successful.
-----	-------	---

Return values

true	The conversion to string was successful
false	The conversion to string was unsuccessful

Implements [AAX_IParameter](#).

14.28.4.66 SetValueWithBool() [2/2]

```
bool AAX_CParameter< bool >::SetValueWithBool (
    bool value ) [virtual]
```

Sets the parameter's value as a bool.

Parameters

out	value	The parameter's real value. Set only if conversion is successful.
-----	-------	---

Return values

true	The conversion from bool was successful
false	The conversion from bool was unsuccessful

Implements [AAX_IParameter](#).

14.28.4.67 SetValueWithInt32() [2/2]

```
bool AAX_CParameter< int32_t >::SetValueWithInt32 (
    int32_t value ) [virtual]
```

Sets the parameter's value as an int32_t.

Parameters

out	value	The parameter's real value. Set only if conversion is successful.
-----	-------	---

Return values

true	The conversion from int32_t was successful
false	The conversion from int32_t was unsuccessful

Implements [AAX_IParameter](#).

14.28.4.68 SetValueWithFloat() [2/2]

```
bool AAX_CParameter< float >::SetValueWithFloat (
    float value ) [virtual]
```

Sets the parameter's value as a float.

Parameters

out	<i>value</i>	The parameter's real value. Set only if conversion is successful.
-----	--------------	---

Return values

<i>true</i>	The conversion from float was successful
<i>false</i>	The conversion from float was unsuccessful

Implements [AAX_IParameter](#).

14.28.4.69 SetValueWithDouble() [2/2]

```
bool AAX_CParameter< double >::SetValueWithDouble (
    double value ) [virtual]
```

Sets the parameter's value as a double.

Parameters

out	<i>value</i>	The parameter's real value. Set only if conversion is successful.
-----	--------------	---

Return values

<i>true</i>	The conversion from double was successful
<i>false</i>	The conversion from double was unsuccessful

Implements [AAX_IParameter](#).

14.28.4.70 SetValueWithString() [2/2]

```
bool AAX_CParameter< AAX_CString >::SetValueWithString (
    const AAX_IString & value ) [virtual]
```

Sets the parameter's value as a string.

Parameters

out	<i>value</i>	The parameter's real value. Set only if conversion is successful.
-----	--------------	---

Return values

<i>true</i>	The conversion from string was successful
<i>false</i>	The conversion from string was unsuccessful

Implements [AAX_IParameter](#).

14.28.4.71 GetNormalizedValueFromBool() [2/2]

```
bool AAX_CParameter< bool >::GetNormalizedValueFromBool (
    bool value,
    double * normalizedValue ) const [virtual]
```

Converts a bool to a normalized parameter value.

Parameters

in	<i>value</i>	A value for the parameter
out	<i>normalizedValue</i>	The normalized parameter value associated with value

Return values

<i>true</i>	The value conversion was successful
<i>false</i>	The value conversion was unsuccessful

Implements [AAX_IParameter](#).

14.28.4.72 GetNormalizedValueFromInt32() [2/2]

```
bool AAX_CParameter< int32_t >::GetNormalizedValueFromInt32 (
    int32_t value,
    double * normalizedValue ) const [virtual]
```

Converts an integer to a normalized parameter value.

Parameters

in	<i>value</i>	A value for the parameter
out	<i>normalizedValue</i>	The normalized parameter value associated with value

Return values

<i>true</i>	The value conversion was successful
<i>false</i>	The value conversion was unsuccessful

Implements [AAX_IParameter](#).

14.28.4.73 GetNormalizedValueFromFloat() [2/2]

```
bool AAX_CParameter< float >::GetNormalizedValueFromFloat (
    float value,
    double * normalizedValue ) const [virtual]
```

Converts a float to a normalized parameter value.

Parameters

in	<i>value</i>	A value for the parameter
out	<i>normalizedValue</i>	The normalized parameter value associated with value

Return values

<i>true</i>	The value conversion was successful
<i>false</i>	The value conversion was unsuccessful

Implements [AAX_IParameter](#).

14.28.4.74 GetNormalizedValueFromDouble() [2/2]

```
bool AAX_CParameter< double >::GetNormalizedValueFromDouble (
    double value,
    double * normalizedValue ) const [virtual]
```

Converts a double to a normalized parameter value.

Parameters

in	<i>value</i>	A value for the parameter
out	<i>normalizedValue</i>	The normalized parameter value associated with value

Return values

<i>true</i>	The value conversion was successful
<i>false</i>	The value conversion was unsuccessful

Implements [AAX_IParameter](#).

14.28.4.75 GetBoolFromNormalizedValue() [2/2]

```
bool AAX_CParameter< bool >::GetBoolFromNormalizedValue (
    double normalizedValue,
    bool * value ) const [virtual]
```

Converts a normalized parameter value to a bool representing the corresponding real value.

Parameters

in	<i>normalizedValue</i>	The normalized value to convert
out	<i>value</i>	The converted value. Set only if conversion is successful.

Return values

<i>true</i>	The conversion to bool was successful
<i>false</i>	The conversion to bool was unsuccessful

Implements [AAX_IParameter](#).

14.28.4.76 GetInt32FromNormalizedValue() [2/2]

```
bool AAX_CParameter< int32_t >::GetInt32FromNormalizedValue (
    double normalizedValue,
    int32_t * value ) const [virtual]
```

Converts a normalized parameter value to an integer representing the corresponding real value.

Parameters

in	<i>normalizedValue</i>	The normalized value to convert
out	<i>value</i>	The converted value. Set only if conversion is successful.

Return values

<i>true</i>	The conversion to int32_t was successful
<i>false</i>	The conversion to int32_t was unsuccessful

Implements [AAX_IParameter](#).

14.28.4.77 GetFloatFromNormalizedValue() [2/2]

```
bool AAX_CParameter< float >::GetFloatFromNormalizedValue (
    double normalizedValue,
    float * value ) const [virtual]
```

Converts a normalized parameter value to a float representing the corresponding real value.

Parameters

in	<i>normalizedValue</i>	The normalized value to convert
out	<i>value</i>	The converted value. Set only if conversion is successful.

Return values

<i>true</i>	The conversion to float was successful
<i>false</i>	The conversion to float was unsuccessful

Implements [AAX_IParameter](#).

14.28.4.78 GetDoubleFromNormalizedValue() [2/2]

```
bool AAX_CParameter< double >::GetDoubleFromNormalizedValue (
    double normalizedValue,
    double * value ) const [virtual]
```

Converts a normalized parameter value to a double representing the corresponding real value.

Parameters

in	<i>normalizedValue</i>	The normalized value to convert
out	<i>value</i>	The converted value. Set only if conversion is successful.

Return values

<i>true</i>	The conversion to double was successful
<i>false</i>	The conversion to double was unsuccessful

Implements [AAX_IParameter](#).

14.28.5 Member Data Documentation

14.28.5.1 mNames

```
template<typename T >  
AAX_CStringAbbreviations AAX_CParameter< T >::mNames [protected]
```

14.28.5.2 mAutomatable

```
template<typename T >  
bool AAX_CParameter< T >::mAutomatable [protected]
```

14.28.5.3 mNumSteps

```
template<typename T >  
uint32_t AAX_CParameter< T >::mNumSteps [protected]
```

14.28.5.4 mControlType

```
template<typename T >  
AAX_EParameterType AAX_CParameter< T >::mControlType [protected]
```

14.28.5.5 mOrientation

```
template<typename T >  
AAX_EParameterOrientation AAX_CParameter< T >::mOrientation [protected]
```

14.28.5.6 mTaperDelegate

```
template<typename T >  
AAX_ITaperDelegate<T>* AAX_CParameter< T >::mTaperDelegate [protected]
```

14.28.5.7 mDisplayDelegate

```
template<typename T >  
AAX_IDisplayDelegate<T>* AAX_CParameter< T >::mDisplayDelegate [protected]
```

14.28.5.8 mAutomationDelegate

```
template<typename T >
AAX_IAutomationDelegate* AAX_CParameter< T >::mAutomationDelegate [protected]
```

14.28.5.9 mNeedNotify

```
template<typename T >
bool AAX_CParameter< T >::mNeedNotify [protected]
```

14.28.5.10 mValue

```
template<typename T >
AAX_CParameterValue<T> AAX_CParameter< T >::mValue [protected]
```

14.28.5.11 mDefaultValue

```
template<typename T >
T AAX_CParameter< T >::mDefaultValue [protected]
```

The documentation for this class was generated from the following file:

- [AAX_CParameter.h](#)

14.29 AAX_CParameterManager Class Reference

```
#include <AAX_CParameterManager.h>
```

Collaboration diagram for AAX_CParameterManager:

14.29.1 Description

A container object for plug-in parameters.

This implementation uses a STL vector to store a plug-in's set of parameters. This class contains a real implementation of the [Parameter Manager](#) (as opposed to a proxy.)

For more information, see [Parameter Manager](#).

Todo Should the Parameter Manager return error codes?

Public Member Functions

- [AAX_CParameterManager](#) ()
- [~AAX_CParameterManager](#) ()
- void [Initialize](#) ([AAX_IAutomationDelegate](#) *iAutomationDelegateUnknown)
Initialize the parameter manager.
- [int32_t](#) [NumParameters](#) () const
Returns the number of parameters in this instance of the parameter manager.
- void [RemoveParameterByID](#) ([AAX_CParamID](#) identifier)
Removes a parameter from the manager.
- void [RemoveAllParameters](#) ()
Removes all parameters from the manager.
- [AAX_IParameter](#) * [GetParameterByID](#) ([AAX_CParamID](#) identifier)
Given a parameter ID, retrieves a reference to the requested parameter.
- const [AAX_IParameter](#) * [GetParameterByID](#) ([AAX_CParamID](#) identifier) const
Given a parameter ID, retrieves a const reference to the requested parameter.
- [AAX_IParameter](#) * [GetParameterByName](#) (const char *name)
Given a parameter name, retrieves a reference to the requested parameter.
- const [AAX_IParameter](#) * [GetParameterByName](#) (const char *name) const
Given a parameter name, retrieves a const reference to the requested parameter.
- [AAX_IParameter](#) * [GetParameter](#) ([int32_t](#) index)
Given a parameter index, retrieves a reference to the requested parameter.
- const [AAX_IParameter](#) * [GetParameter](#) ([int32_t](#) index) const
Given a parameter index, retrieves a const reference to the requested parameter.
- [int32_t](#) [GetParameterIndex](#) ([AAX_CParamID](#) identifier) const
- void [AddParameter](#) ([AAX_IParameter](#) *param)
- void [RemoveParameter](#) ([AAX_IParameter](#) *param)

Protected Attributes

- [AAX_IAutomationDelegate](#) * [mAutomationDelegate](#)
- [std::vector](#)< [AAX_IParameter](#) * > [mParameters](#)
- [std::map](#)< [std::string](#), [AAX_IParameter](#) * > [mParametersMap](#)

14.29.2 Constructor & Destructor Documentation

14.29.2.1 AAX_CParameterManager()

```
AAX_CParameterManager::AAX_CParameterManager ( )
```

14.29.2.2 ~AAX_CParameterManager()

```
AAX_CParameterManager::~~AAX_CParameterManager ( )
```

14.29.3 Member Function Documentation

14.29.3.1 Initialize()

```
void AAX_CParameterManager::Initialize (
    AAX_IAutomationDelegate * iAutomationDelegateUnknown )
```

Initialize the parameter manager.

Called when plug-in instance is first instantiated. This method will initialize the plug-in's automation delegate, among other set-up tasks.

Parameters

in	<i>iAutomationDelegateUnknown</i>	A reference to the plug-in's AAX_IAutomationDelegate interface
----	-----------------------------------	--

14.29.3.2 NumParameters()

```
int32_t AAX_CParameterManager::NumParameters ( ) const
```

Returns the number of parameters in this instance of the parameter manager.

14.29.3.3 RemoveParameterByID()

```
void AAX_CParameterManager::RemoveParameterByID (
    AAX_CParamID identifier )
```

Removes a parameter from the manager.

Todo Should this method return success/failure code?

Parameters

in	<i>identifier</i>	ID of the parameter that will be removed
----	-------------------	--

14.29.3.4 RemoveAllParameters()

```
void AAX_CParameterManager::RemoveAllParameters ( )
```

Removes all parameters from the manager.

Todo Should this method return success/failure code?

14.29.3.5 GetParameterByID() [1/2]

```
AAX_IParameter* AAX_CParameterManager::GetParameterByID (
    AAX_CParamID identifier )
```

Given a parameter ID, retrieves a reference to the requested parameter.

Parameters

in	<i>identifier</i>	ID of the parameter that will be retrieved
----	-------------------	--

Referenced by AAX_CMonolithicParameters::UpdateParameterNormalizedValue().

Here is the caller graph for this function:

14.29.3.6 GetParameterByID() [2/2]

```
const AAX_IParameter* AAX_CParameterManager::GetParameterByID (
    AAX_CParamID identifier ) const
```

Given a parameter ID, retrieves a const reference to the requested parameter.

Parameters

in	<i>identifier</i>	ID of the parameter that will be retrieved
----	-------------------	--

14.29.3.7 GetParameterByName() [1/2]

```
AAX_IParameter* AAX_CParameterManager::GetParameterByName (
    const char * name )
```

Given a parameter name, retrieves a reference to the requested parameter.

Note

Parameter names may be ambiguous

Parameters

in	<i>name</i>	Name of the parameter that will be retrieved
----	-------------	--

14.29.3.8 GetParameterByName() [2/2]

```
const AAX_IParameter* AAX_CParameterManager::GetParameterByName (
    const char * name ) const
```

Given a parameter name, retrieves a const reference to the requested parameter.

Note

Parameter names may be ambiguous

Parameters

in	<i>name</i>	ID of the parameter that will be retrieved
----	-------------	--

14.29.3.9 GetParameter() [1/2]

```
AAX_IParameter* AAX_CParameterManager::GetParameter (
    int32_t index )
```

Given a parameter index, retrieves a reference to the requested parameter.

Parameter indices are incremented in the order that parameters are added to the manager. See [AddParameter\(\)](#).

Parameters

in	<i>index</i>	Index of the parameter that will be retrieved
----	--------------	---

14.29.3.10 GetParameter() [2/2]

```
const AAX_IParameter* AAX_CParameterManager::GetParameter (
    int32_t index ) const
```

Given a parameter index, retrieves a const reference to the requested parameter.

Parameter indices are incremented in the order that parameters are added to the manager. See [AddParameter\(\)](#).

Parameters

in	<i>index</i>	Index of the parameter that will be retrieved
----	--------------	---

14.29.3.11 GetParameterIndex()

```
int32_t AAX_CParameterManager::GetParameterIndex (
    AAX_CParamID identifier ) const
```

Given a parameter ID, retrieves the index for the specified parameter

Parameters

in	<i>identifier</i>	ID of the parameter that will be retrieved
----	-------------------	--

14.29.3.12 AddParameter()

```
void AAX_CParameterManager::AddParameter (
    AAX_IParameter * param )
```

Adds a parameter to the manager

Todo Should this method return success/failure code?

Parameters

in	<i>param</i>	Reference to the parameter that will be added
----	--------------	---

14.29.3.13 RemoveParameter()

```
void AAX_CParameterManager::RemoveParameter (
    AAX_IParameter * param )
```

Removes a parameter to the manager

Todo Should this method return success/failure code?

Parameters

in	<i>param</i>	Reference to the parameter that will be removed
----	--------------	---

14.29.4 Member Data Documentation

14.29.4.1 mAutomationDelegate

[AAX_IAutomationDelegate](#)* AAX_CParameterManager::mAutomationDelegate [protected]

14.29.4.2 mParameters

std::vector<[AAX_IParameter](#)*> AAX_CParameterManager::mParameters [protected]

14.29.4.3 mParametersMap

std::map<std::string, [AAX_IParameter](#)*> AAX_CParameterManager::mParametersMap [protected]

The documentation for this class was generated from the following file:

- [AAX_CParameterManager.h](#)

14.30 AAX_CParameterValue< T > Class Template Reference

```
#include <AAX_CParameter.h>
```

Inheritance diagram for AAX_CParameterValue< T >:

Collaboration diagram for AAX_CParameterValue< T >:

14.30.1 Description

```
template<typename T>
class AAX_CParameterValue< T >
```

Concrete implementation of [AAX_IParameterValue](#).

Used by [AAX_CParameter](#)

Public Types

- enum [Defaults](#) {
[eParameterDefaultMaxIdentifierSize](#) = kAAX_ParameterIdentifierMaxSize ,
[eParameterDefaultMaxIdentifierLength](#) = eParameterDefaultMaxIdentifierSize - 1 }

Public Member Functions

- [AAX_DEFAULT_DTOR_OVERRIDE](#) ([AAX_CParameterValue](#))
- [AAX_DEFAULT_MOVE_CTOR](#) ([AAX_CParameterValue](#))
- [AAX_DEFAULT_MOVE_OPER](#) ([AAX_CParameterValue](#))
- [AAX_DELETE](#) ([AAX_CParameterValue](#) &operator=(const [AAX_CParameterValue](#) &))
- [AAX_CParameterValue](#) ([AAX_CParamID](#) identifier)
Constructs an [AAX_CParameterValue](#) object.
- [AAX_CParameterValue](#) ([AAX_CParamID](#) identifier, const T &value)
Constructs an [AAX_CParameterValue](#) object with a defined initial state.
- [AAX_CParameterValue](#) (const [AAX_CParameterValue](#)< T > &other)
Copy constructor for [AAX_CParameterValue](#).
- const T & [Get](#) () const
Direct access to the template instance's value.
- void [Set](#) (const T &inValue)
Direct access to the template instance's value.
- [AAX_IParameterValue](#) * [Clone](#) () const [AAX_OVERRIDE](#)
Clones the parameter object.
- [AAX_CParamID](#) [Identifier](#) () const [AAX_OVERRIDE](#)
Returns the parameter's unique identifier.
- bool [GetValueAsBool](#) (bool *value) const
Retrieves the parameter's value as a bool.
- bool [GetValueAsInt32](#) (int32_t *value) const
Retrieves the parameter's value as an int32_t.
- bool [GetValueAsFloat](#) (float *value) const
Retrieves the parameter's value as a float.
- bool [GetValueAsDouble](#) (double *value) const
Retrieves the parameter's value as a double.
- bool [GetValueAsString](#) ([AAX_IString](#) *value) const
Retrieves the parameter's value as a string.

Typed accessors

- bool [GetValueAsBool](#) (bool *value) const [AAX_OVERRIDE](#)
Retrieves the parameter's value as a bool.
- bool [GetValueAsInt32](#) (int32_t *value) const [AAX_OVERRIDE](#)
Retrieves the parameter's value as an int32_t.
- bool [GetValueAsFloat](#) (float *value) const [AAX_OVERRIDE](#)
Retrieves the parameter's value as a float.
- bool [GetValueAsDouble](#) (double *value) const [AAX_OVERRIDE](#)
Retrieves the parameter's value as a double.
- bool [GetValueAsString](#) ([AAX_IString](#) *value) const [AAX_OVERRIDE](#)
Retrieves the parameter's value as a string.

14.30.2 Member Enumeration Documentation

14.30.2.1 Defaults

```
template<typename T >
enum AAX_CParameterValue::Defaults
```

Enumerator

eParameterDefaultMaxIdentifierSize	
eParameterDefaultMaxIdentifierLength	

14.30.3 Constructor & Destructor Documentation

14.30.3.1 AAX_CParameterValue() [1/3]

```
template<typename T >
AAX_CParameterValue< T >::AAX_CParameterValue (
    AAX_CParamID identifier ) [explicit]
```

Constructs an [AAX_CParameterValue](#) object.

Parameters

in	<i>identifier</i>	Unique ID for the parameter value, these can only be 31 characters long at most. (the fixed length is a requirement for some optimizations in the host)
----	-------------------	---

Note

The initial state of the parameter value is undefined

14.30.3.2 AAX_CParameterValue() [2/3]

```
template<typename T >
AAX_CParameterValue< T >::AAX_CParameterValue (
    AAX_CParamID identifier,
    const T & value ) [explicit]
```

Constructs an [AAX_CParameterValue](#) object with a defined initial state.

Parameters

in	<i>identifier</i>	Unique ID for the parameter value, these can only be 31 characters long at most. (the fixed length is a requirement for some optimizations in the host)
in	<i>value</i>	Initial state of the parameter value

14.30.3.3 AAX_CParameterValue() [3/3]

```
template<typename T >
AAX_CParameterValue< T >::AAX_CParameterValue (
    const AAX_CParameterValue< T > & other ) [explicit]
```

Copy constructor for [AAX_CParameterValue](#).

14.30.4 Member Function Documentation**14.30.4.1 AAX_DEFAULT_DTOR_OVERRIDE()**

```
template<typename T >
AAX_CParameterValue< T >::AAX_DEFAULT_DTOR_OVERRIDE (
    AAX_CParameterValue< T > )
```

14.30.4.2 AAX_DEFAULT_MOVE_CTOR()

```
template<typename T >
AAX_CParameterValue< T >::AAX_DEFAULT_MOVE_CTOR (
    AAX_CParameterValue< T > )
```

14.30.4.3 AAX_DEFAULT_MOVE_OPER()

```
template<typename T >
AAX_CParameterValue< T >::AAX_DEFAULT_MOVE_OPER (
    AAX_CParameterValue< T > )
```

14.30.4.4 AAX_DELETE()

```
template<typename T >
AAX_CParameterValue< T >::AAX_DELETE (
    AAX_CParameterValue< T > & operator = (const AAX_CParameterValue< T > &) )
```

14.30.4.5 Get()

```
template<typename T >
const T& AAX_CParameterValue< T >::Get ( ) const [inline]
```

Direct access to the template instance's value.

14.30.4.6 Set()

```
template<typename T >
void AAX_CParameterValue< T >::Set (
    const T & inValue ) [inline]
```

Direct access to the template instance's value.

14.30.4.7 Clone()

```
template<typename T >
AAX_IParаметerValue* AAX_CParameterValue< T >::Clone ( ) const [inline], [virtual]
```

Clones the parameter object.

Note

Does NOT set the automation delegate on the clone; ownership of the automation delegate and parameter registration/unregistration stays with the original parameter

Implements [AAX_IParаметerValue](#).

14.30.4.8 Identifier()

```
template<typename T >
AAX_CParamID AAX_CParameterValue< T >::Identifier ( ) const [inline], [virtual]
```

Returns the parameter's unique identifier.

This unique ID is used by the [Parameter Manager](#) and by outside applications to uniquely identify and target control messages. This value may not be changed after the parameter has been constructed.

Implements [AAX_IParаметerValue](#).

14.30.4.9 GetValueAsBool() [1/2]

```
template<typename T >
bool AAX_CParameterValue< T >::GetValueAsBool (
    bool * value ) const [virtual]
```

Retrieves the parameter's value as a bool.

Parameters

out	value	The parameter's real value. Set only if conversion is successful.
-----	-------	---

Return values

true	The conversion to bool was successful
false	The conversion to bool was unsuccessful

Implements [AAX_IParаметerValue](#).

14.30.4.10 GetValueAsInt32() [1/2]

```
template<typename T >
bool AAX_CParameterValue< T >::GetValueAsInt32 (
    int32_t * value ) const [virtual]
```

Retrieves the parameter's value as an int32_t.

Parameters

out	value	The parameter's real value. Set only if conversion is successful.
-----	-------	---

Return values

true	The conversion to int32_t was successful
false	The conversion to int32_t was unsuccessful

Implements [AAX_IParаметerValue](#).

14.30.4.11 GetValueAsFloat() [1/2]

```
template<typename T >
bool AAX_CParameterValue< T >::GetValueAsFloat (
    float * value ) const [virtual]
```

Retrieves the parameter's value as a float.

Parameters

out	value	The parameter's real value. Set only if conversion is successful.
-----	-------	---

Return values

<i>true</i>	The conversion to float was successful
<i>false</i>	The conversion to float was unsuccessful

Implements [AAX_IParameterValue](#).

14.30.4.12 GetValueAsDouble() [1/2]

```
template<typename T >
bool AAX_CParameterValue< T >::GetValueAsDouble (
    double * value ) const [virtual]
```

Retrieves the parameter's value as a double.

Parameters

out	<i>value</i>	The parameter's real value. Set only if conversion is successful.
-----	--------------	---

Return values

<i>true</i>	The conversion to double was successful
<i>false</i>	The conversion to double was unsuccessful

Implements [AAX_IParameterValue](#).

14.30.4.13 GetValueAsString() [1/2]

```
template<typename T >
bool AAX_CParameterValue< T >::GetValueAsString (
    AAX_IString * value ) const [virtual]
```

Retrieves the parameter's value as a string.

Parameters

out	<i>value</i>	The parameter's real value. Set only if conversion is successful.
-----	--------------	---

Return values

<i>true</i>	The conversion to string was successful
<i>false</i>	The conversion to string was unsuccessful

Implements [AAX_IParameterValue](#).

14.30.4.14 GetValueAsBool() [2/2]

```
bool AAX_CParameterValue< bool >::GetValueAsBool (
    bool * value ) const [virtual]
```

Retrieves the parameter's value as a bool.

Parameters

out	value	The parameter's real value. Set only if conversion is successful.
-----	-------	---

Return values

true	The conversion to bool was successful
false	The conversion to bool was unsuccessful

Implements [AAX_IParame](#)terValue.

14.30.4.15 GetValueAsInt32() [2/2]

```
bool AAX_CParameterValue< int32_t >::GetValueAsInt32 (
    int32_t * value ) const [virtual]
```

Retrieves the parameter's value as an int32_t.

Parameters

out	value	The parameter's real value. Set only if conversion is successful.
-----	-------	---

Return values

true	The conversion to int32_t was successful
false	The conversion to int32_t was unsuccessful

Implements [AAX_IParame](#)terValue.

14.30.4.16 GetValueAsFloat() [2/2]

```
bool AAX_CParameterValue< float >::GetValueAsFloat (
    float * value ) const [virtual]
```

Retrieves the parameter's value as a float.

Parameters

out	value	The parameter's real value. Set only if conversion is successful.
-----	-------	---

Return values

true	The conversion to float was successful
false	The conversion to float was unsuccessful

Implements [AAX_IParameterValue](#).

14.30.4.17 GetValueAsDouble() [2/2]

```
bool AAX_CParameterValue< double >::GetValueAsDouble (
    double * value ) const [virtual]
```

Retrieves the parameter's value as a double.

Parameters

out	value	The parameter's real value. Set only if conversion is successful.
-----	-------	---

Return values

true	The conversion to double was successful
false	The conversion to double was unsuccessful

Implements [AAX_IParameterValue](#).

14.30.4.18 GetValueAsString() [2/2]

```
bool AAX_CParameterValue< AAX_CString >::GetValueAsString (
    AAX_IString * value ) const [virtual]
```

Retrieves the parameter's value as a string.

Parameters

out	value	The parameter's real value. Set only if conversion is successful.
-----	-------	---

Return values

true	The conversion to string was successful
false	The conversion to string was unsuccessful

Implements [AAX_IParameterValue](#).

The documentation for this class was generated from the following file:

- [AAX_CParameter.h](#)

14.31 AAX_CPercentDisplayDelegateDecorator< T > Class Template Reference

```
#include <AAX_CPercentDisplayDelegateDecorator.h>
```

Inheritance diagram for AAX_CPercentDisplayDelegateDecorator< T >:

Collaboration diagram for AAX_CPercentDisplayDelegateDecorator< T >:

14.31.1 Description

```
template<typename T>
class AAX_CPercentDisplayDelegateDecorator< T >
```

A percent decorator conforming to [AAX_IDisplayDelegateDecorator](#).

This class is an [AAX_IDisplayDelegateDecorator](#), meaning that it acts as a wrapper for other display delegates or concrete display types. For more information about display delegate decorators in [AAX](#), see [Display delegate decorators](#)

The behavior of this class is to provide string conversion to and from percentage (%) values. When converting a parameter value to a string, it takes the real value and performs a % conversion before passing the value on to a concrete implementation to get a value string. It then adds on the "%" string at the end to signify that the value was converted. This allows something like a gain value to remain internally linear at all times even though its display is converted to a percentage.

The inverse operation is also supported; this class can convert a percentage-formatted string into its associated real value. The string will first be converted to a number, then that number will have the inverse % calculation applied to it to retrieve the parameter's actual value.

Public Member Functions

- [AAX_CPercentDisplayDelegateDecorator](#) (const [AAX_IDisplayDelegate](#)< T > &displayDelegate)
- [AAX_CPercentDisplayDelegateDecorator](#)< T > * [Clone](#) () const [AAX_OVERRIDE](#)
Constructs and returns a copy of the display delegate.
- bool [ValueToString](#) (T value, [AAX_CString](#) *valueString) const [AAX_OVERRIDE](#)
Converts a real parameter value to a string representation.
- bool [ValueToString](#) (T value, int32_t maxNumChars, [AAX_CString](#) *valueString) const [AAX_OVERRIDE](#)
Converts a real parameter value to a string representation using a size hint, useful for control surfaces and other character limited displays.
- bool [StringToValue](#) (const [AAX_CString](#) &valueString, T *value) const [AAX_OVERRIDE](#)
Converts a string to a real parameter value.

14.31.2 Constructor & Destructor Documentation

14.31.2.1 AAX_CPercentDisplayDelegateDecorator()

```
template<typename T >
AAX_CPercentDisplayDelegateDecorator< T >::AAX_CPercentDisplayDelegateDecorator (
    const AAX_IDisplayDelegate< T > & displayDelegate )
```

14.31.3 Member Function Documentation

14.31.3.1 Clone()

```
template<typename T >
AAX_CPercentDisplayDelegateDecorator< T > * AAX_CPercentDisplayDelegateDecorator< T >::Clone
( ) const [virtual]
```

Constructs and returns a copy of the display delegate.

In general, this method's implementation can use a simple copy constructor:

```
template <typename T>
AAX_CSubclassDisplayDelegate<T>*    AAX_CSubclassDisplayDelegate<T>::Clone() const
{
    return new AAX_CSubclassDisplayDelegate(*this);
}
```

Implements [AAX_IDisplayDelegate< T >](#).

14.31.3.2 ValueToString() [1/2]

```
template<typename T >
bool AAX_CPercentDisplayDelegateDecorator< T >::ValueToString (
    T value,
    AAX_CString * valueString ) const [virtual]
```

Converts a real parameter value to a string representation.

Parameters

in	<i>value</i>	The real parameter value that will be converted
out	<i>valueString</i>	A string corresponding to value

Return values

<i>true</i>	The string conversion was successful
-------------	--------------------------------------

Return values

<i>false</i>	The string conversion was unsuccessful
--------------	--

Implements [AAX_IDisplayDelegate< T >](#).

References [AAX_IDisplayDelegateDecorator< T >::ValueToString\(\)](#).

Here is the call graph for this function:

14.31.3.3 ValueToString() [2/2]

```
template<typename T >
bool AAX_CPercentDisplayDelegateDecorator< T >::ValueToString (
    T value,
    int32_t maxNumChars,
    AAX_CString * valueString ) const [virtual]
```

Converts a real parameter value to a string representation using a size hint, useful for control surfaces and other character limited displays.

Parameters

in	<i>value</i>	The real parameter value that will be converted
in	<i>maxNumChars</i>	Size hint for the desired maximum number of characters in the string (not including null termination)
out	<i>valueString</i>	A string corresponding to value

Return values

<i>true</i>	The string conversion was successful
<i>false</i>	The string conversion was unsuccessful

Implements [AAX_IDisplayDelegate< T >](#).

References [AAX_IDisplayDelegateDecorator< T >::ValueToString\(\)](#).

Here is the call graph for this function:

14.31.3.4 StringToValue()

```
template<typename T >
bool AAX_CPercentDisplayDelegateDecorator< T >::StringToValue (
    const AAX_CString & valueString,
    T * value ) const [virtual]
```

Converts a string to a real parameter value.

Parameters

in	<i>valueString</i>	The string that will be converted
out	<i>value</i>	The real parameter value corresponding to valueString

Return values

<i>true</i>	The string conversion was successful
<i>false</i>	The string conversion was unsuccessful

Implements [AAX_IDisplayDelegate< T >](#).

References [AAX_CString::Length\(\)](#), [AAX_IDisplayDelegateDecorator< T >::StringToValue\(\)](#), and [AAX_CString::SubString\(\)](#).

Here is the call graph for this function:

The documentation for this class was generated from the following file:

- [AAX_CPercentDisplayDelegateDecorator.h](#)

14.32 AAX_CPieceWiseLinearTaperDelegate< T, RealPrecision > Class Template Reference

```
#include <AAX_CPieceWiseLinearTaperDelegate.h>
```

Inheritance diagram for [AAX_CPieceWiseLinearTaperDelegate< T, RealPrecision >](#):

Collaboration diagram for [AAX_CPieceWiseLinearTaperDelegate< T, RealPrecision >](#):

14.32.1 Description

```
template<typename T, int32_t RealPrecision = 100>
class AAX_CPieceWiseLinearTaperDelegate< T, RealPrecision >
```

A piece-wise linear taper conforming to [AAX_ITaperDelegate](#).

This taper spaces a parameter's real values in a piecewise linear fashion.

RealPrecision

In addition to its type templatization, this taper includes a precision template parameter. `RealPrecision` is a multiplier that works in conjunction with the `round()` function to limit the precision of the real values provided by this taper. For example, if `RealPrecision` is 1000, it will round to the closest 0.001 when doing any sort of value conversion. If `RealPrecision` is 1, it will round to the nearest integer. If `RealPrecision` is 1000000, it will round to the nearest 0.000001. This is particularly useful for preventing things like 1.9999999 truncating down to 1 instead of rounding up to 2.

To accomplish this behavior, the taper multiplies its unrounded parameter values by `RealPrecision`, rounds the result to the nearest valid value, then divides `RealPrecision` back out.

Rounding will be disabled if `RealPrecision` is set to a value less than 1

Public Member Functions

- [AAX_CPieceWiseLinearTaperDelegate](#) (const double *normalizedValues, const T *realValues, int32_t numValues)
Constructs a Piece-wise Linear Taper with paired normalized and real values.
- [AAX_CPieceWiseLinearTaperDelegate](#) (const [AAX_CPieceWiseLinearTaperDelegate](#) &other)
- [~AAX_CPieceWiseLinearTaperDelegate](#) ()
- [AAX_CPieceWiseLinearTaperDelegate](#)< T, RealPrecision > * [Clone](#) () const [AAX_OVERRIDE](#)
Constructs and returns a copy of the taper delegate.
- T [GetMinimumValue](#) () const [AAX_OVERRIDE](#)
Returns the taper's minimum real value.
- T [GetMaximumValue](#) () const [AAX_OVERRIDE](#)
Returns the taper's maximum real value.
- T [ConstrainRealValue](#) (T value) const [AAX_OVERRIDE](#)
Applies a constraint to the value and returns the constrained value.
- T [NormalizedToReal](#) (double normalizedValue) const [AAX_OVERRIDE](#)
Converts a normalized value to a real value.
- double [RealToNormalized](#) (T realValue) const [AAX_OVERRIDE](#)
Normalizes a real parameter value.

Protected Member Functions

- T [Round](#) (double iValue) const

14.32.2 Constructor & Destructor Documentation

14.32.2.1 AAX_CPieceWiseLinearTaperDelegate() [1/2]

```
template<typename T , int32_t RealPrecision>
AAX_CPieceWiseLinearTaperDelegate< T, RealPrecision >::AAX_CPieceWiseLinearTaperDelegate (
    const double * normalizedValues,
    const T * realValues,
    int32_t numValues )
```

Constructs a Piece-wise Linear Taper with paired normalized and real values.

Note

The parameter's default value should lie within the min to max range.

Parameters

in	<i>normalizedValues</i>	is an array of the normalized values in sorted order. (make sure to include the full normalized range, 0.0-1.0 inclusive)
in	<i>realValues</i>	is an array of the corresponding real values to the normalized values passed in.
in	<i>numValues</i>	is the number of values that have been passed in (i.e. the element length of the other input arrays)

14.32.2.2 AAX_CPieceWiseLinearTaperDelegate() [2/2]

```
template<typename T , int32_t RealPrecision>
AAX_CPieceWiseLinearTaperDelegate< T, RealPrecision >::AAX_CPieceWiseLinearTaperDelegate (
    const AAX_CPieceWiseLinearTaperDelegate< T, RealPrecision > & other )
```

14.32.2.3 ~AAX_CPieceWiseLinearTaperDelegate()

```
template<typename T , int32_t RealPrecision>
AAX_CPieceWiseLinearTaperDelegate< T, RealPrecision >::~~AAX_CPieceWiseLinearTaperDelegate
```

14.32.3 Member Function Documentation

14.32.3.1 Clone()

```
template<typename T , int32_t RealPrecision>
AAX_CPieceWiseLinearTaperDelegate< T, RealPrecision > * AAX_CPieceWiseLinearTaperDelegate< T,
RealPrecision >::Clone ( ) const [virtual]
```

Constructs and returns a copy of the taper delegate.

In general, this method's implementation can use a simple copy constructor:

```
template <typename T>
AAX_CSubclassTaperDelegate<T>* AAX_CSubclassTaperDelegate<T>::Clone() const
{
    return new AAX_CSubclassTaperDelegate(*this);
}
```

Implements [AAX_ITaperDelegate< T >](#).

14.32.3.2 GetMinimumValue()

```
template<typename T , int32_t RealPrecision = 100>
T AAX_CPieceWiseLinearTaperDelegate< T, RealPrecision >::GetMinimumValue ( ) const [inline],
[virtual]
```

Returns the taper's minimum real value.

Implements [AAX_ITaperDelegate< T >](#).

14.32.3.3 GetMaximumValue()

```
template<typename T , int32_t RealPrecision = 100>
T AAX_CPieceWiseLinearTaperDelegate< T, RealPrecision >::GetMaximumValue ( ) const [inline],
[virtual]
```

Returns the taper's maximum real value.

Implements [AAX_ITaperDelegate< T >](#).

14.32.3.4 ConstrainRealValue()

```
template<typename T , int32_t RealPrecision>
T AAX_CPieceWiseLinearTaperDelegate< T, RealPrecision >::ConstrainRealValue (
    T value ) const [virtual]
```

Applies a constraint to the value and returns the constrained value.

This method is useful if the taper requires a constraint beyond simple minimum and maximum real value limits.

Note

This is the function that should actually enforces the constraints in [NormalizeToReal\(\)](#) and [RealToNormalized\(\)](#).

Parameters

in	<i>value</i>	The unconstrained value
----	--------------	-------------------------

Implements [AAX_ITaperDelegate< T >](#).

14.32.3.5 NormalizedToReal()

```
template<typename T , int32_t RealPrecision>
T AAX_CPieceWiseLinearTaperDelegate< T, RealPrecision >::NormalizedToReal (
    double normalizedValue ) const [virtual]
```

Converts a normalized value to a real value.

This is where the actual taper algorithm is implemented.

This function should perform the exact inverse of [RealToNormalized\(\)](#), to within the roundoff precision of the individual taper implementation.

Parameters

in	<i>normalizedValue</i>	The normalized value that will be converted
----	------------------------	---

Implements [AAX_ITaperDelegate< T >](#).

14.32.3.6 RealToNormalized()

```
template<typename T , int32_t RealPrecision>
double AAX_CPieceWiseLinearTaperDelegate< T, RealPrecision >::RealToNormalized (
    T realValue ) const [virtual]
```

Normalizes a real parameter value.

This is where the actual taper algorithm is implemented.

This function should perform the exact inverse of [NormalizedToReal\(\)](#), to within the roundoff precision of the individual taper implementation.

Parameters

in	<i>realValue</i>	The real parameter value that will be normalized
----	------------------	--

Implements [AAX_ITaperDelegate< T >](#).

14.32.3.7 Round()

```
template<typename T , int32_t RealPrecision>
T AAX_CPieceWiseLinearTaperDelegate< T, RealPrecision >::Round (
    double iValue ) const [protected]
```

The documentation for this class was generated from the following file:

- [AAX_CPieceWiseLinearTaperDelegate.h](#)

14.33 AAX_CRangeTaperDelegate< T, RealPrecision > Class Template Reference

```
#include <AAX_CRangeTaperDelegate.h>
```

Inheritance diagram for [AAX_CRangeTaperDelegate< T, RealPrecision >](#):

Collaboration diagram for [AAX_CRangeTaperDelegate< T, RealPrecision >](#):

14.33.1 Description

```
template<typename T, int32_t RealPrecision = 1000>
class AAX_CRangeTaperDelegate< T, RealPrecision >
```

A piecewise-linear taper conforming to [AAX_ITaperDelegate](#).

This taper spaces a parameter's real values between its minimum and maximum using a series of linear regions to create the full mapping between the parameter's real and normalized values.

Here is an example of how this taper can be used:

```
float rangePoints[] = { 0.0, 1.0, 100.0, 1000.0, 2000.0 };
double rangeSteps[] = { 0.1, 1.0, 10.0, 25.0 }; // number of steps per range: 10, 99, 90, 40
const long cNumRanges = sizeof(rangeSteps)/sizeof(rangeSteps[0]);
long numSteps = 0;
for (int i = 0; i < cNumRanges; i++)
{
    numSteps += (rangePoints[i+1] - rangePoints[i]) / rangeSteps[i];
}
AAX_CRangeTaperDelegate<float> nonLinearTaper(rangePoints, rangeSteps, cNumRanges);
float controlValue = 1.5;
double normalized = nonLinearTaper.RealToNormalized(controlValue);
float real = nonLinearTaper.NormalizedToReal(normalized);
```

RealPrecision

In addition to its type templization, this taper includes a precision template parameter. RealPrecision is a multiplier that works in conjunction with the round() function to limit the precision of the real values provided by this taper. For example, if RealPrecision is 1000, it will round to the closest 0.001 when doing any sort of value conversion. If RealPrecision is 1, it will round to the nearest integer. If RealPrecision is 1000000, it will round to the nearest 0.000001. This is particularly useful for preventing things like 1.9999999 truncating down to 1 instead of rounding up to 2.

To accomplish this behavior, the taper multiplies its unrounded parameter values by RealPrecision, rounds the result to the nearest valid value, then divides RealPrecision back out.

Rounding will be disabled if RealPrecision is set to a value less than 1

Public Member Functions

- [AAX_CRangeTaperDelegate](#) (T *range, double *rangeSteps, unsigned long numRanges, bool useSmart← Rounding=true)
 - Constructs a Range Taper with specified minimum and maximum values.*
- [AAX_CRangeTaperDelegate](#) (const [AAX_CRangeTaperDelegate](#) &rhs)
- [AAX_CRangeTaperDelegate](#) & operator= ([AAX_CRangeTaperDelegate](#) &rhs)
- [AAX_CRangeTaperDelegate](#)< T, RealPrecision > * Clone () const [AAX_OVERRIDE](#)
 - Constructs and returns a copy of the taper delegate.*
- T [GetMinimumValue](#) () const [AAX_OVERRIDE](#)
 - Returns the taper's minimum real value.*
- T [GetMaximumValue](#) () const [AAX_OVERRIDE](#)
 - Returns the taper's maximum real value.*
- T [ConstrainRealValue](#) (T value) const [AAX_OVERRIDE](#)
 - Applies a constraint to the value and returns the constrained value.*
- T [NormalizedToReal](#) (double normalizedValue) const [AAX_OVERRIDE](#)
 - Converts a normalized value to a real value.*
- double [RealToNormalized](#) (T realValue) const [AAX_OVERRIDE](#)
 - Normalizes a real parameter value.*

Protected Member Functions

- T [Round](#) (double iValue) const
- T [SmartRound](#) (double value) const

14.33.2 Constructor & Destructor Documentation

14.33.2.1 AAX_CRangeTaperDelegate() [1/2]

```
template<typename T , int32_t RealPrecision>
AAX_CRangeTaperDelegate< T, RealPrecision >::AAX_CRangeTaperDelegate (
    T * range,
    double * rangesSteps,
    unsigned long numRanges,
    bool useSmartRounding = true )
```

Constructs a Range Taper with specified minimum and maximum values.

Note

The parameter's default value should lie within the min to max range.

Parameters

in	<i>range</i>	An array of range endpoints along the taper's mapping range
in	<i>rangesSteps</i>	Step values for each region in the taper's stepwise-linear map. No values in this array may be zero.
in	<i>numRanges</i>	The total number of linear regions in the taper's map
in	<i>useSmartRounding</i>	

Todo Document useSmartRounding parameter

14.33.2.2 AAX_CRangeTaperDelegate() [2/2]

```
template<typename T , int32_t RealPrecision>
AAX_CRangeTaperDelegate< T, RealPrecision >::AAX_CRangeTaperDelegate (
    const AAX_CRangeTaperDelegate< T, RealPrecision > & rhs )
```

14.33.3 Member Function Documentation

14.33.3.1 operator=()

```
template<typename T , int32_t RealPrecision>
AAX_CRangeTaperDelegate< T, RealPrecision > & AAX_CRangeTaperDelegate< T, RealPrecision >↔
::operator= (
    AAX_CRangeTaperDelegate< T, RealPrecision > & rhs )
```

14.33.3.2 Clone()

```
template<typename T , int32_t RealPrecision>
AAX_CRangeTaperDelegate< T, RealPrecision > * AAX_CRangeTaperDelegate< T, RealPrecision >↔
::Clone ( ) const [virtual]
```

Constructs and returns a copy of the taper delegate.

In general, this method's implementation can use a simple copy constructor:

```
template <typename T>
AAX_CSubclassTaperDelegate<T>* AAX_CSubclassTaperDelegate<T>::Clone() const
{
    return new AAX_CSubclassTaperDelegate(*this);
}
```

Implements [AAX_ITaperDelegate< T >](#).

14.33.3.3 GetMinimumValue()

```
template<typename T , int32_t RealPrecision = 1000>
T AAX_CRangeTaperDelegate< T, RealPrecision >::GetMinimumValue ( ) const [inline], [virtual]
```

Returns the taper's minimum real value.

Implements [AAX_ITaperDelegate< T >](#).

14.33.3.4 GetMaximumValue()

```
template<typename T , int32_t RealPrecision = 1000>
T AAX_CRangeTaperDelegate< T, RealPrecision >::GetMaximumValue ( ) const [inline], [virtual]
```

Returns the taper's maximum real value.

Implements [AAX_ITaperDelegate< T >](#).

14.33.3.5 ConstrainRealValue()

```
template<typename T , int32_t RealPrecision>
T AAX_CRangeTaperDelegate< T, RealPrecision >::ConstrainRealValue (
    T value ) const [virtual]
```

Applies a constraint to the value and returns the constrained value.

This method is useful if the taper requires a constraint beyond simple minimum and maximum real value limits.

Note

This is the function that should actually enforces the constraints in [NormalizeToReal\(\)](#) and [RealToNormalized\(\)](#).

Parameters

in	<i>value</i>	The unconstrained value
----	--------------	-------------------------

Implements [AAX_ITaperDelegate< T >](#).

14.33.3.6 NormalizedToReal()

```
template<typename T , int32_t RealPrecision>
T AAX\_CRangeTaperDelegate< T, RealPrecision >::NormalizedToReal (
    double normalizedValue ) const [virtual]
```

Converts a normalized value to a real value.

This is where the actual taper algorithm is implemented.

This function should perform the exact inverse of [RealToNormalized\(\)](#), to within the roundoff precision of the individual taper implementation.

Parameters

in	<i>normalizedValue</i>	The normalized value that will be converted
----	------------------------	---

Implements [AAX_ITaperDelegate< T >](#).

14.33.3.7 RealToNormalized()

```
template<typename T , int32_t RealPrecision>
double AAX\_CRangeTaperDelegate< T, RealPrecision >::RealToNormalized (
    T realValue ) const [virtual]
```

Normalizes a real parameter value.

This is where the actual taper algorithm is implemented.

This function should perform the exact inverse of [NormalizedToReal\(\)](#), to within the roundoff precision of the individual taper implementation.

Parameters

in	<i>realValue</i>	The real parameter value that will be normalized
----	------------------	--

Implements [AAX_ITaperDelegate< T >](#).

14.33.3.8 Round()

```
template<typename T , int32_t RealPrecision>
T AAX_CRangeTaperDelegate< T, RealPrecision >::Round (
    double iValue ) const [protected]
```

14.33.3.9 SmartRound()

```
template<typename T , int32_t RealPrecision>
T AAX_CRangeTaperDelegate< T, RealPrecision >::SmartRound (
    double value ) const [protected]
```

Todo Document

The documentation for this class was generated from the following file:

- [AAX_CRangeTaperDelegate.h](#)

14.34 AAX_CSessionDocumentClient Class Reference

```
#include <AAX_CSessionDocumentClient.h>
```

Inheritance diagram for AAX_CSessionDocumentClient:

Collaboration diagram for AAX_CSessionDocumentClient:

14.34.1 Description

Default implementation of the [AAX_ISessionDocumentClient](#) interface.

Public Member Functions

- [AAX_CSessionDocumentClient](#) (void)
- [~AAX_CSessionDocumentClient](#) (void) [AAX_OVERRIDE](#)

Initialization and uninitialization

- [AAX_Result Initialize](#) ([IACFUnknown](#) *iUnknown) [AAX_OVERRIDE](#)
- [AAX_Result Uninitialize](#) (void) [AAX_OVERRIDE](#)

Session document access

- [AAX_Result SetSessionDocument](#) ([IACFUnknown](#) *iSessionDocument) [AAX_OVERRIDE](#)
Sets or removes a session document.

AAX host and plug-in event notification

- [AAX_Result NotificationReceived](#) ([AAX_CTypeID](#) inNotificationType, const void *inNotificationData, uint32_t inNotificationDataSize) [AAX_OVERRIDE](#)
Notification Hook.

Protected Member Functions

Session document change notifications

- virtual [AAX_Result SessionDocumentWillChange](#) ()
The session document interface is about to be added, replaced, or removed.
- virtual [AAX_Result SessionDocumentChanged](#) ()
The session document interface has been added, replaced, or removed.

Private member accessors

- [AAX_IController](#) * [GetController](#) (void)
Retrieves a reference to the plug-in's controller interface.
- const [AAX_IController](#) * [GetController](#) (void) const
Retrieves a reference to the plug-in's controller interface.
- [AAX_IEffectParameters](#) * [GetEffectParameters](#) (void)
Retrieves a reference to the plug-in's data model interface.
- const [AAX_IEffectParameters](#) * [GetEffectParameters](#) (void) const
Retrieves a reference to the plug-in's data model interface.
- std::shared_ptr< [AAX_ISessionDocument](#) > [GetSessionDocument](#) (void)
Retrieves a reference to the session document interface.
- std::shared_ptr< const [AAX_ISessionDocument](#) > [GetSessionDocument](#) (void) const
Retrieves a reference to the session document interface.

Additional Inherited Members

14.34.2 Constructor & Destructor Documentation

14.34.2.1 AAX_CSessionDocumentClient()

```
AAX_CSessionDocumentClient::AAX_CSessionDocumentClient (
    void )
```

14.34.2.2 ~AAX_CSessionDocumentClient()

```
AAX_CSessionDocumentClient::~~AAX_CSessionDocumentClient (
    void )
```

14.34.3 Member Function Documentation

14.34.3.1 Initialize()

```
AAX_Result AAX_CSessionDocumentClient::Initialize (
    IACFUnknown * iUnknown ) [virtual]
```

Implements [AAX_IACFSessionDocumentClient](#).

14.34.3.2 Uninitialize()

```
AAX_Result AAX_CSessionDocumentClient::Uninitialize (
    void ) [virtual]
```

Implements [AAX_IACFSessionDocumentClient](#).

14.34.3.3 SetSessionDocument()

```
AAX_Result AAX_CSessionDocumentClient::SetSessionDocument (
    IACFUnknown * iSessionDocument ) [virtual]
```

Sets or removes a session document.

Parameters

in	<i>iSessionDocument</i>	Interface supporting at least AAX_IACFSessionDocument , or <code>nullptr</code> to indicate that any session document that is currently held should be released.
----	-------------------------	--

Implements [AAX_IACFSessionDocumentClient](#).

14.34.3.4 NotificationReceived()

```
AAX_Result AAX_CSessionDocumentClient::NotificationReceived (
    AAX_CTypeID inNotificationType,
    const void * inNotificationData,
    uint32_t inNotificationDataSize ) [inline], [virtual]
```

Notification Hook.

Called from the host to deliver notifications to this object.

Look at the [AAX_ENotificationEvent](#) enumeration to see a description of events you can listen for and the data they come with.

Note

- Different notifications are sent to different objects within a plug-in. If you are not seeing an expected notification, try checking the other plug-in objects' [NotificationReceived\(\)](#) methods.

Note

- the host may dispatch notifications synchronously or asynchronously, and calls to this method may occur concurrently on multiple threads.

A plug-in may also dispatch custom notifications using [AAX_IController::SendNotification\(\)](#). Custom notifications will be posted back to the plug-in's other objects which support a [NotificationReceived\(\)](#) method (e.g. the data model).

Parameters

in	<i>inNotificationType</i>	Type of notification being received. Notifications from the host are one of AAX_ENotificationEvent
in	<i>inNotificationData</i>	Block of incoming notification data
in	<i>inNotificationDataSize</i>	Size of <i>inNotificationData</i> , in bytes

Implements [AAX_IACFSessionDocumentClient](#).

References [AAX_SUCCESS](#).

14.34.3.5 SessionDocumentWillChange()

```
virtual AAX\_Result AAX_CSessionDocumentClient::SessionDocumentWillChange ( ) [inline], [protected],
[virtual]
```

The session document interface is about to be added, replaced, or removed.

Custom implementations should stop using the current session document interface, which is about to become invalid.

References [AAX_SUCCESS](#).

14.34.3.6 SessionDocumentChanged()

```
virtual AAX\_Result AAX_CSessionDocumentClient::SessionDocumentChanged ( ) [inline], [protected],
[virtual]
```

The session document interface has been added, replaced, or removed.

Custom implementations should update local references to the session document interface.

References [AAX_SUCCESS](#).

14.34.3.7 GetController() [1/2]

```
AAX_IController* AAX_CSessionDocumentClient::GetController (
    void ) [protected]
```

Retrieves a reference to the plug-in's controller interface.

14.34.3.8 GetController() [2/2]

```
const AAX_IController* AAX_CSessionDocumentClient::GetController (
    void ) const [protected]
```

Retrieves a reference to the plug-in's controller interface.

14.34.3.9 GetEffectParameters() [1/2]

```
AAX_IEffectParameters* AAX_CSessionDocumentClient::GetEffectParameters (
    void ) [protected]
```

Retrieves a reference to the plug-in's data model interface.

14.34.3.10 GetEffectParameters() [2/2]

```
const AAX_IEffectParameters* AAX_CSessionDocumentClient::GetEffectParameters (
    void ) const [protected]
```

Retrieves a reference to the plug-in's data model interface.

14.34.3.11 GetSessionDocument() [1/2]

```
std::shared_ptr<AAX_ISessionDocument> AAX_CSessionDocumentClient::GetSessionDocument (
    void ) [protected]
```

Retrieves a reference to the session document interface.

14.34.3.12 GetSessionDocument() [2/2]

```
std::shared_ptr<const AAX_ISessionDocument> AAX_CSessionDocumentClient::GetSessionDocument (
    void ) const [protected]
```

Retrieves a reference to the session document interface.

The documentation for this class was generated from the following file:

- [AAX_CSessionDocumentClient.h](#)

14.35 AAX_CStateDisplayDelegate< T > Class Template Reference

```
#include <AAX_CStateDisplayDelegate.h>
```

Inheritance diagram for AAX_CStateDisplayDelegate< T >:

Collaboration diagram for AAX_CStateDisplayDelegate< T >:

14.35.1 Description

```
template<typename T>
class AAX_CStateDisplayDelegate< T >
```

A generic display format conforming to [AAX_IDisplayDelegate](#).

This display delegate is similar to [AAX_CNumberDisplayDelegate](#), but does not include precision or spacing templizations.

Public Member Functions

- [AAX_CStateDisplayDelegate](#) (const char *iStateStrings[], T iMinState=0)
Constructor taking a vector of C strings.
- [AAX_CStateDisplayDelegate](#) (int32_t inNumStates, const char *iStateStrings[], T iMinState=0)
Constructor taking a vector of C strings.
- [AAX_CStateDisplayDelegate](#) (const std::vector< [AAX_IString](#) * > &iStateStrings, T iMinState=0)
Constructor taking a vector of [AAX_IString](#) objects.
- [AAX_CStateDisplayDelegate](#) (const [AAX_CStateDisplayDelegate](#) &other)
- [AAX_IDisplayDelegate](#)< T > * [Clone](#) () const [AAX_OVERRIDE](#)
Constructs and returns a copy of the display delegate.
- bool [ValueToString](#) (T value, [AAX_CString](#) *valueString) const [AAX_OVERRIDE](#)
Converts a real parameter value to a string representation.
- bool [ValueToString](#) (T value, int32_t maxNumChars, [AAX_CString](#) *valueString) const [AAX_OVERRIDE](#)
Converts a real parameter value to a string representation using a size hint, useful for control surfaces and other character limited displays.
- bool [StringToValue](#) (const [AAX_CString](#) &valueString, T *value) const [AAX_OVERRIDE](#)
Converts a string to a real parameter value.
- void [AddShortenedStrings](#) (const char *iStateStrings[], int iLength)
- bool [Compare](#) (const [AAX_CString](#) &valueString, const [AAX_CString](#) &stateString) const

14.35.2 Constructor & Destructor Documentation

14.35.2.1 AAX_CStateDisplayDelegate() [1/4]

```
template<typename T >
AAX_CStateDisplayDelegate< T >::AAX_CStateDisplayDelegate (
    const char * iStateStrings[],
    T iMinState = 0 ) [explicit]
```

Constructor taking a vector of C strings.

Each state name will be copied into the display delegate; the C strings may be disposed after construction.

Note

`iStateStrings` must be NULL-terminated

14.35.2.2 AAX_CStateDisplayDelegate() [2/4]

```
template<typename T >
AAX_CStateDisplayDelegate< T >::AAX_CStateDisplayDelegate (
    int32_t inNumStates,
    const char * iStateStrings[],
    T iMinState = 0 ) [explicit]
```

Constructor taking a vector of C strings.

Each state name will be copied into the display delegate; the C strings may be disposed after construction.

State strings will be copied into the display delegate until either a NULL pointer is encountered or `inNumStates` strings have been copied

14.35.2.3 AAX_CStateDisplayDelegate() [3/4]

```
template<typename T >
AAX_CStateDisplayDelegate< T >::AAX_CStateDisplayDelegate (
    const std::vector< AAX_IString * > & iStateStrings,
    T iMinState = 0 ) [explicit]
```

Constructor taking a vector of [AAX_IString](#) objects.

Each [AAX_IString](#) will be copied into the display delegate and may be disposed after construction. The [AAX_IString](#) will not be mutated.

14.35.2.4 AAX_CStateDisplayDelegate() [4/4]

```
template<typename T >
AAX_CStateDisplayDelegate< T >::AAX_CStateDisplayDelegate (
    const AAX_CStateDisplayDelegate< T > & other )
```

14.35.3 Member Function Documentation

14.35.3.1 Clone()

```
template<typename T >
AAX_IDisplayDelegate< T > * AAX_CStateDisplayDelegate< T >::Clone ( ) const [virtual]
```

Constructs and returns a copy of the display delegate.

In general, this method's implementation can use a simple copy constructor:

```
template <typename T>
AAX_CSubclassDisplayDelegate<T>*    AAX_CSubclassDisplayDelegate<T>::Clone() const
{
    return new AAX_CSubclassDisplayDelegate(*this);
}
```

Implements [AAX_IDisplayDelegate< T >](#).

14.35.3.2 ValueToString() [1/2]

```
template<typename T >
bool AAX_CStateDisplayDelegate< T >::ValueToString (
    T value,
    AAX_CString * valueString ) const [virtual]
```

Converts a real parameter value to a string representation.

Parameters

in	<i>value</i>	The real parameter value that will be converted
out	<i>valueString</i>	A string corresponding to value

Return values

<i>true</i>	The string conversion was successful
<i>false</i>	The string conversion was unsuccessful

Implements [AAX_IDisplayDelegate< T >](#).

14.35.3.3 ValueToString() [2/2]

```
template<typename T >
bool AAX_CStateDisplayDelegate< T >::ValueToString (
    T value,
    int32_t maxNumChars,
    AAX_CString * valueString ) const [virtual]
```

Converts a real parameter value to a string representation using a size hint, useful for control surfaces and other character limited displays.

Parameters

in	<i>value</i>	The real parameter value that will be converted
in	<i>maxNumChars</i>	Size hint for the desired maximum number of characters in the string (not including null termination)
out	<i>valueString</i>	A string corresponding to value

Return values

<i>true</i>	The string conversion was successful
<i>false</i>	The string conversion was unsuccessful

Implements [AAX_IDisplayDelegate< T >](#).

14.35.3.4 StringToValue()

```
template<typename T >
bool AAX_CStateDisplayDelegate< T >::StringToValue (
    const AAX_CString & valueString,
    T * value ) const [virtual]
```

Converts a string to a real parameter value.

Parameters

in	<i>valueString</i>	The string that will be converted
out	<i>value</i>	The real parameter value corresponding to valueString

Return values

<i>true</i>	The string conversion was successful
<i>false</i>	The string conversion was unsuccessful

Implements [AAX_IDisplayDelegate< T >](#).

14.35.3.5 AddShortenedStrings()

```
template<typename T >
void AAX_CStateDisplayDelegate< T >::AddShortenedStrings (
    const char * iStateStrings[],
    int iLength )
```

14.35.3.6 Compare()

```
template<typename T >
bool AAX_CStateDisplayDelegate< T >::Compare (
    const AAX_CString & valueString,
    const AAX_CString & stateString ) const
```

The documentation for this class was generated from the following file:

- [AAX_CStateDisplayDelegate.h](#)

14.36 AAX_CStatelessParameter Class Reference

```
#include <AAX_CParameter.h>
```

Inheritance diagram for AAX_CStatelessParameter:

Collaboration diagram for AAX_CStatelessParameter:

14.36.1 Description

A stateless parameter implementation.

This can be useful for mapping event triggers to control surface buttons or to GUI switches.

Public Member Functions

- [AAX_CStatelessParameter](#) ([AAX_CParamID](#) identifier, const [AAX_IString](#) &name, const [AAX_IString](#) &in↔ValueString)
- [AAX_CStatelessParameter](#) (const [AAX_IString](#) &identifier, const [AAX_IString](#) &name, const [AAX_IString](#) &inValueString)
- [AAX_DEFAULT_DTOR_OVERRIDE](#) ([AAX_CStatelessParameter](#))
- [AAX_IParameterValue](#) * [CloneValue](#) () const [AAX_OVERRIDE](#)
Clone the parameter's value to a new [AAX_IParameterValue](#) object.

Identification methods

- [AAX_CParamID Identifier](#) () const [AAX_OVERRIDE](#)
Returns the parameter's unique identifier.
- void [SetName](#) (const [AAX_CString](#) &name) [AAX_OVERRIDE](#)

- Sets the parameter's display name.*
- const [AAX_CString & Name](#) () const [AAX_OVERRIDE](#)
- Returns the parameter's display name.*
- void [AddShortenedName](#) (const [AAX_CString](#) &name) [AAX_OVERRIDE](#)
- Sets the parameter's shortened display name.*
- const [AAX_CString & ShortenedName](#) (int32_t iNumCharacters) const [AAX_OVERRIDE](#)
- Returns the parameter's shortened display name.*
- void [ClearShortenedNames](#) () [AAX_OVERRIDE](#)
- Clears the internal list of shortened display names.*

Automation methods

- bool [Automatable](#) () const [AAX_OVERRIDE](#)
- Returns true if the parameter is automatable, false if it is not.*
- void [SetAutomationDelegate](#) ([AAX_IAutomationDelegate](#) *iAutomationDelegate) [AAX_OVERRIDE](#)
- Sets the automation delegate (if one is required)*
- void [Touch](#) () [AAX_OVERRIDE](#)
- Signals the automation system that a control has been touched.*
- void [Release](#) () [AAX_OVERRIDE](#)
- Signals the automation system that a control has been released.*

Taper methods

- void [SetNormalizedValue](#) (double) [AAX_OVERRIDE](#)
- Sets a parameter value using it's normalized representation.*
- double [GetNormalizedValue](#) () const [AAX_OVERRIDE](#)
- Returns the normalized representation of the parameter's current real value.*
- void [SetNormalizedDefaultValue](#) (double) [AAX_OVERRIDE](#)
- Sets the parameter's default value using its normalized representation.*
- double [GetNormalizedDefaultValue](#) () const [AAX_OVERRIDE](#)
- Returns the normalized representation of the parameter's real default value.*
- void [SetToDefaultValue](#) () [AAX_OVERRIDE](#)
- Restores the state of this parameter to its default value.*
- void [SetNumberOfSteps](#) (uint32_t) [AAX_OVERRIDE](#)
- Sets the number of discrete steps for this parameter.*
- uint32_t [GetNumberOfSteps](#) () const [AAX_OVERRIDE](#)
- Returns the number of discrete steps used by the parameter.*
- uint32_t [GetStepValue](#) () const [AAX_OVERRIDE](#)
- Returns the current step for the current value of the parameter.*
- double [GetNormalizedValueFromStep](#) (uint32_t) const [AAX_OVERRIDE](#)
- Returns the normalized value for a given step.*
- uint32_t [GetStepValueFromNormalizedValue](#) (double) const [AAX_OVERRIDE](#)
- Returns the step value for a normalized value of the parameter.*
- void [SetStepValue](#) (uint32_t) [AAX_OVERRIDE](#)
- Returns the current step for the current value of the parameter.*

Display methods

This functionality is most often used by GUIs, but can also be useful for state serialization.

- bool [GetValueString](#) ([AAX_CString](#) *valueString) const [AAX_OVERRIDE](#)
- Serializes the parameter value into a string.*
- bool [GetValueString](#) (int32_t, [AAX_CString](#) *valueString) const [AAX_OVERRIDE](#)
- Serializes the parameter value into a string, size hint included.*
- bool [GetNormalizedValueFromBool](#) (bool, double *normalizedValue) const [AAX_OVERRIDE](#)
- Converts a bool to a normalized parameter value.*
- bool [GetNormalizedValueFromInt32](#) (int32_t, double *normalizedValue) const [AAX_OVERRIDE](#)
- Converts an integer to a normalized parameter value.*

- bool [GetNormalizedValueFromFloat](#) (float, double *normalizedValue) const [AAX_OVERRIDE](#)
Converts a float to a normalized parameter value.
- bool [GetNormalizedValueFromDouble](#) (double, double *normalizedValue) const [AAX_OVERRIDE](#)
Converts a double to a normalized parameter value.
- bool [GetNormalizedValueFromString](#) (const [AAX_CString](#) &, double *normalizedValue) const [AAX_OVERRIDE](#)
Converts a given string to a normalized parameter value.
- bool [GetBoolFromNormalizedValue](#) (double, bool *value) const [AAX_OVERRIDE](#)
Converts a normalized parameter value to a bool representing the corresponding real value.
- bool [GetInt32FromNormalizedValue](#) (double, int32_t *) const [AAX_OVERRIDE](#)
Converts a normalized parameter value to an integer representing the corresponding real value.
- bool [GetFloatFromNormalizedValue](#) (double, float *) const [AAX_OVERRIDE](#)
Converts a normalized parameter value to a float representing the corresponding real value.
- bool [GetDoubleFromNormalizedValue](#) (double, double *) const [AAX_OVERRIDE](#)
Converts a normalized parameter value to a double representing the corresponding real value.
- bool [GetStringFromNormalizedValue](#) (double, [AAX_CString](#) &valueString) const [AAX_OVERRIDE](#)
Converts a normalized parameter value to a string representing the corresponding real value.
- bool [GetStringFromNormalizedValue](#) (double normalizedValue, int32_t, [AAX_CString](#) &valueString) const [AAX_OVERRIDE](#)
Converts a normalized parameter value to a string representing the corresponding real, size hint included. value.
- bool [SetValueFromString](#) (const [AAX_CString](#) &newValueString) [AAX_OVERRIDE](#)
Converts a string to a real parameter value and sets the parameter to this value.

Typed accessors

- bool [GetValueAsBool](#) (bool *value) const [AAX_OVERRIDE](#)
Retrieves the parameter's value as a bool.
- bool [GetValueAsInt32](#) (int32_t *) const [AAX_OVERRIDE](#)
Retrieves the parameter's value as an int32_t.
- bool [GetValueAsFloat](#) (float *) const [AAX_OVERRIDE](#)
Retrieves the parameter's value as a float.
- bool [GetValueAsDouble](#) (double *) const [AAX_OVERRIDE](#)
Retrieves the parameter's value as a double.
- bool [GetValueAsString](#) ([AAX_IString](#) *) const [AAX_OVERRIDE](#)
Retrieves the parameter's value as a string.
- bool [SetValueWithBool](#) (bool) [AAX_OVERRIDE](#)
Sets the parameter's value as a bool.
- bool [SetValueWithInt32](#) (int32_t) [AAX_OVERRIDE](#)
Sets the parameter's value as an int32_t.
- bool [SetValueWithFloat](#) (float) [AAX_OVERRIDE](#)
Sets the parameter's value as a float.
- bool [SetValueWithDouble](#) (double) [AAX_OVERRIDE](#)
Sets the parameter's value as a double.
- bool [SetValueWithString](#) (const [AAX_IString](#) &value) [AAX_OVERRIDE](#)
Sets the parameter's value as a string.
- void [SetType](#) ([AAX_EParameterType](#)) [AAX_OVERRIDE](#)
Sets the type of this parameter.
- [AAX_EParameterType](#) [GetType](#) () const [AAX_OVERRIDE](#)
Returns the type of this parameter as an AAX_EParameterType.
- void [SetOrientation](#) ([AAX_EParameterOrientation](#)) [AAX_OVERRIDE](#)
Sets the orientation of this parameter.
- [AAX_EParameterOrientation](#) [GetOrientation](#) () const [AAX_OVERRIDE](#)
Returns the orientation of this parameter.
- void [SetTaperDelegate](#) ([AAX_ITaperDelegateBase](#) &, bool) [AAX_OVERRIDE](#)
Sets the parameter's taper delegate.
- void [SetDisplayDelegate](#) ([AAX_IDisplayDelegateBase](#) &) [AAX_OVERRIDE](#)
Sets the parameter's display delegate.

Host interface methods

- [AAX_CStringAbbreviations](#) mNames
- [AAX_CString](#) mID
- [AAX_IAutomationDelegate](#) * mAutomationDelegate
- [AAX_CString](#) mValueString
- void [UpdateNormalizedValue](#) (double) [AAX_OVERRIDE](#)

Sets the parameter's state given a normalized value.

14.36.2 Constructor & Destructor Documentation

14.36.2.1 AAX_CStatelessParameter() [1/2]

```
AAX_CStatelessParameter::AAX_CStatelessParameter (
    AAX_CParamID identifier,
    const AAX_IString & name,
    const AAX_IString & inValueString ) [inline]
```

14.36.2.2 AAX_CStatelessParameter() [2/2]

```
AAX_CStatelessParameter::AAX_CStatelessParameter (
    const AAX_IString & identifier,
    const AAX_IString & name,
    const AAX_IString & inValueString ) [inline]
```

14.36.3 Member Function Documentation

14.36.3.1 AAX_DEFAULT_DTOR_OVERRIDE()

```
AAX_CStatelessParameter::AAX_DEFAULT_DTOR_OVERRIDE (
    AAX_CStatelessParameter )
```

14.36.3.2 CloneValue()

```
AAX_IParameterValue* AAX_CStatelessParameter::CloneValue ( ) const [inline], [virtual]
```

Clone the parameter's value to a new [AAX_IParameterValue](#) object.

The returned object is independent from the [AAX_IParameter](#). For example, changing the state of the returned object will not result in a change to the original [AAX_IParameter](#).

Implements [AAX_IParameter](#).

14.36.3.3 Identifier()

```
AAX_CParamID AAX_CStatelessParameter::Identifier ( ) const [inline], [virtual]
```

Returns the parameter's unique identifier.

This unique ID is used by the [Parameter Manager](#) and by outside applications to uniquely identify and target control messages. This value may not be changed after the parameter has been constructed.

Implements [AAX_IParameter](#).

References [AAX_CString::CString\(\)](#), and [mID](#).

Referenced by [Release\(\)](#), [SetAutomationDelegate\(\)](#), [SetName\(\)](#), and [Touch\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

14.36.3.4 SetName()

```
void AAX_CStatelessParameter::SetName (
    const AAX_CString & name ) [inline], [virtual]
```

Sets the parameter's display name.

This name is used for display only, it is not used for indexing or identifying the parameter. This name may be changed after the parameter has been created, but display name changes may not be recognized by all AAX hosts.

Parameters

in	<i>name</i>	Display name that will be assigned to the parameter
----	-------------	---

Implements [AAX_IParameter](#).

References [Identifier\(\)](#), [mAutomationDelegate](#), [mNames](#), [AAX_IAutomationDelegate::ParameterNameChanged\(\)](#), and [AAX_CStringAbbreviations::SetPrimary\(\)](#).

Here is the call graph for this function:

14.36.3.5 Name()

```
const AAX_CString& AAX_CStatelessParameter::Name ( ) const [inline], [virtual]
```

Returns the parameter's display name.

Note

This method returns a const reference in order to prevent a string copy. Do not cast away the const to change this value.

Implements [AAX_IParameter](#).

References [mNames](#), and [AAX_CStringAbbreviations::Primary\(\)](#).

Here is the call graph for this function:

14.36.3.6 AddShortenedName()

```
void AAX_CStatelessParameter::AddShortenedName (
    const AAX_CString & name ) [inline], [virtual]
```

Sets the parameter's shortened display name.

This name is used for display only, it is not used for indexing or identifying the parameter. These names show up when the host asks for shorter length parameter names for display on Control Surfaces or other string length constrained situations.

Parameters

in	name	Shortened display names that will be assigned to the parameter
----	------	--

Implements [AAX_IParameter](#).

References [AAX_CStringAbbreviations::Add\(\)](#), and [mNames](#).

Here is the call graph for this function:

14.36.3.7 ShortenedName()

```
const AAX_CString& AAX_CStatelessParameter::ShortenedName (
    int32_t iNumCharacters ) const [inline], [virtual]
```

Returns the parameter's shortened display name.

Note

This method returns a const reference in order to prevent a string copy. Do not cast away the const to change this value.

Implements [AAX_IParameter](#).

References [AAX_CStringAbbreviations::Get\(\)](#), and [mNames](#).

Here is the call graph for this function:

14.36.3.8 ClearShortenedNames()

```
void AAX_CStatelessParameter::ClearShortenedNames ( ) [inline], [virtual]
```

Clears the internal list of shortened display names.

Implements [AAX_IParameter](#).

References [AAX_CStringAbbreviations::Clear\(\)](#), and [mNames](#).

Here is the call graph for this function:

14.36.3.9 Automatable()

```
bool AAX_CStatelessParameter::Automatable ( ) const [inline], [virtual]
```

Returns true if the parameter is automatable, false if it is not.

Note

Subclasses that return true in this method must support host-based automation.

Implements [AAX_IParameter](#).

14.36.3.10 SetAutomationDelegate()

```
void AAX_CStatelessParameter::SetAutomationDelegate (
    AAX_IAutomationDelegate * iAutomationDelegate ) [inline], [virtual]
```

Sets the automation delegate (if one is required)

Parameters

in	<i>iAutomationDelegate</i>	A reference to the parameter manager's automation delegate interface
----	----------------------------	--

Implements [AAX_IParameter](#).

References [Identifier\(\)](#), [mAutomationDelegate](#), [AAX_IAutomationDelegate::RegisterParameter\(\)](#), and [AAX_IAutomationDelegate::UnregisterParameter\(\)](#).

Here is the call graph for this function:

14.36.3.11 Touch()

```
void AAX_CStatelessParameter::Touch ( ) [inline], [virtual]
```

Signals the automation system that a control has been touched.

Call this method in response to GUI events that begin editing, such as a mouse down. After this method has been called you are free to call [SetNormalizedValue\(\)](#) as much as you need, e.g. in order to respond to subsequent mouse moved events. Call [Release\(\)](#) to free the parameter for updates from other controls.

Implements [AAX_IParameter](#).

References [Identifier\(\)](#), [mAutomationDelegate](#), and [AAX_IAutomationDelegate::PostTouchRequest\(\)](#).

Here is the call graph for this function:

14.36.3.12 Release()

```
void AAX_CStatelessParameter::Release ( ) [inline], [virtual]
```

Signals the automation system that a control has been released.

Call this method in response to GUI events that complete editing, such as a mouse up. Once this method has been called you should not call [SetNormalizedValue\(\)](#) again until after the next call to [Touch\(\)](#).

Implements [AAX_IParameter](#).

References Identifier(), mAutomationDelegate, and AAX_IAutomationDelegate::PostReleaseRequest().

Here is the call graph for this function:

14.36.3.13 SetNormalizedValue()

```
void AAX_CStatelessParameter::SetNormalizedValue (
    double newNormalizedValue ) [inline], [virtual]
```

Sets a parameter value using it's normalized representation.

For more information regarding normalized values, see [Parameter Manager](#)

Parameters

in	<i>newNormalizedValue</i>	New value (normalized) to which the parameter will be set
----	---------------------------	---

Implements [AAX_IParameter](#).

14.36.3.14 GetNormalizedValue()

```
double AAX_CStatelessParameter::GetNormalizedValue ( ) const [inline], [virtual]
```

Returns the normalized representation of the parameter's current real value.

Implements [AAX_IParameter](#).

14.36.3.15 SetNormalizedDefaultValue()

```
void AAX_CStatelessParameter::SetNormalizedDefaultValue (
    double normalizedDefault ) [inline], [virtual]
```

Sets the parameter's default value using its normalized representation.

Implements [AAX_IParameter](#).

14.36.3.16 GetNormalizedDefaultValue()

```
double AAX_CStatelessParameter::GetNormalizedDefaultValue ( ) const [inline], [virtual]
```

Returns the normalized representation of the parameter's real default value.

Implements [AAX_IParameter](#).

14.36.3.17 SetToDefaultValue()

```
void AAX_CStatelessParameter::SetToDefaultValue ( ) [inline], [virtual]
```

Restores the state of this parameter to its default value.

Implements [AAX_IParameter](#).

14.36.3.18 SetNumberOfSteps()

```
void AAX_CStatelessParameter::SetNumberOfSteps (
    uint32_t numSteps ) [inline], [virtual]
```

Sets the number of discrete steps for this parameter.

Stepped parameter values are useful for discrete parameters and for "jumping" events such as mouse wheels, page up/down, etc. The parameter's step size is used to specify the coarseness of those changes.

Note

numSteps MUST be greater than zero. All other values may be considered an error by the host.

Parameters

in	<i>numSteps</i>	The number of steps that the parameter will use
----	-----------------	---

Implements [AAX_IParameter](#).

14.36.3.19 GetNumberOfSteps()

```
uint32_t AAX_CStatelessParameter::GetNumberOfSteps ( ) const [inline], [virtual]
```

Returns the number of discrete steps used by the parameter.

See [SetNumberOfSteps\(\)](#) for more information about parameter steps.

Implements [AAX_IParameter](#).

14.36.3.20 GetStepValue()

```
uint32_t AAX_CStatelessParameter::GetStepValue ( ) const [inline], [virtual]
```

Returns the current step for the current value of the parameter.

See [SetNumberOfSteps\(\)](#) for more information about parameter steps.

Implements [AAX_IParameter](#).

14.36.3.21 GetNormalizedValueFromStep()

```
double AAX_CStatelessParameter::GetNormalizedValueFromStep (
    uint32_t iStep ) const [inline], [virtual]
```

Returns the normalized value for a given step.

See [SetNumberOfSteps\(\)](#) for more information about parameter steps.

Implements [AAX_IParameter](#).

14.36.3.22 GetStepValueFromNormalizedValue()

```
uint32_t AAX_CStatelessParameter::GetStepValueFromNormalizedValue (
    double normalizedValue ) const [inline], [virtual]
```

Returns the step value for a normalized value of the parameter.

See [SetNumberOfSteps\(\)](#) for more information about parameter steps.

Implements [AAX_IParameter](#).

14.36.3.23 SetStepValue()

```
void AAX_CStatelessParameter::SetStepValue (
    uint32_t iStep ) [inline], [virtual]
```

Returns the current step for the current value of the parameter.

See [SetNumberOfSteps\(\)](#) for more information about parameter steps.

Implements [AAX_IParameter](#).

14.36.3.24 GetValueString() [1/2]

```
bool AAX_CStatelessParameter::GetValueString (
    AAX_CString * valueString ) const [inline], [virtual]
```

Serializes the parameter value into a string.

Parameters

out	<i>valueString</i>	A string representing the parameter's real value
-----	--------------------	--

Return values

<i>true</i>	The string conversion was successful
<i>false</i>	The string conversion was unsuccessful

Implements [AAX_IParameter](#).

References `mValueString`.

14.36.3.25 GetValueString() [2/2]

```
bool AAX_CStatelessParameter::GetValueString (
    int32_t iMaxNumChars,
    AAX_CString * valueString ) const [inline], [virtual]
```

Serializes the parameter value into a string, size hint included.

Parameters

in	<i>iMaxNumChars</i>	A size hint for the size of the string being requested. Useful for control surfaces and other limited area text fields. (make sure that size of desired string also has room for null termination)
out	<i>valueString</i>	A string representing the parameter's real value

Return values

<i>true</i>	The string conversion was successful
<i>false</i>	The string conversion was unsuccessful

Implements [AAX_IParameter](#).

References `GetValueString()`.

Referenced by `GetValueString()`.

Here is the call graph for this function: Here is the caller graph for this function:

14.36.3.26 GetNormalizedValueFromBool()

```
bool AAX_CStatelessParameter::GetNormalizedValueFromBool (
    bool value,
    double * normalizedValue ) const [inline], [virtual]
```

Converts a bool to a normalized parameter value.

Parameters

in	<i>value</i>	A value for the parameter
out	<i>normalizedValue</i>	The normalized parameter value associated with value

Return values

<i>true</i>	The value conversion was successful
<i>false</i>	The value conversion was unsuccessful

Implements [AAX_IParameter](#).

14.36.3.27 GetNormalizedValueFromInt32()

```
bool AAX_CStatelessParameter::GetNormalizedValueFromInt32 (
    int32_t value,
    double * normalizedValue ) const [inline], [virtual]
```

Converts an integer to a normalized parameter value.

Parameters

in	<i>value</i>	A value for the parameter
out	<i>normalizedValue</i>	The normalized parameter value associated with value

Return values

<i>true</i>	The value conversion was successful
<i>false</i>	The value conversion was unsuccessful

Implements [AAX_IParameter](#).

14.36.3.28 GetNormalizedValueFromFloat()

```
bool AAX_CStatelessParameter::GetNormalizedValueFromFloat (
    float value,
    double * normalizedValue ) const [inline], [virtual]
```

Converts a float to a normalized parameter value.

Parameters

in	<i>value</i>	A value for the parameter
out	<i>normalizedValue</i>	The normalized parameter value associated with value

Return values

<i>true</i>	The value conversion was successful
<i>false</i>	The value conversion was unsuccessful

Implements [AAX_IParameter](#).

14.36.3.29 GetNormalizedValueFromDouble()

```
bool AAX_CStatelessParameter::GetNormalizedValueFromDouble (
    double value,
    double * normalizedValue ) const [inline], [virtual]
```

Converts a double to a normalized parameter value.

Parameters

in	<i>value</i>	A value for the parameter
out	<i>normalizedValue</i>	The normalized parameter value associated with value

Return values

<i>true</i>	The value conversion was successful
<i>false</i>	The value conversion was unsuccessful

Implements [AAX_IParameter](#).

14.36.3.30 GetNormalizedValueFromString()

```
bool AAX_CStatelessParameter::GetNormalizedValueFromString (
    const AAX\_CString & valueString,
    double * normalizedValue ) const [inline], [virtual]
```

Converts a given string to a normalized parameter value.

Parameters

in	<i>valueString</i>	A string representing a possible real value for the parameter
out	<i>normalizedValue</i>	The normalized parameter value associated with valueString

Return values

<i>true</i>	The string conversion was successful
<i>false</i>	The string conversion was unsuccessful

Implements [AAX_IParameter](#).

14.36.3.31 GetBoolFromNormalizedValue()

```
bool AAX_CStatelessParameter::GetBoolFromNormalizedValue (
    double normalizedValue,
    bool * value ) const [inline], [virtual]
```

Converts a normalized parameter value to a bool representing the corresponding real value.

Parameters

in	<i>normalizedValue</i>	The normalized value to convert
out	<i>value</i>	The converted value. Set only if conversion is successful.

Return values

<i>true</i>	The conversion to bool was successful
<i>false</i>	The conversion to bool was unsuccessful

Implements [AAX_IParameter](#).

14.36.3.32 GetInt32FromNormalizedValue()

```
bool AAX_CStatelessParameter::GetInt32FromNormalizedValue (
    double normalizedValue,
    int32_t * value ) const [inline], [virtual]
```

Converts a normalized parameter value to an integer representing the corresponding real value.

Parameters

in	<i>normalizedValue</i>	The normalized value to convert
out	<i>value</i>	The converted value. Set only if conversion is successful.

Return values

<i>true</i>	The conversion to int32_t was successful
<i>false</i>	The conversion to int32_t was unsuccessful

Implements [AAX_IParameter](#).

14.36.3.33 GetFloatFromNormalizedValue()

```
bool AAX_CStatelessParameter::GetFloatFromNormalizedValue (
    double normalizedValue,
    float * value ) const [inline], [virtual]
```

Converts a normalized parameter value to a float representing the corresponding real value.

Parameters

in	<i>normalizedValue</i>	The normalized value to convert
out	<i>value</i>	The converted value. Set only if conversion is successful.

Return values

<i>true</i>	The conversion to float was successful
<i>false</i>	The conversion to float was unsuccessful

Implements [AAX_IParameter](#).

14.36.3.34 GetDoubleFromNormalizedValue()

```
bool AAX_CStatelessParameter::GetDoubleFromNormalizedValue (
    double normalizedValue,
    double * value ) const [inline], [virtual]
```

Converts a normalized parameter value to a double representing the corresponding real value.

Parameters

in	<i>normalizedValue</i>	The normalized value to convert
out	<i>value</i>	The converted value. Set only if conversion is successful.

Return values

<i>true</i>	The conversion to double was successful
<i>false</i>	The conversion to double was unsuccessful

Implements [AAX_IParameter](#).

14.36.3.35 GetStringFromNormalizedValue() [1/2]

```
bool AAX_CStatelessParameter::GetStringFromNormalizedValue (
    double normalizedValue,
    AAX_CString & valueString ) const [inline], [virtual]
```

Converts a normalized parameter value to a string representing the corresponding real value.

Parameters

in	<i>normalizedValue</i>	A normalized parameter value
out	<i>valueString</i>	A string representing the parameter value associated with <i>normalizedValue</i>

Return values

<i>true</i>	The string conversion was successful
<i>false</i>	The string conversion was unsuccessful

Implements [AAX_IParameter](#).

References `mValueString`.

14.36.3.36 GetStringFromNormalizedValue() [2/2]

```
bool AAX_CStatelessParameter::GetStringFromNormalizedValue (
    double normalizedValue,
    int32_t iMaxNumChars,
    AAX_CString & valueString ) const [inline], [virtual]
```

Converts a normalized parameter value to a string representing the corresponding real, size hint included. value.

Parameters

in	<i>normalizedValue</i>	A normalized parameter value
in	<i>iMaxNumChars</i>	A size hint for the size of the string being requested. Useful for control surfaces and other limited area text fields. (make sure that size of desired string also has room for null termination)
out	<i>valueString</i>	A string representing the parameter value associated with <i>normalizedValue</i>

Return values

<i>true</i>	The string conversion was successful
<i>false</i>	The string conversion was unsuccessful

Implements [AAX_IParameter](#).

References `GetStringFromNormalizedValue()`.

Referenced by `GetStringFromNormalizedValue()`.

Here is the call graph for this function: Here is the caller graph for this function:

14.36.3.37 SetValueFromString()

```
bool AAX_CStatelessParameter::SetValueFromString (
    const AAX_CString & newValueString ) [inline], [virtual]
```

Converts a string to a real parameter value and sets the parameter to this value.

Parameters

in	<i>newValueString</i>	A string representing the parameter's new real value
----	-----------------------	--

Return values

<i>true</i>	The string conversion was successful
<i>false</i>	The string conversion was unsuccessful

Implements [AAX_IParameter](#).

References `mValueString`.

14.36.3.38 GetValueAsBool()

```
bool AAX_CStatelessParameter::GetValueAsBool (
    bool * value ) const [inline], [virtual]
```

Retrieves the parameter's value as a bool.

Parameters

out	<i>value</i>	The parameter's real value. Set only if conversion is successful.
-----	--------------	---

Return values

<i>true</i>	The conversion to bool was successful
<i>false</i>	The conversion to bool was unsuccessful

Implements [AAX_IParameter](#).

14.36.3.39 GetValueAsInt32()

```
bool AAX_CStatelessParameter::GetValueAsInt32 (
    int32_t * value ) const [inline], [virtual]
```

Retrieves the parameter's value as an `int32_t`.

Parameters

out	<i>value</i>	The parameter's real value. Set only if conversion is successful.
-----	--------------	---

Return values

<i>true</i>	The conversion to int32_t was successful
<i>false</i>	The conversion to int32_t was unsuccessful

Implements [AAX_IParameter](#).

14.36.3.40 GetValueAsFloat()

```
bool AAX_CStatelessParameter::GetValueAsFloat (
    float * value ) const [inline], [virtual]
```

Retrieves the parameter's value as a float.

Parameters

out	<i>value</i>	The parameter's real value. Set only if conversion is successful.
-----	--------------	---

Return values

<i>true</i>	The conversion to float was successful
<i>false</i>	The conversion to float was unsuccessful

Implements [AAX_IParameter](#).

14.36.3.41 GetValueAsDouble()

```
bool AAX_CStatelessParameter::GetValueAsDouble (
    double * value ) const [inline], [virtual]
```

Retrieves the parameter's value as a double.

Parameters

out	<i>value</i>	The parameter's real value. Set only if conversion is successful.
-----	--------------	---

Return values

<i>true</i>	The conversion to double was successful
<i>false</i>	The conversion to double was unsuccessful

Implements [AAX_IParameter](#).

14.36.3.42 GetValueAsString()

```
bool AAX_CStatelessParameter::GetValueAsString (
    AAX_IString * value ) const [inline], [virtual]
```

Retrieves the parameter's value as a string.

Parameters

out	value	The parameter's real value. Set only if conversion is successful.
-----	-------	---

Return values

true	The conversion to string was successful
false	The conversion to string was unsuccessful

Implements [AAX_IParameter](#).

14.36.3.43 SetValueWithBool()

```
bool AAX_CStatelessParameter::SetValueWithBool (
    bool value ) [inline], [virtual]
```

Sets the parameter's value as a bool.

Parameters

out	value	The parameter's real value. Set only if conversion is successful.
-----	-------	---

Return values

true	The conversion from bool was successful
false	The conversion from bool was unsuccessful

Implements [AAX_IParameter](#).

14.36.3.44 SetValueWithInt32()

```
bool AAX_CStatelessParameter::SetValueWithInt32 (
    int32_t value ) [inline], [virtual]
```

Sets the parameter's value as an int32_t.

Parameters

out	value	The parameter's real value. Set only if conversion is successful.
-----	-------	---

Return values

true	The conversion from int32_t was successful
false	The conversion from int32_t was unsuccessful

Implements [AAX_IParameter](#).

14.36.3.45 SetValueWithFloat()

```
bool AAX_CStatelessParameter::SetValueWithFloat (
    float value ) [inline], [virtual]
```

Sets the parameter's value as a float.

Parameters

out	value	The parameter's real value. Set only if conversion is successful.
-----	-------	---

Return values

true	The conversion from float was successful
false	The conversion from float was unsuccessful

Implements [AAX_IParameter](#).

14.36.3.46 SetValueWithDouble()

```
bool AAX_CStatelessParameter::SetValueWithDouble (
    double value ) [inline], [virtual]
```

Sets the parameter's value as a double.

Parameters

out	value	The parameter's real value. Set only if conversion is successful.
-----	-------	---

Return values

true	The conversion from double was successful
false	The conversion from double was unsuccessful

Implements [AAX_IParameter](#).

14.36.3.47 SetValueWithString()

```
bool AAX_CStatelessParameter::SetValueWithString (
    const AAX_IString & value ) [inline], [virtual]
```

Sets the parameter's value as a string.

Parameters

out	<i>value</i>	The parameter's real value. Set only if conversion is successful.
-----	--------------	---

Return values

<i>true</i>	The conversion from string was successful
<i>false</i>	The conversion from string was unsuccessful

Implements [AAX_IParameter](#).

References mValueString.

14.36.3.48 SetType()

```
void AAX_CStatelessParameter::SetType (
    AAX_EParameterType iControlType ) [inline], [virtual]
```

Sets the type of this parameter.

See [GetType](#) for use cases

Parameters

in	<i>iControlType</i>	The parameter's new type as an AAX_EParameterType
----	---------------------	---

Implements [AAX_IParameter](#).

14.36.3.49 GetType()

```
AAX_EParameterType AAX_CStatelessParameter::GetType ( ) const [inline], [virtual]
```

Returns the type of this parameter as an AAX_EParameterType.

Todo Document use cases for control type

Implements [AAX_IParameter](#).

References [AAX_eParameterType_Discrete](#).

14.36.3.50 SetOrientation()

```
void AAX_CStatelessParameter::SetOrientation (
    AAX_EParameterOrientation iOrientation ) [inline], [virtual]
```

Sets the orientation of this parameter.

Parameters

in	<i>iOrientation</i>	The parameter's new orientation
----	---------------------	---------------------------------

Implements [AAX_IParameter](#).

14.36.3.51 GetOrientation()

```
AAX_EParameterOrientation AAX_CStatelessParameter::GetOrientation ( ) const [inline], [virtual]
```

Returns the orientation of this parameter.

Implements [AAX_IParameter](#).

References [AAX_eParameterOrientation_Default](#).

14.36.3.52 SetTaperDelegate()

```
void AAX_CStatelessParameter::SetTaperDelegate (
    AAX_ITaperDelegateBase & inTaperDelegate,
    bool inPreserveValue ) [inline], [virtual]
```

Sets the parameter's taper delegate.

Parameters

in	<i>inTaperDelegate</i>	A reference to the parameter's new taper delegate
in	<i>inPreserveValue</i>	

Todo Document this parameter

Implements [AAX_IParameter](#).

14.36.3.53 SetDisplayDelegate()

```
void AAX_CStatelessParameter::SetDisplayDelegate (
    AAX_IDisplayDelegateBase & inDisplayDelegate ) [inline], [virtual]
```

Sets the parameter's display delegate.

Parameters

in	<i>inDisplayDelegate</i>	A reference to the parameter's new display delegate
----	--------------------------	---

Implements [AAX_IParameter](#).

14.36.3.54 UpdateNormalizedValue()

```
void AAX_CStatelessParameter::UpdateNormalizedValue (
    double newNormalizedValue ) [inline], [virtual]
```

Sets the parameter's state given a normalized value.

This is the second half of the parameter setting operation that is initiated with a call to SetValue(). Parameters should not be set directly using this method; instead, use SetValue().

Parameters

in	<i>newNormalizedValue</i>	Normalized value that will be used to set the parameter's new state
----	---------------------------	---

Implements [AAX_IParameter](#).

14.36.4 Member Data Documentation

14.36.4.1 mNames

```
AAX_CStringAbbreviations AAX_CStatelessParameter::mNames [protected]
```

Referenced by AddShortenedName(), ClearShortenedNames(), Name(), SetName(), and ShortenedName().

14.36.4.2 mID

`AAX_CString` AAX_CStatelessParameter::mID [protected]

Referenced by Identifier().

14.36.4.3 mAutomationDelegate

`AAX_IAutomationDelegate*` AAX_CStatelessParameter::mAutomationDelegate [protected]

Referenced by Release(), SetAutomationDelegate(), SetName(), and Touch().

14.36.4.4 mValueString

`AAX_CString` AAX_CStatelessParameter::mValueString [protected]

Referenced by GetStringFromNormalizedValue(), GetValueString(), SetValueFromString(), and SetValueWithString().

The documentation for this class was generated from the following file:

- [AAX_CParameter.h](#)

14.37 AAX_CStateTaperDelegate< T > Class Template Reference

```
#include <AAX_CStateTaperDelegate.h>
```

Inheritance diagram for AAX_CStateTaperDelegate< T >:

Collaboration diagram for AAX_CStateTaperDelegate< T >:

14.37.1 Description

```
template<typename T>
class AAX_CStateTaperDelegate< T >
```

A linear taper conforming to [AAX_ITaperDelegate](#).

This taper spaces a parameter's real values evenly between its minimum and maximum, with a linear mapping between the parameter's real and normalized values. It is essentially a version of [AAX_CLinearTaperDelegate](#) without that class' additional RealPrecision templatzation.

Public Member Functions

- [AAX_CStateTaperDelegate](#) (T minValue=0, T maxValue=1)
Constructs a State Taper with specified minimum and maximum values.
- [AAX_CStateTaperDelegate](#)< T > * [Clone](#) () const [AAX_OVERRIDE](#)
Constructs and returns a copy of the taper delegate.
- T [GetMinimumValue](#) () const [AAX_OVERRIDE](#)
Returns the taper's minimum real value.
- T [GetMaximumValue](#) () const [AAX_OVERRIDE](#)
Returns the taper's maximum real value.
- T [ConstrainRealValue](#) (T value) const [AAX_OVERRIDE](#)
Applies a constraint to the value and returns the constrained value.
- T [NormalizedToReal](#) (double normalizedValue) const [AAX_OVERRIDE](#)
Converts a normalized value to a real value.
- double [RealToNormalized](#) (T realValue) const [AAX_OVERRIDE](#)
Normalizes a real parameter value.

14.37.2 Constructor & Destructor Documentation

14.37.2.1 AAX_CStateTaperDelegate()

```
template<typename T >
AAX_CStateTaperDelegate< T >::AAX_CStateTaperDelegate (
    T minValue = 0,
    T maxValue = 1 )
```

Constructs a State Taper with specified minimum and maximum values.

Note

The parameter's default value should lie within the min to max range.

Parameters

in	<i>minValue</i>	
in	<i>maxValue</i>	

14.37.3 Member Function Documentation

14.37.3.1 Clone()

```
template<typename T >
AAX_CStateTaperDelegate< T > * AAX\_CStateTaperDelegate< T >::Clone ( ) const [virtual]
```

Constructs and returns a copy of the taper delegate.

In general, this method's implementation can use a simple copy constructor:

```
template <typename T>
AAX_CSubclassTaperDelegate<T>* AAX_CSubclassTaperDelegate<T>::Clone() const
{
    return new AAX_CSubclassTaperDelegate(*this);
}
```

Implements [AAX_ITaperDelegate< T >](#).

14.37.3.2 GetMinimumValue()

```
template<typename T >
T AAX_CStateTaperDelegate< T >::GetMinimumValue ( ) const [inline], [virtual]
```

Returns the taper's minimum real value.

Implements [AAX_ITaperDelegate< T >](#).

14.37.3.3 GetMaximumValue()

```
template<typename T >
T AAX_CStateTaperDelegate< T >::GetMaximumValue ( ) const [inline], [virtual]
```

Returns the taper's maximum real value.

Implements [AAX_ITaperDelegate< T >](#).

14.37.3.4 ConstrainRealValue()

```
template<typename T >
T AAX_CStateTaperDelegate< T >::ConstrainRealValue (
    T value ) const [virtual]
```

Applies a constraint to the value and returns the constrained value.

This method is useful if the taper requires a constraint beyond simple minimum and maximum real value limits.

Note

This is the function that should actually enforces the constraints in [NormalizeToReal\(\)](#) and [RealToNormalized\(\)](#).

Parameters

in	<i>value</i>	The unconstrained value
----	--------------	-------------------------

Implements [AAX_ITaperDelegate< T >](#).

14.37.3.5 NormalizedToReal()

```
template<typename T >
T AAX\_CStateTaperDelegate< T >::NormalizedToReal (
    double normalizedValue ) const [virtual]
```

Converts a normalized value to a real value.

This is where the actual taper algorithm is implemented.

This function should perform the exact inverse of [RealToNormalized\(\)](#), to within the roundoff precision of the individual taper implementation.

Parameters

in	<i>normalizedValue</i>	The normalized value that will be converted
----	------------------------	---

Implements [AAX_ITaperDelegate< T >](#).

14.37.3.6 RealToNormalized()

```
template<typename T >
double AAX\_CStateTaperDelegate< T >::RealToNormalized (
    T realValue ) const [virtual]
```

Normalizes a real parameter value.

This is where the actual taper algorithm is implemented.

This function should perform the exact inverse of [NormalizedToReal\(\)](#), to within the roundoff precision of the individual taper implementation.

Parameters

in	<i>realValue</i>	The real parameter value that will be normalized
----	------------------	--

Implements [AAX_ITaperDelegate< T >](#).

The documentation for this class was generated from the following file:

- [AAX_CStateTaperDelegate.h](#)

14.38 AAX_CString Class Reference

```
#include <AAX_CString.h>
```

Inheritance diagram for AAX_CString:

Collaboration diagram for AAX_CString:

14.38.1 Description

A generic AAX string class with similar functionality to `std::string`

Public Member Functions

- `uint32_t Length () const` [AAX_OVERRIDE](#)
- `uint32_t MaxLength () const` [AAX_OVERRIDE](#)
- `const char * Get () const` [AAX_OVERRIDE](#)
- `void Set (const char *iString)` [AAX_OVERRIDE](#)
- `AAX_IString & operator= (const AAX_IString &iOther)` [AAX_OVERRIDE](#)
- `AAX_IString & operator= (const char *iString)` [AAX_OVERRIDE](#)
- `AAX_CString ()`
- `AAX_CString (const char *str)`
- `AAX_CString (const std::string &str)`
- `AAX_CString (const AAX_CString &other)`
- `AAX_CString (const AAX_IString &other)`
- `AAX_DEFAULT_MOVE_CTOR (AAX_CString)`
- `std::string & StdString ()`
- `const std::string & StdString () const`
- `AAX_CString & operator= (const AAX_CString &other)`
- `AAX_CString & operator= (const std::string &other)`
- `AAX_CString & operator= (AAX_CString &&other)`
- `void Clear ()`
- `bool Empty () const`
- `AAX_CString & Erase (uint32_t pos, uint32_t n)`
- `AAX_CString & Append (const AAX_CString &str)`
- `AAX_CString & Append (const char *str)`
- `AAX_CString & AppendNumber (double number, int32_t precision)`
- `AAX_CString & AppendNumber (int32_t number)`
- `AAX_CString & AppendHex (int32_t number, int32_t width)`
- `AAX_CString & Insert (uint32_t pos, const AAX_CString &str)`
- `AAX_CString & Insert (uint32_t pos, const char *str)`
- `AAX_CString & InsertNumber (uint32_t pos, double number, int32_t precision)`
- `AAX_CString & InsertNumber (uint32_t pos, int32_t number)`
- `AAX_CString & InsertHex (uint32_t pos, int32_t number, int32_t width)`
- `AAX_CString & Replace (uint32_t pos, uint32_t n, const AAX_CString &str)`
- `AAX_CString & Replace (uint32_t pos, uint32_t n, const char *str)`
- `uint32_t FindFirst (const AAX_CString &findStr) const`
- `uint32_t FindFirst (const char *findStr) const`
- `uint32_t FindFirst (char findChar) const`
- `uint32_t FindLast (const AAX_CString &findStr) const`
- `uint32_t FindLast (const char *findStr) const`
- `uint32_t FindLast (char findChar) const`

- const char * [CString](#) () const
- bool [ToDouble](#) (double *oValue) const
- bool [ToInteger](#) (int32_t *oValue) const
- void [SubString](#) (uint32_t pos, uint32_t n, [AAX_IString](#) *outputStr) const
- bool [Equals](#) (const [AAX_CString](#) &other) const
- bool [Equals](#) (const char *other) const
- bool [Equals](#) (const std::string &other) const
- bool [operator==](#) (const [AAX_CString](#) &other) const
- bool [operator==](#) (const char *otherStr) const
- bool [operator==](#) (const std::string &otherStr) const
- bool [operator!=](#) (const [AAX_CString](#) &other) const
- bool [operator!=](#) (const char *otherStr) const
- bool [operator!=](#) (const std::string &otherStr) const
- bool [operator<](#) (const [AAX_CString](#) &other) const
- bool [operator>](#) (const [AAX_CString](#) &other) const
- const char & [operator\[\]](#) (uint32_t index) const
- char & [operator\[\]](#) (uint32_t index)
- [AAX_CString](#) & [operator+=](#) (const [AAX_CString](#) &str)
- [AAX_CString](#) & [operator+=](#) (const std::string &str)
- [AAX_CString](#) & [operator+=](#) (const char *str)

Static Public Attributes

- static const uint32_t [kInvalidIndex](#) = static_cast<uint32_t>(-1)
- static const uint32_t [kMaxStringLength](#) = static_cast<uint32_t>(-2)

Protected Attributes

- std::string [mString](#)

Friends

- std::ostream & [operator<<](#) (std::ostream &os, const [AAX_CString](#) &str)
- std::istream & [operator>>](#) (std::istream &os, [AAX_CString](#) &str)

14.38.2 Constructor & Destructor Documentation

14.38.2.1 [AAX_CString\(\)](#) [1/5]

```
AAX_CString::AAX_CString ( )
```

Constructs an empty string.

14.38.2.2 AAX_CString() [2/5]

```
AAX_CString::AAX_CString (
    const char * str )
```

Implicit conversion constructor: Constructs a string with a const char* pointer to copy.

14.38.2.3 AAX_CString() [3/5]

```
AAX_CString::AAX_CString (
    const std::string & str ) [explicit]
```

Copy constructor: Constructs a string from a std::string. Beware of STL variations across various binaries.

14.38.2.4 AAX_CString() [4/5]

```
AAX_CString::AAX_CString (
    const AAX_CString & other )
```

Copy constructor: Constructs a string with another concrete [AAX_CString](#).

14.38.2.5 AAX_CString() [5/5]

```
AAX_CString::AAX_CString (
    const AAX_IString & other )
```

Copy constructor: Constructs a string from another string that meets the [AAX_IString](#) interface.

14.38.3 Member Function Documentation**14.38.3.1 Length()**

```
uint32_t AAX_CString::Length ( ) const [virtual]
```

Length methods

Implements [AAX_IString](#).

Referenced by [AAX_CBinaryDisplayDelegate< T >::AAX_CBinaryDisplayDelegate\(\)](#), [AAX_CStringAbbreviations::Add\(\)](#), [AAX_CStringAbbreviations::Get\(\)](#), [AAX_CDecibelDisplayDelegateDecorator< T >::StringToValue\(\)](#), [AAX_CPercentDisplayDelegateDecorator< T >::StringToValue\(\)](#), [AAX_CUnitDisplayDelegateDecorator< T >::StringToValue\(\)](#), [AAX_CUnitPrefixDisplayDelegateDecorator< T >::StringToValue\(\)](#), [AAX_CDecibelDisplayDelegateDecorator< T >::ValueToString\(\)](#), [AAX_CNumberDisplayDelegate< T, Precision, SpaceAfter >::ValueToString\(\)](#), and [AAX_CUnitDisplayDelegateDecorator< T >::ValueToString\(\)](#).

Here is the caller graph for this function:

14.38.3.2 MaxLength()

```
uint32_t AAX_CString::MaxLength ( ) const [virtual]
```

Implements [AAX_IString](#).

14.38.3.3 Get()

```
const char* AAX_CString::Get ( ) const [virtual]
```

C string methods

Implements [AAX_IString](#).

Referenced by [AAX::CopyPageTable\(\)](#), [AAX::PageTableParameterNameVariationsAreEqual\(\)](#), and [AAX_CParameter< T >::ShortenedName\(\)](#).

Here is the caller graph for this function:

14.38.3.4 Set()

```
void AAX_CString::Set (
    const char * iString ) [virtual]
```

Implements [AAX_IString](#).

14.38.3.5 operator=() [1/5]

```
AAX\_IString& AAX_CString::operator= (
    const AAX\_IString & iOther ) [virtual]
```

Assignment operators

Implements [AAX_IString](#).

14.38.3.6 operator=() [2/5]

```
AAX\_IString& AAX_CString::operator= (
    const char * iString ) [virtual]
```

Implements [AAX_IString](#).

14.38.3.7 AAX_DEFAULT_MOVE_CTOR()

```
AAX_CString::AAX_DEFAULT_MOVE_CTOR (
    AAX_CString )
```

Default move constructor

14.38.3.8 StdString() [1/2]

```
std::string& AAX_CString::StdString ( )
```

Direct access to a std::string.

14.38.3.9 StdString() [2/2]

```
const std::string& AAX_CString::StdString ( ) const
```

Direct access to a const std::string.

14.38.3.10 operator=() [3/5]

```
AAX_CString& AAX_CString::operator= (
    const AAX_CString & other )
```

Assignment operator from another [AAX_CString](#)

14.38.3.11 operator=() [4/5]

```
AAX_CString& AAX_CString::operator= (
    const std::string & other )
```

Assignment operator from a std::string. Beware of STL variations across various binaries.

14.38.3.12 operator=() [5/5]

```
AAX_CString& AAX_CString::operator= (
    AAX_CString && other )
```

Move operator

14.38.3.13 Clear()

```
void AAX_CString::Clear ( )
```

Referenced by [AAX_CNumberDisplayDelegate< T, Precision, SpaceAfter >::ValueToString\(\)](#).

Here is the caller graph for this function:

14.38.3.14 Empty()

```
bool AAX_CString::Empty ( ) const
```

14.38.3.15 Erase()

```
AAX_CString& AAX_CString::Erase (
    uint32_t pos,
    uint32_t n )
```

Referenced by `AAX_CUnitPrefixDisplayDelegateDecorator< T >::StringToValue()`, and `AAX_CNumberDisplayDelegate< T, Precision, SpaceAfter >::ValueToString()`.

Here is the caller graph for this function:

14.38.3.16 Append() [1/2]

```
AAX_CString& AAX_CString::Append (
    const AAX_CString & str )
```

Referenced by `AAX_IMIDIMessageInfoDelegate::ToString_AppendByteRange()`, `AAX_IMIDIMessageInfoDelegate::ToString_AppendCStr()`, `AAX_CNumberDisplayDelegate< T, Precision, SpaceAfter >::ValueToString()`, and `AAX_CDecibelDisplayDelegateDecorator< T >::ValueToString()`.

Here is the caller graph for this function:

14.38.3.17 Append() [2/2]

```
AAX_CString& AAX_CString::Append (
    const char * str )
```

14.38.3.18 AppendNumber() [1/2]

```
AAX_CString& AAX_CString::AppendNumber (
    double number,
    int32_t precision )
```

Referenced by `AAX_IMIDIMessageInfoDelegate::ToString_AppendNumber()`, and `AAX_CNumberDisplayDelegate< T, Precision, SpaceAfter >::ValueToString()`.

Here is the caller graph for this function:

14.38.3.19 AppendNumber() [2/2]

```
AAX_CString& AAX_CString::AppendNumber (
    int32_t number )
```

14.38.3.20 AppendHex()

```
AAX_CString& AAX_CString::AppendHex (
    int32_t number,
    int32_t width )
```

Referenced by AAX_IMIDIMessageInfoDelegate::ToString_AppendByteRange().

Here is the caller graph for this function:

14.38.3.21 Insert() [1/2]

```
AAX_CString& AAX_CString::Insert (
    uint32_t pos,
    const AAX_CString & str )
```

14.38.3.22 Insert() [2/2]

```
AAX_CString& AAX_CString::Insert (
    uint32_t pos,
    const char * str )
```

14.38.3.23 InsertNumber() [1/2]

```
AAX_CString& AAX_CString::InsertNumber (
    uint32_t pos,
    double number,
    int32_t precision )
```

14.38.3.24 InsertNumber() [2/2]

```
AAX_CString& AAX_CString::InsertNumber (
    uint32_t pos,
    int32_t number )
```

14.38.3.25 InsertHex()

```
AAX_CString& AAX_CString::InsertHex (
    uint32_t pos,
    int32_t number,
    int32_t width )
```

14.38.3.26 Replace() [1/2]

```
AAX_CString& AAX_CString::Replace (
    uint32_t pos,
    uint32_t n,
    const AAX_CString & str )
```

14.38.3.27 Replace() [2/2]

```
AAX_CString& AAX_CString::Replace (
    uint32_t pos,
    uint32_t n,
    const char * str )
```

14.38.3.28 FindFirst() [1/3]

```
uint32_t AAX_CString::FindFirst (
    const AAX_CString & findStr ) const
```

14.38.3.29 FindFirst() [2/3]

```
uint32_t AAX_CString::FindFirst (
    const char * findStr ) const
```

14.38.3.30 FindFirst() [3/3]

```
uint32_t AAX_CString::FindFirst (
    char findChar ) const
```

14.38.3.31 FindLast() [1/3]

```
uint32_t AAX_CString::FindLast (
    const AAX_CString & findStr ) const
```


14.38.3.32 FindLast() [2/3]

```
uint32_t AAX_CString::FindLast (
    const char * findStr ) const
```

14.38.3.33 FindLast() [3/3]

```
uint32_t AAX_CString::FindLast (
    char findChar ) const
```

14.38.3.34 CString()

```
const char* AAX_CString::CString ( ) const
```

Referenced by AAX::CopyPageTable(), and AAX_CStatelessParameter::Identifier().

Here is the caller graph for this function:

14.38.3.35 ToDouble()

```
bool AAX_CString::ToDouble (
    double * oValue ) const
```

Referenced by AAX_CNumberDisplayDelegate< T, Precision, SpaceAfter >::StringToValue().

Here is the caller graph for this function:

14.38.3.36 ToInteger()

```
bool AAX_CString::ToInteger (
    int32_t * oValue ) const
```

14.38.3.37 SubString()

```
void AAX_CString::SubString (
    uint32_t pos,
    uint32_t n,
    AAX_IString * outputStr ) const
```

Referenced by AAX_CDecibelDisplayDelegateDecorator< T >::StringToValue(), AAX_CPercentDisplayDelegateDecorator< T >::StringToValue(), and AAX_CUnitDisplayDelegateDecorator< T >::StringToValue().

Here is the caller graph for this function:

14.38.3.38 Equals() [1/3]

```
bool AAX_CString::Equals (
    const AAX_CString & other ) const [inline]
```

References operator==().

Here is the call graph for this function:

14.38.3.39 Equals() [2/3]

```
bool AAX_CString::Equals (
    const char * other ) const [inline]
```

References operator==().

Here is the call graph for this function:

14.38.3.40 Equals() [3/3]

```
bool AAX_CString::Equals (
    const std::string & other ) const [inline]
```

References operator==().

Here is the call graph for this function:

14.38.3.41 operator==() [1/3]

```
bool AAX_CString::operator==(
    const AAX_CString & other ) const
```

Referenced by Equals().

Here is the caller graph for this function:

14.38.3.42 operator==() [2/3]

```
bool AAX_CString::operator==(
    const char * otherStr ) const
```

14.38.3.43 operator==() [3/3]

```
bool AAX_CString::operator==(
    const std::string & otherStr ) const
```

14.38.3.44 operator"!=()" [1/3]

```
bool AAX_CString::operator!= (
    const AAX_CString & other ) const
```

14.38.3.45 operator"!=()" [2/3]

```
bool AAX_CString::operator!= (
    const char * otherStr ) const
```

14.38.3.46 operator"!=()" [3/3]

```
bool AAX_CString::operator!= (
    const std::string & otherStr ) const
```

14.38.3.47 operator<()

```
bool AAX_CString::operator< (
    const AAX_CString & other ) const
```

14.38.3.48 operator>()

```
bool AAX_CString::operator> (
    const AAX_CString & other ) const
```

14.38.3.49 operator[]() [1/2]

```
const char& AAX_CString::operator[] (
    uint32_t index ) const
```

14.38.3.50 operator[]() [2/2]

```
char& AAX_CString::operator[] (
    uint32_t index )
```

14.38.3.51 operator+=() [1/3]

```
AAX_CString& AAX_CString::operator+= (
    const AAX_CString & str )
```

14.38.3.52 operator+=() [2/3]

```
AAX_CString& AAX_CString::operator+= (
    const std::string & str )
```

14.38.3.53 operator+=() [3/3]

```
AAX_CString& AAX_CString::operator+= (
    const char * str )
```

14.38.4 Friends And Related Function Documentation**14.38.4.1 operator<<**

```
std::ostream& operator<< (
    std::ostream & os,
    const AAX_CString & str ) [friend]
```

output stream operator for concrete [AAX_CString](#)

14.38.4.2 operator>>

```
std::istream& operator>> (
    std::istream & os,
    AAX_CString & str ) [friend]
```

input stream operator for concrete [AAX_CString](#)

14.38.5 Member Data Documentation

14.38.5.1 kInvalidIndex

```
const uint32_t AAX_CString::kInvalidIndex = static_cast<uint32_t>(-1) [static]
```

14.38.5.2 kMaxStringLength

```
const uint32_t AAX_CString::kMaxStringLength = static_cast<uint32_t>(-2) [static]
```

14.38.5.3 mString

```
std::string AAX_CString::mString [protected]
```

The documentation for this class was generated from the following file:

- [AAX_CString.h](#)

14.39 AAX_CStringAbbreviations Class Reference

```
#include <AAX_CString.h>
```

14.39.1 Description

Helper class to store a collection of name abbreviations.

Public Member Functions

- [AAX_CStringAbbreviations](#) (const [AAX_CString](#) &inPrimary)
- void [SetPrimary](#) (const [AAX_CString](#) &inPrimary)
- const [AAX_CString](#) & [Primary](#) () const
- void [Add](#) (const [AAX_CString](#) &inAbbreviation)
- const [AAX_CString](#) & [Get](#) (int32_t inNumCharacters) const
- void [Clear](#) ()

14.39.2 Constructor & Destructor Documentation

14.39.2.1 AAX_CStringAbbreviations()

```
AAX_CStringAbbreviations::AAX_CStringAbbreviations (
    const AAX\_CString & inPrimary ) [inline], [explicit]
```

14.39.3 Member Function Documentation

14.39.3.1 SetPrimary()

```
void AAX_CStringAbbreviations::SetPrimary (
    const AAX\_CString & inPrimary ) [inline]
```

Referenced by `AAX_CStatelessParameter::SetName()`.

Here is the caller graph for this function:

14.39.3.2 Primary()

```
const AAX\_CString& AAX_CStringAbbreviations::Primary ( ) const [inline]
```

Referenced by `AAX_CStatelessParameter::Name()`.

Here is the caller graph for this function:

14.39.3.3 Add()

```
void AAX_CStringAbbreviations::Add (
    const AAX\_CString & inAbbreviation ) [inline]
```

References `AAX_CString::Length()`.

Referenced by `AAX_CStatelessParameter::AddShortenedName()`.

Here is the call graph for this function: Here is the caller graph for this function:

14.39.3.4 Get()

```
const AAX\_CString& AAX_CStringAbbreviations::Get (
    int32_t inNumCharacters ) const [inline]
```

References `AAX_CString::Length()`.

Referenced by `AAX_CStatelessParameter::ShortenedName()`.

Here is the call graph for this function: Here is the caller graph for this function:

14.39.3.5 Clear()

```
void AAX_CStringAbbreviations::Clear ( ) [inline]
```

Referenced by `AAX_CStatelessParameter::ClearShortenedNames()`.

Here is the caller graph for this function:

The documentation for this class was generated from the following file:

- [AAX_CString.h](#)

14.40 AAX_CStringDataBuffer Class Reference

```
#include <AAX_CStringDataBuffer.h>
```

Inheritance diagram for AAX_CStringDataBuffer:

Collaboration diagram for AAX_CStringDataBuffer:

14.40.1 Description

A convenience class for string data buffers.

The data payload is a `char*` C string

Public Member Functions

- [AAX_CStringDataBuffer](#) ([AAX_CTypeID](#) inType, std::string const &inData)
- [AAX_CStringDataBuffer](#) ([AAX_CTypeID](#) inType, std::string &&inData)
- [AAX_CStringDataBuffer](#) ([AAX_CTypeID](#) inType, const char *inData)
- [AAX_CStringDataBuffer](#) ([AAX_CStringDataBuffer](#) const &)=delete
- [AAX_CStringDataBuffer](#) ([AAX_CStringDataBuffer](#) &&)=delete
- [~AAX_CStringDataBuffer](#) (void) [AAX_OVERRIDE](#)=default
- [AAX_CStringDataBuffer](#) & [operator=](#) ([AAX_CStringDataBuffer](#) const &other)=delete
- [AAX_CStringDataBuffer](#) & [operator=](#) ([AAX_CStringDataBuffer](#) &&other)=delete
- [AAX_Result Type](#) ([AAX_CTypeID](#) *oType) const [AAX_OVERRIDE](#)
- [AAX_Result Size](#) (int32_t *oSize) const [AAX_OVERRIDE](#)
- [AAX_Result Data](#) (void const **oBuffer) const [AAX_OVERRIDE](#)

Additional Inherited Members

14.40.2 Constructor & Destructor Documentation

14.40.2.1 AAX_CStringDataBuffer() [1/5]

```
AAX_CStringDataBuffer::AAX_CStringDataBuffer (
    AAX\_CTypeID inType,
    std::string const & inData ) [inline]
```

14.40.2.2 AAX_CStringDataBuffer() [2/5]

```
AAX_CStringDataBuffer::AAX_CStringDataBuffer (
    AAX\_CTypeID inType,
    std::string && inData ) [inline]
```

14.40.2.3 AAX_CStringDataBuffer() [3/5]

```
AAX_CStringDataBuffer::AAX_CStringDataBuffer (
    AAX_CTypeID inType,
    const char * inData ) [inline]
```

14.40.2.4 AAX_CStringDataBuffer() [4/5]

```
AAX_CStringDataBuffer::AAX_CStringDataBuffer (
    AAX_CStringDataBuffer const & ) [delete]
```

14.40.2.5 AAX_CStringDataBuffer() [5/5]

```
AAX_CStringDataBuffer::AAX_CStringDataBuffer (
    AAX_CStringDataBuffer && ) [delete]
```

14.40.2.6 ~AAX_CStringDataBuffer()

```
AAX_CStringDataBuffer::~~AAX_CStringDataBuffer (
    void ) [default]
```

14.40.3 Member Function Documentation

14.40.3.1 operator=() [1/2]

```
AAX_CStringDataBuffer& AAX_CStringDataBuffer::operator= (
    AAX_CStringDataBuffer const & other ) [delete]
```

14.40.3.2 operator=() [2/2]

```
AAX_CStringDataBuffer& AAX_CStringDataBuffer::operator= (
    AAX_CStringDataBuffer && other ) [delete]
```


14.40.3.3 Type()

```
AAX_Result AAX_CStringDataBuffer::Type (
    AAX_CTypeID * oType ) const [inline], [virtual]
```

The type of data contained in this buffer

This identifier must be sufficient for a client that knows the type to correctly interpret and use the data.

Implements [AAX_IACFDataBuffer](#).

References AAX_ERROR_NULL_ARGUMENT, and AAX_SUCCESS.

14.40.3.4 Size()

```
AAX_Result AAX_CStringDataBuffer::Size (
    int32_t * oSize ) const [inline], [virtual]
```

The number of bytes of data in this buffer

Implements [AAX_IACFDataBuffer](#).

References AAX_ERROR_NULL_ARGUMENT, AAX_ERROR_SIGNED_INT_OVERFLOW, and AAX_SUCCESS.

14.40.3.5 Data()

```
AAX_Result AAX_CStringDataBuffer::Data (
    void const ** oBuffer ) const [inline], [virtual]
```

The buffer of data

Implements [AAX_IACFDataBuffer](#).

References AAX_ERROR_NULL_ARGUMENT, and AAX_SUCCESS.

The documentation for this class was generated from the following file:

- [AAX_CStringDataBuffer.h](#)

14.41 AAX_CStringDataBufferOfType< T > Class Template Reference

```
#include <AAX_CStringDataBuffer.h>
```

Inheritance diagram for AAX_CStringDataBufferOfType< T >:

Collaboration diagram for AAX_CStringDataBufferOfType< T >:

14.41.1 Description

```
template<AAX_CTypeID T>
class AAX_CStringDataBufferOfType< T >
```

A convenience class for string data buffers.

The data payload is a `char*` C string

Public Member Functions

- [AAX_CStringDataBufferOfType](#) (std::string const &inData)
- [AAX_CStringDataBufferOfType](#) (std::string &&inData)
- [AAX_CStringDataBufferOfType](#) (const char *inData)
- [AAX_CStringDataBufferOfType](#) ([AAX_CStringDataBufferOfType](#) const &)=delete
- [AAX_CStringDataBufferOfType](#) ([AAX_CStringDataBufferOfType](#) &&)=delete
- [~AAX_CStringDataBufferOfType](#) (void) [AAX_OVERRIDE](#)=default
- [AAX_CStringDataBufferOfType](#) & [operator=](#) ([AAX_CStringDataBufferOfType](#) const &other)=delete
- [AAX_CStringDataBufferOfType](#) & [operator=](#) ([AAX_CStringDataBufferOfType](#) &&other)=delete
- [AAX_Result Type](#) ([AAX_CTypeID](#) *oType) const [AAX_OVERRIDE](#)
- [AAX_Result Size](#) (int32_t *oSize) const [AAX_OVERRIDE](#)
- [AAX_Result Data](#) (void const **oBuffer) const [AAX_OVERRIDE](#)

Additional Inherited Members

14.41.2 Constructor & Destructor Documentation

14.41.2.1 AAX_CStringDataBufferOfType() [1/5]

```
template<AAX_CTypeID T>
AAX_CStringDataBufferOfType< T >::AAX_CStringDataBufferOfType (
    std::string const & inData ) [inline], [explicit]
```

14.41.2.2 AAX_CStringDataBufferOfType() [2/5]

```
template<AAX_CTypeID T>
AAX_CStringDataBufferOfType< T >::AAX_CStringDataBufferOfType (
    std::string && inData ) [inline], [explicit]
```

14.41.2.3 AAX_CStringDataBufferOfType() [3/5]

```
template<AAX_CTypeID T>
AAX_CStringDataBufferOfType< T >::AAX_CStringDataBufferOfType (
    const char * inData ) [inline], [explicit]
```

14.41.2.4 AAX_CStringDataBufferOfType() [4/5]

```
template<AAX_CTypeID T>
AAX_CStringDataBufferOfType< T >::AAX_CStringDataBufferOfType (
    AAX_CStringDataBufferOfType< T > const & ) [delete]
```

14.41.2.5 AAX_CStringDataBufferOfType() [5/5]

```
template<AAX_CTypeID T>
AAX_CStringDataBufferOfType< T >::AAX_CStringDataBufferOfType (
    AAX_CStringDataBufferOfType< T > && ) [delete]
```

14.41.2.6 ~AAX_CStringDataBufferOfType()

```
template<AAX_CTypeID T>
AAX_CStringDataBufferOfType< T >::~~AAX_CStringDataBufferOfType (
    void ) [default]
```

14.41.3 Member Function Documentation**14.41.3.1 operator=()** [1/2]

```
template<AAX_CTypeID T>
AAX_CStringDataBufferOfType& AAX_CStringDataBufferOfType< T >::operator= (
    AAX_CStringDataBufferOfType< T > const & other ) [delete]
```

14.41.3.2 operator=() [2/2]

```
template<AAX_CTypeID T>
AAX_CStringDataBufferOfType& AAX_CStringDataBufferOfType< T >::operator= (
    AAX_CStringDataBufferOfType< T > && other ) [delete]
```

14.41.3.3 Type()

```
template<AAX_CTypeID T>
AAX_Result AAX_CStringDataBufferOfType< T >::Type (
    AAX_CTypeID * oType ) const [inline], [virtual]
```

The type of data contained in this buffer

This identifier must be sufficient for a client that knows the type to correctly interpret and use the data.

Implements [AAX_IACFDataBuffer](#).

References AAX_ERROR_NULL_ARGUMENT, and AAX_SUCCESS.

14.41.3.4 Size()

```
template<AAX_CTypeID T>
AAX_Result AAX_CStringDataBufferOfType< T >::Size (
    int32_t * oSize ) const [inline], [virtual]
```

The number of bytes of data in this buffer

Implements [AAX_IACFDataBuffer](#).

References AAX_ERROR_NULL_ARGUMENT, AAX_ERROR_SIGNED_INT_OVERFLOW, and AAX_SUCCESS.

14.41.3.5 Data()

```
template<AAX_CTypeID T>
AAX_Result AAX_CStringDataBufferOfType< T >::Data (
    void const ** oBuffer ) const [inline], [virtual]
```

The buffer of data

Implements [AAX_IACFDataBuffer](#).

References AAX_ERROR_NULL_ARGUMENT, and AAX_SUCCESS.

The documentation for this class was generated from the following file:

- [AAX_CStringDataBuffer.h](#)

14.42 AAX_CStringDisplayDelegate< T > Class Template Reference

```
#include <AAX_CStringDisplayDelegate.h>
```

Inheritance diagram for AAX_CStringDisplayDelegate< T >:

Collaboration diagram for AAX_CStringDisplayDelegate< T >:

14.42.1 Description

```
template<typename T>
class AAX_CStringDisplayDelegate< T >
```

A string, or list, display format conforming to [AAX_IDisplayDelegate](#).

This display delegate uses a string map to associate parameter values with specific strings. This kind of display delegate is most often used for control string or list parameters, which would internally use an integer parameter type. The int value would then be used as a lookup into this delegate, which would return a string for each valid int value.

Public Member Functions

- [AAX_CStringDisplayDelegate](#) (const std::map< T, [AAX_CString](#) > &stringMap)
Constructor.
- [AAX_CStringDisplayDelegate](#)< T > * [Clone](#) () const [AAX_OVERRIDE](#)
Constructs and returns a copy of the display delegate.
- bool [ValueToString](#) (T value, [AAX_CString](#) *valueString) const [AAX_OVERRIDE](#)
Converts a real parameter value to a string representation.
- bool [ValueToString](#) (T value, int32_t maxNumChars, [AAX_CString](#) *valueString) const [AAX_OVERRIDE](#)
Converts a real parameter value to a string representation using a size hint, useful for control surfaces and other character limited displays.
- bool [StringToValue](#) (const [AAX_CString](#) &valueString, T *value) const [AAX_OVERRIDE](#)
Converts a string to a real parameter value.

Protected Attributes

- std::map< T, [AAX_CString](#) > [mStringMap](#)
- std::map< [AAX_CString](#), T > [mInverseStringMap](#)

14.42.2 Constructor & Destructor Documentation

14.42.2.1 AAX_CStringDisplayDelegate()

```
template<typename T >
AAX_CStringDisplayDelegate< T >::AAX_CStringDisplayDelegate (
    const std::map< T, AAX\_CString > & stringMap )
```

Constructor.

Constructs a String Display Delegate with a provided string map.

Note

The string map should already be populated with value-string pairs, as this constructor will copy the provided map into the delegate object's own memory.

Parameters

in	<i>stringMap</i>	A populated map of value-string pairs
----	------------------	---------------------------------------

References [AAX_CStringDisplayDelegate< T >::mInverseStringMap](#), and [AAX_CStringDisplayDelegate< T >::mStringMap](#).

14.42.3 Member Function Documentation

14.42.3.1 Clone()

```
template<typename T >
AAX_CStringDisplayDelegate< T > * AAX_CStringDisplayDelegate< T >::Clone ( ) const [virtual]
```

Constructs and returns a copy of the display delegate.

In general, this method's implementation can use a simple copy constructor:

```
template <typename T>
AAX_CSubclassDisplayDelegate<T>*    AAX_CSubclassDisplayDelegate<T>::Clone () const
{
    return new AAX_CSubclassDisplayDelegate (*this);
}
```

Implements [AAX_IDisplayDelegate< T >](#).

14.42.3.2 ValueToString() [1/2]

```
template<typename T >
bool AAX_CStringDisplayDelegate< T >::ValueToString (
    T value,
    AAX_CString * valueString ) const [virtual]
```

Converts a real parameter value to a string representation.

Parameters

in	<i>value</i>	The real parameter value that will be converted
out	<i>valueString</i>	A string corresponding to value

Return values

<i>true</i>	The string conversion was successful
<i>false</i>	The string conversion was unsuccessful

Implements [AAX_IDisplayDelegate< T >](#).

14.42.3.3 ValueToString() [2/2]

```
template<typename T >
bool AAX_CStringDisplayDelegate< T >::ValueToString (
    T value,
    int32_t maxNumChars,
    AAX_CString * valueString ) const [virtual]
```

Converts a real parameter value to a string representation using a size hint, useful for control surfaces and other character limited displays.

Parameters

in	<i>value</i>	The real parameter value that will be converted
in	<i>maxNumChars</i>	Size hint for the desired maximum number of characters in the string (not including null termination)
out	<i>valueString</i>	A string corresponding to value

Return values

<i>true</i>	The string conversion was successful
<i>false</i>	The string conversion was unsuccessful

Implements [AAX_IDisplayDelegate< T >](#).

14.42.3.4 StringToValue()

```
template<typename T >
bool AAX_CStringDisplayDelegate< T >::StringToValue (
    const AAX_CString & valueString,
    T * value ) const [virtual]
```

Converts a string to a real parameter value.

Parameters

in	<i>valueString</i>	The string that will be converted
out	<i>value</i>	The real parameter value corresponding to valueString

Return values

<i>true</i>	The string conversion was successful
<i>false</i>	The string conversion was unsuccessful

Implements [AAX_IDisplayDelegate< T >](#).

14.42.4 Member Data Documentation

14.42.4.1 mStringMap

```
template<typename T >
std::map<T, AAX_CString> AAX_CStringDisplayDelegate< T >::mStringMap [protected]
```

Referenced by `AAX_CStringDisplayDelegate< T >::AAX_CStringDisplayDelegate()`.

14.42.4.2 mInverseStringMap

```
template<typename T >
std::map<AAX_CString, T> AAX_CStringDisplayDelegate< T >::mInverseStringMap [protected]
```

Referenced by `AAX_CStringDisplayDelegate< T >::AAX_CStringDisplayDelegate()`.

The documentation for this class was generated from the following file:

- [AAX_CStringDisplayDelegate.h](#)

14.43 AAX_CTask Class Reference

```
#include <AAX_CTask.h>
```

Inheritance diagram for `AAX_CTask`:

Collaboration diagram for `AAX_CTask`:

Public Member Functions

- [ACF_DECLARE_STANDARD_UNKNOWN](#) () ACFMETHOD(InternalQueryInterface)(const [acfiID](#) &riid
- [AAX_DELETE](#) ([AAX_CTask](#) &operator=(const [AAX_CTask](#) &))
- [AAX_CTask](#) ([AAX_CTypeID](#) iType)
- [AAX_DEFAULT_DTOR_OVERRIDE](#) ([AAX_CTask](#))
- [AAX_Result](#) [GetType](#) ([AAX_CTypeID](#) *oType) const [AAX_OVERRIDE](#)
- [AAX_IACFDataBuffer](#) const * [GetArgumentOfType](#) ([AAX_CTypeID](#) iType) const [AAX_OVERRIDE](#)
- [AAX_Result](#) [SetProgress](#) (float iProgress) [AAX_OVERRIDE](#)
- float [GetProgress](#) () const [AAX_OVERRIDE](#)
- [AAX_Result](#) [AddResult](#) ([AAX_IACFDataBuffer](#) const *iResult) [AAX_OVERRIDE](#)
Attach result data to this task.
- [AAX_Result](#) [SetDone](#) ([AAX_TaskCompletionStatus](#) iStatus) [AAX_OVERRIDE](#)
Inform the host that the task is completed.
- [AAX_TaskCompletionStatus](#) [Status](#) () const

Public Attributes

- void **ppvObjOut [AAX_OVERRIDE](#)

14.43.1 Constructor & Destructor Documentation

14.43.1.1 AAX_CTask()

```
AAX_CTask::AAX_CTask (
    AAX\_CTypeID iType ) [explicit]
```

14.43.2 Member Function Documentation

14.43.2.1 ACF_DECLARE_STANDARD_UNKNOWN()

```
AAX_CTask::ACF_DECLARE_STANDARD_UNKNOWN ( ) const &
```

14.43.2.2 AAX_DELETE()

```
AAX_CTask::AAX_DELETE (
    AAX\_CTask & operator = (const AAX\_CTask &) )
```

14.43.2.3 AAX_DEFAULT_DTOR_OVERRIDE()

```
AAX_CTask::AAX_DEFAULT_DTOR_OVERRIDE (
    AAX\_CTask )
```

14.43.2.4 GetType()

```
AAX\_Result AAX_CTask::GetType (
    AAX\_CTypeID * oType ) const [virtual]
```

An identifier defining the type of the requested task

Parameters

out	<i>oType</i>	The type of this task request
-----	--------------	-------------------------------

Implements [AAX_IACFTask](#).

14.43.2.5 GetArgumentOfType()

```
AAX_IACFDataBuffer const* AAX_CTask::GetArgumentOfType (
    AAX_CTypeID iType ) const [virtual]
```

Additional information defining the request, depending on the task type

Parameters

in	<i>iType</i>	The type of argument requested. Possible argument types, if any, and the resulting data buffer format must be defined per task type.
----	--------------	--

Returns

The requested argument data, or nullptr. This data buffer's type ID is expected to match *iType* . The caller takes ownership of this object.

Implements [AAX_IACFTask](#).

14.43.2.6 SetProgress()

```
AAX_Result AAX_CTask::SetProgress (
    float iProgress ) [virtual]
```

Inform the host about the current status of the task

Parameters

in	<i>iProgress</i>	A value between 0 (no progress) and 1 (complete)
----	------------------	--

Implements [AAX_IACFTask](#).

14.43.2.7 GetProgress()

```
float AAX_CTask::GetProgress ( ) const [virtual]
```

Returns the current progress

Implements [AAX_IACFTask](#).

14.43.2.8 AddResult()

```
AAX_Result AAX_CTask::AddResult (
    AAX_IACFDataBuffer const * iResult ) [virtual]
```

Attach result data to this task.

This can be called multiple times to add multiple types of results to a single task.

The host may process the result data immediately or may wait for the task to complete.

The plug-in is expected to release the data buffer upon making this call. At a minimum, the data buffer must not be changed after this call is made. See `ACFPtr::inArg()`

Parameters

in	<i>iResult</i>	A buffer containing the result data. Expected result types, if any, and their data buffer format must be defined per task type.
----	----------------	---

Implements [AAX_IACFTask](#).

14.43.2.9 SetDone()

```
AAX_Result AAX_CTask::SetDone (
    AAX_TaskCompletionStatus iStatus ) [virtual]
```

Inform the host that the task is completed.

If [AAX_SUCCESS](#) is returned, the object should be considered invalid and released by the caller.

Parameters

in	<i>iStatus</i>	The final status of the task. This indicates to the host whether or not the task was performed as requested.
----	----------------	--

Implements [AAX_IACFTask](#).

14.43.2.10 Status()

```
AAX_TaskCompletionStatus AAX_CTask::Status ( ) const [inline]
```

14.43.3 Member Data Documentation

14.43.3.1 AAX_OVERRIDE

```
void** ppvObjOut AAX_CTask::AAX_OVERRIDE
```

The documentation for this class was generated from the following file:

- [AAX_CTask.h](#)

14.44 AAX_CTaskAgent Class Reference

```
#include <AAX_CTaskAgent.h>
```

Inheritance diagram for AAX_CTaskAgent:

Collaboration diagram for AAX_CTaskAgent:

14.44.1 Description

Default implementation of the [AAX_ITaskAgent](#) interface.

This class provides a default implementation of the [AAX_ITaskAgent](#) interface. Your plug-in's task agent implementation should inherit from this class and override the remaining interface functions.

Public Member Functions

- [AAX_CTaskAgent](#) (void)=default
- [~AAX_CTaskAgent](#) (void) [AAX_OVERRIDE](#)

Initialization and uninitialization

- [AAX_Result Initialize](#) (IACFUnknown *iController) [AAX_OVERRIDE](#)
- [AAX_Result Uninitialize](#) (void) [AAX_OVERRIDE](#)

Private member accessors

- [AAX_IController](#) * [GetController](#) (void)
Returns a pointer to the plug-in's controller interface.
- [AAX_IEffectParameters](#) * [GetEffectParameters](#) (void)
Returns a pointer to the plug-in's data model interface.

Task management

- [AAX_Result AddTask](#) ([IACFUnknown](#) *iTask) [AAX_OVERRIDE](#)
Default implemenation of [AddTask\(\)](#)
- [AAX_Result CancelAllTasks](#) () [AAX_OVERRIDE](#)
- virtual [AAX_Result AddTask](#) (std::unique_ptr< [AAX_ITask](#) > iTask)
Convenience method for adding versioned tasks.
- virtual [AAX_Result ReceiveTask](#) (std::unique_ptr< [AAX_ITask](#) > iTask)
Convenience method for adding versioned tasks.

Additional Inherited Members

14.44.2 Constructor & Destructor Documentation

14.44.2.1 AAX_CTaskAgent()

```
AAX_CTaskAgent::AAX_CTaskAgent (
    void ) [default]
```

14.44.2.2 ~AAX_CTaskAgent()

```
AAX_CTaskAgent::~~AAX_CTaskAgent (
    void )
```

14.44.3 Member Function Documentation

14.44.3.1 Initialize()

```
AAX\_Result AAX_CTaskAgent::Initialize (
    IACFUnknown * iController ) [virtual]
```

Initialize the object

Parameters

in	<i>iController</i>	Interface allowing access to other objects in the object graph such as the plug-in's data model.
----	--------------------	--

Implements [AAX_IACFTaskAgent](#).

14.44.3.2 Uninitialize()

```
AAX_Result AAX_CTaskAgent::Uninitialize (
    void ) [virtual]
```

Uninitialize the object

This method should release references to any shared objects

Implements [AAX_IACFTaskAgent](#).

14.44.3.3 AddTask() [1/2]

```
AAX_Result AAX_CTaskAgent::AddTask (
    IACFUnknown * iTask ) [virtual]
```

Default implemenation of [AddTask\(\)](#)

Convenience implementation that converts the [IACFUnknown](#) into an [AAX_ITask](#) . Implementations should override the version that provides an [AAX_ITask](#) object.

Implements [AAX_IACFTaskAgent](#).

14.44.3.4 CancelAllTasks()

```
AAX_Result AAX_CTaskAgent::CancelAllTasks ( ) [virtual]
```

Request that the agent cancel all outstanding tasks

Implements [AAX_IACFTaskAgent](#).

14.44.3.5 AddTask() [2/2]

```
virtual AAX_Result AAX_CTaskAgent::AddTask (
    std::unique_ptr< AAX_ITask > iTask ) [protected], [virtual]
```

Convenience method for adding versioned tasks.

Deprecated Use [ReceiveTask\(\)](#) instead

14.44.3.6 ReceiveTask()

```
virtual AAX_Result AAX_CTaskAgent::ReceiveTask (
    std::unique_ptr< AAX_ITask > iTask ) [protected], [virtual]
```

Convenience method for adding versioned tasks.

14.44.3.7 GetController()

```
AAX_IController* AAX_CTaskAgent::GetController (
    void ) [inline]
```

Returns a pointer to the plug-in's controller interface.

14.44.3.8 GetEffectParameters()

```
AAX_IEffectParameters* AAX_CTaskAgent::GetEffectParameters (
    void ) [inline]
```

Returns a pointer to the plug-in's data model interface.

The documentation for this class was generated from the following file:

- [AAX_CTaskAgent.h](#)

14.45 AAX_CTempoBreakpoint Struct Reference

```
#include <AAX_SessionDocumentTypes.h>
```

Public Attributes

- int64_t [mSampleLocation](#) {0}
- float [mValue](#) {0.f}

14.45.1 Member Data Documentation

14.45.1.1 mSampleLocation

```
int64_t AAX_CTempoBreakpoint::mSampleLocation {0}
```

14.45.1.2 mValue

```
float AAX_CTempoBreakpoint::mValue {0.f}
```

The documentation for this struct was generated from the following file:

- [AAX_SessionDocumentTypes.h](#)

14.46 AAX_CUnitDisplayDelegateDecorator< T > Class Template Reference

```
#include <AAX_CUnitDisplayDelegateDecorator.h>
```

Inheritance diagram for AAX_CUnitDisplayDelegateDecorator< T >:

Collaboration diagram for AAX_CUnitDisplayDelegateDecorator< T >:

14.46.1 Description

```
template<typename T>
class AAX_CUnitDisplayDelegateDecorator< T >
```

A unit type decorator conforming to [AAX_IDisplayDelegateDecorator](#).

This class is an [AAX_IDisplayDelegateDecorator](#), meaning that it acts as a wrapper for other display delegates or concrete display types. For more information about display delegate decorators in [AAX](#), see [Display delegate decorators](#)

The behavior of this class is to decorate parameter value strings with arbitrary units, such as "Hz" or "V". The inverse is also supported, so the unit string is pulled off of value strings when they are converted to real parameter values.

Public Member Functions

- [AAX_CUnitDisplayDelegateDecorator](#) (const [AAX_IDisplayDelegate](#)< T > &displayDelegate, const [AAX_CString](#) &unitString)
Constructor.
- [AAX_CUnitDisplayDelegateDecorator](#)< T > * [Clone](#) () const [AAX_OVERRIDE](#)
Constructs and returns a copy of the display delegate.
- bool [ValueToString](#) (T value, [AAX_CString](#) *valueString) const [AAX_OVERRIDE](#)
Converts a real parameter value to a string representation.
- bool [ValueToString](#) (T value, int32_t maxNumChars, [AAX_CString](#) *valueString) const [AAX_OVERRIDE](#)
Converts a real parameter value to a string representation using a size hint, useful for control surfaces and other character limited displays.
- bool [StringToValue](#) (const [AAX_CString](#) &valueString, T *value) const [AAX_OVERRIDE](#)
Converts a string to a real parameter value.

Protected Attributes

- const [AAX_CString](#) mUnitString

14.46.2 Constructor & Destructor Documentation

14.46.2.1 AAX_CUnitDisplayDelegateDecorator()

```
template<typename T >
AAX_CUnitDisplayDelegateDecorator< T >::AAX_CUnitDisplayDelegateDecorator (
    const AAX_IDisplayDelegate< T > & displayDelegate,
    const AAX_CString & unitString )
```

Constructor.

Along with the standard decorator pattern argument, this class also takes a unit string. This is the string that will be added to the end of valueString.

Parameters

in	<i>displayDelegate</i>	
in	<i>unitString</i>	

14.46.3 Member Function Documentation

14.46.3.1 Clone()

```
template<typename T >
AAX_CUnitDisplayDelegateDecorator< T > * AAX_CUnitDisplayDelegateDecorator< T >::Clone ( )
const [virtual]
```

Constructs and returns a copy of the display delegate.

In general, this method's implementation can use a simple copy constructor:

```
template <typename T>
AAX_CSubclassDisplayDelegate<T>*    AAX_CSubclassDisplayDelegate<T>::Clone() const
{
    return new AAX_CSubclassDisplayDelegate(*this);
}
```

Implements [AAX_IDisplayDelegate< T >](#).

14.46.3.2 ValueToString() [1/2]

```
template<typename T >
bool AAX_CUnitDisplayDelegateDecorator< T >::ValueToString (
    T value,
    AAX_CString * valueString ) const [virtual]
```

Converts a real parameter value to a string representation.

Parameters

in	<i>value</i>	The real parameter value that will be converted
out	<i>valueString</i>	A string corresponding to value

Return values

<i>true</i>	The string conversion was successful
<i>false</i>	The string conversion was unsuccessful

Implements [AAX_IDisplayDelegate< T >](#).

References [AAX_IDisplayDelegateDecorator< T >::ValueToString\(\)](#).

Here is the call graph for this function:

14.46.3.3 ValueToString() [2/2]

```
template<typename T >
bool AAX_CUnitDisplayDelegateDecorator< T >::ValueToString (
    T value,
    int32_t maxNumChars,
    AAX_CString * valueString ) const [virtual]
```

Converts a real parameter value to a string representation using a size hint, useful for control surfaces and other character limited displays.

Parameters

in	<i>value</i>	The real parameter value that will be converted
in	<i>maxNumChars</i>	Size hint for the desired maximum number of characters in the string (not including null termination)
out	<i>valueString</i>	A string corresponding to value

Return values

<i>true</i>	The string conversion was successful
<i>false</i>	The string conversion was unsuccessful

Implements [AAX_IDisplayDelegate< T >](#).

References [AAX_CString::Length\(\)](#), and [AAX_IDisplayDelegateDecorator< T >::ValueToString\(\)](#).

Here is the call graph for this function:

14.46.3.4 StringToValue()

```
template<typename T >
bool AAX_CUnitDisplayDelegateDecorator< T >::StringToValue (
```

```
const AAX_CString & valueString,
T * value ) const [virtual]
```

Converts a string to a real parameter value.

Parameters

in	<i>valueString</i>	The string that will be converted
out	<i>value</i>	The real parameter value corresponding to valueString

Return values

<i>true</i>	The string conversion was successful
<i>false</i>	The string conversion was unsuccessful

Implements [AAX_IDisplayDelegate< T >](#).

References [AAX_CString::Length\(\)](#), [AAX_IDisplayDelegateDecorator< T >::StringToValue\(\)](#), and [AAX_CString::SubString\(\)](#).

Here is the call graph for this function:

14.46.4 Member Data Documentation

14.46.4.1 mUnitString

```
template<typename T >
const AAX_CString AAX_CUnitDisplayDelegateDecorator< T >::mUnitString [protected]
```

The documentation for this class was generated from the following file:

- [AAX_CUnitDisplayDelegateDecorator.h](#)

14.47 AAX_CUnitPrefixDisplayDelegateDecorator< T > Class Template Reference

```
#include <AAX_CUnitPrefixDisplayDelegateDecorator.h>
```

Inheritance diagram for [AAX_CUnitPrefixDisplayDelegateDecorator< T >](#):

Collaboration diagram for [AAX_CUnitPrefixDisplayDelegateDecorator< T >](#):

14.47.1 Description

```
template<typename T>
class AAX_CUnitPrefixDisplayDelegateDecorator< T >
```

A unit prefix decorator conforming to [AAX_IDisplayDelegateDecorator](#).

This class is an [AAX_IDisplayDelegateDecorator](#), meaning that it acts as a wrapper for other display delegates or concrete display types. For more information about display delegate decorators in [AAX](#), see [Display delegate decorators](#)

The behavior of this class is to provide unit prefixes such as the k in kHz or the m in mm. It takes the value passed in and determines if the value is large or small enough to benefit from a unit modifier. If so, it adds that unit prefix character to the display string after scaling the number and calling deeper into the decorator pattern to get the concrete [ValueToString\(\)](#) result.

The inverse is also supported, so if you type 1.5k in a text box and this decorator is in place, it should find the k and multiply the value by 1000 before converting it to a real value.

This decorator supports the following unit prefixes:

- M (mega-)
- k (kilo-)
- m (milli-)
- u (micro-)

Note

This class is not implemented for integer values as the conversions result in fractional numbers. Those would get truncated through the system and be pretty much useless.

Public Member Functions

- [AAX_CUnitPrefixDisplayDelegateDecorator](#) (const [AAX_IDisplayDelegate](#)< T > &displayDelegate)
- [AAX_CUnitPrefixDisplayDelegateDecorator](#)< T > * [Clone](#) () const [AAX_OVERRIDE](#)
Constructs and returns a copy of the display delegate.
- bool [ValueToString](#) (T value, [AAX_CString](#) *valueString) const [AAX_OVERRIDE](#)
Converts a real parameter value to a string representation.
- bool [ValueToString](#) (T value, int32_t maxNumChars, [AAX_CString](#) *valueString) const [AAX_OVERRIDE](#)
Converts a real parameter value to a string representation using a size hint, useful for control surfaces and other character limited displays.
- bool [StringToValue](#) (const [AAX_CString](#) &valueString, T *value) const [AAX_OVERRIDE](#)
Converts a string to a real parameter value.

14.47.2 Constructor & Destructor Documentation

14.47.2.1 AAX_CUnitPrefixDisplayDelegateDecorator()

```
template<typename T >
AAX_CUnitPrefixDisplayDelegateDecorator< T >::AAX_CUnitPrefixDisplayDelegateDecorator (
    const AAX_IDisplayDelegate< T > & displayDelegate )
```

14.47.3 Member Function Documentation

14.47.3.1 Clone()

```
template<typename T >
AAX_CUnitPrefixDisplayDelegateDecorator< T > * AAX_CUnitPrefixDisplayDelegateDecorator< T >↔
::Clone ( ) const [virtual]
```

Constructs and returns a copy of the display delegate.

In general, this method's implementation can use a simple copy constructor:

```
template <typename T>
AAX_CSubclassDisplayDelegate<T>*    AAX_CSubclassDisplayDelegate<T>::Clone() const
{
    return new AAX_CSubclassDisplayDelegate(*this);
}
```

Implements [AAX_IDisplayDelegate< T >](#).

14.47.3.2 ValueToString() [1/2]

```
template<typename T >
bool AAX_CUnitPrefixDisplayDelegateDecorator< T >::ValueToString (
    T value,
    AAX_CString * valueString ) const [virtual]
```

Converts a real parameter value to a string representation.

Parameters

in	<i>value</i>	The real parameter value that will be converted
out	<i>valueString</i>	A string corresponding to value

Return values

<i>true</i>	The string conversion was successful
<i>false</i>	The string conversion was unsuccessful

Implements [AAX_IDisplayDelegate< T >](#).

References [AAX::fabsf\(\)](#), and [AAX_IDisplayDelegateDecorator< T >::ValueToString\(\)](#).

Here is the call graph for this function:

14.47.3.3 ValueToString() [2/2]

```
template<typename T >
bool AAX_CUnitPrefixDisplayDelegateDecorator< T >::ValueToString (
    T value,
    int32_t maxNumChars,
    AAX_CString * valueString ) const [virtual]
```

Converts a real parameter value to a string representation using a size hint, useful for control surfaces and other character limited displays.

Parameters

in	<i>value</i>	The real parameter value that will be converted
in	<i>maxNumChars</i>	Size hint for the desired maximum number of characters in the string (not including null termination)
out	<i>valueString</i>	A string corresponding to value

Return values

<i>true</i>	The string conversion was successful
<i>false</i>	The string conversion was unsuccessful

Implements [AAX_IDisplayDelegate< T >](#).

References [AAX::fabsf\(\)](#), and [AAX_IDisplayDelegateDecorator< T >::ValueToString\(\)](#).

Here is the call graph for this function:

14.47.3.4 StringToValue()

```
template<typename T >
bool AAX_CUnitPrefixDisplayDelegateDecorator< T >::StringToValue (
    const AAX_CString & valueString,
    T * value ) const [virtual]
```

Converts a string to a real parameter value.

Parameters

in	<i>valueString</i>	The string that will be converted
out	<i>value</i>	The real parameter value corresponding to valueString

Return values

<i>true</i>	The string conversion was successful
<i>false</i>	The string conversion was unsuccessful

Implements [AAX_IDisplayDelegate< T >](#).

References [AAX_CString::Erase\(\)](#), [AAX_CString::Length\(\)](#), and [AAX_IDisplayDelegateDecorator< T >::String↔ToValue\(\)](#).

Here is the call graph for this function:

The documentation for this class was generated from the following file:

- [AAX_CUnitPrefixDisplayDelegateDecorator.h](#)

14.48 AAX_FastInterpolatedTableLookup< TFLOAT, DFLOAT > Class Template Reference

```
#include <AAX_FastInterpolatedTableLookup.h>
```

Public Member Functions

- void [SetParameters](#) (int iTableSize, TFLOAT iMin=0.0, TFLOAT iMax=1.0, int iNumTables=1)
Set the table lookup parameters.
- DFLOAT [DoTableLookupExtraFast](#) (const TFLOAT *const iTable, DFLOAT iValue) const
Perform an extra fast table lookup :)
- void [DoTableLookupExtraFastMulti](#) (const TFLOAT *iTable, DFLOAT iValue, DFLOAT *oValues) const
- void [DoTableLookupExtraFast](#) (const TFLOAT *const iTable, const TFLOAT *const inpBuf, DFLOAT *const outBuf, int blockSize)
- TFLOAT [GetMin](#) ()
- TFLOAT [GetMaxMinusMin](#) ()

14.48.1 Member Function Documentation

14.48.1.1 SetParameters()

```
template<class TFLOAT , class DFLOAT >
void AAX_FastInterpolatedTableLookup< TFLOAT, DFLOAT >::SetParameters (
    int iTableSize,
    TFLOAT iMin = 0.0,
    TFLOAT iMax = 1.0,
    int iNumTables = 1 ) [inline]
```

Set the table lookup parameters.

Parameters

in	<i>iTableSize</i>	Size of the lookup table
in	<i>iMin</i>	Minimum input value
in	<i>iMax</i>	Maximum input value
in	<i>iNumTables</i>	Number of tables to index

Note

For future use...

14.48.1.2 DoTableLookupExtraFast() [1/2]

```
template<class TFLOAT , class DFLOAT >
DFLOAT AAX_FastInterpolatedTableLookup< TFLOAT, DFLOAT >::DoTableLookupExtraFast (
    const TFLOAT *const iTable,
    DFLOAT iValue ) const [inline]
```

Perform an extra fast table lookup :)

Parameters

in	<i>iTable</i>	Lookup table
in	<i>iValue</i>	Table value

Note

This version requires that the lookup table is padded with one extra location so we can avoid one of the checks to see if our pointers are out of bounds.

References AAX::FastTrunc2Int32().

Here is the call graph for this function:

14.48.1.3 DoTableLookupExtraFastMulti()

```
template<class TFLOAT , class DFLOAT >
void AAX_FastInterpolatedTableLookup< TFLOAT, DFLOAT >::DoTableLookupExtraFastMulti (
    const TFLOAT * iTable,
    DFLOAT iValue,
    DFLOAT * oValues ) const [inline]
```

References AAX::FastTrunc2Int32().

Here is the call graph for this function:

14.48.1.4 DoTableLookupExtraFast() [2/2]

```
template<class TFLOAT , class DFLOAT >
void AAX_FastInterpolatedTableLookup< TFLOAT, DFLOAT >::DoTableLookupExtraFast (
    const TFLOAT *const iTable,
    const TFLOAT *const inpBuf,
    DFLOAT *const outBuf,
    int blockSize ) [inline]
```


14.48.1.5 GetMin()

```
template<class TFLOAT , class DFLOAT >
TFLOAT AAX_FastInterpolatedTableLookup< TFLOAT, DFLOAT >::GetMin ( ) [inline]
```

14.48.1.6 GetMaxMinusMin()

```
template<class TFLOAT , class DFLOAT >
TFLOAT AAX_FastInterpolatedTableLookup< TFLOAT, DFLOAT >::GetMaxMinusMin ( ) [inline]
```

The documentation for this class was generated from the following files:

- [AAX_FastInterpolatedTableLookup.h](#)
- [AAX_FastInterpolatedTableLookup.hpp](#)

14.49 AAX_IACFAutomationDelegate Class Reference

```
#include <AAX_IACFAutomationDelegate.h>
```

Inheritance diagram for AAX_IACFAutomationDelegate:

Collaboration diagram for AAX_IACFAutomationDelegate:

14.49.1 Description

Versioned interface allowing an AAX plug-in to interact with the host's automation system.

See also

- [Parameter updates](#)
- [AAX_IAutomationDelegate](#)

Public Member Functions

- virtual [AAX_Result RegisterParameter](#) ([AAX_CParamID](#) iParameterID)=0
- virtual [AAX_Result UnregisterParameter](#) ([AAX_CParamID](#) iParameterID)=0
- virtual [AAX_Result PostSetValueRequest](#) ([AAX_CParamID](#) iParameterID, double normalizedValue) const =0
- virtual [AAX_Result PostCurrentValue](#) ([AAX_CParamID](#) iParameterID, double normalizedValue) const =0
- virtual [AAX_Result PostTouchRequest](#) ([AAX_CParamID](#) iParameterID)=0
- virtual [AAX_Result PostReleaseRequest](#) ([AAX_CParamID](#) iParameterID)=0
- virtual [AAX_Result GetTouchState](#) ([AAX_CParamID](#) iParameterID, [AAX_CBoolean](#) *oTouched)=0

14.49.2 Member Function Documentation

14.49.2.1 RegisterParameter()

```
virtual AAX_Result AAX_IACFAutomationDelegate::RegisterParameter (
    AAX_CParamID iParameterID ) [pure virtual]
```

Register a control with the automation system using a char* based control identifier

The automation delegate owns a list of the IDs of all of the parameters that have been registered with it. This list is used to set up listeners for all of the registered parameters such that the automation delegate may update the plug-in when the state of any of the registered parameters have been modified.

See also

[AAX_IAutomationDelegate::UnregisterParameter\(\)](#)

Parameters

in	<i>iParameterID</i>	Parameter ID that is being registered
----	---------------------	---------------------------------------

14.49.2.2 UnregisterParameter()

```
virtual AAX_Result AAX_IACFAutomationDelegate::UnregisterParameter (
    AAX_CParamID iParameterID ) [pure virtual]
```

Unregister a control with the automation system using a char* based control identifier

Note

All registered controls should be unregistered or the system might leak.

See also

[AAX_IAutomationDelegate::RegisterParameter\(\)](#)

Parameters

in	<i>iParameterID</i>	Parameter ID that is being registered
----	---------------------	---------------------------------------

14.49.2.3 PostSetValueRequest()

```
virtual AAX_Result AAX_IACFAutomationDelegate::PostSetValueRequest (
    AAX_CParamID iParameterID,
    double normalizedValue ) const [pure virtual]
```

Submits a request for the given parameter's value to be changed

Parameters

in	<i>iParameterID</i>	ID of the parameter for which a change is requested
in	<i>normalizedValue</i>	The requested new parameter value, formatted as a double and normalized to [0 1]

14.49.2.4 PostCurrentValue()

```
virtual AAX_Result AAX_IACFAutomationDelegate::PostCurrentValue (
    AAX_CParamID iParameterID,
    double normalizedValue ) const [pure virtual]
```

Notifies listeners that a parameter's value has changed

Parameters

in	<i>iParameterID</i>	ID of the parameter that has been updated
in	<i>normalizedValue</i>	The current parameter value, formatted as a double and normalized to [0 1]

14.49.2.5 PostTouchRequest()

```
virtual AAX_Result AAX_IACFAutomationDelegate::PostTouchRequest (
    AAX_CParamID iParameterID ) [pure virtual]
```

Requests that the given parameter be "touched", i.e. locked for updates by the current client

Parameters

in	<i>iParameterID</i>	ID of the parameter that will be touched
----	---------------------	--

14.49.2.6 PostReleaseRequest()

```
virtual AAX_Result AAX_IACFAutomationDelegate::PostReleaseRequest (
    AAX_CParamID iParameterID ) [pure virtual]
```

Requests that the given parameter be "released", i.e. available for updates from any client

Parameters

in	<i>iParameterID</i>	ID of the parameter that will be released
----	---------------------	---

14.49.2.7 GetTouchState()

```
virtual AAX_Result AAX_IACFAutomationDelegate::GetTouchState (
    AAX_CParamID iParameterID,
    AAX_CBoolean * oTouched ) [pure virtual]
```

Gets the current touched state of a parameter

Parameters

in	<i>iParameterID</i>	ID of the parameter that is being queried
out	<i>oTouched</i>	The current touch state of the parameter

The documentation for this class was generated from the following file:

- [AAX_IACFAutomationDelegate.h](#)

14.50 AAX_IACFCollection Class Reference

```
#include <AAX_IACFCollection.h>
```

Inheritance diagram for AAX_IACFCollection:

Collaboration diagram for AAX_IACFCollection:

14.50.1 Description

Versioned interface to represent a plug-in binary's static description.

Public Member Functions

- virtual [AAX_Result AddEffect](#) (const char *inEffectID, [IACFUnknown](#) *inEffectDescriptor)=0
Add an Effect description to the collection.
- virtual [AAX_Result SetManufacturerName](#) (const char *inPackageName)=0
Set the plug-in manufacturer name.
- virtual [AAX_Result AddPackageName](#) (const char *inPackageName)=0
Set the plug-in package name.
- virtual [AAX_Result SetPackageVersion](#) (uint32_t inVersion)=0
Set the plug-in package version number.
- virtual [AAX_Result SetProperties](#) ([IACFUnknown](#) *inProperties)=0
Set the properties of the collection.

14.50.2 Member Function Documentation

14.50.2.1 AddEffect()

```
virtual AAX_Result AAX_IACFCollection::AddEffect (
    const char * inEffectID,
    IACFUnknown * inEffectDescriptor ) [pure virtual]
```

Add an Effect description to the collection.

Each Effect that a plug-in registers with [AAX_ICollection::AddEffect\(\)](#) is considered a completely different user-facing product. For example, in Avid's Dynamics III plug-in the Expander, Compressor, and DeEsser are each registered as separate Effects. All stem format variations within each Effect are registered within that Effect's [AAX_IEffectDescriptor](#) using [AddComponent\(\)](#).

The [AAX_eProperty_ProductID](#) value for all ProcessProcs within a single Effect must be identical.

This method passes ownership of an [AAX_IEffectDescriptor](#) object to the [AAX_ICollection](#). The [AAX_IEffectDescriptor](#) must not be deleted by the AAX plug-in, nor should it be edited in any way after it is passed to the [AAX_ICollection](#).

Parameters

in	<i>inEffectID</i>	The effect ID.
in	<i>inEffectDescriptor</i>	The Effect descriptor.

14.50.2.2 SetManufacturerName()

```
virtual AAX_Result AAX_IACFCollection::SetManufacturerName (
    const char * inPackageName ) [pure virtual]
```

Set the plug-in manufacturer name.

Parameters

in	<i>inPackageName</i>	The name of the manufacturer.
----	----------------------	-------------------------------

14.50.2.3 AddPackageName()

```
virtual AAX_Result AAX_IACFCollection::AddPackageName (
    const char * inPackageName ) [pure virtual]
```

Set the plug-in package name.

May be called multiple times to add abbreviated package names.

Note

Every plug-in must include at least one name variant with 16 or fewer characters, plus a null terminating character. Used for Presets folder.

Parameters

in	<i>inPackageName</i>	The name of the package.
----	----------------------	--------------------------

14.50.2.4 SetPackageVersion()

```
virtual AAX_Result AAX_IACFCollection::SetPackageVersion (
    uint32_t inVersion ) [pure virtual]
```

Set the plug-in package version number.

Parameters

in	<i>inVersion</i>	The package version number.
----	------------------	-----------------------------

14.50.2.5 SetPropertyies()

```
virtual AAX_Result AAX_IACFCollection::SetProperties (
    IACFUnknown * inProperties ) [pure virtual]
```

Set the properties of the collection.

Parameters

in	<i>inProperties</i>	Collection properties
----	---------------------	-----------------------

The documentation for this class was generated from the following file:

- [AAX_IACFCollection.h](#)

14.51 AAX_IACFComponentDescriptor Class Reference

```
#include <AAX_IACFComponentDescriptor.h>
```

Inheritance diagram for AAX_IACFComponentDescriptor:

Collaboration diagram for AAX_IACFComponentDescriptor:

14.51.1 Description

Versioned description interface for an AAX plug-in algorithm callback.

Public Member Functions

- virtual [AAX_Result Clear](#) ()=0
Clears the descriptor.
- virtual [AAX_Result AddReservedField](#) ([AAX_CFieldIndex](#) inFieldIndex, [uint32_t](#) inFieldType)=0
Subscribes a context field to host-provided services or information.
- virtual [AAX_Result AddAudioIn](#) ([AAX_CFieldIndex](#) inFieldIndex)=0
Subscribes an audio input context field.
- virtual [AAX_Result AddAudioOut](#) ([AAX_CFieldIndex](#) inFieldIndex)=0
Subscribes an audio output context field.
- virtual [AAX_Result AddAudioBufferLength](#) ([AAX_CFieldIndex](#) inFieldIndex)=0
Subscribes a buffer length context field.
- virtual [AAX_Result AddSampleRate](#) ([AAX_CFieldIndex](#) inFieldIndex)=0
Subscribes a sample rate context field.
- virtual [AAX_Result AddClock](#) ([AAX_CFieldIndex](#) inFieldIndex)=0
Subscribes a clock context field.
- virtual [AAX_Result AddSideChainIn](#) ([AAX_CFieldIndex](#) inFieldIndex)=0
Subscribes a side-chain input context field.
- virtual [AAX_Result AddDataInPort](#) ([AAX_CFieldIndex](#) inFieldIndex, [uint32_t](#) inPacketSize, [AAX_EDataInPortType](#) inPortType)=0
Adds a custom data port to the algorithm context.
- virtual [AAX_Result AddAuxOutputStem](#) ([AAX_CFieldIndex](#) inFieldIndex, [int32_t](#) inStemFormat, [const char](#) inNameUTF8[])=0
Adds an auxiliary output stem for a plug-in.
- virtual [AAX_Result AddPrivateData](#) ([AAX_CFieldIndex](#) inFieldIndex, [int32_t](#) inDataSize, [uint32_t](#) inOptions=[AAX_ePrivateDataOptions_DefaultOptions](#))=0
Adds a private data port to the algorithm context.
- virtual [AAX_Result AddDmaInstance](#) ([AAX_CFieldIndex](#) inFieldIndex, [AAX_IDma::EMode](#) inDmaMode)=0
Adds a DMA field to the plug-in's context.
- virtual [AAX_Result AddMIDINode](#) ([AAX_CFieldIndex](#) inFieldIndex, [AAX_EMIDINodeType](#) inNodeType, [const char](#) inNodeName[], [uint32_t](#) channelMask)=0
Adds a MIDI node field to the plug-in's context.
- virtual [AAX_Result AddProcessProc_Native](#) ([AAX_CProcessProc](#) inProcessProc, [IACFUnknown](#) *inProperties, [AAX_CInstanceInitProc](#) inInstanceInitProc, [AAX_CBackgroundProc](#) inBackgroundProc, [AAX_CSelector](#) *outProcID)=0
Registers an algorithm processing endpoint (process procedure) for the native architecture.
- virtual [AAX_Result AddProcessProc_TI](#) ([const char](#) inDLLFileNameUTF8[], [const char](#) inProcessProcSymbol[], [IACFUnknown](#) *inProperties, [const char](#) inInstanceInitProcSymbol[], [const char](#) inBackgroundProcSymbol[], [AAX_CSelector](#) *outProcID)=0
Registers an algorithm processing endpoint (process procedure) for the native architecture.
- virtual [AAX_Result AddMeters](#) ([AAX_CFieldIndex](#) inFieldIndex, [const AAX_CTypeID](#) *inMeterIDs, [const uint32_t](#) inMeterCount)=0
Adds a meter field to the plug-in's context.

14.51.2 Member Function Documentation

14.51.2.1 Clear()

```
virtual AAX_Result AAX_IACFComponentDescriptor::Clear ( ) [pure virtual]
```

Clears the descriptor.

Clears the descriptor and readies it for the next algorithm description

14.51.2.2 AddReservedField()

```
virtual AAX_Result AAX_IACFComponentDescriptor::AddReservedField (
    AAX_CFieldIndex inFieldIndex,
    uint32_t inFieldType ) [pure virtual]
```

Subscribes a context field to host-provided services or information.

Note

Currently for internal use only.

Parameters

in	<i>inFieldIndex</i>	Unique identifier for the field, generated using AAX_FIELD_INDEX
in	<i>inFieldType</i>	Type of field that is being added

14.51.2.3 AddAudioIn()

```
virtual AAX_Result AAX_IACFComponentDescriptor::AddAudioIn (
    AAX_CFieldIndex inFieldIndex ) [pure virtual]
```

Subscribes an audio input context field.

Defines an audio in port for host-provided information in the algorithm's context structure.

- Data type: float**
- Data kind: An array of float arrays, one for each input channel

Parameters

in	<i>inFieldIndex</i>	Unique identifier for the field, generated using AAX_FIELD_INDEX
----	---------------------	--

14.51.2.4 AddAudioOut()

```
virtual AAX_Result AAX_IACFComponentDescriptor::AddAudioOut (
    AAX_CFieldIndex inFieldIndex ) [pure virtual]
```

Subscribes an audio output context field.

Defines an audio out port for host-provided information in the algorithm's context structure.

- Data type: float**
- Data kind: An array of float arrays, one for each output channel

Parameters

in	<i>inFieldIndex</i>	Unique identifier for the field, generated using AAX_FIELD_INDEX
----	---------------------	--

14.51.2.5 AddAudioBufferLength()

```
virtual AAX_Result AAX_IACFComponentDescriptor::AddAudioBufferLength (
    AAX_CFieldIndex inFieldIndex ) [pure virtual]
```

Subscribes a buffer length context field.

Defines a buffer length port for host-provided information in the algorithm's context structure.

- Data type: int32_t*
- Data kind: The number of samples in the current audio buffer

Parameters

in	<i>inFieldIndex</i>	Unique identifier for the field, generated using AAX_FIELD_INDEX
----	---------------------	--

14.51.2.6 AddSampleRate()

```
virtual AAX_Result AAX_IACFComponentDescriptor::AddSampleRate (
    AAX_CFieldIndex inFieldIndex ) [pure virtual]
```

Subscribes a sample rate context field.

Defines a sample rate port for host-provided information in the algorithm's context structure.

- Data type: [AAX_CSampleRate](#) *
- Data kind: The current sample rate

Parameters

in	<i>inFieldIndex</i>	Unique identifier for the field, generated using AAX_FIELD_INDEX
----	---------------------	--

14.51.2.7 AddClock()

```
virtual AAX_Result AAX_IACFComponentDescriptor::AddClock (
    AAX_CFieldIndex inFieldIndex ) [pure virtual]
```

Subscribes a clock context field.

Defines a clock port for host-provided information in the algorithm's context structure.

- Data type: [AAX_CTimestamp](#) *
- Data kind: A running counter which increments even when the transport is not playing. The counter increments exactly once per sample quantum.

Host Compatibility Notes As of Pro Tools 11.1, this field may be used in both Native and DSP plug-ins. The DSP clock data is a 16-bit cycling counter. This field was only available for Native plug-ins in previous Pro Tools versions.

Parameters

in	<i>inFieldIndex</i>	Unique identifier for the field, generated using AAX_FIELD_INDEX
----	---------------------	--

14.51.2.8 AddSideChainIn()

```
virtual AAX_Result AAX_IACFComponentDescriptor::AddSideChainIn (
    AAX_CFieldIndex inFieldIndex ) [pure virtual]
```

Subscribes a side-chain input context field.

Defines a side-chain input port for host-provided information in the algorithm's context structure.

- Data type: `int32_t*`
- Data kind: The index of the plug-in's first side-chain input channel within the array of input audio buffers

Parameters

in	<i>inFieldIndex</i>	Unique identifier for the field, generated using AAX_FIELD_INDEX
----	---------------------	--

14.51.2.9 AddDataInPort()

```
virtual AAX_Result AAX_IACFComponentDescriptor::AddDataInPort (
    AAX_CFieldIndex inFieldIndex,
    uint32_t inPacketSize,
    AAX_EDataInPortType inPortType ) [pure virtual]
```

Adds a custom data port to the algorithm context.

Defines a read-only data port for plug-in information in the algorithm's context structure. The plug-in can send information to this port using [AAX_IController::PostPacket\(\)](#).

The host guarantees that all packets will be delivered to this port in the order in which they were posted, up to the point of a packet buffer overflow, though some packets may be dropped depending on the `inPortType` and host implementation.

Note

When a plug-in is operating in offline (AudioSuite) mode, all data ports operate as [AAX_eDataInPortType_Unbuffered](#) ports

Parameters

in	<i>inFieldIndex</i>	Unique identifier for the port, generated using AAX_FIELD_INDEX
in	<i>inPacketSize</i>	Size of the data packets that will be sent to this port
in	<i>inPortType</i>	The requested packet delivery behavior for this port

14.51.2.10 AddAuxOutputStem()

```
virtual AAX_Result AAX_IACFComponentDescriptor::AddAuxOutputStem (
    AAX_CFieldIndex inFieldIndex,
    int32_t inStemFormat,
    const char inNameUTF8[] ) [pure virtual]
```

Adds an auxiliary output stem for a plug-in.

Use this method to add additional output channels to the algorithm context.

The aux output stem audio buffers will be added to the end of the audio outputs array in the order in which they are described. When writing audio data to a specific aux output, find the proper starting channel by accumulating all of the channels of the main output stem format and any previously-described aux output stems.

The plug-in is responsible for providing a meaningful name for each aux outputs. At the very least, individual outputs should be labeled "Output xx", where "xx" is the aux output number as it is defined in the plug-in. The output name should also include the words "mono" and "stereo" to support when users are looking for an output with a specific stem format.

Host Compatibility Notes There is a hard limit to the number of outputs that Pro Tools supports for a single plug-in instance. This limit is currently set at 256 channels, which includes all of the plug-in's output channels in addition to the sum total of all of its aux output stem channels.

Host Compatibility Notes Pro Tools supports only mono and stereo auxiliary output stem formats

Warning

This method will return an error code on hosts which do not support auxiliary output stems. This indicates that the host will not provide audio buffers for auxiliary output stems during processing. A plug-in must not attempt to write data into auxiliary output stem buffers which have not been provided by the host!

Parameters

in	<i>inFieldIndex</i>	DEPRECATED: This parameter is no longer needed by the host, but is included in the interface for binary compatibility
in	<i>inStemFormat</i>	The stem format of the new aux output
in	<i>inNameUTF8</i>	The name of the aux output. This name is static and cannot be changed after the descriptor is submitted to the host

14.51.2.11 AddPrivateData()

```
virtual AAX_Result AAX_IACFComponentDescriptor::AddPrivateData (
    AAX_CFieldIndex inFieldIndex,
    int32_t inDataSize,
    uint32_t inOptions = AAX_ePrivateDataOptions_DefaultOptions ) [pure virtual]
```

Adds a private data port to the algorithm context.

Defines a read/write data port for private state data. Data written to this port will be maintained by the host between calls to the algorithm context.

See also

alg_pd_registration

Parameters

in	<i>inFieldIndex</i>	Unique identifier for the port, generated using AAX_FIELD_INDEX
in	<i>inDataSize</i>	Size of the data packets that will be sent to this port
in	<i>inOptions</i>	Options that define the private data port's behavior

14.51.2.12 AddDmaInstance()

```
virtual AAX_Result AAX_IACFComponentDescriptor::AddDmaInstance (
    AAX_CFieldIndex inFieldIndex,
    AAX_IDma::EMode inDmaMode ) [pure virtual]
```

Adds a DMA field to the plug-in's context.

DMA (direct memory access) provides efficient reads from and writes to external memory on the DSP. DMA behavior is emulated in host-based plug-ins for cross-platform portability.

Note

The order in which DMA instances are added defines their priority and therefore order of execution of DMA operations. In most plug-ins, Scatter fields should be placed first in order to achieve the lowest possible access latency.

For more information, see [Direct Memory Access](#) .

Todo Update the DMA system management such that operation priority can be set arbitrarily

Parameters

in	<i>inFieldIndex</i>	Unique identifier for the field, generated using AAX_FIELD_INDEX
in	<i>inDmaMode</i>	AAX_IDma::EMode that will apply to this field

14.51.2.13 AddMIDINode()

```
virtual AAX_Result AAX_IACFComponentDescriptor::AddMIDINode (
    AAX_CFieldIndex inFieldIndex,
    AAX_EMIDINodeType inNodeType,
    const char inNodeName[],
    uint32_t channelMask ) [pure virtual]
```

Adds a MIDI node field to the plug-in's context.

- Data type: [AAX_IMIDINode](#) *

The resulting MIDI node data will be available both in the algorithm context and in the plug-in's [data model](#) via [UpdateMIDINodes\(\)](#).

To add a MIDI node that is only accessible to the plug-in's data model, use [AAX_IEffectDescriptor::AddControlMIDINode\(\)](#)

Host Compatibility Notes Due to current restrictions MIDI data won't be delivered to DSP algorithms, only to AAX Native.

Parameters

in	<i>inFieldIndex</i>	The ID of the port. MIDI node ports should formatted as a pointer to an AAX_IMIDINode .
in	<i>inNodeType</i>	The type of MIDI node, as AAX_EMIDINodeType
in	<i>inNodeName</i>	The name of the MIDI node as it should appear in the host's UI
in	<i>channelMask</i>	The channel mask for the MIDI node. This parameter specifies used MIDI channels. For Global MIDI nodes, use a mask of AAX_EMidiGlobalNodeSelectors

14.51.2.14 AddProcessProc_Native()

```
virtual AAX_Result AAX_IACFComponentDescriptor::AddProcessProc_Native (
    AAX_CProcessProc inProcessProc,
    IACFUnknown * inProperties,
    AAX_CInstanceInitProc inInstanceInitProc,
    AAX_CBackgroundProc inBackgroundProc,
    AAX_CSelector * outProcID ) [pure virtual]
```

Registers an algorithm processing entrypoint (process procedure) for the native architecture.

Parameters

in	<i>inProcessProc</i>	Symbol for this processing callback
in	<i>inProperties</i>	A property map for this processing callback. The property map's values are copied by the host and associated with the new ProcessProc. The property map contents are unchanged and the map may be re-used when registering additional ProcessProcs.
in	<i>inInstanceInitProc</i>	Initialization routine that will be called when a new instance of the Effect is created. See Algorithm initialization .
in	<i>inBackgroundProc</i>	Background routine that will be called in an idle context within the same address space as the associated process procedure. See Background processing callback
out	<i>outProcID</i>	

Todo document this parameter

14.51.2.15 AddProcessProc_TI()

```
virtual AAX_Result AAX_IACFComponentDescriptor::AddProcessProc_TI (
    const char inDLLFileNameUTF8[],
    const char inProcessProcSymbol[],
    IACFUnknown * inProperties,
    const char inInstanceInitProcSymbol[],
    const char inBackgroundProcSymbol[],
    AAX_CSelector * outProcID ) [pure virtual]
```

Registers an algorithm processing entrypoint (process procedure) for the native architecture.

Parameters

in	<i>inDLLFileNameUTF8</i>	UTF-8 encoded filename for the ELF DLL containing the algorithm code fragment
in	<i>inProcessProcSymbol</i>	Symbol for this processing callback
in	<i>inProperties</i>	A property map for this processing callback. The property map's values are copied by the host and associated with the new ProcessProc. The property map contents are unchanged and the map may be re-used when registering additional ProcessProcs.
in	<i>inInstanceInitProcSymbol</i>	Initialization routine that will be called when a new instance of the Effect is created. Must be included in the same DLL as the main algorithm entrypoint. See Algorithm initialization .

Parameters

in	<i>inBackgroundProcSymbol</i>	Background routine that will be called in an idle context within the same address space as the associated process procedure. Must be included in the same DLL as the main algorithm entrypoint. See Background processing callback
out	<i>outProcID</i>	

Todo document this parameter

14.51.2.16 AddMeters()

```
virtual AAX_Result AAX_IACFComponentDescriptor::AddMeters (
    AAX_CFieldIndex inFieldIndex,
    const AAX_CTypeID * inMeterIDs,
    const uint32_t inMeterCount ) [pure virtual]
```

Adds a meter field to the plug-in's context.

Meter fields include an array of meter tap values, with one tap per meter per context. Only one meter field should be added per Component. Individual meter behaviors can be described at the Effect level.

For more information, see [Plug-in meters](#) .

Parameters

in	<i>inFieldIndex</i>	Unique identifier for the field, generated using AAX_FIELD_INDEX
in	<i>inMeterIDs</i>	Array of 32-bit IDs, one for each meter. Meter IDs must be unique within the Effect.
in	<i>inMeterCount</i>	The number of meters included in this field

The documentation for this class was generated from the following file:

- [AAX_IACFComponentDescriptor.h](#)

14.52 AAX_IACFComponentDescriptor_V2 Class Reference

```
#include <AAX_IACFComponentDescriptor.h>
```

Inheritance diagram for AAX_IACFComponentDescriptor_V2:

Collaboration diagram for AAX_IACFComponentDescriptor_V2:

14.52.1 Description

Versioned description interface for an AAX plug-in algorithm callback.

Public Member Functions

- virtual [AAX_Result AddTemporaryData](#) ([AAX_CFieldIndex](#) inFieldIndex, uint32_t inDataElementSize)=0
Adds a block of data to a context that is not saved between callbacks and is scaled by the system buffer size.

14.52.2 Member Function Documentation

14.52.2.1 AddTemporaryData()

```
virtual AAX\_Result AAX_IACFComponentDescriptor_V2::AddTemporaryData (
    AAX\_CFieldIndex inFieldIndex,
    uint32_t inDataElementSize ) [pure virtual]
```

Adds a block of data to a context that is not saved between callbacks and is scaled by the system buffer size.

This can be very useful if you use block processing and need to store intermediate results. Just specify your base element size and the system will scale the overall block size by the buffer size. For example, to create a buffer of floats that is the length of the block, specify 4 bytes as the elementsize.

This data block does not retain state across callback and can also be reused across instances on memory constrained systems.

Parameters

in	<i>inFieldIndex</i>	Unique identifier for the port, generated using AAX_FIELD_INDEX
in	<i>inDataElementSize</i>	The size of a single piece of data in the block. This number will be multiplied by the processing block size to determine total block size.

The documentation for this class was generated from the following file:

- [AAX_IACFComponentDescriptor.h](#)

14.53 AAX_IACFComponentDescriptor_V3 Class Reference

```
#include <AAX_IACFComponentDescriptor.h>
```

Inheritance diagram for AAX_IACFComponentDescriptor_V3:

Collaboration diagram for AAX_IACFComponentDescriptor_V3:

14.53.1 Description

Versioned description interface for an AAX plug-in algorithm callback.

Public Member Functions

- virtual [AAX_Result AddProcessProc](#) ([IACFUnknown](#) *inProperties, [AAX_CSelector](#) *outProcIDs, int32_t inProcIDsSize)=0
Registers one or more algorithm processing entrypoints (process procedures)

14.53.2 Member Function Documentation

14.53.2.1 AddProcessProc()

```
virtual AAX\_Result AAX_IACFComponentDescriptor_V3::AddProcessProc (
    IACFUnknown * inProperties,
    AAX\_CSelector * outProcIDs,
    int32_t inProcIDsSize ) [pure virtual]
```

Registers one or more algorithm processing entrypoints (process procedures)

Any non-overlapping set of processing entrypoints may be specified. Typically this can be used to specify both Native and TI entrypoints using the same call.

The AAX Library implementation of this method includes backwards compatibility logic to complete the [ProcessProc](#) registration on hosts which do not support this method. Therefore plug-in code may use this single registration routine instead of separate calls to [AddProcessProc_Native\(\)](#), [AddProcessProc_TI\(\)](#), etc. regardless of the host version.

The following properties replace the input arguments to the platform-specific registration methods:

[AddProcessProc_Native\(\)](#) ([AAX_eProperty_PluginID_Native](#), [AAX_eProperty_PluginID_AudioSuite](#))

- [AAX_CProcessProc](#) iProcessProc: [AAX_eProperty_NativeProcessProc](#) (required)
- [AAX_CInstanceInitProc](#) iInstanceInitProc: [AAX_eProperty_NativeInstanceInitProc](#) (optional)
- [AAX_CBackgroundProc](#) iBackgroundProc: [AAX_eProperty_NativeBackgroundProc](#) (optional)

[AddProcessProc_TI\(\)](#) ([AAX_eProperty_PluginID_TI](#))

- const char inDLLFileNameUTF8[]: [AAX_eProperty_TIDLLFileName](#) (required)
- const char iProcessProcSymbol[]: [AAX_eProperty_TIPProcessProc](#) (required)
- const char iInstanceInitProcSymbol[]: [AAX_eProperty_TIInstanceInitProc](#) (optional)
- const char iBackgroundProcSymbol[]: [AAX_eProperty_TIBackgroundProc](#) (optional)

If any platform-specific plug-in ID property is present in iProperties then [AddProcessProc\(\)](#) will check for the required properties for that platform.

Note

[AAX_eProperty_AudioBufferLength](#) will be ignored for the Native and AudioSuite ProcessProcs since it should only be used for AAX DSP.

Parameters

in	<i>inProperties</i>	A property map for this processing callback. The property map's values are copied by the host and associated with the new ProcessProc. The property map contents are unchanged and the map may be re-used when registering additional ProcessProcs.
out	<i>outProcIDs</i>	

Todo document this parameter Returned array will be NULL-terminated

Parameters

in	<i>inProcIDsSize</i>	The size of the array provided to oProcIDs. If oProcIDs is non-NULL but iProcIDsSize is not large enough for all of the registered ProcessProcs (plus one for NULL termination) then this method will fail with AAX_ERROR_ARGUMENT_BUFFER_OVERFLOW
----	----------------------	--

The documentation for this class was generated from the following file:

- [AAX_IACFComponentDescriptor.h](#)

14.54 AAX_IACFController Class Reference

```
#include <AAX_IACFController.h>
```

Inheritance diagram for AAX_IACFController:

Collaboration diagram for AAX_IACFController:

14.54.1 Description

Interface for the AAX host's view of a single instance of an effect. Used by both clients of the AAXHost and by effect components.

Public Member Functions

- virtual [AAX_Result](#) GetEffectID ([AAX_IString](#) *outEffectID) const =0
- virtual [AAX_Result](#) GetSampleRate ([AAX_CSampleRate](#) *outSampleRate) const =0
CALL: Returns the current literal sample rate.
- virtual [AAX_Result](#) GetInputStemFormat ([AAX_EStemFormat](#) *outStemFormat) const =0
CALL: Returns the plug-in's input stem format.
- virtual [AAX_Result](#) GetOutputStemFormat ([AAX_EStemFormat](#) *outStemFormat) const =0
CALL: Returns the plug-in's output stem format.
- virtual [AAX_Result](#) GetSignalLatency (int32_t *outSamples) const =0
CALL: Returns the most recent signal (algorithmic) latency that has been published by the plug-in.
- virtual [AAX_Result](#) GetCycleCount ([AAX_EProperty](#) inWhichCycleCount, [AAX_CPropertyValue](#) *outNumCycles) const =0

- CALL: returns the plug-in's current real-time DSP cycle count.*
- virtual [AAX_Result GetTODLocation](#) ([AAX_CTimeOfDay](#) *outTODLocation) const =0
CALL: Returns the current Time Of Day (TOD) of the system.
- virtual [AAX_Result SetSignalLatency](#) (int32_t inNumSamples)=0
CALL: Submits a request to change the delay compensation value that the host uses to account for the plug-in's signal (algorithmic) latency.
- virtual [AAX_Result SetCycleCount](#) ([AAX_EProperty](#) *inWhichCycleCounts, [AAX_CPropertyValue](#) *iValues, int32_t numValues)=0
CALL: Indicates a change in the plug-in's real-time DSP cycle count.
- virtual [AAX_Result PostPacket](#) ([AAX_CFieldIndex](#) inFieldIndex, const void *inPayloadP, uint32_t inPayloadSize)=0
CALL: Posts a data packet to the host for routing between plug-in components.
- virtual [AAX_Result GetCurrentMeterValue](#) ([AAX_CTypeID](#) inMeterID, float *outMeterValue) const =0
CALL: Retrieves the current value of a host-managed plug-in meter.
- virtual [AAX_Result GetMeterPeakValue](#) ([AAX_CTypeID](#) inMeterID, float *outMeterPeakValue) const =0
CALL: Retrieves the currently held peak value of a host-managed plug-in meter.
- virtual [AAX_Result ClearMeterPeakValue](#) ([AAX_CTypeID](#) inMeterID) const =0
CALL: Clears the peak value from a host-managed plug-in meter.
- virtual [AAX_Result GetMeterClipped](#) ([AAX_CTypeID](#) inMeterID, [AAX_CBoolean](#) *outClipped) const =0
CALL: Retrieves the clipped flag from a host-managed plug-in meter.
- virtual [AAX_Result ClearMeterClipped](#) ([AAX_CTypeID](#) inMeterID) const =0
CALL: Clears the clipped flag from a host-managed plug-in meter.
- virtual [AAX_Result GetMeterCount](#) (uint32_t *outMeterCount) const =0
CALL: Retrieves the number of host-managed meters registered by a plug-in.
- virtual [AAX_Result GetNextMIDIPacket](#) ([AAX_CFieldIndex](#) *outPort, [AAX_CMidiPacket](#) *outPacket)=0
CALL: Retrieves MIDI packets for described MIDI nodes.

14.54.2 Member Function Documentation

14.54.2.1 GetEffectID()

```
virtual AAX\_Result AAX_IACFController::GetEffectID (
    AAX\_IString * outEffectID ) const [pure virtual]
```

14.54.2.2 GetSampleRate()

```
virtual AAX\_Result AAX_IACFController::GetSampleRate (
    AAX\_CSampleRate * outSampleRate ) const [pure virtual]
```

CALL: Returns the current literal sample rate.

Parameters

out	outSampleRate	The current sample rate
-----	---------------	-------------------------

14.54.2.3 GetInputStemFormat()

```
virtual AAX_Result AAX_IACFController::GetInputStemFormat (
    AAX_EStemFormat * outStemFormat ) const [pure virtual]
```

CALL: Returns the plug-in's input stem format.

Parameters

out	<i>outStemFormat</i>	The current input stem format
-----	----------------------	-------------------------------

14.54.2.4 GetOutputStemFormat()

```
virtual AAX_Result AAX_IACFController::GetOutputStemFormat (
    AAX_EStemFormat * outStemFormat ) const [pure virtual]
```

CALL: Returns the plug-in's output stem format.

Parameters

out	<i>outStemFormat</i>	The current output stem format
-----	----------------------	--------------------------------

14.54.2.5 GetSignalLatency()

```
virtual AAX_Result AAX_IACFController::GetSignalLatency (
    int32_t * outSamples ) const [pure virtual]
```

CALL: Returns the most recent signal (algorithmic) latency that has been published by the plug-in.

This method provides the most recently published signal latency. The host may not have updated its delay compensation to match this signal latency yet, so plug-ins that dynamically change their latency using [SetSignalLatency\(\)](#) should always wait for an [AAX_eNotificationEvent_SignalLatencyChanged](#) notification before updating its algorithm to incur this latency.

See also

[SetSignalLatency\(\)](#)

Parameters

out	<i>outSamples</i>	The number of samples of signal delay published by the plug-in
-----	-------------------	--

14.54.2.6 GetCycleCount()

```
virtual AAX_Result AAX_IACFController::GetCycleCount (
    AAX_EProperty inWhichCycleCount,
    AAX_CPropertyValue * outNumCycles ) const [pure virtual]
```

CALL: returns the plug-in's current real-time DSP cycle count.

This method provides the number of cycles that the AAX host expects the DSP plug-in to consume. The host uses this value when allocating DSP resources for the plug-in.

Note

A plug-in should never apply a DSP algorithm with more demanding resource requirements than what is currently accounted for by the host. To set a higher cycle count value, a plug-in must call [AAX_IController::SetCycleCount\(\)](#), then poll [AAX_IController::GetCycleCount\(\)](#) until the new value has been applied. Once the host has recognized the new cycle count value, the plug-in may apply the more demanding algorithm.

Parameters

in	<i>inWhichCycleCount</i>	Selector for the requested cycle count metric. One of: • AAX_eProperty_TI_SharedCycleCount • AAX_eProperty_TI_InstanceCycleCount • AAX_eProperty_TI_MaxInstancesPerChip
in	<i>outNumCycles</i>	The current value of the selected cycle count metric

Todo PLACEHOLDER - NOT CURRENTLY IMPLEMENTED IN HOST

14.54.2.7 GetTODLocation()

```
virtual AAX_Result AAX_IACFController::GetTODLocation (
    AAX_CTimeOfDay * outTODLocation ) const [pure virtual]
```

CALL: Returns the current Time Of Day (TOD) of the system.

This method provides a plug-in the TOD (in samples) of the current system. TOD is the number of samples that the playhead has traversed since the beginning of playback.

Note

The TOD value is the immediate value of the audio engine playhead. This value is incremented within the audio engine's real-time rendering context; it is not synchronized with non-real-time calls to plug-in interface methods.

Parameters

out	<i>outTODLocation</i>	The current Time Of Day as set by the host
-----	-----------------------	--

14.54.2.8 SetSignalLatency()

```
virtual AAX_Result AAX_IACFController::SetSignalLatency (
    int32_t inNumSamples ) [pure virtual]
```

CALL: Submits a request to change the delay compensation value that the host uses to account for the plug-in's signal (algorithmic) latency.

This method is used to request a change in the number of samples that the AAX host expects the plug-in to delay a signal.

The host is not guaranteed to immediately apply the new latency value. A plug-in should avoid incurring an actual algorithmic latency that is different than the latency accounted for by the host.

To set a new latency value, a plug-in must call [AAX_IController::SetSignalLatency\(\)](#), then wait for an [AAX_eNotificationEvent_SignalLatencyChanged](#) notification. Once this notification has been received, [AAX_IController::GetSignalLatency\(\)](#) will reflect the updated latency value and the plug-in should immediately apply any relevant algorithmic changes that alter its latency to this new value.

Warning

Parameters which affect the latency of a plug-in should not be made available for control through automation. This will result in audible glitches when delay compensation is adjusted while playing back automation for these parameters.

Parameters

in	<i>inNumSamples</i>	The number of samples of signal delay that the plug-in requests to incur
----	---------------------	--

14.54.2.9 SetCycleCount()

```
virtual AAX_Result AAX_IACFController::SetCycleCount (
    AAX_EProperty * inWhichCycleCounts,
    AAX_CPropertyValue * iValues,
    int32_t numValues ) [pure virtual]
```

CALL: Indicates a change in the plug-in's real-time DSP cycle count.

This method is used to request a change in the number of cycles that the AAX host expects the DSP plug-in to consume.

Note

A plug-in should never apply a DSP algorithm with more demanding resource requirements than what is currently accounted for by the host. To set a higher cycle count value, a plug-in must call [AAX_IController::SetCycleCount\(\)](#), then poll [AAX_IController::GetCycleCount\(\)](#) until the new value has been applied. Once the host has recognized the new cycle count value, the plug-in may apply the more demanding algorithm.

Parameters

in	<i>inWhichCycleCounts</i>	Array of selectors indicating the specific cycle count metrics that should be set. Each selector must be one of: • AAX_eProperty_TI_SharedCycleCount • AAX_eProperty_TI_InstanceCycleCount • AAX_eProperty_TI_MaxInstancesPerChip
in	<i>iValues</i>	An array of values requested, one for each of the selected cycle count metrics.
in	<i>numValues</i>	The size of <i>iValues</i>

Todo PLACEHOLDER - NOT CURRENTLY IMPLEMENTED IN HOST

14.54.2.10 PostPacket()

```
virtual AAX_Result AAX_IACFController::PostPacket (
    AAX_CFieldIndex inFieldIndex,
    const void * inPayloadP,
    uint32_t inPayloadSize ) [pure virtual]
```

CALL: Posts a data packet to the host for routing between plug-in components.

The posted packet is identified with a [AAX_CFieldIndex](#) packet index value, which is equivalent to the target data port's identifier. The packet's payload must have the expected size for the given packet index / data port, as defined when the port is created in [Describe](#). See [AAX_IComponentDescriptor::AddDataInPort\(\)](#).

Warning

Any data structures that will be passed between platforms (for example, sent to a TI DSP in an AAX DSP plug-in) must be properly data-aligned for compatibility across both platforms. See [AAX_ALIGN_FILE_ALG](#) for more information about guaranteeing cross-platform compatibility of data structures used for algorithm processing.

Note

All calls to this method should be made within the scope of [AAX_IEffectParameters::GenerateCoefficients\(\)](#). Calls from outside this method may result in packets not being delivered. See [PT-206161](#)

Parameters

in	<i>inFieldIndex</i>	The packet's destination port
in	<i>inPayloadP</i>	A pointer to the packet's payload data
in	<i>inPayloadSize</i>	The size, in bytes, of the payload data

14.54.2.11 GetCurrentMeterValue()

```
virtual AAX_Result AAX_IACFController::GetCurrentMeterValue (
    AAX_CTypeID inMeterID,
    float * outMeterValue ) const [pure virtual]
```

CALL: Retrieves the current value of a host-managed plug-in meter.

Parameters

in	<i>inMeterID</i>	ID of the meter that is being queried
out	<i>outMeterValue</i>	The queried meter's current value

14.54.2.12 GetMeterPeakValue()

```
virtual AAX_Result AAX_IACFController::GetMeterPeakValue (
    AAX_CTypeID inMeterID,
    float * outMeterPeakValue ) const [pure virtual]
```

CALL: Retrieves the currently held peak value of a host-managed plug-in meter.

Parameters

in	<i>inMeterID</i>	ID of the meter that is being queried
out	<i>outMeterPeakValue</i>	The queried meter's currently held peak value

14.54.2.13 ClearMeterPeakValue()

```
virtual AAX_Result AAX_IACFController::ClearMeterPeakValue (
    AAX_CTypeID inMeterID ) const [pure virtual]
```

CALL: Clears the peak value from a host-managed plug-in meter.

Parameters

in	<i>inMeterID</i>	ID of the meter that is being cleared
----	------------------	---------------------------------------

14.54.2.14 GetMeterClipped()

```
virtual AAX_Result AAX_IACFController::GetMeterClipped (
    AAX_CTypeID inMeterID,
    AAX_CBoolean * outClipped ) const [pure virtual]
```

CALL: Retrieves the clipped flag from a host-managed plug-in meter.

See [AAX_IComponentDescriptor::AddMeters\(\)](#).

Parameters

in	<i>inMeterID</i>	ID of the meter that is being queried.
out	<i>outClipped</i>	The queried meter's clipped flag.

14.54.2.15 ClearMeterClipped()

```
virtual AAX_Result AAX_IACFController::ClearMeterClipped (
    AAX_CTypeID inMeterID ) const [pure virtual]
```

CALL: Clears the clipped flag from a host-managed plug-in meter.

See [AAX_IComponentDescriptor::AddMeters\(\)](#).

Parameters

in	<i>inMeterID</i>	ID of the meter that is being cleared.
----	------------------	--

14.54.2.16 GetMeterCount()

```
virtual AAX_Result AAX_IACFController::GetMeterCount (
    uint32_t * outMeterCount ) const [pure virtual]
```

CALL: Retrieves the number of host-managed meters registered by a plug-in.

See [AAX_IComponentDescriptor::AddMeters\(\)](#).

Parameters

out	<i>outMeterCount</i>	The number of registered plug-in meters.
-----	----------------------	--

14.54.2.17 GetNextMIDIPacket()

```
virtual AAX_Result AAX_IACFController::GetNextMIDIPacket (
    AAX_CFieldIndex * outPort,
    AAX_CMidiPacket * outPacket ) [pure virtual]
```

CALL: Retrieves MIDI packets for described MIDI nodes.

Parameters

out	<i>outPort</i>	port ID of the MIDI node that has unhandled packet
out	<i>outPacket</i>	The MIDI packet

The documentation for this class was generated from the following file:

- [AAX_IACFController.h](#)

14.55 AAX_IACFController_V2 Class Reference

```
#include <AAX_IACFController.h>
```

Inheritance diagram for AAX_IACFController_V2:

Collaboration diagram for AAX_IACFController_V2:

14.55.1 Description

Interface for the AAX host's view of a single instance of an effect. Used by both clients of the AAXHost and by effect components.

Public Member Functions

- virtual [AAX_Result SendNotification](#) ([AAX_CTypeID](#) inNotificationType, const void *inNotificationData, uint32_t inNotificationDataSize)=0
CALL: Dispatch a notification.
- virtual [AAX_Result GetHybridSignalLatency](#) (int32_t *outSamples) const =0
CALL: Returns the latency between the algorithm normal input samples and the inputs returning from the hybrid component.
- virtual [AAX_Result GetCurrentAutomationTimestamp](#) ([AAX_CTransportCounter](#) *outTimestamp) const =0
CALL: Returns the current automation timestamp if called during the [GenerateCoefficients\(\)](#) call AND the generation of coefficients is being triggered by an automation point instead of immediate changes.
- virtual [AAX_Result GetHostName](#) ([AAX_IString](#) *outHostNameString) const =0
CALL: Returns name of the host application this plug-in instance is being loaded by. This string also typically includes version information.

14.55.2 Member Function Documentation

14.55.2.1 SendNotification()

```
virtual AAX_Result AAX_IACFController_V2::SendNotification (
    AAX_CTypeID inNotificationType,
    const void * inNotificationData,
    uint32_t inNotificationDataSize ) [pure virtual]
```

CALL: Dispatch a notification.

The notification is handled by the host and may be delivered back to other plug-in components such as the GUI or data model (via [AAX_IEffectGUI::NotificationReceived\(\)](#) or [AAX_IEffectParameters::NotificationReceived\(\)](#), respectively) depending on the notification type.

The host may choose to dispatch the posted notification either synchronously or asynchronously.

See the [AAX_ENotificationEvent](#) documentation for more information.

This method is supported by AAX V2 Hosts only. Check the return code on the return of this function. If the error is [AAX_ERROR_UNIMPLEMENTED](#), your plug-in is being loaded into a host that doesn't support this feature.

Parameters

in	<i>inNotificationType</i>	Type of notification to send
in	<i>inNotificationData</i>	Block of notification data
in	<i>inNotificationDataSize</i>	Size of <i>inNotificationData</i> , in bytes

14.55.2.2 GetHybridSignalLatency()

```
virtual AAX_Result AAX_IACFController_V2::GetHybridSignalLatency (
    int32_t * outSamples ) const [pure virtual]
```

CALL: Returns the latency between the algorithm normal input samples and the inputs returning from the hybrid component.

This method provides the number of samples that the AAX host expects the plug-in to delay a signal. The host will use this value when accounting for latency across the system.

Note

This value will generally scale up with sample rate, although it's not a simple multiple due to some fixed overhead. This value will be fixed for any given sample rate regardless of other buffer size settings in the host app.

Parameters

out	<i>outSamples</i>	The number of samples of hybrid signal delay
-----	-------------------	--

14.55.2.3 GetCurrentAutomationTimestamp()

```
virtual AAX_Result AAX_IACFController_V2::GetCurrentAutomationTimestamp (
    AAX_CTransportCounter * outTimestamp ) const [pure virtual]
```

CALL: Returns the current automation timestamp if called during the [GenerateCoefficients\(\)](#) call AND the generation of coefficients is being triggered by an automation point instead of immediate changes.

Note

This function will return 0 if called from outside of [GenerateCoefficients\(\)](#) or if the [GenerateCoefficients\(\)](#) call was initiated due to a non-automated change. In those cases, you can get your sample offset from the transport start using [GetTODLocation\(\)](#).

Parameters

out	<i>outTimestamp</i>	The current coefficient timestamp. Sample count from transport start.
-----	---------------------	---

14.55.2.4 GetHostName()

```
virtual AAX_Result AAX_IACFController_V2::GetHostName (
    AAX_IString * outHostNameString ) const [pure virtual]
```

CALL: Returns name of the host application this plug-in instance is being loaded by. This string also typically includes version information.

Host Compatibility Notes Pro Tools versions from Pro Tools 11.0 to Pro Tools 12.3.1 will return a generic version string to this call. This issue is resolved beginning in Pro Tools 12.4.

Parameters

out	<i>outHostNameString</i>	The name of the current host application.
-----	--------------------------	---

The documentation for this class was generated from the following file:

- [AAX_IACFController.h](#)

14.56 AAX_IACFController_V3 Class Reference

```
#include <AAX_IACFController.h>
```

Inheritance diagram for AAX_IACFController_V3:

Collaboration diagram for AAX_IACFController_V3:

14.56.1 Description

Interface for the AAX host's view of a single instance of an effect. Used by both clients of the AAXHost and by effect components.

Public Member Functions

- virtual [AAX_Result](#) [GetPlugInTargetPlatform](#) ([AAX_CTargetPlatform](#) *outTargetPlatform) const =0
CALL: Returns execution platform type, native or TI.
- virtual [AAX_Result](#) [GetIsAudioSuite](#) ([AAX_CBoolean](#) *outIsAudioSuite) const =0
CALL: Returns true for AudioSuite instances.

14.56.2 Member Function Documentation

14.56.2.1 GetPlugInTargetPlatform()

```
virtual AAX\_Result AAX_IACFController_V3::GetPlugInTargetPlatform (
    AAX\_CTargetPlatform * outTargetPlatform ) const [pure virtual]
```

CALL: Returns execution platform type, native or TI.

Parameters

out	<i>outTargetPlatform</i>	The type of the current execution platform as one of AAX_ETargetPlatform .
-----	--------------------------	--

14.56.2.2 GetIsAudioSuite()

```
virtual AAX\_Result AAX_IACFController_V3::GetIsAudioSuite (
    AAX\_CBoolean * outIsAudioSuite ) const [pure virtual]
```

CALL: Returns true for AudioSuite instances.

Parameters

out	<i>outIsAudioSuite</i>	The boolean flag which indicate true for AudioSuite instances.
-----	------------------------	--

The documentation for this class was generated from the following file:

- [AAX_IACFController.h](#)

14.57 AAX_IACFDataBuffer Class Reference

```
#include <AAX_IACFDataBuffer.h>
```

Inheritance diagram for AAX_IACFDataBuffer:

Collaboration diagram for AAX_IACFDataBuffer:

14.57.1 Description

Versioned interface for reference counted data buffers.

This interface is intended to be used for passing arbitrary blocks of data across the binary boundary and allowing the receiver to take ownership of the allocated memory.

Public Member Functions

- virtual [AAX_Result Type](#) ([AAX_CTypeID](#) *oType) const =0
- virtual [AAX_Result Size](#) (int32_t *oSize) const =0
- virtual [AAX_Result Data](#) (void const **oBuffer) const =0

14.57.2 Member Function Documentation

14.57.2.1 Type()

```
virtual AAX\_Result AAX_IACFDataBuffer::Type (
    AAX\_CTypeID * oType ) const [pure virtual]
```

The type of data contained in this buffer

This identifier must be sufficient for a client that knows the type to correctly interpret and use the data.

Implemented in [AAX_CStringDataBuffer](#), [AAX_CStringDataBufferOfType< T >](#), [AAX_CArrayDataBuffer< D >](#), and [AAX_CArrayDataBufferOfType< T, D >](#).

14.57.2.2 Size()

```
virtual AAX\_Result AAX_IACFDataBuffer::Size (
    int32_t * oSize ) const [pure virtual]
```

The number of bytes of data in this buffer

Implemented in [AAX_CStringDataBuffer](#), [AAX_CStringDataBufferOfType< T >](#), [AAX_CArrayDataBuffer< D >](#), and [AAX_CArrayDataBufferOfType< T, D >](#).

14.57.2.3 Data()

```
virtual AAX_Result AAX_IACFDataBuffer::Data (
    void const ** oBuffer ) const [pure virtual]
```

The buffer of data

Implemented in [AAX_CStringDataBuffer](#), [AAX_CStringDataBufferOfType< T >](#), [AAX_CArrayDataBuffer< D >](#), and [AAX_CArrayDataBufferOfType< T, D >](#).

The documentation for this class was generated from the following file:

- [AAX_IACFDataBuffer.h](#)

14.58 AAX_IACFDescriptionHost Class Reference

```
#include <AAX_IACFDescriptionHost.h>
```

Inheritance diagram for AAX_IACFDescriptionHost:

Collaboration diagram for AAX_IACFDescriptionHost:

14.58.1 Description

Interface to host services provided during plug-in description

Public Member Functions

- virtual [AAX_Result](#) [AcquireFeatureProperties](#) (const [AAX_Feature_UID](#) &inFeatureID, [IACFUnknown](#) **outFeatureProperties)=0

14.58.2 Member Function Documentation

14.58.2.1 AcquireFeatureProperties()

```
virtual AAX_Result AAX_IACFDescriptionHost::AcquireFeatureProperties (
    const AAX_Feature_UID & inFeatureID,
    IACFUnknown ** outFeatureProperties ) [pure virtual]
```

outFeatureProperties must support [AAX_IACFFeatureInfo](#) const methods

See also

[AAX_IDescriptionHost::AcquireFeatureProperties\(\)](#)

The documentation for this class was generated from the following file:

- [AAX_IACFDescriptionHost.h](#)

14.59 AAX_IACFEffectorDescriptor Class Reference

```
#include <AAX_IACFEffectorDescriptor.h>
```

Inheritance diagram for AAX_IACFEffectorDescriptor:

Collaboration diagram for AAX_IACFEffectorDescriptor:

14.59.1 Description

Versioned interface for an [AAX_IEffectDescriptor](#).

Public Member Functions

- virtual [AAX_Result](#) [AddComponent](#) ([IACFUnknown](#) *inComponentDescriptor)=0
Add a component to an instance of a component descriptor.
- virtual [AAX_Result](#) [AddName](#) (const char *inPlugInName)=0
Add a name to the Effect.
- virtual [AAX_Result](#) [AddCategory](#) (uint32_t inCategory)=0
Add a category to your plug-in. See [AAX_EPlugInCategory](#).
- virtual [AAX_Result](#) [AddCategoryBypassParameter](#) (uint32_t inCategory, [AAX_CParamID](#) inParamID)=0
Add a category to your plug-in. See [AAX_EPlugInCategory](#).
- virtual [AAX_Result](#) [AddProcPtr](#) (void *inProcPtr, [AAX_CProcPtrID](#) inProcID)=0
Add a process pointer.
- virtual [AAX_Result](#) [SetProperties](#) ([IACFUnknown](#) *inProperties)=0
Set the properties of a new property map.
- virtual [AAX_Result](#) [AddResourceInfo](#) ([AAX_EResourceType](#) inResourceType, const char *inInfo)=0
Set resource file info.
- virtual [AAX_Result](#) [AddMeterDescription](#) ([AAX_CTypeID](#) inMeterID, const char *inMeterName, [IACFUnknown](#) *inProperties)=0
Add name and property map to meter with given ID.

14.59.2 Member Function Documentation

14.59.2.1 AddComponent()

```
virtual AAX\_Result AAX_IACFEffectorDescriptor::AddComponent (
    IACFUnknown * inComponentDescriptor ) [pure virtual]
```

Add a component to an instance of a component descriptor.

Unlike with [AAX_ICollection::AddEffect\(\)](#), the [AAX_IEffectDescriptor](#) does not take ownership of the [AAX_IComponentDescriptor](#) that is passed to it in this method. The host copies out the contents of this descriptor, and thus the plug-in may re-use the same descriptor object when creating additional similar components.

Parameters

in	<i>inComponentDescriptor</i>	
----	------------------------------	--

14.59.2.2 AddName()

```
virtual AAX_Result AAX_IACEffectDescriptor::AddName (
    const char * inPlugInName ) [pure virtual]
```

Add a name to the Effect.

May be called multiple times to add abbreviated Effect names.

Note

Every Effect must include at least one name variant with 31 or fewer characters, plus a null terminating character

Parameters

in	<i>inPlugInName</i>	The name assigned to the plug-in.
----	---------------------	-----------------------------------

14.59.2.3 AddCategory()

```
virtual AAX_Result AAX_IACEffectDescriptor::AddCategory (
    uint32_t inCategory ) [pure virtual]
```

Add a category to your plug-in. See [AAX_EPlugInCategory](#).

Parameters

in	<i>inCategory</i>	One of the categories for the plug-in.
----	-------------------	--

14.59.2.4 AddCategoryBypassParameter()

```
virtual AAX_Result AAX_IACEffectDescriptor::AddCategoryBypassParameter (
    uint32_t inCategory,
    AAX_CParamID inParamID ) [pure virtual]
```

Add a category to your plug-in. See [AAX_EPlugInCategory](#).

Parameters

in	<i>inCategory</i>	One of the categories for the plug-in.
in	<i>inParamID</i>	The parameter ID of the parameter used to bypass the category section of the plug-in.

14.59.2.5 AddProcPtr()

```
virtual AAX_Result AAX_IACFEffectorDescriptor::AddProcPtr (
    void * inProcPtr,
    AAX_CProcPtrID inProcID ) [pure virtual]
```

Add a process pointer.

Parameters

in	<i>inProcPtr</i>	A process pointer.
in	<i>inProcID</i>	A process ID.

14.59.2.6 SetProperty()

```
virtual AAX_Result AAX_IACFEffectorDescriptor::SetProperties (
    IACFUnknown * inProperties ) [pure virtual]
```

Set the properties of a new property map.

Parameters

in	<i>inProperties</i>	Description
----	---------------------	-------------

14.59.2.7 AddResourceInfo()

```
virtual AAX_Result AAX_IACFEffectorDescriptor::AddResourceInfo (
    AAX_EResourceType inResourceType,
    const char * inInfo ) [pure virtual]
```

Set resource file info.

Parameters

in	<i>inResourceType</i>	See AAX_EResourceType.
in	<i>inInfo</i>	Definition varies on the resource type.

14.59.2.8 AddMeterDescription()

```
virtual AAX_Result AAX_IACFEffectorDescriptor::AddMeterDescription (
    AAX_CTypeID inMeterID,
    const char * inMeterName,
    IACFUnknown * inProperties ) [pure virtual]
```

Add name and property map to meter with given ID.

Parameters

in	<i>inMeterID</i>	The ID of the meter being described.
in	<i>inMeterName</i>	The name of the meter.
in	<i>inProperties</i>	The property map containing meter related data such as meter type, orientation, etc.

The documentation for this class was generated from the following file:

- [AAX_IACFEffectorDescriptor.h](#)

14.60 AAX_IACFEffectorDescriptor_V2 Class Reference

```
#include <AAX_IACFEffectorDescriptor.h>
```

Inheritance diagram for AAX_IACFEffectorDescriptor_V2:

Collaboration diagram for AAX_IACFEffectorDescriptor_V2:

14.60.1 Description

Versioned interface for an [AAX_IEffectDescriptor](#).

Public Member Functions

- virtual [AAX_Result](#) [AddControlMIDINode](#) ([AAX_CTypeID](#) inNodeID, [AAX_EMIDINodeType](#) inNodeType, const char inNodeName[], uint32_t inChannelMask)=0

Add a control MIDI node to the plug-in data model.

14.60.2 Member Function Documentation

14.60.2.1 AddControlMIDINode()

```
virtual AAX_Result AAX_IACFEffectorDescriptor_V2::AddControlMIDINode (
    AAX_CTypeID inNodeID,
    AAX_EMIDINodeType inNodeType,
    const char inNodeName[],
    uint32_t inChannelMask ) [pure virtual]
```

Add a control MIDI node to the plug-in data model.

- This MIDI node may receive note data as well as control data.
- To send MIDI data to the plug-in's algorithm, use [AAX_IComponentDescriptor::AddMIDINode\(\)](#).

See also

[AAX_IACFEffectorParameters_V2::UpdateControlMIDINodes\(\)](#)

Parameters

in	<i>inNodeID</i>	The ID for the new control MIDI node.
in	<i>inNodeType</i>	The type of the node.
in	<i>inNodeName</i>	The name of the node.
in	<i>inChannelMask</i>	The bit mask for required nodes channels (up to 16) or required global events for global node.

The documentation for this class was generated from the following file:

- [AAX_IACFEffectorDescriptor.h](#)

14.61 AAX_IACFEffectorDirectData Class Reference

```
#include <AAX_IACFEffectorDirectData.h>
```

Inheritance diagram for AAX_IACFEffectorDirectData:

Collaboration diagram for AAX_IACFEffectorDirectData:

14.61.1 Description

Optional interface for direct access to a plug-in's alg memory.

Direct data access allows a plug-in to directly manipulate the data in its algorithm's private data blocks. The callback methods in this interface provide a safe context from which this kind of access may be attempted.

Public Member Functions

Initialization and uninitialization

- virtual [AAX_Result Initialize](#) ([IACFUnknown](#) *iController)=0
Main initialization.
- virtual [AAX_Result Uninitialize](#) ()=0
Main uninitialization.

Safe data update callbacks

These callbacks provide a safe context from which to directly access the algorithm's private data blocks. Each callback is called regularly with roughly the schedule of its corresponding [AAX_IEffectParameters](#) counterpart.

Note

Do not attempt to directly access the algorithm's data from outside these callbacks. Instead, use the packet system to route data to the algorithm using the AAX host's buffered data transfer facilities.

- virtual [AAX_Result TimerWakeup](#) ([IACFUnknown](#) *iDataAccessInterface)=0
Periodic wakeup callback for idle-time operations.

14.61.2 Member Function Documentation

14.61.2.1 Initialize()

```
virtual AAX\_Result AAX_IACFEfffectDirectData::Initialize (
    IACFUnknown * iController ) [pure virtual]
```

Main initialization.

Called when the interface is created

Parameters

<code>in</code>	<code>iController</code>	A versioned reference that resolves to an AAX_IController interface
-----------------	--------------------------	---

Implemented in [AAX_CEffectDirectData](#).

14.61.2.2 Uninitialize()

```
virtual AAX\_Result AAX_IACFEfffectDirectData::Uninitialize ( ) [pure virtual]
```

Main uninitialization.

Called when the interface is destroyed.

Implemented in [AAX_CEffectDirectData](#).

14.61.2.3 TimerWakeup()

```
virtual AAX_Result AAX_IACFEEffectDirectData::TimerWakeup (
    IACFUnknown * iDataAccessInterface ) [pure virtual]
```

Periodic wakeup callback for idle-time operations.

Direct alg data updates must be triggered from this method.

This method is called from the host using a non-main thread. In general, it should be driven at approximately one call per 30 ms. However, the wakeup is not guaranteed to be called at any regular interval - for example, it could be held off by a high real-time processing load - and there is no host contract regarding maximum latency between wakeup calls.

This wakeup thread runs continuously and cannot be armed/disarmed or by the plug-in.

Parameters

in	<i>iDataAccessInterface</i>	Reference to the direct access interface.
----	-----------------------------	---

Note

It is not safe to save this address or call the methods in this interface from other threads.

Implemented in [AAX_CEEffectDirectData](#).

The documentation for this class was generated from the following file:

- [AAX_IACFEEffectDirectData.h](#)

14.62 AAX_IACFEEffectDirectData_V2 Class Reference

```
#include <AAX_IACFEEffectDirectData.h>
```

Inheritance diagram for AAX_IACFEEffectDirectData_V2:

Collaboration diagram for AAX_IACFEEffectDirectData_V2:

Public Member Functions

AAX host and plug-in event notification

- virtual [AAX_Result NotificationReceived](#) ([AAX_CTypeID](#) inNotificationType, const void *inNotificationData, uint32_t inNotificationDataSize)=0
Notification Hook.

14.62.1 Member Function Documentation

14.62.1.1 NotificationReceived()

```
virtual AAX_Result AAX_IACFEffEffectDirectData_V2::NotificationReceived (
    AAX_CTypeID inNotificationType,
    const void * inNotificationData,
    uint32_t inNotificationDataSize ) [pure virtual]
```

Notification Hook.

Called from the host to deliver notifications to this object.

Look at the [AAX_ENotificationEvent](#) enumeration to see a description of events you can listen for and the data they come with.

Note

- some notifications are sent only to the plug-in GUI while other notifications are sent only to the plug-in data model. If you are not seeing an expected notification, try checking the other plug-in objects' [NotificationReceived\(\)](#) methods.

Note

- the host may dispatch notifications synchronously or asynchronously, and calls to this method may occur concurrently on multiple threads.

A plug-in may also dispatch custom notifications using [AAX_IController::SendNotification\(\)](#). Custom notifications will be posted back to the plug-in's other objects which support a [NotificationReceived\(\)](#) method (e.g. the GUI).

Parameters

in	<i>inNotificationType</i>	Type of notification being received. Notifications from the host are one of AAX_ENotificationEvent
in	<i>inNotificationData</i>	Block of incoming notification data
in	<i>inNotificationDataSize</i>	Size of <i>inNotificationData</i> , in bytes

Implemented in [AAX_CEffEffectDirectData](#).

The documentation for this class was generated from the following file:

- [AAX_IACFEffEffectDirectData.h](#)

14.63 AAX_IACFEffEffectGUI Class Reference

```
#include <AAX_IACFEffEffectGUI.h>
```

Inheritance diagram for AAX_IACFEffEffectGUI:

Collaboration diagram for AAX_IACFEffEffectGUI:

14.63.1 Description

The interface for a AAX Plug-in's GUI (graphical user interface).

This is the interface for an instance of a plug-in's GUI that gets exposed to the host application. The AAX host interacts with your plug-in's GUI via this interface. See [GUI interface](#).

The plug-in's implementation of this interface is responsible for managing the plug-in's window and graphics objects and for defining the interactions between GUI views and the plug-in's data model.

At [initialization](#), the host provides this interface with a reference to [AAX_IController](#). The GUI may use this controller to retrieve a pointer to the plug-in's [AAX_IEffectParameters](#) interface, allowing the GUI to request changes to the plug-in's data model in response to view events. In addition, the controller provides a means of querying information from the host such as stem format or sample rate

When managing a plug-in's GUI it is important to remember that this is just one of many possible sets of views for the plug-in's parameters. Other views and editors, such as automation lanes or control surfaces, also have the ability to synchronously interact with the plug-in's abstract data model interface. Therefore, the GUI should not take asymmetric control over the data model, act as a secondary data model, or otherwise assume exclusive ownership of the plug-in's state. In general, the data model's abstraction to a pure virtual interface will protect against such aberrations, but this remains an important consideration when managing sophisticated GUI interactions.

You will most likely inherit your implementation of this interface from [AAX_CEffectGUI](#), a default implementation that provides basic GUI functionality and which you can override and customize as needed.

The SDK includes several examples of how the GUI interface may be extended and implemented in order to provide support for third-party frameworks. These examples can be found in the /Extensions/GUI directory in the SDK.

Note

Your implementation of this interface must inherit from [AAX_IEffectGUI](#).

Legacy Porting Notes In the legacy plug-in SDK, these methods were found in CProcess and CEffectProcess. For additional CProcess methods, see [AAX_IEffectParameters](#).

Public Member Functions

Initialization and uninitialization

- virtual [AAX_Result Initialize](#) ([IACFUnknown](#) *iController)=0
Main GUI initialization.
- virtual [AAX_Result Uninitialize](#) ()=0
Main GUI uninitialization.

AAX host and plug-in event notification

- virtual [AAX_Result NotificationReceived](#) ([AAX_CTypeID](#) inNotificationType, const void *inNotificationData, uint32_t inNotificationDataSize)=0
Notification Hook.

View accessors

- virtual [AAX_Result SetViewContainer](#) ([IACFUnknown](#) *iViewContainer)=0
Provides a handle to the main plug-in window.
- virtual [AAX_Result GetViewSize](#) ([AAX_Point](#) *oViewSize) const =0

Retrieves the size of the plug-in window.

GUI update methods

- virtual [AAX_Result Draw](#) ([AAX_Rect](#) *iDrawRect)=0
DEPRECATED, Not called from host any longer. Your chosen graphics framework should be directly handling draw events from the OS.
- virtual [AAX_Result TimerWakeup](#) ()=0
Periodic wakeup callback for idle-time operations.
- virtual [AAX_Result ParameterUpdated](#) ([AAX_CParamID](#) inParamID)=0
Notifies the GUI that a parameter value has changed.

Host interface methods

Miscellaneous methods to provide host-specific functionality

- virtual [AAX_Result GetCustomLabel](#) ([AAX_EPlugInStrings](#) iSelector, [AAX_IString](#) *oString) const =0
Called by host application to retrieve a custom plug-in string.
- virtual [AAX_Result SetControlHighlightInfo](#) ([AAX_CParamID](#) iParameterID, [AAX_CBoolean](#) iIsHighlighted, [AAX_EHighlightColor](#) iColor)=0
Called by host application. Indicates that a control widget should be updated with a highlight color.

14.63.2 Member Function Documentation

14.63.2.1 Initialize()

```
virtual AAX\_Result AAX_IACEffectGUI::Initialize (
    IACFUnknown * iController ) [pure virtual]
```

Main GUI initialization.

Called when the GUI is created

Parameters

in	<i>iController</i>	A versioned reference that resolves to an AAX_IController interface
----	--------------------	---

Implemented in [AAX_CEffectGUI](#).

14.63.2.2 Uninitialize()

```
virtual AAX\_Result AAX_IACEffectGUI::Uninitialize ( ) [pure virtual]
```

Main GUI uninitialization.

Called when the GUI is destroyed. Frees the GUI.

Implemented in [AAX_CEffectGUI](#).

14.63.2.3 NotificationReceived()

```
virtual AAX_Result AAX_IACFEEffectGUI::NotificationReceived (
    AAX_CTypeID inNotificationType,
    const void * inNotificationData,
    uint32_t inNotificationDataSize ) [pure virtual]
```

Notification Hook.

Called from the host to deliver notifications to this object.

Look at the [AAX_ENotificationEvent](#) enumeration to see a description of events you can listen for and the data they come with.

Note

- some notifications are sent only to the plug-in GUI while other notifications are sent only to the plug-in data model. If you are not seeing an expected notification, try checking the other plug-in objects' [NotificationReceived\(\)](#) methods.

Note

- the host may dispatch notifications synchronously or asynchronously, and calls to this method may occur concurrently on multiple threads.

A plug-in may also dispatch custom notifications using [AAX_IController::SendNotification\(\)](#). Custom notifications will be posted back to the plug-in's other objects which support a [NotificationReceived\(\)](#) method (e.g. the data model).

Parameters

in	<i>inNotificationType</i>	Type of notification being received. Notifications from the host are one of AAX_ENotificationEvent
in	<i>inNotificationData</i>	Block of incoming notification data
in	<i>inNotificationDataSize</i>	Size of <i>inNotificationData</i> , in bytes

Implemented in [AAX_CEffectGUI](#).

14.63.2.4 SetViewContainer()

```
virtual AAX_Result AAX_IACFEEffectGUI::SetViewContainer (
    IACFUnknown * iViewContainer ) [pure virtual]
```

Provides a handle to the main plug-in window.

Parameters

in	<i>iViewContainer</i>	An AAX_IViewContainer providing a native handle to the plug-in's window
----	-----------------------	---

Implemented in [AAX_CEffectGUI](#).

14.63.2.5 GetViewSize()

```
virtual AAX_Result AAX_IACFEEffectGUI::GetViewSize (
    AAX_Point * oViewSize ) const [pure virtual]
```

Retrieves the size of the plug-in window.

See also

[AAX_IViewContainer::SetViewSize\(\)](#)

Parameters

out	<i>oViewSize</i>	The size of the plug-in window as a point (width, height)
-----	------------------	---

Implemented in [AAX_CEffectGUI](#).

14.63.2.6 Draw()

```
virtual AAX_Result AAX_IACFEEffectGUI::Draw (
    AAX_Rect * iDrawRect ) [pure virtual]
```

DEPRECATED, Not called from host any longer. Your chosen graphics framework should be directly handling draw events from the OS.

Implemented in [AAX_CEffectGUI](#).

14.63.2.7 TimerWakeup()

```
virtual AAX_Result AAX_IACFEEffectGUI::TimerWakeup ( ) [pure virtual]
```

Periodic wakeup callback for idle-time operations.

GUI animation events such as meter updates should be triggered from this method.

This method is called from the host's main thread. In general, it should be driven at approximately one call per 30 ms. However, the wakeup is not guaranteed to be called at any regular interval - for example, it could be held off by a high real-time processing load - and there is no host contract regarding maximum latency between wakeup calls.

This wakeup runs continuously and cannot be armed/disarmed by the plug-in.

Implemented in [AAX_CEffectGUI](#).

14.63.2.8 ParameterUpdated()

```
virtual AAX_Result AAX_IACFEEffectGUI::ParameterUpdated (
    AAX_CParamID inParamID ) [pure virtual]
```

Notifies the GUI that a parameter value has changed.

This method is called by the host whenever a parameter value has been modified

This method may be called on a non-main thread

Implemented in [AAX_CEffectGUI](#).

14.63.2.9 GetCustomLabel()

```
virtual AAX_Result AAX_IACFEEffectGUI::GetCustomLabel (
    AAX_EPlugInStrings iSelector,
    AAX_IString * oString ) const [pure virtual]
```

Called by host application to retrieve a custom plug-in string.

If no string is provided then the host's default will be used.

Parameters

in	<i>iSelector</i>	The requested strong. One of AAX_EPlugInStrings
out	<i>oString</i>	The plug-in's custom value for the requested string

Implemented in [AAX_CEffectGUI](#).

14.63.2.10 SetControlHighlightInfo()

```
virtual AAX_Result AAX_IACFEEffectGUI::SetControlHighlightInfo (
    AAX_CParamID iParameterID,
    AAX_CBoolean iIsHighlighted,
    AAX_EHighlightColor iColor ) [pure virtual]
```

Called by host application. Indicates that a control widget should be updated with a highlight color.

Todo Document this method

Legacy Porting Notes This method was re-named from `SetControlHighliteInfo()`, its name in the legacy plug-in SDK.

Parameters

in	<i>iParameterID</i>	ID of parameter whose widget(s) must be highlighted
in	<i>isHighlighted</i>	True if turning highlight on, false if turning it off
in	<i>iColor</i>	Desired highlight color. One of AAX_EHighlightColor

Implemented in [AAX_CEffEffectGUI](#).

The documentation for this class was generated from the following file:

- [AAX_IACFEffEffectGUI.h](#)

14.64 AAX_IACFEffEffectParameters Class Reference

```
#include <AAX_IACFEffEffectParameters.h>
```

Inheritance diagram for AAX_IACFEffEffectParameters:

Collaboration diagram for AAX_IACFEffEffectParameters:

14.64.1 Description

The interface for an AAX Plug-in's data model.

This is the interface for an instance of a plug-in's data model that gets exposed to the host application. The AAX host interacts with your plug-in's data model via this interface, which includes methods that store and update of your plug-in's internal data. See [Data model interface](#).

Note

Your implementation of this interface must inherit from [AAX_IEffEffectParameters](#).

Todo Add documentation for expected error state return values

Public Member Functions

Initialization and uninitialization

- virtual [AAX_Result Initialize](#) ([IACFUnknown](#) *iController)=0
Main data model initialization. Called when plug-in instance is first instantiated.
- virtual [AAX_Result Uninitialize](#) ()=0
Main data model uninitialization.

AAX host and plug-in event notification

- virtual [AAX_Result NotificationReceived](#) ([AAX_CTypeID](#) inNotificationType, const void *inNotificationData, uint32_t inNotificationDataSize)=0
Notification Hook.

Parameter information

These methods are used by the AAX host to retrieve information about the plug-in's data model.

For information about adding parameters to the plug-in and otherwise modifying the plug-in's data model, see [AAX_CParameterManager](#). For information about parameters, see [AAX_IParameter](#).

- virtual [AAX_Result](#) [GetNumberOfParameters](#) (int32_t *oNumControls) const =0
CALL: Retrieves the total number of plug-in parameters.
- virtual [AAX_Result](#) [GetMasterBypassParameter](#) (AAX_IString *oIDString) const =0
CALL: Retrieves the ID of the plug-in's Master Bypass parameter.
- virtual [AAX_Result](#) [GetParameterIsAutomatable](#) (AAX_CParamID iParameterID, AAX_CBoolean *oAutomatable) const =0
CALL: Retrieves information about a parameter's automatable status.
- virtual [AAX_Result](#) [GetParameterNumberOfSteps](#) (AAX_CParamID iParameterID, int32_t *oNumSteps) const =0
CALL: Retrieves the number of discrete steps for a parameter.
- virtual [AAX_Result](#) [GetParameterName](#) (AAX_CParamID iParameterID, AAX_IString *oName) const =0
CALL: Retrieves the full name for a parameter.
- virtual [AAX_Result](#) [GetParameterNameOfLength](#) (AAX_CParamID iParameterID, AAX_IString *oName, int32_t iNameLength) const =0
CALL: Retrieves an abbreviated name for a parameter.
- virtual [AAX_Result](#) [GetParameterDefaultNormalizedValue](#) (AAX_CParamID iParameterID, double *oValue) const =0
CALL: Retrieves default value of a parameter.
- virtual [AAX_Result](#) [SetParameterDefaultNormalizedValue](#) (AAX_CParamID iParameterID, double iValue)=0
CALL: Sets the default value of a parameter.
- virtual [AAX_Result](#) [GetParameterType](#) (AAX_CParamID iParameterID, AAX_EParameterType *oParameterType) const =0
CALL: Retrieves the type of a parameter.
- virtual [AAX_Result](#) [GetParameterOrientation](#) (AAX_CParamID iParameterID, AAX_EParameterOrientation *oParameterOrientation) const =0
CALL: Retrieves the orientation that should be applied to a parameter's controls.
- virtual [AAX_Result](#) [GetParameter](#) (AAX_CParamID iParameterID, AAX_IParameter **oParameter)=0
CALL: Retrieves an arbitrary setting within a parameter.
- virtual [AAX_Result](#) [GetParameterIndex](#) (AAX_CParamID iParameterID, int32_t *oControllIndex) const =0
CALL: Retrieves the index of a parameter.
- virtual [AAX_Result](#) [GetParameterIDFromIndex](#) (int32_t iControllIndex, AAX_IString *oParameterIDString) const =0
CALL: Retrieves the ID of a parameter.
- virtual [AAX_Result](#) [GetParameterValueInfo](#) (AAX_CParamID iParameterID, int32_t iSelector, int32_t *oValue) const =0
CALL: Retrieves a property of a parameter.

Parameter setters and getters

These methods are used by the AAX host and by the plug-in's UI to retrieve and modify the values of the plug-in's parameters.

Note

The parameter setters in this section may generate asynchronous requests.

- virtual [AAX_Result](#) [GetParameterValueFromString](#) (AAX_CParamID iParameterID, double *oValue, const AAX_IString &iValueString) const =0
CALL: Converts a value string to a value.
- virtual [AAX_Result](#) [GetParameterStringFromValue](#) (AAX_CParamID iParameterID, double iValue, AAX_IString *oValueString, int32_t iMaxLength) const =0

- CALL: Converts a normalized parameter value into a string representing its corresponding real value.*
- virtual [AAX_Result GetParameterValueString](#) ([AAX_CParamID](#) iParameterID, [AAX_IString](#) *oValueString, [int32_t](#) iMaxLength) const =0
 - CALL: Retrieves the value string associated with a parameter's current value.*
- virtual [AAX_Result GetParameterNormalizedValue](#) ([AAX_CParamID](#) iParameterID, double *oValuePtr) const =0
 - CALL: Retrieves a parameter's current value.*
- virtual [AAX_Result SetParameterNormalizedValue](#) ([AAX_CParamID](#) iParameterID, double iValue)=0
 - CALL: Sets the specified parameter to a new value.*
- virtual [AAX_Result SetParameterNormalizedRelative](#) ([AAX_CParamID](#) iParameterID, double iValue)=0
 - CALL: Sets the specified parameter to a new value relative to its current value.*

Automated parameter helpers

These methods are used to lock and unlock automation system 'resources' when updating automatable parameters.

Note

You should never need to override these methods to extend their behavior beyond what is provided in [AAX_CEffectParameters](#) and [AAX_IParameter](#)

- virtual [AAX_Result TouchParameter](#) ([AAX_CParamID](#) iParameterID)=0
 - "Touches" (locks) a parameter in the automation system to a particular control in preparation for updates*
- virtual [AAX_Result ReleaseParameter](#) ([AAX_CParamID](#) iParameterID)=0
 - Releases a parameter from a "touched" state.*
- virtual [AAX_Result UpdateParameterTouch](#) ([AAX_CParamID](#) iParameterID, [AAX_CBoolean](#) iTouch↔State)=0
 - Sets a "touched" state on a parameter.*

Asynchronous parameter update methods

These methods are called by the AAX host when parameter values have been updated. They are called by the host and can be triggered by other plug-in modules via calls to [AAX_IParameter](#)'s *SetValue* methods, e.g. [SetValueWithFloat\(\)](#)

These methods are responsible for updating parameter values.

Do not call these methods directly! To ensure proper synchronization and to avoid problematic dependency chains, other methods (e.g. [SetParameterNormalizedValue\(\)](#)) and components (e.g. [AAX_IEffectGUI](#)) should always call a *SetValue* method on [AAX_IParameter](#) to update parameter values. The *SetValue* method will properly manage automation locks and other system resources.

- virtual [AAX_Result UpdateParameterNormalizedValue](#) ([AAX_CParamID](#) iParameterID, double iValue, [AAX_EUpdateSource](#) iSource)=0
 - Updates a single parameter's state to its current value.*
- virtual [AAX_Result UpdateParameterNormalizedRelative](#) ([AAX_CParamID](#) iParameterID, double i↔Value)=0
 - Updates a single parameter's state to its current value, as a difference with the parameter's previous value.*
- virtual [AAX_Result GenerateCoefficients](#) ()=0
 - Generates and dispatches new coefficient packets.*

State reset handlers

- virtual [AAX_Result ResetFieldData](#) ([AAX_CFieldIndex](#) inFieldIndex, void *oData, [uint32_t](#) inDataSize) const =0
 - Called by the host to reset a private data field in the plug-in's algorithm.*

Chunk methods

These methods are used to save and restore collections of plug-in state information, known as chunks. Chunks are used by the host when saving or restoring presets and session settings and when providing "compare" functionality for plug-ins.

The default implementation of these methods in [AAX_CEffectParameters](#) supports a single chunk that includes state information for all of the plug-in's registered parameters. Override all of these methods to add support for additional chunks in your plug-in, for example if your plug-in contains any persistent state that is not encapsulated by its set of registered parameters.

Warning

Remember that plug-in chunk data may be loaded on a different platform from the one where it is saved. All data structures in the chunk must be properly data-aligned for compatibility across all platforms that the plug-in supports. See [AAX_ALIGN_FILE_ALG](#) for notes about common cross-platform pitfalls for data structure alignment.

For reference, see also:

- [AAX_CChunkDataParser](#)
- [AAX_SPlugInChunk](#)
- virtual [AAX_Result GetNumberOfChunks](#) (int32_t *oNumChunks) const =0
Retrieves the number of chunks used by this plug-in.
- virtual [AAX_Result GetChunkIDFromIndex](#) (int32_t iIndex, [AAX_CTypeID](#) *oChunkID) const =0
Retrieves the ID associated with a chunk index.
- virtual [AAX_Result GetChunkSize](#) ([AAX_CTypeID](#) iChunkID, uint32_t *oSize) const =0
Get the size of the data structure that can hold all of a chunk's information.
- virtual [AAX_Result GetChunk](#) ([AAX_CTypeID](#) iChunkID, [AAX_SPlugInChunk](#) *oChunk) const =0
Fills a block of data with chunk information representing the plug-in's current state.
- virtual [AAX_Result SetChunk](#) ([AAX_CTypeID](#) iChunkID, const [AAX_SPlugInChunk](#) *iChunk)=0
Restores a set of plug-in parameters based on chunk information.
- virtual [AAX_Result CompareActiveChunk](#) (const [AAX_SPlugInChunk](#) *iChunkP, [AAX_CBoolean](#) *oIsEqual) const =0
Determine if a chunk represents settings that are equivalent to the plug-in's current state.
- virtual [AAX_Result GetNumberOfChanges](#) (int32_t *oNumChanges) const =0
Retrieves the number of parameter changes made since the plug-in's creation.

Thread methods

- virtual [AAX_Result TimerWakeup](#) ()=0
Periodic wakeup callback for idle-time operations.

Auxiliary UI methods

- virtual [AAX_Result GetCurveData](#) ([AAX_CTypeID](#) iCurveType, const float *iValues, uint32_t iNumValues, float *oValues) const =0
Generate a set of output values based on a set of given input values.

Custom data methods

These functions exist as a proxiable way to move data between different modules (e.g. [AAX_IEffectParameters](#) and [AAX_IEffectGUI](#).) Using these, the GUI can query any data through [GetCustomData\(\)](#) with a plug-in defined `typeID`, `void*` and size. This has an advantage over just sharing memory in that this function can work as a remote proxy as we enable those sorts of features later in the platform. Likewise, the GUI can also set arbitrary data on the data model by using the [SetCustomData\(\)](#) function with the same idea.

Note

These are plug-in internal only. They are not called from the host right now, or likely ever.

- virtual [AAX_Result GetCustomData](#) ([AAX_CTypeID](#) iDataBlockID, uint32_t inDataSize, void *oData, uint32_t *oDataWritten) const =0
An optional interface hook for getting custom data from another module.
- virtual [AAX_Result SetCustomData](#) ([AAX_CTypeID](#) iDataBlockID, uint32_t inDataSize, const void *iData) =0
An optional interface hook for setting custom data for use by another module.

MIDI methods

- virtual [AAX_Result DoMIDITransfers](#) ()=0
MIDI update callback.

14.64.2 Member Function Documentation**14.64.2.1 Initialize()**

```
virtual AAX\_Result AAX_IACEffectParameters::Initialize (
    IACFUnknown * iController ) [pure virtual]
```

Main data model initialization. Called when plug-in instance is first instantiated.

Note

Most plug-ins should override [AAX_CEffectParameters::EffectInit\(\)](#) rather than directly overriding this method

Parameters

in	<i>iController</i>	A versioned reference that resolves to an AAX_IController interface
----	--------------------	---

Implemented in [AAX_CEffectParameters](#).

14.64.2.2 Uninitialize()

```
virtual AAX\_Result AAX_IACEffectParameters::Uninitialize ( ) [pure virtual]
```

Main data model uninitialization.

Todo Docs: When exactly is [AAX_IACEffectParameters::Uninitialize\(\)](#) called, and under what conditions?

Implemented in [AAX_CEffectParameters](#).

14.64.2.3 NotificationReceived()

```
virtual AAX_Result AAX_IACFEffParameters::NotificationReceived (
    AAX_CTypeID inNotificationType,
    const void * inNotificationData,
    uint32_t inNotificationDataSize ) [pure virtual]
```

Notification Hook.

Called from the host to deliver notifications to this object.

Look at the [AAX_ENotificationEvent](#) enumeration to see a description of events you can listen for and the data they come with.

Note

- some notifications are sent only to the plug-in GUI while other notifications are sent only to the plug-in data model. If you are not seeing an expected notification, try checking the other plug-in objects' [NotificationReceived\(\)](#) methods.

Note

- the host may dispatch notifications synchronously or asynchronously, and calls to this method may occur concurrently on multiple threads.

A plug-in may also dispatch custom notifications using [AAX_IController::SendNotification\(\)](#). Custom notifications will be posted back to the plug-in's other objects which support a [NotificationReceived\(\)](#) method (e.g. the GUI).

Parameters

in	<i>inNotificationType</i>	Type of notification being received. Notifications from the host are one of AAX_ENotificationEvent
in	<i>inNotificationData</i>	Block of incoming notification data
in	<i>inNotificationDataSize</i>	Size of <i>inNotificationData</i> , in bytes

Implemented in [AAX_CEffectParameters](#).

14.64.2.4 GetNumberOfParameters()

```
virtual AAX_Result AAX_IACFEffParameters::GetNumberOfParameters (
    int32_t * oNumControls ) const [pure virtual]
```

CALL: Retrieves the total number of plug-in parameters.

Parameters

out	<i>oNumControls</i>	The number of parameters in the plug-in's Parameter Manager
-----	---------------------	---

Implemented in [AAX_CEffectParameters](#).

14.64.2.5 GetMasterBypassParameter()

```
virtual AAX_Result AAX_IACEffectParameters::GetMasterBypassParameter (
    AAX_IString * oIDString ) const [pure virtual]
```

CALL: Retrieves the ID of the plug-in's Master Bypass parameter.

This is required if you want our master bypass functionality in the host to hook up to your bypass parameters.

Parameters

out	<i>oIDString</i>	The ID of the plug-in's Master Bypass control
-----	------------------	---

Implemented in [AAX_CEffectParameters](#).

14.64.2.6 GetParameterIsAutomatable()

```
virtual AAX_Result AAX_IACEffectParameters::GetParameterIsAutomatable (
    AAX_CParamID iParameterID,
    AAX_CBoolean * oAutomatable ) const [pure virtual]
```

CALL: Retrieves information about a parameter's automatable status.

Parameters

in	<i>iParameterID</i>	The ID of the parameter that is being queried
out	<i>oAutomatable</i>	True if the queried parameter is automatable, false if it is not

Implemented in [AAX_CEffectParameters](#).

14.64.2.7 GetParameterNumberOfSteps()

```
virtual AAX_Result AAX_IACEffectParameters::GetParameterNumberOfSteps (
    AAX_CParamID iParameterID,
    int32_t * oNumSteps ) const [pure virtual]
```

CALL: Retrieves the number of discrete steps for a parameter.

Note

The value returned for *oNumSteps* MUST be greater than zero. All other values will be considered an error by the host.

Parameters

in	<i>iParameterID</i>	The ID of the parameter that is being queried
out	<i>oNumSteps</i>	The number of steps for this parameter

Implemented in [AAX_CEffectParameters](#).

14.64.2.8 GetParameterName()

```
virtual AAX_Result AAX_IACFEEffectParameters::GetParameterName (
    AAX_CParamID iParameterID,
    AAX_IString * oName ) const [pure virtual]
```

CALL: Retrieves the full name for a parameter.

Parameters

in	<i>iParameterID</i>	The ID of the parameter that is being queried
out	<i>oName</i>	Reference to an AAX_IString owned by the host. The plug-in must set this string equal to the parameter's full name.

Implemented in [AAX_CEffectParameters](#).

14.64.2.9 GetParameterNameOfLength()

```
virtual AAX_Result AAX_IACFEEffectParameters::GetParameterNameOfLength (
    AAX_CParamID iParameterID,
    AAX_IString * oName,
    int32_t iNameLength ) const [pure virtual]
```

CALL: Retrieves an abbreviated name for a parameter.

In general, lengths of 3 through 8 and 31 should be specifically addressed.

Host Compatibility Notes In most cases, the AAX host will call [GetParameterName\(\)](#) or [GetParameterNameOfLength\(\)](#) to retrieve parameter names for display. However, when Pro Tools is retrieving a plug-in name for display on a control surface the XML data stored in the plug-in's page tables will be used in preference to values retrieved from these methods.

Parameters

in	<i>iParameterID</i>	The ID of the parameter that is being queried
out	<i>oName</i>	Reference to an AAX_IString owned by the host. The plug-in must set this string equal to an abbreviated name for the parameter, using <i>iNameLength</i> characters or fewer.
in	<i>iNameLength</i>	The maximum number of characters in <i>oName</i>

Implemented in [AAX_CEffectParameters](#).

14.64.2.10 GetParameterDefaultNormalizedValue()

```
virtual AAX_Result AAX_IACEffectParameters::GetParameterDefaultNormalizedValue (
    AAX_CParamID iParameterID,
    double * oValue ) const [pure virtual]
```

CALL: Retrieves default value of a parameter.

Parameters

in	<i>iParameterID</i>	The ID of the parameter that is being queried
out	<i>oValue</i>	The parameter's default value

Implemented in [AAX_CEffectParameters](#).

14.64.2.11 SetParameterDefaultNormalizedValue()

```
virtual AAX_Result AAX_IACEffectParameters::SetParameterDefaultNormalizedValue (
    AAX_CParamID iParameterID,
    double iValue ) [pure virtual]
```

CALL: Sets the default value of a parameter.

Parameters

in	<i>iParameterID</i>	The ID of the parameter that is being updated
out	<i>iValue</i>	The parameter's new default value

Todo THIS IS NOT CALLED FROM HOST. USEFUL FOR INTERNAL USE ONLY?

Implemented in [AAX_CEffectParameters](#).

14.64.2.12 GetParameterType()

```
virtual AAX_Result AAX_IACEffectParameters::GetParameterType (
    AAX_CParamID iParameterID,
    AAX_EParameterType * oParameterType ) const [pure virtual]
```

CALL: Retrieves the type of a parameter.

Todo The concept of parameter type needs more documentation

Parameters

in	<i>iParameterID</i>	The ID of the parameter that is being queried
out	<i>oParameterType</i>	The parameter's type

Implemented in [AAX_CEffectParameters](#).

14.64.2.13 GetParameterOrientation()

```
virtual AAX_Result AAX_IACFEffectParameters::GetParameterOrientation (
    AAX_CParamID iParameterID,
    AAX_EParameterOrientation * oParameterOrientation ) const [pure virtual]
```

CALL: Retrieves the orientation that should be applied to a parameter's controls.

Todo update this documentation

This method allows you to specify the orientation of knob controls that are managed by the host (e.g. knobs on an attached control surface.)

Here is an example override of this method that reverses the orientation of a control for a parameter:

```
// AAX_IParameter* myBackwardsParameter
if (iParameterID == myBackwardsParameter->Identifier())
{
    *oParameterType =
        AAX_eParameterOrientation_BottomMinTopMax |
        AAX_eParameterOrientation_LeftMinRightMax |
        AAX_eParameterOrientation_RotaryWrapMode |
        AAX_eParameterOrientation_RotaryLeftMinRightMax;
}
```

The orientation options are set according to [AAX_EParameterOrientationBits](#)

Legacy Porting Notes [AAX_IEffectParameters::GetParameterOrientation\(\)](#) corresponds to the [GetControlOrientation\(\)](#) method in the legacy RTAS/TDM SDK.

Parameters

in	<i>iParameterID</i>	The ID of the parameter that is being queried
out	<i>oParameterOrientation</i>	The orientation of the parameter

Implemented in [AAX_CEffectParameters](#).

14.64.2.14 GetParameter()

```
virtual AAX_Result AAX_IACFEffectParameters::GetParameter (
    AAX_CParamID iParameterID,
    AAX_IParameter ** oParameter ) [pure virtual]
```

CALL: Retrieves an arbitrary setting within a parameter.

This is a convenience function for accessing the richer parameter interface from the plug-in's other modules.

Note

This function must not be called by the host; [AAX_IParameter](#) is not safe for passing across the binary boundary with the host!

Parameters

in	<i>iParameterID</i>	The ID of the parameter that is being queried
out	<i>oParameter</i>	A pointer to the returned parameter

Implemented in [AAX_CEffectParameters](#).

14.64.2.15 GetParameterIndex()

```
virtual AAX_Result AAX_IACFEffEffectParameters::GetParameterIndex (
    AAX_CParamID iParameterID,
    int32_t * oControlIndex ) const [pure virtual]
```

CALL: Retrieves the index of a parameter.

Although parameters are normally referenced by their AAX_CParamID, each parameter is also associated with a unique numeric index.

Parameters

in	<i>iParameterID</i>	The ID of the parameter that is being queried
out	<i>oControlIndex</i>	The parameter's numeric index

Implemented in [AAX_CEffectParameters](#).

14.64.2.16 GetParameterIDFromIndex()

```
virtual AAX_Result AAX_IACFEffEffectParameters::GetParameterIDFromIndex (
    int32_t iControlIndex,
    AAX_IString * oParameterIDString ) const [pure virtual]
```

CALL: Retrieves the ID of a parameter.

This method can be used to convert a parameter's unique numeric index to its AAX_CParamID

Parameters

in	<i>iControllIndex</i>	The numeric index of the parameter that is being queried
out	<i>oParameterIDString</i>	Reference to an AAX_IString owned by the host. The plug-in must set this string equal to the parameter's ID.

Implemented in [AAX_CEffectParameters](#).

14.64.2.17 GetParameterValueInfo()

```
virtual AAX_Result AAX_IACFEffectParameters::GetParameterValueInfo (
    AAX_CParamID iParameterID,
    int32_t iSelector,
    int32_t * oValue ) const [pure virtual]
```

CALL: Retrieves a property of a parameter.

This is a general purpose query that is specialized based on the value of *iSelector*. The currently supported selector values are described by [AAX_EParameterValueInfoSelector](#). The meaning of *oValue* is dependent upon *iSelector*.

Parameters

in	<i>iParameterID</i>	The ID of the parameter that is being queried
in	<i>iSelector</i>	The selector of the parameter value to retrieve. See AAX_EParameterValueInfoSelector
out	<i>oValue</i>	The value of the specified parameter

Implemented in [AAX_CEffectParameters](#).

14.64.2.18 GetParameterValueFromString()

```
virtual AAX_Result AAX_IACFEffectParameters::GetParameterValueFromString (
    AAX_CParamID iParameterID,
    double * oValue,
    const AAX_IString & iValueString ) const [pure virtual]
```

CALL: Converts a value string to a value.

This method uses the queried parameter's display delegate and taper to convert a `char*` string into its corresponding value. The formatting of *valueString* must be supported by the parameter's display delegate in order for this call to succeed.

Legacy Porting Notes This method corresponds to `CProcess::MapControlStringToVal()` in the RTAS/TDM SDK

Parameters

in	<i>iParameterID</i>	The ID of the parameter that is being queried
out	<i>oValue</i>	The value associated with valueString
in	<i>iValueString</i>	The formatted value string that will be converted into a value

Implemented in [AAX_CEffectParameters](#).

14.64.2.19 GetParameterStringFromValue()

```
virtual AAX_Result AAX_IACEffectParameters::GetParameterStringFromValue (
    AAX_CParamID iParameterID,
    double iValue,
    AAX_IString * oValueString,
    int32_t iMaxLength ) const [pure virtual]
```

CALL: Converts a normalized parameter value into a string representing its corresponding real value.

This method uses the queried parameter's display delegate and taper to convert a normalized value into the corresponding `char*` value string for its real value.

Legacy Porting Notes This method corresponds to `CProcess::MapControlValToString()` in the RTAS/TDM SDK

Parameters

in	<i>iParameterID</i>	The ID of the parameter that is being queried
in	<i>iValue</i>	The normalized value that will be converted to a formatted valueString
out	<i>oValueString</i>	The formatted value string associated with value
in	<i>iMaxLength</i>	The maximum length of valueString

Implemented in [AAX_CEffectParameters](#).

14.64.2.20 GetParameterValueString()

```
virtual AAX_Result AAX_IACEffectParameters::GetParameterValueString (
    AAX_CParamID iParameterID,
    AAX_IString * oValueString,
    int32_t iMaxLength ) const [pure virtual]
```

CALL: Retrieves the value string associated with a parameter's current value.

This method uses the queried parameter's display delegate and taper to convert its current value into a corresponding `char*` value string.

Parameters

in	<i>iParameterID</i>	The ID of the parameter that is being queried
out	<i>oValueString</i>	The formatted value string associated with the parameter's current value
in	<i>iMaxLength</i>	The maximum length of valueString

Implemented in [AAX_CEffectParameters](#).

14.64.2.21 GetParameterNormalizedValue()

```
virtual AAX_Result AAX_IACFEEffectParameters::GetParameterNormalizedValue (
    AAX_CParamID iParameterID,
    double * oValuePtr ) const [pure virtual]
```

CALL: Retrieves a parameter's current value.

Parameters

in	<i>iParameterID</i>	The ID of the parameter that is being queried
out	<i>oValuePtr</i>	The parameter's current value

Implemented in [AAX_CEffectParameters](#).

14.64.2.22 SetParameterNormalizedValue()

```
virtual AAX_Result AAX_IACFEEffectParameters::SetParameterNormalizedValue (
    AAX_CParamID iParameterID,
    double iValue ) [pure virtual]
```

CALL: Sets the specified parameter to a new value.

[SetParameterNormalizedValue\(\)](#) is responsible for initiating any process that is required in order to update all of the parameter's controls (e.g. in the plug-in's GUI, on control surfaces, in automation lanes, etc.) In most cases, the parameter manager will handle this initiation step.

Parameters

in	<i>iParameterID</i>	The ID of the parameter that is being set
in	<i>iValue</i>	The value to which the parameter should be set

Implemented in [AAX_CEffectParameters](#).

14.64.2.23 SetParameterNormalizedRelative()

```
virtual AAX_Result AAX_IACFEffEffectParameters::SetParameterNormalizedRelative (
    AAX_CParamID iParameterID,
    double iValue ) [pure virtual]
```

CALL: Sets the specified parameter to a new value relative to its current value.

This method is used in cases when a relative control value is more convenient, for example when updating a GUI control using a mouse wheel or the arrow keys. Note that the host may apply the parameter's step size prior to calling [SetParameterNormalizedRelative\(\)](#) in order to determine the correct value for aValue.

[SetParameterNormalizedRelative\(\)](#) can be used to incorporate "wrapping" behavior in a parameter's controls, if desired. If this behavior is not desired, then this method must properly account for overflow of the parameter's normalized value.

[SetParameterNormalizedRelative\(\)](#) is responsible for initiating any process that is required in order to update all of the parameter's controls (e.g. in the plug-in's GUI, on control surfaces, in automation lanes, etc.) In most cases, the parameter manager will handle this initiation step.

See also [UpdateParameterNormalizedRelative\(\)](#).

Todo REMOVE THIS METHOD (?)

Parameters

in	<i>iParameterID</i>	The ID of the parameter that is being queried
in	<i>iValue</i>	The change in value that should be applied to the parameter

Todo NOT CURRENTLY CALLED FROM THE HOST. USEFUL FOR INTERNAL USE ONLY?

Implemented in [AAX_CEffectParameters](#).

14.64.2.24 TouchParameter()

```
virtual AAX_Result AAX_IACFEffEffectParameters::TouchParameter (
    AAX_CParamID iParameterID ) [pure virtual]
```

"Touches" (locks) a parameter in the automation system to a particular control in preparation for updates

This method is called by the Parameter Manager to prime a parameter for receiving new automation data. When an automatable parameter is touched by a control, it will reject input from other controls until it is released.

Note

You should never need to override this method when using [AAX_CEffectParameters](#).

Parameters

in	<i>iParameterID</i>	The parameter that is being touched
----	---------------------	-------------------------------------

Implemented in [AAX_CEffectParameters](#).

14.64.2.25 ReleaseParameter()

```
virtual AAX_Result AAX_IACFEffectParameters::ReleaseParameter (
    AAX_CParamID iParameterID ) [pure virtual]
```

Releases a parameter from a "touched" state.

This method is called by the Parameter Manager to release a parameter so that any control may send updates to the parameter.

Note

You should never need to override this method when using [AAX_CEffectParameters](#).

Parameters

in	<i>iParameterID</i>	The parameter that is being released
----	---------------------	--------------------------------------

Implemented in [AAX_CEffectParameters](#).

14.64.2.26 UpdateParameterTouch()

```
virtual AAX_Result AAX_IACFEffectParameters::UpdateParameterTouch (
    AAX_CParamID iParameterID,
    AAX_CBoolean iTouchState ) [pure virtual]
```

Sets a "touched" state on a parameter.

Note

This method should be overridden when dealing with linked parameters. Do NOT use this method to keep track of touch states. Use the [automation delegate](#) for that.

Parameters

in	<i>iParameterID</i>	The parameter that is changing touch states.
in	<i>iTouchState</i>	The touch state of the parameter.

Implemented in [AAX_CEffectParameters](#).

14.64.2.27 UpdateParameterNormalizedValue()

```
virtual AAX_Result AAX_IACEffectParameters::UpdateParameterNormalizedValue (
    AAX_CParamID iParameterID,
    double iValue,
    AAX_EUpdateSource iSource ) [pure virtual]
```

Updates a single parameter's state to its current value.

Note

Do *not* call this method from the plug-in. This method should be called by the host only. To set parameter values from within the plug-in, use the [AAX_IParameter](#) interface.

Todo FLAGGED FOR CONSIDERATION OF REVISION

Parameters

in	<i>iParameterID</i>	The ID of the parameter that is being updated
in	<i>iValue</i>	The parameter's current value, to which its internal state must be updated
in	<i>iSource</i>	The source of the update

Implemented in [AAX_CMonolithicParameters](#), and [AAX_CEffectParameters](#).

14.64.2.28 UpdateParameterNormalizedRelative()

```
virtual AAX_Result AAX_IACEffectParameters::UpdateParameterNormalizedRelative (
    AAX_CParamID iParameterID,
    double iValue ) [pure virtual]
```

Updates a single parameter's state to its current value, as a difference with the parameter's previous value.

Deprecated This is not called from the host. It *may* still be useful for internal calls within the plug-in, though it should only ever be used to update non-automatable parameters. Automatable parameters should always be updated through the [AAX_IParameter](#) interface, which will ensure proper coordination with other automation clients.

[UpdateParameterNormalizedRelative\(\)](#) can be used to incorporate "wraparound" behavior in a parameter's controls, if desired. If this behavior is not desired, then this method must properly account for overflow of the parameter's normalized value.

See also

[SetParameterNormalizedRelative\(\)](#)

Parameters

in	<i>iParameterID</i>	The ID of the parameter that is being updated
in	<i>iValue</i>	The difference between the parameter's current value and its previous value (normalized). The parameter's state must be updated to reflect this difference.

Implemented in [AAX_CEffectParameters](#).

14.64.2.29 GenerateCoefficients()

```
virtual AAX_Result AAX_IACFEffEffectParameters::GenerateCoefficients ( ) [pure virtual]
```

Generates and dispatches new coefficient packets.

This method is responsible for updating the coefficient packets associated with all parameters whose states have changed since the last call to [GenerateCoefficients\(\)](#). The host may call this method once for every parameter update, or it may "batch" parameter updates such that changes for several parameters are all handled by a single call to [GenerateCoefficients\(\)](#).

For more information on tracking parameters' statuses using the [AAX_CPacketDispatcher](#), helper class, see [AAX_CPacketDispatcher::SetDirty\(\)](#).

Note

Do *not* call this method from the plug-in. This method should be called by the host only. To set parameter values from within the plug-in, use the [AAX_IParameter](#) interface.

Implemented in [AAX_CEffectParameters](#), and [AAX_CMonolithicParameters](#).

14.64.2.30 ResetFieldData()

```
virtual AAX_Result AAX_IACFEffEffectParameters::ResetFieldData (
    AAX_CFieldIndex inFieldIndex,
    void * oData,
    uint32_t inDataSize ) const [pure virtual]
```

Called by the host to reset a private data field in the plug-in's algorithm.

This method is called sequentially for all private data fields on Effect initialization and during any "reset" event, such as priming for a non-real-time render. This method is called before the algorithm's optional initialization callback, and the initialized private data will be available within that callback via its context block.

See also

[Algorithm initialization](#).

Warning

Any data structures that will be passed between platforms (for example, sent to a TI DSP in an AAX DSP plug-in) must be properly data-aligned for compatibility across both platforms. See [AAX_ALIGN_FILE_ALG](#) for more information about guaranteeing cross-platform compatibility of data structures used for algorithm processing.

Parameters

in	<i>inFieldIndex</i>	The index of the field that is being initialized
out	<i>oData</i>	The pre-allocated block of data that should be initialized
in	<i>inDataSize</i>	The size of the data block, in bytes

Implemented in [AAX_CEffectParameters](#), and [AAX_CMonolithicParameters](#).

14.64.2.31 GetNumberOfChunks()

```
virtual AAX_Result AAX_IACFEffEffectParameters::GetNumberOfChunks (
    int32_t * oNumChunks ) const [pure virtual]
```

Retrieves the number of chunks used by this plug-in.

Parameters

out	<i>oNumChunks</i>	The number of distinct chunks used by this plug-in
-----	-------------------	--

Implemented in [AAX_CEffectParameters](#).

14.64.2.32 GetChunkIDFromIndex()

```
virtual AAX_Result AAX_IACFEffEffectParameters::GetChunkIDFromIndex (
    int32_t iIndex,
    AAX_CTypeID * oChunkID ) const [pure virtual]
```

Retrieves the ID associated with a chunk index.

Parameters

in	<i>iIndex</i>	Index of the queried chunk
out	<i>oChunkID</i>	ID of the queried chunk

Implemented in [AAX_CEffectParameters](#).

14.64.2.33 GetChunkSize()

```
virtual AAX_Result AAX_IACFEffEffectParameters::GetChunkSize (
    AAX_CTypeID iChunkID,
    uint32_t * oSize ) const [pure virtual]
```

Get the size of the data structure that can hold all of a chunk's information.

If *chunkID* is one of the plug-in's custom chunks, initialize **size* to the size of the chunk's data in bytes.

This method is invoked every time a chunk is saved, therefore it is possible to have dynamically sized chunks. However, note that each call to [GetChunkSize\(\)](#) will correspond to a following call to [GetChunk\(\)](#). The chunk provided in [GetChunk\(\)](#) *must* have the same size as the *size* provided by [GetChunkSize\(\)](#).

Legacy Porting Notes In [AAX](#), the value provided by [GetChunkSize\(\)](#) should *NOT* include the size of the chunk header. The value should *ONLY* reflect the size of the chunk's data.

Parameters

in	<i>iChunkID</i>	ID of the queried chunk
out	<i>oSize</i>	The chunk's size in bytes

Implemented in [AAX_CEffectParameters](#).

14.64.2.34 GetChunk()

```
virtual AAX_Result AAX_IACFEEffectParameters::GetChunk (
    AAX_CTypeID iChunkID,
    AAX_SPlugInChunk * oChunk ) const [pure virtual]
```

Fills a block of data with chunk information representing the plug-in's current state.

By calling this method, the host is requesting information about the current state of the plug-in. The following chunk fields should be explicitly populated in this method. Other fields will be populated by the host.

- [AAX_SPlugInChunk::fData](#)
- [AAX_SPlugInChunk::fVersion](#)
- [AAX_SPlugInChunk::fName](#) (Optional)
- [AAX_SPlugInChunk::fSize](#) (Data size only)

Warning

Remember that this chunk data may be loaded on a different platform from the one where it is saved. All data structures in the chunk must be properly data-aligned for compatibility across all platforms that the plug-in supports. See [AAX_ALIGN_FILE_ALG](#) for notes about common cross-platform pitfalls for data structure alignment.

Parameters

in	<i>iChunkID</i>	ID of the chunk that should be provided
out	<i>oChunk</i>	A preallocated block of memory that should be populated with the chunk's data.

Implemented in [AAX_CEffectParameters](#).

14.64.2.35 SetChunk()

```
virtual AAX_Result AAX_IACFEffEffectParameters::SetChunk (
    AAX_CTypeID iChunkID,
    const AAX_SPlugInChunk * iChunk ) [pure virtual]
```

Restores a set of plug-in parameters based on chunk information.

By calling this method, the host is attempting to update the plug-in's current state to match the data stored in a chunk. The plug-in should initialize itself to this new state by calling [SetParameterNormalizedValue\(\)](#) for each of the relevant parameters.

Parameters

in	<i>iChunkID</i>	ID of the chunk that is being set
in	<i>iChunk</i>	The chunk

Implemented in [AAX_CEffectParameters](#).

14.64.2.36 CompareActiveChunk()

```
virtual AAX_Result AAX_IACFEffEffectParameters::CompareActiveChunk (
    const AAX_SPlugInChunk * iChunkP,
    AAX_CBoolean * oIsEqual ) const [pure virtual]
```

Determine if a chunk represents settings that are equivalent to the plug-in's current state.

Host Compatibility Notes In Pro Tools, this method will only be called if a prior call to [GetNumberOfChanges\(\)](#) has indicated that the plug-in's state has changed. If the plug-in's current settings are different from the settings in *aChunkP* then the plug-in's Compare Light will be illuminated in the plug-in header, allowing users to toggle between the plug-in's custom state and its saved state.

Parameters

in	<i>iChunkP</i>	The chunk that is to be tested
out	<i>oIsEqual</i>	True if the chunk represents equivalent settings when compared with the plug-in's current state. False if the chunk represents non-equivalent settings

Implemented in [AAX_CEffectParameters](#).

14.64.2.37 TimerWakeup()

```
virtual AAX_Result AAX_IACFEEffectParameters::TimerWakeup ( ) [pure virtual]
```

Periodic wakeup callback for idle-time operations.

This method is called from the host using a non-main thread. In general, it should be driven at approximately one call per 30 ms. However, the wakeup is not guaranteed to be called at any regular interval - for example, it could be held off by a high real-time processing load - and there is no host contract regarding maximum latency between wakeup calls.

This wakeup thread runs continuously and cannot be armed/disarmed or by the plug-in.

Implemented in [AAX_CEffectParameters](#), and [AAX_CMonolithicParameters](#).

14.64.2.38 GetCustomData()

```
virtual AAX_Result AAX_IACFEEffectParameters::GetCustomData (
    AAX_CTypeID iDataBlockID,
    uint32_t inDataSize,
    void * oData,
    uint32_t * oDataWritten ) const [pure virtual]
```

An optional interface hook for getting custom data from another module.

Parameters

in	<i>iDataBlockID</i>	Identifier for the requested block of custom data
in	<i>inDataSize</i>	Size of provided buffer, in bytes
out	<i>oData</i>	Pointer to an allocated buffer. Data will be written here.
out	<i>oDataWritten</i>	The number of bytes actually written

Implemented in [AAX_CEffectParameters](#).

14.64.2.39 SetCustomData()

```
virtual AAX_Result AAX_IACFEEffectParameters::SetCustomData (
    AAX_CTypeID iDataBlockID,
    uint32_t inDataSize,
    const void * iData ) [pure virtual]
```

An optional interface hook for setting custom data for use by another module.

Parameters

in	<i>iDataBlockID</i>	Identifier for the provided block of custom data
in	<i>inDataSize</i>	Size of provided buffer, in bytes
in	<i>iData</i>	Pointer to the data buffer

Implemented in [AAX_CEffectParameters](#).

14.64.2.40 DoMIDITransfers()

```
virtual AAX_Result AAX_IACFEffEffectParameters::DoMIDITransfers ( ) [pure virtual]
```

MIDI update callback.

Call [AAX_IController::GetNextMIDIPacket\(\)](#) from within this method to retrieve and process MIDI packets directly within the Effect's data model. MIDI data will also be delivered to the Effect algorithm.

This method is called regularly by the host, similarly to [AAX_IEffectParameters::TimerWakeup\(\)](#)

Implemented in [AAX_CEffectParameters](#).

The documentation for this class was generated from the following file:

- [AAX_IACFEffEffectParameters.h](#)

14.65 AAX_IACFEffEffectParameters_V2 Class Reference

```
#include <AAX_IACFEffEffectParameters.h>
```

Inheritance diagram for AAX_IACFEffEffectParameters_V2:

Collaboration diagram for AAX_IACFEffEffectParameters_V2:

14.65.1 Description

Supplemental interface for an AAX Plug-in's data model.

This is a supplemental interface for an instance of a plug-in's data model. This interface gets exposed to the host application. Host applications that support AAX versioned features may call into these methods. See [Data model interface](#).

Note

Your implementation of this interface must inherit from [AAX_IEffectParameters](#).

Todo Add documentation for expected error state return values

Public Member Functions

Hybrid audio methods

- virtual [AAX_Result RenderAudio_Hybrid](#) ([AAX_SHybridRenderInfo](#) *ioRenderInfo)=0
Hybrid audio render function.

MIDI methods

- virtual [AAX_Result UpdateMIDINodes](#) ([AAX_CFieldIndex](#) inFieldIndex, [AAX_CMidiPacket](#) &iPacket)=0
MIDI update callback.
- virtual [AAX_Result UpdateControlMIDINodes](#) ([AAX_CTypeID](#) nodeID, [AAX_CMidiPacket](#) &iPacket)=0
MIDI update callback for control MIDI nodes.

14.65.2 Member Function Documentation

14.65.2.1 UpdateMIDINodes()

```
virtual AAX\_Result AAX_IACFEffectParameters_V2::UpdateMIDINodes (
    AAX\_CFieldIndex inFieldIndex,
    AAX\_CMidiPacket & iPacket ) [pure virtual]
```

MIDI update callback.

This method is called by the host for each pending MIDI packet for MIDI nodes in algorithm context structure. Overwrite this method in Plug-In's EffectParameter class if you want to receive MIDI data packets directly in the data model. MIDI data will also be delivered to the Effect algorithm.

The host calls this method in Effects that register one or more MIDI nodes using [AAX_IComponentDescriptor::AddMIDINode\(\)](#). Effects that do not require MIDI data to be sent to the plug-in algorithm should override [UpdateControlMIDINodes\(\)](#).

Parameters

in	<i>inFieldIndex</i>	MIDI node field index in algorithm context structure
in	<i>iPacket</i>	The incoming MIDI packet for the node

Implemented in [AAX_CEffectParameters](#).

14.65.2.2 UpdateControlMIDINodes()

```
virtual AAX\_Result AAX_IACFEffectParameters_V2::UpdateControlMIDINodes (
    AAX\_CTypeID nodeID,
    AAX\_CMidiPacket & iPacket ) [pure virtual]
```

MIDI update callback for control MIDI nodes.

This method is called by the host for each pending MIDI packet for Control MIDI nodes. Overwrite this method in Plug-In's EffectParameter class if you want to receive MIDI data packets directly in the data model.

The host calls this method in Effects that register one or more Control MIDI nodes using [AAX_IEffectDescriptor::AddControlMIDINode\(\)](#). Effects with algorithms that use MIDI data nodes should override [UpdateMIDINodes\(\)](#).

Note

This method will not be called if an Effect includes any MIDI nodes in its algorithm context structure.

Parameters

in	<i>nodeID</i>	Identifier for the MIDI node
in	<i>iPacket</i>	The incoming MIDI packet for the node

Implemented in [AAX_CEffectParameters](#).

The documentation for this class was generated from the following file:

- [AAX_IACFEffEffectParameters.h](#)

14.66 AAX_IACFEffEffectParameters_V3 Class Reference

```
#include <AAX_IACFEffEffectParameters.h>
```

Inheritance diagram for AAX_IACFEffEffectParameters_V3:

Collaboration diagram for AAX_IACFEffEffectParameters_V3:

14.66.1 Description

Supplemental interface for an AAX Plug-in's data model.

This is a supplemental interface for an instance of a plug-in's data model. This interface gets exposed to the host application. Host applications that support AAX versioned features may call into these methods. See [Data model interface](#).

Note

Your implementation of this interface must inherit from [AAX_IEffectParameters](#).

Todo Add documentation for expected error state return values

Public Member Functions

Auxiliary UI methods

- virtual [AAX_Result GetCurveDataMeterIds](#) ([AAX_CTypeID](#) iCurveType, uint32_t *oXMeterId, uint32_t *oYMeterId) const =0
Indicates which meters correspond to the X and Y axes of the EQ or Dynamics graph.
- virtual [AAX_Result GetCurveDataDisplayRange](#) ([AAX_CTypeID](#) iCurveType, float *oXMin, float *oXMax, float *oYMin, float *oYMax) const =0
Determines the range of the graph shown by the plug-in.

The documentation for this class was generated from the following file:

- [AAX_IACFEffEffectParameters.h](#)

14.67 AAX_IACFEffEffectParameters_V4 Class Reference

```
#include <AAX_IACFEffEffectParameters.h>
```

Inheritance diagram for AAX_IACFEffEffectParameters_V4:

Collaboration diagram for AAX_IACFEffEffectParameters_V4:

14.67.1 Description

Supplemental interface for an AAX Plug-in's data model.

This is a supplemental interface for an instance of a plug-in's data model. This interface gets exposed to the host application. Host applications that support AAX versioned features may call into these methods. See [Data model interface](#).

Note

Your implementation of this interface must inherit from [AAX_IEffectParameters](#).

Todo Add documentation for expected error state return values

Public Member Functions

Auxiliary UI methods

- virtual [AAX_Result UpdatePageTable](#) (uint32_t inTableType, int32_t inTablePageSize, [IACFUnknown](#) *iHostUnknown, [IACFUnknown](#) *ioPageTableUnknown) const =0
Allow the plug-in to update its page tables.

14.67.2 Member Function Documentation

14.67.2.1 UpdatePageTable()

```
virtual AAX_Result AAX_IACFEffEffectParameters_V4::UpdatePageTable (
    uint32_t inTableType,
    int32_t inTablePageSize,
    IACFUnknown * iHostUnknown,
    IACFUnknown * ioPageTableUnknown ) const [pure virtual]
```

Allow the plug-in to update its page tables.

Called by the plug-in host, usually in response to a [AAX_eNotificationEvent_ParameterMappingChanged](#) notification sent from the plug-in.

Use this method to change the page table mapping for the plug-in instance or to apply other changes to auxiliary UIs which use the plug-in page tables, such as setting focus to a new page.

See [Page Table Guide](#) for more information about page tables.

Parameters

in	<i>inTableType</i>	Four-char type identifier for the table type (e.g. 'PgTL', 'Av81', etc.)
in	<i>inTablePageSize</i>	Page size for the table
in	<i>iHostUnknown</i>	Unknown interface from the host which may support interfaces providing additional features or information. All interfaces queried from this unknown will be valid only within the scope of this UpdatePageTable() execution and will be relevant for only the current plug-in instance.
in, out	<i>ioPageTableUnknown</i>	Unknown interface which supports AAX_IPageTable . This object represents the page table data which is currently stored by the host for this plug-in instance for the given table type and page size. This data and may be edited within the scope of UpdatePageTable() to change the page table mapping for this plug-in instance.

Returns

This method should return [AAX_ERROR_UNIMPLEMENTED](#) if the plug-in does not implement it or when no change is requested by the plug-in. This allows optimizations to be used in the host when no UI update is required following this call.

See also

[AAX_eNotificationEvent_ParameterMappingChanged](#)

Implemented in [AAX_CEffectParameters](#).

The documentation for this class was generated from the following file:

- [AAX_IACFEffEffectParameters.h](#)

14.68 AAX_IACFFeatureInfo Class Reference

```
#include <AAX_IACFFeatureInfo.h>
```

Inheritance diagram for [AAX_IACFFeatureInfo](#):

Collaboration diagram for [AAX_IACFFeatureInfo](#):

14.68.1 Description

Information about host support for a particular feature

Acquired using [AAX_IACFDescriptionHost::AcquireFeatureProperties\(\)](#)

This interface is shared between multiple features. The specific feature which this object represents is the feature whose ID was used in the call to acquire this interface.

See the feature UID documentation for which properties support additional property map data

IID: [IID_IAAXFeatureInfoV1](#)

Note

Do not [QueryInterface\(\)](#) for [IID_IAAXFeatureInfoV1](#) since this does not indicate which specific feature is being requested. Instead, use [AAX_IDescriptionHost::AcquireFeatureProperties\(\)](#)

Public Member Functions

- virtual [AAX_Result](#) [SupportLevel](#) ([AAX_ESupportLevel](#) *oSupportLevel) const =0
- virtual [AAX_Result](#) [AcquireProperties](#) ([IACFUnknown](#) **outProperties)=0

14.68.2 Member Function Documentation

14.68.2.1 SupportLevel()

```
virtual AAX\_Result AAX\_IACFFeatureInfo::SupportLevel (  
    AAX\_ESupportLevel * oSupportLevel ) const    [pure virtual]
```

Determine the level of support for this feature by the host

Note

The host will not provide an underlying [AAX_IACFFeatureInfo](#) interface for features which it does not recognize at all, resulting in a [AAX_ERROR_NULL_OBJECT](#) error code

See also

[AAX_IFeatureInfo::SupportLevel\(\)](#)

Determine the level of support for this feature by the host

Note

The host will not provide an underlying [AAX_IACFFeatureInfo](#) interface for features which it does not recognize at all, resulting in a [AAX_ERROR_NULL_OBJECT](#) error code

14.68.2.2 AcquireProperties()

```
virtual AAX_Result AAX_IACFFeatureInfo::AcquireProperties (
    IACFUnknown ** outProperties ) [pure virtual]
```

`outProperties` must support [AAX_IACFPropertyMap](#) const methods

See also

[AAX_IFeatureInfo::AcquireProperties\(\)](#)

The documentation for this class was generated from the following file:

- [AAX_IACFFeatureInfo.h](#)

14.69 AAX_IACFHostProcessor Class Reference

```
#include <AAX_IACFHostProcessor.h>
```

Inheritance diagram for [AAX_IACFHostProcessor](#):

Collaboration diagram for [AAX_IACFHostProcessor](#):

14.69.1 Description

Versioned interface for an AAX host processing component.

Note

This interface gets exposed to the host application. See [AAX_CHostProcessor](#) for method documentation.

Legacy Porting Notes This interface provides offline processing features analogous to the legacy AudioSuite plug-in architecture

Public Member Functions

- virtual [AAX_Result Initialize](#) ([IACFUnknown](#) *iController)=0
Host Processor initialization.
- virtual [AAX_Result Uninitialize](#) ()=0
Host Processor teardown.
- virtual [AAX_Result InitOutputBounds](#) (int64_t iSrcStart, int64_t iSrcEnd, int64_t *oDstStart, int64_t *oDstEnd)=0
Sets the processing region.
- virtual [AAX_Result SetLocation](#) (int64_t iSample)=0
Updates the relative sample location of the current processing frame.
- virtual [AAX_Result RenderAudio](#) (const float *const inAudioIns[], int32_t inAudioInCount, float *const iAudioOuts[], int32_t iAudioOutCount, int32_t *ioWindowSize)=0
Perform the signal processing.
- virtual [AAX_Result PreRender](#) (int32_t inAudioInCount, int32_t iAudioOutCount, int32_t iWindowSize)=0
Invoked right before the start of a Preview or Render pass.
- virtual [AAX_Result PostRender](#) ()=0
Invoked at the end of a Render pass.
- virtual [AAX_Result AnalyzeAudio](#) (const float *const inAudioIns[], int32_t inAudioInCount, int32_t *ioWindowSize)=0
Override this method if the plug-in needs to analyze the audio prior to a Render pass.
- virtual [AAX_Result PreAnalyze](#) (int32_t inAudioInCount, int32_t iWindowSize)=0
Invoked right before the start of an Analysis pass.
- virtual [AAX_Result PostAnalyze](#) ()=0
Invoked at the end of an Analysis pass.

14.69.2 Member Function Documentation

14.69.2.1 Initialize()

```
virtual AAX\_Result AAX_IACFHostProcessor::Initialize (
    IACFUnknown * iController ) [pure virtual]
```

Host Processor initialization.

Parameters

in	<i>iController</i>	A versioned reference that can be resolved to both an AAX_IController interface and an AAX_IHostProcessorDelegate
----	--------------------	---

Implemented in [AAX_CHostProcessor](#).

14.69.2.2 Uninitialize()

```
virtual AAX\_Result AAX_IACFHostProcessor::Uninitialize ( ) [pure virtual]
```

Host Processor teardown.

Implemented in [AAX_CHostProcessor](#).

14.69.2.3 InitOutputBounds()

```
virtual AAX\_Result AAX_IACFHostProcessor::InitOutputBounds (
    int64_t iSrcStart,
    int64_t iSrcEnd,
    int64_t * oDstStart,
    int64_t * oDstEnd ) [pure virtual]
```

Sets the processing region.

This method allows offline processing plug-ins to vary the length and/or start/end points of the audio processing region.

This method is called in a few different scenarios:

- Before an analyze, process or preview of data begins.
- At the end of every preview loop.
- After the user makes a new data selection on the timeline.

Plug-ins that inherit from [AAX_CHostProcessor](#) should not override this method. Instead, use the following convenience functions:

- To retrieve the length or boundaries of the processing region, use [GetInputRange\(\)](#), [GetSrcStart\(\)](#), etc.
- To change the boundaries of the processing region before processing begins, use [AAX_CHostProcessor::TranslateOutputBound](#)

Note

Currently, a host processor may not randomly access samples outside of the boundary defined by `oDstStart` and `oDstEnd`.

Legacy Porting Notes DAE no longer makes use of the `mStartBound` and `mEndBounds` member variables that existed in the legacy RTAS/TDM SDK. Use `oDstStart` and `oDstEnd` instead (preferably by overriding `TranslateOutputBounds()`.)

Parameters

in	<i>iSrcStart</i>	The selection start of the user selected region. This is will always return 0 for a given selection on the timeline.
in	<i>iSrcEnd</i>	The selection end of the user selected region. This will always return the value of the selection length on the timeline.
in	<i>oDstStart</i>	The starting sample location in the output audio region. By default, this is the same as <code>iSrcStart</code> .
in	<i>oDstEnd</i>	The ending sample location in the output audio region. By default, this is the same as <code>iSrcEnd</code> .

Implemented in [AAX_CHostProcessor](#).

14.69.2.4 SetLocation()

```
virtual AAX_Result AAX_IACFHostProcessor::SetLocation (
    int64_t iSample ) [pure virtual]
```

Updates the relative sample location of the current processing frame.

This method is called by the host to update the relative sample location of the current processing frame.

Note

Plug-ins should not override this method; instead, use [AAX_CHostProcessor::GetLocation\(\)](#) to retrieve the current relative sample location.

Parameters

in	<i>iSample</i>	The sample location of the first sample in the current processing frame relative to the beginning of the full processing buffer
----	----------------	---

Implemented in [AAX_CHostProcessor](#).

14.69.2.5 RenderAudio()

```
virtual AAX_Result AAX_IACFHostProcessor::RenderAudio (
    const float *const inAudioIns[],
```

```

    int32_t inAudioInCount,
    float *const iAudioOuts[],
    int32_t iAudioOutCount,
    int32_t * ioWindowSize ) [pure virtual]

```

Perform the signal processing.

This method is called by the host to invoke the plug-in's signal processing.

Legacy Porting Notes This method is a replacement for the AudioSuite `ProcessAudio` method

Parameters

in	<i>inAudioIn</i>	Input audio buffer
in	<i>inAudioInCount</i>	The number if input channels
in	<i>iAudioOuts</i>	The number of output channels
in	<i>iAudioOutCount</i>	A user defined destination end of the ingested audio
in	<i>ioWindowSize</i>	Window buffer length of the received audio

Implemented in [AAX_CHostProcessor](#).

14.69.2.6 PreRender()

```

virtual AAX_Result AAX_IACFHostProcessor::PreRender (
    int32_t inAudioInCount,
    int32_t iAudioOutCount,
    int32_t iWindowSize ) [pure virtual]

```

Invoked right before the start of a Preview or Render pass.

This method is called by the host to allow a plug-in to make any initializations before processing actually begins. Upon a Preview pass, PreRender will also be called at the beginning of every "loop".

See also

[AAX_eProcessingState_StartPass](#), [AAX_eProcessingState_BeginPassGroup](#)

Parameters

in	<i>inAudioInCount</i>	The number if input channels
in	<i>iAudioOutCount</i>	The number of output channels
in	<i>iWindowSize</i>	Window buffer length of the ingested audio

Implemented in [AAX_CHostProcessor](#).

14.69.2.7 PostRender()

```
virtual AAX_Result AAX_IACFHostProcessor::PostRender ( ) [pure virtual]
```

Invoked at the end of a Render pass.

Note

Upon a Preview pass, PostRender will not be called until Preview has stopped.

See also

[AAX_eProcessingState_StopPass](#), [AAX_eProcessingState_EndPassGroup](#)

Implemented in [AAX_CHostProcessor](#).

14.69.2.8 AnalyzeAudio()

```
virtual AAX_Result AAX_IACFHostProcessor::AnalyzeAudio (
    const float *const inAudioIns[],
    int32_t inAudioInCount,
    int32_t * ioWindowSize ) [pure virtual]
```

Override this method if the plug-in needs to analyze the audio prior to a Render pass.

Use this after declaring the appropriate properties in Describe. See [AAX_eProperty_RequiresAnalysis](#) and [AAX_eProperty_OptionalAnalysis](#)

To request an analysis pass from within a plug-in, use [AAX_IHostProcessorDelegate::ForceAnalyze\(\)](#)

Legacy Porting Notes Ported from AudioSuite's AnalyzeAudio(bool isMasterBypassed) method

Parameters

in	<i>inAudioIns</i>	Input audio buffer
in	<i>inAudioInCount</i>	The number of input channels
in	<i>ioWindowSize</i>	Window buffer length of the ingested audio

Implemented in [AAX_CHostProcessor](#).

14.69.2.9 PreAnalyze()

```
virtual AAX_Result AAX_IACFHostProcessor::PreAnalyze (
    int32_t inAudioInCount,
    int32_t iWindowSize ) [pure virtual]
```

Invoked right before the start of an Analysis pass.

This method is called by the host to allow a plug-in to make any initializations before an Analysis pass actually begins.

See also

[AAX_eProcessingState_StartPass](#), [AAX_eProcessingState_BeginPassGroup](#)

Parameters

in	<i>inAudioInCount</i>	The number if input channels
in	<i>iWindowSize</i>	Window buffer length of the ingested audio

Implemented in [AAX_CHostProcessor](#).

14.69.2.10 PostAnalyze()

```
virtual AAX_Result AAX_IACFHostProcessor::PostAnalyze ( ) [pure virtual]
```

Invoked at the end of an Analysis pass.

Note

In Pro Tools, a long execution time for this method will hold off the main application thread and cause a visible hang. If the plug-in must perform any long running tasks before initiating processing then it is best to perform these tasks in [AAX_IHostProcessor::PreRender\(\)](#)

See also

[AAX_eProcessingState_StopPass](#), [AAX_eProcessingState_EndPassGroup](#)

Implemented in [AAX_CHostProcessor](#).

The documentation for this class was generated from the following file:

- [AAX_IACFHostProcessor.h](#)

14.70 AAX_IACFHostProcessor_V2 Class Reference

```
#include <AAX_IACFHostProcessor.h>
```

Inheritance diagram for AAX_IACFHostProcessor_V2:

Collaboration diagram for AAX_IACFHostProcessor_V2:

14.70.1 Description

Supplemental interface for an AAX host processing component.

Note

This interface gets exposed to the host application. See [AAX_CHostProcessor](#) for method documentation.

Public Member Functions

- virtual [AAX_Result](#) [GetClipNameSuffix](#) (int32_t inMaxLength, [AAX_IString](#) *outString) const =0
Called by host application to retrieve a custom string to be appended to the clip name.

14.70.2 Member Function Documentation

14.70.2.1 GetClipNameSuffix()

```
virtual AAX\_Result AAX_IACFHostProcessor_V2::GetClipNameSuffix (
    int32_t inMaxLength,
    AAX\_IString * outString ) const    [pure virtual]
```

Called by host application to retrieve a custom string to be appended to the clip name.

If no string is provided then the host's default will be used.

Parameters

in	<i>inMaxLength</i>	The maximum allowed string length, not including the NULL terminating char
out	<i>outString</i>	Add a value to this string to provide a custom clip suffix

Implemented in [AAX_CHostProcessor](#).

The documentation for this class was generated from the following file:

- [AAX_IACFHostProcessor.h](#)

14.71 AAX_IACFHostProcessorDelegate Class Reference

```
#include <AAX_IACFHostProcessorDelegate.h>
```

Inheritance diagram for AAX_IACFHostProcessorDelegate:

Collaboration diagram for AAX_IACFHostProcessorDelegate:

14.71.1 Description

Versioned interface for host methods specific to offline processing.

Public Member Functions

- virtual [AAX_Result GetAudio](#) (const float *const inAudioIns[], int32_t inAudioInCount, int64_t inLocation, int32_t *ioNumSamples)=0
CALL: Randomly access audio from the timeline.
- virtual int32_t [GetSideChainInputNum](#) ()=0
CALL: Returns the index of the side chain input buffer.

14.71.2 Member Function Documentation

14.71.2.1 GetAudio()

```
virtual AAX\_Result AAX_IACFHostProcessorDelegate::GetAudio (
    const float *const inAudioIns[],
    int32_t inAudioInCount,
    int64_t inLocation,
    int32_t * ioNumSamples ) [pure virtual]
```

CALL: Randomly access audio from the timeline.

Called from within [AAX_IHostProcessor::RenderAudio\(\)](#), this method fills a buffer of samples with randomly-accessed data from the current input processing region on the timeline, including any extra samples such as processing "handles".

Note

Plug-ins that use this feature must set [AAX_eProperty_UsesRandomAccess](#) to `true`

It is not possible to retrieve samples from outside of the current input processing region

Always check the return value of this method before using the randomly-accessed samples

Parameters

in	<i>inAudioIns</i>	Timeline audio buffer(s). This must be set to <i>inAudioIns</i> from AAX_IHostProcessor::RenderAudio()
in	<i>inAudioInCount</i>	Number of buffers in <i>inAudioIns</i> . This must be set to <i>inAudioInCount</i> from AAX_IHostProcessor::RenderAudio()
in	<i>inLocation</i>	A sample location relative to the beginning of the currently processed region, e.g. a value of 0 corresponds to the timeline location returned by AAX_CHostProcessor::GetSrcStart()
in, out	<i>ioNumSamples</i>	• Input: The maximum number of samples to read. • Output: The actual number of samples that were read from the timeline

14.71.2.2 GetSideChainInputNum()

```
virtual int32_t AAX_IACFHostProcessorDelegate::GetSideChainInputNum ( ) [pure virtual]
```

CALL: Returns the index of the side chain input buffer.

Called from within [AAX_IHostProcessor::RenderAudio\(\)](#), this method returns the index of the side chain input sample buffer within `inAudioIns`.

The documentation for this class was generated from the following file:

- [AAX_IACFHostProcessorDelegate.h](#)

14.72 AAX_IACFHostProcessorDelegate_V2 Class Reference

```
#include <AAX_IACFHostProcessorDelegate.h>
```

Inheritance diagram for AAX_IACFHostProcessorDelegate_V2:

Collaboration diagram for AAX_IACFHostProcessorDelegate_V2:

14.72.1 Description

Versioned interface for host methods specific to offline processing.

Public Member Functions

- virtual [AAX_Result ForceAnalyze](#) ()=0
CALL: Request an analysis pass.

14.72.2 Member Function Documentation

14.72.2.1 ForceAnalyze()

```
virtual AAX\_Result AAX_IACFHostProcessorDelegate_V2::ForceAnalyze ( ) [pure virtual]
```

CALL: Request an analysis pass.

Call this method to request an analysis pass from within the plug-in. Most plug-ins should rely on the host to trigger analysis passes when appropriate. However, plug-ins that require an analysis pass a) outside of the context of host-driven render or analysis, or b) when internal plug-in data changes may need to call [ForceAnalyze\(\)](#).

The documentation for this class was generated from the following file:

- [AAX_IACFHostProcessorDelegate.h](#)

14.73 AAX_IACFHostProcessorDelegate_V3 Class Reference

```
#include <AAX_IACFHostProcessorDelegate.h>
```

Inheritance diagram for AAX_IACFHostProcessorDelegate_V3:

Collaboration diagram for AAX_IACFHostProcessorDelegate_V3:

14.73.1 Description

Versioned interface for host methods specific to offline processing.

Public Member Functions

- virtual [AAX_Result ForceProcess](#) ()=0
CALL: Request a process pass.

14.73.2 Member Function Documentation

14.73.2.1 ForceProcess()

```
virtual AAX\_Result AAX_IACFHostProcessorDelegate_V3::ForceProcess ( ) [pure virtual]
```

CALL: Request a process pass.

Call this method to request a process pass from within the plug-in. If [AAX_eProperty_RequiresAnalysis](#) is defined, the resulting process pass will be preceded by an analysis pass. This method should only be used in rare circumstances by plug-ins that must launch processing outside of the normal host AudioSuite workflow.

The documentation for this class was generated from the following file:

- [AAX_IACFHostProcessorDelegate.h](#)

14.74 AAX_IACFHostServices Class Reference

```
#include <AAX_IACFHostServices.h>
```

Inheritance diagram for AAX_IACFHostServices:

Collaboration diagram for AAX_IACFHostServices:

14.74.1 Description

Versioned interface to diagnostic and debugging services provided by the AAX host.

Public Member Functions

- virtual [AAX_Result Assert](#) (const char *iFile, int32_t iLine, const char *iNote)=0
- virtual [AAX_Result Trace](#) (int32_t iPriority, const char *iMessage)=0

Log a trace message.

14.74.2 Member Function Documentation

14.74.2.1 Assert()

```
virtual AAX\_Result AAX_IACFHostServices::Assert (
    const char * iFile,
    int32_t iLine,
    const char * iNote ) [pure virtual]
```

Deprecated Legacy version of [AAX_IACFHostServices_V3::HandleAssertFailure\(\)](#) implemented by older hosts

Prior to [AAX_IACFHostServices_V3::HandleAssertFailure\(\)](#), the [AAX_ASSERT](#) macro, a wrapper around [Assert\(\)](#), was only compiled into debug plug-in builds. [AAX_ASSERT](#) is now compiled in to all plug-in builds and the original debug-only form is available through [AAX_DEBUGASSERT](#).

Because the implementation of [Assert\(\)](#) in the host is not aware of the plug-in's build configuration, older hosts implemented this method with a warning dialog in all cases. Newer hosts - those which implement [HandleAssertFailure\(\)](#) - will log assertion failures but will not present any user dialog in shipping builds of the host software.

In order to prevent assertion failure dialogs from appearing to users who run new builds of plug-ins containing [AAX_ASSERT](#) calls in older hosts the deprecated [Assert\(\)](#) method should only be called from debug plug-in builds.

14.74.2.2 Trace()

```
virtual AAX\_Result AAX_IACFHostServices::Trace (
    int32_t iPriority,
    const char * iMessage ) [pure virtual]
```

Log a trace message.

Parameters

in	<i>iPriority</i>	Priority of the trace, used for log filtering. One of kAAX_Trace_Priority_Low , kAAX_Trace_Priority_Normal , kAAX_Trace_Priority_High
in	<i>iMessage</i>	Message string to log

The documentation for this class was generated from the following file:

- [AAX_IACFHostServices.h](#)

14.75 AAX_IACFHostServices_V2 Class Reference

```
#include <AAX_IACFHostServices.h>
```

Inheritance diagram for AAX_IACFHostServices_V2:

Collaboration diagram for AAX_IACFHostServices_V2:

14.75.1 Description

V2 of versioned interface to diagnostic and debugging services provided by the AAX host.

Public Member Functions

- virtual [AAX_Result StackTrace](#) (int32_t iTracePriority, int32_t iStackTracePriority, const char *iMessage)=0
Log a trace message or a stack trace.

14.75.2 Member Function Documentation

14.75.2.1 StackTrace()

```
virtual AAX\_Result AAX_IACFHostServices_V2::StackTrace (
    int32_t iTracePriority,
    int32_t iStackTracePriority,
    const char * iMessage ) [pure virtual]
```

Log a trace message or a stack trace.

If the logging output filtering is set to include logs with `iStackTracePriority` then both the logging message and a stack trace will be emitted, regardless of `iTracePriority`.

If the logging output filtering is set to include logs with `iTracePriority` but to exclude logs with `iStackTracePriority` then this will emit a normal log with no stack trace.

Parameters

in	<i>iTracePriority</i>	Priority of the trace, used for log filtering. One of kAAX_Trace_Priority_Low , kAAX_Trace_Priority_Normal , kAAX_Trace_Priority_High
in	<i>iStackTracePriority</i>	Priority of the stack trace, used for log filtering. One of kAAX_Trace_Priority_Low , kAAX_Trace_Priority_Normal , kAAX_Trace_Priority_High
in	<i>iMessage</i>	Message string to log

The documentation for this class was generated from the following file:

- [AAX_IACFHostServices.h](#)

14.76 AAX_IACFHostServices_V3 Class Reference

```
#include <AAX_IACFHostServices.h>
```

Inheritance diagram for AAX_IACFHostServices_V3:

Collaboration diagram for AAX_IACFHostServices_V3:

14.76.1 Description

V3 of versioned interface to diagnostic and debugging services provided by the AAX host.

Public Member Functions

- virtual [AAX_Result HandleAssertFailure](#) (const char *iFile, int32_t iLine, const char *iNote, int32_t iFlags) const =0
Handle an assertion failure.

14.76.2 Member Function Documentation

14.76.2.1 HandleAssertFailure()

```
virtual AAX\_Result AAX_IACFHostServices_V3::HandleAssertFailure (
    const char * iFile,
    int32_t iLine,
    const char * iNote,
    int32_t iFlags ) const [pure virtual]
```

Handle an assertion failure.

Use this method to delegate assertion failure handling to the host

Use `iFlags` to request that specific behavior be included when handling the failure. This request may not be fulfilled by the host, and absence of a flag does not preclude the host from using that behavior when handling the failure.

Parameters

in	<i>iFile</i>	The name of the file containing the assert check. Usually <code>__FILE__</code>
in	<i>iLine</i>	The line number of the assert check. Usually <code>__LINE__</code>
in	<i>iNote</i>	Text to display related to the assert. Usually the condition which failed
in	<i>iFlags</i>	Bitfield of AAX_EAssertFlags to request specific handling behavior

The documentation for this class was generated from the following file:

- [AAX_IACFHostServices.h](#)

14.77 AAX_IACFPageTable Class Reference

`#include <AAX_IACFPageTable.h>`

Inheritance diagram for AAX_IACFPageTable:

Collaboration diagram for AAX_IACFPageTable:

14.77.1 Description

Versioned interface to the host's representation of a plug-in instance's page table.

Public Member Functions

- virtual [AAX_Result Clear](#) ()=0
Clears all parameter mappings from the table.
- virtual [AAX_Result Empty](#) (AAX_CBoolean &oEmpty) const =0
Indicates whether the table is empty.
- virtual [AAX_Result GetNumPages](#) (int32_t &oNumPages) const =0
Get the number of pages currently in this table.
- virtual [AAX_Result InsertPage](#) (int32_t iPage)=0
Insert a new empty page before the page at index iPage.
- virtual [AAX_Result RemovePage](#) (int32_t iPage)=0
Remove the page at index iPage.
- virtual [AAX_Result GetNumMappedParameterIDs](#) (int32_t iPage, int32_t &oNumParameterIdentifiers) const =0
Returns the total number of parameter IDs which are mapped to a page.
- virtual [AAX_Result ClearMappedParameter](#) (int32_t iPage, int32_t iIndex)=0
Clear the parameter at a particular index in this table.
- virtual [AAX_Result GetMappedParameterID](#) (int32_t iPage, int32_t iIndex, AAX_IString &oParameterIdentifier) const =0
Get the parameter identifier which is currently mapped to an index in this table.
- virtual [AAX_Result MapParameterID](#) (AAX_CParamID iParameterIdentifier, int32_t iPage, int32_t iIndex)=0
Map a parameter to this table.

14.77.2 Member Function Documentation

14.77.2.1 Clear()

```
virtual AAX_Result AAX_IACFPageTable::Clear ( ) [pure virtual]
```

Clears all parameter mappings from the table.

This method does not clear any parameter name variations from the table. For that, use [AAX_IPageTable::ClearParameterNameVariations](#) or [AAX_IPageTable::ClearNameVariationsForParameter](#)

14.77.2.2 Empty()

```
virtual AAX_Result AAX_IACFPageTable::Empty (
    AAX_CBoolean & oEmpty ) const [pure virtual]
```

Indicates whether the table is empty.

A table is empty if it contains no pages. A table which contains pages but no parameter assignments is not empty. A table which has associated parameter name variations but no pages is empty.

Parameters

out	<i>oEmpty</i>	true if this table is empty
-----	---------------	-----------------------------

14.77.2.3 GetNumPages()

```
virtual AAX_Result AAX_IACFPageTable::GetNumPages (
    int32_t & oNumPages ) const [pure virtual]
```

Get the number of pages currently in this table.

Parameters

out	<i>oNumPages</i>	The number of pages which are present in the page table. Some pages might not contain any parameter assignments.
-----	------------------	--

14.77.2.4 InsertPage()

```
virtual AAX_Result AAX_IACFPageTable::InsertPage (
    int32_t iPage ) [pure virtual]
```

Insert a new empty page before the page at index *iPage*.

Returns

[AAX_ERROR_INVALID_ARGUMENT](#) if *iPage* is greater than the total number of pages

Parameters

in	<i>iPage</i>	The insertion point page index
----	--------------	--------------------------------

14.77.2.5 RemovePage()

```
virtual AAX_Result AAX_IACFPageTable::RemovePage (
    int32_t iPage ) [pure virtual]
```

Remove the page at index *iPage*.

Returns

[AAX_ERROR_INVALID_ARGUMENT](#) if *iPage* is greater than the index of the last existing page

Parameters

in	<i>iPage</i>	The target page index
----	--------------	-----------------------

14.77.2.6 GetNumMappedParameterIDs()

```
virtual AAX_Result AAX_IACFPageTable::GetNumMappedParameterIDs (
    int32_t iPage,
    int32_t & oNumParameterIdentifiers ) const [pure virtual]
```

Returns the total number of parameter IDs which are mapped to a page.

Note

The number of mapped parameter IDs does not correspond to the actual slot indices of the parameter assignments. For example, a page could have three total parameter assignments with parameters mapped to slots 2, 4, and 6.

Returns

[AAX_ERROR_INVALID_ARGUMENT](#) if *iPage* is greater than the index of the last existing page

Parameters

in	<i>iPage</i>	The target page index
out	<i>oNumParameterIdentifiers</i>	The number of parameter identifiers which are mapped to the target page

14.77.2.7 ClearMappedParameter()

```
virtual AAX_Result AAX_IACFPageTable::ClearMappedParameter (
    int32_t iPage,
    int32_t iIndex ) [pure virtual]
```

Clear the parameter at a particular index in this table.

Returns

[AAX_SUCCESS](#) even if no parameter was mapped at the given index (the index is still clear)

Parameters

in	<i>iPage</i>	The target page index
in	<i>iIndex</i>	The target parameter slot index within the target page

14.77.2.8 GetMappedParameterID()

```
virtual AAX\_Result AAX_IACFPageTable::GetMappedParameterID (
    int32_t iPage,
    int32_t iIndex,
    AAX\_IString & oParameterIdentifier ) const [pure virtual]
```

Get the parameter identifier which is currently mapped to an index in this table.

Returns

[AAX_ERROR_INVALID_ARGUMENT](#) if no parameter is mapped at the specified page/index location

Parameters

in	<i>iPage</i>	The target page index
in	<i>iIndex</i>	The target parameter slot index within the target page
out	<i>oParameterIdentifier</i>	The identifier used for the mapped parameter in the page table (may be parameter name or ID)

14.77.2.9 MapParameterID()

```
virtual AAX\_Result AAX_IACFPageTable::MapParameterID (
    AAX\_CParamID iParameterIdentifier,
    int32_t iPage,
    int32_t iIndex ) [pure virtual]
```

Map a parameter to this table.

If *iParameterIdentifier* is an empty string then the parameter assignment will be cleared

Returns

[AAX_ERROR_NULL_ARGUMENT](#) if *iParameterIdentifier* is null

[AAX_ERROR_INVALID_ARGUMENT](#) if *iPage* is greater than the index of the last existing page

[AAX_ERROR_INVALID_ARGUMENT](#) if *iIndex* is negative

Parameters

in	<i>iParameterIdentifier</i>	The identifier for the parameter which will be mapped
in	<i>iPage</i>	The target page index
in	<i>iIndex</i>	The target parameter slot index within the target page

The documentation for this class was generated from the following file:

- [AAX_IACFPageTable.h](#)

14.78 AAX_IACFPageTable_V2 Class Reference

```
#include <AAX_IACFPageTable.h>
```

Inheritance diagram for AAX_IACFPageTable_V2:

Collaboration diagram for AAX_IACFPageTable_V2:

14.78.1 Description

Versioned interface to the host's representation of a plug-in instance's page table.

Public Member Functions

- virtual [AAX_Result GetNumParametersWithNameVariations](#) (int32_t &oNumParameterIdentifiers) const =0
- virtual [AAX_Result GetNameVariationParameterIDAtIndex](#) (int32_t iIndex, [AAX_IString](#) &oParameterIdentifier) const =0
- virtual [AAX_Result GetNumNameVariationsForParameter](#) ([AAX_CPageTableParamID](#) iParameterIdentifier, int32_t &oNumVariations) const =0
- virtual [AAX_Result GetParameterNameVariationAtIndex](#) ([AAX_CPageTableParamID](#) iParameterIdentifier, int32_t iIndex, [AAX_IString](#) &oNameVariation, int32_t &oLength) const =0
- virtual [AAX_Result GetParameterNameVariationOfLength](#) ([AAX_CPageTableParamID](#) iParameterIdentifier, int32_t iLength, [AAX_IString](#) &oNameVariation) const =0
- virtual [AAX_Result ClearParameterNameVariations](#) ()=0
- virtual [AAX_Result ClearNameVariationsForParameter](#) ([AAX_CPageTableParamID](#) iParameterIdentifier)=0
- virtual [AAX_Result SetParameterNameVariation](#) ([AAX_CPageTableParamID](#) iParameterIdentifier, const [AAX_IString](#) &iNameVariation, int32_t iLength)=0

14.78.2 Member Function Documentation

14.78.2.1 GetNumParametersWithNameVariations()

```
virtual AAX_Result AAX_IACFPageTable_V2::GetNumParametersWithNameVariations (
    int32_t & oNumParameterIdentifiers ) const [pure virtual]
```

Get the number of parameters with name variations defined for the current table type

Provides the number of parameters with `lt;ControlNameVariationslt;` which are explicitly defined for the current page table type.

Note

Normally parameter name variations are only used with the 'PgTL' table type

See also

- [AAX_IPageTable::GetNameVariationParameterIDAtIndex\(\)](#)

Parameters

out	<i>oNumParameterIdentifiers</i>	The number of parameters with name variations explicitly associated with the current table type.
-----	---------------------------------	--

14.78.2.2 GetNameVariationParameterIDAtIndex()

```
virtual AAX_Result AAX_IACFPageTable_V2::GetNameVariationParameterIDAtIndex (
    int32_t iIndex,
    AAX_IString & oParameterIdentifier ) const [pure virtual]
```

Get the identifier for a parameter with name variations defined for the current table type

Note

Normally parameter name variations are only used with the 'PgTL' table type

See also

- [AAX_IPageTable::GetNumParametersWithNameVariations\(\)](#)

Parameters

in	<i>iIndex</i>	The target parameter index within the list of parameters with explicit name variations defined for this table type.
out	<i>oParameterIdentifier</i>	The identifier used for the parameter in the page table name variations list (may be parameter name or ID)

14.78.2.3 GetNumNameVariationsForParameter()

```
virtual AAX_Result AAX_IACFPageTable_V2::GetNumNameVariationsForParameter (
    AAX_CPageTableParamID iParameterIdentifier,
    int32_t & oNumVariations ) const [pure virtual]
```

Get the number of name variations defined for a parameter

Provides the number of `lt;ControlNameVariationslt;` which are explicitly defined for `iParameterIdentifier` for the current page table type. No fallback logic is used to resolve this to the list of variations which would actually be used for an attached control surface if no explicit variations are defined for the current table type.

Note

Normally parameter name variations are only used with the 'PgTL' table type

See also

- [AAX_IPageTable::GetParameterNameVariationAtIndex\(\)](#)

Returns

[AAX_SUCCESS](#) and provides zero to `oNumVariations` if `iParameterIdentifier` is not found

Parameters

in	<i>iParameterIdentifier</i>	The identifier for the parameter
out	<i>oNumVariations</i>	The number of name variations which are defined for this parameter and explicitly associated with the current table type.

14.78.2.4 GetParameterNameVariationAtIndex()

```
virtual AAX_Result AAX_IACFPageTable_V2::GetParameterNameVariationAtIndex (
    AAX_CPageTableParamID iParameterIdentifier,
    int32_t iIndex,
    AAX_IString & oNameVariation,
    int32_t & oLength ) const [pure virtual]
```

Get a parameter name variation from the page table

Only returns `lt;ControlNameVariationslt;` which are explicitly defined for the current page table type. No fallback logic is used to resolve this to the abbreviation which would actually be shown on an attached control surface if no explicit variation is defined for the current table type.

Note

Normally parameter name variations are only used with the 'PgTL' table type

See also

- [AAX_IPageTable::GetNumNameVariationsForParameter\(\)](#)

See also

- [AAX_IPageTable::GetParameterNameVariationOfLength\(\)](#)

Returns

[AAX_ERROR_NO_ABBREVIATED_PARAMETER_NAME](#) if no suitable variation is defined for this table

[AAX_ERROR_ARGUMENT_OUT_OF_RANGE](#) if `iIndex` is out of range

Parameters

in	<i>iParameterIdentifier</i>	The identifier for the parameter
in	<i>iIndex</i>	Index of the name variation
out	<i>oNameVariation</i>	The name variation, if one is explicitly defined for this table type
out	<i>oLength</i>	The length value for this name variation. This corresponds to the variation's <code>sz</code> attribute in the page table XML and may be different from the string length of <code>iNameVariation</code> .

14.78.2.5 GetParameterNameVariationOfLength()

```
virtual AAX_Result AAX_IACFPageTable_V2::GetParameterNameVariationOfLength (
    AAX_CPageTableParamID iParameterIdentifier,
    int32_t iLength,
    AAX_IString & oNameVariation ) const [pure virtual]
```

Get a parameter name variation of a particular length from the page table

Only returns `lt;ControlNameVariationslt;` which are explicitly defined of `iLength` for the current page table type. No fallback logic is used to resolve this to the abbreviation which would actually be shown on an attached control surface if no explicit variation is defined for the specified length or current table type.

Note

Normally parameter name variations are only used with the 'PgTL' table type

See also

- [AAX_IPageTable::GetParameterNameVariationAtIndex\(\)](#)

Returns

[AAX_ERROR_NO_ABBREVIATED_PARAMETER_NAME](#) if no suitable variation is defined for this table

Parameters

in	<i>iParameterIdentifier</i>	The identifier for the parameter
in	<i>iLength</i>	The variation length to check, i.e. the <code>sz</code> attribute for the name variation in the page table XML
out	<i>oNameVariation</i>	The name variation, if one is explicitly defined for this table type and <code>iLength</code>

14.78.2.6 ClearParameterNameVariations()

```
virtual AAX\_Result AAX_IACFPageTable_V2::ClearParameterNameVariations ( ) [pure virtual]
```

Clears all name variations for the current page table type

Note

Normally parameter name variations are only used with the 'PgTL' table type

See also

[AAX_IPageTable::Clear\(\)](#)

[AAX_IPageTable::ClearNameVariationsForParameter\(\)](#)

14.78.2.7 ClearNameVariationsForParameter()

```
virtual AAX\_Result AAX_IACFPageTable_V2::ClearNameVariationsForParameter (
    AAX\_CPageTableParamID iParameterIdentifier ) [pure virtual]
```

Clears all name variations for a single parameter for the current page table type

Warning

This will invalidate the list of parameter name variations indices, i.e. the parameter identifier associated with each index by [AAX_IPageTable::GetNameVariationParameterIDAtIndex\(\)](#)

Note

Normally parameter name variations are only used with the 'PgTL' table type

See also

[AAX_IPageTable::Clear\(\)](#)

[AAX_IPageTable::ClearParameterNameVariations\(\)](#)

Returns

[AAX_SUCCESS](#) and provides zero to `oNumVariations` if `iParameterIdentifier` is not found

Parameters

in	<i>iParameterIdentifier</i>	The identifier for the parameter
----	-----------------------------	----------------------------------

14.78.2.8 SetParameterNameVariation()

```
virtual AAX_Result AAX_IACFPageTable_V2::SetParameterNameVariation (
    AAX_CPageTableParamID iParameterIdentifier,
    const AAX_IString & iNameVariation,
    int32_t iLength ) [pure virtual]
```

Sets a name variation explicitly for the current page table type

This will add a new name variation or overwrite the existing name variation with the same length which is defined for the current table type.

Warning

If no name variation previously existed for this parameter then this will invalidate the list of parameter name variations indices, i.e. the parameter identifier associated with each index by [AAX_IPageTable::GetNameVariationParameterIDAt](#)

Note

Normally parameter name variations are only used with the 'PgTL' table type

Returns

AAX_ERROR_INVALID_ARGUMENT if *iNameVariation* is empty or if *iLength* is less than zero

Parameters

in	<i>iParameterIdentifier</i>	The identifier for the parameter
in	<i>iNameVariation</i>	The new parameter name variation
in	<i>iLength</i>	The length value for this name variation. This corresponds to the variation's <i>sz</i> attribute in the page table XML and is not required to match the length of <i>iNameVariation</i> .

The documentation for this class was generated from the following file:

- [AAX_IACFPageTable.h](#)

14.79 AAX_IACFPageTableController Class Reference

```
#include <AAX_IACFPageTableController.h>
```

Inheritance diagram for AAX_IACFPageTableController:

Collaboration diagram for AAX_IACFPageTableController:

14.79.1 Description

Interface for host operations related to the page tables for this plug-in.

Note

In the AAX Library, access to this interface is provided through [AAX_IController](#)

Public Member Functions

- virtual [AAX_Result](#) [CopyTableForEffect](#) ([AAX_CPropertyValue](#) inManufacturerID, [AAX_CPropertyValue](#) inProductID, [AAX_CPropertyValue](#) inPlugInID, uint32_t inTableType, int32_t inTablePageSize, [IACFUnknown](#) *oPageTable) const =0
- virtual [AAX_Result](#) [CopyTableOfLayoutForEffect](#) (const char *inEffectID, const char *inLayoutName, uint32_t inTableType, int32_t inTablePageSize, [IACFUnknown](#) *oPageTable) const =0

14.79.2 Member Function Documentation

14.79.2.1 CopyTableForEffect()

```
virtual AAX\_Result AAX_IACFPageTableController::CopyTableForEffect (
    AAX\_CPropertyValue inManufacturerID,
    AAX\_CPropertyValue inProductID,
    AAX\_CPropertyValue inPlugInID,
    uint32_t inTableType,
    int32_t inTablePageSize,
    IACFUnknown * oPageTable ) const [pure virtual]
```

Copy the current page table data for a particular plug-in type.

The host will reject the copy and return an error if the requested plug-in type is unknown, if inTableType is unknown or if inTablePageSize is not a supported size for the given table type.

The host may also restrict plug-ins to only copying page table data from certain plug-in types, such as plug-ins from the same manufacturer or plug-in types within the same effect.

See [Page Table Guide](#) for more information about page tables.

Returns

[AAX_ERROR_NULL_ARGUMENT](#) if oPageTable is null

[AAX_ERROR_INVALID_ARGUMENT](#) if no valid page table mapping can be created due to the specified arguments

Parameters

in	inManufacturerID	Manufacturer ID of the desired plug-in type
in	inProductID	Product ID of the desired plug-in type
in	inPlugInID	Type ID of the desired plug-in type (AAX_eProperty_PlugInID_Native , AAX_eProperty_PlugInID_Tl)
in	inTableType	Four-char type identifier for the requested table type (e.g. 'PgTL', 'Av81', etc.)
in	inTablePageSize	Page size for the requested table. Some tables support multiple page sizes.
out	oPageTable	The page table object to which the page table data should be copied

See also

[AAX_IController::CreateTableCopyForEffect\(\)](#)

14.79.2.2 CopyTableOfLayoutForEffect()

```
virtual AAX\_Result AAX_IACFPageTableController::CopyTableOfLayoutForEffect (
    const char * inEffectID,
    const char * inLayoutName,
    uint32_t inTableType,
    int32_t inTablePageSize,
    IACFUnknown * oPageTable ) const [pure virtual]
```

Copy the current page table data for a particular plug-in effect and page table layout.

The host will reject the copy and return an error if the requested effect ID is unknown or if *inLayoutName* is not a valid layout name for the page tables registered for the effect.

The host may also restrict plug-ins to only copying page table data from certain effects, such as effects registered within the current [AAX](#) plug-in bundle.

See [Page Table Guide](#) for more information about page tables.

Returns

[AAX_ERROR_NULL_ARGUMENT](#) if *inEffectID*, *inLayoutName*, or *oPageTable* is null

[AAX_ERROR_INVALID_ARGUMENT](#) if no valid page table mapping can be created due to the specified arguments

Parameters

in	<i>inEffectID</i>	Effect ID for the desired effect. See AAX_ICollection::AddEffect()
in	<i>inLayoutName</i>	Page table layout name ("name" attribute of the <code>PTLayout</code> XML tag)
in	<i>inTableType</i>	Four-char type identifier for the requested table type (e.g. 'PgTL', 'Av81', etc.)
in	<i>inTablePageSize</i>	Page size for the requested table. Some tables support multiple page sizes.
out	<i>oPageTable</i>	The page table object to which the page table data should be copied. <i>oPageTable</i> must support AAX_IACFPageTable

See also

[AAX_IController::CreateTableCopyForLayout\(\)](#)

The documentation for this class was generated from the following file:

- [AAX_IACFPageTableController.h](#)

14.80 AAX_IACFPageTableController_V2 Class Reference

```
#include <AAX_IACFPageTableController.h>
```

Inheritance diagram for AAX_IACFPageTableController_V2:

Collaboration diagram for AAX_IACFPageTableController_V2:

14.80.1 Description

Interface for host operations related to the page tables for this plug-in.

Note

In the AAX Library, access to this interface is provided through [AAX_IController](#)

Public Member Functions

- virtual [AAX_Result](#) [CopyTableForEffectFromFile](#) (const char *inPageTableFilePath, [AAX_ETextEncoding](#) inFilePathEncoding, [AAX_CPropertyValue](#) inManufacturerID, [AAX_CPropertyValue](#) inProductID, [AAX_CPropertyValue](#) inPlugInID, uint32_t inTableType, int32_t inTablePageSize, [IACFUnknown](#) *oPageTable) const =0
- virtual [AAX_Result](#) [CopyTableOfLayoutFromFile](#) (const char *inPageTableFilePath, [AAX_ETextEncoding](#) inFilePathEncoding, const char *inLayoutName, uint32_t inTableType, int32_t inTablePageSize, [IACFUnknown](#) *oPageTable) const =0

14.80.2 Member Function Documentation

14.80.2.1 CopyTableForEffectFromFile()

```
virtual AAX\_Result AAX_IACFPageTableController_V2::CopyTableForEffectFromFile (
    const char * inPageTableFilePath,
    AAX\_ETextEncoding inFilePathEncoding,
    AAX\_CPropertyValue inManufacturerID,
    AAX\_CPropertyValue inProductID,
    AAX\_CPropertyValue inPlugInID,
    uint32_t inTableType,
    int32_t inTablePageSize,
    IACFUnknown * oPageTable ) const [pure virtual]
```

Returns

[AAX_ERROR_NULL_ARGUMENT](#) if inPageTableFilePath or oPageTable is null

[AAX_ERROR_UNSUPPORTED_ENCODING](#) if inFilePathEncoding has unsupported encoding value

[AAX_ERROR_INVALID_ARGUMENT](#) if no valid page table mapping can be created due to the specified arguments

Parameters

in	<i>inPageTableFilePath</i>	Path to XML page table file.
in	<i>inFilePathEncoding</i>	File path text encoding.
in	<i>inManufacturerID</i>	Manufacturer ID of the desired plug-in type
in	<i>inProductID</i>	Product ID of the desired plug-in type
in	<i>inPlugInID</i>	Type ID of the desired plug-in type (AAX_eProperty_PlugInID_Native , AAX_eProperty_PlugInID_TI)
in	<i>inTableType</i>	Four-char type identifier for the requested table type (e.g. 'PgTL', 'Av81', etc.)
in	<i>inTablePageSize</i>	Page size for the requested table. Some tables support multiple page sizes.
out	<i>oPageTable</i>	The page table object to which the page table data should be copied. <i>oPageTable</i> must support AAX_IACFPageTable

See also

[AAX_IController::CreateTableCopyForEffect\(\)](#)

14.80.2.2 CopyTableOfLayoutFromFile()

```
virtual AAX\_Result AAX_IACFPageTableController_V2::CopyTableOfLayoutFromFile (
    const char * inPageTableFilePath,
    AAX\_ETextEncoding inFilePathEncoding,
    const char * inLayoutName,
    uint32_t inTableType,
    int32_t inTablePageSize,
    IACFUnknown * oPageTable ) const [pure virtual]
```

Returns

[AAX_ERROR_NULL_ARGUMENT](#) if *inPageTableFilePath*, *inLayoutName*, or *oPageTable* is null

[AAX_ERROR_UNSUPPORTED_ENCODING](#) if *inFilePathEncoding* has unsupported encoding value

[AAX_ERROR_INVALID_ARGUMENT](#) if no valid page table mapping can be created due to the specified arguments

Parameters

in	<i>inPageTableFilePath</i>	Path to XML page table file.
in	<i>inFilePathEncoding</i>	File path text encoding.
in	<i>inLayoutName</i>	Page table layout name ("name" attribute of the PTLayout XML tag)
in	<i>inTableType</i>	Four-char type identifier for the requested table type (e.g. 'PgTL', 'Av81', etc.)
in	<i>inTablePageSize</i>	Page size for the requested table. Some tables support multiple page sizes.
out	<i>oPageTable</i>	The page table object to which the page table data should be copied. <i>oPageTable</i> must support AAX_IACFPageTable

See also

[AAX_IController::CreateTableCopyForLayout\(\)](#)

The documentation for this class was generated from the following file:

- [AAX_IACFPageTableController.h](#)

14.81 AAX_IACFPrivateDataAccess Class Reference

```
#include <AAX_IACFPrivateDataAccess.h>
```

Inheritance diagram for AAX_IACFPrivateDataAccess:

Collaboration diagram for AAX_IACFPrivateDataAccess:

14.81.1 Description

Interface for the AAX host's data access functionality.

Public Member Functions

- virtual [AAX_Result](#) [ReadPortDirect](#) ([AAX_CFieldIndex](#) inFieldIndex, const uint32_t inOffset, const uint32_t inSize, void *outBuffer)=0
Read data directly from DSP at the given port.
- virtual [AAX_Result](#) [WritePortDirect](#) ([AAX_CFieldIndex](#) inFieldIndex, const uint32_t inOffset, const uint32_t inSize, const void *inBuffer)=0
Write data directly to DSP at the given port.

14.81.2 Member Function Documentation

14.81.2.1 ReadPortDirect()

```
virtual AAX\_Result AAX_IACFPrivateDataAccess::ReadPortDirect (
    AAX\_CFieldIndex inFieldIndex,
    const uint32_t inOffset,
    const uint32_t inSize,
    void * outBuffer ) [pure virtual]
```

Read data directly from DSP at the given port.

Note

Blocking

Parameters

in	<i>inFieldIndex</i>	The port to read from.
in	<i>inOffset</i>	Offset into data to start reading.
in	<i>inSize</i>	Amount of data to read (in bytes).
out	<i>outBuffer</i>	Pointer to storage for data to be read into.

14.81.2.2 WritePortDirect()

```
virtual AAX_Result AAX_IACFPrivateDataAccess::WritePortDirect (
    AAX_CFieldIndex inFieldIndex,
    const uint32_t inOffset,
    const uint32_t inSize,
    const void * inBuffer ) [pure virtual]
```

Write data directly to DSP at the given port.

Note

Blocking

Parameters

in	<i>inFieldIndex</i>	The port to write to.
in	<i>inOffset</i>	Offset into data to begin writing.
in	<i>inSize</i>	Amount of data to write (in bytes).
in	<i>inBuffer</i>	Pointer to data being written.

The documentation for this class was generated from the following file:

- [AAX_IACFPrivateDataAccess.h](#)

14.82 AAX_IACFPropertyMap Class Reference

```
#include <AAX_IACFPropertyMap.h>
```

Inheritance diagram for AAX_IACFPropertyMap:

Collaboration diagram for AAX_IACFPropertyMap:

14.82.1 Description

Versioned interface for an [AAX_IPropertyMap](#).

Public Member Functions

- virtual [AAX_CBoolean](#) [GetProperty](#) ([AAX_EProperty](#) inProperty, [AAX_CPropertyValue](#) *outValue) const =0
Get a property value from a property map.
- virtual [AAX_Result](#) [AddProperty](#) ([AAX_EProperty](#) inProperty, [AAX_CPropertyValue](#) inValue)=0
Add a property to a property map.
- virtual [AAX_Result](#) [RemoveProperty](#) ([AAX_EProperty](#) inProperty)=0
Remove a property from a property map.

14.82.2 Member Function Documentation

14.82.2.1 GetProperty()

```
virtual AAX\_CBoolean AAX_IACFPropertyMap::GetProperty (
    AAX\_EProperty inProperty,
    AAX\_CPropertyValue * outValue ) const [pure virtual]
```

Get a property value from a property map.

Returns true if the selected property is supported, false if it is not

Parameters

in	<i>inProperty</i>	The property ID
out	<i>outValue</i>	The property value

14.82.2.2 AddProperty()

```
virtual AAX\_Result AAX_IACFPropertyMap::AddProperty (
    AAX\_EProperty inProperty,
    AAX\_CPropertyValue inValue ) [pure virtual]
```

Add a property to a property map.

Note

This method may return an error if adding the property was unsuccessful. If there is a failure when adding a required property then registration of the relevant description element must be abandoned and the plug-in's description logic should proceed to the next element.

Parameters

in	<i>inProperty</i>	The property ID.
in	<i>inValue</i>	

14.82.2.3 RemoveProperty()

```
virtual AAX_Result AAX_IACFPropertyMap::RemoveProperty (
    AAX_EProperty inProperty ) [pure virtual]
```

Remove a property from a property map.

Parameters

in	<i>inProperty</i>	The property ID.
----	-------------------	------------------

The documentation for this class was generated from the following file:

- [AAX_IACFPropertyMap.h](#)

14.83 AAX_IACFPropertyMap_V2 Class Reference

```
#include <AAX_IACFPropertyMap.h>
```

Inheritance diagram for AAX_IACFPropertyMap_V2:

Collaboration diagram for AAX_IACFPropertyMap_V2:

14.83.1 Description

Versioned interface for an [AAX_IPropertyMap](#).

Public Member Functions

- virtual [AAX_Result](#) [AddPropertyWithIDArray](#) ([AAX_EProperty](#) inProperty, const [AAX_SPlugInIdentifierTriad](#) *inPluginIDs, uint32_t inNumPluginIDs)=0
Add an array of plug-in IDs to a property map.
- virtual [AAX_CBoolean](#) [GetPropertyWithIDArray](#) ([AAX_EProperty](#) inProperty, const [AAX_SPlugInIdentifierTriad](#) **outPluginIDs, uint32_t *outNumPluginIDs) const =0
Get an array of plug-in IDs from a property map.

14.83.2 Member Function Documentation

14.83.2.1 AddPropertyWithIDArray()

```
virtual AAX_Result AAX_IACFPropertyMap_V2::AddPropertyWithIDArray (
    AAX_EProperty inProperty,
    const AAX_SPlugInIdentifierTriad * inPluginIDs,
    uint32_t inNumPluginIDs ) [pure virtual]
```

Add an array of plug-in IDs to a property map.

Parameters

in	<i>inProperty</i>	The property ID.
in	<i>inPluginIDs</i>	An array of AAX_SPlugInIdentifierTriad
in	<i>inNumPluginIDs</i>	The length of iPluginIDs

14.83.2.2 GetPropertyWithIDArray()

```
virtual AAX\_CBoolean AAX_IACFPropertyMap_V2::GetPropertyWithIDArray (
    AAX\_EProperty inProperty,
    const AAX\_SPlugInIdentifierTriad ** outPluginIDs,
    uint32_t * outNumPluginIDs ) const [pure virtual]
```

Get an array of plug-in IDs from a property map.

Parameters

in	<i>inProperty</i>	The property ID.
out	<i>outPluginIDs</i>	A pointer that will be set to reference an array of AAX_SPlugInIdentifierTriad
in	<i>outNumPluginIDs</i>	The length of oPluginIDs

The documentation for this class was generated from the following file:

- [AAX_IACFPropertyMap.h](#)

14.84 AAX_IACFPropertyMap_V3 Class Reference

```
#include <AAX_IACFPropertyMap.h>
```

Inheritance diagram for AAX_IACFPropertyMap_V3:

Collaboration diagram for AAX_IACFPropertyMap_V3:

14.84.1 Description

Versioned interface for an [AAX_IPropertyMap](#).

Public Member Functions

- virtual [AAX_CBoolean](#) GetProperty64 ([AAX_EProperty](#) inProperty, [AAX_CPropertyValue64](#) *outValue) const =0
Get a property value from a property map.
- virtual [AAX_Result](#) AddProperty64 ([AAX_EProperty](#) inProperty, [AAX_CPropertyValue64](#) inValue)=0
Add a property to a property map.

14.84.2 Member Function Documentation

14.84.2.1 GetProperty64()

```
virtual AAX_CBoolean AAX_IACFPropertyMap_V3::GetProperty64 (
    AAX_EProperty inProperty,
    AAX_CPropertyValue64 * outValue ) const [pure virtual]
```

Get a property value from a property map.

Returns true if the selected property is supported, false if it is not

Parameters

in	<i>inProperty</i>	The property ID
out	<i>outValue</i>	The property value

14.84.2.2 AddProperty64()

```
virtual AAX_Result AAX_IACFPropertyMap_V3::AddProperty64 (
    AAX_EProperty inProperty,
    AAX_CPropertyValue64 inValue ) [pure virtual]
```

Add a property to a property map.

Note

This method may return an error if adding the property was unsuccessful. If there is a failure when adding a required property then registration of the relevant description element must be abandoned and the plug-in's description logic should proceed to the next element.

Parameters

in	<i>inProperty</i>	The property ID.
in	<i>inValue</i>	

The documentation for this class was generated from the following file:

- [AAX_IACFPropertyMap.h](#)

14.85 AAX_IACFSessionDocument Class Reference

```
#include <AAX_IACFSessionDocument.h>
```

Inheritance diagram for AAX_IACFSessionDocument:

Collaboration diagram for AAX_IACFSessionDocument:

14.85.1 Description

Interface representing information in a host session document.

Plug-in implementations should use [AAX_ISessionDocument](#) , which provides specific convenience methods for supported data types.

Public Member Functions

- virtual [AAX_Result](#) [GetDocumentData](#) ([AAX_DocumentData_UID](#) const &inDataType, [IACFUnknown](#) **outData)=0

Get data from the document.

14.85.2 Member Function Documentation

14.85.2.1 GetDocumentData()

```
virtual AAX\_Result AAX_IACFSessionDocument::GetDocumentData (
    AAX\_DocumentData\_UID const & inDataType,
    IACFUnknown ** outData ) [pure virtual]
```

Get data from the document.

Get document data of a generic type

Similar to [QueryInterface\(\)](#) but uses a data type identifier rather than a true IID

The provided interface has already had a reference added, so be careful not to add an additional reference:

```
ACFPtr<MyType> ptr;
IACFUnknown * docDataPtr{nullptr};
if (AAX_SUCCESS == doc->GetDocumentData(dataUID, &docDataPtr) && docDataPtr) {
    ptr.attach(std::static_cast<MyType*>(docDataPtr)); // attach does not AddRef
}
```

Parameters

in	<i>inDataType</i>	The type of the document data requested
out	<i>outData</i>	An interface providing the requested data, or <code>nullptr</code> if the host does not support or cannot provide the requested data type. The reference count has been incremented on this object on behalf of the caller, so the caller must not add an additional reference count and must decrement the reference count on this object to release it. For information about which interface to expect for each requested data type, see the documentation for that data type.

The documentation for this class was generated from the following file:

- [AAX_IACFSessionDocument.h](#)

14.86 AAX_IACFSessionDocumentClient Class Reference

```
#include <AAX_IACFSessionDocumentClient.h>
```

Inheritance diagram for AAX_IACFSessionDocumentClient:

Collaboration diagram for AAX_IACFSessionDocumentClient:

14.86.1 Description

Interface representing a client of the session document interface.

For example, a plug-in implementation that makes calls on the session document interface provided by the host.

Public Member Functions

Initialization and uninitialization

- virtual [AAX_Result Initialize](#) ([IACFUnknown](#) *iUnknown)=0
- virtual [AAX_Result Uninitialize](#) (void)=0

Session document access

- virtual [AAX_Result SetSessionDocument](#) ([IACFUnknown](#) *iSessionDocument)=0
Sets or removes a session document.

AAX host and plug-in event notification

- virtual [AAX_Result NotificationReceived](#) ([AAX_CTypeID](#) inNotificationType, const void *inNotificationData, uint32_t inNotificationDataSize)=0
Notification Hook.

14.86.2 Member Function Documentation

14.86.2.1 Initialize()

```
virtual AAX\_Result AAX_IACFSessionDocumentClient::Initialize (
    IACFUnknown * iUnknown ) [pure virtual]
```

Implemented in [AAX_CSessionDocumentClient](#).

14.86.2.2 Uninitialize()

```
virtual AAX_Result AAX_IACFSessionDocumentClient::Uninitialize (
    void ) [pure virtual]
```

Implemented in [AAX_CSessionDocumentClient](#).

14.86.2.3 SetSessionDocument()

```
virtual AAX_Result AAX_IACFSessionDocumentClient::SetSessionDocument (
    IACFUnknown * iSessionDocument ) [pure virtual]
```

Sets or removes a session document.

Parameters

in	<i>iSessionDocument</i>	Interface supporting at least AAX_IACFSessionDocument , or nullptr to indicate that any session document that is currently held should be released.
----	-------------------------	---

Implemented in [AAX_CSessionDocumentClient](#).

14.86.2.4 NotificationReceived()

```
virtual AAX_Result AAX_IACFSessionDocumentClient::NotificationReceived (
    AAX_CTypeID inNotificationType,
    const void * inNotificationData,
    uint32_t inNotificationDataSize ) [pure virtual]
```

Notification Hook.

Called from the host to deliver notifications to this object.

Look at the [AAX_ENotificationEvent](#) enumeration to see a description of events you can listen for and the data they come with.

Note

- Different notifications are sent to different objects within a plug-in. If you are not seeing an expected notification, try checking the other plug-in objects' [NotificationReceived\(\)](#) methods.

Note

- the host may dispatch notifications synchronously or asynchronously, and calls to this method may occur concurrently on multiple threads.

A plug-in may also dispatch custom notifications using [AAX_IController::SendNotification\(\)](#). Custom notifications will be posted back to the plug-in's other objects which support a [NotificationReceived\(\)](#) method (e.g. the data model).

Parameters

in	<i>inNotificationType</i>	Type of notification being received. Notifications from the host are one of AAX_ENotificationEvent
in	<i>inNotificationData</i>	Block of incoming notification data
in	<i>inNotificationDataSize</i>	Size of <i>inNotificationData</i> , in bytes

Implemented in [AAX_CSessionDocumentClient](#).

The documentation for this class was generated from the following file:

- [AAX_IACFSessionDocumentClient.h](#)

14.87 AAX_IACFTask Class Reference

```
#include <AAX_IACFTask.h>
```

Inheritance diagram for AAX_IACFTask:

Collaboration diagram for AAX_IACFTask:

14.87.1 Description

Versioned interface for an asynchronous task.

:Implemented by the AAX Host

Used by the [task agent](#).

This interface describes a task request and provides a way for the agent to express one or more results of the task as well as the progress of the task.

This interface is open-ended for both inputs and outputs. The host and agent must use common definitions for specific task types, their possible arguments, and the expected results.

Public Member Functions

- virtual [AAX_Result](#) [GetType](#) ([AAX_CTypeID](#) *oType) const =0
- virtual [AAX_IACFDataBuffer](#) const * [GetArgumentOfType](#) ([AAX_CTypeID](#) iType) const =0
- virtual [AAX_Result](#) [SetProgress](#) (float iProgress)=0
- virtual float [GetProgress](#) () const =0
- virtual [AAX_Result](#) [AddResult](#) ([AAX_IACFDataBuffer](#) const *iResult)=0
Attach result data to this task.
- virtual [AAX_Result](#) [SetDone](#) ([AAX_TaskCompletionStatus](#) iStatus)=0
Inform the host that the task is completed.

14.87.2 Member Function Documentation

14.87.2.1 GetType()

```
virtual AAX\_Result AAX_IACFTask::GetType (
    AAX\_CTypeID * oType ) const [pure virtual]
```

An identifier defining the type of the requested task

Parameters

out	<i>oType</i>	The type of this task request
-----	--------------	-------------------------------

Implemented in [AAX_CTask](#).

14.87.2.2 GetArgumentOfType()

```
virtual AAX\_IACFDataBuffer const* AAX_IACFTask::GetArgumentOfType (
    AAX\_CTypeID iType ) const [pure virtual]
```

Additional information defining the request, depending on the task type

Parameters

in	<i>iType</i>	The type of argument requested. Possible argument types, if any, and the resulting data buffer format must be defined per task type.
----	--------------	--

Returns

The requested argument data, or nullptr. This data buffer's type ID is expected to match *iType* . The caller takes ownership of this object.

Implemented in [AAX_CTask](#).

14.87.2.3 SetProgress()

```
virtual AAX\_Result AAX_IACFTask::SetProgress (
    float iProgress ) [pure virtual]
```

Inform the host about the current status of the task

Parameters

in	<i>iProgress</i>	A value between 0 (no progress) and 1 (complete)
----	------------------	--

Implemented in [AAX_CTask](#).

14.87.2.4 GetProgress()

```
virtual float AAX_IACFTask::GetProgress ( ) const [pure virtual]
```

Returns the current progress

Implemented in [AAX_CTask](#).

14.87.2.5 AddResult()

```
virtual AAX_Result AAX_IACFTask::AddResult (
    AAX_IACFDataBuffer const * iResult ) [pure virtual]
```

Attach result data to this task.

This can be called multiple times to add multiple types of results to a single task.

The host may process the result data immediately or may wait for the task to complete.

The plug-in is expected to release the data buffer upon making this call. At a minimum, the data buffer must not be changed after this call is made. See `ACFPtr::inArg()`

Parameters

in	<i>iResult</i>	A buffer containing the result data. Expected result types, if any, and their data buffer format must be defined per task type.
----	----------------	---

Implemented in [AAX_CTask](#).

14.87.2.6 SetDone()

```
virtual AAX_Result AAX_IACFTask::SetDone (
    AAX_TaskCompletionStatus iStatus ) [pure virtual]
```

Inform the host that the task is completed.

If [AAX_SUCCESS](#) is returned, the object should be considered invalid and released by the caller.

Parameters

in	<i>iStatus</i>	The final status of the task. This indicates to the host whether or not the task was performed as requested.
----	----------------	--

Implemented in [AAX_CTask](#).

The documentation for this class was generated from the following file:

- [AAX_IACFTask.h](#)

14.88 AAX_IACFTaskAgent Class Reference

```
#include <AAX_IACFTaskAgent.h>
```

Inheritance diagram for AAX_IACFTaskAgent:

Collaboration diagram for AAX_IACFTaskAgent:

14.88.1 Description

Versioned interface for a component that accepts task requests.

:Implemented by the Plug-In

The task agent is expected to complete the requested tasks asynchronously and to provide progress and completion details via calls on the [AAX_IACFTask](#) interface as the tasks proceed.

See also

[AAX_ITask](#)

Public Member Functions

Initialization and uninitialization

- virtual [AAX_Result Initialize](#) ([IACFUnknown](#) *iController)=0
- virtual [AAX_Result Uninitialize](#) ()=0

Task management

- virtual [AAX_Result AddTask](#) ([IACFUnknown](#) *iTask)=0
- virtual [AAX_Result CancelAllTasks](#) ()=0

14.88.2 Member Function Documentation

14.88.2.1 Initialize()

```
virtual AAX\_Result AAX_IACFTaskAgent::Initialize (
    IACFUnknown * iController ) [pure virtual]
```

Initialize the object

Parameters

in	<i>iController</i>	Interface allowing access to other objects in the object graph such as the plug-in's data model.
----	--------------------	--

Implemented in [AAX_CTaskAgent](#).

14.88.2.2 Uninitialize()

```
virtual AAX_Result AAX_IACFTaskAgent::Uninitialize ( ) [pure virtual]
```

Uninitialize the object

This method should release references to any shared objects

Implemented in [AAX_CTaskAgent](#).

14.88.2.3 AddTask()

```
virtual AAX_Result AAX_IACFTaskAgent::AddTask (
    IACFUnknown * iTask ) [pure virtual]
```

Request that the agent perform a task

Parameters

in	<i>iTask</i>	The task to perform. The agent must retain a reference to this task if it will be used beyond the scope of this method. This object should support at least AAX_IACFTask .
----	--------------	--

Implemented in [AAX_CTaskAgent](#).

14.88.2.4 CancelAllTasks()

```
virtual AAX_Result AAX_IACFTaskAgent::CancelAllTasks ( ) [pure virtual]
```

Request that the agent cancel all outstanding tasks

Implemented in [AAX_CTaskAgent](#).

The documentation for this class was generated from the following file:

- [AAX_IACFTaskAgent.h](#)

14.89 AAX_IACFTransport Class Reference

```
#include <AAX_IACFTransport.h>
```

Inheritance diagram for AAX_IACFTransport:

Collaboration diagram for AAX_IACFTransport:

14.89.1 Description

Versioned interface to get information about the host's transport state.

Public Member Functions

- virtual [AAX_Result GetCurrentTempo](#) (double *TempoBPM) const =0
CALL: Gets the current tempo.
- virtual [AAX_Result GetCurrentMeter](#) (int32_t *MeterNumerator, int32_t *MeterDenominator) const =0
CALL: Gets the current meter.
- virtual [AAX_Result IsTransportPlaying](#) (bool *isPlaying) const =0
CALL: Indicates whether or not the transport is playing back.
- virtual [AAX_Result GetCurrentTickPosition](#) (int64_t *TickPosition) const =0
CALL: Gets the current tick position.
- virtual [AAX_Result GetCurrentLoopPosition](#) (bool *bLooping, int64_t *LoopStartTick, int64_t *LoopEndTick) const =0
CALL: Gets current information on loop playback.
- virtual [AAX_Result GetCurrentNativeSampleLocation](#) (int64_t *SampleLocation) const =0
CALL: Gets the current playback location of the native audio engine.
- virtual [AAX_Result GetCustomTickPosition](#) (int64_t *oTickPosition, int64_t iSampleLocation) const =0
CALL: Given an absolute sample position, gets the corresponding tick position.
- virtual [AAX_Result GetBarBeatPosition](#) (int32_t *Bars, int32_t *Beats, int64_t *DisplayTicks, int64_t SampleLocation) const =0
CALL: Given an absolute sample position, gets the corresponding bar and beat position.
- virtual [AAX_Result GetTicksPerQuarter](#) (uint32_t *ticks) const =0
CALL: Retrieves the number of ticks per quarter note.
- virtual [AAX_Result GetCurrentTicksPerBeat](#) (uint32_t *ticks) const =0
CALL: Retrieves the number of ticks per beat.

14.89.2 Member Function Documentation

14.89.2.1 GetCurrentTempo()

```
virtual AAX\_Result AAX_IACFTransport::GetCurrentTempo (
    double * TempoBPM ) const [pure virtual]
```

CALL: Gets the current tempo.

Returns the tempo corresponding to the current position of the transport counter

Note

The resolution of the tempo returned here is based on the host's tempo resolution, so it will match the tempo displayed in the host. Use [GetCurrentTicksPerBeat\(\)](#) to calculate the tempo resolution note.

Parameters

out	<i>TempoBPM</i>	The current tempo in beats per minute
-----	-----------------	---------------------------------------

14.89.2.2 GetCurrentMeter()

```
virtual AAX_Result AAX_IACFTransport::GetCurrentMeter (
    int32_t * MeterNumerator,
    int32_t * MeterDenominator ) const [pure virtual]
```

CALL: Gets the current meter.

Returns the meter corresponding to the current position of the transport counter

Parameters

out	<i>MeterNumerator</i>	The numerator portion of the meter
out	<i>MeterDenominator</i>	The denominator portion of the meter

14.89.2.3 IsTransportPlaying()

```
virtual AAX_Result AAX_IACFTransport::IsTransportPlaying (
    bool * isPlaying ) const [pure virtual]
```

CALL: Indicates whether or not the transport is playing back.

Parameters

out	<i>isPlaying</i>	true if the transport is currently in playback
-----	------------------	--

14.89.2.4 GetCurrentTickPosition()

```
virtual AAX_Result AAX_IACFTransport::GetCurrentTickPosition (
    int64_t * TickPosition ) const [pure virtual]
```

CALL: Gets the current tick position.

Returns the current tick position corresponding to the current transport position. One "Tick" is represented here as 1/960000 of a quarter note. That is, there are 960,000 of these ticks in a quarter note.

Host Compatibility Notes The tick resolution here is different than that of the tick displayed in Pro Tools. "Display ticks" (as they are called) are 1/960 of a quarter note.

Parameters

out	<i>TickPosition</i>	The tick position value
-----	---------------------	-------------------------

14.89.2.5 GetCurrentLoopPosition()

```
virtual AAX_Result AAX_IACFTransport::GetCurrentLoopPosition (
    bool * bLooping,
    int64_t * LoopStartTick,
    int64_t * LoopEndTick ) const [pure virtual]
```

CALL: Gets current information on loop playback.

Host Compatibility Notes This does not indicate anything about the status of the "Loop Record" option. Even when the host is configured to loop playback, looping may not occur if certain conditions are not met (i.e. the length of the selection is too short)

Parameters

out	<i>bLooping</i>	true if the host is configured to loop playback
out	<i>LoopStartTick</i>	The starting tick position of the selection being looped (see GetCurrentTickPosition())
out	<i>LoopEndTick</i>	The ending tick position of the selection being looped (see GetCurrentTickPosition())

14.89.2.6 GetCurrentNativeSampleLocation()

```
virtual AAX_Result AAX_IACFTransport::GetCurrentNativeSampleLocation (
    int64_t * SampleLocation ) const [pure virtual]
```

CALL: Gets the current playback location of the native audio engine.

When called from a ProcessProc render callback, this method will provide the absolute sample location at the beginning of the callback's audio buffers.

When called from [AAX_IEffectParameters::RenderAudio_Hybrid\(\)](#), this method will provide the absolute sample location for the samples in the method's **output** audio buffers. To calculate the absolute sample location for the samples in the method's input buffers (i.e. the timeline location where the samples originated) subtract the value provided by [AAX_IController::GetHybridSignalLatency\(\)](#) from this value.

When called from a non-real-time thread, this method will provide the current location of the samples being processed by the plug-in's ProcessProc on its real-time processing thread.

Note

This method only returns a value during playback. It cannot be used to determine, e.g., the location of the timeline selector while the host is not in playback.

Parameters

out	<i>SampleLocation</i>	Absolute sample location of the first sample in the current native processing buffer
-----	-----------------------	--

14.89.2.7 GetCustomTickPosition()

```
virtual AAX_Result AAX_IACFTransport::GetCustomTickPosition (
    int64_t * oTickPosition,
    int64_t iSampleLocation ) const [pure virtual]
```

CALL: Given an absolute sample position, gets the corresponding tick position.

Host Compatibility Notes There is a minor performance cost associated with using this API in Pro Tools. It should not be used excessively without need.

Parameters

out	<i>oTickPosition</i>	the timeline tick position corresponding to <i>iSampleLocation</i>
in	<i>iSampleLocation</i>	An absolute sample location (see GetCurrentNativeSampleLocation())

14.89.2.8 GetBarBeatPosition()

```
virtual AAX_Result AAX_IACFTransport::GetBarBeatPosition (
    int32_t * Bars,
    int32_t * Beats,
    int64_t * DisplayTicks,
    int64_t SampleLocation ) const [pure virtual]
```

CALL: Given an absolute sample position, gets the corresponding bar and beat position.

Host Compatibility Notes There is a minor performance cost associated with using this API in Pro Tools. It should not be used excessively without need.

Parameters

out	<i>Bars</i>	The bar corresponding to <i>SampleLocation</i>
out	<i>Beats</i>	The beat corresponding to <i>SampleLocation</i>
out	<i>DisplayTicks</i>	The ticks corresponding to <i>SampleLocation</i>
in	<i>SampleLocation</i>	An absolute sample location (see GetCurrentNativeSampleLocation())

14.89.2.9 GetTicksPerQuarter()

```
virtual AAX_Result AAX_IACFTransport::GetTicksPerQuarter (
    uint32_t * ticks ) const [pure virtual]
```

CALL: Retrieves the number of ticks per quarter note.

Parameters

out	ticks	
-----	-------	--

14.89.2.10 GetCurrentTicksPerBeat()

```
virtual AAX_Result AAX_IACFTransport::GetCurrentTicksPerBeat (
    uint32_t * ticks ) const [pure virtual]
```

CALL: Retrieves the number of ticks per beat.

Parameters

out	ticks	
-----	-------	--

The documentation for this class was generated from the following file:

- [AAX_IACFTransport.h](#)

14.90 AAX_IACFTransport_V2 Class Reference

```
#include <AAX_IACFTransport.h>
```

Inheritance diagram for AAX_IACFTransport_V2:

Collaboration diagram for AAX_IACFTransport_V2:

14.90.1 Description

Versioned interface to get information about the host's transport state.

Public Member Functions

- virtual [AAX_Result GetTimelineSelectionStartPosition](#) (int64_t *oSampleLocation) const =0
CALL: Retrieves the absolute sample position of the beginning of the current transport selection.
- virtual [AAX_Result GetTimeCodeInfo](#) (AAX_EFrameRate *oFrameRate, int32_t *oOffset) const =0
CALL: Retrieves the current time code frame rate and offset.
- virtual [AAX_Result GetFeetFramesInfo](#) (AAX_EFeetFramesRate *oFeetFramesRate, int64_t *oOffset) const =0
CALL: Retrieves the current timecode feet/frames rate and offset.
- virtual [AAX_Result IsMetronomeEnabled](#) (int32_t *isEnabled) const =0
Sets isEnabled to true if the metronome is enabled.

14.90.2 Member Function Documentation

14.90.2.1 GetTimelineSelectionStartPosition()

```
virtual AAX_Result AAX_IACFTransport_V2::GetTimelineSelectionStartPosition (
    int64_t * oSampleLocation ) const [pure virtual]
```

CALL: Retrieves the absolute sample position of the beginning of the current transport selection.

Note

This method is part of the [version 2 transport interface](#)

Parameters

out	<i>oSampleLocation</i>	
-----	------------------------	--

14.90.2.2 GetTimeCodeInfo()

```
virtual AAX_Result AAX_IACFTransport_V2::GetTimeCodeInfo (
    AAX_EFrameRate * oFrameRate,
    int32_t * oOffset ) const [pure virtual]
```

CALL: Retrieves the current time code frame rate and offset.

Note

This method is part of the [version 2 transport interface](#)

Parameters

out	<i>oFrameRate</i>	
out	<i>oOffset</i>	

14.90.2.3 GetFeetFramesInfo()

```
virtual AAX_Result AAX_IACFTransport_V2::GetFeetFramesInfo (
    AAX_EFeetFramesRate * oFeetFramesRate,
    int64_t * oOffset ) const [pure virtual]
```

CALL: Retrieves the current timecode feet/frames rate and offset.

Note

This method is part of the [version 2 transport interface](#)

Parameters

out	<i>oFeetFramesRate</i>	
out	<i>oOffset</i>	

14.90.2.4 IsMetronomeEnabled()

```
virtual AAX\_Result AAX_IACFTransport_V2::IsMetronomeEnabled (
    int32_t * isEnabled ) const [pure virtual]
```

Sets isEnabled to true if the metronome is enabled.

Note

This method is part of the [version 2 transport interface](#)

Parameters

out	<i>isEnabled</i>	
-----	------------------	--

The documentation for this class was generated from the following file:

- [AAX_IACFTransport.h](#)

14.91 AAX_IACFTransport_V3 Class Reference

```
#include <AAX_IACFTransport.h>
```

Inheritance diagram for AAX_IACFTransport_V3:

Collaboration diagram for AAX_IACFTransport_V3:

14.91.1 Description

Versioned interface to get information about the host's transport state.

Public Member Functions

- virtual [AAX_Result](#) GetHDTimeCodeInfo ([AAX_EFrameRate](#) *oHDFrameRate, int64_t *oHDOffset) const =0
CALL: Retrieves the current HD time code frame rate and offset.

14.91.2 Member Function Documentation

14.91.2.1 GetHDTimeCodeInfo()

```
virtual AAX_Result AAX_IACFTransport_V3::GetHDTimeCodeInfo (
    AAX_EFrameRate * oHDFrameRate,
    int64_t * oHDOffset ) const [pure virtual]
```

CALL: Retrieves the current HD time code frame rate and offset.

Note

This method is part of the [version 3 transport interface](#)

Parameters

out	<i>oHDFrameRate</i>	
out	<i>oHDOffset</i>	

The documentation for this class was generated from the following file:

- [AAX_IACFTransport.h](#)

14.92 AAX_IACFTransport_V4 Class Reference

```
#include <AAX_IACFTransport.h>
```

Inheritance diagram for AAX_IACFTransport_V4:

Collaboration diagram for AAX_IACFTransport_V4:

14.92.1 Description

Versioned interface to get information about the host's transport state.

Public Member Functions

- virtual [AAX_Result GetTimelineSelectionEndPosition](#) (int64_t *oSampleLocation) const =0
CALL: Retrieves the absolute sample position of the end of the current transport selection.

14.92.2 Member Function Documentation

14.92.2.1 GetTimelineSelectionEndPosition()

```
virtual AAX_Result AAX_IACFTransport_V4::GetTimelineSelectionEndPosition (
    int64_t * oSampleLocation ) const [pure virtual]
```

CALL: Retrieves the absolute sample position of the end of the current transport selection.

Note

This method is part of the [version 4 transport interface](#)

Parameters

out	<i>oSampleLocation</i>	
-----	------------------------	--

The documentation for this class was generated from the following file:

- [AAX_IACFTransport.h](#)

14.93 AAX_IACFTransport_V5 Class Reference

```
#include <AAX_IACFTransport.h>
```

Inheritance diagram for AAX_IACFTransport_V5:

Collaboration diagram for AAX_IACFTransport_V5:

14.93.1 Description

Versioned interface to get information about the host's transport state.

Public Member Functions

- virtual [AAX_Result GetKeySignature](#) (int64_t iSampleLocation, uint32_t *oKeySignature) const =0
CALL: Retrieves the key signature at a sample location.

14.93.2 Member Function Documentation

14.93.2.1 GetKeySignature()

```
virtual AAX_Result AAX_IACFTransport_V5::GetKeySignature (
    int64_t iSampleLocation,
    uint32_t * oKeySignature ) const [pure virtual]
```

CALL: Retrieves the key signature at a sample location.

The signature is provided as a bitfield:

- 31-20: Chromatic scale elements, ordered MSB (root) to LSB
- 19-4: (Reserved)
- 3-0: Root note (C natural = 0)

For example

```
* D# Major
* Ionian D#
* 0b 101011010101 0000 00000000 0000 0011
*
* E Phrygian
* Phrygian E
* 0b 110101011010 0000 00000000 0000 0100
*
* Chromatic
* Chromatic C
* 0b 111111111111 0000 00000000 0000 0000
*
```

The documentation for this class was generated from the following file:

- [AAX_IACFTransport.h](#)

14.94 AAX_IACFTransportControl Class Reference

```
#include <AAX_IACFTransportControl.h>
```

Inheritance diagram for AAX_IACFTransportControl:

Collaboration diagram for AAX_IACFTransportControl:

14.94.1 Description

Versioned interface to control the host's transport state.

Public Member Functions

- virtual [AAX_Result RequestTransportStart](#) ()=0
CALL: Request that the host transport start playback.
- virtual [AAX_Result RequestTransportStop](#) ()=0
CALL: Request that the host transport stop playback.

14.94.2 Member Function Documentation

14.94.2.1 RequestTransportStart()

```
virtual AAX\_Result AAX_IACFTransportControl::RequestTransportStart ( ) [pure virtual]
```

CALL: Request that the host transport start playback.

Note

This method is part of the [AAX_IACFTransportControl](#) interface

14.94.2.2 RequestTransportStop()

```
virtual AAX\_Result AAX_IACFTransportControl::RequestTransportStop ( ) [pure virtual]
```

CALL: Request that the host transport stop playback.

Note

This method is part of the [AAX_IACFTransportControl](#) interface

The documentation for this class was generated from the following file:

- [AAX_IACFTransportControl.h](#)

14.95 AAX_IACFViewContainer Class Reference

```
#include <AAX_IACFViewContainer.h>
```

Inheritance diagram for AAX_IACFViewContainer:

Collaboration diagram for AAX_IACFViewContainer:

14.95.1 Description

Interface for the AAX host's view of a single instance of an effect. Used by both clients of the host app and by effect components.

See also

[AAX_IViewContainer](#)

Public Member Functions

View and GUI state queries

- virtual `int32_t GetType ()=0`
Returns the raw view type as one of [AAX_EViewContainer_Type](#).
- virtual `void * GetPtr ()=0`
Returns a pointer to the raw view.
- virtual `AAX_Result GetModifiers (uint32_t *outModifiers)=0`
Queries the host for the current [modifier keys](#).

View change requests

- virtual `AAX_Result SetViewSize (AAX_Point &inSize)=0`
Request a change to the main view size.

Host event handlers

- virtual `AAX_Result HandleParameterMouseDown (AAX_CParamID inParamID, uint32_t inModifiers)=0`
Alert the host to a mouse down event.
- virtual `AAX_Result HandleParameterMouseDrag (AAX_CParamID inParamID, uint32_t inModifiers)=0`
Alert the host to a mouse drag event.
- virtual `AAX_Result HandleParameterMouseUp (AAX_CParamID inParamID, uint32_t inModifiers)=0`
Alert the host to a mouse up event.

14.95.2 Member Function Documentation

14.95.2.1 GetType()

```
virtual int32_t AAX_IACFViewContainer::GetType ( ) [pure virtual]
```

Returns the raw view type as one of [AAX_EViewContainer_Type](#).

14.95.2.2 GetPtr()

```
virtual void* AAX_IACFViewContainer::GetPtr ( ) [pure virtual]
```

Returns a pointer to the raw view.

14.95.2.3 GetModifiers()

```
virtual AAX_Result AAX_IACFViewContainer::GetModifiers (
    uint32_t * outModifiers ) [pure virtual]
```

Queries the host for the current [modifier keys](#).

This method returns a bit mask with bits set for each of the currently active modifier keys. This method does not return the state of the [AAX_eModifiers_SecondaryButton](#).

Host Compatibility Notes Although this method allows plug-ins to acquire the current state of the Windows key (normally blocked by Pro Tools), plug-ins should not use key combinations that require this key.

Parameters

out	<i>outModifiers</i>	Current modifiers as a bitmask of AAX_EModifiers
-----	---------------------	--

14.95.2.4 SetViewSize()

```
virtual AAX_Result AAX_IACFViewContainer::SetViewSize (
    AAX_Point & inSize ) [pure virtual]
```

Request a change to the main view size.

Note

- For compatibility with the smallest supported displays, plug-in GUI dimensions should not exceed 749x617 pixels, or 749x565 pixels for plug-ins with sidechain support.

Parameters

in	<i>inSize</i>	The new size to which the plug-in view should be set
----	---------------	--

14.95.2.5 HandleParameterMouseDown()

```
virtual AAX_Result AAX_IACFViewContainer::HandleParameterMouseDown (
    AAX_CParamID inParamID,
    uint32_t inModifiers ) [pure virtual]
```

Alert the host to a mouse down event.

Parameters

in	<i>inParamID</i>	ID of the parameter whose control is being edited
in	<i>inModifiers</i>	A bitmask of AAX_EModifiers values

14.95.2.6 HandleParameterMouseDrag()

```
virtual AAX_Result AAX_IACFViewContainer::HandleParameterMouseDrag (
    AAX_CParamID inParamID,
    uint32_t inModifiers ) [pure virtual]
```

Alert the host to a mouse drag event.

Warning

The host may return [AAX_ERROR_UNIMPLEMENTED](#) for this event even if the host did handle the corresponding mouse down event. A plug-in should ignore any following mouse drag and mouse up events that correspond to a host-managed mouse down event. ([PTSW-195209](#) / [PT-218474](#))

Parameters

in	<i>inParamID</i>	ID of the parameter whose control is being edited
in	<i>inModifiers</i>	A bitmask of AAX_EModifiers values

14.95.2.7 HandleParameterMouseUp()

```
virtual AAX\_Result AAX_IACFViewContainer::HandleParameterMouseUp (
    AAX\_CParamID inParamID,
    uint32_t inModifiers ) [pure virtual]
```

Alert the host to a mouse up event.

Warning

The host may return [AAX_ERROR_UNIMPLEMENTED](#) for this event even if the host did handle the corresponding mouse down event. A plug-in should ignore any following mouse drag and mouse up events that correspond to a host-managed mouse down event. ([PTSW-195209](#) / [PT-218474](#))

Parameters

in	<i>inParamID</i>	ID of the parameter whose control is being edited
in	<i>inModifiers</i>	A bitmask of AAX_EModifiers values

The documentation for this class was generated from the following file:

- [AAX_IACFViewContainer.h](#)

14.96 AAX_IACFViewContainer_V2 Class Reference

```
#include <AAX_IACFViewContainer.h>
```

Inheritance diagram for AAX_IACFViewContainer_V2:

Collaboration diagram for AAX_IACFViewContainer_V2:

14.96.1 Description

Supplemental interface for the AAX host's view of a single instance of an effect. Used by both clients of the host app and by effect components.

See also

[AAX_IViewContainer](#)

Public Member Functions

Host event handlers

- virtual [AAX_Result HandleMultipleParametersMouseDown](#) (const [AAX_CParamID](#) *inParamIDs, uint32_t inNumOfParams, uint32_t inModifiers)=0
Alert the host to a mouse down event.
- virtual [AAX_Result HandleMultipleParametersMouseDrag](#) (const [AAX_CParamID](#) *inParamIDs, uint32_t inNumOfParams, uint32_t inModifiers)=0
Alert the host to a mouse drag event.
- virtual [AAX_Result HandleMultipleParametersMouseUp](#) (const [AAX_CParamID](#) *inParamIDs, uint32_t inNumOfParams, uint32_t inModifiers)=0
Alert the host to a mouse up event.

14.96.2 Member Function Documentation

14.96.2.1 HandleMultipleParametersMouseDown()

```
virtual AAX\_Result AAX_IACFViewContainer_V2::HandleMultipleParametersMouseDown (
    const AAX\_CParamID * inParamIDs,
    uint32_t inNumOfParams,
    uint32_t inModifiers ) [pure virtual]
```

Alert the host to a mouse down event.

Parameters

in	<i>inParamIDs</i>	IDs of the parameters that belong to the same GUI element whose controls are being edited
in	<i>inNumOfParams</i>	Number of parameter IDS
in	<i>inModifiers</i>	A bitmask of AAX_EModifiers values

14.96.2.2 HandleMultipleParametersMouseDrag()

```
virtual AAX\_Result AAX_IACFViewContainer_V2::HandleMultipleParametersMouseDrag (
    const AAX\_CParamID * inParamIDs,
    uint32_t inNumOfParams,
    uint32_t inModifiers ) [pure virtual]
```

Alert the host to a mouse drag event.

Warning

The host may return [AAX_ERROR_UNIMPLEMENTED](#) for this event even if the host did handle the corresponding mouse down event. A plug-in should ignore any following mouse drag and mouse up events that correspond to a host-managed mouse down event. ([PTSW-195209](#) / [PT-218474](#))

Parameters

in	<i>inParamIDs</i>	IDs of the parameters that belong to the same GUI element whose controls are being edited
in	<i>inNumOfParams</i>	Number of parameter IDS
in	<i>inModifiers</i>	A bitmask of AAX_EModifiers values

14.96.2.3 HandleMultipleParametersMouseUp()

```
virtual AAX\_Result AAX_IACFViewContainer_V2::HandleMultipleParametersMouseUp (
    const AAX\_CParamID * inParamIDs,
    uint32_t inNumOfParams,
    uint32_t inModifiers ) [pure virtual]
```

Alert the host to a mouse up event.

Warning

The host may return [AAX_ERROR_UNIMPLEMENTED](#) for this event even if the host did handle the corresponding mouse down event. A plug-in should ignore any following mouse drag and mouse up events that correspond to a host-managed mouse down event. ([PTSW-195209](#) / [PT-218474](#))

Parameters

in	<i>inParamIDs</i>	IDs of the parameters that belong to the same GUI element whose controls are being edited
in	<i>inNumOfParams</i>	Number of parameter IDS
in	<i>inModifiers</i>	A bitmask of AAX_EModifiers values

The documentation for this class was generated from the following file:

- [AAX_IACFViewContainer.h](#)

14.97 AAX_IACFViewContainer_V3 Class Reference

```
#include <AAX_IACFViewContainer.h>
```

Inheritance diagram for AAX_IACFViewContainer_V3:

Collaboration diagram for AAX_IACFViewContainer_V3:

14.97.1 Description

Additional methods to track mouse as it moves over controls.

See also

[AAX_IViewContainer](#)

Public Member Functions

Host event handlers

- virtual [AAX_Result](#) [HandleParameterMouseEnter](#) ([AAX_CParamID](#) inParamID, uint32_t inModifiers)=0
Alert the host to a mouse enter event to the parameter's control.
- virtual [AAX_Result](#) [HandleParameterMouseExit](#) ([AAX_CParamID](#) inParamID, uint32_t inModifiers)=0
Alert the host to a mouse exit event from the parameter's control.

14.97.2 Member Function Documentation

14.97.2.1 HandleParameterMouseEnter()

```
virtual AAX\_Result AAX_IACFViewContainer_V3::HandleParameterMouseEnter (
    AAX\_CParamID inParamID,
    uint32_t inModifiers ) [pure virtual]
```

Alert the host to a mouse enter event to the parameter's control.

Parameters

in	<i>inParamID</i>	ID of the parameter whose control is being entered
in	<i>inModifiers</i>	A bitmask of AAX_EModifiers values

Returns [AAX_SUCCESS](#) if event was processed successfully, otherwise an [AAX_ERROR](#) code

14.97.2.2 HandleParameterMouseExit()

```
virtual AAX\_Result AAX_IACFViewContainer_V3::HandleParameterMouseExit (
    AAX\_CParamID inParamID,
    uint32_t inModifiers ) [pure virtual]
```

Alert the host to a mouse exit event from the parameter's control.

Parameters

in	<i>inParamID</i>	ID of the parameter whose control is being exited
in	<i>inModifiers</i>	A bitmask of AAX_EModifiers values

Returns [AAX_SUCCESS](#) if event was processed successfully, otherwise an [AAX_ERROR](#) code

The documentation for this class was generated from the following file:

- [AAX_IACFViewContainer.h](#)

14.98 AAX_IAutomationDelegate Class Reference

```
#include <AAX_IAutomationDelegate.h>
```

Inheritance diagram for AAX_IAutomationDelegate:

14.98.1 Description

Interface allowing an AAX plug-in to interact with the host's event system.

Implemented by the AAX Host

This delegate provides a means of interacting with the host's event system in order to ensure that events such as parameter updates are properly arbitrated and broadcast to all listeners. The automation delegate is used regardless of whether or not an individual parameter is "automatable" or "automation-enabled".

A parameter must be registered with the automation delegate in order for updates to the parameter's control in the plug-in's GUI or other controller (control surface, etc.) to be successfully processed by the host and sent to the [AAX_IEffectParameters](#) object.

The parameter identifiers used by this interface correspond to the control IDs used to identify parameters in the [Parameter Manager](#).

Public Member Functions

- virtual [~AAX_IAutomationDelegate](#) ()
- virtual [AAX_Result RegisterParameter](#) ([AAX_CParamID](#) iParameterID)=0
- virtual [AAX_Result UnregisterParameter](#) ([AAX_CParamID](#) iParameterID)=0
- virtual [AAX_Result PostSetValueRequest](#) ([AAX_CParamID](#) iParameterID, double normalizedValue) const =0
- virtual [AAX_Result PostCurrentValue](#) ([AAX_CParamID](#) iParameterID, double normalizedValue) const =0
- virtual [AAX_Result PostTouchRequest](#) ([AAX_CParamID](#) iParameterID)=0
- virtual [AAX_Result PostReleaseRequest](#) ([AAX_CParamID](#) iParameterID)=0
- virtual [AAX_Result GetTouchState](#) ([AAX_CParamID](#) iParameterID, [AAX_CBoolean](#) *oTouched)=0
- virtual [AAX_Result ParameterNameChanged](#) ([AAX_CParamID](#) iParameterID)=0

14.98.2 Constructor & Destructor Documentation

14.98.2.1 ~AAX_IAutomationDelegate()

```
virtual AAX_IAutomationDelegate::~~AAX_IAutomationDelegate ( ) [inline], [virtual]
```

14.98.3 Member Function Documentation

14.98.3.1 RegisterParameter()

```
virtual AAX_Result AAX_IAutomationDelegate::RegisterParameter (
    AAX_CParamID iParameterID ) [pure virtual]
```

Register a control with the automation system using a char* based control identifier

The automation delegate owns a list of the IDs of all of the parameters that have been registered with it. This list is used to set up listeners for all of the registered parameters such that the automation delegate may update the plug-in when the state of any of the registered parameters have been modified.

See also

[AAX_IAutomationDelegate::UnregisterParameter\(\)](#)

Parameters

in	<i>iParameterID</i>	Parameter ID that is being registered
----	---------------------	---------------------------------------

Implemented in [AAX_VAutomationDelegate](#).

Referenced by `AAX_CParameter< T >::SetAutomationDelegate()`, and `AAX_CStatelessParameter::SetAutomationDelegate()`.

Here is the caller graph for this function:

14.98.3.2 UnregisterParameter()

```
virtual AAX_Result AAX_IAutomationDelegate::UnregisterParameter (
    AAX_CParamID iParameterID ) [pure virtual]
```

Unregister a control with the automation system using a char* based control identifier

Note

All registered controls should be unregistered or the system might leak.

See also

[AAX_IAutomationDelegate::RegisterParameter\(\)](#)

Parameters

in	<i>iParameterID</i>	Parameter ID that is being registered
----	---------------------	---------------------------------------

Implemented in [AAX_VAutomationDelegate](#).

Referenced by `AAX_CStatelessParameter::SetAutomationDelegate()`.

Here is the caller graph for this function:

14.98.3.3 PostSetValueRequest()

```
virtual AAX_Result AAX_IAutomationDelegate::PostSetValueRequest (
    AAX_CParamID iParameterID,
    double normalizedValue ) const [pure virtual]
```

Submits a request for the given parameter's value to be changed

Parameters

in	<i>iParameterID</i>	ID of the parameter for which a change is requested
in	<i>normalizedValue</i>	The requested new parameter value, formatted as a double and normalized to [0 1]

Implemented in [AAX_VAutomationDelegate](#).

14.98.3.4 PostCurrentValue()

```
virtual AAX_Result AAX_IAutomationDelegate::PostCurrentValue (
    AAX_CParamID iParameterID,
    double normalizedValue ) const [pure virtual]
```

Notifies listeners that a parameter's value has changed

Parameters

in	<i>iParameterID</i>	ID of the parameter that has been updated
in	<i>normalizedValue</i>	The current parameter value, formatted as a double and normalized to [0 1]

Implemented in [AAX_VAutomationDelegate](#).

14.98.3.5 PostTouchRequest()

```
virtual AAX_Result AAX_IAutomationDelegate::PostTouchRequest (
    AAX_CParamID iParameterID ) [pure virtual]
```

Requests that the given parameter be "touched", i.e. locked for updates by the current client

Parameters

in	<i>iParameterID</i>	ID of the parameter that will be touched
----	---------------------	--

Implemented in [AAX_VAutomationDelegate](#).

Referenced by [AAX_CStatelessParameter::Touch\(\)](#).

Here is the caller graph for this function:

14.98.3.6 PostReleaseRequest()

```
virtual AAX_Result AAX_IAutomationDelegate::PostReleaseRequest (
    AAX_CParamID iParameterID ) [pure virtual]
```

Requests that the given parameter be "released", i.e. available for updates from any client

Parameters

in	<i>iParameterID</i>	ID of the parameter that will be released
----	---------------------	---

Implemented in [AAX_VAutomationDelegate](#).

Referenced by [AAX_CStatelessParameter::Release\(\)](#).

Here is the caller graph for this function:

14.98.3.7 GetTouchState()

```
virtual AAX_Result AAX_IAutomationDelegate::GetTouchState (
    AAX_CParamID iParameterID,
    AAX_CBoolean * oTouched ) [pure virtual]
```

Gets the current touched state of a parameter

Parameters

in	<i>iParameterID</i>	ID of the parameter that is being queried
out	<i>oTouched</i>	The current touch state of the parameter

Implemented in [AAX_VAutomationDelegate](#).

14.98.3.8 ParameterNameChanged()

```
virtual AAX_Result AAX_IAutomationDelegate::ParameterNameChanged (
    AAX_CParamID iParameterID ) [pure virtual]
```

Notify listeners that the parameter's display name has changed

Note that this is not part of the underlying automation delegate interface with the host; it is converted on the AAX side to a notification posted to the host via the [AAX_IController](#) .

Parameters

in	<i>iParameterID</i>	ID of the parameter that has been updated
----	---------------------	---

Implemented in [AAX_VAutomationDelegate](#).

Referenced by `AAX_CStatelessParameter::SetName()`.

Here is the caller graph for this function:

The documentation for this class was generated from the following file:

- [AAX_IAutomationDelegate.h](#)

14.99 AAX_ICollection Class Reference

```
#include <AAX_ICollection.h>
```

Inheritance diagram for AAX_ICollection:

14.99.1 Description

Interface to represent a plug-in binary's static description.

:Implemented by the AAX Host

The [AAX_ICollection](#) interface provides a creation function for new plug-in descriptors, which in turn provides access to the various interfaces necessary for describing a plug-in. When a plug-in description is complete, it is added to the collection via the [AddEffect](#) method. The [AAX_ICollection](#) interface also provides some additional description methods that are used to describe the overall plug-in package. These methods can be used to describe the plug-in package's name, the name of the plug-in's manufacturer, and the plug-in package version.

Legacy Porting Notes The information in [AAX_ICollection](#) is roughly analogous to the information provided by `CProcessGroup` in the legacy plug-in library

Public Member Functions

- virtual [~AAX_ICollection](#) ()
- virtual [AAX_IEffectDescriptor * NewDescriptor](#) ()=0
Create a new Effect descriptor.
- virtual [AAX_Result AddEffect](#) (const char *inEffectID, [AAX_IEffectDescriptor](#) *inEffectDescriptor)=0
Add an Effect description to the collection.
- virtual [AAX_Result SetManufacturerName](#) (const char *inPackageName)=0
Set the plug-in manufacturer name.
- virtual [AAX_Result AddPackageName](#) (const char *inPackageName)=0
Set the plug-in package name.
- virtual [AAX_Result SetPackageVersion](#) (uint32_t inVersion)=0
Set the plug-in package version number.
- virtual [AAX_IPropertyMap * NewPropertyMap](#) ()=0
Create a new property map.
- virtual [AAX_Result SetProperties](#) ([AAX_IPropertyMap](#) *inProperties)=0
Set the properties of the collection.
- virtual [AAX_Result GetHostVersion](#) (uint32_t *outVersion) const =0
Get the current version of the host.
- virtual [AAX_IDescriptionHost * DescriptionHost](#) ()=0
- virtual const [AAX_IDescriptionHost * DescriptionHost](#) () const =0
- virtual [IACFDefinition * HostDefinition](#) () const =0

14.99.2 Constructor & Destructor Documentation

14.99.2.1 ~AAX_ICollection()

```
virtual AAX_ICollection::~~AAX_ICollection ( ) [inline], [virtual]
```

14.99.3 Member Function Documentation

14.99.3.1 NewDescriptor()

```
virtual AAX_IEffectDescriptor* AAX_ICollection::NewDescriptor ( ) [pure virtual]
```

Create a new Effect descriptor.

Implemented in [AAX_VCollection](#).

14.99.3.2 AddEffect()

```
virtual AAX_Result AAX_ICollection::AddEffect (
    const char * inEffectID,
    AAX_IEffectDescriptor * inEffectDescriptor ) [pure virtual]
```

Add an Effect description to the collection.

Each Effect that a plug-in registers with [AAX_ICollection::AddEffect\(\)](#) is considered a completely different user-facing product. For example, in Avid's Dynamics III plug-in the Expander, Compressor, and DeEsser are each registered as separate Effects. All stem format variations within each Effect are registered within that Effect's [AAX_IEffectDescriptor](#) using [AddComponent\(\)](#).

The [AAX_eProperty_ProductID](#) value for all ProcessProcs within a single Effect must be identical.

This method passes ownership of an [AAX_IEffectDescriptor](#) object to the [AAX_ICollection](#). The [AAX_IEffectDescriptor](#) must not be deleted by the AAX plug-in, nor should it be edited in any way after it is passed to the [AAX_ICollection](#).

Parameters

in	<i>inEffectID</i>	The effect ID.
in	<i>inEffectDescriptor</i>	The Effect descriptor.

Implemented in [AAX_VCollection](#).

14.99.3.3 SetManufacturerName()

```
virtual AAX_Result AAX_ICollection::SetManufacturerName (
    const char * inPackageName ) [pure virtual]
```

Set the plug-in manufacturer name.

Parameters

in	<i>inPackageName</i>	The name of the manufacturer.
----	----------------------	-------------------------------

Implemented in [AAX_VCollection](#).

14.99.3.4 AddPackageName()

```
virtual AAX_Result AAX_ICollection::AddPackageName (
    const char * inPackageName ) [pure virtual]
```

Set the plug-in package name.

May be called multiple times to add abbreviated package names.

Note

Every plug-in must include at least one name variant with 16 or fewer characters, plus a null terminating character. Used for Presets folder.

Parameters

in	<i>inPackageName</i>	The name of the package.
----	----------------------	--------------------------

Implemented in [AAX_VCollection](#).

14.99.3.5 SetPackageVersion()

```
virtual AAX_Result AAX_ICollection::SetPackageVersion (
    uint32_t inVersion ) [pure virtual]
```

Set the plug-in package version number.

Parameters

in	<i>inVersion</i>	The package version numner.
----	------------------	-----------------------------

Implemented in [AAX_VCollection](#).

14.99.3.6 NewPropertyMap()

```
virtual AAX_IPropertyMap* AAX_ICollection::NewPropertyMap ( ) [pure virtual]
```

Create a new property map.

Implemented in [AAX_VCollection](#).

14.99.3.7 SetPropertyMap()

```
virtual AAX_Result AAX_ICollection::SetProperties (
    AAX_IPropertyMap * inProperties ) [pure virtual]
```

Set the properties of the collection.

Parameters

in	<i>inProperties</i>	Collection properties
----	---------------------	-----------------------

Implemented in [AAX_VCollection](#).

14.99.3.8 GetHostVersion()

```
virtual AAX_Result AAX_ICollection::GetHostVersion (
    uint32_t * outVersion ) const [pure virtual]
```

Get the current version of the host.

See [AAXATTR_Client_Version](#) for information about the version data format

Warning

Do not use this method to infer host feature support. Instead, use [AAX_IDescriptionHost](#) to query the host for specific features.

AAX plugin description details are cached and might not be re-scanned even if the host version changes. Be careful to avoid changing any cacheable information based on this data. See [Host feature and environment checks](#) for more information.

Parameters

in	<i>outVersion</i>	Host version
----	-------------------	--------------

Implemented in [AAX_VCollection](#).

14.99.3.9 DescriptionHost() [1/2]

```
virtual AAX\_IDescriptionHost\* AAX_ICollection::DescriptionHost ( ) [pure virtual]
```

Get a pointer to an [AAX_IDescriptionHost](#), if supported by the host

This interface is served by the [AAX_ICollection](#) in order to avoid requiring a new method prototype for the [GetEffectDescriptions\(\)](#) method called from the AAX Library.

See also

[AAX_UIDs.h](#) for available feature UIDs, e.g. [AAXATTR_ClientFeature_AuxOutputStem](#)

Warning

AAX plugin description details are cached and might not be re-scanned even if attributes of the host description change. Be careful to avoid changing any cacheable information based on this data. See [Host feature and environment checks](#) for more information.

Implemented in [AAX_VCollection](#).

14.99.3.10 DescriptionHost() [2/2]

```
virtual const AAX\_IDescriptionHost\* AAX_ICollection::DescriptionHost ( ) const [pure virtual]
```

Get a pointer to an [AAX_IDescriptionHost](#), if supported by the host

This interface is served by the [AAX_ICollection](#) in order to avoid requiring a new method prototype for the [GetEffectDescriptions\(\)](#) method called from the AAX Library.

See also

[AAX_UIDs.h](#) for available feature UIDs, e.g. [AAXATTR_ClientFeature_AuxOutputStem](#)

Warning

AAX plugin description details are cached and might not be re-scanned even if attributes of the host description change. Be careful to avoid changing any cacheable information based on this data. See [Host feature and environment checks](#) for more information.

Implemented in [AAX_VCollection](#).

14.99.3.11 HostDefinition()

```
virtual IACFDefinition* AAX_ICollection::HostDefinition ( ) const [pure virtual]
```

Get a pointer to an [IACFDefinition](#), if supported by the host

This interface is served by the [AAX_ICollection](#) in order to avoid requiring a new method prototype for the [GetEffectDescriptions\(\)](#) method called from the AAX Library.

See also

[AAX_UIDs.h](#) for available host attribute UIDs, e.g. [AAXATTR_Client_Level](#)

Warning

AAX plugin description details are cached and might not be re-scanned even if attributes of the host definition change, for example the host level. Be careful to avoid changing any cacheable information based on this data. See [Host feature and environment checks](#) for more information.

The implementation of [AAX_ICollection](#) owns the referenced object. No AddRef occurs.

[IACFDefinition::DefineAttribute\(\)](#) is not supported on this object

Implemented in [AAX_VCollection](#).

The documentation for this class was generated from the following file:

- [AAX_ICollection.h](#)

14.100 AAX_IComponentDescriptor Class Reference

```
#include <AAX_IComponentDescriptor.h>
```

Inheritance diagram for [AAX_IComponentDescriptor](#):

14.100.1 Description

Description interface for an AAX plug-in component.

[:Implemented by the AAX Host](#)

This is an abstract interface containing everything needed to describe a single algorithm of an Effect. For more information about algorithm processing in AAX plug-ins, see [Real-time algorithm callback](#).

Public Member Functions

- virtual [~AAX_IComponentDescriptor](#) ()
- virtual [AAX_Result Clear](#) ()=0
Clears the descriptor.
- virtual [AAX_Result AddAudioIn](#) ([AAX_CFieldIndex](#) inFieldIndex)=0
Subscribes an audio input context field.
- virtual [AAX_Result AddAudioOut](#) ([AAX_CFieldIndex](#) inFieldIndex)=0
Subscribes an audio output context field.
- virtual [AAX_Result AddAudioBufferLength](#) ([AAX_CFieldIndex](#) inFieldIndex)=0
Subscribes a buffer length context field.
- virtual [AAX_Result AddSampleRate](#) ([AAX_CFieldIndex](#) inFieldIndex)=0
Subscribes a sample rate context field.
- virtual [AAX_Result AddClock](#) ([AAX_CFieldIndex](#) inFieldIndex)=0
Subscribes a clock context field.
- virtual [AAX_Result AddSideChainIn](#) ([AAX_CFieldIndex](#) inFieldIndex)=0
Subscribes a side-chain input context field.
- virtual [AAX_Result AddDataInPort](#) ([AAX_CFieldIndex](#) inFieldIndex, [uint32_t](#) inPacketSize, [AAX_EDataInPortType](#) inPortType=[AAX_eDataInPortType_Buffered](#))=0
Adds a custom data port to the algorithm context.
- virtual [AAX_Result AddAuxOutputStem](#) ([AAX_CFieldIndex](#) inFieldIndex, [int32_t](#) inStemFormat, [const char](#) inNameUTF8[])=0
Adds an auxiliary output stem for a plug-in.
- virtual [AAX_Result AddPrivateData](#) ([AAX_CFieldIndex](#) inFieldIndex, [int32_t](#) inDataSize, [uint32_t](#) inOptions=[AAX_ePrivateDataOptions_DefaultOptions](#))=0
Adds a private data port to the algorithm context.
- virtual [AAX_Result AddTemporaryData](#) ([AAX_CFieldIndex](#) inFieldIndex, [uint32_t](#) inDataElementSize)=0
Adds a block of data to a context that is not saved between callbacks and is scaled by the system buffer size.
- virtual [AAX_Result AddDmaInstance](#) ([AAX_CFieldIndex](#) inFieldIndex, [AAX_IDma::EMode](#) inDmaMode)=0
Adds a DMA field to the plug-in's context.
- virtual [AAX_Result AddMeters](#) ([AAX_CFieldIndex](#) inFieldIndex, [const AAX_CTypeID](#) *inMeterIDs, [const uint32_t](#) inMeterCount)=0
Adds a meter field to the plug-in's context.
- virtual [AAX_Result AddMIDINode](#) ([AAX_CFieldIndex](#) inFieldIndex, [AAX_EMIDINodeType](#) inNodeType, [const char](#) inNodeName[], [uint32_t](#) channelMask)=0
Adds a MIDI node field to the plug-in's context.
- virtual [AAX_Result AddReservedField](#) ([AAX_CFieldIndex](#) inFieldIndex, [uint32_t](#) inFieldType)=0
Subscribes a context field to host-provided services or information.
- virtual [AAX_IPropertyMap * NewPropertyMap](#) () [const](#) =0
Creates a new, empty property map.
- virtual [AAX_IPropertyMap * DuplicatePropertyMap](#) ([AAX_IPropertyMap](#) *inPropertyMap) [const](#) =0
Creates a new property map using an existing property map.
- virtual [AAX_Result AddProcessProc_Native](#) ([AAX_CProcessProc](#) inProcessProc, [AAX_IPropertyMap](#) *inProperties=NULL, [AAX_CInstanceInitProc](#) inInstanceInitProc=NULL, [AAX_CBackgroundProc](#) inBackgroundProc=NULL, [AAX_CSelector](#) *outProcID=NULL)=0
Registers an algorithm processing entrypoint (process procedure) for the native architecture.
- virtual [AAX_Result AddProcessProc_TI](#) ([const char](#) inDLLFileNameUTF8[], [const char](#) inProcessProcSymbol[], [AAX_IPropertyMap](#) *inProperties, [const char](#) inInstanceInitProcSymbol[]=NULL, [const char](#) inBackgroundProcSymbol[]=NULL, [AAX_CSelector](#) *outProcID=NULL)=0
Registers an algorithm processing entrypoint (process procedure) for the native architecture.
- virtual [AAX_Result AddProcessProc](#) ([AAX_IPropertyMap](#) *inProperties, [AAX_CSelector](#) *outProcIDs=NULL, [int32_t](#) inProcIDsSize=0)=0

Registers one or more algorithm processing entrypoints (process procedures)

- `template<typename aContextType >`
`AAX_Result AddProcessProc_Native (void(AAX_CALLBACK *inProcessProc)(aContextType *const inInstancesBegin[], const void *inInstancesEnd), AAX_IPropertyMap *inProperties=NULL, int32_t(AAX_CALLBACK *inInstanceInitProc)(const aContextType *inInstanceContextPtr, AAX_EComponentInstanceInitAction inAction)=NULL, int32_t(AAX_CALLBACK *inBackgroundProc)(void)=NULL)`

Registers an algorithm processing entrypoint (process procedure) for the native architecture.

14.100.2 Constructor & Destructor Documentation

14.100.2.1 ~AAX_IComponentDescriptor()

```
virtual AAX_IComponentDescriptor::~~AAX_IComponentDescriptor ( ) [inline], [virtual]
```

14.100.3 Member Function Documentation

14.100.3.1 Clear()

```
virtual AAX_Result AAX_IComponentDescriptor::Clear ( ) [pure virtual]
```

Clears the descriptor.

Clears the descriptor and readies it for the next algorithm description

Implemented in [AAX_VComponentDescriptor](#).

14.100.3.2 AddAudioIn()

```
virtual AAX_Result AAX_IComponentDescriptor::AddAudioIn (
    AAX_CFieldIndex inFieldIndex ) [pure virtual]
```

Subscribes an audio input context field.

Defines an audio in port for host-provided information in the algorithm's context structure.

- Data type: float**
- Data kind: An array of float arrays, one for each input channel

Parameters

in	<i>inFieldIndex</i>	Unique identifier for the field, generated using AAX_FIELD_INDEX
----	---------------------	--

Implemented in [AAX_VComponentDescriptor](#).

Referenced by `AAX_CMonolithicParameters::StaticDescribe()`.

Here is the caller graph for this function:

14.100.3.3 AddAudioOut()

```
virtual AAX\_Result AAX_IComponentDescriptor::AddAudioOut (
    AAX\_CFieldIndex inFieldIndex ) [pure virtual]
```

Subscribes an audio output context field.

Defines an audio out port for host-provided information in the algorithm's context structure.

- Data type: `float**`
- Data kind: An array of float arrays, one for each output channel

Parameters

in	<i>inFieldIndex</i>	Unique identifier for the field, generated using AAX_FIELD_INDEX
----	---------------------	--

Implemented in [AAX_VComponentDescriptor](#).

Referenced by `AAX_CMonolithicParameters::StaticDescribe()`.

Here is the caller graph for this function:

14.100.3.4 AddAudioBufferLength()

```
virtual AAX\_Result AAX_IComponentDescriptor::AddAudioBufferLength (
    AAX\_CFieldIndex inFieldIndex ) [pure virtual]
```

Subscribes a buffer length context field.

Defines a buffer length port for host-provided information in the algorithm's context structure.

- Data type: `int32_t*`
- Data kind: The number of samples in the current audio buffer

Parameters

in	<i>inFieldIndex</i>	Unique identifier for the field, generated using AAX_FIELD_INDEX
----	---------------------	--

Implemented in [AAX_VComponentDescriptor](#).

Referenced by `AAX_CMonolithicParameters::StaticDescribe()`.

Here is the caller graph for this function:

14.100.3.5 AddSampleRate()

```
virtual AAX_Result AAX_IComponentDescriptor::AddSampleRate (
    AAX_CFieldIndex inFieldIndex ) [pure virtual]
```

Subscribes a sample rate context field.

Defines a sample rate port for host-provided information in the algorithm's context structure.

- Data type: [AAX_CSampleRate](#) *
- Data kind: The current sample rate

Parameters

in	<i>inFieldIndex</i>	Unique identifier for the field, generated using AAX_FIELD_INDEX
----	---------------------	--

Implemented in [AAX_VComponentDescriptor](#).

14.100.3.6 AddClock()

```
virtual AAX_Result AAX_IComponentDescriptor::AddClock (
    AAX_CFieldIndex inFieldIndex ) [pure virtual]
```

Subscribes a clock context field.

Defines a clock port for host-provided information in the algorithm's context structure.

- Data type: [AAX_CTimestamp](#) *
- Data kind: A running counter which increments even when the transport is not playing. The counter increments exactly once per sample quantum.

Host Compatibility Notes As of Pro Tools 11.1, this field may be used in both Native and DSP plug-ins. The DSP clock data is a 16-bit cycling counter. This field was only available for Native plug-ins in previous Pro Tools versions.

Parameters

in	<i>inFieldIndex</i>	Unique identifier for the field, generated using AAX_FIELD_INDEX
----	---------------------	--

Implemented in [AAX_VComponentDescriptor](#).

Referenced by `AAX_CMonoIthicParameters::StaticDescribe()`.

Here is the caller graph for this function:

14.100.3.7 AddSideChainIn()

```
virtual AAX_Result AAX_IComponentDescriptor::AddSideChainIn (
    AAX_CFieldIndex inFieldIndex ) [pure virtual]
```

Subscribes a side-chain input context field.

Defines a side-chain input port for host-provided information in the algorithm's context structure.

- Data type: `int32_t*`
- Data kind: The index of the plug-in's first side-chain input channel within the array of input audio buffers

Parameters

in	<i>inFieldIndex</i>	Unique identifier for the field, generated using AAX_FIELD_INDEX
----	---------------------	--

Implemented in [AAX_VComponentDescriptor](#).

14.100.3.8 AddDataInPort()

```
virtual AAX_Result AAX_IComponentDescriptor::AddDataInPort (
    AAX_CFieldIndex inFieldIndex,
    uint32_t inPacketSize,
    AAX_EDataInPortType inPortType = AAX_eDataInPortType_Buffered ) [pure virtual]
```

Adds a custom data port to the algorithm context.

Defines a read-only data port for plug-in information in the algorithm's context structure. The plug-in can send information to this port using [AAX_IController::PostPacket\(\)](#).

The host guarantees that all packets will be delivered to this port in the order in which they were posted, up to the point of a packet buffer overflow, though some packets may be dropped depending on the `inPortType` and host implementation.

Note

When a plug-in is operating in offline (AudioSuite) mode, all data ports operate as [AAX_eDataInPortType_Unbuffered](#) ports

Parameters

in	<i>inFieldIndex</i>	Unique identifier for the port, generated using AAX_FIELD_INDEX
in	<i>inPacketSize</i>	Size of the data packets that will be sent to this port
in	<i>inPortType</i>	The requested packet delivery behavior for this port

Implemented in [AAX_VComponentDescriptor](#).

Referenced by `AAX_CMonolithicParameters::StaticDescribe()`.

Here is the caller graph for this function:

14.100.3.9 AddAuxOutputStem()

```
virtual AAX_Result AAX_IComponentDescriptor::AddAuxOutputStem (
    AAX_CFieldIndex inFieldIndex,
    int32_t inStemFormat,
    const char inNameUTF8[] ) [pure virtual]
```

Adds an auxiliary output stem for a plug-in.

Use this method to add additional output channels to the algorithm context.

The aux output stem audio buffers will be added to the end of the audio outputs array in the order in which they are described. When writing audio data to a specific aux output, find the proper starting channel by accumulating all of the channels of the main output stem format and any previously-described aux output stems.

The plug-in is responsible for providing a meaningful name for each aux outputs. At the very least, individual outputs should be labeled "Output xx", where "xx" is the aux output number as it is defined in the plug-in. The output name should also include the words "mono" and "stereo" to support when users are looking for an output with a specific stem format.

Host Compatibility Notes There is a hard limit to the number of outputs that Pro Tools supports for a single plug-in instance. This limit is currently set at 256 channels, which includes all of the plug-in's output channels in addition to the sum total of all of its aux output stem channels.

Host Compatibility Notes Pro Tools supports only mono and stereo auxiliary output stem formats

Warning

This method will return an error code on hosts which do not support auxiliary output stems. This indicates that the host will not provide audio buffers for auxiliary output stems during processing. A plug-in must not attempt to write data into auxiliary output stem buffers which have not been provided by the host!

Parameters

in	<i>inFieldIndex</i>	DEPRECATED: This parameter is no longer needed by the host, but is included in the interface for binary compatibility
in	<i>inStemFormat</i>	The stem format of the new aux output
in	<i>inNameUTF8</i>	The name of the aux output. This name is static and cannot be changed after the descriptor is submitted to the host

Implemented in [AAX_VComponentDescriptor](#).

Referenced by `AAX_CMonolithicParameters::StaticDescribe()`.

Here is the caller graph for this function:

14.100.3.10 AddPrivateData()

```
virtual AAX_Result AAX_IComponentDescriptor::AddPrivateData (
    AAX_CFieldIndex inFieldIndex,
    int32_t inDataSize,
    uint32_t inOptions = AAX_ePrivateDataOptions_DefaultOptions ) [pure virtual]
```

Adds a private data port to the algorithm context.

Defines a read/write data port for private state data. Data written to this port will be maintained by the host between calls to the algorithm context.

See also

`alg_pd_registration`

Parameters

in	<i>inFieldIndex</i>	Unique identifier for the port, generated using AAX_FIELD_INDEX
in	<i>inDataSize</i>	Size of the data packets that will be sent to this port
in	<i>inOptions</i>	Options that define the private data port's behavior

Implemented in [AAX_VComponentDescriptor](#).

Referenced by `AAX_CMonolithicParameters::StaticDescribe()`.

Here is the caller graph for this function:

14.100.3.11 AddTemporaryData()

```
virtual AAX_Result AAX_IComponentDescriptor::AddTemporaryData (
    AAX_CFieldIndex inFieldIndex,
    uint32_t inDataElementSize ) [pure virtual]
```

Adds a block of data to a context that is not saved between callbacks and is scaled by the system buffer size.

This can be very useful if you use block processing and need to store intermediate results. Just specify your base element size and the system will scale the overall block size by the buffer size. For example, to create a buffer of floats that is the length of the block, specify 4 bytes as the elementsize.

This data block does not retain state across callback and can also be reused across instances on memory constrained systems.

Parameters

in	<i>inFieldIndex</i>	Unique identifier for the port, generated using AAX_FIELD_INDEX
in	<i>inDataElementSize</i>	The size of a single piece of data in the block. This number will be multiplied by the processing block size to determine total block size.

Implemented in [AAX_VComponentDescriptor](#).

14.100.3.12 AddDmaInstance()

```
virtual AAX_Result AAX_IComponentDescriptor::AddDmaInstance (
    AAX_CFieldIndex inFieldIndex,
    AAX_IDma::EMode inDmaMode ) [pure virtual]
```

Adds a DMA field to the plug-in's context.

DMA (direct memory access) provides efficient reads from and writes to external memory on the DSP. DMA behavior is emulated in host-based plug-ins for cross-platform portability.

Note

The order in which DMA instances are added defines their priority and therefore order of execution of DMA operations. In most plug-ins, Scatter fields should be placed first in order to achieve the lowest possible access latency.

For more information, see [Direct Memory Access](#) .

Todo Update the DMA system management such that operation priority can be set arbitrarily

Parameters

in	<i>inFieldIndex</i>	Unique identifier for the field, generated using AAX_FIELD_INDEX
in	<i>inDmaMode</i>	AAX_IDma::EMode that will apply to this field

Implemented in [AAX_VComponentDescriptor](#).

14.100.3.13 AddMeters()

```
virtual AAX_Result AAX_IComponentDescriptor::AddMeters (
    AAX_CFieldIndex inFieldIndex,
    const AAX_CTypeID * inMeterIDs,
    const uint32_t inMeterCount ) [pure virtual]
```

Adds a meter field to the plug-in's context.

Meter fields include an array of meter tap values, with one tap per meter per context. Only one meter field should be added per Component. Individual meter behaviors can be described at the Effect level.

For more information, see [Plug-in meters](#) .

Parameters

in	<i>inFieldIndex</i>	Unique identifier for the field, generated using AAX_FIELD_INDEX
in	<i>inMeterIDs</i>	Array of 32-bit IDs, one for each meter. Meter IDs must be unique within the Effect.
in	<i>inMeterCount</i>	The number of meters included in this field

Implemented in [AAX_VComponentDescriptor](#).

Referenced by `AAX_CMonolithicParameters::StaticDescribe()`.

Here is the caller graph for this function:

14.100.3.14 AddMIDINode()

```
virtual AAX_Result AAX_IComponentDescriptor::AddMIDINode (
    AAX_CFieldIndex inFieldIndex,
    AAX_EMIDINodeType inNodeType,
    const char inNodeName[],
    uint32_t channelMask ) [pure virtual]
```

Adds a MIDI node field to the plug-in's context.

- Data type: [AAX_IMIDINode](#) *

The resulting MIDI node data will be available both in the algorithm context and in the plug-in's [data model](#) via [UpdateMIDINodes\(\)](#).

To add a MIDI node that is only accessible to the plug-in's data model, use [AAX_IEffectDescriptor::AddControlMIDINode\(\)](#)

Host Compatibility Notes Due to current restrictions MIDI data won't be delivered to DSP algorithms, only to AAX Native.

Parameters

in	<i>inFieldIndex</i>	The ID of the port. MIDI node ports should be formatted as a pointer to an AAX_IMIDINode .
in	<i>inNodeType</i>	The type of MIDI node, as AAX_EMIDINodeType
in	<i>inNodeName</i>	The name of the MIDI node as it should appear in the host's UI
in	<i>channelMask</i>	The channel mask for the MIDI node. This parameter specifies used MIDI channels. For Global MIDI nodes, use a mask of AAX_EMidiGlobalNodeSelectors

Implemented in [AAX_VComponentDescriptor](#).

Referenced by `AAX_CMonolithicParameters::StaticDescribe()`.

Here is the caller graph for this function:

14.100.3.15 AddReservedField()

```
virtual AAX_Result AAX_IComponentDescriptor::AddReservedField (
    AAX_CFieldIndex inFieldIndex,
    uint32_t inFieldType ) [pure virtual]
```

Subscribes a context field to host-provided services or information.

Note

Currently for internal use only.

Parameters

in	<i>inFieldIndex</i>	Unique identifier for the field, generated using AAX_FIELD_INDEX
in	<i>inFieldType</i>	Type of field that is being added

Implemented in [AAX_VComponentDescriptor](#).

14.100.3.16 NewPropertyMap()

```
virtual AAX\_IPropertyMap* AAX_IComponentDescriptor::NewPropertyMap ( ) const [pure virtual]
```

Creates a new, empty property map.

The component descriptor owns the reference to the resulting property map, and the underlying property map is destroyed when the component descriptor is released.

Implemented in [AAX_VComponentDescriptor](#).

Referenced by [AAX_CMonolithicParameters::StaticDescribe\(\)](#).

Here is the caller graph for this function:

14.100.3.17 DuplicatePropertyMap()

```
virtual AAX\_IPropertyMap* AAX_IComponentDescriptor::DuplicatePropertyMap (
    AAX\_IPropertyMap * inPropertyMap ) const [pure virtual]
```

Creates a new property map using an existing property map.

The component descriptor owns the reference to the resulting property map, and the underlying property map is destroyed when the component descriptor is released.

Parameters

in	<i>inPropertyMap</i>	The property values in this map will be copied into the new map
----	----------------------	---

Implemented in [AAX_VComponentDescriptor](#).

14.100.3.18 AddProcessProc_Native() [1/2]

```
virtual AAX\_Result AAX_IComponentDescriptor::AddProcessProc_Native (
    AAX\_CProcessProc inProcessProc,
    AAX\_IPropertyMap * inProperties = NULL,
    AAX\_CInstanceInitProc inInstanceInitProc = NULL,
```

```
AAX_CBackgroundProc inBackgroundProc = NULL,  
AAX_CSelector * outProcID = NULL ) [pure virtual]
```

Registers an algorithm processing entrypoint (process procedure) for the native architecture.

Parameters

in	<i>inProcessProc</i>	Symbol for this processing callback
in	<i>inProperties</i>	A property map for this processing callback. The property map's values are copied by the host and associated with the new ProcessProc. The property map contents are unchanged and the map may be re-used when registering additional ProcessProcs.
in	<i>inInstanceInitProc</i>	Initialization routine that will be called when a new instance of the Effect is created. See Algorithm initialization .
in	<i>inBackgroundProc</i>	Background routine that will be called in an idle context within the same address space as the associated process procedure. See Background processing callback
out	<i>outProcID</i>	

Todo document this parameter

Implemented in [AAX_VComponentDescriptor](#).

Referenced by `AddProcessProc_Native()`, and `AAX_CMonolithicParameters::StaticDescribe()`.

Here is the caller graph for this function:

14.100.3.19 AddProcessProc_TI()

```
virtual AAX_Result AAX_IComponentDescriptor::AddProcessProc_TI (
    const char inDLLFileNameUTF8[],
    const char inProcessProcSymbol[],
    AAX_IPropertyMap * inProperties,
    const char inInstanceInitProcSymbol[] = NULL,
    const char inBackgroundProcSymbol[] = NULL,
    AAX_CSelector * outProcID = NULL ) [pure virtual]
```

Registers an algorithm processing entrypoint (process procedure) for the native architecture.

Parameters

in	<i>inDLLFileNameUTF8</i>	UTF-8 encoded filename for the ELF DLL containing the algorithm code fragment
in	<i>inProcessProcSymbol</i>	Symbol for this processing callback
in	<i>inProperties</i>	A property map for this processing callback. The property map's values are copied by the host and associated with the new ProcessProc. The property map contents are unchanged and the map may be re-used when registering additional ProcessProcs.
in	<i>inInstanceInitProcSymbol</i>	Initialization routine that will be called when a new instance of the Effect is created. Must be included in the same DLL as the main algorithm entrypoint. See Algorithm initialization .
in	<i>inBackgroundProcSymbol</i>	Background routine that will be called in an idle context within the same address space as the associated process procedure. Must be included in the same DLL as the main algorithm entrypoint. See Background processing callback
out	<i>outProcID</i>	

Todo document this parameter

Implemented in [AAX_VComponentDescriptor](#).

14.100.3.20 AddProcessProc()

```
virtual AAX_Result AAX_IComponentDescriptor::AddProcessProc (
    AAX_IPropertyMap * inProperties,
    AAX_CSelector * outProcIDs = NULL,
    int32_t inProcIDsSize = 0 ) [pure virtual]
```

Registers one or more algorithm processing entrypoints (process procedures)

Any non-overlapping set of processing entrypoints may be specified. Typically this can be used to specify both Native and TI entrypoints using the same call.

The AAX Library implementation of this method includes backwards compatibility logic to complete the ProcessProc registration on hosts which do not support this method. Therefore plug-in code may use this single registration routine instead of separate calls to [AddProcessProc_Native\(\)](#), [AddProcessProc_TI\(\)](#), etc. regardless of the host version.

The following properties replace the input arguments to the platform-specific registration methods:

[AddProcessProc_Native\(\)](#) ([AAX_eProperty_PluginID_Native](#), [AAX_eProperty_PluginID_AudioSuite](#))

- [AAX_CProcessProc](#) iProcessProc: [AAX_eProperty_NativeProcessProc](#) (required)
- [AAX_CInstanceInitProc](#) iInstanceInitProc: [AAX_eProperty_NativeInstanceInitProc](#) (optional)
- [AAX_CBackgroundProc](#) iBackgroundProc: [AAX_eProperty_NativeBackgroundProc](#) (optional)

[AddProcessProc_TI\(\)](#) ([AAX_eProperty_PluginID_TI](#))

- const char inDLLFileNameUTF8[]: [AAX_eProperty_TIDLLFileName](#) (required)
- const char iProcessProcSymbol[]: [AAX_eProperty_TIProcessProc](#) (required)
- const char iInstanceInitProcSymbol[]: [AAX_eProperty_TIInstanceInitProc](#) (optional)
- const char iBackgroundProcSymbol[]: [AAX_eProperty_TIBackgroundProc](#) (optional)

If any platform-specific plug-in ID property is present in iProperties then [AddProcessProc\(\)](#) will check for the required properties for that platform.

Note

[AAX_eProperty_AudioBufferLength](#) will be ignored for the Native and AudioSuite ProcessProcs since it should only be used for AAX DSP.

Parameters

in	<i>inProperties</i>	A property map for this processing callback. The property map's values are copied by the host and associated with the new ProcessProc. The property map contents are unchanged and the map may be re-used when registering additional ProcessProcs.
out	<i>outProcIDs</i>	

Todo document this parameter Returned array will be NULL-terminated

Parameters

in	<i>inProcIDsSize</i>	The size of the array provided to oProcIDs. If oProcIDs is non-NULL but iProcIDsSize is not large enough for all of the registered ProcessProcs (plus one for NULL termination) then this method will fail with AAX_ERROR_ARGUMENT_BUFFER_OVERFLOW
----	----------------------	--

Implemented in [AAX_VComponentDescriptor](#).

14.100.3.21 AddProcessProc_Native() [2/2]

```
template<typename aContextType >
AAX_Result AAX_IComponentDescriptor::AddProcessProc_Native (
    void(AAX_CALLBACK *inProcessProc)(aContextType *const inInstancesBegin[], const
void *inInstancesEnd) ,
    AAX_IPropertyMap * inProperties = NULL,
    int32_t(AAX_CALLBACK *inInstanceInitProc)(const aContextType *inInstanceContext←
Ptr, AAX_EComponentInstanceInitAction inAction) = NULL,
    int32_t(AAX_CALLBACK *inBackgroundProc)(void) = NULL ) [inline]
```

Registers an algorithm processing entrypoint (process procedure) for the native architecture.

This template provides an [AAX_CALLBACK](#) based interface to the [AddProcessProc_Native](#) method.

See also

[AAX_IComponentDescriptor::AddProcessProc_Native\(AAX_CProcessProc,AAX_IPropertyMap*,AAX_CInstanceInitProc,AAX_](#)

Parameters

in	<i>inProperties</i>	A property map for this processing callback. The property map's values are copied by the host and associated with the new ProcessProc. The property map contents are unchanged and the map may be re-used when registering additional ProcessProcs.
----	---------------------	---

References [AddProcessProc_Native\(\)](#).

Here is the call graph for this function:

The documentation for this class was generated from the following file:

- [AAX_IComponentDescriptor.h](#)

14.101 AAX_IContainer Class Reference

```
#include <AAX_IContainer.h>
```

Inheritance diagram for AAX_IContainer:

14.101.1 Description

Abstract container interface

Public Types

- enum [EStatus](#) {
 [eStatus_Success](#) = 0 ,
 [eStatus_Overflow](#) = 1 ,
 [eStatus_NotInitialized](#) = 2 ,
 [eStatus_Unavailable](#) = 3 ,
 [eStatus_Unsupported](#) = 4 }

Public Member Functions

- virtual [~AAX_IContainer](#) ()
- virtual void [Clear](#) ()=0

14.101.2 Member Enumeration Documentation

14.101.2.1 EStatus

```
enum AAX\_IContainer::EStatus
```

Enumerator

eStatus_Success	Operation succeeded.
eStatus_Overflow	Internal buffer overflow.
eStatus_NotInitialized	Uninitialized container.
eStatus_Unavailable	An internal resource was not available.
eStatus_Unsupported	Operation is unsupported.

14.101.3 Constructor & Destructor Documentation

14.101.3.1 ~AAX_IContainer()

```
virtual AAX_IContainer::~~AAX_IContainer ( ) [inline], [virtual]
```

14.101.4 Member Function Documentation

14.101.4.1 Clear()

```
virtual void AAX_IContainer::Clear ( ) [pure virtual]
```

Clear the container

Implemented in [AAX_IPointerQueue< T >](#), [AAX_CAtomicQueue< T, S >](#), [AAX_CAtomicQueue< TNumberedParamStateList, 256 >](#) and [AAX_CAtomicQueue< const TParamValPair, 16 *kSynchronizedParameterQueueSize >](#).

The documentation for this class was generated from the following file:

- [AAX_IContainer.h](#)

14.102 AAX_IController Class Reference

```
#include <AAX_IController.h>
```

Inheritance diagram for AAX_IController:

14.102.1 Description

Interface for the AAX host's view of a single instance of an effect. Used by both clients of the AAX host and by effect components.

:Implemented by the AAX Host

Public Member Functions

- virtual [~AAX_IController](#) (void)

Host information getters

Call these methods to retrieve environment and run-time information from the AAX host.

- virtual [AAX_Result GetEffectID](#) ([AAX_IString](#) *outEffectID) const =0
CALL: Returns the current literal sample rate.
- virtual [AAX_Result GetSampleRate](#) ([AAX_CSampleRate](#) *outSampleRate) const =0
CALL: Returns the current literal sample rate.
- virtual [AAX_Result GetInputStemFormat](#) ([AAX_EStemFormat](#) *outStemFormat) const =0
CALL: Returns the plug-in's input stem format.
- virtual [AAX_Result GetOutputStemFormat](#) ([AAX_EStemFormat](#) *outStemFormat) const =0
CALL: Returns the plug-in's output stem format.
- virtual [AAX_Result GetSignalLatency](#) (int32_t *outSamples) const =0
CALL: Returns the most recent signal (algorithmic) latency that has been published by the plug-in.
- virtual [AAX_Result GetCycleCount](#) ([AAX_EProperty](#) inWhichCycleCount, [AAX_CPropertyValue](#) *out← NumCycles) const =0
CALL: returns the plug-in's current real-time DSP cycle count.
- virtual [AAX_Result GetTODLocation](#) ([AAX_CTimeOfDay](#) *outTODLocation) const =0
CALL: Returns the current Time Of Day (TOD) of the system.

Host information setters

Call these methods to set dynamic plug-in run-time information on the AAX host.

- virtual [AAX_Result SetSignalLatency](#) (int32_t inNumSamples)=0
CALL: Submits a request to change the delay compensation value that the host uses to account for the plug-in's signal (algorithmic) latency.
- virtual [AAX_Result SetCycleCount](#) ([AAX_EProperty](#) *inWhichCycleCounts, [AAX_CPropertyValue](#) *i← Values, int32_t numValues)=0
CALL: Indicates a change in the plug-in's real-time DSP cycle count.

Posting methods

Call these methods to post new plug-in information to the host's data management system.

- virtual [AAX_Result PostPacket](#) ([AAX_CFieldIndex](#) inFieldIndex, const void *inPayloadP, uint32_t in← PayloadSize)=0
CALL: Posts a data packet to the host for routing between plug-in components.

Notification methods

Call these methods to send events among plug-in components

- virtual [AAX_Result SendNotification](#) ([AAX_CTypeID](#) inNotificationType, const void *inNotificationData, uint32_t inNotificationDataSize)=0
CALL: Dispatch a notification.
- virtual [AAX_Result SendNotification](#) ([AAX_CTypeID](#) inNotificationType)=0
CALL: Sends an event to the GUI (no payload)

Metering methods

Methods to access the plug-in's host-managed metering information.

See also

Plug-in meters

- virtual [AAX_Result](#) [GetCurrentMeterValue](#) ([AAX_CTypeID](#) inMeterID, float *outMeterValue) const =0
CALL: Retrieves the current value of a host-managed plug-in meter.
- virtual [AAX_Result](#) [GetMeterPeakValue](#) ([AAX_CTypeID](#) inMeterID, float *outMeterPeakValue) const =0
CALL: Retrieves the currently held peak value of a host-managed plug-in meter.
- virtual [AAX_Result](#) [ClearMeterPeakValue](#) ([AAX_CTypeID](#) inMeterID) const =0
CALL: Clears the peak value from a host-managed plug-in meter.
- virtual [AAX_Result](#) [GetMeterCount](#) (uint32_t *outMeterCount) const =0
CALL: Retrieves the number of host-managed meters registered by a plug-in.
- virtual [AAX_Result](#) [GetMeterClipped](#) ([AAX_CTypeID](#) inMeterID, [AAX_CBoolean](#) *outClipped) const =0
CALL: Retrieves the clipped flag from a host-managed plug-in meter.
- virtual [AAX_Result](#) [ClearMeterClipped](#) ([AAX_CTypeID](#) inMeterID) const =0
CALL: Clears the clipped flag from a host-managed plug-in meter.

MIDI methods

Methods to access the plug-in's host-managed MIDI information.

- virtual [AAX_Result](#) [GetNextMIDIPacket](#) ([AAX_CFieldIndex](#) *outPort, [AAX_CMidiPacket](#) *outPacket)=0
CALL: Retrieves MIDI packets for described MIDI nodes.
- virtual [AAX_Result](#) [GetHybridSignalLatency](#) (int32_t *outSamples) const =0
CALL: Returns the latency between the algorithm normal input samples and the inputs returning from the hybrid component.
- virtual [AAX_Result](#) [GetCurrentAutomationTimestamp](#) ([AAX_CTransportCounter](#) *outTimestamp) const =0
CALL: Returns the current automation timestamp if called during the [GenerateCoefficients\(\)](#) call AND the generation of coefficients is being triggered by an automation point instead of immediate changes.
- virtual [AAX_Result](#) [GetHostName](#) ([AAX_IString](#) *outHostNameString) const =0
CALL: Returns name of the host application this plug-in instance is being loaded by. This string also typically includes version information.
- virtual [AAX_Result](#) [GetPlugInTargetPlatform](#) ([AAX_CTargetPlatform](#) *outTargetPlatform) const =0
CALL: Returns execution platform type, native or TI.
- virtual [AAX_Result](#) [GetIsAudioSuite](#) ([AAX_CBoolean](#) *outIsAudioSuite) const =0
CALL: Returns true for AudioSuite instances.
- virtual [AAX_IPageTable](#) * [CreateTableCopyForEffect](#) ([AAX_CPropertyValue](#) inManufacturerID, [AAX_CPropertyValue](#) inProductID, [AAX_CPropertyValue](#) inPlugInID, uint32_t inTableType, int32_t inTablePageSize) const =0
Copy the current page table data for a particular plug-in type.
- virtual [AAX_IPageTable](#) * [CreateTableCopyForLayout](#) (const char *inEffectID, const char *inLayoutName, uint32_t inTableType, int32_t inTablePageSize) const =0
Copy the current page table data for a particular plug-in effect and page table layout.
- virtual [AAX_IPageTable](#) * [CreateTableCopyForEffectFromFile](#) (const char *inPageTableFilePath, [AAX_ETextEncoding](#) inFilePathEncoding, [AAX_CPropertyValue](#) inManufacturerID, [AAX_CPropertyValue](#) inProductID, [AAX_CPropertyValue](#) inPlugInID, uint32_t inTableType, int32_t inTablePageSize) const =0
Copy the current page table data for a particular plug-in type.
- virtual [AAX_IPageTable](#) * [CreateTableCopyForLayoutFromFile](#) (const char *inPageTableFilePath, [AAX_ETextEncoding](#) inFilePathEncoding, const char *inLayoutName, uint32_t inTableType, int32_t inTablePageSize) const =0
Copy the current page table data for a particular plug-in effect and page table layout.

14.102.2 Constructor & Destructor Documentation

14.102.2.1 ~AAX_IController()

```
virtual AAX_IController::~AAX_IController (
    void ) [inline], [virtual]
```

14.102.3 Member Function Documentation

14.102.3.1 GetEffectID()

```
virtual AAX_Result AAX_IController::GetEffectID (
    AAX_IString * outEffectID ) const [pure virtual]
```

Implemented in [AAX_VController](#).

14.102.3.2 GetSampleRate()

```
virtual AAX_Result AAX_IController::GetSampleRate (
    AAX_CSampleRate * outSampleRate ) const [pure virtual]
```

CALL: Returns the current literal sample rate.

Parameters

out	<i>outSampleRate</i>	The current sample rate
-----	----------------------	-------------------------

Implemented in [AAX_VController](#).

14.102.3.3 GetInputStemFormat()

```
virtual AAX_Result AAX_IController::GetInputStemFormat (
    AAX_EStemFormat * outStemFormat ) const [pure virtual]
```

CALL: Returns the plug-in's input stem format.

Parameters

out	<i>outStemFormat</i>	The current input stem format
-----	----------------------	-------------------------------

Implemented in [AAX_VController](#).

14.102.3.4 GetOutputStemFormat()

```
virtual AAX_Result AAX_IController::GetOutputStemFormat (
    AAX_EStemFormat * outStemFormat ) const [pure virtual]
```

CALL: Returns the plug-in's output stem format.

Parameters

out	<i>outStemFormat</i>	The current output stem format
-----	----------------------	--------------------------------

Implemented in [AAX_VController](#).

14.102.3.5 GetSignalLatency()

```
virtual AAX_Result AAX_IController::GetSignalLatency (
    int32_t * outSamples ) const [pure virtual]
```

CALL: Returns the most recent signal (algorithmic) latency that has been published by the plug-in.

This method provides the most recently published signal latency. The host may not have updated its delay compensation to match this signal latency yet, so plug-ins that dynamically change their latency using [SetSignalLatency\(\)](#) should always wait for an [AAX_eNotificationEvent_SignalLatencyChanged](#) notification before updating its algorithm to incur this latency.

See also

[SetSignalLatency\(\)](#)

Parameters

out	<i>outSamples</i>	The number of samples of signal delay published by the plug-in
-----	-------------------	--

Implemented in [AAX_VController](#).

14.102.3.6 GetCycleCount()

```
virtual AAX_Result AAX_IController::GetCycleCount (
    AAX_EProperty inWhichCycleCount,
    AAX_CPropertyValue * outNumCycles ) const [pure virtual]
```

CALL: returns the plug-in's current real-time DSP cycle count.

This method provides the number of cycles that the AAX host expects the DSP plug-in to consume. The host uses this value when allocating DSP resources for the plug-in.

Note

A plug-in should never apply a DSP algorithm with more demanding resource requirements than what is currently accounted for by the host. To set a higher cycle count value, a plug-in must call [AAX_IController::SetCycleCount\(\)](#), then poll [AAX_IController::GetCycleCount\(\)](#) until the new value has been applied. Once the host has recognized the new cycle count value, the plug-in may apply the more demanding algorithm.

Parameters

in	<i>inWhichCycleCount</i>	Selector for the requested cycle count metric. One of: • AAX_eProperty_TI_SharedCycleCount • AAX_eProperty_TI_InstanceCycleCount • AAX_eProperty_TI_MaxInstancesPerChip
in	<i>outNumCycles</i>	The current value of the selected cycle count metric

Todo PLACEHOLDER - NOT CURRENTLY IMPLEMENTED IN HOST

Implemented in [AAX_VController](#).

14.102.3.7 GetTODLocation()

```
virtual AAX_Result AAX_IController::GetTODLocation (
    AAX_CTimeOfDay * outTODLocation ) const [pure virtual]
```

CALL: Returns the current Time Of Day (TOD) of the system.

This method provides a plug-in the TOD (in samples) of the current system. TOD is the number of samples that the playhead has traversed since the beginning of playback.

Note

The TOD value is the immediate value of the audio engine playhead. This value is incremented within the audio engine's real-time rendering context; it is not synchronized with non-real-time calls to plug-in interface methods.

Parameters

out	<i>outTODLocation</i>	The current Time Of Day as set by the host
-----	-----------------------	--

Implemented in [AAX_VController](#).

14.102.3.8 SetSignalLatency()

```
virtual AAX_Result AAX_IController::SetSignalLatency (
    int32_t inNumSamples ) [pure virtual]
```

CALL: Submits a request to change the delay compensation value that the host uses to account for the plug-in's signal (algorithmic) latency.

This method is used to request a change in the number of samples that the AAX host expects the plug-in to delay a signal.

The host is not guaranteed to immediately apply the new latency value. A plug-in should avoid incurring an actual algorithmic latency that is different than the latency accounted for by the host.

To set a new latency value, a plug-in must call [AAX_IController::SetSignalLatency\(\)](#), then wait for an [AAX_eNotificationEvent_SignalLatencyChanged](#) notification. Once this notification has been received, [AAX_IController::GetSignalLatency\(\)](#) will reflect the updated latency value and the plug-in should immediately apply any relevant algorithmic changes that alter its latency to this new value.

Warning

Parameters which affect the latency of a plug-in should not be made available for control through automation. This will result in audible glitches when delay compensation is adjusted while playing back automation for these parameters.

Parameters

<i>in</i>	<i>inNumSamples</i>	The number of samples of signal delay that the plug-in requests to incur
-----------	---------------------	--

Implemented in [AAX_VController](#).

14.102.3.9 SetCycleCount()

```
virtual AAX_Result AAX_IController::SetCycleCount (
    AAX_EProperty * inWhichCycleCounts,
    AAX_CPropertyValue * iValues,
    int32_t numValues ) [pure virtual]
```

CALL: Indicates a change in the plug-in's real-time DSP cycle count.

This method is used to request a change in the number of cycles that the AAX host expects the DSP plug-in to consume.

Note

A plug-in should never apply a DSP algorithm with more demanding resource requirements than what is currently accounted for by the host. To set a higher cycle count value, a plug-in must call [AAX_IController::SetCycleCount\(\)](#), then poll [AAX_IController::GetCycleCount\(\)](#) until the new value has been applied. Once the host has recognized the new cycle count value, the plug-in may apply the more demanding algorithm.

Parameters

in	<i>inWhichCycleCounts</i>	Array of selectors indicating the specific cycle count metrics that should be set. Each selector must be one of: • AAX_eProperty_TI_SharedCycleCount • AAX_eProperty_TI_InstanceCycleCount • AAX_eProperty_TI_MaxInstancesPerChip
in	<i>iValues</i>	An array of values requested, one for each of the selected cycle count metrics.
in	<i>numValues</i>	The size of <i>iValues</i>

Todo PLACEHOLDER - NOT CURRENTLY IMPLEMENTED IN HOST

Implemented in [AAX_VController](#).

14.102.3.10 PostPacket()

```
virtual AAX_Result AAX_IController::PostPacket (
    AAX_CFieldIndex inFieldIndex,
    const void * inPayloadP,
    uint32_t inPayloadSize ) [pure virtual]
```

CALL: Posts a data packet to the host for routing between plug-in components.

The posted packet is identified with a [AAX_CFieldIndex](#) packet index value, which is equivalent to the target data port's identifier. The packet's payload must have the expected size for the given packet index / data port, as defined when the port is created in [Describe](#). See [AAX_IComponentDescriptor::AddDataInPort\(\)](#).

Warning

Any data structures that will be passed between platforms (for example, sent to a TI DSP in an AAX DSP plug-in) must be properly data-aligned for compatibility across both platforms. See [AAX_ALIGN_FILE_ALG](#) for more information about guaranteeing cross-platform compatibility of data structures used for algorithm processing.

Note

All calls to this method should be made within the scope of [AAX_IEffectParameters::GenerateCoefficients\(\)](#). Calls from outside this method may result in packets not being delivered. See [PT-206161](#)

Parameters

in	<i>inFieldIndex</i>	The packet's destination port
in	<i>inPayloadP</i>	A pointer to the packet's payload data
in	<i>inPayloadSize</i>	The size, in bytes, of the payload data

Implemented in [AAX_VController](#).

Referenced by [AAX_CMonoIthicParameters::GenerateCoefficients\(\)](#).

Here is the caller graph for this function:

14.102.3.11 SendNotification() [1/2]

```
virtual AAX_Result AAX_IController::SendNotification (
    AAX_CTypeID inNotificationType,
    const void * inNotificationData,
    uint32_t inNotificationDataSize ) [pure virtual]
```

CALL: Dispatch a notification.

The notification is handled by the host and may be delivered back to other plug-in components such as the GUI or data model (via [AAX_IEffectGUI::NotificationReceived\(\)](#) or [AAX_IEffectParameters::NotificationReceived\(\)](#), respectively) depending on the notification type.

The host may choose to dispatch the posted notification either synchronously or asynchronously.

See the [AAX_ENotificationEvent](#) documentation for more information.

This method is supported by AAX V2 Hosts only. Check the return code on the return of this function. If the error is [AAX_ERROR_UNIMPLEMENTED](#), your plug-in is being loaded into a host that doesn't support this feature.

Parameters

in	<i>inNotificationType</i>	Type of notification to send
in	<i>inNotificationData</i>	Block of notification data
in	<i>inNotificationDataSize</i>	Size of <i>inNotificationData</i> , in bytes

Implemented in [AAX_VController](#).

14.102.3.12 SendNotification() [2/2]

```
virtual AAX_Result AAX_IController::SendNotification (
    AAX_CTypeID inNotificationType ) [pure virtual]
```

CALL: Sends an event to the GUI (no payload)

This version of the notification method is a convenience for notifications which do not take any payload data. Internally, it simply calls [AAX_IController::SendNotification\(AAX_CTypeID, const void*, uint32_t\)](#) with a null payload.

Parameters

in	<i>inNotificationType</i>	Type of notification to send
----	---------------------------	------------------------------

Implemented in [AAX_VController](#).

14.102.3.13 GetCurrentMeterValue()

```
virtual AAX_Result AAX_IController::GetCurrentMeterValue (
    AAX_CTypeID inMeterID,
    float * outMeterValue ) const [pure virtual]
```

CALL: Retrieves the current value of a host-managed plug-in meter.

Parameters

in	<i>inMeterID</i>	ID of the meter that is being queried
out	<i>outMeterValue</i>	The queried meter's current value

Implemented in [AAX_VController](#).

14.102.3.14 GetMeterPeakValue()

```
virtual AAX_Result AAX_IController::GetMeterPeakValue (
    AAX_CTypeID inMeterID,
    float * outMeterPeakValue ) const [pure virtual]
```

CALL: Retrieves the currently held peak value of a host-managed plug-in meter.

Parameters

in	<i>inMeterID</i>	ID of the meter that is being queried
out	<i>outMeterPeakValue</i>	The queried meter's currently held peak value

Implemented in [AAX_VController](#).

14.102.3.15 ClearMeterPeakValue()

```
virtual AAX_Result AAX_IController::ClearMeterPeakValue (
    AAX_CTypeID inMeterID ) const [pure virtual]
```

CALL: Clears the peak value from a host-managed plug-in meter.

Parameters

in	<i>inMeterID</i>	ID of the meter that is being cleared
----	------------------	---------------------------------------

Implemented in [AAX_VController](#).

14.102.3.16 GetMeterCount()

```
virtual AAX_Result AAX_IController::GetMeterCount (
    uint32_t * outMeterCount ) const [pure virtual]
```

CALL: Retrieves the number of host-managed meters registered by a plug-in.

See [AAX_IComponentDescriptor::AddMeters\(\)](#).

Parameters

out	<i>outMeterCount</i>	The number of registered plug-in meters.
-----	----------------------	--

Implemented in [AAX_VController](#).

14.102.3.17 GetMeterClipped()

```
virtual AAX_Result AAX_IController::GetMeterClipped (
    AAX_CTypeID inMeterID,
    AAX_CBoolean * outClipped ) const [pure virtual]
```

CALL: Retrieves the clipped flag from a host-managed plug-in meter.

See [AAX_IComponentDescriptor::AddMeters\(\)](#).

Parameters

in	<i>inMeterID</i>	ID of the meter that is being queried.
out	<i>outClipped</i>	The queried meter's clipped flag.

Implemented in [AAX_VController](#).

14.102.3.18 ClearMeterClipped()

```
virtual AAX_Result AAX_IController::ClearMeterClipped (
    AAX_CTypeID inMeterID ) const [pure virtual]
```

CALL: Clears the clipped flag from a host-managed plug-in meter.

See [AAX_IComponentDescriptor::AddMeters\(\)](#).

Parameters

in	<i>inMeterID</i>	ID of the meter that is being cleared.
----	------------------	--

Implemented in [AAX_VController](#).

14.102.3.19 GetNextMIDIPacket()

```
virtual AAX_Result AAX_IController::GetNextMIDIPacket (
    AAX_CFieldIndex * outPort,
    AAX_CMidiPacket * outPacket ) [pure virtual]
```

CALL: Retrieves MIDI packets for described MIDI nodes.

Parameters

out	<i>outPort</i>	port ID of the MIDI node that has unhandled packet
out	<i>outPacket</i>	The MIDI packet

Implemented in [AAX_VController](#).

14.102.3.20 GetCurrentAutomationTimestamp()

```
virtual AAX_Result AAX_IController::GetCurrentAutomationTimestamp (
    AAX_CTransportCounter * outTimestamp ) const [pure virtual]
```

CALL: Returns the current automation timestamp if called during the [GenerateCoefficients\(\)](#) call AND the generation of coefficients is being triggered by an automation point instead of immediate changes.

Note

This function will return 0 if called from outside of [GenerateCoefficients\(\)](#) or if the [GenerateCoefficients\(\)](#) call was initiated due to a non-automated change. In those cases, you can get your sample offset from the transport start using [GetTODLocation\(\)](#).

Parameters

out	<i>outTimestamp</i>	The current coefficient timestamp. Sample count from transport start.
-----	---------------------	---

Implemented in [AAX_VController](#).

14.102.3.21 GetHostName()

```
virtual AAX_Result AAX_IController::GetHostName (
    AAX_IString * outHostNameString ) const [pure virtual]
```

CALL: Returns name of the host application this plug-in instance is being loaded by. This string also typically includes version information.

Host Compatibility Notes Pro Tools versions from Pro Tools 11.0 to Pro Tools 12.3.1 will return a generic version string to this call. This issue is resolved beginning in Pro Tools 12.4.

Parameters

out	<i>outHostNameString</i>	The name of the current host application.
-----	--------------------------	---

Implemented in [AAX_VController](#).

14.102.3.22 GetPlugInTargetPlatform()

```
virtual AAX_Result AAX_IController::GetPlugInTargetPlatform (
    AAX_CTargetPlatform * outTargetPlatform ) const [pure virtual]
```

CALL: Returns execution platform type, native or TI.

Parameters

out	<i>outTargetPlatform</i>	The type of the current execution platform as one of AAX_ETargetPlatform .
-----	--------------------------	--

Implemented in [AAX_VController](#).

14.102.3.23 GetIsAudioSuite()

```
virtual AAX_Result AAX_IController::GetIsAudioSuite (
    AAX_CBoolean * outIsAudioSuite ) const [pure virtual]
```

CALL: Returns true for AudioSuite instances.

Parameters

out	<i>outIsAudioSuite</i>	The boolean flag which indicate true for AudioSuite instances.
-----	------------------------	--

Implemented in [AAX_VController](#).

14.102.3.24 CreateTableCopyForEffect()

```
virtual AAX_IPageTable* AAX_IController::CreateTableCopyForEffect (
    AAX_CPropertyValue inManufacturerID,
    AAX_CPropertyValue inProductID,
    AAX_CPropertyValue inPlugInID,
    uint32_t inTableType,
    int32_t inTablePageSize ) const [pure virtual]
```

Copy the current page table data for a particular plug-in type.

The host may restrict plug-ins to only copying page table data from certain plug-in types, such as plug-ins from the same manufacturer or plug-in types within the same effect.

See [Page Table Guide](#) for more information about page tables.

Returns

A new page table object to which the requested page table data has been copied. Ownership of this object passes to the caller.

a null pointer if the requested plug-in type is unknown, if `inTableType` is unknown or if `inTablePageSize` is not a supported size for the given table type.

Parameters

in	<i>inManufacturerID</i>	Manufacturer ID of the desired plug-in type
in	<i>inProductID</i>	Product ID of the desired plug-in type
in	<i>inPlugInID</i>	Type ID of the desired plug-in type (AAX_eProperty_PlugInID_Native , AAX_eProperty_PlugInID_TI)
in	<i>inTableType</i>	Four-char type identifier for the requested table type (e.g. 'PgTL', 'Av81', etc.)
in	<i>inTablePageSize</i>	Page size for the requested table. Some tables support multiple page sizes.

Implemented in [AAX_VController](#).

14.102.3.25 CreateTableCopyForLayout()

```
virtual AAX_IPageTable* AAX_IController::CreateTableCopyForLayout (
    const char * inEffectID,
    const char * inLayoutName,
    uint32_t inTableType,
    int32_t inTablePageSize ) const [pure virtual]
```

Copy the current page table data for a particular plug-in effect and page table layout.

The host may restrict plug-ins to only copying page table data from certain effects, such as effects registered within the current [AAX](#) plug-in bundle.

See [Page Table Guide](#) for more information about page tables.

Returns

A new page table object to which the requested page table data has been copied. Ownership of this object passes to the caller.

a null pointer if the requested effect ID is unknown or if `inLayoutName` is not a valid layout name for the page tables registered for the effect.

Parameters

in	<i>inEffectID</i>	Effect ID for the desired effect. See AAX_ICollection::AddEffect()
in	<i>inLayoutName</i>	Page table layout name ("name" attribute of the <code>PTLayout</code> XML tag)
in	<i>inTableType</i>	Four-char type identifier for the requested table type (e.g. 'PgTL', 'Av81', etc.)
in	<i>inTablePageSize</i>	Page size for the requested table. Some tables support multiple page sizes.

Implemented in [AAX_VController](#).

14.102.3.26 CreateTableCopyForEffectFromFile()

```
virtual AAX\_IPageTable\* AAX_IController::CreateTableCopyForEffectFromFile (
    const char * inPageTableFilePath,
    AAX\_ETextEncoding inFilePathEncoding,
    AAX\_CPropertyValue inManufacturerID,
    AAX\_CPropertyValue inProductID,
    AAX\_CPropertyValue inPlugInID,
    uint32_t inTableType,
    int32_t inTablePageSize ) const [pure virtual]
```

Copy the current page table data for a particular plug-in type.

Returns

A new page table object to which the requested page table data has been copied. Ownership of this object passes to the caller.

a null pointer if the requested plug-in type is unknown, if *inTableType* is unknown or if *inTablePageSize* is not a supported size for the given table type.

Parameters

in	<i>inPageTableFilePath</i>	Path to XML page table file.
in	<i>inFilePathEncoding</i>	File path text encoding.
in	<i>inManufacturerID</i>	Manufacturer ID of the desired plug-in type
in	<i>inProductID</i>	Product ID of the desired plug-in type
in	<i>inPlugInID</i>	Type ID of the desired plug-in type (AAX_eProperty_PlugInID_Native , AAX_eProperty_PlugInID_TI)
in	<i>inTableType</i>	Four-char type identifier for the requested table type (e.g. 'PgTL', 'Av81', etc.)
in	<i>inTablePageSize</i>	Page size for the requested table. Some tables support multiple page sizes.

Implemented in [AAX_VController](#).

14.102.3.27 CreateTableCopyForLayoutFromFile()

```
virtual AAX\_IPageTable\* AAX_IController::CreateTableCopyForLayoutFromFile (
    const char * inPageTableFilePath,
```

```
AAX_ETextEncoding inFilePathEncoding,
const char * inLayoutName,
uint32_t inTableType,
int32_t inTablePageSize ) const [pure virtual]
```

Copy the current page table data for a particular plug-in effect and page table layout.

Returns

A new page table object to which the requested page table data has been copied. Ownership of this object passes to the caller.

a null pointer if `inLayoutName` is not a valid layout name for the page tables file.

Parameters

in	<i>inPageTableFilePath</i>	Path to XML page table file.
in	<i>inFilePathEncoding</i>	File path text encoding.
in	<i>inLayoutName</i>	Page table layout name ("name" attribute of the <code>PTLayout</code> XML tag)
in	<i>inTableType</i>	Four-char type identifier for the requested table type (e.g. 'PgTL', 'Av81', etc.)
in	<i>inTablePageSize</i>	Page size for the requested table. Some tables support multiple page sizes.

Implemented in [AAX_VController](#).

The documentation for this class was generated from the following file:

- [AAX_IController.h](#)

14.103 AAX_IDataBuffer Class Reference

```
#include <AAX_IDataBuffer.h>
```

Inheritance diagram for `AAX_IDataBuffer`:

Collaboration diagram for `AAX_IDataBuffer`:

14.103.1 Description

Interface for reference counted data buffers.

This interface is intended to be used for passing arbitrary blocks of data across the binary boundary and allowing the receiver to take ownership of the allocated memory.

Public Member Functions

- [ACF_DECLARE_STANDARD_UNKNOWN](#) () ACFMETHOD(InternalQueryInterface)(const [acfiID](#) &riid
- [AAX_DELETE](#) ([AAX_IDataBuffer](#) &operator=(const [AAX_IDataBuffer](#) &))

Public Attributes

- void **ppvObjOut [AAX_OVERRIDE](#)

14.103.2 Member Function Documentation

14.103.2.1 ACF_DECLARE_STANDARD_UNKNOWN()

```
AAX_IDataBuffer::ACF_DECLARE_STANDARD_UNKNOWN ( ) const &
```

14.103.2.2 AAX_DELETE()

```
AAX_IDataBuffer::AAX_DELETE (
    AAX\_IDataBuffer & operator = (const AAX\_IDataBuffer &) )
```

14.103.3 Member Data Documentation

14.103.3.1 AAX_OVERRIDE

```
void** ppvObjOut AAX_IDataBuffer::AAX_OVERRIDE
```

Initial value:

```
{
    if (riid == IID_IAAXDataBufferV1)
    {
        *ppvObjOut = static_cast<IACFUnknown *>(this);
        ( static_cast<IACFUnknown *>(*ppvObjOut) )->AddRef();
        return ACF_OK;
    }

    return this->CACFUnknown::InternalQueryInterface(riid, ppvObjOut)
```

The documentation for this class was generated from the following file:

- [AAX_IDataBuffer.h](#)

14.104 AAX_IDataBufferWrapper Class Reference

```
#include <AAX_IDataBufferWrapper.h>
```

Inheritance diagram for AAX_IDataBufferWrapper:

14.104.1 Description

Wrapper for an [AAX_IDataBuffer](#).

Like [AAX_IController](#) and similar classes, this class provides a non-ACF interface matching an ACF interface, in this case [AAX_IACFDataBuffer](#).

The implementation of this interface will contain a reference counted pointer to the underlying ACF interface. This interface may be extended with convenience functions that are not required on the underlying ACF interface.

Public Member Functions

- virtual [~AAX_IDataBufferWrapper](#) ()=default
- virtual [AAX_Result Type](#) ([AAX_CTypeID](#) *oType) const =0
- virtual [AAX_Result Size](#) (int32_t *oSize) const =0
- virtual [AAX_Result Data](#) (void const **oBuffer) const =0

14.104.2 Constructor & Destructor Documentation

14.104.2.1 ~AAX_IDataBufferWrapper()

```
virtual AAX_IDataBufferWrapper::~AAX_IDataBufferWrapper ( ) [virtual], [default]
```

14.104.3 Member Function Documentation

14.104.3.1 Type()

```
virtual AAX_Result AAX_IDataBufferWrapper::Type (
    AAX_CTypeID * oType ) const [pure virtual]
```

The type of data contained in this buffer

This identifier must be sufficient for a client that knows the type to correctly interpret and use the data.

Implemented in [AAX_VDataBufferWrapper](#).

14.104.3.2 Size()

```
virtual AAX_Result AAX_IDataBufferWrapper::Size (
    int32_t * oSize ) const [pure virtual]
```

The number of bytes of data in this buffer

Implemented in [AAX_VDataBufferWrapper](#).

14.104.3.3 Data()

```
virtual AAX\_Result AAX_IDataBufferWrapper::Data (
    void const ** oBuffer ) const [pure virtual]
```

The buffer of data

Implemented in [AAX_VDataBufferWrapper](#).

The documentation for this class was generated from the following file:

- [AAX_IDataBufferWrapper.h](#)

14.105 AAX_IDescriptionHost Class Reference

```
#include <AAX_IDescriptionHost.h>
```

Inheritance diagram for AAX_IDescriptionHost:

14.105.1 Description

Interface to host services provided during plug-in description

Warning

AAX plugin description details are cached and might not be re-scanned even if attributes of the host description change. Be careful to avoid changing any cacheable information based on this data. See [Host feature and environment checks](#) for more information.

Public Member Functions

- virtual [~AAX_IDescriptionHost](#) ()
- virtual const [AAX_IFeatureInfo](#) * [AcquireFeatureProperties](#) (const [AAX_Feature_UID](#) &inFeatureID) const =0

14.105.2 Constructor & Destructor Documentation

14.105.2.1 ~AAX_IDescriptionHost()

```
virtual AAX_IDescriptionHost::~~AAX_IDescriptionHost ( ) [inline], [virtual]
```

14.105.3 Member Function Documentation

14.105.3.1 AcquireFeatureProperties()

```
virtual const AAX\_IFeatureInfo* AAX_IDescriptionHost::AcquireFeatureProperties (
    const AAX\_Feature\_UID & inFeatureID ) const [pure virtual]
```

Get the client's feature object for a given feature ID

Similar to `QueryInterface()` but uses a feature identifier rather than a true IID

Ownership of the returned object is passed to the caller; the caller is responsible for destroying the object, e.g. by capturing the returned object in a smart pointer.

```
// AAX_IDescriptionHost* descHost
std::unique_ptr<const AAX\_IFeatureInfo> featureInfoPtr(descHost->AcquireFeatureProperties(someFeatureUID);
```

Returns

An [AAX_IFeatureInfo](#) interface with access to the host's feature properties for this feature.

NULL if the desired feature was not found or if an error occurred

Note

May return an [AAX_IFeatureInfo](#) object with limited method support, which would return an error such as [AAX_ERROR_NULL_OBJECT](#) or [AAX_ERROR_UNIMPLEMENTED](#) to interface calls.

If no [AAX_IFeatureInfo](#) is provided then that may mean that the host is unaware of the feature, or it may mean that the host is aware of the feature but has not implemented the AAX feature support interface for this feature yet.

Parameters

in	<i>inFeatureID</i>	Identifier of the requested feature
----	--------------------	-------------------------------------

Implemented in [AAX_VDescriptionHost](#).

The documentation for this class was generated from the following file:

- [AAX_IDescriptionHost.h](#)

14.106 AAX_IDisplayDelegate< T > Class Template Reference

```
#include <AAX_IDisplayDelegate.h>
```

Inheritance diagram for `AAX_IDisplayDelegate< T >`:

Collaboration diagram for `AAX_IDisplayDelegate< T >`:

14.106.1 Description

```
template<typename T>
class AAX_IDisplayDelegate< T >
```

Classes for parameter value string conversion.

Display delegate interface template

Display delegates are used to convert real parameter values to and from their formatted string representations. All display delegates implement the [AAX_IDisplayDelegate](#) interface, which contains two conversion functions:

```
virtual bool ValueToString(T value, std::string& valueString) const = 0;
virtual bool StringToValue(const std::string& valueString, T& value) const = 0;
```

14.106.2 Display delegate decorators

The AAX SDK utilizes a decorator pattern in order to provide code re-use while accounting for a wide variety of possible parameter display formats. The SDK includes a number of sample display delegate decorator classes.

Each concrete display delegate decorator implements [AAX_IDisplayDelegateDecorator](#) and adheres to the decorator pattern. The decorator pattern allows multiple display behaviors to be composited or wrapped together at run time. For instance it is possible to implement a dBV (dB Volts) decorator, by wrapping an [AAX_CDecibelDisplayDelegateDecorator](#) with an [AAX_CUnitDisplayDelegateDecorator](#).

14.106.2.1 Display delegate decorator implementation

By implementing [AAX_IDisplayDelegateDecorator](#), each concrete display delegate decorator class implements the full [AAX_IDisplayDelegate](#) interface. In addition, it retains a pointer to the [AAX_IDisplayDelegateDecorator](#) that it wraps. When the decorator performs a conversion, it calls into its wrapped class so that the wrapped decorator may apply its own conversion formatting. By repeating this pattern in each decorator, all of the decorator subclasses call into their "wrapper" in turn, resulting in a final string to which all of the decorators' conversions have been applied in sequence.

Here is the relevant implementation from [AAX_IDisplayDelegateDecorator](#) :

```
template <typename T>
AAX_IDisplayDelegateDecorator<T>::AAX_IDisplayDelegateDecorator(const AAX_IDisplayDelegate<T>&
    displayDelegate) :
    AAX_IDisplayDelegate<T>(),
    mWrappedDisplayDelegate(displayDelegate.Clone())
{
}
template <typename T>
bool AAX_IDisplayDelegateDecorator<T>::ValueToString(T value, AAX_CString* valueString) const
{
    return mWrappedDisplayDelegate->ValueToString(value, valueString);
}
template <typename T>
bool AAX_IDisplayDelegateDecorator<T>::StringToValue(const AAX_CString& valueString, T* value) const
{
    return mWrappedDisplayDelegate->StringToValue(valueString, value);
}
```

14.106.2.2 Decibel decorator example

Here is a concrete example of how a decibel decorator might be implemented

```
template <typename T>
bool AAX_CDecibelDisplayDelegateDecorator<T>::ValueToString(T value, AAX_CString* valueString) const
{
    if (value <= 0)
    {
        *valueString = AAX_CString("--- dB");
        return true;
    }

    value = 20*log10(value);
    bool succeeded = AAX_IDisplayDelegateDecorator<T>::ValueToString(value, valueString);
    *valueString += AAX_CString("dB");
    return succeeded;
}
```

Notice in this example that the [ValueToString\(\)](#) method is called in the parent class, [AAX_IDisplayDelegateDecorator](#). This results in a call into the wrapped class' implementation of [ValueToString\(\)](#), which converts the decorated value to a redecorated string, and so forth for additional decorators.

Public Member Functions

- virtual [AAX_IDisplayDelegate](#) * [Clone](#) () const =0
Constructs and returns a copy of the display delegate.
- virtual bool [ValueToString](#) (T value, [AAX_CString](#) *valueString) const =0
Converts a real parameter value to a string representation.
- virtual bool [ValueToString](#) (T value, int32_t maxNumChars, [AAX_CString](#) *valueString) const =0
Converts a real parameter value to a string representation using a size hint, useful for control surfaces and other character limited displays.
- virtual bool [StringToValue](#) (const [AAX_CString](#) &valueString, T *value) const =0
Converts a string to a real parameter value.

14.106.3 Member Function Documentation

14.106.3.1 Clone()

```
template<typename T >
virtual AAX\_IDisplayDelegate* AAX\_IDisplayDelegate< T >::Clone ( ) const [pure virtual]
```

Constructs and returns a copy of the display delegate.

In general, this method's implementation can use a simple copy constructor:

```
template <typename T>
AAX_CSubclassDisplayDelegate<T>* AAX_CSubclassDisplayDelegate<T>::Clone() const
{
    return new AAX_CSubclassDisplayDelegate(*this);
}
```

Implemented in [AAX_IDisplayDelegateDecorator< T >](#), [AAX_CUnitPrefixDisplayDelegateDecorator< T >](#), [AAX_CUnitDisplayDelegateDecorator< T >](#), [AAX_CStringDisplayDelegate< T >](#), [AAX_CStateDisplayDelegate< T >](#), [AAX_CPercentDisplayDelegateDecorator< T >](#), [AAX_CNumberDisplayDelegate< T, Precision, SpaceAfter >](#), [AAX_CDecibelDisplayDelegateDecorator< T >](#), and [AAX_CBinaryDisplayDelegate< T >](#).

Referenced by [AAX_CParameter< T >::SetDisplayDelegate\(\)](#).

Here is the caller graph for this function:

14.106.3.2 ValueToString() [1/2]

```
template<typename T >
virtual bool AAX_IDisplayDelegate< T >::ValueToString (
    T value,
    AAX_CString * valueString ) const [pure virtual]
```

Converts a real parameter value to a string representation.

Parameters

in	<i>value</i>	The real parameter value that will be converted
out	<i>valueString</i>	A string corresponding to value

Return values

<i>true</i>	The string conversion was successful
<i>false</i>	The string conversion was unsuccessful

Implemented in [AAX_IDisplayDelegateDecorator< T >](#), [AAX_CUnitPrefixDisplayDelegateDecorator< T >](#), [AAX_CUnitDisplayDelegateDecorator< T >](#), [AAX_CStringDisplayDelegate< T >](#), [AAX_CStateDisplayDelegate< T >](#), [AAX_CPercentDisplayDelegateDecorator< T >](#), [AAX_CNumberDisplayDelegate< T, Precision, SpaceAfter >](#), [AAX_CDecibelDisplayDelegateDecorator< T >](#), and [AAX_CBinaryDisplayDelegate< T >](#).

14.106.3.3 ValueToString() [2/2]

```
template<typename T >
virtual bool AAX_IDisplayDelegate< T >::ValueToString (
    T value,
    int32_t maxNumChars,
    AAX_CString * valueString ) const [pure virtual]
```

Converts a real parameter value to a string representation using a size hint, useful for control surfaces and other character limited displays.

Parameters

in	<i>value</i>	The real parameter value that will be converted
in	<i>maxNumChars</i>	Size hint for the desired maximum number of characters in the string (not including null termination)
out	<i>valueString</i>	A string corresponding to value

Return values

<i>true</i>	The string conversion was successful
<i>false</i>	The string conversion was unsuccessful

Implemented in [AAX_IDisplayDelegateDecorator< T >](#), [AAX_CUnitPrefixDisplayDelegateDecorator< T >](#),

[AAX_CUnitDisplayDelegateDecorator< T >](#), [AAX_CStringDisplayDelegate< T >](#), [AAX_CStateDisplayDelegate< T >](#), [AAX_CPercentDisplayDelegateDecorator< T >](#), [AAX_CNumberDisplayDelegate< T, Precision, SpaceAfter >](#), [AAX_CDecibelDisplayDelegateDecorator< T >](#), and [AAX_CBinaryDisplayDelegate< T >](#).

14.106.3.4 StringToValue()

```
template<typename T >
virtual bool AAX_IDisplayDelegate< T >::StringToValue (
    const AAX_CString & valueString,
    T * value ) const [pure virtual]
```

Converts a string to a real parameter value.

Parameters

in	<i>valueString</i>	The string that will be converted
out	<i>value</i>	The real parameter value corresponding to valueString

Return values

<i>true</i>	The string conversion was successful
<i>false</i>	The string conversion was unsuccessful

Implemented in [AAX_IDisplayDelegateDecorator< T >](#), [AAX_CUnitPrefixDisplayDelegateDecorator< T >](#), [AAX_CUnitDisplayDelegateDecorator< T >](#), [AAX_CStringDisplayDelegate< T >](#), [AAX_CStateDisplayDelegate< T >](#), [AAX_CPercentDisplayDelegateDecorator< T >](#), [AAX_CNumberDisplayDelegate< T, Precision, SpaceAfter >](#), [AAX_CDecibelDisplayDelegateDecorator< T >](#), and [AAX_CBinaryDisplayDelegate< T >](#).

The documentation for this class was generated from the following file:

- [AAX_IDisplayDelegate.h](#)

14.107 AAX_IDisplayDelegateBase Class Reference

```
#include <AAX_IDisplayDelegate.h>
```

Inheritance diagram for AAX_IDisplayDelegateBase:

14.107.1 Description

Defines the display behavior for a parameter.

This interface represents a delegate class to be used in conjunction with [AAX_IParameter](#). [AAX_IParameter](#) delegates all conversion operations between strings and real parameter values to classes that meet this interface. You can think of [AAX_ITaperDelegate](#) subclasses as simple string serialization routines that enable a specific string conversions for an arbitrary parameter.

For more information about how parameter delegates operate, see the [AAX_ITaperDelegate](#) and [Parameter Manager](#) documentation.

Note

This class is *not* part of the AAX ABI and must not be passed between the plug-in and the host.

Public Member Functions

- virtual [~AAX_IDisplayDelegateBase](#) ()
Virtual destructor.

14.107.2 Constructor & Destructor Documentation

14.107.2.1 ~AAX_IDisplayDelegateBase()

```
virtual AAX_IDisplayDelegateBase::~~AAX_IDisplayDelegateBase ( ) [inline], [virtual]
```

Virtual destructor.

Note

This destructor MUST be virtual to prevent memory leaks.

The documentation for this class was generated from the following file:

- [AAX_IDisplayDelegate.h](#)

14.108 AAX_IDisplayDelegateDecorator< T > Class Template Reference

```
#include <AAX_IDisplayDelegateDecorator.h>
```

Inheritance diagram for AAX_IDisplayDelegateDecorator< T >:

Collaboration diagram for AAX_IDisplayDelegateDecorator< T >:

14.108.1 Description

```
template<typename T>
class AAX_IDisplayDelegateDecorator< T >
```

The base class for all concrete display delegate decorators.

The AAX parameter display strategy uses a decorator pattern for parameter value formatting. This approach allows developers to maximize code re-use across display delegates with many different kinds of varying formatting, all without creating interdependencies between the different display delegates themselves.

For more information, see [Display delegate decorators](#). For even more information, about the Decorator design pattern, please consult the GOF design patterns book.

Note

This class is *not* part of the AAX ABI and must not be passed between the plug-in and the host.

Public Member Functions

- [AAX_IDisplayDelegateDecorator](#) (const [AAX_IDisplayDelegate](#)< T > &displayDelegate)
Constructor.
- [AAX_IDisplayDelegateDecorator](#) (const [AAX_IDisplayDelegateDecorator](#) &other)
Copy constructor.
- [~AAX_IDisplayDelegateDecorator](#) () [AAX_OVERRIDE](#)
Virtual destructor.
- [AAX_IDisplayDelegateDecorator](#)< T > * [Clone](#) () const [AAX_OVERRIDE](#)
Constructs and returns a copy of the display delegate decorator.
- bool [ValueToString](#) (T value, [AAX_CString](#) *valueString) const [AAX_OVERRIDE](#)
Converts a string to a real parameter value.
- bool [ValueToString](#) (T value, int32_t maxNumChars, [AAX_CString](#) *valueString) const [AAX_OVERRIDE](#)
Converts a string to a real parameter value with a size constraint.
- bool [StringToValue](#) (const [AAX_CString](#) &valueString, T *value) const [AAX_OVERRIDE](#)
Converts a string to a real parameter value.

14.108.2 Constructor & Destructor Documentation

14.108.2.1 AAX_IDisplayDelegateDecorator() [1/2]

```
template<typename T >
AAX_IDisplayDelegateDecorator< T >::AAX_IDisplayDelegateDecorator (
    const AAX_IDisplayDelegate< T > & displayDelegate )
```

Constructor.

This class implements the decorator pattern, which is a sort of wrapper. The object that is being wrapped is passed into this constructor. This object is passed by reference because it must be copied to prevent any potential memory ambiguities.

This constructor sets the local mWrappedDisplayDelegate member to a clone of the provided [AAX_IDisplayDelegate](#).

Parameters

in	<i>displayDelegate</i>	The decorated display delegate.
----	------------------------	---------------------------------

14.108.2.2 AAX_IDisplayDelegateDecorator() [2/2]

```
template<typename T >
AAX_IDisplayDelegateDecorator< T >::AAX_IDisplayDelegateDecorator (
    const AAX_IDisplayDelegateDecorator< T > & other )
```

Copy constructor.

This class implements the decorator pattern, which is a sort of wrapper. The object that is being wrapped is passed into this constructor. This object is passed by reference because it must be copied to prevent any potential memory ambiguities.

This constructor sets the local mWrappedDisplayDelegate member to a clone of the provided [AAX_IDisplayDelegateDecorator](#), allowing multiply-decorated display delegates.

Parameters

in	other	The display delegate decorator that will be set as the wrapped delegate of this object
----	-------	--

14.108.2.3 ~AAX_IDisplayDelegateDecorator()

```
template<typename T >
AAX_IDisplayDelegateDecorator< T >::~~AAX_IDisplayDelegateDecorator
```

Virtual destructor.

Note

This destructor must be overridden here in order to delete the wrapped display delegate object upon decorator destruction.

14.108.3 Member Function Documentation

14.108.3.1 Clone()

```
template<typename T >
AAX_IDisplayDelegateDecorator< T > * AAX_IDisplayDelegateDecorator< T >::Clone [virtual]
```

Constructs and returns a copy of the display delegate decorator.

In general, this method's implementation can use a simple copy constructor:

```
template <typename T>
AAX_CSubclassDisplayDelegate<T>*    AAX_CSubclassDisplayDelegate<T>::Clone() const
{
    return new AAX_CSubclassDisplayDelegate(*this);
}
```

Note

This is an idiomatic method in the decorator pattern, so watch for potential problems if this method is ever changed or removed.

Implements [AAX_IDisplayDelegate< T >](#).

14.108.3.2 ValueToString() [1/2]

```
template<typename T >
bool AAX_IDisplayDelegateDecorator< T >::ValueToString (
    T value,
    AAX_CString * valueString ) const [virtual]
```

Converts a string to a real parameter value.

Override of the [AAX_IDisplayDelegate](#) implementation to call into the wrapped object. Display delegate decorators should call into this implementation to pass [ValueToString\(\)](#) calls on to the wrapped object after applying their own value-to-string decoration.

Parameters

in	<i>valueString</i>	The string that will be converted
out	<i>value</i>	The real parameter value corresponding to valueString

Return values

<i>true</i>	The string conversion was successful
<i>false</i>	The string conversion was unsuccessful

Implements [AAX_IDisplayDelegate< T >](#).

Referenced by [AAX_CDecibelDisplayDelegateDecorator< T >::ValueToString\(\)](#), [AAX_CPercentDisplayDelegateDecorator< T >::ValueToString\(\)](#), [AAX_CUnitDisplayDelegateDecorator< T >::ValueToString\(\)](#), and [AAX_CUnitPrefixDisplayDelegateDecorator< T >::ValueToString\(\)](#).

Here is the caller graph for this function:

14.108.3.3 ValueToString() [2/2]

```
template<typename T >
bool AAX_IDisplayDelegateDecorator< T >::ValueToString (
    T value,
    int32_t maxNumChars,
    AAX_CString * valueString ) const [virtual]
```

Converts a string to a real parameter value with a size constraint.

Override of the [AAX_IDisplayDelegate](#) implementation to call into the wrapped object. Display delegate decorators should call into this implementation to pass [ValueToString\(\)](#) calls on to the wrapped object after applying their own value-to-string decoration.

Parameters

in	<i>valueString</i>	The string that will be converted
in	<i>maxNumChars</i>	Size hint for the desired maximum number of characters in the string (not including null termination)
out	<i>value</i>	The real parameter value corresponding to valueString

Return values

<i>true</i>	The string conversion was successful
<i>false</i>	The string conversion was unsuccessful

Implements [AAX_IDisplayDelegate< T >](#).

14.108.3.4 StringToValue()

```
template<typename T >
bool AAX_IDisplayDelegateDecorator< T >::StringToValue (
    const AAX_CString & valueString,
    T * value ) const [virtual]
```

Converts a string to a real parameter value.

Override of the DisplayDecorator implementation to call into the wrapped object. Display delegate decorators should call into this implementation to pass [StringToValue\(\)](#) calls on to the wrapped object after applying their own string-to-value decoding.

Parameters

in	<i>valueString</i>	The string that will be converted
out	<i>value</i>	The real parameter value corresponding to valueString

Return values

<i>true</i>	The string conversion was successful
<i>false</i>	The string conversion was unsuccessful

Implements [AAX_IDisplayDelegate< T >](#).

Referenced by [AAX_CDecibelDisplayDelegateDecorator< T >::StringToValue\(\)](#), [AAX_CPercentDisplayDelegateDecorator< T >::StringToValue\(\)](#), [AAX_CUnitDisplayDelegateDecorator< T >::StringToValue\(\)](#), and [AAX_CUnitPrefixDisplayDelegateDecorator< T >::StringToValue\(\)](#).

Here is the caller graph for this function:

The documentation for this class was generated from the following file:

- [AAX_IDisplayDelegateDecorator.h](#)

14.109 AAX_IDma Class Reference

```
#include <AAX_IDma.h>
```

14.109.1 Description

Cross-platform interface for access to the host's direct memory access (DMA) facilities.

[:Implemented by the AAX Host](#)

This interface is provided via a DMA port in the plug-in's algorithm context.

See also

[AAX_IComponentDescriptor::AddDmaInstance\(\)](#)

[Direct Memory Access](#)

Public Types

- enum [EState](#) {
[eState_Error](#) = -1 ,
[eState_Init](#) = 0 ,
[eState_Running](#) = 1 ,
[eState_Complete](#) = 2 ,
[eState_Pending](#) = 3 }
- enum [EMode](#) {
[eMode_Error](#) = -1 ,
[eMode_Burst](#) = 6 ,
[eMode_Gather](#) = 10 ,
[eMode_Scatter](#) = 11 }

DMA mode IDs.

Public Member Functions

- virtual [~AAX_IDma](#) ()

Basic DMA operation

- virtual [AAX_Result AAX_DMA_API PostRequest](#) ()=0
Posts the transfer request to the DMA server.
- virtual [int32_t AAX_DMA_API IsTransferComplete](#) ()=0
Query whether a transfer has completed.
- virtual [AAX_Result AAX_DMA_API SetDmaState](#) ([EState](#) iState)=0
Sets the DMA State.
- virtual [EState AAX_DMA_API GetDmaState](#) () const =0
Inquire to find the state of the DMA instance.
- virtual [EMode AAX_DMA_API GetDmaMode](#) () const =0
Inquire to find the mode of the DMA instance.

Methods for Burst operation

Use these methods in conjunction with [AAX_IDma::eMode_Burst](#)

- virtual [AAX_Result AAX_DMA_API SetSrc](#) ([int8_t](#) *iSrc)=0
Sets the address of the source buffer.
- virtual [int8_t *AAX_DMA_API GetSrc](#) ()=0
Gets the address of the source buffer.

- virtual [AAX_Result AAX_DMA_API SetDst](#) (int8_t *iDst)=0
Sets the address of the destination buffer.
- virtual int8_t *[AAX_DMA_API GetDst](#) ()=0
Gets the address of the destination buffer.
- virtual [AAX_Result AAX_DMA_API SetBurstLength](#) (int32_t iBurstLengthBytes)=0
Sets the length of each burst.
- virtual int32_t [AAX_DMA_API GetBurstLength](#) ()=0
Gets the length of each burst.
- virtual [AAX_Result AAX_DMA_API SetNumBursts](#) (int32_t iNumBursts)=0
Sets the number of bursts to perform before giving up priority to other DMA transfers.
- virtual int32_t [AAX_DMA_API GetNumBursts](#) ()=0
Gets the number of bursts to perform before giving up priority to other DMA transfers.
- virtual [AAX_Result AAX_DMA_API SetTransferSize](#) (int32_t iTransferSizeBytes)=0
Sets the size of the whole transfer.
- virtual int32_t [AAX_DMA_API GetTransferSize](#) ()=0
Gets the size of the whole transfer, in Bytes.

Methods for Scatter and Gather operation

Use these methods in conjunction with [AAX_IDma::eMode_Scatter](#) and [AAX_IDma::eMode_Gather](#)

- virtual [AAX_Result AAX_DMA_API SetFifoBuffer](#) (int8_t *iFifoBase)=0
Sets the address of the FIFO buffer for the DMA transfer (usually the external memory block)
- virtual int8_t *[AAX_DMA_API GetFifoBuffer](#) ()=0
Gets the address of the FIFO buffer for the DMA transfer.
- virtual [AAX_Result AAX_DMA_API SetLinearBuffer](#) (int8_t *iLinearBase)=0
Sets the address of the linear buffer for the DMA transfer (usually the internal memory block)
- virtual int8_t *[AAX_DMA_API GetLinearBuffer](#) ()=0
Gets the address of the linear buffer for the DMA transfer.
- virtual [AAX_Result AAX_DMA_API SetOffsetTable](#) (const int32_t *iOffsetTable)=0
Sets the offset table for the DMA transfer.
- virtual const int32_t *[AAX_DMA_API GetOffsetTable](#) ()=0
Gets the offset table for the DMA transfer.
- virtual [AAX_Result AAX_DMA_API SetNumOffsets](#) (int32_t iNumOffsets)=0
Sets the number of offsets in the offset table.
- virtual int32_t [AAX_DMA_API GetNumOffsets](#) ()=0
Gets the number of offsets in the offset table.
- virtual [AAX_Result AAX_DMA_API SetBaseOffset](#) (int32_t iBaseOffsetBytes)=0
Sets the relative base offset into the FIFO where transfers will begin.
- virtual int32_t [AAX_DMA_API GetBaseOffset](#) ()=0
Gets the relative base offset into the FIFO where transfers will begin.
- virtual [AAX_Result AAX_DMA_API SetFifoSize](#) (int32_t iSizeBytes)=0
Sets the size of the FIFO buffer, in bytes.
- virtual int32_t [AAX_DMA_API GetFifoSize](#) ()=0
Gets the size of the FIFO buffer, in bytes.

14.109.2 Member Enumeration Documentation

14.109.2.1 EState

```
enum AAX\_IDma::EState
```

Enumerator

eState_Error	
eState_Init	
eState_Running	
eState_Complete	
eState_Pending	

14.109.2.2 EMode

enum [AAX_IDma::EMode](#)

DMA mode IDs.

These IDs are used to bind DMA context fields to a particular DMA mode when describing the fields with [AAX_IComponentDescriptor::AddDmaInstance\(\)](#)

Enumerator

eMode_Error	
eMode_Burst	Burst mode (uncommon)
eMode_Gather	Gather mode.
eMode_Scatter	Scatter mode.

14.109.3 Constructor & Destructor Documentation**14.109.3.1 ~AAX_IDma()**

```
virtual AAX_IDma::~~AAX_IDma ( ) [inline], [virtual]
```

14.109.4 Member Function Documentation**14.109.4.1 PostRequest()**

```
virtual AAX\_Result AAX\_DMA\_API AAX_IDma::PostRequest ( ) [pure virtual]
```

Posts the transfer request to the DMA server.

Note

Whichever mode this method is called on first will be the first mode to start transferring. Most plug-ins should therefore call this method for their Scatter DMA fields before their Gather DMA fields so that the scattered data is available as quickly as possible for future gathers.

Returns

AAX_SUCCESS on success

14.109.4.2 IsTransferComplete()

```
virtual int32_t AAX_DMA_API AAX_IDma::IsTransferComplete ( ) [pure virtual]
```

Query whether a transfer has completed.

A return value of false indicates an error, and that the DMA missed its cycle count deadline

Note

This function should not be used for polling within a Process loop! Instead, it can be used as a test for DMA failure. This test is usually performed via a Debug-only assert.

Todo Clarify return value meaning – ambiguity in documentation

Returns

true if all pending transfers are complete

false if pending transfers are not complete

14.109.4.3 SetDmaState()

```
virtual AAX_Result AAX_DMA_API AAX_IDma::SetDmaState (
    EState iState ) [pure virtual]
```

Sets the DMA State.

Note

This method is part of the host interface and should not be used by plug-ins

Returns

AAX_SUCCESS on success

14.109.4.4 GetDmaState()

```
virtual EState AAX_DMA_API AAX_IDma::GetDmaState ( ) const [pure virtual]
```

Inquire to find the state of the DMA instance.

14.109.4.5 GetDmaMode()

```
virtual EMode AAX_DMA_API AAX_IDma::GetDmaMode ( ) const [pure virtual]
```

Inquire to find the mode of the DMA instance.

This value does not change, so there is no setter.

14.109.4.6 SetSrc()

```
virtual AAX_Result AAX_DMA_API AAX_IDma::SetSrc (
    int8_t * iSrc ) [pure virtual]
```

Sets the address of the source buffer.

Parameters

in	<i>iSrc</i>	Address of the location in the source buffer where the read transfer should begin
----	-------------	---

Returns

AAX_SUCCESS on success

14.109.4.7 GetSrc()

```
virtual int8_t* AAX_DMA_API AAX_IDma::GetSrc ( ) [pure virtual]
```

Gets the address of the source buffer.

14.109.4.8 SetDst()

```
virtual AAX_Result AAX_DMA_API AAX_IDma::SetDst (
    int8_t * iDst ) [pure virtual]
```

Sets the address of the destination buffer.

Parameters

in	<i>iDst</i>	Address of the location in the destination buffer where the write transfer should begin
----	-------------	---

Returns

AAX_SUCCESS on success

14.109.4.9 GetDst()

```
virtual int8_t* AAX_DMA_API AAX_IDma::GetDst ( ) [pure virtual]
```

Gets the address of the destination buffer.

14.109.4.10 SetBurstLength()

```
virtual AAX_Result AAX_DMA_API AAX_IDma::SetBurstLength (
    int32_t iBurstLengthBytes ) [pure virtual]
```

Sets the length of each burst.

Note

Burst length must be between 1 and 64 Bytes, inclusive

64-Byte transfers are recommended for the fastest overall transfer speed

Returns

AAX_SUCCESS on success

14.109.4.11 GetBurstLength()

```
virtual int32_t AAX_DMA_API AAX_IDma::GetBurstLength ( ) [pure virtual]
```

Gets the length of each burst.

14.109.4.12 SetNumBursts()

```
virtual AAX_Result AAX_DMA_API AAX_IDma::SetNumBursts (
    int32_t iNumBursts ) [pure virtual]
```

Sets the number of bursts to perform before giving up priority to other DMA transfers.

Valid values are 1, 2, 4, or 16.

The full transmission may be broken up into several series of bursts, and thus the total size of the data being transferred is not bounded by the number of bursts times the burst length.

Parameters

in	<i>iNumBursts</i>	The number of bursts
----	-------------------	----------------------

Returns

AAX_SUCCESS on success

14.109.4.13 GetNumBursts()

```
virtual int32_t AAX_DMA_API AAX_IDma::GetNumBursts ( ) [pure virtual]
```

Gets the number of bursts to perform before giving up priority to other DMA transfers.

14.109.4.14 SetTransferSize()

```
virtual AAX_Result AAX_DMA_API AAX_IDma::SetTransferSize (
    int32_t iTransferSizeBytes ) [pure virtual]
```

Sets the size of the whole transfer.

Parameters

in	<i>iTransferSizeBytes</i>	The transfer size, in Bytes
----	---------------------------	-----------------------------

Returns

AAX_SUCCESS on success

14.109.4.15 GetTransferSize()

```
virtual int32_t AAX_DMA_API AAX_IDma::GetTransferSize ( ) [pure virtual]
```

Gets the size of the whole transfer, in Bytes.

14.109.4.16 SetFifoBuffer()

```
virtual AAX_Result AAX_DMA_API AAX_IDma::SetFifoBuffer (
    int8_t * iFifoBase ) [pure virtual]
```

Sets the address of the FIFO buffer for the DMA transfer (usually the external memory block)

Returns

AAX_SUCCESS on success

14.109.4.17 GetFifoBuffer()

```
virtual int8_t* AAX_DMA_API AAX_IDma::GetFifoBuffer ( ) [pure virtual]
```

Gets the address of the FIFO buffer for the DMA transfer.

14.109.4.18 SetLinearBuffer()

```
virtual AAX_Result AAX_DMA_API AAX_IDma::SetLinearBuffer (
    int8_t * iLinearBase ) [pure virtual]
```

Sets the address of the linear buffer for the DMA transfer (usually the internal memory block)

Returns

AAX_SUCCESS on success

14.109.4.19 GetLinearBuffer()

```
virtual int8_t* AAX_DMA_API AAX_IDma::GetLinearBuffer ( ) [pure virtual]
```

Gets the address of the linear buffer for the DMA transfer.

14.109.4.20 SetOffsetTable()

```
virtual AAX_Result AAX_DMA_API AAX_IDma::SetOffsetTable (
    const int32_t * iOffsetTable ) [pure virtual]
```

Sets the offset table for the DMA transfer.

The offset table provides a list of Byte-aligned memory offsets into the FIFO buffer. The transfer will be broken into a series of individual bursts, each beginning at the specified offset locations within the FIFO buffer. The size of each burst is set by [SetBurstLength\(\)](#).

See also

[AAX_IDma::SetNumOffsets\(\)](#)

[AAX_IDma::SetBaseOffset\(\)](#)

Returns

AAX_SUCCESS on success

14.109.4.21 GetOffsetTable()

```
virtual const int32_t* AAX_DMA_API AAX_IDma::GetOffsetTable ( ) [pure virtual]
```

Gets the offset table for the DMA transfer.

14.109.4.22 SetNumOffsets()

```
virtual AAX_Result AAX_DMA_API AAX_IDma::SetNumOffsets (
    int32_t iNumOffsets ) [pure virtual]
```

Sets the number of offsets in the offset table.

See also

[AAX_IDma::SetOffsetTable\(\)](#)

Returns

AAX_SUCCESS on success

14.109.4.23 GetNumOffsets()

```
virtual int32_t AAX\_DMA\_API AAX_IDma::GetNumOffsets ( ) [pure virtual]
```

Gets the number of offsets in the offset table.

14.109.4.24 SetBaseOffset()

```
virtual AAX\_Result AAX\_DMA\_API AAX_IDma::SetBaseOffset (
    int32_t iBaseOffsetBytes ) [pure virtual]
```

Sets the relative base offset into the FIFO where transfers will begin.

The base offset will be added to each value in the offset table in order to determine the starting offset within the FIFO buffer for each burst.

See also

[AAX_IDma::SetOffsetTable\(\)](#)

Returns

AAX_SUCCESS on success

14.109.4.25 GetBaseOffset()

```
virtual int32_t AAX\_DMA\_API AAX_IDma::GetBaseOffset ( ) [pure virtual]
```

Gets the relative base offset into the FIFO where transfers will begin.

14.109.4.26 SetFifoSize()

```
virtual AAX\_Result AAX\_DMA\_API AAX_IDma::SetFifoSize (
    int32_t iSizeBytes ) [pure virtual]
```

Sets the size of the FIFO buffer, in bytes.

Note

The FIFO buffer must be padded with at least enough memory to accommodate one burst, as defined by [SetBurstLength\(\)](#).

Returns

AAX_SUCCESS on success

14.109.4.27 GetFifoSize()

```
virtual int32_t AAX_DMA_API AAX_IDma::GetFifoSize ( ) [pure virtual]
```

Gets the size of the FIFO buffer, in bytes.

The documentation for this class was generated from the following file:

- [AAX_IDma.h](#)

14.110 AAX_IEffectDescriptor Class Reference

```
#include <AAX_IEffectDescriptor.h>
```

Inheritance diagram for AAX_IEffectDescriptor:

14.110.1 Description

Description interface for an effect's (plug-in type's) components.

:Implemented by the AAX Host

Each Effect represents a different "type" of plug-in. The host will present different Effects to the user as separate products, even if they are derived from the same [AAX_ICollection](#) description.

See also

[AAX_ICollection::AddEffect\(\)](#)

Public Member Functions

- virtual [~AAX_IEffectDescriptor](#) ()
- virtual [AAX_IComponentDescriptor](#) * [NewComponentDescriptor](#) ()=0
Create an instance of a component descriptor.
- virtual [AAX_Result](#) [AddComponent](#) ([AAX_IComponentDescriptor](#) *inComponentDescriptor)=0
Add a component to an instance of a component descriptor.
- virtual [AAX_Result](#) [AddName](#) (const char *inPlugInName)=0
Add a name to the Effect.
- virtual [AAX_Result](#) [AddCategory](#) (uint32_t inCategory)=0
Add a category to your plug-in. See [AAX_EPlugInCategory](#).
- virtual [AAX_Result](#) [AddCategoryBypassParameter](#) (uint32_t inCategory, [AAX_CParamID](#) inParamID)=0
Add a category to your plug-in. See [AAX_EPlugInCategory](#).
- virtual [AAX_Result](#) [AddProcPtr](#) (void *inProcPtr, [AAX_CProcPtrID](#) inProcID)=0
Add a process pointer.
- virtual [AAX_IPropertyMap](#) * [NewPropertyMap](#) ()=0
Create a new property map.
- virtual [AAX_Result](#) [SetProperties](#) ([AAX_IPropertyMap](#) *inProperties)=0
Set the properties of a new property map.
- virtual [AAX_Result](#) [AddResourceInfo](#) ([AAX_EResourceType](#) inResourceType, const char *inInfo)=0
Set resource file info.
- virtual [AAX_Result](#) [AddMeterDescription](#) ([AAX_CTypeID](#) inMeterID, const char *inMeterName, [AAX_IPropertyMap](#) *inProperties)=0
Add name and property map to meter with given ID.
- virtual [AAX_Result](#) [AddControlMIDINode](#) ([AAX_CTypeID](#) inNodeID, [AAX_EMIDINodeType](#) inNodeType, const char inNodeName[], uint32_t inChannelMask)=0
Add a control MIDI node to the plug-in data model.

14.110.2 Constructor & Destructor Documentation

14.110.2.1 ~AAX_IEffectDescriptor()

```
virtual AAX_IEffectDescriptor::~~AAX_IEffectDescriptor ( ) [inline], [virtual]
```

14.110.3 Member Function Documentation

14.110.3.1 NewComponentDescriptor()

```
virtual AAX_IComponentDescriptor* AAX_IEffectDescriptor::NewComponentDescriptor ( ) [pure virtual]
```

Create an instance of a component descriptor.

Implemented in [AAX_VEffectDescriptor](#).

Referenced by [AAX_CMonolithicParameters::StaticDescribe\(\)](#).

Here is the caller graph for this function:

14.110.3.2 AddComponent()

```
virtual AAX_Result AAX_IEffectDescriptor::AddComponent (
    AAX_IComponentDescriptor * inComponentDescriptor ) [pure virtual]
```

Add a component to an instance of a component descriptor.

Unlike with [AAX_ICollection::AddEffect\(\)](#), the [AAX_IEffectDescriptor](#) does not take ownership of the [AAX_IComponentDescriptor](#) that is passed to it in this method. The host copies out the contents of this descriptor, and thus the plug-in may re-use the same descriptor object when creating additional similar components.

Parameters

in	<i>inComponentDescriptor</i>	
----	------------------------------	--

Implemented in [AAX_VEffectDescriptor](#).

Referenced by [AAX_CMonolithicParameters::StaticDescribe\(\)](#).

Here is the caller graph for this function:

14.110.3.3 AddName()

```
virtual AAX_Result AAX_IEffectDescriptor::AddName (
    const char * inPlugInName ) [pure virtual]
```

Add a name to the Effect.

May be called multiple times to add abbreviated Effect names.

Note

Every Effect must include at least one name variant with 31 or fewer characters, plus a null terminating character

Parameters

in	<i>inPlugInName</i>	The name assigned to the plug-in.
----	---------------------	-----------------------------------

Implemented in [AAX_VEffectDescriptor](#).

14.110.3.4 AddCategory()

```
virtual AAX_Result AAX_IEffectDescriptor::AddCategory (
    uint32_t inCategory ) [pure virtual]
```

Add a category to your plug-in. See [AAX_EPlugInCategory](#).

Parameters

in	<i>inCategory</i>	One of the categories for the plug-in.
----	-------------------	--

Implemented in [AAX_VEffectDescriptor](#).

14.110.3.5 AddCategoryBypassParameter()

```
virtual AAX_Result AAX_IEffectDescriptor::AddCategoryBypassParameter (
    uint32_t inCategory,
    AAX_CParamID inParamID ) [pure virtual]
```

Add a category to your plug-in. See [AAX_EPlugInCategory](#).

Parameters

in	<i>inCategory</i>	One of the categories for the plug-in.
in	<i>inParamID</i>	The parameter ID of the parameter used to bypass the category section of the plug-in.

Implemented in [AAX_VEffectDescriptor](#).

14.110.3.6 AddProcPtr()

```
virtual AAX_Result AAX_IEffectDescriptor::AddProcPtr (
    void * inProcPtr,
    AAX_CProcPtrID inProcID ) [pure virtual]
```

Add a process pointer.

Parameters

in	<i>inProcPtr</i>	A process pointer.
in	<i>inProcID</i>	A process ID.

Implemented in [AAX_VEffectDescriptor](#).

14.110.3.7 NewPropertyMap()

```
virtual AAX_IPropertyMap* AAX_IEffectDescriptor::NewPropertyMap ( ) [pure virtual]
```

Create a new property map.

Implemented in [AAX_VEffectDescriptor](#).

14.110.3.8 SetProperties()

```
virtual AAX_Result AAX_IEffectDescriptor::SetProperties (
    AAX_IPropertyMap * inProperties ) [pure virtual]
```

Set the properties of a new property map.

Parameters

in	<i>inProperties</i>	Description
----	---------------------	-------------

Implemented in [AAX_VEffectDescriptor](#).

14.110.3.9 AddResourceInfo()

```
virtual AAX_Result AAX_IEffectDescriptor::AddResourceInfo (
```

```

    AAX_EResourceType inResourceType,
    const char * inInfo ) [pure virtual]

```

Set resource file info.

Parameters

in	<i>inResourceType</i>	See AAX_EResourceType.
in	<i>inInfo</i>	Definition varies on the resource type.

Implemented in [AAX_VEffectDescriptor](#).

14.110.3.10 AddMeterDescription()

```

virtual AAX_Result AAX_IEffectDescriptor::AddMeterDescription (
    AAX_CTypeID inMeterID,
    const char * inMeterName,
    AAX_IPropertyMap * inProperties ) [pure virtual]

```

Add name and property map to meter with given ID.

Parameters

in	<i>inMeterID</i>	The ID of the meter being described.
in	<i>inMeterName</i>	The name of the meter.
in	<i>inProperties</i>	The property map containing meter related data such as meter type, orientation, etc.

Implemented in [AAX_VEffectDescriptor](#).

14.110.3.11 AddControlMIDINode()

```

virtual AAX_Result AAX_IEffectDescriptor::AddControlMIDINode (
    AAX_CTypeID inNodeID,
    AAX_EMIDINodeType inNodeType,
    const char inNodeName[],
    uint32_t inChannelMask ) [pure virtual]

```

Add a control MIDI node to the plug-in data model.

- This MIDI node may receive note data as well as control data.
- To send MIDI data to the plug-in's algorithm, use [AAX_IComponentDescriptor::AddMIDINode\(\)](#).

See also

[AAX_IACFEffectParameters_V2::UpdateControlMIDINodes\(\)](#)

Parameters

in	<i>inNodeID</i>	The ID for the new control MIDI node.
in	<i>inNodeType</i>	The type of the node.
in	<i>inNodeName</i>	The name of the node.
in	<i>inChannelMask</i>	The bit mask for required nodes channels (up to 16) or required global events for global node.

Implemented in [AAX_VEffectDescriptor](#).

The documentation for this class was generated from the following file:

- [AAX_IEffectDescriptor.h](#)

14.111 AAX_IEffectDirectData Class Reference

```
#include <AAX_IEffectDirectData.h>
```

Inheritance diagram for AAX_IEffectDirectData:

Collaboration diagram for AAX_IEffectDirectData:

14.111.1 Description

The interface for a AAX Plug-in's direct data interface.

:Implemented by the Plug-In

This is the interface for an instance of a plug-in's direct data interface that gets exposed to the host application. A plug-in needs to inherit from this interface and override all of the virtual functions to support direct data access functionality.

Direct data access allows a plug-in to directly manipulate the data in its algorithm's private data blocks. The callback methods in this interface provide a safe context from which this kind of access may be attempted.

Note

This class always inherits from the latest version of the interface and thus requires any subclass to implement all the methods in the latest version of the interface.

See [AAX_IACFEffectDirectData](#) for further information.

Public Member Functions

- [ACF_DECLARE_STANDARD_UNKNOWN](#) () ACFMETHOD(InternalQueryInterface)(const [acfiID](#) &riid
- [AAX_DELETE](#) (AAX_IEffectDirectData &operator=(const [AAX_IEffectDirectData](#) &))

Public Attributes

- void **ppvObjOut [override](#)

14.111.2 Member Function Documentation

14.111.2.1 ACF_DECLARE_STANDARD_UNKNOWN()

```
AAX_IEffectDirectData::ACF_DECLARE_STANDARD_UNKNOWN ( ) const &
```

14.111.2.2 AAX_DELETE()

```
AAX_IEffectDirectData::AAX_DELETE (
    AAX_IEffectDirectData & operator = (const AAX_IEffectDirectData &) )
```

14.111.3 Member Data Documentation

14.111.3.1 override

```
void** ppvObjOut AAX_IEffectDirectData::override
```

The documentation for this class was generated from the following file:

- [AAX_IEffectDirectData.h](#)

14.112 AAX_IEffectGUI Class Reference

```
#include <AAX_IEffectGUI.h>
```

Inheritance diagram for AAX_IEffectGUI:

Collaboration diagram for AAX_IEffectGUI:

14.112.1 Description

The interface for a AAX Plug-in's user interface.

:Implemented by the Plug-In

This is the interface for an instance of a plug-in's GUI that gets exposed to the host application. You need to inherit from this interface and override all of the virtual functions to create a plug-in GUI.

To create the GUI for an AAX plug-in it is required that you inherit from this interface and override all of the virtual functions from [AAX_IACFEffctGUI](#). In nearly all cases you will be able to take advantage of the implementations in the AAX library's [AAX_CEffectGUI](#) class and only override the few specific methods that you want to explicitly customize.

Note

This class always inherits from the latest version of the interface and thus requires any subclass to implement all the methods in the latest version of the interface.

Public Member Functions

- [ACF_DECLARE_STANDARD_UNKNOWN](#) () ACFMETHOD(InternalQueryInterface)(const [acflID](#) &riid
- [AAX_DELETE](#) (AAX_IEffectGUI &operator=(const [AAX_IEffectGUI](#) &))

Public Attributes

- void **ppvObjOut [override](#)

14.112.2 Member Function Documentation

14.112.2.1 ACF_DECLARE_STANDARD_UNKNOWN()

```
AAX_IEffectGUI::ACF_DECLARE_STANDARD_UNKNOWN ( ) const &
```

14.112.2.2 AAX_DELETE()

```
AAX_IEffectGUI::AAX_DELETE (
    AAX\_IEffectGUI & operator = (const AAX\_IEffectGUI &) )
```

14.112.3 Member Data Documentation

14.112.3.1 override

```
void** ppvObjOut AAX_IEffectGUI::override
```

The documentation for this class was generated from the following file:

- [AAX_IEffectGUI.h](#)

14.113 AAX_IEffectParameters Class Reference

```
#include <AAX_IEffectParameters.h>
```

Inheritance diagram for AAX_IEffectParameters:

Collaboration diagram for AAX_IEffectParameters:

14.113.1 Description

The interface for an AAX Plug-in's data model.

:Implemented by the Plug-In

The interface for an instance of a plug-in's data model. A plug-in's implementation of this interface is responsible for creating the plug-in's set of parameters and for defining how the plug-in will respond when these parameters are changed via control updates or preset loads. In order for information to be routed from the plug-in's data model to its algorithm, the parameters that are created here must be registered with the host in the plug-in's [Description callback](#).

At [initialization](#), the host provides this interface with a reference to [AAX_IController](#), which provides access from the data model back to the host. This reference provides a means of querying information from the host such as stem format or sample rate, and is also responsible for communication between the data model and the plug-in's (decoupled) algorithm. See [Real-time algorithm callback](#).

You will most likely inherit your implementation of this interface from [AAX_CEffectParameters](#), a default implementation that provides basic data model functionality such as adding custom parameters, setting control values, restoring state, generating coefficients, etc., which you can override and customize as needed.

The following tags appear in the descriptions for methods of this class and its derived classes:

- **CALL**: Components in the plug-in should call this method to get / set data in the data model.

Note

- This class always inherits from the latest version of the interface and thus requires any subclass to implement all the methods in the latest version of the interface. The current version of [AAX_CEffectParameters](#) provides a convenient default implementation for all methods in the latest interface.
- Except where noted otherwise, the parameter values referenced by the methods in this interface are normalized values. See [Parameter Manager](#) for more information.

Legacy Porting Notes In the legacy plug-in SDK, these methods were found in `CProcess` and `CEffect↔Process`. For additional `CProcess` methods, see [AAX_IEffectGUI](#).

14.113.2 Related classes

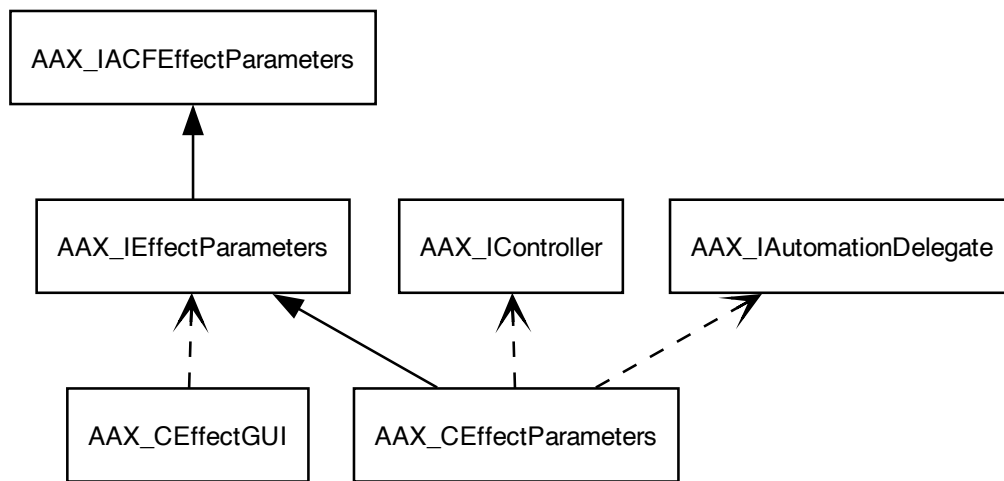


Figure 14.3 Classes related to AAX_IEffectParameters by inheritance or composition

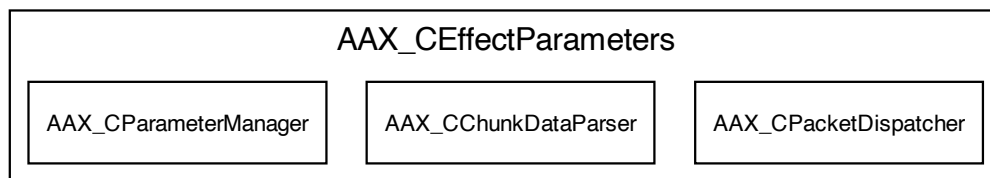


Figure 14.4 Classes owned as member objects of AAX_CEffectParameters

Public Member Functions

- [ACF_DECLARE_STANDARD_UNKNOWN](#) () ACFMETHOD(InternalQueryInterface)(const [acfiID](#) &riid
- [AAX_DELETE](#) (AAX_IEffectParameters &operator=(const [AAX_IEffectParameters](#) &))

Public Attributes

- void **ppvObjOut [override](#)

14.113.3 Member Function Documentation

14.113.3.1 ACF_DECLARE_STANDARD_UNKNOWN()

```
AAX_IEffectParameters::ACF_DECLARE_STANDARD_UNKNOWN ( ) const &
```

14.113.3.2 AAX_DELETE()

```
AAX_IEffectParameters::AAX_DELETE (
    AAX_IEffectParameters & operator = (const AAX_IEffectParameters &) )
```

14.113.4 Member Data Documentation

14.113.4.1 override

```
void** ppvObjOut AAX_IEffectParameters::override
```

The documentation for this class was generated from the following file:

- [AAX_IEffectParameters.h](#)

14.114 AAX_IFeatureInfo Class Reference

```
#include <AAX_IFeatureInfo.h>
```

Inheritance diagram for AAX_IFeatureInfo:

Public Member Functions

- virtual [~AAX_IFeatureInfo](#) ()
- virtual [AAX_Result SupportLevel](#) ([AAX_ESupportLevel](#) &oSupportLevel) const =0
- virtual const [AAX_IPropertyMap](#) * [AcquireProperties](#) () const =0
- virtual const [AAX_Feature_UID](#) & [ID](#) () const =0

14.114.1 Constructor & Destructor Documentation

14.114.1.1 ~AAX_IFeatureInfo()

```
virtual AAX_IFeatureInfo::~~AAX_IFeatureInfo ( ) [inline], [virtual]
```

14.114.2 Member Function Documentation

14.114.2.1 SupportLevel()

```
virtual AAX\_Result AAX_IFeatureInfo::SupportLevel (
    AAX\_ESupportLevel & oSupportLevel ) const [pure virtual]
```

Determine the level of support for this feature by the host

Note

The host will not provide an underlying [AAX_IACFFeatureInfo](#) interface for features which it does not recognize at all, resulting in a [AAX_ERROR_NULL_OBJECT](#) error code

Implemented in [AAX_VFeatureInfo](#).

14.114.2.2 AcquireProperties()

```
virtual const AAX\_IPropertyMap* AAX_IFeatureInfo::AcquireProperties ( ) const [pure virtual]
```

Additional properties providing details of the feature support

See the feature's UID for documentation of which features provide additional properties

Ownership of the returned object is passed to the caller; the caller is responsible for destroying the object, e.g. by capturing the returned object in a smart pointer.

```
// AAX\_IFeatureInfo* featureInfo
std::unique_ptr<const AAX\_IPropertyMap> featurePropertiesPtr(featureInfo->AcquireProperties());
```

Returns

An [AAX_IPropertyMap](#) interface with access to the host's properties for this feature.

NULL if the desired feature was not found or if an error occurred

Note

May return an [AAX_IPropertyMap](#) object with limited method support, which would return an error such as [AAX_ERROR_NULL_OBJECT](#) or [AAX_ERROR_UNIMPLEMENTED](#) to interface calls.

Implemented in [AAX_VFeatureInfo](#).

14.114.2.3 ID()

```
virtual const AAX_Feature_UID& AAX_IFeatureInfo::ID ( ) const [pure virtual]
```

Returns the ID of the feature which this object represents

Implemented in [AAX_VFeatureInfo](#).

The documentation for this class was generated from the following file:

- [AAX_IFeatureInfo.h](#)

14.115 AAX_IHostProcessor Class Reference

```
#include <AAX_IHostProcessor.h>
```

Inheritance diagram for AAX_IHostProcessor:

Collaboration diagram for AAX_IHostProcessor:

14.115.1 Description

Base class for the host processor interface.

:Implemented by the Plug-In

Note

This class always inherits from the latest version of the interface and thus requires any subclass to implement all the methods in the latest version of the interface. Most plug-ins will inherit from the [AAX_CHostProcessor](#) convenience class.

Public Member Functions

- [ACF_DECLARE_STANDARD_UNKNOWN](#) () ACFMETHOD(InternalQueryInterface)(const [acfiID](#) &riid
- [AAX_DELETE](#) (AAX_IHostProcessor &operator=(const [AAX_IHostProcessor](#) &))

Public Attributes

- void **ppvObjOut [override](#)

14.115.2 Member Function Documentation

14.115.2.1 ACF_DECLARE_STANDARD_UNKNOWN()

```
AAX_IHostProcessor::ACF_DECLARE_STANDARD_UNKNOWN ( ) const &
```

14.115.2.2 AAX_DELETE()

```
AAX_IHostProcessor::AAX_DELETE (
    AAX_IHostProcessor & operator = (const AAX_IHostProcessor &) )
```

14.115.3 Member Data Documentation

14.115.3.1 override

```
void** ppvObjOut AAX_IHostProcessor::override
```

The documentation for this class was generated from the following file:

- [AAX_IHostProcessor.h](#)

14.116 AAX_IHostProcessorDelegate Class Reference

```
#include <AAX_IHostProcessorDelegate.h>
```

Inheritance diagram for AAX_IHostProcessorDelegate:

14.116.1 Description

Versioned interface for host methods specific to offline processing.

:Implemented by the AAX Host

The host provides a host processor delegate to a plug-in's [host processor](#) object at initialization. The host processor object may make calls to this object to get information about the current render pass or to affect the plug-in's offline processing behavior.

Public Member Functions

- virtual [~AAX_IHostProcessorDelegate](#) ()
- virtual [AAX_Result GetAudio](#) (const float *const inAudioIns[], int32_t inAudioInCount, int64_t inLocation, int32_t *ioNumSamples)=0
CALL: Randomly access audio from the timeline.
- virtual int32_t [GetSideChainInputNum](#) ()=0
CALL: Returns the index of the side chain input buffer.
- virtual [AAX_Result ForceAnalyze](#) ()=0
CALL: Request an analysis pass.
- virtual [AAX_Result ForceProcess](#) ()=0
CALL: Request a process pass.

14.116.2 Constructor & Destructor Documentation

14.116.2.1 ~AAX_IHostProcessorDelegate()

```
virtual AAX_IHostProcessorDelegate::~~AAX_IHostProcessorDelegate ( ) [inline], [virtual]
```

14.116.3 Member Function Documentation

14.116.3.1 GetAudio()

```
virtual AAX_Result AAX_IHostProcessorDelegate::GetAudio (
    const float *const inAudioIns[],
    int32_t inAudioInCount,
    int64_t inLocation,
    int32_t * ioNumSamples ) [pure virtual]
```

CALL: Randomly access audio from the timeline.

Called from within [AAX_IHostProcessor::RenderAudio\(\)](#), this method fills a buffer of samples with randomly-accessed data from the current input processing region on the timeline, including any extra samples such as processing "handles".

Note

Plug-ins that use this feature must set [AAX_eProperty_UsesRandomAccess](#) to `true`

It is not possible to retrieve samples from outside of the current input processing region

Always check the return value of this method before using the randomly-accessed samples

Parameters

in	<i>inAudioIns</i>	Timeline audio buffer(s). This must be set to <code>inAudioIns</code> from AAX_IHostProcessor::RenderAudio()
in	<i>inAudioInCount</i>	Number of buffers in <code>inAudioIns</code> . This must be set to <code>inAudioInCount</code> from AAX_IHostProcessor::RenderAudio()
in	<i>inLocation</i>	A sample location relative to the beginning of the currently processed region, e.g. a value of 0 corresponds to the timeline location returned by AAX_CHostProcessor::GetSrcStart()
in, out	<i>ioNumSamples</i>	• Input: The maximum number of samples to read. • Output: The actual number of samples that were read from the timeline

Implemented in [AAX_VHostProcessorDelegate](#).

14.116.3.2 GetSideChainInputNum()

```
virtual int32_t AAX_IHostProcessorDelegate::GetSideChainInputNum ( ) [pure virtual]
```

CALL: Returns the index of the side chain input buffer.

Called from within [AAX_IHostProcessor::RenderAudio\(\)](#), this method returns the index of the side chain input sample buffer within `inAudioIns`.

Implemented in [AAX_VHostProcessorDelegate](#).

14.116.3.3 ForceAnalyze()

```
virtual AAX_Result AAX_IHostProcessorDelegate::ForceAnalyze ( ) [pure virtual]
```

CALL: Request an analysis pass.

Call this method to request an analysis pass from within the plug-in. Most plug-ins should rely on the host to trigger analysis passes when appropriate. However, plug-ins that require an analysis pass a) outside of the context of host-driven render or analysis, or b) when internal plug-in data changes may need to call [ForceAnalyze\(\)](#).

Implemented in [AAX_VHostProcessorDelegate](#).

14.116.3.4 ForceProcess()

```
virtual AAX\_Result AAX_IHostProcessorDelegate::ForceProcess ( ) [pure virtual]
```

CALL: Request a process pass.

Call this method to request a process pass from within the plug-in. If [AAX_eProperty_RequiresAnalysis](#) is defined, the resulting process pass will be preceded by an analysis pass. This method should only be used in rare circumstances by plug-ins that must launch processing outside of the normal host AudioSuite workflow.

Implemented in [AAX_VHostProcessorDelegate](#).

The documentation for this class was generated from the following file:

- [AAX_IHostProcessorDelegate.h](#)

14.117 AAX_IHostServices Class Reference

```
#include <AAX_IHostServices.h>
```

Inheritance diagram for AAX_IHostServices:

14.117.1 Description

Interface to diagnostic and debugging services provided by the AAX host.

:Implemented by the AAX Host

See also

[AAX_IACFHostServices](#)

Public Member Functions

- virtual [~AAX_IHostServices](#) ()
- virtual [AAX_Result HandleAssertFailure](#) (const char *iFile, int32_t iLine, const char *iNote, int32_t iFlags) const =0
Handle an assertion failure.
- virtual [AAX_Result Trace](#) (int32_t iPriority, const char *iMessage) const =0
Log a trace message.
- virtual [AAX_Result StackTrace](#) (int32_t iTracePriority, int32_t iStackTracePriority, const char *iMessage) const =0
Log a trace message or a stack trace.

14.117.2 Constructor & Destructor Documentation

14.117.2.1 ~AAX_IHostServices()

```
virtual AAX_IHostServices::~~AAX_IHostServices ( ) [inline], [virtual]
```

14.117.3 Member Function Documentation

14.117.3.1 HandleAssertFailure()

```
virtual AAX_Result AAX_IHostServices::HandleAssertFailure (
    const char * iFile,
    int32_t iLine,
    const char * iNote,
    int32_t iFlags ) const [pure virtual]
```

Handle an assertion failure.

Use this method to delegate assertion failure handling to the host

Use `iFlags` to request that specific behavior be included when handling the failure. This request may not be fulfilled by the host, and absence of a flag does not preclude the host from using that behavior when handling the failure.

Parameters

in	<i>iFile</i>	The name of the file containing the assert check. Usually <code>__FILE__</code>
in	<i>iLine</i>	The line number of the assert check. Usually <code>__LINE__</code>
in	<i>iNote</i>	Text to display related to the assert. Usually the condition which failed
in	<i>iFlags</i>	Bitfield of AAX_EAssertFlags to request specific handling behavior

Implemented in [AAX_VHostServices](#).

14.117.3.2 Trace()

```
virtual AAX_Result AAX_IHostServices::Trace (
    int32_t iPriority,
    const char * iMessage ) const [pure virtual]
```

Log a trace message.

Parameters

in	<i>iPriority</i>	Priority of the trace, used for log filtering. One of kAAX_Trace_Priority_Low , kAAX_Trace_Priority_Normal , kAAX_Trace_Priority_High
in	<i>iMessage</i>	Message string to log

Implemented in [AAX_VHostServices](#).

14.117.3.3 StackTrace()

```
virtual AAX_Result AAX_IHostServices::StackTrace (
    int32_t iTracePriority,
    int32_t iStackTracePriority,
    const char * iMessage ) const [pure virtual]
```

Log a trace message or a stack trace.

If the logging output filtering is set to include logs with `iStackTracePriority` then both the logging message and a stack trace will be emitted, regardless of `iTracePriority`.

If the logging output filtering is set to include logs with `iTracePriority` but to exclude logs with `iStackTracePriority` then this will emit a normal log with no stack trace.

Parameters

in	<i>iTracePriority</i>	Priority of the trace, used for log filtering. One of kAAX_Trace_Priority_Low , kAAX_Trace_Priority_Normal , kAAX_Trace_Priority_High
in	<i>iStackTracePriority</i>	Priority of the stack trace, used for log filtering. One of kAAX_Trace_Priority_Low , kAAX_Trace_Priority_Normal , kAAX_Trace_Priority_High
in	<i>iMessage</i>	Message string to log

Implemented in [AAX_VHostServices](#).

The documentation for this class was generated from the following file:

- [AAX_IHostServices.h](#)

14.118 AAX_IHostTaskAgent Class Reference

```
#include <AAX_IHostTaskAgent.h>
```

Inheritance diagram for `AAX_IHostTaskAgent`:

14.118.1 Description

Interface to access an [AAX_IACFTaskAgent](#) object implemented by the host.

The plugin may use this interface to add tasks to the host's agent, requesting that the host perform actions.

See also

[AAX_IACFTaskAgent](#)

Public Member Functions

- virtual [~AAX_IHostTaskAgent](#) ()=default
- virtual [AAX_Result Initialize](#) ([IACFUnknown](#) *iController)=0
- virtual [AAX_Result Uninitialize](#) ()=0
- virtual [AAX_Result AddTask](#) ([IACFUnknown](#) *iTask)=0
- virtual [AAX_Result CancelAllTasks](#) ()=0

14.118.2 Constructor & Destructor Documentation

14.118.2.1 ~AAX_IHostTaskAgent()

```
virtual AAX_IHostTaskAgent::~~AAX_IHostTaskAgent ( ) [virtual], [default]
```

14.118.3 Member Function Documentation

14.118.3.1 Initialize()

```
virtual AAX\_Result AAX_IHostTaskAgent::Initialize (
    IACFUnknown * iController ) [pure virtual]
```

Implemented in [AAX_VHostTaskAgent](#).

14.118.3.2 Uninitialize()

```
virtual AAX\_Result AAX_IHostTaskAgent::Uninitialize ( ) [pure virtual]
```

Implemented in [AAX_VHostTaskAgent](#).

14.118.3.3 AddTask()

```
virtual AAX\_Result AAX_IHostTaskAgent::AddTask (
    IACFUnknown * iTask ) [pure virtual]
```

Implemented in [AAX_VHostTaskAgent](#).

14.118.3.4 CancelAllTasks()

```
virtual AAX_Result AAX_IHostTaskAgent::CancelAllTasks ( ) [pure virtual]
```

Implemented in [AAX_VHostTaskAgent](#).

The documentation for this class was generated from the following file:

- [AAX_IHostTaskAgent.h](#)

14.119 AAX_IMIDIMessageInfoDelegate Class Reference

Public Member Functions

- virtual [~AAX_IMIDIMessageInfoDelegate](#) ()
- virtual uint32_t [Mask](#) () const =0
- virtual uint32_t [Length](#) () const =0
- virtual [AAX_CString ToString](#) (uint32_t inLength, const uint8_t *inData) const =0
- virtual bool [Accepts](#) (uint32_t inLength, const uint8_t *inData) const

Protected Member Functions

- bool [Accepts_ExactStatus](#) (uint32_t inLength, const uint8_t *inData) const

Static Protected Member Functions

- static void [ToString_AppendNumber](#) (const char *inLabel, int32_t inData, [AAX_CString](#) &outString)
- static void [ToString_AppendCStr](#) (const char *inLabel, const char *inCStr, [AAX_CString](#) &outString)
- static void [ToString_AppendByteRange](#) (const char *inLabel, const uint8_t *inData, int32_t inNumBytes, [AAX_CString](#) &outString)
- static void [ToString_AppendValid](#) (bool inCheck, [AAX_CString](#) &outString)

14.119.1 Constructor & Destructor Documentation

14.119.1.1 ~AAX_IMIDIMessageInfoDelegate()

```
virtual AAX_IMIDIMessageInfoDelegate::~~AAX_IMIDIMessageInfoDelegate ( ) [inline], [virtual]
```

14.119.2 Member Function Documentation

14.119.2.1 Mask()

```
virtual uint32_t AAX_IMIDIMessageInfoDelegate::Mask ( ) const [pure virtual]
```

Referenced by `Accepts_ExactStatus()`.

Here is the caller graph for this function:

14.119.2.2 Length()

```
virtual uint32_t AAX_IMIDIMessageInfoDelegate::Length ( ) const [pure virtual]
```

Referenced by `Accepts()`.

Here is the caller graph for this function:

14.119.2.3 ToString()

```
virtual AAX_CString AAX_IMIDIMessageInfoDelegate::ToString (
    uint32_t inLength,
    const uint8_t * inData ) const [pure virtual]
```

14.119.2.4 Accepts()

```
virtual bool AAX_IMIDIMessageInfoDelegate::Accepts (
    uint32_t inLength,
    const uint8_t * inData ) const [inline], [virtual]
```

References `Length()`.

Referenced by `Accepts_ExactStatus()`.

Here is the call graph for this function: Here is the caller graph for this function:

14.119.2.5 Accepts_ExactStatus()

```
bool AAX_IMIDIMessageInfoDelegate::Accepts_ExactStatus (
    uint32_t inLength,
    const uint8_t * inData ) const [inline], [protected]
```

References `Accepts()`, and `Mask()`.

Here is the call graph for this function:

14.119.2.6 ToString_AppendNumber()

```
static void AAX_IMIDIMessageInfoDelegate::ToString_AppendNumber (
    const char * inLabel,
    int32_t inData,
    AAX_CString & outString ) [inline], [static], [protected]
```

References AAX_CString::AppendNumber().

Here is the call graph for this function:

14.119.2.7 ToString_AppendCStr()

```
static void AAX_IMIDIMessageInfoDelegate::ToString_AppendCStr (
    const char * inLabel,
    const char * inCStr,
    AAX_CString & outString ) [inline], [static], [protected]
```

References AAX_CString::Append().

Referenced by ToString_AppendValid().

Here is the call graph for this function: Here is the caller graph for this function:

14.119.2.8 ToString_AppendByteRange()

```
static void AAX_IMIDIMessageInfoDelegate::ToString_AppendByteRange (
    const char * inLabel,
    const uint8_t * inData,
    int32_t inNumBytes,
    AAX_CString & outString ) [inline], [static], [protected]
```

References AAX_CString::Append(), and AAX_CString::AppendHex().

Here is the call graph for this function:

14.119.2.9 ToString_AppendValid()

```
static void AAX_IMIDIMessageInfoDelegate::ToString_AppendValid (
    bool inCheck,
    AAX_CString & outString ) [inline], [static], [protected]
```

References ToString_AppendCStr().

Here is the call graph for this function:

The documentation for this class was generated from the following file:

- [AAX_MIDILogging.cpp](#)

14.120 AAX_IMIDINode Class Reference

```
#include <AAX_IMIDINode.h>
```

14.120.1 Description

Interface for accessing information in a MIDI node.

:Implemented by the AAX Host

See also

[AAX_IComponentDescriptor::AddMIDINode](#)

Public Member Functions

- virtual [~AAX_IMIDINode](#) ()
- virtual [AAX_CMidiStream](#) * [GetNodeBuffer](#) ()=0
Returns a MIDI stream data structure.
- virtual [AAX_Result](#) [PostMIDIPacket](#) ([AAX_CMidiPacket](#) *packet)=0
Posts an [AAX_CMidiPacket](#) to an output MIDI node.
- virtual [AAX_ITransport](#) * [GetTransport](#) ()=0
Returns a transport object.

14.120.2 Constructor & Destructor Documentation

14.120.2.1 ~AAX_IMIDINode()

```
virtual AAX_IMIDINode::~~AAX_IMIDINode ( ) [inline], [virtual]
```

14.120.3 Member Function Documentation

14.120.3.1 GetNodeBuffer()

```
virtual AAX\_CMidiStream* AAX_IMIDINode::GetNodeBuffer ( ) [pure virtual]
```

Returns a MIDI stream data structure.

14.120.3.2 PostMIDIPacket()

```
virtual AAX_Result AAX_IMIDINode::PostMIDIPacket (
    AAX_CMidiPacket * packet ) [pure virtual]
```

Posts an [AAX_CMidiPacket](#) to an output MIDI node.

Host Compatibility Notes Pro Tools supports the following MIDI events from plug-ins:

- NoteOn
- NoteOff
- Pitch bend
- Polyphonic key pressure
- Bank select (controller #0)
- Program change (no bank)
- Channel pressure

Parameters

in	<i>packet</i>	The MIDI packet to be pushed to a MIDI output node
----	---------------	--

14.120.3.3 GetTransport()

```
virtual AAX_ITransport* AAX_IMIDINode::GetTransport ( ) [pure virtual]
```

Returns a transport object.

Warning

The returned interface is not versioned. Calling a method on this interface that is not supported by the host will result in undefined behavior, usually a crash. You must either check the host version before using this interface or limit the use of this interface to [V1 Transport interface](#) methods.

Wherever possible, use a versioned Transport object such as the one created in [AAX_CEffectParameters::Initialize\(\)](#) rather than this unversioned interface.

The documentation for this class was generated from the following file:

- [AAX_IMIDINode.h](#)

14.121 AAX_IPacketHandler Struct Reference

```
#include <AAX_CPacketDispatcher.h>
```

Inheritance diagram for [AAX_IPacketHandler](#):

14.121.1 Description

Callback container used by [AAX_CPacketDispatcher](#).

Public Member Functions

- virtual [~AAX_IPacketHandler](#) ()
- virtual [AAX_IPacketHandler](#) * [Clone](#) () const =0
- virtual [AAX_Result](#) [Call](#) ([AAX_CParamID](#) inParamID, [AAX_CPacket](#) &ioPacket) const =0

14.121.2 Constructor & Destructor Documentation

14.121.2.1 ~AAX_IPacketHandler()

```
virtual AAX_IPacketHandler::~~AAX_IPacketHandler ( ) [inline], [virtual]
```

14.121.3 Member Function Documentation

14.121.3.1 Clone()

```
virtual AAX\_IPacketHandler* AAX_IPacketHandler::Clone ( ) const [pure virtual]
```

Implemented in [AAX_CPacketHandler< TWorker >](#).

14.121.3.2 Call()

```
virtual AAX\_Result AAX_IPacketHandler::Call (
    AAX\_CParamID inParamID,
    AAX\_CPacket & ioPacket ) const [pure virtual]
```

Implemented in [AAX_CPacketHandler< TWorker >](#).

The documentation for this struct was generated from the following file:

- [AAX_CPacketDispatcher.h](#)

14.122 AAX_IPageTable Class Reference

```
#include <AAX_IPageTable.h>
```

Inheritance diagram for AAX_IPageTable:

14.122.1 Description

Interface to the host's representation of a plug-in instance's page table.

See also

[AAX_IEffectParameters::UpdatePageTable\(\)](#)

Public Member Functions

- virtual [~AAX_IPageTable](#) ()
Virtual destructor.
- virtual [AAX_Result Clear](#) ()=0
Clears all parameter mappings from the table.
- virtual [AAX_Result Empty](#) ([AAX_CBoolean](#) &oEmpty) const =0
Indicates whether the table is empty.
- virtual [AAX_Result GetNumPages](#) (int32_t &oNumPages) const =0
Get the number of pages currently in this table.
- virtual [AAX_Result InsertPage](#) (int32_t iPage)=0
Insert a new empty page before the page at index iPage.
- virtual [AAX_Result RemovePage](#) (int32_t iPage)=0
Remove the page at index iPage.
- virtual [AAX_Result GetNumMappedParameterIDs](#) (int32_t iPage, int32_t &oNumParameterIdentifiers) const =0
Returns the total number of parameter IDs which are mapped to a page.
- virtual [AAX_Result ClearMappedParameter](#) (int32_t iPage, int32_t iIndex)=0
Clear the parameter at a particular index in this table.
- virtual [AAX_Result GetMappedParameterID](#) (int32_t iPage, int32_t iIndex, [AAX_IString](#) &oParameterIdentifier) const =0
Get the parameter identifier which is currently mapped to an index in this table.
- virtual [AAX_Result MapParameterID](#) ([AAX_CPageTableParamID](#) iParameterIdentifier, int32_t iPage, int32_t iIndex)=0
Map a parameter to this table.
- virtual [AAX_Result GetNumParametersWithNameVariations](#) (int32_t &oNumParameterIdentifiers) const =0
- virtual [AAX_Result GetNameVariationParameterIDAtIndex](#) (int32_t iIndex, [AAX_IString](#) &oParameterIdentifier) const =0
- virtual [AAX_Result GetNumNameVariationsForParameter](#) ([AAX_CPageTableParamID](#) iParameterIdentifier, int32_t &oNumVariations) const =0
- virtual [AAX_Result GetParameterNameVariationAtIndex](#) ([AAX_CPageTableParamID](#) iParameterIdentifier, int32_t iIndex, [AAX_IString](#) &oNameVariation, int32_t &oLength) const =0
- virtual [AAX_Result GetParameterNameVariationOfLength](#) ([AAX_CPageTableParamID](#) iParameterIdentifier, int32_t iLength, [AAX_IString](#) &oNameVariation) const =0
- virtual [AAX_Result ClearParameterNameVariations](#) ()=0
- virtual [AAX_Result ClearNameVariationsForParameter](#) ([AAX_CPageTableParamID](#) iParameterIdentifier)=0
- virtual [AAX_Result SetParameterNameVariation](#) ([AAX_CPageTableParamID](#) iParameterIdentifier, const [AAX_IString](#) &iNameVariation, int32_t iLength)=0

14.122.2 Constructor & Destructor Documentation

14.122.2.1 ~AAX_IPageTable()

```
virtual AAX_IPageTable::~~AAX_IPageTable ( ) [inline], [virtual]
```

Virtual destructor.

Note

This destructor MUST be virtual to prevent memory leaks.

14.122.3 Member Function Documentation

14.122.3.1 Clear()

```
virtual AAX_Result AAX_IPageTable::Clear ( ) [pure virtual]
```

Clears all parameter mappings from the table.

This method does not clear any parameter name variations from the table. For that, use [AAX_IPageTable::ClearParameterNameVariations](#) or [AAX_IPageTable::ClearNameVariationsForParameter](#)()

Implemented in [AAX_VPageTable](#).

14.122.3.2 Empty()

```
virtual AAX_Result AAX_IPageTable::Empty (
    AAX_CBoolean & oEmpty ) const [pure virtual]
```

Indicates whether the table is empty.

A table is empty if it contains no pages. A table which contains pages but no parameter assignments is not empty. A table which has associated parameter name variations but no pages is empty.

Parameters

out	<i>oEmpty</i>	true if this table is empty
-----	---------------	-----------------------------

Implemented in [AAX_VPageTable](#).

14.122.3.3 GetNumPages()

```
virtual AAX_Result AAX_IPageTable::GetNumPages (
    int32_t & oNumPages ) const [pure virtual]
```

Get the number of pages currently in this table.

Parameters

out	<i>oNumPages</i>	The number of pages which are present in the page table. Some pages might not contain any parameter assignments.
-----	------------------	--

Implemented in [AAX_VPageTable](#).

14.122.3.4 InsertPage()

```
virtual AAX_Result AAX_IPageTable::InsertPage (
    int32_t iPage ) [pure virtual]
```

Insert a new empty page before the page at index *iPage*.

Returns

[AAX_ERROR_INVALID_ARGUMENT](#) if *iPage* is greater than the total number of pages

Parameters

in	<i>iPage</i>	The insertion point page index
----	--------------	--------------------------------

Implemented in [AAX_VPageTable](#).

14.122.3.5 RemovePage()

```
virtual AAX_Result AAX_IPageTable::RemovePage (
    int32_t iPage ) [pure virtual]
```

Remove the page at index *iPage*.

Returns

[AAX_ERROR_INVALID_ARGUMENT](#) if *iPage* is greater than the index of the last existing page

Parameters

in	<i>iPage</i>	The target page index
----	--------------	-----------------------

Implemented in [AAX_VPageTable](#).

14.122.3.6 GetNumMappedParameterIDs()

```
virtual AAX_Result AAX_IPageTable::GetNumMappedParameterIDs (
    int32_t iPage,
    int32_t & oNumParameterIdentifiers ) const [pure virtual]
```

Returns the total number of parameter IDs which are mapped to a page.

Note

The number of mapped parameter IDs does not correspond to the actual slot indices of the parameter assignments. For example, a page could have three total parameter assignments with parameters mapped to slots 2, 4, and 6.

Returns

[AAX_ERROR_INVALID_ARGUMENT](#) if *iPage* is greater than the index of the last existing page

Parameters

in	<i>iPage</i>	The target page index
out	<i>oNumParameterIdentifiers</i>	The number of parameter identifiers which are mapped to the target page

Implemented in [AAX_VPageTable](#).

14.122.3.7 ClearMappedParameter()

```
virtual AAX_Result AAX_IPageTable::ClearMappedParameter (
    int32_t iPage,
    int32_t iIndex ) [pure virtual]
```

Clear the parameter at a particular index in this table.

Returns

[AAX_SUCCESS](#) even if no parameter was mapped at the given index (the index is still clear)

Parameters

in	<i>iPage</i>	The target page index
in	<i>iIndex</i>	The target parameter slot index within the target page

Implemented in [AAX_VPageTable](#).

14.122.3.8 GetMappedParameterID()

```
virtual AAX_Result AAX_IPageTable::GetMappedParameterID (
    int32_t iPage,
    int32_t iIndex,
    AAX_IString & oParameterIdentifier ) const [pure virtual]
```

Get the parameter identifier which is currently mapped to an index in this table.

Returns

[AAX_ERROR_INVALID_ARGUMENT](#) if no parameter is mapped at the specified page/index location

Parameters

in	<i>iPage</i>	The target page index
in	<i>iIndex</i>	The target parameter slot index within the target page
out	<i>oParameterIdentifier</i>	The identifier used for the mapped parameter in the page table (may be parameter name or ID)

Implemented in [AAX_VPageTable](#).

14.122.3.9 MapParameterID()

```
virtual AAX_Result AAX_IPageTable::MapParameterID (
    AAX_CPageTableParamID iParameterIdentifier,
    int32_t iPage,
    int32_t iIndex ) [pure virtual]
```

Map a parameter to this table.

If *iParameterIdentifier* is an empty string then the parameter assignment will be cleared

Returns

[AAX_ERROR_NULL_ARGUMENT](#) if *iParameterIdentifier* is null

[AAX_ERROR_INVALID_ARGUMENT](#) if *iPage* is greater than the index of the last existing page

[AAX_ERROR_INVALID_ARGUMENT](#) if *iIndex* is negative

Parameters

in	<i>iParameterIdentifier</i>	The identifier for the parameter which will be mapped
in	<i>iPage</i>	The target page index
in	<i>iIndex</i>	The target parameter slot index within the target page

Implemented in [AAX_VPageTable](#).

14.122.3.10 GetNumParametersWithNameVariations()

```
virtual AAX_Result AAX_IPageTable::GetNumParametersWithNameVariations (
    int32_t & oNumParameterIdentifiers ) const [pure virtual]
```

Get the number of parameters with name variations defined for the current table type

Provides the number of parameters with `lt;ControlNameVariationslt;` which are explicitly defined for the current page table type.

Note

Normally parameter name variations are only used with the 'PgTL' table type

See also

- [AAX_IPageTable::GetNameVariationParameterIDAtIndex\(\)](#)

Parameters

out	<i>oNumParameterIdentifiers</i>	The number of parameters with name variations explicitly associated with the current table type.
-----	---------------------------------	--

Implemented in [AAX_VPageTable](#).

14.122.3.11 GetNameVariationParameterIDAtIndex()

```
virtual AAX_Result AAX_IPageTable::GetNameVariationParameterIDAtIndex (
    int32_t iIndex,
    AAX_IString & oParameterIdentifier ) const [pure virtual]
```

Get the identifier for a parameter with name variations defined for the current table type

Note

Normally parameter name variations are only used with the 'PgTL' table type

See also

- [AAX_IPageTable::GetNumParametersWithNameVariations\(\)](#)

Parameters

in	<i>iIndex</i>	The target parameter index within the list of parameters with explicit name variations defined for this table type.
out	<i>oParameterIdentifier</i>	The identifier used for the parameter in the page table name variations list (may be parameter name or ID)

Implemented in [AAX_VPageTable](#).

14.122.3.12 GetNumNameVariationsForParameter()

```
virtual AAX_Result AAX_IPageTable::GetNumNameVariationsForParameter (
    AAX_CPageTableParamID iParameterIdentifier,
    int32_t & oNumVariations ) const [pure virtual]
```

Get the number of name variations defined for a parameter

Provides the number of `lt;ControlNameVariationslt;` which are explicitly defined for `iParameterIdentifier` for the current page table type. No fallback logic is used to resolve this to the list of variations which would actually be used for an attached control surface if no explicit variations are defined for the current table type.

Note

Normally parameter name variations are only used with the 'PgTL' table type

See also

- [AAX_IPageTable::GetParameterNameVariationAtIndex\(\)](#)

Returns

[AAX_SUCCESS](#) and provides zero to `oNumVariations` if `iParameterIdentifier` is not found

Parameters

in	<i>iParameterIdentifier</i>	The identifier for the parameter
out	<i>oNumVariations</i>	The number of name variations which are defined for this parameter and explicitly associated with the current table type.

Implemented in [AAX_VPageTable](#).

14.122.3.13 GetParameterNameVariationAtIndex()

```
virtual AAX_Result AAX_IPageTable::GetParameterNameVariationAtIndex (
    AAX_CPageTableParamID iParameterIdentifier,
    int32_t iIndex,
    AAX_IString & oNameVariation,
    int32_t & oLength ) const [pure virtual]
```

Get a parameter name variation from the page table

Only returns `lt;ControlNameVariationslt;` which are explicitly defined for the current page table type. No fallback logic is used to resolve this to the abbreviation which would actually be shown on an attached control surface if no explicit variation is defined for the current table type.

Note

Normally parameter name variations are only used with the 'PgTL' table type

See also

- [AAX_IPageTable::GetNumNameVariationsForParameter\(\)](#)

See also

- [AAX_IPageTable::GetParameterNameVariationOfLength\(\)](#)

Returns

[AAX_ERROR_NO_ABBREVIATED_PARAMETER_NAME](#) if no suitable variation is defined for this table

[AAX_ERROR_ARGUMENT_OUT_OF_RANGE](#) if `iIndex` is out of range

Parameters

in	<i>iParameterIdentifier</i>	The identifier for the parameter
in	<i>iIndex</i>	Index of the name variation
out	<i>oNameVariation</i>	The name variation, if one is explicitly defined for this table type
out	<i>oLength</i>	The length value for this name variation. This corresponds to the variation's <code>sz</code> attribute in the page table XML and may be different from the string length of <code>iNameVariation</code> .

Implemented in [AAX_VPageTable](#).

14.122.3.14 GetParameterNameVariationOfLength()

```
virtual AAX_Result AAX_IPageTable::GetParameterNameVariationOfLength (
    AAX_CPageTableParamID iParameterIdentifier,
    int32_t iLength,
    AAX_IString & oNameVariation ) const [pure virtual]
```

Get a parameter name variation of a particular length from the page table

Only returns `lt;ControlNameVariationslt;` which are explicitly defined of `iLength` for the current page table type. No fallback logic is used to resolve this to the abbreviation which would actually be shown on an attached control surface if no explicit variation is defined for the specified length or current table type.

Note

Normally parameter name variations are only used with the 'PgTL' table type

See also

- [AAX_IPageTable::GetParameterNameVariationAtIndex\(\)](#)

Returns

[AAX_ERROR_NO_ABBREVIATED_PARAMETER_NAME](#) if no suitable variation is defined for this table

Parameters

in	<i>iParameterIdentifier</i>	The identifier for the parameter
in	<i>iLength</i>	The variation length to check, i.e. the <i>sz</i> attribute for the name variation in the page table XML
out	<i>oNameVariation</i>	The name variation, if one is explicitly defined for this table type and <i>iLength</i>

Implemented in [AAX_VPageTable](#).

14.122.3.15 ClearParameterNameVariations()

```
virtual AAX_Result AAX_IPageTable::ClearParameterNameVariations ( ) [pure virtual]
```

Clears all name variations for the current page table type

Note

Normally parameter name variations are only used with the 'PgTL' table type

See also

[AAX_IPageTable::Clear\(\)](#)

[AAX_IPageTable::ClearNameVariationsForParameter\(\)](#)

Implemented in [AAX_VPageTable](#).

14.122.3.16 ClearNameVariationsForParameter()

```
virtual AAX_Result AAX_IPageTable::ClearNameVariationsForParameter (
    AAX_CPageTableParamID iParameterIdentifier ) [pure virtual]
```

Clears all name variations for a single parameter for the current page table type

Warning

This will invalidate the list of parameter name variations indices, i.e. the parameter identifier associated with each index by [AAX_IPageTable::GetNameVariationParameterIDAtIndex\(\)](#)

Note

Normally parameter name variations are only used with the 'PgTL' table type

See also

[AAX_IPageTable::Clear\(\)](#)

[AAX_IPageTable::ClearParameterNameVariations\(\)](#)

Returns

[AAX_SUCCESS](#) and provides zero to *oNumVariations* if *iParameterIdentifier* is not found

Parameters

in	<i>iParameterIdentifier</i>	The identifier for the parameter
----	-----------------------------	----------------------------------

Implemented in [AAX_VPageTable](#).

14.122.3.17 SetParameterNameVariation()

```
virtual AAX_Result AAX_IPageTable::SetParameterNameVariation (
    AAX_CPageTableParamID iParameterIdentifier,
    const AAX_IString & iNameVariation,
    int32_t iLength ) [pure virtual]
```

Sets a name variation explicitly for the current page table type

This will add a new name variation or overwrite the existing name variation with the same length which is defined for the current table type.

Warning

If no name variation previously existed for this parameter then this will invalidate the list of parameter name variations indices, i.e. the parameter identifier associated with each index by [AAX_IPageTable::GetNameVariationParameterIDat](#)

Note

Normally parameter name variations are only used with the 'PgTL' table type

Returns

AAX_ERROR_INVALID_ARGUMENT if *iNameVariation* is empty or if *iLength* is less than zero

Parameters

in	<i>iParameterIdentifier</i>	The identifier for the parameter
in	<i>iNameVariation</i>	The new parameter name variation
in	<i>iLength</i>	The length value for this name variation. This corresponds to the variation's <i>sz</i> attribute in the page table XML and is not required to match the length of <i>iNameVariation</i> .

Implemented in [AAX_VPageTable](#).

The documentation for this class was generated from the following file:

- [AAX_IPageTable.h](#)

14.123 AAX_IParameter Class Reference

```
#include <AAX_IParameter.h>
```

Inheritance diagram for AAX_IParameter:

14.123.1 Description

The base interface for all normalizable plug-in parameters.

:Internal to the AAX SDK

This class is an outside interface for an arbitrarily typed parameter. The subclasses of this generic interface hold the parameter's state and conversion functionality.

Note

This class is *not* part of the AAX ABI and must not be passed between the plug-in and the host. Version checking is recommended when passing references to this interface between plug-in modules (e.g. between the data model and the GUI)

Public Member Functions

- virtual [~AAX_IParameter](#) ()
Virtual destructor.
- virtual [AAX_IParameterValue](#) * [CloneValue](#) () const =0
Clone the parameter's value to a new [AAX_IParameterValue](#) object.

Identification methods

- virtual [AAX_CParamID](#) [Identifier](#) () const =0
Returns the parameter's unique identifier.
- virtual void [SetName](#) (const [AAX_CString](#) &name)=0
Sets the parameter's display name.
- virtual const [AAX_CString](#) & [Name](#) () const =0
Returns the parameter's display name.
- virtual void [AddShortenedName](#) (const [AAX_CString](#) &name)=0
Sets the parameter's shortened display name.
- virtual const [AAX_CString](#) & [ShortenedName](#) (int32_t iNumCharacters) const =0
Returns the parameter's shortened display name.
- virtual void [ClearShortenedNames](#) ()=0
Clears the internal list of shortened display names.

Automation methods

- virtual bool [Automatable](#) () const =0
Returns true if the parameter is automatable, false if it is not.
- virtual void [SetAutomationDelegate](#) ([AAX_IAutomationDelegate](#) *iAutomationDelegate)=0
Sets the automation delegate (if one is required)
- virtual void [Touch](#) ()=0
Signals the automation system that a control has been touched.

- virtual void [Release](#) ()=0
Signals the automation system that a control has been released.

Taper methods

- virtual void [SetNormalizedValue](#) (double newNormalizedValue)=0
Sets a parameter value using it's normalized representation.
- virtual double [GetNormalizedValue](#) () const =0
Returns the normalized representation of the parameter's current real value.
- virtual void [SetNormalizedDefaultValue](#) (double normalizedDefault)=0
Sets the parameter's default value using its normalized representation.
- virtual double [GetNormalizedDefaultValue](#) () const =0
Returns the normalized representation of the parameter's real default value.
- virtual void [SetToDefaultValue](#) ()=0
Restores the state of this parameter to its default value.
- virtual void [SetNumberOfSteps](#) (uint32_t numSteps)=0
Sets the number of discrete steps for this parameter.
- virtual uint32_t [GetNumberOfSteps](#) () const =0
Returns the number of discrete steps used by the parameter.
- virtual uint32_t [GetStepValue](#) () const =0
Returns the current step for the current value of the parameter.
- virtual double [GetNormalizedValueFromStep](#) (uint32_t iStep) const =0
Returns the normalized value for a given step.
- virtual uint32_t [GetStepValueFromNormalizedValue](#) (double normalizedValue) const =0
Returns the step value for a normalized value of the parameter.
- virtual void [SetStepValue](#) (uint32_t iStep)=0
Returns the current step for the current value of the parameter.

Display methods

This functionality is most often used by GUIs, but can also be useful for state serialization.

- virtual bool [GetValueString](#) (AAX_CString *valueString) const =0
Serializes the parameter value into a string.
- virtual bool [GetValueString](#) (int32_t iMaxNumChars, AAX_CString *valueString) const =0
Serializes the parameter value into a string, size hint included.
- virtual bool [GetNormalizedValueFromBool](#) (bool value, double *normalizedValue) const =0
Converts a bool to a normalized parameter value.
- virtual bool [GetNormalizedValueFromInt32](#) (int32_t value, double *normalizedValue) const =0
Converts an integer to a normalized parameter value.
- virtual bool [GetNormalizedValueFromFloat](#) (float value, double *normalizedValue) const =0
Converts a float to a normalized parameter value.
- virtual bool [GetNormalizedValueFromDouble](#) (double value, double *normalizedValue) const =0
Converts a double to a normalized parameter value.
- virtual bool [GetNormalizedValueFromString](#) (const AAX_CString &valueString, double *normalizedValue) const =0
Converts a given string to a normalized parameter value.
- virtual bool [GetBoolFromNormalizedValue](#) (double normalizedValue, bool *value) const =0
Converts a normalized parameter value to a bool representing the corresponding real value.
- virtual bool [GetInt32FromNormalizedValue](#) (double normalizedValue, int32_t *value) const =0
Converts a normalized parameter value to an integer representing the corresponding real value.
- virtual bool [GetFloatFromNormalizedValue](#) (double normalizedValue, float *value) const =0
Converts a normalized parameter value to a float representing the corresponding real value.
- virtual bool [GetDoubleFromNormalizedValue](#) (double normalizedValue, double *value) const =0
Converts a normalized parameter value to a double representing the corresponding real value.
- virtual bool [GetStringFromNormalizedValue](#) (double normalizedValue, AAX_CString &valueString) const =0
Converts a normalized parameter value to a string representing the corresponding real value.

- virtual bool [GetStringFromNormalizedValue](#) (double normalizedValue, int32_t iMaxNumChars, [AAX_CString](#) &valueString) const =0
Converts a normalized parameter value to a string representing the corresponding real, size hint included. value.
- virtual bool [SetValueFromString](#) (const [AAX_CString](#) &newValueString)=0
Converts a string to a real parameter value and sets the parameter to this value.

Typed accessors

- virtual bool [GetValueAsBool](#) (bool *value) const =0
Retrieves the parameter's value as a bool.
- virtual bool [GetValueAsInt32](#) (int32_t *value) const =0
Retrieves the parameter's value as an int32_t.
- virtual bool [GetValueAsFloat](#) (float *value) const =0
Retrieves the parameter's value as a float.
- virtual bool [GetValueAsDouble](#) (double *value) const =0
Retrieves the parameter's value as a double.
- virtual bool [GetValueAsString](#) ([AAX_IString](#) *value) const =0
Retrieves the parameter's value as a string.
- virtual bool [SetValueWithBool](#) (bool value)=0
Sets the parameter's value as a bool.
- virtual bool [SetValueWithInt32](#) (int32_t value)=0
Sets the parameter's value as an int32_t.
- virtual bool [SetValueWithFloat](#) (float value)=0
Sets the parameter's value as a float.
- virtual bool [SetValueWithDouble](#) (double value)=0
Sets the parameter's value as a double.
- virtual bool [SetValueWithString](#) (const [AAX_IString](#) &value)=0
Sets the parameter's value as a string.
- virtual void [SetType](#) ([AAX_EParameterType](#) iControlType)=0
Sets the type of this parameter.
- virtual [AAX_EParameterType](#) [GetType](#) () const =0
Returns the type of this parameter as an AAX_EParameterType.
- virtual void [SetOrientation](#) ([AAX_EParameterOrientation](#) iOrientation)=0
Sets the orientation of this parameter.
- virtual [AAX_EParameterOrientation](#) [GetOrientation](#) () const =0
Returns the orientation of this parameter.
- virtual void [SetTaperDelegate](#) ([AAX_ITaperDelegateBase](#) &inTaperDelegate, bool inPreserveValue)=0
Sets the parameter's taper delegate.
- virtual void [SetDisplayDelegate](#) ([AAX_IDisplayDelegateBase](#) &inDisplayDelegate)=0
Sets the parameter's display delegate.

Host interface methods

- virtual void [UpdateNormalizedValue](#) (double newNormalizedValue)=0
Sets the parameter's state given a normalized value.

14.123.2 Constructor & Destructor Documentation

14.123.2.1 ~AAX_IParameter()

```
virtual AAX_IParameter::~~AAX_IParameter ( ) [inline], [virtual]
```

Virtual destructor.

Note

This destructor MUST be virtual to prevent memory leaks.

14.123.3 Member Function Documentation

14.123.3.1 CloneValue()

```
virtual AAX\_IParacterValue* AAX_IParacter::CloneValue ( ) const [pure virtual]
```

Clone the parameter's value to a new [AAX_IParacterValue](#) object.

The returned object is independent from the [AAX_IParacter](#). For example, changing the state of the returned object will not result in a change to the original [AAX_IParacter](#).

Implemented in [AAX_CStatelessParameter](#), and [AAX_CParameter< T >](#).

Referenced by [AAX_CMonolithicParameters::GenerateCoefficients\(\)](#).

Here is the caller graph for this function:

14.123.3.2 Identifier()

```
virtual AAX\_CParamID AAX_IParacter::Identifier ( ) const [pure virtual]
```

Returns the parameter's unique identifier.

This unique ID is used by the [Parameter Manager](#) and by outside applications to uniquely identify and target control messages. This value may not be changed after the parameter has been constructed.

Implemented in [AAX_CStatelessParameter](#), and [AAX_CParameter< T >](#).

Referenced by [AAX_CMonolithicParameters::AddSynchronizedParameter\(\)](#), and [AAX_CMonolithicParameters::↔GenerateCoefficients\(\)](#).

Here is the caller graph for this function:

14.123.3.3 SetName()

```
virtual void AAX_IParacter::SetName (
    const AAX\_CString & name ) [pure virtual]
```

Sets the parameter's display name.

This name is used for display only, it is not used for indexing or identifying the parameter. This name may be changed after the parameter has been created, but display name changes may not be recognized by all AAX hosts.

Parameters

in	<i>name</i>	Display name that will be assigned to the parameter
----	-------------	---

Implemented in [AAX_CStatelessParameter](#), and [AAX_CParameter< T >](#).

14.123.3.4 Name()

```
virtual const AAX_CString& AAX_IPParameter::Name ( ) const [pure virtual]
```

Returns the parameter's display name.

Note

This method returns a const reference in order to prevent a string copy. Do not cast away the const to change this value.

Implemented in [AAX_CStatelessParameter](#), and [AAX_CParameter< T >](#).

14.123.3.5 AddShortenedName()

```
virtual void AAX_IPParameter::AddShortenedName (
    const AAX_CString & name ) [pure virtual]
```

Sets the parameter's shortened display name.

This name is used for display only, it is not used for indexing or identifying the parameter. These names show up when the host asks for shorter length parameter names for display on Control Surfaces or other string length constrained situations.

Parameters

in	<i>name</i>	Shortened display names that will be assigned to the parameter
----	-------------	--

Implemented in [AAX_CStatelessParameter](#), and [AAX_CParameter< T >](#).

14.123.3.6 ShortenedName()

```
virtual const AAX_CString& AAX_IPParameter::ShortenedName (
    int32_t iNumCharacters ) const [pure virtual]
```

Returns the parameter's shortened display name.

Note

This method returns a const reference in order to prevent a string copy. Do not cast away the const to change this value.

Implemented in [AAX_CStatelessParameter](#), and [AAX_CParameter< T >](#).

14.123.3.7 ClearShortenedNames()

```
virtual void AAX_IParacter::ClearShortenedNames ( ) [pure virtual]
```

Clears the internal list of shortened display names.

Implemented in [AAX_CStatelessParameter](#), and [AAX_CParameter< T >](#).

14.123.3.8 Automatable()

```
virtual bool AAX_IParacter::Automatable ( ) const [pure virtual]
```

Returns true if the parameter is automatable, false if it is not.

Note

Subclasses that return true in this method must support host-based automation.

Implemented in [AAX_CStatelessParameter](#), and [AAX_CParameter< T >](#).

Referenced by [AAX_CMonolithicParameters::AddSynchronizedParameter\(\)](#).

Here is the caller graph for this function:

14.123.3.9 SetAutomationDelegate()

```
virtual void AAX_IParacter::SetAutomationDelegate (
    AAX_IAutomationDelegate * iAutomationDelegate ) [pure virtual]
```

Sets the automation delegate (if one is required)

Parameters

in	<i>iAutomationDelegate</i>	A reference to the parameter manager's automation delegate interface
----	----------------------------	--

Implemented in [AAX_CStatelessParameter](#), and [AAX_CParameter< T >](#).

14.123.3.10 Touch()

```
virtual void AAX_IParacter::Touch ( ) [pure virtual]
```

Signals the automation system that a control has been touched.

Call this method in response to GUI events that begin editing, such as a mouse down. After this method has been called you are free to call [SetNormalizedValue\(\)](#) as much as you need, e.g. in order to respond to subsequent mouse moved events. Call [Release\(\)](#) to free the parameter for updates from other controls.

Implemented in [AAX_CStatelessParameter](#), and [AAX_CParameter< T >](#).

14.123.3.11 Release()

```
virtual void AAX_IPParameter::Release ( ) [pure virtual]
```

Signals the automation system that a control has been released.

Call this method in response to GUI events that complete editing, such as a mouse up. Once this method has been called you should not call [SetNormalizedValue\(\)](#) again until after the next call to [Touch\(\)](#).

Implemented in [AAX_CStatelessParameter](#), and [AAX_CParameter< T >](#).

14.123.3.12 SetNormalizedValue()

```
virtual void AAX_IPParameter::SetNormalizedValue (
    double newNormalizedValue ) [pure virtual]
```

Sets a parameter value using it's normalized representation.

For more information regarding normalized values, see [Parameter Manager](#)

Parameters

in	<i>newNormalizedValue</i>	New value (normalized) to which the parameter will be set
----	---------------------------	---

Implemented in [AAX_CStatelessParameter](#), and [AAX_CParameter< T >](#).

14.123.3.13 GetNormalizedValue()

```
virtual double AAX_IPParameter::GetNormalizedValue ( ) const [pure virtual]
```

Returns the normalized representation of the parameter's current real value.

Implemented in [AAX_CStatelessParameter](#), and [AAX_CParameter< T >](#).

14.123.3.14 SetNormalizedDefaultValue()

```
virtual void AAX_IPParameter::SetNormalizedDefaultValue (
    double normalizedDefault ) [pure virtual]
```

Sets the parameter's default value using its normalized representation.

Implemented in [AAX_CStatelessParameter](#), and [AAX_CParameter< T >](#).

14.123.3.15 GetNormalizedDefaultValue()

```
virtual double AAX_IParacter::GetNormalizedDefaultValue ( ) const [pure virtual]
```

Returns the normalized representation of the parameter's real default value.

Implemented in [AAX_CStatelessParameter](#), and [AAX_CParameter< T >](#).

14.123.3.16 SetToDefaultValue()

```
virtual void AAX_IParacter::SetToDefaultValue ( ) [pure virtual]
```

Restores the state of this parameter to its default value.

Implemented in [AAX_CStatelessParameter](#), and [AAX_CParameter< T >](#).

14.123.3.17 SetNumberOfSteps()

```
virtual void AAX_IParacter::SetNumberOfSteps (
    uint32_t numSteps ) [pure virtual]
```

Sets the number of discrete steps for this parameter.

Stepped parameter values are useful for discrete parameters and for "jumping" events such as mouse wheels, page up/down, etc. The parameter's step size is used to specify the coarseness of those changes.

Note

numSteps MUST be greater than zero. All other values may be considered an error by the host.

Parameters

in	<i>numSteps</i>	The number of steps that the parameter will use
----	-----------------	---

Implemented in [AAX_CStatelessParameter](#), and [AAX_CParameter< T >](#).

14.123.3.18 GetNumberOfSteps()

```
virtual uint32_t AAX_IParacter::GetNumberOfSteps ( ) const [pure virtual]
```

Returns the number of discrete steps used by the parameter.

See [SetNumberOfSteps\(\)](#) for more information about parameter steps.

Implemented in [AAX_CStatelessParameter](#), and [AAX_CParameter< T >](#).

14.123.3.19 GetStepValue()

```
virtual uint32_t AAX_IParameter::GetStepValue ( ) const [pure virtual]
```

Returns the current step for the current value of the parameter.

See [SetNumberOfSteps\(\)](#) for more information about parameter steps.

Implemented in [AAX_CStatelessParameter](#), and [AAX_CParameter< T >](#).

14.123.3.20 GetNormalizedValueFromStep()

```
virtual double AAX_IParameter::GetNormalizedValueFromStep (
    uint32_t iStep ) const [pure virtual]
```

Returns the normalized value for a given step.

See [SetNumberOfSteps\(\)](#) for more information about parameter steps.

Implemented in [AAX_CStatelessParameter](#), and [AAX_CParameter< T >](#).

14.123.3.21 GetStepValueFromNormalizedValue()

```
virtual uint32_t AAX_IParameter::GetStepValueFromNormalizedValue (
    double normalizedValue ) const [pure virtual]
```

Returns the step value for a normalized value of the parameter.

See [SetNumberOfSteps\(\)](#) for more information about parameter steps.

Implemented in [AAX_CStatelessParameter](#), and [AAX_CParameter< T >](#).

14.123.3.22 SetStepValue()

```
virtual void AAX_IParameter::SetStepValue (
    uint32_t iStep ) [pure virtual]
```

Returns the current step for the current value of the parameter.

See [SetNumberOfSteps\(\)](#) for more information about parameter steps.

Implemented in [AAX_CStatelessParameter](#), and [AAX_CParameter< T >](#).

14.123.3.23 GetValueString() [1/2]

```
virtual bool AAX_IParameter::GetValueString (
    AAX_CString * valueString ) const [pure virtual]
```

Serializes the parameter value into a string.

Parameters

out	<i>valueString</i>	A string representing the parameter's real value
-----	--------------------	--

Return values

<i>true</i>	The string conversion was successful
<i>false</i>	The string conversion was unsuccessful

Implemented in [AAX_CStatelessParameter](#), and [AAX_CParameter< T >](#).

14.123.3.24 GetValueString() [2/2]

```
virtual bool AAX_IParacter::GetValueString (
    int32_t iMaxNumChars,
    AAX_CString * valueString ) const [pure virtual]
```

Serializes the parameter value into a string, size hint included.

Parameters

in	<i>iMaxNumChars</i>	A size hint for the size of the string being requested. Useful for control surfaces and other limited area text fields. (make sure that size of desired string also has room for null termination)
out	<i>valueString</i>	A string representing the parameter's real value

Return values

<i>true</i>	The string conversion was successful
<i>false</i>	The string conversion was unsuccessful

Implemented in [AAX_CStatelessParameter](#), and [AAX_CParameter< T >](#).

14.123.3.25 GetNormalizedValueFromBool()

```
virtual bool AAX_IParacter::GetNormalizedValueFromBool (
    bool value,
    double * normalizedValue ) const [pure virtual]
```

Converts a bool to a normalized parameter value.

Parameters

in	<i>value</i>	A value for the parameter
out	<i>normalizedValue</i>	The normalized parameter value associated with value

Return values

<i>true</i>	The value conversion was successful
<i>false</i>	The value conversion was unsuccessful

Implemented in [AAX_CStatelessParameter](#), [AAX_CParameter< T >](#), and [AAX_CParameter< T >](#).

14.123.3.26 GetNormalizedValueFromInt32()

```
virtual bool AAX_IParameter::GetNormalizedValueFromInt32 (
    int32_t value,
    double * normalizedValue ) const [pure virtual]
```

Converts an integer to a normalized parameter value.

Parameters

in	<i>value</i>	A value for the parameter
out	<i>normalizedValue</i>	The normalized parameter value associated with value

Return values

<i>true</i>	The value conversion was successful
<i>false</i>	The value conversion was unsuccessful

Implemented in [AAX_CStatelessParameter](#), [AAX_CParameter< T >](#), and [AAX_CParameter< T >](#).

14.123.3.27 GetNormalizedValueFromFloat()

```
virtual bool AAX_IParameter::GetNormalizedValueFromFloat (
    float value,
    double * normalizedValue ) const [pure virtual]
```

Converts a float to a normalized parameter value.

Parameters

in	<i>value</i>	A value for the parameter
out	<i>normalizedValue</i>	The normalized parameter value associated with value

Return values

<i>true</i>	The value conversion was successful
<i>false</i>	The value conversion was unsuccessful

Implemented in [AAX_CStatelessParameter](#), [AAX_CParameter< T >](#), and [AAX_CParameter< T >](#).

14.123.3.28 GetNormalizedValueFromDouble()

```
virtual bool AAX_IPParameter::GetNormalizedValueFromDouble (
    double value,
    double * normalizedValue ) const [pure virtual]
```

Converts a double to a normalized parameter value.

Parameters

in	<i>value</i>	A value for the parameter
out	<i>normalizedValue</i>	The normalized parameter value associated with value

Return values

<i>true</i>	The value conversion was successful
<i>false</i>	The value conversion was unsuccessful

Implemented in [AAX_CStatelessParameter](#), [AAX_CParameter< T >](#), and [AAX_CParameter< T >](#).

14.123.3.29 GetNormalizedValueFromString()

```
virtual bool AAX_IPParameter::GetNormalizedValueFromString (
    const AAX\_CString & valueString,
    double * normalizedValue ) const [pure virtual]
```

Converts a given string to a normalized parameter value.

Parameters

in	<i>valueString</i>	A string representing a possible real value for the parameter
out	<i>normalizedValue</i>	The normalized parameter value associated with valueString

Return values

<i>true</i>	The string conversion was successful
<i>false</i>	The string conversion was unsuccessful

Implemented in [AAX_CParameter< T >](#), and [AAX_CStatelessParameter](#).

14.123.3.30 GetBoolFromNormalizedValue()

```
virtual bool AAX_IParameter::GetBoolFromNormalizedValue (
    double normalizedValue,
    bool * value ) const [pure virtual]
```

Converts a normalized parameter value to a bool representing the corresponding real value.

Parameters

in	<i>normalizedValue</i>	The normalized value to convert
out	<i>value</i>	The converted value. Set only if conversion is successful.

Return values

<i>true</i>	The conversion to bool was successful
<i>false</i>	The conversion to bool was unsuccessful

Implemented in [AAX_CStatelessParameter](#), [AAX_CParameter< T >](#), and [AAX_CParameter< T >](#).

14.123.3.31 GetInt32FromNormalizedValue()

```
virtual bool AAX_IParameter::GetInt32FromNormalizedValue (
    double normalizedValue,
    int32_t * value ) const [pure virtual]
```

Converts a normalized parameter value to an integer representing the corresponding real value.

Parameters

in	<i>normalizedValue</i>	The normalized value to convert
out	<i>value</i>	The converted value. Set only if conversion is successful.

Return values

<i>true</i>	The conversion to int32_t was successful
<i>false</i>	The conversion to int32_t was unsuccessful

Implemented in [AAX_CStatelessParameter](#), [AAX_CParameter< T >](#), and [AAX_CParameter< T >](#).

14.123.3.32 GetFloatFromNormalizedValue()

```
virtual bool AAX_IParameter::GetFloatFromNormalizedValue (
    double normalizedValue,
    float * value ) const [pure virtual]
```

Converts a normalized parameter value to a float representing the corresponding real value.

Parameters

in	<i>normalizedValue</i>	The normalized value to convert
out	<i>value</i>	The converted value. Set only if conversion is successful.

Return values

<i>true</i>	The conversion to float was successful
<i>false</i>	The conversion to float was unsuccessful

Implemented in [AAX_CStatelessParameter](#), [AAX_CParameter< T >](#), and [AAX_CParameter< T >](#).

14.123.3.33 GetDoubleFromNormalizedValue()

```
virtual bool AAX_IParacter::GetDoubleFromNormalizedValue (
    double normalizedValue,
    double * value ) const [pure virtual]
```

Converts a normalized parameter value to a double representing the corresponding real value.

Parameters

in	<i>normalizedValue</i>	The normalized value to convert
out	<i>value</i>	The converted value. Set only if conversion is successful.

Return values

<i>true</i>	The conversion to double was successful
<i>false</i>	The conversion to double was unsuccessful

Implemented in [AAX_CStatelessParameter](#), [AAX_CParameter< T >](#), and [AAX_CParameter< T >](#).

14.123.3.34 GetStringFromNormalizedValue() [1/2]

```
virtual bool AAX_IParacter::GetStringFromNormalizedValue (
    double normalizedValue,
    AAX_CString & valueString ) const [pure virtual]
```

Converts a normalized parameter value to a string representing the corresponding real value.

Parameters

in	<i>normalizedValue</i>	A normalized parameter value
out	<i>valueString</i>	A string representing the parameter value associated with <i>normalizedValue</i>

Return values

<i>true</i>	The string conversion was successful
<i>false</i>	The string conversion was unsuccessful

Implemented in [AAX_CStatelessParameter](#), and [AAX_CParameter< T >](#).

14.123.3.35 GetStringFromNormalizedValue() [2/2]

```
virtual bool AAX_IParameter::GetStringFromNormalizedValue (
    double normalizedValue,
    int32_t iMaxNumChars,
    AAX_CString & valueString ) const [pure virtual]
```

Converts a normalized parameter value to a string representing the corresponding real, size hint included. *value*.

Parameters

in	<i>normalizedValue</i>	A normalized parameter value
in	<i>iMaxNumChars</i>	A size hint for the size of the string being requested. Useful for control surfaces and other limited area text fields. (make sure that size of desired string also has room for null termination)
out	<i>valueString</i>	A string representing the parameter value associated with <i>normalizedValue</i>

Return values

<i>true</i>	The string conversion was successful
<i>false</i>	The string conversion was unsuccessful

Implemented in [AAX_CStatelessParameter](#), and [AAX_CParameter< T >](#).

14.123.3.36 SetValueFromString()

```
virtual bool AAX_IParameter::SetValueFromString (
    const AAX_CString & newValueString ) [pure virtual]
```

Converts a string to a real parameter value and sets the parameter to this value.

Parameters

in	<i>newValueString</i>	A string representing the parameter's new real value
----	-----------------------	--

Return values

<i>true</i>	The string conversion was successful
<i>false</i>	The string conversion was unsuccessful

Implemented in [AAX_CStatelessParameter](#), and [AAX_CParameter< T >](#).

14.123.3.37 GetValueAsBool()

```
virtual bool AAX_IParacter::GetValueAsBool (
    bool * value ) const [pure virtual]
```

Retrieves the parameter's value as a bool.

Parameters

out	<i>value</i>	The parameter's real value. Set only if conversion is successful.
-----	--------------	---

Return values

<i>true</i>	The conversion to bool was successful
<i>false</i>	The conversion to bool was unsuccessful

Implemented in [AAX_CStatelessParameter](#), and [AAX_CParameter< T >](#).

14.123.3.38 GetValueAsInt32()

```
virtual bool AAX_IParacter::GetValueAsInt32 (
    int32_t * value ) const [pure virtual]
```

Retrieves the parameter's value as an int32_t.

Parameters

out	<i>value</i>	The parameter's real value. Set only if conversion is successful.
-----	--------------	---

Return values

<i>true</i>	The conversion to int32_t was successful
<i>false</i>	The conversion to int32_t was unsuccessful

Implemented in [AAX_CParameter< T >](#), and [AAX_CStatelessParameter](#).

14.123.3.39 GetValueAsFloat()

```
virtual bool AAX_IParameter::GetValueAsFloat (
    float * value ) const [pure virtual]
```

Retrieves the parameter's value as a float.

Parameters

out	<i>value</i>	The parameter's real value. Set only if conversion is successful.
-----	--------------	---

Return values

<i>true</i>	The conversion to float was successful
<i>false</i>	The conversion to float was unsuccessful

Implemented in [AAX_CParameter< T >](#), and [AAX_CStatelessParameter](#).

14.123.3.40 GetValueAsDouble()

```
virtual bool AAX_IParameter::GetValueAsDouble (
    double * value ) const [pure virtual]
```

Retrieves the parameter's value as a double.

Parameters

out	<i>value</i>	The parameter's real value. Set only if conversion is successful.
-----	--------------	---

Return values

<i>true</i>	The conversion to double was successful
<i>false</i>	The conversion to double was unsuccessful

Implemented in [AAX_CParameter< T >](#), and [AAX_CStatelessParameter](#).

14.123.3.41 GetValueAsString()

```
virtual bool AAX_IParameter::GetValueAsString (
    AAX\_IString * value ) const [pure virtual]
```

Retrieves the parameter's value as a string.

Parameters

out	value	The parameter's real value. Set only if conversion is successful.
-----	-------	---

Return values

true	The conversion to string was successful
false	The conversion to string was unsuccessful

Implemented in [AAX_CParameter< T >](#), [AAX_CStatelessParameter](#), and [AAX_CParameter< T >](#).

14.123.3.42 SetValueWithBool()

```
virtual bool AAX_IParameter::SetValueWithBool (
    bool value ) [pure virtual]
```

Sets the parameter's value as a bool.

Parameters

out	value	The parameter's real value. Set only if conversion is successful.
-----	-------	---

Return values

true	The conversion from bool was successful
false	The conversion from bool was unsuccessful

Implemented in [AAX_CStatelessParameter](#), [AAX_CParameter< T >](#), and [AAX_CParameter< T >](#).

14.123.3.43 SetValueWithInt32()

```
virtual bool AAX_IParameter::SetValueWithInt32 (
    int32_t value ) [pure virtual]
```

Sets the parameter's value as an int32_t.

Parameters

out	value	The parameter's real value. Set only if conversion is successful.
-----	-------	---

Return values

true	The conversion from int32_t was successful
false	The conversion from int32_t was unsuccessful

Implemented in [AAX_CStatelessParameter](#), [AAX_CParameter< T >](#), and [AAX_CParameter< T >](#).

14.123.3.44 SetValueWithFloat()

```
virtual bool AAX_IParameter::SetValueWithFloat (
    float value ) [pure virtual]
```

Sets the parameter's value as a float.

Parameters

out	<i>value</i>	The parameter's real value. Set only if conversion is successful.
-----	--------------	---

Return values

<i>true</i>	The conversion from float was successful
<i>false</i>	The conversion from float was unsuccessful

Implemented in [AAX_CStatelessParameter](#), [AAX_CParameter< T >](#), and [AAX_CParameter< T >](#).

14.123.3.45 SetValueWithDouble()

```
virtual bool AAX_IParameter::SetValueWithDouble (
    double value ) [pure virtual]
```

Sets the parameter's value as a double.

Parameters

out	<i>value</i>	The parameter's real value. Set only if conversion is successful.
-----	--------------	---

Return values

<i>true</i>	The conversion from double was successful
<i>false</i>	The conversion from double was unsuccessful

Implemented in [AAX_CStatelessParameter](#), [AAX_CParameter< T >](#), and [AAX_CParameter< T >](#).

14.123.3.46 SetValueWithString()

```
virtual bool AAX_IParameter::SetValueWithString (
    const AAX\_IString & value ) [pure virtual]
```

Sets the parameter's value as a string.

Parameters

out	<i>value</i>	The parameter's real value. Set only if conversion is successful.
-----	--------------	---

Return values

<i>true</i>	The conversion from string was successful
<i>false</i>	The conversion from string was unsuccessful

Implemented in [AAX_CStatelessParameter](#), [AAX_CParameter< T >](#), and [AAX_CParameter< T >](#).

14.123.3.47 SetType()

```
virtual void AAX_IParameter::SetType (
    AAX_EParameterType iControlType ) [pure virtual]
```

Sets the type of this parameter.

See [GetType](#) for use cases

Parameters

in	<i>iControlType</i>	The parameter's new type as an AAX_EParameterType
----	---------------------	---

Implemented in [AAX_CStatelessParameter](#), and [AAX_CParameter< T >](#).

14.123.3.48 GetType()

```
virtual AAX_EParameterType AAX_IParameter::GetType ( ) const [pure virtual]
```

Returns the type of this parameter as an AAX_EParameterType.

Todo Document use cases for control type

Implemented in [AAX_CStatelessParameter](#), and [AAX_CParameter< T >](#).

14.123.3.49 SetOrientation()

```
virtual void AAX_IParameter::SetOrientation (
    AAX_EParameterOrientation iOrientation ) [pure virtual]
```

Sets the orientation of this parameter.

Parameters

in	<i>iOrientation</i>	The parameter's new orientation
----	---------------------	---------------------------------

Implemented in [AAX_CStatelessParameter](#), and [AAX_CParameter< T >](#).

14.123.3.50 GetOrientation()

```
virtual AAX\_EParameterOrientation AAX_IParameter::GetOrientation ( ) const [pure virtual]
```

Returns the orientation of this parameter.

Implemented in [AAX_CStatelessParameter](#), and [AAX_CParameter< T >](#).

14.123.3.51 SetTaperDelegate()

```
virtual void AAX_IParameter::SetTaperDelegate (
    AAX\_ITaperDelegateBase & inTaperDelegate,
    bool inPreserveValue ) [pure virtual]
```

Sets the parameter's taper delegate.

Parameters

in	<i>inTaperDelegate</i>	A reference to the parameter's new taper delegate
in	<i>inPreserveValue</i>	

Todo Document this parameter

Implemented in [AAX_CParameter< T >](#), and [AAX_CStatelessParameter](#).

14.123.3.52 SetDisplayDelegate()

```
virtual void AAX_IParameter::SetDisplayDelegate (
    AAX\_IDisplayDelegateBase & inDisplayDelegate ) [pure virtual]
```

Sets the parameter's display delegate.

Parameters

in	<i>inDisplayDelegate</i>	A reference to the parameter's new display delegate
----	--------------------------	---

Implemented in [AAX_CParameter< T >](#), and [AAX_CStatelessParameter](#).

14.123.3.53 UpdateNormalizedValue()

```
virtual void AAX_IParameter::UpdateNormalizedValue (
    double newNormalizedValue ) [pure virtual]
```

Sets the parameter's state given a normalized value.

This is the second half of the parameter setting operation that is initiated with a call to SetValue(). Parameters should not be set directly using this method; instead, use SetValue().

Parameters

in	<i>newNormalizedValue</i>	Normalized value that will be used to set the parameter's new state
----	---------------------------	---

Implemented in [AAX_CStatelessParameter](#), and [AAX_CParameter< T >](#).

The documentation for this class was generated from the following file:

- [AAX_IParameter.h](#)

14.124 AAX_IParameterValue Class Reference

```
#include <AAX_IParameter.h>
```

Inheritance diagram for AAX_IParameterValue:

14.124.1 Description

An abstract interface representing a parameter value of arbitrary type.

:Internal to the AAX SDK

See also

[AAX_IParameter](#)

Public Member Functions

- virtual [~AAX_IParаметerValue](#) ()
Virtual destructor.
- virtual [AAX_IParаметerValue](#) * [Clone](#) () const =0
Clones the parameter object.
- virtual [AAX_CParamID Identifier](#) () const =0
Returns the parameter's unique identifier.

Typed accessors

- virtual bool [GetValueAsBool](#) (bool *value) const =0
Retrieves the parameter's value as a bool.
- virtual bool [GetValueAsInt32](#) (int32_t *value) const =0
Retrieves the parameter's value as an int32_t.
- virtual bool [GetValueAsFloat](#) (float *value) const =0
Retrieves the parameter's value as a float.
- virtual bool [GetValueAsDouble](#) (double *value) const =0
Retrieves the parameter's value as a double.
- virtual bool [GetValueAsString](#) ([AAX_IString](#) *value) const =0
Retrieves the parameter's value as a string.

14.124.2 Constructor & Destructor Documentation

14.124.2.1 ~AAX_IParаметerValue()

```
virtual AAX_IParаметerValue::~~AAX_IParаметerValue ( ) [inline], [virtual]
```

Virtual destructor.

Note

This destructor MUST be virtual to prevent memory leaks.

14.124.3 Member Function Documentation

14.124.3.1 Clone()

```
virtual AAX_IParаметerValue* AAX_IParаметerValue::Clone ( ) const [pure virtual]
```

Clones the parameter object.

Note

Does NOT set the automation delegate on the clone; ownership of the automation delegate and parameter registration/unregistration stays with the original parameter

Implemented in [AAX_CParameterValue< T >](#).

14.124.3.2 Identifier()

```
virtual AAX_CParamID AAX_IParameterValue::Identifier ( ) const [pure virtual]
```

Returns the parameter's unique identifier.

This unique ID is used by the [Parameter Manager](#) and by outside applications to uniquely identify and target control messages. This value may not be changed after the parameter has been constructed.

Implemented in [AAX_CParameterValue< T >](#).

14.124.3.3 GetValueAsBool()

```
virtual bool AAX_IParameterValue::GetValueAsBool (
    bool * value ) const [pure virtual]
```

Retrieves the parameter's value as a bool.

Parameters

out	value	The parameter's real value. Set only if conversion is successful.
-----	-------	---

Return values

true	The conversion to bool was successful
false	The conversion to bool was unsuccessful

Implemented in [AAX_CParameterValue< T >](#), and [AAX_CParameterValue< T >](#).

14.124.3.4 GetValueAsInt32()

```
virtual bool AAX_IParameterValue::GetValueAsInt32 (
    int32_t * value ) const [pure virtual]
```

Retrieves the parameter's value as an int32_t.

Parameters

out	value	The parameter's real value. Set only if conversion is successful.
-----	-------	---

Return values

true	The conversion to int32_t was successful
false	The conversion to int32_t was unsuccessful

Implemented in [AAX_CParameterValue< T >](#), and [AAX_CParameterValue< T >](#).

14.124.3.5 GetValueAsFloat()

```
virtual bool AAX_IParаметerValue::GetValueAsFloat (
    float * value ) const [pure virtual]
```

Retrieves the parameter's value as a float.

Parameters

out	<i>value</i>	The parameter's real value. Set only if conversion is successful.
-----	--------------	---

Return values

<i>true</i>	The conversion to float was successful
<i>false</i>	The conversion to float was unsuccessful

Implemented in [AAX_CParameterValue< T >](#), and [AAX_CParameterValue< T >](#).

14.124.3.6 GetValueAsDouble()

```
virtual bool AAX_IParаметerValue::GetValueAsDouble (
    double * value ) const [pure virtual]
```

Retrieves the parameter's value as a double.

Parameters

out	<i>value</i>	The parameter's real value. Set only if conversion is successful.
-----	--------------	---

Return values

<i>true</i>	The conversion to double was successful
<i>false</i>	The conversion to double was unsuccessful

Implemented in [AAX_CParameterValue< T >](#), and [AAX_CParameterValue< T >](#).

14.124.3.7 GetValueAsString()

```
virtual bool AAX_IParаметerValue::GetValueAsString (
    AAX\_IString * value ) const [pure virtual]
```

Retrieves the parameter's value as a string.

Parameters

out	value	The parameter's real value. Set only if conversion is successful.
-----	-------	---

Return values

true	The conversion to string was successful
false	The conversion to string was unsuccessful

Implemented in [AAX_CParameterValue< T >](#), and [AAX_CParameterValue< T >](#).

The documentation for this class was generated from the following file:

- [AAX_IParameter.h](#)

14.125 AAX_IPointerQueue< T > Class Template Reference

```
#include <AAX_IPointerQueue.h>
```

Inheritance diagram for AAX_IPointerQueue< T >:

Collaboration diagram for AAX_IPointerQueue< T >:

14.125.1 Description

```
template<typename T>
class AAX_IPointerQueue< T >
```

Abstract interface for a basic FIFO queue of pointers-to-objects

Public Types

- typedef T [template_type](#)
The type used for this template instance.
- typedef T * [value_type](#)
The type of values stored in this queue.

Public Member Functions

- virtual [~AAX_IPointerQueue](#) ()
- virtual void [Clear](#) ()=0
- virtual [AAX_IContainer::EStatus Push](#) ([value_type](#) inElem)=0
- virtual [value_type Pop](#) ()=0
- virtual [value_type Peek](#) () const =0

14.125.2 Member Typedef Documentation

14.125.2.1 `template_type`

```
template<typename T >
typedef T AAX\_IPointerQueue< T >::template_type
```

The type used for this template instance.

14.125.2.2 `value_type`

```
template<typename T >
typedef T* AAX\_IPointerQueue< T >::value_type
```

The type of values stored in this queue.

14.125.3 Constructor & Destructor Documentation

14.125.3.1 `~AAX_IPointerQueue()`

```
template<typename T >
virtual AAX\_IPointerQueue< T >::~~AAX_IPointerQueue ( ) [inline], [virtual]
```

14.125.4 Member Function Documentation

14.125.4.1 `Clear()`

```
template<typename T >
virtual void AAX\_IPointerQueue< T >::Clear ( ) [pure virtual]
```

Note

This operation is NOT atomic

This does NOT call the destructor for any pointed-to elements; it only clears the pointer values in the queue

Implements [AAX_IContainer](#).

Implemented in [AAX_CAtomicQueue< T, S >](#), [AAX_CAtomicQueue< TNumberedParamStateList, 256 >](#), and [AAX_CAtomicQueue< const TParamValPair, 16 *kSynchronizedParameterQueueSize >](#).

14.125.4.2 Push()

```
template<typename T >
virtual AAX_IContainer::EStatus AAX_IPointerQueue< T >::Push (
    value_type inElem ) [pure virtual]
```

Push an element onto the queue

Call from: Write thread

Returns

[AAX_IContainer::EStatus_Success](#) if the push succeeded

Implemented in [AAX_CAtomicQueue< T, S >](#), [AAX_CAtomicQueue< TNumberedParamStateList, 256 >](#), and [AAX_CAtomicQueue< const TParamValPair, 16 *kSynchronizedParameterQueueSize >](#).

14.125.4.3 Pop()

```
template<typename T >
virtual value_type AAX_IPointerQueue< T >::Pop ( ) [pure virtual]
```

Pop the front element from the queue

Call from: Read thread

Returns

NULL if no element is available

Implemented in [AAX_CAtomicQueue< T, S >](#), [AAX_CAtomicQueue< TNumberedParamStateList, 256 >](#), and [AAX_CAtomicQueue< const TParamValPair, 16 *kSynchronizedParameterQueueSize >](#).

14.125.4.4 Peek()

```
template<typename T >
virtual value_type AAX_IPointerQueue< T >::Peek ( ) const [pure virtual]
```

Get the current top element without popping it off of the queue

Call from: Read thread

Note

This value will change if another thread calls [Pop\(\)](#)

Implemented in [AAX_CAtomicQueue< T, S >](#), [AAX_CAtomicQueue< TNumberedParamStateList, 256 >](#), and [AAX_CAtomicQueue< const TParamValPair, 16 *kSynchronizedParameterQueueSize >](#).

The documentation for this class was generated from the following file:

- [AAX_IPointerQueue.h](#)

14.126 AAX_IPrivateDataAccess Class Reference

```
#include <AAX_IPrivateDataAccess.h>
```

Inheritance diagram for AAX_IPrivateDataAccess:

14.126.1 Description

Interface to data access provided by host to plug-in.

:Implemented by the AAX Host

WARNING: [AAX_IPrivateDataAccess](#) objects are not reference counted and are not guaranteed to exist beyond the scope of the method(s) they are passed into.

See also

[AAX_IACFEEffectDirectData::TimerWakeup](#)

Public Member Functions

- virtual [~AAX_IPrivateDataAccess](#) ()
- virtual [AAX_Result ReadPortDirect](#) ([AAX_CFieldIndex](#) inFieldIndex, const uint32_t inOffset, const uint32_t inSize, void *outBuffer)=0
Read data directly from DSP at the given port.
- virtual [AAX_Result WritePortDirect](#) ([AAX_CFieldIndex](#) inFieldIndex, const uint32_t inOffset, const uint32_t inSize, const void *inBuffer)=0
Write data directly to DSP at the given port.

14.126.2 Constructor & Destructor Documentation

14.126.2.1 ~AAX_IPrivateDataAccess()

```
virtual AAX_IPrivateDataAccess::~~AAX_IPrivateDataAccess ( ) [inline], [virtual]
```

14.126.3 Member Function Documentation

14.126.3.1 ReadPortDirect()

```
virtual AAX\_Result AAX_IPrivateDataAccess::ReadPortDirect (
    AAX\_CFieldIndex inFieldIndex,
    const uint32_t inOffset,
    const uint32_t inSize,
    void * outBuffer ) [pure virtual]
```

Read data directly from DSP at the given port.

Note

Blocking

Parameters

in	<i>inFieldIndex</i>	The port to read from.
in	<i>inOffset</i>	Offset into data to start reading.
in	<i>inSize</i>	Amount of data to read (in bytes).
out	<i>outBuffer</i>	Pointer to storage for data to be read into.

Implemented in [AAX_VPrivateDataAccess](#).

14.126.3.2 WritePortDirect()

```
virtual AAX_Result AAX_IPrivateDataAccess::WritePortDirect (
    AAX_CFieldIndex inFieldIndex,
    const uint32_t inOffset,
    const uint32_t inSize,
    const void * inBuffer ) [pure virtual]
```

Write data directly to DSP at the given port.

Note

Blocking

Parameters

in	<i>inFieldIndex</i>	The port to write to.
in	<i>inOffset</i>	Offset into data to begin writing.
in	<i>inSize</i>	Amount of data to write (in bytes).
in	<i>inBuffer</i>	Pointer to data being written.

Implemented in [AAX_VPrivateDataAccess](#).

The documentation for this class was generated from the following file:

- [AAX_IPrivateDataAccess.h](#)

14.127 AAX_IPropertyMap Class Reference

```
#include <AAX_IPropertyMap.h>
```

Inheritance diagram for AAX_IPropertyMap:

14.127.1 Description

Generic plug-in description property map.

:Implemented by the AAX Host

Property Maps are used to associate specific sets of properties with plug-in description interfaces. For example, an audio processing component might register mono and stereo callbacks, or Native and TI callbacks, assigning each `ProcessProc` the applicable property mapping. This allows the host to determine the correct callback to use depending on the environment in which the plug-in is instantiated.

AAX does not require that every value in AAX IPropertyMap be assigned by the developer. Unassigned properties do not have defined default values; if a specific value is not assigned to one of an element's properties then the element must support any value for that property. For example, if an audio processing component does not define its callback's audio buffer length property, the host will assume that the callback will support any buffer length.

- To create a new property map: [AAX_IComponentDescriptor::NewPropertyMap\(\)](#)
- To copy an existing property map: [AAX_IComponentDescriptor::DuplicatePropertyMap\(\)](#)

Public Member Functions

- virtual [~AAX_IPropertyMap](#) ()
- virtual [AAX_CBoolean](#) [GetProperty](#) ([AAX_EProperty](#) inProperty, [AAX_CPropertyValue](#) *outValue) const =0
Get a property value from a property map.
- virtual [AAX_CBoolean](#) [GetPointerProperty](#) ([AAX_EProperty](#) inProperty, const void **outValue) const =0
Get a property value from a property map with a pointer-sized value.
- virtual [AAX_Result](#) [AddProperty](#) ([AAX_EProperty](#) inProperty, [AAX_CPropertyValue](#) inValue)=0
Add a property to a property map.
- virtual [AAX_Result](#) [AddPointerProperty](#) ([AAX_EProperty](#) inProperty, const void *inValue)=0
Add a property to a property map with a pointer-sized value.
- virtual [AAX_Result](#) [AddPointerProperty](#) ([AAX_EProperty](#) inProperty, const char *inValue)=0
Add a property to a property map with a pointer-sized value.
- virtual [AAX_Result](#) [RemoveProperty](#) ([AAX_EProperty](#) inProperty)=0
Remove a property from a property map.
- virtual [AAX_Result](#) [AddPropertyWithIDArray](#) ([AAX_EProperty](#) inProperty, const [AAX_SPlugInIdentifierTriad](#) *inPluginIDs, uint32_t inNumPluginIDs)=0
Add an array of plug-in IDs to a property map.
- virtual [AAX_CBoolean](#) [GetPropertyWithIDArray](#) ([AAX_EProperty](#) inProperty, const [AAX_SPlugInIdentifierTriad](#) **outPluginIDs, uint32_t *outNumPluginIDs) const =0
Get an array of plug-in IDs from a property map.
- virtual [IACFUnknown](#) * [GetUnknown](#) ()=0

14.127.2 Constructor & Destructor Documentation

14.127.2.1 ~AAX_IPropertyMap()

```
virtual AAX_IPropertyMap::~~AAX_IPropertyMap ( ) [inline], [virtual]
```

14.127.3 Member Function Documentation

14.127.3.1 GetProperty()

```
virtual AAX_CBoolean AAX_IPropertyMap::GetProperty (
    AAX_EProperty inProperty,
    AAX_CPropertyValue * outValue ) const [pure virtual]
```

Get a property value from a property map.

Returns true if the selected property is supported, false if it is not

Parameters

in	<i>inProperty</i>	The property ID
out	<i>outValue</i>	The property value

Implemented in [AAX_VPropertyMap](#).

14.127.3.2 GetPointerProperty()

```
virtual AAX_CBoolean AAX_IPropertyMap::GetPointerProperty (
    AAX_EProperty inProperty,
    const void ** outValue ) const [pure virtual]
```

Get a property value from a property map with a pointer-sized value.

Returns true if the selected property is supported, false if it is not

Parameters

in	<i>inProperty</i>	The property ID
out	<i>outValue</i>	The property value

Implemented in [AAX_VPropertyMap](#).

14.127.3.3 AddProperty()

```
virtual AAX_Result AAX_IPropertyMap::AddProperty (
    AAX_EProperty inProperty,
    AAX_CPropertyValue inValue ) [pure virtual]
```

Add a property to a property map.

Note

This method may return an error if adding the property was unsuccessful. If there is a failure when adding a required property then registration of the relevant description element must be abandoned and the plug-in's description logic should proceed to the next element.

Parameters

in	<i>inProperty</i>	The property ID.
in	<i>inValue</i>	

Implemented in [AAX_VPropertyMap](#).

Referenced by `AAX_CMonolithicParameters::StaticDescribe()`.

Here is the caller graph for this function:

14.127.3.4 AddPointerProperty() [1/2]

```
virtual AAX_Result AAX_IPropertyMap::AddPointerProperty (
    AAX_EProperty inProperty,
    const void * inValue ) [pure virtual]
```

Add a property to a property map with a pointer-sized value.

Use this method to add properties which require a pointer-sized value. Do not use this method to add a property unless a pointer-sized value is explicitly specified in the property documentation.

Note

This method may return an error if adding the property was unsuccessful. If there is a failure when adding a required property then registration of the relevant description element must be abandoned and the plug-in's description logic should proceed to the next element.

Parameters

in	<i>inProperty</i>	The property ID.
in	<i>inValue</i>	

Implemented in [AAX_VPropertyMap](#).

14.127.3.5 AddPointerProperty() [2/2]

```
virtual AAX_Result AAX_IPropertyMap::AddPointerProperty (
    AAX_EProperty inProperty,
    const char * inValue ) [pure virtual]
```

Add a property to a property map with a pointer-sized value.

Use this method to add properties which require a pointer-sized value. Do not use this method to add a property unless a pointer-sized value is explicitly specified in the property documentation.

Note

This method may return an error if adding the property was unsuccessful. If there is a failure when adding a required property then registration of the relevant description element must be abandoned and the plug-in's description logic should proceed to the next element.

Parameters

in	<i>inProperty</i>	The property ID.
in	<i>inValue</i>	

Implemented in [AAX_VPropertyMap](#).

14.127.3.6 RemoveProperty()

```
virtual AAX_Result AAX_IPropertyMap::RemoveProperty (
    AAX_EProperty inProperty ) [pure virtual]
```

Remove a property from a property map.

Parameters

in	<i>inProperty</i>	The property ID.
----	-------------------	------------------

Implemented in [AAX_VPropertyMap](#).

14.127.3.7 AddPropertyWithIDArray()

```
virtual AAX_Result AAX_IPropertyMap::AddPropertyWithIDArray (
    AAX_EProperty inProperty,
    const AAX_SPlugInIdentifierTriad * inPluginIDs,
    uint32_t inNumPluginIDs ) [pure virtual]
```

Add an array of plug-in IDs to a property map.

Parameters

in	<i>inProperty</i>	The property ID.
in	<i>inPluginIDs</i>	An array of AAX_SPlugInIdentifierTriad
in	<i>inNumPluginIDs</i>	The length of iPluginIDs

Implemented in [AAX_VPropertyMap](#).

14.127.3.8 GetPropertyWithIDArray()

```
virtual AAX_CBoolean AAX_IPropertyMap::GetPropertyWithIDArray (
    AAX_EProperty inProperty,
    const AAX_SPlugInIdentifierTriad ** outPluginIDs,
    uint32_t * outNumPluginIDs ) const [pure virtual]
```

Get an array of plug-in IDs from a property map.

Parameters

in	<i>inProperty</i>	The property ID.
out	<i>outPluginIDs</i>	A pointer that will be set to reference an array of AAX_SPlugInIdentifierTriad
in	<i>outNumPluginIDs</i>	The length of oPluginIDs

Implemented in [AAX_VPropertyMap](#).

14.127.3.9 GetIUnknown()

```
virtual IACFUnknown* AAX_IPropertyMap::GetIUnknown ( ) [pure virtual]
```

Returns the most up-to-date underlying interface

Implemented in [AAX_VPropertyMap](#).

The documentation for this class was generated from the following file:

- [AAX_IPropertyMap.h](#)

14.128 AAX_ISessionDocument Class Reference

```
#include <AAX_ISessionDocument.h>
```

Inheritance diagram for AAX_ISessionDocument:

14.128.1 Description

Interface representing information in a host session document.

This interface wraps the versioned interfaces defined in [AAX_IACFSessionDocument.h](#) and provides additional convenience functions providing session data back in the expected format.

See also

[AAX_ISessionDocumentClient](#)

Classes

- class [TempoMap](#)

Public Member Functions

- virtual [~AAX_I_SessionDocument](#) ()=default
- virtual bool [Valid](#) () const =0
Check whether this session document is valid.
- virtual std::unique_ptr< [TempoMap](#) const > [GetTempoMap](#) ()=0
Get a copy of the document's tempo map.
- virtual [AAX_Result](#) [GetDocumentData](#) ([AAX_DocumentData_UID](#) const &inDataType, [IACFUnknown](#) **outData)=0

14.128.2 Constructor & Destructor Documentation

14.128.2.1 ~AAX_I_SessionDocument()

```
virtual AAX_I_SessionDocument::~~AAX_I_SessionDocument ( ) [virtual], [default]
```

14.128.3 Member Function Documentation

14.128.3.1 Valid()

```
virtual bool AAX_I_SessionDocument::Valid ( ) const [pure virtual]
```

Check whether this session document is valid.

Implemented in [AAX_V_SessionDocument](#).

14.128.3.2 GetTempoMap()

```
virtual std::unique_ptr<TempoMap const> AAX_I_SessionDocument::GetTempoMap ( ) [pure virtual]
```

Get a copy of the document's tempo map.

Returns

A [TempoMap](#) interface representing a copy of the current tempo map.

`nullptr` if the host does not support tempo map data or if an error occurred.

Implemented in [AAX_V_SessionDocument](#).

14.128.3.3 GetDocumentData()

```
virtual AAX_Result AAX_I_SessionDocument::GetDocumentData (
    AAX_DocumentData_UID const & inDataType,
    IACFUnknown ** outData ) [pure virtual]
```

Get document data of a generic type

Similar to `QueryInterface()` but uses a data type identifier rather than a true IID

The provided interface has already had a reference added, so be careful not to add an additional reference:

```
ACFPtr<MyType> ptr;
IACFUnknown * docDataPtr{nullptr};
if (AAX_SUCCESS == doc->GetDocumentData(dataUID, &docDataPtr) && docDataPtr) {
    ptr.attach(std::static_cast<MyType*>(docDataPtr)); // attach does not AddRef
}
```

Parameters

in	<i>inDataType</i>	The type of the document data requested
out	<i>outData</i>	An interface providing the requested data, or <code>nullptr</code> if the host does not support or cannot provide the requested data type. The reference count has been incremented on this object on behalf of the caller, so the caller must not add an additional reference count and must decrement the reference count on this object to release it. For information about which interface to expect for each requested data type, see the documentation for that data type.

Implemented in [AAX_V_SessionDocument](#).

The documentation for this class was generated from the following file:

- [AAX_I_SessionDocument.h](#)

14.129 AAX_I_SessionDocumentClient Class Reference

```
#include <AAX_I_SessionDocumentClient.h>
```

Inheritance diagram for `AAX_I_SessionDocumentClient`:

Collaboration diagram for `AAX_I_SessionDocumentClient`:

14.129.1 Description

Interface representing a client of the session document interface.

For example, a plug-in implementation that makes calls on the session document interface provided by the host.

Public Member Functions

- [ACF_DECLARE_STANDARD_UNKNOWN\(\)](#) `ACFMETHOD(InternalQueryInterface)(const acfIID &riid`
- [AAX_DELETE](#) (`AAX_I_SessionDocumentClient &operator=(const AAX_I_SessionDocumentClient &))`

Public Attributes

- void **ppvObjOut [override](#)

14.129.2 Member Function Documentation

14.129.2.1 ACF_DECLARE_STANDARD_UNKNOWN()

```
AAX_ISessionDocumentClient::ACF_DECLARE_STANDARD_UNKNOWN ( ) const &
```

14.129.2.2 AAX_DELETE()

```
AAX_ISessionDocumentClient::AAX_DELETE (
    AAX\_ISessionDocumentClient & operator = (const AAX\_ISessionDocumentClient &) )
```

14.129.3 Member Data Documentation

14.129.3.1 override

```
void** ppvObjOut AAX_ISessionDocumentClient::override
```

The documentation for this class was generated from the following file:

- [AAX_ISessionDocumentClient.h](#)

14.130 AAX_IString Class Reference

```
#include <AAX_IString.h>
```

Inheritance diagram for AAX_IString:

14.130.1 Description

A simple string container that can be passed across a binary boundary. This class, for simplicity, is not versioned and thus can never change.

For a real string implementation, see [AAX_CString](#), which inherits from this interface, but provides a much richer string interface.

This object is not versioned with ACF for a variety of reasons, but the biggest implication of that is that THIS INTERFACE CAN NEVER CHANGE!

Public Member Functions

- virtual [~AAX_IString](#) ()
- virtual uint32_t [Length](#) () const =0
- virtual uint32_t [MaxLength](#) () const =0
- virtual const char * [Get](#) () const =0
- virtual void [Set](#) (const char *iString)=0
- virtual [AAX_IString](#) & [operator=](#) (const [AAX_IString](#) &iOther)=0
- virtual [AAX_IString](#) & [operator=](#) (const char *iString)=0

14.130.2 Constructor & Destructor Documentation

14.130.2.1 ~AAX_IString()

```
virtual AAX_IString::~~AAX_IString ( ) [inline], [virtual]
```

Virtual Destructor

14.130.3 Member Function Documentation

14.130.3.1 Length()

```
virtual uint32_t AAX_IString::Length ( ) const [pure virtual]
```

Length methods

Implemented in [AAX_CString](#).

Referenced by [AAX::String2Binary\(\)](#).

Here is the caller graph for this function:

14.130.3.2 MaxLength()

```
virtual uint32_t AAX_IString::MaxLength ( ) const [pure virtual]
```

Implemented in [AAX_CString](#).

14.130.3.3 Get()

```
virtual const char* AAX_IString::Get ( ) const [pure virtual]
```

C string methods

Implemented in [AAX_CString](#).

Referenced by `AAX::IsEffectIDEqual()`, and `AAX::String2Binary()`.

Here is the caller graph for this function:

14.130.3.4 Set()

```
virtual void AAX_IString::Set (
    const char * iString ) [pure virtual]
```

Implemented in [AAX_CString](#).

14.130.3.5 operator=() [1/2]

```
virtual AAX\_IString& AAX_IString::operator= (
    const AAX\_IString & iOther ) [pure virtual]
```

Assignment operators

Implemented in [AAX_CString](#).

14.130.3.6 operator=() [2/2]

```
virtual AAX\_IString& AAX_IString::operator= (
    const char * iString ) [pure virtual]
```

Implemented in [AAX_CString](#).

The documentation for this class was generated from the following file:

- [AAX_IString.h](#)

14.131 AAX_ITaperDelegate< T > Class Template Reference

```
#include <AAX_ITaperDelegate.h>
```

Inheritance diagram for `AAX_ITaperDelegate< T >`:

Collaboration diagram for `AAX_ITaperDelegate< T >`:

14.131.1 Description

```
template<typename T>
class AAX_ITaperDelegate< T >
```

Classes for conversion to and from normalized parameter values.

Taper delegate interface template

Taper delegates are used to convert real parameter values to and from their normalized representations. All taper delegates implement the `AAX_ITaperDelegate<T>` interface template, which contains two conversion functions:

```
virtual T      NormalizedToReal(double normalizedValue) const = 0;
virtual double RealToNormalized(T realValue) const = 0;
```

In addition, tapers may incorporate logical value constraints via the following interface methods:

```
virtual T      GetMaximumValue() const = 0;
virtual T      GetMinimumValue() const = 0;
virtual T      ConstrainRealValue(T value) const = 0;
```

For more information, see the [AAX_ITaperDelegate](#) class documentation.

Public Member Functions

- virtual [AAX_ITaperDelegate * Clone](#) () const =0
Constructs and returns a copy of the taper delegate.
- virtual T [GetMaximumValue](#) () const =0
Returns the taper's maximum real value.
- virtual T [GetMinimumValue](#) () const =0
Returns the taper's minimum real value.
- virtual T [ConstrainRealValue](#) (T value) const =0
Applies a constraint to the value and returns the constrained value.
- virtual T [NormalizedToReal](#) (double normalizedValue) const =0
Converts a normalized value to a real value.
- virtual double [RealToNormalized](#) (T realValue) const =0
Normalizes a real parameter value.

14.131.2 Member Function Documentation

14.131.2.1 Clone()

```
template<typename T >
virtual AAX_ITaperDelegate* AAX_ITaperDelegate< T >::Clone ( ) const [pure virtual]
```

Constructs and returns a copy of the taper delegate.

In general, this method's implementation can use a simple copy constructor:

```
template <typename T>
AAX_CSubclassTaperDelegate<T>* AAX_CSubclassTaperDelegate<T>::Clone() const
{
    return new AAX_CSubclassTaperDelegate(*this);
}
```

Implemented in [AAX_CStateTaperDelegate< T >](#), [AAX_CRangeTaperDelegate< T, RealPrecision >](#), [AAX_CPieceWiseLinearTaperDelegate< T, RealPrecision >](#), [AAX_CLogTaperDelegate< T, RealPrecision >](#), [AAX_CLinearTaperDelegate< T, RealPrecision >](#), and [AAX_CBinaryTaperDelegate](#)

Referenced by `AAX_CParameter< T >::SetTaperDelegate()`.

Here is the caller graph for this function:

14.131.2.2 GetMaximumValue()

```
template<typename T >
virtual T AAX_ITaperDelegate< T >::GetMaximumValue ( ) const [pure virtual]
```

Returns the taper's maximum real value.

Implemented in [AAX_CStateTaperDelegate< T >](#), [AAX_CRangeTaperDelegate< T, RealPrecision >](#), [AAX_CPieceWiseLinearTaper](#), [AAX_CLogTaperDelegate< T, RealPrecision >](#), [AAX_CLinearTaperDelegate< T, RealPrecision >](#), and [AAX_CBinaryTaperDelegate](#).

14.131.2.3 GetMinimumValue()

```
template<typename T >
virtual T AAX_ITaperDelegate< T >::GetMinimumValue ( ) const [pure virtual]
```

Returns the taper's minimum real value.

Implemented in [AAX_CStateTaperDelegate< T >](#), [AAX_CRangeTaperDelegate< T, RealPrecision >](#), [AAX_CPieceWiseLinearTaper](#), [AAX_CLogTaperDelegate< T, RealPrecision >](#), [AAX_CLinearTaperDelegate< T, RealPrecision >](#), and [AAX_CBinaryTaperDelegate](#).

14.131.2.4 ConstrainRealValue()

```
template<typename T >
virtual T AAX_ITaperDelegate< T >::ConstrainRealValue (
    T value ) const [pure virtual]
```

Applies a constraint to the value and returns the constrained value.

This method is useful if the taper requires a constraint beyond simple minimum and maximum real value limits.

Note

This is the function that should actually enforces the constraints in [NormalizeToReal\(\)](#) and [RealToNormalized\(\)](#).

Parameters

in	<i>value</i>	The unconstrained value
----	--------------	-------------------------

Implemented in [AAX_CStateTaperDelegate< T >](#), [AAX_CRangeTaperDelegate< T, RealPrecision >](#), [AAX_CPieceWiseLinearTaper](#), [AAX_CLogTaperDelegate< T, RealPrecision >](#), [AAX_CLinearTaperDelegate< T, RealPrecision >](#), and [AAX_CBinaryTaperDelegate](#).

14.131.2.5 NormalizedToReal()

```
template<typename T >
virtual T AAX_ITaperDelegate< T >::NormalizedToReal (
    double normalizedValue ) const [pure virtual]
```

Converts a normalized value to a real value.

This is where the actual taper algorithm is implemented.

This function should perform the exact inverse of [RealToNormalized\(\)](#), to within the roundoff precision of the individual taper implementation.

Parameters

in	<i>normalizedValue</i>	The normalized value that will be converted
----	------------------------	---

Implemented in [AAX_CStateTaperDelegate< T >](#), [AAX_CRangeTaperDelegate< T, RealPrecision >](#), [AAX_CPieceWiseLinearTaper](#), [AAX_CLogTaperDelegate< T, RealPrecision >](#), [AAX_CLinearTaperDelegate< T, RealPrecision >](#), and [AAX_CBinaryTaperDelegate](#)

14.131.2.6 RealToNormalized()

```
template<typename T >
virtual double AAX_ITaperDelegate< T >::RealToNormalized (
    T realValue ) const [pure virtual]
```

Normalizes a real parameter value.

This is where the actual taper algorithm is implemented.

This function should perform the exact inverse of [NormalizedToReal\(\)](#), to within the roundoff precision of the individual taper implementation.

Parameters

in	<i>realValue</i>	The real parameter value that will be normalized
----	------------------	--

Implemented in [AAX_CStateTaperDelegate< T >](#), [AAX_CRangeTaperDelegate< T, RealPrecision >](#), [AAX_CPieceWiseLinearTaper](#), [AAX_CLogTaperDelegate< T, RealPrecision >](#), [AAX_CLinearTaperDelegate< T, RealPrecision >](#), and [AAX_CBinaryTaperDelegate](#)

The documentation for this class was generated from the following file:

- [AAX_ITaperDelegate.h](#)

14.132 AAX_ITaperDelegateBase Class Reference

```
#include <AAX_ITaperDelegate.h>
```

Inheritance diagram for AAX_ITaperDelegateBase:

14.132.1 Description

Defines the taper conversion behavior for a parameter.

:Internal to the AAX SDK

This interface represents a delegate class to be used in conjunction with [AAX_IParameter](#). [AAX_IParameter](#) delegates all conversion operations between normalized and real parameter values to classes that meet this interface. You can think of [AAX_ITaperDelegate](#) subclasses as simple taper conversion routines that enable a specific taper or range conversion function on an arbitrary parameter.

To demonstrate the use of this interface, we will examine a simple call routine into a parameter:

1. The host application calls into the plug-in's [AAX_CParameterManager](#) with a Parameter ID and a new normalized parameter value. This new value could be coming from an automation lane, a control surface, or any other parameter control; from the plug-in's perspective, these are all identical.
2. The [AAX_CParameterManager](#) finds the specified [AAX_CParameter](#) and calls [AAX_IParameter::SetNormalizedValue\(\)](#) on that parameter
3. [AAX_IParameter::SetNormalizedValue\(\)](#) results in a call into the parameter's concrete taper delegate to convert the normalized value to a real value.

Using this pattern, the parameter manager is able to use real parameter values without actually knowing how to perform the conversion between normalized and real values.

The inverse of the above example can also happen, e.g. when a control is updated from within the data model. In this case, the parameter can call into its concrete taper delegate in order to normalize the updated value, which can then be passed on to any observers that require normalized values, such as the host app.

For more information about the parameter manager, see the [Parameter Manager](#) documentation page.

Public Member Functions

- virtual [~AAX_ITaperDelegateBase](#) ()
Virtual destructor.

14.132.2 Constructor & Destructor Documentation

14.132.2.1 ~AAX_ITaperDelegateBase()

```
virtual AAX_ITaperDelegateBase::~AAX_ITaperDelegateBase ( ) [inline], [virtual]
```

Virtual destructor.

Note

This destructor MUST be virtual to prevent memory leaks.

The documentation for this class was generated from the following file:

- [AAX_ITaperDelegate.h](#)

14.133 AAX_ITask Class Reference

```
#include <AAX_ITask.h>
```

Inheritance diagram for AAX_ITask:

14.133.1 Description

Interface representing a request to perform a task.

:Implemented by the AAX Host

Used by the [task agent](#).

This interface describes a task request and provides a way for the agent to express one or more results of the task as well as the progress of the task.

This interface is open-ended for both inputs and outputs. The host and agent must use common definitions for specific task types, their possible arguments, and the expected results.

Public Member Functions

- virtual [~AAX_ITask](#) ()=default
- virtual [AAX_Result](#) [GetType](#) ([AAX_CTypeID](#) *oType) const =0
- virtual [AAX_IACFDataBuffer](#) const * [GetArgumentOfType](#) ([AAX_CTypeID](#) iType) const =0
- virtual [AAX_Result](#) [SetProgress](#) (float iProgress)=0
- virtual float [GetProgress](#) () const =0
- virtual [AAX_Result](#) [AddResult](#) ([AAX_IACFDataBuffer](#) const *iResult)=0
 - Attach result data to this task.*
- virtual [AAX_ITask](#) * [SetDone](#) ([AAX_TaskCompletionStatus](#) iStatus)=0
 - Inform the host that the task is completed.*

14.133.2 Constructor & Destructor Documentation

14.133.2.1 ~AAX_ITask()

```
virtual AAX_ITask::~~AAX_ITask ( ) [virtual], [default]
```

14.133.3 Member Function Documentation

14.133.3.1 GetType()

```
virtual AAX_Result AAX_ITask::GetType (
    AAX_CTypeID * oType ) const [pure virtual]
```

An identifier defining the type of the requested task

Parameters

out	<i>oType</i>	The type of this task request
-----	--------------	-------------------------------

Implemented in [AAX_VTask](#).

14.133.3.2 GetArgumentOfType()

```
virtual AAX_IACFDataBuffer const* AAX_ITask::GetArgumentOfType (
    AAX_CTypeID iType ) const [pure virtual]
```

Additional information defining the request, depending on the task type

Parameters

in	<i>iType</i>	The type of argument requested. Possible argument types, if any, and the resulting data buffer format must be defined per task type.
----	--------------	--

Returns

The requested argument data, or nullptr. This data buffer's type ID is expected to match *iType*. The caller takes ownership of this object.

Implemented in [AAX_VTask](#).

14.133.3.3 SetProgress()

```
virtual AAX_Result AAX_ITask::SetProgress (
    float iProgress ) [pure virtual]
```

Inform the host about the current status of the task

Parameters

in	<i>iProgress</i>	A value between 0 (no progress) and 1 (complete)
----	------------------	--

Implemented in [AAX_VTask](#).

14.133.3.4 GetProgress()

```
virtual float AAX_ITask::GetProgress ( ) const [pure virtual]
```

Returns the current progress

Implemented in [AAX_VTask](#).

14.133.3.5 AddResult()

```
virtual AAX_Result AAX_ITask::AddResult (
    AAX_IACFDataBuffer const * iResult ) [pure virtual]
```

Attach result data to this task.

This can be called multiple times to add multiple types of results to a single task.

The host may process the result data immediately or may wait for the task to complete.

The plug-in is expected to release the data buffer upon making this call. At a minimum, the data buffer must not be changed after this call is made. See `ACFPtr::inArg()`

Parameters

in	<i>iResult</i>	A buffer containing the result data. Expected result types, if any, and their data buffer format must be defined per task type.
----	----------------	---

Implemented in [AAX_VTask](#).

14.133.3.6 SetDone()

```
virtual AAX_ITask* AAX_ITask::SetDone (
```

```
AAX_TaskCompletionStatus iStatus ) [pure virtual]
```

Inform the host that the task is completed.

If successful, returns a null pointer. Otherwise, returns a pointer back to the same object. This is the expected usage pattern:

```
// release the task on success, retain it on failure
myTask = myTask->SetDone(status);
```

Parameters

in	<i>iStatus</i>	The final status of the task. This indicates to the host whether or not the task was performed as requested.
----	----------------	--

Implemented in [AAX_VTask](#).

The documentation for this class was generated from the following file:

- [AAX_ITask.h](#)

14.134 AAX_ITaskAgent Class Reference

```
#include <AAX_ITaskAgent.h>
```

Inheritance diagram for AAX_ITaskAgent:

Collaboration diagram for AAX_ITaskAgent:

14.134.1 Description

Interface for a component that accepts task requests.

:Implemented by the Plug-In

The task agent is expected to complete the requested tasks asynchronously and to provide progress and completion details via calls on the [AAX_IACFTask](#) interface as the tasks proceed.

See also

[AAX_ITask](#)

Public Member Functions

- [ACF_DECLARE_STANDARD_UNKNOWN](#) () ACFMETHOD(InternalQueryInterface)(const [acflID](#) &riid
- [AAX_DELETE](#) (AAX_ITaskAgent &operator=(const [AAX_ITaskAgent](#) &))

Public Attributes

- void **ppvObjOut [AAX_OVERRIDE](#)

14.134.2 Member Function Documentation

14.134.2.1 ACF_DECLARE_STANDARD_UNKNOWN()

```
AAX_ITaskAgent::ACF_DECLARE_STANDARD_UNKNOWN ( ) const &
```

14.134.2.2 AAX_DELETE()

```
AAX_ITaskAgent::AAX_DELETE (
    AAX_ITaskAgent & operator = (const AAX_ITaskAgent &) )
```

14.134.3 Member Data Documentation

14.134.3.1 AAX_OVERRIDE

```
void** ppvObjOut AAX_ITaskAgent::AAX_OVERRIDE
```

The documentation for this class was generated from the following file:

- [AAX_ITaskAgent.h](#)

14.135 AAX_ITransport Class Reference

```
#include <AAX_ITransport.h>
```

Inheritance diagram for AAX_ITransport:

14.135.1 Description

Interface to information about the host's transport state.

:Implemented by the AAX Host

Plug-ins that use this interface should describe [AAX_eProperty_UsesTransport](#) as 1

Classes that inherit from [AAX_CEffectParameters](#) or [AAX_CEffectGUI](#) can use [AAX_CEffectParameters::Transport\(\)](#) / [AAX_CEffectGUI::Transport\(\)](#) to access this interface. This interface is used as a local interface to the [AAX_VTransport](#) versioned implementation, which dispatches calls to the appropriate host-supplied versioned transport interface depending on which features are supported by the host. See [AAX_CEffectParameters::Initialize\(\)](#) for an example.

A copy of this interface may also be obtained directly from the host using [AAX_IMIDINode::GetTransport\(\)](#). However, in this case the interface is not versioned, so the host and the plugin may not agree on the interface. This can lead to undefined behavior. See the documentation at [AAX_IMIDINode::GetTransport\(\)](#) for more information.

Public Member Functions

- virtual `~AAX_ITransport ()`
Virtual destructor.
- virtual `AAX_Result GetCurrentTempo (double *TempoBPM) const =0`
CALL: Gets the current tempo.
- virtual `AAX_Result GetCurrentMeter (int32_t *MeterNumerator, int32_t *MeterDenominator) const =0`
CALL: Gets the current meter.
- virtual `AAX_Result IsTransportPlaying (bool *isPlaying) const =0`
CALL: Indicates whether or not the transport is playing back.
- virtual `AAX_Result GetCurrentTickPosition (int64_t *TickPosition) const =0`
CALL: Gets the current tick position.
- virtual `AAX_Result GetCurrentLoopPosition (bool *bLooping, int64_t *LoopStartTick, int64_t *LoopEndTick) const =0`
CALL: Gets current information on loop playback.
- virtual `AAX_Result GetCurrentNativeSampleLocation (int64_t *SampleLocation) const =0`
CALL: Gets the current playback location of the native audio engine.
- virtual `AAX_Result GetCustomTickPosition (int64_t *oTickPosition, int64_t iSampleLocation) const =0`
CALL: Given an absolute sample position, gets the corresponding tick position.
- virtual `AAX_Result GetBarBeatPosition (int32_t *Bars, int32_t *Beats, int64_t *DisplayTicks, int64_t SampleLocation) const =0`
CALL: Given an absolute sample position, gets the corresponding bar and beat position.
- virtual `AAX_Result GetTicksPerQuarter (uint32_t *ticks) const =0`
CALL: Retrieves the number of ticks per quarter note.
- virtual `AAX_Result GetCurrentTicksPerBeat (uint32_t *ticks) const =0`
CALL: Retrieves the number of ticks per beat.
- virtual `AAX_Result GetTimelineSelectionStartPosition (int64_t *oSampleLocation) const =0`
CALL: Retrieves the absolute sample position of the beginning of the current transport selection.
- virtual `AAX_Result GetTimeCodeInfo (AAX_EFrameRate *oFrameRate, int32_t *oOffset) const =0`
CALL: Retrieves the current time code frame rate and offset.
- virtual `AAX_Result GetFeetFramesInfo (AAX_EFeetFramesRate *oFeetFramesRate, int64_t *oOffset) const =0`
CALL: Retrieves the current timecode feet/frames rate and offset.
- virtual `AAX_Result IsMetronomeEnabled (int32_t *isEnabled) const =0`
Sets isEnabled to true if the metronome is enabled.
- virtual `AAX_Result GetHDTimeCodeInfo (AAX_EFrameRate *oHDFrameRate, int64_t *oHDOffset) const =0`
CALL: Retrieves the current HD time code frame rate and offset.
- virtual `AAX_Result RequestTransportStart ()=0`
CALL: Request that the host transport start playback.
- virtual `AAX_Result RequestTransportStop ()=0`
CALL: Request that the host transport stop playback.
- virtual `AAX_Result GetTimelineSelectionEndPosition (int64_t *oSampleLocation) const =0`
CALL: Retrieves the absolute sample position of the end of the current transport selection.
- virtual `AAX_Result GetKeySignature (int64_t iSampleLocation, uint32_t *oKeySignature) const =0`
CALL: Retrieves the key signature at a sample location.

14.135.2 Constructor & Destructor Documentation

14.135.2.1 ~AAX_ITransport()

```
virtual AAX_ITransport::~~AAX_ITransport ( ) [inline], [virtual]
```

Virtual destructor.

Note

This destructor MUST be virtual to prevent memory leaks.

14.135.3 Member Function Documentation

14.135.3.1 GetCurrentTempo()

```
virtual AAX_Result AAX_ITransport::GetCurrentTempo (
    double * TempoBPM ) const [pure virtual]
```

CALL: Gets the current tempo.

Returns the tempo corresponding to the current position of the transport counter

Note

The resolution of the tempo returned here is based on the host's tempo resolution, so it will match the tempo displayed in the host. Use [GetCurrentTicksPerBeat\(\)](#) to calculate the tempo resolution note.

Parameters

out	<i>TempoBPM</i>	The current tempo in beats per minute
-----	-----------------	---------------------------------------

Implemented in [AAX_VTransport](#).

14.135.3.2 GetCurrentMeter()

```
virtual AAX_Result AAX_ITransport::GetCurrentMeter (
    int32_t * MeterNumerator,
    int32_t * MeterDenominator ) const [pure virtual]
```

CALL: Gets the current meter.

Returns the meter corresponding to the current position of the transport counter

Parameters

out	<i>MeterNumerator</i>	The numerator portion of the meter
out	<i>MeterDenominator</i>	The denominator portion of the meter

Implemented in [AAX_VTransport](#).

14.135.3.3 IsTransportPlaying()

```
virtual AAX_Result AAX_ITransport::IsTransportPlaying (
    bool * isPlaying ) const [pure virtual]
```

CALL: Indicates whether or not the transport is playing back.

Parameters

out	<i>isPlaying</i>	true if the transport is currently in playback
-----	------------------	--

Implemented in [AAX_VTransport](#).

14.135.3.4 GetCurrentTickPosition()

```
virtual AAX_Result AAX_ITransport::GetCurrentTickPosition (
    int64_t * TickPosition ) const [pure virtual]
```

CALL: Gets the current tick position.

Returns the current tick position corresponding to the current transport position. One "Tick" is represented here as 1/960000 of a quarter note. That is, there are 960,000 of these ticks in a quarter note.

Host Compatibility Notes The tick resolution here is different than that of the tick displayed in Pro Tools. "Display ticks" (as they are called) are 1/960 of a quarter note.

Parameters

out	<i>TickPosition</i>	The tick position value
-----	---------------------	-------------------------

Implemented in [AAX_VTransport](#).

14.135.3.5 GetCurrentLoopPosition()

```
virtual AAX_Result AAX_ITransport::GetCurrentLoopPosition (
    bool * bLooping,
    int64_t * LoopStartTick,
    int64_t * LoopEndTick ) const [pure virtual]
```

CALL: Gets current information on loop playback.

Host Compatibility Notes This does not indicate anything about the status of the "Loop Record" option. Even when the host is configured to loop playback, looping may not occur if certain conditions are not met (i.e. the length of the selection is too short)

Parameters

out	<i>bLooping</i>	true if the host is configured to loop playback
out	<i>LoopStartTick</i>	The starting tick position of the selection being looped (see GetCurrentTickPosition())
out	<i>LoopEndTick</i>	The ending tick position of the selection being looped (see GetCurrentTickPosition())

Implemented in [AAX_VTransport](#).

14.135.3.6 GetCurrentNativeSampleLocation()

```
virtual AAX_Result AAX_ITransport::GetCurrentNativeSampleLocation (
    int64_t * SampleLocation ) const [pure virtual]
```

CALL: Gets the current playback location of the native audio engine.

When called from a ProcessProc render callback, this method will provide the absolute sample location at the beginning of the callback's audio buffers.

When called from [AAX_IEffectParameters::RenderAudio_Hybrid\(\)](#), this method will provide the absolute sample location for the samples in the method's **output** audio buffers. To calculate the absolute sample location for the samples in the method's input buffers (i.e. the timeline location where the samples originated) subtract the value provided by [AAX_IController::GetHybridSignalLatency\(\)](#) from this value.

When called from a non-real-time thread, this method will provide the current location of the samples being processed by the plug-in's ProcessProc on its real-time processing thread.

Note

This method only returns a value during playback. It cannot be used to determine, e.g., the location of the timeline selector while the host is not in playback.

Parameters

out	<i>SampleLocation</i>	Absolute sample location of the first sample in the current native processing buffer
-----	-----------------------	--

Implemented in [AAX_VTransport](#).

14.135.3.7 GetCustomTickPosition()

```
virtual AAX_Result AAX_ITransport::GetCustomTickPosition (
    int64_t * oTickPosition,
    int64_t iSampleLocation ) const [pure virtual]
```

CALL: Given an absolute sample position, gets the corresponding tick position.

Host Compatibility Notes There is a minor performance cost associated with using this API in Pro Tools. It should not be used excessively without need.

Parameters

out	<i>oTickPosition</i>	the timeline tick position corresponding to <code>iSampleLocation</code>
in	<i>iSampleLocation</i>	An absolute sample location (see GetCurrentNativeSampleLocation())

Implemented in [AAX_VTransport](#).

14.135.3.8 GetBarBeatPosition()

```
virtual AAX_Result AAX_ITransport::GetBarBeatPosition (
    int32_t * Bars,
    int32_t * Beats,
    int64_t * DisplayTicks,
    int64_t SampleLocation ) const [pure virtual]
```

CALL: Given an absolute sample position, gets the corresponding bar and beat position.

Host Compatibility Notes There is a minor performance cost associated with using this API in Pro Tools. It should not be used excessively without need.

Parameters

out	<i>Bars</i>	The bar corresponding to <code>SampleLocation</code>
out	<i>Beats</i>	The beat corresponding to <code>SampleLocation</code>
out	<i>DisplayTicks</i>	The ticks corresponding to <code>SampleLocation</code>
in	<i>SampleLocation</i>	An absolute sample location (see GetCurrentNativeSampleLocation())

Implemented in [AAX_VTransport](#).

14.135.3.9 GetTicksPerQuarter()

```
virtual AAX_Result AAX_ITransport::GetTicksPerQuarter (
    uint32_t * ticks ) const [pure virtual]
```

CALL: Retrieves the number of ticks per quarter note.

Parameters

out	<i>ticks</i>	
-----	--------------	--

Implemented in [AAX_VTransport](#).

14.135.3.10 GetCurrentTicksPerBeat()

```
virtual AAX_Result AAX_ITransport::GetCurrentTicksPerBeat (
    uint32_t * ticks ) const [pure virtual]
```

CALL: Retrieves the number of ticks per beat.

Parameters

out	<i>ticks</i>	
-----	--------------	--

Implemented in [AAX_VTransport](#).

14.135.3.11 GetTimelineSelectionStartPosition()

```
virtual AAX_Result AAX_ITransport::GetTimelineSelectionStartPosition (
    int64_t * oSampleLocation ) const [pure virtual]
```

CALL: Retrieves the absolute sample position of the beginning of the current transport selection.

Note

This method is part of the [version 2 transport interface](#)

Parameters

out	<i>oSampleLocation</i>	
-----	------------------------	--

Implemented in [AAX_VTransport](#).

14.135.3.12 GetTimeCodeInfo()

```
virtual AAX_Result AAX_ITransport::GetTimeCodeInfo (
    AAX_EFrameRate * oFrameRate,
    int32_t * oOffset ) const [pure virtual]
```

CALL: Retrieves the current time code frame rate and offset.

Note

This method is part of the [version 2 transport interface](#)

Parameters

out	<i>oFrameRate</i>	
out	<i>oOffset</i>	

Implemented in [AAX_VTransport](#).

14.135.3.13 GetFeetFramesInfo()

```
virtual AAX_Result AAX_ITransport::GetFeetFramesInfo (
    AAX_EFeetFramesRate * oFeetFramesRate,
    int64_t * oOffset ) const [pure virtual]
```

CALL: Retrieves the current timecode feet/frames rate and offset.

Note

This method is part of the [version 2 transport interface](#)

Parameters

out	<i>oFeetFramesRate</i>	
out	<i>oOffset</i>	

Implemented in [AAX_VTransport](#).

14.135.3.14 IsMetronomeEnabled()

```
virtual AAX_Result AAX_ITransport::IsMetronomeEnabled (
    int32_t * isEnabled ) const [pure virtual]
```

Sets isEnabled to true if the metronome is enabled.

Note

This method is part of the [version 2 transport interface](#)

Parameters

out	<i>isEnabled</i>	
-----	------------------	--

Implemented in [AAX_VTransport](#).

14.135.3.15 GetHDTIMECodeInfo()

```
virtual AAX_Result AAX_ITransport::GetHDTIMECodeInfo (
    AAX_EFrameRate * oHDFrameRate,
    int64_t * oHDOffset ) const [pure virtual]
```

CALL: Retrieves the current HD time code frame rate and offset.

Note

This method is part of the [version 3 transport interface](#)

Parameters

out	<i>oHDFrameRate</i>	
out	<i>oHDOffset</i>	

Implemented in [AAX_VTransport](#).

14.135.3.16 RequestTransportStart()

```
virtual AAX_Result AAX_ITransport::RequestTransportStart ( ) [pure virtual]
```

CALL: Request that the host transport start playback.

Note

This method is part of the [AAX_IACFTransportControl](#) interface

Implemented in [AAX_VTransport](#).

14.135.3.17 RequestTransportStop()

```
virtual AAX_Result AAX_ITransport::RequestTransportStop ( ) [pure virtual]
```

CALL: Request that the host transport stop playback.

Note

This method is part of the [AAX_IACFTransportControl](#) interface

Implemented in [AAX_VTransport](#).

14.135.3.18 GetTimelineSelectionEndPosition()

```
virtual AAX_Result AAX_ITransport::GetTimelineSelectionEndPosition (
    int64_t * oSampleLocation ) const [pure virtual]
```

CALL: Retrieves the absolute sample position of the end of the current transport selection.

Note

This method is part of the [version 4 transport interface](#)

Parameters

out	<i>oSampleLocation</i>	
-----	------------------------	--

Implemented in [AAX_VTransport](#).

14.135.3.19 GetKeySignature()

```
virtual AAX_Result AAX_ITransport::GetKeySignature (
    int64_t iSampleLocation,
    uint32_t * oKeySignature ) const [pure virtual]
```

CALL: Retrieves the key signature at a sample location.

The signature is provided as a bitfield:

- 31-20: Chromatic scale elements, ordered MSB (root) to LSB
- 19-4: (Reserved)
- 3-0: Root note (C natural = 0)

For example

```
* D# Major
*   Ionian                                D#
* 0b 101011010101 0000 00000000 0000 0011
*
* E Phrygian
*   Phrygian                             E
* 0b 110101011010 0000 00000000 0000 0100
*
* Chromatic
*   Chromatic                             C
* 0b 111111111111 0000 00000000 0000 0000
*
```

Implemented in [AAX_VTransport](#).

The documentation for this class was generated from the following file:

- [AAX_ITransport.h](#)

14.136 AAX_IViewContainer Class Reference

```
#include <AAX_IViewContainer.h>
```

Inheritance diagram for AAX_IViewContainer:

14.136.1 Description

Interface for the AAX host's view of a single instance of an effect. Used both by clients of the AAX host and by effect components.

Implemented by the AAX Host

Public Member Functions

- virtual [~AAX_IViewContainer](#) (void)

View and GUI state queries

- virtual int32_t [GetType](#) ()=0
Returns the raw view type as one of [AAX_EViewContainer_Type](#).
- virtual void * [GetPtr](#) ()=0
Returns a pointer to the raw view.
- virtual [AAX_Result](#) [GetModifiers](#) (uint32_t *outModifiers)=0
Queries the host for the current [modifier keys](#).

View change requests

- virtual [AAX_Result](#) [SetViewSize](#) ([AAX_Point](#) &inSize)=0
Request a change to the main view size.

Host event handlers

These methods are used to pass plug-in GUI events to the host for handling. Events should always be passed on in this way when there is a possibility of the host overriding the event with its own behavior.

For example, in Pro Tools a command-control-option-click on any automatable plug-in parameter editor should bring up that parameter's automation pop-up menu, and a control-right click should display the parameter's automation lane in the Pro Tools Edit window. In order for Pro Tools to handle these events, the plug-in must pass them on using [HandleParameterMouseDown\(\)](#)

For each of these methods:

- [AAX_SUCCESS](#) is returned if the event was successfully handled by the host. In most cases, no further action will be required from the plug-in after the host successfully handles an event.
- [AAX_ERROR_UNIMPLEMENTED](#) is returned if the event was not handled by the host. In this case, the plug-in should perform its own event handling.
- virtual [AAX_Result](#) [HandleParameterMouseDown](#) ([AAX_CParamID](#) inParamID, uint32_t inModifiers)=0
Alert the host to a mouse down event.
- virtual [AAX_Result](#) [HandleParameterMouseDrag](#) ([AAX_CParamID](#) inParamID, uint32_t inModifiers)=0
Alert the host to a mouse drag event.
- virtual [AAX_Result](#) [HandleParameterMouseUp](#) ([AAX_CParamID](#) inParamID, uint32_t inModifiers)=0
Alert the host to a mouse up event.
- virtual [AAX_Result](#) [HandleParameterMouseEnter](#) ([AAX_CParamID](#) inParamID, uint32_t inModifiers)=0
Alert the host to a mouse enter event to the parameter's control.
- virtual [AAX_Result](#) [HandleParameterMouseExit](#) ([AAX_CParamID](#) inParamID, uint32_t inModifiers)=0
Alert the host to a mouse exit event from the parameter's control.
- virtual [AAX_Result](#) [HandleMultipleParametersMouseDown](#) (const [AAX_CParamID](#) *inParamIDs, uint32_t inNumOfParams, uint32_t inModifiers)=0
Alert the host to a mouse down event.
- virtual [AAX_Result](#) [HandleMultipleParametersMouseDrag](#) (const [AAX_CParamID](#) *inParamIDs, uint32_t inNumOfParams, uint32_t inModifiers)=0
Alert the host to a mouse drag event.
- virtual [AAX_Result](#) [HandleMultipleParametersMouseUp](#) (const [AAX_CParamID](#) *inParamIDs, uint32_t inNumOfParams, uint32_t inModifiers)=0
Alert the host to a mouse up event.

14.136.2 Constructor & Destructor Documentation

14.136.2.1 ~AAX_IViewContainer()

```
virtual AAX_IViewContainer::~~AAX_IViewContainer (
    void ) [inline], [virtual]
```

14.136.3 Member Function Documentation

14.136.3.1 GetType()

```
virtual int32_t AAX_IViewContainer::GetType ( ) [pure virtual]
```

Returns the raw view type as one of [AAX_EViewContainer_Type](#).

Implemented in [AAX_VViewContainer](#).

14.136.3.2 GetPtr()

```
virtual void* AAX_IViewContainer::GetPtr ( ) [pure virtual]
```

Returns a pointer to the raw view.

Implemented in [AAX_VViewContainer](#).

14.136.3.3 GetModifiers()

```
virtual AAX_Result AAX_IViewContainer::GetModifiers (
    uint32_t * outModifiers ) [pure virtual]
```

Queries the host for the current [modifier keys](#).

This method returns a bit mask with bits set for each of the currently active modifier keys. This method does not return the state of the [AAX_eModifiers_SecondaryButton](#).

Host Compatibility Notes Although this method allows plug-ins to acquire the current state of the Windows key (normally blocked by Pro Tools), plug-ins should not use key combinations that require this key.

Parameters

out	<i>outModifiers</i>	Current modifiers as a bitmask of AAX_EModifiers
-----	---------------------	--

Implemented in [AAX_VViewContainer](#).

14.136.3.4 SetViewSize()

```
virtual AAX\_Result AAX_IViewContainer::SetViewSize (
    AAX\_Point & inSize ) [pure virtual]
```

Request a change to the main view size.

Note

- For compatibility with the smallest supported displays, plug-in GUI dimensions should not exceed 749x617 pixels, or 749x565 pixels for plug-ins with sidechain support.

Parameters

in	<i>inSize</i>	The new size to which the plug-in view should be set
----	---------------	--

Implemented in [AAX_VViewContainer](#).

14.136.3.5 HandleParameterMouseDown()

```
virtual AAX\_Result AAX_IViewContainer::HandleParameterMouseDown (
    AAX\_CParamID inParamID,
    uint32_t inModifiers ) [pure virtual]
```

Alert the host to a mouse down event.

Parameters

in	<i>inParamID</i>	ID of the parameter whose control is being edited
in	<i>inModifiers</i>	A bitmask of AAX_EModifiers values

Implemented in [AAX_VViewContainer](#).

14.136.3.6 HandleParameterMouseDrag()

```
virtual AAX\_Result AAX_IViewContainer::HandleParameterMouseDrag (
    AAX\_CParamID inParamID,
    uint32_t inModifiers ) [pure virtual]
```

Alert the host to a mouse drag event.

Warning

The host may return [AAX_ERROR_UNIMPLEMENTED](#) for this event even if the host did handle the corresponding mouse down event. A plug-in should ignore any following mouse drag and mouse up events that correspond to a host-managed mouse down event. ([PTSW-195209](#) / [PT-218474](#))

Parameters

in	<i>inParamID</i>	ID of the parameter whose control is being edited
in	<i>inModifiers</i>	A bitmask of AAX_EModifiers values

Implemented in [AAX_VViewContainer](#).

14.136.3.7 HandleParameterMouseUp()

```
virtual AAX_Result AAX_IViewContainer::HandleParameterMouseUp (
    AAX_CParamID inParamID,
    uint32_t inModifiers ) [pure virtual]
```

Alert the host to a mouse up event.

Warning

The host may return [AAX_ERROR_UNIMPLEMENTED](#) for this event even if the host did handle the corresponding mouse down event. A plug-in should ignore any following mouse drag and mouse up events that correspond to a host-managed mouse down event. ([PTSW-195209](#) / [PT-218474](#))

Parameters

in	<i>inParamID</i>	ID of the parameter whose control is being edited
in	<i>inModifiers</i>	A bitmask of AAX_EModifiers values

Implemented in [AAX_VViewContainer](#).

14.136.3.8 HandleParameterMouseEnter()

```
virtual AAX_Result AAX_IViewContainer::HandleParameterMouseEnter (
    AAX_CParamID inParamID,
    uint32_t inModifiers ) [pure virtual]
```

Alert the host to a mouse enter event to the parameter's control.

Parameters

in	<i>inParamID</i>	ID of the parameter whose control is being entered
in	<i>inModifiers</i>	A bitmask of AAX_EModifiers values

Returns AAX_SUCCESS if event was processed successfully, otherwise an AAX_ERROR code

Implemented in [AAX_VViewContainer](#).

14.136.3.9 HandleParameterMouseExit()

```
virtual AAX_Result AAX_IViewContainer::HandleParameterMouseExit (
    AAX_CParamID inParamID,
    uint32_t inModifiers ) [pure virtual]
```

Alert the host to a mouse exit event from the parameter's control.

Parameters

in	<i>inParamID</i>	ID of the parameter whose control is being exited
in	<i>inModifiers</i>	A bitmask of AAX_EModifiers values

Returns AAX_SUCCESS if event was processed successfully, otherwise an AAX_ERROR code

Implemented in [AAX_VViewContainer](#).

14.136.3.10 HandleMultipleParametersMouseDown()

```
virtual AAX_Result AAX_IViewContainer::HandleMultipleParametersMouseDown (
    const AAX_CParamID * inParamIDs,
    uint32_t inNumOfParams,
    uint32_t inModifiers ) [pure virtual]
```

Alert the host to a mouse down event.

Parameters

in	<i>inParamIDs</i>	IDs of the parameters that belong to the same GUI element whose controls are being edited
in	<i>inNumOfParams</i>	Number of parameter IDS
in	<i>inModifiers</i>	A bitmask of AAX_EModifiers values

Implemented in [AAX_VViewContainer](#).

14.136.3.11 HandleMultipleParametersMouseDown()

```
virtual AAX_Result AAX_IViewContainer::HandleMultipleParametersMouseDown (
    const AAX_CParamID * inParamIDs,
    uint32_t inNumOfParams,
    uint32_t inModifiers ) [pure virtual]
```

Alert the host to a mouse drag event.

Warning

The host may return [AAX_ERROR_UNIMPLEMENTED](#) for this event even if the host did handle the corresponding mouse down event. A plug-in should ignore any following mouse drag and mouse up events that correspond to a host-managed mouse down event. ([PTSW-195209](#) / [PT-218474](#))

Parameters

in	<i>inParamIDs</i>	IDs of the parameters that belong to the same GUI element whose controls are being edited
in	<i>inNumOfParams</i>	Number of parameter IDS
in	<i>inModifiers</i>	A bitmask of AAX_EModifiers values

Implemented in [AAX_VViewContainer](#).

14.136.3.12 HandleMultipleParametersMouseUp()

```
virtual AAX_Result AAX_IViewContainer::HandleMultipleParametersMouseUp (
    const AAX_CParamID * inParamIDs,
    uint32_t inNumOfParams,
    uint32_t inModifiers ) [pure virtual]
```

Alert the host to a mouse up event.

Warning

The host may return [AAX_ERROR_UNIMPLEMENTED](#) for this event even if the host did handle the corresponding mouse down event. A plug-in should ignore any following mouse drag and mouse up events that correspond to a host-managed mouse down event. ([PTSW-195209](#) / [PT-218474](#))

Parameters

in	<i>inParamIDs</i>	IDs of the parameters that belong to the same GUI element whose controls are being edited
in	<i>inNumOfParams</i>	Number of parameter IDS
in	<i>inModifiers</i>	A bitmask of AAX_EModifiers values

Implemented in [AAX_VViewContainer](#).

The documentation for this class was generated from the following file:

- [AAX_ImageViewContainer.h](#)

14.137 AAX_Map Class Reference

```
#include <AAX_Map.h>
```

Public Member Functions

- [AAX_Map](#) ()
- [~AAX_Map](#) ()
- void [SetCoefficients](#) (int aSize, double *aInpX, double *aInpY)
- void [GetCoefficient](#) (int aIndex, double *aOutX, double *aOutY)
- int [GetUpperBoundIndex](#) (double inp)
- double [GetX](#) (int aIndex)
- double [GetY](#) (int aIndex)
- double [GetFirstX](#) ()
- double [GetFirstY](#) ()
- double [GetLastX](#) ()
- double [GetLastY](#) ()
- int [GetSize](#) ()

14.137.1 Constructor & Destructor Documentation

14.137.1.1 AAX_Map()

```
AAX_Map::AAX_Map ( ) [inline]
```

14.137.1.2 ~AAX_Map()

```
AAX_Map::~~AAX_Map ( ) [inline]
```

14.137.2 Member Function Documentation

14.137.2.1 SetCoefficients()

```
void AAX_Map::SetCoefficients (
    int aSize,
    double * aInpX,
    double * aInpY )
```


14.137.2.2 GetCoefficient()

```
void AAX_Map::GetCoefficient (
    int aIndex,
    double * aOutX,
    double * aOutY )
```

14.137.2.3 GetUpperBoundIndex()

```
int AAX_Map::GetUpperBoundIndex (
    double inp )
```

14.137.2.4 GetX()

```
double AAX_Map::GetX (
    int aIndex ) [inline]
```

14.137.2.5 GetY()

```
double AAX_Map::GetY (
    int aIndex ) [inline]
```

14.137.2.6 GetFirstX()

```
double AAX_Map::GetFirstX ( ) [inline]
```

14.137.2.7 GetFirstY()

```
double AAX_Map::GetFirstY ( ) [inline]
```

14.137.2.8 GetLastX()

```
double AAX_Map::GetLastX ( ) [inline]
```

14.137.2.9 GetLastY()

```
double AAX_Map::GetLastY ( ) [inline]
```

14.137.2.10 GetSize()

```
int AAX_Map::GetSize ( ) [inline]
```

The documentation for this class was generated from the following file:

- [AAX_Map.h](#)

14.138 AAX_Point Struct Reference

```
#include <AAX_GUITypes.h>
```

14.138.1 Description

Data structure representing a two-dimensional coordinate point.

Comparison operators give preference to `vert`

Public Member Functions

- [AAX_Point](#) (float v, float h)
- [AAX_Point](#) (void)

Public Attributes

- float [vert](#)
- float [horz](#)

14.138.2 Constructor & Destructor Documentation

14.138.2.1 AAX_Point() [1/2]

```
AAX_Point::AAX_Point (
    float v,
    float h ) [inline]
```

14.138.2.2 AAX_Point() [2/2]

```
AAX_Point::AAX_Point (
    void ) [inline]
```

14.138.3 Member Data Documentation

14.138.3.1 vert

```
float AAX_Point::vert
```

Referenced by `operator<()`, `operator<=()`, and `operator==()`.

14.138.3.2 horz

```
float AAX_Point::horz
```

Referenced by `operator<()`, `operator<=()`, and `operator==()`.

The documentation for this struct was generated from the following file:

- [AAX_GUITypes.h](#)

14.139 AAX_Rect Struct Reference

```
#include <AAX_GUITypes.h>
```

14.139.1 Description

Data structure representing a rectangle in a two-dimensional coordinate plane.

Public Member Functions

- [AAX_Rect](#) (float t, float l, float w, float h)
- [AAX_Rect](#) (void)

Public Attributes

- float [top](#)
- float [left](#)
- float [width](#)
- float [height](#)

14.139.2 Constructor & Destructor Documentation

14.139.2.1 AAX_Rect() [1/2]

```
AAX_Rect::AAX_Rect (
    float t,
    float l,
    float w,
    float h ) [inline]
```

14.139.2.2 AAX_Rect() [2/2]

```
AAX_Rect::AAX_Rect (
    void ) [inline]
```

14.139.3 Member Data Documentation

14.139.3.1 top

```
float AAX_Rect::top
```

Referenced by operator==(.).

14.139.3.2 left

```
float AAX_Rect::left
```

Referenced by operator==(.).

14.139.3.3 width

```
float AAX_Rect::width
```

Referenced by operator==(.).

14.139.3.4 height

```
float AAX_Rect::height
```

Referenced by operator==().

The documentation for this struct was generated from the following file:

- [AAX_GUITypes.h](#)

14.140 AAX_SHybridRenderInfo Struct Reference

```
#include <AAX_IACFEEffectParameters.h>
```

14.140.1 Description

Hybrid render processing context.

See also

[AAX_IACFEEffectParameters_V2::RenderAudio_Hybrid\(\)](#)

Public Attributes

- float ** [mAudioInputs](#)
- int32_t * [mNumAudioInputs](#)
- float ** [mAudioOutputs](#)
- int32_t * [mNumAudioOutputs](#)
- int32_t * [mNumSamples](#)
- [AAX_CTimestamp](#) * [mClock](#)

14.140.2 Member Data Documentation

14.140.2.1 mAudioInputs

```
float** AAX_SHybridRenderInfo::mAudioInputs
```

14.140.2.2 mNumAudioInputs

```
int32_t* AAX_SHybridRenderInfo::mNumAudioInputs
```

14.140.2.3 mAudioOutputs

```
float** AAX_SHybridRenderInfo::mAudioOutputs
```

14.140.2.4 mNumAudioOutputs

```
int32_t* AAX_SHybridRenderInfo::mNumAudioOutputs
```

14.140.2.5 mNumSamples

```
int32_t* AAX_SHybridRenderInfo::mNumSamples
```

14.140.2.6 mClock

```
AAX_CTimestamp* AAX_SHybridRenderInfo::mClock
```

The documentation for this struct was generated from the following file:

- [AAX_IACFEffEffectParameters.h](#)

14.141 AAX_SInstrumentPrivateData Struct Reference

```
#include <AAX_CMonolithicParameters.h>
```

Collaboration diagram for AAX_SInstrumentPrivateData:

14.141.1 Description

Utility struct for [AAX_CMonolithicParameters](#).

This is an implementation detail of [AAX_CMonolithicParameters](#); you should never need to interact with this structure directly.

Public Attributes

- [AAX_CMonolithicParameters](#) * [mMonolithicParametersPtr](#)

A pointer to the instrument's data model.

14.141.2 Member Data Documentation

14.141.2.1 mMonolithicParametersPtr

`AAX_CMonolithicParameters* AAX_SInstrumentPrivateData::mMonolithicParametersPtr`

A pointer to the instrument's data model.

You should never need to use this since the data model is available directly from within the virtual `AAX_CMonolithicParameters::RenderAudio()` function.

Referenced by `AAX_CMonolithicParameters::ResetFieldData()`, and `AAX_CMonolithicParameters::StaticRenderAudio()`.

The documentation for this struct was generated from the following file:

- [AAX_CMonolithicParameters.h](#)

14.142 AAX_SInstrumentRenderInfo Struct Reference

```
#include <AAX_CMonolithicParameters.h>
```

Collaboration diagram for AAX_SInstrumentRenderInfo:

14.142.1 Description

Information used to parameterize `AAX_CMonolithicParameters::RenderAudio()`

Public Attributes

- `float ** mAudioInputs`
Audio input buffers.
- `float ** mAudioOutputs`
Audio output buffers, including any aux output stems.
- `int32_t * mNumSamples`
Number of samples in each buffer. Bounded as per [AAE_EAudioBufferLengthNative](#). The exact value can vary from buffer to buffer.
- `AAX_CTimestamp * mClock`
Pointer to the global running time clock.
- `AAX_IMIDINode * mInputNode`
Buffered local MIDI input node. Used for incoming MIDI messages directed to the instrument.
- `AAX_IMIDINode * mGlobalNode`
Buffered global MIDI input node. Used for global events like beat updates in metronomes.
- `AAX_IMIDINode * mTransportNode`
Transport MIDI node. Used for querying the state of the MIDI transport.
- `AAX_IMIDINode * mAdditionalInputMIDINodes [kMaxAdditionalMIDINodes]`
List of additional input MIDI nodes, if your plugin needs them.
- `AAX_SInstrumentPrivateData * mPrivateData`
Struct containing private data relating to the instance. You should not need to use this data.
- `float ** mMeters`
Array of meter taps. One meter value should be entered per tap for each render call.
- `int64_t * mCurrentStateNum`
State counter.

14.142.2 Member Data Documentation

14.142.2.1 mAudioInputs

`float** AAX_SInstrumentRenderInfo::mAudioInputs`

Audio input buffers.

14.142.2.2 mAudioOutputs

`float** AAX_SInstrumentRenderInfo::mAudioOutputs`

Audio output buffers, including any aux output stems.

14.142.2.3 mNumSamples

`int32_t* AAX_SInstrumentRenderInfo::mNumSamples`

Number of samples in each buffer. Bounded as per [AAE_EAudioBufferLengthNative](#). The exact value can vary from buffer to buffer.

14.142.2.4 mClock

`AAX_CTimestamp* AAX_SInstrumentRenderInfo::mClock`

Pointer to the global running time clock.

14.142.2.5 mInputNode

`AAX_IMIDINode* AAX_SInstrumentRenderInfo::mInputNode`

Buffered local MIDI input node. Used for incoming MIDI messages directed to the instrument.

14.142.2.6 mGlobalNode

[AAX_IMIDINode*](#) AAX_SInstrumentRenderInfo::mGlobalNode

Buffered global MIDI input node. Used for global events like beat updates in metronomes.

14.142.2.7 mTransportNode

[AAX_IMIDINode*](#) AAX_SInstrumentRenderInfo::mTransportNode

Transport MIDI node. Used for querying the state of the MIDI transport.

14.142.2.8 mAdditionalInputMIDINodes

[AAX_IMIDINode*](#) AAX_SInstrumentRenderInfo::mAdditionalInputMIDINodes[[kMaxAdditionalMIDINodes](#)]

List of additional input MIDI nodes, if your plugin needs them.

14.142.2.9 mPrivateData

[AAX_SInstrumentPrivateData*](#) AAX_SInstrumentRenderInfo::mPrivateData

Struct containing private data relating to the instance. You should not need to use this data.

14.142.2.10 mMeters

[float**](#) AAX_SInstrumentRenderInfo::mMeters

Array of meter taps. One meter value should be entered per tap for each render call.

14.142.2.11 mCurrentStateNum

[int64_t*](#) AAX_SInstrumentRenderInfo::mCurrentStateNum

State counter.

The documentation for this struct was generated from the following file:

- [AAX_CMonolithicParameters.h](#)

14.143 AAX_SInstrumentSetupInfo Struct Reference

```
#include <AAX_CMonolithicParameters.h>
```

14.143.1 Description

Information used to describe the instrument.

See also

[AAX_CMonolithicParameters::StaticDescribe\(\)](#)

Public Member Functions

- [AAX_SInstrumentSetupInfo \(\)](#)

Default constructor.

Public Attributes

- bool [mNeedsGlobalMIDI](#)
Does the instrument use a global MIDI input node?
- const char * [mGlobalMIDINodeName](#)
Name of the global MIDI node, if used.
- uint32_t [mGlobalMIDIEventMask](#)
Global MIDI node event mask of [AAX_EMidiGlobalNodeSelectors](#), if used.
- bool [mNeedsInputMIDI](#)
Does the instrument use a local MIDI input node?
- const char * [mInputMIDINodeName](#)
Name of the MIDI input node, if used.
- uint32_t [mInputMIDIChannelMask](#)
MIDI input node channel mask, if used.
- int32_t [mNumAdditionalInputMIDINodes](#)
Number of additional input MIDI Nodes. These will all share the same channelMask and base MIDINodeName, but the names will be appended with numbers 2,3,4,...
- bool [mNeedsTransport](#)
Does the instrument use the transport interface?
- const char * [mTransportMIDINodeName](#)
Name of the MIDI transport node, if used.
- int32_t [mNumMeters](#)
Number of meter taps used by the instrument. Must match the size of [mMeterIDs](#).
- const [AAX_CTypeID](#) * [mMeterIDs](#)
Array of meter IDs.
- int32_t [mNumAuxOutputStems](#)
Number of aux output stems for the plug-in.
- const char * [mAuxOutputStemNames](#) [kMaxAuxOutputStems]
Names of the aux output stems.
- [AAX_EStemFormat](#) [mAuxOutputStemFormats](#) [kMaxAuxOutputStems]
Stem formats for the output stems.
- [AAX_EStemFormat](#) [mHybridInputStemFormat](#)

- Hybrid input stem format*
 - [AAX_EStemFormat mHybridOutputStemFormat](#)
 - Hybrid output stem format*
 - [AAX_EStemFormat mInputStemFormat](#)
 - Input stem format*
 - [AAX_EStemFormat mOutputStemFormat](#)
 - Output stem format*
 - [bool mUseHostGeneratedGUI](#)
 - Allow Pro Tools or other host to generate a generic GUI. This can be useful for early development.*
 - [bool mCanBypass](#)
 - Can this instrument be bypassed?*
 - [AAX_CTypeID mManufacturerID](#)
 - Manufacturer ID*
 - [AAX_CTypeID mProductID](#)
 - Product ID*
 - [AAX_CTypeID mPluginID](#)
 - Plug-In (Type) ID*
 - [AAX_CTypeID mAudiosuiteID](#)
 - AudioSuite ID*
 - [AAX_CBoolean mMultiMonoSupport](#)

14.143.2 Constructor & Destructor Documentation

14.143.2.1 AAX_SInstrumentSetupInfo()

```
AAX_SInstrumentSetupInfo::AAX_SInstrumentSetupInfo ( ) [inline]
```

Default constructor.

Use this constructor if you want to enable a sub-set of features and don't need to fill out the whole struct.

References [AAX_eStemFormat_Mono](#), [AAX_eStemFormat_None](#), [kMaxAuxOutputStems](#), [mAudiosuiteID](#), [mAuxOutputStemFormats](#), [mAuxOutputStemNames](#), [mCanBypass](#), [mGlobalMIDIEventMask](#), [mGlobalMIDINodeName](#), [mHybridInputStemFormat](#), [mHybridOutputStemFormat](#), [mInputMIDIChannelMask](#), [mInputMIDINodeName](#), [mInputStemFormat](#), [mManufacturerID](#), [mMeterIDs](#), [mMultiMonoSupport](#), [mNeedsGlobalMIDI](#), [mNeedsInputMIDI](#), [mNeedsTransport](#), [mNumAdditionalInputMIDINodes](#), [mNumAuxOutputStems](#), [mNumMeters](#), [mOutputStemFormat](#), [mPluginID](#), [mProductID](#), [mTransportMIDINodeName](#), and [mUseHostGeneratedGUI](#).

14.143.3 Member Data Documentation

14.143.3.1 mNeedsGlobalMIDI

```
bool AAX_SInstrumentSetupInfo::mNeedsGlobalMIDI
```

Does the instrument use a global MIDI input node?

See also

[MIDI](#)

Referenced by `AAX_SInstrumentSetupInfo()`, and `AAX_CMonolithicParameters::StaticDescribe()`.

14.143.3.2 mGlobalMIDINodeName

```
const char* AAX_SInstrumentSetupInfo::mGlobalMIDINodeName
```

Name of the global MIDI node, if used.

See also

[MIDI](#)

Referenced by `AAX_SInstrumentSetupInfo()`, and `AAX_CMonolithicParameters::StaticDescribe()`.

14.143.3.3 mGlobalMIDIEventMask

```
uint32_t AAX_SInstrumentSetupInfo::mGlobalMIDIEventMask
```

Global MIDI node event mask of [AAX_EMidiGlobalNodeSelectors](#), if used.

See also

[MIDI](#)

Referenced by `AAX_SInstrumentSetupInfo()`, and `AAX_CMonolithicParameters::StaticDescribe()`.

14.143.3.4 mNeedsInputMIDI

```
bool AAX_SInstrumentSetupInfo::mNeedsInputMIDI
```

Does the instrument use a local MIDI input node?

See also

[MIDI](#)

Referenced by `AAX_SInstrumentSetupInfo()`, and `AAX_CMonolithicParameters::StaticDescribe()`.

14.143.3.5 mInputMIDINodeName

```
const char* AAX_SInstrumentSetupInfo::mInputMIDINodeName
```

Name of the MIDI input node, if used.

See also

[MIDI](#)

Referenced by `AAX_SInstrumentSetupInfo()`, and `AAX_CMonolithicParameters::StaticDescribe()`.

14.143.3.6 mInputMIDIChannelMask

```
uint32_t AAX_SInstrumentSetupInfo::mInputMIDIChannelMask
```

MIDI input node channel mask, if used.

See also

[MIDI](#)

Referenced by `AAX_SInstrumentSetupInfo()`, and `AAX_CMonolithicParameters::StaticDescribe()`.

14.143.3.7 mNumAdditionalInputMIDINodes

```
int32_t AAX_SInstrumentSetupInfo::mNumAdditionalInputMIDINodes
```

Number of additional input MIDI Nodes. These will all share the same channelMask and base MIDINodeName, but the names will be appended with numbers 2,3,4,...

See also

[MIDI](#)

Referenced by `AAX_SInstrumentSetupInfo()`, and `AAX_CMonolithicParameters::StaticDescribe()`.

14.143.3.8 mNeedsTransport

```
bool AAX_SInstrumentSetupInfo::mNeedsTransport
```

Does the instrument use the transport interface?

See also

[AAX_ITransport](#)

Referenced by `AAX_SInstrumentSetupInfo()`, and `AAX_CMonolithicParameters::StaticDescribe()`.

14.143.3.9 mTransportMIDINodeName

```
const char* AAX_SInstrumentSetupInfo::mTransportMIDINodeName
```

Name of the MIDI transport node, if used.

See also

[AAX_ITransport](#)

Referenced by `AAX_SInstrumentSetupInfo()`.

14.143.3.10 mNumMeters

```
int32_t AAX_SInstrumentSetupInfo::mNumMeters
```

Number of meter taps used by the instrument. Must match the size of [mMeterIDs](#).

See also

[Plug-in meters](#)

Referenced by `AAX_SInstrumentSetupInfo()`, and `AAX_CMonolithicParameters::StaticDescribe()`.

14.143.3.11 mMeterIDs

```
const AAX_CTypeID* AAX_SInstrumentSetupInfo::mMeterIDs
```

Array of meter IDs.

See also

[Plug-in meters](#)

Referenced by `AAX_SInstrumentSetupInfo()`, and `AAX_CMonolithicParameters::StaticDescribe()`.

14.143.3.12 mNumAuxOutputStems

```
int32_t AAX_SInstrumentSetupInfo::mNumAuxOutputStems
```

Number of aux output stems for the plug-in.

See also

[Auxiliary Output Stems](#)

Referenced by `AAX_SInstrumentSetupInfo()`, and `AAX_CMonolithicParameters::StaticDescribe()`.

14.143.3.13 mAuxOutputStemNames

```
const char* AAX_SInstrumentSetupInfo::mAuxOutputStemNames[kMaxAuxOutputStems]
```

Names of the aux output stems.

See also

[Auxiliary Output Stems](#)

Referenced by `AAX_SInstrumentSetupInfo()`, and `AAX_CMonolithicParameters::StaticDescribe()`.

14.143.3.14 mAuxOutputStemFormats

```
AAX_EStemFormat AAX_SInstrumentSetupInfo::mAuxOutputStemFormats[kMaxAuxOutputStems]
```

Stem formats for the output stems.

See also

[Auxiliary Output Stems](#)

Referenced by `AAX_SInstrumentSetupInfo()`, and `AAX_CMonolithicParameters::StaticDescribe()`.

14.143.3.15 mHybridInputStemFormat

```
AAX_EStemFormat AAX_SInstrumentSetupInfo::mHybridInputStemFormat
```

[Hybrid input stem format](#)

A plug-in that defines this value must also define `mHybridOutputStemFormat` and implement `AAX_IEffectParameters::RenderAudio_Hybrid()`

See also

[Hybrid Processing architecture](#)

Referenced by `AAX_SInstrumentSetupInfo()`, and `AAX_CMonolithicParameters::StaticDescribe()`.

14.143.3.16 mHybridOutputStemFormat

[AAX_EStemFormat](#) [AAX_SInstrumentSetupInfo::mHybridOutputStemFormat](#)

[Hybrid output stem format](#)

A plug-in that defines this value must also define [mHybridInputStemFormat](#) and implement [AAX_IEffectParameters::RenderAudio](#).

See also

[Hybrid Processing architecture](#)

Referenced by [AAX_SInstrumentSetupInfo\(\)](#), and [AAX_CMonolithicParameters::StaticDescribe\(\)](#).

14.143.3.17 mInputStemFormat

[AAX_EStemFormat](#) [AAX_SInstrumentSetupInfo::mInputStemFormat](#)

[Input stem format](#)

Referenced by [AAX_SInstrumentSetupInfo\(\)](#), and [AAX_CMonolithicParameters::StaticDescribe\(\)](#).

14.143.3.18 mOutputStemFormat

[AAX_EStemFormat](#) [AAX_SInstrumentSetupInfo::mOutputStemFormat](#)

[Output stem format](#)

Referenced by [AAX_SInstrumentSetupInfo\(\)](#), and [AAX_CMonolithicParameters::StaticDescribe\(\)](#).

14.143.3.19 mUseHostGeneratedGUI

[bool](#) [AAX_SInstrumentSetupInfo::mUseHostGeneratedGUI](#)

Allow Pro Tools or other host to generate a generic GUI. This can be useful for early development.

See also

[AAX_eProperty_UsesClientGUI](#)

Referenced by [AAX_SInstrumentSetupInfo\(\)](#), and [AAX_CMonolithicParameters::StaticDescribe\(\)](#).

14.143.3.20 mCanBypass

`bool AAX_SInstrumentSetupInfo::mCanBypass`

Can this instrument be bypassed?

See also

[AAX_eProperty_CanBypass](#)

Referenced by `AAX_SInstrumentSetupInfo()`, and `AAX_CMonolithicParameters::StaticDescribe()`.

14.143.3.21 mManufacturerID

`AAX_CTypeID AAX_SInstrumentSetupInfo::mManufacturerID`

[Manufacturer ID](#)

Referenced by `AAX_SInstrumentSetupInfo()`, and `AAX_CMonolithicParameters::StaticDescribe()`.

14.143.3.22 mProductID

`AAX_CTypeID AAX_SInstrumentSetupInfo::mProductID`

[Product ID](#)

Referenced by `AAX_SInstrumentSetupInfo()`, and `AAX_CMonolithicParameters::StaticDescribe()`.

14.143.3.23 mPluginID

`AAX_CTypeID AAX_SInstrumentSetupInfo::mPluginID`

[Plug-In \(Type\) ID](#)

Referenced by `AAX_SInstrumentSetupInfo()`, and `AAX_CMonolithicParameters::StaticDescribe()`.

14.143.3.24 mAudiosuiteID

`AAX_CTypeID AAX_SInstrumentSetupInfo::mAudiosuiteID`

[AudioSuite ID](#)

Referenced by `AAX_SInstrumentSetupInfo()`, and `AAX_CMonolithicParameters::StaticDescribe()`.

14.143.3.25 mMultiMonoSupport

[AAX_CBoolean](#) AAX_SInstrumentSetupInfo::mMultiMonoSupport

Multi-mono support

Note

It is recommended to un-set the `mMultiMonoSupport` flag for VIs and other plug-ins which rely on non-global MIDI input. For more information see [AAX_eProperty_Constraint_MultiMonoSupport](#)

Referenced by `AAX_SInstrumentSetupInfo()`, and `AAX_CMonolithicParameters::StaticDescribe()`.

The documentation for this struct was generated from the following file:

- [AAX_CMonolithicParameters.h](#)

14.144 AAX_SPlugInChunk Struct Reference

```
#include <AAX.h>
```

14.144.1 Description

Plug-in chunk header + data.

See also

[AAX_SPlugInChunkHeader](#)

Public Attributes

- [int32_t fSize](#)
The size of the chunk's [fData](#) member.
- [int32_t fVersion](#)
The chunk's version.
- [AAX_CTypeID fManufacturerID](#)
The Plug-In's manufacturer ID.
- [AAX_CTypeID fProductID](#)
The Plug-In file's product ID.
- [AAX_CTypeID fPlugInID](#)
The ID of a particular Plug-In within the file.
- [AAX_CTypeID fChunkID](#)
The ID of a particular Plug-In chunk.
- `unsigned char fName [32]`
A user defined name for this chunk.
- `char fData [1]`
The chunk's data.

14.144.2 Member Data Documentation

14.144.2.1 fSize

```
int32_t AAX_SPlugInChunk::fSize
```

The size of the chunk's [fData](#) member.

14.144.2.2 fVersion

```
int32_t AAX_SPlugInChunk::fVersion
```

The chunk's version.

14.144.2.3 fManufacturerID

```
AAX\_CTypeID AAX_SPlugInChunk::fManufacturerID
```

The Plug-In's manufacturer ID.

14.144.2.4 fProductID

```
AAX\_CTypeID AAX_SPlugInChunk::fProductID
```

The Plug-In file's product ID.

14.144.2.5 fPlugInID

```
AAX\_CTypeID AAX_SPlugInChunk::fPlugInID
```

The ID of a particular Plug-In within the file.

14.144.2.6 fChunkID

```
AAX_CTypeID AAX_SPlugInChunk::fChunkID
```

The ID of a particular Plug-In chunk.

14.144.2.7 fName

```
unsigned char AAX_SPlugInChunk::fName[32]
```

A user defined name for this chunk.

14.144.2.8 fData

```
char AAX_SPlugInChunk::fData[1]
```

The chunk's data.

Note

The fixed-size array definition here is historical, but misleading. Plug-ins actually write off the end of this block and are allowed to as long as they don't exceed their reported size.

The documentation for this struct was generated from the following file:

- [AAX.h](#)

14.145 AAX_SPlugInChunkHeader Struct Reference

```
#include <AAX.h>
```

14.145.1 Description

Plug-in chunk header.

Legacy Porting Notes To ensure compatibility with TDM/RTAS plug-ins whose implementation requires `fSize` to be equal to the size of the chunk's header plus its data, AAE performs some behind-the-scenes record keeping.

The following actions are only taken for AAX plug-ins, so, e.g., if a chunk is stored by an RTAS or TDM plug-in that reports data+header size in `fSize` and this chunk is then loaded by the AAX version of the plug-in, the header size will be cached as-is from the legacy plug-in and will be subtracted out before the chunk data is passed to the AAX plug-in. If a chunk is stored by an AAX plug-in and is then loaded by a legacy plug-in, the legacy plug-in will receive the cached plug-in header with `fSize` equal to the data+header size.

These are the special actions that AAE takes to ensure backwards-compatibility when handling AAX chunk data:

- When AAE retrieves the size of a chunk from an AAX plug-in using `GetChunkSize()`, it adds the chunk header size to the amount of memory that it allocates for the chunk
- When AAE retrieves a chunk from an AAX plug-in using `GetChunk()`, it adds the chunk header size to `fChunkSize` before caching the chunk
- Before calling `SetChunk()` or `CompareActiveChunk()`, AAE subtracts the chunk header size from the cached chunk's header's `fChunkSize` member

Public Attributes

- `int32_t fSize`
The size of the chunk's `fData` member.
- `int32_t fVersion`
The chunk's version.
- `AAX_CTypeID fManufacturerID`
The Plug-In's manufacturer ID.
- `AAX_CTypeID fProductID`
The Plug-In file's product ID.
- `AAX_CTypeID fPlugInID`
The ID of a particular Plug-In within the file.
- `AAX_CTypeID fChunkID`
The ID of a particular Plug-In chunk.
- `unsigned char fName [32]`
A user defined name for this chunk.

14.145.2 Member Data Documentation

14.145.2.1 fSize

```
int32_t AAX_SPlugInChunkHeader::fSize
```

The size of the chunk's [fData](#) member.

14.145.2.2 fVersion

```
int32_t AAX_SPlugInChunkHeader::fVersion
```

The chunk's version.

14.145.2.3 fManufacturerID

```
AAX_CTypeID AAX_SPlugInChunkHeader::fManufacturerID
```

The Plug-In's manufacturer ID.

14.145.2.4 fProductID

```
AAX_CTypeID AAX_SPlugInChunkHeader::fProductID
```

The Plug-In file's product ID.

14.145.2.5 fPlugInID

```
AAX_CTypeID AAX_SPlugInChunkHeader::fPlugInID
```

The ID of a particular Plug-In within the file.

14.145.2.6 fChunkID

```
AAX_CTypeID AAX_SPlugInChunkHeader::fChunkID
```

The ID of a particular Plug-In chunk.

14.145.2.7 fName

```
unsigned char AAX_SPlugInChunkHeader::fName[32]
```

A user defined name for this chunk.

The documentation for this struct was generated from the following file:

- [AAX.h](#)

14.146 AAX_SPlugInIdentifierTriad Struct Reference

```
#include <AAX.h>
```

14.146.1 Description

Plug-in Identifier Triad.

This set of identifiers are what uniquely identify a particular plug-in type.

Public Attributes

- [AAX_CTypeID mManufacturerID](#)
The Plug-In's manufacturer ID.
- [AAX_CTypeID mProductID](#)
The Plug-In's product (Effect) ID.
- [AAX_CTypeID mPlugInID](#)
The ID of a specific type in the product (Effect)

14.146.2 Member Data Documentation

14.146.2.1 mManufacturerID

```
AAX\_CTypeID AAX_SPlugInIdentifierTriad::mManufacturerID
```

The Plug-In's manufacturer ID.

Referenced by `AAX::AsStringIDTriad()`.

14.146.2.2 mProductID

[AAX_CTypeID](#) AAX_SPlugInIdentifierTriad::mProductID

The Plug-In's product (Effect) ID.

Referenced by `AAX::AsStringIDTriad()`.

14.146.2.3 mPlugInID

[AAX_CTypeID](#) AAX_SPlugInIdentifierTriad::mPlugInID

The ID of a specific type in the product (Effect)

Referenced by `AAX::AsStringIDTriad()`.

The documentation for this struct was generated from the following file:

- [AAX.h](#)

14.147 AAX_StLock_Guard Class Reference

```
#include <AAX_CMutex.h>
```

14.147.1 Description

Helper class for working with mutex.

Public Member Functions

- [AAX_StLock_Guard](#) ([AAX_CMutex](#) &iMutex)
- [~AAX_StLock_Guard](#) ()

14.147.2 Constructor & Destructor Documentation

14.147.2.1 AAX_StLock_Guard()

```
AAX_StLock_Guard::AAX_StLock_Guard (
    AAX\_CMutex & iMutex ) [inline], [explicit]
```

References `AAX_CMutex::Lock()`.

Here is the call graph for this function:

14.147.2.2 ~AAX_StLock_Guard()

```
AAX_StLock_Guard::~~AAX_StLock_Guard ( ) [inline]
```

References `AAX_CMutex::Unlock()`.

Here is the call graph for this function:

The documentation for this class was generated from the following file:

- [AAX_CMutex.h](#)

14.148 AAX_TransportStateInfo_V1 Struct Reference

```
#include <AAX_TransportTypes.h>
```

14.148.1 Description

Helper structure for payload data described transport state information.

Public Member Functions

- [AAX_TransportStateInfo_V1 \(\)](#)
- `std::string ToString () const`

Public Attributes

- [AAX_ETransportState mTransportState](#)
- [AAX_ERecordMode mRecordMode](#)
- [AAX_CBoolean mIsRecordEnabled](#)
- [AAX_CBoolean mIsRecording](#)
- [AAX_CBoolean mIsLoopEnabled](#)

14.148.2 Constructor & Destructor Documentation

14.148.2.1 AAX_TransportStateInfo_V1()

```
AAX_TransportStateInfo_V1::AAX_TransportStateInfo_V1 ( ) [inline]
```

14.148.3 Member Function Documentation

14.148.3.1 ToString()

```
std::string AAX_TransportStateInfo_V1::ToString ( ) const [inline]
```

References `mlsLoopEnabled`, `mlsRecordEnabled`, `mlsRecording`, `mRecordMode`, and `mTransportState`.

14.148.4 Member Data Documentation

14.148.4.1 mTransportState

```
AAX_ETransportState AAX_TransportStateInfo_V1::mTransportState
```

Referenced by `operator==()`, and `ToString()`.

14.148.4.2 mRecordMode

```
AAX_ERecordMode AAX_TransportStateInfo_V1::mRecordMode
```

Referenced by `operator==()`, and `ToString()`.

14.148.4.3 mIsRecordEnabled

```
AAX_CBoolean AAX_TransportStateInfo_V1::mIsRecordEnabled
```

Referenced by `operator==()`, and `ToString()`.

14.148.4.4 mIsRecording

```
AAX_CBoolean AAX_TransportStateInfo_V1::mIsRecording
```

Referenced by `operator==()`, and `ToString()`.

14.148.4.5 mIsLoopEnabled

```
AAX_CBoolean AAX_TransportStateInfo_V1::mIsLoopEnabled
```

Referenced by `operator==()`, and `ToString()`.

The documentation for this struct was generated from the following file:

- [AAX_TransportTypes.h](#)

14.149 AAX_VAutomationDelegate Class Reference

```
#include <AAX_VAutomationDelegate.h>
```

Inheritance diagram for AAX_VAutomationDelegate:

Collaboration diagram for AAX_VAutomationDelegate:

14.149.1 Description

Version-managed concrete [automation delegate](#) class.

Public Member Functions

- [AAX_VAutomationDelegate](#) ([IACFUnknown](#) *pUnknown)
- [~AAX_VAutomationDelegate](#) () [AAX_OVERRIDE](#)
- [IACFUnknown](#) * [GetUnknown](#) () const
- [AAX_Result](#) [RegisterParameter](#) ([AAX_CParamID](#) iParameterID) [AAX_OVERRIDE](#)
- [AAX_Result](#) [UnregisterParameter](#) ([AAX_CParamID](#) iParameterID) [AAX_OVERRIDE](#)
- [AAX_Result](#) [PostSetValueRequest](#) ([AAX_CParamID](#) iParameterID, double iNormalizedValue) const [AAX_OVERRIDE](#)
- [AAX_Result](#) [PostCurrentValue](#) ([AAX_CParamID](#) iParameterID, double iNormalizedValue) const [AAX_OVERRIDE](#)
- [AAX_Result](#) [PostTouchRequest](#) ([AAX_CParamID](#) iParameterID) [AAX_OVERRIDE](#)
- [AAX_Result](#) [PostReleaseRequest](#) ([AAX_CParamID](#) iParameterID) [AAX_OVERRIDE](#)
- [AAX_Result](#) [GetTouchState](#) ([AAX_CParamID](#) iParameterID, [AAX_CBoolean](#) *outTouched) [AAX_OVERRIDE](#)
- [AAX_Result](#) [ParameterNameChanged](#) ([AAX_CParamID](#) iParameterID) [AAX_OVERRIDE](#)

14.149.2 Constructor & Destructor Documentation

14.149.2.1 AAX_VAutomationDelegate()

```
AAX_VAutomationDelegate::AAX_VAutomationDelegate (
    IACFUnknown * pUnknown )
```

14.149.2.2 ~AAX_VAutomationDelegate()

```
AAX_VAutomationDelegate::~~AAX_VAutomationDelegate ( )
```

14.149.3 Member Function Documentation

14.149.3.1 GetUnknown()

```
IACFUnknown* AAX_VAutomationDelegate::GetUnknown ( ) const [inline]
```

14.149.3.2 RegisterParameter()

```
AAX_Result AAX_VAutomationDelegate::RegisterParameter (
    AAX_CParamID iParameterID ) [virtual]
```

Register a control with the automation system using a char* based control identifier

The automation delegate owns a list of the IDs of all of the parameters that have been registered with it. This list is used to set up listeners for all of the registered parameters such that the automation delegate may update the plug-in when the state of any of the registered parameters have been modified.

See also

[AAX_IAutomationDelegate::UnregisterParameter\(\)](#)

Parameters

in	<i>iParameterID</i>	Parameter ID that is being registered
----	---------------------	---------------------------------------

Implements [AAX_IAutomationDelegate](#).

14.149.3.3 UnregisterParameter()

```
AAX_Result AAX_VAutomationDelegate::UnregisterParameter (
    AAX_CParamID iParameterID ) [virtual]
```

Unregister a control with the automation system using a char* based control identifier

Note

All registered controls should be unregistered or the system might leak.

See also

[AAX_IAutomationDelegate::RegisterParameter\(\)](#)

Parameters

in	<i>iParameterID</i>	Parameter ID that is being registered
----	---------------------	---------------------------------------

Implements [AAX_IAutomationDelegate](#).

14.149.3.4 PostSetValueRequest()

```
AAX_Result AAX_VAutomationDelegate::PostSetValueRequest (
    AAX_CParamID iParameterID,
    double normalizedValue ) const [virtual]
```

Submits a request for the given parameter's value to be changed

Parameters

in	<i>iParameterID</i>	ID of the parameter for which a change is requested
in	<i>normalizedValue</i>	The requested new parameter value, formatted as a double and normalized to [0 1]

Implements [AAX_IAutomationDelegate](#).

14.149.3.5 PostCurrentValue()

```
AAX_Result AAX_VAutomationDelegate::PostCurrentValue (
    AAX_CParamID iParameterID,
    double normalizedValue ) const [virtual]
```

Notifies listeners that a parameter's value has changed

Parameters

in	<i>iParameterID</i>	ID of the parameter that has been updated
in	<i>normalizedValue</i>	The current parameter value, formatted as a double and normalized to [0 1]

Implements [AAX_IAutomationDelegate](#).

14.149.3.6 PostTouchRequest()

```
AAX_Result AAX_VAutomationDelegate::PostTouchRequest (
    AAX_CParamID iParameterID ) [virtual]
```

Requests that the given parameter be "touched", i.e. locked for updates by the current client

Parameters

in	<i>iParameterID</i>	ID of the parameter that will be touched
----	---------------------	--

Implements [AAX_IAutomationDelegate](#).

14.149.3.7 PostReleaseRequest()

```
AAX_Result AAX_VAutomationDelegate::PostReleaseRequest (
    AAX_CParamID iParameterID ) [virtual]
```

Requests that the given parameter be "released", i.e. available for updates from any client

Parameters

in	<i>iParameterID</i>	ID of the parameter that will be released
----	---------------------	---

Implements [AAX_IAutomationDelegate](#).

14.149.3.8 GetTouchState()

```
AAX_Result AAX_VAutomationDelegate::GetTouchState (
    AAX_CParamID iParameterID,
    AAX_CBoolean * oTouched ) [virtual]
```

Gets the current touched state of a parameter

Parameters

in	<i>iParameterID</i>	ID of the parameter that is being queried
out	<i>oTouched</i>	The current touch state of the parameter

Implements [AAX_IAutomationDelegate](#).

14.149.3.9 ParameterNameChanged()

```
AAX_Result AAX_VAutomationDelegate::ParameterNameChanged (
    AAX_CParamID iParameterID ) [virtual]
```

Notify listeners that the parameter's display name has changed

Note that this is not part of the underlying automation delegate interface with the host; it is converted on the AAX side to a notification posted to the host via the [AAX_IController](#).

Parameters

in	<i>iParameterID</i>	ID of the parameter that has been updated
----	---------------------	---

Implements [AAX_IAutomationDelegate](#).

The documentation for this class was generated from the following file:

- [AAX_VAutomationDelegate.h](#)

14.150 AAX_VCollection Class Reference

```
#include <AAX_VCollection.h>
```

Inheritance diagram for AAX_VCollection:

Collaboration diagram for AAX_VCollection:

14.150.1 Description

Version-managed concrete [AAX_ICollection](#) class.

Public Member Functions

- [AAX_VCollection](#) ([IACFUnknown](#) *pUnkHost)
- [~AAX_VCollection](#) () [AAX_OVERRIDE](#)
- [AAX_IEffectDescriptor](#) * [NewDescriptor](#) () [AAX_OVERRIDE](#)
Create a new Effect descriptor.
- [AAX_Result](#) [AddEffect](#) (const char *inEffectID, [AAX_IEffectDescriptor](#) *inEffectDescriptor) [AAX_OVERRIDE](#)
Add an Effect description to the collection.
- [AAX_Result](#) [SetManufacturerName](#) (const char *inPackageName) [AAX_OVERRIDE](#)
Set the plug-in manufacturer name.
- [AAX_Result](#) [AddPackageName](#) (const char *inPackageName) [AAX_OVERRIDE](#)
Set the plug-in package name.
- [AAX_Result](#) [SetPackageVersion](#) (uint32_t inVersion) [AAX_OVERRIDE](#)
Set the plug-in package version number.
- [AAX_IPropertyMap](#) * [NewPropertyMap](#) () [AAX_OVERRIDE](#)
Create a new property map.
- [AAX_Result](#) [SetProperties](#) ([AAX_IPropertyMap](#) *inProperties) [AAX_OVERRIDE](#)
Set the properties of the collection.
- [AAX_Result](#) [GetHostVersion](#) (uint32_t *outVersion) const [AAX_OVERRIDE](#)
Get the current version of the host.
- [AAX_IDescriptionHost](#) * [DescriptionHost](#) () [AAX_OVERRIDE](#)
- const [AAX_IDescriptionHost](#) * [DescriptionHost](#) () const [AAX_OVERRIDE](#)
- [IACFDefinition](#) * [HostDefinition](#) () const [AAX_OVERRIDE](#)
- [IACFPluginDefinition](#) * [GetUnknown](#) () const

14.150.2 Constructor & Destructor Documentation

14.150.2.1 AAX_VCollection()

```
AAX_VCollection::AAX_VCollection (
    IACFUnknown * pUnkHost )
```

14.150.2.2 ~AAX_VCollection()

```
AAX_VCollection::~~AAX_VCollection ( )
```

14.150.3 Member Function Documentation

14.150.3.1 NewDescriptor()

```
AAX_IEffectDescriptor* AAX_VCollection::NewDescriptor ( ) [virtual]
```

Create a new Effect descriptor.

This implementation retains each generated [AAX_IEffectDescriptor](#) and destroys the descriptor upon [AAX_VCollection](#) destruction

Create a new Effect descriptor.

Implements [AAX_ICollection](#).

14.150.3.2 AddEffect()

```
AAX_Result AAX_VCollection::AddEffect (
    const char * inEffectID,
    AAX_IEffectDescriptor * inEffectDescriptor ) [virtual]
```

Add an Effect description to the collection.

Each Effect that a plug-in registers with [AAX_ICollection::AddEffect\(\)](#) is considered a completely different user-facing product. For example, in Avid's Dynamics III plug-in the Expander, Compressor, and DeEsser are each registered as separate Effects. All stem format variations within each Effect are registered within that Effect's [AAX_IEffectDescriptor](#) using [AddComponent\(\)](#).

The [AAX_eProperty_ProductID](#) value for all ProcessProcs within a single Effect must be identical.

This method passes ownership of an [AAX_IEffectDescriptor](#) object to the [AAX_ICollection](#). The [AAX_IEffectDescriptor](#) must not be deleted by the AAX plug-in, nor should it be edited in any way after it is passed to the [AAX_ICollection](#).

Parameters

in	<i>inEffectID</i>	The effect ID.
in	<i>inEffectDescriptor</i>	The Effect descriptor.

Implements [AAX_ICollection](#).

14.150.3.3 SetManufacturerName()

```
AAX_Result AAX_VCollection::SetManufacturerName (
    const char * inPackageName ) [virtual]
```

Set the plug-in manufacturer name.

Parameters

in	<i>inPackageName</i>	The name of the manufacturer.
----	----------------------	-------------------------------

Implements [AAX_ICollection](#).

14.150.3.4 AddPackageName()

```
AAX_Result AAX_VCollection::AddPackageName (
    const char * inPackageName ) [virtual]
```

Set the plug-in package name.

May be called multiple times to add abbreviated package names.

Note

Every plug-in must include at least one name variant with 16 or fewer characters, plus a null terminating character. Used for Presets folder.

Parameters

in	<i>inPackageName</i>	The name of the package.
----	----------------------	--------------------------

Implements [AAX_ICollection](#).

14.150.3.5 SetPackageVersion()

```
AAX_Result AAX_VCollection::SetPackageVersion (
    uint32_t inVersion ) [virtual]
```

Set the plug-in package version number.

Parameters

in	<i>inVersion</i>	The package version number.
----	------------------	-----------------------------

Implements [AAX_ICollection](#).

14.150.3.6 NewPropertyMap()

```
AAX_IPropertyMap* AAX_VCollection::NewPropertyMap ( ) [virtual]
```

Create a new property map.

Implements [AAX_ICollection](#).

14.150.3.7 SetPropertyMap()

```
AAX_Result AAX_VCollection::SetProperties (
    AAX_IPropertyMap * inProperties ) [virtual]
```

Set the properties of the collection.

Parameters

in	<i>inProperties</i>	Collection properties
----	---------------------	-----------------------

Implements [AAX_ICollection](#).

14.150.3.8 GetHostVersion()

```
AAX_Result AAX_VCollection::GetHostVersion (
    uint32_t * outVersion ) const [virtual]
```

Get the current version of the host.

See [AAXATTR_Client_Version](#) for information about the version data format

Warning

Do not use this method to infer host feature support. Instead, use [AAX_IDescriptionHost](#) to query the host for specific features.

AAX plugin description details are cached and might not be re-scanned even if the host version changes. Be careful to avoid changing any cacheable information based on this data. See [Host feature and environment checks](#) for more information.

Parameters

in	outVersion	Host version
----	------------	--------------

Implements [AAX_ICollection](#).

14.150.3.9 DescriptionHost() [1/2]

```
AAX_IDescriptionHost* AAX_VCollection::DescriptionHost ( ) [virtual]
```

Get a pointer to an [AAX_IDescriptionHost](#), if supported by the host

This interface is served by the [AAX_ICollection](#) in order to avoid requiring a new method prototype for the [GetEffectDescriptions\(\)](#) method called from the AAX Library.

See also

[AAX_UIDs.h](#) for available feature UIDs, e.g. [AAXATTR_ClientFeature_AuxOutputStem](#)

Warning

AAX plugin description details are cached and might not be re-scanned even if attributes of the host description change. Be careful to avoid changing any cacheable information based on this data. See [Host feature and environment checks](#) for more information.

Implements [AAX_ICollection](#).

14.150.3.10 DescriptionHost() [2/2]

```
const AAX_IDescriptionHost* AAX_VCollection::DescriptionHost ( ) const [virtual]
```

Get a pointer to an [AAX_IDescriptionHost](#), if supported by the host

This interface is served by the [AAX_ICollection](#) in order to avoid requiring a new method prototype for the [GetEffectDescriptions\(\)](#) method called from the AAX Library.

See also

[AAX_UIDs.h](#) for available feature UIDs, e.g. [AAXATTR_ClientFeature_AuxOutputStem](#)

Warning

AAX plugin description details are cached and might not be re-scanned even if attributes of the host description change. Be careful to avoid changing any cacheable information based on this data. See [Host feature and environment checks](#) for more information.

Implements [AAX_ICollection](#).

14.150.3.11 HostDefinition()

```
IACFDefinition* AAX_VCollection::HostDefinition ( ) const [virtual]
```

Get a pointer to an [IACFDefinition](#), if supported by the host

This interface is served by the [AAX_ICollection](#) in order to avoid requiring a new method prototype for the [GetEffectDescriptions\(\)](#) method called from the AAX Library.

See also

[AAX_UIDs.h](#) for available host attribute UIDs, e.g. [AAXATTR_Client_Level](#)

Warning

AAX plugin description details are cached and might not be re-scanned even if attributes of the host definition change, for example the host level. Be careful to avoid changing any cacheable information based on this data. See [Host feature and environment checks](#) for more information.

The implementation of [AAX_ICollection](#) owns the referenced object. No AddRef occurs.

[IACFDefinition::DefineAttribute\(\)](#) is not supported on this object

Implements [AAX_ICollection](#).

14.150.3.12 GetUnknown()

```
IACFPluginDefinition* AAX_VCollection::GetUnknown ( ) const
```

The documentation for this class was generated from the following file:

- [AAX_VCollection.h](#)

14.151 AAX_VComponentDescriptor Class Reference

```
#include <AAX_VComponentDescriptor.h>
```

Inheritance diagram for [AAX_VComponentDescriptor](#):

Collaboration diagram for [AAX_VComponentDescriptor](#):

14.151.1 Description

Version-managed concrete [AAX_IComponentDescriptor](#) class.

Public Member Functions

- [AAX_VComponentDescriptor](#) (IACFUnknown *pUnkHost)
- [~AAX_VComponentDescriptor](#) () [AAX_OVERRIDE](#)
- [AAX_Result Clear](#) () [AAX_OVERRIDE](#)
Clears the descriptor.
- [AAX_Result AddReservedField](#) (AAX_CFieldIndex inFieldIndex, uint32_t inFieldType) [AAX_OVERRIDE](#)
Subscribes a context field to host-provided services or information.
- [AAX_Result AddAudioIn](#) (AAX_CFieldIndex inFieldIndex) [AAX_OVERRIDE](#)
Subscribes an audio input context field.
- [AAX_Result AddAudioOut](#) (AAX_CFieldIndex inFieldIndex) [AAX_OVERRIDE](#)
Subscribes an audio output context field.
- [AAX_Result AddAudioBufferLength](#) (AAX_CFieldIndex inFieldIndex) [AAX_OVERRIDE](#)
Subscribes a buffer length context field.
- [AAX_Result AddSampleRate](#) (AAX_CFieldIndex inFieldIndex) [AAX_OVERRIDE](#)
Subscribes a sample rate context field.
- [AAX_Result AddClock](#) (AAX_CFieldIndex inFieldIndex) [AAX_OVERRIDE](#)
Subscribes a clock context field.
- [AAX_Result AddSideChainIn](#) (AAX_CFieldIndex inFieldIndex) [AAX_OVERRIDE](#)
Subscribes a side-chain input context field.
- [AAX_Result AddDataInPort](#) (AAX_CFieldIndex inFieldIndex, uint32_t inPacketSize, AAX_EDataInPortType inPortType) [AAX_OVERRIDE](#)
Adds a custom data port to the algorithm context.
- [AAX_Result AddAuxOutputStem](#) (AAX_CFieldIndex inFieldIndex, int32_t inStemFormat, const char inNameUTF8[]) [AAX_OVERRIDE](#)
Adds an auxiliary output stem for a plug-in.
- [AAX_Result AddPrivateData](#) (AAX_CFieldIndex inFieldIndex, int32_t inDataSize, uint32_t inOptions) [AAX_OVERRIDE](#)
Adds a private data port to the algorithm context.
- [AAX_Result AddTemporaryData](#) (AAX_CFieldIndex inFieldIndex, uint32_t inDataElementSize) [AAX_OVERRIDE](#)
Adds a block of data to a context that is not saved between callbacks and is scaled by the system buffer size.
- [AAX_Result AddDmaInstance](#) (AAX_CFieldIndex inFieldIndex, AAX_IDma::EMode inDmaMode) [AAX_OVERRIDE](#)
Adds a DMA field to the plug-in's context.
- [AAX_Result AddMeters](#) (AAX_CFieldIndex inFieldIndex, const AAX_CTypeID *inMeterIDs, const uint32_t inMeterCount) [AAX_OVERRIDE](#)
Adds a meter field to the plug-in's context.
- [AAX_Result AddMIDINode](#) (AAX_CFieldIndex inFieldIndex, AAX_EMIDINodeType inNodeType, const char inNodeName[], uint32_t channelMask) [AAX_OVERRIDE](#)
Adds a MIDI node field to the plug-in's context.
- [AAX_IPropertyMap * NewPropertyMap](#) () const [AAX_OVERRIDE](#)
Creates a new, empty property map.
- [AAX_IPropertyMap * DuplicatePropertyMap](#) (AAX_IPropertyMap *inPropertyMap) const [AAX_OVERRIDE](#)
Creates a new property map using an existing property map.
- virtual [AAX_Result AddProcessProc_Native](#) (AAX_CProcessProc inProcessProc, AAX_IPropertyMap *inProperties=NULL, AAX_CInstanceInitProc inInstanceInitProc=NULL, AAX_CBackgroundProc inBackgroundProc=NULL, AAX_CSelector *outProcID=NULL) [AAX_OVERRIDE](#)
Registers an algorithm processing endpoint (process procedure) for the native architecture.
- virtual [AAX_Result AddProcessProc_TI](#) (const char inDLLFileNameUTF8[], const char inProcessProcSymbol[], AAX_IPropertyMap *inProperties=NULL, const char inInstanceInitProcSymbol[]=NULL, const char inBackgroundProcSymbol[]=NULL, AAX_CSelector *outProcID=NULL) [AAX_OVERRIDE](#)
Registers an algorithm processing endpoint (process procedure) for the native architecture.
- virtual [AAX_Result AddProcessProc](#) (AAX_IPropertyMap *inProperties, AAX_CSelector *outProcIDs=NULL, int32_t inProcIDsSize=0) [AAX_OVERRIDE](#)
Registers one or more algorithm processing endpoints (process procedures)
- [IACFUnknown * GetIUnknown](#) (void) const

Friends

- class [AAX_VPropertyMap](#)

14.151.2 Constructor & Destructor Documentation

14.151.2.1 AAX_VComponentDescriptor()

```
AAX_VComponentDescriptor::AAX_VComponentDescriptor (
    IACFUnknown * pUnkHost )
```

14.151.2.2 ~AAX_VComponentDescriptor()

```
AAX_VComponentDescriptor::~~AAX_VComponentDescriptor ( )
```

14.151.3 Member Function Documentation

14.151.3.1 Clear()

```
AAX_Result AAX_VComponentDescriptor::Clear ( ) [virtual]
```

Clears the descriptor.

Clears the descriptor and readies it for the next algorithm description

Implements [AAX_IComponentDescriptor](#).

14.151.3.2 AddReservedField()

```
AAX_Result AAX_VComponentDescriptor::AddReservedField (
    AAX_CFieldIndex inFieldIndex,
    uint32_t inFieldType ) [virtual]
```

Subscribes a context field to host-provided services or information.

Note

Currently for internal use only.

Parameters

in	<i>inFieldIndex</i>	Unique identifier for the field, generated using AAX_FIELD_INDEX
in	<i>inFieldType</i>	Type of field that is being added

Implements [AAX_IComponentDescriptor](#).

14.151.3.3 AddAudioIn()

```
AAX_Result AAX_VComponentDescriptor::AddAudioIn (
    AAX_CFieldIndex inFieldIndex ) [virtual]
```

Subscribes an audio input context field.

Defines an audio in port for host-provided information in the algorithm's context structure.

- Data type: float**
- Data kind: An array of float arrays, one for each input channel

Parameters

in	<i>inFieldIndex</i>	Unique identifier for the field, generated using AAX_FIELD_INDEX
----	---------------------	--

Implements [AAX_IComponentDescriptor](#).

14.151.3.4 AddAudioOut()

```
AAX_Result AAX_VComponentDescriptor::AddAudioOut (
    AAX_CFieldIndex inFieldIndex ) [virtual]
```

Subscribes an audio output context field.

Defines an audio out port for host-provided information in the algorithm's context structure.

- Data type: float**
- Data kind: An array of float arrays, one for each output channel

Parameters

in	<i>inFieldIndex</i>	Unique identifier for the field, generated using AAX_FIELD_INDEX
----	---------------------	--

Implements [AAX_IComponentDescriptor](#).

14.151.3.5 AddAudioBufferLength()

```
AAX_Result AAX_VComponentDescriptor::AddAudioBufferLength (
    AAX_CFieldIndex inFieldIndex ) [virtual]
```

Subscribes a buffer length context field.

Defines a buffer length port for host-provided information in the algorithm's context structure.

- Data type: int32_t*
- Data kind: The number of samples in the current audio buffer

Parameters

in	<i>inFieldIndex</i>	Unique identifier for the field, generated using AAX_FIELD_INDEX
----	---------------------	--

Implements [AAX_IComponentDescriptor](#).

14.151.3.6 AddSampleRate()

```
AAX_Result AAX_VComponentDescriptor::AddSampleRate (
    AAX_CFieldIndex inFieldIndex ) [virtual]
```

Subscribes a sample rate context field.

Defines a sample rate port for host-provided information in the algorithm's context structure.

- Data type: [AAX_CSampleRate](#) *
- Data kind: The current sample rate

Parameters

in	<i>inFieldIndex</i>	Unique identifier for the field, generated using AAX_FIELD_INDEX
----	---------------------	--

Implements [AAX_IComponentDescriptor](#).

14.151.3.7 AddClock()

```
AAX_Result AAX_VComponentDescriptor::AddClock (
```



```
AAX_CFieldIndex inFieldIndex ) [virtual]
```

Subscribes a clock context field.

Defines a clock port for host-provided information in the algorithm's context structure.

- Data type: [AAX_CTimestamp](#) *
- Data kind: A running counter which increments even when the transport is not playing. The counter increments exactly once per sample quantum.

Host Compatibility Notes As of Pro Tools 11.1, this field may be used in both Native and DSP plug-ins. The DSP clock data is a 16-bit cycling counter. This field was only available for Native plug-ins in previous Pro Tools versions.

Parameters

in	<i>inFieldIndex</i>	Unique identifier for the field, generated using AAX_FIELD_INDEX
----	---------------------	--

Implements [AAX_IComponentDescriptor](#).

14.151.3.8 AddSideChainIn()

```
AAX_Result AAX_VComponentDescriptor::AddSideChainIn (
    AAX_CFieldIndex inFieldIndex ) [virtual]
```

Subscribes a side-chain input context field.

Defines a side-chain input port for host-provided information in the algorithm's context structure.

- Data type: int32_t*
- Data kind: The index of the plug-in's first side-chain input channel within the array of input audio buffers

Parameters

in	<i>inFieldIndex</i>	Unique identifier for the field, generated using AAX_FIELD_INDEX
----	---------------------	--

Implements [AAX_IComponentDescriptor](#).

14.151.3.9 AddDataInPort()

```
AAX_Result AAX_VComponentDescriptor::AddDataInPort (
    AAX_CFieldIndex inFieldIndex,
```

```
uint32_t inPacketSize,
AAX_EDataInPortType inPortType ) [virtual]
```

Adds a custom data port to the algorithm context.

Defines a read-only data port for plug-in information in the algorithm's context structure. The plug-in can send information to this port using [AAX_IController::PostPacket\(\)](#).

The host guarantees that all packets will be delivered to this port in the order in which they were posted, up to the point of a packet buffer overflow, though some packets may be dropped depending on the `inPortType` and host implementation.

Note

When a plug-in is operating in offline (AudioSuite) mode, all data ports operate as [AAX_eDataInPortType_Unbuffered](#) ports

Parameters

in	<i>inFieldIndex</i>	Unique identifier for the port, generated using AAX_FIELD_INDEX
in	<i>inPacketSize</i>	Size of the data packets that will be sent to this port
in	<i>inPortType</i>	The requested packet delivery behavior for this port

Implements [AAX_IComponentDescriptor](#).

14.151.3.10 AddAuxOutputStem()

```
AAX_Result AAX_VComponentDescriptor::AddAuxOutputStem (
    AAX_CFieldIndex inFieldIndex,
    int32_t inStemFormat,
    const char inNameUTF8[] ) [virtual]
```

Adds an auxiliary output stem for a plug-in.

Use this method to add additional output channels to the algorithm context.

The aux output stem audio buffers will be added to the end of the audio outputs array in the order in which they are described. When writing audio data to a specific aux output, find the proper starting channel by accumulating all of the channels of the main output stem format and any previously-described aux output stems.

The plug-in is responsible for providing a meaningful name for each aux outputs. At the very least, individual outputs should be labeled "Output xx", where "xx" is the aux output number as it is defined in the plug-in. The output name should also include the words "mono" and "stereo" to support when users are looking for an output with a specific stem format.

Host Compatibility Notes There is a hard limit to the number of outputs that Pro Tools supports for a single plug-in instance. This limit is currently set at 256 channels, which includes all of the plug-in's output channels in addition to the sum total of all of its aux output stem channels.

Host Compatibility Notes Pro Tools supports only mono and stereo auxiliary output stem formats

Warning

This method will return an error code on hosts which do not support auxiliary output stems. This indicates that the host will not provide audio buffers for auxiliary output stems during processing. A plug-in must not attempt to write data into auxiliary output stem buffers which have not been provided by the host!

Parameters

in	<i>inFieldIndex</i>	DEPRECATED: This parameter is no longer needed by the host, but is included in the interface for binary compatibility
in	<i>inStemFormat</i>	The stem format of the new aux output
in	<i>inNameUTF8</i>	The name of the aux output. This name is static and cannot be changed after the descriptor is submitted to the host

Implements [AAX_IComponentDescriptor](#).

14.151.3.11 AddPrivateData()

```
AAX_Result AAX_VComponentDescriptor::AddPrivateData (
    AAX_CFieldIndex inFieldIndex,
    int32_t inDataSize,
    uint32_t inOptions ) [virtual]
```

Adds a private data port to the algorithm context.

Defines a read/write data port for private state data. Data written to this port will be maintained by the host between calls to the algorithm context.

See also

`alg_pd_registration`

Parameters

in	<i>inFieldIndex</i>	Unique identifier for the port, generated using AAX_FIELD_INDEX
in	<i>inDataSize</i>	Size of the data packets that will be sent to this port
in	<i>inOptions</i>	Options that define the private data port's behavior

Implements [AAX_IComponentDescriptor](#).

14.151.3.12 AddTemporaryData()

```
AAX_Result AAX_VComponentDescriptor::AddTemporaryData (
    AAX_CFieldIndex inFieldIndex,
    uint32_t inDataElementSize ) [virtual]
```

Adds a block of data to a context that is not saved between callbacks and is scaled by the system buffer size.

This can be very useful if you use block processing and need to store intermediate results. Just specify your base element size and the system will scale the overall block size by the buffer size. For example, to create a buffer of floats that is the length of the block, specify 4 bytes as the elementsize.

This data block does not retain state across callback and can also be reused across instances on memory constrained systems.

Parameters

in	<i>inFieldIndex</i>	Unique identifier for the port, generated using AAX_FIELD_INDEX
in	<i>inDataElementSize</i>	The size of a single piece of data in the block. This number will be multiplied by the processing block size to determine total block size.

Implements [AAX_IComponentDescriptor](#).

14.151.3.13 AddDmaInstance()

```
AAX_Result AAX_VComponentDescriptor::AddDmaInstance (
    AAX_CFieldIndex inFieldIndex,
    AAX_IDma::EMode inDmaMode ) [virtual]
```

Adds a DMA field to the plug-in's context.

DMA (direct memory access) provides efficient reads from and writes to external memory on the DSP. DMA behavior is emulated in host-based plug-ins for cross-platform portability.

Note

The order in which DMA instances are added defines their priority and therefore order of execution of DMA operations. In most plug-ins, Scatter fields should be placed first in order to achieve the lowest possible access latency.

For more information, see [Direct Memory Access](#) .

Todo Update the DMA system management such that operation priority can be set arbitrarily

Parameters

in	<i>inFieldIndex</i>	Unique identifier for the field, generated using AAX_FIELD_INDEX
in	<i>inDmaMode</i>	AAX_IDma::EMode that will apply to this field

Implements [AAX_IComponentDescriptor](#).

14.151.3.14 AddMeters()

```
AAX_Result AAX_VComponentDescriptor::AddMeters (
    AAX_CFieldIndex inFieldIndex,
    const AAX_CTypeID * inMeterIDs,
    const uint32_t inMeterCount ) [virtual]
```

Adds a meter field to the plug-in's context.

Meter fields include an array of meter tap values, with one tap per meter per context. Only one meter field should be added per Component. Individual meter behaviors can be described at the Effect level.

For more information, see [Plug-in meters](#) .

Parameters

in	<i>inFieldIndex</i>	Unique identifier for the field, generated using AAX_FIELD_INDEX
in	<i>inMeterIDs</i>	Array of 32-bit IDs, one for each meter. Meter IDs must be unique within the Effect.
in	<i>inMeterCount</i>	The number of meters included in this field

Implements [AAX_IComponentDescriptor](#).

14.151.3.15 AddMIDINode()

```
AAX_Result AAX_VComponentDescriptor::AddMIDINode (
    AAX_CFieldIndex inFieldIndex,
    AAX_EMIDINodeType inNodeType,
    const char inNodeName[],
    uint32_t channelMask ) [virtual]
```

Adds a MIDI node field to the plug-in's context.

- Data type: [AAX_IMIDINode](#) *

The resulting MIDI node data will be available both in the algorithm context and in the plug-in's [data model](#) via [UpdateMIDINodes\(\)](#).

To add a MIDI node that is only accessible to the plug-in's data model, use [AAX_IEffectDescriptor::AddControlMIDINode\(\)](#)

Host Compatibility Notes Due to current restrictions MIDI data won't be delivered to DSP algorithms, only to AAX Native.

Parameters

in	<i>inFieldIndex</i>	The ID of the port. MIDI node ports should formatted as a pointer to an AAX_IMIDINode .
in	<i>inNodeType</i>	The type of MIDI node, as AAX_EMIDINodeType
in	<i>inNodeName</i>	The name of the MIDI node as it should appear in the host's UI
in	<i>channelMask</i>	The channel mask for the MIDI node. This parameter specifies used MIDI channels. For Global MIDI nodes, use a mask of AAX_EMidiGlobalNodeSelectors

Implements [AAX_IComponentDescriptor](#).

14.151.3.16 NewPropertyMap()

```
AAX_IPropertyMap* AAX_VComponentDescriptor::NewPropertyMap ( ) const [virtual]
```

Creates a new, empty property map.

The component descriptor owns the reference to the resulting property map, and the underlying property map is destroyed when the component descriptor is released.

This implementation retains each generated [AAX_IPropertyMap](#) and destroys the property map upon [AAX_VComponentDescriptor](#) destruction

Implements [AAX_IComponentDescriptor](#).

14.151.3.17 DuplicatePropertyMap()

```
AAX_IPropertyMap* AAX_VComponentDescriptor::DuplicatePropertyMap (
    AAX_IPropertyMap * inPropertyMap ) const [virtual]
```

Creates a new property map using an existing property map.

The component descriptor owns the reference to the resulting property map, and the underlying property map is destroyed when the component descriptor is released.

Parameters

in	<i>inPropertyMap</i>	The property values in this map will be copied into the new map
----	----------------------	---

This implementation retains each generated [AAX_IPropertyMap](#) and destroys the property map upon [AAX_VComponentDescriptor](#) destruction

Implements [AAX_IComponentDescriptor](#).

14.151.3.18 AddProcessProc_Native()

```
virtual AAX_Result AAX_VComponentDescriptor::AddProcessProc_Native (
    AAX_CProcessProc inProcessProc,
    AAX_IPropertyMap * inProperties = NULL,
    AAX_CInstanceInitProc inInstanceInitProc = NULL,
    AAX_CBackgroundProc inBackgroundProc = NULL,
    AAX_CSelector * outProcID = NULL ) [virtual]
```

Registers an algorithm processing entrypoint (process procedure) for the native architecture.

Parameters

in	<i>inProcessProc</i>	Symbol for this processing callback
in	<i>inProperties</i>	A property map for this processing callback. The property map's values are copied by the host and associated with the new ProcessProc. The property map contents are unchanged and the map may be re-used when registering additional ProcessProcs.
in	<i>inInstanceInitProc</i>	Initialization routine that will be called when a new instance of the Effect is created. See Algorithm initialization .
in	<i>inBackgroundProc</i>	Background routine that will be called in an idle context within the same address space as the associated process procedure. See Background processing callback
Generated by Doxygen out	<i>outProcID</i>	

Todo document this parameter

Implements [AAX_IComponentDescriptor](#).

14.151.3.19 AddProcessProc_TI()

```
virtual AAX_Result AAX_VComponentDescriptor::AddProcessProc_TI (
    const char inDLLFileNameUTF8[],
    const char inProcessProcSymbol[],
    AAX_IPropertyMap * inProperties = NULL,
    const char inInstanceInitProcSymbol[] = NULL,
    const char inBackgroundProcSymbol[] = NULL,
    AAX_CSelector * outProcID = NULL ) [virtual]
```

Registers an algorithm processing entrypoint (process procedure) for the native architecture.

Parameters

in	<i>inDLLFileNameUTF8</i>	UTF-8 encoded filename for the ELF DLL containing the algorithm code fragment
in	<i>inProcessProcSymbol</i>	Symbol for this processing callback
in	<i>inProperties</i>	A property map for this processing callback. The property map's values are copied by the host and associated with the new ProcessProc. The property map contents are unchanged and the map may be re-used when registering additional ProcessProcs.
in	<i>inInstanceInitProcSymbol</i>	Initialization routine that will be called when a new instance of the Effect is created. Must be included in the same DLL as the main algorithm entrypoint. See Algorithm initialization .
in	<i>inBackgroundProcSymbol</i>	Background routine that will be called in an idle context within the same address space as the associated process procedure. Must be included in the same DLL as the main algorithm entrypoint. See Background processing callback
out	<i>outProcID</i>	

Todo document this parameter

Implements [AAX_IComponentDescriptor](#).

14.151.3.20 AddProcessProc()

```
virtual AAX_Result AAX_VComponentDescriptor::AddProcessProc (
    AAX_IPropertyMap * inProperties,
    AAX_CSelector * outProcIDs = NULL,
    int32_t inProcIDsSize = 0 ) [virtual]
```

Registers one or more algorithm processing entrypoints (process procedures)

Any non-overlapping set of processing entrypoints may be specified. Typically this can be used to specify both Native and TI entrypoints using the same call.

The AAX Library implementation of this method includes backwards compatibility logic to complete the ProcessProc registration on hosts which do not support this method. Therefore plug-in code may use this single registration routine instead of separate calls to [AddProcessProc_Native\(\)](#), [AddProcessProc_TI\(\)](#), etc. regardless of the host version.

The following properties replace the input arguments to the platform-specific registration methods:

[AddProcessProc_Native\(\)](#) ([AAX_eProperty_PluginID_Native](#), [AAX_eProperty_PluginID_AudioSuite](#))

- [AAX_CProcessProc](#) *iProcessProc*: [AAX_eProperty_NativeProcessProc](#) (required)
- [AAX_CInstanceInitProc](#) *iInstanceInitProc*: [AAX_eProperty_NativeInstanceInitProc](#) (optional)
- [AAX_CBackgroundProc](#) *iBackgroundProc*: [AAX_eProperty_NativeBackgroundProc](#) (optional)

[AddProcessProc_TI\(\)](#) ([AAX_eProperty_PluginID_TI](#))

- `const char inDLLFileNameUTF8[]`: [AAX_eProperty_TIDLLFileName](#) (required)
- `const char iProcessProcSymbol[]`: [AAX_eProperty_TIPProcessProc](#) (required)
- `const char iInstanceInitProcSymbol[]`: [AAX_eProperty_TIInstanceInitProc](#) (optional)
- `const char iBackgroundProcSymbol[]`: [AAX_eProperty_TIBackgroundProc](#) (optional)

If any platform-specific plug-in ID property is present in *iProperties* then [AddProcessProc\(\)](#) will check for the required properties for that platform.

Note

[AAX_eProperty_AudioBufferLength](#) will be ignored for the Native and AudioSuite ProcessProcs since it should only be used for AAX DSP.

Parameters

in	<i>inProperties</i>	A property map for this processing callback. The property map's values are copied by the host and associated with the new ProcessProc. The property map contents are unchanged and the map may be re-used when registering additional ProcessProcs.
out	<i>outProcIDs</i>	

Todo document this parameter Returned array will be NULL-terminated

Parameters

in	<i>inProcIDsSize</i>	The size of the array provided to <i>oProcIDs</i> . If <i>oProcIDs</i> is non-NULL but <i>iProcIDsSize</i> is not large enough for all of the registered ProcessProcs (plus one for NULL termination) then this method will fail with AAX_ERROR_ARGUMENT_BUFFER_OVERFLOW
----	----------------------	--

Implements [AAX_IComponentDescriptor](#).

14.151.3.21 GetIUnknown()

```
IACFUnknown* AAX_VComponentDescriptor::GetIUnknown (
    void ) const
```

14.151.4 Friends And Related Function Documentation

14.151.4.1 AAX_VPropertyMap

```
friend class AAX\_VPropertyMap [friend]
```

The documentation for this class was generated from the following file:

- [AAX_VComponentDescriptor.h](#)

14.152 AAX_VController Class Reference

```
#include <AAX_VController.h>
```

Inheritance diagram for AAX_VController:

Collaboration diagram for AAX_VController:

14.152.1 Description

Version-managed concrete [Controller](#) class.

For usage information, see [Host-provided interfaces](#)

Public Member Functions

- [AAX_VController](#) ([IACFUnknown](#) *pUnknown)
- [~AAX_VController](#) () override
- [AAX_Result](#) [GetEffectID](#) ([AAX_IString](#) *outEffectID) const [AAX_OVERRIDE](#)
- [AAX_Result](#) [GetSampleRate](#) ([AAX_CSampleRate](#) *outSampleRate) const [AAX_OVERRIDE](#)
CALL: Returns the current literal sample rate.
- [AAX_Result](#) [GetInputStemFormat](#) ([AAX_EStemFormat](#) *outStemFormat) const [AAX_OVERRIDE](#)
CALL: Returns the plug-in's input stem format.
- [AAX_Result](#) [GetOutputStemFormat](#) ([AAX_EStemFormat](#) *outStemFormat) const [AAX_OVERRIDE](#)
CALL: Returns the plug-in's output stem format.
- [AAX_Result](#) [GetSignalLatency](#) ([int32_t](#) *outSamples) const [AAX_OVERRIDE](#)
CALL: Returns the most recent signal (algorithmic) latency that has been published by the plug-in.
- [AAX_Result](#) [GetHybridSignalLatency](#) ([int32_t](#) *outSamples) const [AAX_OVERRIDE](#)
CALL: Returns the latency between the algorithm normal input samples and the inputs returning from the hybrid component.
- [AAX_Result](#) [GetPlugInTargetPlatform](#) ([AAX_CTargetPlatform](#) *outTargetPlatform) const [AAX_OVERRIDE](#)
CALL: Returns execution platform type, native or TI.
- [AAX_Result](#) [GetIsAudioSuite](#) ([AAX_CBoolean](#) *outIsAudioSuite) const [AAX_OVERRIDE](#)
CALL: Returns true for AudioSuite instances.
- [AAX_Result](#) [GetCycleCount](#) ([AAX_EProperty](#) inWhichCycleCount, [AAX_CPropertyValue](#) *outNumCycles) const [AAX_OVERRIDE](#)
CALL: returns the plug-in's current real-time DSP cycle count.
- [AAX_Result](#) [GetTODLocation](#) ([AAX_CTimeOfDay](#) *outTODLocation) const [AAX_OVERRIDE](#)
CALL: Returns the current Time Of Day (TOD) of the system.
- [AAX_Result](#) [GetCurrentAutomationTimestamp](#) ([AAX_CTransportCounter](#) *outTimestamp) const [AAX_OVERRIDE](#)
CALL: Returns the current automation timestamp if called during the [GenerateCoefficients\(\)](#) call AND the generation of coefficients is being triggered by an automation point instead of immediate changes.
- [AAX_Result](#) [GetHostName](#) ([AAX_IString](#) *outHostNameString) const [AAX_OVERRIDE](#)
CALL: Returns name of the host application this plug-in instance is being loaded by. This string also typically includes version information.
- [AAX_Result](#) [SetSignalLatency](#) ([int32_t](#) inNumSamples) [AAX_OVERRIDE](#)
CALL: Submits a request to change the delay compensation value that the host uses to account for the plug-in's signal (algorithmic) latency.
- [AAX_Result](#) [SetCycleCount](#) ([AAX_EProperty](#) *inWhichCycleCounts, [AAX_CPropertyValue](#) *iValues, [int32_t](#) numValues) [AAX_OVERRIDE](#)
CALL: Indicates a change in the plug-in's real-time DSP cycle count.
- [AAX_Result](#) [PostPacket](#) ([AAX_CFieldIndex](#) inFieldIndex, const void *inPayloadP, [uint32_t](#) inPayloadSize) [AAX_OVERRIDE](#)
CALL: Posts a data packet to the host for routing between plug-in components.
- [AAX_Result](#) [SendNotification](#) ([AAX_CTypeID](#) inNotificationType, const void *inNotificationData, [uint32_t](#) inNotificationDataSize) [AAX_OVERRIDE](#)
CALL: Dispatch a notification.
- [AAX_Result](#) [SendNotification](#) ([AAX_CTypeID](#) inNotificationType) [AAX_OVERRIDE](#)
CALL: Sends an event to the GUI (no payload)
Note
Not an AAX interface method
- [AAX_Result](#) [GetCurrentMeterValue](#) ([AAX_CTypeID](#) inMeterID, float *outMeterValue) const [AAX_OVERRIDE](#)
CALL: Retrieves the current value of a host-managed plug-in meter.
- [AAX_Result](#) [GetMeterPeakValue](#) ([AAX_CTypeID](#) inMeterID, float *outMeterPeakValue) const [AAX_OVERRIDE](#)
CALL: Retrieves the currently held peak value of a host-managed plug-in meter.
- [AAX_Result](#) [ClearMeterPeakValue](#) ([AAX_CTypeID](#) inMeterID) const [AAX_OVERRIDE](#)

- CALL: Clears the peak value from a host-managed plug-in meter.*
- [AAX_Result GetMeterClipped](#) ([AAX_CTypeID](#) inMeterID, [AAX_CBoolean](#) *outClipped) const [AAX_OVERRIDE](#)
- CALL: Retrieves the clipped flag from a host-managed plug-in meter.*
- [AAX_Result ClearMeterClipped](#) ([AAX_CTypeID](#) inMeterID) const [AAX_OVERRIDE](#)
- CALL: Clears the clipped flag from a host-managed plug-in meter.*
- [AAX_Result GetMeterCount](#) (uint32_t *outMeterCount) const [AAX_OVERRIDE](#)
- CALL: Retrieves the number of host-managed meters registered by a plug-in.*
- [AAX_Result GetNextMIDIPacket](#) ([AAX_CFieldIndex](#) *outPort, [AAX_CMidiPacket](#) *outPacket) [AAX_OVERRIDE](#)
- CALL: Retrieves MIDI packets for described MIDI nodes.*
- [AAX_IPageTable * CreateTableCopyForEffect](#) ([AAX_CPropertyValue](#) inManufacturerID, [AAX_CPropertyValue](#) inProductID, [AAX_CPropertyValue](#) inPlugInID, uint32_t inTableType, int32_t inTablePageSize) const [AAX_OVERRIDE](#)
- Copy the current page table data for a particular plug-in type.*
- [AAX_IPageTable * CreateTableCopyForLayout](#) (const char *inEffectID, const char *inLayoutName, uint32_t inTableType, int32_t inTablePageSize) const [AAX_OVERRIDE](#)
- Copy the current page table data for a particular plug-in effect and page table layout.*
- [AAX_IPageTable * CreateTableCopyForEffectFromFile](#) (const char *inPageTableFilePath, [AAX_ETextEncoding](#) inFilePathEncoding, [AAX_CPropertyValue](#) inManufacturerID, [AAX_CPropertyValue](#) inProductID, [AAX_CPropertyValue](#) inPlugInID, uint32_t inTableType, int32_t inTablePageSize) const [AAX_OVERRIDE](#)
- Copy the current page table data for a particular plug-in type.*
- [AAX_IPageTable * CreateTableCopyForLayoutFromFile](#) (const char *inPageTableFilePath, [AAX_ETextEncoding](#) inFilePathEncoding, const char *inLayoutName, uint32_t inTableType, int32_t inTablePageSize) const [AAX_OVERRIDE](#)
- Copy the current page table data for a particular plug-in effect and page table layout.*

14.152.2 Constructor & Destructor Documentation

14.152.2.1 AAX_VController()

```
AAX_VController::AAX_VController (
    IAcfUnknown * pUnknown )
```

14.152.2.2 ~AAX_VController()

```
AAX_VController::~AAX_VController ( ) [override]
```

14.152.3 Member Function Documentation

14.152.3.1 GetEffectID()

```
AAX_Result AAX_VController::GetEffectID (
    AAX_IString * outEffectID ) const [virtual]
```

Implements [AAX_IController](#).

14.152.3.2 GetSampleRate()

```
AAX_Result AAX_VController::GetSampleRate (
    AAX_CSampleRate * outSampleRate ) const [virtual]
```

CALL: Returns the current literal sample rate.

Parameters

out	<i>outSampleRate</i>	The current sample rate
-----	----------------------	-------------------------

Implements [AAX_IController](#).

14.152.3.3 GetInputStemFormat()

```
AAX_Result AAX_VController::GetInputStemFormat (
    AAX_EStemFormat * outStemFormat ) const [virtual]
```

CALL: Returns the plug-in's input stem format.

Parameters

out	<i>outStemFormat</i>	The current input stem format
-----	----------------------	-------------------------------

Implements [AAX_IController](#).

14.152.3.4 GetOutputStemFormat()

```
AAX_Result AAX_VController::GetOutputStemFormat (
    AAX_EStemFormat * outStemFormat ) const [virtual]
```

CALL: Returns the plug-in's output stem format.

Parameters

out	<i>outStemFormat</i>	The current output stem format
-----	----------------------	--------------------------------

Implements [AAX_IController](#).

14.152.3.5 GetSignalLatency()

```
AAX_Result AAX_VController::GetSignalLatency (
    int32_t * outSamples ) const [virtual]
```

CALL: Returns the most recent signal (algorithmic) latency that has been published by the plug-in.

This method provides the most recently published signal latency. The host may not have updated its delay compensation to match this signal latency yet, so plug-ins that dynamically change their latency using [SetSignalLatency\(\)](#) should always wait for an [AAX_eNotificationEvent_SignalLatencyChanged](#) notification before updating its algorithm to incur this latency.

See also

[SetSignalLatency\(\)](#)

Parameters

out	<i>outSamples</i>	The number of samples of signal delay published by the plug-in
-----	-------------------	--

Implements [AAX_IController](#).

14.152.3.6 GetHybridSignalLatency()

```
AAX_Result AAX_VController::GetHybridSignalLatency (
    int32_t * outSamples ) const [virtual]
```

CALL: Returns the latency between the algorithm normal input samples and the inputs returning from the hybrid component.

This method provides the number of samples that the AAX host expects the plug-in to delay a signal. The host will use this value when accounting for latency across the system.

Note

This value will generally scale up with sample rate, although it's not a simple multiple due to some fixed overhead. This value will be fixed for any given sample rate regardless of other buffer size settings in the host app.

Parameters

out	<i>outSamples</i>	The number of samples of hybrid signal delay
-----	-------------------	--

Implements [AAX_IController](#).

14.152.3.7 GetPlugInTargetPlatform()

```
AAX_Result AAX_VController::GetPlugInTargetPlatform (
    AAX_CTargetPlatform * outTargetPlatform ) const [virtual]
```

CALL: Returns execution platform type, native or TI.

Parameters

out	<i>outTargetPlatform</i>	The type of the current execution platform as one of AAX_ETargetPlatform .
-----	--------------------------	--

Implements [AAX_IController](#).

14.152.3.8 GetIsAudioSuite()

```
AAX_Result AAX_VController::GetIsAudioSuite (
    AAX_CBoolean * outIsAudioSuite ) const [virtual]
```

CALL: Returns true for AudioSuite instances.

Parameters

out	<i>outIsAudioSuite</i>	The boolean flag which indicate true for AudioSuite instances.
-----	------------------------	--

Implements [AAX_IController](#).

14.152.3.9 GetCycleCount()

```
AAX_Result AAX_VController::GetCycleCount (
    AAX_EProperty inWhichCycleCount,
    AAX_CPropertyValue * outNumCycles ) const [virtual]
```

CALL: returns the plug-in's current real-time DSP cycle count.

This method provides the number of cycles that the AAX host expects the DSP plug-in to consume. The host uses this value when allocating DSP resources for the plug-in.

Note

A plug-in should never apply a DSP algorithm with more demanding resource requirements than what is currently accounted for by the host. To set a higher cycle count value, a plug-in must call [AAX_IController::SetCycleCount\(\)](#), then poll [AAX_IController::GetCycleCount\(\)](#) until the new value has been applied. Once the host has recognized the new cycle count value, the plug-in may apply the more demanding algorithm.

Parameters

in	<i>inWhichCycleCount</i>	Selector for the requested cycle count metric. One of: • AAX_eProperty_TI_SharedCycleCount • AAX_eProperty_TI_InstanceCycleCount • AAX_eProperty_TI_MaxInstancesPerChip
in	<i>outNumCycles</i>	The current value of the selected cycle count metric

Todo PLACEHOLDER - NOT CURRENTLY IMPLEMENTED IN HOST

Implements [AAX_IController](#).

14.152.3.10 GetTODLocation()

```
AAX_Result AAX_VController::GetTODLocation (
    AAX_CTimeOfDay * outTODLocation ) const [virtual]
```

CALL: Returns the current Time Of Day (TOD) of the system.

This method provides a plug-in the TOD (in samples) of the current system. TOD is the number of samples that the playhead has traversed since the beginning of playback.

Note

The TOD value is the immediate value of the audio engine playhead. This value is incremented within the audio engine's real-time rendering context; it is not synchronized with non-real-time calls to plug-in interface methods.

Parameters

out	<i>outTODLocation</i>	The current Time Of Day as set by the host
-----	-----------------------	--

Implements [AAX_IController](#).

14.152.3.11 GetCurrentAutomationTimestamp()

```
AAX_Result AAX_VController::GetCurrentAutomationTimestamp (
    AAX_CTransportCounter * outTimestamp ) const [virtual]
```

CALL: Returns the current automation timestamp if called during the [GenerateCoefficients\(\)](#) call AND the generation of coefficients is being triggered by an automation point instead of immediate changes.

Note

This function will return 0 if called from outside of [GenerateCoefficients\(\)](#) or if the [GenerateCoefficients\(\)](#) call was initiated due to a non-automated change. In those cases, you can get your sample offset from the transport start using [GetTODLocation\(\)](#).

Parameters

out	<i>outTimestamp</i>	The current coefficient timestamp. Sample count from transport start.
-----	---------------------	---

Implements [AAX_IController](#).

14.152.3.12 GetHostName()

```
AAX_Result AAX_VController::GetHostName (
    AAX_IString * outHostNameString ) const [virtual]
```

CALL: Returns name of the host application this plug-in instance is being loaded by. This string also typically includes version information.

Host Compatibility Notes Pro Tools versions from Pro Tools 11.0 to Pro Tools 12.3.1 will return a generic version string to this call. This issue is resolved beginning in Pro Tools 12.4.

Parameters

out	<i>outHostNameString</i>	The name of the current host application.
-----	--------------------------	---

Implements [AAX_IController](#).

14.152.3.13 SetSignalLatency()

```
AAX_Result AAX_VController::SetSignalLatency (
    int32_t inNumSamples ) [virtual]
```

CALL: Submits a request to change the delay compensation value that the host uses to account for the plug-in's signal (algorithmic) latency.

This method is used to request a change in the number of samples that the AAX host expects the plug-in to delay a signal.

The host is not guaranteed to immediately apply the new latency value. A plug-in should avoid incurring an actual algorithmic latency that is different than the latency accounted for by the host.

To set a new latency value, a plug-in must call [AAX_IController::SetSignalLatency\(\)](#), then wait for an [AAX_eNotificationEvent_SignalLatencyChanged](#) notification. Once this notification has been received, [AAX_IController::GetSignalLatency\(\)](#) will reflect the updated latency value and the plug-in should immediately apply any relevant algorithmic changes that alter its latency to this new value.

Warning

Parameters which affect the latency of a plug-in should not be made available for control through automation. This will result in audible glitches when delay compensation is adjusted while playing back automation for these parameters.

Parameters

in	<i>inNumSamples</i>	The number of samples of signal delay that the plug-in requests to incur
----	---------------------	--

Implements [AAX_IController](#).

14.152.3.14 SetCycleCount()

```
AAX_Result AAX_VController::SetCycleCount (
    AAX_EProperty * inWhichCycleCounts,
    AAX_CPropertyValue * iValues,
    int32_t numValues ) [virtual]
```

CALL: Indicates a change in the plug-in's real-time DSP cycle count.

This method is used to request a change in the number of cycles that the AAX host expects the DSP plug-in to consume.

Note

A plug-in should never apply a DSP algorithm with more demanding resource requirements than what is currently accounted for by the host. To set a higher cycle count value, a plug-in must call [AAX_IController::SetCycleCount\(\)](#), then poll [AAX_IController::GetCycleCount\(\)](#) until the new value has been applied. Once the host has recognized the new cycle count value, the plug-in may apply the more demanding algorithm.

Parameters

in	<i>inWhichCycleCounts</i>	Array of selectors indicating the specific cycle count metrics that should be set. Each selector must be one of: AAX_eProperty_TI_SharedCycleCount AAX_eProperty_TI_InstanceCycleCount AAX_eProperty_TI_MaxInstancesPerChip
in	<i>iValues</i>	An array of values requested, one for each of the selected cycle count metrics.
in	<i>numValues</i>	The size of <i>iValues</i>

Todo PLACEHOLDER - NOT CURRENTLY IMPLEMENTED IN HOST

Implements [AAX_IController](#).

14.152.3.15 PostPacket()

```
AAX_Result AAX_VController::PostPacket (
    AAX_CFieldIndex inFieldIndex,
    const void * inPayloadP,
    uint32_t inPayloadSize ) [virtual]
```

CALL: Posts a data packet to the host for routing between plug-in components.

The posted packet is identified with a [AAX_CFieldIndex](#) packet index value, which is equivalent to the target data port's identifier. The packet's payload must have the expected size for the given packet index / data port, as defined when the port is created in [Describe](#). See [AAX_IComponentDescriptor::AddDataInPort\(\)](#).

Warning

Any data structures that will be passed between platforms (for example, sent to a TI DSP in an AAX DSP plug-in) must be properly data-aligned for compatibility across both platforms. See [AAX_ALIGN_FILE_ALG](#) for more information about guaranteeing cross-platform compatibility of data structures used for algorithm processing.

Note

All calls to this method should be made within the scope of [AAX_IEffectParameters::GenerateCoefficients\(\)](#). Calls from outside this method may result in packets not being delivered. See [PT-206161](#)

Parameters

in	<i>inFieldIndex</i>	The packet's destination port
in	<i>inPayloadP</i>	A pointer to the packet's payload data
in	<i>inPayloadSize</i>	The size, in bytes, of the payload data

Implements [AAX_IController](#).

14.152.3.16 SendNotification() [1/2]

```
AAX_Result AAX_VController::SendNotification (
    AAX_CTypeID inNotificationType,
    const void * inNotificationData,
    uint32_t inNotificationDataSize ) [virtual]
```

CALL: Dispatch a notification.

The notification is handled by the host and may be delivered back to other plug-in components such as the GUI or data model (via [AAX_IEffectGUI::NotificationReceived\(\)](#) or [AAX_IEffectParameters::NotificationReceived\(\)](#), respectively) depending on the notification type.

The host may choose to dispatch the posted notification either synchronously or asynchronously.

See the [AAX_ENotificationEvent](#) documentation for more information.

This method is supported by AAX V2 Hosts only. Check the return code on the return of this function. If the error is [AAX_ERROR_UNIMPLEMENTED](#), your plug-in is being loaded into a host that doesn't support this feature.

Parameters

in	<i>inNotificationType</i>	Type of notification to send
in	<i>inNotificationData</i>	Block of notification data
in	<i>inNotificationDataSize</i>	Size of <i>inNotificationData</i> , in bytes

Implements [AAX_IController](#).

14.152.3.17 SendNotification() [2/2]

```
AAX_Result AAX_VController::SendNotification (
    AAX_CTypeID inNotificationType ) [virtual]
```

CALL: Sends an event to the GUI (no payload)

Note

Not an [AAX](#) interface method

This version of the notification method is a convenience for notifications which do not take any payload data. Internally, it simply calls [AAX_IController::SendNotification\(AAX_CTypeID, const void*, uint32_t\)](#) with a null payload.

Parameters

in	<i>inNotificationType</i>	Type of notification to send
----	---------------------------	------------------------------

Note

Not an [AAX](#) interface method

Implements [AAX_IController](#).

14.152.3.18 GetCurrentMeterValue()

```
AAX_Result AAX_VController::GetCurrentMeterValue (
    AAX_CTypeID inMeterID,
    float * outMeterValue ) const [virtual]
```

CALL: Retrieves the current value of a host-managed plug-in meter.

Parameters

in	<i>inMeterID</i>	ID of the meter that is being queried
out	<i>outMeterValue</i>	The queried meter's current value

Implements [AAX_IController](#).

14.152.3.19 GetMeterPeakValue()

```
AAX_Result AAX_VController::GetMeterPeakValue (
    AAX_CTypeID inMeterID,
    float * outMeterPeakValue ) const [virtual]
```

CALL: Retrieves the currently held peak value of a host-managed plug-in meter.

Parameters

in	<i>inMeterID</i>	ID of the meter that is being queried
out	<i>outMeterPeakValue</i>	The queried meter's currently held peak value

Implements [AAX_IController](#).

14.152.3.20 ClearMeterPeakValue()

```
AAX_Result AAX_VController::ClearMeterPeakValue (
    AAX_CTypeID inMeterID ) const [virtual]
```

CALL: Clears the peak value from a host-managed plug-in meter.

Parameters

in	<i>inMeterID</i>	ID of the meter that is being cleared
----	------------------	---------------------------------------

Implements [AAX_IController](#).

14.152.3.21 GetMeterClipped()

```
AAX_Result AAX_VController::GetMeterClipped (
    AAX_CTypeID inMeterID,
    AAX_CBoolean * outClipped ) const [virtual]
```

CALL: Retrieves the clipped flag from a host-managed plug-in meter.

See [AAX_IComponentDescriptor::AddMeters\(\)](#).

Parameters

in	<i>inMeterID</i>	ID of the meter that is being queried.
out	<i>outClipped</i>	The queried meter's clipped flag.

Implements [AAX_IController](#).

14.152.3.22 ClearMeterClipped()

```
AAX_Result AAX_VController::ClearMeterClipped (
    AAX_CTypeID inMeterID ) const [virtual]
```

CALL: Clears the clipped flag from a host-managed plug-in meter.

See [AAX_IComponentDescriptor::AddMeters\(\)](#).

Parameters

in	<i>inMeterID</i>	ID of the meter that is being cleared.
----	------------------	--

Implements [AAX_IController](#).

14.152.3.23 GetMeterCount()

```
AAX_Result AAX_VController::GetMeterCount (
    uint32_t * outMeterCount ) const [virtual]
```

CALL: Retrieves the number of host-managed meters registered by a plug-in.

See [AAX_IComponentDescriptor::AddMeters\(\)](#).

Parameters

out	<i>outMeterCount</i>	The number of registered plug-in meters.
-----	----------------------	--

Implements [AAX_IController](#).

14.152.3.24 GetNextMIDIpacket()

```
AAX_Result AAX_VController::GetNextMIDIpacket (
    AAX_CFieldIndex * outPort,
    AAX_CMidiPacket * outPacket ) [virtual]
```

CALL: Retrieves MIDI packets for described MIDI nodes.

Parameters

out	<i>outPort</i>	port ID of the MIDI node that has unhandled packet
out	<i>outPacket</i>	The MIDI packet

Implements [AAX_IController](#).

14.152.3.25 CreateTableCopyForEffect()

```
AAX_IPageTable* AAX_VController::CreateTableCopyForEffect (
    AAX_CPropertyValue inManufacturerID,
    AAX_CPropertyValue inProductID,
    AAX_CPropertyValue inPlugInID,
    uint32_t inTableType,
    int32_t inTablePageSize ) const [virtual]
```

Copy the current page table data for a particular plug-in type.

The host may restrict plug-ins to only copying page table data from certain plug-in types, such as plug-ins from the same manufacturer or plug-in types within the same effect.

See [Page Table Guide](#) for more information about page tables.

Returns

A new page table object to which the requested page table data has been copied. Ownership of this object passes to the caller.

a null pointer if the requested plug-in type is unknown, if `inTableType` is unknown or if `inTablePageSize` is not a supported size for the given table type.

Parameters

in	<i>inManufacturerID</i>	Manufacturer ID of the desired plug-in type
in	<i>inProductID</i>	Product ID of the desired plug-in type
in	<i>inPlugInID</i>	Type ID of the desired plug-in type (AAX_eProperty_PlugInID_Native , AAX_eProperty_PlugInID_TI)
in	<i>inTableType</i>	Four-char type identifier for the requested table type (e.g. 'PgTL', 'Av81', etc.)
in	<i>inTablePageSize</i>	Page size for the requested table. Some tables support multiple page sizes.

Implements [AAX_IController](#).

14.152.3.26 CreateTableCopyForLayout()

```
AAX_IPageTable* AAX_VController::CreateTableCopyForLayout (
    const char * inEffectID,
    const char * inLayoutName,
    uint32_t inTableType,
    int32_t inTablePageSize ) const [virtual]
```

Copy the current page table data for a particular plug-in effect and page table layout.

The host may restrict plug-ins to only copying page table data from certain effects, such as effects registered within the current [AAX](#) plug-in bundle.

See [Page Table Guide](#) for more information about page tables.

Returns

A new page table object to which the requested page table data has been copied. Ownership of this object passes to the caller.

a null pointer if the requested effect ID is unknown or if `inLayoutName` is not a valid layout name for the page tables registered for the effect.

Parameters

in	<i>inEffectID</i>	Effect ID for the desired effect. See AAX_ICollection::AddEffect()
in	<i>inLayoutName</i>	Page table layout name ("name" attribute of the <code>PTLayout</code> XML tag)
in	<i>inTableType</i>	Four-char type identifier for the requested table type (e.g. 'PgTL', 'Av81', etc.)
in	<i>inTablePageSize</i>	Page size for the requested table. Some tables support multiple page sizes.

Implements [AAX_IController](#).

14.152.3.27 CreateTableCopyForEffectFromFile()

```
AAX_IPageTable* AAX_VController::CreateTableCopyForEffectFromFile (
    const char * inPageTableFilePath,
    AAX_ETextEncoding inFilePathEncoding,
    AAX_CPropertyValue inManufacturerID,
    AAX_CPropertyValue inProductID,
    AAX_CPropertyValue inPlugInID,
    uint32_t inTableType,
    int32_t inTablePageSize ) const [virtual]
```

Copy the current page table data for a particular plug-in type.

Returns

A new page table object to which the requested page table data has been copied. Ownership of this object passes to the caller.

a null pointer if the requested plug-in type is unknown, if `inTableType` is unknown or if `inTablePageSize` is not a supported size for the given table type.

Parameters

in	<i>inPageTableFilePath</i>	Path to XML page table file.
in	<i>inFilePathEncoding</i>	File path text encoding.
in	<i>inManufacturerID</i>	Manufacturer ID of the desired plug-in type
in	<i>inProductID</i>	Product ID of the desired plug-in type
in	<i>inPlugInID</i>	Type ID of the desired plug-in type (AAX_eProperty_PlugInID_Native , AAX_eProperty_PlugInID_TI)
in	<i>inTableType</i>	Four-char type identifier for the requested table type (e.g. 'PgTL', 'Av81', etc.)
in	<i>inTablePageSize</i>	Page size for the requested table. Some tables support multiple page sizes.

Implements [AAX_IController](#).

14.152.3.28 CreateTableCopyForLayoutFromFile()

```
AAX_IPageTable* AAX_VController::CreateTableCopyForLayoutFromFile (
    const char * inPageTableFilePath,
    AAX_ETextEncoding inFilePathEncoding,
    const char * inLayoutName,
    uint32_t inTableType,
    int32_t inTablePageSize ) const [virtual]
```

Copy the current page table data for a particular plug-in effect and page table layout.

Returns

A new page table object to which the requested page table data has been copied. Ownership of this object passes to the caller.

a null pointer if `inLayoutName` is not a valid layout name for the page tables file.

Parameters

in	<i>inPageTableFilePath</i>	Path to XML page table file.
in	<i>inFilePathEncoding</i>	File path text encoding.
in	<i>inLayoutName</i>	Page table layout name ("name" attribute of the <code>PTLayout</code> XML tag)
in	<i>inTableType</i>	Four-char type identifier for the requested table type (e.g. 'PgTL', 'Av81', etc.)
in	<i>inTablePageSize</i>	Page size for the requested table. Some tables support multiple page sizes.

Implements [AAX_IController](#).

The documentation for this class was generated from the following file:

- [AAX_VController.h](#)

14.153 AAX_VDataBufferWrapper Class Reference

```
#include <AAX_VDataBufferWrapper.h>
```

Inheritance diagram for `AAX_VDataBufferWrapper`:

Collaboration diagram for `AAX_VDataBufferWrapper`:

14.153.1 Description

Wrapper for an [AAX_IDataBuffer](#).

Like [AAX_IController](#) and similar classes, this class provides a non-ACF interface matching an ACF interface, in this case [AAX_IACFDataBuffer](#).

The implementation of this interface will contain a reference counted pointer to the underlying ACF interface. This interface may be extended with convenience functions that are not required on the underlying ACF interface.

Public Member Functions

- [AAX_VDataBufferWrapper](#) ([IACFUnknown](#) *iUnknown)
- [~AAX_VDataBufferWrapper](#) () [AAX_OVERRIDE](#)
- [AAX_Result Type](#) ([AAX_CTypeID](#) *oType) const [AAX_OVERRIDE](#)
- [AAX_Result Size](#) ([int32_t](#) *oSize) const [AAX_OVERRIDE](#)
- [AAX_Result Data](#) (void const **oBuffer) const [AAX_OVERRIDE](#)

14.153.2 Constructor & Destructor Documentation

14.153.2.1 AAX_VDataBufferWrapper()

```
AAX_VDataBufferWrapper::AAX_VDataBufferWrapper (
    IACFUnknown * iUnknown ) [explicit]
```

14.153.2.2 ~AAX_VDataBufferWrapper()

```
AAX_VDataBufferWrapper::~~AAX_VDataBufferWrapper ( )
```

14.153.3 Member Function Documentation

14.153.3.1 Type()

```
AAX\_Result AAX_VDataBufferWrapper::Type (
    AAX\_CTypeID * oType ) const [virtual]
```

The type of data contained in this buffer

This identifier must be sufficient for a client that knows the type to correctly interpret and use the data.

Implements [AAX_IDataBufferWrapper](#).

14.153.3.2 Size()

```
AAX\_Result AAX_VDataBufferWrapper::Size (
    int32\_t * oSize ) const [virtual]
```

The number of bytes of data in this buffer

Implements [AAX_IDataBufferWrapper](#).

14.153.3.3 Data()

```
AAX_Result AAX_VDataBufferWrapper::Data (
    void const ** oBuffer ) const [virtual]
```

The buffer of data

Implements [AAX_IDataBufferWrapper](#).

The documentation for this class was generated from the following file:

- [AAX_VDataBufferWrapper.h](#)

14.154 AAX_VDescriptionHost Class Reference

```
#include <AAX_VDescriptionHost.h>
```

Inheritance diagram for AAX_VDescriptionHost:

Collaboration diagram for AAX_VDescriptionHost:

14.154.1 Description

Versioned wrapper for access to host service interfaces provided during plug-in description

This object aggregates access to [AAX_IACFDescriptionHost](#) and [IACFDefinition](#), with support depending on the interface support level of the [IACFUnknown](#) which is passed to this object upon creation.

Public Member Functions

- [AAX_VDescriptionHost](#) ([IACFUnknown](#) *pUnknown)
- [~AAX_VDescriptionHost](#) () [AAX_OVERRIDE](#)
- const [AAX_IFeatureInfo](#) * [AcquireFeatureProperties](#) (const [AAX_Feature_UID](#) &inFeatureID) const [AAX_OVERRIDE](#)
- bool [Supported](#) () const
- [AAX_IACFDescriptionHost](#) * [DescriptionHost](#) ()
- const [AAX_IACFDescriptionHost](#) * [DescriptionHost](#) () const
- [IACFDefinition](#) * [HostDefinition](#) () const

14.154.2 Constructor & Destructor Documentation

14.154.2.1 AAX_VDescriptionHost()

```
AAX_VDescriptionHost::AAX_VDescriptionHost (
    IACFUnknown * pUnknown ) [explicit]
```

14.154.2.2 ~AAX_VDescriptionHost()

```
AAX_VDescriptionHost::~~AAX_VDescriptionHost ( )
```

14.154.3 Member Function Documentation

14.154.3.1 AcquireFeatureProperties()

```
const AAX_IFeatureInfo* AAX_VDescriptionHost::AcquireFeatureProperties (
    const AAX_Feature_UID & inFeatureID ) const [virtual]
```

Get the client's feature object for a given feature ID

Similar to `QueryInterface()` but uses a feature identifier rather than a true IID

Ownership of the returned object is passed to the caller; the caller is responsible for destroying the object, e.g. by capturing the returned object in a smart pointer.

```
// AAX_IDescriptionHost* descHost
std::unique_ptr<const AAX_IFeatureInfo> featureInfoPtr(descHost->AcquireFeatureProperties(someFeatureUID);
```

Returns

An [AAX_IFeatureInfo](#) interface with access to the host's feature properties for this feature.

NULL if the desired feature was not found or if an error occurred

Note

May return an [AAX_IFeatureInfo](#) object with limited method support, which would return an error such as [AAX_ERROR_NULL_OBJECT](#) or [AAX_ERROR_UNIMPLEMENTED](#) to interface calls.

If no [AAX_IFeatureInfo](#) is provided then that may mean that the host is unaware of the feature, or it may mean that the host is aware of the feature but has not implemented the AAX feature support interface for this feature yet.

Parameters

in	<i>inFeatureID</i>	Identifier of the requested feature
----	--------------------	-------------------------------------

Implements [AAX_IDescriptionHost](#).

14.154.3.2 Supported()

```
bool AAX_VDescriptionHost::Supported ( ) const [inline]
```

14.154.3.3 DescriptionHost() [1/2]

```
AAX_IACFDescriptionHost* AAX_VDescriptionHost::DescriptionHost ( ) [inline]
```

14.154.3.4 DescriptionHost() [2/2]

```
const AAX_IACFDescriptionHost* AAX_VDescriptionHost::DescriptionHost ( ) const [inline]
```

14.154.3.5 HostDefinition()

```
IACFDefinition* AAX_VDescriptionHost::HostDefinition ( ) const [inline]
```

The documentation for this class was generated from the following file:

- [AAX_VDescriptionHost.h](#)

14.155 AAX_VEffectDescriptor Class Reference

```
#include <AAX_VEffectDescriptor.h>
```

Inheritance diagram for AAX_VEffectDescriptor:

Collaboration diagram for AAX_VEffectDescriptor:

14.155.1 Description

Version-managed concrete [AAX_IEffectDescriptor](#) class.

Public Member Functions

- [AAX_VEffectDescriptor](#) ([IACFUnknown](#) *pUnkHost)
- [~AAX_VEffectDescriptor](#) () [AAX_OVERRIDE](#)
- [AAX_IComponentDescriptor](#) * [NewComponentDescriptor](#) () [AAX_OVERRIDE](#)
Create an instance of a component descriptor.
- [AAX_Result](#) [AddComponent](#) ([AAX_IComponentDescriptor](#) *inComponentDescriptor) [AAX_OVERRIDE](#)
Add a component to an instance of a component descriptor.
- [AAX_Result](#) [AddName](#) (const char *inPlugInName) [AAX_OVERRIDE](#)
Add a name to the Effect.
- [AAX_Result](#) [AddCategory](#) (uint32_t inCategory) [AAX_OVERRIDE](#)
Add a category to your plug-in. See [AAX_EPlugInCategory](#).
- [AAX_Result](#) [AddCategoryBypassParameter](#) (uint32_t inCategory, [AAX_CParamID](#) inParamID) [AAX_OVERRIDE](#)
Add a category to your plug-in. See [AAX_EPlugInCategory](#).
- [AAX_Result](#) [AddProcPtr](#) (void *inProcPtr, [AAX_CProcPtrID](#) inProcID) [AAX_OVERRIDE](#)
Add a process pointer.
- [AAX_IPropertyMap](#) * [NewPropertyMap](#) () [AAX_OVERRIDE](#)
Create a new property map.
- [AAX_Result](#) [SetProperties](#) ([AAX_IPropertyMap](#) *inProperties) [AAX_OVERRIDE](#)
Set the properties of a new property map.
- [AAX_Result](#) [AddResourceInfo](#) ([AAX_EResourceType](#) inResourceType, const char *inInfo) [AAX_OVERRIDE](#)
Set resource file info.
- [AAX_Result](#) [AddMeterDescription](#) ([AAX_CTypeID](#) inMeterID, const char *inMeterName, [AAX_IPropertyMap](#) *inProperties) [AAX_OVERRIDE](#)
Add name and property map to meter with given ID.
- [AAX_Result](#) [AddControlMIDINode](#) ([AAX_CTypeID](#) inNodeID, [AAX_EMIDINodeType](#) inNodeType, const char inNodeName[], uint32_t inChannelMask) [AAX_OVERRIDE](#)
Add a control MIDI node to the plug-in data model.
- [IACFUnknown](#) * [GetIUnknown](#) (void) const

14.155.2 Constructor & Destructor Documentation

14.155.2.1 AAX_VEffectDescriptor()

```
AAX_VEffectDescriptor::AAX_VEffectDescriptor (
    IACFUnknown * pUnkHost )
```

14.155.2.2 ~AAX_VEffectDescriptor()

```
AAX_VEffectDescriptor::~~AAX_VEffectDescriptor ( )
```

14.155.3 Member Function Documentation

14.155.3.1 NewComponentDescriptor()

```
AAX_IComponentDescriptor* AAX_VEffectDescriptor::NewComponentDescriptor ( ) [virtual]
```

Create an instance of a component descriptor.

This implementation retains each generated [AAX_IComponentDescriptor](#) and destroys the property map upon [AAX_VEffectDescriptor](#) destruction

Implements [AAX_IEffectDescriptor](#).

14.155.3.2 AddComponent()

```
AAX_Result AAX_VEffectDescriptor::AddComponent (
    AAX_IComponentDescriptor * inComponentDescriptor ) [virtual]
```

Add a component to an instance of a component descriptor.

Unlike with [AAX_ICollection::AddEffect\(\)](#), the [AAX_IEffectDescriptor](#) does not take ownership of the [AAX_IComponentDescriptor](#) that is passed to it in this method. The host copies out the contents of this descriptor, and thus the plug-in may re-use the same descriptor object when creating additional similar components.

Parameters

in	<i>inComponentDescriptor</i>	
----	------------------------------	--

Implements [AAX_IEffectDescriptor](#).

14.155.3.3 AddName()

```
AAX_Result AAX_VEffectDescriptor::AddName (
    const char * inPlugInName ) [virtual]
```

Add a name to the Effect.

May be called multiple times to add abbreviated Effect names.

Note

Every Effect must include at least one name variant with 31 or fewer characters, plus a null terminating character

Parameters

in	<i>inPlugInName</i>	The name assigned to the plug-in.
----	---------------------	-----------------------------------

Implements [AAX_IEffectDescriptor](#).

14.155.3.4 AddCategory()

```
AAX_Result AAX_VEffectDescriptor::AddCategory (
    uint32_t inCategory ) [virtual]
```

Add a category to your plug-in. See [AAX_EPlugInCategory](#).

Parameters

in	<i>inCategory</i>	One of the categories for the plug-in.
----	-------------------	--

Implements [AAX_IEffectDescriptor](#).

14.155.3.5 AddCategoryBypassParameter()

```
AAX_Result AAX_VEffectDescriptor::AddCategoryBypassParameter (
    uint32_t inCategory,
    AAX_CParamID inParamID ) [virtual]
```

Add a category to your plug-in. See [AAX_EPlugInCategory](#).

Parameters

in	<i>inCategory</i>	One of the categories for the plug-in.
in	<i>inParamID</i>	The parameter ID of the parameter used to bypass the category section of the plug-in.

Implements [AAX_IEffectDescriptor](#).

14.155.3.6 AddProcPtr()

```
AAX_Result AAX_VEffectDescriptor::AddProcPtr (
    void * inProcPtr,
    AAX_CProcPtrID inProcID ) [virtual]
```

Add a process pointer.

Parameters

in	<i>inProcPtr</i>	A process pointer.
in	<i>inProcID</i>	A process ID.

Implements [AAX_IEffectDescriptor](#).

14.155.3.7 NewPropertyMap()

```
AAX_IPropertyMap* AAX_VEffectDescriptor::NewPropertyMap ( ) [virtual]
```

Create a new property map.

This implementation retains each generated [AAX_IPropertyMap](#) and destroys the property map upon [AAX_VEffectDescriptor](#) destruction

Implements [AAX_IEffectDescriptor](#).

14.155.3.8 SetPropertyes()

```
AAX_Result AAX_VEffectDescriptor::SetProperties (
    AAX_IPropertyMap * inProperties ) [virtual]
```

Set the properties of a new property map.

Parameters

in	<i>inProperties</i>	Description
----	---------------------	-------------

Implements [AAX_IEffectDescriptor](#).

14.155.3.9 AddResourceInfo()

```
AAX_Result AAX_VEffectDescriptor::AddResourceInfo (
    AAX_EResourceType inResourceType,
    const char * inInfo ) [virtual]
```

Set resource file info.

Parameters

in	<i>inResourceType</i>	See AAX_EResourceType .
in	<i>inInfo</i>	Definition varies on the resource type.

Implements [AAX_IEffectDescriptor](#).

14.155.3.10 AddMeterDescription()

```
AAX_Result AAX_VEffectDescriptor::AddMeterDescription (
    AAX_CTypeID inMeterID,
    const char * inMeterName,
    AAX_IPropertyMap * inProperties ) [virtual]
```

Add name and property map to meter with given ID.

Parameters

in	<i>inMeterID</i>	The ID of the meter being described.
in	<i>inMeterName</i>	The name of the meter.
in	<i>inProperties</i>	The property map containing meter related data such as meter type, orientation, etc.

Implements [AAX_IEffectDescriptor](#).

14.155.3.11 AddControlMIDINode()

```
AAX_Result AAX_VEffectDescriptor::AddControlMIDINode (
    AAX_CTypeID inNodeID,
    AAX_EMIDINodeType inNodeType,
    const char inNodeName[],
    uint32_t inChannelMask ) [virtual]
```

Add a control MIDI node to the plug-in data model.

- This MIDI node may receive note data as well as control data.
- To send MIDI data to the plug-in's algorithm, use [AAX_IComponentDescriptor::AddMIDINode\(\)](#).

See also

[AAX_IACFEffParameters_V2::UpdateControlMIDINodes\(\)](#)

Parameters

in	<i>inNodeID</i>	The ID for the new control MIDI node.
in	<i>inNodeType</i>	The type of the node.
in	<i>inNodeName</i>	The name of the node.
in	<i>inChannelMask</i>	The bit mask for required nodes channels (up to 16) or required global events for global node.

Implements [AAX_IEffectDescriptor](#).

14.155.3.12 GetUnknown()

```
IACFUnknown* AAX_VEffectDescriptor::GetUnknown (
    void ) const
```

The documentation for this class was generated from the following file:

- [AAX_VEffectDescriptor.h](#)

14.156 AAX_VFeatureInfo Class Reference

```
#include <AAX_VFeatureInfo.h>
```

Inheritance diagram for AAX_VFeatureInfo:

Collaboration diagram for AAX_VFeatureInfo:

14.156.1 Description

Concrete implementation of [AAX_IFeatureInfo](#), which provides a version-controlled interface to host feature information

Public Member Functions

- [AAX_VFeatureInfo](#) ([IACFUnknown](#) *pUnknown, const [AAX_Feature_UID](#) &inFeatureID)
- [~AAX_VFeatureInfo](#) () [AAX_OVERRIDE](#)
- [AAX_Result](#) SupportLevel ([AAX_ESupportLevel](#) &oSupportLevel) const [AAX_OVERRIDE](#)
- const [AAX_IPropertyMap](#) * [AcquireProperties](#) () const [AAX_OVERRIDE](#)
- const [AAX_Feature_UID](#) & ID () const [AAX_OVERRIDE](#)

14.156.2 Constructor & Destructor Documentation**14.156.2.1 AAX_VFeatureInfo()**

```
AAX_VFeatureInfo::AAX_VFeatureInfo (
    IACFUnknown * pUnknown,
    const AAX\_Feature\_UID & inFeatureID ) [explicit]
```

14.156.2.2 ~AAX_VFeatureInfo()

```
AAX_VFeatureInfo::~~AAX_VFeatureInfo ( )
```

14.156.3 Member Function Documentation

14.156.3.1 SupportLevel()

```
AAX_Result AAX_VFeatureInfo::SupportLevel (
    AAX_ESupportLevel & oSupportLevel ) const [virtual]
```

Determine the level of support for this feature by the host

Note

The host will not provide an underlying [AAX_IACFFeatureInfo](#) interface for features which it does not recognize at all, resulting in a [AAX_ERROR_NULL_OBJECT](#) error code

Implements [AAX_IFeatureInfo](#).

14.156.3.2 AcquireProperties()

```
const AAX_IPropertyMap* AAX_VFeatureInfo::AcquireProperties ( ) const [virtual]
```

Additional properties providing details of the feature support

See the feature's UID for documentation of which features provide additional properties

Ownership of the returned object is passed to the caller; the caller is responsible for destroying the object, e.g. by capturing the returned object in a smart pointer.

```
// AAX_IFeatureInfo* featureInfo
std::unique_ptr<const AAX_IPropertyMap> featurePropertiesPtr(featureInfo->AcquireProperties());
```

Returns

An [AAX_IPropertyMap](#) interface with access to the host's properties for this feature.

NULL if the desired feature was not found or if an error occurred

Note

May return an [AAX_IPropertyMap](#) object with limited method support, which would return an error such as [AAX_ERROR_NULL_OBJECT](#) or [AAX_ERROR_UNIMPLEMENTED](#) to interface calls.

Implements [AAX_IFeatureInfo](#).

14.156.3.3 ID()

```
const AAX_Feature_UID& AAX_VFeatureInfo::ID ( ) const [virtual]
```

Returns the ID of the feature which this object represents

Implements [AAX_IFeatureInfo](#).

The documentation for this class was generated from the following file:

- [AAX_VFeatureInfo.h](#)

14.157 AAX_VHostProcessorDelegate Class Reference

```
#include <AAX_VHostProcessorDelegate.h>
```

Inheritance diagram for AAX_VHostProcessorDelegate:

Collaboration diagram for AAX_VHostProcessorDelegate:

14.157.1 Description

Version-managed concrete [Host Processor delegate](#) class.

Public Member Functions

- [AAX_VHostProcessorDelegate](#) ([IACFUnknown](#) *pUnknown)
- [AAX_Result GetAudio](#) (const float *const inAudioIns[], int32_t inAudioInCount, int64_t inLocation, int32_t *ioNumSamples) [AAX_OVERRIDE](#)
CALL: Randomly access audio from the timeline.
- int32_t [GetSideChainInputNum](#) () [AAX_OVERRIDE](#)
CALL: Returns the index of the side chain input buffer.
- [AAX_Result ForceAnalyze](#) () [AAX_OVERRIDE](#)
CALL: Request an analysis pass.
- [AAX_Result ForceProcess](#) () [AAX_OVERRIDE](#)
CALL: Request a process pass.

14.157.2 Constructor & Destructor Documentation

14.157.2.1 AAX_VHostProcessorDelegate()

```
AAX_VHostProcessorDelegate::AAX_VHostProcessorDelegate (
    IACFUnknown * pUnknown )
```

14.157.3 Member Function Documentation

14.157.3.1 GetAudio()

```
AAX_Result AAX_VHostProcessorDelegate::GetAudio (
    const float *const inAudioIns[],
    int32_t inAudioInCount,
    int64_t inLocation,
    int32_t * ioNumSamples ) [virtual]
```

CALL: Randomly access audio from the timeline.

Called from within [AAX_IHostProcessor::RenderAudio\(\)](#), this method fills a buffer of samples with randomly-accessed data from the current input processing region on the timeline, including any extra samples such as processing "handles".

Note

Plug-ins that use this feature must set [AAX_eProperty_UsesRandomAccess](#) to `true`

It is not possible to retrieve samples from outside of the current input processing region

Always check the return value of this method before using the randomly-accessed samples

Parameters

in	<i>inAudioIns</i>	Timeline audio buffer(s). This must be set to <i>inAudioIns</i> from AAX_IHostProcessor::RenderAudio()
in	<i>inAudioInCount</i>	Number of buffers in <i>inAudioIns</i> . This must be set to <i>inAudioInCount</i> from AAX_IHostProcessor::RenderAudio()
in	<i>inLocation</i>	A sample location relative to the beginning of the currently processed region, e.g. a value of 0 corresponds to the timeline location returned by AAX_CHostProcessor::GetSrcStart()
in, out	<i>ioNumSamples</i>	- Input: The maximum number of samples to read. - Output: The actual number of samples that were read from the timeline

Implements [AAX_IHostProcessorDelegate](#).

14.157.3.2 GetSideChainInputNum()

```
int32_t AAX_VHostProcessorDelegate::GetSideChainInputNum ( ) [virtual]
```

CALL: Returns the index of the side chain input buffer.

Called from within [AAX_IHostProcessor::RenderAudio\(\)](#), this method returns the index of the side chain input sample buffer within *inAudioIns*.

Implements [AAX_IHostProcessorDelegate](#).

14.157.3.3 ForceAnalyze()

`AAX_Result AAX_VHostProcessorDelegate::ForceAnalyze ( ) [virtual]`

CALL: Request an analysis pass.

Call this method to request an analysis pass from within the plug-in. Most plug-ins should rely on the host to trigger analysis passes when appropriate. However, plug-ins that require an analysis pass a) outside of the context of host-driven render or analysis, or b) when internal plug-in data changes may need to call `ForceAnalyze()`.

Implements [AAX_IHostProcessorDelegate](#).

14.157.3.4 ForceProcess()

`AAX_Result AAX_VHostProcessorDelegate::ForceProcess ( ) [virtual]`

CALL: Request a process pass.

Call this method to request a process pass from within the plug-in. If [AAX_eProperty_RequiresAnalysis](#) is defined, the resulting process pass will be preceded by an analysis pass. This method should only be used in rare circumstances by plug-ins that must launch processing outside of the normal host AudioSuite workflow.

Implements [AAX_IHostProcessorDelegate](#).

The documentation for this class was generated from the following file:

- [AAX_VHostProcessorDelegate.h](#)

14.158 AAX_VHostServices Class Reference

```
#include <AAX_VHostServices.h>
```

Inheritance diagram for AAX_VHostServices:

Collaboration diagram for AAX_VHostServices:

14.158.1 Description

Version-managed concrete [AAX_IHostServices](#) class.

Public Member Functions

- [AAX_VHostServices](#) ([IACFUnknown](#) *pUnkHost)
- [~AAX_VHostServices](#) ()
- [AAX_Result HandleAssertFailure](#) (const char *iFile, int32_t iLine, const char *iNote, int32_t iFlags) const [AAX_OVERRIDE](#)
Handle an assertion failure.
- [AAX_Result Trace](#) (int32_t iPriority, const char *iMessage) const [AAX_OVERRIDE](#)
Log a trace message.
- [AAX_Result StackTrace](#) (int32_t iTracePriority, int32_t iStackTracePriority, const char *iMessage) const [AAX_OVERRIDE](#)
Log a trace message or a stack trace.

14.158.2 Constructor & Destructor Documentation

14.158.2.1 AAX_VHostServices()

```
AAX_VHostServices::AAX_VHostServices (
    IACFUnknown * pUnkHost )
```

14.158.2.2 ~AAX_VHostServices()

```
AAX_VHostServices::~~AAX_VHostServices ( )
```

14.158.3 Member Function Documentation

14.158.3.1 HandleAssertFailure()

```
AAX_Result AAX_VHostServices::HandleAssertFailure (
    const char * iFile,
    int32_t iLine,
    const char * iNote,
    int32_t iFlags ) const [virtual]
```

Handle an assertion failure.

Use this method to delegate assertion failure handling to the host

Use `iFlags` to request that specific behavior be included when handling the failure. This request may not be fulfilled by the host, and absence of a flag does not preclude the host from using that behavior when handling the failure.

Parameters

in	<i>iFile</i>	The name of the file containing the assert check. Usually <code>__FILE__</code>
in	<i>iLine</i>	The line number of the assert check. Usually <code>__LINE__</code>
in	<i>iNote</i>	Text to display related to the assert. Usually the condition which failed
in	<i>iFlags</i>	Bitfield of AAX_EAssertFlags to request specific handling behavior

Implements [AAX_IHostServices](#).

14.158.3.2 Trace()

```
AAX_Result AAX_VHostServices::Trace (
    int32_t iPriority,
    const char * iMessage ) const [virtual]
```

Log a trace message.

Parameters

in	<i>iPriority</i>	Priority of the trace, used for log filtering. One of kAAX_Trace_Priority_Low , kAAX_Trace_Priority_Normal , kAAX_Trace_Priority_High
in	<i>iMessage</i>	Message string to log

Implements [AAX_IHostServices](#).

14.158.3.3 StackTrace()

```
AAX_Result AAX_VHostServices::StackTrace (
    int32_t iTracePriority,
    int32_t iStackTracePriority,
    const char * iMessage ) const [virtual]
```

Log a trace message or a stack trace.

If the logging output filtering is set to include logs with `iStackTracePriority` then both the logging message and a stack trace will be emitted, regardless of `iTracePriority`.

If the logging output filtering is set to include logs with `iTracePriority` but to exclude logs with `iStackTracePriority` then this will emit a normal log with no stack trace.

Parameters

in	<i>iTracePriority</i>	Priority of the trace, used for log filtering. One of kAAX_Trace_Priority_Low , kAAX_Trace_Priority_Normal , kAAX_Trace_Priority_High
in	<i>iStackTracePriority</i>	Priority of the stack trace, used for log filtering. One of kAAX_Trace_Priority_Low , kAAX_Trace_Priority_Normal , kAAX_Trace_Priority_High
in	<i>iMessage</i>	Message string to log

Implements [AAX_IHostServices](#).

The documentation for this class was generated from the following file:

- [AAX_VHostServices.h](#)

14.159 AAX_VHostTaskAgent Class Reference

```
#include <AAX_VHostTaskAgent.h>
```

Inheritance diagram for AAX_VHostTaskAgent:

Collaboration diagram for AAX_VHostTaskAgent:

Public Member Functions

- [AAX_VHostTaskAgent](#) ([IACFUnknown](#) *iUnknown)
- [~AAX_VHostTaskAgent](#) () override
- [AAX_Result Initialize](#) ([IACFUnknown](#) *iController) override
- [AAX_Result Uninitialize](#) () override
- [AAX_Result AddTask](#) ([IACFUnknown](#) *iTask) override
- [AAX_Result CancelAllTasks](#) () override

14.159.1 Constructor & Destructor Documentation

14.159.1.1 AAX_VHostTaskAgent()

```
AAX_VHostTaskAgent::AAX_VHostTaskAgent (
    IACFUnknown * iUnknown ) [explicit]
```

14.159.1.2 ~AAX_VHostTaskAgent()

```
AAX_VHostTaskAgent::~~AAX_VHostTaskAgent ( ) [override]
```

14.159.2 Member Function Documentation

14.159.2.1 Initialize()

```
AAX\_Result AAX_VHostTaskAgent::Initialize (
    IACFUnknown * iController ) [override], [virtual]
```

Implements [AAX_IHostTaskAgent](#).

14.159.2.2 Uninitialize()

```
AAX\_Result AAX_VHostTaskAgent::Uninitialize ( ) [override], [virtual]
```

Implements [AAX_IHostTaskAgent](#).

14.159.2.3 AddTask()

```
AAX_Result AAX_VHostTaskAgent::AddTask (
    IACFUnknown * iTask ) [override], [virtual]
```

Implements [AAX_IHostTaskAgent](#).

14.159.2.4 CancelAllTasks()

```
AAX_Result AAX_VHostTaskAgent::CancelAllTasks ( ) [override], [virtual]
```

Implements [AAX_IHostTaskAgent](#).

The documentation for this class was generated from the following file:

- [AAX_VHostTaskAgent.h](#)

14.160 AAX_VPageTable Class Reference

```
#include <AAX_VPageTable.h>
```

Inheritance diagram for AAX_VPageTable:

Collaboration diagram for AAX_VPageTable:

14.160.1 Description

Version-managed concrete [AAX_IPageTable](#) class.

Public Member Functions

- [AAX_VPageTable](#) ([IACFUnknown](#) *pUnknown)
- [~AAX_VPageTable](#) () [AAX_OVERRIDE](#)
- [AAX_Result Clear](#) () [AAX_OVERRIDE](#)
Clears all parameter mappings from the table.
- [AAX_Result Empty](#) ([AAX_CBoolean](#) &oEmpty) const [AAX_OVERRIDE](#)
Indicates whether the table is empty.
- [AAX_Result GetNumPages](#) ([int32_t](#) &oNumPages) const [AAX_OVERRIDE](#)
Get the number of pages currently in this table.
- [AAX_Result InsertPage](#) ([int32_t](#) iPage) [AAX_OVERRIDE](#)
Insert a new empty page before the page at index iPage.
- [AAX_Result RemovePage](#) ([int32_t](#) iPage) [AAX_OVERRIDE](#)
Remove the page at index iPage.
- [AAX_Result GetNumMappedParameterIDs](#) ([int32_t](#) iPage, [int32_t](#) &oNumParameterIdentifiers) const [AAX_OVERRIDE](#)
Returns the total number of parameter IDs which are mapped to a page.

- [AAX_Result ClearMappedParameter](#) (int32_t iPage, int32_t iIndex) [AAX_OVERRIDE](#)
Clear the parameter at a particular index in this table.
- [AAX_Result GetMappedParameterID](#) (int32_t iPage, int32_t iIndex, [AAX_IString](#) &oParameterIdentifier) const [AAX_OVERRIDE](#)
Get the parameter identifier which is currently mapped to an index in this table.
- [AAX_Result MapParameterID](#) ([AAX_CParamID](#) iParameterIdentifier, int32_t iPage, int32_t iIndex) [AAX_OVERRIDE](#)
Map a parameter to this table.
- [AAX_Result GetNumParametersWithNameVariations](#) (int32_t &oNumParameterIdentifiers) const [AAX_OVERRIDE](#)
- [AAX_Result GetNameVariationParameterIDAtIndex](#) (int32_t iIndex, [AAX_IString](#) &oParameterIdentifier) const [AAX_OVERRIDE](#)
- [AAX_Result GetNumNameVariationsForParameter](#) ([AAX_CPageTableParamID](#) iParameterIdentifier, int32_t &oNumVariations) const [AAX_OVERRIDE](#)
- [AAX_Result GetParameterNameVariationAtIndex](#) ([AAX_CPageTableParamID](#) iParameterIdentifier, int32_t iIndex, [AAX_IString](#) &oNameVariation, int32_t &oLength) const [AAX_OVERRIDE](#)
- [AAX_Result GetParameterNameVariationOfLength](#) ([AAX_CPageTableParamID](#) iParameterIdentifier, int32_t iLength, [AAX_IString](#) &oNameVariation) const [AAX_OVERRIDE](#)
- [AAX_Result ClearParameterNameVariations](#) () [AAX_OVERRIDE](#)
- [AAX_Result ClearNameVariationsForParameter](#) ([AAX_CPageTableParamID](#) iParameterIdentifier) [AAX_OVERRIDE](#)
- [AAX_Result SetParameterNameVariation](#) ([AAX_CPageTableParamID](#) iParameterIdentifier, const [AAX_IString](#) &iNameVariation, int32_t iLength) [AAX_OVERRIDE](#)
- const [IACFUnknown](#) * [AsUnknown](#) () const
- [IACFUnknown](#) * [AsUnknown](#) ()
- bool [IsSupported](#) () const

14.160.2 Constructor & Destructor Documentation

14.160.2.1 AAX_VPageTable()

```
AAX_VPageTable::AAX_VPageTable (
    IACFUnknown * pUnknown )
```

14.160.2.2 ~AAX_VPageTable()

```
AAX_VPageTable::~~AAX_VPageTable ( )
```

14.160.3 Member Function Documentation

14.160.3.1 Clear()

```
AAX_Result AAX_VPageTable::Clear ( ) [virtual]
```

Clears all parameter mappings from the table.

This method does not clear any parameter name variations from the table. For that, use [AAX_IPageTable::ClearParameterNameVariations](#) or [AAX_IPageTable::ClearNameVariationsForParameter\(\)](#)

Implements [AAX_IPageTable](#).

14.160.3.2 Empty()

```
AAX_Result AAX_VPageTable::Empty (
    AAX_CBoolean & oEmpty ) const [virtual]
```

Indicates whether the table is empty.

A table is empty if it contains no pages. A table which contains pages but no parameter assignments is not empty. A table which has associated parameter name variations but no pages is empty.

Parameters

out	<i>oEmpty</i>	true if this table is empty
-----	---------------	-----------------------------

Implements [AAX_IPageTable](#).

14.160.3.3 GetNumPages()

```
AAX_Result AAX_VPageTable::GetNumPages (
    int32_t & oNumPages ) const [virtual]
```

Get the number of pages currently in this table.

Parameters

out	<i>oNumPages</i>	The number of pages which are present in the page table. Some pages might not contain any parameter assignments.
-----	------------------	--

Implements [AAX_IPageTable](#).

14.160.3.4 InsertPage()

```
AAX_Result AAX_VPageTable::InsertPage (
    int32_t iPage ) [virtual]
```

Insert a new empty page before the page at index `iPage`.

Returns

[AAX_ERROR_INVALID_ARGUMENT](#) if `iPage` is greater than the total number of pages

Parameters

in	<i>iPage</i>	The insertion point page index
----	--------------	--------------------------------

Implements [AAX_IPageTable](#).

14.160.3.5 RemovePage()

```
AAX_Result AAX_VPageTable::RemovePage (
    int32_t iPage ) [virtual]
```

Remove the page at index `iPage`.

Returns

[AAX_ERROR_INVALID_ARGUMENT](#) if `iPage` is greater than the index of the last existing page

Parameters

in	<i>iPage</i>	The target page index
----	--------------	-----------------------

Implements [AAX_IPageTable](#).

14.160.3.6 GetNumMappedParameterIDs()

```
AAX_Result AAX_VPageTable::GetNumMappedParameterIDs (
    int32_t iPage,
    int32_t & oNumParameterIdentifiers ) const [virtual]
```

Returns the total number of parameter IDs which are mapped to a page.

Note

The number of mapped parameter IDs does not correspond to the actual slot indices of the parameter assignments. For example, a page could have three total parameter assignments with parameters mapped to slots 2, 4, and 6.

Returns

[AAX_ERROR_INVALID_ARGUMENT](#) if `iPage` is greater than the index of the last existing page

Parameters

in	<i>iPage</i>	The target page index
out	<i>oNumParameterIdentifiers</i>	The number of parameter identifiers which are mapped to the target page

Implements [AAX_IPageTable](#).

14.160.3.7 ClearMappedParameter()

```
AAX_Result AAX_VPageTable::ClearMappedParameter (
    int32_t iPage,
    int32_t iIndex ) [virtual]
```

Clear the parameter at a particular index in this table.

Returns

[AAX_SUCCESS](#) even if no parameter was mapped at the given index (the index is still clear)

Parameters

in	<i>iPage</i>	The target page index
in	<i>iIndex</i>	The target parameter slot index within the target page

Implements [AAX_IPageTable](#).

14.160.3.8 GetMappedParameterID()

```
AAX_Result AAX_VPageTable::GetMappedParameterID (
    int32_t iPage,
    int32_t iIndex,
    AAX_IString & oParameterIdentifier ) const [virtual]
```

Get the parameter identifier which is currently mapped to an index in this table.

Returns

[AAX_ERROR_INVALID_ARGUMENT](#) if no parameter is mapped at the specified page/index location

Parameters

in	<i>iPage</i>	The target page index
in	<i>iIndex</i>	The target parameter slot index within the target page
out	<i>oParameterIdentifier</i>	The identifier used for the mapped parameter in the page table (may be parameter name or ID)

Implements [AAX_IPageTable](#).

14.160.3.9 MapParameterID()

```
AAX_Result AAX_VPageTable::MapParameterID (
    AAX_CParamID iParameterIdentifier,
    int32_t iPage,
    int32_t iIndex ) [virtual]
```

Map a parameter to this table.

If `iParameterIdentifier` is an empty string then the parameter assignment will be cleared

Returns

[AAX_ERROR_NULL_ARGUMENT](#) if `iParameterIdentifier` is null

[AAX_ERROR_INVALID_ARGUMENT](#) if `iPage` is greater than the index of the last existing page

[AAX_ERROR_INVALID_ARGUMENT](#) if `iIndex` is negative

Parameters

in	<i>iParameterIdentifier</i>	The identifier for the parameter which will be mapped
in	<i>iPage</i>	The target page index
in	<i>iIndex</i>	The target parameter slot index within the target page

Implements [AAX_IPageTable](#).

14.160.3.10 GetNumParametersWithNameVariations()

```
AAX_Result AAX_VPageTable::GetNumParametersWithNameVariations (
    int32_t & oNumParameterIdentifiers ) const [virtual]
```

Get the number of parameters with name variations defined for the current table type

Provides the number of parameters with `lt;ControlNameVariationslt;` which are explicitly defined for the current page table type.

Note

Normally parameter name variations are only used with the 'PgTL' table type

See also

- [AAX_IPageTable::GetNameVariationParameterIDAtIndex\(\)](#)

Parameters

out	<i>oNumParameterIdentifiers</i>	The number of parameters with name variations explicitly associated with the current table type.
-----	---------------------------------	--

Implements [AAX_IPageTable](#).

14.160.3.11 GetNameVariationParameterIDAtIndex()

```
AAX_Result AAX_VPageTable::GetNameVariationParameterIDAtIndex (
    int32_t iIndex,
    AAX_IString & oParameterIdentifier ) const [virtual]
```

Get the identifier for a parameter with name variations defined for the current table type

Note

Normally parameter name variations are only used with the 'PgTL' table type

See also

- [AAX_IPageTable::GetNumParametersWithNameVariations\(\)](#)

Parameters

in	<i>iIndex</i>	The target parameter index within the list of parameters with explicit name variations defined for this table type.
out	<i>oParameterIdentifier</i>	The identifier used for the parameter in the page table name variations list (may be parameter name or ID)

Implements [AAX_IPageTable](#).

14.160.3.12 GetNumNameVariationsForParameter()

```
AAX_Result AAX_VPageTable::GetNumNameVariationsForParameter (
    AAX_CPageTableParamID iParameterIdentifier,
    int32_t & oNumVariations ) const [virtual]
```

Get the number of name variations defined for a parameter

Provides the number of `lt;ControlNameVariationslt;` which are explicitly defined for `iParameterIdentifier` for the current page table type. No fallback logic is used to resolve this to the list of variations which would actually be used for an attached control surface if no explicit variations are defined for the current table type.

Note

Normally parameter name variations are only used with the 'PgTL' table type

See also

- [AAX_IPageTable::GetParameterNameVariationAtIndex\(\)](#)

Returns

[AAX_SUCCESS](#) and provides zero to `oNumVariations` if `iParameterIdentifier` is not found

Parameters

in	<i>iParameterIdentifier</i>	The identifier for the parameter
out	<i>oNumVariations</i>	The number of name variations which are defined for this parameter and explicitly associated with the current table type.

Implements [AAX_IPageTable](#).

14.160.3.13 GetParameterNameVariationAtIndex()

```
AAX_Result AAX_VPageTable::GetParameterNameVariationAtIndex (
    AAX_CPageTableParamID iParameterIdentifier,
    int32_t iIndex,
    AAX_IString & oNameVariation,
    int32_t & oLength ) const [virtual]
```

Get a parameter name variation from the page table

Only returns `lt;ControlNameVariationslt;` which are explicitly defined for the current page table type. No fallback logic is used to resolve this to the abbreviation which would actually be shown on an attached control surface if no explicit variation is defined for the current table type.

Note

Normally parameter name variations are only used with the 'PgTL' table type

See also

- [AAX_IPageTable::GetNumNameVariationsForParameter\(\)](#)

See also

- [AAX_IPageTable::GetParameterNameVariationOfLength\(\)](#)

Returns

[AAX_ERROR_NO_ABBREVIATED_PARAMETER_NAME](#) if no suitable variation is defined for this table
[AAX_ERROR_ARGUMENT_OUT_OF_RANGE](#) if `iIndex` is out of range

Parameters

in	<i>iParameterIdentifier</i>	The identifier for the parameter
in	<i>iIndex</i>	Index of the name variation
out	<i>oNameVariation</i>	The name variation, if one is explicitly defined for this table type
out	<i>oLength</i>	The length value for this name variation. This corresponds to the variation's <i>sz</i> attribute in the page table XML and may be different from the string length of <i>iNameVariation</i> .

Implements [AAX_IPageTable](#).

14.160.3.14 GetParameterNameVariationOfLength()

```
AAX_Result AAX_VPageTable::GetParameterNameVariationOfLength (
    AAX_CPageTableParamID iParameterIdentifier,
    int32_t iLength,
    AAX_IString & oNameVariation ) const [virtual]
```

Get a parameter name variation of a particular length from the page table

Only returns `lt;ControlNameVariationslt;` which are explicitly defined of `iLength` for the current page table type. No fallback logic is used to resolve this to the abbreviation which would actually be shown on an attached control surface if no explicit variation is defined for the specified length or current table type.

Note

Normally parameter name variations are only used with the 'PgTL' table type

See also

- [AAX_IPageTable::GetParameterNameVariationAtIndex\(\)](#)

Returns

[AAX_ERROR_NO_ABBREVIATED_PARAMETER_NAME](#) if no suitable variation is defined for this table

Parameters

in	<i>iParameterIdentifier</i>	The identifier for the parameter
in	<i>iLength</i>	The variation length to check, i.e. the <i>sz</i> attribute for the name variation in the page table XML
out	<i>oNameVariation</i>	The name variation, if one is explicitly defined for this table type and <i>iLength</i>

Implements [AAX_IPageTable](#).

14.160.3.15 ClearParameterNameVariations()

```
AAX_Result AAX_VPageTable::ClearParameterNameVariations ( ) [virtual]
```

Clears all name variations for the current page table type

Note

Normally parameter name variations are only used with the 'PgTL' table type

See also

[AAX_IPageTable::Clear\(\)](#)

[AAX_IPageTable::ClearNameVariationsForParameter\(\)](#)

Implements [AAX_IPageTable](#).

14.160.3.16 ClearNameVariationsForParameter()

```
AAX_Result AAX_VPageTable::ClearNameVariationsForParameter (
    AAX_CPageTableParamID iParameterIdentifier ) [virtual]
```

Clears all name variations for a single parameter for the current page table type

Warning

This will invalidate the list of parameter name variations indices, i.e. the parameter identifier associated with each index by [AAX_IPageTable::GetNameVariationParameterIDAtIndex\(\)](#)

Note

Normally parameter name variations are only used with the 'PgTL' table type

See also

[AAX_IPageTable::Clear\(\)](#)

[AAX_IPageTable::ClearParameterNameVariations\(\)](#)

Returns

[AAX_SUCCESS](#) and provides zero to oNumVariations if iParameterIdentifier is not found

Parameters

in	<i>iParameterIdentifier</i>	The identifier for the parameter
----	-----------------------------	----------------------------------

Implements [AAX_IPageTable](#).

14.160.3.17 SetParameterNameVariation()

```
AAX_Result AAX_VPageTable::SetParameterNameVariation (
    AAX_CPageTableParamID iParameterIdentifier,
    const AAX_IString & iNameVariation,
    int32_t iLength ) [virtual]
```

Sets a name variation explicitly for the current page table type

This will add a new name variation or overwrite the existing name variation with the same length which is defined for the current table type.

Warning

If no name variation previously existed for this parameter then this will invalidate the list of parameter name variations indices, i.e. the parameter identifier associated with each index by [AAX_IPageTable::GetNameVariationParameterIDAt](#)

Note

Normally parameter name variations are only used with the 'PgTL' table type

Returns

AAX_ERROR_INVALID_ARGUMENT if *iNameVariation* is empty or if *iLength* is less than zero

Parameters

in	<i>iParameterIdentifier</i>	The identifier for the parameter
in	<i>iNameVariation</i>	The new parameter name variation
in	<i>iLength</i>	The length value for this name variation. This corresponds to the variation's <i>sz</i> attribute in the page table XML and is not required to match the length of <i>iNameVariation</i> .

Implements [AAX_IPageTable](#).

14.160.3.18 AsUnknown() [1/2]

```
const IACFUnknown* AAX_VPageTable::AsUnknown ( ) const [inline]
```

Returns the latest supported versioned ACF interface (e.g. an [AAX_IACFPageTable](#)) which is wrapped by this [AAX_IPageTable](#)

14.160.3.19 AsUnknown() [2/2]

```
IACFUnknown* AAX_VPageTable::AsUnknown ( ) [inline]
```

Returns the latest supported versioned ACF interface (e.g. an [AAX_IACFPageTable](#)) which is wrapped by this [AAX_IPageTable](#)

14.160.3.20 IsSupported()

```
bool AAX_VPageTable::IsSupported ( ) const [inline]
```

The documentation for this class was generated from the following file:

- [AAX_VPageTable.h](#)

14.161 AAX_VPrivateDataAccess Class Reference

```
#include <AAX_VPrivateDataAccess.h>
```

Inheritance diagram for AAX_VPrivateDataAccess:

Collaboration diagram for AAX_VPrivateDataAccess:

14.161.1 Description

Version-managed concrete [AAX_IPrivateDataAccess](#) class.

Public Member Functions

- [AAX_VPrivateDataAccess](#) ([IACFUnknown](#) *pUnknown)
- [~AAX_VPrivateDataAccess](#) () [AAX_OVERRIDE](#)
- [AAX_Result ReadPortDirect](#) ([AAX_CFieldIndex](#) inFieldIndex, const uint32_t inOffset, const uint32_t inSize, void *outBuffer) [AAX_OVERRIDE](#)
Read data directly from DSP at the given port.
- [AAX_Result WritePortDirect](#) ([AAX_CFieldIndex](#) inFieldIndex, const uint32_t inOffset, const uint32_t inSize, const void *inBuffer) [AAX_OVERRIDE](#)
Write data directly to DSP at the given port.

14.161.2 Constructor & Destructor Documentation**14.161.2.1 AAX_VPrivateDataAccess()**

```
AAX_VPrivateDataAccess::AAX_VPrivateDataAccess (
    IACFUnknown * pUnknown )
```

14.161.2.2 ~AAX_VPrivateDataAccess()

```
AAX_VPrivateDataAccess::~AAX_VPrivateDataAccess ( )
```

14.161.3 Member Function Documentation**14.161.3.1 ReadPortDirect()**

```
AAX_Result AAX_VPrivateDataAccess::ReadPortDirect (
    AAX_CFieldIndex inFieldIndex,
    const uint32_t inOffset,
    const uint32_t inSize,
    void * outBuffer ) [virtual]
```

Read data directly from DSP at the given port.

Note

Blocking

Parameters

in	<i>inFieldIndex</i>	The port to read from.
in	<i>inOffset</i>	Offset into data to start reading.
in	<i>inSize</i>	Amount of data to read (in bytes).
out	<i>outBuffer</i>	Pointer to storage for data to be read into.

Implements [AAX_IPrivateDataAccess](#).

14.161.3.2 WritePortDirect()

```
AAX_Result AAX_VPrivateDataAccess::WritePortDirect (
    AAX_CFieldIndex inFieldIndex,
    const uint32_t inOffset,
    const uint32_t inSize,
    const void * inBuffer ) [virtual]
```

Write data directly to DSP at the given port.

Note

Blocking

Parameters

in	<i>inFieldIndex</i>	The port to write to.
in	<i>inOffset</i>	Offset into data to begin writing.
in	<i>inSize</i>	Amount of data to write (in bytes).
in	<i>inBuffer</i>	Pointer to data being written.

Implements [AAX_IPrivateDataAccess](#).

The documentation for this class was generated from the following file:

- [AAX_VPrivateDataAccess.h](#)

14.162 AAX_VPropertyMap Class Reference

```
#include <AAX_VPropertyMap.h>
```

Inheritance diagram for AAX_VPropertyMap:

Collaboration diagram for AAX_VPropertyMap:

14.162.1 Description

Version-managed concrete [AAX_IPropertyMap](#) class.

Public Member Functions

- [~AAX_VPropertyMap](#) (void) [AAX_OVERRIDE](#)
- [AAX_CBoolean](#) [GetProperty](#) ([AAX_EProperty](#) inProperty, [AAX_CPropertyValue](#) *outValue) const [AAX_OVERRIDE](#)
Get a property value from a property map.
- [AAX_CBoolean](#) [GetPointerProperty](#) ([AAX_EProperty](#) inProperty, const void **outValue) const [AAX_OVERRIDE](#)
Get a property value from a property map with a pointer-sized value.
- [AAX_Result](#) [AddProperty](#) ([AAX_EProperty](#) inProperty, [AAX_CPropertyValue](#) inValue) [AAX_OVERRIDE](#)
Add a property to a property map.
- [AAX_Result](#) [AddPointerProperty](#) ([AAX_EProperty](#) inProperty, const void *inValue) [AAX_OVERRIDE](#)
Add a property to a property map with a pointer-sized value.
- [AAX_Result](#) [AddPointerProperty](#) ([AAX_EProperty](#) inProperty, const char *inValue) [AAX_OVERRIDE](#)
Add a property to a property map with a pointer-sized value.
- [AAX_Result](#) [RemoveProperty](#) ([AAX_EProperty](#) inProperty) [AAX_OVERRIDE](#)
Remove a property from a property map.
- [AAX_Result](#) [AddPropertyWithIDArray](#) ([AAX_EProperty](#) inProperty, const [AAX_SPluginIdentifierTriad](#) *inPluginIDs, uint32_t inNumPluginIDs) [AAX_OVERRIDE](#)
Add an array of plug-in IDs to a property map.
- [AAX_CBoolean](#) [GetPropertyWithIDArray](#) ([AAX_EProperty](#) inProperty, const [AAX_SPluginIdentifierTriad](#) **outPluginIDs, uint32_t *outNumPluginIDs) const [AAX_OVERRIDE](#)
Get an array of plug-in IDs from a property map.
- [IACFUnknown](#) * [GetUnknown](#) () [AAX_OVERRIDE](#)

Static Public Member Functions

- static [AAX_VPropertyMap](#) * [Create](#) ([IACFUnknown](#) *inComponentFactory)
inComponentFactory must support IID_IACFComponentFactory - otherwise NULL is returned
- static [AAX_VPropertyMap](#) * [Acquire](#) ([IACFUnknown](#) *inPropertyMapUnknown)
inPropertyMapUnknown must support at least one [AAX_IPropertyMap](#) interface - otherwise an [AAX_VPropertyMap](#) object with no backing interface is returned

14.162.2 Constructor & Destructor Documentation

14.162.2.1 ~AAX_VPropertyMap()

```
AAX_VPropertyMap::~~AAX_VPropertyMap (
    void )
```

14.162.3 Member Function Documentation

14.162.3.1 Create()

```
static AAX\_VPropertyMap* AAX_VPropertyMap::Create (
    IACFUnknown * inComponentFactory ) [static]
```

inComponentFactory must support IID_IACFComponentFactory - otherwise NULL is returned

14.162.3.2 Acquire()

```
static AAX\_VPropertyMap* AAX_VPropertyMap::Acquire (
    IACFUnknown * inPropertyMapUnknown ) [static]
```

inPropertyMapUnknown must support at least one [AAX_IPropertyMap](#) interface - otherwise an [AAX_VPropertyMap](#) object with no backing interface is returned

14.162.3.3 GetProperty()

```
AAX\_CBoolean AAX_VPropertyMap::GetProperty (
    AAX\_EProperty inProperty,
    AAX\_CPropertyValue * outValue ) const [virtual]
```

Get a property value from a property map.

Returns true if the selected property is supported, false if it is not

Parameters

in	<i>inProperty</i>	The property ID
out	<i>outValue</i>	The property value

Implements [AAX_IPropertyMap](#).

14.162.3.4 GetPointerProperty()

```
AAX_Boolean AAX_VPropertyMap::GetPointerProperty (
    AAX_EProperty inProperty,
    const void ** outValue ) const [virtual]
```

Get a property value from a property map with a pointer-sized value.

Returns true if the selected property is supported, false if it is not

Parameters

in	<i>inProperty</i>	The property ID
out	<i>outValue</i>	The property value

Implements [AAX_IPropertyMap](#).

14.162.3.5 AddProperty()

```
AAX_Result AAX_VPropertyMap::AddProperty (
    AAX_EProperty inProperty,
    AAX_CPropertyValue inValue ) [virtual]
```

Add a property to a property map.

Note

This method may return an error if adding the property was unsuccessful. If there is a failure when adding a required property then registration of the relevant description element must be abandoned and the plug-in's description logic should proceed to the next element.

Parameters

in	<i>inProperty</i>	The property ID.
in	<i>inValue</i>	

Implements [AAX_IPropertyMap](#).

14.162.3.6 AddPointerProperty() [1/2]

```
AAX_Result AAX_VPropertyMap::AddPointerProperty (
    AAX_EProperty inProperty,
    const void * inValue ) [virtual]
```

Add a property to a property map with a pointer-sized value.

Use this method to add properties which require a pointer-sized value. Do not use this method to add a property unless a pointer-sized value is explicitly specified in the property documentation.

Note

This method may return an error if adding the property was unsuccessful. If there is a failure when adding a required property then registration of the relevant description element must be abandoned and the plug-in's description logic should proceed to the next element.

Parameters

in	<i>inProperty</i>	The property ID.
in	<i>inValue</i>	

Implements [AAX_IPropertyMap](#).

14.162.3.7 AddPointerProperty() [2/2]

```
AAX_Result AAX_VPropertyMap::AddPointerProperty (
    AAX_EProperty inProperty,
    const char * inValue ) [virtual]
```

Add a property to a property map with a pointer-sized value.

Use this method to add properties which require a pointer-sized value. Do not use this method to add a property unless a pointer-sized value is explicitly specified in the property documentation.

Note

This method may return an error if adding the property was unsuccessful. If there is a failure when adding a required property then registration of the relevant description element must be abandoned and the plug-in's description logic should proceed to the next element.

Parameters

in	<i>inProperty</i>	The property ID.
in	<i>inValue</i>	

Implements [AAX_IPropertyMap](#).

14.162.3.8 RemoveProperty()

```
AAX_Result AAX_VPropertyMap::RemoveProperty (
    AAX_EProperty inProperty ) [virtual]
```

Remove a property from a property map.

Parameters

in	<i>inProperty</i>	The property ID.
----	-------------------	------------------

Implements [AAX_IPropertyMap](#).

14.162.3.9 AddPropertyWithIDArray()

```
AAX_Result AAX_VPropertyMap::AddPropertyWithIDArray (
    AAX_EProperty inProperty,
    const AAX_SPlugInIdentifierTriad * inPluginIDs,
    uint32_t inNumPluginIDs ) [virtual]
```

Add an array of plug-in IDs to a property map.

Parameters

in	<i>inProperty</i>	The property ID.
in	<i>inPluginIDs</i>	An array of AAX_SPlugInIdentifierTriad
in	<i>inNumPluginIDs</i>	The length of iPluginIDs

Implements [AAX_IPropertyMap](#).

14.162.3.10 GetPropertyWithIDArray()

```
AAX_CBoolean AAX_VPropertyMap::GetPropertyWithIDArray (
    AAX_EProperty inProperty,
    const AAX_SPlugInIdentifierTriad ** outPluginIDs,
    uint32_t * outNumPluginIDs ) const [virtual]
```

Get an array of plug-in IDs from a property map.

Parameters

in	<i>inProperty</i>	The property ID.
out	<i>outPluginIDs</i>	A pointer that will be set to reference an array of AAX_SPlugInIdentifierTriad
in	<i>outNumPluginIDs</i>	The length of oPluginIDs

Implements [AAX_IPropertyMap](#).

14.162.3.11 GetIUnknown()

```
IACFUnknown* AAX_VPropertyMap::GetIUnknown ( ) [virtual]
```

Returns the most up-to-date underlying interface

Implements [AAX_IPropertyMap](#).

The documentation for this class was generated from the following file:

- [AAX_VPropertyMap.h](#)

14.163 AAX_VSessionDocument Class Reference

```
#include <AAX_VSessionDocument.h>
```

Inheritance diagram for AAX_VSessionDocument:

Collaboration diagram for AAX_VSessionDocument:

Classes

- class [VTempoMap](#)

Public Member Functions

- [AAX_VSessionDocument](#) ([IACFUnknown](#) *iUnknown)
- [~AAX_VSessionDocument](#) () [AAX_OVERRIDE](#)
- void [Clear](#) ()
Release all interface references.
- bool [Valid](#) () const [AAX_OVERRIDE](#)
Check whether this session document is valid.
- std::unique_ptr< [AAX_ISessionDocument::TempoMap](#) const > [GetTempoMap](#) () [AAX_OVERRIDE](#)
Get a copy of the document's tempo map.
- [AAX_Result](#) [GetDocumentData](#) ([AAX_DocumentData_UID](#) const &inDataType, [IACFUnknown](#) **outData) [AAX_OVERRIDE](#)

14.163.1 Constructor & Destructor Documentation

14.163.1.1 AAX_VSessionDocument()

```
AAX_VSessionDocument::AAX_VSessionDocument (
    IACFUnknown * iUnknown ) [explicit]
```

14.163.1.2 ~AAX_VSessionDocument()

```
AAX_VSessionDocument::~~AAX_VSessionDocument ( )
```

14.163.2 Member Function Documentation

14.163.2.1 Clear()

```
void AAX_VSessionDocument::Clear ( )
```

Release all interface references.

14.163.2.2 Valid()

```
bool AAX_VSessionDocument::Valid ( ) const [virtual]
```

Check whether this session document is valid.

Implements [AAX_ISessionDocument](#).

14.163.2.3 GetTempoMap()

```
std::unique_ptr<AAX\_ISessionDocument::TempoMap const> AAX_VSessionDocument::GetTempoMap ( )
[virtual]
```

Get a copy of the document's tempo map.

Returns

A [TempoMap](#) interface representing a copy of the current tempo map.

`nullptr` if the host does not support tempo map data or if an error occurred.

Implements [AAX_ISessionDocument](#).

14.163.2.4 GetDocumentData()

```
AAX_Result AAX_VSessionDocument::GetDocumentData (
    AAX_DocumentData_UID const & inDataType,
    IACFUnknown ** outData ) [virtual]
```

Get document data of a generic type

Similar to `QueryInterface()` but uses a data type identifier rather than a true IID

The provided interface has already had a reference added, so be careful not to add an additional reference:

```
ACFPtr<MyType> ptr;
IACFUnknown * docDataPtr{nullptr};
if (AAX_SUCCESS == doc->GetDocumentData(dataUID, &docDataPtr) && docDataPtr) {
    ptr.attach(std::static_cast<MyType*>(docDataPtr)); // attach does not AddRef
}
```

Parameters

in	<i>inDataType</i>	The type of the document data requested
out	<i>outData</i>	An interface providing the requested data, or <code>nullptr</code> if the host does not support or cannot provide the requested data type. The reference count has been incremented on this object on behalf of the caller, so the caller must not add an additional reference count and must decrement the reference count on this object to release it. For information about which interface to expect for each requested data type, see the documentation for that data type.

Implements [AAX_ISessionDocument](#).

The documentation for this class was generated from the following file:

- [AAX_VSessionDocument.h](#)

14.164 AAX_VTask Class Reference

```
#include <AAX_VTask.h>
```

Inheritance diagram for AAX_VTask:

Collaboration diagram for AAX_VTask:

14.164.1 Description

Version-managed concrete [AAX_ITask](#).

Public Member Functions

- [AAX_VTask](#) (*IACFUnknown* *pUnknown)
- [~AAX_VTask](#) () [AAX_OVERRIDE](#)
- [AAX_Result](#) [GetType](#) (*AAX_CTypeID* *oType) const [AAX_OVERRIDE](#)
- [AAX_IACFDataBuffer](#) const * [GetArgumentOfType](#) (*AAX_CTypeID* iType) const [AAX_OVERRIDE](#)
- [AAX_Result](#) [SetProgress](#) (float iProgress) [AAX_OVERRIDE](#)
- float [GetProgress](#) () const [AAX_OVERRIDE](#)
- [AAX_Result](#) [AddResult](#) ([AAX_IACFDataBuffer](#) const *iResult) [AAX_OVERRIDE](#)
Attach result data to this task.
- [AAX_ITask](#) * [SetDone](#) (*AAX_TaskCompletionStatus* iStatus) [AAX_OVERRIDE](#)
Inform the host that the task is completed.

14.164.2 Constructor & Destructor Documentation

14.164.2.1 AAX_VTask()

```
AAX_VTask::AAX_VTask (
    IACFUnknown * pUnknown ) [explicit]
```

14.164.2.2 ~AAX_VTask()

```
AAX_VTask::~AAX_VTask ( )
```

14.164.3 Member Function Documentation**14.164.3.1 GetType()**

```
AAX_Result AAX_VTask::GetType (
    AAX_CTypeID * oType ) const [virtual]
```

An identifier defining the type of the requested task

Parameters

out	<i>oType</i>	The type of this task request
-----	--------------	-------------------------------

Implements [AAX_ITask](#).

14.164.3.2 GetArgumentOfType()

```
AAX_IACFDataBuffer const* AAX_VTask::GetArgumentOfType (
    AAX_CTypeID iType ) const [virtual]
```

Additional information defining the request, depending on the task type

Parameters

in	<i>iType</i>	The type of argument requested. Possible argument types, if any, and the resulting data buffer format must be defined per task type.
----	--------------	--

Returns

The requested argument data, or nullptr. This data buffer's type ID is expected to match *iType*. The caller takes ownership of this object.

Implements [AAX_ITask](#).

14.164.3.3 SetProgress()

```
AAX_Result AAX_VTask::SetProgress (
    float iProgress ) [virtual]
```

Inform the host about the current status of the task

Parameters

in	<i>iProgress</i>	A value between 0 (no progress) and 1 (complete)
----	------------------	--

Implements [AAX_ITask](#).

14.164.3.4 GetProgress()

```
float AAX_VTask::GetProgress ( ) const [virtual]
```

Returns the current progress

Implements [AAX_ITask](#).

14.164.3.5 AddResult()

```
AAX_Result AAX_VTask::AddResult (
    AAX_IACFDataBuffer const * iResult ) [virtual]
```

Attach result data to this task.

This can be called multiple times to add multiple types of results to a single task.

The host may process the result data immediately or may wait for the task to complete.

The plug-in is expected to release the data buffer upon making this call. At a minimum, the data buffer must not be changed after this call is made. See `ACFPtr::inArg()`

Parameters

in	<i>iResult</i>	A buffer containing the result data. Expected result types, if any, and their data buffer format must be defined per task type.
----	----------------	---

Implements [AAX_ITask](#).

14.164.3.6 SetDone()

```
AAX_ITask* AAX_VTask::SetDone (
    AAX_TaskCompletionStatus iStatus ) [virtual]
```

Inform the host that the task is completed.

If successful, returns a null pointer. Otherwise, returns a pointer back to the same object. This is the expected usage pattern:

```
// release the task on success, retain it on failure
myTask = myTask->SetDone(status);
```

Parameters

in	<i>iStatus</i>	The final status of the task. This indicates to the host whether or not the task was performed as requested.
----	----------------	--

Implements [AAX_ITask](#).

The documentation for this class was generated from the following file:

- [AAX_VTask.h](#)

14.165 AAX_VTransport Class Reference

```
#include <AAX_VTransport.h>
```

Inheritance diagram for AAX_VTransport:

Collaboration diagram for AAX_VTransport:

14.165.1 Description

Version-managed concrete [AAX_ITransport](#) class.

Public Member Functions

- [AAX_VTransport](#) ([IACFUnknown](#) *pUnknown)
- [~AAX_VTransport](#) () [AAX_OVERRIDE](#)
- [AAX_Result GetCurrentTempo](#) (double *TempoBPM) const [AAX_OVERRIDE](#)
CALL: Gets the current tempo.
- [AAX_Result GetCurrentMeter](#) (int32_t *MeterNumerator, int32_t *MeterDenominator) const [AAX_OVERRIDE](#)
CALL: Gets the current meter.
- [AAX_Result IsTransportPlaying](#) (bool *isPlaying) const [AAX_OVERRIDE](#)
CALL: Indicates whether or not the transport is playing back.
- [AAX_Result GetCurrentTickPosition](#) (int64_t *TickPosition) const [AAX_OVERRIDE](#)

- CALL: Gets the current tick position.*
- [AAX_Result GetCurrentLoopPosition](#) (bool *bLooping, int64_t *LoopStartTick, int64_t *LoopEndTick) const [AAX_OVERRIDE](#)
- CALL: Gets current information on loop playback.*
- [AAX_Result GetCurrentNativeSampleLocation](#) (int64_t *SampleLocation) const [AAX_OVERRIDE](#)
- CALL: Gets the current playback location of the native audio engine.*
- [AAX_Result GetCustomTickPosition](#) (int64_t *oTickPosition, int64_t iSampleLocation) const [AAX_OVERRIDE](#)
- CALL: Given an absolute sample position, gets the corresponding tick position.*
- [AAX_Result GetBarBeatPosition](#) (int32_t *Bars, int32_t *Beats, int64_t *DisplayTicks, int64_t SampleLocation) const [AAX_OVERRIDE](#)
- CALL: Given an absolute sample position, gets the corresponding bar and beat position.*
- [AAX_Result GetTicksPerQuarter](#) (uint32_t *ticks) const [AAX_OVERRIDE](#)
- CALL: Retrieves the number of ticks per quarter note.*
- [AAX_Result GetCurrentTicksPerBeat](#) (uint32_t *ticks) const [AAX_OVERRIDE](#)
- CALL: Retrieves the number of ticks per beat.*
- [AAX_Result GetTimelineSelectionStartPosition](#) (int64_t *oSampleLocation) const [AAX_OVERRIDE](#)
- CALL: Retrieves the absolute sample position of the beginning of the current transport selection.*
- [AAX_Result GetTimeCodeInfo](#) (AAX_EFrameRate *oFrameRate, int32_t *oOffset) const [AAX_OVERRIDE](#)
- CALL: Retrieves the current time code frame rate and offset.*
- [AAX_Result GetFeetFramesInfo](#) (AAX_EFeetFramesRate *oFeetFramesRate, int64_t *oOffset) const [AAX_OVERRIDE](#)
- CALL: Retrieves the current timecode feet/frames rate and offset.*
- [AAX_Result IsMetronomeEnabled](#) (int32_t *isEnabled) const [AAX_OVERRIDE](#)
- Sets isEnabled to true if the metronome is enabled.*
- [AAX_Result GetHDTIMECodeInfo](#) (AAX_EFrameRate *oHDFFrameRate, int64_t *oHDOffset) const [AAX_OVERRIDE](#)
- CALL: Retrieves the current HD time code frame rate and offset.*
- [AAX_Result GetTimelineSelectionEndPosition](#) (int64_t *oSampleLocation) const [AAX_OVERRIDE](#)
- CALL: Retrieves the absolute sample position of the end of the current transport selection.*
- [AAX_Result GetKeySignature](#) (int64_t iSampleLocation, uint32_t *oKeySignature) const [AAX_OVERRIDE](#)
- CALL: Retrieves the key signature at a sample location.*
- [AAX_Result RequestTransportStart](#) () [AAX_OVERRIDE](#)
- CALL: Request that the host transport start playback.*
- [AAX_Result RequestTransportStop](#) () [AAX_OVERRIDE](#)
- CALL: Request that the host transport stop playback.*

14.165.2 Constructor & Destructor Documentation

14.165.2.1 AAX_VTransport()

```
AAX_VTransport::AAX_VTransport (
    IACFUnknown * pUnknown )
```

14.165.2.2 ~AAX_VTransport()

```
AAX_VTransport::~AAX_VTransport ( )
```

14.165.3 Member Function Documentation

14.165.3.1 GetCurrentTempo()

```
AAX_Result AAX_VTransport::GetCurrentTempo (
    double * TempoBPM ) const [virtual]
```

CALL: Gets the current tempo.

Returns the tempo corresponding to the current position of the transport counter

Note

The resolution of the tempo returned here is based on the host's tempo resolution, so it will match the tempo displayed in the host. Use [GetCurrentTicksPerBeat\(\)](#) to calculate the tempo resolution note.

Parameters

out	<i>TempoBPM</i>	The current tempo in beats per minute
-----	-----------------	---------------------------------------

Implements [AAX_ITransport](#).

14.165.3.2 GetCurrentMeter()

```
AAX_Result AAX_VTransport::GetCurrentMeter (
    int32_t * MeterNumerator,
    int32_t * MeterDenominator ) const [virtual]
```

CALL: Gets the current meter.

Returns the meter corresponding to the current position of the transport counter

Parameters

out	<i>MeterNumerator</i>	The numerator portion of the meter
out	<i>MeterDenominator</i>	The denominator portion of the meter

Implements [AAX_ITransport](#).

14.165.3.3 IsTransportPlaying()

```
AAX_Result AAX_VTransport::IsTransportPlaying (
    bool * isPlaying ) const [virtual]
```

CALL: Indicates whether or not the transport is playing back.

Parameters

out	<i>isPlaying</i>	true if the transport is currently in playback
-----	------------------	--

Implements [AAX_ITransport](#).

14.165.3.4 GetCurrentTickPosition()

```
AAX_Result AAX_VTransport::GetCurrentTickPosition (
    int64_t * TickPosition ) const [virtual]
```

CALL: Gets the current tick position.

Returns the current tick position corresponding to the current transport position. One "Tick" is represented here as 1/960000 of a quarter note. That is, there are 960,000 of these ticks in a quarter note.

Host Compatibility Notes The tick resolution here is different than that of the tick displayed in Pro Tools. "Display ticks" (as they are called) are 1/960 of a quarter note.

Parameters

out	<i>TickPosition</i>	The tick position value
-----	---------------------	-------------------------

Implements [AAX_ITransport](#).

14.165.3.5 GetCurrentLoopPosition()

```
AAX_Result AAX_VTransport::GetCurrentLoopPosition (
    bool * bLooping,
    int64_t * LoopStartTick,
    int64_t * LoopEndTick ) const [virtual]
```

CALL: Gets current information on loop playback.

Host Compatibility Notes This does not indicate anything about the status of the "Loop Record" option. Even when the host is configured to loop playback, looping may not occur if certain conditions are not met (i.e. the length of the selection is too short)

Parameters

out	<i>bLooping</i>	true if the host is configured to loop playback
out	<i>LoopStartTick</i>	The starting tick position of the selection being looped (see GetCurrentTickPosition())
out	<i>LoopEndTick</i>	The ending tick position of the selection being looped (see GetCurrentTickPosition())

Implements [AAX_ITransport](#).

14.165.3.6 GetCurrentNativeSampleLocation()

```
AAX_Result AAX_VTransport::GetCurrentNativeSampleLocation (
    int64_t * SampleLocation ) const [virtual]
```

CALL: Gets the current playback location of the native audio engine.

When called from a ProcessProc render callback, this method will provide the absolute sample location at the beginning of the callback's audio buffers.

When called from [AAX_IEffectParameters::RenderAudio_Hybrid\(\)](#), this method will provide the absolute sample location for the samples in the method's **output** audio buffers. To calculate the absolute sample location for the samples in the method's input buffers (i.e. the timeline location where the samples originated) subtract the value provided by [AAX_IController::GetHybridSignalLatency\(\)](#) from this value.

When called from a non-real-time thread, this method will provide the current location of the samples being processed by the plug-in's ProcessProc on its real-time processing thread.

Note

This method only returns a value during playback. It cannot be used to determine, e.g., the location of the timeline selector while the host is not in playback.

Parameters

out	<i>SampleLocation</i>	Absolute sample location of the first sample in the current native processing buffer
-----	-----------------------	--

Implements [AAX_ITransport](#).

14.165.3.7 GetCustomTickPosition()

```
AAX_Result AAX_VTransport::GetCustomTickPosition (
    int64_t * oTickPosition,
    int64_t iSampleLocation ) const [virtual]
```

CALL: Given an absolute sample position, gets the corresponding tick position.

Host Compatibility Notes There is a minor performance cost associated with using this API in Pro Tools. It should not be used excessively without need.

Parameters

out	<i>oTickPosition</i>	the timeline tick position corresponding to <i>iSampleLocation</i>
in	<i>iSampleLocation</i>	An absolute sample location (see GetCurrentNativeSampleLocation())

Implements [AAX_ITransport](#).

14.165.3.8 GetBarBeatPosition()

```
AAX_Result AAX_VTransport::GetBarBeatPosition (
    int32_t * Bars,
    int32_t * Beats,
    int64_t * DisplayTicks,
    int64_t SampleLocation ) const [virtual]
```

CALL: Given an absolute sample position, gets the corresponding bar and beat position.

Host Compatibility Notes There is a minor performance cost associated with using this API in Pro Tools. It should not be used excessively without need.

Parameters

out	<i>Bars</i>	The bar corresponding to <i>SampleLocation</i>
out	<i>Beats</i>	The beat corresponding to <i>SampleLocation</i>
out	<i>DisplayTicks</i>	The ticks corresponding to <i>SampleLocation</i>
in	<i>SampleLocation</i>	An absolute sample location (see GetCurrentNativeSampleLocation())

Implements [AAX_ITransport](#).

14.165.3.9 GetTicksPerQuarter()

```
AAX_Result AAX_VTransport::GetTicksPerQuarter (
    uint32_t * ticks ) const [virtual]
```

CALL: Retrieves the number of ticks per quarter note.

Parameters

out	<i>ticks</i>	
-----	--------------	--

Implements [AAX_ITransport](#).

14.165.3.10 GetCurrentTicksPerBeat()

```
AAX_Result AAX_VTransport::GetCurrentTicksPerBeat (
    uint32_t * ticks ) const [virtual]
```

CALL: Retrieves the number of ticks per beat.

Parameters

out	<i>ticks</i>	
-----	--------------	--

Implements [AAX_ITransport](#).

14.165.3.11 GetTimelineSelectionStartPosition()

```
AAX_Result AAX_VTransport::GetTimelineSelectionStartPosition (
    int64_t * oSampleLocation ) const [virtual]
```

CALL: Retrieves the absolute sample position of the beginning of the current transport selection.

Note

This method is part of the [version 2 transport interface](#)

Parameters

out	<i>oSampleLocation</i>	
-----	------------------------	--

Implements [AAX_ITransport](#).

14.165.3.12 GetTimeCodeInfo()

```
AAX_Result AAX_VTransport::GetTimeCodeInfo (
    AAX_EFrameRate * oFrameRate,
    int32_t * oOffset ) const [virtual]
```

CALL: Retrieves the current time code frame rate and offset.

Note

This method is part of the [version 2 transport interface](#)

Parameters

out	<i>oFrameRate</i>	
out	<i>oOffset</i>	

Implements [AAX_ITransport](#).

14.165.3.13 GetFeetFramesInfo()

```
AAX_Result AAX_VTransport::GetFeetFramesInfo (
    AAX_EFeetFramesRate * oFeetFramesRate,
    int64_t * oOffset ) const [virtual]
```

CALL: Retrieves the current timecode feet/frames rate and offset.

Note

This method is part of the [version 2 transport interface](#)

Parameters

out	<i>oFeetFramesRate</i>	
out	<i>oOffset</i>	

Implements [AAX_ITransport](#).

14.165.3.14 IsMetronomeEnabled()

```
AAX_Result AAX_VTransport::IsMetronomeEnabled (
    int32_t * isEnabled ) const [virtual]
```

Sets isEnabled to true if the metronome is enabled.

Note

This method is part of the [version 2 transport interface](#)

Parameters

out	<i>isEnabled</i>	
-----	------------------	--

Implements [AAX_ITransport](#).

14.165.3.15 GetHDTimeCodeInfo()

```
AAX_Result AAX_VTransport::GetHDTimeCodeInfo (
    AAX_EFrameRate * oHDFrameRate,
    int64_t * oHDOffset ) const [virtual]
```

CALL: Retrieves the current HD time code frame rate and offset.

Note

This method is part of the [version 3 transport interface](#)

Parameters

out	<i>oHDFrameRate</i>	
out	<i>oHDOffset</i>	

Implements [AAX_ITransport](#).

14.165.3.16 GetTimelineSelectionEndPosition()

```
AAX_Result AAX_VTransport::GetTimelineSelectionEndPosition (
    int64_t * oSampleLocation ) const [virtual]
```

CALL: Retrieves the absolute sample position of the end of the current transport selection.

Note

This method is part of the [version 4 transport interface](#)

Parameters

out	<i>oSampleLocation</i>	
-----	------------------------	--

Implements [AAX_ITransport](#).

14.165.3.17 GetKeySignature()

```
AAX_Result AAX_VTransport::GetKeySignature (
    int64_t iSampleLocation,
    uint32_t * oKeySignature ) const [virtual]
```

CALL: Retrieves the key signature at a sample location.

The signature is provided as a bitfield:

- 31-20: Chromatic scale elements, ordered MSB (root) to LSB
- 19-4: (Reserved)
- 3-0: Root note (C natural = 0)

For example

```

*   D# Major
*       Ionian
*   0b 101011010101 0000 00000000 0000 0011
*
*   E Phrygian
*       Phrygian
*   0b 110101011010 0000 00000000 0000 0100
*
*   Chromatic
*       Chromatic
*   0b 111111111111 0000 00000000 0000 0000
*

```

Implements [AAX_ITransport](#).

14.165.3.18 RequestTransportStart()

```
AAX_Result AAX_VTransport::RequestTransportStart ( ) [virtual]
```

CALL: Request that the host transport start playback.

Note

This method is part of the [AAX_IACFTransportControl](#) interface

Implements [AAX_ITransport](#).

14.165.3.19 RequestTransportStop()

```
AAX_Result AAX_VTransport::RequestTransportStop ( ) [virtual]
```

CALL: Request that the host transport stop playback.

Note

This method is part of the [AAX_IACFTransportControl](#) interface

Implements [AAX_ITransport](#).

The documentation for this class was generated from the following file:

- [AAX_VTransport.h](#)

14.166 AAX_VViewContainer Class Reference

```
#include <AAX_VViewContainer.h>
```

Inheritance diagram for AAX_VViewContainer:

Collaboration diagram for AAX_VViewContainer:

14.166.1 Description

Version-managed concrete [AAX_UIViewContainer](#) class.

Public Member Functions

- [AAX_VViewContainer](#) ([IACFUnknown](#) *pUnknown)
- [~AAX_VViewContainer](#) () [AAX_OVERRIDE](#)
- [int32_t GetType](#) () [AAX_OVERRIDE](#)
Returns the raw view type as one of [AAX_EViewContainer_Type](#).
- [void * GetPtr](#) () [AAX_OVERRIDE](#)
Returns a pointer to the raw view.
- [AAX_Result GetModifiers](#) ([uint32_t](#) *outModifiers) [AAX_OVERRIDE](#)
*Queries the host for the current *modifier keys*.*
- [AAX_Result SetViewSize](#) ([AAX_Point](#) &inSize) [AAX_OVERRIDE](#)
Request a change to the main view size.
- [AAX_Result HandleParameterMouseDown](#) ([AAX_CParamID](#) inParamID, [uint32_t](#) inModifiers) [AAX_OVERRIDE](#)
Alert the host to a mouse down event.
- [AAX_Result HandleParameterMouseDrag](#) ([AAX_CParamID](#) inParamID, [uint32_t](#) inModifiers) [AAX_OVERRIDE](#)
Alert the host to a mouse drag event.
- [AAX_Result HandleParameterMouseUp](#) ([AAX_CParamID](#) inParamID, [uint32_t](#) inModifiers) [AAX_OVERRIDE](#)
Alert the host to a mouse up event.
- [AAX_Result HandleParameterMouseEnter](#) ([AAX_CParamID](#) inParamID, [uint32_t](#) inModifiers) [AAX_OVERRIDE](#)
Alert the host to a mouse enter event to the parameter's control.
- [AAX_Result HandleParameterMouseExit](#) ([AAX_CParamID](#) inParamID, [uint32_t](#) inModifiers) [AAX_OVERRIDE](#)
Alert the host to a mouse exit event from the parameter's control.
- [AAX_Result HandleMultipleParametersMouseDown](#) (const [AAX_CParamID](#) *inParamIDs, [uint32_t](#) inNumOfParams, [uint32_t](#) inModifiers) [AAX_OVERRIDE](#)
Alert the host to a mouse down event.
- [AAX_Result HandleMultipleParametersMouseDrag](#) (const [AAX_CParamID](#) *inParamIDs, [uint32_t](#) inNumOfParams, [uint32_t](#) inModifiers) [AAX_OVERRIDE](#)
Alert the host to a mouse drag event.
- [AAX_Result HandleMultipleParametersMouseUp](#) (const [AAX_CParamID](#) *inParamIDs, [uint32_t](#) inNumOfParams, [uint32_t](#) inModifiers) [AAX_OVERRIDE](#)
Alert the host to a mouse up event.

14.166.2 Constructor & Destructor Documentation

14.166.2.1 AAX_VViewContainer()

```
AAX_VViewContainer::AAX_VViewContainer (
    IACFUnknown * pUnknown )
```

14.166.2.2 ~AAX_VViewContainer()

```
AAX_VViewContainer::~~AAX_VViewContainer ( )
```

14.166.3 Member Function Documentation**14.166.3.1 GetType()**

```
int32_t AAX_VViewContainer::GetType ( ) [virtual]
```

Returns the raw view type as one of [AAX_EViewContainer_Type](#).

Implements [AAX_IViewContainer](#).

14.166.3.2 GetPtr()

```
void* AAX_VViewContainer::GetPtr ( ) [virtual]
```

Returns a pointer to the raw view.

Implements [AAX_IViewContainer](#).

14.166.3.3 GetModifiers()

```
AAX_Result AAX_VViewContainer::GetModifiers (
    uint32_t * outModifiers ) [virtual]
```

Queries the host for the current [modifier keys](#).

This method returns a bit mask with bits set for each of the currently active modifier keys. This method does not return the state of the [AAX_eModifiers_SecondaryButton](#).

Host Compatibility Notes Although this method allows plug-ins to acquire the current state of the Windows key (normally blocked by Pro Tools), plug-ins should not use key combinations that require this key.

Parameters

out	outModifiers	Current modifiers as a bitmask of AAX_EModifiers
-----	--------------	--

Implements [AAX_IViewContainer](#).

14.166.3.4 SetViewSize()

```
AAX_Result AAX_VViewContainer::SetViewSize (
    AAX_Point & inSize ) [virtual]
```

Request a change to the main view size.

Note

- For compatibility with the smallest supported displays, plug-in GUI dimensions should not exceed 749x617 pixels, or 749x565 pixels for plug-ins with sidechain support.

Parameters

in	<i>inSize</i>	The new size to which the plug-in view should be set
----	---------------	--

Implements [AAX_IViewContainer](#).

14.166.3.5 HandleParameterMouseDown()

```
AAX_Result AAX_VViewContainer::HandleParameterMouseDown (
    AAX_CParamID inParamID,
    uint32_t inModifiers ) [virtual]
```

Alert the host to a mouse down event.

Parameters

in	<i>inParamID</i>	ID of the parameter whose control is being edited
in	<i>inModifiers</i>	A bitmask of AAX_EModifiers values

Implements [AAX_IViewContainer](#).

14.166.3.6 HandleParameterMouseDrag()

```
AAX_Result AAX_VViewContainer::HandleParameterMouseDrag (
    AAX_CParamID inParamID,
    uint32_t inModifiers ) [virtual]
```

Alert the host to a mouse drag event.

Warning

The host may return [AAX_ERROR_UNIMPLEMENTED](#) for this event even if the host did handle the corresponding mouse down event. A plug-in should ignore any following mouse drag and mouse up events that correspond to a host-managed mouse down event. ([PTSW-195209](#) / [PT-218474](#))

Parameters

in	<i>inParamID</i>	ID of the parameter whose control is being edited
in	<i>inModifiers</i>	A bitmask of AAX_EModifiers values

Implements [AAX_IViewContainer](#).

14.166.3.7 HandleParameterMouseUp()

```
AAX_Result AAX_VViewContainer::HandleParameterMouseUp (
    AAX_CParamID inParamID,
    uint32_t inModifiers ) [virtual]
```

Alert the host to a mouse up event.

Warning

The host may return [AAX_ERROR_UNIMPLEMENTED](#) for this event even if the host did handle the corresponding mouse down event. A plug-in should ignore any following mouse drag and mouse up events that correspond to a host-managed mouse down event. ([PTSW-195209](#) / [PT-218474](#))

Parameters

in	<i>inParamID</i>	ID of the parameter whose control is being edited
in	<i>inModifiers</i>	A bitmask of AAX_EModifiers values

Implements [AAX_IViewContainer](#).

14.166.3.8 HandleParameterMouseEnter()

```
AAX_Result AAX_VViewContainer::HandleParameterMouseEnter (
    AAX_CParamID inParamID,
    uint32_t inModifiers ) [virtual]
```

Alert the host to a mouse enter event to the parameter's control.

Parameters

in	<i>inParamID</i>	ID of the parameter whose control is being entered
in	<i>inModifiers</i>	A bitmask of AAX_EModifiers values

Returns [AAX_SUCCESS](#) if event was processed successfully, otherwise an [AAX_ERROR](#) code

Implements [AAX_IViewContainer](#).

14.166.3.9 HandleParameterMouseExit()

```
AAX_Result AAX_VViewContainer::HandleParameterMouseExit (
    AAX_CParamID inParamID,
    uint32_t inModifiers ) [virtual]
```

Alert the host to a mouse exit event from the parameter's control.

Parameters

in	<i>inParamID</i>	ID of the parameter whose control is being exited
in	<i>inModifiers</i>	A bitmask of AAX_EModifiers values

Returns [AAX_SUCCESS](#) if event was processed successfully, otherwise an [AAX_ERROR](#) code

Implements [AAX_IViewContainer](#).

14.166.3.10 HandleMultipleParametersMouseDown()

```
AAX_Result AAX_VViewContainer::HandleMultipleParametersMouseDown (
    const AAX_CParamID * inParamIDs,
    uint32_t inNumOfParams,
    uint32_t inModifiers ) [virtual]
```

Alert the host to a mouse down event.

Parameters

in	<i>inParamIDs</i>	IDs of the parameters that belong to the same GUI element whose controls are being edited
in	<i>inNumOfParams</i>	Number of parameter IDS
in	<i>inModifiers</i>	A bitmask of AAX_EModifiers values

Implements [AAX_IViewContainer](#).

14.166.3.11 HandleMultipleParametersMouseDrag()

```
AAX_Result AAX_VViewContainer::HandleMultipleParametersMouseDrag (
    const AAX_CParamID * inParamIDs,
    uint32_t inNumOfParams,
    uint32_t inModifiers ) [virtual]
```

Alert the host to a mouse drag event.

Warning

The host may return [AAX_ERROR_UNIMPLEMENTED](#) for this event even if the host did handle the corresponding mouse down event. A plug-in should ignore any following mouse drag and mouse up events that correspond to a host-managed mouse down event. ([PTSW-195209](#) / [PT-218474](#))

Parameters

in	<i>inParamIDs</i>	IDs of the parameters that belong to the same GUI element whose controls are being edited
in	<i>inNumOfParams</i>	Number of parameter IDS
in	<i>inModifiers</i>	A bitmask of AAX_EModifiers values

Implements [AAX_IViewContainer](#).

14.166.3.12 HandleMultipleParametersMouseUp()

```
AAX_Result AAX_VViewContainer::HandleMultipleParametersMouseUp (
    const AAX_CParamID * inParamIDs,
    uint32_t inNumOfParams,
    uint32_t inModifiers ) [virtual]
```

Alert the host to a mouse up event.

Warning

The host may return [AAX_ERROR_UNIMPLEMENTED](#) for this event even if the host did handle the corresponding mouse down event. A plug-in should ignore any following mouse drag and mouse up events that correspond to a host-managed mouse down event. ([PTSW-195209](#) / [PT-218474](#))

Parameters

in	<i>inParamIDs</i>	IDs of the parameters that belong to the same GUI element whose controls are being edited
in	<i>inNumOfParams</i>	Number of parameter IDS
in	<i>inModifiers</i>	A bitmask of AAX_EModifiers values

Implements [AAX_IViewContainer](#).

The documentation for this class was generated from the following file:

- [AAX_VViewContainer.h](#)

14.167 AAX::Exception::Any Class Reference

```
#include <AAX_Exception.h>
```

Inheritance diagram for AAX::Exception::Any:

14.167.1 Description

Base class for AAX exceptions

This class is defined within the AAX Library and is always handled within the AAX plug-in. Objects of this class are never passed between the plug-in and the AAX host.

The definition of this class may change between versions of the AAX SDK. This class does not include any form of version safety for cross-version compatibility.

Warning

Do not use multiple inheritance in any sub-classes within the [AAX::Exception::Any](#) inheritance tree

Never pass exceptions across the library boundary to the AAX host

Public Member Functions

- virtual [~Any](#) ()
- template<class C >
 [Any](#) (const C &inWhat)
- template<class C1 , class C2 , class C3 >
 [Any](#) (const C1 &inWhat, const C2 &inFunction, const C3 &inLine)
- [Any](#) (const [Any](#) &inOther)
- [Any](#) & operator= (const [Any](#) &inOther)
- [AAX_DEFAULT_MOVE_CTOR](#) ([Any](#))
- [AAX_DEFAULT_MOVE_OPER](#) ([Any](#))
- const std::string & [What](#) () const
- const std::string & [Desc](#) () const
- const std::string & [Function](#) () const
- const std::string & [Line](#) () const

14.167.2 Constructor & Destructor Documentation

14.167.2.1 ~Any()

```
virtual AAX::Exception::Any::~~Any ( ) [inline], [virtual]
```

14.167.2.2 Any() [1/3]

```
template<class C >
AAX::Exception::Any::Any (
    const C & inWhat ) [inline], [explicit]
```

Explicit conversion from a string-like object

14.167.2.3 Any() [2/3]

```
template<class C1 , class C2 , class C3 >
AAX::Exception::Any::Any (
    const C1 & inWhat,
    const C2 & inFunction,
    const C3 & inLine ) [inline], [explicit]
```

Explicit conversion from a string-like object with function name and line number

14.167.2.4 Any() [3/3]

```
AAX::Exception::Any::Any (
    const Any & inOther ) [inline]
```

14.167.3 Member Function Documentation**14.167.3.1 operator=()**

```
Any& AAX::Exception::Any::operator= (
    const Any & inOther ) [inline]
```

14.167.3.2 AAX_DEFAULT_MOVE_CTOR()

```
AAX::Exception::Any::AAX_DEFAULT_MOVE_CTOR (
    Any )
```

14.167.3.3 AAX_DEFAULT_MOVE_OPER()

```
AAX::Exception::Any::AAX_DEFAULT_MOVE_OPER (
    Any )
```

14.167.3.4 What()

```
const std::string& AAX::Exception::Any::What ( ) const [inline]
```

Referenced by `AAX::AsString()`.

Here is the caller graph for this function:

14.167.3.5 Desc()

```
const std::string& AAX::Exception::Any::Desc ( ) const [inline]
```

14.167.3.6 Function()

```
const std::string& AAX::Exception::Any::Function ( ) const [inline]
```

14.167.3.7 Line()

```
const std::string& AAX::Exception::Any::Line ( ) const [inline]
```

The documentation for this class was generated from the following file:

- [AAX_Exception.h](#)

14.168 AAX_CChunkDataParser::DataValue Struct Reference

```
#include <AAX_CChunkDataParser.h>
```

Collaboration diagram for AAX_CChunkDataParser::DataValue:

Public Member Functions

- [DataValue](#) ()

Public Attributes

- [int32_t mDataType](#)
- [AAX_CString mDataName](#)
name of the stored data
- [int64_t mIntValue](#)
used if this [DataValue](#) is not a string
- [AAX_CString mStringValue](#)
used if this [DataValue](#) is a string

14.168.1 Constructor & Destructor Documentation

14.168.1.1 DataValue()

```
AAX_CChunkDataParser::DataValue::DataValue ( ) [inline]
```

14.168.2 Member Data Documentation

14.168.2.1 mDataType

```
int32_t AAX_CChunkDataParser::DataValue::mDataType
```

14.168.2.2 mDataName

```
AAX_CString AAX_CChunkDataParser::DataValue::mDataName
```

name of the stored data

14.168.2.3 mIntValue

```
int64_t AAX_CChunkDataParser::DataValue::mIntValue
```

used if this [DataValue](#) is not a string

14.168.2.4 mStringValue

```
AAX_CString AAX_CChunkDataParser::DataValue::mStringValue
```

used if this [DataValue](#) is a string

The documentation for this struct was generated from the following file:

- [AAX_CChunkDataParser.h](#)

14.169 IACFDefinition Interface Reference

Inheritance diagram for IACFDefinition:

Collaboration diagram for IACFDefinition:

14.169.1 Description

Publicly inherits from IACFUnknown. This abstract interface is used to indentify all of the plug-in components in the host.

Remarks

This interface is the base class for both plug-in and component definitions. All defined attributes are read only.

Note

This interface does not provide any attribute enumeration. You must know the uid of the associated with the attribute that you need to find.

This interface is implemented by the host. The plug-in will use this interface to define optional attributes for both plug-in and component implementations classes.

Public Member Functions

- virtual ACFRESULT ACFMETHODCALLTYPE [DefineAttribute](#) (const [acfUID](#) &attributeID, const [acfUID](#) &typeID, const void *attrData, acfUInt32 attrDataSize)=0
Add a read only attribute to the definition.
- virtual ACFRESULT ACFMETHODCALLTYPE [GetAttributeInfo](#) (const [acfUID](#) &attributeID, [acfUID](#) *typeID, acfUInt32 *attrDataSize)=0
Returns information about the given attribute.
- virtual ACFRESULT ACFMETHODCALLTYPE [CopyAttribute](#) (const [acfUID](#) &attributeID, const [acfUID](#) &typeID, void *attrData, acfUInt32 attrDataSize)=0
Copy the a given attribute.

14.169.2 Member Function Documentation

14.169.2.1 DefineAttribute()

```
virtual ACFRESULT ACFMETHODCALLTYPE IACFDefinition::DefineAttribute (
    const acfUID & attributeID,
    const acfUID & typeID,
    const void * attrData,
    acfUInt32 attrDataSize ) [pure virtual]
```

Add a read only attribute to the definition.

DefineAttribute

Remarks

Use the method to define additional global attributes for you component. This method will fail if the attribute has already been defined.

Parameters

<i>attributeID</i>	Unique identifier for attribute
<i>typeID</i>	Indicates the type of the attribute data
<i>attrData</i>	Pointer to buffer that contains the attribute data
<i>attrDataSize</i>	Size of the attribute buffer

14.169.2.2 GetAttributeInfo()

```
virtual ACFRESULT ACFMETHODCALLTYPE IACFDefinition::GetAttributeInfo (
    const acfUID & attributeID,
    acfUID * typeID,
    acfUInt32 * attrDataSize ) [pure virtual]
```

Returns information about the given attribute.

Remarks

Use this method to retrieve the type and size of a given attribute.

Parameters

<i>attributeID</i>	Unique identifier for attribute
<i>typeID</i>	Indicates the type of the attribute data
<i>attrDataSize</i>	Size of the attribute data

14.169.2.3 CopyAttribute()

```
virtual ACFRESULT ACFMETHODCALLTYPE IACFDefinition::CopyAttribute (
    const acfUID & attributeID,
    const acfUID & typeID,
    void * attrData,
    acfUInt32 attrDataSize ) [pure virtual]
```

Copy the a given attribute.

CopyAttribute

Remarks

Use this method to access the contents of a given attribute.

Parameters

<i>attributeID</i>	Unique identifier for attribute
<i>typeID</i>	Indicates the type of the attribute data
<i>attrData</i>	Pointer to buffer to copy the attribute data
<i>attrDataSize</i>	Size of the attribute buffer

The documentation for this interface was generated from the following file:

- [AAX_ACFInterface.doxygen](#)

14.170 IACFUnknown Interface Reference

Inheritance diagram for IACFUnknown:

14.170.1 Description

COM compatible IUnknown C++ interface.

Remarks

The methods of the [IACFUnknown](#) interface, implemented by all ACF objects, supports general inter-object protocol negotiation via the `QueryInterface` method, and object lifetime management with the `AddRef` and `Release` methods.

Note

Because `AddRef` and `Release` are not required to return accurate values, callers of these methods must not use the return values to determine if an object is still valid or has been destroyed. (Standard M*cr*s*ft disclaimer)

For further information please refer to the Microsoft documentation for IUnknown.

Note

This class will work only with compilers that can produce COM-compatible object layouts for C++ classes. egcs can not do this. Metrowerks can do this (if you subclass from `__comobject`).

Public Member Functions

- virtual `BEGIN_ACFINTERFACE ACFRESULT ACFMETHODCALLTYPE QueryInterface` (const `acfIID` &iid, void **ppOut)=0
Returns pointers to supported interfaces.
- virtual `acfUInt32 ACFMETHODCALLTYPE AddRef` (void)=0
Increments reference count.
- virtual `acfUInt32 ACFMETHODCALLTYPE Release` (void)=0
Decrements reference count.

14.170.2 Member Function Documentation

14.170.2.1 QueryInterface()

```
virtual BEGIN_ACFINTERFACE ACFRESULT ACFMETHODCALLTYPE IACFUnknown::QueryInterface (
    const acfIID & iid,
    void ** ppOut ) [pure virtual]
```

Returns pointers to supported interfaces.

Remarks

The `QueryInterface` method gives a client access to alternate interfaces implemented by an object. The returned interface pointer will have already had its reference count incremented so the caller will be required to call the `Release` method.

Parameters

<i>iid</i>	Identifier of the requested interface
<i>ppOut</i>	Address of variable that receives the interface pointer associated with iid.

14.170.2.2 AddRef()

```
virtual acfUInt32 ACFMETHODCALLTYPE IACFUnknown::AddRef (
    void ) [pure virtual]
```

Increments reference count.

Remarks

The AddRef method should be called every time a new copy of an interface is made. When this copy is no longer referenced it must be released with the Release method.

14.170.2.3 Release()

```
virtual acfUInt32 ACFMETHODCALLTYPE IACFUnknown::Release (
    void ) [pure virtual]
```

Decrements reference count.

Remarks

Use this method to decrement the reference count. When the reference count reaches zero the object that implements the interface will be deleted.

The documentation for this interface was generated from the following file:

- [AAX_ACFInterface.doxygen](#)

14.171 AAX::Exception::ResultError Class Reference

```
#include <AAX_Exception.h>
```

Inheritance diagram for AAX::Exception::ResultError:

Collaboration diagram for AAX::Exception::ResultError:

14.171.1 Description

[Exception](#) class for [AAX_EError](#) results

Public Member Functions

- [ResultError](#) ([AAX_Result](#) inWhatResult)
- template<class C >
[ResultError](#) ([AAX_Result](#) inWhatResult, const C &inFunction)
- template<class C1 , class C2 >
[ResultError](#) ([AAX_Result](#) inWhatResult, const C1 &inFunction, const C2 &inLine)
- [ResultError](#) (const [ResultError](#) &inOther)
- [AAX_Result Result](#) () const

Static Public Member Functions

- static std::string [FormatResult](#) ([AAX_Result](#) inResult)

14.171.2 Constructor & Destructor Documentation

14.171.2.1 ResultError() [1/4]

```
AAX::Exception::ResultError::ResultError (
    AAX\_Result inWhatResult ) [inline], [explicit]
```

14.171.2.2 ResultError() [2/4]

```
template<class C >
AAX::Exception::ResultError::ResultError (
    AAX\_Result inWhatResult,
    const C & inFunction ) [inline], [explicit]
```

14.171.2.3 ResultError() [3/4]

```
template<class C1 , class C2 >
AAX::Exception::ResultError::ResultError (
    AAX\_Result inWhatResult,
    const C1 & inFunction,
    const C2 & inLine ) [inline], [explicit]
```

14.171.2.4 ResultError() [4/4]

```
AAX::Exception::ResultError::ResultError (
    const ResultError & inOther ) [inline]
```

14.171.3 Member Function Documentation

14.171.3.1 FormatResult()

```
static std::string AAX::Exception::ResultError::FormatResult (
    AAX_Result inResult ) [inline], [static]
```

References `AAX::AsStringInt32()`, and `AAX::AsStringResult()`.

Here is the call graph for this function:

14.171.3.2 Result()

```
AAX_Result AAX::Exception::ResultError::Result ( ) const [inline]
```

The documentation for this class was generated from the following file:

- [AAX_Exception.h](#)

14.172 SAutoArray< T > Struct Template Reference

Public Member Functions

- [SAutoArray](#) ()
- [~SAutoArray](#) ()
- void [Reset](#) (T *inData)
- T * [Get](#) ()

14.172.1 Constructor & Destructor Documentation

14.172.1.1 SAutoArray()

```
template<typename T >
SAutoArray< T >::SAutoArray ( ) [inline]
```

14.172.1.2 ~SAutoArray()

```
template<typename T >
SAutoArray< T >::~~SAutoArray ( ) [inline]
```

References SAutoArray< T >::Reset().

Here is the call graph for this function:

14.172.2 Member Function Documentation

14.172.2.1 Reset()

```
template<typename T >
void SAutoArray< T >::Reset (
    T * inData ) [inline]
```

Referenced by SAutoArray< T >::~SAutoArray().

Here is the caller graph for this function:

14.172.2.2 Get()

```
template<typename T >
T* SAutoArray< T >::Get ( ) [inline]
```

The documentation for this struct was generated from the following file:

- [AAX_MIDILogging.cpp](#)

14.173 AAX_I_SessionDocument::TempoMap Class Reference

```
#include <AAX_I_SessionDocument.h>
```

Inheritance diagram for AAX_I_SessionDocument::TempoMap:

Public Member Functions

- virtual [~TempoMap](#) ()=default
- virtual int32_t [Size](#) () const =0
- virtual [AAX_CTempoBreakpoint](#) const * [Data](#) () const =0

14.173.1 Constructor & Destructor Documentation

14.173.1.1 ~TempoMap()

```
virtual AAX_I_SessionDocument::TempoMap::~~TempoMap ( ) [virtual], [default]
```

14.173.2 Member Function Documentation

14.173.2.1 Size()

```
virtual int32_t AAX_I_SessionDocument::TempoMap::Size ( ) const [pure virtual]
```

Implemented in [AAX_VSessionDocument::VTempoMap](#).

14.173.2.2 Data()

```
virtual AAX_CTempoBreakpoint const* AAX_I_SessionDocument::TempoMap::Data ( ) const [pure virtual]
```

Implemented in [AAX_VSessionDocument::VTempoMap](#).

The documentation for this class was generated from the following file:

- [AAX_I_SessionDocument.h](#)

14.174 AAX_VSessionDocument::VTempoMap Class Reference

```
#include <AAX_VSessionDocument.h>
```

Inheritance diagram for AAX_VSessionDocument::VTempoMap:

Collaboration diagram for AAX_VSessionDocument::VTempoMap:

Public Member Functions

- [~VTempoMap](#) () [AAX_OVERRIDE](#)
- [VTempoMap](#) (IACFUnknown &inDataBuffer)
- [int32_t Size](#) () const [AAX_OVERRIDE](#)
- [AAX_CTempoBreakpoint](#) const * [Data](#) () const [AAX_OVERRIDE](#)

14.174.1 Constructor & Destructor Documentation

14.174.1.1 ~VTempoMap()

```
AAX_VSessionDocument::VTempoMap::~VTempoMap ( )
```

14.174.1.2 VTempoMap()

```
AAX_VSessionDocument::VTempoMap::VTempoMap (
    IACFUnknown & inDataBuffer ) [explicit]
```

14.174.2 Member Function Documentation

14.174.2.1 Size()

```
int32_t AAX_VSessionDocument::VTempoMap::Size ( ) const [virtual]
```

Implements [AAX_ISessionDocument::TempoMap](#).

14.174.2.2 Data()

```
AAX_CTempoBreakpoint const* AAX_VSessionDocument::VTempoMap::Data ( ) const [virtual]
```

Implements [AAX_ISessionDocument::TempoMap](#).

The documentation for this class was generated from the following file:

- [AAX_VSessionDocument.h](#)

Chapter 15

File Documentation

15.1 AAX.h File Reference

```
#include <stdint.h>
#include <stddef.h>
#include "AAX_PreStructAlignmentHelper.h"
#include "AAX_Push2ByteStructAlignment.h"
#include "AAX_PostStructAlignmentHelper.h"
#include "AAX_PopStructAlignment.h"
#include "AAX_Version.h"
#include "AAX_Enums.h"
#include "AAX_Errors.h"
#include "AAX_Properties.h"
```

15.1.1 Description

Various utility definitions for AAX.

Classes

- struct [AAX_SPlugInChunkHeader](#)
Plug-in chunk header.
- struct [AAX_SPlugInChunk](#)
Plug-in chunk header + data.
- struct [AAX_SPlugInIdentifierTriad](#)
Plug-in Identifier Triad.
- struct [AAX_CMidiPacket](#)
Packet structure for MIDI data.
- struct [AAX_CMidiStream](#)
MIDI stream data structure used by [AAX_IMIDINode](#).

Macros

C++ compiler macros

- #define `TI_VERSION` 0
Preprocessor flag indicating compilation for TI.
- #define `AAX_CPP11_SUPPORT` 1
Preprocessor toggle for code which requires C++11 compiler support.

C++ keyword macros

Use these macros for keywords which may not be supported on all compilers

Warning

Be careful when using these macros; they are a workaround and the fallback versions of the macros are not guaranteed to provide identical behavior to the fully-supported versions. Always consider the code which will be generated in each case!

If your code is protected with PACE Fusion and you are using a PACE SDK prior to v4 then you must explicitly define `AAX_CPP11_SUPPORT` 0 in your project's preprocessor settings to avoid encountering source failover caused by AAX header includes with exotic syntax.

- #define `AAX_OVERRIDE` `override`
override keyword macro
- #define `AAX_FINAL` `final`
final keyword macro
- #define `AAX_DEFAULT_DTOR`(X) `~X()` = default
- #define `AAX_DEFAULT_DTOR_OVERRIDE`(X) `~X()` `override` = default
- #define `AAX_DEFAULT_CTOR`(X) `X()` = default
default keyword macro for a class default constructor
- #define `AAX_DEFAULT_COPY_CTOR`(X) `X(const X&)` = default
default keyword macro for a class copy constructor
- #define `AAX_DEFAULT_ASGN_OPER`(X) `X& operator=(const X&)` = default
default keyword macro for a class assignment operator
- #define `AAX_DELETE`(X) `X = delete`
delete keyword macro
- #define `AAX_DEFAULT_MOVE_CTOR`(X) `X(X&&)` = default
default keyword macro for a class move constructor
- #define `AAX_DEFAULT_MOVE_OPER`(X) `X& operator=(X&&)` = default
default keyword macro for a class move-assignment operator
- #define `AAX_CONSTEXPR` `constexpr`
constexpr keyword macro
- #define `AAX_UNIQUE_PTR`(X) `std::unique_ptr<X>`

Pointer definitions

- #define `AAXPointer_32bit` 1
When `AAX_PointerSize` == `AAXPointer_32bit` this is a 32-bit build.
- #define `AAXPointer_64bit` 2
When `AAX_PointerSize` == `AAXPointer_64bit` this is a 64-bit build.
- #define `AAX_PointerSize` `AAXPointer_32bit`
Use this definition to check the pointer size in the current build.

Alignment macros

Use these macros to define struct packing alignment for data structures that will be sent across binary or platform boundaries.

```
#include AAX_ALIGN_FILE_BEGIN
#include AAX_ALIGN_FILE_HOST
#include AAX_ALIGN_FILE_END
// Structure definition
#include AAX_ALIGN_FILE_BEGIN
#include AAX_ALIGN_FILE_RESET
#include AAX_ALIGN_FILE_END
```

See the documentation for each macro for individual usage notes and warnings

- `#define AAX_ALIGN_FILE_HOST "AAX_Push2ByteStructAlignment.h"`
Macro to set alignment for data structures that are shared with the host.
- `#define AAX_ALIGN_FILE_ALG "AAX_Push8ByteStructAlignment.h"`
Macro to set alignment for data structures that are used in the alg.
- `#define AAX_ALIGN_FILE_RESET "AAX_PopStructAlignment.h"`
Macro to reset alignment back to default.
- `#define AAX_ALIGN_FILE_BEGIN "AAX_PreStructAlignmentHelper.h"`
Wrapper macro used for warning suppression.
- `#define AAX_ALIGN_FILE_END "AAX_PostStructAlignmentHelper.h"`
Wrapper macro used for warning suppression.
- `#define AAX_CALLBACK`
- `#define AAX_PREPROCESSOR_CONCAT_HELPER(X, Y) X ## Y`
- `#define AAX_PREPROCESSOR_CONCAT(X, Y) AAX_PREPROCESSOR_CONCAT_HELPER(X, Y)`
- `#define AAX_FIELD_INDEX(aContextType, aMember) ((AAX_CFieldIndex) (offsetof (aContextType, aMember) / sizeof (void *)))`
Compute the index used to address a context field.
- `typedef int32_t AAX_CIndex`
- `typedef AAX_CIndex AAX_CCount`
- `typedef uint8_t AAX_CBoolean`
Cross-compiler boolean type used by AAX interfaces.
- `typedef uint32_t AAX_CSelector`
- `typedef int64_t AAX_CTimestamp`
Time stamp value. Measured against the DAE clock (see [AAX_IComponentDescriptor::AddClock\(\)](#))
- `typedef int64_t AAX_CTimeOfDay`
Hardware running clock value. MIDI packet time stamps are measured against this clock. This is actually the same as TransportCounter, but kept for compatibility.
- `typedef int64_t AAX_CTransportCounter`
Offset of samples from transport start. Same as TimeOfDay, but added for new interfaces as TimeOfDay is a confusing name.
- `typedef float AAX_CSampleRate`
Literal sample rate value used by the [sample rate field](#). For [AAX_eProperty_SampleRate](#), use a mask of [AAX_ESampleRateMask](#).
- `typedef uint32_t AAX_CTypeID`
Matches type of OSType used in classic plugins.
- `typedef int32_t AAX_Result`
- `typedef int32_t AAX_CPropertyValue`
32-bit property values
- `typedef int64_t AAX_CPropertyValue64`
64-bit property values
- `typedef AAX_CPropertyValue AAX_CPointerPropertyValue`
Pointer-sized property values.

- typedef int32_t [AAX_CTargetPlatform](#)
Matches type of [target platform](#).
- typedef [AAX_CIndex](#) [AAX_CFieldIndex](#)
Not used by AAX plug-ins (except in [AAX_FIELD_INDEX](#) macro)
- typedef [AAX_CSelector](#) [AAX_CComponentID](#)
- typedef [AAX_CSelector](#) [AAX_CMeterID](#)
- typedef const char * [AAX_CParamID](#)
Parameter identifier.
- typedef [AAX_CParamID](#) [AAX_CPageTableParamID](#)
Parameter identifier used in a page table.
- typedef const char * [AAX_CEffectID](#)
URL-style Effect identifier. Must be unique among all registered effects in the collection.
- typedef [_acfUID](#) [acfUID](#)
- typedef [acfUID](#) [AAX_Feature_UID](#)
- typedef const float *const * [AAX_CAudioInPort](#)
A cross-platform alignment macro to ensure a data type is aligned properly.
- typedef float *const * [AAX_CAudioOutPort](#)
AAX algorithm audio output port data type
- typedef float *const [AAX_CMeterPort](#)
AAX algorithm meter port data type
- typedef struct [AAX_SPlugInChunkHeader](#) [AAX_SPlugInChunkHeader](#)
- typedef struct [AAX_SPlugInChunk](#) [AAX_SPlugInChunk](#)
- typedef struct [AAX_SPlugInChunk](#) * [AAX_SPlugInChunkPtr](#)
- typedef struct [AAX_SPlugInIdentifierTriad](#) [AAX_SPlugInIdentifierTriad](#)
- typedef struct [AAX_SPlugInIdentifierTriad](#) * [AAX_SPlugInIdentifierTriadPtr](#)
- [AAX_CONSTEXPR](#) size_t [kAAX_ParameterIdentifierMaxSize](#) = 32

15.1.2 Macro Definition Documentation

15.1.2.1 TI_VERSION

```
#define TI_VERSION 0
```

Preprocessor flag indicating compilation for TI.

15.1.2.2 AAX_CPP11_SUPPORT

```
#define AAX_CPP11_SUPPORT 1
```

Preprocessor toggle for code which requires C++11 compiler support.

15.1.2.3 AAX_OVERRIDE

```
#define AAX_OVERRIDE override
```

override keyword macro

15.1.2.4 AAX_FINAL

```
#define AAX_FINAL final
```

final keyword macro

15.1.2.5 AAX_DEFAULT_DTOR

```
#define AAX_DEFAULT_DTOR(  
    X ) ~X() = default
```

15.1.2.6 AAX_DEFAULT_DTOR_OVERRIDE

```
#define AAX_DEFAULT_DTOR_OVERRIDE(  
    X ) ~X() override = default
```

15.1.2.7 AAX_DEFAULT_CTOR

```
#define AAX_DEFAULT_CTOR(  
    X ) X() = default
```

default keyword macro for a class default constructor

15.1.2.8 AAX_DEFAULT_COPY_CTOR

```
#define AAX_DEFAULT_COPY_CTOR(  
    X ) X(const X&) = default
```

default keyword macro for a class copy constructor

15.1.2.9 AAX_DEFAULT_ASGN_OPER

```
#define AAX_DEFAULT_ASGN_OPER(  
    X ) X& operator=(const X&) = default
```

default keyword macro for a class assignment operator

15.1.2.10 AAX_DELETE

```
#define AAX_DELETE(  
    X ) X = delete
```

delete keyword macro

Warning

The non-C++11 version of this macro assumes `public` declaration access

15.1.2.11 AAX_DEFAULT_MOVE_CTOR

```
#define AAX_DEFAULT_MOVE_CTOR(  
    X ) X(X&&) = default
```

default keyword macro for a class move constructor

15.1.2.12 AAX_DEFAULT_MOVE_OPER

```
#define AAX_DEFAULT_MOVE_OPER(  
    X ) X& operator=(X&&) = default
```

default keyword macro for a class move-assignment operator

15.1.2.13 AAX_CONSTEXPR

```
#define AAX_CONSTEXPR constexpr
```

constexpr keyword macro

15.1.2.14 AAX_UNIQUE_PTR

```
#define AAX_UNIQUE_PTR(  
    X ) std::unique_ptr<X>
```

15.1.2.15 AAXPointer_32bit

```
#define AAXPointer_32bit 1
```

When `AAX_PointerSize == AAXPointer_32bit` this is a 32-bit build.

15.1.2.16 AAXPointer_64bit

```
#define AAXPointer_64bit 2
```

When `AAX_PointerSize == AAXPointer_64bit` this is a 64-bit build.

15.1.2.17 AAX_PointerSize

```
#define AAX_PointerSize AAXPointer\_32bit
```

Use this definition to check the pointer size in the current build.

See also

[AAXPointer_32bit](#)

[AAXPointer_64bit](#)

15.1.2.18 AAX_ALIGN_FILE_HOST

```
#define AAX_ALIGN_FILE_HOST "AAX_Push2ByteStructAlignment.h"
```

Macro to set alignment for data structures that are shared with the host.

This macro is used to set alignment for data structures that are part of the AAX ABI. You should not need to use this macro for any custom data structures in your plug-in.

15.1.2.19 AAX_ALIGN_FILE_ALG

```
#define AAX_ALIGN_FILE_ALG "AAX_Push8ByteStructAlignment.h"
```

Macro to set alignment for data structures that are used in the alg.

IMPORTANT: Be very careful to maintain correct data alignment when sending data structures between platforms.

Warning

- This macro does not guarantee data alignment compatibility for data structures which include base classes/structs or virtual functions. The MSVC, GCC and LLVM/clang, and CCS (TI) compilers do not support data structure cross-compatibility for these types of structures. clang will now present a warning when these macros are used on any such structures: `#pragma ms_struct` can not be used with dynamic classes or structures
- Struct Member Alignment (/Zp) on Microsoft compilers must be set to a minimum of 8-byte packing in order for this macro to function properly. For more information, see this MSDN article:
<http://msdn.microsoft.com/en-us/library/ms253935.aspx>

15.1.2.20 AAX_ALIGN_FILE_RESET

```
#define AAX_ALIGN_FILE_RESET "AAX_PopStructAlignment.h"
```

Macro to reset alignment back to default.

15.1.2.21 AAX_ALIGN_FILE_BEGIN

```
#define AAX_ALIGN_FILE_BEGIN "AAX_PreStructAlignmentHelper.h"
```

Wrapper macro used for warning suppression.

This wrapper is required in llvm 10.0 and later due to the addition of the `-Wpragma-pack` warning. This is a useful compiler warning but it is awkward to properly suppress in cases where we are intentionally including only part of the push/pop sequence in a single file, as with the `AAX_ALIGN_FILE_XXX` macros.

15.1.2.22 AAX_ALIGN_FILE_END

```
#define AAX_ALIGN_FILE_END "AAX_PostStructAlignmentHelper.h"
```

Wrapper macro used for warning suppression.

This wrapper is required in llvm 10.0 and later due to the addition of the `-Wpragma-pack` warning. This is a useful compiler warning but it is awkward to properly suppress in cases where we are intentionally including only part of the push/pop sequence in a single file, as with the `AAX_ALIGN_FILE_XXX` macros.

15.1.2.23 AAX_CALLBACK

```
#define AAX_CALLBACK
```

15.1.2.24 AAX_PREPROCESSOR_CONCAT_HELPER

```
#define AAX_PREPROCESSOR_CONCAT_HELPER(  
    X,  
    Y ) X ## Y
```

15.1.2.25 AAX_PREPROCESSOR_CONCAT

```
#define AAX_PREPROCESSOR_CONCAT(  
    X,  
    Y ) AAX_PREPROCESSOR_CONCAT_HELPER(X,Y)
```

15.1.2.26 AAX_FIELD_INDEX

```
#define AAX_FIELD_INDEX(  
    aContextType,  
    aMember ) ((AAX_CFieldIndex) (offsetof (aContextType, aMember) / sizeof (void  
*)) )
```

Compute the index used to address a context field.

This macro expands to a constant expression suitable for use in enumerator definitions and case labels so `int32_t` as `aMember` is a constant specifier.

Parameters

in	<i>aContextType</i>	The name of context type
in	<i>aMember</i>	The name or other specifier of a field of that context type

15.1.3 Typedef Documentation

15.1.3.1 AAX_CIndex

```
typedef int32_t AAX_CIndex
```

Todo Not used by AAX plug-ins (except as [AAX_CFieldIndex](#))

15.1.3.2 AAX_CCount

```
typedef AAX\_CIndex AAX\_CCount
```

Todo Not used by AAX plug-ins

15.1.3.3 AAX_CBoolean

```
typedef uint8_t AAX\_CBoolean
```

Cross-compiler boolean type used by AAX interfaces.

15.1.3.4 AAX_CSelector

```
typedef uint32_t AAX\_CSelector
```

Todo Clean up usage; currently used for a variety of ID-related values

15.1.3.5 AAX_CTimestamp

```
typedef int64_t AAX\_CTimestamp
```

Time stamp value. Measured against the DAE clock (see [AAX_IComponentDescriptor::AddClock\(\)](#))

15.1.3.6 AAX_CTimeOfDay

```
typedef int64_t AAX\_CTimeOfDay
```

Hardware running clock value. MIDI packet time stamps are measured against this clock. This is actually the same as TransportCounter, but kept for compatibility.

15.1.3.7 AAX_CTransportCounter

```
typedef int64_t AAX_CTransportCounter
```

Offset of samples from transport start. Same as TimeOfDay, but added for new interfaces as TimeOfDay is a confusing name.

15.1.3.8 AAX_CSampleRate

```
typedef float AAX_CSampleRate
```

Literal sample rate value used by the [sample rate field](#). For [AAX_eProperty_SampleRate](#), use a mask of [AAX_ESampleRateMask](#).

See also

[sampleRateInMask](#)

15.1.3.9 AAX_CTypeID

```
typedef uint32_t AAX_CTypeID
```

Matches type of OSType used in classic plugins.

15.1.3.10 AAX_Result

```
typedef int32_t AAX_Result
```

15.1.3.11 AAX_CPropertyValue

```
typedef int32_t AAX_CPropertyValue
```

32-bit property values

Use this property value type for all properties unless otherwise specified by the property documentation

15.1.3.12 AAX_CPropertyValue64

```
typedef int64_t AAX_CPropertyValue64
```

64-bit property values

Do not use this value type unless specified explicitly in the property documentation

15.1.3.13 AAX_CPointerPropertyValue

```
typedef AAX_CPropertyValue AAX_CPointerPropertyValue
```

Pointer-sized property values.

Do not use this value type unless specified explicitly in the property documentation

15.1.3.14 AAX_CTargetPlatform

```
typedef int32_t AAX_CTargetPlatform
```

Matches type of [target platform](#).

15.1.3.15 AAX_CFieldIndex

```
typedef AAX_CIndex AAX_CFieldIndex
```

Not used by AAX plug-ins (except in [AAX_FIELD_INDEX](#) macro)

15.1.3.16 AAX_CComponentID

```
typedef AAX_CSelector AAX_CComponentID
```

Todo Not used by AAX plug-ins

15.1.3.17 AAX_CMeterID

```
typedef AAX_CSelector AAX_CMeterID
```

Todo Not used by AAX plug-ins

15.1.3.18 AAX_CParamID

```
typedef const char* AAX_CParamID
```

Parameter identifier.

Note

While this is a string, it must be less than 32 characters in length. (strlen of 31 or less)

See also

[kAAX_ParameterIdentifierMaxSize](#)

15.1.3.19 AAX_CPageTableParamID

```
typedef AAX_CParamID AAX_CPageTableParamID
```

Parameter identifier used in a page table.

May be a parameter ID or a parameter name string depending on the page table formatting. Must be less than 32 characters in length (strlen of 31 or less.)

See also

[Parameter identifiers](#) in the [Page Table Guide](#)

15.1.3.20 AAX_CEffectID

```
typedef const char* AAX_CEffectID
```

URL-style Effect identifier. Must be unique among all registered effects in the collection.

15.1.3.21 acfUID

```
typedef _acfUID acfUID
```

15.1.3.22 AAX_Feature_UID

```
typedef acfUID AAX_Feature_UID
```

Identifier for AAX features

See [AAX_IDescriptionHost::AcquireFeatureProperties\(\)](#) and [AAX_IFeatureInfo](#)

15.1.3.23 AAX_CAudioInPort

```
typedef const float* const* AAX_CAudioInPort
```

A cross-platform alignment macro to ensure a data type is aligned properly.

AAX algorithm audio input port data type

Audio input ports are provided with a pointer to an array of const audio buffers, with one buffer provided per input or side chain channel.

Todo Not used directly by AAX plug-ins

15.1.3.24 AAX_CAudioOutPort

```
typedef float* const* AAX_CAudioOutPort
```

AAX algorithm audio output port data type

Audio output ports are provided with a pointer to an array of audio buffers, with one buffer provided per output or auxiliary output channel.

Todo Not used directly by AAX plug-ins

15.1.3.25 AAX_CMeterPort

```
typedef float* const AAX_CMeterPort
```

AAX algorithm meter port data type

Meter output ports are provided with a pointer to an array of floats, with one float provided per meter tap. The algorithm is responsible for setting these to the corresponding per-buffer peak sample values.

Todo Not used directly by AAX plug-ins

15.1.3.26 AAX_SPlugInChunkHeader

```
typedef struct AAX_SPlugInChunkHeader AAX_SPlugInChunkHeader
```

15.1.3.27 AAX_SPlugInChunk

```
typedef struct AAX_SPlugInChunk AAX_SPlugInChunk
```

15.1.3.28 AAX_SPlugInChunkPtr

```
typedef struct AAX_SPlugInChunk * AAX_SPlugInChunkPtr
```

15.1.3.29 AAX_SPlugInIdentifierTriad

```
typedef struct AAX_SPlugInIdentifierTriad AAX_SPlugInIdentifierTriad
```

15.1.3.30 AAX_SPlugInIdentifierTriadPtr

```
typedef struct AAX_SPlugInIdentifierTriad * AAX_SPlugInIdentifierTriadPtr
```

15.1.4 Variable Documentation

15.1.4.1 kAAX_ParameterIdentifierMaxSize

```
AAX_CONSTEXPR size_t kAAX_ParameterIdentifierMaxSize = 32
```

Maximum size for a [AAX_CParamID](#) including the null-terminating character

15.2 AAX_ACFInterface.doxygen File Reference

Classes

- struct [_acfUID](#)
- interface [IACFUnknown](#)
COM compatible IUnknown C++ interface.
- interface [IACFDefinition](#)
Publicly inherits from IACFUnknown. This abstract interface is used to indentify all of the plug-in components in the host.

Typedefs

- typedef struct [_acfUID](#) [acfUID](#)
GUID compatible structure for ACF.
- typedef [acfUID](#) [acfIID](#)
IID compatible structure for ACF.

15.2.1 Typedef Documentation

15.2.1.1 acfUID

[acfUID](#)

GUID compatible structure for ACF.

15.2.1.2 acfIID

[acfIID](#)

IID compatible structure for ACF.

15.3 AAX_AdditionalFeatures_Algorithm.doxygen File Reference

15.4 AAX_AdditionalFeatures_AOSandSidechain.doxygen File Reference

15.5 AAX_AdditionalFeatures_CurveDisplays.doxygen File Reference

15.6 AAX_AdditionalFeatures_Hybrid.doxygen File Reference

15.7 AAX_AdditionalFeatures_Meters.doxygen File Reference

15.8 AAX_AdditionalFeatures_MIDI.doxygen File Reference

15.9 AAX_AdditionalFeatures_PropertiesFile.doxygen File Reference

15.10 AAX_Alignment.h File Reference

```
#include <stddef.h>
```

15.10.1 Description

Alignment malloc and free methods for optimization.

Namespaces

- [AAX](#)

Functions

- void [AAX::alignFree](#) (void *p)
- template<class T >
T * [AAX::alignMalloc](#) (int iArraySize, int iAlignment)

15.11 AAX_Assert.h File Reference

```
#include "AAX_Enums.h"
#include "AAX_CHostServices.h"
```

15.11.1 Description

Declarations for cross-platform AAX_ASSERT, AAX_TRACE and related facilities.

- [AAX_ASSERT\(condition \)](#) - If the condition is `false` triggers some manner of warning, e.g. a dialog in a developer build or a DigiTrace log in a shipping build. May be used on host or TI.
- [AAX_DEBUGASSERT\(condition \)](#) - Variant of [AAX_ASSERT](#) which is only active in debug builds of the plug-in.
- [AAX_TRACE_RELEASE\(iPriority, iMessageStr \[,params...\] \)](#) - Traces a printf- style message to the DigiTrace log file. Enabled using the `DTF_AAXPLUGINS` DigiTrace facility.
- [AAX_TRACE\(iPriority, iMessageStr \[, params...\] \)](#) - Variant of [AAX_TRACE_RELEASE](#) which only emits logs in debug builds of the plug-in.
- [AAX_STACKTRACE_RELEASE\(iPriority, iMessageStr \[,params...\] \)](#) - Similar to [AAX_TRACE_RELEASE](#) but prints a stack trace as well as a log message
- [AAX_STACKTRACE\(iPriority, iMessageStr \[,params...\] \)](#) - Variant of [AAX_STACKTRACE_RELEASE](#) which only emits logs in debug builds of the plug-in.
- [AAX_TRACEORSTACKTRACE_RELEASE\(iTracePriority, iStackTracePriority, iMessageStr \[,params...\] \)](#) - Combination of [AAX_TRACE_RELEASE](#) and [AAX_STACKTRACE_RELEASE](#); a stack trace is emitted if logging is enabled at `iStackTracePriority`. Otherwise, if logging is enabled at `iTracePriority` then emits a log.

For all trace macros:

`iPriority` is one of

- [kAAX_Trace_Priority_Low](#)
- [kAAX_Trace_Priority_Normal](#)
- [kAAX_Trace_Priority_High](#)
- [kAAX_Trace_Priority_Critical](#)

These correspond to how the trace messages are filtered using [DigiTrace](#).

Note

Disabling the `DTF_AAXPLUGINS` facility will slightly reduce the overhead of trace statements and chip communication on HDX systems.

=====

Macros

- `#define kAAX_Trace_Priority_None AAX_eTracePriorityHost_None`
- `#define kAAX_Trace_Priority_Critical AAX_eTracePriorityHost_Critical`
- `#define kAAX_Trace_Priority_High AAX_eTracePriorityHost_High`
- `#define kAAX_Trace_Priority_Normal AAX_eTracePriorityHost_Normal`
- `#define kAAX_Trace_Priority_Low AAX_eTracePriorityHost_Low`
- `#define kAAX_Trace_Priority_Lowest AAX_eTracePriorityHost_Lowest`
- `#define AAX_TRACE_RELEASE(iPriority, ...)`
Print a trace statement to the log.
- `#define AAX_STACKTRACE_RELEASE(iPriority, ...)`
Print a stack trace statement to the log.
- `#define AAX_TRACEORSTACKTRACE_RELEASE(iTracePriority, iStackTracePriority, ...)`
Print a trace statement with an optional stack trace to the log.
- `#define AAX_ASSERT(condition)`
Asserts that a condition is true and logs an error if the condition is false.
- `#define AAX_DEBUGASSERT(condition) do { ; } while (0)`
Asserts that a condition is true and logs an error if the condition is false (debug plug-in builds only)
- `#define AAX_TRACE(iPriority, ...) do { ; } while (0)`
Print a trace statement to the log (debug plug-in builds only)
- `#define AAX_STACKTRACE(iPriority, ...) do { ; } while (0)`
Print a stack trace statement to the log (debug builds only)
- `#define AAX_TRACEORSTACKTRACE(iTracePriority, iStackTracePriority, ...) do { ; } while (0)`
Print a trace statement with an optional stack trace to the log (debug builds only)

Typedefs

- `typedef AAX_ETracePriorityHost AAX_ETracePriority`

15.11.2 Macro Definition Documentation

15.11.2.1 kAAX_Trace_Priority_None

```
#define kAAX_Trace_Priority_None AAX_eTracePriorityHost_None
```

15.11.2.2 kAAX_Trace_Priority_Critical

```
#define kAAX_Trace_Priority_Critical AAX_eTracePriorityHost_Critical
```

15.11.2.3 kAAX_Trace_Priority_High

```
#define kAAX_Trace_Priority_High AAX_eTracePriorityHost_High
```

15.11.2.4 kAAX_Trace_Priority_Normal

```
#define kAAX_Trace_Priority_Normal AAX_eTracePriorityHost_Normal
```

15.11.2.5 kAAX_Trace_Priority_Low

```
#define kAAX_Trace_Priority_Low AAX_eTracePriorityHost_Low
```

15.11.2.6 kAAX_Trace_Priority_Lowest

```
#define kAAX_Trace_Priority_Lowest AAX_eTracePriorityHost_Lowest
```

15.11.2.7 AAX_TRACE_RELEASE

```
#define AAX_TRACE_RELEASE(
    iPriority,
    ... )
```

Value:

```
{ \
    AAX_CHostServices::Trace ( iPriority, __VA_ARGS__ ); \
};
```

Print a trace statement to the log.

Use this macro to print a trace statement to the log file. This macro will be included in all builds of the plug-in.

Notes

- This macro is compatible with bost host and embedded ([AAX DSP](#)) environments
- Subject to a total line limit of 256 chars

Usage Each invocation of this macro takes a trace priority and a `printf`-style logging string.

Because output from this macro will be enabled on end users' systems under certain tracing configurations, logs should always be formatted with some standard information to avoid confusion between logs from different plug-ins. This is the recommended formatting for `AAX_TRACE_RELEASE` logs:

```
[Manufacturer name] [Plug-in name] [Plug-in version][logging text (indented)]
```

For example:

```
AAX_TRACE_RELEASE(kAAX_Trace_Priority_Normal, "%s %s %s;\tMy float: %f, My C-string: %s",
    "MyCompany", "MyPlugIn", "1.0.2", myFloat, myCString);
```

See also

[DigiTrace Guide](#)

15.11.2.8 AAX_STACKTRACE_RELEASE

```
#define AAX_STACKTRACE_RELEASE(
    iPriority,
    ... )
```

Value:

```
{ \
    AAX_CHostServices::StackTrace ( iPriority, iPriority, __VA_ARGS__ ); \
};
```

Print a stack trace statement to the log.

See also

[AAX_TRACE_RELEASE](#)

15.11.2.9 AAX_TRACEORSTACKTRACE_RELEASE

```
#define AAX_TRACEORSTACKTRACE_RELEASE (
    iTracePriority,
    iStackTracePriority,
    ... )
```

Value:

```
{ \
    AAX_CHostServices::StackTrace ( iTracePriority, iStackTracePriority, __VA_ARGS__ ); \
};
```

Print a trace statement with an optional stack trace to the log.

Parameters

in	<i>iTracePriority</i>	The log priority at which the trace statement will be printed
in	<i>iStackTracePriority</i>	The log priority at which the stack trace will be printed

See also

[AAX_TRACE_RELEASE](#)

15.11.2.10 AAX_ASSERT

```
#define AAX_ASSERT(
    condition )
```

Value:

```
{ \
    if( ! ( condition ) ) { \
        AAX_CHostServices::HandleAssertFailure( __FILE__, __LINE__, #condition,
(int32_t)AAX_eAssertFlags_Log ); \
    } \
};
```

Asserts that a condition is true and logs an error if the condition is false.

Notes

- This macro will be compiled out of release builds.
- This macro is compatible with bost host and embedded ([AAX DSP](#)) environments.

Usage Each invocation of this macro takes a single argument, which is interpreted as a `bool`.
[AAX_ASSERT](#)(desiredValue == variableUnderTest);

15.11.2.11 AAX_DEBUGASSERT

```
#define AAX_DEBUGASSERT(  
    condition ) do { ; } while (0)
```

Asserts that a condition is true and logs an error if the condition is false (debug plug-in builds only)

See also

[AAX_ASSERT](#)

15.11.2.12 AAX_TRACE

```
#define AAX_TRACE(  
    iPriority,  
    ... ) do { ; } while (0)
```

Print a trace statement to the log (debug plug-in builds only)

Use this macro to print a trace statement to the log file from debug builds of a plug-in.

Notes

- This macro will be compiled out of release builds
- This macro is compatible with bost host and embedded ([AAX DSP](#)) environments
- Subject to a total line limit of 256 chars

Usage Each invocation of this macro takes a trace priority and a `printf`-style logging string. For example:
`AAX_TRACE(kAAX_Trace_Priority_Normal, "My float: %f, My C-string: %s", myFloat, myCString);`

See also

[DigiTrace Guide](#)

15.11.2.13 AAX_STACKTRACE

```
#define AAX_STACKTRACE(  
    iPriority,  
    ... ) do { ; } while (0)
```

Print a stack trace statement to the log (debug builds only)

See also

[AAX_TRACE](#)

15.11.2.14 AAX_TRACEORSTACKTRACE

```
#define AAX_TRACEORSTACKTRACE(  
    iTracePriority,  
    iStackTracePriority,  
    ... ) do { ; } while (0)
```

Print a trace statement with an optional stack trace to the log (debug builds only)

Parameters

in	<i>iTracePriority</i>	The log priority at which the trace statement will be printed
in	<i>iStackTracePriority</i>	The log priority at which the stack trace will be printed

See also

[AAX_TRACE](#)

15.11.3 Typedef Documentation

15.11.3.1 AAX_ETracePriority

```
typedef AAX_ETracePriorityHost AAX_ETracePriority
```

15.12 AAX_Atomic.h File Reference

```
#include "AAX.h"
#include <stdint.h>
```

15.12.1 Description

Atomic operation utilities.

Macros

- #define [AAX_ATOMIC_H_](#)

Functions

- uint32_t [AAX_CALLBACK AAX_Atomic_IncThenGet_32](#) (uint32_t &ioData)
Increments a 32-bit value and returns the result.
- uint32_t [AAX_CALLBACK AAX_Atomic_DecThenGet_32](#) (uint32_t &ioData)
Decrements a 32-bit value and returns the result.
- uint32_t [AAX_CALLBACK AAX_Atomic_Exchange_32](#) (volatile uint32_t &ioValue, uint32_t inExchange↔Value)
Return the original value of ioValue and then set it to inExchangeValue.
- uint64_t [AAX_CALLBACK AAX_Atomic_Exchange_64](#) (volatile uint64_t &ioValue, uint64_t inExchange↔Value)
Return the original value of ioValue and then set it to inExchangeValue.

- `template<typename TPointer >`
`TPointer *AAX_CALLBACK AAX_Atomic_Exchange_Pointer (TPointer *&ioValue, TPointer *inExchange↵`
`Value)`
Perform an exchange operation on a pointer value.
- `bool AAX_CALLBACK AAX_Atomic_CompareAndExchange_32 (volatile uint32_t &ioValue, uint32_t in↵`
`CompareValue, uint32_t inExchangeValue)`
Perform a compare and exchange operation on a 32-bit value.
- `bool AAX_CALLBACK AAX_Atomic_CompareAndExchange_64 (volatile uint64_t &ioValue, uint64_t in↵`
`CompareValue, uint64_t inExchangeValue)`
Perform a compare and exchange operation on a 64-bit value.
- `template<typename TPointer >`
`bool AAX_CALLBACK AAX_Atomic_CompareAndExchange_Pointer (TPointer *&ioValue, TPointer *in↵`
`CompareValue, TPointer *inExchangeValue)`
Perform a compare and exchange operation on a pointer value.
- `template<typename TPointer >`
`TPointer *AAX_CALLBACK AAX_Atomic_Load_Pointer (TPointer const *const volatile *inValue)`
Atomically loads a pointer value.

15.12.2 Macro Definition Documentation

15.12.2.1 AAX_ATOMIC_H_

```
#define AAX_ATOMIC_H_
```

15.12.3 Function Documentation

15.12.3.1 AAX_Atomic_IncThenGet_32()

```
uint32_t AAX_CALLBACK AAX_Atomic_IncThenGet_32 (
    uint32_t & ioData )
```

Increments a 32-bit value and returns the result.

15.12.3.2 AAX_Atomic_DecThenGet_32()

```
uint32_t AAX_CALLBACK AAX_Atomic_DecThenGet_32 (
    uint32_t & ioData )
```

Decrements a 32-bit value and returns the result.

15.12.3.3 AAX_Atomic_Exchange_32()

```
uint32_t AAX_CALLBACK AAX_Atomic_Exchange_32 (
    volatile uint32_t & ioValue,
    uint32_t inExchangeValue )
```

Return the original value of ioValue and then set it to inExchangeValue.

Referenced by AAX_Atomic_Exchange_Pointer().

Here is the caller graph for this function:

15.13 AAX_AuxInterface_DirectData.doxygen File Reference

15.14 AAX_AuxInterface_HostProcessor.doxygen File Reference

15.15 AAX_AuxInterface_TaskAgent.doxygen File Reference

15.16 AAX_BugList.doxygen File Reference

15.17 AAX_Callbacks.h File Reference

```
#include "AAX.h"
```

15.17.1 Description

AAX callback prototypes and ProcPtr definitions

Classes

- class [AAX_Component< aContextType >](#)
Empty class containing type declarations for the AAX algorithm and associated callbacks.

Typedefs

- typedef [IACFUnknown](#) *[AAX_CALLBACK](#) * [AAXCreateObjectProc](#)(void)
- typedef [AAX_Component](#)< void >::CProcessProc [AAX_CProcessProc](#)
A user-defined callback that AAX calls to process data packets and/or audio.
- typedef [AAX_Component](#)< void >::CPacketAllocator [AAX_CPacketAllocator](#)
Used by AAX_SchedulePacket()
- typedef [AAX_Component](#)< void >::CInstanceInitProc [AAX_CInstanceInitProc](#)
A user-defined callback that AAX calls to notify the component that an instance is being added or removed.
- typedef [AAX_Component](#)< void >::CBackgroundProc [AAX_CBackgroundProc](#)
A user-defined callback that AAX calls in the AAX Idle time.
- typedef [AAX_Component](#)< void >::CInitPrivateDataProc [AAX_CInitPrivateDataProc](#)
A user-defined callback to initialize a private data block.

Enumerations

- enum `AAX_CProcPtrID` {
`kAAX_ProcPtrID_Create_EffectParameters` = 0 ,
`kAAX_ProcPtrID_Create_EffectGUI` = 1 ,
`kAAX_ProcPtrID_Create_HostProcessor` = 3 ,
`kAAX_ProcPtrID_Create_EffectDirectData` = 5 ,
`kAAX_ProcPtrID_Create_TaskAgent` = 6 ,
`kAAX_ProcPtrID_Create_SessionDocumentClient` = 7 }

15.17.2 Typedef Documentation

15.17.2.1 AAXCreateObjectProc

```
typedef IACFUnknown* AAX_CALLBACK* AAXCreateObjectProc(void)
```

15.17.2.2 AAX_CProcessProc

```
typedef AAX_Component<void>::CProcessProc AAX_CProcessProc
```

A user-defined callback that AAX calls to process data packets and/or audio.

iContextPtrsBegin

A vector of context pointers. Each element points to the context for one instance of this component. `iContextPtrsEnd` gives the upper bound of the vector and `(iContextPtrsEnd - iContextPtrsBegin)` gives the count.

iContextPtrsEnd

The upper bound of the vector at `iContextPtrsBegin`. `(iContextPtrsEnd - iContextPtrsBegin)` gives the count of this vector.

The instance vector was originally NULL-terminated in earlier versions of this API. However, the STL-style begin/end pattern was suggested as a more general representation that could, for instance, allow a vector to be split for parallel processing.

15.17.2.3 AAX_CPacketAllocator

```
typedef AAX_Component<void>::CPacketAllocator AAX_CPacketAllocator
```

Used by `AAX_SchedulePacket()`

Deprecated

A `AAX_CProcessProc` that calls `AAX_SchedulePacket()` must include a `AAX_CPacketAllocator` field in its context and register that field with `AAX`. `AAX` will then populate that field with a `AAX_CPacketAllocator` to pass to `AAX_SchedulePacket()`.

See also

`AAX_SchedulePacket()`

15.17.2.4 AAX_CInstanceInitProc

```
typedef AAX_Component<void>::CInstanceInitProc AAX_CInstanceInitProc
```

A user-defined callback that AAX calls to notify the component that an instance is being added or removed.

This optional callback allows the component to keep per-instance data. It's called before the instance appears in the list supplied to CProcessProc, and then after the instance is removed from the list.

iInstanceContextPtr

A pointer to the context data structure of the instance being added or removed from the processing list.

iAction

Indicates the action that triggered the init callback, e.g. whether the instance is being added or removed.

Return values

<i>Should</i>	return 0 on success, anything else on failure. Failure will prevent the instance from being created.
---------------	--

15.17.2.5 AAX_CBackgroundProc

```
typedef AAX_Component<void>::CBackgroundProc AAX_CBackgroundProc
```

A user-defined callback that AAX calls in the AAX Idle time.

This optional callback allows the component to do background processing in whatever manner the plug-in developer desires

Return values

<i>Should</i>	return 0 on success, anything else on failure. Failure will cause the AAX host to signal an error up the callchain.
---------------	---

15.17.2.6 AAX_CInitPrivateDataProc

```
typedef AAX_Component<void>::CInitPrivateDataProc AAX_CInitPrivateDataProc
```

A user-defined callback to initialize a private data block.

Deprecated

A component that requires private data supplies [AAX_CInitPrivateDataProc](#) callbacks to set its private data to the state it should be in at the start of audio. The component first declares one or more pointers to private data in its context. It then registers each such field with AAX along with its data size, various other attributes, and a [AAX_CInitPrivateDataProc](#). The [AAX_CInitPrivateDataProc](#) always runs on the host system, not the DSP. AAX allocates storage for each private data block and calls its associated [AAX_CInitPrivateDataProc](#) to initialize it. If the component's [AAX_CProcessProc](#) runs on external hardware, AAX initializes private data blocks on the host system and copies them to the remote system.

See also

`alg_pd_init`

inFieldIndex

The port ID of the block to be initialized, as generated by [AAX_FIELD_INDEX\(\)](#). A component can register a separate [AAX_CInitPrivateDataProc](#) for each of its private data blocks, or it can use fewer functions that switch on `inFieldIndex`.

inNewBlock

A pointer to the block to be initialized. If the component runs externally, AAX will copy this block to the remote system after it is initialized.

inSize

The size of the block to be initialized. If a component has multiple private blocks that only need to be zeroed out, say, it can use a single [AAX_CInitPrivateDataProc](#) for all of these blocks that zeroes them out according to `inSize`.

inController

A pointer to the current Effect instance's [AAX_IController](#).

Note

Do not directly reference data from this interface when populating `iNewBlock`. The data in this block must be fully self-contained to ensure portability to a new device or memory space.

Deprecated

15.17.3 Enumeration Type Documentation

15.17.3.1 AAX_CProcPtrID

enum [AAX_CProcPtrID](#)

Enumerator

kAAX_ProcPtrID_Create_EffectParameters	AAX_IEffectParameters creation procedure
kAAX_ProcPtrID_Create_EffectGUI	AAX_IEffectGUI creation procedure
kAAX_ProcPtrID_Create_HostProcessor	AAX_IHostProcessor creation procedure
kAAX_ProcPtrID_Create_EffectDirectData	AAX_IEffectDirectData creation procedure, used by plug-ins that want direct access to their alg memory
kAAX_ProcPtrID_Create_TaskAgent	AAX_ITaskAgent creation procedure, used by plug-ins that want to process task requests made by the host.
kAAX_ProcPtrID_Create_SessionDocumentClient	AAX_ISessionDocumentClient creation procedure

15.18 AAX_CArrayDataBuffer.h File Reference

```
#include "AAX_IDataBuffer.h"
#include "AAX.h"
#include <string>
#include <limits>
#include <type_traits>
```

Classes

- class [AAX_CArrayDataBufferOfType< T, D >](#)
A convenience class for array data buffers.
- class [AAX_CArrayDataBuffer< D >](#)
A convenience class for array data buffers.

Macros

- #define [AAX_CArrayDataBuffer_H](#)

15.18.1 Macro Definition Documentation

15.18.1.1 AAX_CArrayDataBuffer_H

```
#define AAX_CArrayDataBuffer_H
```

15.19 AAX_CAtomicQueue.h File Reference

```
#include "AAX_IPointerQueue.h"
#include "AAX_Atomic.h"
#include "AAX_CMutex.h"
#include <cstring>
```

15.19.1 Description

Atomic, non-blocking queue.

Classes

- class [AAX_CAtomicQueue< T, S >](#)

15.20 AAX_CAutoreleasePool.h File Reference

15.20.1 Description

Autorelease pool helper utility.

Classes

- class [AAX_CAutoreleasePool](#)

Macros

- `#define` [_AAX_CAUTORELEASEPOOL_H_](#)

15.20.2 Macro Definition Documentation

15.20.2.1 `_AAX_CAUTORELEASEPOOL_H_`

```
#define _AAX_CAUTORELEASEPOOL_H_
```

15.21 AAX_CBinaryDisplayDelegate.h File Reference

```
#include "AAX_IDisplayDelegate.h"  
#include "AAX_CString.h"  
#include <vector>  
#include <algorithm>
```

15.21.1 Description

A binary display delegate.

Classes

- class [AAX_CBinaryDisplayDelegate< T >](#)
A binary display format conforming to [AAX_IDisplayDelegate](#).

15.22 AAX_CBinaryTaperDelegate.h File Reference

```
#include "AAX_ITaperDelegate.h"
```

15.22.1 Description

A binary taper delegate.

Classes

- class [AAX_CBinaryTaperDelegate< T >](#)
A binary taper conforming to [AAX_ITaperDelegate](#).

15.23 AAX_CChunkDataParser.h File Reference

```
#include "AAX.h"  
#include "AAX_CString.h"  
#include <vector>
```

15.23.1 Description

Parser utility for plugin chunks.

Classes

- class [AAX_CChunkDataParser](#)
Parser utility for plugin chunks.
- struct [AAX_CChunkDataParser::DataValue](#)

Namespaces

- [AAX_ChunkDataParserDefs](#)
Constants used by ChunkDataParser class.

Macros

- `#define` [AAX_CHUNKDATAPARSER_H](#)

Variables

- `const int32_t AAX_ChunkDataParserDefs::FLOAT_TYPE = 1`
- `const char AAX_ChunkDataParserDefs::FLOAT_STRING_IDENTIFIER [] = "f_"`
- `const int32_t AAX_ChunkDataParserDefs::LONG_TYPE = 2`
- `const char AAX_ChunkDataParserDefs::LONG_STRING_IDENTIFIER [] = "l_"`
- `const int32_t AAX_ChunkDataParserDefs::DOUBLE_TYPE = 3`
- `const char AAX_ChunkDataParserDefs::DOUBLE_STRING_IDENTIFIER [] = "d_"`
- `const size_t AAX_ChunkDataParserDefs::DOUBLE_TYPE_SIZE = 8`
- `const size_t AAX_ChunkDataParserDefs::DOUBLE_TYPE_INCR = 8`
- `const int32_t AAX_ChunkDataParserDefs::SHORT_TYPE = 4`
- `const char AAX_ChunkDataParserDefs::SHORT_STRING_IDENTIFIER [] = "s_"`
- `const size_t AAX_ChunkDataParserDefs::SHORT_TYPE_SIZE = 2`
- `const size_t AAX_ChunkDataParserDefs::SHORT_TYPE_INCR = 4`
- `const int32_t AAX_ChunkDataParserDefs::STRING_TYPE = 5`
- `const char AAX_ChunkDataParserDefs::STRING_STRING_IDENTIFIER [] = "r_"`
- `const size_t AAX_ChunkDataParserDefs::MAX_STRINGDATA_LENGTH = 255`
- `const size_t AAX_ChunkDataParserDefs::DEFAULT32BIT_TYPE_SIZE = 4`
- `const size_t AAX_ChunkDataParserDefs::DEFAULT32BIT_TYPE_INCR = 4`
- `const size_t AAX_ChunkDataParserDefs::STRING_IDENTIFIER_SIZE = 2`
- `const int32_t AAX_ChunkDataParserDefs::NAME_NOT_FOUND = -1`
- `const size_t AAX_ChunkDataParserDefs::MAX_NAME_LENGTH = 255`
- `const int32_t AAX_ChunkDataParserDefs::BUILD_DATA_FAILED = -333`
- `const int32_t AAX_ChunkDataParserDefs::HEADER_SIZE = 4`
- `const int32_t AAX_ChunkDataParserDefs::VERSION_ID_1 = 0x01010101`

15.23.2 Macro Definition Documentation

15.23.2.1 AAX_CHUNKDATAPARSER_H

```
#define AAX_CHUNKDATAPARSER_H
```

15.24 AAX_CDecibelDisplayDelegateDecorator.h File Reference

```
#include "AAX_IDisplayDelegateDecorator.h"
#include <cmath>
```

15.24.1 Description

A decibel display delegate.

Classes

- class [AAX_CDecibelDisplayDelegateDecorator< T >](#)
A percent decorator conforming to [AAX_IDisplayDelegateDecorator](#).

15.25 AAX_CEffectDirectData.h File Reference

```
#include "AAX_IEffectDirectData.h"
```

15.25.1 Description

A default implementation of the [AAX_IEffectDirectData](#) interface.

Classes

- class [AAX_CEffectDirectData](#)
Default implementation of the [AAX_IEffectDirectData](#) interface.

Macros

- #define [AAX_CEFFECTDIRECTDATA_H](#)

15.25.2 Macro Definition Documentation

15.25.2.1 AAX_CEFFECTDIRECTDATA_H

```
#define AAX_CEFFECTDIRECTDATA_H
```

15.26 AAX_CEffectGUI.h File Reference

```
#include "AAX_IEffectGUI.h"  
#include "AAX_IACFEffectParameters.h"  
#include <string>  
#include <vector>  
#include <map>  
#include <memory>
```

15.26.1 Description

A default implementation of the [AAX_IEffectGUI](#) interface.

Classes

- class [AAX_CEffectGUI](#)
Default implementation of the [AAX_IEffectGUI](#) interface.

15.27 AAX_CEffectParameters.h File Reference

```
#include "AAX_IEffectParameters.h"
#include "AAX_IPageTable.h"
#include "AAX_CString.h"
#include "AAX_CChunkDataParser.h"
#include "AAX_CParameterManager.h"
#include "AAX_CPacketDispatcher.h"
#include <set>
#include <string>
#include <vector>
```

15.27.1 Description

A default implementation of the `AAX_IeffectParameters` interface.

Classes

- class [AAX_CEffectParameters](#)
Default implementation of the [AAX_IEffectParameters](#) interface.

Functions

- `int32_t` [NormalizedToInt32](#) (`double normalizedValue`)
- `double` [Int32ToNormalized](#) (`int32_t value`)
- `double` [BoolToNormalized](#) (`bool value`)

Variables

- [AAX_CParamID cPreviewID](#)
- [AAX_CParamID cDefaultMasterBypassID](#)

15.27.2 Function Documentation

15.27.2.1 NormalizedToInt32()

```
int32_t NormalizedToInt32 (
    double normalizedValue )
```

15.27.2.2 Int32ToNormalized()

```
double Int32ToNormalized (
    int32_t value )
```

15.27.2.3 BoolToNormalized()

```
double BoolToNormalized (
    bool value )
```

15.27.3 Variable Documentation

15.27.3.1 cPreviewID

[AAX_CParamID](#) cPreviewID

15.27.3.2 cDefaultMasterBypassID

[AAX_CParamID](#) cDefaultMasterBypassID

15.28 AAX_CHostProcessor.h File Reference

```
#include "AAX_IEffectParameters.h"
#include "AAX_IHostProcessor.h"
#include "ACFPtr.h"
```

15.28.1 Description

Concrete implementation of the [AAX_IHostProcessor](#) interface for non-real-time processing.

Classes

- class [AAX_CHostProcessor](#)

Concrete implementation of the [AAX_IHostProcessor](#) interface for non-real-time processing.

15.29 AAX_CHostServices.h File Reference

```
#include "AAX.h"
#include "AAX_Enums.h"
```

15.29.1 Description

Concrete implementation of the [AAX_IHostServices](#) interface.

Classes

- class [AAX_CHostServices](#)
Method access to a singleton implementation of the [AAX_IHostServices](#) interface.

15.30 AAX_CLinearTaperDelegate.h File Reference

```
#include "AAX_ITaperDelegate.h"
#include "AAX.h"
#include <cmath>
```

15.30.1 Description

A linear taper delegate.

Classes

- class [AAX_CLinearTaperDelegate](#)< T, RealPrecision >
A linear taper conforming to [AAX_ITaperDelegate](#).

15.31 AAX_CLogTaperDelegate.h File Reference

```
#include "AAX_ITaperDelegate.h"
#include "AAX_UtillsNative.h"
#include "AAX.h"
#include <cmath>
```

15.31.1 Description

A log taper delegate.

Classes

- class [AAX_CLogTaperDelegate< T, RealPrecision >](#)
A logarithmic taper conforming to [AAX_ITaperDelegate](#).

15.32 AAX_CMonolithicParameters.cpp File Reference

```
#include "AAX_CMonolithicParameters.h"
#include "AAX_Exception.h"
```

15.33 AAX_CMonolithicParameters.h File Reference

```
#include "AAX_CEffectParameters.h"
#include "AAX_IEffectDescriptor.h"
#include "AAX_IComponentDescriptor.h"
#include "AAX_IPropertyMap.h"
#include "AAX_CAtomicQueue.h"
#include "AAX_IParameter.h"
#include "AAX_IMIDINode.h"
#include "AAX_IString.h"
#include <set>
#include <list>
#include <utility>
```

15.33.1 Description

A convenience class extending [AAX_CEffectParameters](#) for monolithic instruments.

Classes

- struct [AAX_SInstrumentSetupInfo](#)
Information used to describe the instrument.
- struct [AAX_SInstrumentPrivateData](#)
Utility struct for [AAX_CMonolithicParameters](#).
- struct [AAX_SInstrumentRenderInfo](#)
Information used to parameterize [AAX_CMonolithicParameters::RenderAudio\(\)](#)
- class [AAX_CMonolithicParameters](#)
Extension of the [AAX_CEffectParameters](#) class for monolithic VIs and effects.

Macros

- #define [kMaxAdditionalMIDINodes](#) 15
- #define [kMaxAuxOutputStems](#) 32
- #define [kSynchronizedParameterQueueSize](#) 32

15.33.2 Macro Definition Documentation

15.33.2.1 kMaxAdditionalMIDINodes

```
#define kMaxAdditionalMIDINodes 15
```

15.33.2.2 kMaxAuxOutputStems

```
#define kMaxAuxOutputStems 32
```

15.33.2.3 kSynchronizedParameterQueueSize

```
#define kSynchronizedParameterQueueSize 32
```

15.34 AAX_CMutex.h File Reference

15.34.1 Description

Mutex.

Classes

- class [AAX_CMutex](#)
Mutex with try lock functionality.
- class [AAX_StLock_Guard](#)
Helper class for working with mutex.

15.35 AAX_CNumberDisplayDelegate.h File Reference

```
#include "AAX_IDisplayDelegate.h"  
#include "AAX_CString.h"
```

15.35.1 Description

A number display delegate.

Classes

- class [AAX_CNumberDisplayDelegate< T, Precision, SpaceAfter >](#)
A numeric display format conforming to [AAX_IDisplayDelegate](#).

15.36 AAX_CommonConversions.h File Reference

```
#include <math.h>
#include "AAX.h"
```

Functions

- double [GainToDB](#) (double aGain)
Convert Gain to dB.
- double [DBToGain](#) (double dB)
Convert dB to Gain.
- double [LongToDouble](#) (int32_t aLong)
Convert Long to Double.
- int32_t [DoubleToLong](#) (double aDouble)
convert floating point equivalent back to int32_t
- int32_t [DoubleToDSPCoef](#) (double d, double max=[k56kFloatPosMax](#), double min=[k56kFloatNegMax](#))
Convert Double to DSPCoef.
- double [DSPCoefToDouble](#) (int32_t c, int32_t max=[k56kFracPosMax](#), int32_t min=[k56kFracNegMax](#))
Convert DSPCoef to Double.
- double [ThirtyTwoBitDSPCoefToDouble](#) (int32_t c)
ThirtyTwoBitDSPCoefToDouble.
- int32_t [DoubleTo32BitDSPCoefRnd](#) (double d)
DoubleTo32BitDSPCoefRnd.
- int32_t [DoubleTo32BitDSPCoef](#) (double d)
- int32_t [DoubleToDSPCoefRnd](#) (double d, double max, double min)

Variables

- const int32_t [k32BitPosMax](#) = 0x7FFFFFFF
- const int32_t [k32BitAbsMax](#) = 0x80000000
- const int32_t [k32BitNegMax](#) = 0x80000000
- const int32_t [k56kFracPosMax](#) = 0x007FFFFF
- const int32_t [k56kFracAbsMax](#) = 0x00800000
- const int32_t [k56kFracHalf](#) = 0x00400000
- const int32_t [k56kFracNegOne](#) = 0xFF800000
- const int32_t [k56kFracNegMax](#) = [k56kFracNegOne](#)
- const int32_t [k56kFracZero](#) = 0x00000000
- const double [kOneOver56kFracAbsMax](#) = 1.0/double([k56kFracAbsMax](#))
- const double [k56kFloatPosMax](#) = double([k56kFracPosMax](#))/double([k56kFracAbsMax](#))
- const double [k56kFloatNegMax](#) = -1.0
- const double [kNeg144DB](#) = -144.0
- const double [kNeg144Gain](#) = 6.3095734448019324943436013662234e-8

15.36.1 Function Documentation

15.36.1.1 GainToDB()

```
double GainToDB (
    double aGain ) [inline]
```

Convert Gain to dB.

Todo This should be incorporated into parameters' tapers and not called separately

References kNeg144DB.

15.36.1.2 DBToGain()

```
double DBToGain (
    double dB ) [inline]
```

Convert dB to Gain.

Todo This should be incorporated into parameters' tapers and not called separately

15.36.1.3 LongToDouble()

```
double LongToDouble (
    int32_t aLong ) [inline]
```

Convert Long to Double.

LongToDouble: convert 24 bit fixed point in a int32_t to floating point equivalent

References k56kFracNegMax, k56kFracPosMax, and kOneOver56kFracAbsMax.

15.36.1.4 DoubleToLong()

```
int32_t DoubleToLong (
    double aDouble )
```

convert floating point equivalent back to int32_t

15.36.1.5 DoubleToDSPCoef()

```
int32_t DoubleToDSPCoef (
    double d,
    double max = k56kFloatPosMax,
    double min = k56kFloatNegMax ) [inline]
```

Convert Double to DSPCoef.

References k56kFracAbsMax, k56kFracNegMax, and k56kFracPosMax.

Referenced by DoubleTo32BitDSPCoefRnd().

Here is the caller graph for this function:

15.36.1.6 DSPCoefToDouble()

```
double DSPCoefToDouble (
    int32_t c,
    int32_t max = k56kFracPosMax,
    int32_t min = k56kFracNegMax ) [inline]
```

Convert DSPCoef to Double.

References k56kFracNegMax, k56kFracPosMax, and kOneOver56kFracAbsMax.

Referenced by ThirtyTwoBitDSPCoefToDouble().

Here is the caller graph for this function:

15.36.1.7 ThirtyTwoBitDSPCoefToDouble()

```
double ThirtyTwoBitDSPCoefToDouble (
    int32_t c ) [inline]
```

ThirtyTwoBitDSPCoefToDouble.

References DSPCoefToDouble(), k32BitNegMax, and k32BitPosMax.

Here is the call graph for this function:

15.36.1.8 DoubleTo32BitDSPCoefRnd()

```
int32_t DoubleTo32BitDSPCoefRnd (
    double d ) [inline]
```

DoubleTo32BitDSPCoefRnd.

References DoubleToDSPCoef(), k32BitNegMax, and k32BitPosMax.

Here is the call graph for this function:

15.36.1.9 DoubleTo32BitDSPCoef()

```
int32_t DoubleTo32BitDSPCoef (
    double d )
```

15.36.1.10 DoubleToDSPCoefRnd()

```
int32_t DoubleToDSPCoefRnd (
    double d,
    double max,
    double min )
```

15.36.2 Variable Documentation

15.36.2.1 k32BitPosMax

```
const int32_t k32BitPosMax = 0x7FFFFFFF
```

Referenced by DoubleTo32BitDSPCoefRnd(), and ThirtyTwoBitDSPCoefToDouble().

15.36.2.2 k32BitAbsMax

```
const int32_t k32BitAbsMax = 0x80000000
```

15.36.2.3 k32BitNegMax

```
const int32_t k32BitNegMax = 0x80000000
```

Referenced by DoubleTo32BitDSPCoefRnd(), and ThirtyTwoBitDSPCoefToDouble().

15.36.2.4 k56kFracPosMax

```
const int32_t k56kFracPosMax = 0x007FFFFF
```

Referenced by DoubleToDSPCoef(), DSPCoefToDouble(), and LongToDouble().

15.36.2.5 k56kFracAbsMax

```
const int32_t k56kFracAbsMax = 0x00800000
```

Referenced by DoubleToDSPCoef().

15.36.2.6 k56kFracHalf

```
const int32_t k56kFracHalf = 0x00400000
```

15.36.2.7 k56kFracNegOne

```
const int32_t k56kFracNegOne = 0xFF800000
```

15.36.2.8 k56kFracNegMax

```
const int32_t k56kFracNegMax = k56kFracNegOne
```

Referenced by DoubleToDSPCoef(), DSPCoefToDouble(), and LongToDouble().

15.36.2.9 k56kFracZero

```
const int32_t k56kFracZero = 0x00000000
```

15.36.2.10 kOneOver56kFracAbsMax

```
const double kOneOver56kFracAbsMax = 1.0/double(k56kFracAbsMax)
```

Referenced by DSPCoefToDouble(), and LongToDouble().

15.36.2.11 k56kFloatPosMax

```
const double k56kFloatPosMax = double(k56kFracPosMax)/double(k56kFracAbsMax)
```

15.36.2.12 k56kFloatNegMax

```
const double k56kFloatNegMax = -1.0
```

15.36.2.13 kNeg144DB

```
const double kNeg144DB = -144.0
```

Referenced by GainToDB().

15.36.2.14 kNeg144Gain

```
const double kNeg144Gain = 6.3095734448019324943436013662234e-8
```

15.37 AAX_CommonInterface_Algorithm.doxygen File Reference

15.38 AAX_CommonInterface_Communication.doxygen File Reference

15.39 AAX_CommonInterface_DataModel.doxygen File Reference

15.40 AAX_CommonInterface_Describe.doxygen File Reference

Functions

- [AAX_Result GetEffectDescriptions](#) ([AAX_ICollection](#) *inCollection)

The plug-in's static Description entrypoint.

15.41 AAX_CommonInterface_FormatSpecification.doxygen File Reference

15.42 AAX_CommonInterface_GUI.doxygen File Reference

15.43 AAX_Constants.h File Reference

```
#include <cmath>
```

15.43.1 Description

Signal processing constants.

Namespaces

- [AAX](#)

Macros

- `#define AAX_CONSTANTS_H`

Enumerations

- enum [AAX::ESampleRates](#) {
 [AAX::e44100SampleRate](#) = 44100 ,
 [AAX::e48000SampleRate](#) = 48000 ,
 [AAX::e88200SampleRate](#) = 88200 ,
 [AAX::e96000SampleRate](#) = 96000 ,
 [AAX::e176400SampleRate](#) = 176400 ,
 [AAX::e192000SampleRate](#) = 192000 }

Variables

- const int [AAX::cBigEndian](#) =0
- const int [AAX::cLittleEndian](#) =1
- const double [AAX::cPi](#) = 3.1415926535897932384626433832795
- const double [AAX::cTwoPi](#) = 6.2831853071795862319959269370884
- const double [AAX::cHalfPi](#) = 1.5707963267948965579989817342721
- const double [AAX::cQuarterPi](#) = 0.78539816339744827899949086713605
- const double [AAX::cRootTwo](#) = 1.4142135623730950488016887242097
- const double [AAX::cOneOverRootTwo](#) = 0.70710678118654752440084436210485
- const double [AAX::cPos3dB](#) =1.4142135623730950488016887242097
- const double [AAX::cNeg3dB](#) =0.70710678118654752440084436210485
- const double [AAX::cPos6dB](#) =2.0
- const double [AAX::cNeg6dB](#) =0.5
- const double [AAX::cNormalizeLongToAmplitudeOneHalf](#) = 0.00000000023283064365386962890625
- const double [AAX::cNormalizeLongToAmplitudeOne](#) = 1.0/double(1<<31)
- const double [AAX::cMilli](#) =0.001
- const double [AAX::cMicro](#) =0.001*0.001
- const double [AAX::cNano](#) =0.001*0.001*0.001
- const double [AAX::cPico](#) =0.001*0.001*0.001*0.001
- const double [AAX::cKilo](#) =1000.0
- const double [AAX::cMega](#) =1000.0*1000.0
- const double [AAX::cGiga](#) =1000.0*1000.0*1000.0

15.43.2 Macro Definition Documentation

15.43.2.1 AAX_CONSTANTS_H

```
#define AAX_CONSTANTS_H
```

15.44 AAX_CPacketDispatcher.h File Reference

```
#include "AAX.h"  
#include "AAX_IController.h"  
#include "AAX_CMutex.h"  
#include <string>  
#include <map>
```

15.44.1 Description

Helper classes related to posting AAX packets and handling parameter update events.

Classes

- class [AAX_CPacket](#)
Container for packet-related data.
- struct [AAX_IPacketHandler](#)
Callback container used by [AAX_CPacketDispatcher](#).
- class [AAX_CPacketHandler< TWorker >](#)
Callback container used by [AAX_CPacketDispatcher](#).
- class [AAX_CPacketDispatcher](#)
Helper class for managing AAX packet posting.

15.45 AAX_CParameter.h File Reference

```
#include "AAX_Assert.h"  
#include "AAX_IParameter.h"  
#include "AAX_ITaperDelegate.h"  
#include "AAX_IDisplayDelegate.h"  
#include "AAX_IAutomationDelegate.h"  
#include "AAX_CString.h"  
#include <cstring>  
#include <list>  
#include <map>
```

15.45.1 Description

Generic implementation of an [AAX_IParameter](#).

Classes

- class [AAX_CParameterValue< T >](#)
Concrete implementation of [AAX_IPParameterValue](#).
- class [AAX_CParameter< T >](#)
Generic implementation of an [AAX_IPParameter](#).
- class [AAX_CStatelessParameter](#)
A stateless parameter implementation.

15.46 AAX_CParameterManager.h File Reference

```
#include "AAX_CParameter.h"  
#include "AAX.h"  
#include <vector>  
#include <map>
```

15.46.1 Description

A container object for plug-in parameters.

Classes

- class [AAX_CParameterManager](#)
A container object for plug-in parameters.

15.47 AAX_CPercentDisplayDelegateDecorator.h File Reference

```
#include "AAX_IDisplayDelegateDecorator.h"  
#include <cmath>
```

15.47.1 Description

A percent display delegate decorator.

Classes

- class [AAX_CPercentDisplayDelegateDecorator< T >](#)
A percent decorator conforming to [AAX_IDisplayDelegateDecorator](#).

Macros

- `#define` [AAX_CPERCENTDISPLAYDELEGATEDECORATOR_H](#)

15.47.2 Macro Definition Documentation

15.47.2.1 AAX_CPERCENTDISPLAYDELEGATEDECORATOR_H

```
#define AAX_CPERCENTDISPLAYDELEGATEDECORATOR_H
```

15.48 AAX_CPieceWiseLinearTaperDelegate.h File Reference

```
#include "AAX_ITaperDelegate.h"  
#include "AAX.h"  
#include <cmath>
```

15.48.1 Description

A piece-wise linear taper delegate.

Classes

- class [AAX_CPieceWiseLinearTaperDelegate< T, RealPrecision >](#)
A piece-wise linear taper conforming to [AAX_ITaperDelegate](#).

15.49 AAX_CRangeTaperDelegate.h File Reference

```
#include "AAX_ITaperDelegate.h"  
#include "AAX.h"  
#include <cmath>  
#include <vector>
```

15.49.1 Description

A range taper delegate decorator.

Classes

- class [AAX_CRangeTaperDelegate< T, RealPrecision >](#)
A piecewise-linear taper conforming to [AAX_ITaperDelegate](#).

15.50 AAX_CSessionDocumentClient.h File Reference

```
#include "AAX_ISessionDocumentClient.h"  
#include <memory>
```

Classes

- class [AAX_CSessionDocumentClient](#)
Default implementation of the [AAX_ISessionDocumentClient](#) interface.

Macros

- #define [AAX_CSessionDocumentClient_H](#)

15.50.1 Macro Definition Documentation

15.50.1.1 AAX_CSessionDocumentClient_H

```
#define AAX_CSessionDocumentClient_H
```

15.51 AAX_CStateDisplayDelegate.h File Reference

```
#include "AAX_IDisplayDelegate.h"  
#include "AAX_CString.h"  
#include <vector>
```

15.51.1 Description

A state display delegate.

Classes

- class [AAX_CStateDisplayDelegate< T >](#)
A generic display format conforming to [AAX_IDisplayDelegate](#).

15.52 AAX_CStateTaperDelegate.h File Reference

```
#include "AAX_ITaperDelegate.h"  
#include "AAX.h"  
#include <cmath>
```

15.52.1 Description

A state taper delegate (similar to a linear taper delegate.)

Classes

- class [AAX_CStateTaperDelegate< T >](#)
A linear taper conforming to [AAX_ITaperDelegate](#).

15.53 AAX_CString.h File Reference

```
#include "AAX_IString.h"
#include "AAX.h"
#include <string>
#include <map>
```

15.53.1 Description

A generic AAX string class with similar functionality to `std::string`.

Classes

- class [AAX_CString](#)
A generic AAX string class with similar functionality to `std::string`
- class [AAX_CStringAbbreviations](#)
Helper class to store a collection of name abbreviations.

Macros

- `#define` [AAX_CSTRING_H](#)

Functions

- [AAX_CString operator+](#) ([AAX_CString](#) lhs, const [AAX_CString](#) &rhs)
- [AAX_CString operator+](#) ([AAX_CString](#) lhs, const char *rhs)
- [AAX_CString operator+](#) (const char *lhs, const [AAX_CString](#) &rhs)

15.53.2 Macro Definition Documentation

15.53.2.1 AAX_CSTRING_H

```
#define AAX_CSTRING_H
```

15.53.3 Function Documentation

15.53.3.1 operator+() [1/3]

```
AAX_CString operator+ (
    AAX_CString lhs,
    const AAX_CString & rhs ) [inline]
```

15.53.3.2 operator+() [2/3]

```
AAX_CString operator+ (
    AAX_CString lhs,
    const char * rhs ) [inline]
```

15.53.3.3 operator+() [3/3]

```
AAX_CString operator+ (
    const char * lhs,
    const AAX_CString & rhs ) [inline]
```

15.54 AAX_CStringDataBuffer.h File Reference

```
#include "AAX_IDataBuffer.h"
#include "AAX.h"
#include <string>
#include <limits>
#include <type_traits>
```

Classes

- class [AAX_CStringDataBufferOfType< T >](#)
A convenience class for string data buffers.
- class [AAX_CStringDataBuffer](#)
A convenience class for string data buffers.

Macros

- `#define` [AAX_CStringDataBuffer_H](#)

15.54.1 Macro Definition Documentation

15.54.1.1 AAX_CStringDataBuffer_H

```
#define AAX_CStringDataBuffer_H
```

15.55 AAX_CStringDisplayDelegate.h File Reference

```
#include "AAX_IDisplayDelegate.h"  
#include <sstream>  
#include <map>
```

15.55.1 Description

A string display delegate.

Classes

- class [AAX_CStringDisplayDelegate< T >](#)
A string, or list, display format conforming to [AAX_IDisplayDelegate](#).

15.56 AAX_CTask.h File Reference

```
#include "AAX_IACFTask.h"  
#include "CACFUnknown.h"
```

15.56.1 Description

A default implementation of the [AAX_IACFTask](#) interface.

Classes

- class [AAX_CTask](#)

Macros

- #define [AAX_CTask_H](#)

15.56.2 Macro Definition Documentation

15.56.2.1 AAX_CTask_H

```
#define AAX_CTask_H
```

15.57 AAX_CTaskAgent.h File Reference

```
#include "AAX_ITaskAgent.h"  
#include <memory>
```

15.57.1 Description

A default implementation of the [AAX_ITaskAgent](#) interface.

Classes

- class [AAX_CTaskAgent](#)
Default implementation of the [AAX_ITaskAgent](#) interface.

15.58 AAX_CUnitDisplayDelegateDecorator.h File Reference

```
#include "AAX_IDisplayDelegateDecorator.h"
```

15.58.1 Description

A unit display delgate decorator.

Classes

- class [AAX_CUnitDisplayDelegateDecorator< T >](#)
A unit type decorator conforming to [AAX_IDisplayDelegateDecorator](#).

15.59 AAX_CUnitPrefixDisplayDelegateDecorator.h File Reference

```
#include "AAX_IDisplayDelegateDecorator.h"
```

15.59.1 Description

A unit prefix display delegate decorator.

Classes

- class [AAX_CUnitPrefixDisplayDelegateDecorator< T >](#)
A unit prefix decorator conforming to [AAX_IDisplayDelegateDecorator](#).

15.60 AAX_Denormal.h File Reference

```
#include "AAX.h"
#include "AAX_PlatformOptimizationConstants.h"
#include <limits>
#include <math.h>
```

15.60.1 Description

Signal processing utilities for denormal/subnormal floating point numbers.

Namespaces

- [AAX](#)

Macros

- #define [AAX_DENORMAL_H](#)
- #define [AAX_SCOPE_COMPUTE_DENORMALS\(\)](#) do {} while(0)
Sets the run-time environment to handle denormal floats within the scope of this macro.
- #define [AAX_SCOPE_DENORMALS_AS_ZERO\(\)](#) do {} while(0)
Sets the run-time environment to treat denormal floats as zero within the scope of this macro.

Functions

- void [AAX::DeDenormal](#) (double &iValue)
Clamps very small floating point values to zero.
- void [AAX::DeDenormal](#) (float &iValue)
Clamps very small floating point values to zero.
- void [AAX::DeDenormalFine](#) (float &iValue)
- void [AAX::FilterDenormals](#) (float *inSamples, int32_t inLength)
Round all denormal/subnormal samples in a buffer to zero.

Variables

- const double `AAX::cDenormalAvoidanceOffset` =3.0e-34
- const float `AAX::cFloatDenormalAvoidanceOffset` =3.0e-20f

15.60.2 Macro Definition Documentation

15.60.2.1 AAX_DENORMAL_H

```
#define AAX_DENORMAL_H
```

15.60.2.2 AAX_SCOPE_COMPUTE_DENORMALS

```
#define AAX_SCOPE_COMPUTE_DENORMALS( ) do {} while(0)
```

Sets the run-time environment to handle denormal floats within the scope of this macro.

The host sets the denormal policy for all `AAX` threads and may use settings which treat denormal float values as zero (DAZ+FZ). This macro forces non-DAZ behavior.

15.60.2.3 AAX_SCOPE_DENORMALS_AS_ZERO

```
#define AAX_SCOPE_DENORMALS_AS_ZERO( ) do {} while(0)
```

Sets the run-time environment to treat denormal floats as zero within the scope of this macro.

The host sets the denormal policy for all `AAX` threads and may already use settings which treat denormal float values as zero (DAZ+FZ). This macro forces DAZ behavior.

15.61 AAX_DigiTrace_Guide.doxygen File Reference

15.62 AAX_DistributingYourPlugIn.doxygen File Reference

15.63 AAX_DocsDirectory.doxygen File Reference

15.64 AAX_EndianSwap.h File Reference

```
#include <algorithm>
```

15.64.1 Description

Utility functions for byte-swapping. Used by [AAX_CChunkDataParser](#).

Macros

- `#define` [ENDIANSWAP_H](#)

Functions

- `template<class T >`
`void` [AAX_EndianSwapInPlace](#) (`T *theDataP`)
Byte swap data in-place.
- `template<class T >`
`T` [AAX_EndianSwap](#) (`T theData`)
Make a byte-swapped copy of data.
- `template<class T >`
`void` [AAX_BigEndianNativeSwapInPlace](#) (`T *theDataP`)
Convert data in-place between Big Endian and native byte ordering.
- `template<class T >`
`T` [AAX_BigEndianNativeSwap](#) (`T theData`)
Copy and convert data between Big Endian and native byte ordering.
- `template<class T >`
`void` [AAX_LittleEndianNativeSwapInPlace](#) (`T *theDataP`)
Convert data in-place from the native byte ordering to Little Endian byte ordering.
- `template<class T >`
`T` [AAX_LittleEndianNativeSwap](#) (`T theData`)
Copy and convert data from the native byte ordering to Little Endian byte ordering.
- `template<class Iter >`
`void` [AAX_EndianSwapSequenceInPlace](#) (`Iter beginI, Iter endI`)
Byte swap a sequence of data in-place.
- `template<class Iter >`
`void` [AAX_BigEndianNativeSwapSequenceInPlace](#) (`Iter beginI, Iter endI`)
Convert an sequence of data in-place between Big Endian and native byte ordering.
- `template<class Iter >`
`void` [AAX_LittleEndianNativeSwapSequenceInPlace](#) (`Iter beginI, Iter endI`)
Convert an sequence of data in-place from the native byte ordering to Little Endian byte ordering.

15.64.2 Macro Definition Documentation

15.64.2.1 ENDIANSWAP_H

```
#define ENDIANSWAP_H
```

15.64.3 Function Documentation

15.64.3.1 AAX_EndianSwapInPlace()

```
template<class T >
void AAX_EndianSwapInPlace (
    T * theDataP ) [inline]
```

Byte swap data in-place.

Referenced by AAX_BigEndianNativeSwapInPlace(), AAX_EndianSwap(), AAX_EndianSwapSequenceInPlace(), and AAX_LittleEndianNativeSwapInPlace().

Here is the caller graph for this function:

15.64.3.2 AAX_EndianSwap()

```
template<class T >
T AAX_EndianSwap (
    T theData ) [inline]
```

Make a byte-swapped copy of data.

References AAX_EndianSwapInPlace().

Here is the call graph for this function:

15.64.3.3 AAX_BigEndianNativeSwapInPlace()

```
template<class T >
void AAX_BigEndianNativeSwapInPlace (
    T * theDataP ) [inline]
```

Convert data in-place between Big Endian and native byte ordering.

References AAX_EndianSwapInPlace().

Referenced by AAX_BigEndianNativeSwap().

Here is the call graph for this function: Here is the caller graph for this function:

15.64.3.4 AAX_BigEndianNativeSwap()

```
template<class T >
T AAX_BigEndianNativeSwap (
    T theData ) [inline]
```

Copy and convert data between Big Endian and native byte ordering.

References AAX_BigEndianNativeSwapInPlace().

Here is the call graph for this function:

15.64.3.5 AAX_LittleEndianNativeSwapInPlace()

```
template<class T >
void AAX_LittleEndianNativeSwapInPlace (
    T * theDataP ) [inline]
```

Convert data in-place from the native byte ordering to Little Endian byte ordering.

References AAX_EndianSwapInPlace().

Referenced by AAX_LittleEndianNativeSwap().

Here is the call graph for this function: Here is the caller graph for this function:

15.64.3.6 AAX_LittleEndianNativeSwap()

```
template<class T >
T AAX_LittleEndianNativeSwap (
    T theData ) [inline]
```

Copy and convert data from the native byte ordering to Little Endian byte ordering.

References AAX_LittleEndianNativeSwapInPlace().

Here is the call graph for this function:

15.64.3.7 AAX_EndianSwapSequenceInPlace()

```
template<class Iter >
void AAX_EndianSwapSequenceInPlace (
    Iter beginI,
    Iter endI ) [inline]
```

Byte swap a sequence of data in-place.

References AAX_EndianSwapInPlace().

Referenced by AAX_BigEndianNativeSwapSequenceInPlace(), and AAX_LittleEndianNativeSwapSequenceInPlace().

Here is the call graph for this function: Here is the caller graph for this function:

15.64.3.8 AAX_BigEndianNativeSwapSequenceInPlace()

```
template<class Iter >
void AAX_BigEndianNativeSwapSequenceInPlace (
    Iter beginI,
    Iter endI ) [inline]
```

Convert an sequence of data in-place between Big Endian and native byte ordering.

References AAX_EndianSwapSequenceInPlace().

Here is the call graph for this function:

15.64.3.9 AAX_LittleEndianNativeSwapSequenceInPlace()

```
template<class Iter >
void AAX_LittleEndianNativeSwapSequenceInPlace (
    Iter beginI,
    Iter endI ) [inline]
```

Convert an sequence of data in-place from the native byte ordering to Little Endian byte ordering.

References [AAX_EndianSwapSequenceInPlace\(\)](#).

Here is the call graph for this function:

15.65 AAX_Enums.h File Reference

```
#include <stdint.h>
```

15.65.1 Description

Utility functions for byte-swapping. Used by [AAX_CChunkDataParser](#).

Macros

- #define [AAX_INT32_MIN](#) (-2147483647 - 1) /** minimum signed 32 bit value */
- #define [AAX_INT32_MAX](#) 2147483647 /** maximum signed 32 bit value */
- #define [AAX_UINT32_MIN](#) 0U /** minimum unsigned 32 bit value */
- #define [AAX_UINT32_MAX](#) 4294967295U /** maximum unsigned 32 bit value */
- #define [AAX_INT16_MIN](#) (-32767 - 1) /** minimum signed 16 bit value */
- #define [AAX_INT16_MAX](#) 32767 /** maximum signed 16 bit value */
- #define [AAX_UINT16_MIN](#) 0U /** minimum unsigned 16 bit value */
- #define [AAX_UINT16_MAX](#) 65535U /** maximum unsigned 16 bit value */
- #define [AAX_ENUM_SIZE_CHECK](#)(x) extern int __enumSizeCheck[2*(sizeof(uint32_t)==sizeof(x)) - 1]
Macro to ensure enum type consistency across binaries.
- #define [AAX_STEM_FORMAT](#)(aIndex, aChannelCount) (static_cast<uint32_t>((static_cast<uint16_t>(aIndex) << 16) | (aChannelCount >= [AAX_UINT16_MIN](#)) && (aChannelCount <= 0xFFFF) ? aChannelCount & 0xFFFF : 0x0000))
- #define [AAX_STEM_FORMAT_CHANNEL_COUNT](#)(aStemFormat) (static_cast<uint16_t>(aStemFormat & 0xFFFF))
- #define [AAX_STEM_FORMAT_INDEX](#)(aStemFormat) (static_cast<int16_t>((aStemFormat >> 16) & 0xFFFF))

Typedefs

- typedef enum [AAX_EParameterType](#) [AAX_EParameterType](#)
FIC stuff that I can't include without DAE library dependence.
- typedef int32_t [AAX_EParameterOrientation](#)
Typedef for a bitfield of [AAX_EParameterOrientationBits](#) values.

Enumerations

- enum `AAX_EHighlightColor` {
`AAX_eHighlightColor_Red` = 0 ,
`AAX_eHighlightColor_Blue` = 1 ,
`AAX_eHighlightColor_Green` = 2 ,
`AAX_eHighlightColor_Yellow` = 3 ,
`AAX_eHighlightColor_Num` }
Highlight color selector.
- enum `AAX_ETracePriorityHost` {
`AAX_eTracePriorityHost_None` = 0 ,
`AAX_eTracePriorityHost_Critical` = 0x10000000 ,
`AAX_eTracePriorityHost_High` = 0x08000000 ,
`AAX_eTracePriorityHost_Normal` = 0x04000000 ,
`AAX_eTracePriorityHost_Low` = 0x02000000 ,
`AAX_eTracePriorityHost_Lowest` = 0x01000000 }
Platform-specific tracing priorities.
- enum `AAX_ETracePriorityDSP` {
`AAX_eTracePriorityDSP_None` = 0 ,
`AAX_eTracePriorityDSP_Assert` = 1 ,
`AAX_eTracePriorityDSP_High` = 2 ,
`AAX_eTracePriorityDSP_Normal` = 3 ,
`AAX_eTracePriorityDSP_Low` = 4 }
Platform-specific tracing priorities.
- enum `AAX_EModifiers` {
`AAX_eModifiers_None` = 0 ,
`AAX_eModifiers_Shift` = (1 << 0) ,
`AAX_eModifiers_Control` = (1 << 1) ,
`AAX_eModifiers_Option` = (1 << 2) ,
`AAX_eModifiers_Command` = (1 << 3) ,
`AAX_eModifiers_SecondaryButton` = (1 << 4) ,
`AAX_eModifiers_Alt` = `AAX_eModifiers_Option` ,
`AAX_eModifiers_Cntl` = `AAX_eModifiers_Command` ,
`AAX_eModifiers_WINKEY` = `AAX_eModifiers_Control` }
Modifier key definitions used by AAX API.
- enum `AAX_EAudioBufferLength` {
`AAX_eAudioBufferLength_Undefined` = -1 ,
`AAX_eAudioBufferLength_1` = 0 ,
`AAX_eAudioBufferLength_2` = 1 ,
`AAX_eAudioBufferLength_4` = 2 ,
`AAX_eAudioBufferLength_8` = 3 ,
`AAX_eAudioBufferLength_16` = 4 ,
`AAX_eAudioBufferLength_32` = 5 ,
`AAX_eAudioBufferLength_64` = 6 ,
`AAX_eAudioBufferLength_128` = 7 ,
`AAX_eAudioBufferLength_256` = 8 ,
`AAX_eAudioBufferLength_512` = 9 ,
`AAX_eAudioBufferLength_1024` = 10 ,
`AAX_eAudioBufferLength_Max` = `AAX_eAudioBufferLength_1024` }
Generic buffer length definitions.
- enum `AAX_EAudioBufferLengthDSP` {
`AAX_eAudioBufferLengthDSP_Default` = `AAX_eAudioBufferLength_4` ,
`AAX_eAudioBufferLengthDSP_4` = `AAX_eAudioBufferLength_4` ,
`AAX_eAudioBufferLengthDSP_16` = `AAX_eAudioBufferLength_16` ,
`AAX_eAudioBufferLengthDSP_32` = `AAX_eAudioBufferLength_32` ,
`AAX_eAudioBufferLengthDSP_64` = `AAX_eAudioBufferLength_64` ,
`AAX_eAudioBufferLengthDSP_Max` = `AAX_eAudioBufferLengthDSP_64` }

Currently supported processing buffer length definitions for AAX DSP hosts.

- enum [AAE_EAudioBufferLengthNative](#) {
[AAX_eAudioBufferLengthNative_Min](#) = AAX_eAudioBufferLength_32 ,
[AAX_eAudioBufferLengthNative_Max](#) = AAX_eAudioBufferLength_Max }

Processing buffer length definitions for Native AAX hosts.

- enum [AAX_EMaxAudioSuiteTracks](#) { [AAX_eMaxAudioSuiteTracks](#) = 48 }

The maximum number of tracks that an AAX host will process in a non-real-time context.

- enum [AAX_EStemFormat](#) {
[AAX_eStemFormat_Mono](#) = AAX_STEM_FORMAT (0, 1) ,
[AAX_eStemFormat_Stereo](#) = AAX_STEM_FORMAT (1, 2) ,
[AAX_eStemFormat_LCR](#) = AAX_STEM_FORMAT (2, 3) ,
[AAX_eStemFormat_LCRS](#) = AAX_STEM_FORMAT (3, 4) ,
[AAX_eStemFormat_Quad](#) = AAX_STEM_FORMAT (4, 4) ,
[AAX_eStemFormat_5_0](#) = AAX_STEM_FORMAT (5, 5) ,
[AAX_eStemFormat_5_1](#) = AAX_STEM_FORMAT (6, 6) ,
[AAX_eStemFormat_6_0](#) = AAX_STEM_FORMAT (7, 6) ,
[AAX_eStemFormat_6_1](#) = AAX_STEM_FORMAT (8, 7) ,
[AAX_eStemFormat_7_0_SDDS](#) = AAX_STEM_FORMAT (9, 7) ,
[AAX_eStemFormat_7_1_SDDS](#) = AAX_STEM_FORMAT (10, 8) ,
[AAX_eStemFormat_7_0_DTS](#) = AAX_STEM_FORMAT (11, 7) ,
[AAX_eStemFormat_7_1_DTS](#) = AAX_STEM_FORMAT (12, 8) ,
[AAX_eStemFormat_7_0_2](#) = AAX_STEM_FORMAT (20, 9) ,
[AAX_eStemFormat_7_1_2](#) = AAX_STEM_FORMAT (13, 10) ,
[AAX_eStemFormat_5_0_2](#) = AAX_STEM_FORMAT (21, 7) ,
[AAX_eStemFormat_5_1_2](#) = AAX_STEM_FORMAT (22, 8) ,
[AAX_eStemFormat_5_0_4](#) = AAX_STEM_FORMAT (23, 9) ,
[AAX_eStemFormat_5_1_4](#) = AAX_STEM_FORMAT (24, 10) ,
[AAX_eStemFormat_7_0_4](#) = AAX_STEM_FORMAT (25, 11) ,
[AAX_eStemFormat_7_1_4](#) = AAX_STEM_FORMAT (26, 12) ,
[AAX_eStemFormat_7_0_6](#) = AAX_STEM_FORMAT (35, 13) ,
[AAX_eStemFormat_7_1_6](#) = AAX_STEM_FORMAT (36, 14) ,
[AAX_eStemFormat_9_0_4](#) = AAX_STEM_FORMAT (27, 13) ,
[AAX_eStemFormat_9_1_4](#) = AAX_STEM_FORMAT (28, 14) ,
[AAX_eStemFormat_9_0_6](#) = AAX_STEM_FORMAT (29, 15) ,
[AAX_eStemFormat_9_1_6](#) = AAX_STEM_FORMAT (30, 16) ,
[AAX_eStemFormat_Ambi_1_ACN](#) = AAX_STEM_FORMAT (14, 4) ,
[AAX_eStemFormat_Ambi_2_ACN](#) = AAX_STEM_FORMAT (18, 9) ,
[AAX_eStemFormat_Ambi_3_ACN](#) = AAX_STEM_FORMAT (19, 16) ,
[AAX_eStemFormat_Ambi_4_ACN](#) = AAX_STEM_FORMAT (31, 25) ,
[AAX_eStemFormat_Ambi_5_ACN](#) = AAX_STEM_FORMAT (32, 36) ,
[AAX_eStemFormat_Ambi_6_ACN](#) = AAX_STEM_FORMAT (33, 49) ,
[AAX_eStemFormat_Ambi_7_ACN](#) = AAX_STEM_FORMAT (34, 64) ,
[AAX_eStemFormatNum](#) = 37 ,
[AAX_eStemFormat_None](#) = AAX_STEM_FORMAT (-100, 0) ,
[AAX_eStemFormat_Any](#) = AAX_STEM_FORMAT (-1, 0) ,
[AAX_eStemFormat_INT32_MAX](#) = AAX_INT32_MAX }

Stem format definitions.

- enum [AAX_EPlugInCategory](#) {
[AAX_ePlugInCategory_None](#) = 0x00000000 ,
[AAX_ePlugInCategory_EQ](#) = 0x00000001 ,
[AAX_ePlugInCategory_Dynamics](#) = 0x00000002 ,
[AAX_ePlugInCategory_PitchShift](#) = 0x00000004 ,
[AAX_ePlugInCategory_Reverb](#) = 0x00000008 ,
[AAX_ePlugInCategory_Delay](#) = 0x00000010 ,
[AAX_ePlugInCategory_Modulation](#) = 0x00000020 ,
[AAX_ePlugInCategory_Harmonic](#) = 0x00000040 ,
[AAX_ePlugInCategory_NoiseReduction](#) = 0x00000080 ,

```

AAX_ePlugInCategory_Dither = 0x00000100 ,
AAX_ePlugInCategory_SoundField = 0x00000200 ,
AAX_ePlugInCategory_HWGenerators = 0x00000400 ,
AAX_ePlugInCategory_SWGenerators = 0x00000800 ,
AAX_ePlugInCategory_WrappedPlugin = 0x00001000 ,
AAX_EPlugInCategory_Effect = 0x00002000 ,
AAX_ePlugInCategory_Example = AAX_EPlugInCategory_Effect ,
AAX_EPlugInCategory_MIDIEffect = 0x00010000 ,
AAX_ePlugInCategory_INT32_MAX = AAX_INT32_MAX }

```

Effect category definitions.

- enum `AAX_EPlugInStrings` {
 - `AAX_ePlugInStrings_Analysis` = 0 ,
 - `AAX_ePlugInStrings_MonoMode` = 1 ,
 - `AAX_ePlugInStrings_MultiInputMode` = 2 ,
 - `AAX_ePlugInStrings_RegionByRegionAnalysis` = 3 ,
 - `AAX_ePlugInStrings_AllSelectedRegionsAnalysis` = 4 ,
 - `AAX_ePlugInStrings_RegionName` = 5 ,
 - `AAX_ePlugInStrings_ClipName` = 5 ,
 - `AAX_ePlugInStrings_Progress` = 6 ,
 - `AAX_ePlugInStrings_PluginFileName` = 7 ,
 - `AAX_ePlugInStrings_Preview` = 8 ,
 - `AAX_ePlugInStrings_Process` = 9 ,
 - `AAX_ePlugInStrings_Bypass` = 10 ,
 - `AAX_ePlugInStrings_ClipNameSuffix` = 11 ,
 - `AAX_ePlugInStrings_INT32_MAX` = AAX_INT32_MAX }

Effect string identifiers.

- enum `AAX_EMeterOrientation` {
 - `AAX_eMeterOrientation_Default` = 0 ,
 - `AAX_eMeterOrientation_BottomLeft` = AAX_eMeterOrientation_Default ,
 - `AAX_eMeterOrientation_TopRight` = 1 ,
 - `AAX_eMeterOrientation_Center` = 2 ,
 - `AAX_eMeterOrientation_PhaseDot` = 3 }

Meter orientation.

- enum `AAX_EMeterBallisticType` {
 - `AAX_eMeterBallisticType_Host` = 0 ,
 - `AAX_eMeterBallisticType_NoDecay` = 1 }

Meter ballistics type.

- enum `AAX_EMeterType` {
 - `AAX_eMeterType_Input` = 0 ,
 - `AAX_eMeterType_Output` = 1 ,
 - `AAX_eMeterType_CLGain` = 2 ,
 - `AAX_eMeterType_EGGain` = 3 ,
 - `AAX_eMeterType_Analysis` = 4 ,
 - `AAX_eMeterType_Other` = 5 ,
 - `AAX_eMeterType_None` = 31 }

Meter type.

- enum `AAX_ECurveType` {
 - `AAX_eCurveType_None` = 0 ,
 - `AAX_eCurveType_EQ` = 'AXeq' ,
 - `AAX_eCurveType_Dynamics` = 'AXdy' ,
 - `AAX_eCurveType_Reduction` = 'AXdr' }

Different Curve Types that can be queried from the Host.

- enum `AAX_EResourceType` {
 - `AAX_eResourceType_None` = 0 ,
 - `AAX_eResourceType_PageTable` ,
 - `AAX_eResourceType_PageTableDir` }

Types of resources that can be added to an Effect's description.

- enum `AAX_ENotificationEvent` {
`AAX_eNotificationEvent_InsertPositionChanged` = 'AXip' ,
`AAX_eNotificationEvent_TrackNameChanged` = 'AXtn' ,
`AAX_eNotificationEvent_TrackUIDChanged` = 'AXtu' ,
`AAX_eNotificationEvent_TrackPositionChanged` = 'AXtp' ,
`AAX_eNotificationEvent_AlgorithmMoved` = 'AXam' ,
`AAX_eNotificationEvent_GUIOpened` = 'AXgo' ,
`AAX_eNotificationEvent_GUIClosed` = 'AXgc' ,
`AAX_eNotificationEvent_ASProcessingState` = 'AXPr' ,
`AAX_eNotificationEvent_ASPreviewState` = 'ASpv' ,
`AAX_eNotificationEvent_SessionBeingOpened` = 'AXso' ,
`AAX_eNotificationEvent_PresetOpened` = 'AXpo' ,
`AAX_eNotificationEvent_EnteringOfflineMode` = 'AXof' ,
`AAX_eNotificationEvent_ExitingOfflineMode` = 'AXox' ,
`AAX_eNotificationEvent_SessionPathChanged` = 'AXsp' ,
`AAX_eNotificationEvent_SignalLatencyChanged` = 'AXsl' ,
`AAX_eNotificationEvent_DelayCompensationState` = 'AXdc' ,
`AAX_eNotificationEvent_CycleCountChanged` = 'AXcc' ,
`AAX_eNotificationEvent_MaxViewSizeChanged` = 'AXws' ,
`AAX_eNotificationEvent_SideChainBeingConnected` = 'AXsc' ,
`AAX_eNotificationEvent_SideChainBeingDisconnected` = 'AXsd' ,
`AAX_eNotificationEvent_NoiseFloorChanged` = 'AXnf' ,
`AAX_eNotificationEvent_ParameterMappingChanged` = 'AXpm' ,
`AAX_eNotificationEvent_ParameterNameChanged` = 'AXpn' ,
`AAX_eNotificationEvent_HostModeChanged` = 'AXHm' ,
`AAX_eNotificationEvent_PriorSettingsInvalid` = 'AXps' ,
`AAX_eNotificationEvent_LogState` = 'AXls' ,
`AAX_eNotificationEvent_TransportStateChanged` = 'AXts' ,
`AAX_eNotificationEvent_HostLocale` = 'AXLc' }

Events IDs for AAX notifications.

- enum `AAX_EHostModeBits` {
`AAX_eHostModeBits_None` = 0 ,
`AAX_eHostModeBits_Live` = (1 << 0) }

Host mode.

- enum `AAX_EHostMode` {
`AAX_eHostMode_Show` = `AAX_eHostModeBits_Live` ,
`AAX_eHostMode_Config` = `AAX_eHostModeBits_None` }

DEPRECATED.

- enum `AAX_EPrivateDataOptions` {
`AAX_ePrivateDataOptions_DefaultOptions` = 0 ,
`AAX_ePrivateDataOptions_KeepOnReset` = (1 << 0) ,
`AAX_ePrivateDataOptions_External` = (1 << 1) ,
`AAX_ePrivateDataOptions_Align8` = (1 << 2) ,
`AAX_ePrivateDataOptions_INT32_MAX` = `AAX_INT32_MAX` }

Options for algorithm private data fields.

- enum `AAX_EConstraintLocationMask` {
`AAX_eConstraintLocationMask_None` = 0 ,
`AAX_eConstraintLocationMask_DataModel` = (1 << 0) ,
`AAX_eConstraintLocationMask_DLLChipAffinity` = (1 << 1) }

Property values to describe location constraints placed on the plug-in's algorithm component (ProcessProc)

- enum `AAX_EConstraintTopology` {
`AAX_eConstraintTopology_None` = 0 ,
`AAX_eConstraintTopology_Monolithic` = 1 }

Property values to describe the topology of the plug-in's modules (e.g. data model, GUI.)

- enum `AAX_EComponentInstanceInitAction` {
`AAX_eComponentInstanceInitAction_AddingNewInstance` = 0 ,
`AAX_eComponentInstanceInitAction_RemovingInstance` = 1 ,
`AAX_eComponentInstanceInitAction_ResetInstance` = 2 }

Selector indicating the action that occurred to prompt a component initialization callback.

- enum `AAX_ESampleRateMask` {
`AAX_eSampleRateMask_No` = 0 ,
`AAX_eSampleRateMask_44100` = (1 << 0) ,
`AAX_eSampleRateMask_48000` = (1 << 1) ,
`AAX_eSampleRateMask_88200` = (1 << 2) ,
`AAX_eSampleRateMask_96000` = (1 << 3) ,
`AAX_eSampleRateMask_176400` = (1 << 4) ,
`AAX_eSampleRateMask_192000` = (1 << 5) ,
`AAX_eSampleRateMask_All` = `AAX_INT32_MAX` }

Property values to describe various sample rates.

- enum `AAX_EParameterType` {
`AAX_eParameterType_Discrete` ,
`AAX_eParameterType_Continuous` }

FIC stuff that I can't include without DAE library dependence.

- enum `AAX_EParameterOrientationBits` {
`AAX_eParameterOrientation_Default` = 0 ,
`AAX_eParameterOrientation_BottomMinTopMax` = 0 ,
`AAX_eParameterOrientation_TopMinBottomMax` = 1 ,
`AAX_eParameterOrientation_LeftMinRightMax` = 0 ,
`AAX_eParameterOrientation_RightMinLeftMax` = 2 ,
`AAX_eParameterOrientation_RotarySingleDotMode` = 0 ,
`AAX_eParameterOrientation_RotaryBoostCutMode` = 4 ,
`AAX_eParameterOrientation_RotaryWrapMode` = 8 ,
`AAX_eParameterOrientation_RotarySpreadMode` = 12 ,
`AAX_eParameterOrientation_RotaryLeftMinRightMax` = 0 ,
`AAX_eParameterOrientation_RotaryRightMinLeftMax` = 16 }

Visual Orientation of a parameter.

- enum `AAX_EParameterValueInfoSelector` {
`AAX_ePageTable_EQ_Band_Type` = 0 ,
`AAX_ePageTable_EQ_InCircuitPolarity` = 1 ,
`AAX_ePageTable_UseAlternateControl` = 2 }

Query type selectors for use with `AAX_IEffectParameters::GetParameterValueInfo()`

- enum `AAX_EEQBandTypes` {
`AAX_eEQBandType_HighPass` = 0 ,
`AAX_eEQBandType_LowShelf` = 1 ,
`AAX_eEQBandType_Parametric` = 2 ,
`AAX_eEQBandType_HighShelf` = 3 ,
`AAX_eEQBandType_LowPass` = 4 ,
`AAX_eEQBandType_Notch` = 5 }

Definitions of band types for EQ page table.

- enum `AAX_EEQInCircuitPolarity` {
`AAX_eEQInCircuitPolarity_Enabled` = 0 ,
`AAX_eEQInCircuitPolarity_Bypassed` = 1 ,
`AAX_eEQInCircuitPolarity_Disabled` = 2 }

Definitions for band in/out for EQ page table.

- enum `AAX_EUseAlternateControl` {
`AAX_eUseAlternateControl_No` = 0 ,
`AAX_eUseAlternateControl_Yes` = 1 }

Definitions for Use Alternate Control parameter.

- enum `AAX_EMIDINodeType` {
`AAX_eMIDINodeType_LocalInput` = 0 ,

```
AAX_eMIDINodeType_LocalOutput = 1 ,
AAX_eMIDINodeType_Global = 2 ,
AAX_eMIDINodeType_Transport = 3 }
```

MIDI node types.

- enum [AAX_EUpdateSource](#) {
[AAX_eUpdateSource_Unspecified](#) = 0 ,
[AAX_eUpdateSource_Parameter](#) = 1 ,
[AAX_eUpdateSource_Chunk](#) = 2 ,
[AAX_eUpdateSource_Delay](#) = 3 }

Source for values passed into [UpdateParameterNormalizedValue\(\)](#).

- enum [AAX_EDataInPortType](#) {
[AAX_eDataInPortType_Unbuffered](#) = 0 ,
[AAX_eDataInPortType_Buffered](#) = 1 ,
[AAX_eDataInPortType_Incremental](#) = 2 }

Property value for whether a data in port should be buffered or not.

- enum [AAX_EFrameRate](#) {
[AAX_eFrameRate_Undeclared](#) = 0 ,
[AAX_eFrameRate_24Frame](#) = 1 ,
[AAX_eFrameRate_25Frame](#) = 2 ,
[AAX_eFrameRate_2997NonDrop](#) = 3 ,
[AAX_eFrameRate_2997DropFrame](#) = 4 ,
[AAX_eFrameRate_30NonDrop](#) = 5 ,
[AAX_eFrameRate_30DropFrame](#) = 6 ,
[AAX_eFrameRate_23976](#) = 7 ,
[AAX_eFrameRate_47952](#) = 8 ,
[AAX_eFrameRate_48Frame](#) = 9 ,
[AAX_eFrameRate_50Frame](#) = 10 ,
[AAX_eFrameRate_5994NonDrop](#) = 11 ,
[AAX_eFrameRate_5994DropFrame](#) = 12 ,
[AAX_eFrameRate_60NonDrop](#) = 13 ,
[AAX_eFrameRate_60DropFrame](#) = 14 ,
[AAX_eFrameRate_100Frame](#) = 15 ,
[AAX_eFrameRate_11988NonDrop](#) = 16 ,
[AAX_eFrameRate_11988DropFrame](#) = 17 ,
[AAX_eFrameRate_120NonDrop](#) = 18 ,
[AAX_eFrameRate_120DropFrame](#) = 19 }

FrameRate types.

- enum [AAX_EFeetFramesRate](#) {
[AAX_eFeetFramesRate_23976](#) = 0 ,
[AAX_eFeetFramesRate_24](#) = 1 ,
[AAX_eFeetFramesRate_25](#) = 2 }

FeetFramesRate types.

- enum [AAX_EMidiGlobalNodeSelectors](#) {
[AAX_eMIDIClick](#) = 1 << 0 ,
[AAX_eMIDIMtc](#) = 1 << 1 ,
[AAX_eMIDIBeatClock](#) = 1 << 2 }

The Global MIDI Node Selectors.

- enum [AAX_EPreviewState](#) {
[AAX_ePreviewState_Stop](#) = 0 ,
[AAX_ePreviewState_Start](#) = 1 }

Offline preview states for use with [AAX_eNotificationEvent_ASPreviewState](#).

- enum [AAX_EProcessingState](#) {
[AAX_eProcessingState_StopPass](#) = 2 ,
[AAX_eProcessingState_StartPass](#) = 3 ,
[AAX_eProcessingState_EndPassGroup](#) = 4 ,
[AAX_eProcessingState_BeginPassGroup](#) = 5 ,

```
AAX_eProcessingState_Stop = AAX_eProcessingState_StopPass ,
AAX_eProcessingState_Start = AAX_eProcessingState_StartPass }
```

Offline preview states for use with [AAX_eNotificationEvent_ASProcessingState](#).

- enum [AAX_ETargetPlatform](#) {
[kAAX_eTargetPlatform_None](#) = 0 ,
[kAAX_eTargetPlatform_Native](#) = 1 ,
[kAAX_eTargetPlatform_TI](#) = 2 ,
[kAAX_eTargetPlatform_External](#) = 3 ,
[kAAX_eTargetPlatform_Count](#) = 5 }

Describes what platform the component runs on.

- enum [AAX_ESupportLevel](#) {
[AAX_eSupportLevel_Uninitialized](#) = 0 ,
[AAX_eSupportLevel_Unsupported](#) = 1 ,
[AAX_eSupportLevel_Supported](#) = 2 ,
[AAX_eSupportLevel_Disabled](#) = 3 ,
[AAX_eSupportLevel_ByProperty](#) = 4 }
- enum [AAX_EHostLevel](#) {
[AAX_eHostLevel_Unknown](#) = 0 ,
[AAX_eHostLevel_Standard](#) = 1 ,
[AAX_eHostLevel_Entry](#) = 2 ,
[AAX_eHostLevel_Intermediate](#) = 3 }

Host levels.

- enum [AAX_ETextEncoding](#) {
[AAX_eTextEncoding_Undefined](#) = -1 ,
[AAX_eTextEncoding_UTF8](#) = 0 ,
[AAX_eTextEncoding_Num](#) }

Describes possible string encodings.

- enum [AAX_EAssertFlags](#) {
[AAX_eAssertFlags_Default](#) = 0 ,
[AAX_eAssertFlags_Log](#) = 1 << 0 ,
[AAX_eAssertFlags_Dialog](#) = 1 << 1 }

Flags for use with [AAX_IHostServices::HandleAssertFailure\(\)](#)

- enum [AAX_ETransportState](#) {
[AAX_eTransportState_Unknown](#) = 0 ,
[AAX_eTransportState_Stopping](#) = 1 ,
[AAX_eTransportState_Stop](#) = 2 ,
[AAX_eTransportState_Paused](#) = 3 ,
[AAX_eTransportState_Play](#) = 4 ,
[AAX_eTransportState_FastForward](#) = 5 ,
[AAX_eTransportState_Rewind](#) = 6 ,
[AAX_eTransportState_Scrub](#) = 11 ,
[AAX_eTransportState_Shuttle](#) = 12 ,
[AAX_eTransportState_Num](#) }

Used to indicate the current transport state of the host. This is the global transport state; it does not indicate a track-specific state.

- enum [AAX_ERecordMode](#) {
[AAX_eRecordMode_Unknown](#) = 0 ,
[AAX_eRecordMode_None](#) = 1 ,
[AAX_eRecordMode_Normal](#) = 2 ,
[AAX_eRecordMode_Destructive](#) = 3 ,
[AAX_eRecordMode_QuickPunch](#) = 4 ,
[AAX_eRecordMode_TrackPunch](#) = 5 ,
[AAX_eRecordMode_Num](#) }

Used to indicate the current record mode of the host. This is the global record mode; it does not indicate a track-specific state.

Functions

- [AAX_ENUM_SIZE_CHECK \(AAX_EHighlightColor\)](#)
- [AAX_ENUM_SIZE_CHECK \(AAX_ETracePriorityHost\)](#)
- [AAX_ENUM_SIZE_CHECK \(AAX_ETracePriorityDSP\)](#)
- [AAX_ENUM_SIZE_CHECK \(AAX_EModifiers\)](#)
- [AAX_ENUM_SIZE_CHECK \(AAX_EAudioBufferLength\)](#)
- [AAX_ENUM_SIZE_CHECK \(AAX_EAudioBufferLengthDSP\)](#)
- [AAX_ENUM_SIZE_CHECK \(AAE_EAudioBufferLengthNative\)](#)
- [AAX_ENUM_SIZE_CHECK \(AAX_EMaxAudioSuiteTracks\)](#)
- [AAX_ENUM_SIZE_CHECK \(AAX_EStemFormat\)](#)
- [AAX_ENUM_SIZE_CHECK \(AAX_EPlugInCategory\)](#)
- [AAX_ENUM_SIZE_CHECK \(AAX_EPlugInStrings\)](#)
- [AAX_ENUM_SIZE_CHECK \(AAX_EMeterOrientation\)](#)
- [AAX_ENUM_SIZE_CHECK \(AAX_EMeterBallisticType\)](#)
- [AAX_ENUM_SIZE_CHECK \(AAX_EMeterType\)](#)
- [AAX_ENUM_SIZE_CHECK \(AAX_ECurveType\)](#)
- [AAX_ENUM_SIZE_CHECK \(AAX_EResourceType\)](#)
- [AAX_ENUM_SIZE_CHECK \(AAX_ENotificationEvent\)](#)
- [AAX_ENUM_SIZE_CHECK \(AAX_EHostModeBits\)](#)
- [AAX_ENUM_SIZE_CHECK \(AAX_EHostMode\)](#)
- [AAX_ENUM_SIZE_CHECK \(AAX_EPrivateDataOptions\)](#)
- [AAX_ENUM_SIZE_CHECK \(AAX_EConstraintLocationMask\)](#)
- [AAX_ENUM_SIZE_CHECK \(AAX_EConstraintTopology\)](#)
- [AAX_ENUM_SIZE_CHECK \(AAX_EComponentInstanceInitAction\)](#)
- [AAX_ENUM_SIZE_CHECK \(AAX_ESampleRateMask\)](#)
- `uint8_t sampleRateInMask (float inSR, uint32_t iMask)`
Determines whether a particular [AAX_CSampleRate](#) is present in a given mask of [AAX_ESampleRateMask](#).
- `float getLowestSampleRateInMask (uint32_t iMask)`
Converts from a mask of [AAX_ESampleRateMask](#) to the lowest supported [AAX_CSampleRate](#) value in Hz.
- `uint32_t getMaskForSampleRate (float inSR)`
Returns the [AAX_ESampleRateMask](#) selector for a literal sample rate.
- [AAX_ENUM_SIZE_CHECK \(AAX_EParameterType\)](#)
- [AAX_ENUM_SIZE_CHECK \(AAX_EParameterOrientationBits\)](#)
- [AAX_ENUM_SIZE_CHECK \(AAX_EParameterValueInfoSelector\)](#)
- [AAX_ENUM_SIZE_CHECK \(AAX_EEQBandTypes\)](#)
- [AAX_ENUM_SIZE_CHECK \(AAX_EEQInCircuitPolarity\)](#)
- [AAX_ENUM_SIZE_CHECK \(AAX_EUseAlternateControl\)](#)
- [AAX_ENUM_SIZE_CHECK \(AAX_EMIDINodeType\)](#)
- [AAX_ENUM_SIZE_CHECK \(AAX_EUpdateSource\)](#)
- [AAX_ENUM_SIZE_CHECK \(AAX_EDataInPortType\)](#)
- [AAX_ENUM_SIZE_CHECK \(AAX_EFrameRate\)](#)
- [AAX_ENUM_SIZE_CHECK \(AAX_EFeetFramesRate\)](#)
- [AAX_ENUM_SIZE_CHECK \(AAX_EMidiGlobalNodeSelectors\)](#)
- [AAX_ENUM_SIZE_CHECK \(AAX_EPreviewState\)](#)
- [AAX_ENUM_SIZE_CHECK \(AAX_EProcessingState\)](#)
- [AAX_ENUM_SIZE_CHECK \(AAX_ETargetPlatform\)](#)
- [AAX_ENUM_SIZE_CHECK \(AAX_ESupportLevel\)](#)
- [AAX_ENUM_SIZE_CHECK \(AAX_EHostLevel\)](#)
- [AAX_ENUM_SIZE_CHECK \(AAX_ETextEncoding\)](#)
- [AAX_ENUM_SIZE_CHECK \(AAX_EAssertFlags\)](#)
- [AAX_ENUM_SIZE_CHECK \(AAX_ETransportState\)](#)
- [AAX_ENUM_SIZE_CHECK \(AAX_ERecordMode\)](#)

15.65.2 Macro Definition Documentation

15.65.2.1 AAX_INT32_MIN

```
#define AAX_INT32_MIN (-2147483647 - 1) /** minimum signed 32 bit value */
```

15.65.2.2 AAX_INT32_MAX

```
#define AAX_INT32_MAX 2147483647 /** maximum signed 32 bit value */
```

15.65.2.3 AAX_UINT32_MIN

```
#define AAX_UINT32_MIN 0U /** minimum unsigned 32 bit value */
```

15.65.2.4 AAX_UINT32_MAX

```
#define AAX_UINT32_MAX 4294967295U /** maximum unsigned 32 bit value */
```

15.65.2.5 AAX_INT16_MIN

```
#define AAX_INT16_MIN (-32767 - 1) /** minimum signed 16 bit value */
```

15.65.2.6 AAX_INT16_MAX

```
#define AAX_INT16_MAX 32767 /** maximum signed 16 bit value */
```

15.65.2.7 AAX_UINT16_MIN

```
#define AAX_UINT16_MIN 0U /** minimum unsigned 16 bit value */
```

15.65.2.8 AAX_UINT16_MAX

```
#define AAX_UINT16_MAX 65535U /** maximum unsigned 16 bit value */
```

15.65.2.9 AAX_ENUM_SIZE_CHECK

```
#define AAX_ENUM_SIZE_CHECK(
    x ) extern int __enumSizeCheck[ 2*(sizeof(uint32_t)==sizeof(x)) - 1]
```

Macro to ensure enum type consistency across binaries.

15.65.2.10 AAX_STEM_FORMAT

```
#define AAX_STEM_FORMAT(
    aIndex,
    aChannelCount ) ( static_cast<uint32_t>( ( static_cast<uint16_t>(aIndex) << 16
) | ( (aChannelCount >= AAX_UINT16_MIN) && (aChannelCount <= 0xFFFF) ? aChannelCount & 0xFFFF
: 0x0000 ) ) )
```

15.65.2.11 AAX_STEM_FORMAT_CHANNEL_COUNT

```
#define AAX_STEM_FORMAT_CHANNEL_COUNT(
    aStemFormat ) ( static_cast<uint16_t>( aStemFormat & 0xFFFF ) )
```

15.65.2.12 AAX_STEM_FORMAT_INDEX

```
#define AAX_STEM_FORMAT_INDEX(
    aStemFormat ) ( static_cast<int16_t>( ( aStemFormat >> 16 ) & 0xFFFF ) )
```

15.65.3 Typedef Documentation

15.65.3.1 AAX_EParameterType

```
typedef enum AAX_EParameterType AAX_EParameterType
```

FIC stuff that I can't include without DAE library dependence.

Legacy Porting Notes Values must match unnamed type enum in FicTDMControl.h

Todo FLAGGED FOR REMOVAL

15.65.3.2 AAX_EParameterOrientation

```
typedef int32_t AAX_EParameterOrientation
```

Typedef for a bitfield of [AAX_EParameterOrientationBits](#) values.

15.65.4 Enumeration Type Documentation

15.65.4.1 AAX_EHighlightColor

```
enum AAX_EHighlightColor
```

Highlight color selector.

See also

[AAX_IEffectGUI::SetControlHighlightInfo\(\)](#)

Enumerator

AAX_eHighlightColor_Red	
AAX_eHighlightColor_Blue	
AAX_eHighlightColor_Green	
AAX_eHighlightColor_Yellow	
AAX_eHighlightColor_Num	

15.65.4.2 AAX_ETracePriorityHost

```
enum AAX_ETracePriorityHost
```

Platform-specific tracing priorities.

Use the generic `EAXX_Trace_Priority` in plug-ins for cross-platform tracing (see [AAX_Assert.h](#))

Enumerator

AAX_eTracePriorityHost_None	
AAX_eTracePriorityHost_Critical	
AAX_eTracePriorityHost_High	
AAX_eTracePriorityHost_Normal	
AAX_eTracePriorityHost_Low	
AAX_eTracePriorityHost_Lowest	

15.65.4.3 AAX_ETracePriorityDSP

enum [AAX_ETracePriorityDSP](#)

Platform-specific tracing priorities.

Use the generic `EAXX_Trace_Priority` in plug-ins for cross-platform tracing (see [AAX_Assert.h](#))

Enumerator

<code>AAX_eTracePriorityDSP_None</code>	
<code>AAX_eTracePriorityDSP_Assert</code>	
<code>AAX_eTracePriorityDSP_High</code>	
<code>AAX_eTracePriorityDSP_Normal</code>	
<code>AAX_eTracePriorityDSP_Low</code>	

15.65.4.4 AAX_EModifiers

enum [AAX_EModifiers](#)

Modifier key definitions used by AAX API.

Enumerator

<code>AAX_eModifiers_None</code>	
<code>AAX_eModifiers_Shift</code>	Shift.
<code>AAX_eModifiers_Control</code>	Control on Mac, Winkey/Start on PC.
<code>AAX_eModifiers_Option</code>	Option on Mac, Alt on PC.
<code>AAX_eModifiers_Command</code>	Command on Mac, Ctrl on PC.
<code>AAX_eModifiers_SecondaryButton</code>	Secondary mouse button.
<code>AAX_eModifiers_Alt</code>	Option on Mac, Alt on PC.
<code>AAX_eModifiers_Cntl</code>	Command on Mac, Cntl on PC.
<code>AAX_eModifiers_WINKEY</code>	Control on Mac, WINKEY on PC.

15.65.4.5 AAX_EAudioBufferLength

enum [AAX_EAudioBufferLength](#)

Generic buffer length definitions.

These enum values can be used to calculate literal values as powers of two:

```
(1 << AAX\_eAudioBufferLength\_16) == 16;
```

See also

[AAX_EAudioBufferLengthDSP](#)

[AAE_EAudioBufferLengthNative](#)

Enumerator

AAX_eAudioBufferLength_Undefined	
AAX_eAudioBufferLength_1	
AAX_eAudioBufferLength_2	
AAX_eAudioBufferLength_4	
AAX_eAudioBufferLength_8	
AAX_eAudioBufferLength_16	
AAX_eAudioBufferLength_32	
AAX_eAudioBufferLength_64	
AAX_eAudioBufferLength_128	
AAX_eAudioBufferLength_256	
AAX_eAudioBufferLength_512	
AAX_eAudioBufferLength_1024	
AAX_eAudioBufferLength_Max	Maximum buffer length for ProcessProc processing buffers. Audio buffers for other methods, such as the high-latency render callback for AAX Hybrid or the offline render callback for Host Processor effects, may contain more samples than AAX_eAudioBufferLength_Max.

15.65.4.6 AAX_EAudioBufferLengthDSP

enum [AAX_EAudioBufferLengthDSP](#)

Currently supported processing buffer length definitions for AAX DSP hosts.

AAX DSP decks must support at least these buffer lengths. All AAX DSP algorithm ProcessProcs must support exactly one of these buffer lengths.

See also

[AAX_eProperty_DSP_AudioBufferLength](#)

Enumerator

AAX_eAudioBufferLengthDSP_Default	
AAX_eAudioBufferLengthDSP_4	
AAX_eAudioBufferLengthDSP_16	
AAX_eAudioBufferLengthDSP_32	
AAX_eAudioBufferLengthDSP_64	
AAX_eAudioBufferLengthDSP_Max	

15.65.4.7 AAE_EAudioBufferLengthNative

enum [AAE_EAudioBufferLengthNative](#)

Processing buffer length definitions for Native AAX hosts.

All AAX Native plug-ins must support variable buffer lengths. The buffer lengths that a host will use are constrained by the values in this enum. All Native buffer lengths will be powers of two, as per [AAX_EAudioBufferLength](#)

See also

[AAX_eProperty_DSP_AudioBufferLength](#)

Enumerator

AAX_eAudioBufferLengthNative_Min	Minimum Native buffer length.
AAX_eAudioBufferLengthNative_Max	Maximum Native buffer length.

15.65.4.8 AAX_EMaxAudioSuiteTracks

enum [AAX_EMaxAudioSuiteTracks](#)

The maximum number of tracks that an AAX host will process in a non-real-time context.

See also

[AAX_eProperty_NumberOfInputs](#) and [AAX_eProperty_NumberOfOutputs](#)

Enumerator

AAX_eMaxAudioSuiteTracks	
--------------------------	--

15.65.4.9 AAX_EStemFormat

enum [AAX_EStemFormat](#)

Stem format definitions.

A stem format combines a channel count with a semantic meaning for each channel. Usually this is the speaker or speaker position associated with the data in the channel. The meanings of each channel in each stem format (i.e. channel orders) are listed below.

Not all stem formats are supported by all AAX plug-in hosts. An effect may describe support for any stem format combination which it supports and the host will ignore any configurations which it cannot support.

Note

When defining stem format support in [AAX_IHostProcessor](#) effects do not use stem format properties or values. Instead, use [AAX_eProperty_NumberOfInputs](#) and [AAX_eProperty_NumberOfOutputs](#) with integer channel count values.

See also

- [AAX_eProperty_InputStemFormat](#)
- [AAX_eProperty_OutputStemFormat](#)
- [AAX_eProperty_HybridInputStemFormat](#)
- [AAX_eProperty_HybridOutputStemFormat](#)
- [AAX_eProperty_SideChainStemFormat](#)

Enumerator

AAX_eStemFormat_Mono	M.
AAX_eStemFormat_Stereo	L R.
AAX_eStemFormat_LCR	L C R.
AAX_eStemFormat_LCRS	L C R S.
AAX_eStemFormat_Quad	L R Ls Rs.
AAX_eStemFormat_5_0	L C R Ls Rs.
AAX_eStemFormat_5_1	L C R Ls Rs LFE.
AAX_eStemFormat_6_0	L C R Ls Cs Rs.
AAX_eStemFormat_6_1	L C R Ls Cs Rs LFE.
AAX_eStemFormat_7_0_SDDS	L Lc C Rc R Ls Rs.
AAX_eStemFormat_7_1_SDDS	L Lc C Rc R Ls Rs LFE.
AAX_eStemFormat_7_0_DTS	L C R Lss Rss Lsr Rsr.
AAX_eStemFormat_7_1_DTS	L C R Lss Rss Lsr Rsr LFE.
AAX_eStemFormat_7_0_2	L C R Lss Rss Lsr Rsr Lts Rts.
AAX_eStemFormat_7_1_2	L C R Lss Rss Lsr Rsr LFE Lts Rts.
AAX_eStemFormat_5_0_2	L C R Ls Rs Ltm Rtm.
AAX_eStemFormat_5_1_2	L C R Ls Rs LFE Ltm Rtm.
AAX_eStemFormat_5_0_4	L C R Ls Rs Ltf Rtf Ltr Rtr.
AAX_eStemFormat_5_1_4	L C R Ls Rs LFE Ltf Rtf Ltr Rtr.
AAX_eStemFormat_7_0_4	L C R Lss Rss Lsr Rsr Ltf Rtf Ltr Rtr.
AAX_eStemFormat_7_1_4	L C R Lss Rss Lsr Rsr LFE Ltf Rtf Ltr Rtr.
AAX_eStemFormat_7_0_6	L C R Lss Rss Lsr Rsr Ltf Rtf Ltm Rtm Ltr Rtr.
AAX_eStemFormat_7_1_6	L C R Lss Rss Lsr Rsr LFE Ltf Rtf Ltm Rtm Ltr Rtr.
AAX_eStemFormat_9_0_4	L C R Lw Rw Lss Rss Lsr Rsr Ltf Rtf Ltr Rtr.
AAX_eStemFormat_9_1_4	L C R Lw Rw Lss Rss Lsr Rsr LFE Ltf Rtf Ltr Rtr.
AAX_eStemFormat_9_0_6	L C R Lw Rw Lss Rss Lsr Rsr Ltf Rtf Ltm Rtm Ltr Rtr.
AAX_eStemFormat_9_1_6	L C R Lw Rw Lss Rss Lsr Rsr LFE Ltf Rtf Ltm Rtm Ltr Rtr.
AAX_eStemFormat_Ambi_1_ACN	Ambisonics: first-order with ACN channel order and SN3D (AmbiX) normalization.
AAX_eStemFormat_Ambi_2_ACN	Ambisonics: second-order with ACN channel order and SN3D (AmbiX) normalization.
AAX_eStemFormat_Ambi_3_ACN	Ambisonics: third-order with ACN channel order and SN3D (AmbiX) normalization.
AAX_eStemFormat_Ambi_4_ACN	Ambisonics: fourth-order with ACN channel order and SN3D (AmbiX) normalization.
AAX_eStemFormat_Ambi_5_ACN	Ambisonics: fifth-order with ACN channel order and SN3D (AmbiX) normalization.

Enumerator

AAX_eStemFormat_Ambi_6_ACN	Ambisonics: sixth-order with ACN channel order and SN3D (AmbiX) normalization.
AAX_eStemFormat_Ambi_7_ACN	Ambisonics: seventh-order with ACN channel order and SN3D (AmbiX) normalization.
AAX_eStemFormatNum	
AAX_eStemFormat_None	
AAX_eStemFormat_Any	
AAX_eStemFormat_INT32_MAX	

15.65.4.10 AAX_EPlugInCategory

enum [AAX_EPlugInCategory](#)

Effect category definitions.

Used with [AAX_IEffectDescriptor::AddCategory\(\)](#) to categorize an Effect.

These values are bitwise-exclusive and may be used in a bitmask to define multiple categories:

```
myCategory = AAX_ePlugInCategory_EQ | AAX_ePlugInCategory_Dynamics;
```

Note

The host may handle plug-ins with different categories in different manners, e.g. replacing "analyze" with "reverse" for offline processing of delays and reverbs.

Enumerator

AAX_ePlugInCategory_None	
AAX_ePlugInCategory_EQ	Equalization.
AAX_ePlugInCategory_Dynamics	Compressor, expander, limiter, etc.
AAX_ePlugInCategory_PitchShift	Pitch processing.
AAX_ePlugInCategory_Reverb	Reverberation and room/space simulation.
AAX_ePlugInCategory_Delay	Delay and echo.
AAX_ePlugInCategory_Modulation	Phasing, flanging, chorus, etc.
AAX_ePlugInCategory_Harmonic	Distortion, saturation, and harmonic enhancement.
AAX_ePlugInCategory_NoiseReduction	Noise reduction.
AAX_ePlugInCategory_Dither	Dither, noise shaping, etc.
AAX_ePlugInCategory_SoundField	Pan, auto-pan, upmix and downmix, and surround handling.
AAX_ePlugInCategory_HWGenerators	Fixed hardware audio sources such as SampleCell.
AAX_ePlugInCategory_SWGenerators	Virtual instruments, metronomes, and other software audio sources.
AAX_ePlugInCategory_WrappedPlugin	All plug-ins wrapped by a third party wrapper (i.e. VST to RTAS wrapper), except for VI plug-ins which should be mapped to AAX_PlugInCategory_SWGenerators.
AAX_EPlugInCategory_Effect	Special effects.
AAX_ePlugInCategory_Example	
AAX_EPlugInCategory_MIDIEffect	MIDI effects.
AAX_ePlugInCategory_INT32_MAX	

15.65.4.11 AAX_EPlugInStrings

enum [AAX_EPlugInStrings](#)

Effect string identifiers.

The AAX host may associate certain plug-in display strings with these identifiers.

See also

[AAX_IEffectGUI::GetCustomLabel\(\)](#)

Enumerator

AAX_ePlugInStrings_Analysis	"Analyze" button label (AudioSuite) Legacy Porting Notes Was pluginStrings_Analysis in the RTAS/TDM SDK
AAX_ePlugInStrings_MonoMode	"Mono Mode" selector label (AudioSuite) Legacy Porting Notes Was pluginStrings_MonoMode in the RTAS/TDM SDK
AAX_ePlugInStrings_MultiInputMode	"Multi-Input Mode" selector label (AudioSuite) Legacy Porting Notes Was pluginStrings_Multi↔InputMode in the RTAS/TDM SDK
AAX_ePlugInStrings_RegionByRegionAnalysis	"Clip-by-Clip Analysis" selector label (AudioSuite) Legacy Porting Notes Was pluginStrings_Region↔ByRegionAnalysis in the RTAS/TDM SDK
AAX_ePlugInStrings_AllSelectedRegionsAnalysis	"Whole File Analysis" selector label (AudioSuite) Legacy Porting Notes Was pluginStrings_All↔SelectedRegionsAnalysis in the RTAS/TDM SDK
AAX_ePlugInStrings_RegionName	Deprecated
AAX_ePlugInStrings_ClipName	Clip name label (AudioSuite). This value will replace the clip's name. See also AAX_ePlugInStrings_ClipNameSuffix Legacy Porting Notes Was pluginStrings_RegionName in the RTAS/TDM SDK

Enumerator

AAX_ePlugInStrings_Progress	Progress bar label (AudioSuite) Host Compatibility Notes Not currently supported by Pro Tools Legacy Porting Notes Was pluginStrings_Progress in the RTAS/TDM SDK
AAX_ePlugInStrings_PluginFileName	Deprecated
AAX_ePlugInStrings_Preview	Deprecated
AAX_ePlugInStrings_Process	"Render" button label (AudioSuite) Legacy Porting Notes Was pluginStrings_Process in the RTAS/TDM SDK
AAX_ePlugInStrings_Bypass	"Bypass" button label (AudioSuite) Legacy Porting Notes Was pluginStrings_Bypass in the RTAS/TDM SDK
AAX_ePlugInStrings_ClipNameSuffix	Clip name label suffix (AudioSuite). This value will be appended to the clip's name, vs AAX_ePlugInStrings_ClipName which will replace the clip's name completely.
AAX_ePlugInStrings_INT32_MAX	

15.65.4.12 AAX_EMeterOrientation

enum [AAX_EMeterOrientation](#)

Meter orientation.

Use this enum in conjunction with the [AAX_eProperty_Meter_Orientation](#) property

For more information about meters in [AAX](#), see [Plug-in meters](#)

Enumerator

AAX_eMeterOrientation_Default	
AAX_eMeterOrientation_BottomLeft	the default orientation
AAX_eMeterOrientation_TopRight	Some dynamics plug-in orient their gain reduction like so.
AAX_eMeterOrientation_Center	A plug-in that does gain increase and decrease may want this. meter values less than 0x40000000 would display downward from the mid-point. meter values greater than 0x40000000 would display upward from the mid-point.
AAX_eMeterOrientation_PhaseDot	linear scale, displays 2 dots around the value (currently D-Control only)

15.65.4.13 AAX_EMeterBallisticType

enum [AAX_EMeterBallisticType](#)

Meter ballistics type.

Use this enum in conjunction with the [AAX_eProperty_Meter_Ballistics](#) property

For more information about meters in [AAX](#), see [Plug-in meters](#)

Enumerator

AAX_eMeterBallisticType_Host	The ballistics follow the host settings.
AAX_eMeterBallisticType_NoDecay	No decay ballistics.

15.65.4.14 AAX_EMeterType

enum [AAX_EMeterType](#)

Meter type.

Use this enum in conjunction with the [AAX_eProperty_Meter_Type](#) property

For more information about meters in [AAX](#), see [Plug-in meters](#)

Enumerator

AAX_eMeterType_Input	e.g. Your typical input meter (possibly after an input gain stage)
AAX_eMeterType_Output	e.g. Your typical output meter (possibly after an output gain stage)
AAX_eMeterType_CLGain	e.g. Compressor/Limiter gain reduction
AAX_eMeterType_EGGain	e.g. Expander/Gate gain reduction
AAX_eMeterType_Analysis	e.g. multi-band amplitude from a Spectrum analyzer
AAX_eMeterType_Other	e.g. a meter that does not fit in any of the above categories
AAX_eMeterType_None	For internal host use only.

15.65.4.15 AAX_EResourceType

enum [AAX_EResourceType](#)

Types of resources that can be added to an Effect's description.

See also

[AAX_IEffectDescriptor::AddResourceInfo\(\)](#)

Enumerator

AAX_eResourceType_None	
AAX_eResourceType_PageTable	The file name of the page table xml file
AAX_eResourceType_PageTableDir	The absolute path to the directory containing the plug-in's page table xml file(s) Defaults to *.aaxplugin/Contents/Resources

15.65.4.16 AAX_ENotificationEvent

enum [AAX_ENotificationEvent](#)

Events IDs for AAX notifications.

- Notifications listed with *Sent by: Host* are dispatched by the AAX host and may be received in one or more of
- [AAX_IEffectParameters::NotificationReceived\(\)](#)
- [AAX_IEffectGUI::NotificationReceived\(\)](#)
- [AAX_IEffectDirectData::NotificationReceived\(\)](#)

The host will choose which components are registered to receive each event type. See the documentation for each event type for more information.

Note

All 'AX__' four-char IDs are reserved for the AAX specification

Enumerator

AAX_eNotificationEvent_InsertPositionChanged	(not currently sent) The zero-indexed insert position of this plug-in instance within its track <i>Data: int32_t</i> <i>Sent by: Host</i>
AAX_eNotificationEvent_TrackNameChanged	(const AAX_IString) The current name of this plug-in instance's track Host Compatibility Notes Supported in Pro Tools 11.2 and higher Not supported by Media Composer <i>Data: const AAX_IString</i> <i>Sent by: Host</i>
AAX_eNotificationEvent_TrackUIDChanged	(not currently sent) The current UID of this plug-in instance's track <i>Data: const uint8_t[16]</i> <i>Sent by: Host</i>
AAX_eNotificationEvent_TrackPositionChanged	(not currently sent) The current position index of this plug-in instance's track <i>Data: int32_t</i> <i>Sent by: Host</i>

Enumerator

AAX_eNotificationEvent_AlgorithmMoved	Not currently sent. <i>Data: none</i> <i>Sent by: Host</i>
AAX_eNotificationEvent_GUIOpened	Not currently sent. <i>Data: none</i> <i>Sent by: Host</i>
AAX_eNotificationEvent_GUIClosed	Not currently sent. <i>Data: none</i> <i>Sent by: Host</i>
AAX_eNotificationEvent_ASProcessingState	AudioSuite processing state change notification. One of AAX_EProcessingState. Host Compatibility Notes Supported in Pro Tools 11 and higher Not supported by Media Composer <i>Data: int32_t</i> <i>Sent by: Host</i>
AAX_eNotificationEvent_ASPreviewState	AudioSuite preview state change notification. One of AAX_EPreviewState. Legacy Porting Notes Replacement for <code>SetPreviewState()</code> Host Compatibility Notes Supported in Pro Tools 11 and higher Not supported by Media Composer <i>Data: int32_t</i> <i>Sent by: Host</i>
AAX_eNotificationEvent_SessionBeingOpened	Tell the plug-in that chunk data is coming from a PTX. Host Compatibility Notes Supported in Pro Tools 11 and higher Not supported by Media Composer <i>Data: none</i> <i>Sent by: Host</i>
AAX_eNotificationEvent_PresetOpened	Tell the plug-in that chunk data is coming from a TFX. Host Compatibility Notes Supported in Pro Tools 11 and higher <i>Data: none</i> <i>Sent by: Host</i>
AAX_eNotificationEvent_EnteringOfflineMode	Entering offline processing mode (i.e. offline bounce) Host Compatibility Notes Supported in Pro Tools 11 and higher <i>Data: none</i> <i>Sent by: Host</i>

Enumerator

AAX_eNotificationEvent_ExitingOfflineMode	Exiting offline processing mode (i.e. offline bounce) Host Compatibility Notes Supported in Pro Tools 11 and higher <i>Data: none</i> <i>Sent by: Host</i>
AAX_eNotificationEvent_SessionPathChanged	A string representing the path of the current session. Host Compatibility Notes Supported in Pro Tools 11.1 and higher <i>Data: const AAX_IString</i> <i>Sent by: Host</i>
AAX_eNotificationEvent_SignalLatencyChanged	The host has changed its latency compensation for this plug-in instance. Note This notification may be sent redundantly just after plug-in instantiation when the AAX_eProperty_LatencyContribution property is described. Host Compatibility Notes Supported in Pro Tools 11.1 and higher <i>Data: none</i> <i>Sent by: Host</i>
AAX_eNotificationEvent_DelayCompensationState	The host's delay compensation state has changed. This notification refers to the host's delay compensation feature as a whole, rather than the specific delay compensation state for the plug-in. Possible values: 0 (disabled), 1 (enabled) Plug-ins may need to monitor the host's delay compensation state because, while delay compensation is disabled, the host will never change the plug-in's accounted latency and, therefore, will never dispatch AAX_eNotificationEvent_SignalLatencyChanged to the plug-in following a call to AAX_IController::SetSignalLatency(). Host Compatibility Notes Supported in Pro Tools 12.6 and higher <i>Data: int32_t</i> <i>Sent by: Host</i>
AAX_eNotificationEvent_CycleCountChanged	(not currently sent) The host has changed its DSP cycle allocation for this plug-in instance <i>Data: none</i> <i>Sent by: Host</i>

Enumerator

AAX_eNotificationEvent_MaxViewSizeChanged	Tell the plug-in the maximum allowed GUI dimensions. Delivered to the plugin's AAX_IEffectGUI::NotificationReceived() Host Compatibility Notes Supported in Pro Tools 11.1 and higher <i>Data: const AAX_Point</i> <i>Sent by: Host</i>
AAX_eNotificationEvent_SideChainBeingConnected	Tell the plug-in about connection of the sidechain input. Host Compatibility Notes Supported in Pro Tools 11.1 and higher <i>Data: none</i> <i>Sent by: Host</i>
AAX_eNotificationEvent_SideChainBeingDisconnected	Tell the plug-in about disconnection of the sidechain input. Host Compatibility Notes Supported in Pro Tools 11.1 and higher <i>Data: none</i> <i>Sent by: Host</i>
AAX_eNotificationEvent_NoiseFloorChanged	The plug-in's noise floor level. The notification data is the new absolute noise floor level generated by the plug-in, as amplitude. For example, a plug-in generating a noise floor at -80 dB (amplitude) would provide 0.0001 in the notification data. Signal below the level of the plug-in's noise floor may be ignored by host features such as Dynamic Plug-In Processing, which detect whether or not there is any signal being generated by the plug-in <i>Data: double</i> <i>Sent by: Plug-in</i>
AAX_eNotificationEvent_ParameterMappingChanged	Notify the host that some aspect of the parameters' mapping has changed. To respond to this notification, the host will call AAX_IEffectParameters::UpdatePageTable() to update its cached page tables. <i>Data: none</i> <i>Sent by: Plug-in</i>
AAX_eNotificationEvent_ParameterNameChanged	Notify the host that one or more parameters' display names have changed. The payload is the parameter's ID. The payload size must be at least as large as the ID string, including the null termination character, and no larger than the size of the buffer containing the AAX_CParamID . Host Compatibility Notes Supported in Pro Tools 2023.3 and higher <i>Data: const AAX_CParamID</i> <i>Sent by: Plug-in</i>

Enumerator

AAX_eNotificationEvent_HostModeChanged	Notify the plug-in about Host mode changing. Host Compatibility Notes Supported in Venue 5.6 and higher <i>Data: AAX_EHostModeBits</i> <i>Sent by: Host</i>
AAX_eNotificationEvent_PriorSettingsInvalid	Previously-saved settings may no longer restore the captured state. Use this notification when a change occurs which may cause a different state to be restored by saved settings, and in particular by a saved setting representing the plug-in's state just prior to the change. For example, a plug-in which restricts certain types of state changes when the host is in AAX_eHostModeBits_Live mode should post an AAX_eNotificationEvent_PriorSettingsInvalid notification when this part of the plug-in state is changed manually by the user; if plug-in settings captured prior to this manual change are later set on the plug-in while the host is in live mode then some part of the settings change will be blocked and the captured state will not be perfectly restored. Host Compatibility Notes Supported in Venue 5.6 and higher <i>Data: none</i> <i>Sent by: Plug-in</i>
AAX_eNotificationEvent_LogState	Notify plug-in to log current state. Plug-in implementation specific Host Compatibility Notes Pro Tools currently only sends this notification to the Direct Data object in the plug-in <i>Data: none</i> <i>Sent by: Host</i>
AAX_eNotificationEvent_TransportStateChanged	Notify plug-in that the TransportState was changed. Host Compatibility Notes Supported in Pro Tools 2021.10 and higher <i>Data: AAX_TransportStateInfo_V1</i> <i>Sent by: Host</i>

Enumerator

AAX_eNotificationEvent_HostLocale	Tell the plug-in the current host language setting. Data is sent as a string. The format is a two-part code based on RFC 4646. The values follow Microsoft's formatting for CultureInfo culture names as described in http://msdn.microsoft.com/en-us/library/system.globalization.cultureinfo%28VS.80%29.aspx Examples: • en-US: English (US) • ja-JP: Japanese • ko-KR: Korean • fr-FR: French • it-IT: Italian • de-DE: German • es-ES: Spanish These exceptions to the specification are used by Pro Tools: • zh-CHS: Simplified Chinese • zh-CN: Traditional Chinese Note Currently in Pro Tools the language setting will remain consistent throughout the lifetime of the plugin instance. Delivered to the plugin's AAX_IEffectGUI::NotificationReceived() and AAX_IEffectParameters::NotificationReceived() Host Compatibility Notes Supported in Pro Tools 2024.3 and higher <i>Data:</i> <code>const AAX_IString</code> <i>Sent by:</i> Host
-----------------------------------	--

15.65.4.17 AAX_EHostModeBits

enum [AAX_EHostModeBits](#)

Host mode.

Host Compatibility Notes Supported in Venue 5.6 and higher

Enumerator

AAX_eHostModeBits_None	No special host mode, e.g. Pro Tools normal operation, Venue Config mode.
AAX_eHostModeBits_Live	The host is in a live playback mode, e.g. Venue Show mode - inserts are live and must not allow state changes which interrupt audio processing.

15.65.4.18 AAX_EHostMode

enum [AAX_EHostMode](#)

DEPRECATED.

Use [AAX_EHostModeBits](#)

Warning

The values of these modes have changed as of AAX SDK 2.3.1 from the definitions originally published in AAX SDK 2.3.0

Deprecated This enum is deprecated and will be removed in a future release.

Enumerator

AAX_eHostMode_Show	Deprecated Use AAX_eHostModeBits_Live
AAX_eHostMode_Config	Deprecated Use AAX_eHostModeBits_None

15.65.4.19 AAX_EPrivateDataOptions

enum [AAX_EPrivateDataOptions](#)

Options for algorithm private data fields.

Enumerator

AAX_ePrivateDataOptions_DefaultOptions	
AAX_ePrivateDataOptions_KeepOnReset	Retain data upon plug-in reset. Warning Not currently implemented. If this functionality is desired, the recommended workaround is to cache the desired private data to be set during AAX_IEffectParameters::ResetFieldData() .

Enumerator

AAX_ePrivateDataOptions_External	Place the block in external memory (internal by default)
AAX_ePrivateDataOptions_Align8	Place the block in mem aligned by 64 bits.
AAX_ePrivateDataOptions_INT32_MAX	

15.65.4.20 AAX_EConstraintLocationMask

enum [AAX_EConstraintLocationMask](#)

Property values to describe location constraints placed on the plug-in's algorithm component (`ProcessProc`)

See also

[AAX_eProperty_Constraint_Location](#)

Enumerator

AAX_eConstraintLocationMask_None	No constraint placed on component's location.
AAX_eConstraintLocationMask_DataModel	This <code>ProcessProc</code> must be co-located with the plug-in's data model object.
AAX_eConstraintLocationMask_DLLChipAffinity	This <code>ProcessProc</code> should be instantiated on the same chip as other effects that use the same DLL. This constraint is only applicable to DSP algorithms This property should only be used when absolutely required, as it will constrain the DSP manager and reduce overall DSP plug-in instance counts on the system. Host Compatibility Notes This constraint is supported in Pro Tools 10.2 and higher

15.65.4.21 AAX_EConstraintTopology

enum [AAX_EConstraintTopology](#)

Property values to describe the topology of the plug-in's modules (e.g. data model, GUI.)

See also

[AAX_eProperty_Constraint_Topology](#)

Enumerator

AAX_eConstraintTopology_None	No constraint placed on plug-in's topology.
AAX_eConstraintTopology_Monolithic	All plug-in modules (e.g. data model, GUI) must be co-located and non-relocatable.

15.65.4.22 AAX_EComponentInstanceInitAction

enum [AAX_EComponentInstanceInitAction](#)

Selector indicating the action that occurred to prompt a component initialization callback.

See also

[AAX_CInstanceInitProc](#)

Enumerator

AAX_eComponentInstanceInitAction_AddingNewInstance	
AAX_eComponentInstanceInitAction_RemovingInstance	
AAX_eComponentInstanceInitAction_ResetInstance	

15.65.4.23 AAX_ESampleRateMask

enum [AAX_ESampleRateMask](#)

Property values to describe various sample rates.

These values may be used as a bitmask, so e.g. a particular Effect may declare compatibility with `AAX_eSampleRateMask_44100 | AAX_eSampleRateMask_48000`

See also

[AAX_eProperty_SampleRate](#)

Enumerator

AAX_eSampleRateMask_No	
AAX_eSampleRateMask_44100	
AAX_eSampleRateMask_48000	
AAX_eSampleRateMask_88200	
AAX_eSampleRateMask_96000	
AAX_eSampleRateMask_176400	
AAX_eSampleRateMask_192000	
AAX_eSampleRateMask_All	

15.65.4.24 AAX_EParameterType

enum [AAX_EParameterType](#)

FIC stuff that I can't include without DAE library dependence.

Legacy Porting Notes Values must match unnamed type enum in FicTDMControl.h

Todo FLAGGED FOR REMOVAL

Enumerator

AAX_eParameterType_Discrete	Legacy Porting Notes Matches kDAE_DiscreteValues
AAX_eParameterType_Continuous	Legacy Porting Notes Matches kDAE_ContinuousValues

15.65.4.25 AAX_EParameterOrientationBits

enum [AAX_EParameterOrientationBits](#)

Visual Orientation of a parameter.

Todo FLAGGED FOR REVISION

Enumerator

AAX_eParameterOrientation_Default	
AAX_eParameterOrientation_BottomMinTopMax	
AAX_eParameterOrientation_TopMinBottomMax	
AAX_eParameterOrientation_LeftMinRightMax	
AAX_eParameterOrientation_RightMinLeftMax	
AAX_eParameterOrientation_RotarySingleDotMode	
AAX_eParameterOrientation_RotaryBoostCutMode	
AAX_eParameterOrientation_RotaryWrapMode	
AAX_eParameterOrientation_RotarySpreadMode	
AAX_eParameterOrientation_RotaryLeftMinRightMax	
AAX_eParameterOrientation_RotaryRightMinLeftMax	

15.65.4.26 AAX_EParameterValueInfoSelector

enum [AAX_EParameterValueInfoSelector](#)

Query type selectors for use with [AAX_IEffectParameters::GetParameterValueInfo\(\)](#)

See also

[AAX_EEQBandTypes](#)

[AAX_EEQInCircuitPolarity](#)

[AAX_EUseAlternateControl](#)

Legacy Porting Notes converted from `EControlValueInfo` in the legacy SDK

Enumerator

<code>AAX_ePageTable_EQ_Band_Type</code>	EQ filter band type. Possible response values are listed in AAX_EEQBandTypes Legacy Porting Notes converted from <code>eDigi_PageTable_EQ_Band_Type</code> in the legacy SDK
<code>AAX_ePageTable_EQ_InCircuitPolarity</code>	Description of whether a particular EQ band is active. Possible response values are listed in AAX_EEQInCircuitPolarity Legacy Porting Notes converted from <code>eDigi_PageTable_↔EQ_InCircuitPolarity</code> in the legacy SDK
<code>AAX_ePageTable_UseAlternateControl</code>	Description of whether an alternate parameter should be used for a given slot. For example, some control surfaces support Q/Slope encoders. Using an alternate control mechanism, plug-ins mapped to these devices can assign a different slope control to the alternate slot and have it coexist with a Q control for each band. This is only applicable when mapping separate parameters to the same encoder; if the Q and Slope controls are implemented as the same parameter object in the plug-in then customization is not needed. Possible response values are listed in AAX_EUseAlternateControl Legacy Porting Notes converted from <code>eDigi_PageTable_↔UseAlternateControl</code> in the legacy SDK

15.65.4.27 AAX_EEQBandTypes

enum [AAX_EEQBandTypes](#)

Definitions of band types for EQ page table.

For the [AAX_ePageTable_EQ_Band_Type](#) parameter value info selector

Enumerator

AAX_eEQBandType_HighPass	Freq, Slope
AAX_eEQBandType_LowShelf	Freq, Gain, Slope
AAX_eEQBandType_Parametric	Freq, Gain, Q
AAX_eEQBandType_HighShelf	Freq, Gain, Slope
AAX_eEQBandType_LowPass	Freq, Slope
AAX_eEQBandType_Notch	Freq, Q

15.65.4.28 AAX_EEQInCircuitPolarity

enum [AAX_EEQInCircuitPolarity](#)

Definitions for band in/out for EQ page table.

For the AAX_ePageTable_EQ_InCircuitPolarity parameter value selector

Enumerator

AAX_eEQInCircuitPolarity_Enabled	EQ band is in the signal path and enabled
AAX_eEQInCircuitPolarity_Bypassed	EQ band is in the signal path but bypassed/off
AAX_eEQInCircuitPolarity_Disabled	EQ band is completely removed from signal path

15.65.4.29 AAX_EUseAlternateControl

enum [AAX_EUseAlternateControl](#)

Definitions for Use Alternate Control parameter.

For the AAX_ePageTable_UseAlternateControl parameter value info selector

Enumerator

AAX_eUseAlternateControl_No	
AAX_eUseAlternateControl_Yes	

15.65.4.30 AAX_EMIDINodeType

enum [AAX_EMIDINodeType](#)

MIDI node types.

See also

[AAX_IComponentDescriptor::AddMIDINode\(\)](#)

Enumerator

AAX_eMIDINodeType_LocalInput	Local MIDI input. Local MIDI input nodes receive MIDI by accessing AAX_CMidiStream buffers filled with MIDI messages. These buffers of MIDI data are available within the algorithm context with data corresponding to the current audio buffer being computed. The Effect can step through this buffer like a "script" to respond to MIDI events within the audio callback. Legacy Porting Notes Corresponds to RTAS Buffered MIDI input nodes in the legacy SDK
AAX_eMIDINodeType_LocalOutput	Local MIDI output. Local MIDI output nodes send MIDI by filling buffers with MIDI messages. Messages posted to MIDI output nodes will be available in the host as MIDI streams, routable to MIDI track inputs and elsewhere. Data posted to a MIDI output buffer will be timed to correspond with the current audio buffer being processed. MIDI outputs support custom timestamping relative to the first sample of the audio buffer. The delivery of variable length SysEx messages is also supported. There are no buffer size limitations for output of SysEx messages. To post a MIDI output buffer, an Effect must construct a series of AAX_CMidiPacket objects and place them in the output buffer provided in the port's AAX_CMidiStream Legacy Porting Notes Corresponds to RTAS Buffered MIDI output nodes in the legacy SDK
AAX_eMIDINodeType_Global	Global MIDI node. Global MIDI nodes allow an Effect to receive streaming global MIDI data like MIDI Time Code, MIDI Beat Clock, and host-specific message formats such as the Click messages used in Pro Tools. The specific kind of data that will be received by a Global MIDI node is specified using a mask of AAX_EMidiGlobalNodeSelectors values. Global MIDI nodes are like local MIDI nodes, except they do not show up as assignable outputs in the host. Instead the MIDI data is automatically routed to the plug-in, without the user making any connections. The buffer of data provided via a Global MIDI node may be shared between all currently active Effect instances, and this node may include both explicitly requested data and data not requested by the current Effect. For example, if one plug-in requests MTC and another plug-in requests Click, all plug-ins connected to this global node will get both MTC and Click messages in the shared buffer. Legacy Porting Notes Corresponds to RTAS Shared Buffer global nodes in the legacy SDK
AAX_eMIDINodeType_Transport	Transport node. Call AAX_IMIDINode::GetTransport() on this node to access the AAX_ITransport interface. Warning See warning at AAX_IMIDINode::GetTransport() regarding use of this interface

15.65.4.31 AAX_EUpdateSource

enum [AAX_EUpdateSource](#)

Source for values passed into [UpdateParameterNormalizedValue\(\)](#).

Enumerator

AAX_eUpdateSource_Unspecified	Parameter updates of unknown / unspecified origin, currently including all updates from control surfaces, GUI edit events, and edits originating in the plug-in outside of the context of UpdateParameterNormalizedValue() or SetChunk() .
AAX_eUpdateSource_Parameter	Parameter updates originating (via AAX_IAutomationDelegate::PostSetValueRequest()) within the scope of UpdateParameterNormalizedValue() .
AAX_eUpdateSource_Chunk	Parameter updates originating (via AAX_IAutomationDelegate::PostSetValueRequest()) within the scope of SetChunk() .
AAX_eUpdateSource_Delay	:Not Used by AAX Plug-Ins

15.65.4.32 AAX_EDataInPortType

enum [AAX_EDataInPortType](#)

Property value for whether a data in port should be buffered or not.

See also

[AAX_IComponentDescriptor::AddDataInPort\(\)](#)

Enumerator

AAX_eDataInPortType_Unbuffered	Data port is unbuffered; the most recently posted packet is always delivered to the alg proc
AAX_eDataInPortType_Buffered	Data port is buffered both on the host and DSP and packets are updated to the current timestamp with every alg proc call Data delivered to alg proc always reflects the latest posted packet that has a timestamp at or before the current processing buffer

Enumerator

AAX_eDataInPortType_Incremental	Data port is buffered both on the host and DSP and packets are updated only once per alg proc call Since only one packet is delivered at a time, all packets will be delivered to the alg proc unless an internal buffer overflow occurs Note If multiple packets are posted to this port <i>before</i> the initial call to the alg proc, only the latest packet will be delivered to the first call to the alg proc. Thereafter, all packets will be delivered incrementally. Host Compatibility Notes Supported in Pro Tools 12.5 and higher; when AAX_eDataInPortType_Incremental is not supported the port will be treated as AAX_eDataInPortType_Unbuffered
---------------------------------	---

15.65.4.33 AAX_EFrameRate

enum [AAX_EFrameRate](#)

FrameRate types.

See also

[AAX_ITransport::GetTimeCodeInfo\(\)](#)

[AAX_ITransport::GetHDTIMECodeInfo\(\)](#)

Enumerator

AAX_eFrameRate_Undeclared	
AAX_eFrameRate_24Frame	
AAX_eFrameRate_25Frame	
AAX_eFrameRate_2997NonDrop	
AAX_eFrameRate_2997DropFrame	
AAX_eFrameRate_30NonDrop	
AAX_eFrameRate_30DropFrame	
AAX_eFrameRate_23976	
AAX_eFrameRate_47952	
AAX_eFrameRate_48Frame	
AAX_eFrameRate_50Frame	
AAX_eFrameRate_5994NonDrop	
AAX_eFrameRate_5994DropFrame	
AAX_eFrameRate_60NonDrop	
AAX_eFrameRate_60DropFrame	
AAX_eFrameRate_100Frame	
AAX_eFrameRate_11988NonDrop	
AAX_eFrameRate_11988DropFrame	
AAX_eFrameRate_120NonDrop	
AAX_eFrameRate_120DropFrame	

15.65.4.34 AAX_EFeetFramesRate

enum [AAX_EFeetFramesRate](#)

FeetFramesRate types.

See also

[AAX_ITransport::GetFeetFramesInfo\(\)](#)

Enumerator

AAX_eFeetFramesRate_23976	
AAX_eFeetFramesRate_24	
AAX_eFeetFramesRate_25	

15.65.4.35 AAX_EMidiGlobalNodeSelectors

enum [AAX_EMidiGlobalNodeSelectors](#)

The Global MIDI Node Selectors.

These selectors are used in the *channelMask* argument of [AAX_IComponentDescriptor::AddMIDINode\(\)](#) and [AAX_IEffectDescriptor::AddControlMIDINode\(\)](#) to request one or more kinds of global data.

Enumerator

AAX_eMIDIClick	Selector to request click messages. The click messages are special 2-byte messages encoded as follows: • Accented click: Note on pitch 0 (0x90 0x00) • Unaccented click: Note on pitch 1 (0x90 0x01) Note No <i>Note Off</i> messages are ever sent. This isn't up-to-spec MIDI data, just a way of encoding click events.
AAX_eMIDIMtc	Selector to request MIDI Time Code (MTC) data. The Standard MIDI Time Code format.
AAX_eMIDIBeatClock	Selector to request MIDI Beat Clock (MBC) messages. This includes Song Position Pointer, Start/Stop/Continue, and Midi Clock (F8).

15.65.4.36 AAX_EPreviewState

enum [AAX_EPreviewState](#)

Offline preview states for use with [AAX_eNotificationEvent_ASPreviewState](#).

Note

Do not perform any non-trivial processing within the notification handler. Instead, use the processing state notification to inform the processing that is performed in methods such as [PreRender\(\)](#).

Enumerator

AAX_ePreviewState_Stop	Offline preview has ended. For Host Processor plug-ins, this notification is sent just before the final call to PostRender() , or after analysis is complete for plug-ins with analysis-only preview.
AAX_ePreviewState_Start	Offline preview is beginning. For Host Processor plug-ins, this notification is sent before any calls to PreAnalyze() or to PreRender() .

15.65.4.37 AAX_EProcessingState

enum [AAX_EProcessingState](#)

Offline preview states for use with [AAX_eNotificationEvent_ASProcessingState](#).

Note

Do not perform any non-trivial processing within the notification handler. Instead, use the processing state notification to inform the processing that is performed in methods such as [PreRender\(\)](#).

Enumerator

AAX_eProcessingState_StopPass	A single offline processing pass has ended. A single offline processing pass is an analysis and/or render applied to a set of channels in parallel. For Host Processor plug-ins, this notification is sent just before the final call to PostRender() , or after analysis is complete for analysis-only offline plug-ins.
AAX_eProcessingState_StartPass	A single offline processing pass is beginning. A single offline processing pass is an analysis and/or render applied to a set of channels in parallel. For Host Processor plug-ins, this notification is sent before any calls to PreAnalyze() , PreRender() , or InitOutputBounds() for each processing pass.
AAX_eProcessingState_EndPassGroup	An offline processing pass group has completed. An offline processing pass group is a full set of analysis and/or render passes applied to the complete set of input channels. Host Compatibility Notes AudioSuite pass group notifications are supported starting in Pro Tools 12.0

Enumerator

AAX_eProcessingState_BeginPassGroup	An offline processing pass group is beginning. An offline processing pass group is a full set of analysis and/or render passes applied to the complete set of input channels. Host Compatibility Notes AudioSuite pass group notifications are supported starting in Pro Tools 12.0
AAX_eProcessingState_Stop	Deprecated
AAX_eProcessingState_Start	Deprecated

15.65.4.38 AAX_ETargetPlatform

enum [AAX_ETargetPlatform](#)

Describes what platform the component runs on.

Enumerator

kAAX_eTargetPlatform_None	
kAAX_eTargetPlatform_Native	
kAAX_eTargetPlatform_TI	
kAAX_eTargetPlatform_External	
kAAX_eTargetPlatform_Count	

15.65.4.39 AAX_ESupportLevel

enum [AAX_ESupportLevel](#)

Feature support indicators

See also

[AAX_IDescriptionHost::AcquireFeatureProperties\(\)](#)

Note

: There is no value defined for unknown features. Instead, unknown features are indicated by [AcquireFeatureProperties\(\)](#) providing a null [AAX_IFeatureInfo](#) in response to a request using the unknown feature UID

Enumerator

AAX_eSupportLevel_Uninitialized	An uninitialized AAX_ESupportLevel
AAX_eSupportLevel_Unsupported	The feature is known but explicitly not supported
AAX_eSupportLevel_Supported	
AAX_eSupportLevel_Disabled	
AAX_eSupportLevel_ByProperty	

15.65.4.40 AAX_EHostLevel

enum [AAX_EHostLevel](#)

Host levels.

Some AAX software hosts support different levels which are sold as separate products. For example, there may be an entry-level version of a product as well as a full version.

The level of a host may impact the user experience, workflows, or the availability of certain plug-ins. For example, some entry-level hosts are restricted to loading only specific plug-ins.

Typically an AAX plug-in should not need to query this information or change its behavior based on the level of the host.

Warning

AAX plugin description details are cached and are not re-scanned if the host level changes. Be careful to avoid changing any cacheable information based on this data.

See also

[AAXATTR_Client_Level](#)

Enumerator

AAX_eHostLevel_Unknown	
AAX_eHostLevel_Standard	Standard host level.
AAX_eHostLevel_Entry	Entry-level host.
AAX_eHostLevel_Intermediate	Intermediate-level host.

15.65.4.41 AAX_ETextEncoding

enum [AAX_ETextEncoding](#)

Describes possible string encodings.

Enumerator

AAX_eTextEncoding_Undefined	
AAX_eTextEncoding_UTF8	UTF-8 string encoding.
AAX_eTextEncoding_Num	

15.65.4.42 AAX_EAssertFlags

enum [AAX_EAssertFlags](#)

Flags for use with [AAX_IHostServices::HandleAssertFailure\(\)](#)

Enumerator

AAX_eAssertFlags_Default	No special handler requested.
AAX_eAssertFlags_Log	Logging requested.
AAX_eAssertFlags_Dialog	User-visible modal alert dialog requested.

15.65.4.43 AAX_ETransportState

enum [AAX_ETransportState](#)

Used to indicate the current transport state of the host. This is the global transport state; it does not indicate a track-specific state.

Enumerator

AAX_eTransportState_Unknown	
AAX_eTransportState_Stopping	
AAX_eTransportState_Stop	
AAX_eTransportState_Paused	
AAX_eTransportState_Play	
AAX_eTransportState_FastForward	
AAX_eTransportState_Rewind	
AAX_eTransportState_Scrub	
AAX_eTransportState_Shuttle	
AAX_eTransportState_Num	

15.65.4.44 AAX_ERecordMode

enum [AAX_ERecordMode](#)

Used to indicate the current record mode of the host. This is the global record mode; it does not indicate a track-specific state.

Enumerator

AAX_eRecordMode_Unknown	
AAX_eRecordMode_None	
AAX_eRecordMode_Normal	
AAX_eRecordMode_Destructive	
AAX_eRecordMode_QuickPunch	
AAX_eRecordMode_TrackPunch	
AAX_eRecordMode_Num	

15.65.5 Function Documentation

15.65.5.1 AAX_ENUM_SIZE_CHECK() [1/45]

```
AAX_ENUM_SIZE_CHECK (
    AAX_EHighlightColor )
```

15.65.5.2 AAX_ENUM_SIZE_CHECK() [2/45]

```
AAX_ENUM_SIZE_CHECK (
    AAX_ETracePriorityHost )
```

15.65.5.3 AAX_ENUM_SIZE_CHECK() [3/45]

```
AAX_ENUM_SIZE_CHECK (
    AAX_ETracePriorityDSP )
```

15.65.5.4 AAX_ENUM_SIZE_CHECK() [4/45]

```
AAX_ENUM_SIZE_CHECK (
    AAX_EModifiers )
```

15.65.5.5 AAX_ENUM_SIZE_CHECK() [5/45]

```
AAX_ENUM_SIZE_CHECK (
    AAX_EAudioBufferLength )
```

15.65.5.6 AAX_ENUM_SIZE_CHECK() [6/45]

```
AAX_ENUM_SIZE_CHECK (
    AAX_EAudioBufferLengthDSP )
```

15.65.5.7 AAX_ENUM_SIZE_CHECK() [7/45]

```
AAX_ENUM_SIZE_CHECK (
    AAE_EAudioBufferLengthNative )
```

15.65.5.8 AAX_ENUM_SIZE_CHECK() [8/45]

```
AAX_ENUM_SIZE_CHECK (
    AAX_EMaxAudioSuiteTracks )
```

15.65.5.9 AAX_ENUM_SIZE_CHECK() [9/45]

```
AAX_ENUM_SIZE_CHECK (
    AAX_EStemFormat )
```

15.65.5.10 AAX_ENUM_SIZE_CHECK() [10/45]

```
AAX_ENUM_SIZE_CHECK (
    AAX_EPlugInCategory )
```

15.65.5.11 AAX_ENUM_SIZE_CHECK() [11/45]

```
AAX_ENUM_SIZE_CHECK (
    AAX_EPlugInStrings )
```

15.65.5.12 AAX_ENUM_SIZE_CHECK() [12/45]

```
AAX_ENUM_SIZE_CHECK (
    AAX_EMeterOrientation )
```

15.65.5.13 AAX_ENUM_SIZE_CHECK() [13/45]

```
AAX_ENUM_SIZE_CHECK (
    AAX_EMeterBallisticType )
```

15.65.5.14 AAX_ENUM_SIZE_CHECK() [14/45]

```
AAX_ENUM_SIZE_CHECK (
    AAX_EMeterType )
```

15.65.5.15 AAX_ENUM_SIZE_CHECK() [15/45]

```
AAX_ENUM_SIZE_CHECK (
    AAX_ECurveType )
```

15.65.5.16 AAX_ENUM_SIZE_CHECK() [16/45]

```
AAX_ENUM_SIZE_CHECK (
    AAX_EResourceType )
```

15.65.5.17 AAX_ENUM_SIZE_CHECK() [17/45]

```
AAX_ENUM_SIZE_CHECK (
    AAX_ENotificationEvent )
```

15.65.5.18 AAX_ENUM_SIZE_CHECK() [18/45]

```
AAX_ENUM_SIZE_CHECK (
    AAX_EHostModeBits )
```

15.65.5.19 AAX_ENUM_SIZE_CHECK() [19/45]

```
AAX_ENUM_SIZE_CHECK (
    AAX_EHostMode )
```

15.65.5.20 AAX_ENUM_SIZE_CHECK() [20/45]

```
AAX_ENUM_SIZE_CHECK (
    AAX_EPrivateDataOptions )
```

15.65.5.21 AAX_ENUM_SIZE_CHECK() [21/45]

```
AAX_ENUM_SIZE_CHECK (
    AAX_EConstraintLocationMask )
```

15.65.5.22 AAX_ENUM_SIZE_CHECK() [22/45]

```
AAX_ENUM_SIZE_CHECK (
    AAX_EConstraintTopology )
```

15.65.5.23 AAX_ENUM_SIZE_CHECK() [23/45]

```
AAX_ENUM_SIZE_CHECK (
    AAX_EComponentInstanceInitAction )
```

15.65.5.24 AAX_ENUM_SIZE_CHECK() [24/45]

```
AAX_ENUM_SIZE_CHECK (
    AAX_ESampleRateMask )
```


15.65.5.25 sampleRateInMask()

```
uint8_t sampleRateInMask (
    float inSR,
    uint32_t iMask ) [inline]
```

Determines whether a particular [AAX_CSAMPLE_RATE](#) is present in a given mask of [AAX_ESAMPLE_RATE_MASK](#).

See also

[kAAX_Property_SampleRate](#)

References [AAX_eSampleRateMask_176400](#), [AAX_eSampleRateMask_192000](#), [AAX_eSampleRateMask_44100](#), [AAX_eSampleRateMask_48000](#), [AAX_eSampleRateMask_88200](#), and [AAX_eSampleRateMask_96000](#).

15.65.5.26 getLowestSampleRateInMask()

```
float getLowestSampleRateInMask (
    uint32_t iMask ) [inline]
```

Converts from a mask of [AAX_ESAMPLE_RATE_MASK](#) to the lowest supported [AAX_CSAMPLE_RATE](#) value in Hz.

References [AAX_eSampleRateMask_176400](#), [AAX_eSampleRateMask_192000](#), [AAX_eSampleRateMask_44100](#), [AAX_eSampleRateMask_48000](#), [AAX_eSampleRateMask_88200](#), and [AAX_eSampleRateMask_96000](#).

15.65.5.27 getMaskForSampleRate()

```
uint32_t getMaskForSampleRate (
    float inSR ) [inline]
```

Returns the [AAX_ESAMPLE_RATE_MASK](#) selector for a literal sample rate.

The given rate must be an exact match with one of the available selectors. If no exact match is found then [AAX_eSampleRateMask_No](#) is returned.

References [AAX_eSampleRateMask_176400](#), [AAX_eSampleRateMask_192000](#), [AAX_eSampleRateMask_44100](#), [AAX_eSampleRateMask_48000](#), [AAX_eSampleRateMask_88200](#), [AAX_eSampleRateMask_96000](#), and [AAX_eSampleRateMask_No](#).

15.65.5.28 AAX_ENUM_SIZE_CHECK() [25/45]

```
AAX_ENUM_SIZE_CHECK (
    AAX_EParameterType )
```

15.65.5.29 AAX_ENUM_SIZE_CHECK() [26/45]

```
AAX_ENUM_SIZE_CHECK (
    AAX_EParameterOrientationBits )
```

15.65.5.30 AAX_ENUM_SIZE_CHECK() [27/45]

```
AAX_ENUM_SIZE_CHECK (
    AAX_EParameterValueInfoSelector )
```

15.65.5.31 AAX_ENUM_SIZE_CHECK() [28/45]

```
AAX_ENUM_SIZE_CHECK (
    AAX_EEQBandTypes )
```

15.65.5.32 AAX_ENUM_SIZE_CHECK() [29/45]

```
AAX_ENUM_SIZE_CHECK (
    AAX_EEQInCircuitPolarity )
```

15.65.5.33 AAX_ENUM_SIZE_CHECK() [30/45]

```
AAX_ENUM_SIZE_CHECK (
    AAX_EUseAlternateControl )
```

15.65.5.34 AAX_ENUM_SIZE_CHECK() [31/45]

```
AAX_ENUM_SIZE_CHECK (
    AAX_EMIDINodeType )
```

15.65.5.35 AAX_ENUM_SIZE_CHECK() [32/45]

```
AAX_ENUM_SIZE_CHECK (
    AAX_EUpdateSource )
```

15.65.5.36 AAX_ENUM_SIZE_CHECK() [33/45]

```
AAX_ENUM_SIZE_CHECK (
    AAX_EDataInPortType )
```

15.65.5.37 AAX_ENUM_SIZE_CHECK() [34/45]

```
AAX_ENUM_SIZE_CHECK (
    AAX_EFrameRate )
```

15.65.5.38 AAX_ENUM_SIZE_CHECK() [35/45]

```
AAX_ENUM_SIZE_CHECK (
    AAX_EFeetFramesRate )
```

15.65.5.39 AAX_ENUM_SIZE_CHECK() [36/45]

```
AAX_ENUM_SIZE_CHECK (
    AAX_EMidiGlobalNodeSelectors )
```

15.65.5.40 AAX_ENUM_SIZE_CHECK() [37/45]

```
AAX_ENUM_SIZE_CHECK (
    AAX_EPreviewState )
```

15.65.5.41 AAX_ENUM_SIZE_CHECK() [38/45]

```
AAX_ENUM_SIZE_CHECK (
    AAX_EProcessingState )
```

15.65.5.42 AAX_ENUM_SIZE_CHECK() [39/45]

```
AAX_ENUM_SIZE_CHECK (
    AAX_ETargetPlatform )
```

15.65.5.43 AAX_ENUM_SIZE_CHECK() [40/45]

```
AAX_ENUM_SIZE_CHECK (
    AAX_ESupportLevel )
```

15.65.5.44 AAX_ENUM_SIZE_CHECK() [41/45]

```
AAX_ENUM_SIZE_CHECK (
    AAX_EHostLevel )
```

15.65.5.45 AAX_ENUM_SIZE_CHECK() [42/45]

```
AAX_ENUM_SIZE_CHECK (
    AAX_ETextEncoding )
```

15.65.5.46 AAX_ENUM_SIZE_CHECK() [43/45]

```
AAX_ENUM_SIZE_CHECK (
    AAX_EAssertFlags )
```

15.65.5.47 AAX_ENUM_SIZE_CHECK() [44/45]

```
AAX_ENUM_SIZE_CHECK (
    AAX_ETransportState )
```

15.65.5.48 AAX_ENUM_SIZE_CHECK() [45/45]

```
AAX_ENUM_SIZE_CHECK (
    AAX_ERecordMode )
```

15.66 AAX_EnvironmentUtilities.h File Reference

```
#include <cstdlib>
```

15.66.1 Description

Useful environment definitions for AAX.

Namespaces

- [AAX](#)

15.67 AAX_Errors.h File Reference

```
#include "AAX_Enums.h"
```

15.67.1 Description

Definitions of error codes used by AAX plug-ins.

Enumerations

- enum [AAX_EError](#) {
 [AAX_SUCCESS](#) = 0 ,
 [AAX_ERROR_INVALID_PARAMETER_ID](#) = -20001 ,
 [AAX_ERROR_INVALID_STRING_CONVERSION](#) = -20002 ,
 [AAX_ERROR_INVALID_METER_INDEX](#) = -20003 ,
 [AAX_ERROR_NULL_OBJECT](#) = -20004 ,
 [AAX_ERROR_OLDER_VERSION](#) = -20005 ,
 [AAX_ERROR_INVALID_CHUNK_INDEX](#) = -20006 ,
 [AAX_ERROR_INVALID_CHUNK_ID](#) = -20007 ,
 [AAX_ERROR_INCORRECT_CHUNK_SIZE](#) = -20008 ,
 [AAX_ERROR_UNIMPLEMENTED](#) = -20009 ,
 [AAX_ERROR_INVALID_PARAMETER_INDEX](#) = -20010 ,
 [AAX_ERROR_NOT_INITIALIZED](#) = -20011 ,
 [AAX_ERROR_ACF_ERROR](#) = -20012 ,
 [AAX_ERROR_INVALID_METER_TYPE](#) = -20013 ,
 [AAX_ERROR_CONTEXT_ALREADY_HAS_METERS](#) = -20014 ,
 [AAX_ERROR_NULL_COMPONENT](#) = -20015 ,
 [AAX_ERROR_PORT_ID_OUT_OF_RANGE](#) = -20016 ,
 [AAX_ERROR_FIELD_TYPE_DOES_NOT_SUPPORT_DIRECT_ACCESS](#) = -20017 ,
 [AAX_ERROR_DIRECT_ACCESS_OUT_OF_BOUNDS](#) = -20018 ,
 [AAX_ERROR_FIFO_FULL](#) = -20019 ,
 [AAX_ERROR_INITIALIZING_PACKET_STREAM_THREAD](#) = -20020 ,
 [AAX_ERROR_POST_PACKET_FAILED](#) = -20021 ,
 [AAX_RESULT_PACKET_STREAM_NOT_EMPTY](#) = -20022 ,
 [AAX_RESULT_ADD_FIELD_UNSUPPORTED_FIELD_TYPE](#) = -20023 ,
 [AAX_ERROR_MIXER_THREAD_FALLING_BEHIND](#) = -20024 ,
 [AAX_ERROR_INVALID_FIELD_INDEX](#) = -20025 ,
 [AAX_ERROR_MALFORMED_CHUNK](#) = -20026 ,
 [AAX_ERROR_TOD_BEHIND](#) = -20027 ,
 [AAX_RESULT_NEW_PACKET_POSTED](#) = -20028 ,
 [AAX_ERROR_PLUGIN_NOT_AUTHORIZED](#) = -20029 ,
}

```

AAX_ERROR_PLUGIN_NULL_PARAMETER = -20030 ,
AAX_ERROR_NOTIFICATION_FAILED = -20031 ,
AAX_ERROR_INVALID_VIEW_SIZE = -20032 ,
AAX_ERROR_SIGNED_INT_OVERFLOW = -20033 ,
AAX_ERROR_NO_COMPONENTS = -20034 ,
AAX_ERROR_DUPLICATE_EFFECT_ID = -20035 ,
AAX_ERROR_DUPLICATE_TYPE_ID = -20036 ,
AAX_ERROR_EMPTY_EFFECT_NAME = -20037 ,
AAX_ERROR_UNKNOWN_PLUGIN = -20038 ,
AAX_ERROR_PROPERTY_UNDEFINED = -20039 ,
AAX_ERROR_INVALID_PATH = -20040 ,
AAX_ERROR_UNKNOWN_ID = -20041 ,
AAX_ERROR_UNKNOWN_EXCEPTION = -20042 ,
AAX_ERROR_INVALID_ARGUMENT = -20043 ,
AAX_ERROR_NULL_ARGUMENT = -20044 ,
AAX_ERROR_INVALID_INTERNAL_DATA = -20045 ,
AAX_ERROR_ARGUMENT_BUFFER_OVERFLOW = -20046 ,
AAX_ERROR_UNSUPPORTED_ENCODING = -20047 ,
AAX_ERROR_UNEXPECTED_EFFECT_ID = -20048 ,
AAX_ERROR_NO_ABBREVIATED_PARAMETER_NAME = -20049 ,
AAX_ERROR_ARGUMENT_OUT_OF_RANGE = -20050 ,
AAX_ERROR_PRINT_FAILURE = -20051 ,
AAX_ERROR_PLUGIN_BEGIN = -20600 ,
AAX_ERROR_PLUGIN_END = -21000 }

```

Functions

- [AAX_ENUM_SIZE_CHECK](#) ([AAX_EError](#))

15.67.2 Enumeration Type Documentation

15.67.2.1 AAX_EError

enum [AAX_EError](#)

[AAX](#) result codes

Enumerator

AAX_SUCCESS	
AAX_ERROR_INVALID_PARAMETER_ID	
AAX_ERROR_INVALID_STRING_CONVERSION	
AAX_ERROR_INVALID_METER_INDEX	
AAX_ERROR_NULL_OBJECT	
AAX_ERROR_OLDER_VERSION	
AAX_ERROR_INVALID_CHUNK_INDEX	
AAX_ERROR_INVALID_CHUNK_ID	
AAX_ERROR_INCORRECT_CHUNK_SIZE	
AAX_ERROR_UNIMPLEMENTED	
AAX_ERROR_INVALID_PARAMETER_INDEX	

Enumerator

AAX_ERROR_NOT_INITIALIZED	
AAX_ERROR_ACF_ERROR	
AAX_ERROR_INVALID_METER_TYPE	
AAX_ERROR_CONTEXT_ALREADY_HAS_↔ METERS	
AAX_ERROR_NULL_COMPONENT	
AAX_ERROR_PORT_ID_OUT_OF_RANGE	
AAX_ERROR_FIELD_TYPE_DOES_NOT_↔ SUPPORT_DIRECT_ACCESS	
AAX_ERROR_DIRECT_ACCESS_OUT_OF_↔ BOUNDS	
AAX_ERROR_FIFO_FULL	
AAX_ERROR_INITIALIZING_PACKET_STREAM_↔ THREAD	
AAX_ERROR_POST_PACKET_FAILED	
AAX_RESULT_PACKET_STREAM_NOT_EMPTY	
AAX_RESULT_ADD_FIELD_UNSUPPORTED_↔ FIELD_TYPE	
AAX_ERROR_MIXER_THREAD_FALLING_BEHIND	
AAX_ERROR_INVALID_FIELD_INDEX	
AAX_ERROR_MALFORMED_CHUNK	
AAX_ERROR_TOD_BEHIND	
AAX_RESULT_NEW_PACKET_POSTED	
AAX_ERROR_PLUGIN_NOT_AUTHORIZED	
AAX_ERROR_PLUGIN_NULL_PARAMETER	
AAX_ERROR_NOTIFICATION_FAILED	
AAX_ERROR_INVALID_VIEW_SIZE	
AAX_ERROR_SIGNED_INT_OVERFLOW	
AAX_ERROR_NO_COMPONENTS	
AAX_ERROR_DUPLICATE_EFFECT_ID	
AAX_ERROR_DUPLICATE_TYPE_ID	
AAX_ERROR_EMPTY_EFFECT_NAME	
AAX_ERROR_UNKNOWN_PLUGIN	
AAX_ERROR_PROPERTY_UNDEFINED	The plugin does not explicitly define a value for a queried property.
AAX_ERROR_INVALID_PATH	
AAX_ERROR_UNKNOWN_ID	
AAX_ERROR_UNKNOWN_EXCEPTION	An AAX plug-in should return this to the host if an unknown exception is caught. Exceptions should never be passed to the host.
AAX_ERROR_INVALID_ARGUMENT	One or more input parameters are invalid; all output parameters are left unchanged.
AAX_ERROR_NULL_ARGUMENT	One or more required pointer arguments are null.
AAX_ERROR_INVALID_INTERNAL_DATA	Some part of the internal data required by the method is invalid. See also AAX_ERROR_NOT_INITIALIZED
AAX_ERROR_ARGUMENT_BUFFER_OVERFLOW	A buffer argument was not large enough to hold the data which must be placed within it.

Enumerator

AAX_ERROR_UNSUPPORTED_ENCODING	Unsupported input argument text encoding.
AAX_ERROR_UNEXPECTED_EFFECT_ID	Encountered an effect ID with a different value from what was expected.
AAX_ERROR_NO_ABBREVIATED_PARAMETER_NAME	No parameter name abbreviation with the requested properties has been defined.
AAX_ERROR_ARGUMENT_OUT_OF_RANGE	One or more input parameters are out of the expected range, e.g. an index argument that is negative or exceeds the number of elements.
AAX_ERROR_PRINT_FAILURE	A failure occurred in a "print" library call such as <code>printf</code> .
AAX_ERROR_PLUGIN_BEGIN	Custom plug-in error codes may be placed in the range (AAX_ERROR_PLUGIN_END , AAX_ERROR_PLUGIN_BEGIN].
AAX_ERROR_PLUGIN_END	Custom plug-in error codes may be placed in the range (AAX_ERROR_PLUGIN_END , AAX_ERROR_PLUGIN_BEGIN].

15.67.3 Function Documentation

15.67.3.1 AAX_ENUM_SIZE_CHECK()

```
AAX_ENUM_SIZE_CHECK (
    AAX_EError )
```

15.68 AAX_Exception.h File Reference

```
#include "AAX_Assert.h"
#include "AAX_StringUtilities.h"
#include "AAX.h"
#include <exception>
#include <string>
#include <set>
```

15.68.1 Description

AAX SDK exception classes and utilities

Classes

- class [AAX::Exception::Any](#)
- class [AAX::Exception::ResultError](#)
- class [AAX_CheckedResult](#)
- class [AAX_AggregateResult](#)

Namespaces

- [AAX](#)
- [AAX::Exception](#)

AAX exception classes

Macros

- `#define AAX_SWALLOW(...)`
Executes X in a try/catch block that catches [AAX_CheckedResult](#) exceptions.
- `#define AAX_SWALLOW_MULT(...)`
Executes X in a try/catch block that catches [AAX_CheckedResult](#) exceptions.
- `#define AAX_CAPTURE(X, ...)`
Executes Y in a try/catch block that catches [AAX::Exception::ResultError](#) exceptions and captures the result.
- `#define AAX_CAPTURE_MULT(X, ...)`
Executes Y in a try/catch block that catches [AAX::Exception::ResultError](#) exceptions and captures the result.

Functions

- `std::string AAX::AsString (const char *inStr)`
- `const std::string & AAX::AsString (const std::string &inStr)`
- `const std::string & AAX::AsString (const Exception::Any &inStr)`

15.68.2 Macro Definition Documentation

15.68.2.1 AAX_SWALLOW

```
#define AAX_SWALLOW(  
    ...  
)
```

Value:

```
try { if(true) { ( __VA_ARGS__ ); } } \
catch (const AAX\_CheckedResult::Exception& AAX\_PREPROCESSOR\_CONCAT(ex,__LINE__)) { \
    AAX\_TRACE\_RELEASE(kAAX\_Trace\_Priority\_High, "%s line %d (%s) exception caught: %s (swallowed)",  
    __FILE__, __LINE__, __FUNCTION__, AAX\_PREPROCESSOR\_CONCAT(ex,__LINE__).What().c_str()); \
} do {} while (false)
```

Executes X in a try/catch block that catches [AAX_CheckedResult](#) exceptions.

Catches exceptions thrown from [AAX_CheckedResult](#) only - other exceptions require an explicit catch.

```
AAX\_CheckedResult cr;  
cr = NecessaryFunc1();  
AAX\_SWALLOW(cr = FailableFunc());  
cr = NecessaryFunc2();
```

15.68.2.2 AAX_SWALLOW_MULT

```
#define AAX_SWALLOW_MULT(
    ... )
```

Value:

```
try { if(true) { __VA_ARGS__ } } \
catch (const AAX_CheckedResult::Exception& AAX_PREPROCESSOR_CONCAT(ex,__LINE__)) { \
AAX_TRACE_RELEASE(kAAX_Trace_Priority_High, "%s line %d (%s) exception caught: %s (swallowed)", __FILE__,
    __LINE__, __FUNCTION__, AAX_PREPROCESSOR_CONCAT(ex,__LINE__).What().c_str()); \
} do {} while (false)
```

Executes *X* in a try/catch block that catches [AAX_CheckedResult](#) exceptions.

Version of [AAX_SWALLOW](#) for multi-line input.

Catches exceptions thrown from [AAX_CheckedResult](#) only - other exceptions require an explicit catch.

```
AAX_CheckedResult cr;
cr = NecessaryFunc();
AAX_SWALLOW_MULT(
    cr = FailableFunc1();
    cr = FailableFunc2(); // may not execute
    cr = FailableFunc3(); // may not execute
);
cr = NecessaryFunc2();
```

15.68.2.3 AAX_CAPTURE

```
#define AAX_CAPTURE(
    X,
    ... )
```

Value:

```
try { if(true) { ( __VA_ARGS__ ); } } \
catch (const AAX::Exception::ResultError& AAX_PREPROCESSOR_CONCAT(ex,__LINE__)) { \
AAX_TRACE_RELEASE(kAAX_Trace_Priority_High, "%s line %d (%s) exception caught: %s (captured)", __FILE__,
    __LINE__, __FUNCTION__, AAX_PREPROCESSOR_CONCAT(ex,__LINE__).What().c_str()); \
(X) = AAX_PREPROCESSOR_CONCAT(ex,__LINE__).Result(); \
} do {} while (false)
```

Executes *Y* in a try/catch block that catches [AAX::Exception::ResultError](#) exceptions and captures the result.

Catches exceptions thrown from [AAX_CheckedResult](#) and other [AAX::Exception::ResultError](#) exceptions.

X must be an [AAX_Result](#)

```
AAX_Result result = AAX_SUCCESS;
AAX_CAPTURE(result, ResultErrorThrowingFunc());
// result now holds the error code thrown by ThrowingFunc()
AAX_CheckedResult cr;
AAX_CAPTURE(result, cr = FailableFunc());
```

15.68.2.4 AAX_CAPTURE_MULT

```
#define AAX_CAPTURE_MULT(
    X,
    ... )
```

Value:

```
try { if(true) { __VA_ARGS__ } } \
catch (const AAX_CheckedResult::Exception& AAX_PREPROCESSOR_CONCAT(ex,__LINE__)) { \
AAX_TRACE_RELEASE(kAAX_Trace_Priority_High, "%s line %d (%s) exception caught: %s (captured)", __FILE__, \
    __LINE__, __FUNCTION__, AAX_PREPROCESSOR_CONCAT(ex,__LINE__).What().c_str()); \
(X) = AAX_PREPROCESSOR_CONCAT(ex,__LINE__).Result(); \
} do {} while (false)
```

Executes *Y* in a try/catch block that catches [AAX::Exception::ResultError](#) exceptions and captures the result.

Version of [AAX_CAPTURE](#) for multi-line input.

Catches exceptions thrown from [AAX_CheckedResult](#) and other [AAX::Exception::ResultError](#) exceptions.

X must be an [AAX_Result](#) or an implicitly convertible type

```
AAX_Result result = AAX_SUCCESS;
AAX_CAPTURE_MULT(result,
    MaybeThrowingFunc1();
    MaybeThrowingFunc2();

    // can use AAX_CheckedResult within AAX_CAPTURE_MULT
    AAX_CheckedResult cr;
    cr = FailableFunc1();
    cr = FailableFunc2();
    cr = FailableFunc3();
);
// result now holds the value of the last thrown error
return result;
```

15.69 AAX_Exports.cpp File Reference

```
#include "AAX_Init.h"
#include "AAX.h"
#include "acfunknown.h"
#include "acfresult.h"
```

Macros

- `#define AAX\_EXPORT extern "C" __declspec(dllexport) ACFRESULT __stdcall`

Functions

- [AAX_EXPORT ACFRegisterPlugin](#) ([IACFUnknown](#) *pUnkHostVoid, [IACFPluginDefinition](#) **ppPlugin↔
DefinitionVoid)
The main plug-in registration method.
- [AAX_EXPORT ACFRegisterComponent](#) ([IACFUnknown](#) *pUnkHost, [acfUInt32](#) index, [IACFComponent](#)↔
Definition **ppComponentDefinition)
Registers a specific component in the DLL.
- [AAX_EXPORT ACFGetClassFactory](#) ([IACFUnknown](#) *pUnkHost, const [acfCLSID](#) &clsid, const [acfIID](#) &iid,
void **ppOut)
Gets the factory for a given class ID.

- [AAX_EXPORT ACFCanUnloadNow](#) ([IACFUnknown](#) *pUnkHost)
Determines whether or not the host may unload the DLL.
- [AAX_EXPORT ACStartup](#) ([IACFUnknown](#) *pUnkHost)
DLL initialization routine.
- [AAX_EXPORT ACShutdown](#) ([IACFUnknown](#) *pUnkHost)
DLL shutdown routine.
- [AAX_EXPORT ACGetSDKVersion](#) (acfUInt64 *oSDKVersion)
Returns the DLL's SDK version.

15.69.1 Macro Definition Documentation

15.69.1.1 AAX_EXPORT

```
#define AAX_EXPORT extern "C" __declspec(dllexport) ACFRESULT __stdcall
```

15.69.2 Function Documentation

15.69.2.1 ACFRegisterPlugin()

```
ACFAPI ACFRegisterPlugin (
    IACFUnknown * pUnkHost,
    IACFPluginDefinition ** ppPluginDefinition )
```

The main plug-in registration method.

References [AAXRegisterPlugin\(\)](#).

Here is the call graph for this function:

15.69.2.2 ACFRegisterComponent()

```
ACFAPI ACFRegisterComponent (
    IACFUnknown * pUnkHost,
    acfUInt32 index,
    IACFComponentDefinition ** ppComponentDefinition )
```

Registers a specific component in the DLL.

References [AAXRegisterComponent\(\)](#).

Here is the call graph for this function:

15.69.2.3 ACFGetClassFactory()

```
ACFAPI ACFGetClassFactory (
    IACFUnknown * pUnkHost,
    const acfCLSID & clsid,
    const acfIID & iid,
    void ** ppOut )
```

Gets the factory for a given class ID.

References AAXGetClassFactory().

Here is the call graph for this function:

15.69.2.4 ACFCanUnloadNow()

```
ACFAPI ACFCanUnloadNow (
    IACFUnknown * pUnkHost )
```

Determines whether or not the host may unload the DLL.

References AAXCanUnloadNow().

Here is the call graph for this function:

15.69.2.5 ACFStartup()

```
ACFAPI ACFStartup (
    IACFUnknown * pUnkHost )
```

DLL initialization routine.

References AAXStartup().

Here is the call graph for this function:

15.69.2.6 ACFShtutdown()

```
ACFAPI ACFShtutdown (
    IACFUnknown * pUnkHost )
```

DLL shutdown routine.

References AAXShutdown().

Here is the call graph for this function:

15.69.2.7 ACFGetSDKVersion()

```
ACF_API ACFGetSDKVersion (
    acfUInt64 * oSDKVersion )
```

Returns the DLL's SDK version.

References AAXGetSDKVersion().

Here is the call graph for this function:

15.70 AAX_FastInterpolatedTableLookup.h File Reference

```
#include "AAX_Quantize.h"
#include <AAX_ALIGN_FILE_BEGIN>
#include <AAX_ALIGN_FILE_ALG>
#include <AAX_ALIGN_FILE_END>
#include <AAX_ALIGN_FILE_RESET>
#include "AAX_FastInterpolatedTableLookup.hpp"
```

15.70.1 Description

A set of functions that provide lookup table functionality. Not necessarily optimized for TI, but used internally.

Classes

- class [AAX_FastInterpolatedTableLookup< TFLOAT, DFLOAT >](#)

Macros

- `#define` [AAX_FASTINTERPOLATEDTABLELOOKUP_H](#)

15.70.2 Macro Definition Documentation

15.70.2.1 AAX_FASTINTERPOLATEDTABLELOOKUP_H

```
#define AAX_FASTINTERPOLATEDTABLELOOKUP_H
```

15.71 AAX_FastInterpolatedTableLookup.hpp File Reference

```
#include "AAX_Quantize.h"
```

15.72 AAX_FastPow.h File Reference

```
#include <cmath>
#include "AAX.h"
```

15.72.1 Description

Set of functions to optimize pow.

To use:

```
const int kPowTableExtent = 9;           // Lower values are less precise. 9 is the maximum
float powTableH[kPowTableSize];
float powTableL[kPowTableSize];
int radix = 2;                           // This should be whatever radix you want. Ie: radix ^ exp
PowFastSetTable( powTableH, kPowExtent, kPowExtent, false ); // Set the high table
PowFastSetTable( powTableL, kPowExtent*2, kPowExtent, true ); // Set the low table
..
result = powFastLookup(exp, log(2) / log(radix), powTableH, powTableL);
```

Namespaces

- [AAX](#)

Macros

- `#define \_AAX\_FASTPOW\_H\_`

Variables

- const unsigned int [AAX::kPowExtent](#) = 9
- const unsigned int [AAX::kPowTableSize](#) = 1 << kPowExtent

15.72.2 Macro Definition Documentation

15.72.2.1 [_AAX_FASTPOW_H_](#)

```
#define \_AAX\_FASTPOW\_H\_
```

15.73 AAX_Getting_Started_Guide.doxygen File Reference

15.74 AAX_GUITypes.h File Reference

```
#include "AAX.h"
#include "AAX_PreStructAlignmentHelper.h"
#include "AAX_Push2ByteStructAlignment.h"
#include "AAX_PostStructAlignmentHelper.h"
#include "AAX_PopStructAlignment.h"
```

15.74.1 Description

Constants and other definitions used by AAX plug-in GUIs.

Classes

- struct [AAX_Point](#)
Data structure representing a two-dimensional coordinate point.
- struct [AAX_Rect](#)
Data structure representing a rectangle in a two-dimensional coordinate plane.

Typedefs

- typedef struct [AAX_Point](#) [AAX_Point](#)
Data structure representing a two-dimensional coordinate point.
- typedef struct [AAX_Rect](#) [AAX_Rect](#)
Data structure representing a rectangle in a two-dimensional coordinate plane.
- typedef enum [AAX_EViewContainer_Type](#) [AAX_EViewContainer_Type](#)
Type of [view container](#).

Enumerations

- enum [AAX_EViewContainer_Type](#) {
[AAX_eViewContainer_Type_NULL](#) = 0 ,
[AAX_eViewContainer_Type_NSView](#) = 1 ,
[AAX_eViewContainer_Type_UIView](#) = 2 ,
[AAX_eViewContainer_Type_HWND](#) = 3 }
Type of [view container](#).

Functions

- bool [operator==](#) (const [AAX_Point](#) &p1, const [AAX_Point](#) &p2)
- bool [operator!=](#) (const [AAX_Point](#) &p1, const [AAX_Point](#) &p2)
- bool [operator<](#) (const [AAX_Point](#) &p1, const [AAX_Point](#) &p2)
- bool [operator<=](#) (const [AAX_Point](#) &p1, const [AAX_Point](#) &p2)
- bool [operator>](#) (const [AAX_Point](#) &p1, const [AAX_Point](#) &p2)
- bool [operator>=](#) (const [AAX_Point](#) &p1, const [AAX_Point](#) &p2)
- bool [operator==](#) (const [AAX_Rect](#) &r1, const [AAX_Rect](#) &r2)
- bool [operator!=](#) (const [AAX_Rect](#) &r1, const [AAX_Rect](#) &r2)
- [AAX_ENUM_SIZE_CHECK](#) ([AAX_EViewContainer_Type](#))

15.74.2 Typedef Documentation

15.74.2.1 AAX_Point

```
typedef struct AAX_Point AAX_Point
```

Data structure representing a two-dimensional coordinate point.

Comparison operators give preference to `vert`

15.74.2.2 AAX_Rect

```
typedef struct AAX_Rect AAX_Rect
```

Data structure representing a rectangle in a two-dimensional coordinate plane.

15.74.2.3 AAX_EViewContainer_Type

```
typedef enum AAX_EViewContainer_Type AAX_EViewContainer_Type
```

Type of [view container](#).

See also

[AAX_IViewContainer::GetType\(\)](#)

15.74.3 Enumeration Type Documentation

15.74.3.1 AAX_EViewContainer_Type

```
enum AAX_EViewContainer_Type
```

Type of [view container](#).

See also

[AAX_IViewContainer::GetType\(\)](#)

Enumerator

AAX_eViewContainer_Type_NULL	
AAX_eViewContainer_Type_NSView	
AAX_eViewContainer_Type_UIView	
AAX_eViewContainer_Type_HWND	

15.74.4 Function Documentation

15.74.4.1 operator==() [1/2]

```
bool operator== (
    const AAX_Point & p1,
    const AAX_Point & p2 ) [inline]
```

References AAX_Point::horz, and AAX_Point::vert.

15.74.4.2 operator!=() [1/2]

```
bool operator!= (
    const AAX_Point & p1,
    const AAX_Point & p2 ) [inline]
```

15.74.4.3 operator<()

```
bool operator< (
    const AAX_Point & p1,
    const AAX_Point & p2 ) [inline]
```

References AAX_Point::horz, and AAX_Point::vert.

15.74.4.4 operator<=()

```
bool operator<= (
    const AAX_Point & p1,
    const AAX_Point & p2 ) [inline]
```

References AAX_Point::horz, and AAX_Point::vert.

15.74.4.5 operator>()

```
bool operator> (
    const AAX_Point & p1,
    const AAX_Point & p2 ) [inline]
```

15.74.4.6 operator>=()

```
bool operator>= (
    const AAX_Point & p1,
    const AAX_Point & p2 ) [inline]
```

15.74.4.7 operator==() [2/2]

```
bool operator== (
    const AAX_Rect & r1,
    const AAX_Rect & r2 ) [inline]
```

References AAX_Rect::height, AAX_Rect::left, AAX_Rect::top, and AAX_Rect::width.

15.74.4.8 operator!=() [2/2]

```
bool operator!= (
    const AAX_Rect & r1,
    const AAX_Rect & r2 ) [inline]
```

15.74.4.9 AAX_ENUM_SIZE_CHECK()

```
AAX_ENUM_SIZE_CHECK (
    AAX_EViewContainer_Type )
```

15.75 AAX_HostSupport.doxygen File Reference

15.76 AAX_IACFAutomationDelegate.h File Reference

```
#include "AAX.h"
#include "acfunknown.h"
```

15.76.1 Description

Versioned interface allowing an AAX plug-in to interact with the host's automation system.

Classes

- class [AAX_IACFAutomationDelegate](#)

Versioned interface allowing an AAX plug-in to interact with the host's automation system.

15.77 AAX_IACFCollection.h File Reference

```
#include "acfbaseapi.h"
```

15.77.1 Description

Versioned interface to represent a plug-in binary's static description.

Classes

- class [AAX_IACFCollection](#)

Versioned interface to represent a plug-in binary's static description.

15.78 AAX_IACFComponentDescriptor.h File Reference

```
#include "AAX.h"  
#include "AAX_Callbacks.h"  
#include "AAX_IDma.h"  
#include "acfunknown.h"
```

15.78.1 Description

Versioned description interface for an AAX plug-in algorithm callback.

Classes

- class [AAX_IACFComponentDescriptor](#)

Versioned description interface for an AAX plug-in algorithm callback.

- class [AAX_IACFComponentDescriptor_V2](#)

Versioned description interface for an AAX plug-in algorithm callback.

- class [AAX_IACFComponentDescriptor_V3](#)

Versioned description interface for an AAX plug-in algorithm callback.

15.79 AAX_IACFController.h File Reference

```
#include "AAX.h"  
#include "acfunknown.h"
```

15.79.1 Description

Interface for the AAX host's view of a single instance of an effect. Used by both clients of the AAXHost and by effect components.

Classes

- class [AAX_IACFController](#)
Interface for the AAX host's view of a single instance of an effect. Used by both clients of the AAXHost and by effect components.
- class [AAX_IACFController_V2](#)
Interface for the AAX host's view of a single instance of an effect. Used by both clients of the AAXHost and by effect components.
- class [AAX_IACFController_V3](#)
Interface for the AAX host's view of a single instance of an effect. Used by both clients of the AAXHost and by effect components.

15.80 AAX_IACFDataBuffer.h File Reference

```
#include "AAX.h"  
#include "acfunknown.h"
```

Classes

- class [AAX_IACFDataBuffer](#)
Versioned interface for reference counted data buffers.

Macros

- `#define` [AAX_IACFDataBuffer_H](#)

15.80.1 Macro Definition Documentation

15.80.1.1 AAX_IACFDataBuffer_H

```
#define AAX_IACFDataBuffer_H
```

15.81 AAX_IACFDescriptionHost.h File Reference

```
#include "AAX.h"  
#include "acfbaseapi.h"  
#include "acfunknown.h"
```

Classes

- class [AAX_IACFDescriptionHost](#)

15.82 AAX_IACFEffector.h File Reference

```
#include "AAX.h"
#include "AAX_Callbacks.h"
#include "acfunknown.h"
```

15.82.1 Description

Versioned interface for an [AAX_IEffectDescriptor](#).

Classes

- class [AAX_IACFEffector](#)
Versioned interface for an [AAX_IEffectDescriptor](#).
- class [AAX_IACFEffector_V2](#)
Versioned interface for an [AAX_IEffectDescriptor](#).

15.83 AAX_IACFEffectorDirectData.h File Reference

```
#include "AAX.h"
#include "acfunknown.h"
```

15.83.1 Description

The direct data access interface that gets exposed to the host application.

Classes

- class [AAX_IACFEffectorDirectData](#)
Optional interface for direct access to a plug-in's alg memory.
- class [AAX_IACFEffectorDirectData_V2](#)

15.84 AAX_IACFEffectorGUI.h File Reference

```
#include "AAX.h"
#include "AAX_GUITypes.h"
#include "AAX_IString.h"
#include "acfunknown.h"
```

15.84.1 Description

The GUI interface that gets exposed to the host application.

Classes

- class [AAX_IACFEfectGUI](#)

The interface for a AAX Plug-in's GUI (graphical user interface).

15.85 AAX_IACFEfectParameters.h File Reference

```
#include "AAX.h"
#include "acfunknown.h"
```

15.85.1 Description

The data model interface that is exposed to the host application.

Classes

- class [AAX_IACFEfectParameters](#)
The interface for an AAX Plug-in's data model.
- struct [AAX_SHybridRenderInfo](#)
Hybrid render processing context.
- class [AAX_IACFEfectParameters_V2](#)
Supplemental interface for an AAX Plug-in's data model.
- class [AAX_IACFEfectParameters_V3](#)
Supplemental interface for an AAX Plug-in's data model.
- class [AAX_IACFEfectParameters_V4](#)
Supplemental interface for an AAX Plug-in's data model.

15.86 AAX_IACFFeatureInfo.h File Reference

```
#include "AAX.h"
#include "acfunknown.h"
```

Classes

- class [AAX_IACFFeatureInfo](#)

15.87 AAX_IACFHostProcessor.h File Reference

```
#include "AAX.h"  
#include "acfunknown.h"
```

15.87.1 Description

The host processor interface that is exposed to the host application.

Classes

- class [AAX_IACFHostProcessor](#)
Versioned interface for an AAX host processing component.
- class [AAX_IACFHostProcessor_V2](#)
Supplemental interface for an AAX host processing component.

15.88 AAX_IACFHostProcessorDelegate.h File Reference

```
#include "AAX.h"  
#include "acfunknown.h"
```

Classes

- class [AAX_IACFHostProcessorDelegate](#)
Versioned interface for host methods specific to offline processing.
- class [AAX_IACFHostProcessorDelegate_V2](#)
Versioned interface for host methods specific to offline processing.
- class [AAX_IACFHostProcessorDelegate_V3](#)
Versioned interface for host methods specific to offline processing.

15.89 AAX_IACFHostServices.h File Reference

```
#include "AAX.h"  
#include "acfunknown.h"
```

Classes

- class [AAX_IACFHostServices](#)
Versioned interface to diagnostic and debugging services provided by the AAX host.
- class [AAX_IACFHostServices_V2](#)
V2 of versioned interface to diagnostic and debugging services provided by the AAX host.
- class [AAX_IACFHostServices_V3](#)
V3 of versioned interface to diagnostic and debugging services provided by the AAX host.

15.90 AAX_IACFPageTable.h File Reference

```
#include "AAX.h"  
#include "acfunknown.h"
```

Classes

- class [AAX_IACFPageTable](#)
Versioned interface to the host's representation of a plug-in instance's page table.
- class [AAX_IACFPageTable_V2](#)
Versioned interface to the host's representation of a plug-in instance's page table.

15.91 AAX_IACFPageTableController.h File Reference

```
#include "AAX.h"  
#include "acfunknown.h"
```

Classes

- class [AAX_IACFPageTableController](#)
Interface for host operations related to the page tables for this plug-in.
- class [AAX_IACFPageTableController_V2](#)
Interface for host operations related to the page tables for this plug-in.

15.92 AAX_IACFPrivateDataAccess.h File Reference

```
#include "AAX.h"  
#include "acfunknown.h"
```

15.92.1 Description

Interface for the AAX host's data access functionality.

Classes

- class [AAX_IACFPrivateDataAccess](#)
Interface for the AAX host's data access functionality.

15.93 AAX_IACFPropertyMap.h File Reference

```
#include "AAX.h"  
#include "acfunknown.h"
```

15.93.1 Description

Versioned interface for an [AAX_IPropertyMap](#).

Classes

- class [AAX_IACFPropertyMap](#)
Versioned interface for an [AAX_IPropertyMap](#).
- class [AAX_IACFPropertyMap_V2](#)
Versioned interface for an [AAX_IPropertyMap](#).
- class [AAX_IACFPropertyMap_V3](#)
Versioned interface for an [AAX_IPropertyMap](#).

15.94 AAX_IACFSessionDocument.h File Reference

```
#include "AAX_UIDs.h"  
#include "AAX.h"  
#include "acfunknown.h"
```

Classes

- class [AAX_IACFSessionDocument](#)
Interface representing information in a host session document.

Macros

- `#define` [AAX_IACFSessionDocument_H](#)

15.94.1 Macro Definition Documentation

15.94.1.1 AAX_IACFSessionDocument_H

```
#define AAX_IACFSessionDocument_H
```

15.95 AAX_IACFSessionDocumentClient.h File Reference

```
#include "AAX_UIDs.h"
#include "AAX.h"
#include "acfunknown.h"
```

Classes

- class [AAX_IACFSessionDocumentClient](#)
Interface representing a client of the session document interface.

Macros

- #define [AAX_IACFSessionDocumentClient_H](#)

15.95.1 Macro Definition Documentation

15.95.1.1 AAX_IACFSessionDocumentClient_H

```
#define AAX_IACFSessionDocumentClient_H
```

15.96 AAX_IACFTask.h File Reference

```
#include "AAX.h"
#include "acfunknown.h"
```

15.96.1 Description

Defines the interface representing an asynchronous task.

Classes

- class [AAX_IACFTask](#)
Versioned interface for an asynchronous task.

Macros

- #define [AAX_IACFTask_H](#)

Enumerations

- enum class [AAX_TaskCompletionStatus](#) : int32_t {
 [None](#) = 0 ,
 [Done](#) = 1 ,
 [Canceled](#) = 2 ,
 [Error](#) = 3 }

15.96.2 Macro Definition Documentation

15.96.2.1 AAX_IACFTask_H

```
#define AAX_IACFTask_H
```

15.97 AAX_IACFTaskAgent.h File Reference

```
#include "AAX.h"  
#include "acfunknown.h"
```

Classes

- class [AAX_IACFTaskAgent](#)
Versioned interface for a component that accepts task requests.

Macros

- #define [AAX_IACFTaskAgent_H](#)

15.97.1 Macro Definition Documentation

15.97.1.1 AAX_IACFTaskAgent_H

```
#define AAX_IACFTaskAgent_H
```

15.98 AAX_IACFTTransport.h File Reference

```
#include "AAX.h"  
#include "AAX_Enums.h"  
#include "acfunknown.h"
```

15.98.1 Description

Interface for accessing the host's transport state.

Classes

- class [AAX_IACFTransport](#)
Versioned interface to get information about the host's transport state.
- class [AAX_IACFTransport_V2](#)
Versioned interface to get information about the host's transport state.
- class [AAX_IACFTransport_V3](#)
Versioned interface to get information about the host's transport state.
- class [AAX_IACFTransport_V4](#)
Versioned interface to get information about the host's transport state.
- class [AAX_IACFTransport_V5](#)
Versioned interface to get information about the host's transport state.

15.99 AAX_IACFTransportControl.h File Reference

```
#include "AAX.h"  
#include "AAX_Enums.h"  
#include "acfunknown.h"
```

15.99.1 Description

Interface for control over the host's transport state.

Classes

- class [AAX_IACFTransportControl](#)
Versioned interface to control the host's transport state.

15.100 AAX_IACFViewContainer.h File Reference

```
#include "AAX_GUITypes.h"  
#include "AAX.h"  
#include "acfunknown.h"
```

15.100.1 Description

Interface for the AAX host's view of a single instance of an effect. Used by both clients of the AAXHost and by effect components.

Classes

- class [AAX_IACFViewContainer](#)
Interface for the AAX host's view of a single instance of an effect. Used by both clients of the host app and by effect components.
- class [AAX_IACFViewContainer_V2](#)
Supplemental interface for the AAX host's view of a single instance of an effect. Used by both clients of the host app and by effect components.
- class [AAX_IACFViewContainer_V3](#)
Additional methods to track mouse as it moves over controls.

15.101 AAX_IAutomationDelegate.h File Reference

```
#include "AAX.h"
```

15.101.1 Description

Interface allowing an AAX plug-in to interact with the host's automation system.

Classes

- class [AAX_IAutomationDelegate](#)
Interface allowing an AAX plug-in to interact with the host's event system.

15.102 AAX_ICollection.h File Reference

```
#include "AAX.h"
```

15.102.1 Description

Interface to represent a plug-in binary's static description.

Classes

- class [AAX_ICollection](#)
Interface to represent a plug-in binary's static description.

15.103 AAX_IComponentDescriptor.h File Reference

```
#include "AAX.h"  
#include "AAX_IDma.h"  
#include "AAX_Callbacks.h"
```

15.103.1 Description

Description interface for an AAX plug-in algorithm.

Classes

- class [AAX_IComponentDescriptor](#)
Description interface for an AAX plug-in component.

15.104 AAX_IContainer.h File Reference

15.104.1 Description

Abstract container interface.

Classes

- class [AAX_IContainer](#)

15.105 AAX_IController.h File Reference

```
#include "AAX_Properties.h"  
#include "AAX_IString.h"  
#include "AAX.h"  
#include <memory>
```

15.105.1 Description

Interface for the AAX host's view of a single instance of an effect.

Classes

- class [AAX_IController](#)
Interface for the AAX host's view of a single instance of an effect. Used by both clients of the AAX host and by effect components.

15.106 AAX_IDataBuffer.h File Reference

```
#include "AAX_IACFDataBuffer.h"  
#include "AAX.h"  
#include "CACFUnknown.h"  
#include "AAX_UIDs.h"  
#include "acfextras.h"
```

Classes

- class [AAX_IDataBuffer](#)
Interface for reference counted data buffers.

Macros

- #define [AAX_IDataBuffer_H](#)

15.106.1 Macro Definition Documentation

15.106.1.1 AAX_IDataBuffer_H

```
#define AAX_IDataBuffer_H
```

15.107 AAX_IDataBufferWrapper.h File Reference

```
#include "AAX.h"
```

Classes

- class [AAX_IDataBufferWrapper](#)
Wrapper for an [AAX_IDataBuffer](#).

Macros

- #define [AAX_IDATABUFFERWRAPPER_H](#)

15.107.1 Macro Definition Documentation

15.107.1.1 AAX_IDATABUFFERWRAPPER_H

```
#define AAX_IDATABUFFERWRAPPER_H
```

15.108 AAX_IDescriptionHost.h File Reference

```
#include "AAX.h"
```


Classes

- class [AAX_IDescriptionHost](#)

15.109 AAX_IDisplayDelegate.h File Reference

```
#include "AAX.h"
```

15.109.1 Description

Defines the display behavior for a parameter.

Classes

- class [AAX_IDisplayDelegateBase](#)
Defines the display behavior for a parameter.
- class [AAX_IDisplayDelegate< T >](#)
Classes for parameter value string conversion.

15.110 AAX_IDisplayDelegateDecorator.h File Reference

```
#include "AAX_IDisplayDelegate.h"
```

15.110.1 Description

The base class for all concrete display delegate decorators.

Classes

- class [AAX_IDisplayDelegateDecorator< T >](#)
The base class for all concrete display delegate decorators.

15.111 AAX_IDma.h File Reference

```
#include "AAX.h"
```

15.111.1 Description

Cross-platform interface for access to the host's direct memory access (DMA) facilities.

Classes

- class [AAX_IDma](#)

Cross-platform interface for access to the host's direct memory access (DMA) facilities.

Macros

- #define [AAX_IDMA_H](#)
- #define [AAX_DMA_API](#)

15.111.2 Macro Definition Documentation

15.111.2.1 AAX_IDMA_H

```
#define AAX_IDMA_H
```

15.111.2.2 AAX_DMA_API

```
#define AAX_DMA_API
```

15.112 AAX_IEffectDescriptor.h File Reference

```
#include "AAX.h"  
#include "AAX_Callbacks.h"
```

15.112.1 Description

Description interface for an effect's (plug-in type's) components.

Classes

- class [AAX_IEffectDescriptor](#)

Description interface for an effect's (plug-in type's) components.

15.113 AAX_IEffectDirectData.h File Reference

```
#include "AAX_IACFEffectDirectData.h"  
#include "AAX.h"  
#include "CACFUnknown.h"
```

15.113.1 Description

Optional interface for direct access to alg memory.

Classes

- class [AAX_IEffectDirectData](#)

The interface for a AAX Plug-in's direct data interface.

15.114 AAX_IEffectGUI.h File Reference

```
#include "AAX_IACFEEffectGUI.h"
#include "AAX.h"
#include "CACFUnknown.h"
```

15.114.1 Description

The interface for a AAX Plug-in's user interface.

Classes

- class [AAX_IEffectGUI](#)

The interface for a AAX Plug-in's user interface.

15.115 AAX_IEffectParameters.h File Reference

```
#include "AAX_IACFEEffectParameters.h"
#include "AAX.h"
#include "CACFUnknown.h"
```

15.115.1 Description

The interface for an AAX Plug-in's data model.

Classes

- class [AAX_IEffectParameters](#)

The interface for an AAX Plug-in's data model.

15.116 AAX_IFeatureInfo.h File Reference

```
#include "AAX.h"
```

Classes

- class [AAX_IFeatureInfo](#)

15.117 AAX_IHostProcessor.h File Reference

```
#include "AAX_IACFHostProcessor.h"  
#include "AAX.h"  
#include "CACFUnknown.h"
```

15.117.1 Description

Base class for the host processor interface which is extended by plugin code.

Classes

- class [AAX_IHostProcessor](#)
Base class for the host processor interface.

15.118 AAX_IHostProcessorDelegate.h File Reference

```
#include "AAX.h"
```

15.118.1 Description

Interface allowing plug-in's HostProcessor to interact with the host's side.

Classes

- class [AAX_IHostProcessorDelegate](#)
Versioned interface for host methods specific to offline processing.

15.119 AAX_IHostServices.h File Reference

```
#include "AAX.h"
```

15.119.1 Description

Various host services.

Classes

- class [AAX_IHostServices](#)

Interface to diagnostic and debugging services provided by the AAX host.

15.120 AAX_IHostTaskAgent.h File Reference

```
#include "AAX.h"  
#include "CACFUnknown.h"
```

15.120.1 Description

Interface to access an [AAX_IACFTaskAgent](#) object implemented by the host.

Version-managed concrete HostTaskAgent class.

Classes

- class [AAX_IHostTaskAgent](#)

Interface to access an [AAX_IACFTaskAgent](#) object implemented by the host.

Macros

- #define [AAX_IHostTaskAgent_H](#)

15.120.2 Macro Definition Documentation

15.120.2.1 AAX_IHostTaskAgent_H

```
#define AAX_IHostTaskAgent_H
```

15.121 AAX_IMIDINode.h File Reference

```
#include "AAX.h"  
#include "AAX_ITransport.h"
```

15.121.1 Description

Declaration of the base MIDI Node interface.

Author

by Andriy Goshko

Classes

- class [AAX_IMIDINode](#)
Interface for accessing information in a MIDI node.

15.122 AAX_Init.h File Reference

```
#include "AAX.h"
#include "acfbasetypes.h"
```

15.122.1 Description

AAX library implementations of required plug-in initialization, registration, and tear-down methods.

Functions

- [AAX_Result AAXRegisterPlugin](#) ([IACFUnknown](#) *pUnkHost, [IACFPluginDefinition](#) **ppPluginDefinition)
The main plug-in registration method.
- [AAX_Result AAXRegisterComponent](#) ([IACFUnknown](#) *pUnkHost, [acfUInt32](#) index, [IACFComponentDefinition](#) **ppComponentDefinition)
Registers a specific component in the DLL.
- [AAX_Result AAXGetClassFactory](#) ([IACFUnknown](#) *pUnkHost, const [acfCLSID](#) &clsid, const [acfIID](#) &iid, void **ppOut)
Gets the factory for a given class ID.
- [AAX_Result AAXCanUnloadNow](#) ([IACFUnknown](#) *pUnkHost)
Determines whether or not the host may unload the DLL.
- [AAX_Result AAXStartup](#) ([IACFUnknown](#) *pUnkHost)
DLL initialization routine.
- [AAX_Result AAXShutdown](#) ([IACFUnknown](#) *pUnkHost)
DLL shutdown routine.
- [AAX_Result AAXGetSDKVersion](#) ([acfUInt64](#) *oSDKVersion)
Returns the DLL's SDK version.

15.122.2 Function Documentation

15.122.2.1 AAXRegisterComponent()

```
AAX_Result AAXRegisterComponent (
    IACFUnknown * pUnkHost,
    acfUInt32 index,
    IACFComponentDefinition ** ppComponentDefinition )
```

Registers a specific component in the DLL.

The implementation of this method in the AAX library simply sets *ppComponentDefinition to NULL and returns [AAX_SUCCESS](#).

Wrapped by [ACFRegisterComponent\(\)](#)

Referenced by [ACFRegisterComponent\(\)](#).

Here is the caller graph for this function:

15.122.2.2 AAXGetClassFactory()

```
AAX_Result AAXGetClassFactory (
    IACFUnknown * pUnkHost,
    const acfCLSID & clsid,
    const acfIID & iid,
    void ** ppOut )
```

Gets the factory for a given class ID.

This method is required by ACF but is not supported by [AAX](#). Therefore the implementation of this method in the AAX library simply sets *ppOut to NULL and returns [AAX_ERROR_UNIMPLEMENTED](#).

Wrapped by [ACFGetClassFactory\(\)](#)

Referenced by [ACFGetClassFactory\(\)](#).

Here is the caller graph for this function:

15.122.2.3 AAXCanUnloadNow()

```
AAX_Result AAXCanUnloadNow (
    IACFUnknown * pUnkHost )
```

Determines whether or not the host may unload the DLL.

The implementation of this method in the AAX library returns the result of `GetActiveObjectCount()` as an [AAX_Result](#), with zero active objects interpreted as [AAX_SUCCESS](#) (see [CACFUnknown.h](#))

Wrapped by [ACFCanUnloadNow\(\)](#)

Referenced by [ACFCanUnloadNow\(\)](#).

Here is the caller graph for this function:

15.122.2.4 AAXStartup()

```
AAX_Result AAXStartup (
    IACFUnknown * pUnkHost )
```

DLL initialization routine.

Called once at init time. The implementation of this method in the AAX library uses `pUnkHost` as an `IACFComponentFactory` to initialize global services (see `acfbaseapi.h`)

Wrapped by [ACFStartup\(\)](#)

Referenced by `ACFStartup()`.

Here is the caller graph for this function:

15.122.2.5 AAXShutdown()

```
AAX_Result AAXShutdown (
    IACFUnknown * pUnkHost )
```

DLL shutdown routine.

Called once before unloading the DLL. The implementation of this method in the AAX library tears down any globally initialized state and releases any globally retained resources.

Wrapped by [ACFShutdown\(\)](#)

Referenced by `ACFShutdown()`.

Here is the caller graph for this function:

15.122.2.6 AAXGetSDKVersion()

```
AAX_Result AAXGetSDKVersion (
    acfUInt64 * oSDKVersion )
```

Returns the DLL's SDK version.

The implementation of this method in the AAX library provides a 64-bit value in which the upper 32 bits represent the SDK version and the lower 32 bits represent the revision number of the SDK. See [AAX_Version.h](#)

Wrapped by [ACFGetSDKVersion\(\)](#)

Referenced by `ACFGetSDKVersion()`.

Here is the caller graph for this function:

15.123 AAX_InstrumentParameters.doxygen File Reference

15.124 AAX_InterfaceList.doxygen File Reference

15.125 AAX_IPageTable.h File Reference

```
#include "AAX.h"  
#include "AAX_IString.h"
```

Classes

- class [AAX_IPageTable](#)
Interface to the host's representation of a plug-in instance's page table.

15.126 AAX_IParameter.h File Reference

```
#include "AAX.h"
```

15.126.1 Description

The base interface for all normalizable plug-in parameters.

Classes

- class [AAX_IParameterValue](#)
An abstract interface representing a parameter value of arbitrary type.
- class [AAX_IParameter](#)
The base interface for all normalizable plug-in parameters.

15.127 AAX_IPointerQueue.h File Reference

```
#include "AAX_IContainer.h"
```

15.127.1 Description

Abstract interface for a basic FIFO queue of pointers-to-objects.

Classes

- class [AAX_IPointerQueue< T >](#)

15.128 AAX_IPrivateDataAccess.h File Reference

```
#include "AAX.h"
```

15.128.1 Description

Interface to data access provided by host to plug-in.

Classes

- class [AAX_IPrivateDataAccess](#)
Interface to data access provided by host to plug-in.

15.129 AAX_IPropertyMap.h File Reference

```
#include "AAX_Properties.h"  
#include "AAX.h"
```

15.129.1 Description

Generic plug-in description property map.

Classes

- class [AAX_IPropertyMap](#)
Generic plug-in description property map.

15.130 AAX_ISessionDocument.h File Reference

```
#include "AAX_SessionDocumentTypes.h"  
#include "AAX_UIDs.h"  
#include "AAX.h"  
#include <memory>
```

Classes

- class [AAX_I_SessionDocument](#)
Interface representing information in a host session document.
- class [AAX_I_SessionDocument::TempoMap](#)

Macros

- #define [AAX_I_SessionDocument_H](#)

15.130.1 Macro Definition Documentation

15.130.1.1 AAX_I_SessionDocument_H

```
#define AAX_I_SessionDocument_H
```

15.131 AAX_I_SessionDocumentClient.h File Reference

```
#include "AAX_IACFSessionDocumentClient.h"  
#include "AAX.h"  
#include "CACFUnknown.h"
```

Classes

- class [AAX_I_SessionDocumentClient](#)
Interface representing a client of the session document interface.

Macros

- #define [AAX_I_SessionDocumentClient_H](#)

15.131.1 Macro Definition Documentation

15.131.1.1 AAX_I_SessionDocumentClient_H

```
#define AAX_I_SessionDocumentClient_H
```

15.132 AAX_IString.h File Reference

```
#include "AAX.h"
```

15.132.1 Description

An AAX string interface.

Classes

- class [AAX_IString](#)

A simple string container that can be passed across a binary boundary. This class, for simplicity, is not versioned and thus can never change.

15.133 AAX_ITaperDelegate.h File Reference

15.133.1 Description

Defines the taper conversion behavior for a parameter.

Classes

- class [AAX_ITaperDelegateBase](#)

Defines the taper conversion behavior for a parameter.

- class [AAX_ITaperDelegate< T >](#)

Classes for conversion to and from normalized parameter values.

15.134 AAX_ITask.h File Reference

```
#include "AAX_IACFTask.h"  
#include "AAX.h"
```

Classes

- class [AAX_ITask](#)

Interface representing a request to perform a task.

Macros

- #define [AAX_ITask_H](#)

15.134.1 Macro Definition Documentation

15.134.1.1 AAX_ITask_H

```
#define AAX_ITask_H
```

15.135 AAX_ITaskAgent.h File Reference

```
#include "AAX_IACFTaskAgent.h"  
#include "AAX.h"  
#include "CACFUnknown.h"
```

Classes

- class [AAX_ITaskAgent](#)
Interface for a component that accepts task requests.

15.136 AAX_ITransport.h File Reference

```
#include "AAX.h"  
#include "AAX_Enums.h"
```

15.136.1 Description

The interface for query ProTools transport information.

Note

To use this interface plug-in must describe AAX_eProperty_UsesMIDI property

Classes

- class [AAX_ITransport](#)
Interface to information about the host's transport state.

15.137 AAX_UIViewContainer.h File Reference

```
#include "AAX_GUITypes.h"  
#include "AAX.h"
```

15.137.1 Description

Interface for the AAX host's view of a single instance of an effect.

Classes

- class [AAX_IViewContainer](#)

Interface for the AAX host's view of a single instance of an effect. Used both by clients of the AAX host and by effect components.

15.138 AAX_LinkedParameters.doxygen File Reference

15.139 AAX_Map.h File Reference

```
#include "AAX.h"
#include <AAX_ALIGN_FILE_BEGIN>
#include <AAX_ALIGN_FILE_ALG>
#include <AAX_ALIGN_FILE_END>
#include <AAX_ALIGN_FILE_RESET>
```

15.139.1 Description

Author

Mykola Kryvonos

Classes

- class [AAX_Map](#)

Macros

- #define [AAX_MAP_H](#)

15.139.2 Macro Definition Documentation

15.139.2.1 AAX_MAP_H

```
#define AAX_MAP_H
```

15.140 AAX_Media_Composer_Guide.doxygen File Reference

15.141 AAX_MIDILogging.cpp File Reference

```
#include "AAX_MIDILogging.h"
#include "AAX_CString.h"
#include "AAX_Assert.h"
#include <map>
#include <vector>
#include <algorithm>
```

Classes

- struct [SAutoArray< T >](#)
- class [AAX_IMIDIMessageInfoDelegate](#)

15.142 AAX_MIDILogging.h File Reference

```
#include "AAX.h"
```

15.142.1 Description

Utilities for logging MIDI data.

Namespaces

- [AAX](#)

Functions

MIDI logging utilities

- void [AAX::AsStringMIDIStream_Debug](#) (const [AAX_CMidiStream](#) &inStream, char *outBuffer, int32_t in↔ BufferSize)

15.143 AAX_MIDIUtilities.h File Reference

```
#include "AAX.h"
```

15.143.1 Description

Utilities for managing MIDI data.

Namespaces

- [AAX](#)

Enumerations

- enum [AAX::EStatusNibble](#) {
[AAX::eStatusNibble_NoteOff](#) = 0x80 ,
[AAX::eStatusNibble_NoteOn](#) = 0x90 ,
[AAX::eStatusNibble_KeyPressure](#) = 0xA0 ,
[AAX::eStatusNibble_ControlChange](#) = 0xB0 ,
[AAX::eStatusNibble_ChannelMode](#) = 0xB0 ,
[AAX::eStatusNibble_ProgramChange](#) = 0xC0 ,
[AAX::eStatusNibble_ChannelPressure](#) = 0xD0 ,
[AAX::eStatusNibble_PitchBend](#) = 0xE0 ,
[AAX::eStatusNibble_SystemCommon](#) = 0xF0 ,
[AAX::eStatusNibble_SystemRealTime](#) = 0xF0 }
Values for the status nibble in a MIDI packet.
- enum [AAX::EStatusByte](#) {
[AAX::eStatusByte_SysExBegin](#) = 0xF0 ,
[AAX::eStatusByte_MTCQuarterFrame](#) = 0xF1 ,
[AAX::eStatusByte_SongPosition](#) = 0xF2 ,
[AAX::eStatusByte_SongSelect](#) = 0xF3 ,
[AAX::eStatusByte_TuneRequest](#) = 0xF6 ,
[AAX::eStatusByte_SysExEnd](#) = 0xF7 ,
[AAX::eStatusByte_TimingClock](#) = 0xF8 ,
[AAX::eStatusByte_Start](#) = 0xFA ,
[AAX::eStatusByte_Continue](#) = 0xFB ,
[AAX::eStatusByte_Stop](#) = 0xFC ,
[AAX::eStatusByte_ActiveSensing](#) = 0xFE ,
[AAX::eStatusByte_Reset](#) = 0xFF }
Values for the status byte in a MIDI packet.
- enum [AAX::EChannelModeData](#) {
[AAX::eChannelModeData_AllSoundOff](#) = 120 ,
[AAX::eChannelModeData_ResetControllers](#) = 121 ,
[AAX::eChannelModeData_LocalControl](#) = 122 ,
[AAX::eChannelModeData_AllNotesOff](#) = 123 ,
[AAX::eChannelModeData_OmniOff](#) = 124 ,
[AAX::eChannelModeData_OmniOn](#) = 125 ,
[AAX::eChannelModeData_PolyOff](#) = 126 ,
[AAX::eChannelModeData_PolyOn](#) = 127 }
Values for the first data byte in a Channel Mode Message MIDI packet.
- enum [AAX::ESpecialData](#) {
[AAX::eSpecialData_AccentedClick](#) = 0x00 ,
[AAX::eSpecialData_UnaccentedClick](#) = 0x01 }
Special message data for the first data byte in a message.

Functions

- bool [AAX::IsNoteOn](#) (const [AAX_CMidiPacket](#) *inPacket)
Returns true if inPacket is a Note On message.
- bool [AAX::IsNoteOff](#) (const [AAX_CMidiPacket](#) *inPacket)
Returns true if inPacket is a Note Off message, or a Note On message with velocity zero.
- bool [AAX::IsAllNotesOff](#) (const [AAX_CMidiPacket](#) *inPacket)
Returns true if inPacket is an All Sound Off or All Notes Off message.
- bool [AAX::IsAccentedClick](#) (const [AAX_CMidiPacket](#) *inPacket)
Returns true if inPacket is a special Pro Tools accented click message.
- bool [AAX::IsUnaccentedClick](#) (const [AAX_CMidiPacket](#) *inPacket)
Returns true if inPacket is a special Pro Tools unaccented click message.
- bool [AAX::IsClick](#) (const [AAX_CMidiPacket](#) *inPacket)
Returns true if inPacket is a special Pro Tools click message.

15.144 AAX_MiscUtils.h File Reference

```
#include "AAX_PlatformOptimizationConstants.h"
#include "AAX_Constants.h"
#include "AAX_Denormal.h"
```

15.144.1 Description

Miscellaneous signal processing utilities.

Namespaces

- [AAX](#)

Macros

- #define [AAX_MISCUTILS_H](#)
- #define [AAX_ALIGNMENT_HINT](#)(a, b)
Currently only functional on TI, these word alignments will provide better performance on TI.
- #define [AAX_WORD_ALIGNED_HINT](#)(a)
- #define [AAX_DWORD_ALIGNED_HINT](#)(a)
- #define [AAX_LO](#)(x) x
These macros are used on TI to convert 2 single words accesses to one double word access to provide additional optimization.
- #define [AAX_HI](#)(x) *((const_cast<float*>(&x))+1)
- #define [AAX_INT_LO](#)(x) x
- #define [AAX_INT_HI](#)(x) *((const_cast<int32_t*>(reinterpret_cast<const int32_t*>(&x)))+1)

Functions

- template<class GFLOAT >
GFLOAT [AAX::ClampToZero](#) (GFLOAT iValue, GFLOAT iClampThreshold)
- void [AAX::ZeroMemorySW](#) (void *iPointer, int iNumBytes)
- void [AAX::ZeroMemoryDW](#) (void *iPointer, int iNumBytes)
- template<typename T , int N>
void [AAX::Fill](#) (T *iArray, const T *iVal)
- template<typename T , int M, int N>
void [AAX::Fill](#) (T *iArray, const T *iVal)
- template<typename T , int L, int M, int N>
void [AAX::Fill](#) (T *iArray, const T *iVal)
- double [AAX::fabs](#) (double iVal)
- float [AAX::fabs](#) (float iVal)
- float [AAX::fabsf](#) (float iVal)
- template<class T >
T [AAX::AbsMax](#) (const T &iValue, const T &iMax)
- template<class T >
T [AAX::MinMax](#) (const T &iValue, const T &iMin, const T &iMax)
- template<class T >
T [AAX::Max](#) (const T &iValue1, const T &iValue2)
- template<class T >
T [AAX::Min](#) (const T &iValue1, const T &iValue2)
- template<class T >
T [AAX::Sign](#) (const T &iValue)
- double [AAX::PolyEval](#) (double x, const double *coefs, int numCoefs)
- double [AAX::CeilLog2](#) (double iValue)
- void [AAX::SinCosMix](#) (float aLinearMix, float &aSinMix, float &aCosMix)

15.144.2 Macro Definition Documentation

15.144.2.1 AAX_MISCUTILS_H

```
#define AAX_MISCUTILS_H
```

15.144.2.2 AAX_ALIGNMENT_HINT

```
#define AAX_ALIGNMENT_HINT(  
    a,  
    b )
```

Currently only functional on TI, these word alignments will provide better performance on TI.

15.144.2.3 AAX_WORD_ALIGNED_HINT

```
#define AAX_WORD_ALIGNED_HINT(  
    a )
```

15.144.2.4 AAX_DWORD_ALIGNED_HINT

```
#define AAX_DWORD_ALIGNED_HINT(  
    a )
```

15.144.2.5 AAX_LO

```
#define AAX_LO(  
    x ) x
```

These macros are used on TI to convert 2 single words accesses to one double word access to provide additional optimization.

15.144.2.6 AAX_HI

```
#define AAX_HI(  
    x ) *((const_cast<float*>(&x))+1)
```

15.144.2.7 AAX_INT_LO

```
#define AAX_INT_LO(  
    x ) x
```

15.144.2.8 AAX_INT_HI

```
#define AAX_INT_HI(  
    x ) *((const_cast<int32_t*>(reinterpret_cast<const int32_t*>(&x)))+1)
```

15.145 AAX_OtherExtensions.doxygen File Reference

15.146 AAX_Page_Table_Guide.doxygen File Reference

15.147 AAX_PageTableUtilities.h File Reference

```
#include "AAX_CString.h"  
#include "AAX.h"
```

Namespaces

- [AAX](#)

Functions

- `template<class T1 , class T2 >`
`bool AAX::PageTableParameterMappingsAreEqual (const T1 &inL, const T2 &inR)`
- `template<class T1 , class T2 >`
`bool AAX::PageTableParameterNameVariationsAreEqual (const T1 &inL, const T2 &inR)`
- `template<class T1 , class T2 >`
`bool AAX::PageTablesAreEqual (const T1 &inL, const T2 &inR)`
- `template<class T >`
`void AAX::CopyPageTable (T &to, const T &from)`
- `template<class T >`
`std::vector< std::pair< int32_t, int32_t > > AAX::FindParameterMappingsInPageTable (const T &inTable, AAX\_CParamID inParameterID)`
- `template<class T >`
`void AAX::ClearMappedParameterByID (T &ioTable, AAX\_CParamID inParameterID)`

15.148 AAX_ParameterAutomation.doxygen File Reference

15.149 AAX_ParameterManager.doxygen File Reference

15.150 AAX_ParameterUpdateProtocol.doxygen File Reference

15.151 AAX_ParameterUpdateTiming.doxygen File Reference

15.152 AAX_PlatformOptimizationConstants.h File Reference

15.152.1 Description

Constants descriptor...

Macros

- `#define` [AAX_PLATFORMOPTIMIZATIONCONSTANTS_H](#)

15.152.2 Macro Definition Documentation

15.152.2.1 AAX_PLATFORMOPTIMIZATIONCONSTANTS_H

```
#define AAX_PLATFORMOPTIMIZATIONCONSTANTS_H
```

15.153 AAX_PluginBundleLocation.h File Reference

15.153.1 Description

Utilities for interacting with the host OS.

Namespaces

- [AAX](#)

Functions

Filesystem utilities

- `bool` [AAX::GetPathToPluginBundle](#) (const char *iBundleName, int iMaxLength, char *oModuleName)
Retrieve the file path of the .aaxplugin bundle.

15.154 AAX_PopStructAlignment.h File Reference

15.154.1 Description

Resets (pops) the struct alignment to its previous value.

See also

[AAX_ALIGN_HOST](#)
[AAX_ALIGN_ALG](#)
[AAX_ALIGN_RESET](#)

Note

Inclusion of this file is mandatory after any 'push' inclusion.
Some compilers do not properly "pop" alignment, so nesting push/pop inclusions is not allowed.

See also

[AAX_Push2ByteStructAlignment.h](#)
[AAX_Push4ByteStructAlignment.h](#)
[AAX_Push8ByteStructAlignment.h](#)

15.155 AAX_PostStructAlignmentHelper.h File Reference

15.155.1 Description

Helper file for data alignment macros.

15.156 AAX_PreStructAlignmentHelper.h File Reference

15.156.1 Description

Helper file for data alignment macros.

15.157 AAX_Pro_Tools_Guide.doxygen File Reference

15.158 AAX_Properties.h File Reference

```
#include "AAX.h"
```

15.158.1 Description

Contains IDs for properties that can be added to an [AAX_IPropertyMap](#).

Enumerations

- enum [AAX_EProperty](#) : int32_t {
 - [AAX_eProperty_NoID](#) = 0 ,
 - [AAX_eProperty_MinProp](#) = 10 ,
 - [AAX_eProperty_PluginSpecPropsBase](#) = 10 ,
 - [AAX_eProperty_ManufacturerID](#) = 11 ,
 - [AAX_eProperty_ProductID](#) = 12 ,
 - [AAX_eProperty_PluginID_Native](#) = 13 ,
 - [AAX_eProperty_PluginID_RTAS](#) = [AAX_eProperty_PluginID_Native](#) ,
 - [AAX_eProperty_PluginID_AudioSuite](#) = 14 ,
 - [AAX_eProperty_PluginID_TI](#) = 15 ,
 - [AAX_eProperty_PluginID_NoProcessing](#) = 16 ,
 - [AAX_eProperty_PluginID_Deprecated](#) = 18 ,
 - [AAX_eProperty_Deprecated_Plugin_List](#) = 21 ,
 - [AAX_eProperty_Related_DSP_Plugin_List](#) = 22 ,
 - [AAX_eProperty_Related_Native_Plugin_List](#) = 23 ,
 - [AAX_eProperty_Deprecated_DSP_Plugin_List](#) = 24 ,
 - [AAX_eProperty_Deprecated_Native_Plugin_List](#) = [AAX_eProperty_Deprecated_Plugin_List](#) ,
 - [AAX_eProperty_PluginID_ExternalProcessor](#) = 25 ,
 - [AAX_eProperty_ExternalProcessorTypeID](#) = 26 ,
 - [AAX_eProperty_ProcessProcPropsBase](#) = 35 ,
 - [AAX_eProperty_NativeProcessProc](#) = 36 ,

AAX_eProperty_NativeInstanceInitProc = 37 ,
AAX_eProperty_NativeBackgroundProc = 38 ,
AAX_eProperty_TIDLLFileName = 39 ,
AAX_eProperty_TIProcessProc = 40 ,
AAX_eProperty_TIInstanceInitProc = 41 ,
AAX_eProperty_TIBackgroundProc = 42 ,
AAX_eProperty_GeneralPropsBase = 50 ,
AAX_eProperty_InputStemFormat = 51 ,
AAX_eProperty_OutputStemFormat = 52 ,
AAX_eProperty_DSP_AudioBufferLength = 54 ,
AAX_eProperty_AudioBufferLength = AAX_eProperty_DSP_AudioBufferLength ,
AAX_eProperty_LatencyContribution = 56 ,
AAX_eProperty_SampleRate = 58 ,
AAX_eProperty_CanBypass = 60 ,
AAX_eProperty_SideChainStemFormat = 61 ,
AAX_eProperty_TI_SharedCycleCount = 62 ,
AAX_eProperty_TI_InstanceCycleCount = 63 ,
AAX_eProperty_TI_MaxInstancesPerChip = 64 ,
AAX_eProperty_TI_ForceAllowChipSharing = 65 ,
AAX_eProperty_AlwaysBypass = 75 ,
AAX_eProperty_ShowInMenus = 76 ,
AAX_eProperty_HybridOutputStemFormat = 90 ,
AAX_eProperty_HybridInputStemFormat = 91 ,
AAX_eProperty_AudiosuitePropsBase = 100 ,
AAX_eProperty_UsesRandomAccess = 101 ,
AAX_eProperty_RequiresAnalysis = 102 ,
AAX_eProperty_OptionalAnalysis = 103 ,
AAX_eProperty_AllowPreviewWithoutAnalysis = 104 ,
AAX_eProperty_DestinationTrack = 105 ,
AAX_eProperty_RequestsAllTrackData = 106 ,
AAX_eProperty_ContinuousOnly = 107 ,
AAX_eProperty_MultiInputModeOnly = 108 ,
AAX_eProperty_DisablePreview = 110 ,
AAX_eProperty_DoesntIncrOutputSample = 112 ,
AAX_eProperty_NumberOfInputs = 113 ,
AAX_eProperty_NumberOfOutputs = 114 ,
AAX_eProperty_DisableHandles = 115 ,
AAX_eProperty_SupportsSideChainInput = 116 ,
AAX_eProperty_NeedsOutputDithered = 117 ,
AAX_eProperty_DisableAudioSuiteReverse = 118 ,
AAX_eProperty_MaxASProp ,
AAX_eProperty_GUIBase = 150 ,
AAX_eProperty_UsesClientGUI = 151 ,
AAX_eProperty_MaxGUIProp ,
AAX_eProperty_MeterBase = 199 ,
AAX_eProperty_Meter_Type = 200 ,
AAX_eProperty_Meter_Orientation = 201 ,
AAX_eProperty_Meter_Ballistics = 202 ,
AAX_eProperty_MaxMeterProp ,
AAX_eProperty_ConstraintBase = 299 ,
AAX_eProperty_Constraint_Location = 300 ,
AAX_eProperty_Constraint_Topology = 301 ,
AAX_eProperty_Constraint_NeverUnload = 302 ,
AAX_eProperty_Constraint_NeverCache = 303 ,
AAX_eProperty_Constraint_MultiMonoSupport = 304 ,
AAX_eProperty_MaxConstraintProp ,
AAX_eProperty_FeaturesBase = 305 ,
AAX_eProperty_SupportsSaveRestore = 305 ,

```

AAX_eProperty_UsesTransport = 306 ,
AAX_eProperty_StoreXMLPageTablesByEffect = 307 ,
AAX_eProperty_StoreXMLPageTablesByType = AAX_eProperty_StoreXMLPageTablesByEffect ,
AAX_eProperty_RequiresChunkCallsOnMainThread = 308 ,
AAX_eProperty_ObservesTransportState = 309 ,
AAX_eProperty_UsesTransportControl = 311 ,
AAX_eProperty_MaxFeaturesProp ,
AAX_eProperty_ConstraintBase_2 = 350 ,
AAX_eProperty_Constraint_AlwaysProcess = 351 ,
AAX_eProperty_Constraint_DoNotApplyDefaultSettings = 352 ,
AAX_eProperty_MaxConstraintProp_2 ,
AAX_eProperty_DebugPropertiesBase = 400 ,
AAX_eProperty_EnableHostDebugLogs = 401 ,
AAX_eProperty_MaxProp ,
AAX_eProperty_MaxCap = 10000 }

```

The list of properties that can be added to an [AAX_IPropertyMap](#).

Functions

- [AAX_ENUM_SIZE_CHECK](#) ([AAX_EProperty](#))

15.158.2 Enumeration Type Documentation

15.158.2.1 AAX_EProperty

```
enum AAX\_EProperty : int32_t
```

The list of properties that can be added to an [AAX_IPropertyMap](#).

See [AAX_IPropertyMap::AddProperty\(\)](#) for more information

Sections

- [Plug-In spec properties](#)
- [ProcessProc properties](#)
- [General properties](#)
- [TI-specific properties](#)
- [Offline \(AudioSuite\) properties](#)
- [GUI properties](#)
- [Meter properties](#)
- [Plug-in management constraints](#)

Legacy Porting Notes These property IDs are somewhat analogous to the pluginGestalt system in the legacy SDK, and several [AAX_EProperty](#) values correlate directly with a corresponding legacy plug-in gestalt.

Legacy Porting Notes To ensure session interchange compatibility, make sure the 4 character IDs for [AAX_eProperty_ManufacturerID](#), [AAX_eProperty_ProductID](#), [AAX_eProperty_PluginID_Native](#), and [AAX_eProperty_PluginID_AudioSuite](#) are identical to the legacy SDK's counterpart.

Enumerator

AAX_eProperty_NoID	
AAX_eProperty_MinProp	
AAX_eProperty_PluginSpecPropsBase	
AAX_eProperty_ManufacturerID	Four-character osid-style manufacturer identifier. Should be registered with Avid, and must be identical for all plug-ins from the same manufacturer. • Apply this property at the ProcessProc level for plug-ins that support audio processing using a ProcessProc callback, or at the Effect level for all other plug-ins. Legacy Porting Notes For legacy plug-in session compatibility, this ID should match the Manufacturer ID used in the corresponding legacy plug-ins.
AAX_eProperty_ProductID	Four-character osid-style Effect identifier. Must be identical for all ProcessProcs within a single Effect. • Apply this property at the ProcessProc level for plug-ins that support audio processing using a ProcessProc callback, or at the Effect level for all other plug-ins. Legacy Porting Notes For legacy plug-in session compatibility, this ID should match the Product ID used in the corresponding legacy plug-in.

Enumerator

AAX_eProperty_PluginID_Native	Four-character osid-style plug-in type identifier for real-time native audio Effects. All registered plug-in type IDs (AAX_eProperty_PluginID_Native, AAX_eProperty_PluginID_AudioSuite, AAX_eProperty_PluginID_TI, etc.) must be unique across all ProcessProcs registered within a single Effect. Warning As with all plug-in ID properties, this value must remain constant across all releases of the plug-in which support this Effect configuration. The value of this property should be stored in a constant rather than being calculated at run-time in order to avoid unresolvable compatibility issues with saved sessions which can occur if an ID value is accidentally changed between two plug-in version releases. • Apply this property at the ProcessProc level Legacy Porting Notes For legacy plug-in session compatibility, this ID should match the Type ID used in the corresponding legacy RTAS plug-in Types.
AAX_eProperty_PluginID_RTAS	Deprecated Use AAX_eProperty_PluginID_Native
AAX_eProperty_PluginID_AudioSuite	Four-character osid-style plug-in type identifier for offline native audio Effects. All registered plug-in type IDs (AAX_eProperty_PluginID_Native, AAX_eProperty_PluginID_AudioSuite, AAX_eProperty_PluginID_TI, etc.) must be unique across all ProcessProcs registered within a single Effect. • Apply this property at the ProcessProc level for plug-ins that support audio processing using a ProcessProc callback, or at the Effect level for all other AudioSuite plug-ins (e.g. those that use the AAX_IHostProcessor interface.) Legacy Porting Notes For legacy plug-in session compatibility, this ID should match the Type ID used in the corresponding legacy AudioSuite plug-in Types.

Enumerator

AAX_eProperty_PluginID_TI	Four-character osid-style plug-in type identifier for real-time TI-accelerated audio Effect types. All registered plug-in type IDs (AAX_eProperty_PluginID_Native, AAX_eProperty_PluginID_AudioSuite, AAX_eProperty_PluginID_TI, etc.) must be unique across all ProcessProcs registered within a single Effect. Warning As with all plug-in ID properties, this value must remain constant across all releases of the plug-in which support this Effect configuration. The value of this property should be stored in a constant rather than being calculated at run-time in order to avoid unresolvable compatibility issues with saved sessions which can occur of an ID value is accidentally changed between two plug-in version releases. • Apply this property at the ProcessProc level Legacy Porting Notes For legacy plug-in session compatibility, this ID should match the Type ID used in the corresponding legacy TDM plug-in Types.
AAX_eProperty_PluginID_NoProcessing	Four-character osid-style plug-in type identifier for Effect types that do not process audio. All registered plug-in type IDs (AAX_eProperty_PluginID_Native, AAX_eProperty_PluginID_AudioSuite, AAX_eProperty_PluginID_TI, etc.) must be unique across all ProcessProcs registered within a single Effect. Warning As with all plug-in ID properties, this value must remain constant across all releases of the plug-in which support this Effect configuration. The value of this property should be stored in a constant rather than being calculated at run-time in order to avoid unresolvable compatibility issues with saved sessions which can occur of an ID value is accidentally changed between two plug-in version releases. • Apply this property at the Effect level

Enumerator

AAX_eProperty_PluginID_Deprecated	Four-character osid-style plug-in type identifier for a corresponding deprecated type. Only one deprecated effect ID may correspond to each valid (non-deprecated) effect ID. To associate a plug-in type with more than one deprecated type, use the following properties instead: • AAX_eProperty_Deprecated_DSP_Plugin_List • AAX_eProperty_Deprecated_Native_Plugin_List • Apply this property at the ProcessProc level
AAX_eProperty_Deprecated_Plugin_List	Deprecated Use AAX_eProperty_Deprecated_Native_Plugin_List and AAX_eProperty_Deprecated_DSP_Plugin_List. See AAX_eProperty_PluginID_RTAS for an example.
AAX_eProperty_Related_DSP_Plugin_List	Specify a list of DSP plug-ins that are related to a plug-in type. • For example, use this property inside a Native process to tell the host that this plug-in can be used in place of a DSP version. • This property must be applied at the ProcessProc level and used with the AAX_IPropertyMap::AddPropertyWithIDArray method, which takes a list of full plug-in identifier specification triads (ManufacturerID, ProductID, PluginID)
AAX_eProperty_Related_Native_Plugin_List	Specify a list of Native plug-ins that are related to a plug-in type. • This property must be applied at the ProcessProc level and used with the AAX_IPropertyMap::AddPropertyWithIDArray method, which takes a list of full plug-in identifier specification triads (ManufacturerID, ProductID, PluginID)
AAX_eProperty_Deprecated_DSP_Plugin_List	Specify a list of DSP plug-ins that are deprecated by a new plug-in type. • This property must be applied at the ProcessProc level and used with the AddPropertyWithIDArray, which is a list of full plug-in specs (ManufacturerID, ProductID, PluginID)

Enumerator

AAX_eProperty_Deprecated_Native_Plugin_List	Specify a list of Native plug-ins that are deprecated by a new plug-in type. This property must be applied at the ProcessProc level and used with the AddPropertyWithIDArray, which is a list of full plug-in specs (ManufacturerID, ProductID, PluginID)
AAX_eProperty_PluginID_ExternalProcessor	Four-character osid-style plug-in type identifier for audio effects rendered on external hardware. Note This property is not currently used by any AAX plug-in host software All registered plug-in type IDs must be unique across all ProcessProcs registered within a single Effect. Warning As with all plug-in ID properties, this value must remain constant across all releases of the plug-in which support this Effect configuration. The value of this property should be stored in a constant rather than being calculated at run-time in order to avoid unresolvable compatibility issues with saved sessions which can occur if an ID value is accidentally changed between two plug-in version releases. Apply this property at the ProcessProc level
AAX_eProperty_ExternalProcessorTypeID	Identifier for the type of the external processor hardware. See also AAX_eProperty_PluginID_ExternalProcessor The value of this property will be specific to the external processor hardware. Currently there are no public external processor hardware type IDs. Apply this property at the ProcessProc level
AAX_eProperty_ProcessProcPropsBase	
AAX_eProperty_NativeProcessProc	Address of a native effect's ProcessProc callback Data type: AAX_CProcessProc For use with AAX_IComponentDescriptor::AddProcessProc()
AAX_eProperty_NativeInstanceInitProc	Address of a native effect's instance initialization callback Data type: AAX_CInstanceInitProc For use with AAX_IComponentDescriptor::AddProcessProc()

Enumerator

AAX_eProperty_NativeBackgroundProc	Address of a native effect's background callback Data type: AAX_CBackgroundProc For use with AAX_IComponentDescriptor::AddProcessProc()
AAX_eProperty_TIDLLFileName	Name of the DLL for a TI effect Data type: UTF-8 C-string For use with AAX_IComponentDescriptor::AddProcessProc()
AAX_eProperty_TIProcessProc	Name of a TI effect's ProcessProc callback Data type: C-string For use with AAX_IComponentDescriptor::AddProcessProc()
AAX_eProperty_TIInstanceInitProc	Name of a TI effect's instance initialization callback Data type: C-string For use with AAX_IComponentDescriptor::AddProcessProc()
AAX_eProperty_TIBackgroundProc	Name of a TI effect's background callback Data type: C-string For use with AAX_IComponentDescriptor::AddProcessProc()
AAX_eProperty_GeneralPropsBase	_____
AAX_eProperty_InputStemFormat	Input stem format. One of AAX_EStemFormat . • Apply this property at the ProcessProc level For offline processing, use AAX_eProperty_NumberOfInputs
AAX_eProperty_OutputStemFormat	Output stem format. One of AAX_EStemFormat . • Apply this property at the ProcessProc level For offline processing, use AAX_eProperty_NumberOfOutputs
AAX_eProperty_DSP_AudioBufferLength	Audio buffer length for DSP processing callbacks. One of AAX_EAudioBufferLengthDSP . • Apply this property at the ProcessProc level • This property is only applicable to DSP algorithms
AAX_eProperty_AudioBufferLength	Deprecated Use AAX_eProperty_DSP_AudioBufferLength

Enumerator

AAX_eProperty_LatencyContribution	Default latency contribution of a given processing callback, in samples. • Apply this property at the ProcessProc level Unlike most properties, an Effect's latency contribution may also be changed dynamically at runtime. This is done via AAX_IController::SetSignalLatency(). Dynamic latency reporting may not be recognized by the host application in all circumstances, however, so Effects should <i>always</i> define any nonzero initial latency value using AAX_eProperty_LatencyContribution Host Compatibility Notes Maximum delay compensation limits will vary from host to host. If your plug-in exceeds the delay compensation sample limit for a given AAX host then you should note this limitation in your user documentation. Example limits: • Pro Tools 9 and higher: 16,383 samples at 44.1/48 kHz, 32,767 samples at 88.2/96 kHz, or 65,534 samples at 176.4/192 kHz • Media Composer 8.1 and higher: 16,383 samples at 44.1/48 kHz, 32,767 samples at 88.2/96 kHz
AAX_eProperty_SampleRate	Specifies which sample rates the Effect supports. A mask of AAX_ESampleRateMask. • Apply this property at the ProcessProc level See also AAX_IComponentDescriptor::AddSampleRate()

Enumerator

AAX_eProperty_CanBypass	The plug-in supports a Master Bypass control. • Apply this property at the ProcessProc level Nearly all AAX plug-ins should set this property to <code>true</code> Set this property to <code>false</code> (0) to disable Master Bypass for plug-ins that cannot be bypassed, such as fold-down plug-ins that convert to a narrower channel format. Legacy Porting Notes Was <code>pluginGestalt_CanBypass</code>.
AAX_eProperty_SideChainStemFormat	Side chain stem format. One of AAX_EStemFormat. Host Compatibility Notes Currently Pro Tools supports only AAX_eStemFormat_Mono side chain inputs • Apply this property at the ProcessProc level Host Compatibility Notes <code>AAX_eProperty_SideChainStemFormat</code> is not currently implemented in DAE or AAE
AAX_eProperty_TI_SharedCycleCount	Shared cycle count (outer, per clump, loop overhead) • Apply this property at the ProcessProc level • This property is only applicable to DSP algorithms
AAX_eProperty_TI_InstanceCycleCount	Instance cycle count (inner, per instance, loop overhead) • Apply this property at the ProcessProc level • This property is only applicable to DSP algorithms
AAX_eProperty_TI_MaxInstancesPerChip	Maximum number of instances of this plug-in that can be loaded on a chip. This property is only used for DMA and background thread-enabled plug-ins. • Apply this property at the ProcessProc level • This property is only applicable to DSP algorithms

Enumerator

AAX_eProperty_TI_ForceAllowChipSharing	Allow different plug-in types to share the same DSP even if AAX_eProperty_TI_MaxInstancesPerChip is declared. In general, this is not desired behavior. However, this can be useful if your plug-in instance counts are bound by a system constraint other than CPU usage and you require chip-sharing between instances of different types of the plug-in. Note In addition to defining this property, the types which will share allocations on the same DSP chip must be compiled into the same ELF DLL file. • Apply this property at the ProcessProc level • This property is only applicable to DSP algorithms
AAX_eProperty_AlwaysBypass	The plug-in never alters its audio signal, audio output is always equal to audio input. • Apply this property at the ProcessProc level Setting this property allows host to optimize audio routing and reduce audio latency.
AAX_eProperty_ShowInMenus	Indicates whether or not the plug-in should be shown in insert menus. • Apply this property to show or hide the plug-in from the Pro Tools insert menus. • This property value is <code>true</code> by default.
AAX_eProperty_HybridOutputStemFormat	Hybrid Output stem format. One of AAX_EStemFormat. This property represents the stem format for the audio channels that are sent from the ProcessProc callback to the AAX_IEffectParameters::RenderAudio_Hybrid() method • Apply this property at the ProcessProc level • Normally plugins will set this to the same thing as AAX_eProperty_InputStemFormat
AAX_eProperty_HybridInputStemFormat	Hybrid Input stem format. One of AAX_EStemFormat. This property represents the stem format for the audio channels that are sent from the AAX_IEffectParameters::RenderAudio_Hybrid() method to the ProcessProc callback • Apply this property at the ProcessProc level • Normally plugins will set this to the same thing as AAX_eProperty_OutputStemFormat
AAX_eProperty_AudiosuitePropsBase	

Enumerator

AAX_eProperty_UsesRandomAccess	The Effect requires random access to audio data. • Apply this property at the AAX_IEffectDescriptor level • This property is only applicable to Host Processor algorithms Legacy Porting Notes Was pluginGestalt_Uses↔ RandomAccess
AAX_eProperty_RequiresAnalysis	The Effect requires an analysis pass. • Apply this property at the AAX_IEffectDescriptor level • This property is only applicable to offline processing Legacy Porting Notes Was pluginGestalt_↔ RequiresAnalysis
AAX_eProperty_OptionalAnalysis	The Effect supports an analysis pass, but does not require it. Host Compatibility Notes In Media Composer, optional analysis will also be performed automatically before each channel is rendered. See MCDEV-2904 • Apply this property at the AAX_IEffectDescriptor level • This property is only applicable to offline processing Legacy Porting Notes Was pluginGestalt_↔ OptionalAnalysis
AAX_eProperty_AllowPreviewWithoutAnalysis	The Effect requires analysis, but is also allowed to preview. • Apply this property at the AAX_IEffectDescriptor level • This property is only applicable to offline processing Legacy Porting Notes Was pluginGestalt_↔ AnalyzeOnTheFly

Enumerator

AAX_eProperty_DestinationTrack	Informs the host application to reassign output to a different track. • Apply this property at the AAX_IEffectDescriptor level • This property is only applicable to offline processing Host Compatibility Notes This property is not supported on Media Composer Legacy Porting Notes Was pluginGestalt_↔ DestinationTrack
AAX_eProperty_RequestsAllTrackData	The host should make all of the processed track's data available to the Effect. • Apply this property at the AAX_IEffectDescriptor level • This property is only applicable to Host Processor algorithms Legacy Porting Notes Was pluginGestalt_↔ RequestsAllTrackData
AAX_eProperty_ContinuousOnly	The Effect only processes on continuous data and does not support 'clip by clip' rendering. • Apply this property at the AAX_IEffectDescriptor level • This property is only applicable to offline processing Legacy Porting Notes Was pluginGestalt_↔ ContinuousOnly
AAX_eProperty_MultiInputModeOnly	The Effect wants multi-input mode only (no mono mode option) Note See bug PT-258560 / PT-256919 • Apply this property at the AAX_IEffectDescriptor level • This property is only applicable to offline processing Legacy Porting Notes Was pluginGestalt_Multi↔ InputModeOnly

Enumerator

AAX_eProperty_DisablePreview	The Effect does not support preview. • Apply this property at the AAX_IEffectDescriptor level • This property is only applicable to offline processing Legacy Porting Notes Was pluginGestalt_Disable↔ Preview
AAX_eProperty_DoesntIncrOutputSample	The Effect may not increment its output sample during some rendering calls. • Apply this property at the AAX_IEffectDescriptor level • This property is only applicable to Host Processor algorithms Legacy Porting Notes Was pluginGestalt_Doesnt↔ IncrOutputSample
AAX_eProperty_NumberOfInputs	The number of input channels that the plug-in supports. • Apply this property at the AAX_IEffectDescriptor level • This property is only applicable to Host Processor algorithms For real-time processing, use AAX_eProperty_InputStemFormat
AAX_eProperty_NumberOfOutputs	The number of output channels that the plug-in supports. • Apply this property at the AAX_IEffectDescriptor level • This property is only applicable to Host Processor algorithms For real-time processing, use AAX_eProperty_OutputStemFormat
AAX_eProperty_DisableHandles	Prevents the application of rendered region handles by the host. • Apply this property at the AAX_IEffectDescriptor level • This property is only applicable to offline processing

Enumerator

AAX_eProperty_SupportsSideChainInput	Tells the host that the plug-in supports side chain inputs. • Apply this property at the AAX_IEffectDescriptor level • This property is only applicable to offline processing
AAX_eProperty_NeedsOutputDithered	Requests that the host apply dithering to the Effect's output. • Apply this property at the AAX_IEffectDescriptor level • This property is only applicable to offline processing Legacy Porting Notes Was pluginGestalt_Needs↔ OutputDithered
AAX_eProperty_DisableAudioSuiteReverse	The plug-in supports audiosuite reverse. By default, all reverb and delay plug-ins support this feature. If a plug-in needs to opt out of this feature, they can set this property to true. • Apply this property at the AAX_IEffectDescriptor level • This property is only applicable to offline processing
AAX_eProperty_MaxASProp	
AAX_eProperty_GUIBase	
AAX_eProperty_UsesClientGUI	Requests a host-generated GUI based on the Effect's parameters. Use this property while your plug-in is in development to test the plug-in's data model and algorithm before its GUI has been created, or when troubleshooting problems to isolate the data model and algorithm operation from the plug-in's GUI. • Apply this property at the ProcessProc level Host Compatibility Notes Currently supported by Pro Tools only Note See PTSW-189725 / PT-218397
AAX_eProperty_MaxGUIProp	
AAX_eProperty_MeterBase	
AAX_eProperty_Meter_Type	Indicates meter type as one of AAX_EMeterType. • Apply this property at the AAX_IEffectDescriptor::AddMeterDescription() level

Enumerator

AAX_eProperty_Meter_Orientation	Indicates meter orientation as one of AAX_EMeterOrientation. Apply this property at the AAX_IEffectDescriptor::AddMeterDescription() level
AAX_eProperty_Meter_Ballistics	Indicates meter ballistics preference as one of AAX_EMeterBallisticType. Apply this property at the AAX_IEffectDescriptor::AddMeterDescription() level
AAX_eProperty_MaxMeterProp	
AAX_eProperty_ConstraintBase	
AAX_eProperty_Constraint_Location	Constraint on the algorithm's location, as a mask of AAX_EConstraintLocationMask. Apply this property at the ProcessProc level
AAX_eProperty_Constraint_Topology	Constraint on the topology of the Effect's modules, as one of AAX_EConstraintTopology. Apply this property at the AAX_IEffectDescriptor level
AAX_eProperty_Constraint_NeverUnload	Tells the host that it should never unload the plug-in binary. Apply this property at the AAX_IEffectDescriptor level Host Compatibility Notes AAX_eProperty_Constraint_NeverUnload is not currently implemented in DAE or AAE
AAX_eProperty_Constraint_NeverCache	Tells the host that it should never cache the plug-in binary. Only use this if required as there is a performance penalty on launch to not use the Cache. Set this property to 1, if you really need to not cache. Default is 0. The most common reason for a plug-in to require this constraint is if the plug-in's configuration can change based on external conditions. Most of the data contained in the plug-in's description routine is cached, so if the plug-in description can change between launches of the host application then the plug-in should apply this constraint to prevent the host from using stale description information. Apply this property at the AAX_ICollection level

Enumerator

AAX_eProperty_Constraint_MultiMonoSupport	Indicates whether or not the plug-in supports multi-mono configurations (<code>true/false</code>) Note Multi-mono mode may not work as expected for VIs and other plug-ins which rely on non-global MIDI input. Depending on the host, multi-mono instances may not all be automatically connected to the same MIDI port upon instantiation. Therefore it is recommended to set this property to 0 for any plug-ins if this lack of automatic connection may confuse users. • Apply this property at the ProcessProc level
AAX_eProperty_MaxConstraintProp	
AAX_eProperty_FeaturesBase	
AAX_eProperty_SupportsSaveRestore	Indicates whether or not the plug-in supports Save/Restore features. (<code>true/false</code>) • Apply this property to show or hide the Settings section in the plug-in window. • This property value is true by default. Legacy Porting Notes Was <code>pluginGestalt_↔ SupportsSaveRestore</code>
AAX_eProperty_UsesTransport	Indicates whether or not the plug-in uses transport requests. (<code>true/false</code>) • Apply this property if your plug-in uses AAX_ITransport class. • Apply this property at the AAX_IEffectDescriptor level
AAX_eProperty_StoreXMLPageTablesByEffect	This property specifies whether the plug-in bundle contains an XML file per plug-in type. AAX plug-ins always provide XML page table data via external files referenced by AAX_eResourceType_PageTable. If AAX_eProperty_StoreXMLPageTablesByEffect is not defined or is set to 0 (the default) then the host may assume that all Effects in the collection use the same XML page table file. If this property is set to a non-zero value, the plug-in may describe a different AAX_eResourceType_PageTable for each separate Effect. • Apply this property at the AAX_ICollection level
AAX_eProperty_StoreXMLPageTablesByType	Deprecated Use AAX_eProperty_StoreXMLPageTablesByEffect

Enumerator

AAX_eProperty_RequiresChunkCallsOnMainThread	Indicates whether the plug-in supports SetChunk and GetChunk calls on threads other than the main thread. It is actually important for plug-ins to support these calls on non-main threads, so that is the default. However, in response to a few companies having issues with this, we have decided to support this constraint for now. property value should be set to true if you need Chunk calls on the main thread. Values: 0 (off, default), 1 (on) • Apply this property at the AAX_IEffectDescriptor level
AAX_eProperty_ObservesTransportState	Indicates whether the plug-in subscribes to the TransportStateChanged notification to receive transport info. property value should be set to true if you need subscribe to the TransportStateNotification. Values: 0 (off, default), 1 (on) • Apply this property at the AAX_IEffectDescriptor level
AAX_eProperty_UsesTransportControl	Indicates whether or not the plug-in uses transport control requests. (<code>true/false</code>) • Apply this property if your plug-in uses AAX_IACFTransportControl methods in the AAX_ITransport class. • Apply this property at the AAX_IEffectDescriptor level
AAX_eProperty_MaxFeaturesProp	
AAX_eProperty_ConstraintBase_2	

Enumerator

AAX_eProperty_Constraint_AlwaysProcess	Indicates that the plug-in's processing should never be disabled by the host (<code>true/false</code>) Some hosts will disable processing for plug-in chains in certain circumstances to conserve system resources, e.g. when the chains' output drops to silence for an extended period. Note This property may impact performance of other plug-ins. For example, the Dynamic Plug-In Processing feature in Pro Tools operates over chains of plug-ins rather than single instances; any plug-in that defines AAX_eProperty_Constraint_AlwaysProcess will force its entire signal chain to continue processing. Therefore it is important to avoid using this property unless features such as Dynamic Plug-In Processing are actually interfering in some way with the operation of the plug-in. • This property value is false by default. • Apply this property at the AAX_IEffectDescriptor level
AAX_eProperty_Constraint_DoNotApplyDefault↔ Settings	Requests that the host does not send default settings chunks to the plug-in after instantiation (<code>true/false</code>) Some hosts will apply the plug-in's default settings via chunks after creating a new plug-in instance as a way to ensure that the all new plug-in instances are initialized to the same state. If a plug-in can make this guarantee itself and does not wish to receive any default settings chunks from the host after instantiation then it may set this property. Support for this property is not guaranteed; the plug-in must be able to handle default settings chunk application even if this property is set, or clearly document the plug-in's host compatibility. Note See bug PT-284916 • Apply this property at the AAX_IEffectDescriptor level #
AAX_eProperty_MaxConstraintProp_2	
AAX_eProperty_DebugPropertiesBase	

Enumerator

AAX_eProperty_EnableHostDebugLogs	Enables host debug logging for this plug-in. This logging is made via DigiTrace using the DTF_AAXHOST facility, generally at DTP_LOW priority • It is recommended to set this property to 1 for debug builds and to 0 for release builds of a plug-in • Apply this property at the AAX_IEffectDescriptor level
AAX_eProperty_MaxProp	
AAX_eProperty_MaxCap	

15.158.3 Function Documentation

15.158.3.1 AAX_ENUM_SIZE_CHECK()

```
AAX_ENUM_SIZE_CHECK (
    AAX_EProperty )
```

15.159 AAX_Push2ByteStructAlignment.h File Reference

15.159.1 Description

Set the struct alignment to 2-byte. This file will throw an error on platforms that do not support 2-byte alignment (i.e. TI DSPs)

When setting the alignment for a struct in order to match a particular environment (e.g. host/plug-in binary compatibility) the following macros are recommended:

- [AAX_ALIGN_FILE_HOST](#)
- [AAX_ALIGN_FILE_ALG](#)
- [AAX_ALIGN_FILE_RESET](#)

15.159.2 Usage notes

- Always follow an inclusion of this file with a matching inclusion of [AAX_PopStructAlignment.h](#)
- Do not place other file `#include` after this file. For example:

```
// HeaderFile1.h
#include AAX_Push2ByteStructAlignment.h
#include HeaderFile2.h // this file now has 2-byte alignment also!!
// HeaderFile1.h definitions...
#include AAX_PopStructAlignment.h
// end HeaderFile1.h
```

This will cause problems if HeaderFile2.h is included elsewhere without the 2-byte alignment which will manifest as hard to find run-time bugs. The proper usage is:

```
// HeaderFile1.h
#include HeaderFile2.h
#include AAX_Push2ByteStructAlignment.h
// HeaderFile1.h definitions...
#include AAX_PopStructAlignment.h
// end HeaderFile1.h
```

See also

[AAX_Push4ByteStructAlignment.h](#)
[AAX_Push8ByteStructAlignment.h](#)
[AAX_PopStructAlignment.h](#)

15.160 AAX_Push4ByteStructAlignment.h File Reference

15.160.1 Description

Set the struct alignment to 4-byte.

When setting the alignment for a struct in order to match a particular environment (e.g. host/plugin binary compatibility) the following macros are recommended:

- [AAX_ALIGN_FILE_HOST](#)
- [AAX_ALIGN_FILE_ALG](#)
- [AAX_ALIGN_FILE_RESET](#)

15.160.2 Usage notes

- Always follow an inclusion of this file with a matching inclusion of [AAX_PopStructAlignment.h](#)
- Do not place other file `#include` after this file. For example:

```
// HeaderFile1.h
#include AAX_Push4ByteStructAlignment.h
#include HeaderFile2.h // this file now has 4-byte alignment also!!
// HeaderFile1.h definitions...
#include AAX_PopStructAlignment.h
// end HeaderFile1.h
```

This will cause problems if HeaderFile2.h is included elsewhere without the 4-byte alignment which will manifest as hard to find run-time bugs. The proper usage is:

```
// HeaderFile1.h
#include HeaderFile2.h
#include AAX_Push4ByteStructAlignment.h
// HeaderFile1.h definitions...
#include AAX_PopStructAlignment.h
// end HeaderFile1.h
```

See also

[AAX_Push2ByteStructAlignment.h](#)
[AAX_Push8ByteStructAlignment.h](#)
[AAX_PopStructAlignment.h](#)

15.161 AAX_Push8ByteStructAlignment.h File Reference

15.161.1 Description

Set the struct alignment to 8-byte.

When setting the alignment for a struct in order to match a particular environment (e.g. host/plugin binary compatibility) the following macros are recommended:

- [AAX_ALIGN_FILE_HOST](#)
- [AAX_ALIGN_FILE_ALG](#)
- [AAX_ALIGN_FILE_RESET](#)

15.161.2 Usage notes

- Always follow an inclusion of this file with a matching inclusion of [AAX_PopStructAlignment.h](#)
- Do not place other file `#include` after this file. For example:

```
// HeaderFile1.h
#include AAX_Push8ByteStructAlignment.h
#include HeaderFile2.h // this file now has 8-byte alignment also!!
// HeaderFile1.h definitions...
#include AAX_PopStructAlignment.h
// end HeaderFile1.h
```

This will cause problems if HeaderFile2.h is included elsewhere without the 8-byte alignment which will manifest as hard to find run-time bugs. The proper usage is:

```
// HeaderFile1.h
#include HeaderFile2.h
#include AAX_Push8ByteStructAlignment.h
// HeaderFile1.h definitions...
#include AAX_PopStructAlignment.h
// end HeaderFile1.h
```

See also

[AAX_Push2ByteStructAlignment.h](#)

[AAX_Push4ByteStructAlignment.h](#)

[AAX_PopStructAlignment.h](#)

15.162 AAX_Quantize.h File Reference

```
#include "AAX.h"
#include "AAX_PlatformOptimizationConstants.h"
#include "AAX_Constants.h"
#include <xmmintrin.h>
#include <pmmintrin.h>
#include <tmmintrin.h>
```

15.162.1 Description

Quantization utilities.

Namespaces

- [AAX](#)

Macros

- `#define` [AAX_QUANTIZE_H](#)

Functions

- `int32_t` [AAX::FastRound2Int32](#) (double iVal)
Round to Int32.
- `int32_t` [AAX::FastRound2Int32](#) (float iVal)
Round to Int32.
- `int32_t` [AAX::FastRndDbl2Int32](#) (double iVal)
- `int32_t` [AAX::FastTrunc2Int32](#) (double iVal)
Float to Int conversion with truncation.
- `int32_t` [AAX::FastTrunc2Int32](#) (float iVal)
Float to Int conversion with truncation.
- `int64_t` [AAX::FastRound2Int64](#) (double iVal)
Round to Int64.

15.162.2 Macro Definition Documentation

15.162.2.1 AAX_QUANTIZE_H

```
#define AAX_QUANTIZE_H
```

15.163 AAX_RandomGen.h File Reference

```
#include <stdlib.h>
#include <time.h>
#include <stdint.h>
#include "AAX_PlatformOptimizationConstants.h"
#include "AAX_Constants.h"
```

15.163.1 Description

Functions for calculating pseudo-random numbers.

Namespaces

- [AAX](#)

Macros

- `#define` [AAX_RANDOMGEN_H](#)

Functions

- `int32_t` [AAX::GetInt32RPDF](#) (`int32_t *iSeed`)
- `int32_t` [AAX::GetFastInt32RPDF](#) (`int32_t *iSeed`)
CALL: Calculate pseudo-random 32 bit number based on linear congruential method.
- `float` [AAX::GetRPDFWithAmplitudeOneHalf](#) (`int32_t *iSeed`)
- `float` [AAX::GetRPDFWithAmplitudeOne](#) (`int32_t *iSeed`)
- `float` [AAX::GetFastRPDFWithAmplitudeOne](#) (`int32_t *iSeed`)
- `float` [AAX::GetTPDFWithAmplitudeOne](#) (`int32_t *iSeed`)

Variables

- `const float` [AAX::cSeedDivisor](#) = 1/127773.0f
- `const int32_t` [AAX::cInitialSeedValue](#) = 0x00F54321

15.163.2 Macro Definition Documentation

15.163.2.1 AAX_RANDOMGEN_H

```
#define AAX_RANDOMGEN_H
```

15.164 AAX_RealTimePerformance.doxygen File Reference

15.165 AAX_RelatedTypes.doxygen File Reference

15.166 AAX_SampleRateUtils.h File Reference

15.166.1 Description

Description.

Enumerations

- enum [ESRUtils](#) {
 - [eSRUtils_48kRangeCoarse](#) = 48000 ,
 - [eSRUtils_96kRangeCoarse](#) = 96000 ,
 - [eSRUtils_192kRangeCoarse](#) = 192000 ,
 - [eSRUtils_48kRangeMin](#) = 0 ,
 - [eSRUtils_48kRangeMax](#) = 51000 ,
 - [eSRUtils_96kRangeMin](#) = [eSRUtils_48kRangeMax](#)+1 ,
 - [eSRUtils_96kRangeMax](#) = 102000 ,
 - [eSRUtils_192kRangeMin](#) = [eSRUtils_96kRangeMax](#)+1 ,
 - [eSRUtils_192kRangeMax](#) = 204000 ,
 - [eSRUtils_48kIndex](#) = 0 ,
 - [eSRUtils_96kIndex](#) = 1 ,
 - [eSRUtils_192kIndex](#) = 2 }

Functions

- int [CoarseSampleRate](#) (int iRate)
- int [CoarseSampleRateFactor](#) (int iRate)
- int [CoarseSampleRateIndex](#) (int iRate)

15.166.2 Enumeration Type Documentation

15.166.2.1 ESRUtils

enum [ESRUtils](#)

Enumerator

eSRUtils_48kRangeCoarse	
eSRUtils_96kRangeCoarse	
eSRUtils_192kRangeCoarse	
eSRUtils_48kRangeMin	
eSRUtils_48kRangeMax	
eSRUtils_96kRangeMin	
eSRUtils_96kRangeMax	
eSRUtils_192kRangeMin	
eSRUtils_192kRangeMax	
eSRUtils_48kIndex	
eSRUtils_96kIndex	
eSRUtils_192kIndex	

15.166.3 Function Documentation

15.166.3.1 CoarseSampleRate()

```
int CoarseSampleRate (
    int iRate ) [inline]
```

References eSRUtils_192kRangeCoarse, eSRUtils_192kRangeMax, eSRUtils_192kRangeMin, eSRUtils_48kRangeCoarse, eSRUtils_48kRangeMax, eSRUtils_48kRangeMin, eSRUtils_96kRangeCoarse, eSRUtils_96kRangeMax, and eSRUtils_96kRangeMin.

Referenced by CoarseSampleRateFactor(), and CoarseSampleRateIndex().

Here is the caller graph for this function:

15.166.3.2 CoarseSampleRateFactor()

```
int CoarseSampleRateFactor (
    int iRate ) [inline]
```

References CoarseSampleRate(), and eSRUtils_48kRangeCoarse.

Here is the call graph for this function:

15.166.3.3 CoarseSampleRateIndex()

```
int CoarseSampleRateIndex (
    int iRate ) [inline]
```

References CoarseSampleRate(), eSRUtils_192kRangeCoarse, eSRUtils_48kRangeCoarse, and eSRUtils_96kRangeCoarse.

Here is the call graph for this function:

15.167 AAX_SDK_ChangeLog.doxygen File Reference

15.168 AAX_SDK_ExamplePlugIns.doxygen File Reference

15.169 AAX_SDK_GUIExtensions.doxygen File Reference

15.170 AAX_SessionDocumentTypes.h File Reference

```
#include "AAX.h"
#include <stdint.h>
#include "AAX_PreStructAlignmentHelper.h"
#include "AAX_Push2ByteStructAlignment.h"
#include "AAX_PostStructAlignmentHelper.h"
#include "AAX_PopStructAlignment.h"
```


Classes

- struct [AAX_CTempoBreakpoint](#)

Macros

- `#define` [AAX_SessionDocumentTypes_H](#)

Variables

- [AAX_CONSTEXPR AAX_CTypeID kAAX_DataBufferType_TempoBreakpointArray](#) = 'AXtB'

15.170.1 Macro Definition Documentation

15.170.1.1 AAX_SessionDocumentTypes_H

```
#define AAX_SessionDocumentTypes_H
```

15.170.2 Variable Documentation

15.170.2.1 kAAX_DataBufferType_TempoBreakpointArray

```
AAX_CONSTEXPR AAX_CTypeID kAAX_DataBufferType_TempoBreakpointArray = 'AXtB'
```

15.171 AAX_SliderConversions.h File Reference

```
#include "AAX.h"  
#include <algorithm>  
#include <stdint.h>
```

15.171.1 Description

Legacy utilities for converting parameter values to and from the normalized full-scale 32-bit fixed domain that was used for RTAS/TDM plug-ins.

Legacy Porting Notes These utilities may be required in order to maintain settings chunk compatibility with plug-ins that were ported from the legacy RTAS/TDM format.

Note

AAX does not provide facilities for converting to and from extended80 data types. If you use these types in your plug-in settings then you must provide your own chunk data parsing routines.

Macros

- #define [AAX_SLIDERCONVERSIONS_H](#)
- #define [AAX_LIMIT](#)(v1, firstVal, secondVal) ((secondVal > firstVal) ? (std::max)((std::min)(v1,secondVal),firstVal) : (std::min)((std::max)(v1,secondVal),firstVal))

Functions

- int32_t [LongControlToNewRange](#) (int32_t aValue, int32_t rangeMin, int32_t rangeMax)
- int32_t [LongToLongControl](#) (int32_t aValue, int32_t rangeMin, int32_t rangeMax)
Convert from int32_t control value 0x80000000...0x7FFFFFFF to a int32_t ranging from rangeMin to rangeMax (linear)
- double [LongControlToDouble](#) (int32_t aValue, double firstVal, double secondVal)
Convert from int32_t control value 0x80000000...0x7FFFFFFF to an double ranging from firstVal to secondVal (linear)
- int32_t [DoubleToLongControl](#) (double aValue, double firstVal, double secondVal)
Convert from an double ranging from firstVal to secondVal (linear) to int32_t control value 0x80000000...0x7FFFFFFF
- int32_t [DoubleToLongControlNonlinear](#) (double aValue, double *minVal, double *rangePercent, int32_t numRanges)
- double [LongControlToDoubleNonlinear](#) (int32_t aValue, double *minVal, double *rangePercent, int32_t numRanges)
- double [LongControlToLogDouble](#) (int32_t aValue, double minVal, double maxVal)
Convert from int32_t control value 0x80000000...0x7FFFFFFF to an double ranging from minVal to maxVal (logarithmic)
- int32_t [LogDoubleToLongControl](#) (double aValue, double minVal, double maxVal)
Convert from an double ranging from minVal to maxVal (logarithmic) to int32_t control value 0x80000000...0x7FFFFFFF

15.171.2 Macro Definition Documentation

15.171.2.1 AAX_SLIDERCONVERSIONS_H

```
#define AAX_SLIDERCONVERSIONS_H
```

15.171.2.2 AAX_LIMIT

```
#define AAX_LIMIT(  
    v1,  
    firstVal,  
    secondVal ) ( (secondVal > firstVal) ? (std::max)((std::min)(v1,secondVal),firstVal)  
Val) : (std::min)((std::max)(v1,secondVal),firstVal) )
```

15.171.3 Function Documentation

15.171.3.1 LongControlToNewRange()

```
int32_t LongControlToNewRange (
    int32_t aValue,
    int32_t rangeMin,
    int32_t rangeMax )
```

15.171.3.2 LongToLongControl()

```
int32_t LongToLongControl (
    int32_t aValue,
    int32_t rangeMin,
    int32_t rangeMax )
```

Convert from int32_t control value 0x80000000...0x7FFFFFFF to a int32_t ranging from rangeMin to rangeMax (linear)

15.171.3.3 LongControlToDouble()

```
double LongControlToDouble (
    int32_t aValue,
    double firstVal,
    double secondVal )
```

Convert from int32_t control value 0x80000000...0x7FFFFFFF to an double ranging from firstVal to secondVal (linear)

15.171.3.4 DoubleToLongControl()

```
int32_t DoubleToLongControl (
    double aValue,
    double firstVal,
    double secondVal )
```

Convert from an double ranging from firstVal to secondVal (linear) to int32_t control value 0x80000000...0x7FFFFFFF.

15.171.3.5 DoubleToLongControlNonlinear()

```
int32_t DoubleToLongControlNonlinear (
    double aValue,
    double * minVal,
    double * rangePercent,
    int32_t numRanges )
```

15.171.3.6 LongControlToDoubleNonlinear()

```
double LongControlToDoubleNonlinear (
    int32_t aValue,
    double * minVal,
    double * rangePercent,
    int32_t numRanges )
```

15.171.3.7 LongControlToLogDouble()

```
double LongControlToLogDouble (
    int32_t aValue,
    double minVal,
    double maxVal )
```

Convert from int32_t control value 0x80000000...0x7FFFFFFF to an double ranging from minVal to maxVal (logarithmic)

Note

This is LOGARITHMIC, so minVal & maxVal have to be > zero!

15.171.3.8 LogDoubleToLongControl()

```
int32_t LogDoubleToLongControl (
    double aValue,
    double minVal,
    double maxVal )
```

Convert from an double ranging from minVal to maxVal (logarithmic) to int32_t control value 0x80000000...0x7FFFFFFF.

Note

This is LOGARITHMIC, so minVal & maxVal have to be > zero!

15.172 AAX_StringUtilities.h File Reference

```
#include "AAX.h"
#include "AAX_Enums.h"
#include <string>
#include "AAX_StringUtilities.hpp"
```

15.172.1 Description

Various string utility definitions for AAX Native.

Namespaces

- [AAX](#)

Macros

- `#define` [AAXLibrary_AAX_StringUtilities_h](#)

Functions

- void [AAX::GetCStringOfLength](#) (char *stringOut, const char *stringIn, int32_t aMaxChars)
=====
- int32_t [AAX::Caseless_strcmp](#) (const char *cs, const char *ct)
- std::string [AAX::Binary2String](#) (uint32_t binaryValue, int32_t numBits)
- uint32_t [AAX::String2Binary](#) (const [AAX_IString](#) &s)
- bool [AAX::IsASCII](#) (char inChar)
- bool [AAX::IsFourCharASCII](#) (uint32_t inFourChar)
- std::string [AAX::AsStringFourChar](#) (uint32_t inFourChar)
- std::string [AAX::AsStringPropertyValue](#) ([AAX_EProperty](#) inProperty, [AAX_CPropertyValue](#) inPropertyValue)
- std::string [AAX::AsStringInt32](#) (int32_t inInt32)
- std::string [AAX::AsStringUInt32](#) (uint32_t inUInt32)
- std::string [AAX::AsStringIDTriad](#) (const [AAX_SPlugInIdentifierTriad](#) &inIDTriad)
- std::string [AAX::AsStringStemFormat](#) ([AAX_EStemFormat](#) inStemFormat, bool inAbbreviate=false)
- std::string [AAX::AsStringStemChannel](#) ([AAX_EStemFormat](#) inStemFormat, uint32_t inChannelIndex, bool inAbbreviate)
- std::string [AAX::AsStringResult](#) ([AAX_Result](#) inResult)
- std::string [AAX::AsStringSupportLevel](#) ([AAX_ESupportLevel](#) inSupportLevel)

15.172.2 Macro Definition Documentation

15.172.2.1 AAXLibrary_AAX_StringUtilities_h

```
#define AAXLibrary_AAX_StringUtilities_h
```

15.173 AAX_StringUtilities.hpp File Reference

```
#include "AAX_IString.h"
#include "AAX_Errors.h"
#include "AAX_Assert.h"
#include <cstdlib>
#include <cstring>
#include <algorithm>
#include <sstream>
#include <string>
#include <vector>
```

Namespaces

- [AAX](#)
- [AAX::internal](#)

Macros

- `#define DEFINE\_AAX\_ERROR\_STRING(X) if (X == inResult) { return std::string(#X); }`

Functions

- `template<typename T >
std::string AAX::internal::ToHexadecimal (T inValue, bool inLeadingZeros=false)`

15.173.1 Macro Definition Documentation

15.173.1.1 [DEFINE_AAX_ERROR_STRING](#)

```
#define DEFINE\_AAX\_ERROR\_STRING(  
    X ) if (X == inResult) { return std::string(#X); }
```

15.174 [AAX_TI_Guide.doxygen](#) File Reference

15.175 [AAX_TransportTypes.h](#) File Reference

```
#include "AAX.h"  
#include <string>  
#include <sstream>  
#include "AAX_PreStructAlignmentHelper.h"  
#include "AAX_Push2ByteStructAlignment.h"  
#include "AAX_PostStructAlignmentHelper.h"  
#include "AAX_PopStructAlignment.h"
```

15.175.1 Description

Structures, enums and other definitions used in transport.

Classes

- struct [AAX_TransportStateInfo_V1](#)

Functions

- bool `operator==` (const `AAX_TransportStateInfo_V1` &state1, const `AAX_TransportStateInfo_V1` &state2)
- bool `operator!=` (const `AAX_TransportStateInfo_V1` &state1, const `AAX_TransportStateInfo_V1` &state2)

15.175.2 Function Documentation

15.175.2.1 `operator==()`

```
bool operator== (
    const AAX_TransportStateInfo_V1 & state1,
    const AAX_TransportStateInfo_V1 & state2 ) [inline]
```

References `AAX_TransportStateInfo_V1::mIsLoopEnabled`, `AAX_TransportStateInfo_V1::mIsRecordEnabled`, `AAX_TransportStateInfo_V1::mIsRecording`, `AAX_TransportStateInfo_V1::mRecordMode`, and `AAX_TransportStateInfo_V1::mTransportState`.

15.175.2.2 `operator!=()`

```
bool operator!= (
    const AAX_TransportStateInfo_V1 & state1,
    const AAX_TransportStateInfo_V1 & state2 ) [inline]
```

15.176 AAX_Troubleshooting.doxygen File Reference

15.177 AAX_UIDs.h File Reference

```
#include "acfbasetypes.h"
#include "defineacfuid.h"
#include "acfuids.h"
```

15.177.1 Description

Unique identifiers for AAX/ACF interfaces.

Variables

AAX Host interface IDs

- const [acfiID AAXCompID_HostServices](#)
ACF component ID for [AAX_IHostServices](#) components.
- const [acfiID IID_IAAXHostServicesV1](#)
ACF interface ID for [AAX_IACFHostServices](#).
- const [acfiID IID_IAAXHostServicesV2](#)
ACF interface ID for [AAX_IACFHostServices_V2](#).
- const [acfiID IID_IAAXHostServicesV3](#)
ACF interface ID for [AAX_IACFHostServices_V3](#).
- const [acfiID AAXCompID_AAXCollection](#)
ACF component ID for [AAX_ICollection](#) components.
- const [acfiID IID_IAAXCollectionV1](#)
ACF interface ID for [AAX_IACFCollection](#).
- const [acfiID AAXCompID_AAXEffectDescriptor](#)
ACF component ID for [AAX_IEffectDescriptor](#) components.
- const [acfiID IID_IAAXEffectDescriptorV1](#)
ACF interface ID for [AAX_IACFEffectDescriptor](#).
- const [acfiID IID_IAAXEffectDescriptorV2](#)
ACF interface ID for [AAX_IACFEffectDescriptor_V2](#).
- const [acfiID AAXCompID_AAXComponentDescriptor](#)
ACF component ID for [AAX_IComponentDescriptor](#) components.
- const [acfiID IID_IAAXComponentDescriptorV1](#)
ACF interface ID for [AAX_IACFComponentDescriptor](#).
- const [acfiID IID_IAAXComponentDescriptorV2](#)
ACF interface ID for [AAX_IACFComponentDescriptor_V2](#).
- const [acfiID IID_IAAXComponentDescriptorV3](#)
ACF interface ID for [AAX_IACFComponentDescriptor_V3](#).
- const [acfiID AAXCompID_AAXPropertyMap](#)
ACF component ID for [AAX_IPropertyMap](#) components.
- const [acfiID IID_IAAXPropertyMapV1](#)
ACF interface ID for [AAX_IACFPropertyMap](#).
- const [acfiID IID_IAAXPropertyMapV2](#)
ACF interface ID for [AAX_IACFPropertyMap_V2](#).
- const [acfiID IID_IAAXPropertyMapV3](#)
ACF interface ID for [AAX_IACFPropertyMap_V3](#).
- const [acfiID AAXCompID_HostProcessorDelegate](#)
ACF component ID for [AAX_IHostProcessorDelegate](#) components.
- const [acfiID IID_IAAXHostProcessorDelegateV1](#)
ACF interface ID for [AAX_IACFHostProcessorDelegate](#).
- const [acfiID IID_IAAXHostProcessorDelegateV2](#)
ACF interface ID for [AAX_IACFHostProcessorDelegate_V2](#).
- const [acfiID IID_IAAXHostProcessorDelegateV3](#)
ACF interface ID for [AAX_IACFHostProcessorDelegate_V3](#).
- const [acfiID AAXCompID_AutomationDelegate](#)
ACF component ID for [AAX_IAutomationDelegate](#) components.
- const [acfiID IID_IAAXAutomationDelegateV1](#)
ACF interface ID for [AAX_IACFAutomationDelegate](#).
- const [acfiID AAXCompID_Controller](#)
ACF component ID for [AAX_IController](#) components.
- const [acfiID IID_IAAXControllerV1](#)
ACF interface ID for [AAX_IACFController](#).
- const [acfiID IID_IAAXControllerV2](#)
ACF interface ID for [AAX_IACFController_V2](#).
- const [acfiID IID_IAAXControllerV3](#)
ACF interface ID for [AAX_IACFController_V3](#).
- const [acfiID AAXCompID_PageTableController](#)

- ACF component ID for AAX page table controller components.*

 - const [acfiID IID_IAAXPageTableController](#)

ACF interface ID for [AAX_IACFPageTableController](#).
- const [acfiID IID_IAAXPageTableControllerV2](#)

ACF interface ID for [AAX_IACFPageTableController_V2](#).
- const [acfiID AAXCompID_PrivateDataAccess](#)

ACF component ID for [AAX_IPrivateDataAccess](#) components.
- const [acfiID IID_IAAXPrivateDataAccessV1](#)

ACF interface ID for [AAX_IACFPrivateDataAccess](#).
- const [acfiID AAXCompID_ViewContainer](#)

ACF component ID for [AAX_IViewContainer](#) components.
- const [acfiID IID_IAAXViewContainerV1](#)

ACF interface ID for [AAX_IACFViewContainer](#).
- const [acfiID IID_IAAXViewContainerV2](#)

ACF interface ID for [AAX_IACFViewContainer_V2](#).
- const [acfiID IID_IAAXViewContainerV3](#)

ACF interface ID for [AAX_IACFViewContainer_V3](#).
- const [acfiID AAXCompID_Transport](#)

ACF component ID for [AAX_ITransport](#) components.
- const [acfiID IID_IAAXTransportV1](#)

ACF interface ID for [AAX_IACFTransport](#).
- const [acfiID IID_IAAXTransportV2](#)

ACF interface ID for [AAX_IACFTransport_V2](#).
- const [acfiID IID_IAAXTransportV3](#)

ACF interface ID for [AAX_IACFTransport_V3](#).
- const [acfiID IID_IAAXTransportV4](#)

ACF interface ID for [AAX_IACFTransport_V4](#).
- const [acfiID IID_IAAXTransportV5](#)

ACF interface ID for [AAX_IACFTransport_V5](#).
- const [acfiID AAXCompID_TransportControl](#)

ACF component ID for [AAX_ITransportControl](#) components (accessed via [AAX_ITransport](#))
- const [acfiID IID_IAAXTransportControlV1](#)

ACF interface ID for [AAX_IACFTransportControl](#).
- const [acfiID AAXCompID_PageTable](#)

ACF component ID for [AAX_IPageTable](#) components.
- const [acfiID IID_IAAXPageTableV1](#)

ACF interface ID for [AAX_IACFPageTable](#).
- const [acfiID IID_IAAXPageTableV2](#)

ACF interface ID for [AAX_IACFPageTable_V2](#).
- const [acfiID AAX_CompID_DescriptionHost](#)

ACF component ID for [AAX_IDescriptionHost](#) components.
- const [acfiID IID_IAAXDescriptionHostV1](#)

ACF interface ID for [AAX_IACFDescriptionHost](#).
- const [acfiID AAX_CompID_FeatureInfo](#)

ACF component ID for [AAX_IFeatureInfo](#) components.
- const [acfiID IID_IAAXFeatureInfoV1](#)

ACF interface ID for [AAX_IACFFeatureInfo](#).
- const [acfiID AAXCompID_Task](#)

ACF component ID for [AAX_ITask](#) components.
- const [acfiID IID_IAAXTaskV1](#)

ACF interface ID for [AAX_IACFTask](#).
- const [acfiID AAXCompID_SessionDocument](#)

ACF component ID for [AAX_ISessionDocument](#) components.
- const [acfiID IID_IAAXSessionDocumentV1](#)

ACF interface ID for [AAX_IACFSessionDocument](#).

AAX plug-in interface IDs

- const [acfiID AAXCompID_EffectParameters](#)
ACF component ID for [AAX_IEffectParameters](#) components.
- const [acfiID IID_IAAXEffectParametersV1](#)
ACF interface ID for [AAX_IACFEffectParameters](#).
- const [acfiID IID_IAAXEffectParametersV2](#)
ACF interface ID for [AAX_IACFEffectParameters_V2](#).
- const [acfiID IID_IAAXEffectParametersV3](#)
ACF interface ID for [AAX_IACFEffectParameters_V3](#).
- const [acfiID IID_IAAXEffectParametersV4](#)
ACF interface ID for [AAX_IACFEffectParameters_V4](#).
- const [acfiID AAXCompID_HostProcessor](#)
ACF component ID for [AAX_IHostProcessor](#) components.
- const [acfiID IID_IAAXHostProcessorV1](#)
ACF interface ID for [AAX_IACFHostProcessor](#).
- const [acfiID IID_IAAXHostProcessorV2](#)
ACF interface ID for [AAX_IACFHostProcessor_V2](#).
- const [acfiID AAXCompID_EffectGUI](#)
ACF component ID for [AAX_IEffectGUI](#) components.
- const [acfiID IID_IAAXEffectGUIV1](#)
ACF interface ID for [AAX_IACFEffectGUI](#).
- const [acfiID AAXCompID_EffectDirectData](#)
ACF component ID for [AAX_IEffectDirectData](#) components.
- const [acfiID IID_IAAXEffectDirectDataV1](#)
ACF interface ID for [AAX_IACFEffectDirectData](#).
- const [acfiID IID_IAAXEffectDirectDataV2](#)
- const [acfiID AAXCompID_TaskAgent](#)
ACF component ID for [AAX_ITaskAgent](#) components.
- const [acfiID IID_IAAXTaskAgentV1](#)
ACF interface ID for [AAX_IACFTaskAgent](#).
- const [acfiID AAXCompID_SessionDocumentClient](#)
ACF component ID for [AAX_ISessionDocumentClient](#) components.
- const [acfiID IID_IAAXSessionDocumentClientV1](#)
ACF interface ID for [AAX_IACFSessionDocumentClient](#).

Other AAX interface IDs

- const [acfiID AAXCompID_DataBuffer](#)
ACF component ID for [AAX_IDataBuffer](#) components.
- const [acfiID IID_IAAXDataBufferV1](#)
ACF interface ID for [AAX_IACFDataBuffer](#).

AAX host attributes

- const [acfUID AAXATTR_Client_Level](#)
Client application level.
- const [acfUID AAXATTR_Client_Version](#)
Client application version.

AAX Feature UIDs

- using [AAX_Feature_UID](#) = [acfUID](#)
- const [AAX_Feature_UID AAXATTR_ClientFeature_StemFormat](#)
Client stem format feature support.
- const [AAX_Feature_UID AAXATTR_ClientFeature_AuxOutputStem](#)
*Client *Auxiliary Output Stem* feature support.*
- const [AAX_Feature_UID AAXATTR_ClientFeature_SideChainInput](#)
- const [AAX_Feature_UID AAXATTR_ClientFeature_MIDI](#)
*Client *MIDI* feature support.*

AAX document data type UUIDs

- using [AAX_DocumentData_UUID](#) = acfUUID
- const [AAX_DocumentData_UUID](#) [AAX_DocumentDataType_TempoMap](#)

15.177.2 Typedef Documentation

15.177.2.1 AAX_Feature_UUID

using [AAX_Feature_UUID](#) = acfUUID

Identifier for AAX features

See [AAX_IDescriptionHost::AcquireFeatureProperties\(\)](#) and [AAX_IFeatureInfo](#)

15.177.2.2 AAX_DocumentData_UUID

using [AAX_DocumentData_UUID](#) = acfUUID

Identifier for AAX document data types

See also

[AAX_IACFSessionDocument](#)

15.177.3 Variable Documentation

15.177.3.1 AAXCompID_HostServices

const [acfIID](#) [AAXCompID_HostServices](#)

ACF component ID for [AAX_IHostServices](#) components.

15.177.3.2 IID_IAAXHostServicesV1

const [acfIID](#) [IID_IAAXHostServicesV1](#)

ACF interface ID for [AAX_IACFHostServices](#).

15.177.3.3 IID_IAAXHostServicesV2

```
const acfIID IID_IAAXHostServicesV2
```

ACF interface ID for [AAX_IACFHostServices_V2](#).

15.177.3.4 IID_IAAXHostServicesV3

```
const acfIID IID_IAAXHostServicesV3
```

ACF interface ID for [AAX_IACFHostServices_V3](#).

15.177.3.5 AAXCompID_AAXCollection

```
const acfIID AAXCompID_AAXCollection
```

ACF component ID for [AAX_ICollection](#) components.

15.177.3.6 IID_IAAXCollectionV1

```
const acfIID IID_IAAXCollectionV1
```

ACF interface ID for [AAX_IACFCollection](#).

15.177.3.7 AAXCompID_AAXEffectDescriptor

```
const acfIID AAXCompID_AAXEffectDescriptor
```

ACF component ID for [AAX_IEffectDescriptor](#) components.

15.177.3.8 IID_IAAXEffectDescriptorV1

```
const acfIID IID_IAAXEffectDescriptorV1
```

ACF interface ID for [AAX_IACFEffectDescriptor](#).

15.177.3.9 IID_IAAXEffectDescriptorV2

```
const acfIID IID_IAAXEffectDescriptorV2
```

ACF interface ID for [AAX_IACFEffectDescriptor_V2](#).

15.177.3.10 AAXCompID_AAXComponentDescriptor

```
const acfIID AAXCompID_AAXComponentDescriptor
```

ACF component ID for [AAX_IComponentDescriptor](#) components.

15.177.3.11 IID_IAAXComponentDescriptorV1

```
const acfIID IID_IAAXComponentDescriptorV1
```

ACF interface ID for [AAX_IACFComponentDescriptor](#).

15.177.3.12 IID_IAAXComponentDescriptorV2

```
const acfIID IID_IAAXComponentDescriptorV2
```

ACF interface ID for [AAX_IACFComponentDescriptor_V2](#).

15.177.3.13 IID_IAAXComponentDescriptorV3

```
const acfIID IID_IAAXComponentDescriptorV3
```

ACF interface ID for [AAX_IACFComponentDescriptor_V3](#).

15.177.3.14 AAXCompID_AAXPropertyMap

```
const acfIID AAXCompID_AAXPropertyMap
```

ACF component ID for [AAX_IPropertyMap](#) components.

15.177.3.15 IID_IAAXPropertyMapV1

```
const acfIID IID_IAAXPropertyMapV1
```

ACF interface ID for [AAX_IACFPropertyMap](#).

15.177.3.16 IID_IAAXPropertyMapV2

```
const acfIID IID_IAAXPropertyMapV2
```

ACF interface ID for [AAX_IACFPropertyMap_V2](#).

15.177.3.17 IID_IAAXPropertyMapV3

```
const acfIID IID_IAAXPropertyMapV3
```

ACF interface ID for [AAX_IACFPropertyMap_V3](#).

15.177.3.18 AAXCompID_HostProcessorDelegate

```
const acfIID AAXCompID_HostProcessorDelegate
```

ACF component ID for [AAX_IHostProcessorDelegate](#) components.

15.177.3.19 IID_IAAXHostProcessorDelegateV1

```
const acfIID IID_IAAXHostProcessorDelegateV1
```

ACF interface ID for [AAX_IACFHostProcessorDelegate](#).

15.177.3.20 IID_IAAXHostProcessorDelegateV2

```
const acfIID IID_IAAXHostProcessorDelegateV2
```

ACF interface ID for [AAX_IACFHostProcessorDelegate_V2](#).

15.177.3.21 IID_IAAXHostProcessorDelegateV3

```
const acfIID IID_IAAXHostProcessorDelegateV3
```

ACF interface ID for [AAX_IACFHostProcessorDelegate_V3](#).

15.177.3.22 AAXCompID_AutomationDelegate

```
const acfIID AAXCompID_AutomationDelegate
```

ACF component ID for [AAX_IAutomationDelegate](#) components.

15.177.3.23 IID_IAAXAutomationDelegateV1

```
const acfIID IID_IAAXAutomationDelegateV1
```

ACF interface ID for [AAX_IACFAutomationDelegate](#).

15.177.3.24 AAXCompID_Controller

```
const acfIID AAXCompID_Controller
```

ACF component ID for [AAX_IController](#) components.

15.177.3.25 IID_IAAXControllerV1

```
const acfIID IID_IAAXControllerV1
```

ACF interface ID for [AAX_IACFController](#).

15.177.3.26 IID_IAAXControllerV2

```
const acfIID IID_IAAXControllerV2
```

ACF interface ID for [AAX_IACFController_V2](#).

15.177.3.27 IID_IAAXControllerV3

```
const acfIID IID_IAAXControllerV3
```

ACF interface ID for [AAX_IACFController_V3](#).

15.177.3.28 AAXCompID_PageTableController

```
const acfIID AAXCompID_PageTableController
```

ACF component ID for AAX page table controller components.

15.177.3.29 IID_IAAXPageTableController

```
const acfIID IID_IAAXPageTableController
```

ACF interface ID for [AAX_IACFPageTableController](#).

15.177.3.30 IID_IAAXPageTableControllerV2

```
const acfIID IID_IAAXPageTableControllerV2
```

ACF interface ID for [AAX_IACFPageTableController_V2](#).

15.177.3.31 AAXCompID_PrivateDataAccess

```
const acfIID AAXCompID_PrivateDataAccess
```

ACF component ID for [AAX_IPrivateDataAccess](#) components.

15.177.3.32 IID_IAAXPrivateDataAccessV1

```
const acfIID IID_IAAXPrivateDataAccessV1
```

ACF interface ID for [AAX_IACFPrivateDataAccess](#).

15.177.3.33 AAXCompID_ViewContainer

```
const acfIID AAXCompID_ViewContainer
```

ACF component ID for [AAX_IViewContainer](#) components.

15.177.3.34 IID_IAAXViewContainerV1

```
const acfIID IID_IAAXViewContainerV1
```

ACF interface ID for [AAX_IACFViewContainer](#).

15.177.3.35 IID_IAAXViewContainerV2

```
const acfIID IID_IAAXViewContainerV2
```

ACF interface ID for [AAX_IACFViewContainer_V2](#).

15.177.3.36 IID_IAAXViewContainerV3

```
const acfIID IID_IAAXViewContainerV3
```

ACF interface ID for [AAX_IACFViewContainer_V3](#).

15.177.3.37 AAXCompID_Transport

```
const acfIID AAXCompID_Transport
```

ACF component ID for [AAX_ITransport](#) components.

15.177.3.38 IID_IAAXTransportV1

```
const acfIID IID_IAAXTransportV1
```

ACF interface ID for [AAX_IACFTransport](#).

15.177.3.39 IID_IAAXTransportV2

```
const acfIID IID_IAAXTransportV2
```

ACF interface ID for [AAX_IACFTransport_V2](#).

15.177.3.40 IID_IAAXTransportV3

```
const acfIID IID_IAAXTransportV3
```

ACF interface ID for [AAX_IACFTransport_V3](#).

15.177.3.41 IID_IAAXTransportV4

```
const acfIID IID_IAAXTransportV4
```

ACF interface ID for [AAX_IACFTransport_V4](#).

15.177.3.42 IID_IAAXTransportV5

```
const acfIID IID_IAAXTransportV5
```

ACF interface ID for [AAX_IACFTransport_V5](#).

15.177.3.43 AAXCompID_TransportControl

```
const acfIID AAXCompID_TransportControl
```

ACF component ID for AAX_ITransportControl components (accessed via [AAX_ITransport](#))

15.177.3.44 IID_IAAXTransportControlV1

```
const acfIID IID_IAAXTransportControlV1
```

ACF interface ID for [AAX_IACFTransportControl](#).

15.177.3.45 AAXCompID_PageTable

```
const acfIID AAXCompID_PageTable
```

ACF component ID for [AAX_IPageTable](#) components.

15.177.3.46 IID_IAAXPageTableV1

```
const acfIID IID_IAAXPageTableV1
```

ACF interface ID for [AAX_IACFPageTable](#).

15.177.3.47 IID_IAAXPageTableV2

```
const acfIID IID_IAAXPageTableV2
```

ACF interface ID for [AAX_IACFPageTable_V2](#).

15.177.3.48 AAX_CompID_DescriptionHost

```
const acfIID AAX_CompID_DescriptionHost
```

ACF component ID for [AAX_IDescriptionHost](#) components.

15.177.3.49 IID_IAAXDescriptionHostV1

```
const acfIID IID_IAAXDescriptionHostV1
```

ACF interface ID for [AAX_IACFDescriptionHost](#).

15.177.3.50 AAX_CompID_FeatureInfo

```
const acfIID AAX_CompID_FeatureInfo
```

ACF component ID for [AAX_IFeatureInfo](#) components.

15.177.3.51 IID_IAAXFeatureInfoV1

```
const acfIID IID_IAAXFeatureInfoV1
```

ACF interface ID for [AAX_IACFFeatureInfo](#).

15.177.3.52 AAXCompID_Task

```
const acfIID AAXCompID_Task
```

ACF component ID for [AAX_ITask](#) components.

15.177.3.53 IID_IAAXTaskV1

```
const acfIID IID_IAAXTaskV1
```

ACF interface ID for [AAX_IACFTask](#).

15.177.3.54 AAXCompID_SessionDocument

```
const acfIID AAXCompID_SessionDocument
```

ACF component ID for [AAX_ISessionDocument](#) components.

15.177.3.55 IID_IAAXSessionDocumentV1

```
const acfIID IID_IAAXSessionDocumentV1
```

ACF interface ID for [AAX_IACFSessionDocument](#).

15.177.3.56 AAXCompID_EffectParameters

```
const acfIID AAXCompID_EffectParameters
```

ACF component ID for [AAX_IEffectParameters](#) components.

15.177.3.57 IID_IAAXEffectParametersV1

```
const acfIID IID_IAAXEffectParametersV1
```

ACF interface ID for [AAX_IACFEffectParameters](#).

15.177.3.58 IID_IAAXEffectParametersV2

```
const acfIID IID_IAAXEffectParametersV2
```

ACF interface ID for [AAX_IACFEffectParameters_V2](#).

15.177.3.59 IID_IAAXEffectParametersV3

```
const acfIID IID_IAAXEffectParametersV3
```

ACF interface ID for [AAX_IACFEffectParameters_V3](#).

15.177.3.60 IID_IAAXEffectParametersV4

```
const acfIID IID_IAAXEffectParametersV4
```

ACF interface ID for [AAX_IACFEffectParameters_V4](#).

15.177.3.61 AAXCompID_HostProcessor

```
const acfIID AAXCompID_HostProcessor
```

ACF component ID for [AAX_IHostProcessor](#) components.

15.177.3.62 IID_IAAXHostProcessorV1

```
const acfIID IID_IAAXHostProcessorV1
```

ACF interface ID for [AAX_IACFHostProcessor](#).

15.177.3.63 IID_IAAXHostProcessorV2

```
const acfIID IID_IAAXHostProcessorV2
```

ACF interface ID for [AAX_IACFHostProcessor_V2](#).

15.177.3.64 AAXCompID_EffectGUI

```
const acfIID AAXCompID_EffectGUI
```

ACF component ID for [AAX_IEffectGUI](#) components.

15.177.3.65 IID_IAAXEffectGUIV1

```
const acfIID IID_IAAXEffectGUIV1
```

ACF interface ID for [AAX_IACFEffectGUI](#).

15.177.3.66 AAXCompID_EffectDirectData

```
const acfIID AAXCompID_EffectDirectData
```

ACF component ID for [AAX_IEffectDirectData](#) components.

15.177.3.67 IID_IAAXEffectDirectDataV1

```
const acfIID IID_IAAXEffectDirectDataV1
```

ACF interface ID for [AAX_IACFEffectDirectData](#).

15.177.3.68 IID_IAAXEffectDirectDataV2

```
const acfIID IID_IAAXEffectDirectDataV2
```

15.177.3.69 AAXCompID_TaskAgent

```
const acfIID AAXCompID_TaskAgent
```

ACF component ID for [AAX_ITaskAgent](#) components.

15.177.3.70 IID_IAAXTaskAgentV1

```
const acfIID IID_IAAXTaskAgentV1
```

ACF interface ID for [AAX_IACFTaskAgent](#).

15.177.3.71 AAXCompID_SessionDocumentClient

```
const acfIID AAXCompID_SessionDocumentClient
```

ACF component ID for [AAX_ISessionDocumentClient](#) components.

15.177.3.72 IID_IAAXSessionDocumentClientV1

```
const acfIID IID_IAAXSessionDocumentClientV1
```

ACF interface ID for [AAX_IACFSessionDocumentClient](#).

15.177.3.73 AAXCompID_DataBuffer

```
const acfIID AAXCompID_DataBuffer
```

ACF component ID for [AAX_IDataBuffer](#) components.

15.177.3.74 IID_IAAXDataBufferV1

```
const acfIID IID_IAAXDataBufferV1
```

ACF interface ID for [AAX_IACFDataBuffer](#).

15.177.3.75 AAXATTR_ClientFeature_StemFormat

AAXATTR_ClientFeature_StemFormat

Client stem format feature support.

To determine the client's support for specific stem formats, use the property map

Property map contents Key: [AAX_EStemFormat](#) values Value: [AAX_ESupportLevel](#) value; if undefined then no information is available

15.177.3.76 AAXATTR_ClientFeature_AuxOutputStem

AAXATTR_ClientFeature_AuxOutputStem

Client [Auxiliary Output Stem](#) feature support.

Client [Side Chain](#) feature support.

Plug-ins must detect when a host does not support AOS in order to avoid running off the end of the output audio buffer list in the audio algorithm.

[AddAuxOutputStem\(\)](#) will return an error for hosts that do not support this feature, so typically a feature support query using this [AAX_Feature_UID](#) is not required.

15.177.3.77 AAXATTR_ClientFeature_SideChainInput

const [AAX_Feature_UID](#) AAXATTR_ClientFeature_SideChainInput

15.177.3.78 AAXATTR_ClientFeature_MIDI

AAXATTR_ClientFeature_MIDI

Client [MIDI](#) feature support.

15.177.3.79 AAXATTR_Client_Level

AAXATTR_Client_Level

Client application level.

Type: `uint32_t` (`ACFTypeID_UInt32`) Value: one of [AAX_EHostLevel](#)

Query using the host's [IACFDefinition](#)

15.177.3.80 AAXATTR_Client_Version

AAXATTR_Client_Version

Client application version.

Type: `uint32_t` (`ACFTypeID_UInt32`)

The value contains the host version in 3 sections:

- First section - 16 bits - major version
- Second section - 8 bits - minor version
- Third section - 8 bits - revision version.

e.g. for 2023.3.1 (major.minor.revision):

```
major - 00000111111100111
minor - 00000011
revision - 00000001
```

in a result value this would be represented as : 000001111111001110000001100000001, or in decimal: 132580097

Query using the host's [IACFDefinition](#)

15.177.3.81 AAX_DocumentDataType_TempoMap

AAX_DocumentDataType_TempoMap

The session tempo map

Provides an [AAX_IACFDataBuffer](#) containing a list of [AAX_CTempoBreakpoint](#) elements.

15.178 AAX_UtilsNative.h File Reference

```
#include "AAX_CString.h"
#include "AAX_IString.h"
#include "AAX_Assert.h"
#include "AAX.h"
#include <cmath>
#include <string.h>
```

15.178.1 Description

Various utility definitions for AAX Native.

Namespaces

- [AAX](#)

Macros

- `#define \_AAX\_UTILSNATIVE\_H\_`

Functions

- double [AAX::SafeLog](#) (double aValue)
Double-precision safe log function. Returns zero for input values that are ≤ 0.0 .
- float [AAX::SafeLogf](#) (float aValue)
Single-precision safe log function. Returns zero for input values that are ≤ 0.0 .
- [AAX_CBoolean AAX::IsParameterIDEqual](#) ([AAX_CParamID](#) iParam1, [AAX_CParamID](#) iParam2)
Helper function to check if two parameter IDs are equivalent.
- [AAX_CBoolean AAX::IsEffectIDEqual](#) (const [AAX_IString](#) *iEffectID1, const [AAX_IString](#) *iEffectID2)
Helper function to check if two Effect IDs are equivalent.
- [AAX_CBoolean AAX::IsAvidNotification](#) ([AAX_CTypeID](#) inNotificationID)
Helper function to check if a notification ID is reserved for host notifications.

15.178.2 Macro Definition Documentation

15.178.2.1 [_AAX_UTILSNATIVE_H_](#)

```
#define \_AAX\_UTILSNATIVE\_H\_
```

15.179 AAX_VAutomationDelegate.h File Reference

```
#include "AAX_IAutomationDelegate.h"
#include "AAX_IACFAutomationDelegate.h"
#include "ACFPtr.h"
```

15.179.1 Description

Version-managed concrete AutomationDelegate class.

Classes

- class [AAX_VAutomationDelegate](#)
Version-managed concrete [automation delegate](#) class.

15.180 AAX_VCollection.h File Reference

```
#include "AAX.h"
#include "AAX_ICollection.h"
#include "AAX_IACFCollection.h"
#include "AAX_VDescriptionHost.h"
#include "acfunknown.h"
#include "ACFPtr.h"
#include <set>
```

15.180.1 Description

Version-managed concrete Collection class.

Classes

- class [AAX_VCollection](#)
Version-managed concrete [AAX_ICollection](#) class.

15.181 AAX_VComponentDescriptor.h File Reference

```
#include "AAX_IComponentDescriptor.h"
#include "AAX_IDma.h"
#include "AAX_IACFComponentDescriptor.h"
#include "acfunknown.h"
#include "ACFPtr.h"
#include <set>
```

15.181.1 Description

Version-managed concrete ComponentDescriptor class.

Classes

- class [AAX_VComponentDescriptor](#)
Version-managed concrete [AAX_IComponentDescriptor](#) class.

15.182 AAX_VController.h File Reference

```
#include "AAX_IController.h"
#include "AAX_IACFController.h"
#include "ACFPtr.h"
```

15.182.1 Description

Version-managed concrete Controller class.

Classes

- class [AAX_VController](#)
Version-managed concrete [Controller](#) class.

15.183 AAX_VDataBufferWrapper.h File Reference

```
#include "AAX_IDataBufferWrapper.h"  
#include "ACFPtr.h"
```

Classes

- class [AAX_VDataBufferWrapper](#)
Wrapper for an [AAX_IDataBuffer](#).

Macros

- #define [AAX_VDATABUFFERWRAPPER_H](#)

15.183.1 Macro Definition Documentation

15.183.1.1 AAX_VDATABUFFERWRAPPER_H

```
#define AAX_VDATABUFFERWRAPPER_H
```

15.184 AAX_VDescriptionHost.h File Reference

```
#include "AAX_IDescriptionHost.h"  
#include "ACFPtr.h"
```

Classes

- class [AAX_VDescriptionHost](#)

15.185 AAX_VEffectDescriptor.h File Reference

```
#include "AAX.h"
#include "AAX_IEffectDescriptor.h"
#include "AAX_IACFEffectDescriptor.h"
#include "acfunknown.h"
#include "ACFPtr.h"
#include <set>
#include <map>
```

15.185.1 Description

Version-managed concrete EffectDescriptor class.

Classes

- class [AAX_VEffectDescriptor](#)
Version-managed concrete [AAX_IEffectDescriptor](#) class.

15.186 AAX_VENUE_Guide.doxygen File Reference

15.187 AAX_Version.h File Reference

15.187.1 Description

Version stamp header for the AAX SDK.

This file defines a unique number that can be used to identify the version of the AAX SDK

Macros

- #define [_AAX_VERSION_H_](#)
- #define [AAX_SDK_VERSION](#) (0x0208)
The SDK's version number.
- #define [AAX_SDK_CURRENT_REVISION](#) (20208001)
An atomic revision number for the source included in this SDK.
- #define [AAX_SDK_1p0p1_REVISION](#) (3712639)
- #define [AAX_SDK_1p0p2_REVISION](#) (3780585)
- #define [AAX_SDK_1p0p3_REVISION](#) (3895859)
- #define [AAX_SDK_1p0p4_REVISION](#) (4333589)
- #define [AAX_SDK_1p0p5_REVISION](#) (4598560)
- #define [AAX_SDK_1p0p6_REVISION](#) (5051497)
- #define [AAX_SDK_1p5p0_REVISION](#) (5740047)
- #define [AAX_SDK_2p0b1_REVISION](#) (6169787)
- #define [AAX_SDK_2p0p0_REVISION](#) (6307708)
- #define [AAX_SDK_2p0p1_REVISION](#) (6361692)

- `#define AAX_SDK_2p1p0_REVISION ( 7820991 )`
- `#define AAX_SDK_2p1p1_REVISION ( 8086416 )`
- `#define AAX_SDK_2p2p0_REVISION ( 9967334 )`
- `#define AAX_SDK_2p2p1_REVISION ( 10693954 )`
- `#define AAX_SDK_2p2p2_REVISION ( 11819832 )`
- `#define AAX_SDK_2p3p0_REVISION ( 12546840 )`
- `#define AAX_SDK_2p3p1_REVISION ( 13200373 )`
- `#define AAX_SDK_2p3p2_REVISION ( 14017972 )`
- `#define AAX_SDK_2p4p0_REVISION ( 20204000 )`
- `#define AAX_SDK_2p4p1_REVISION ( 20204010 )`
- `#define AAX_SDK_2p5p0_REVISION ( 20205000 )`
- `#define AAX_SDK_2p6p0_REVISION ( 20206000 )`
- `#define AAX_SDK_2p6p1_REVISION ( 20206001 )`
- `#define AAX_SDK_2p7p0_REVISION ( 20207000 )`
- `#define AAX_SDK_2p8p0_REVISION ( 20208000 )`
- `#define AAX_SDK_2p8p1_REVISION ( 20208001 )`

15.187.2 Macro Definition Documentation

15.187.2.1 `_AAX_VERSION_H_`

```
#define _AAX_VERSION_H_
```

15.187.2.2 `AAX_SDK_VERSION`

```
#define AAX_SDK_VERSION ( 0x0208 )
```

The SDK's version number.

This version number is generally updated only when changes have been made to the AAX binary interface

- The first byte is the major version number
- The second byte is the minor version number

For example:

- SDK 1.0.5 > 0x0100
- SDK 10.2.1 > 0x0A02

15.187.2.3 AAX_SDK_CURRENT_REVISION

```
#define AAX_SDK_CURRENT_REVISION ( 20208001 )
```

An atomic revision number for the source included in this SDK.

15.187.2.4 AAX_SDK_1p0p1_REVISION

```
#define AAX_SDK_1p0p1_REVISION ( 3712639 )
```

15.187.2.5 AAX_SDK_1p0p2_REVISION

```
#define AAX_SDK_1p0p2_REVISION ( 3780585 )
```

15.187.2.6 AAX_SDK_1p0p3_REVISION

```
#define AAX_SDK_1p0p3_REVISION ( 3895859 )
```

15.187.2.7 AAX_SDK_1p0p4_REVISION

```
#define AAX_SDK_1p0p4_REVISION ( 4333589 )
```

15.187.2.8 AAX_SDK_1p0p5_REVISION

```
#define AAX_SDK_1p0p5_REVISION ( 4598560 )
```

15.187.2.9 AAX_SDK_1p0p6_REVISION

```
#define AAX_SDK_1p0p6_REVISION ( 5051497 )
```

15.187.2.10 AAX_SDK_1p5p0_REVISION

```
#define AAX_SDK_1p5p0_REVISION ( 5740047 )
```

15.187.2.11 AAX_SDK_2p0b1_REVISION

```
#define AAX_SDK_2p0b1_REVISION ( 6169787 )
```

15.187.2.12 AAX_SDK_2p0p0_REVISION

```
#define AAX_SDK_2p0p0_REVISION ( 6307708 )
```

15.187.2.13 AAX_SDK_2p0p1_REVISION

```
#define AAX_SDK_2p0p1_REVISION ( 6361692 )
```

15.187.2.14 AAX_SDK_2p1p0_REVISION

```
#define AAX_SDK_2p1p0_REVISION ( 7820991 )
```

15.187.2.15 AAX_SDK_2p1p1_REVISION

```
#define AAX_SDK_2p1p1_REVISION ( 8086416 )
```

15.187.2.16 AAX_SDK_2p2p0_REVISION

```
#define AAX_SDK_2p2p0_REVISION ( 9967334 )
```

15.187.2.17 AAX_SDK_2p2p1_REVISION

```
#define AAX_SDK_2p2p1_REVISION ( 10693954 )
```


15.187.2.18 AAX_SDK_2p2p2_REVISION

```
#define AAX_SDK_2p2p2_REVISION ( 11819832 )
```

15.187.2.19 AAX_SDK_2p3p0_REVISION

```
#define AAX_SDK_2p3p0_REVISION ( 12546840 )
```

15.187.2.20 AAX_SDK_2p3p1_REVISION

```
#define AAX_SDK_2p3p1_REVISION ( 13200373 )
```

15.187.2.21 AAX_SDK_2p3p2_REVISION

```
#define AAX_SDK_2p3p2_REVISION ( 14017972 )
```

15.187.2.22 AAX_SDK_2p4p0_REVISION

```
#define AAX_SDK_2p4p0_REVISION ( 20204000 )
```

15.187.2.23 AAX_SDK_2p4p1_REVISION

```
#define AAX_SDK_2p4p1_REVISION ( 20204010 )
```

15.187.2.24 AAX_SDK_2p5p0_REVISION

```
#define AAX_SDK_2p5p0_REVISION ( 20205000 )
```

15.187.2.25 AAX_SDK_2p6p0_REVISION

```
#define AAX_SDK_2p6p0_REVISION ( 20206000 )
```

15.187.2.26 AAX_SDK_2p6p1_REVISION

```
#define AAX_SDK_2p6p1_REVISION ( 20206001 )
```

15.187.2.27 AAX_SDK_2p7p0_REVISION

```
#define AAX_SDK_2p7p0_REVISION ( 20207000 )
```

15.187.2.28 AAX_SDK_2p8p0_REVISION

```
#define AAX_SDK_2p8p0_REVISION ( 20208000 )
```

15.187.2.29 AAX_SDK_2p8p1_REVISION

```
#define AAX_SDK_2p8p1_REVISION ( 20208001 )
```

15.188 AAX_VFeatureInfo.h File Reference

```
#include "AAX_IFeatureInfo.h"  
#include "ACFPtr.h"  
#include "acfbasetypes.h"
```

Classes

- class [AAX_VFeatureInfo](#)

15.189 AAX_VHostProcessorDelegate.h File Reference

```
#include "AAX_IHostProcessorDelegate.h"  
#include "AAX_IACFHostProcessorDelegate.h"  
#include "ACFPtr.h"
```

15.189.1 Description

Version-managed concrete HostProcessorDelegate class.

Classes

- class [AAX_VHostProcessorDelegate](#)
Version-managed concrete [Host Processor delegate](#) class.

15.190 AAX_VHostServices.h File Reference

```
#include "AAX_IHostServices.h"
#include "AAX.h"
#include "acfunknown.h"
#include "ACFPtr.h"
#include "AAX_IACFHostServices.h"
```

15.190.1 Description

Version-managed concrete HostServices class.

Classes

- class [AAX_VHostServices](#)
Version-managed concrete [AAX_IHostServices](#) class.

15.191 AAX_VHostTaskAgent.h File Reference

```
#include "AAX_IHostTaskAgent.h"
#include "ACFPtr.h"
```

Classes

- class [AAX_VHostTaskAgent](#)

Macros

- #define [AAX_VHostTaskAgent_H](#)

15.191.1 Macro Definition Documentation

15.191.1.1 AAX_VHostTaskAgent_H

```
#define AAX_VHostTaskAgent_H
```

15.192 AAX_VPageTable.h File Reference

```
#include "AAX_IPageTable.h"  
#include "AAX_IACFPageTable.h"  
#include "ACFPtr.h"
```

Classes

- class [AAX_VPageTable](#)
Version-managed concrete [AAX_IPageTable](#) class.

15.193 AAX_VPrivateDataAccess.h File Reference

```
#include "AAX_IPrivateDataAccess.h"  
#include "AAX_IACFPPrivateDataAccess.h"  
#include "ACFPtr.h"
```

15.193.1 Description

Version-managed concrete PrivateDataAccess class.

Classes

- class [AAX_VPrivateDataAccess](#)
Version-managed concrete [AAX_IPrivateDataAccess](#) class.

15.194 AAX_VPropertyMap.h File Reference

```
#include "AAX_IPropertyMap.h"  
#include "AAX_IACFPropertyMap.h"  
#include "AAX.h"  
#include "acfunknown.h"  
#include "ACFPtr.h"  
#include <map>
```

15.194.1 Description

Version-managed concrete PropertyMap class.

Classes

- class [AAX_VPropertyMap](#)
Version-managed concrete [AAX_IPropertyMap](#) class.

15.195 AAX_VSessionDocument.h File Reference

```
#include "AAX_ISessionDocument.h"  
#include "ACFPtr.h"
```

Classes

- class [AAX_VSessionDocument](#)
- class [AAX_VSessionDocument::VTempoMap](#)

Macros

- #define [AAX_VSessionDocument_H](#)

15.195.1 Macro Definition Documentation

15.195.1.1 AAX_VSessionDocument_H

```
#define AAX_VSessionDocument_H
```

15.196 AAX_VTask.h File Reference

```
#include "AAX_ITask.h"  
#include "AAX.h"  
#include "ACFPtr.h"
```

Classes

- class [AAX_VTask](#)
Version-managed concrete [AAX_ITask](#).

Macros

- #define [AAX_VTask_H](#)

15.196.1 Macro Definition Documentation

15.196.1.1 AAX_VTask_H

```
#define AAX_VTask_H
```

15.197 AAX_VTransport.h File Reference

```
#include "AAX_ITransport.h"  
#include "AAX_IACFTransport.h"  
#include "AAX_IACFTransportControl.h"  
#include "ACFPtr.h"
```

15.197.1 Description

Version-managed concrete Transport class.

Classes

- class [AAX_VTransport](#)
Version-managed concrete [AAX_ITransport](#) class.

15.198 AAX_VViewContainer.h File Reference

```
#include "AAX_IViewContainer.h"  
#include "AAX_IACFViewContainer.h"  
#include "ACFPtr.h"
```

15.198.1 Description

Version-managed concrete ViewContainer class.

Classes

- class [AAX_VViewContainer](#)
Version-managed concrete [AAX_IViewContainer](#) class.

15.199 DSH_Guide.doxygen File Reference

15.200 DTT_Guide.doxygen File Reference

15.201 ReadMe.doxygen File Reference

Index

- [_AAX_CAUTORELEASEPOOL_H_](#)
 - [AAX_CAutoreleasePool.h, 1132](#)
- [_AAX_FASTPOW_H_](#)
 - [AAX_FastPow.h, 1219](#)
- [_AAX_UTILSNATIVE_H_](#)
 - [AAX_UtillsNative.h, 1310](#)
- [_AAX_VERSION_H_](#)
 - [AAX_Version.h, 1314](#)
- [_acfUID, 403](#)
 - [Data1, 403](#)
 - [Data2, 403](#)
 - [Data3, 403](#)
 - [Data4, 403](#)
- [~AAX_AggregateResult](#)
 - [AAX_AggregateResult, 405](#)
- [~AAX_CArrayDataBuffer](#)
 - [AAX_CArrayDataBuffer< D >, 408](#)
- [~AAX_CArrayDataBufferOfType](#)
 - [AAX_CArrayDataBufferOfType< T, D >, 411](#)
- [~AAX_CAtomicQueue](#)
 - [AAX_CAtomicQueue< T, S >, 414](#)
- [~AAX_CAutoreleasePool](#)
 - [AAX_CAutoreleasePool, 416](#)
- [~AAX_CChunkDataParser](#)
 - [AAX_CChunkDataParser, 426](#)
- [~AAX_CEffectDirectData](#)
 - [AAX_CEffectDirectData, 435](#)
- [~AAX_CEffectGUI](#)
 - [AAX_CEffectGUI, 440](#)
- [~AAX_CEffectParameters](#)
 - [AAX_CEffectParameters, 453](#)
- [~AAX_CHostProcessor](#)
 - [AAX_CHostProcessor, 488](#)
- [~AAX_CMonolithicParameters](#)
 - [AAX_CMonolithicParameters, 511](#)
- [~AAX_CMutex](#)
 - [AAX_CMutex, 517](#)
- [~AAX_CPacket](#)
 - [AAX_CPacket, 523](#)
- [~AAX_CPacketDispatcher](#)
 - [AAX_CPacketDispatcher, 525](#)
- [~AAX_CParameter](#)
 - [AAX_CParameter< T >, 537](#)
- [~AAX_CParameterManager](#)
 - [AAX_CParameterManager, 567](#)
- [~AAX_CPieceWiseLinearTaperDelegate](#)
 - [AAX_CPieceWiseLinearTaperDelegate< T, Real-Precision >, 586](#)
- [~AAX_CSessionDocumentClient](#)
 - [AAX_CSessionDocumentClient, 594](#)
- [~AAX_CStringDataBuffer](#)
 - [AAX_CStringDataBuffer, 644](#)
- [~AAX_CStringDataBufferOfType](#)
 - [AAX_CStringDataBufferOfType< T >, 647](#)
- [~AAX_CTaskAgent](#)
 - [AAX_CTaskAgent, 657](#)
- [~AAX_CheckedResult](#)
 - [AAX_CheckedResult, 484](#)
- [~AAX_IAutomationDelegate](#)
 - [AAX_IAutomationDelegate, 799](#)
- [~AAX_ICollection](#)
 - [AAX_ICollection, 804](#)
- [~AAX_IComponentDescriptor](#)
 - [AAX_IComponentDescriptor, 810](#)
- [~AAX_IContainer](#)
 - [AAX_IContainer, 823](#)
- [~AAX_IController](#)
 - [AAX_IController, 826](#)
- [~AAX_IDataBufferWrapper](#)
 - [AAX_IDataBufferWrapper, 842](#)
- [~AAX_IDescriptionHost](#)
 - [AAX_IDescriptionHost, 843](#)
- [~AAX_IDisplayDelegateBase](#)
 - [AAX_IDisplayDelegateBase, 849](#)
- [~AAX_IDisplayDelegateDecorator](#)
 - [AAX_IDisplayDelegateDecorator< T >, 851](#)
- [~AAX_IDma](#)
 - [AAX_IDma, 856](#)
- [~AAX_IEffectDescriptor](#)
 - [AAX_IEffectDescriptor, 865](#)
- [~AAX_IFeatureInfo](#)
 - [AAX_IFeatureInfo, 874](#)
- [~AAX_IHostProcessorDelegate](#)
 - [AAX_IHostProcessorDelegate, 878](#)
- [~AAX_IHostServices](#)
 - [AAX_IHostServices, 880](#)
- [~AAX_IHostTaskAgent](#)
 - [AAX_IHostTaskAgent, 883](#)
- [~AAX_IMIDIMessageInfoDelegate](#)
 - [AAX_IMIDIMessageInfoDelegate, 884](#)
- [~AAX_IMIDINode](#)
 - [AAX_IMIDINode, 887](#)
- [~AAX_IPacketHandler](#)
 - [AAX_IPacketHandler, 889](#)
- [~AAX_IPageTable](#)
 - [AAX_IPageTable, 891](#)
- [~AAX_IParameter](#)
 - [AAX_IParameter, 902](#)

- ~AAX_IParameterValue
 - AAX_IParameterValue, 922
- ~AAX_IPointerQueue
 - AAX_IPointerQueue< T >, 926
- ~AAX_IPrivateDataAccess
 - AAX_IPrivateDataAccess, 928
- ~AAX_IPropertyMap
 - AAX_IPropertyMap, 930
- ~AAX_ISessionDocument
 - AAX_ISessionDocument, 935
- ~AAX_IString
 - AAX_IString, 938
- ~AAX_ITaperDelegateBase
 - AAX_ITaperDelegateBase, 943
- ~AAX_ITask
 - AAX_ITask, 945
- ~AAX_ITransport
 - AAX_ITransport, 949
- ~AAX_IViewContainer
 - AAX_IViewContainer, 959
- ~AAX_Map
 - AAX_Map, 964
- ~AAX_StLock_Guard
 - AAX_StLock_Guard, 988
- ~AAX_VAutomationDelegate
 - AAX_VAutomationDelegate, 991
- ~AAX_VCollection
 - AAX_VCollection, 996
- ~AAX_VComponentDescriptor
 - AAX_VComponentDescriptor, 1002
- ~AAX_VController
 - AAX_VController, 1016
- ~AAX_VDataBufferWrapper
 - AAX_VDataBufferWrapper, 1030
- ~AAX_VDescriptionHost
 - AAX_VDescriptionHost, 1031
- ~AAX_VEffectDescriptor
 - AAX_VEffectDescriptor, 1034
- ~AAX_VFeatureInfo
 - AAX_VFeatureInfo, 1039
- ~AAX_VHostServices
 - AAX_VHostServices, 1044
- ~AAX_VHostTaskAgent
 - AAX_VHostTaskAgent, 1046
- ~AAX_VPageTable
 - AAX_VPageTable, 1048
- ~AAX_VPrivateDataAccess
 - AAX_VPrivateDataAccess, 1058
- ~AAX_VPropertyMap
 - AAX_VPropertyMap, 1061
- ~AAX_VSessionDocument
 - AAX_VSessionDocument, 1066
- ~AAX_VTask
 - AAX_VTask, 1069
- ~AAX_VTransport
 - AAX_VTransport, 1072
- ~AAX_VViewContainer
 - AAX_VViewContainer, 1082
- ~Any
 - AAX::Exception::Any, 1089
- ~SAutoArray
 - SAutoArray< T >, 1098
- ~TempoMap
 - AAX_ISessionDocument::TempoMap, 1099
- ~VTempoMap
 - AAX_VSessionDocument::VTempoMap, 1100
- AAE_EAudioBufferLengthNative
 - AAX_Enums.h, 1174
- AAX, 371
 - AbsMax, 388
 - alignFree, 385
 - alignMalloc, 385
 - AsString, 377
 - AsStringFourChar, 382
 - AsStringIDTriad, 383
 - AsStringInt32, 382
 - AsStringPropertyValue, 382
 - AsStringResult, 383
 - AsStringStemChannel, 383
 - AsStringStemFormat, 383
 - AsStringSupportLevel, 384
 - AsStringUInt32, 382
 - Binary2String, 381
 - Caseless_strcmp, 381
 - cBigEndian, 393
 - cDenormalAvoidanceOffset, 396
 - CeilLog2, 390
 - cFloatDenormalAvoidanceOffset, 396
 - cGiga, 396
 - cHalfPi, 394
 - cInitialSeedValue, 397
 - cKilo, 396
 - ClampToZero, 387
 - ClearMappedParameterByID, 380
 - cLittleEndian, 393
 - cMega, 396
 - cMicro, 395
 - cMilli, 395
 - cNano, 396
 - cNeg3dB, 395
 - cNeg6dB, 395
 - cNormalizeLongToAmplitudeOne, 395
 - cNormalizeLongToAmplitudeOneHalf, 395
 - cOneOverRootTwo, 394
 - CopyPageTable, 380
 - cPi, 394
 - cPico, 396
 - cPos3dB, 394
 - cPos6dB, 395
 - cQuarterPi, 394
 - cRootTwo, 394
 - cSeedDivisor, 397
 - cTwoPi, 394
 - DeDenormal, 386
 - DeDenormalFine, 386
 - e176400SampleRate, 377

- e192000SampleRate, [377](#)
- e44100SampleRate, [377](#)
- e48000SampleRate, [377](#)
- e88200SampleRate, [377](#)
- e96000SampleRate, [377](#)
- EChannelModeData, [376](#)
- eChannelModeData_AllNotesOff, [376](#)
- eChannelModeData_AllSoundOff, [376](#)
- eChannelModeData_LocalControl, [376](#)
- eChannelModeData_OmniOff, [376](#)
- eChannelModeData_OmniOn, [376](#)
- eChannelModeData_PolyOff, [376](#)
- eChannelModeData_PolyOn, [376](#)
- eChannelModeData_ResetControllers, [376](#)
- ESampleRates, [377](#)
- ESpecialData, [376](#)
- eSpecialData_AccentedClick, [376](#)
- eSpecialData_UnaccentedClick, [376](#)
- EStatusByte, [375](#)
- eStatusByte_ActiveSensing, [376](#)
- eStatusByte_Continue, [376](#)
- eStatusByte_MTCQuarterFrame, [376](#)
- eStatusByte_Reset, [376](#)
- eStatusByte_SongPosition, [376](#)
- eStatusByte_SongSelect, [376](#)
- eStatusByte_Start, [376](#)
- eStatusByte_Stop, [376](#)
- eStatusByte_SysExBegin, [375](#)
- eStatusByte_SysExEnd, [376](#)
- eStatusByte_TimingClock, [376](#)
- eStatusByte_TuneRequest, [376](#)
- EStatusNibble, [375](#)
- eStatusNibble_ChannelMode, [375](#)
- eStatusNibble_ChannelPressure, [375](#)
- eStatusNibble_ControlChange, [375](#)
- eStatusNibble_KeyPressure, [375](#)
- eStatusNibble_NoteOff, [375](#)
- eStatusNibble_NoteOn, [375](#)
- eStatusNibble_PitchBend, [375](#)
- eStatusNibble_ProgramChange, [375](#)
- eStatusNibble_SystemCommon, [375](#)
- eStatusNibble_SystemRealTime, [375](#)
- fabs, [388](#)
- fabsf, [388](#)
- FastRndDbf2Int32, [391](#)
- FastRound2Int32, [390](#)
- FastRound2Int64, [392](#)
- FastTrunc2Int32, [391](#)
- Fill, [387](#), [388](#)
- FilterDenormals, [386](#)
- FindParameterMappingsInPageTable, [380](#)
- GetCStringOfLength, [381](#)
- GetFastInt32RPDF, [392](#)
- GetFastRPDFWithAmplitudeOne, [393](#)
- GetInt32RPDF, [392](#)
- GetRPDFWithAmplitudeOne, [393](#)
- GetRPDFWithAmplitudeOneHalf, [393](#)
- GetTPDFWithAmplitudeOne, [393](#)
- IsAccentedClick, [378](#)
- IsAllNotesOff, [378](#)
- IsASCII, [381](#)
- IsAvidNotification, [385](#)
- IsClick, [379](#)
- IsEffectIDEqual, [385](#)
- IsFourCharASCII, [382](#)
- IsNoteOff, [378](#)
- IsNoteOn, [377](#)
- IsParameterIDEqual, [385](#)
- IsUnaccentedClick, [378](#)
- kPowExtent, [397](#)
- kPowTableSize, [397](#)
- Max, [389](#)
- Min, [389](#)
- MinMax, [389](#)
- PageTableParameterMappingsAreEqual, [379](#)
- PageTableParameterNameVariationsAreEqual, [379](#)
- PageTablesAreEqual, [379](#)
- PolyEval, [389](#)
- SafeLog, [384](#)
- SafeLogf, [384](#)
- Sign, [389](#)
- SinCosMix, [390](#)
- String2Binary, [381](#)
- ZeroMemoryDW, [387](#)
- ZeroMemorySW, [387](#)
- AAX communication protocols, [79](#)
- AAX Format Specification, [81](#)
- AAX Host Guides, [150](#)
- AAX Interfaces, [289](#)
- AAX Library features, [106](#)
- AAX SDK Manual, [47](#)
- AAX.h, [1103](#)
 - AAX_ALIGN_FILE_ALG, [1109](#)
 - AAX_ALIGN_FILE_BEGIN, [1110](#)
 - AAX_ALIGN_FILE_END, [1110](#)
 - AAX_ALIGN_FILE_HOST, [1109](#)
 - AAX_ALIGN_FILE_RESET, [1110](#)
 - AAX_CALLBACK, [1110](#)
 - AAX_CAudioInPort, [1116](#)
 - AAX_CAudioOutPort, [1116](#)
 - AAX_CBoolean, [1112](#)
 - AAX_CComponentID, [1114](#)
 - AAX_CCount, [1112](#)
 - AAX_CEffectID, [1115](#)
 - AAX_CFieldIndex, [1114](#)
 - AAX_CIndex, [1111](#)
 - AAX_CMeterID, [1114](#)
 - AAX_CMeterPort, [1116](#)
 - AAX_CONSTEXPR, [1108](#)
 - AAX_CPageTableParamID, [1115](#)
 - AAX_CParamID, [1114](#)
 - AAX_CPointerPropertyValue, [1113](#)
 - AAX_CPP11_SUPPORT, [1106](#)
 - AAX_CPropertyValue, [1113](#)
 - AAX_CPropertyValue64, [1113](#)

- AAX_CSAMPLERate, 1113
- AAX_CSelector, 1112
- AAX_CTargetPlatform, 1114
- AAX_CTimeOfDay, 1112
- AAX_CTimestamp, 1112
- AAX_CTransportCounter, 1112
- AAX_CTypeID, 1113
- AAX_DEFAULT_ASGN_OPER, 1107
- AAX_DEFAULT_COPY_CTOR, 1107
- AAX_DEFAULT_CTOR, 1107
- AAX_DEFAULT_DTOR, 1107
- AAX_DEFAULT_DTOR_OVERRIDE, 1107
- AAX_DEFAULT_MOVE_CTOR, 1108
- AAX_DEFAULT_MOVE_OPER, 1108
- AAX_DELETE, 1108
- AAX_Feature_UID, 1115
- AAX_FIELD_INDEX, 1111
- AAX_FINAL, 1107
- AAX_OVERRIDE, 1106
- AAX_PointerSize, 1109
- AAX_PREPROCESSOR_CONCAT, 1111
- AAX_PREPROCESSOR_CONCAT_HELPER, 1111
- AAX_Result, 1113
- AAX_SPlugInChunk, 1117
- AAX_SPlugInChunkHeader, 1116
- AAX_SPlugInChunkPtr, 1117
- AAX_SPlugInIdentifierTriad, 1117
- AAX_SPlugInIdentifierTriadPtr, 1117
- AAX_UNIQUE_PTR, 1108
- AAXPointer_32bit, 1109
- AAXPointer_64bit, 1109
- acfUID, 1115
- kAAX_ParameterIdentifierMaxSize, 1117
- TI_VERSION, 1106
- AAX::Exception, 397
- AAX::Exception::Any, 1088
 - ~Any, 1089
 - AAX_DEFAULT_MOVE_CTOR, 1090
 - AAX_DEFAULT_MOVE_OPER, 1090
 - Any, 1089, 1090
 - Desc, 1090
 - Function, 1091
 - Line, 1091
 - operator=, 1090
 - What, 1090
- AAX::Exception::ResultError, 1096
 - FormatResult, 1098
 - Result, 1098
 - ResultError, 1097
- AAX::internal, 398
 - ToHexadecimal, 398
- AAX_ACFInterface.doxygen, 1117
 - acfIID, 1118
 - acfUID, 1118
- AAX_AdditionalFeatures_Algorithm.doxygen, 1118
- AAX_AdditionalFeatures_AOSandSidechain.doxygen, 1118
- AAX_AdditionalFeatures_CurveDisplays.doxygen, 1118
- AAX_AdditionalFeatures_Hybrid.doxygen, 1118
- AAX_AdditionalFeatures_Meters.doxygen, 1118
- AAX_AdditionalFeatures_MIDI.doxygen, 1118
- AAX_AdditionalFeatures_PropertiesFile.doxygen, 1118
- AAX_AggregateResult, 404
 - ~AAX_AggregateResult, 405
 - AAX_AggregateResult, 405
 - Check, 405
 - Clear, 405
 - LastFailure, 406
 - NumAttempted, 406
 - NumFailed, 406
 - NumSucceeded, 406
 - operator AAX_Result, 405
 - operator=, 405
- AAX_ALIGN_FILE_ALG
 - AAX.h, 1109
- AAX_ALIGN_FILE_BEGIN
 - AAX.h, 1110
- AAX_ALIGN_FILE_END
 - AAX.h, 1110
- AAX_ALIGN_FILE_HOST
 - AAX.h, 1109
- AAX_ALIGN_FILE_RESET
 - AAX.h, 1110
- AAX_Alignment.h, 1118
- AAX_ALIGNMENT_HINT
 - AAX_MiscUtils.h, 1254
- AAX_ASSERT
 - AAX_Assert.h, 1123
- AAX_Assert.h, 1119
 - AAX_ASSERT, 1123
 - AAX_DEBUGASSERT, 1123
 - AAX_ETracePriority, 1125
 - AAX_STACKTRACE, 1124
 - AAX_STACKTRACE_RELEASE, 1122
 - AAX_TRACE, 1124
 - AAX_TRACE_RELEASE, 1121
 - AAX_TRACEORSTACKTRACE, 1124
 - AAX_TRACEORSTACKTRACE_RELEASE, 1122
 - kAAX_Trace_Priority_Critical, 1121
 - kAAX_Trace_Priority_High, 1121
 - kAAX_Trace_Priority_Low, 1121
 - kAAX_Trace_Priority_Lowest, 1121
 - kAAX_Trace_Priority_None, 1120
 - kAAX_Trace_Priority_Normal, 1121
- AAX_Atomic.h, 1125
 - AAX_Atomic_DecThenGet_32, 1126
 - AAX_Atomic_Exchange_32, 1126
 - AAX_ATOMIC_H_, 1126
 - AAX_Atomic_IncThenGet_32, 1126
- AAX_Atomic_DecThenGet_32
 - AAX_Atomic.h, 1126
- AAX_Atomic_Exchange_32
 - AAX_Atomic.h, 1126
- AAX_ATOMIC_H_
 - AAX_Atomic.h, 1126

- AAX_Atomic_IncThenGet_32
 - AAX_Atomic.h, 1126
- AAX_AuxInterface_DirectData.doxygen, 1127
- AAX_AuxInterface_HostProcessor.doxygen, 1127
- AAX_AuxInterface_TaskAgent.doxygen, 1127
- AAX_BigEndianNativeSwap
 - AAX_EndianSwap.h, 1159
- AAX_BigEndianNativeSwapInPlace
 - AAX_EndianSwap.h, 1159
- AAX_BigEndianNativeSwapSequenceInPlace
 - AAX_EndianSwap.h, 1160
- AAX_BugList.doxygen, 1127
- AAX_CALLBACK
 - AAX.h, 1110
- AAX_Callbacks.h, 1127
 - AAX_CBackgroundProc, 1129
 - AAX_CInitPrivateDataProc, 1129
 - AAX_CInstanceInitProc, 1128
 - AAX_CPacketAllocator, 1128
 - AAX_CProcessProc, 1128
 - AAX_CProcPtrID, 1130
 - AAXCreateObjectProc, 1128
 - kAAX_ProcPtrID_Create_EffectDirectData, 1131
 - kAAX_ProcPtrID_Create_EffectGUI, 1131
 - kAAX_ProcPtrID_Create_EffectParameters, 1131
 - kAAX_ProcPtrID_Create_HostProcessor, 1131
 - kAAX_ProcPtrID_Create_SessionDocumentClient, 1131
 - kAAX_ProcPtrID_Create_TaskAgent, 1131
- AAX_CAPTURE
 - AAX_Exception.h, 1214
- AAX_CAPTURE_MULT
 - AAX_Exception.h, 1214
- AAX_CArrayDataBuffer
 - AAX_CArrayDataBuffer< D >, 407, 408
- AAX_CArrayDataBuffer< D >, 406
 - ~AAX_CArrayDataBuffer, 408
 - AAX_CArrayDataBuffer, 407, 408
 - Data, 409
 - operator=, 408
 - Size, 409
 - Type, 408
- AAX_CArrayDataBuffer.h, 1131
 - AAX_CArrayDataBuffer_H, 1131
- AAX_CArrayDataBuffer_H
 - AAX_CArrayDataBuffer.h, 1131
- AAX_CArrayDataBufferOfType
 - AAX_CArrayDataBufferOfType< T, D >, 410, 411
- AAX_CArrayDataBufferOfType< T, D >, 409
 - ~AAX_CArrayDataBufferOfType, 411
 - AAX_CArrayDataBufferOfType, 410, 411
 - Data, 412
 - operator=, 411
 - Size, 412
 - Type, 411
- AAX_CAtomicQueue
 - AAX_CAtomicQueue< T, S >, 414
- AAX_CAtomicQueue< T, S >, 412
 - ~AAX_CAtomicQueue, 414
 - AAX_CAtomicQueue, 414
 - Clear, 414
 - Peek, 415
 - Pop, 415
 - Push, 414
 - template_size, 416
 - template_type, 413
 - value_type, 414
- AAX_CAtomicQueue.h, 1131
- AAX_CAudioInPort
 - AAX.h, 1116
- AAX_CAudioOutPort
 - AAX.h, 1116
- AAX_CAutoreleasePool, 416
 - ~AAX_CAutoreleasePool, 416
 - AAX_CAutoreleasePool, 416
- AAX_CAutoreleasePool.h, 1132
 - _AAX_CAUTORELEASEPOOL_H_, 1132
- AAX_CBackgroundProc
 - AAX_Callbacks.h, 1129
- AAX_CBinaryDisplayDelegate
 - AAX_CBinaryDisplayDelegate< T >, 417, 418
- AAX_CBinaryDisplayDelegate< T >, 417
 - AAX_CBinaryDisplayDelegate, 417, 418
 - AddShortenedStrings, 420
 - Clone, 418
 - StringToValue, 419
 - ValueToString, 418, 419
- AAX_CBinaryDisplayDelegate.h, 1132
- AAX_CBinaryTaperDelegate
 - AAX_CBinaryTaperDelegate< T >, 421
- AAX_CBinaryTaperDelegate< T >, 420
 - AAX_CBinaryTaperDelegate, 421
 - Clone, 421
 - ConstrainRealValue, 422
 - GetMaximumValue, 421
 - GetMinimumValue, 422
 - NormalizedToReal, 422
 - RealToNormalized, 423
- AAX_CBinaryTaperDelegate.h, 1133
- AAX_CBoolean
 - AAX.h, 1112
- AAX_CChunkDataParser, 423
 - ~AAX_CChunkDataParser, 426
 - AAX_CChunkDataParser, 425
 - AddDouble, 426
 - AddFloat, 426
 - AddInt16, 426
 - AddInt32, 426
 - AddString, 427
 - Clear, 429
 - FindDouble, 427
 - FindFloat, 427
 - FindInt16, 427
 - FindInt32, 427
 - FindName, 429
 - FindString, 428

- GetChunkData, [428](#)
- GetChunkDataSize, [428](#)
- GetChunkVersion, [428](#)
- IsEmpty, [428](#)
- LoadChunk, [429](#)
- mChunkData, [430](#)
- mChunkVersion, [430](#)
- mDataValues, [430](#)
- mLastFoundIndex, [430](#)
- ReplaceDouble, [428](#)
- WordAlign, [429](#)
- AAX_CChunkDataParser.h, [1133](#)
- AAX_CHUNKDATAPARSER_H, [1134](#)
- AAX_CChunkDataParser::DataValue, [1091](#)
- DataValue, [1091](#)
- mDataName, [1092](#)
- mDataType, [1092](#)
- mIntValue, [1092](#)
- mStringValue, [1092](#)
- AAX_CComponentID
- AAX.h, [1114](#)
- AAX_CCount
- AAX.h, [1112](#)
- AAX_CDecibelDisplayDelegateDecorator
- AAX_CDecibelDisplayDelegateDecorator< T >, [431](#)
- AAX_CDecibelDisplayDelegateDecorator< T >, [430](#)
- AAX_CDecibelDisplayDelegateDecorator, [431](#)
- Clone, [431](#)
- StringToValue, [433](#)
- ValueToString, [432](#)
- AAX_CDecibelDisplayDelegateDecorator.h, [1134](#)
- AAX_CEffectDirectData, [434](#)
- ~AAX_CEffectDirectData, [435](#)
- AAX_CEffectDirectData, [435](#)
- Controller, [437](#)
- EffectParameters, [437](#)
- Initialize, [435](#)
- Initialize_PrivateDataAccess, [437](#)
- NotificationReceived, [436](#)
- TimerWakeup, [436](#)
- TimerWakeup_PrivateDataAccess, [437](#)
- Uninitialize, [435](#)
- AAX_CEffectDirectData.h, [1135](#)
- AAX_CEFFECTDIRECTDATA_H, [1135](#)
- AAX_CEffectDirectData_H
- AAX_CEffectDirectData.h, [1135](#)
- AAX_CEffectGUI, [438](#)
- ~AAX_CEffectGUI, [440](#)
- AAX_CEffectGUI, [440](#)
- CreateViewContainer, [444](#)
- CreateViewContents, [444](#)
- DeleteViewContainer, [444](#)
- Draw, [442](#)
- GetController, [445](#)
- GetCustomLabel, [443](#)
- GetEffectParameters, [445](#)
- GetViewContainer, [446](#)
- GetViewContainerPtr, [446](#)
- GetViewContainerType, [446](#)
- GetViewSize, [442](#)
- Initialize, [440](#)
- NotificationReceived, [440](#)
- ParameterUpdated, [443](#)
- SetControlHighlightInfo, [444](#)
- SetViewContainer, [441](#)
- TimerWakeup, [442](#)
- Transport, [446](#)
- Uninitialize, [440](#)
- UpdateAllParameters, [445](#)
- AAX_CEffectGUI.h, [1135](#)
- AAX_CEffectID
- AAX.h, [1115](#)
- AAX_CEffectParameters, [447](#)
- ~AAX_CEffectParameters, [453](#)
- AAX_CEffectParameters, [453](#)
- AutomationDelegate, [478](#)
- BuildChunkData, [480](#)
- CompareActiveChunk, [470](#)
- Controller, [477](#)
- DoMIDITransfers, [476](#)
- EffectInit, [479](#)
- FilterParameterIDOnSave, [480](#)
- GenerateCoefficients, [467](#)
- GetChunk, [469](#)
- GetChunkIDFromIndex, [469](#)
- GetChunkSize, [469](#)
- GetCurveData, [472](#)
- GetCurveDataDisplayRange, [473](#)
- GetCurveDataMeterIds, [473](#)
- GetCustomData, [475](#)
- GetMasterBypassParameter, [455](#)
- GetNumberOfChanges, [471](#)
- GetNumberOfChunks, [468](#)
- GetNumberOfParameters, [455](#)
- GetParameter, [459](#)
- GetParameterDefaultNormalizedValue, [457](#)
- GetParameterIDFromIndex, [461](#)
- GetParameterIndex, [459](#)
- GetParameterIsAutomatable, [455](#)
- GetParameterName, [456](#)
- GetParameterNameOfLength, [456](#)
- GetParameterNormalizedValue, [463](#)
- GetParameterNumberOfSteps, [456](#)
- GetParameterOrientation, [458](#)
- GetParameterStringFromValue, [462](#)
- GetParameterType, [458](#)
- GetParameterValueFromString, [462](#)
- GetParameterValueInfo, [461](#)
- GetParameterValueString, [463](#)
- Initialize, [453](#)
- IsParameterLinkReady, [479](#)
- IsParameterTouched, [479](#)
- mChunkParser, [481](#)
- mChunkSize, [481](#)
- mFilteredParameters, [481](#)

- mNumChunkedParameters, 481
- mNumPluginChanges, 480
- mPacketDispatcher, 481
- mParameterManager, 481
- NotificationReceived, 454
- operator=, 453
- ReleaseParameter, 465
- RenderAudio_Hybrid, 477
- ResetFieldData, 467
- SetChunk, 470
- SetCustomData, 475
- SetDisplayDelegate, 478
- SetParameterDefaultNormalizedValue, 457
- SetParameterNormalizedRelative, 464
- SetParameterNormalizedValue, 463
- SetTaperDelegate, 478
- TimerWakeup, 471
- TouchParameter, 464
- Transport, 478
- Uninitialize, 454
- UpdateControlMIDINodes, 476
- UpdateMIDINodes, 476
- UpdatePageTable, 474, 479
- UpdateParameterNormalizedRelative, 466
- UpdateParameterNormalizedValue, 466
- UpdateParameterTouch, 465
- AAX_CEffectParameters.h, 1136
 - BoolToNormalized, 1137
 - cDefaultMasterBypassID, 1137
 - cPreviewID, 1137
 - Int32ToNormalized, 1136
 - NormalizedToInt32, 1136
- AAX_CFieldIndex
 - AAX.h, 1114
- AAX_CheckedResult, 482
 - ~AAX_CheckedResult, 484
 - AAX_CheckedResult, 484
 - AddAcceptedResult, 484
 - Clear, 485
 - Exception, 484
 - LastError, 485
 - operator AAX_Result, 485
 - operator=, 485
 - operator| =, 485
 - ResetAcceptedResults, 484
- AAX_CHostProcessor, 486
 - ~AAX_CHostProcessor, 488
 - AAX_CHostProcessor, 488
 - AnalyzeAudio, 491
 - Controller, 496
 - EffectParameters, 497
 - GetAudio, 496
 - GetClipNameSuffix, 493
 - GetDstEnd, 495
 - GetDstStart, 495
 - GetEffectParameters, 493
 - GetHostProcessorDelegate, 493, 494
 - GetInputRange, 494
 - GetLocation, 494
 - GetOutputRange, 494
 - GetSideChainInputNum, 496
 - GetSrcEnd, 494
 - GetSrcStart, 494
 - HostProcessorDelegate, 496, 497
 - Initialize, 488
 - InitOutputBounds, 489
 - PostAnalyze, 492
 - PostRender, 491
 - PreAnalyze, 492
 - PreRender, 491
 - RenderAudio, 490
 - SetLocation, 490
 - TranslateOutputBounds, 495
 - Uninitialize, 488
- AAX_CHostProcessor.h, 1137
- AAX_CHostServices, 497
 - HandleAssertFailure, 498
 - Set, 498
 - StackTrace, 499
 - Trace, 498
- AAX_CHostServices.h, 1138
- AAX_CHUNKDATAPARSER_H
 - AAX_CChunkDataParser.h, 1134
- AAX_ChunkDataParserDefs, 398
 - BUILD_DATA_FAILED, 401
 - DEFAULT32BIT_TYPE_INCR, 401
 - DEFAULT32BIT_TYPE_SIZE, 400
 - DOUBLE_STRING_IDENTIFIER, 399
 - DOUBLE_TYPE, 399
 - DOUBLE_TYPE_INCR, 399
 - DOUBLE_TYPE_SIZE, 399
 - FLOAT_STRING_IDENTIFIER, 399
 - FLOAT_TYPE, 399
 - HEADER_SIZE, 401
 - LONG_STRING_IDENTIFIER, 399
 - LONG_TYPE, 399
 - MAX_NAME_LENGTH, 401
 - MAX_STRINGDATA_LENGTH, 400
 - NAME_NOT_FOUND, 401
 - SHORT_STRING_IDENTIFIER, 400
 - SHORT_TYPE, 400
 - SHORT_TYPE_INCR, 400
 - SHORT_TYPE_SIZE, 400
 - STRING_IDENTIFIER_SIZE, 401
 - STRING_STRING_IDENTIFIER, 400
 - STRING_TYPE, 400
 - VERSION_ID_1, 401
- AAX_CIndex
 - AAX.h, 1111
- AAX_CInitPrivateDataProc
 - AAX_Callbacks.h, 1129
- AAX_CInstanceInitProc
 - AAX_Callbacks.h, 1128
- AAX_CLinearTaperDelegate
 - AAX_CLinearTaperDelegate< T, RealPrecision >, 500

- AAX_CLinearTaperDelegate< T, RealPrecision >, 499
 - AAX_CLinearTaperDelegate, 500
 - Clone, 501
 - ConstrainRealValue, 501
 - GetMaximumValue, 501
 - GetMinimumValue, 501
 - NormalizedToReal, 502
 - RealToNormalized, 502
 - Round, 502
- AAX_CLinearTaperDelegate.h, 1138
- AAX_CLogTaperDelegate
 - AAX_CLogTaperDelegate< T, RealPrecision >, 504
- AAX_CLogTaperDelegate< T, RealPrecision >, 503
 - AAX_CLogTaperDelegate, 504
 - Clone, 504
 - ConstrainRealValue, 505
 - GetMaximumValue, 505
 - GetMinimumValue, 505
 - NormalizedToReal, 506
 - RealToNormalized, 506
 - Round, 507
- AAX_CLogTaperDelegate.h, 1138
- AAX_CMeterID
 - AAX.h, 1114
- AAX_CMeterPort
 - AAX.h, 1116
- AAX_CMidiPacket, 507
 - mData, 508
 - mIsImmediate, 508
 - mLength, 507
 - mTimestamp, 507
- AAX_CMidiStream, 508
 - mBuffer, 509
 - mBufferSize, 509
- AAX_CMonolithicParameters, 509
 - ~AAX_CMonolithicParameters, 511
 - AAX_CMonolithicParameters, 511
 - AddSynchronizedParameter, 512
 - GenerateCoefficients, 513
 - RenderAudio, 512
 - ResetFieldData, 514
 - StaticDescribe, 515
 - StaticRenderAudio, 516
 - TimerWakeup, 514
 - TParamValPair, 511
 - UpdateParameterNormalizedValue, 513
- AAX_CMonolithicParameters.cpp, 1139
- AAX_CMonolithicParameters.h, 1139
 - kMaxAdditionalMIDINodes, 1140
 - kMaxAuxOutputStems, 1140
 - kSynchronizedParameterQueueSize, 1140
- AAX_CMutex, 516
 - ~AAX_CMutex, 517
 - AAX_CMutex, 517
 - Lock, 517
 - Try_Lock, 517
 - Unlock, 517
- AAX_CMutex.h, 1140
- AAX_CNumberDisplayDelegate< T, Precision, SpaceAfter >, 518
 - Clone, 518
 - StringToValue, 520
 - ValueToString, 518, 520
- AAX_CNumberDisplayDelegate.h, 1140
- AAX_CommonConversions.h, 1141
 - DBToGain, 1142
 - DoubleTo32BitDSPCoef, 1143
 - DoubleTo32BitDSPCoefRnd, 1143
 - DoubleToDSPCoef, 1142
 - DoubleToDSPCoefRnd, 1144
 - DoubleToLong, 1142
 - DSPCoefToDouble, 1143
 - GainToDB, 1142
 - k32BitAbsMax, 1144
 - k32BitNegMax, 1144
 - k32BitPosMax, 1144
 - k56kFloatNegMax, 1145
 - k56kFloatPosMax, 1145
 - k56kFracAbsMax, 1144
 - k56kFracHalf, 1145
 - k56kFracNegMax, 1145
 - k56kFracNegOne, 1145
 - k56kFracPosMax, 1144
 - k56kFracZero, 1145
 - kNeg144DB, 1146
 - kNeg144Gain, 1146
 - kOneOver56kFracAbsMax, 1145
 - LongToDouble, 1142
 - ThirtyTwoBitDSPCoefToDouble, 1143
- AAX_CommonInterface_Algorithm.doxygen, 1146
- AAX_CommonInterface_Communication.doxygen, 1146
- AAX_CommonInterface_DataModel.doxygen, 1146
- AAX_CommonInterface_Describe.doxygen, 1146
- AAX_CommonInterface_FormatSpecification.doxygen, 1146
- AAX_CommonInterface_GUI.doxygen, 1146
- AAX_CompID_DescriptionHost
 - AAX_UIDs.h, 1303
- AAX_CompID_FeatureInfo
 - AAX_UIDs.h, 1303
- AAX_Component< aContextType >, 521
 - CBackgroundProc, 522
 - CInitPrivateDataProc, 522
 - CInstanceInitProc, 522
 - CPacketAllocator, 522
 - CProcessProc, 522
- AAX_Constants.h, 1146
 - AAX_CONSTANTS_H, 1147
- AAX_CONSTANTS_H
 - AAX_Constants.h, 1147
- AAX_CONSTEXPR
 - AAX.h, 1108
- AAX_CPacket, 523
 - ~AAX_CPacket, 523
 - AAX_CPacket, 523

- GetID, [524](#)
- GetPtr, [523](#), [524](#)
- GetSize, [524](#)
- IsDirty, [524](#)
- SetDirty, [524](#)
- AAX_CPacketAllocator
 - AAX_Callbacks.h, [1128](#)
- AAX_CPacketDispatcher, [524](#)
 - ~AAX_CPacketDispatcher, [525](#)
 - AAX_CPacketDispatcher, [525](#)
 - Dispatch, [526](#)
 - GenerateSingleValuePacket, [527](#)
 - Initialize, [525](#)
 - RegisterPacket, [526](#)
 - SetDirty, [526](#)
- AAX_CPacketDispatcher.h, [1148](#)
- AAX_CPacketHandler
 - AAX_CPacketHandler< TWorker >, [528](#)
- AAX_CPacketHandler< TWorker >, [527](#)
 - AAX_CPacketHandler, [528](#)
 - Call, [528](#)
 - Clone, [528](#)
 - fpt, [529](#)
 - fptEx, [529](#)
 - pt2Object, [529](#)
- AAX_CPageTableParamID
 - AAX.h, [1115](#)
- AAX_CParameter
 - AAX_CParameter< T >, [535](#), [536](#)
- AAX_CParameter< T >, [529](#)
 - ~AAX_CParameter, [537](#)
 - AAX_CParameter, [535](#), [536](#)
 - AAX_DEFAULT_MOVE_CTOR, [537](#)
 - AAX_DEFAULT_MOVE_OPER, [537](#)
 - AAX_DELETE, [537](#), [538](#)
 - AddShortenedName, [539](#)
 - Automatable, [552](#)
 - ClearShortenedNames, [540](#)
 - CloneValue, [538](#)
 - Defaults, [534](#)
 - DisplayDelegate, [558](#)
 - eParameterDefaultNumStepsContinuous, [535](#)
 - eParameterDefaultNumStepsDiscrete, [535](#)
 - eParameterTypeBool, [534](#)
 - eParameterTypeCustom, [534](#)
 - eParameterTypeFloat, [534](#)
 - eParameterTypeInt32, [534](#)
 - eParameterTypeUndefined, [534](#)
 - GetBoolFromNormalizedValue, [548](#), [563](#)
 - GetDefaultValue, [558](#)
 - GetDoubleFromNormalizedValue, [549](#), [564](#)
 - GetFloatFromNormalizedValue, [549](#), [563](#)
 - GetInt32FromNormalizedValue, [548](#), [563](#)
 - GetNormalizedDefaultValue, [540](#)
 - GetNormalizedValue, [541](#)
 - GetNormalizedValueFromBool, [546](#), [561](#)
 - GetNormalizedValueFromDouble, [547](#), [562](#)
 - GetNormalizedValueFromFloat, [547](#), [562](#)
 - GetNormalizedValueFromInt32, [546](#), [561](#)
 - GetNormalizedValueFromStep, [542](#)
 - GetNormalizedValueFromString, [548](#)
 - GetNumberOfSteps, [541](#)
 - GetOrientation, [544](#)
 - GetStepValue, [542](#)
 - GetStepValueFromNormalizedValue, [542](#)
 - GetStringFromNormalizedValue, [550](#)
 - GetType, [543](#)
 - GetValue, [557](#)
 - GetValueAsBool, [552](#)
 - GetValueAsDouble, [554](#)
 - GetValueAsFloat, [553](#)
 - GetValueAsInt32, [553](#)
 - GetValueAsString, [554](#), [558](#)
 - GetValueString, [545](#)
 - Identifier, [538](#)
 - mAutomatable, [565](#)
 - mAutomationDelegate, [565](#)
 - mControlType, [565](#)
 - mDefaultValue, [566](#)
 - mDisplayDelegate, [565](#)
 - mNames, [564](#)
 - mNeedNotify, [566](#)
 - mNumSteps, [565](#)
 - mOrientation, [565](#)
 - mTaperDelegate, [565](#)
 - mValue, [566](#)
 - Name, [539](#)
 - Release, [552](#)
 - SetAutomationDelegate, [551](#)
 - SetDefaultValue, [558](#)
 - SetDisplayDelegate, [544](#)
 - SetName, [538](#)
 - SetNormalizedDefaultValue, [540](#)
 - SetNormalizedValue, [540](#)
 - SetNumberOfSteps, [541](#)
 - SetOrientation, [543](#)
 - SetStepValue, [542](#)
 - SetTaperDelegate, [544](#)
 - SetToDefaultValue, [540](#)
 - SetType, [543](#)
 - SetValue, [557](#)
 - SetValueFromString, [551](#)
 - SetValueWithBool, [555](#), [559](#)
 - SetValueWithDouble, [556](#), [560](#)
 - SetValueWithFloat, [555](#), [560](#)
 - SetValueWithInt32, [555](#), [559](#)
 - SetValueWithString, [556](#), [560](#)
 - ShortenedName, [539](#)
 - TaperDelegate, [558](#)
 - Touch, [552](#)
 - Type, [534](#)
 - UpdateNormalizedValue, [557](#)
- AAX_CParameter.h, [1148](#)
- AAX_CParameterManager, [566](#)
 - ~AAX_CParameterManager, [567](#)
- AAX_CParameterManager, [567](#)

- AddParameter, 571
- GetParameter, 570
- GetParameterByID, 569
- GetParameterByName, 569, 570
- GetParameterIndex, 571
- Initialize, 568
- mAutomationDelegate, 572
- mParameters, 572
- mParametersMap, 572
- NumParameters, 568
- RemoveAllParameters, 568
- RemoveParameter, 571
- RemoveParameterByID, 568
- AAX_CParameterManager.h, 1149
- AAX_CParameterValue
 - AAX_CParameterValue< T >, 574, 575
- AAX_CParameterValue< T >, 572
 - AAX_CParameterValue, 574, 575
 - AAX_DEFAULT_DTOR_OVERRIDE, 575
 - AAX_DEFAULT_MOVE_CTOR, 575
 - AAX_DEFAULT_MOVE_OPER, 575
 - AAX_DELETE, 575
 - Clone, 576
 - Defaults, 574
 - eParameterDefaultMaxIdentifierLength, 574
 - eParameterDefaultMaxIdentifierSize, 574
 - Get, 576
 - GetValueAsBool, 576, 579
 - GetValueAsDouble, 578, 580
 - GetValueAsFloat, 577, 579
 - GetValueAsInt32, 577, 579
 - GetValueAsString, 578, 580
 - Identifier, 576
 - Set, 576
- AAX_CParamID
 - AAX.h, 1114
- AAX_CPercentDisplayDelegateDecorator
 - AAX_CPercentDisplayDelegateDecorator< T >, 582
- AAX_CPercentDisplayDelegateDecorator< T >, 581
 - AAX_CPercentDisplayDelegateDecorator, 582
 - Clone, 582
 - StringToValue, 583
 - ValueToString, 582, 583
- AAX_CPercentDisplayDelegateDecorator.h, 1149
 - AAX_CPERCENTDISPLAYDELEGATEDECORATOR_H, 1150
- AAX_CPERCENTDISPLAYDELEGATEDECORATOR_H
 - AAX_CPercentDisplayDelegateDecorator.h, 1150
- AAX_CPieceWiseLinearTaperDelegate
 - AAX_CPieceWiseLinearTaperDelegate< T, RealPrecision >, 585, 586
- AAX_CPieceWiseLinearTaperDelegate< T, RealPrecision >, 584
 - ~AAX_CPieceWiseLinearTaperDelegate, 586
 - AAX_CPieceWiseLinearTaperDelegate, 585, 586
 - Clone, 586
 - ConstrainRealValue, 587
 - GetMaximumValue, 586
 - GetMinimumValue, 586
 - NormalizedToReal, 587
 - RealToNormalized, 588
 - Round, 588
- AAX_CPieceWiseLinearTaperDelegate.h, 1150
- AAX_CPointerPropertyValue
 - AAX.h, 1113
- AAX_CPP11_SUPPORT
 - AAX.h, 1106
- AAX_CProcessProc
 - AAX_Callbacks.h, 1128
- AAX_CProcPtrID
 - AAX_Callbacks.h, 1130
- AAX_CPropertyValue
 - AAX.h, 1113
- AAX_CPropertyValue64
 - AAX.h, 1113
- AAX_CRangeTaperDelegate
 - AAX_CRangeTaperDelegate< T, RealPrecision >, 590
- AAX_CRangeTaperDelegate< T, RealPrecision >, 588
 - AAX_CRangeTaperDelegate, 590
 - Clone, 591
 - ConstrainRealValue, 591
 - GetMaximumValue, 591
 - GetMinimumValue, 591
 - NormalizedToReal, 592
 - operator=, 590
 - RealToNormalized, 592
 - Round, 592
 - SmartRound, 593
- AAX_CRangeTaperDelegate.h, 1150
- AAX_CSampleRate
 - AAX.h, 1113
- AAX_CSelector
 - AAX.h, 1112
- AAX_CSessionDocumentClient, 593
 - ~AAX_CSessionDocumentClient, 594
 - AAX_CSessionDocumentClient, 594
 - GetController, 596, 597
 - GetEffectParameters, 597
 - GetSessionDocument, 597
 - Initialize, 594
 - NotificationReceived, 595
 - SessionDocumentChanged, 596
 - SessionDocumentWillChange, 596
 - SetSessionDocument, 595
 - Uninitialize, 595
- AAX_CSessionDocumentClient.h, 1151
 - AAX_CSessionDocumentClient_H, 1151
- AAX_CSessionDocumentClient_H
 - AAX_CSessionDocumentClient.h, 1151
- AAX_CStateDisplayDelegate
 - AAX_CStateDisplayDelegate< T >, 599
- AAX_CStateDisplayDelegate< T >, 598
 - AAX_CStateDisplayDelegate, 599
 - AddShortenedStrings, 601

- Clone, [600](#)
- Compare, [602](#)
- StringToValue, [601](#)
- ValueToString, [600](#)
- AAX_CStateDisplayDelegate.h, [1151](#)
- AAX_CStatelessParameter, [602](#)
 - AAX_CStatelessParameter, [605](#)
 - AAX_DEFAULT_DTOR_OVERRIDE, [605](#)
 - AddShortenedName, [606](#)
 - Automatable, [607](#)
 - ClearShortenedNames, [607](#)
 - CloneValue, [605](#)
 - GetBoolFromNormalizedValue, [615](#)
 - GetDoubleFromNormalizedValue, [616](#)
 - GetFloatFromNormalizedValue, [615](#)
 - GetInt32FromNormalizedValue, [615](#)
 - GetNormalizedDefaultValue, [609](#)
 - GetNormalizedValue, [609](#)
 - GetNormalizedValueFromBool, [612](#)
 - GetNormalizedValueFromDouble, [614](#)
 - GetNormalizedValueFromFloat, [613](#)
 - GetNormalizedValueFromInt32, [613](#)
 - GetNormalizedValueFromStep, [611](#)
 - GetNormalizedValueFromString, [614](#)
 - GetNumberOfSteps, [610](#)
 - GetOrientation, [623](#)
 - GetStepValue, [610](#)
 - GetStepValueFromNormalizedValue, [611](#)
 - GetStringFromNormalizedValue, [616](#), [617](#)
 - GetType, [622](#)
 - GetValueAsBool, [618](#)
 - GetValueAsDouble, [619](#)
 - GetValueAsFloat, [619](#)
 - GetValueAsInt32, [618](#)
 - GetValueAsString, [619](#)
 - GetValueString, [611](#), [612](#)
 - Identifier, [605](#)
 - mAutomationDelegate, [625](#)
 - mID, [624](#)
 - mNames, [624](#)
 - mValueString, [625](#)
 - Name, [606](#)
 - Release, [608](#)
 - SetAutomationDelegate, [608](#)
 - SetDisplayDelegate, [624](#)
 - SetName, [606](#)
 - SetNormalizedDefaultValue, [609](#)
 - SetNormalizedValue, [609](#)
 - SetNumberOfSteps, [610](#)
 - SetOrientation, [623](#)
 - SetStepValue, [611](#)
 - SetTaperDelegate, [623](#)
 - SetToDefaultValue, [610](#)
 - SetType, [622](#)
 - SetValueFromString, [617](#)
 - SetValueWithBool, [620](#)
 - SetValueWithDouble, [621](#)
 - SetValueWithFloat, [621](#)
 - SetValueWithInt32, [620](#)
 - SetValueWithString, [622](#)
 - ShortenedName, [607](#)
 - Touch, [608](#)
 - UpdateNormalizedValue, [624](#)
- AAX_CStateTaperDelegate
 - AAX_CStateTaperDelegate< T >, [626](#)
- AAX_CStateTaperDelegate< T >, [625](#)
 - AAX_CStateTaperDelegate, [626](#)
 - Clone, [626](#)
 - ConstrainRealValue, [627](#)
 - GetMaximumValue, [627](#)
 - GetMinimumValue, [627](#)
 - NormalizedToReal, [628](#)
 - RealToNormalized, [628](#)
- AAX_CStateTaperDelegate.h, [1151](#)
- AAX_CString, [629](#)
 - AAX_CString, [630](#), [631](#)
 - AAX_DEFAULT_MOVE_CTOR, [632](#)
 - Append, [634](#)
 - AppendHex, [634](#)
 - AppendNumber, [634](#)
 - Clear, [633](#)
 - CString, [637](#)
 - Empty, [633](#)
 - Equals, [637](#), [638](#)
 - Erase, [634](#)
 - FindFirst, [636](#)
 - FindLast, [636](#), [637](#)
 - Get, [632](#)
 - Insert, [635](#)
 - InsertHex, [635](#)
 - InsertNumber, [635](#)
 - kInvalidIndex, [640](#)
 - kMaxStringLength, [641](#)
 - Length, [631](#)
 - MaxLength, [631](#)
 - mString, [641](#)
 - operator!=, [638](#), [639](#)
 - operator<, [639](#)
 - operator<<, [640](#)
 - operator>, [639](#)
 - operator>>, [640](#)
 - operator+&, [639](#), [640](#)
 - operator=, [632](#), [633](#)
 - operator==, [638](#)
 - operator[], [639](#)
 - Replace, [635](#), [636](#)
 - Set, [632](#)
 - StdString, [633](#)
 - SubString, [637](#)
 - ToDouble, [637](#)
 - ToInteger, [637](#)
- AAX_CString.h, [1152](#)
 - AAX_CSTRING_H, [1152](#)
 - operator+, [1153](#)
- AAX_CSTRING_H
 - AAX_CString.h, [1152](#)

- AAX_CStringAbbreviations, 641
 - AAX_CStringAbbreviations, 641
 - Add, 642
 - Clear, 642
 - Get, 642
 - Primary, 642
 - SetPrimary, 642
- AAX_CStringDataBuffer, 643
 - ~AAX_CStringDataBuffer, 644
 - AAX_CStringDataBuffer, 643, 644
 - Data, 645
 - operator=, 644
 - Size, 645
 - Type, 644
- AAX_CStringDataBuffer.h, 1153
 - AAX_CStringDataBuffer_H, 1154
- AAX_CStringDataBuffer_H
 - AAX_CStringDataBuffer.h, 1154
- AAX_CStringDataBufferOfType
 - AAX_CStringDataBufferOfType< T >, 646, 647
- AAX_CStringDataBufferOfType< T >, 645
 - ~AAX_CStringDataBufferOfType, 647
 - AAX_CStringDataBufferOfType, 646, 647
 - Data, 648
 - operator=, 647
 - Size, 648
 - Type, 647
- AAX_CStringDisplayDelegate
 - AAX_CStringDisplayDelegate< T >, 649
- AAX_CStringDisplayDelegate< T >, 648
 - AAX_CStringDisplayDelegate, 649
 - Clone, 650
 - mInverseStringMap, 652
 - mStringMap, 652
 - StringToValue, 651
 - ValueToString, 650, 651
- AAX_CStringDisplayDelegate.h, 1154
- AAX_CTargetPlatform
 - AAX.h, 1114
- AAX_CTask, 652
 - AAX_CTask, 653
 - AAX_DEFAULT_DTOR_OVERRIDE, 653
 - AAX_DELETE, 653
 - AAX_OVERRIDE, 656
 - ACF_DECLARE_STANDARD_UNKNOWN, 653
 - AddResult, 655
 - GetArgumentOfType, 654
 - GetProgress, 654
 - GetType, 653
 - SetDone, 655
 - SetProgress, 654
 - Status, 655
- AAX_CTask.h, 1154
 - AAX_CTask_H, 1155
- AAX_CTask_H
 - AAX_CTask.h, 1155
- AAX_CTaskAgent, 656
 - ~AAX_CTaskAgent, 657
- AAX_CTaskAgent, 657
 - AddTask, 658
 - CancelAllTasks, 658
 - GetController, 659
 - GetEffectParameters, 659
 - Initialize, 657
 - ReceiveTask, 658
 - Uninitialize, 658
- AAX_CTaskAgent.h, 1155
- AAX_CTempoBreakpoint, 659
 - mSampleLocation, 659
 - mValue, 659
- AAX_CTimeOfDay
 - AAX.h, 1112
- AAX_CTimestamp
 - AAX.h, 1112
- AAX_CTransportCounter
 - AAX.h, 1112
- AAX_CTypeID
 - AAX.h, 1113
- AAX_CUnitDisplayDelegateDecorator
 - AAX_CUnitDisplayDelegateDecorator< T >, 661
- AAX_CUnitDisplayDelegateDecorator< T >, 660
 - AAX_CUnitDisplayDelegateDecorator, 661
 - Clone, 661
 - mUnitString, 663
 - StringToValue, 662
 - ValueToString, 661, 662
- AAX_CUnitDisplayDelegateDecorator.h, 1155
- AAX_CUnitPrefixDisplayDelegateDecorator
 - AAX_CUnitPrefixDisplayDelegateDecorator< T >, 664
- AAX_CUnitPrefixDisplayDelegateDecorator< T >, 663
 - AAX_CUnitPrefixDisplayDelegateDecorator, 664
 - Clone, 665
 - StringToValue, 666
 - ValueToString, 665, 666
- AAX_CUnitPrefixDisplayDelegateDecorator.h, 1156
- AAX_DEBUGASSERT
 - AAX_Assert.h, 1123
- AAX_DEFAULT_ASGN_OPER
 - AAX.h, 1107
- AAX_DEFAULT_COPY_CTOR
 - AAX.h, 1107
- AAX_DEFAULT_CTOR
 - AAX.h, 1107
- AAX_DEFAULT_DTOR
 - AAX.h, 1107
- AAX_DEFAULT_DTOR_OVERRIDE
 - AAX.h, 1107
- AAX_CParameterValue< T >, 575
- AAX_CStatelessParameter, 605
- AAX_CTask, 653
- AAX_DEFAULT_MOVE_CTOR
 - AAX.h, 1108
- AAX::Exception::Any, 1090
- AAX_CParameter< T >, 537
- AAX_CParameterValue< T >, 575

- AAX_CString, [632](#)
- AAX_DEFAULT_MOVE_OPER
 - AAX.h, [1108](#)
 - AAX::Exception::Any, [1090](#)
 - AAX_CParameter< T >, [537](#)
 - AAX_CParameterValue< T >, [575](#)
- AAX_DELETE
 - AAX.h, [1108](#)
 - AAX_CParameter< T >, [537](#), [538](#)
 - AAX_CParameterValue< T >, [575](#)
 - AAX_CTask, [653](#)
 - AAX_IDataBuffer, [841](#)
 - AAX_IEffectDirectData, [870](#)
 - AAX_IEffectGUI, [871](#)
 - AAX_IEffectParameters, [874](#)
 - AAX_IHostProcessor, [877](#)
 - AAX_ISessionDocumentClient, [937](#)
 - AAX_ITaskAgent, [948](#)
- AAX_Denormal.h, [1156](#)
 - AAX_DENORMAL_H, [1157](#)
 - AAX_SCOPE_COMPUTE_DENORMALS, [1157](#)
 - AAX_SCOPE_DENORMALS_AS_ZERO, [1157](#)
- AAX_DENORMAL_H
 - AAX_Denormal.h, [1157](#)
- AAX_DigiTrace_Guide.doxygen, [1157](#)
- AAX_DistributingYourPlugIn.doxygen, [1157](#)
- AAX_DMA_API
 - AAX_IDma.h, [1238](#)
- AAX_DocsDirectory.doxygen, [1157](#)
- AAX_DocumentData_UID
 - AAX_UIDs.h, [1295](#)
- AAX_DocumentDataType_TempoMap
 - AAX_UIDs.h, [1309](#)
- AAX_DWORD_ALIGNED_HINT
 - AAX_MiscUtils.h, [1255](#)
- AAX_EAssertFlags
 - AAX_Enums.h, [1200](#)
- AAX_eAssertFlags_Default
 - AAX_Enums.h, [1200](#)
- AAX_eAssertFlags_Dialog
 - AAX_Enums.h, [1200](#)
- AAX_eAssertFlags_Log
 - AAX_Enums.h, [1200](#)
- AAX_EAudioBufferLength
 - AAX_Enums.h, [1173](#)
- AAX_eAudioBufferLength_1
 - AAX_Enums.h, [1174](#)
- AAX_eAudioBufferLength_1024
 - AAX_Enums.h, [1174](#)
- AAX_eAudioBufferLength_128
 - AAX_Enums.h, [1174](#)
- AAX_eAudioBufferLength_16
 - AAX_Enums.h, [1174](#)
- AAX_eAudioBufferLength_2
 - AAX_Enums.h, [1174](#)
- AAX_eAudioBufferLength_256
 - AAX_Enums.h, [1174](#)
- AAX_eAudioBufferLength_32
 - AAX_Enums.h, [1174](#)
- AAX_Enums.h, [1174](#)
- AAX_eAudioBufferLength_4
 - AAX_Enums.h, [1174](#)
- AAX_eAudioBufferLength_512
 - AAX_Enums.h, [1174](#)
- AAX_eAudioBufferLength_64
 - AAX_Enums.h, [1174](#)
- AAX_eAudioBufferLength_8
 - AAX_Enums.h, [1174](#)
- AAX_eAudioBufferLength_Max
 - AAX_Enums.h, [1174](#)
- AAX_eAudioBufferLength_Undefined
 - AAX_Enums.h, [1174](#)
- AAX_EAudioBufferLengthDSP
 - AAX_Enums.h, [1174](#)
- AAX_eAudioBufferLengthDSP_16
 - AAX_Enums.h, [1174](#)
- AAX_eAudioBufferLengthDSP_32
 - AAX_Enums.h, [1174](#)
- AAX_eAudioBufferLengthDSP_4
 - AAX_Enums.h, [1174](#)
- AAX_eAudioBufferLengthDSP_64
 - AAX_Enums.h, [1174](#)
- AAX_eAudioBufferLengthDSP_Default
 - AAX_Enums.h, [1174](#)
- AAX_eAudioBufferLengthDSP_Max
 - AAX_Enums.h, [1174](#)
- AAX_eAudioBufferLengthNative_Max
 - AAX_Enums.h, [1175](#)
- AAX_eAudioBufferLengthNative_Min
 - AAX_Enums.h, [1175](#)
- AAX_EComponentInstanceInitAction
 - AAX_Enums.h, [1189](#)
- AAX_eComponentInstanceInitAction_AddingNewInstance
 - AAX_Enums.h, [1189](#)
- AAX_eComponentInstanceInitAction_RemovingInstance
 - AAX_Enums.h, [1189](#)
- AAX_eComponentInstanceInitAction_ResetInstance
 - AAX_Enums.h, [1189](#)
- AAX_EConstraintLocationMask
 - AAX_Enums.h, [1188](#)
- AAX_eConstraintLocationMask_DataModel
 - AAX_Enums.h, [1188](#)
- AAX_eConstraintLocationMask_DLLChipAffinity
 - AAX_Enums.h, [1188](#)
- AAX_eConstraintLocationMask_None
 - AAX_Enums.h, [1188](#)
- AAX_EConstraintTopology
 - AAX_Enums.h, [1188](#)
- AAX_eConstraintTopology_Monolithic
 - AAX_Enums.h, [1189](#)
- AAX_eConstraintTopology_None
 - AAX_Enums.h, [1189](#)
- AAX_ECurveType
 - EQ and Dynamics Curve Displays, [91](#)
- AAX_eCurveType_Dynamics
 - EQ and Dynamics Curve Displays, [92](#)
- AAX_eCurveType_EQ

- EQ and Dynamics Curve Displays, [92](#)
- AAX_eCurveType_None
 - EQ and Dynamics Curve Displays, [92](#)
- AAX_eCurveType_Reduction
 - EQ and Dynamics Curve Displays, [92](#)
- AAX_EDataInPortType
 - AAX_Enums.h, [1194](#)
- AAX_eDataInPortType_Buffered
 - AAX_Enums.h, [1194](#)
- AAX_eDataInPortType_Incremental
 - AAX_Enums.h, [1195](#)
- AAX_eDataInPortType_Unbuffered
 - AAX_Enums.h, [1194](#)
- AAX_eEQBandType_HighPass
 - AAX_Enums.h, [1192](#)
- AAX_eEQBandType_HighShelf
 - AAX_Enums.h, [1192](#)
- AAX_eEQBandType_LowPass
 - AAX_Enums.h, [1192](#)
- AAX_eEQBandType_LowShelf
 - AAX_Enums.h, [1192](#)
- AAX_eEQBandType_Notch
 - AAX_Enums.h, [1192](#)
- AAX_eEQBandType_Parametric
 - AAX_Enums.h, [1192](#)
- AAX_EEQBandTypes
 - AAX_Enums.h, [1191](#)
- AAX_EEQInCircuitPolarity
 - AAX_Enums.h, [1192](#)
- AAX_eEQInCircuitPolarity_Bypassed
 - AAX_Enums.h, [1192](#)
- AAX_eEQInCircuitPolarity_Disabled
 - AAX_Enums.h, [1192](#)
- AAX_eEQInCircuitPolarity_Enabled
 - AAX_Enums.h, [1192](#)
- AAX_EError
 - AAX_Errors.h, [1210](#)
- AAX_EFeetFramesRate
 - AAX_Enums.h, [1196](#)
- AAX_eFeetFramesRate_23976
 - AAX_Enums.h, [1196](#)
- AAX_eFeetFramesRate_24
 - AAX_Enums.h, [1196](#)
- AAX_eFeetFramesRate_25
 - AAX_Enums.h, [1196](#)
- AAX_EFrameRate
 - AAX_Enums.h, [1195](#)
- AAX_eFrameRate_100Frame
 - AAX_Enums.h, [1195](#)
- AAX_eFrameRate_11988DropFrame
 - AAX_Enums.h, [1195](#)
- AAX_eFrameRate_11988NonDrop
 - AAX_Enums.h, [1195](#)
- AAX_eFrameRate_120DropFrame
 - AAX_Enums.h, [1195](#)
- AAX_eFrameRate_120NonDrop
 - AAX_Enums.h, [1195](#)
- AAX_eFrameRate_23976
 - AAX_Enums.h, [1195](#)
- AAX_eFrameRate_24Frame
 - AAX_Enums.h, [1195](#)
- AAX_eFrameRate_25Frame
 - AAX_Enums.h, [1195](#)
- AAX_eFrameRate_2997DropFrame
 - AAX_Enums.h, [1195](#)
- AAX_eFrameRate_2997NonDrop
 - AAX_Enums.h, [1195](#)
- AAX_eFrameRate_30DropFrame
 - AAX_Enums.h, [1195](#)
- AAX_eFrameRate_30NonDrop
 - AAX_Enums.h, [1195](#)
- AAX_eFrameRate_47952
 - AAX_Enums.h, [1195](#)
- AAX_eFrameRate_48Frame
 - AAX_Enums.h, [1195](#)
- AAX_eFrameRate_50Frame
 - AAX_Enums.h, [1195](#)
- AAX_eFrameRate_5994DropFrame
 - AAX_Enums.h, [1195](#)
- AAX_eFrameRate_5994NonDrop
 - AAX_Enums.h, [1195](#)
- AAX_eFrameRate_60DropFrame
 - AAX_Enums.h, [1195](#)
- AAX_eFrameRate_60NonDrop
 - AAX_Enums.h, [1195](#)
- AAX_eFrameRate_Undeclared
 - AAX_Enums.h, [1195](#)
- AAX_EHighlightColor
 - AAX_Enums.h, [1172](#)
- AAX_eHighlightColor_Blue
 - AAX_Enums.h, [1172](#)
- AAX_eHighlightColor_Green
 - AAX_Enums.h, [1172](#)
- AAX_eHighlightColor_Num
 - AAX_Enums.h, [1172](#)
- AAX_eHighlightColor_Red
 - AAX_Enums.h, [1172](#)
- AAX_eHighlightColor_Yellow
 - AAX_Enums.h, [1172](#)
- AAX_EHostLevel
 - AAX_Enums.h, [1199](#)
- AAX_eHostLevel_Entry
 - AAX_Enums.h, [1199](#)
- AAX_eHostLevel_Intermediate
 - AAX_Enums.h, [1199](#)
- AAX_eHostLevel_Standard
 - AAX_Enums.h, [1199](#)
- AAX_eHostLevel_Unknown
 - AAX_Enums.h, [1199](#)
- AAX_EHostMode
 - AAX_Enums.h, [1187](#)
- AAX_eHostMode_Config
 - AAX_Enums.h, [1187](#)
- AAX_eHostMode_Show
 - AAX_Enums.h, [1187](#)
- AAX_EHostModeBits

- AAX_Enums.h, [1186](#)
- AAX_eHostModeBits_Live
 - AAX_Enums.h, [1187](#)
- AAX_eHostModeBits_None
 - AAX_Enums.h, [1187](#)
- AAX_EMaxAudioSuiteTracks
 - AAX_Enums.h, [1175](#)
- AAX_eMaxAudioSuiteTracks
 - AAX_Enums.h, [1175](#)
- AAX_EMeterBallisticType
 - AAX_Enums.h, [1180](#)
- AAX_eMeterBallisticType_Host
 - AAX_Enums.h, [1180](#)
- AAX_eMeterBallisticType_NoDecay
 - AAX_Enums.h, [1180](#)
- AAX_EMeterOrientation
 - AAX_Enums.h, [1179](#)
- AAX_eMeterOrientation_BottomLeft
 - AAX_Enums.h, [1179](#)
- AAX_eMeterOrientation_Center
 - AAX_Enums.h, [1179](#)
- AAX_eMeterOrientation_Default
 - AAX_Enums.h, [1179](#)
- AAX_eMeterOrientation_PhaseDot
 - AAX_Enums.h, [1179](#)
- AAX_eMeterOrientation_TopRight
 - AAX_Enums.h, [1179](#)
- AAX_EMeterType
 - AAX_Enums.h, [1180](#)
- AAX_eMeterType_Analysis
 - AAX_Enums.h, [1180](#)
- AAX_eMeterType_CLGain
 - AAX_Enums.h, [1180](#)
- AAX_eMeterType_EGGain
 - AAX_Enums.h, [1180](#)
- AAX_eMeterType_Input
 - AAX_Enums.h, [1180](#)
- AAX_eMeterType_None
 - AAX_Enums.h, [1180](#)
- AAX_eMeterType_Other
 - AAX_Enums.h, [1180](#)
- AAX_eMeterType_Output
 - AAX_Enums.h, [1180](#)
- AAX_eMIDIBeatClock
 - AAX_Enums.h, [1196](#)
- AAX_eMIDIClick
 - AAX_Enums.h, [1196](#)
- AAX_EMidiGlobalNodeSelectors
 - AAX_Enums.h, [1196](#)
- AAX_eMIDIMtc
 - AAX_Enums.h, [1196](#)
- AAX_EMIDINodeType
 - AAX_Enums.h, [1192](#)
- AAX_eMIDINodeType_Global
 - AAX_Enums.h, [1193](#)
- AAX_eMIDINodeType_LocalInput
 - AAX_Enums.h, [1193](#)
- AAX_eMIDINodeType_LocalOutput
 - AAX_Enums.h, [1193](#)
- AAX_Enums.h, [1193](#)
- AAX_eMIDINodeType_Transport
 - AAX_Enums.h, [1193](#)
- AAX_EModifiers
 - AAX_Enums.h, [1173](#)
- AAX_eModifiers_Alt
 - AAX_Enums.h, [1173](#)
- AAX_eModifiers_Cntl
 - AAX_Enums.h, [1173](#)
- AAX_eModifiers_Command
 - AAX_Enums.h, [1173](#)
- AAX_eModifiers_Control
 - AAX_Enums.h, [1173](#)
- AAX_eModifiers_None
 - AAX_Enums.h, [1173](#)
- AAX_eModifiers_Option
 - AAX_Enums.h, [1173](#)
- AAX_eModifiers_SecondaryButton
 - AAX_Enums.h, [1173](#)
- AAX_eModifiers_Shift
 - AAX_Enums.h, [1173](#)
- AAX_eModifiers_WINKEY
 - AAX_Enums.h, [1173](#)
- AAX_EndianSwap
 - AAX_EndianSwap.h, [1159](#)
- AAX_EndianSwap.h, [1157](#)
 - AAX_BigEndianNativeSwap, [1159](#)
 - AAX_BigEndianNativeSwapInPlace, [1159](#)
 - AAX_BigEndianNativeSwapSequenceInPlace, [1160](#)
 - AAX_EndianSwap, [1159](#)
 - AAX_EndianSwapInPlace, [1159](#)
 - AAX_EndianSwapSequenceInPlace, [1160](#)
 - AAX_LittleEndianNativeSwap, [1160](#)
 - AAX_LittleEndianNativeSwapInPlace, [1159](#)
 - AAX_LittleEndianNativeSwapSequenceInPlace, [1160](#)
- ENDIANSWAP_H, [1158](#)
- AAX_EndianSwapInPlace
 - AAX_EndianSwap.h, [1159](#)
- AAX_EndianSwapSequenceInPlace
 - AAX_EndianSwap.h, [1160](#)
- AAX_ENotificationEvent
 - AAX_Enums.h, [1181](#)
- AAX_eNotificationEvent_AlgorithmMoved
 - AAX_Enums.h, [1182](#)
- AAX_eNotificationEvent_ASPreviewState
 - AAX_Enums.h, [1182](#)
- AAX_eNotificationEvent_ASProcessingState
 - AAX_Enums.h, [1182](#)
- AAX_eNotificationEvent_CycleCountChanged
 - AAX_Enums.h, [1183](#)
- AAX_eNotificationEvent_DelayCompensationState
 - AAX_Enums.h, [1183](#)
- AAX_eNotificationEvent_EnteringOfflineMode
 - AAX_Enums.h, [1182](#)
- AAX_eNotificationEvent_ExitingOfflineMode
 - AAX_Enums.h, [1183](#)

- AAX_eNotificationEvent_GUIClosed
 - AAX_Enums.h, [1182](#)
- AAX_eNotificationEvent_GUIOpened
 - AAX_Enums.h, [1182](#)
- AAX_eNotificationEvent_HostLocale
 - AAX_Enums.h, [1186](#)
- AAX_eNotificationEvent_HostModeChanged
 - AAX_Enums.h, [1185](#)
- AAX_eNotificationEvent_InsertPositionChanged
 - AAX_Enums.h, [1181](#)
- AAX_eNotificationEvent_LogState
 - AAX_Enums.h, [1185](#)
- AAX_eNotificationEvent_MaxViewSizeChanged
 - AAX_Enums.h, [1184](#)
- AAX_eNotificationEvent_NoiseFloorChanged
 - AAX_Enums.h, [1184](#)
- AAX_eNotificationEvent_ParameterMappingChanged
 - AAX_Enums.h, [1184](#)
- AAX_eNotificationEvent_ParameterNameChanged
 - AAX_Enums.h, [1184](#)
- AAX_eNotificationEvent_PresetOpened
 - AAX_Enums.h, [1182](#)
- AAX_eNotificationEvent_PriorSettingsInvalid
 - AAX_Enums.h, [1185](#)
- AAX_eNotificationEvent_SessionBeingOpened
 - AAX_Enums.h, [1182](#)
- AAX_eNotificationEvent_SessionPathChanged
 - AAX_Enums.h, [1183](#)
- AAX_eNotificationEvent_SideChainBeingConnected
 - AAX_Enums.h, [1184](#)
- AAX_eNotificationEvent_SideChainBeingDisconnected
 - AAX_Enums.h, [1184](#)
- AAX_eNotificationEvent_SignalLatencyChanged
 - AAX_Enums.h, [1183](#)
- AAX_eNotificationEvent_TrackNameChanged
 - AAX_Enums.h, [1181](#)
- AAX_eNotificationEvent_TrackPositionChanged
 - AAX_Enums.h, [1181](#)
- AAX_eNotificationEvent_TrackUIDChanged
 - AAX_Enums.h, [1181](#)
- AAX_eNotificationEvent_TransportStateChanged
 - AAX_Enums.h, [1185](#)
- AAX_ENUM_SIZE_CHECK
 - AAX_Enums.h, [1171](#), [1201–1208](#)
 - AAX_Errors.h, [1212](#)
 - AAX_GUITypes.h, [1223](#)
 - AAX_Properties.h, [1278](#)
- AAX_Enums.h, [1161](#)
 - AAE_EAudioBufferLengthNative, [1174](#)
 - AAX_EAssertFlags, [1200](#)
 - AAX_eAssertFlags_Default, [1200](#)
 - AAX_eAssertFlags_Dialog, [1200](#)
 - AAX_eAssertFlags_Log, [1200](#)
 - AAX_EAudioBufferLength, [1173](#)
 - AAX_eAudioBufferLength_1, [1174](#)
 - AAX_eAudioBufferLength_1024, [1174](#)
 - AAX_eAudioBufferLength_128, [1174](#)
 - AAX_eAudioBufferLength_16, [1174](#)
 - AAX_eAudioBufferLength_2, [1174](#)
 - AAX_eAudioBufferLength_256, [1174](#)
 - AAX_eAudioBufferLength_32, [1174](#)
 - AAX_eAudioBufferLength_4, [1174](#)
 - AAX_eAudioBufferLength_512, [1174](#)
 - AAX_eAudioBufferLength_64, [1174](#)
 - AAX_eAudioBufferLength_8, [1174](#)
 - AAX_eAudioBufferLength_Max, [1174](#)
 - AAX_eAudioBufferLength_Undefined, [1174](#)
 - AAX_EAudioBufferLengthDSP, [1174](#)
 - AAX_eAudioBufferLengthDSP_16, [1174](#)
 - AAX_eAudioBufferLengthDSP_32, [1174](#)
 - AAX_eAudioBufferLengthDSP_4, [1174](#)
 - AAX_eAudioBufferLengthDSP_64, [1174](#)
 - AAX_eAudioBufferLengthDSP_Default, [1174](#)
 - AAX_eAudioBufferLengthDSP_Max, [1174](#)
 - AAX_eAudioBufferLengthNative_Max, [1175](#)
 - AAX_eAudioBufferLengthNative_Min, [1175](#)
 - AAX_EComponentInstanceInitAction, [1189](#)
 - AAX_eComponentInstanceInitAction_AddingNewInstance, [1189](#)
 - AAX_eComponentInstanceInitAction_RemovingInstance, [1189](#)
 - AAX_eComponentInstanceInitAction_ResetInstance, [1189](#)
 - AAX_EConstraintLocationMask, [1188](#)
 - AAX_eConstraintLocationMask_DataModel, [1188](#)
 - AAX_eConstraintLocationMask_DLLChipAffinity, [1188](#)
 - AAX_eConstraintLocationMask_None, [1188](#)
 - AAX_EConstraintTopology, [1188](#)
 - AAX_eConstraintTopology_Monolithic, [1189](#)
 - AAX_eConstraintTopology_None, [1189](#)
 - AAX_EDataInPortType, [1194](#)
 - AAX_eDataInPortType_Buffered, [1194](#)
 - AAX_eDataInPortType_Incremental, [1195](#)
 - AAX_eDataInPortType_Unbuffered, [1194](#)
 - AAX_eEQBandType_HighPass, [1192](#)
 - AAX_eEQBandType_HighShelf, [1192](#)
 - AAX_eEQBandType_LowPass, [1192](#)
 - AAX_eEQBandType_LowShelf, [1192](#)
 - AAX_eEQBandType_Notch, [1192](#)
 - AAX_eEQBandType_Parametric, [1192](#)
 - AAX_EEQBandTypes, [1191](#)
 - AAX_EEQInCircuitPolarity, [1192](#)
 - AAX_eEQInCircuitPolarity_Bypassed, [1192](#)
 - AAX_eEQInCircuitPolarity_Disabled, [1192](#)
 - AAX_eEQInCircuitPolarity_Enabled, [1192](#)
 - AAX_EFeetFramesRate, [1196](#)
 - AAX_eFeetFramesRate_23976, [1196](#)
 - AAX_eFeetFramesRate_24, [1196](#)
 - AAX_eFeetFramesRate_25, [1196](#)
 - AAX_EFrameRate, [1195](#)
 - AAX_eFrameRate_100Frame, [1195](#)
 - AAX_eFrameRate_11988DropFrame, [1195](#)
 - AAX_eFrameRate_11988NonDrop, [1195](#)
 - AAX_eFrameRate_120DropFrame, [1195](#)
 - AAX_eFrameRate_120NonDrop, [1195](#)

- AAX_eFrameRate_23976, [1195](#)
- AAX_eFrameRate_24Frame, [1195](#)
- AAX_eFrameRate_25Frame, [1195](#)
- AAX_eFrameRate_2997DropFrame, [1195](#)
- AAX_eFrameRate_2997NonDrop, [1195](#)
- AAX_eFrameRate_30DropFrame, [1195](#)
- AAX_eFrameRate_30NonDrop, [1195](#)
- AAX_eFrameRate_47952, [1195](#)
- AAX_eFrameRate_48Frame, [1195](#)
- AAX_eFrameRate_50Frame, [1195](#)
- AAX_eFrameRate_5994DropFrame, [1195](#)
- AAX_eFrameRate_5994NonDrop, [1195](#)
- AAX_eFrameRate_60DropFrame, [1195](#)
- AAX_eFrameRate_60NonDrop, [1195](#)
- AAX_eFrameRate_Undeclared, [1195](#)
- AAX_EHighlightColor, [1172](#)
- AAX_eHighlightColor_Blue, [1172](#)
- AAX_eHighlightColor_Green, [1172](#)
- AAX_eHighlightColor_Num, [1172](#)
- AAX_eHighlightColor_Red, [1172](#)
- AAX_eHighlightColor_Yellow, [1172](#)
- AAX_EHostLevel, [1199](#)
- AAX_eHostLevel_Entry, [1199](#)
- AAX_eHostLevel_Intermediate, [1199](#)
- AAX_eHostLevel_Standard, [1199](#)
- AAX_eHostLevel_Unknown, [1199](#)
- AAX_EHostMode, [1187](#)
- AAX_eHostMode_Config, [1187](#)
- AAX_eHostMode_Show, [1187](#)
- AAX_EHostModeBits, [1186](#)
- AAX_eHostModeBits_Live, [1187](#)
- AAX_eHostModeBits_None, [1187](#)
- AAX_EMaxAudioSuiteTracks, [1175](#)
- AAX_eMaxAudioSuiteTracks, [1175](#)
- AAX_EMeterBallisticType, [1180](#)
- AAX_eMeterBallisticType_Host, [1180](#)
- AAX_eMeterBallisticType_NoDecay, [1180](#)
- AAX_EMeterOrientation, [1179](#)
- AAX_eMeterOrientation_BottomLeft, [1179](#)
- AAX_eMeterOrientation_Center, [1179](#)
- AAX_eMeterOrientation_Default, [1179](#)
- AAX_eMeterOrientation_PhaseDot, [1179](#)
- AAX_eMeterOrientation_TopRight, [1179](#)
- AAX_EMeterType, [1180](#)
- AAX_eMeterType_Analysis, [1180](#)
- AAX_eMeterType_CLGain, [1180](#)
- AAX_eMeterType_EGGain, [1180](#)
- AAX_eMeterType_Input, [1180](#)
- AAX_eMeterType_None, [1180](#)
- AAX_eMeterType_Other, [1180](#)
- AAX_eMeterType_Output, [1180](#)
- AAX_eMIDIBeatClock, [1196](#)
- AAX_eMIDIClick, [1196](#)
- AAX_EMidiGlobalNodeSelectors, [1196](#)
- AAX_eMIDIMtc, [1196](#)
- AAX_EMIDINodeType, [1192](#)
- AAX_eMIDINodeType_Global, [1193](#)
- AAX_eMIDINodeType_LocalInput, [1193](#)
- AAX_eMIDINodeType_LocalOutput, [1193](#)
- AAX_eMIDINodeType_Transport, [1193](#)
- AAX_EModifiers, [1173](#)
- AAX_eModifiers_Alt, [1173](#)
- AAX_eModifiers_Cntl, [1173](#)
- AAX_eModifiers_Command, [1173](#)
- AAX_eModifiers_Control, [1173](#)
- AAX_eModifiers_None, [1173](#)
- AAX_eModifiers_Option, [1173](#)
- AAX_eModifiers_SecondaryButton, [1173](#)
- AAX_eModifiers_Shift, [1173](#)
- AAX_eModifiers_WINKEY, [1173](#)
- AAX_ENotificationEvent, [1181](#)
- AAX_eNotificationEvent_AlgorithmMoved, [1182](#)
- AAX_eNotificationEvent_ASPreviewState, [1182](#)
- AAX_eNotificationEvent_ASProcessingState, [1182](#)
- AAX_eNotificationEvent_CycleCountChanged, [1183](#)
- AAX_eNotificationEvent_DelayCompensationState, [1183](#)
- AAX_eNotificationEvent_EnteringOfflineMode, [1182](#)
- AAX_eNotificationEvent_ExitingOfflineMode, [1183](#)
- AAX_eNotificationEvent_GUIClosed, [1182](#)
- AAX_eNotificationEvent_GUIOpened, [1182](#)
- AAX_eNotificationEvent_HostLocale, [1186](#)
- AAX_eNotificationEvent_HostModeChanged, [1185](#)
- AAX_eNotificationEvent_InsertPositionChanged, [1181](#)
- AAX_eNotificationEvent_LogState, [1185](#)
- AAX_eNotificationEvent_MaxViewSizeChanged, [1184](#)
- AAX_eNotificationEvent_NoiseFloorChanged, [1184](#)
- AAX_eNotificationEvent_ParameterMappingChanged, [1184](#)
- AAX_eNotificationEvent_ParameterNameChanged, [1184](#)
- AAX_eNotificationEvent_PresetOpened, [1182](#)
- AAX_eNotificationEvent_PriorSettingsInvalid, [1185](#)
- AAX_eNotificationEvent_SessionBeingOpened, [1182](#)
- AAX_eNotificationEvent_SessionPathChanged, [1183](#)
- AAX_eNotificationEvent_SideChainBeingConnected, [1184](#)
- AAX_eNotificationEvent_SideChainBeingDisconnected, [1184](#)
- AAX_eNotificationEvent_SignalLatencyChanged, [1183](#)
- AAX_eNotificationEvent_TrackNameChanged, [1181](#)
- AAX_eNotificationEvent_TrackPositionChanged, [1181](#)
- AAX_eNotificationEvent_TrackUIDChanged, [1181](#)
- AAX_eNotificationEvent_TransportStateChanged, [1185](#)
- AAX_ENUM_SIZE_CHECK, [1171](#), [1201–1208](#)

- AAX_ePageTable_EQ_Band_Type, 1191
- AAX_ePageTable_EQ_InCircuitPolarity, 1191
- AAX_ePageTable_UseAlternateControl, 1191
- AAX_EParameterOrientation, 1171
- AAX_eParameterOrientation_BottomMinTopMax, 1190
- AAX_eParameterOrientation_Default, 1190
- AAX_eParameterOrientation_LeftMinRightMax, 1190
- AAX_eParameterOrientation_RightMinLeftMax, 1190
- AAX_eParameterOrientation_RotaryBoostCutMode, 1190
- AAX_eParameterOrientation_RotaryLeftMinRightMax, 1190
- AAX_eParameterOrientation_RotaryRightMinLeftMax, 1190
- AAX_eParameterOrientation_RotarySingleDotMode, 1190
- AAX_eParameterOrientation_RotarySpreadMode, 1190
- AAX_eParameterOrientation_RotaryWrapMode, 1190
- AAX_eParameterOrientation_TopMinBottomMax, 1190
- AAX_EParameterOrientationBits, 1190
- AAX_EParameterType, 1171, 1190
- AAX_eParameterType_Continuous, 1190
- AAX_eParameterType_Discrete, 1190
- AAX_EParameterValueInfoSelector, 1190
- AAX_EPlugInCategory, 1177
- AAX_ePlugInCategory_Delay, 1177
- AAX_ePlugInCategory_Dither, 1177
- AAX_ePlugInCategory_Dynamics, 1177
- AAX_EPlugInCategory_Effect, 1177
- AAX_ePlugInCategory_EQ, 1177
- AAX_ePlugInCategory_Example, 1177
- AAX_ePlugInCategory_Harmonic, 1177
- AAX_ePlugInCategory_HWGenerators, 1177
- AAX_ePlugInCategory_INT32_MAX, 1177
- AAX_EPlugInCategory_MIDIEffect, 1177
- AAX_ePlugInCategory_Modulation, 1177
- AAX_ePlugInCategory_NoiseReduction, 1177
- AAX_ePlugInCategory_None, 1177
- AAX_ePlugInCategory_PitchShift, 1177
- AAX_ePlugInCategory_Reverb, 1177
- AAX_ePlugInCategory_SoundField, 1177
- AAX_ePlugInCategory_SWGenerators, 1177
- AAX_ePlugInCategory_WrappedPlugin, 1177
- AAX_EPlugInStrings, 1178
- AAX_ePlugInStrings_AllSelectedRegionsAnalysis, 1178
- AAX_ePlugInStrings_Analysis, 1178
- AAX_ePlugInStrings_Bypass, 1179
- AAX_ePlugInStrings_ClipName, 1178
- AAX_ePlugInStrings_ClipNameSuffix, 1179
- AAX_ePlugInStrings_INT32_MAX, 1179
- AAX_ePlugInStrings_MonoMode, 1178
- AAX_ePlugInStrings_MultiInputMode, 1178
- AAX_ePlugInStrings_PluginFileName, 1179
- AAX_ePlugInStrings_Preview, 1179
- AAX_ePlugInStrings_Process, 1179
- AAX_ePlugInStrings_Progress, 1179
- AAX_ePlugInStrings_RegionByRegionAnalysis, 1178
- AAX_ePlugInStrings_RegionName, 1178
- AAX_EPreviewState, 1196
- AAX_ePreviewState_Start, 1197
- AAX_ePreviewState_Stop, 1197
- AAX_EPrivateDataOptions, 1187
- AAX_ePrivateDataOptions_Align8, 1188
- AAX_ePrivateDataOptions_DefaultOptions, 1187
- AAX_ePrivateDataOptions_External, 1188
- AAX_ePrivateDataOptions_INT32_MAX, 1188
- AAX_ePrivateDataOptions_KeepOnReset, 1187
- AAX_EProcessingState, 1197
- AAX_eProcessingState_BeginPassGroup, 1198
- AAX_eProcessingState_EndPassGroup, 1197
- AAX_eProcessingState_Start, 1198
- AAX_eProcessingState_StartPass, 1197
- AAX_eProcessingState_Stop, 1198
- AAX_eProcessingState_StopPass, 1197
- AAX_ERecordMode, 1200
- AAX_eRecordMode_Destructive, 1201
- AAX_eRecordMode_None, 1201
- AAX_eRecordMode_Normal, 1201
- AAX_eRecordMode_Num, 1201
- AAX_eRecordMode_QuickPunch, 1201
- AAX_eRecordMode_TrackPunch, 1201
- AAX_eRecordMode_Unknown, 1201
- AAX_EResourceType, 1180
- AAX_eResourceType_None, 1181
- AAX_eResourceType_PageTable, 1181
- AAX_eResourceType_PageTableDir, 1181
- AAX_ESampleRateMask, 1189
- AAX_eSampleRateMask_176400, 1189
- AAX_eSampleRateMask_192000, 1189
- AAX_eSampleRateMask_44100, 1189
- AAX_eSampleRateMask_48000, 1189
- AAX_eSampleRateMask_88200, 1189
- AAX_eSampleRateMask_96000, 1189
- AAX_eSampleRateMask_All, 1189
- AAX_eSampleRateMask_No, 1189
- AAX_EStemFormat, 1175
- AAX_eStemFormat_5_0, 1176
- AAX_eStemFormat_5_0_2, 1176
- AAX_eStemFormat_5_0_4, 1176
- AAX_eStemFormat_5_1, 1176
- AAX_eStemFormat_5_1_2, 1176
- AAX_eStemFormat_5_1_4, 1176
- AAX_eStemFormat_6_0, 1176
- AAX_eStemFormat_6_1, 1176
- AAX_eStemFormat_7_0_2, 1176
- AAX_eStemFormat_7_0_4, 1176
- AAX_eStemFormat_7_0_6, 1176
- AAX_eStemFormat_7_0_DTS, 1176

- AAX_eStemFormat_7_0_SDDS, 1176
- AAX_eStemFormat_7_1_2, 1176
- AAX_eStemFormat_7_1_4, 1176
- AAX_eStemFormat_7_1_6, 1176
- AAX_eStemFormat_7_1_DTS, 1176
- AAX_eStemFormat_7_1_SDDS, 1176
- AAX_eStemFormat_9_0_4, 1176
- AAX_eStemFormat_9_0_6, 1176
- AAX_eStemFormat_9_1_4, 1176
- AAX_eStemFormat_9_1_6, 1176
- AAX_eStemFormat_Ambi_1_ACN, 1176
- AAX_eStemFormat_Ambi_2_ACN, 1176
- AAX_eStemFormat_Ambi_3_ACN, 1176
- AAX_eStemFormat_Ambi_4_ACN, 1176
- AAX_eStemFormat_Ambi_5_ACN, 1176
- AAX_eStemFormat_Ambi_6_ACN, 1177
- AAX_eStemFormat_Ambi_7_ACN, 1177
- AAX_eStemFormat_Any, 1177
- AAX_eStemFormat_INT32_MAX, 1177
- AAX_eStemFormat_LCR, 1176
- AAX_eStemFormat_LCRS, 1176
- AAX_eStemFormat_Mono, 1176
- AAX_eStemFormat_None, 1177
- AAX_eStemFormat_Quad, 1176
- AAX_eStemFormat_Stereo, 1176
- AAX_eStemFormatNum, 1177
- AAX_ESupportLevel, 1198
- AAX_eSupportLevel_ByProperty, 1199
- AAX_eSupportLevel_Disabled, 1199
- AAX_eSupportLevel_Supported, 1199
- AAX_eSupportLevel_Uninitialized, 1199
- AAX_eSupportLevel_Unsupported, 1199
- AAX_ETargetPlatform, 1198
- AAX_ETextEncoding, 1199
- AAX_eTextEncoding_Num, 1200
- AAX_eTextEncoding_Undefined, 1200
- AAX_eTextEncoding_UTF8, 1200
- AAX_ETracePriorityDSP, 1173
- AAX_eTracePriorityDSP_Assert, 1173
- AAX_eTracePriorityDSP_High, 1173
- AAX_eTracePriorityDSP_Low, 1173
- AAX_eTracePriorityDSP_None, 1173
- AAX_eTracePriorityDSP_Normal, 1173
- AAX_ETracePriorityHost, 1172
- AAX_eTracePriorityHost_Critical, 1172
- AAX_eTracePriorityHost_High, 1172
- AAX_eTracePriorityHost_Low, 1172
- AAX_eTracePriorityHost_Lowest, 1172
- AAX_eTracePriorityHost_None, 1172
- AAX_eTracePriorityHost_Normal, 1172
- AAX_ETransportState, 1200
- AAX_eTransportState_FastForward, 1200
- AAX_eTransportState_Num, 1200
- AAX_eTransportState_Paused, 1200
- AAX_eTransportState_Play, 1200
- AAX_eTransportState_Rewind, 1200
- AAX_eTransportState_Scrub, 1200
- AAX_eTransportState_Shuttle, 1200
- AAX_eTransportState_Stop, 1200
- AAX_eTransportState_Stopping, 1200
- AAX_eTransportState_Unknown, 1200
- AAX_EUpdateSource, 1194
- AAX_eUpdateSource_Chunk, 1194
- AAX_eUpdateSource_Delay, 1194
- AAX_eUpdateSource_Parameter, 1194
- AAX_eUpdateSource_Unspecified, 1194
- AAX_EUseAlternateControl, 1192
- AAX_eUseAlternateControl_No, 1192
- AAX_eUseAlternateControl_Yes, 1192
- AAX_INT16_MAX, 1170
- AAX_INT16_MIN, 1170
- AAX_INT32_MAX, 1170
- AAX_INT32_MIN, 1170
- AAX_STEM_FORMAT, 1171
- AAX_STEM_FORMAT_CHANNEL_COUNT, 1171
- AAX_STEM_FORMAT_INDEX, 1171
- AAX_UINT16_MAX, 1170
- AAX_UINT16_MIN, 1170
- AAX_UINT32_MAX, 1170
- AAX_UINT32_MIN, 1170
- getLowestSampleRateInMask, 1205
- getMaskForSampleRate, 1205
- kAAX_eTargetPlatform_Count, 1198
- kAAX_eTargetPlatform_External, 1198
- kAAX_eTargetPlatform_Native, 1198
- kAAX_eTargetPlatform_None, 1198
- kAAX_eTargetPlatform_TI, 1198
- sampleRateInMask, 1204
- AAX_EnvironmentUtilities.h, 1208
- AAX_ePageTable_EQ_Band_Type
 - AAX_Enums.h, 1191
- AAX_ePageTable_EQ_InCircuitPolarity
 - AAX_Enums.h, 1191
- AAX_ePageTable_UseAlternateControl
 - AAX_Enums.h, 1191
- AAX_EParameterOrientation
 - AAX_Enums.h, 1171
- AAX_eParameterOrientation_BottomMinTopMax
 - AAX_Enums.h, 1190
- AAX_eParameterOrientation_Default
 - AAX_Enums.h, 1190
- AAX_eParameterOrientation_LeftMinRightMax
 - AAX_Enums.h, 1190
- AAX_eParameterOrientation_RightMinLeftMax
 - AAX_Enums.h, 1190
- AAX_eParameterOrientation_RotaryBoostCutMode
 - AAX_Enums.h, 1190
- AAX_eParameterOrientation_RotaryLeftMinRightMax
 - AAX_Enums.h, 1190
- AAX_eParameterOrientation_RotaryRightMinLeftMax
 - AAX_Enums.h, 1190
- AAX_eParameterOrientation_RotarySingleDotMode
 - AAX_Enums.h, 1190
- AAX_eParameterOrientation_RotarySpreadMode
 - AAX_Enums.h, 1190
- AAX_eParameterOrientation_RotaryWrapMode

- AAX_Enums.h, [1190](#)
- AAX_eParameterOrientation_TopMinBottomMax
 - AAX_Enums.h, [1190](#)
- AAX_EParameterOrientationBits
 - AAX_Enums.h, [1190](#)
- AAX_EParameterType
 - AAX_Enums.h, [1171](#), [1190](#)
- AAX_eParameterType_Continuous
 - AAX_Enums.h, [1190](#)
- AAX_eParameterType_Discrete
 - AAX_Enums.h, [1190](#)
- AAX_EParameterValueInfoSelector
 - AAX_Enums.h, [1190](#)
- AAX_EPlugInCategory
 - AAX_Enums.h, [1177](#)
- AAX_ePlugInCategory_Delay
 - AAX_Enums.h, [1177](#)
- AAX_ePlugInCategory_Dither
 - AAX_Enums.h, [1177](#)
- AAX_ePlugInCategory_Dynamics
 - AAX_Enums.h, [1177](#)
- AAX_EPlugInCategory_Effect
 - AAX_Enums.h, [1177](#)
- AAX_ePlugInCategory_EQ
 - AAX_Enums.h, [1177](#)
- AAX_ePlugInCategory_Example
 - AAX_Enums.h, [1177](#)
- AAX_ePlugInCategory_Harmonic
 - AAX_Enums.h, [1177](#)
- AAX_ePlugInCategory_HWGenerators
 - AAX_Enums.h, [1177](#)
- AAX_ePlugInCategory_INT32_MAX
 - AAX_Enums.h, [1177](#)
- AAX_EPlugInCategory_MIDIEffect
 - AAX_Enums.h, [1177](#)
- AAX_ePlugInCategory_Modulation
 - AAX_Enums.h, [1177](#)
- AAX_ePlugInCategory_NoiseReduction
 - AAX_Enums.h, [1177](#)
- AAX_ePlugInCategory_None
 - AAX_Enums.h, [1177](#)
- AAX_ePlugInCategory_PitchShift
 - AAX_Enums.h, [1177](#)
- AAX_ePlugInCategory_Reverb
 - AAX_Enums.h, [1177](#)
- AAX_ePlugInCategory_SoundField
 - AAX_Enums.h, [1177](#)
- AAX_ePlugInCategory_SWGenerators
 - AAX_Enums.h, [1177](#)
- AAX_ePlugInCategory_WrappedPlugin
 - AAX_Enums.h, [1177](#)
- AAX_EPlugInStrings
 - AAX_Enums.h, [1178](#)
- AAX_ePlugInStrings_AllSelectedRegionsAnalysis
 - AAX_Enums.h, [1178](#)
- AAX_ePlugInStrings_Analysis
 - AAX_Enums.h, [1178](#)
- AAX_ePlugInStrings_Bypass
 - AAX_Enums.h, [1179](#)
- AAX_ePlugInStrings_ClipName
 - AAX_Enums.h, [1178](#)
- AAX_ePlugInStrings_ClipNameSuffix
 - AAX_Enums.h, [1179](#)
- AAX_ePlugInStrings_INT32_MAX
 - AAX_Enums.h, [1179](#)
- AAX_ePlugInStrings_MonoMode
 - AAX_Enums.h, [1178](#)
- AAX_ePlugInStrings_MultiInputMode
 - AAX_Enums.h, [1178](#)
- AAX_ePlugInStrings_PlugInFileName
 - AAX_Enums.h, [1179](#)
- AAX_ePlugInStrings_Preview
 - AAX_Enums.h, [1179](#)
- AAX_ePlugInStrings_Process
 - AAX_Enums.h, [1179](#)
- AAX_ePlugInStrings_Progress
 - AAX_Enums.h, [1179](#)
- AAX_ePlugInStrings_RegionByRegionAnalysis
 - AAX_Enums.h, [1178](#)
- AAX_ePlugInStrings_RegionName
 - AAX_Enums.h, [1178](#)
- AAX_EPreviewState
 - AAX_Enums.h, [1196](#)
- AAX_ePreviewState_Start
 - AAX_Enums.h, [1197](#)
- AAX_ePreviewState_Stop
 - AAX_Enums.h, [1197](#)
- AAX_EPrivateDataOptions
 - AAX_Enums.h, [1187](#)
- AAX_ePrivateDataOptions_Align8
 - AAX_Enums.h, [1188](#)
- AAX_ePrivateDataOptions_DefaultOptions
 - AAX_Enums.h, [1187](#)
- AAX_ePrivateDataOptions_External
 - AAX_Enums.h, [1188](#)
- AAX_ePrivateDataOptions_INT32_MAX
 - AAX_Enums.h, [1188](#)
- AAX_ePrivateDataOptions_KeepOnReset
 - AAX_Enums.h, [1187](#)
- AAX_EProcessingState
 - AAX_Enums.h, [1197](#)
- AAX_eProcessingState_BeginPassGroup
 - AAX_Enums.h, [1198](#)
- AAX_eProcessingState_EndPassGroup
 - AAX_Enums.h, [1197](#)
- AAX_eProcessingState_Start
 - AAX_Enums.h, [1198](#)
- AAX_eProcessingState_StartPass
 - AAX_Enums.h, [1197](#)
- AAX_eProcessingState_Stop
 - AAX_Enums.h, [1198](#)
- AAX_eProcessingState_StopPass
 - AAX_Enums.h, [1197](#)
- AAX_EProperty
 - AAX_Properties.h, [1260](#)
- AAX_eProperty_AllowPreviewWithoutAnalysis

- AAX_Properties.h, [1270](#)
- AAX_eProperty_AlwaysBypass
 - AAX_Properties.h, [1269](#)
- AAX_eProperty_AudioBufferLength
 - AAX_Properties.h, [1266](#)
- AAX_eProperty_AudiosuitePropsBase
 - AAX_Properties.h, [1269](#)
- AAX_eProperty_CanBypass
 - AAX_Properties.h, [1268](#)
- AAX_eProperty_Constraint_AlwaysProcess
 - AAX_Properties.h, [1277](#)
- AAX_eProperty_Constraint_DoNotApplyDefaultSettings
 - AAX_Properties.h, [1277](#)
- AAX_eProperty_Constraint_Location
 - AAX_Properties.h, [1274](#)
- AAX_eProperty_Constraint_MultiMonoSupport
 - AAX_Properties.h, [1275](#)
- AAX_eProperty_Constraint_NeverCache
 - AAX_Properties.h, [1274](#)
- AAX_eProperty_Constraint_NeverUnload
 - AAX_Properties.h, [1274](#)
- AAX_eProperty_Constraint_Topology
 - AAX_Properties.h, [1274](#)
- AAX_eProperty_ConstraintBase
 - AAX_Properties.h, [1274](#)
- AAX_eProperty_ConstraintBase_2
 - AAX_Properties.h, [1276](#)
- AAX_eProperty_ContinuousOnly
 - AAX_Properties.h, [1271](#)
- AAX_eProperty_DebugPropertiesBase
 - AAX_Properties.h, [1277](#)
- AAX_eProperty_Deprecated_DSP_Plugin_List
 - AAX_Properties.h, [1264](#)
- AAX_eProperty_Deprecated_Native_Plugin_List
 - AAX_Properties.h, [1265](#)
- AAX_eProperty_Deprecated_Plugin_List
 - AAX_Properties.h, [1264](#)
- AAX_eProperty_DestinationTrack
 - AAX_Properties.h, [1271](#)
- AAX_eProperty_DisableAudioSuiteReverse
 - AAX_Properties.h, [1273](#)
- AAX_eProperty_DisableHandles
 - AAX_Properties.h, [1272](#)
- AAX_eProperty_DisablePreview
 - AAX_Properties.h, [1272](#)
- AAX_eProperty_DoesntIncrOutputSample
 - AAX_Properties.h, [1272](#)
- AAX_eProperty_DSP_AudioBufferLength
 - AAX_Properties.h, [1266](#)
- AAX_eProperty_EnableHostDebugLogs
 - AAX_Properties.h, [1278](#)
- AAX_eProperty_ExternalProcessorTypeID
 - AAX_Properties.h, [1265](#)
- AAX_eProperty_FeaturesBase
 - AAX_Properties.h, [1275](#)
- AAX_eProperty_GeneralPropsBase
 - AAX_Properties.h, [1266](#)
- AAX_eProperty_GUIBase
 - AAX_Properties.h, [1273](#)
- AAX_eProperty_HybridInputStemFormat
 - AAX_Properties.h, [1269](#)
- AAX_eProperty_HybridOutputStemFormat
 - AAX_Properties.h, [1269](#)
- AAX_eProperty_InputStemFormat
 - AAX_Properties.h, [1266](#)
- AAX_eProperty_LatencyContribution
 - AAX_Properties.h, [1267](#)
- AAX_eProperty_ManufacturerID
 - AAX_Properties.h, [1261](#)
- AAX_eProperty_MaxASProp
 - AAX_Properties.h, [1273](#)
- AAX_eProperty_MaxCap
 - AAX_Properties.h, [1278](#)
- AAX_eProperty_MaxConstraintProp
 - AAX_Properties.h, [1275](#)
- AAX_eProperty_MaxConstraintProp_2
 - AAX_Properties.h, [1277](#)
- AAX_eProperty_MaxFeaturesProp
 - AAX_Properties.h, [1276](#)
- AAX_eProperty_MaxGUIProp
 - AAX_Properties.h, [1273](#)
- AAX_eProperty_MaxMeterProp
 - AAX_Properties.h, [1274](#)
- AAX_eProperty_MaxProp
 - AAX_Properties.h, [1278](#)
- AAX_eProperty_Meter_Ballistics
 - AAX_Properties.h, [1274](#)
- AAX_eProperty_Meter_Orientation
 - AAX_Properties.h, [1274](#)
- AAX_eProperty_Meter_Type
 - AAX_Properties.h, [1273](#)
- AAX_eProperty_MeterBase
 - AAX_Properties.h, [1273](#)
- AAX_eProperty_MinProp
 - AAX_Properties.h, [1261](#)
- AAX_eProperty_MultiInputModeOnly
 - AAX_Properties.h, [1271](#)
- AAX_eProperty_NativeBackgroundProc
 - AAX_Properties.h, [1266](#)
- AAX_eProperty_NativeInstanceInitProc
 - AAX_Properties.h, [1265](#)
- AAX_eProperty_NativeProcessProc
 - AAX_Properties.h, [1265](#)
- AAX_eProperty_NeedsOutputDithered
 - AAX_Properties.h, [1273](#)
- AAX_eProperty_NoID
 - AAX_Properties.h, [1261](#)
- AAX_eProperty_NumberOfInputs
 - AAX_Properties.h, [1272](#)
- AAX_eProperty_NumberOfOutputs
 - AAX_Properties.h, [1272](#)
- AAX_eProperty_ObservesTransportState
 - AAX_Properties.h, [1276](#)
- AAX_eProperty_OptionalAnalysis
 - AAX_Properties.h, [1270](#)
- AAX_eProperty_OutputStemFormat

AAX_Properties.h, [1266](#)
 AAX_eProperty_PluginID_AudioSuite
 AAX_Properties.h, [1262](#)
 AAX_eProperty_PluginID_Deprecated
 AAX_Properties.h, [1264](#)
 AAX_eProperty_PluginID_ExternalProcessor
 AAX_Properties.h, [1265](#)
 AAX_eProperty_PluginID_Native
 AAX_Properties.h, [1262](#)
 AAX_eProperty_PluginID_NoProcessing
 AAX_Properties.h, [1263](#)
 AAX_eProperty_PluginID_RTAS
 AAX_Properties.h, [1262](#)
 AAX_eProperty_PluginID_TI
 AAX_Properties.h, [1263](#)
 AAX_eProperty_PluginSpecPropsBase
 AAX_Properties.h, [1261](#)
 AAX_eProperty_ProcessProcPropsBase
 AAX_Properties.h, [1265](#)
 AAX_eProperty_ProductID
 AAX_Properties.h, [1261](#)
 AAX_eProperty_Related_DSP_Plugin_List
 AAX_Properties.h, [1264](#)
 AAX_eProperty_Related_Native_Plugin_List
 AAX_Properties.h, [1264](#)
 AAX_eProperty_RequestsAllTrackData
 AAX_Properties.h, [1271](#)
 AAX_eProperty_RequiresAnalysis
 AAX_Properties.h, [1270](#)
 AAX_eProperty_RequiresChunkCallsOnMainThread
 AAX_Properties.h, [1276](#)
 AAX_eProperty_SampleRate
 AAX_Properties.h, [1267](#)
 AAX_eProperty_ShowInMenus
 AAX_Properties.h, [1269](#)
 AAX_eProperty_SideChainStemFormat
 AAX_Properties.h, [1268](#)
 AAX_eProperty_StoreXMLPageTablesByEffect
 AAX_Properties.h, [1275](#)
 AAX_eProperty_StoreXMLPageTablesByType
 AAX_Properties.h, [1275](#)
 AAX_eProperty_SupportsSaveRestore
 AAX_Properties.h, [1275](#)
 AAX_eProperty_SupportsSideChainInput
 AAX_Properties.h, [1273](#)
 AAX_eProperty_TI_ForceAllowChipSharing
 AAX_Properties.h, [1269](#)
 AAX_eProperty_TI_InstanceCycleCount
 AAX_Properties.h, [1268](#)
 AAX_eProperty_TI_MaxInstancesPerChip
 AAX_Properties.h, [1268](#)
 AAX_eProperty_TI_SharedCycleCount
 AAX_Properties.h, [1268](#)
 AAX_eProperty_TIBackgroundProc
 AAX_Properties.h, [1266](#)
 AAX_eProperty_TIDLLFileName
 AAX_Properties.h, [1266](#)
 AAX_eProperty_TIInstanceInitProc

 AAX_Properties.h, [1266](#)
 AAX_eProperty_TIProcessProc
 AAX_Properties.h, [1266](#)
 AAX_eProperty_UsesClientGUI
 AAX_Properties.h, [1273](#)
 AAX_eProperty_UsesRandomAccess
 AAX_Properties.h, [1270](#)
 AAX_eProperty_UsesTransport
 AAX_Properties.h, [1275](#)
 AAX_eProperty_UsesTransportControl
 AAX_Properties.h, [1276](#)
 AAX_ERecordMode
 AAX_Enums.h, [1200](#)
 AAX_eRecordMode_Destructive
 AAX_Enums.h, [1201](#)
 AAX_eRecordMode_None
 AAX_Enums.h, [1201](#)
 AAX_eRecordMode_Normal
 AAX_Enums.h, [1201](#)
 AAX_eRecordMode_Num
 AAX_Enums.h, [1201](#)
 AAX_eRecordMode_QuickPunch
 AAX_Enums.h, [1201](#)
 AAX_eRecordMode_TrackPunch
 AAX_Enums.h, [1201](#)
 AAX_eRecordMode_Unknown
 AAX_Enums.h, [1201](#)
 AAX_EResourceType
 AAX_Enums.h, [1180](#)
 AAX_eResourceType_None
 AAX_Enums.h, [1181](#)
 AAX_eResourceType_PageTable
 AAX_Enums.h, [1181](#)
 AAX_eResourceType_PageTableDir
 AAX_Enums.h, [1181](#)
 AAX_ERROR_ACF_ERROR
 AAX_Errors.h, [1211](#)
 AAX_ERROR_ARGUMENT_BUFFER_OVERFLOW
 AAX_Errors.h, [1211](#)
 AAX_ERROR_ARGUMENT_OUT_OF_RANGE
 AAX_Errors.h, [1212](#)
 AAX_ERROR_CONTEXT_ALREADY_HAS_METERS
 AAX_Errors.h, [1211](#)
 AAX_ERROR_DIRECT_ACCESS_OUT_OF_BOUNDS
 AAX_Errors.h, [1211](#)
 AAX_ERROR_DUPLICATE_EFFECT_ID
 AAX_Errors.h, [1211](#)
 AAX_ERROR_DUPLICATE_TYPE_ID
 AAX_Errors.h, [1211](#)
 AAX_ERROR_EMPTY_EFFECT_NAME
 AAX_Errors.h, [1211](#)
 AAX_ERROR_FIELD_TYPE_DOES_NOT_SUPPORT_DIRECT_ACCESS
 AAX_Errors.h, [1211](#)
 AAX_ERROR_FIFO_FULL
 AAX_Errors.h, [1211](#)
 AAX_ERROR_INCORRECT_CHUNK_SIZE
 AAX_Errors.h, [1210](#)
 AAX_ERROR_INITIALIZING_PACKET_STREAM_THREAD

AAX_Errors.h, [1211](#)
AAX_ERROR_INVALID_ARGUMENT
AAX_Errors.h, [1211](#)
AAX_ERROR_INVALID_CHUNK_ID
AAX_Errors.h, [1210](#)
AAX_ERROR_INVALID_CHUNK_INDEX
AAX_Errors.h, [1210](#)
AAX_ERROR_INVALID_FIELD_INDEX
AAX_Errors.h, [1211](#)
AAX_ERROR_INVALID_INTERNAL_DATA
AAX_Errors.h, [1211](#)
AAX_ERROR_INVALID_METER_INDEX
AAX_Errors.h, [1210](#)
AAX_ERROR_INVALID_METER_TYPE
AAX_Errors.h, [1211](#)
AAX_ERROR_INVALID_PARAMETER_ID
AAX_Errors.h, [1210](#)
AAX_ERROR_INVALID_PARAMETER_INDEX
AAX_Errors.h, [1210](#)
AAX_ERROR_INVALID_PATH
AAX_Errors.h, [1211](#)
AAX_ERROR_INVALID_STRING_CONVERSION
AAX_Errors.h, [1210](#)
AAX_ERROR_INVALID_VIEW_SIZE
AAX_Errors.h, [1211](#)
AAX_ERROR_MALFORMED_CHUNK
AAX_Errors.h, [1211](#)
AAX_ERROR_MIXER_THREAD_FALLING_BEHIND
AAX_Errors.h, [1211](#)
AAX_ERROR_NO_ABBREVIATED_PARAMETER_NAME
AAX_Errors.h, [1212](#)
AAX_ERROR_NO_COMPONENTS
AAX_Errors.h, [1211](#)
AAX_ERROR_NOT_INITIALIZED
AAX_Errors.h, [1211](#)
AAX_ERROR_NOTIFICATION_FAILED
AAX_Errors.h, [1211](#)
AAX_ERROR_NULL_ARGUMENT
AAX_Errors.h, [1211](#)
AAX_ERROR_NULL_COMPONENT
AAX_Errors.h, [1211](#)
AAX_ERROR_NULL_OBJECT
AAX_Errors.h, [1210](#)
AAX_ERROR_OLDER_VERSION
AAX_Errors.h, [1210](#)
AAX_ERROR_PLUGIN_BEGIN
AAX_Errors.h, [1212](#)
AAX_ERROR_PLUGIN_END
AAX_Errors.h, [1212](#)
AAX_ERROR_PLUGIN_NOT_AUTHORIZED
AAX_Errors.h, [1211](#)
AAX_ERROR_PLUGIN_NULL_PARAMETER
AAX_Errors.h, [1211](#)
AAX_ERROR_PORT_ID_OUT_OF_RANGE
AAX_Errors.h, [1211](#)
AAX_ERROR_POST_PACKET_FAILED
AAX_Errors.h, [1211](#)
AAX_ERROR_PRINT_FAILURE
AAX_Errors.h, [1212](#)
AAX_ERROR_PROPERTY_UNDEFINED
AAX_Errors.h, [1211](#)
AAX_ERROR_SIGNED_INT_OVERFLOW
AAX_Errors.h, [1211](#)
AAX_ERROR_TOD_BEHIND
AAX_Errors.h, [1211](#)
AAX_ERROR_UNEXPECTED_EFFECT_ID
AAX_Errors.h, [1212](#)
AAX_ERROR_UNIMPLEMENTED
AAX_Errors.h, [1210](#)
AAX_ERROR_UNKNOWN_EXCEPTION
AAX_Errors.h, [1211](#)
AAX_ERROR_UNKNOWN_ID
AAX_Errors.h, [1211](#)
AAX_ERROR_UNKNOWN_PLUGIN
AAX_Errors.h, [1211](#)
AAX_ERROR_UNSUPPORTED_ENCODING
AAX_Errors.h, [1212](#)
AAX_Errors.h, [1209](#)
AAX_EError, [1210](#)
AAX_ENUM_SIZE_CHECK, [1212](#)
AAX_ERROR_ACF_ERROR, [1211](#)
AAX_ERROR_ARGUMENT_BUFFER_OVERFLOW,
[1211](#)
AAX_ERROR_ARGUMENT_OUT_OF_RANGE,
[1212](#)
AAX_ERROR_CONTEXT_ALREADY_HAS_METERS,
[1211](#)
AAX_ERROR_DIRECT_ACCESS_OUT_OF_BOUNDS,
[1211](#)
AAX_ERROR_DUPLICATE_EFFECT_ID, [1211](#)
AAX_ERROR_DUPLICATE_TYPE_ID, [1211](#)
AAX_ERROR_EMPTY_EFFECT_NAME, [1211](#)
AAX_ERROR_FIELD_TYPE_DOES_NOT_SUPPORT_DIRECT_AC
[1211](#)
AAX_ERROR_FIFO_FULL, [1211](#)
AAX_ERROR_INCORRECT_CHUNK_SIZE, [1210](#)
AAX_ERROR_INITIALIZING_PACKET_STREAM_THREAD,
[1211](#)
AAX_ERROR_INVALID_ARGUMENT, [1211](#)
AAX_ERROR_INVALID_CHUNK_ID, [1210](#)
AAX_ERROR_INVALID_CHUNK_INDEX, [1210](#)
AAX_ERROR_INVALID_FIELD_INDEX, [1211](#)
AAX_ERROR_INVALID_INTERNAL_DATA, [1211](#)
AAX_ERROR_INVALID_METER_INDEX, [1210](#)
AAX_ERROR_INVALID_METER_TYPE, [1211](#)
AAX_ERROR_INVALID_PARAMETER_ID, [1210](#)
AAX_ERROR_INVALID_PARAMETER_INDEX,
[1210](#)
AAX_ERROR_INVALID_PATH, [1211](#)
AAX_ERROR_INVALID_STRING_CONVERSION,
[1210](#)
AAX_ERROR_INVALID_VIEW_SIZE, [1211](#)
AAX_ERROR_MALFORMED_CHUNK, [1211](#)
AAX_ERROR_MIXER_THREAD_FALLING_BEHIND,
[1211](#)

AAX_ERROR_NO_ABBREVIATED_PARAMETER_NAME, [1212](#)
 AAX_ERROR_NO_COMPONENTS, [1211](#)
 AAX_ERROR_NOT_INITIALIZED, [1211](#)
 AAX_ERROR_NOTIFICATION_FAILED, [1211](#)
 AAX_ERROR_NULL_ARGUMENT, [1211](#)
 AAX_ERROR_NULL_COMPONENT, [1211](#)
 AAX_ERROR_NULL_OBJECT, [1210](#)
 AAX_ERROR_OLDER_VERSION, [1210](#)
 AAX_ERROR_PLUGIN_BEGIN, [1212](#)
 AAX_ERROR_PLUGIN_END, [1212](#)
 AAX_ERROR_PLUGIN_NOT_AUTHORIZED, [1211](#)
 AAX_ERROR_PLUGIN_NULL_PARAMETER, [1211](#)
 AAX_ERROR_PORT_ID_OUT_OF_RANGE, [1211](#)
 AAX_ERROR_POST_PACKET_FAILED, [1211](#)
 AAX_ERROR_PRINT_FAILURE, [1212](#)
 AAX_ERROR_PROPERTY_UNDEFINED, [1211](#)
 AAX_ERROR_SIGNED_INT_OVERFLOW, [1211](#)
 AAX_ERROR_TOD_BEHIND, [1211](#)
 AAX_ERROR_UNEXPECTED_EFFECT_ID, [1212](#)
 AAX_ERROR_UNIMPLEMENTED, [1210](#)
 AAX_ERROR_UNKNOWN_EXCEPTION, [1211](#)
 AAX_ERROR_UNKNOWN_ID, [1211](#)
 AAX_ERROR_UNKNOWN_PLUGIN, [1211](#)
 AAX_ERROR_UNSUPPORTED_ENCODING, [1212](#)
 AAX_RESULT_ADD_FIELD_UNSUPPORTED_FIELD, [1211](#)
 AAX_RESULT_NEW_PACKET_POSTED, [1211](#)
 AAX_RESULT_PACKET_STREAM_NOT_EMPTY, [1211](#)
 AAX_SUCCESS, [1210](#)
 AAX_eSampleRateMask
 AAX_Enums.h, [1189](#)
 AAX_eSampleRateMask_176400
 AAX_Enums.h, [1189](#)
 AAX_eSampleRateMask_192000
 AAX_Enums.h, [1189](#)
 AAX_eSampleRateMask_44100
 AAX_Enums.h, [1189](#)
 AAX_eSampleRateMask_48000
 AAX_Enums.h, [1189](#)
 AAX_eSampleRateMask_88200
 AAX_Enums.h, [1189](#)
 AAX_eSampleRateMask_96000
 AAX_Enums.h, [1189](#)
 AAX_eSampleRateMask_All
 AAX_Enums.h, [1189](#)
 AAX_eSampleRateMask_No
 AAX_Enums.h, [1189](#)
 AAX_eStemFormat
 AAX_Enums.h, [1175](#)
 AAX_eStemFormat_5_0
 AAX_Enums.h, [1176](#)
 AAX_eStemFormat_5_0_2
 AAX_Enums.h, [1176](#)
 AAX_eStemFormat_5_0_4
 AAX_Enums.h, [1176](#)
 AAX_eStemFormat_5_1
 AAX_Enums.h, [1176](#)
 AAX_eStemFormat_5_1_2
 AAX_Enums.h, [1176](#)
 AAX_eStemFormat_5_1_4
 AAX_Enums.h, [1176](#)
 AAX_eStemFormat_6_0
 AAX_Enums.h, [1176](#)
 AAX_eStemFormat_6_1
 AAX_Enums.h, [1176](#)
 AAX_eStemFormat_7_0_2
 AAX_Enums.h, [1176](#)
 AAX_eStemFormat_7_0_4
 AAX_Enums.h, [1176](#)
 AAX_eStemFormat_7_0_6
 AAX_Enums.h, [1176](#)
 AAX_eStemFormat_7_0_DTS
 AAX_Enums.h, [1176](#)
 AAX_eStemFormat_7_0_SDDS
 AAX_Enums.h, [1176](#)
 AAX_eStemFormat_7_1_2
 AAX_Enums.h, [1176](#)
 AAX_eStemFormat_7_1_4
 AAX_Enums.h, [1176](#)
 AAX_eStemFormat_7_1_6
 AAX_Enums.h, [1176](#)
 AAX_eStemFormat_7_1_DTS
 AAX_Enums.h, [1176](#)
 AAX_eStemFormat_7_1_SDDS
 AAX_Enums.h, [1176](#)
 AAX_eStemFormat_9_0_4
 AAX_Enums.h, [1176](#)
 AAX_eStemFormat_9_0_6
 AAX_Enums.h, [1176](#)
 AAX_eStemFormat_9_1_4
 AAX_Enums.h, [1176](#)
 AAX_eStemFormat_9_1_6
 AAX_Enums.h, [1176](#)
 AAX_eStemFormat_Ambi_1_ACN
 AAX_Enums.h, [1176](#)
 AAX_eStemFormat_Ambi_2_ACN
 AAX_Enums.h, [1176](#)
 AAX_eStemFormat_Ambi_3_ACN
 AAX_Enums.h, [1176](#)
 AAX_eStemFormat_Ambi_4_ACN
 AAX_Enums.h, [1176](#)
 AAX_eStemFormat_Ambi_5_ACN
 AAX_Enums.h, [1176](#)
 AAX_eStemFormat_Ambi_6_ACN
 AAX_Enums.h, [1177](#)
 AAX_eStemFormat_Ambi_7_ACN
 AAX_Enums.h, [1177](#)
 AAX_eStemFormat_Any
 AAX_Enums.h, [1177](#)
 AAX_eStemFormat_INT32_MAX
 AAX_Enums.h, [1177](#)

- AAX_eStemFormat_LCR
 - AAX_Enums.h, [1176](#)
- AAX_eStemFormat_LCRS
 - AAX_Enums.h, [1176](#)
- AAX_eStemFormat_Mono
 - AAX_Enums.h, [1176](#)
- AAX_eStemFormat_None
 - AAX_Enums.h, [1177](#)
- AAX_eStemFormat_Quad
 - AAX_Enums.h, [1176](#)
- AAX_eStemFormat_Stereo
 - AAX_Enums.h, [1176](#)
- AAX_eStemFormatNum
 - AAX_Enums.h, [1177](#)
- AAX_ESupportLevel
 - AAX_Enums.h, [1198](#)
- AAX_eSupportLevel_ByProperty
 - AAX_Enums.h, [1199](#)
- AAX_eSupportLevel_Disabled
 - AAX_Enums.h, [1199](#)
- AAX_eSupportLevel_Supported
 - AAX_Enums.h, [1199](#)
- AAX_eSupportLevel_Uninitialized
 - AAX_Enums.h, [1199](#)
- AAX_eSupportLevel_Unsupported
 - AAX_Enums.h, [1199](#)
- AAX_ETargetPlatform
 - AAX_Enums.h, [1198](#)
- AAX_ETextEncoding
 - AAX_Enums.h, [1199](#)
- AAX_eTextEncoding_Num
 - AAX_Enums.h, [1200](#)
- AAX_eTextEncoding_Undefined
 - AAX_Enums.h, [1200](#)
- AAX_eTextEncoding_UTF8
 - AAX_Enums.h, [1200](#)
- AAX_ETracePriority
 - AAX_Assert.h, [1125](#)
- AAX_ETracePriorityDSP
 - AAX_Enums.h, [1173](#)
- AAX_eTracePriorityDSP_Assert
 - AAX_Enums.h, [1173](#)
- AAX_eTracePriorityDSP_High
 - AAX_Enums.h, [1173](#)
- AAX_eTracePriorityDSP_Low
 - AAX_Enums.h, [1173](#)
- AAX_eTracePriorityDSP_None
 - AAX_Enums.h, [1173](#)
- AAX_eTracePriorityDSP_Normal
 - AAX_Enums.h, [1173](#)
- AAX_ETracePriorityHost
 - AAX_Enums.h, [1172](#)
- AAX_eTracePriorityHost_Critical
 - AAX_Enums.h, [1172](#)
- AAX_eTracePriorityHost_High
 - AAX_Enums.h, [1172](#)
- AAX_eTracePriorityHost_Low
 - AAX_Enums.h, [1172](#)
- AAX_eTracePriorityHost_Lowest
 - AAX_Enums.h, [1172](#)
- AAX_eTracePriorityHost_None
 - AAX_Enums.h, [1172](#)
- AAX_eTracePriorityHost_Normal
 - AAX_Enums.h, [1172](#)
- AAX_ETransportState
 - AAX_Enums.h, [1200](#)
- AAX_eTransportState_FastForward
 - AAX_Enums.h, [1200](#)
- AAX_eTransportState_Num
 - AAX_Enums.h, [1200](#)
- AAX_eTransportState_Paused
 - AAX_Enums.h, [1200](#)
- AAX_eTransportState_Play
 - AAX_Enums.h, [1200](#)
- AAX_eTransportState_Rewind
 - AAX_Enums.h, [1200](#)
- AAX_eTransportState_Scrub
 - AAX_Enums.h, [1200](#)
- AAX_eTransportState_Shuttle
 - AAX_Enums.h, [1200](#)
- AAX_eTransportState_Stop
 - AAX_Enums.h, [1200](#)
- AAX_eTransportState_Stopping
 - AAX_Enums.h, [1200](#)
- AAX_eTransportState_Unknown
 - AAX_Enums.h, [1200](#)
- AAX_EUpdateSource
 - AAX_Enums.h, [1194](#)
- AAX_eUpdateSource_Chunk
 - AAX_Enums.h, [1194](#)
- AAX_eUpdateSource_Delay
 - AAX_Enums.h, [1194](#)
- AAX_eUpdateSource_Parameter
 - AAX_Enums.h, [1194](#)
- AAX_eUpdateSource_Unspecified
 - AAX_Enums.h, [1194](#)
- AAX_EUseAlternateControl
 - AAX_Enums.h, [1192](#)
- AAX_eUseAlternateControl_No
 - AAX_Enums.h, [1192](#)
- AAX_eUseAlternateControl_Yes
 - AAX_Enums.h, [1192](#)
- AAX_EViewContainer_Type
 - AAX_GUITypes.h, [1221](#)
- AAX_eViewContainer_Type_HWND
 - AAX_GUITypes.h, [1221](#)
- AAX_eViewContainer_Type_NSView
 - AAX_GUITypes.h, [1221](#)
- AAX_eViewContainer_Type_NULL
 - AAX_GUITypes.h, [1221](#)
- AAX_eViewContainer_Type_UIView
 - AAX_GUITypes.h, [1221](#)
- AAX_Exception.h, [1212](#)
 - AAX_CAPTURE, [1214](#)
 - AAX_CAPTURE_MULT, [1214](#)
 - AAX_SWALLOW, [1213](#)

- AAX_SWALLOW_MULT, 1213
- AAX_EXPORT
 - AAX_Exports.cpp, 1216
- AAX_Exports.cpp, 1215
 - AAX_EXPORT, 1216
 - ACFCanUnloadNow, 1217
 - ACFGetClassFactory, 1216
 - ACFGetSDKVersion, 1217
 - ACFRegisterComponent, 1216
 - ACFRegisterPlugin, 1216
 - ACFShutdown, 1217
 - ACFStartup, 1217
- AAX_FastInterpolatedTableLookup< TFloat, DFloat>, 667
 - DoTableLookupExtraFast, 668
 - DoTableLookupExtraFastMulti, 668
 - GetMaxMinusMin, 669
 - GetMin, 668
 - SetParameters, 667
- AAX_FastInterpolatedTableLookup.h, 1218
 - AAX_FASTINTERPOLATEDTABLELOOKUP_H, 1218
- AAX_FastInterpolatedTableLookup.hpp, 1218
- AAX_FASTINTERPOLATEDTABLELOOKUP_H
 - AAX_FastInterpolatedTableLookup.h, 1218
- AAX_FastPow.h, 1219
 - _AAX_FASTPOW_H_, 1219
- AAX_Feature_UID
 - AAX.h, 1115
 - AAX_UIDs.h, 1295
- AAX_FIELD_INDEX
 - AAX.h, 1111
- AAX_FINAL
 - AAX.h, 1107
- AAX_Getting_Started_Guide.doxygen, 1219
- AAX_GUITypes.h, 1219
 - AAX_ENUM_SIZE_CHECK, 1223
 - AAX_EViewContainer_Type, 1221
 - AAX_eViewContainer_Type_HWND, 1221
 - AAX_eViewContainer_Type_NSView, 1221
 - AAX_eViewContainer_Type_NULL, 1221
 - AAX_eViewContainer_Type_UIView, 1221
 - AAX_Point, 1220
 - AAX_Rect, 1221
 - operator!=, 1222, 1223
 - operator<, 1222
 - operator<=, 1222
 - operator>, 1222
 - operator>=, 1222
 - operator==, 1222, 1223
- AAX_HI
 - AAX_MiscUtils.h, 1255
- AAX_HostSupport.doxygen, 1223
- AAX_IACFAutomationDelegate, 669
 - GetTouchState, 671
 - PostCurrentValue, 671
 - PostReleaseRequest, 671
 - PostSetValueRequest, 670
- PostTouchRequest, 671
- RegisterParameter, 669
- UnregisterParameter, 670
- AAX_IACFAutomationDelegate.h, 1223
- AAX_IACFCollection, 672
 - AddEffect, 672
 - AddPackageName, 673
 - SetManufacturerName, 673
 - SetPackageVersion, 674
 - SetProperties, 674
- AAX_IACFCollection.h, 1224
- AAX_IACFComponentDescriptor, 674
 - AddAudioBufferLength, 677
 - AddAudioIn, 676
 - AddAudioOut, 676
 - AddAuxOutputStem, 679
 - AddClock, 678
 - AddDataInPort, 678
 - AddDmaInstance, 680
 - AddMeters, 683
 - AddMIDINode, 681
 - AddPrivateData, 680
 - AddProcessProc_Native, 681
 - AddProcessProc_TI, 682
 - AddReservedField, 676
 - AddSampleRate, 677
 - AddSideChainIn, 678
 - Clear, 675
- AAX_IACFComponentDescriptor.h, 1224
- AAX_IACFComponentDescriptor_V2, 683
 - AddTemporaryData, 684
- AAX_IACFComponentDescriptor_V3, 684
 - AddProcessProc, 685
- AAX_IACFController, 686
 - ClearMeterClipped, 693
 - ClearMeterPeakValue, 692
 - GetCurrentMeterValue, 692
 - GetCycleCount, 689
 - GetEffectID, 687
 - GetInputStemFormat, 688
 - GetMeterClipped, 693
 - GetMeterCount, 693
 - GetMeterPeakValue, 692
 - GetNextMIDIpacket, 693
 - GetOutputStemFormat, 688
 - GetSampleRate, 687
 - GetSignalLatency, 688
 - GetTODLocation, 689
 - PostPacket, 691
 - SetCycleCount, 690
 - SetSignalLatency, 690
- AAX_IACFController.h, 1224
- AAX_IACFController_V2, 694
 - GetCurrentAutomationTimestamp, 695
 - GetHostName, 696
 - GetHybridSignalLatency, 695
 - SendNotification, 694
- AAX_IACFController_V3, 696

- GetIsAudioSuite, 697
- GetPluginTargetPlatform, 697
- AAX_IACFDataBuffer, 698
 - Data, 698
 - Size, 698
 - Type, 698
- AAX_IACFDataBuffer.h, 1225
 - AAX_IACFDataBuffer_H, 1225
- AAX_IACFDataBuffer_H
 - AAX_IACFDataBuffer.h, 1225
- AAX_IACFDescriptionHost, 699
 - AcquireFeatureProperties, 699
- AAX_IACFDescriptionHost.h, 1225
- AAX_IACFEffectDescriptor, 700
 - AddCategory, 701
 - AddCategoryBypassParameter, 701
 - AddComponent, 700
 - AddMeterDescription, 703
 - AddName, 701
 - AddProcPtr, 702
 - AddResourceInfo, 702
 - SetProperties, 702
- AAX_IACFEffectDescriptor.h, 1226
- AAX_IACFEffectDescriptor_V2, 703
 - AddControlMIDINode, 703
- AAX_IACFEffectDirectData, 704
 - Initialize, 705
 - TimerWakeup, 705
 - Uninitialize, 705
- AAX_IACFEffectDirectData.h, 1226
- AAX_IACFEffectDirectData_V2, 706
 - NotificationReceived, 706
- AAX_IACFEffectGUI, 707
 - Draw, 711
 - GetCustomLabel, 712
 - GetViewSize, 710
 - Initialize, 709
 - NotificationReceived, 709
 - ParameterUpdated, 711
 - SetControlHighlightInfo, 712
 - SetViewContainer, 710
 - TimerWakeup, 711
 - Uninitialize, 709
- AAX_IACFEffectGUI.h, 1226
- AAX_IACFEffectParameters, 713
 - CompareActiveChunk, 733
 - DoMIDITransfers, 735
 - GenerateCoefficients, 730
 - GetChunk, 732
 - GetChunkIDFromIndex, 731
 - GetChunkSize, 731
 - GetCustomData, 734
 - GetMasterBypassParameter, 718
 - GetNumberOfChunks, 731
 - GetNumberOfParameters, 718
 - GetParameter, 722
 - GetParameterDefaultNormalizedValue, 720
 - GetParameterIDFromIndex, 723
 - GetParameterIndex, 723
 - GetParameterIsAutomatable, 719
 - GetParameterName, 720
 - GetParameterNameOfLength, 720
 - GetParameterNormalizedValue, 726
 - GetParameterNumberOfSteps, 719
 - GetParameterOrientation, 722
 - GetParameterStringFromValue, 725
 - GetParameterType, 721
 - GetParameterValueFromString, 724
 - GetParameterValueInfo, 724
 - GetParameterValueString, 725
 - Initialize, 717
 - NotificationReceived, 717
 - ReleaseParameter, 728
 - ResetFieldData, 730
 - SetChunk, 733
 - SetCustomData, 734
 - SetParameterDefaultNormalizedValue, 721
 - SetParameterNormalizedRelative, 726
 - SetParameterNormalizedValue, 726
 - TimerWakeup, 733
 - TouchParameter, 727
 - Uninitialize, 717
 - UpdateParameterNormalizedRelative, 729
 - UpdateParameterNormalizedValue, 729
 - UpdateParameterTouch, 728
- AAX_IACFEffectParameters.h, 1227
- AAX_IACFEffectParameters_V2, 735
 - UpdateControlMIDINodes, 736
 - UpdateMIDINodes, 736
- AAX_IACFEffectParameters_V3, 737
- AAX_IACFEffectParameters_V4, 738
 - UpdatePageTable, 738
- AAX_IACFFeatureInfo, 739
 - AcquireProperties, 740
 - SupportLevel, 740
- AAX_IACFFeatureInfo.h, 1227
- AAX_IACFHostProcessor, 741
 - AnalyzeAudio, 745
 - Initialize, 742
 - InitOutputBounds, 742
 - PostAnalyze, 746
 - PostRender, 744
 - PreAnalyze, 745
 - PreRender, 744
 - RenderAudio, 743
 - SetLocation, 743
 - Uninitialize, 742
- AAX_IACFHostProcessor.h, 1228
- AAX_IACFHostProcessor_V2, 746
 - GetClipNameSuffix, 747
- AAX_IACFHostProcessorDelegate, 747
 - GetAudio, 748
 - GetSideChainInputNum, 749
- AAX_IACFHostProcessorDelegate.h, 1228
- AAX_IACFHostProcessorDelegate_V2, 749
 - ForceAnalyze, 749

- AAX_IACFHostProcessorDelegate_V3, 750
 - ForceProcess, 750
- AAX_IACFHostServices, 750
 - Assert, 751
 - Trace, 751
- AAX_IACFHostServices.h, 1228
- AAX_IACFHostServices_V2, 752
 - StackTrace, 752
- AAX_IACFHostServices_V3, 753
 - HandleAssertFailure, 753
- AAX_IACFPageTable, 754
 - Clear, 754
 - ClearMappedParameter, 756
 - Empty, 754
 - GetMappedParameterID, 757
 - GetNumMappedParameterIDs, 756
 - GetNumPages, 755
 - InsertPage, 755
 - MapParameterID, 757
 - RemovePage, 755
- AAX_IACFPageTable.h, 1229
- AAX_IACFPageTable_V2, 758
 - ClearNameVariationsForParameter, 762
 - ClearParameterNameVariations, 762
 - GetNameVariationParameterIDAtIndex, 759
 - GetNumNameVariationsForParameter, 759
 - GetNumParametersWithNameVariations, 758
 - GetParameterNameVariationAtIndex, 760
 - GetParameterNameVariationOfLength, 761
 - SetParameterNameVariation, 763
- AAX_IACFPageTableController, 763
 - CopyTableForEffect, 764
 - CopyTableOfLayoutForEffect, 765
- AAX_IACFPageTableController.h, 1229
- AAX_IACFPageTableController_V2, 766
 - CopyTableForEffectFromFile, 766
 - CopyTableOfLayoutFromFile, 767
- AAX_IACFPrivateDataAccess, 768
 - ReadPortDirect, 768
 - WritePortDirect, 769
- AAX_IACFPrivateDataAccess.h, 1229
- AAX_IACFPropertyMap, 769
 - AddProperty, 770
 - GetProperty, 770
 - RemoveProperty, 771
- AAX_IACFPropertyMap.h, 1230
- AAX_IACFPropertyMap_V2, 771
 - AddPropertyWithIDArray, 771
 - GetPropertyWithIDArray, 772
- AAX_IACFPropertyMap_V3, 772
 - AddProperty64, 773
 - GetProperty64, 773
- AAX_IACFSessionDocument, 773
 - GetDocumentData, 774
- AAX_IACFSessionDocument.h, 1230
 - AAX_IACFSessionDocument_H, 1230
- AAX_IACFSessionDocument_H
 - AAX_IACFSessionDocument.h, 1230
- AAX_IACFSessionDocumentClient, 775
 - Initialize, 775
 - NotificationReceived, 776
 - SetSessionDocument, 776
 - Uninitialize, 775
- AAX_IACFSessionDocumentClient.h, 1231
 - AAX_IACFSessionDocumentClient_H, 1231
- AAX_IACFSessionDocumentClient_H
 - AAX_IACFSessionDocumentClient.h, 1231
- AAX_IACFTask, 777
 - AddResult, 779
 - GetArgumentOfType, 778
 - GetProgress, 778
 - GetType, 777
 - SetDone, 779
 - SetProgress, 778
- AAX_IACFTask.h, 1231
 - AAX_IACFTask_H, 1232
- AAX_IACFTask_H
 - AAX_IACFTask.h, 1232
- AAX_IACFTaskAgent, 780
 - AddTask, 781
 - CancelAllTasks, 781
 - Initialize, 780
 - Uninitialize, 781
- AAX_IACFTaskAgent.h, 1232
 - AAX_IACFTaskAgent_H, 1232
- AAX_IACFTaskAgent_H
 - AAX_IACFTaskAgent.h, 1232
- AAX_IACFTTransport, 781
 - GetBarBeatPosition, 785
 - GetCurrentLoopPosition, 784
 - GetCurrentMeter, 783
 - GetCurrentNativeSampleLocation, 784
 - GetCurrentTempo, 782
 - GetCurrentTickPosition, 783
 - GetCurrentTicksPerBeat, 786
 - GetCustomTickPosition, 785
 - GetTicksPerQuarter, 785
 - IsTransportPlaying, 783
- AAX_IACFTTransport.h, 1232
- AAX_IACFTTransport_V2, 786
 - GetFeetFramesInfo, 787
 - GetTimeCodeInfo, 787
 - GetTimelineSelectionStartPosition, 787
 - IsMetronomeEnabled, 788
- AAX_IACFTTransport_V3, 788
 - GetHDTTimeCodeInfo, 789
- AAX_IACFTTransport_V4, 789
 - GetTimelineSelectionEndPosition, 789
- AAX_IACFTTransport_V5, 790
 - GetKeySignature, 790
- AAX_IACFTTransportControl, 791
 - RequestTransportStart, 792
 - RequestTransportStop, 792
- AAX_IACFTTransportControl.h, 1233
- AAX_IACFViewContainer, 792
 - GetModifiers, 793

- GetPtr, [793](#)
- GetType, [793](#)
- HandleParameterMouseDown, [794](#)
- HandleParameterMouseDrag, [794](#)
- HandleParameterMouseUp, [795](#)
- SetViewSize, [794](#)
- AAX_IACFViewContainer.h, [1233](#)
- AAX_IACFViewContainer_V2, [795](#)
 - HandleMultipleParametersMouseDown, [796](#)
 - HandleMultipleParametersMouseDrag, [796](#)
 - HandleMultipleParametersMouseUp, [797](#)
- AAX_IACFViewContainer_V3, [797](#)
 - HandleParameterMouseEnter, [798](#)
 - HandleParameterMouseExit, [798](#)
- AAX_IAutomationDelegate, [799](#)
 - ~AAX_IAutomationDelegate, [799](#)
 - GetTouchState, [802](#)
 - ParameterNameChanged, [802](#)
 - PostCurrentValue, [801](#)
 - PostReleaseRequest, [801](#)
 - PostSetValueRequest, [800](#)
 - PostTouchRequest, [801](#)
 - RegisterParameter, [799](#)
 - UnregisterParameter, [800](#)
- AAX_IAutomationDelegate.h, [1234](#)
- AAX_ICollection, [803](#)
 - ~AAX_ICollection, [804](#)
 - AddEffect, [804](#)
 - AddPackageName, [805](#)
 - DescriptionHost, [807](#)
 - GetHostVersion, [806](#)
 - HostDefinition, [807](#)
 - NewDescriptor, [804](#)
 - NewPropertyMap, [806](#)
 - SetManufacturerName, [804](#)
 - SetPackageVersion, [805](#)
 - SetProperties, [806](#)
- AAX_ICollection.h, [1234](#)
- AAX_IComponentDescriptor, [808](#)
 - ~AAX_IComponentDescriptor, [810](#)
 - AddAudioBufferLength, [811](#)
 - AddAudioIn, [810](#)
 - AddAudioOut, [811](#)
 - AddAuxOutputStem, [814](#)
 - AddClock, [812](#)
 - AddDataInPort, [813](#)
 - AddDmaInstance, [815](#)
 - AddMeters, [816](#)
 - AddMIDINode, [817](#)
 - AddPrivateData, [814](#)
 - AddProcessProc, [821](#)
 - AddProcessProc_Native, [818, 822](#)
 - AddProcessProc_TI, [820](#)
 - AddReservedField, [817](#)
 - AddSampleRate, [812](#)
 - AddSideChainIn, [813](#)
 - AddTemporaryData, [815](#)
 - Clear, [810](#)
 - DuplicatePropertyMap, [818](#)
 - NewPropertyMap, [818](#)
- AAX_IComponentDescriptor.h, [1234](#)
- AAX_IContainer, [823](#)
 - ~AAX_IContainer, [823](#)
 - Clear, [824](#)
 - EStatus, [823](#)
 - eStatus_NotInitialized, [823](#)
 - eStatus_Overflow, [823](#)
 - eStatus_Success, [823](#)
 - eStatus_Unavailable, [823](#)
 - eStatus_Unsupported, [823](#)
- AAX_IContainer.h, [1235](#)
- AAX_IController, [824](#)
 - ~AAX_IController, [826](#)
 - ClearMeterClipped, [834](#)
 - ClearMeterPeakValue, [833](#)
 - CreateTableCopyForEffect, [836](#)
 - CreateTableCopyForEffectFromFile, [839](#)
 - CreateTableCopyForLayout, [837](#)
 - CreateTableCopyForLayoutFromFile, [839](#)
 - GetCurrentAutomationTimestamp, [835](#)
 - GetCurrentMeterValue, [833](#)
 - GetCycleCount, [828](#)
 - GetEffectID, [827](#)
 - GetHostName, [835](#)
 - GetInputStemFormat, [827](#)
 - GetIsAudioSuite, [836](#)
 - GetMeterClipped, [834](#)
 - GetMeterCount, [834](#)
 - GetMeterPeakValue, [833](#)
 - GetNextMIDIPacket, [835](#)
 - GetOutputStemFormat, [827](#)
 - GetPlugInTargetPlatform, [836](#)
 - GetSampleRate, [827](#)
 - GetSignalLatency, [828](#)
 - GetTODLocation, [829](#)
 - PostPacket, [831](#)
 - SendNotification, [832](#)
 - SetCycleCount, [830](#)
 - SetSignalLatency, [829](#)
- AAX_IController.h, [1235](#)
- AAX_IDataBuffer, [840](#)
 - AAX_DELETE, [841](#)
 - AAX_OVERRIDE, [841](#)
 - ACF_DECLARE_STANDARD_UNKNOWN, [841](#)
- AAX_IDataBuffer.h, [1235](#)
 - AAX_IDataBuffer_H, [1236](#)
- AAX_IDataBuffer_H
 - AAX_IDataBuffer.h, [1236](#)
- AAX_IDataBufferWrapper, [841](#)
 - ~AAX_IDataBufferWrapper, [842](#)
 - Data, [842](#)
 - Size, [842](#)
 - Type, [842](#)
- AAX_IDataBufferWrapper.h, [1236](#)
 - AAX_IDATABUFFERWRAPPER_H, [1236](#)
- AAX_IDATABUFFERWRAPPER_H

- AAX_IDataBufferWrapper.h, 1236
- AAX_IDescriptionHost, 843
 - ~AAX_IDescriptionHost, 843
 - AcquireFeatureProperties, 843
- AAX_IDescriptionHost.h, 1236
- AAX_IDisplayDelegate< T >, 844
 - Clone, 846
 - StringToValue, 848
 - ValueToString, 846, 847
- AAX_IDisplayDelegate.h, 1237
- AAX_IDisplayDelegateBase, 848
 - ~AAX_IDisplayDelegateBase, 849
- AAX_IDisplayDelegateDecorator
 - AAX_IDisplayDelegateDecorator< T >, 850
- AAX_IDisplayDelegateDecorator< T >, 849
 - ~AAX_IDisplayDelegateDecorator, 851
 - AAX_IDisplayDelegateDecorator, 850
 - Clone, 851
 - StringToValue, 853
 - ValueToString, 851, 852
- AAX_IDisplayDelegateDecorator.h, 1237
- AAX_IDma, 853
 - ~AAX_IDma, 856
 - EMode, 856
 - eMode_Burst, 856
 - eMode_Error, 856
 - eMode_Gather, 856
 - eMode_Scatter, 856
 - EState, 855
 - eState_Complete, 856
 - eState_Error, 856
 - eState_Init, 856
 - eState_Pending, 856
 - eState_Running, 856
 - GetBaseOffset, 863
 - GetBurstLength, 859
 - GetDmaMode, 858
 - GetDmaState, 857
 - GetDst, 859
 - GetFifoBuffer, 861
 - GetFifoSize, 863
 - GetLinearBuffer, 861
 - GetNumBursts, 860
 - GetNumOffsets, 862
 - GetOffsetTable, 862
 - GetSrc, 858
 - GetTransferSize, 860
 - IsTransferComplete, 857
 - PostRequest, 856
 - SetBaseOffset, 863
 - SetBurstLength, 859
 - SetDmaState, 857
 - SetDst, 858
 - SetFifoBuffer, 860
 - SetFifoSize, 863
 - SetLinearBuffer, 861
 - SetNumBursts, 859
 - SetNumOffsets, 862
 - SetOffsetTable, 861
 - SetSrc, 858
 - SetTransferSize, 860
- AAX_IDma.h, 1237
 - AAX_DMA_API, 1238
 - AAX_IDMA_H, 1238
- AAX_IDMA_H
 - AAX_IDma.h, 1238
- AAX_IEffectDescriptor, 864
 - ~AAX_IEffectDescriptor, 865
 - AddCategory, 866
 - AddCategoryBypassParameter, 866
 - AddComponent, 865
 - AddControlMIDINode, 868
 - AddMeterDescription, 868
 - AddName, 865
 - AddProcPtr, 867
 - AddResourceInfo, 867
 - NewComponentDescriptor, 865
 - NewPropertyMap, 867
 - SetProperties, 867
- AAX_IEffectDescriptor.h, 1238
- AAX_IEffectDirectData, 869
 - AAX_DELETE, 870
 - ACF_DECLARE_STANDARD_UNKNOWN, 870
 - override, 870
- AAX_IEffectDirectData.h, 1238
- AAX_IEffectGUI, 870
 - AAX_DELETE, 871
 - ACF_DECLARE_STANDARD_UNKNOWN, 871
 - override, 871
- AAX_IEffectGUI.h, 1239
- AAX_IEffectParameters, 872
 - AAX_DELETE, 874
 - ACF_DECLARE_STANDARD_UNKNOWN, 873
 - override, 874
- AAX_IEffectParameters.h, 1239
- AAX_IFeatureInfo, 874
 - ~AAX_IFeatureInfo, 874
 - AcquireProperties, 875
 - ID, 875
 - SupportLevel, 875
- AAX_IFeatureInfo.h, 1240
- AAX_IHostProcessor, 876
 - AAX_DELETE, 877
 - ACF_DECLARE_STANDARD_UNKNOWN, 876
 - override, 877
- AAX_IHostProcessor.h, 1240
- AAX_IHostProcessorDelegate, 877
 - ~AAX_IHostProcessorDelegate, 878
 - ForceAnalyze, 879
 - ForceProcess, 879
 - GetAudio, 878
 - GetSideChainInputNum, 879
- AAX_IHostProcessorDelegate.h, 1240
- AAX_IHostServices, 880
 - ~AAX_IHostServices, 880
 - HandleAssertFailure, 881

- StackTrace, 882
- Trace, 881
- AAX_IHostServices.h, 1240
- AAX_IHostTaskAgent, 882
 - ~AAX_IHostTaskAgent, 883
 - AddTask, 883
 - CancelAllTasks, 883
 - Initialize, 883
 - Uninitialize, 883
- AAX_IHostTaskAgent.h, 1241
 - AAX_IHostTaskAgent_H, 1241
- AAX_IHostTaskAgent_H
 - AAX_IHostTaskAgent.h, 1241
- AAX_IMIDIMessageInfoDelegate, 884
 - ~AAX_IMIDIMessageInfoDelegate, 884
 - Accepts, 885
 - Accepts_ExactStatus, 885
 - Length, 885
 - Mask, 884
 - ToString, 885
 - ToString_AppendByteRange, 886
 - ToString_AppendCStr, 886
 - ToString_AppendNumber, 885
 - ToString_AppendValid, 886
- AAX_IMIDIINode, 887
 - ~AAX_IMIDIINode, 887
 - GetNodeBuffer, 887
 - GetTransport, 888
 - PostMIDIIPacket, 887
- AAX_IMIDIINode.h, 1241
- AAX_Init.h, 1242
 - AAXCanUnloadNow, 1243
 - AAXGetClassFactory, 1243
 - AAXGetSDKVersion, 1244
 - AAXRegisterComponent, 1242
 - AAXShutdown, 1244
 - AAXStartup, 1243
- AAX_InstrumentParameters.doxygen, 1245
- AAX_INT16_MAX
 - AAX_Enums.h, 1170
- AAX_INT16_MIN
 - AAX_Enums.h, 1170
- AAX_INT32_MAX
 - AAX_Enums.h, 1170
- AAX_INT32_MIN
 - AAX_Enums.h, 1170
- AAX_INT_HI
 - AAX_MiscUtils.h, 1255
- AAX_INT_LO
 - AAX_MiscUtils.h, 1255
- AAX_InterfaceList.doxygen, 1245
- AAX_IPacketHandler, 888
 - ~AAX_IPacketHandler, 889
 - Call, 889
 - Clone, 889
- AAX_IPageTable, 890
 - ~AAX_IPageTable, 891
 - Clear, 891
 - ClearMappedParameter, 893
 - ClearNameVariationsForParameter, 898
 - ClearParameterNameVariations, 898
 - Empty, 891
 - GetMappedParameterID, 894
 - GetNameVariationParameterIDAtIndex, 895
 - GetNumMappedParameterIDs, 893
 - GetNumNameVariationsForParameter, 896
 - GetNumPages, 891
 - GetNumParametersWithNameVariations, 895
 - GetParameterNameVariationAtIndex, 896
 - GetParameterNameVariationOfLength, 897
 - InsertPage, 892
 - MapParameterID, 894
 - RemovePage, 892
 - SetParameterNameVariation, 899
- AAX_IPageTable.h, 1245
- AAX_IParameter, 900
 - ~AAX_IParameter, 902
 - AddShortenedName, 904
 - Automatable, 905
 - ClearShortenedNames, 904
 - CloneValue, 903
 - GetBoolFromNormalizedValue, 911
 - GetDoubleFromNormalizedValue, 913
 - GetFloatFromNormalizedValue, 912
 - GetInt32FromNormalizedValue, 912
 - GetNormalizedDefaultValue, 906
 - GetNormalizedValue, 906
 - GetNormalizedValueFromBool, 909
 - GetNormalizedValueFromDouble, 911
 - GetNormalizedValueFromFloat, 910
 - GetNormalizedValueFromInt32, 910
 - GetNormalizedValueFromStep, 908
 - GetNormalizedValueFromString, 911
 - GetNumberOfSteps, 907
 - GetOrientation, 920
 - GetStepValue, 907
 - GetStepValueFromNormalizedValue, 908
 - GetStringFromNormalizedValue, 913, 914
 - GetType, 919
 - GetValueAsBool, 915
 - GetValueAsDouble, 916
 - GetValueAsFloat, 915
 - GetValueAsInt32, 915
 - GetValueAsString, 916
 - GetValueString, 908, 909
 - Identifier, 903
 - Name, 904
 - Release, 905
 - SetAutomationDelegate, 905
 - SetDisplayDelegate, 920
 - SetName, 903
 - SetNormalizedDefaultValue, 906
 - SetNormalizedValue, 906
 - SetNumberOfSteps, 907
 - SetOrientation, 919
 - SetStepValue, 908

- SetTaperDelegate, [920](#)
- SetToDefaultValue, [907](#)
- SetType, [919](#)
- SetValueFromString, [914](#)
- SetValueWithBool, [917](#)
- SetValueWithDouble, [918](#)
- SetValueWithFloat, [918](#)
- SetValueWithInt32, [917](#)
- SetValueWithString, [918](#)
- ShortenedName, [904](#)
- Touch, [905](#)
- UpdateNormalizedValue, [921](#)
- AAX_IParameter.h, [1245](#)
- AAX_IParameterValue, [921](#)
 - ~AAX_IParameterValue, [922](#)
 - Clone, [922](#)
 - GetValueAsBool, [923](#)
 - GetValueAsDouble, [924](#)
 - GetValueAsFloat, [924](#)
 - GetValueAsInt32, [923](#)
 - GetValueAsString, [924](#)
 - Identifier, [922](#)
- AAX_IPointerQueue< T >, [925](#)
 - ~AAX_IPointerQueue, [926](#)
 - Clear, [926](#)
 - Peek, [927](#)
 - Pop, [927](#)
 - Push, [926](#)
 - template_type, [926](#)
 - value_type, [926](#)
- AAX_IPointerQueue.h, [1245](#)
- AAX_IPrivateDataAccess, [928](#)
 - ~AAX_IPrivateDataAccess, [928](#)
 - ReadPortDirect, [928](#)
 - WritePortDirect, [929](#)
- AAX_IPrivateDataAccess.h, [1246](#)
- AAX_IPropertyMap, [929](#)
 - ~AAX_IPropertyMap, [930](#)
 - AddPointerProperty, [932](#)
 - AddProperty, [931](#)
 - AddPropertyWithIDArray, [933](#)
 - GetUnknown, [934](#)
 - GetPointerProperty, [931](#)
 - GetProperty, [931](#)
 - GetPropertyWithIDArray, [933](#)
 - RemoveProperty, [933](#)
- AAX_IPropertyMap.h, [1246](#)
- AAX_ISessionDocument, [934](#)
 - ~AAX_ISessionDocument, [935](#)
 - GetDocumentData, [935](#)
 - GetTempoMap, [935](#)
 - Valid, [935](#)
- AAX_ISessionDocument.h, [1246](#)
 - AAX_ISessionDocument_H, [1247](#)
- AAX_ISessionDocument::TempoMap, [1099](#)
 - ~TempoMap, [1099](#)
 - Data, [1100](#)
 - Size, [1100](#)
- AAX_ISessionDocument_H
 - AAX_ISessionDocument.h, [1247](#)
- AAX_ISessionDocumentClient, [936](#)
 - AAX_DELETE, [937](#)
 - ACF_DECLARE_STANDARD_UNKNOWN, [937](#)
 - override, [937](#)
- AAX_ISessionDocumentClient.h, [1247](#)
 - AAX_ISessionDocumentClient_H, [1247](#)
- AAX_ISessionDocumentClient_H
 - AAX_ISessionDocumentClient.h, [1247](#)
- AAX_IString, [937](#)
 - ~AAX_IString, [938](#)
 - Get, [938](#)
 - Length, [938](#)
 - MaxLength, [938](#)
 - operator=, [939](#)
 - Set, [939](#)
- AAX_IString.h, [1248](#)
- AAX_ITaperDelegate< T >, [939](#)
 - Clone, [940](#)
 - ConstrainRealValue, [941](#)
 - GetMaximumValue, [940](#)
 - GetMinimumValue, [941](#)
 - NormalizedToReal, [941](#)
 - RealToNormalized, [942](#)
- AAX_ITaperDelegate.h, [1248](#)
- AAX_ITaperDelegateBase, [942](#)
 - ~AAX_ITaperDelegateBase, [943](#)
- AAX_ITask, [944](#)
 - ~AAX_ITask, [945](#)
 - AddResult, [946](#)
 - GetArgumentOfType, [945](#)
 - GetProgress, [946](#)
 - GetType, [945](#)
 - SetDone, [946](#)
 - SetProgress, [945](#)
- AAX_ITask.h, [1248](#)
 - AAX_ITask_H, [1249](#)
- AAX_ITask_H
 - AAX_ITask.h, [1249](#)
- AAX_ITaskAgent, [947](#)
 - AAX_DELETE, [948](#)
 - AAX_OVERRIDE, [948](#)
 - ACF_DECLARE_STANDARD_UNKNOWN, [948](#)
- AAX_ITaskAgent.h, [1249](#)
- AAX_ITransport, [948](#)
 - ~AAX_ITransport, [949](#)
 - GetBarBeatPosition, [953](#)
 - GetCurrentLoopPosition, [951](#)
 - GetCurrentMeter, [950](#)
 - GetCurrentNativeSampleLocation, [952](#)
 - GetCurrentTempo, [950](#)
 - GetCurrentTickPosition, [951](#)
 - GetCurrentTicksPerBeat, [954](#)
 - GetCustomTickPosition, [952](#)
 - GetFeetFramesInfo, [955](#)
 - GetHDTIME_CODE_INFO, [955](#)
 - GetKeySignature, [957](#)

- GetTicksPerQuarter, [953](#)
- GetTimeCodeInfo, [954](#)
- GetTimelineSelectionEndPosition, [956](#)
- GetTimelineSelectionStartPosition, [954](#)
- IsMetronomeEnabled, [955](#)
- IsTransportPlaying, [951](#)
- RequestTransportStart, [956](#)
- RequestTransportStop, [956](#)
- AAX_ITransport.h, [1249](#)
- AAX_IViewContainer, [957](#)
 - ~AAX_IViewContainer, [959](#)
 - GetModifiers, [959](#)
 - GetPtr, [959](#)
 - GetType, [959](#)
 - HandleMultipleParametersMouseDown, [962](#)
 - HandleMultipleParametersMouseDrag, [962](#)
 - HandleMultipleParametersMouseUp, [963](#)
 - HandleParameterMouseDown, [960](#)
 - HandleParameterMouseDrag, [960](#)
 - HandleParameterMouseEnter, [961](#)
 - HandleParameterMouseExit, [962](#)
 - HandleParameterMouseUp, [961](#)
 - SetViewSize, [960](#)
- AAX_IViewContainer.h, [1249](#)
- AAX_LIMIT
 - AAX_SliderConversions.h, [1286](#)
- AAX_LinkedParameters.doxygen, [1250](#)
- AAX_LittleEndianNativeSwap
 - AAX_EndianSwap.h, [1160](#)
- AAX_LittleEndianNativeSwapInPlace
 - AAX_EndianSwap.h, [1159](#)
- AAX_LittleEndianNativeSwapSequenceInPlace
 - AAX_EndianSwap.h, [1160](#)
- AAX_LO
 - AAX_MiscUtils.h, [1255](#)
- AAX_Map, [964](#)
 - ~AAX_Map, [964](#)
 - AAX_Map, [964](#)
 - GetCoefficient, [964](#)
 - GetFirstX, [965](#)
 - GetFirstY, [965](#)
 - GetLastX, [965](#)
 - GetLastY, [965](#)
 - GetSize, [966](#)
 - GetUpperBoundIndex, [965](#)
 - GetX, [965](#)
 - GetY, [965](#)
 - SetCoefficients, [964](#)
- AAX_Map.h, [1250](#)
 - AAX_MAP_H, [1250](#)
- AAX_MAP_H
 - AAX_Map.h, [1250](#)
- AAX_Media_Composer_Guide.doxygen, [1251](#)
- AAX_MIDILogging.cpp, [1251](#)
- AAX_MIDILogging.h, [1251](#)
- AAX_MIDIUtilities.h, [1251](#)
- AAX_MiscUtils.h, [1253](#)
 - AAX_ALIGNMENT_HINT, [1254](#)
 - AAX_DWORD_ALIGNED_HINT, [1255](#)
 - AAX_HI, [1255](#)
 - AAX_INT_HI, [1255](#)
 - AAX_INT_LO, [1255](#)
 - AAX_LO, [1255](#)
 - AAX_MISCUTILS_H, [1254](#)
 - AAX_WORD_ALIGNED_HINT, [1254](#)
- AAX_MISCUTILS_H
 - AAX_MiscUtils.h, [1254](#)
- AAX_OtherExtensions.doxygen, [1256](#)
- AAX_OVERRIDE
 - AAX.h, [1106](#)
 - AAX_CTask, [656](#)
 - AAX_IDataBuffer, [841](#)
 - AAX_ITaskAgent, [948](#)
- AAX_Page_Table_Guide.doxygen, [1256](#)
- AAX_PageTableUtilities.h, [1256](#)
- AAX_ParameterAutomation.doxygen, [1256](#)
- AAX_ParameterManager.doxygen, [1256](#)
- AAX_ParameterUpdateProtocol.doxygen, [1256](#)
- AAX_ParameterUpdateTiming.doxygen, [1256](#)
- AAX_PlatformOptimizationConstants.h, [1256](#)
 - AAX_PLATFORMOPTIMIZATIONCONSTANTS_H, [1257](#)
- AAX_PLATFORMOPTIMIZATIONCONSTANTS_H
 - AAX_PlatformOptimizationConstants.h, [1257](#)
- AAX_PluginBundleLocation.h, [1257](#)
- AAX_Point, [966](#)
 - AAX_GUITypes.h, [1220](#)
 - AAX_Point, [966](#)
 - horz, [967](#)
 - vert, [967](#)
- AAX_PointerSize
 - AAX.h, [1109](#)
- AAX_PopStructAlignment.h, [1257](#)
- AAX_PostStructAlignmentHelper.h, [1258](#)
- AAX_PREPROCESSOR_CONCAT
 - AAX.h, [1111](#)
- AAX_PREPROCESSOR_CONCAT_HELPER
 - AAX.h, [1111](#)
- AAX_PreStructAlignmentHelper.h, [1258](#)
- AAX_Pro_Tools_Guide.doxygen, [1258](#)
- AAX_Properties.h, [1258](#)
 - AAX_ENUM_SIZE_CHECK, [1278](#)
 - AAX_EProperty, [1260](#)
 - AAX_eProperty_AllowPreviewWithoutAnalysis, [1270](#)
 - AAX_eProperty_AlwaysBypass, [1269](#)
 - AAX_eProperty_AudioBufferLength, [1266](#)
 - AAX_eProperty_AudiosuitePropsBase, [1269](#)
 - AAX_eProperty_CanBypass, [1268](#)
 - AAX_eProperty_Constraint_AlwaysProcess, [1277](#)
 - AAX_eProperty_Constraint_DoNotApplyDefaultSettings, [1277](#)
 - AAX_eProperty_Constraint_Location, [1274](#)
 - AAX_eProperty_Constraint_MultiMonoSupport, [1275](#)
 - AAX_eProperty_Constraint_NeverCache, [1274](#)

- AAX_eProperty_Constraint_NeverUnload, 1274
- AAX_eProperty_Constraint_Topology, 1274
- AAX_eProperty_ConstraintBase, 1274
- AAX_eProperty_ConstraintBase_2, 1276
- AAX_eProperty_ContinuousOnly, 1271
- AAX_eProperty_DebugPropertiesBase, 1277
- AAX_eProperty_Deprecated_DSP_Plugin_List, 1264
- AAX_eProperty_Deprecated_Native_Plugin_List, 1265
- AAX_eProperty_Deprecated_Plugin_List, 1264
- AAX_eProperty_DestinationTrack, 1271
- AAX_eProperty_DisableAudioSuiteReverse, 1273
- AAX_eProperty_DisableHandles, 1272
- AAX_eProperty_DisablePreview, 1272
- AAX_eProperty_DoesntIncrOutputSample, 1272
- AAX_eProperty_DSP_AudioBufferLength, 1266
- AAX_eProperty_EnableHostDebugLogs, 1278
- AAX_eProperty_ExternalProcessorTypeID, 1265
- AAX_eProperty_FeaturesBase, 1275
- AAX_eProperty_GeneralPropsBase, 1266
- AAX_eProperty_GUIBase, 1273
- AAX_eProperty_HybridInputStemFormat, 1269
- AAX_eProperty_HybridOutputStemFormat, 1269
- AAX_eProperty_InputStemFormat, 1266
- AAX_eProperty_LatencyContribution, 1267
- AAX_eProperty_ManufacturerID, 1261
- AAX_eProperty_MaxASProp, 1273
- AAX_eProperty_MaxCap, 1278
- AAX_eProperty_MaxConstraintProp, 1275
- AAX_eProperty_MaxConstraintProp_2, 1277
- AAX_eProperty_MaxFeaturesProp, 1276
- AAX_eProperty_MaxGUIProp, 1273
- AAX_eProperty_MaxMeterProp, 1274
- AAX_eProperty_MaxProp, 1278
- AAX_eProperty_Meter_Ballistics, 1274
- AAX_eProperty_Meter_Orientation, 1274
- AAX_eProperty_Meter_Type, 1273
- AAX_eProperty_MeterBase, 1273
- AAX_eProperty_MinProp, 1261
- AAX_eProperty_MultiInputModeOnly, 1271
- AAX_eProperty_NativeBackgroundProc, 1266
- AAX_eProperty_NativeInstanceInitProc, 1265
- AAX_eProperty_NativeProcessProc, 1265
- AAX_eProperty_NeedsOutputDithered, 1273
- AAX_eProperty_NoID, 1261
- AAX_eProperty_NumberOfInputs, 1272
- AAX_eProperty_NumberOfOutputs, 1272
- AAX_eProperty_ObservesTransportState, 1276
- AAX_eProperty_OptionalAnalysis, 1270
- AAX_eProperty_OutputStemFormat, 1266
- AAX_eProperty_PluginID_AudioSuite, 1262
- AAX_eProperty_PluginID_Deprecated, 1264
- AAX_eProperty_PluginID_ExternalProcessor, 1265
- AAX_eProperty_PluginID_Native, 1262
- AAX_eProperty_PluginID_NoProcessing, 1263
- AAX_eProperty_PluginID_RTAS, 1262
- AAX_eProperty_PluginID_TI, 1263
- AAX_eProperty_PluginSpecPropsBase, 1261
- AAX_eProperty_ProcessProcPropsBase, 1265
- AAX_eProperty_ProductID, 1261
- AAX_eProperty_Related_DSP_Plugin_List, 1264
- AAX_eProperty_Related_Native_Plugin_List, 1264
- AAX_eProperty_RequestsAllTrackData, 1271
- AAX_eProperty_RequiresAnalysis, 1270
- AAX_eProperty_RequiresChunkCallsOnMainThread, 1276
- AAX_eProperty_SampleRate, 1267
- AAX_eProperty_ShowInMenus, 1269
- AAX_eProperty_SideChainStemFormat, 1268
- AAX_eProperty_StoreXMLPageTablesByEffect, 1275
- AAX_eProperty_StoreXMLPageTablesByType, 1275
- AAX_eProperty_SupportsSaveRestore, 1275
- AAX_eProperty_SupportsSideChainInput, 1273
- AAX_eProperty_TI_ForceAllowChipSharing, 1269
- AAX_eProperty_TI_InstanceCycleCount, 1268
- AAX_eProperty_TI_MaxInstancesPerChip, 1268
- AAX_eProperty_TI_SharedCycleCount, 1268
- AAX_eProperty_TIBackgroundProc, 1266
- AAX_eProperty_TIDLLFileName, 1266
- AAX_eProperty_TIInstanceInitProc, 1266
- AAX_eProperty_TIProcessProc, 1266
- AAX_eProperty_UsesClientGUI, 1273
- AAX_eProperty_UsesRandomAccess, 1270
- AAX_eProperty_UsesTransport, 1275
- AAX_eProperty_UsesTransportControl, 1276
- AAX_Push2ByteStructAlignment.h, 1278
- AAX_Push4ByteStructAlignment.h, 1279
- AAX_Push8ByteStructAlignment.h, 1280
- AAX_Quantize.h, 1280
 - AAX_QUANTIZE_H, 1281
- AAX_QUANTIZE_H
 - AAX_Quantize.h, 1281
- AAX_RandomGen.h, 1281
 - AAX_RANDOMGEN_H, 1282
- AAX_RANDOMGEN_H
 - AAX_RandomGen.h, 1282
- AAX_RealTimePerformance.doxygen, 1282
- AAX_Rect, 967
 - AAX_GUITypes.h, 1221
 - AAX_Rect, 968
 - height, 968
 - left, 968
 - top, 968
 - width, 968
- AAX_RelatedTypes.doxygen, 1282
- AAX_Result
 - AAX.h, 1113
- AAX_RESULT_ADD_FIELD_UNSUPPORTED_FIELD_TYPE
 - AAX_Errors.h, 1211
- AAX_RESULT_NEW_PACKET_POSTED
 - AAX_Errors.h, 1211
- AAX_RESULT_PACKET_STREAM_NOT_EMPTY

- AAX_Errors.h, [1211](#)
- AAX_SampleRateUtils.h, [1282](#)
 - CoarseSampleRate, [1283](#)
 - CoarseSampleRateFactor, [1284](#)
 - CoarseSampleRateIndex, [1284](#)
 - ESRUtils, [1283](#)
 - eSRUtils_192kIndex, [1283](#)
 - eSRUtils_192kRangeCoarse, [1283](#)
 - eSRUtils_192kRangeMax, [1283](#)
 - eSRUtils_192kRangeMin, [1283](#)
 - eSRUtils_48kIndex, [1283](#)
 - eSRUtils_48kRangeCoarse, [1283](#)
 - eSRUtils_48kRangeMax, [1283](#)
 - eSRUtils_48kRangeMin, [1283](#)
 - eSRUtils_96kIndex, [1283](#)
 - eSRUtils_96kRangeCoarse, [1283](#)
 - eSRUtils_96kRangeMax, [1283](#)
 - eSRUtils_96kRangeMin, [1283](#)
- AAX_SCOPE_COMPUTE_DENORMALS
 - AAX_Denormal.h, [1157](#)
- AAX_SCOPE_DENORMALS_AS_ZERO
 - AAX_Denormal.h, [1157](#)
- AAX_SDK_1p0p1_REVISION
 - AAX_Version.h, [1315](#)
- AAX_SDK_1p0p2_REVISION
 - AAX_Version.h, [1315](#)
- AAX_SDK_1p0p3_REVISION
 - AAX_Version.h, [1315](#)
- AAX_SDK_1p0p4_REVISION
 - AAX_Version.h, [1315](#)
- AAX_SDK_1p0p5_REVISION
 - AAX_Version.h, [1315](#)
- AAX_SDK_1p0p6_REVISION
 - AAX_Version.h, [1315](#)
- AAX_SDK_1p5p0_REVISION
 - AAX_Version.h, [1315](#)
- AAX_SDK_2p0b1_REVISION
 - AAX_Version.h, [1316](#)
- AAX_SDK_2p0p0_REVISION
 - AAX_Version.h, [1316](#)
- AAX_SDK_2p0p1_REVISION
 - AAX_Version.h, [1316](#)
- AAX_SDK_2p1p0_REVISION
 - AAX_Version.h, [1316](#)
- AAX_SDK_2p1p1_REVISION
 - AAX_Version.h, [1316](#)
- AAX_SDK_2p2p0_REVISION
 - AAX_Version.h, [1316](#)
- AAX_SDK_2p2p1_REVISION
 - AAX_Version.h, [1316](#)
- AAX_SDK_2p2p2_REVISION
 - AAX_Version.h, [1316](#)
- AAX_SDK_2p3p0_REVISION
 - AAX_Version.h, [1317](#)
- AAX_SDK_2p3p1_REVISION
 - AAX_Version.h, [1317](#)
- AAX_SDK_2p3p2_REVISION
 - AAX_Version.h, [1317](#)
- AAX_SDK_2p4p0_REVISION
 - AAX_Version.h, [1317](#)
- AAX_SDK_2p4p1_REVISION
 - AAX_Version.h, [1317](#)
- AAX_SDK_2p5p0_REVISION
 - AAX_Version.h, [1317](#)
- AAX_SDK_2p6p0_REVISION
 - AAX_Version.h, [1317](#)
- AAX_SDK_2p6p1_REVISION
 - AAX_Version.h, [1317](#)
- AAX_SDK_2p7p0_REVISION
 - AAX_Version.h, [1318](#)
- AAX_SDK_2p8p0_REVISION
 - AAX_Version.h, [1318](#)
- AAX_SDK_2p8p1_REVISION
 - AAX_Version.h, [1318](#)
- AAX_SDK_ChangeLog.doxygen, [1284](#)
- AAX_SDK_CURRENT_REVISION
 - AAX_Version.h, [1314](#)
- AAX_SDK_ExamplePlugIns.doxygen, [1284](#)
- AAX_SDK_GUIExtensions.doxygen, [1284](#)
- AAX_SDK_VERSION
 - AAX_Version.h, [1314](#)
- AAX_SessionDocumentTypes.h, [1284](#)
 - AAX_SessionDocumentTypes_H, [1285](#)
 - kAAX_DataBufferType_TempoBreakpointArray, [1285](#)
- AAX_SessionDocumentTypes_H
 - AAX_SessionDocumentTypes.h, [1285](#)
- AAX_SHybridRenderInfo, [969](#)
 - mAudioInputs, [969](#)
 - mAudioOutputs, [969](#)
 - mClock, [970](#)
 - mNumAudioInputs, [969](#)
 - mNumAudioOutputs, [970](#)
 - mNumSamples, [970](#)
- AAX_SInstrumentPrivateData, [970](#)
 - mMonolithicParametersPtr, [971](#)
- AAX_SInstrumentRenderInfo, [971](#)
 - mAdditionalInputMIDINodes, [973](#)
 - mAudioInputs, [972](#)
 - mAudioOutputs, [972](#)
 - mClock, [972](#)
 - mCurrentStateNum, [973](#)
 - mGlobalNode, [972](#)
 - mInputNode, [972](#)
 - mMeters, [973](#)
 - mNumSamples, [972](#)
 - mPrivateData, [973](#)
 - mTransportNode, [973](#)
- AAX_SInstrumentSetupInfo, [974](#)
 - AAX_SInstrumentSetupInfo, [975](#)
 - mAudiosuiteID, [981](#)
 - mAuxOutputStemFormats, [979](#)
 - mAuxOutputStemNames, [978](#)
 - mCanBypass, [980](#)
 - mGlobalMIDIEventMask, [976](#)
 - mGlobalMIDINodeName, [976](#)

- mHybridInputStemFormat, [979](#)
- mHybridOutputStemFormat, [979](#)
- mInputMIDChannelMask, [977](#)
- mInputMIDNodeName, [976](#)
- mInputStemFormat, [980](#)
- mManufacturerID, [981](#)
- mMeterIDs, [978](#)
- mMultiMonoSupport, [981](#)
- mNeedsGlobalMIDI, [975](#)
- mNeedsInputMIDI, [976](#)
- mNeedsTransport, [977](#)
- mNumAdditionalInputMIDINodes, [977](#)
- mNumAuxOutputStems, [978](#)
- mNumMeters, [978](#)
- mOutputStemFormat, [980](#)
- mPluginID, [981](#)
- mProductID, [981](#)
- mTransportMIDNodeName, [977](#)
- mUseHostGeneratedGUI, [980](#)
- AAX_SliderConversions.h, [1285](#)
 - AAX_LIMIT, [1286](#)
 - AAX_SLIDERCONVERSIONS_H, [1286](#)
 - DoubleToLongControl, [1287](#)
 - DoubleToLongControlNonlinear, [1287](#)
 - LogDoubleToLongControl, [1288](#)
 - LongControlToDouble, [1287](#)
 - LongControlToDoubleNonlinear, [1287](#)
 - LongControlToLogDouble, [1288](#)
 - LongControlToNewRange, [1286](#)
 - LongToLongControl, [1287](#)
- AAX_SLIDERCONVERSIONS_H
 - AAX_SliderConversions.h, [1286](#)
- AAX_SPlugInChunk, [982](#)
 - AAX.h, [1117](#)
 - fChunkID, [983](#)
 - fData, [984](#)
 - fManufacturerID, [983](#)
 - fName, [984](#)
 - fPluginID, [983](#)
 - fProductID, [983](#)
 - fSize, [983](#)
 - fVersion, [983](#)
- AAX_SPlugInChunkHeader, [984](#)
 - AAX.h, [1116](#)
 - fChunkID, [986](#)
 - fManufacturerID, [986](#)
 - fName, [986](#)
 - fPluginID, [986](#)
 - fProductID, [986](#)
 - fSize, [985](#)
 - fVersion, [986](#)
- AAX_SPlugInChunkPtr
 - AAX.h, [1117](#)
- AAX_SPlugInIdentifierTriad, [987](#)
 - AAX.h, [1117](#)
 - mManufacturerID, [987](#)
 - mPluginID, [988](#)
 - mProductID, [987](#)
 - AAX_SPlugInIdentifierTriadPtr
 - AAX.h, [1117](#)
- AAX_STACKTRACE
 - AAX_Assert.h, [1124](#)
- AAX_STACKTRACE_RELEASE
 - AAX_Assert.h, [1122](#)
- AAX_STEM_FORMAT
 - AAX_Enums.h, [1171](#)
- AAX_STEM_FORMAT_CHANNEL_COUNT
 - AAX_Enums.h, [1171](#)
- AAX_STEM_FORMAT_INDEX
 - AAX_Enums.h, [1171](#)
- AAX_StLock_Guard, [988](#)
 - ~AAX_StLock_Guard, [988](#)
 - AAX_StLock_Guard, [988](#)
- AAX_StringUtilities.h, [1288](#)
 - AAXLibrary_AAX_StringUtilities_h, [1289](#)
- AAX_StringUtilities.hpp, [1289](#)
 - DEFINE_AAX_ERROR_STRING, [1290](#)
- AAX_SUCCESS
 - AAX_Errors.h, [1210](#)
- AAX_SWALLOW
 - AAX_Exception.h, [1213](#)
- AAX_SWALLOW_MULT
 - AAX_Exception.h, [1213](#)
- AAX_TaskCompletionStatus
 - Task agent interface, [105](#)
- AAX_TI_Guide.doxygen, [1290](#)
- AAX_TRACE
 - AAX_Assert.h, [1124](#)
- AAX_TRACE_RELEASE
 - AAX_Assert.h, [1121](#)
- AAX_TRACEORSTACKTRACE
 - AAX_Assert.h, [1124](#)
- AAX_TRACEORSTACKTRACE_RELEASE
 - AAX_Assert.h, [1122](#)
- AAX_TransportStateInfo_V1, [989](#)
 - AAX_TransportStateInfo_V1, [989](#)
 - mIsLoopEnabled, [990](#)
 - mIsRecordEnabled, [990](#)
 - mIsRecording, [990](#)
 - mRecordMode, [990](#)
 - mTransportState, [990](#)
 - ToString, [989](#)
- AAX_TransportTypes.h, [1290](#)
 - operator!=, [1291](#)
 - operator==, [1291](#)
- AAX_Troubleshooting.doxygen, [1291](#)
- AAX_UIDs.h, [1291](#)
 - AAX_ComplID_DescriptionHost, [1303](#)
 - AAX_ComplID_FeatureInfo, [1303](#)
 - AAX_DocumentData_UID, [1295](#)
 - AAX_DocumentDataType_TempoMap, [1309](#)
 - AAX_Feature_UID, [1295](#)
 - AAXATTR_Client_Level, [1308](#)
 - AAXATTR_Client_Version, [1308](#)
 - AAXATTR_ClientFeature_AuxOutputStem, [1308](#)
 - AAXATTR_ClientFeature_MIDI, [1308](#)

- AAXATTR_ClientFeature_SideChainInput, 1308
- AAXATTR_ClientFeature_StemFormat, 1307
- AAXCompID_AAXCollection, 1296
- AAXCompID_AAXComponentDescriptor, 1297
- AAXCompID_AAXEffectDescriptor, 1296
- AAXCompID_AAXPropertyMap, 1297
- AAXCompID_AutomationDelegate, 1299
- AAXCompID_Controller, 1299
- AAXCompID_DataBuffer, 1307
- AAXCompID_EffectDirectData, 1306
- AAXCompID_EffectGUI, 1306
- AAXCompID_EffectParameters, 1304
- AAXCompID_HostProcessor, 1305
- AAXCompID_HostProcessorDelegate, 1298
- AAXCompID_HostServices, 1295
- AAXCompID_PageTable, 1302
- AAXCompID_PageTableController, 1300
- AAXCompID_PrivateDataAccess, 1300
- AAXCompID_SessionDocument, 1304
- AAXCompID_SessionDocumentClient, 1307
- AAXCompID_Task, 1304
- AAXCompID_TaskAgent, 1306
- AAXCompID_Transport, 1301
- AAXCompID_TransportControl, 1302
- AAXCompID_ViewContainer, 1300
- IID_IAAXAutomationDelegateV1, 1299
- IID_IAAXCollectionV1, 1296
- IID_IAAXComponentDescriptorV1, 1297
- IID_IAAXComponentDescriptorV2, 1297
- IID_IAAXComponentDescriptorV3, 1297
- IID_IAAXControllerV1, 1299
- IID_IAAXControllerV2, 1299
- IID_IAAXControllerV3, 1299
- IID_IAAXDataBufferV1, 1307
- IID_IAAXDescriptionHostV1, 1303
- IID_IAAXEffectDescriptorV1, 1296
- IID_IAAXEffectDescriptorV2, 1296
- IID_IAAXEffectDirectDataV1, 1306
- IID_IAAXEffectDirectDataV2, 1306
- IID_IAAXEffectGUIV1, 1306
- IID_IAAXEffectParametersV1, 1304
- IID_IAAXEffectParametersV2, 1305
- IID_IAAXEffectParametersV3, 1305
- IID_IAAXEffectParametersV4, 1305
- IID_IAAXFeatureInfoV1, 1303
- IID_IAAXHostProcessorDelegateV1, 1298
- IID_IAAXHostProcessorDelegateV2, 1298
- IID_IAAXHostProcessorDelegateV3, 1298
- IID_IAAXHostProcessorV1, 1305
- IID_IAAXHostProcessorV2, 1305
- IID_IAAXHostServicesV1, 1295
- IID_IAAXHostServicesV2, 1295
- IID_IAAXHostServicesV3, 1296
- IID_IAAXPageTableController, 1300
- IID_IAAXPageTableControllerV2, 1300
- IID_IAAXPageTableV1, 1303
- IID_IAAXPageTableV2, 1303
- IID_IAAXPrivateDataAccessV1, 1300
- IID_IAAXPropertyMapV1, 1297
- IID_IAAXPropertyMapV2, 1298
- IID_IAAXPropertyMapV3, 1298
- IID_IAAXSessionDocumentClientV1, 1307
- IID_IAAXSessionDocumentV1, 1304
- IID_IAAXTaskAgentV1, 1307
- IID_IAAXTaskV1, 1304
- IID_IAAXTransportControlV1, 1302
- IID_IAAXTransportV1, 1301
- IID_IAAXTransportV2, 1301
- IID_IAAXTransportV3, 1302
- IID_IAAXTransportV4, 1302
- IID_IAAXTransportV5, 1302
- IID_IAAXViewContainerV1, 1301
- IID_IAAXViewContainerV2, 1301
- IID_IAAXViewContainerV3, 1301
- AAX_UINT16_MAX
 - AAX_Enums.h, 1170
- AAX_UINT16_MIN
 - AAX_Enums.h, 1170
- AAX_UINT32_MAX
 - AAX_Enums.h, 1170
- AAX_UINT32_MIN
 - AAX_Enums.h, 1170
- AAX_UNIQUE_PTR
 - AAX.h, 1108
- AAX_UtilsNative.h, 1309
 - _AAX_UTILSNATIVE_H_, 1310
- AAX_VAutomationDelegate, 991
 - ~AAX_VAutomationDelegate, 991
 - AAX_VAutomationDelegate, 991
 - GetTouchState, 994
 - GetUnknown, 991
 - ParameterNameChanged, 994
 - PostCurrentValue, 993
 - PostReleaseRequest, 994
 - PostSetValueRequest, 993
 - PostTouchRequest, 993
 - RegisterParameter, 992
 - UnregisterParameter, 992
- AAX_VAutomationDelegate.h, 1310
- AAX_VCollection, 995
 - ~AAX_VCollection, 996
 - AAX_VCollection, 995
 - AddEffect, 996
 - AddPackageName, 997
 - DescriptionHost, 999
 - GetHostVersion, 998
 - GetUnknown, 1000
 - HostDefinition, 999
 - NewDescriptor, 996
 - NewPropertyMap, 998
 - SetManufacturerName, 997
 - SetPackageVersion, 997
 - SetProperties, 998
- AAX_VCollection.h, 1311
- AAX_VComponentDescriptor, 1000
 - ~AAX_VComponentDescriptor, 1002

- AAX_VComponentDescriptor, 1002
- AAX_VPropertyMap, 1014
- AddAudioBufferLength, 1004
- AddAudioIn, 1003
- AddAudioOut, 1003
- AddAuxOutputStem, 1006
- AddClock, 1004
- AddDataInPort, 1005
- AddDmaInstance, 1008
- AddMeters, 1008
- AddMIDINode, 1010
- AddPrivateData, 1007
- AddProcessProc, 1012
- AddProcessProc_Native, 1011
- AddProcessProc_TI, 1012
- AddReservedField, 1002
- AddSampleRate, 1004
- AddSideChainIn, 1005
- AddTemporaryData, 1007
- Clear, 1002
- DuplicatePropertyMap, 1011
- GetUnknown, 1014
- NewPropertyMap, 1010
- AAX_VComponentDescriptor.h, 1311
- AAX_VController, 1014
 - ~AAX_VController, 1016
 - AAX_VController, 1016
 - ClearMeterClipped, 1026
 - ClearMeterPeakValue, 1025
 - CreateTableCopyForEffect, 1027
 - CreateTableCopyForEffectFromFile, 1028
 - CreateTableCopyForLayout, 1027
 - CreateTableCopyForLayoutFromFile, 1029
 - GetCurrentAutomationTimestamp, 1020
 - GetCurrentMeterValue, 1024
 - GetCycleCount, 1019
 - GetEffectID, 1016
 - GetHostName, 1021
 - GetHybridSignalLatency, 1018
 - GetInputStemFormat, 1017
 - GetIsAudioSuite, 1019
 - GetMeterClipped, 1025
 - GetMeterCount, 1026
 - GetMeterPeakValue, 1025
 - GetNextMIDIAPacket, 1026
 - GetOutputStemFormat, 1017
 - GetPlugInTargetPlatform, 1019
 - GetSampleRate, 1017
 - GetSignalLatency, 1018
 - GetTODLocation, 1020
 - PostPacket, 1022
 - SendNotification, 1023, 1024
 - SetCycleCount, 1022
 - SetSignalLatency, 1021
- AAX_VController.h, 1311
- AAX_VDataBufferWrapper, 1029
 - ~AAX_VDataBufferWrapper, 1030
 - AAX_VDataBufferWrapper, 1030
 - Data, 1030
 - Size, 1030
 - Type, 1030
- AAX_VDataBufferWrapper.h, 1312
 - AAX_VDATABUFFERWRAPPER_H, 1312
- AAX_VDATABUFFERWRAPPER_H
 - AAX_VDataBufferWrapper.h, 1312
- AAX_VDescriptionHost, 1031
 - ~AAX_VDescriptionHost, 1031
 - AAX_VDescriptionHost, 1031
 - AcquireFeatureProperties, 1032
 - DescriptionHost, 1032, 1033
 - HostDefinition, 1033
 - Supported, 1032
- AAX_VDescriptionHost.h, 1312
- AAX_VEffectDescriptor, 1033
 - ~AAX_VEffectDescriptor, 1034
 - AAX_VEffectDescriptor, 1034
 - AddCategory, 1036
 - AddCategoryBypassParameter, 1036
 - AddComponent, 1035
 - AddControlMIDINode, 1038
 - AddMeterDescription, 1037
 - AddName, 1035
 - AddProcPtr, 1036
 - AddResourceInfo, 1037
 - GetUnknown, 1038
 - NewComponentDescriptor, 1034
 - NewPropertyMap, 1037
 - SetProperties, 1037
- AAX_VEffectDescriptor.h, 1313
- AAX_VENUE_Guide.doxygen, 1313
- AAX_Version.h, 1313
 - _AAX_VERSION_H_, 1314
 - AAX_SDK_1p0p1_REVISION, 1315
 - AAX_SDK_1p0p2_REVISION, 1315
 - AAX_SDK_1p0p3_REVISION, 1315
 - AAX_SDK_1p0p4_REVISION, 1315
 - AAX_SDK_1p0p5_REVISION, 1315
 - AAX_SDK_1p0p6_REVISION, 1315
 - AAX_SDK_1p5p0_REVISION, 1315
 - AAX_SDK_2p0b1_REVISION, 1316
 - AAX_SDK_2p0p0_REVISION, 1316
 - AAX_SDK_2p0p1_REVISION, 1316
 - AAX_SDK_2p1p0_REVISION, 1316
 - AAX_SDK_2p1p1_REVISION, 1316
 - AAX_SDK_2p2p0_REVISION, 1316
 - AAX_SDK_2p2p1_REVISION, 1316
 - AAX_SDK_2p2p2_REVISION, 1316
 - AAX_SDK_2p3p0_REVISION, 1317
 - AAX_SDK_2p3p1_REVISION, 1317
 - AAX_SDK_2p3p2_REVISION, 1317
 - AAX_SDK_2p4p0_REVISION, 1317
 - AAX_SDK_2p4p1_REVISION, 1317
 - AAX_SDK_2p5p0_REVISION, 1317
 - AAX_SDK_2p6p0_REVISION, 1317
 - AAX_SDK_2p6p1_REVISION, 1317
 - AAX_SDK_2p7p0_REVISION, 1318

- AAX_SDK_2p8p0_REVISION, 1318
- AAX_SDK_2p8p1_REVISION, 1318
- AAX_SDK_CURRENT_REVISION, 1314
- AAX_SDK_VERSION, 1314
- AAX_VFeatureInfo, 1039
 - ~AAX_VFeatureInfo, 1039
 - AAX_VFeatureInfo, 1039
 - AcquireProperties, 1040
 - ID, 1040
 - SupportLevel, 1040
- AAX_VFeatureInfo.h, 1318
- AAX_VHostProcessorDelegate, 1041
 - AAX_VHostProcessorDelegate, 1041
 - ForceAnalyze, 1042
 - ForceProcess, 1043
 - GetAudio, 1042
 - GetSideChainInputNum, 1042
- AAX_VHostProcessorDelegate.h, 1318
- AAX_VHostServices, 1043
 - ~AAX_VHostServices, 1044
 - AAX_VHostServices, 1044
 - HandleAssertFailure, 1044
 - StackTrace, 1045
 - Trace, 1044
- AAX_VHostServices.h, 1319
- AAX_VHostTaskAgent, 1045
 - ~AAX_VHostTaskAgent, 1046
 - AAX_VHostTaskAgent, 1046
 - AddTask, 1046
 - CancelAllTasks, 1047
 - Initialize, 1046
 - Uninitialize, 1046
- AAX_VHostTaskAgent.h, 1319
 - AAX_VHostTaskAgent_H, 1319
- AAX_VHostTaskAgent_H
 - AAX_VHostTaskAgent.h, 1319
- AAX_VPageTable, 1047
 - ~AAX_VPageTable, 1048
 - AAX_VPageTable, 1048
 - AsUnknown, 1057
 - Clear, 1048
 - ClearMappedParameter, 1051
 - ClearNameVariationsForParameter, 1056
 - ClearParameterNameVariations, 1055
 - Empty, 1049
 - GetMappedParameterID, 1051
 - GetNameVariationParameterIDAtIndex, 1053
 - GetNumMappedParameterIDs, 1050
 - GetNumNameVariationsForParameter, 1053
 - GetNumPages, 1049
 - GetNumParametersWithNameVariations, 1052
 - GetParameterNameVariationAtIndex, 1054
 - GetParameterNameVariationOfLength, 1055
 - InsertPage, 1049
 - IsSupported, 1058
 - MapParameterID, 1052
 - RemovePage, 1050
 - SetParameterNameVariation, 1057
- AAX_VPageTable.h, 1320
- AAX_VPrivateDataAccess, 1058
 - ~AAX_VPrivateDataAccess, 1058
 - AAX_VPrivateDataAccess, 1058
 - ReadPortDirect, 1059
 - WritePortDirect, 1059
- AAX_VPrivateDataAccess.h, 1320
- AAX_VPropertyMap, 1060
 - ~AAX_VPropertyMap, 1061
 - AAX_VComponentDescriptor, 1014
 - Acquire, 1061
 - AddPointerProperty, 1062, 1063
 - AddProperty, 1062
 - AddPropertyWithIDArray, 1064
 - Create, 1061
 - GetIUnknown, 1065
 - GetPointerProperty, 1062
 - GetProperty, 1061
 - GetPropertyWithIDArray, 1064
 - RemoveProperty, 1063
- AAX_VPropertyMap.h, 1320
- AAX_VSessionDocument, 1065
 - ~AAX_VSessionDocument, 1066
 - AAX_VSessionDocument, 1065
 - Clear, 1066
 - GetDocumentData, 1066
 - GetTempoMap, 1066
 - Valid, 1066
- AAX_VSessionDocument.h, 1321
 - AAX_VSessionDocument_H, 1321
- AAX_VSessionDocument::VTempoMap, 1100
 - ~VTempoMap, 1100
 - Data, 1101
 - Size, 1101
 - VTempoMap, 1101
- AAX_VSessionDocument_H
 - AAX_VSessionDocument.h, 1321
- AAX_VTask, 1068
 - ~AAX_VTask, 1069
 - AAX_VTask, 1068
 - AddResult, 1070
 - GetArgumentOfType, 1069
 - GetProgress, 1070
 - GetType, 1069
 - SetDone, 1071
 - SetProgress, 1070
- AAX_VTask.h, 1321
 - AAX_VTask_H, 1322
- AAX_VTask_H
 - AAX_VTask.h, 1322
- AAX_VTransport, 1071
 - ~AAX_VTransport, 1072
 - AAX_VTransport, 1072
 - GetBarBeatPosition, 1077
 - GetCurrentLoopPosition, 1075
 - GetCurrentMeter, 1073
 - GetCurrentNativeSampleLocation, 1076
 - GetCurrentTempo, 1073

- GetCurrentTickPosition, [1075](#)
- GetCurrentTicksPerBeat, [1077](#)
- GetCustomTickPosition, [1076](#)
- GetFeetFramesInfo, [1078](#)
- GetHDTTimeCodeInfo, [1079](#)
- GetKeySignature, [1080](#)
- GetTicksPerQuarter, [1077](#)
- GetTimeCodeInfo, [1078](#)
- GetTimelineSelectionEndPosition, [1080](#)
- GetTimelineSelectionStartPosition, [1078](#)
- IsMetronomeEnabled, [1079](#)
- IsTransportPlaying, [1073](#)
- RequestTransportStart, [1081](#)
- RequestTransportStop, [1081](#)
- AAX_VTransport.h, [1322](#)
- AAX_VViewContainer, [1081](#)
 - ~AAX_VViewContainer, [1082](#)
 - AAX_VViewContainer, [1082](#)
 - GetModifiers, [1083](#)
 - GetPtr, [1083](#)
 - GetType, [1083](#)
 - HandleMultipleParametersMouseDown, [1087](#)
 - HandleMultipleParametersMouseDrag, [1087](#)
 - HandleMultipleParametersMouseUp, [1088](#)
 - HandleParameterMouseDown, [1084](#)
 - HandleParameterMouseDrag, [1084](#)
 - HandleParameterMouseEnter, [1086](#)
 - HandleParameterMouseExit, [1086](#)
 - HandleParameterMouseUp, [1086](#)
 - SetViewSize, [1084](#)
- AAX_VViewContainer.h, [1322](#)
- AAX_WORD_ALIGNED_HINT
 - AAX_MiscUtils.h, [1254](#)
- AAXATTR_Client_Level
 - AAX_UIDs.h, [1308](#)
- AAXATTR_Client_Version
 - AAX_UIDs.h, [1308](#)
- AAXATTR_ClientFeature_AuxOutputStem
 - AAX_UIDs.h, [1308](#)
- AAXATTR_ClientFeature_MIDI
 - AAX_UIDs.h, [1308](#)
- AAXATTR_ClientFeature_SideChainInput
 - AAX_UIDs.h, [1308](#)
- AAXATTR_ClientFeature_StemFormat
 - AAX_UIDs.h, [1307](#)
- AAXCanUnloadNow
 - AAX_Init.h, [1243](#)
- AAXCompID_AAXCollection
 - AAX_UIDs.h, [1296](#)
- AAXCompID_AAXComponentDescriptor
 - AAX_UIDs.h, [1297](#)
- AAXCompID_AAXEffectDescriptor
 - AAX_UIDs.h, [1296](#)
- AAXCompID_AAXPropertyMap
 - AAX_UIDs.h, [1297](#)
- AAXCompID_AutomationDelegate
 - AAX_UIDs.h, [1299](#)
- AAXCompID_Controller
 - AAX_UIDs.h, [1299](#)
- AAXCompID_DataBuffer
 - AAX_UIDs.h, [1307](#)
- AAXCompID_EffectDirectData
 - AAX_UIDs.h, [1306](#)
- AAXCompID_EffectGUI
 - AAX_UIDs.h, [1306](#)
- AAXCompID_EffectParameters
 - AAX_UIDs.h, [1304](#)
- AAXCompID_HostProcessor
 - AAX_UIDs.h, [1305](#)
- AAXCompID_HostProcessorDelegate
 - AAX_UIDs.h, [1298](#)
- AAXCompID_HostServices
 - AAX_UIDs.h, [1295](#)
- AAXCompID_PageTable
 - AAX_UIDs.h, [1302](#)
- AAXCompID_PageTableController
 - AAX_UIDs.h, [1300](#)
- AAXCompID_PrivateDataAccess
 - AAX_UIDs.h, [1300](#)
- AAXCompID_SessionDocument
 - AAX_UIDs.h, [1304](#)
- AAXCompID_SessionDocumentClient
 - AAX_UIDs.h, [1307](#)
- AAXCompID_Task
 - AAX_UIDs.h, [1304](#)
- AAXCompID_TaskAgent
 - AAX_UIDs.h, [1306](#)
- AAXCompID_Transport
 - AAX_UIDs.h, [1301](#)
- AAXCompID_TransportControl
 - AAX_UIDs.h, [1302](#)
- AAXCompID_ViewContainer
 - AAX_UIDs.h, [1300](#)
- AAXCreateObjectProc
 - AAX_Callbacks.h, [1128](#)
- AAXGetClassFactory
 - AAX_Init.h, [1243](#)
- AAXGetSDKVersion
 - AAX_Init.h, [1244](#)
- AAXLibrary_AAX_StringUtilities_h
 - AAX_StringUtilities.h, [1289](#)
- AAXPointer_32bit
 - AAX.h, [1109](#)
- AAXPointer_64bit
 - AAX.h, [1109](#)
- AAXRegisterComponent
 - AAX_Init.h, [1242](#)
- AAXRegisterPlugin
 - Description callback, [67](#)
- AAXShutdown
 - AAX_Init.h, [1244](#)
- AAXStartup
 - AAX_Init.h, [1243](#)
- AbsMax
 - AAX, [388](#)
- Accepts

- AAX_IMIDIMessageInfoDelegate, [885](#)
- Accepts_ExactStatus
 - AAX_IMIDIMessageInfoDelegate, [885](#)
- ACF Elements, [149](#)
- ACF_DECLARE_STANDARD_UNKNOWN
 - AAX_CTask, [653](#)
 - AAX_IDataBuffer, [841](#)
 - AAX_IEffectDirectData, [870](#)
 - AAX_IEffectGUI, [871](#)
 - AAX_IEffectParameters, [873](#)
 - AAX_IHostProcessor, [876](#)
 - AAX_ISessionDocumentClient, [937](#)
 - AAX_ITaskAgent, [948](#)
- ACFCanUnloadNow
 - AAX_Exports.cpp, [1217](#)
- ACFGetClassFactory
 - AAX_Exports.cpp, [1216](#)
- ACFGetSDKVersion
 - AAX_Exports.cpp, [1217](#)
- acfIID
 - AAX_ACFInterface.doxygen, [1118](#)
- ACFRegisterComponent
 - AAX_Exports.cpp, [1216](#)
- ACFRegisterPlugin
 - AAX_Exports.cpp, [1216](#)
- ACFShutdown
 - AAX_Exports.cpp, [1217](#)
- ACFStartup
 - AAX_Exports.cpp, [1217](#)
- acfUID
 - AAX.h, [1115](#)
 - AAX_ACFInterface.doxygen, [1118](#)
- Acquire
 - AAX_VPropertyMap, [1061](#)
- AcquireFeatureProperties
 - AAX_IACFDescriptionHost, [699](#)
 - AAX_IDescriptionHost, [843](#)
 - AAX_VDescriptionHost, [1032](#)
- AcquireProperties
 - AAX_IACFFeatureInfo, [740](#)
 - AAX_IFeatureInfo, [875](#)
 - AAX_VFeatureInfo, [1040](#)
- Add
 - AAX_CStringAbbreviations, [642](#)
- AddAcceptedResult
 - AAX_CheckedResult, [484](#)
- AddAudioBufferLength
 - AAX_IACFComponentDescriptor, [677](#)
 - AAX_IComponentDescriptor, [811](#)
 - AAX_VComponentDescriptor, [1004](#)
- AddAudioIn
 - AAX_IACFComponentDescriptor, [676](#)
 - AAX_IComponentDescriptor, [810](#)
 - AAX_VComponentDescriptor, [1003](#)
- AddAudioOut
 - AAX_IACFComponentDescriptor, [676](#)
 - AAX_IComponentDescriptor, [811](#)
 - AAX_VComponentDescriptor, [1003](#)
- AddAuxOutputStem
 - AAX_IACFComponentDescriptor, [679](#)
 - AAX_IComponentDescriptor, [814](#)
 - AAX_VComponentDescriptor, [1006](#)
- AddCategory
 - AAX_IACFEffectorDescriptor, [701](#)
 - AAX_IEffectDescriptor, [866](#)
 - AAX_VEffectDescriptor, [1036](#)
- AddCategoryBypassParameter
 - AAX_IACFEffectorDescriptor, [701](#)
 - AAX_IEffectDescriptor, [866](#)
 - AAX_VEffectDescriptor, [1036](#)
- AddClock
 - AAX_IACFComponentDescriptor, [678](#)
 - AAX_IComponentDescriptor, [812](#)
 - AAX_VComponentDescriptor, [1004](#)
- AddComponent
 - AAX_IACFEffectorDescriptor, [700](#)
 - AAX_IEffectDescriptor, [865](#)
 - AAX_VEffectDescriptor, [1035](#)
- AddControlMIDINode
 - AAX_IACFEffectorDescriptor_V2, [703](#)
 - AAX_IEffectDescriptor, [868](#)
 - AAX_VEffectDescriptor, [1038](#)
- AddDataInPort
 - AAX_IACFComponentDescriptor, [678](#)
 - AAX_IComponentDescriptor, [813](#)
 - AAX_VComponentDescriptor, [1005](#)
- AddDmaInstance
 - AAX_IACFComponentDescriptor, [680](#)
 - AAX_IComponentDescriptor, [815](#)
 - AAX_VComponentDescriptor, [1008](#)
- AddDouble
 - AAX_CChunkDataParser, [426](#)
- AddEffect
 - AAX_IACFCollection, [672](#)
 - AAX_ICollection, [804](#)
 - AAX_VCollection, [996](#)
- AddFloat
 - AAX_CChunkDataParser, [426](#)
- AddInt16
 - AAX_CChunkDataParser, [426](#)
- AddInt32
 - AAX_CChunkDataParser, [426](#)
- Additional AAX features, [82](#)
- Additional Topics, [112](#)
- AddMeterDescription
 - AAX_IACFEffectorDescriptor, [703](#)
 - AAX_IEffectDescriptor, [868](#)
 - AAX_VEffectDescriptor, [1037](#)
- AddMeters
 - AAX_IACFComponentDescriptor, [683](#)
 - AAX_IComponentDescriptor, [816](#)
 - AAX_VComponentDescriptor, [1008](#)
- AddMIDINode
 - AAX_IACFComponentDescriptor, [681](#)
 - AAX_IComponentDescriptor, [817](#)
 - AAX_VComponentDescriptor, [1010](#)

- AddName
 - AAX_IACFEffectorDescriptor, 701
 - AAX_IEffectDescriptor, 865
 - AAX_VEffectDescriptor, 1035
- AddPackageName
 - AAX_IACFCollection, 673
 - AAX_ICollection, 805
 - AAX_VCollection, 997
- AddParameter
 - AAX_CParameterManager, 571
- AddPointerProperty
 - AAX_IPropertyMap, 932
 - AAX_VPropertyMap, 1062, 1063
- AddPrivateData
 - AAX_IACFComponentDescriptor, 680
 - AAX_IComponentDescriptor, 814
 - AAX_VComponentDescriptor, 1007
- AddProcessProc
 - AAX_IACFComponentDescriptor_V3, 685
 - AAX_IComponentDescriptor, 821
 - AAX_VComponentDescriptor, 1012
- AddProcessProc_Native
 - AAX_IACFComponentDescriptor, 681
 - AAX_IComponentDescriptor, 818, 822
 - AAX_VComponentDescriptor, 1011
- AddProcessProc_TI
 - AAX_IACFComponentDescriptor, 682
 - AAX_IComponentDescriptor, 820
 - AAX_VComponentDescriptor, 1012
- AddProcPtr
 - AAX_IACFEffectorDescriptor, 702
 - AAX_IEffectDescriptor, 867
 - AAX_VEffectDescriptor, 1036
- AddProperty
 - AAX_IACFPropertyMap, 770
 - AAX_IPropertyMap, 931
 - AAX_VPropertyMap, 1062
- AddProperty64
 - AAX_IACFPropertyMap_V3, 773
- AddPropertyWithIDArray
 - AAX_IACFPropertyMap_V2, 771
 - AAX_IPropertyMap, 933
 - AAX_VPropertyMap, 1064
- AddRef
 - IACFUnknown, 1096
- AddReservedField
 - AAX_IACFComponentDescriptor, 676
 - AAX_IComponentDescriptor, 817
 - AAX_VComponentDescriptor, 1002
- AddResourceInfo
 - AAX_IACFEffectorDescriptor, 702
 - AAX_IEffectDescriptor, 867
 - AAX_VEffectDescriptor, 1037
- AddResult
 - AAX_CTask, 655
 - AAX_IACFTask, 779
 - AAX_ITask, 946
 - AAX_VTask, 1070
- AddSampleRate
 - AAX_IACFComponentDescriptor, 677
 - AAX_IComponentDescriptor, 812
 - AAX_VComponentDescriptor, 1004
- AddShortenedName
 - AAX_CParameter< T >, 539
 - AAX_CStatelessParameter, 606
 - AAX_IParameter, 904
- AddShortenedStrings
 - AAX_CBinaryDisplayDelegate< T >, 420
 - AAX_CStateDisplayDelegate< T >, 601
- AddSideChainIn
 - AAX_IACFComponentDescriptor, 678
 - AAX_IComponentDescriptor, 813
 - AAX_VComponentDescriptor, 1005
- AddString
 - AAX_CChunkDataParser, 427
- AddSynchronizedParameter
 - AAX_CMonolithicParameters, 512
- AddTask
 - AAX_CTaskAgent, 658
 - AAX_IACFTaskAgent, 781
 - AAX_IHostTaskAgent, 883
 - AAX_VHostTaskAgent, 1046
- AddTemporaryData
 - AAX_IACFComponentDescriptor_V2, 684
 - AAX_IComponentDescriptor, 815
 - AAX_VComponentDescriptor, 1007
- alignFree
 - AAX, 385
- alignMalloc
 - AAX, 385
- AnalyzeAudio
 - AAX_CHostProcessor, 491
 - AAX_IACFHostProcessor, 745
- Any
 - AAX::Exception::Any, 1089, 1090
- Append
 - AAX_CString, 634
- AppendHex
 - AAX_CString, 634
- AppendNumber
 - AAX_CString, 634
- Assert
 - AAX_IACFHostServices, 751
- AsString
 - AAX, 377
- AsStringFourChar
 - AAX, 382
- AsStringIDTriad
 - AAX, 383
- AsStringInt32
 - AAX, 382
- AsStringMIDIStream_Debug
 - Other Extensions, 280
- AsStringPropertyValue
 - AAX, 382
- AsStringResult

- AAX, [383](#)
- AsStringStemChannel
 - AAX, [383](#)
- AsStringStemFormat
 - AAX, [383](#)
- AsStringSupportLevel
 - AAX, [384](#)
- AsStringUInt32
 - AAX, [382](#)
- AsUnknown
 - AAX_VPageTable, [1057](#)
- Automatable
 - AAX_CParameter< T >, [552](#)
 - AAX_CStatelessParameter, [607](#)
 - AAX_IParameter, [905](#)
- AutomationDelegate
 - AAX_CEffectParameters, [478](#)
- Auxiliary Output Stems, [83](#)
- Background processing callback, [84](#)
- Basic parameter update sequences, [127](#)
- Binary2String
 - AAX, [381](#)
- BoolToNormalized
 - AAX_CEffectParameters.h, [1137](#)
- BUILD_DATA_FAILED
 - AAX_ChunkDataParserDefs, [401](#)
- BuildChunkData
 - AAX_CEffectParameters, [480](#)
- Call
 - AAX_CPacketHandler< TWorker >, [528](#)
 - AAX_IPacketHandler, [889](#)
- CancelAllTasks
 - AAX_CTaskAgent, [658](#)
 - AAX_IACFTaskAgent, [781](#)
 - AAX_IHostTaskAgent, [883](#)
 - AAX_VHostTaskAgent, [1047](#)
- Canceled
 - Task agent interface, [106](#)
- Caseless_strcmp
 - AAX, [381](#)
- CBackgroundProc
 - AAX_Component< aContextType >, [522](#)
- cBigEndian
 - AAX, [393](#)
- cDefaultMasterBypassID
 - AAX_CEffectParameters.h, [1137](#)
- cDenormalAvoidanceOffset
 - AAX, [396](#)
- CeilLog2
 - AAX, [390](#)
- cFloatDenormalAvoidanceOffset
 - AAX, [396](#)
- cGiga
 - AAX, [396](#)
- cHalfPi
 - AAX, [394](#)
- Change Log, [331](#)
- Check
 - AAX_AggregateResult, [405](#)
- clnitialSeedValue
 - AAX, [397](#)
- ClnitPrivateDataProc
 - AAX_Component< aContextType >, [522](#)
- ClnstanceInitProc
 - AAX_Component< aContextType >, [522](#)
- cKilo
 - AAX, [396](#)
- ClampToZero
 - AAX, [387](#)
- Clear
 - AAX_AggregateResult, [405](#)
 - AAX_CAtomicQueue< T, S >, [414](#)
 - AAX_CChunkDataParser, [429](#)
 - AAX_CheckedResult, [485](#)
 - AAX_CString, [633](#)
 - AAX_CStringAbbreviations, [642](#)
 - AAX_IACFComponentDescriptor, [675](#)
 - AAX_IACFPageTable, [754](#)
 - AAX_IComponentDescriptor, [810](#)
 - AAX_IContainer, [824](#)
 - AAX_IPageTable, [891](#)
 - AAX_IPointerQueue< T >, [926](#)
 - AAX_VComponentDescriptor, [1002](#)
 - AAX_VPageTable, [1048](#)
 - AAX_VSessionDocument, [1066](#)
- ClearMappedParameter
 - AAX_IACFPageTable, [756](#)
 - AAX_IPageTable, [893](#)
 - AAX_VPageTable, [1051](#)
- ClearMappedParameterByID
 - AAX, [380](#)
- ClearMeterClipped
 - AAX_IACFController, [693](#)
 - AAX_IController, [834](#)
 - AAX_VController, [1026](#)
- ClearMeterPeakValue
 - AAX_IACFController, [692](#)
 - AAX_IController, [833](#)
 - AAX_VController, [1025](#)
- ClearNameVariationsForParameter
 - AAX_IACFPageTable_V2, [762](#)
 - AAX_IPageTable, [898](#)
 - AAX_VPageTable, [1056](#)
- ClearParameterNameVariations
 - AAX_IACFPageTable_V2, [762](#)
 - AAX_IPageTable, [898](#)
 - AAX_VPageTable, [1055](#)
- ClearShortenedNames
 - AAX_CParameter< T >, [540](#)
 - AAX_CStatelessParameter, [607](#)
 - AAX_IParameter, [904](#)
- cLittleEndian
 - AAX, [393](#)
- Clone
 - AAX_CBinaryDisplayDelegate< T >, [418](#)

- AAX_CBinaryTaperDelegate< T >, [421](#)
- AAX_CDecibelDisplayDelegateDecorator< T >, [431](#)
- AAX_CLinearTaperDelegate< T, RealPrecision >, [501](#)
- AAX_CLogTaperDelegate< T, RealPrecision >, [504](#)
- AAX_CNumberDisplayDelegate< T, Precision, SpaceAfter >, [518](#)
- AAX_CPacketHandler< TWorker >, [528](#)
- AAX_CParameterValue< T >, [576](#)
- AAX_CPercentDisplayDelegateDecorator< T >, [582](#)
- AAX_CPieceWiseLinearTaperDelegate< T, RealPrecision >, [586](#)
- AAX_CRangeTaperDelegate< T, RealPrecision >, [591](#)
- AAX_CStateDisplayDelegate< T >, [600](#)
- AAX_CStateTaperDelegate< T >, [626](#)
- AAX_CStringDisplayDelegate< T >, [650](#)
- AAX_CUnitDisplayDelegateDecorator< T >, [661](#)
- AAX_CUnitPrefixDisplayDelegateDecorator< T >, [665](#)
- AAX_IDisplayDelegate< T >, [846](#)
- AAX_IDisplayDelegateDecorator< T >, [851](#)
- AAX_IPacketHandler, [889](#)
- AAX_IPParameterValue, [922](#)
- AAX_ITaperDelegate< T >, [940](#)
- CloneValue
 - AAX_CParameter< T >, [538](#)
 - AAX_CStatelessParameter, [605](#)
 - AAX_IPParameter, [903](#)
- cMega
 - AAX, [396](#)
- cMicro
 - AAX, [395](#)
- cMilli
 - AAX, [395](#)
- cNano
 - AAX, [396](#)
- cNeg3dB
 - AAX, [395](#)
- cNeg6dB
 - AAX, [395](#)
- cNormalizeLongToAmplitudeOne
 - AAX, [395](#)
- cNormalizeLongToAmplitudeOneHalf
 - AAX, [395](#)
- CoarseSampleRate
 - AAX_SampleRateUtils.h, [1283](#)
- CoarseSampleRateFactor
 - AAX_SampleRateUtils.h, [1284](#)
- CoarseSampleRateIndex
 - AAX_SampleRateUtils.h, [1284](#)
- Compare
 - AAX_CStateDisplayDelegate< T >, [602](#)
- CompareActiveChunk
 - AAX_CEffectParameters, [470](#)
- AAX_IACFEffParameters, [733](#)
- cOneOverRootTwo
 - AAX, [394](#)
- ConstrainRealValue
 - AAX_CBinaryTaperDelegate< T >, [422](#)
 - AAX_CLinearTaperDelegate< T, RealPrecision >, [501](#)
 - AAX_CLogTaperDelegate< T, RealPrecision >, [505](#)
 - AAX_CPieceWiseLinearTaperDelegate< T, RealPrecision >, [587](#)
 - AAX_CRangeTaperDelegate< T, RealPrecision >, [591](#)
 - AAX_CStateTaperDelegate< T >, [627](#)
 - AAX_ITaperDelegate< T >, [941](#)
- Controller
 - AAX_CEffectDirectData, [437](#)
 - AAX_CEffectParameters, [477](#)
 - AAX_CHostProcessor, [496](#)
- CopyAttribute
 - IACFDefinition, [1094](#)
- CopyPageTable
 - AAX, [380](#)
- CopyTableForEffect
 - AAX_IACFPageTableController, [764](#)
- CopyTableForEffectFromFile
 - AAX_IACFPageTableController_V2, [766](#)
- CopyTableOfLayoutForEffect
 - AAX_IACFPageTableController, [765](#)
- CopyTableOfLayoutFromFile
 - AAX_IACFPageTableController_V2, [767](#)
- Core AAX Interface, [58](#)
- CPacketAllocator
 - AAX_Component< aContextType >, [522](#)
- cPi
 - AAX, [394](#)
- cPico
 - AAX, [396](#)
- cPos3dB
 - AAX, [394](#)
- cPos6dB
 - AAX, [395](#)
- cPreviewID
 - AAX_CEffectParameters.h, [1137](#)
- CProcessProc
 - AAX_Component< aContextType >, [522](#)
- cQuarterPi
 - AAX, [394](#)
- Create
 - AAX_VPropertyMap, [1061](#)
- CreateTableCopyForEffect
 - AAX_IController, [836](#)
 - AAX_VController, [1027](#)
- CreateTableCopyForEffectFromFile
 - AAX_IController, [839](#)
 - AAX_VController, [1028](#)
- CreateTableCopyForLayout
 - AAX_IController, [837](#)

- AAX_VController, [1027](#)
- CreateTableCopyForLayoutFromFile
 - AAX_IController, [839](#)
 - AAX_VController, [1029](#)
- CreateViewContainer
 - AAX_CEffectGUI, [444](#)
- CreateViewContents
 - AAX_CEffectGUI, [444](#)
- cRootTwo
 - AAX, [394](#)
- cSeedDivisor
 - AAX, [397](#)
- CString
 - AAX_CString, [637](#)
- cTwoPi
 - AAX, [394](#)
- Data
 - AAX_CArrayDataBuffer< D >, [409](#)
 - AAX_CArrayDataBufferOfType< T, D >, [412](#)
 - AAX_CStringDataBuffer, [645](#)
 - AAX_CStringDataBufferOfType< T >, [648](#)
 - AAX_IACFDataBuffer, [698](#)
 - AAX_IDataBufferWrapper, [842](#)
 - AAX_ISessionDocument::TempoMap, [1100](#)
 - AAX_VDataBufferWrapper, [1030](#)
 - AAX_VSessionDocument::VTempoMap, [1101](#)
- Data model interface, [75](#)
- Data model overrides interface, [77](#)
 - GetNumberOfChanges, [77](#)
- Data1
 - _acfUID, [403](#)
- Data2
 - _acfUID, [403](#)
- Data3
 - _acfUID, [403](#)
- Data4
 - _acfUID, [403](#)
- DataValue
 - AAX_CChunkDataParser::DataValue, [1091](#)
- DBToGain
 - AAX_CommonConversions.h, [1142](#)
- DeDenormal
 - AAX, [386](#)
- DeDenormalFine
 - AAX, [386](#)
- DEFAULT32BIT_TYPE_INCR
 - AAX_ChunkDataParserDefs, [401](#)
- DEFAULT32BIT_TYPE_SIZE
 - AAX_ChunkDataParserDefs, [400](#)
- Defaults
 - AAX_CParameter< T >, [534](#)
 - AAX_CParameterValue< T >, [574](#)
- DEFINE_AAX_ERROR_STRING
 - AAX_StringUtilities.hpp, [1290](#)
- DefineAttribute
 - IACFDefinition, [1093](#)
- DeleteViewContainer
 - AAX_CEffectGUI, [444](#)
- Desc
 - AAX::Exception::Any, [1090](#)
- Description callback, [59](#)
 - AAXRegisterPlugin, [67](#)
 - GetEffectDescriptions, [68](#)
- DescriptionHost
 - AAX_ICollection, [807](#)
 - AAX_VCollection, [999](#)
 - AAX_VDescriptionHost, [1032](#), [1033](#)
- DigiTrace Guide, [257](#)
- Direct data access interface, [87](#)
- Direct Memory Access, [86](#)
- Dispatch
 - AAX_CPacketDispatcher, [526](#)
- Display delegate decorators, [111](#)
- Display delegates, [109](#)
- DisplayDelegate
 - AAX_CParameter< T >, [558](#)
- Distributing Your AAX Plug-In, [285](#)
- DoMIDITransfers
 - AAX_CEffectParameters, [476](#)
 - AAX_IACFEffectParameters, [735](#)
- Done
 - Task agent interface, [106](#)
- DoTableLookupExtraFast
 - AAX_FastInterpolatedTableLookup< TFLOAT, DFLOAT >, [668](#)
- DoTableLookupExtraFastMulti
 - AAX_FastInterpolatedTableLookup< TFLOAT, DFLOAT >, [668](#)
- DOUBLE_STRING_IDENTIFIER
 - AAX_ChunkDataParserDefs, [399](#)
- DOUBLE_TYPE
 - AAX_ChunkDataParserDefs, [399](#)
- DOUBLE_TYPE_INCR
 - AAX_ChunkDataParserDefs, [399](#)
- DOUBLE_TYPE_SIZE
 - AAX_ChunkDataParserDefs, [399](#)
- DoubleTo32BitDSPCoef
 - AAX_CommonConversions.h, [1143](#)
- DoubleTo32BitDSPCoefRnd
 - AAX_CommonConversions.h, [1143](#)
- DoubleToDSPCoef
 - AAX_CommonConversions.h, [1142](#)
- DoubleToDSPCoefRnd
 - AAX_CommonConversions.h, [1144](#)
- DoubleToLong
 - AAX_CommonConversions.h, [1142](#)
- DoubleToLongControl
 - AAX_SliderConversions.h, [1287](#)
- DoubleToLongControlNonlinear
 - AAX_SliderConversions.h, [1287](#)
- Draw
 - AAX_CEffectGUI, [442](#)
 - AAX_IACFEffectGUI, [711](#)
- DSH Guide, [268](#)
- DSH_Guide.doxygen, [1322](#)
- DSPCoefToDouble

- AAX_CommonConversions.h, [1143](#)
- DTT Guide, [274](#)
- DTT_Guide.doxygen, [1322](#)
- DuplicatePropertyMap
 - AAX_IComponentDescriptor, [818](#)
 - AAX_VComponentDescriptor, [1011](#)
- e176400SampleRate
 - AAX, [377](#)
- e192000SampleRate
 - AAX, [377](#)
- e44100SampleRate
 - AAX, [377](#)
- e48000SampleRate
 - AAX, [377](#)
- e88200SampleRate
 - AAX, [377](#)
- e96000SampleRate
 - AAX, [377](#)
- EChannelModeData
 - AAX, [376](#)
- eChannelModeData_AllNotesOff
 - AAX, [376](#)
- eChannelModeData_AllSoundOff
 - AAX, [376](#)
- eChannelModeData_LocalControl
 - AAX, [376](#)
- eChannelModeData_OmniOff
 - AAX, [376](#)
- eChannelModeData_OmniOn
 - AAX, [376](#)
- eChannelModeData_PolyOff
 - AAX, [376](#)
- eChannelModeData_PolyOn
 - AAX, [376](#)
- eChannelModeData_ResetControllers
 - AAX, [376](#)
- EffectInit
 - AAX_CEffectParameters, [479](#)
- EffectParameters
 - AAX_CEffectDirectData, [437](#)
 - AAX_CHostProcessor, [497](#)
- EMode
 - AAX_IDma, [856](#)
- eMode_Burst
 - AAX_IDma, [856](#)
- eMode_Error
 - AAX_IDma, [856](#)
- eMode_Gather
 - AAX_IDma, [856](#)
- eMode_Scatter
 - AAX_IDma, [856](#)
- Empty
 - AAX_CString, [633](#)
 - AAX_IACFPageTable, [754](#)
 - AAX_IPageTable, [891](#)
 - AAX_VPageTable, [1049](#)
- ENDIANSWAP_H
 - AAX_EndianSwap.h, [1158](#)
- eParameterDefaultMaxIdentifierLength
 - AAX_CParameterValue< T >, [574](#)
- eParameterDefaultMaxIdentifierSize
 - AAX_CParameterValue< T >, [574](#)
- eParameterDefaultNumStepsContinuous
 - AAX_CParameter< T >, [535](#)
- eParameterDefaultNumStepsDiscrete
 - AAX_CParameter< T >, [535](#)
- eParameterTypeBool
 - AAX_CParameter< T >, [534](#)
- eParameterTypeCustom
 - AAX_CParameter< T >, [534](#)
- eParameterTypeFloat
 - AAX_CParameter< T >, [534](#)
- eParameterTypeInt32
 - AAX_CParameter< T >, [534](#)
- eParameterTypeUndefined
 - AAX_CParameter< T >, [534](#)
- EQ and Dynamics Curve Displays, [89](#)
 - AAX_ECType, [91](#)
 - AAX_eCurveType_Dynamics, [92](#)
 - AAX_eCurveType_EQ, [92](#)
 - AAX_eCurveType_None, [92](#)
 - AAX_eCurveType_Reduction, [92](#)
 - GetCurveData, [92](#)
 - GetCurveDataDisplayRange, [93](#)
 - GetCurveDataMeterIds, [93](#)
- Equals
 - AAX_CString, [637](#), [638](#)
- Erase
 - AAX_CString, [634](#)
- Error
 - Task agent interface, [106](#)
- ESampleRates
 - AAX, [377](#)
- ESpecialData
 - AAX, [376](#)
- eSpecialData_AccentedClick
 - AAX, [376](#)
- eSpecialData_UnaccentedClick
 - AAX, [376](#)
- ESRUtils
 - AAX_SampleRateUtils.h, [1283](#)
- eSRUtils_192kIndex
 - AAX_SampleRateUtils.h, [1283](#)
- eSRUtils_192kRangeCoarse
 - AAX_SampleRateUtils.h, [1283](#)
- eSRUtils_192kRangeMax
 - AAX_SampleRateUtils.h, [1283](#)
- eSRUtils_192kRangeMin
 - AAX_SampleRateUtils.h, [1283](#)
- eSRUtils_48kIndex
 - AAX_SampleRateUtils.h, [1283](#)
- eSRUtils_48kRangeCoarse
 - AAX_SampleRateUtils.h, [1283](#)
- eSRUtils_48kRangeMax
 - AAX_SampleRateUtils.h, [1283](#)
- eSRUtils_48kRangeMin

- AAX_SampleRateUtils.h, [1283](#)
- eSRUtils_96kIndex
 - AAX_SampleRateUtils.h, [1283](#)
- eSRUtils_96kRangeCoarse
 - AAX_SampleRateUtils.h, [1283](#)
- eSRUtils_96kRangeMax
 - AAX_SampleRateUtils.h, [1283](#)
- eSRUtils_96kRangeMin
 - AAX_SampleRateUtils.h, [1283](#)
- EState
 - AAX_IDma, [855](#)
- eState_Complete
 - AAX_IDma, [856](#)
- eState_Error
 - AAX_IDma, [856](#)
- eState_Init
 - AAX_IDma, [856](#)
- eState_Pending
 - AAX_IDma, [856](#)
- eState_Running
 - AAX_IDma, [856](#)
- EStatus
 - AAX_IContainer, [823](#)
- eStatus_NotInitialized
 - AAX_IContainer, [823](#)
- eStatus_Overflow
 - AAX_IContainer, [823](#)
- eStatus_Success
 - AAX_IContainer, [823](#)
- eStatus_Unavailable
 - AAX_IContainer, [823](#)
- eStatus_Unsupported
 - AAX_IContainer, [823](#)
- EStatusByte
 - AAX, [375](#)
- eStatusByte_ActiveSensing
 - AAX, [376](#)
- eStatusByte_Continue
 - AAX, [376](#)
- eStatusByte_MTCQuarterFrame
 - AAX, [376](#)
- eStatusByte_Reset
 - AAX, [376](#)
- eStatusByte_SongPosition
 - AAX, [376](#)
- eStatusByte_SongSelect
 - AAX, [376](#)
- eStatusByte_Start
 - AAX, [376](#)
- eStatusByte_Stop
 - AAX, [376](#)
- eStatusByte_SysExBegin
 - AAX, [375](#)
- eStatusByte_SysExEnd
 - AAX, [376](#)
- eStatusByte_TimingClock
 - AAX, [376](#)
- eStatusByte_TuneRequest
 - AAX, [376](#)
- EStatusNibble
 - AAX, [375](#)
- eStatusNibble_ChannelMode
 - AAX, [375](#)
- eStatusNibble_ChannelPressure
 - AAX, [375](#)
- eStatusNibble_ControlChange
 - AAX, [375](#)
- eStatusNibble_KeyPressure
 - AAX, [375](#)
- eStatusNibble_NoteOff
 - AAX, [375](#)
- eStatusNibble_NoteOn
 - AAX, [375](#)
- eStatusNibble_PitchBend
 - AAX, [375](#)
- eStatusNibble_ProgramChange
 - AAX, [375](#)
- eStatusNibble_SystemCommon
 - AAX, [375](#)
- eStatusNibble_SystemRealTime
 - AAX, [375](#)
- Example Plug-Ins, [351](#)
- Exception
 - AAX_CheckedResult, [484](#)
- Extensions, [279](#)
- fabs
 - AAX, [388](#)
- fabsf
 - AAX, [388](#)
- FastRndDbl2Int32
 - AAX, [391](#)
- FastRound2Int32
 - AAX, [390](#)
- FastRound2Int64
 - AAX, [392](#)
- FastTrunc2Int32
 - AAX, [391](#)
- fChunkID
 - AAX_SPlugInChunk, [983](#)
 - AAX_SPlugInChunkHeader, [986](#)
- fData
 - AAX_SPlugInChunk, [984](#)
- Fill
 - AAX, [387](#), [388](#)
- FilterDenormals
 - AAX, [386](#)
- FilterParameterIDOnSave
 - AAX_CEffectParameters, [480](#)
- FindDouble
 - AAX_CChunkDataParser, [427](#)
- FindFirst
 - AAX_CString, [636](#)
- FindFloat
 - AAX_CChunkDataParser, [427](#)
- FindInt16
 - AAX_CChunkDataParser, [427](#)

- FindInt32
 - AAX_CChunkDataParser, [427](#)
- FindLast
 - AAX_CString, [636](#), [637](#)
- FindName
 - AAX_CChunkDataParser, [429](#)
- FindParameterMappingsInPageTable
 - AAX, [380](#)
- FindString
 - AAX_CChunkDataParser, [428](#)
- FLOAT_STRING_IDENTIFIER
 - AAX_ChunkDataParserDefs, [399](#)
- FLOAT_TYPE
 - AAX_ChunkDataParserDefs, [399](#)
- fManufacturerID
 - AAX_SPlugInChunk, [983](#)
 - AAX_SPlugInChunkHeader, [986](#)
- fName
 - AAX_SPlugInChunk, [984](#)
 - AAX_SPlugInChunkHeader, [986](#)
- ForceAnalyze
 - AAX_IACFHostProcessorDelegate_V2, [749](#)
 - AAX_IHostProcessorDelegate, [879](#)
 - AAX_VHostProcessorDelegate, [1042](#)
- ForceProcess
 - AAX_IACFHostProcessorDelegate_V3, [750](#)
 - AAX_IHostProcessorDelegate, [879](#)
 - AAX_VHostProcessorDelegate, [1043](#)
- FormatResult
 - AAX::Exception::ResultError, [1098](#)
- fPlugInID
 - AAX_SPlugInChunk, [983](#)
 - AAX_SPlugInChunkHeader, [986](#)
- fProductID
 - AAX_SPlugInChunk, [983](#)
 - AAX_SPlugInChunkHeader, [986](#)
- fpt
 - AAX_CPacketHandler< TWorker >, [529](#)
- fptEx
 - AAX_CPacketHandler< TWorker >, [529](#)
- fSize
 - AAX_SPlugInChunk, [983](#)
 - AAX_SPlugInChunkHeader, [985](#)
- Function
 - AAX::Exception::Any, [1091](#)
- fVersion
 - AAX_SPlugInChunk, [983](#)
 - AAX_SPlugInChunkHeader, [986](#)
- GainToDB
 - AAX_CommonConversions.h, [1142](#)
- GenerateCoefficients
 - AAX_CEffectParameters, [467](#)
 - AAX_CMonolithicParameters, [513](#)
 - AAX_IACFEEffectParameters, [730](#)
- GenerateSingleValuePacket
 - AAX_CPacketDispatcher, [527](#)
- Get
 - AAX_CParameterValue< T >, [576](#)
 - AAX_CString, [632](#)
 - AAX_CStringAbbreviations, [642](#)
 - AAX_IString, [938](#)
 - SAutoArray< T >, [1099](#)
- GetArgumentOfType
 - AAX_CTask, [654](#)
 - AAX_IACFTask, [778](#)
 - AAX_ITask, [945](#)
 - AAX_VTask, [1069](#)
- GetAttributeInfo
 - IACFDefinition, [1094](#)
- GetAudio
 - AAX_CHostProcessor, [496](#)
 - AAX_IACFHostProcessorDelegate, [748](#)
 - AAX_IHostProcessorDelegate, [878](#)
 - AAX_VHostProcessorDelegate, [1042](#)
- GetBarBeatPosition
 - AAX_IACFTransport, [785](#)
 - AAX_ITransport, [953](#)
 - AAX_VTransport, [1077](#)
- GetBaseOffset
 - AAX_IDma, [863](#)
- GetBoolFromNormalizedValue
 - AAX_CParameter< T >, [548](#), [563](#)
 - AAX_CStatelessParameter, [615](#)
 - AAX_IParameter, [911](#)
- GetBurstLength
 - AAX_IDma, [859](#)
- GetChunk
 - AAX_CEffectParameters, [469](#)
 - AAX_IACFEEffectParameters, [732](#)
- GetChunkData
 - AAX_CChunkDataParser, [428](#)
- GetChunkDataSize
 - AAX_CChunkDataParser, [428](#)
- GetChunkIDFromIndex
 - AAX_CEffectParameters, [469](#)
 - AAX_IACFEEffectParameters, [731](#)
- GetChunkSize
 - AAX_CEffectParameters, [469](#)
 - AAX_IACFEEffectParameters, [731](#)
- GetChunkVersion
 - AAX_CChunkDataParser, [428](#)
- GetClipNameSuffix
 - AAX_CHostProcessor, [493](#)
 - AAX_IACFHostProcessor_V2, [747](#)
- GetCoefficient
 - AAX_Map, [964](#)
- GetController
 - AAX_CEffectGUI, [445](#)
 - AAX_CSessionDocumentClient, [596](#), [597](#)
 - AAX_CTaskAgent, [659](#)
- GetCStringOfLength
 - AAX, [381](#)
- GetCurrentAutomationTimestamp
 - AAX_IACFController_V2, [695](#)
 - AAX_IController, [835](#)
 - AAX_VController, [1020](#)

- GetCurrentLoopPosition
 - AAX_IACFTransport, [784](#)
 - AAX_ITransport, [951](#)
 - AAX_VTransport, [1075](#)
- GetCurrentMeter
 - AAX_IACFTransport, [783](#)
 - AAX_ITransport, [950](#)
 - AAX_VTransport, [1073](#)
- GetCurrentMeterValue
 - AAX_IACFController, [692](#)
 - AAX_IController, [833](#)
 - AAX_VController, [1024](#)
- GetCurrentNativeSampleLocation
 - AAX_IACFTransport, [784](#)
 - AAX_ITransport, [952](#)
 - AAX_VTransport, [1076](#)
- GetCurrentTempo
 - AAX_IACFTransport, [782](#)
 - AAX_ITransport, [950](#)
 - AAX_VTransport, [1073](#)
- GetCurrentTickPosition
 - AAX_IACFTransport, [783](#)
 - AAX_ITransport, [951](#)
 - AAX_VTransport, [1075](#)
- GetCurrentTicksPerBeat
 - AAX_IACFTransport, [786](#)
 - AAX_ITransport, [954](#)
 - AAX_VTransport, [1077](#)
- GetCurveData
 - AAX_CEffectParameters, [472](#)
 - EQ and Dynamics Curve Displays, [92](#)
- GetCurveDataDisplayRange
 - AAX_CEffectParameters, [473](#)
 - EQ and Dynamics Curve Displays, [93](#)
- GetCurveDataMeterIds
 - AAX_CEffectParameters, [473](#)
 - EQ and Dynamics Curve Displays, [93](#)
- GetCustomData
 - AAX_CEffectParameters, [475](#)
 - AAX_IACFEffEffectParameters, [734](#)
- GetCustomLabel
 - AAX_CEffectGUI, [443](#)
 - AAX_IACFEffEffectGUI, [712](#)
- GetCustomTickPosition
 - AAX_IACFTransport, [785](#)
 - AAX_ITransport, [952](#)
 - AAX_VTransport, [1076](#)
- GetCycleCount
 - AAX_IACFController, [689](#)
 - AAX_IController, [828](#)
 - AAX_VController, [1019](#)
- GetDefaultValue
 - AAX_CParameter< T >, [558](#)
- GetDmaMode
 - AAX_IDma, [858](#)
- GetDmaState
 - AAX_IDma, [857](#)
- GetDocumentData
 - AAX_IACFSessionDocument, [774](#)
 - AAX_ISessionDocument, [935](#)
 - AAX_VSessionDocument, [1066](#)
- GetDoubleFromNormalizedValue
 - AAX_CParameter< T >, [549](#), [564](#)
 - AAX_CStatelessParameter, [616](#)
 - AAX_IParameter, [913](#)
- GetDst
 - AAX_IDma, [859](#)
- GetDstEnd
 - AAX_CHostProcessor, [495](#)
- GetDstStart
 - AAX_CHostProcessor, [495](#)
- GetEffectDescriptions
 - Description callback, [68](#)
- GetEffectID
 - AAX_IACFController, [687](#)
 - AAX_IController, [827](#)
 - AAX_VController, [1016](#)
- GetEffectParameters
 - AAX_CEffectGUI, [445](#)
 - AAX_CHostProcessor, [493](#)
 - AAX_CSessionDocumentClient, [597](#)
 - AAX_CTaskAgent, [659](#)
- GetFastInt32RPDF
 - AAX, [392](#)
- GetFastRPDFWithAmplitudeOne
 - AAX, [393](#)
- GetFeetFramesInfo
 - AAX_IACFTransport_V2, [787](#)
 - AAX_ITransport, [955](#)
 - AAX_VTransport, [1078](#)
- GetFifoBuffer
 - AAX_IDma, [861](#)
- GetFifoSize
 - AAX_IDma, [863](#)
- GetFirstX
 - AAX_Map, [965](#)
- GetFirstY
 - AAX_Map, [965](#)
- GetFloatFromNormalizedValue
 - AAX_CParameter< T >, [549](#), [563](#)
 - AAX_CStatelessParameter, [615](#)
 - AAX_IParameter, [912](#)
- GetHDTTimeCodeInfo
 - AAX_IACFTransport_V3, [789](#)
 - AAX_ITransport, [955](#)
 - AAX_VTransport, [1079](#)
- GetHostName
 - AAX_IACFController_V2, [696](#)
 - AAX_IController, [835](#)
 - AAX_VController, [1021](#)
- GetHostProcessorDelegate
 - AAX_CHostProcessor, [493](#), [494](#)
- GetHostVersion
 - AAX_ICollection, [806](#)
 - AAX_VCollection, [998](#)
- GetHybridSignalLatency

- AAX_IACFController_V2, [695](#)
- AAX_VController, [1018](#)
- Hybrid Processing architecture, [96](#)
- GetID
 - AAX_CPacket, [524](#)
- GetInputRange
 - AAX_CHostProcessor, [494](#)
- GetInputStemFormat
 - AAX_IACFController, [688](#)
 - AAX_IController, [827](#)
 - AAX_VController, [1017](#)
- GetInt32FromNormalizedValue
 - AAX_CParameter< T >, [548](#), [563](#)
 - AAX_CStatelessParameter, [615](#)
 - AAX_IParameter, [912](#)
- GetInt32RPDF
 - AAX, [392](#)
- GetIsAudioSuite
 - AAX_IACFController_V3, [697](#)
 - AAX_IController, [836](#)
 - AAX_VController, [1019](#)
- GetIsUnknown
 - AAX_IPropertyMap, [934](#)
 - AAX_VCollection, [1000](#)
 - AAX_VComponentDescriptor, [1014](#)
 - AAX_VEffectDescriptor, [1038](#)
 - AAX_VPropertyMap, [1065](#)
- GetKeySignature
 - AAX_IACFTransport_V5, [790](#)
 - AAX_ITransport, [957](#)
 - AAX_VTransport, [1080](#)
- GetLastX
 - AAX_Map, [965](#)
- GetLastY
 - AAX_Map, [965](#)
- GetLinearBuffer
 - AAX_IDma, [861](#)
- GetLocation
 - AAX_CHostProcessor, [494](#)
- getLowestSampleRateInMask
 - AAX_Enums.h, [1205](#)
- GetMappedParameterID
 - AAX_IACFPageTable, [757](#)
 - AAX_IPageTable, [894](#)
 - AAX_VPageTable, [1051](#)
- getMaskForSampleRate
 - AAX_Enums.h, [1205](#)
- GetMasterBypassParameter
 - AAX_CEffectParameters, [455](#)
 - AAX_IACFEffectParameters, [718](#)
- GetMaximumValue
 - AAX_CBinaryTaperDelegate< T >, [421](#)
 - AAX_CLinearTaperDelegate< T, RealPrecision >, [501](#)
 - AAX_CLogTaperDelegate< T, RealPrecision >, [505](#)
 - AAX_CPieceWiseLinearTaperDelegate< T, RealPrecision >, [586](#)
- AAX_CRangeTaperDelegate< T, RealPrecision >, [591](#)
- AAX_CStateTaperDelegate< T >, [627](#)
- AAX_ITaperDelegate< T >, [940](#)
- GetMaxMinusMin
 - AAX_FastInterpolatedTableLookup< TFLOAT, DFLOAT >, [669](#)
- GetMeterClipped
 - AAX_IACFController, [693](#)
 - AAX_IController, [834](#)
 - AAX_VController, [1025](#)
- GetMeterCount
 - AAX_IACFController, [693](#)
 - AAX_IController, [834](#)
 - AAX_VController, [1026](#)
- GetMeterPeakValue
 - AAX_IACFController, [692](#)
 - AAX_IController, [833](#)
 - AAX_VController, [1025](#)
- GetMin
 - AAX_FastInterpolatedTableLookup< TFLOAT, DFLOAT >, [668](#)
- GetMinimumValue
 - AAX_CBinaryTaperDelegate< T >, [422](#)
 - AAX_CLinearTaperDelegate< T, RealPrecision >, [501](#)
 - AAX_CLogTaperDelegate< T, RealPrecision >, [505](#)
 - AAX_CPieceWiseLinearTaperDelegate< T, RealPrecision >, [586](#)
 - AAX_CRangeTaperDelegate< T, RealPrecision >, [591](#)
 - AAX_CStateTaperDelegate< T >, [627](#)
 - AAX_ITaperDelegate< T >, [941](#)
- GetModifiers
 - AAX_IACFViewContainer, [793](#)
 - AAX_IViewContainer, [959](#)
 - AAX_VViewContainer, [1083](#)
- GetNameVariationParameterIDAtIndex
 - AAX_IACFPageTable_V2, [759](#)
 - AAX_IPageTable, [895](#)
 - AAX_VPageTable, [1053](#)
- GetNextMIDIpacket
 - AAX_IACFController, [693](#)
 - AAX_IController, [835](#)
 - AAX_VController, [1026](#)
- GetNodeBuffer
 - AAX_IMIDINode, [887](#)
- GetNormalizedDefaultValue
 - AAX_CParameter< T >, [540](#)
 - AAX_CStatelessParameter, [609](#)
 - AAX_IParameter, [906](#)
- GetNormalizedValue
 - AAX_CParameter< T >, [541](#)
 - AAX_CStatelessParameter, [609](#)
 - AAX_IParameter, [906](#)
- GetNormalizedValueFromBool
 - AAX_CParameter< T >, [546](#), [561](#)

- AAX_CStatelessParameter, 612
- AAX_IParameter, 909
- GetNormalizedValueFromDouble
 - AAX_CParameter< T >, 547, 562
 - AAX_CStatelessParameter, 614
 - AAX_IParameter, 911
- GetNormalizedValueFromFloat
 - AAX_CParameter< T >, 547, 562
 - AAX_CStatelessParameter, 613
 - AAX_IParameter, 910
- GetNormalizedValueFromInt32
 - AAX_CParameter< T >, 546, 561
 - AAX_CStatelessParameter, 613
 - AAX_IParameter, 910
- GetNormalizedValueFromStep
 - AAX_CParameter< T >, 542
 - AAX_CStatelessParameter, 611
 - AAX_IParameter, 908
- GetNormalizedValueFromString
 - AAX_CParameter< T >, 548
 - AAX_CStatelessParameter, 614
 - AAX_IParameter, 911
- GetNumberOfChanges
 - AAX_CEffectParameters, 471
 - Data model overrides interface, 77
- GetNumberOfChunks
 - AAX_CEffectParameters, 468
 - AAX_IACFEEffectParameters, 731
- GetNumberOfParameters
 - AAX_CEffectParameters, 455
 - AAX_IACFEEffectParameters, 718
- GetNumberOfSteps
 - AAX_CParameter< T >, 541
 - AAX_CStatelessParameter, 610
 - AAX_IParameter, 907
- GetNumBursts
 - AAX_IDma, 860
- GetNumMappedParameterIDs
 - AAX_IACFPageTable, 756
 - AAX_IPageTable, 893
 - AAX_VPageTable, 1050
- GetNumNameVariationsForParameter
 - AAX_IACFPageTable_V2, 759
 - AAX_IPageTable, 896
 - AAX_VPageTable, 1053
- GetNumOffsets
 - AAX_IDma, 862
- GetNumPages
 - AAX_IACFPageTable, 755
 - AAX_IPageTable, 891
 - AAX_VPageTable, 1049
- GetNumParametersWithNameVariations
 - AAX_IACFPageTable_V2, 758
 - AAX_IPageTable, 895
 - AAX_VPageTable, 1052
- GetOffsetTable
 - AAX_IDma, 862
- GetOrientation
 - AAX_CParameter< T >, 544
 - AAX_CStatelessParameter, 623
 - AAX_IParameter, 920
- GetOutputRange
 - AAX_CHostProcessor, 494
- GetOutputStemFormat
 - AAX_IACFController, 688
 - AAX_IController, 827
 - AAX_VController, 1017
- GetParameter
 - AAX_CEffectParameters, 459
 - AAX_CParameterManager, 570
 - AAX_IACFEEffectParameters, 722
- GetParameterByID
 - AAX_CParameterManager, 569
- GetParameterByName
 - AAX_CParameterManager, 569, 570
- GetParameterDefaultNormalizedValue
 - AAX_CEffectParameters, 457
 - AAX_IACFEEffectParameters, 720
- GetParameterIDFromIndex
 - AAX_CEffectParameters, 461
 - AAX_IACFEEffectParameters, 723
- GetParameterIndex
 - AAX_CEffectParameters, 459
 - AAX_CParameterManager, 571
 - AAX_IACFEEffectParameters, 723
- GetParameterIsAutomatable
 - AAX_CEffectParameters, 455
 - AAX_IACFEEffectParameters, 719
- GetParameterName
 - AAX_CEffectParameters, 456
 - AAX_IACFEEffectParameters, 720
- GetParameterNameOfLength
 - AAX_CEffectParameters, 456
 - AAX_IACFEEffectParameters, 720
- GetParameterNameVariationAtIndex
 - AAX_IACFPageTable_V2, 760
 - AAX_IPageTable, 896
 - AAX_VPageTable, 1054
- GetParameterNameVariationOfLength
 - AAX_IACFPageTable_V2, 761
 - AAX_IPageTable, 897
 - AAX_VPageTable, 1055
- GetParameterNormalizedValue
 - AAX_CEffectParameters, 463
 - AAX_IACFEEffectParameters, 726
- GetParameterNumberOfSteps
 - AAX_CEffectParameters, 456
 - AAX_IACFEEffectParameters, 719
- GetParameterOrientation
 - AAX_CEffectParameters, 458
 - AAX_IACFEEffectParameters, 722
- GetParameterStringFromValue
 - AAX_CEffectParameters, 462
 - AAX_IACFEEffectParameters, 725
- GetParameterType
 - AAX_CEffectParameters, 458

- AAX_IACEffectParameters, 721
- GetParameterValueFromString
 - AAX_CEffectParameters, 462
 - AAX_IACEffectParameters, 724
- GetParameterValueInfo
 - AAX_CEffectParameters, 461
 - AAX_IACEffectParameters, 724
- GetParameterValueString
 - AAX_CEffectParameters, 463
 - AAX_IACEffectParameters, 725
- GetPathToPlugInBundle
 - Other Extensions, 281
- GetPlugInTargetPlatform
 - AAX_IACFController_V3, 697
 - AAX_IController, 836
 - AAX_VController, 1019
- GetPointerProperty
 - AAX_IPropertyMap, 931
 - AAX_VPropertyMap, 1062
- GetProgress
 - AAX_CTask, 654
 - AAX_IACFTask, 778
 - AAX_ITask, 946
 - AAX_VTask, 1070
- GetProperty
 - AAX_IACFPropertyMap, 770
 - AAX_IPropertyMap, 931
 - AAX_VPropertyMap, 1061
- GetProperty64
 - AAX_IACFPropertyMap_V3, 773
- GetPropertyWithIDArray
 - AAX_IACFPropertyMap_V2, 772
 - AAX_IPropertyMap, 933
 - AAX_VPropertyMap, 1064
- GetPtr
 - AAX_CPacket, 523, 524
 - AAX_IACFViewContainer, 793
 - AAX_IViewContainer, 959
 - AAX_VViewContainer, 1083
- GetRPDFWithAmplitudeOne
 - AAX, 393
- GetRPDFWithAmplitudeOneHalf
 - AAX, 393
- GetSampleRate
 - AAX_IACFController, 687
 - AAX_IController, 827
 - AAX_VController, 1017
- GetSessionDocument
 - AAX_CSessionDocumentClient, 597
- GetSideChainInputNum
 - AAX_CHostProcessor, 496
 - AAX_IACFHostProcessorDelegate, 749
 - AAX_IHostProcessorDelegate, 879
 - AAX_VHostProcessorDelegate, 1042
- GetSignalLatency
 - AAX_IACFController, 688
 - AAX_IController, 828
 - AAX_VController, 1018
- GetSize
 - AAX_CPacket, 524
 - AAX_Map, 966
- GetSrc
 - AAX_IDma, 858
- GetSrcEnd
 - AAX_CHostProcessor, 494
- GetSrcStart
 - AAX_CHostProcessor, 494
- GetStepValue
 - AAX_CParameter< T >, 542
 - AAX_CStatelessParameter, 610
 - AAX_IParameter, 907
- GetStepValueFromNormalizedValue
 - AAX_CParameter< T >, 542
 - AAX_CStatelessParameter, 611
 - AAX_IParameter, 908
- GetStringFromNormalizedValue
 - AAX_CParameter< T >, 550
 - AAX_CStatelessParameter, 616, 617
 - AAX_IParameter, 913, 914
- GetTempoMap
 - AAX_ISessionDocument, 935
 - AAX_VSessionDocument, 1066
- GetTicksPerQuarter
 - AAX_IACFTransport, 785
 - AAX_ITransport, 953
 - AAX_VTransport, 1077
- GetTimeCodeInfo
 - AAX_IACFTransport_V2, 787
 - AAX_ITransport, 954
 - AAX_VTransport, 1078
- GetTimelineSelectionEndPosition
 - AAX_IACFTransport_V4, 789
 - AAX_ITransport, 956
 - AAX_VTransport, 1080
- GetTimelineSelectionStartPosition
 - AAX_IACFTransport_V2, 787
 - AAX_ITransport, 954
 - AAX_VTransport, 1078
- Getting Started with AAX, 50
- GetTODLocation
 - AAX_IACFController, 689
 - AAX_IController, 829
 - AAX_VController, 1020
- GetTouchState
 - AAX_IACFAutomationDelegate, 671
 - AAX_IAutomationDelegate, 802
 - AAX_VAutomationDelegate, 994
- GetTPDFWithAmplitudeOne
 - AAX, 393
- GetTransferSize
 - AAX_IDma, 860
- GetTransport
 - AAX_IMIDINode, 888
- GetType
 - AAX_CParameter< T >, 543
 - AAX_CStatelessParameter, 622

- AAX_CTask, [653](#)
- AAX_IACFTask, [777](#)
- AAX_IACFViewContainer, [793](#)
- AAX_IParameter, [919](#)
- AAX_ITask, [945](#)
- AAX_IViewContainer, [959](#)
- AAX_VTask, [1069](#)
- AAX_VViewContainer, [1083](#)
- GetUnknown
 - AAX_VAutomationDelegate, [991](#)
- GetUpperBoundIndex
 - AAX_Map, [965](#)
- GetValue
 - AAX_CParameter< T >, [557](#)
- GetValueAsBool
 - AAX_CParameter< T >, [552](#)
 - AAX_CParameterValue< T >, [576](#), [579](#)
 - AAX_CStatelessParameter, [618](#)
 - AAX_IParameter, [915](#)
 - AAX_IParameterValue, [923](#)
- GetValueAsDouble
 - AAX_CParameter< T >, [554](#)
 - AAX_CParameterValue< T >, [578](#), [580](#)
 - AAX_CStatelessParameter, [619](#)
 - AAX_IParameter, [916](#)
 - AAX_IParameterValue, [924](#)
- GetValueAsFloat
 - AAX_CParameter< T >, [553](#)
 - AAX_CParameterValue< T >, [577](#), [579](#)
 - AAX_CStatelessParameter, [619](#)
 - AAX_IParameter, [915](#)
 - AAX_IParameterValue, [924](#)
- GetValueAsInt32
 - AAX_CParameter< T >, [553](#)
 - AAX_CParameterValue< T >, [577](#), [579](#)
 - AAX_CStatelessParameter, [618](#)
 - AAX_IParameter, [915](#)
 - AAX_IParameterValue, [923](#)
- GetValueAsString
 - AAX_CParameter< T >, [554](#), [558](#)
 - AAX_CParameterValue< T >, [578](#), [580](#)
 - AAX_CStatelessParameter, [619](#)
 - AAX_IParameter, [916](#)
 - AAX_IParameterValue, [924](#)
- GetValueString
 - AAX_CParameter< T >, [545](#)
 - AAX_CStatelessParameter, [611](#), [612](#)
 - AAX_IParameter, [908](#), [909](#)
- GetViewContainer
 - AAX_CEffectGUI, [446](#)
- GetViewContainerPtr
 - AAX_CEffectGUI, [446](#)
- GetViewContainerType
 - AAX_CEffectGUI, [446](#)
- GetViewSize
 - AAX_CEffectGUI, [442](#)
 - AAX_IACFEffEffectGUI, [710](#)
- GetX
 - AAX_Map, [965](#)
- GetY
 - AAX_Map, [965](#)
- GUI Extensions, [279](#)
- GUI interface, [78](#)
- HandleAssertFailure
 - AAX_CHostServices, [498](#)
 - AAX_IACFHostServices_V3, [753](#)
 - AAX_IHostServices, [881](#)
 - AAX_VHostServices, [1044](#)
- HandleMultipleParametersMouseDown
 - AAX_IACFViewContainer_V2, [796](#)
 - AAX_IViewContainer, [962](#)
 - AAX_VViewContainer, [1087](#)
- HandleMultipleParametersMouseDrag
 - AAX_IACFViewContainer_V2, [796](#)
 - AAX_IViewContainer, [962](#)
 - AAX_VViewContainer, [1087](#)
- HandleMultipleParametersMouseUp
 - AAX_IACFViewContainer_V2, [797](#)
 - AAX_IViewContainer, [963](#)
 - AAX_VViewContainer, [1088](#)
- HandleParameterMouseDown
 - AAX_IACFViewContainer, [794](#)
 - AAX_IViewContainer, [960](#)
 - AAX_VViewContainer, [1084](#)
- HandleParameterMouseDrag
 - AAX_IACFViewContainer, [794](#)
 - AAX_IViewContainer, [960](#)
 - AAX_VViewContainer, [1084](#)
- HandleParameterMouseEnter
 - AAX_IACFViewContainer_V3, [798](#)
 - AAX_IViewContainer, [961](#)
 - AAX_VViewContainer, [1086](#)
- HandleParameterMouseExit
 - AAX_IACFViewContainer_V3, [798](#)
 - AAX_IViewContainer, [962](#)
 - AAX_VViewContainer, [1086](#)
- HandleParameterMouseUp
 - AAX_IACFViewContainer, [795](#)
 - AAX_IViewContainer, [961](#)
 - AAX_VViewContainer, [1086](#)
- HDX DSP Guide, [180](#)
- HEADER_SIZE
 - AAX_ChunkDataParserDefs, [401](#)
- height
 - AAX_Rect, [968](#)
- horz
 - AAX_Point, [967](#)
- Host Support, [291](#)
- HostDefinition
 - AAX_ICollection, [807](#)
 - AAX_VCollection, [999](#)
 - AAX_VDescriptionHost, [1033](#)
- HostProcessorDelegate
 - AAX_CHostProcessor, [496](#), [497](#)
- Hybrid Processing architecture, [94](#)
 - GetHybridSignalLatency, [96](#)

- RenderAudio_Hybrid, [96](#)
- IACFDefinition, [1092](#)
 - CopyAttribute, [1094](#)
 - DefineAttribute, [1093](#)
 - GetAttributeInfo, [1094](#)
- IACFUnknown, [1095](#)
 - AddRef, [1096](#)
 - QueryInterface, [1095](#)
 - Release, [1096](#)
- ID
 - AAX_IFeatureInfo, [875](#)
 - AAX_VFeatureInfo, [1040](#)
- Identifier
 - AAX_CParameter< T >, [538](#)
 - AAX_CParameterValue< T >, [576](#)
 - AAX_CStatelessParameter, [605](#)
 - AAX_IPParameter, [903](#)
 - AAX_IPParameterValue, [922](#)
- IID_IAAXAutomationDelegateV1
 - AAX_UIDs.h, [1299](#)
- IID_IAAXCollectionV1
 - AAX_UIDs.h, [1296](#)
- IID_IAAXComponentDescriptorV1
 - AAX_UIDs.h, [1297](#)
- IID_IAAXComponentDescriptorV2
 - AAX_UIDs.h, [1297](#)
- IID_IAAXComponentDescriptorV3
 - AAX_UIDs.h, [1297](#)
- IID_IAAXControllerV1
 - AAX_UIDs.h, [1299](#)
- IID_IAAXControllerV2
 - AAX_UIDs.h, [1299](#)
- IID_IAAXControllerV3
 - AAX_UIDs.h, [1299](#)
- IID_IAAXDataBufferV1
 - AAX_UIDs.h, [1307](#)
- IID_IAAXDescriptionHostV1
 - AAX_UIDs.h, [1303](#)
- IID_IAAXEffectDescriptorV1
 - AAX_UIDs.h, [1296](#)
- IID_IAAXEffectDescriptorV2
 - AAX_UIDs.h, [1296](#)
- IID_IAAXEffectDirectDataV1
 - AAX_UIDs.h, [1306](#)
- IID_IAAXEffectDirectDataV2
 - AAX_UIDs.h, [1306](#)
- IID_IAAXEffectGUIV1
 - AAX_UIDs.h, [1306](#)
- IID_IAAXEffectParametersV1
 - AAX_UIDs.h, [1304](#)
- IID_IAAXEffectParametersV2
 - AAX_UIDs.h, [1305](#)
- IID_IAAXEffectParametersV3
 - AAX_UIDs.h, [1305](#)
- IID_IAAXEffectParametersV4
 - AAX_UIDs.h, [1305](#)
- IID_IAAXFeatureInfoV1
 - AAX_UIDs.h, [1303](#)
- IID_IAAXHostProcessorDelegateV1
 - AAX_UIDs.h, [1298](#)
- IID_IAAXHostProcessorDelegateV2
 - AAX_UIDs.h, [1298](#)
- IID_IAAXHostProcessorDelegateV3
 - AAX_UIDs.h, [1298](#)
- IID_IAAXHostProcessorV1
 - AAX_UIDs.h, [1305](#)
- IID_IAAXHostProcessorV2
 - AAX_UIDs.h, [1305](#)
- IID_IAAXHostServicesV1
 - AAX_UIDs.h, [1295](#)
- IID_IAAXHostServicesV2
 - AAX_UIDs.h, [1295](#)
- IID_IAAXHostServicesV3
 - AAX_UIDs.h, [1296](#)
- IID_IAAXPageTableController
 - AAX_UIDs.h, [1300](#)
- IID_IAAXPageTableControllerV2
 - AAX_UIDs.h, [1300](#)
- IID_IAAXPageTableV1
 - AAX_UIDs.h, [1303](#)
- IID_IAAXPageTableV2
 - AAX_UIDs.h, [1303](#)
- IID_IAAXPrivateDataAccessV1
 - AAX_UIDs.h, [1300](#)
- IID_IAAXPropertyMapV1
 - AAX_UIDs.h, [1297](#)
- IID_IAAXPropertyMapV2
 - AAX_UIDs.h, [1298](#)
- IID_IAAXPropertyMapV3
 - AAX_UIDs.h, [1298](#)
- IID_IAAXSessionDocumentClientV1
 - AAX_UIDs.h, [1307](#)
- IID_IAAXSessionDocumentV1
 - AAX_UIDs.h, [1304](#)
- IID_IAAXTaskAgentV1
 - AAX_UIDs.h, [1307](#)
- IID_IAAXTaskV1
 - AAX_UIDs.h, [1304](#)
- IID_IAAXTransportControlV1
 - AAX_UIDs.h, [1302](#)
- IID_IAAXTransportV1
 - AAX_UIDs.h, [1301](#)
- IID_IAAXTransportV2
 - AAX_UIDs.h, [1301](#)
- IID_IAAXTransportV3
 - AAX_UIDs.h, [1302](#)
- IID_IAAXTransportV4
 - AAX_UIDs.h, [1302](#)
- IID_IAAXTransportV5
 - AAX_UIDs.h, [1302](#)
- IID_IAAXViewContainerV1
 - AAX_UIDs.h, [1301](#)
- IID_IAAXViewContainerV2
 - AAX_UIDs.h, [1301](#)
- IID_IAAXViewContainerV3
 - AAX_UIDs.h, [1301](#)

- Initialize
 - AAX_CEffectDirectData, [435](#)
 - AAX_CEffectGUI, [440](#)
 - AAX_CEffectParameters, [453](#)
 - AAX_CHostProcessor, [488](#)
 - AAX_CPacketDispatcher, [525](#)
 - AAX_CParameterManager, [568](#)
 - AAX_CSessionDocumentClient, [594](#)
 - AAX_CTaskAgent, [657](#)
 - AAX_IACFEfffectDirectData, [705](#)
 - AAX_IACFEfffectGUI, [709](#)
 - AAX_IACFEfffectParameters, [717](#)
 - AAX_IACFHostProcessor, [742](#)
 - AAX_IACFSessionDocumentClient, [775](#)
 - AAX_IACFTaskAgent, [780](#)
 - AAX_IHostTaskAgent, [883](#)
 - AAX_VHostTaskAgent, [1046](#)
- Initialize_PrivateDataAccess
 - AAX_CEffectDirectData, [437](#)
- InitOutputBounds
 - AAX_CHostProcessor, [489](#)
 - AAX_IACFHostProcessor, [742](#)
- Insert
 - AAX_CString, [635](#)
- InsertHex
 - AAX_CString, [635](#)
- InsertNumber
 - AAX_CString, [635](#)
- InsertPage
 - AAX_IACFPageTable, [755](#)
 - AAX_IPageTable, [892](#)
 - AAX_VPageTable, [1049](#)
- Int32ToNormalized
 - AAX_CEffectParameters.h, [1136](#)
- IsAccentedClick
 - AAX, [378](#)
- IsAllNotesOff
 - AAX, [378](#)
- IsASCII
 - AAX, [381](#)
- IsAvidNotification
 - AAX, [385](#)
- IsClick
 - AAX, [379](#)
- IsDirty
 - AAX_CPacket, [524](#)
- IsEffectIDEqual
 - AAX, [385](#)
- IsEmpty
 - AAX_CChunkDataParser, [428](#)
- IsFourCharASCII
 - AAX, [382](#)
- IsMetronomeEnabled
 - AAX_IACFTransport_V2, [788](#)
 - AAX_ITransport, [955](#)
 - AAX_VTransport, [1079](#)
- IsNoteOff
 - AAX, [378](#)
- IsNoteOn
 - AAX, [377](#)
- IsParameterIDEqual
 - AAX, [385](#)
- IsParameterLinkReady
 - AAX_CEffectParameters, [479](#)
- IsParameterTouched
 - AAX_CEffectParameters, [479](#)
- IsSupported
 - AAX_VPageTable, [1058](#)
- IsTransferComplete
 - AAX_IDma, [857](#)
- IsTransportPlaying
 - AAX_IACFTransport, [783](#)
 - AAX_ITransport, [951](#)
 - AAX_VTransport, [1073](#)
- IsUnaccentedClick
 - AAX, [378](#)
- k32BitAbsMax
 - AAX_CommonConversions.h, [1144](#)
- k32BitNegMax
 - AAX_CommonConversions.h, [1144](#)
- k32BitPosMax
 - AAX_CommonConversions.h, [1144](#)
- k56kFloatNegMax
 - AAX_CommonConversions.h, [1145](#)
- k56kFloatPosMax
 - AAX_CommonConversions.h, [1145](#)
- k56kFracAbsMax
 - AAX_CommonConversions.h, [1144](#)
- k56kFracHalf
 - AAX_CommonConversions.h, [1145](#)
- k56kFracNegMax
 - AAX_CommonConversions.h, [1145](#)
- k56kFracNegOne
 - AAX_CommonConversions.h, [1145](#)
- k56kFracPosMax
 - AAX_CommonConversions.h, [1144](#)
- k56kFracZero
 - AAX_CommonConversions.h, [1145](#)
- kAAX_DataBufferType_TempoBreakpointArray
 - AAX_SessionDocumentTypes.h, [1285](#)
- kAAX_eTargetPlatform_Count
 - AAX_Enums.h, [1198](#)
- kAAX_eTargetPlatform_External
 - AAX_Enums.h, [1198](#)
- kAAX_eTargetPlatform_Native
 - AAX_Enums.h, [1198](#)
- kAAX_eTargetPlatform_None
 - AAX_Enums.h, [1198](#)
- kAAX_eTargetPlatform_TI
 - AAX_Enums.h, [1198](#)
- kAAX_ParameterIdentifierMaxSize
 - AAX.h, [1117](#)
- kAAX_ProcPtrID_Create_EffectDirectData
 - AAX_Callbacks.h, [1131](#)
- kAAX_ProcPtrID_Create_EffectGUI
 - AAX_Callbacks.h, [1131](#)

- kAAX_ProcPtrID_Create_EffectParameters
 - AAX_Callbacks.h, [1131](#)
- kAAX_ProcPtrID_Create_HostProcessor
 - AAX_Callbacks.h, [1131](#)
- kAAX_ProcPtrID_Create_SessionDocumentClient
 - AAX_Callbacks.h, [1131](#)
- kAAX_ProcPtrID_Create_TaskAgent
 - AAX_Callbacks.h, [1131](#)
- kAAX_Trace_Priority_Critical
 - AAX_Assert.h, [1121](#)
- kAAX_Trace_Priority_High
 - AAX_Assert.h, [1121](#)
- kAAX_Trace_Priority_Low
 - AAX_Assert.h, [1121](#)
- kAAX_Trace_Priority_Lowest
 - AAX_Assert.h, [1121](#)
- kAAX_Trace_Priority_None
 - AAX_Assert.h, [1120](#)
- kAAX_Trace_Priority_Normal
 - AAX_Assert.h, [1121](#)
- kInvalidIndex
 - AAX_CString, [640](#)
- kMaxAdditionalMIDINodes
 - AAX_CMonolithicParameters.h, [1140](#)
- kMaxAuxOutputStems
 - AAX_CMonolithicParameters.h, [1140](#)
- kMaxStringLength
 - AAX_CString, [641](#)
- kNeg144DB
 - AAX_CommonConversions.h, [1146](#)
- kNeg144Gain
 - AAX_CommonConversions.h, [1146](#)
- Known Issues, [297](#)
- kOneOver56kFracAbsMax
 - AAX_CommonConversions.h, [1145](#)
- kPowExtent
 - AAX, [397](#)
- kPowTableSize
 - AAX, [397](#)
- kSynchronizedParameterQueueSize
 - AAX_CMonolithicParameters.h, [1140](#)
- LastError
 - AAX_CheckedResult, [485](#)
- LastFailure
 - AAX_AggregateResult, [406](#)
- left
 - AAX_Rect, [968](#)
- Length
 - AAX_CString, [631](#)
 - AAX_IMIDIMessageInfoDelegate, [885](#)
 - AAX_IString, [938](#)
- Line
 - AAX::Exception::Any, [1091](#)
- Linked parameter update sequences, [136](#)
- Linked parameters, [131](#)
- LoadChunk
 - AAX_CChunkDataParser, [429](#)
- Lock
 - AAX_CMutex, [517](#)
- LogDoubleToLongControl
 - AAX_SliderConversions.h, [1288](#)
- LONG_STRING_IDENTIFIER
 - AAX_ChunkDataParserDefs, [399](#)
- LONG_TYPE
 - AAX_ChunkDataParserDefs, [399](#)
- LongControlToDouble
 - AAX_SliderConversions.h, [1287](#)
- LongControlToDoubleNonlinear
 - AAX_SliderConversions.h, [1287](#)
- LongControlToLogDouble
 - AAX_SliderConversions.h, [1288](#)
- LongControlToNewRange
 - AAX_SliderConversions.h, [1286](#)
- LongToDouble
 - AAX_CommonConversions.h, [1142](#)
- LongToLongControl
 - AAX_SliderConversions.h, [1287](#)
- mAdditionalInputMIDINodes
 - AAX_SInstrumentRenderInfo, [973](#)
- MapParameterID
 - AAX_IACFPPageTable, [757](#)
 - AAX_IPageTable, [894](#)
 - AAX_VPageTable, [1052](#)
- Mask
 - AAX_IMIDIMessageInfoDelegate, [884](#)
- mAudioInputs
 - AAX_SHybridRenderInfo, [969](#)
 - AAX_SInstrumentRenderInfo, [972](#)
- mAudioOutputs
 - AAX_SHybridRenderInfo, [969](#)
 - AAX_SInstrumentRenderInfo, [972](#)
- mAudiosuiteID
 - AAX_SInstrumentSetupInfo, [981](#)
- mAutomatable
 - AAX_CParameter< T >, [565](#)
- mAutomationDelegate
 - AAX_CParameter< T >, [565](#)
 - AAX_CParameterManager, [572](#)
 - AAX_CStatelessParameter, [625](#)
- mAuxOutputStemFormats
 - AAX_SInstrumentSetupInfo, [979](#)
- mAuxOutputStemNames
 - AAX_SInstrumentSetupInfo, [978](#)
- Max
 - AAX, [389](#)
- MAX_NAME_LENGTH
 - AAX_ChunkDataParserDefs, [401](#)
- MAX_STRINGDATA_LENGTH
 - AAX_ChunkDataParserDefs, [400](#)
- MaxLength
 - AAX_CString, [631](#)
 - AAX_IString, [938](#)
- mBuffer
 - AAX_CMidiStream, [509](#)
- mBufferSize
 - AAX_CMidiStream, [509](#)

- mCanBypass
 - AAX_SInstrumentSetupInfo, 980
- mChunkData
 - AAX_CChunkDataParser, 430
- mChunkParser
 - AAX_CEffectParameters, 481
- mChunkSize
 - AAX_CEffectParameters, 481
- mChunkVersion
 - AAX_CChunkDataParser, 430
- mClock
 - AAX_SHybridRenderInfo, 970
 - AAX_SInstrumentRenderInfo, 972
- mControlType
 - AAX_CParameter< T >, 565
- mCurrentStateNum
 - AAX_SInstrumentRenderInfo, 973
- mData
 - AAX_CMidiPacket, 508
- mDataName
 - AAX_CChunkDataParser::DataValue, 1092
- mDataType
 - AAX_CChunkDataParser::DataValue, 1092
- mDataValues
 - AAX_CChunkDataParser, 430
- mDefaultValue
 - AAX_CParameter< T >, 566
- mDisplayDelegate
 - AAX_CParameter< T >, 565
- Media Composer Guide, 171
- mFilteredParameters
 - AAX_CEffectParameters, 481
- mGlobalMIDIEventMask
 - AAX_SInstrumentSetupInfo, 976
- mGlobalMIDI nodeName
 - AAX_SInstrumentSetupInfo, 976
- mGlobalNode
 - AAX_SInstrumentRenderInfo, 972
- mHybridInputStemFormat
 - AAX_SInstrumentSetupInfo, 979
- mHybridOutputStemFormat
 - AAX_SInstrumentSetupInfo, 979
- mID
 - AAX_CStatelessParameter, 624
- MIDI, 99
- Min
 - AAX, 389
- MinMax
 - AAX, 389
- mInputMIDIChannelMask
 - AAX_SInstrumentSetupInfo, 977
- mInputMIDI nodeName
 - AAX_SInstrumentSetupInfo, 976
- mInputNode
 - AAX_SInstrumentRenderInfo, 972
- mInputStemFormat
 - AAX_SInstrumentSetupInfo, 980
- mIntValue
 - AAX_CChunkDataParser::DataValue, 1092
- mInverseStringMap
 - AAX_CStringDisplayDelegate< T >, 652
- mIsImmediate
 - AAX_CMidiPacket, 508
- mIsLoopEnabled
 - AAX_TransportStateInfo_V1, 990
- mIsRecordEnabled
 - AAX_TransportStateInfo_V1, 990
- mIsRecording
 - AAX_TransportStateInfo_V1, 990
- mLastFoundIndex
 - AAX_CChunkDataParser, 430
- mLength
 - AAX_CMidiPacket, 507
- mManufacturerID
 - AAX_SInstrumentSetupInfo, 981
 - AAX_SPlugInIdentifierTriad, 987
- mMeterIDs
 - AAX_SInstrumentSetupInfo, 978
- mMeters
 - AAX_SInstrumentRenderInfo, 973
- mMonolithicParametersPtr
 - AAX_SInstrumentPrivateData, 971
- mMultiMonoSupport
 - AAX_SInstrumentSetupInfo, 981
- mNames
 - AAX_CParameter< T >, 564
 - AAX_CStatelessParameter, 624
- mNeedNotify
 - AAX_CParameter< T >, 566
- mNeedsGlobalMIDI
 - AAX_SInstrumentSetupInfo, 975
- mNeedsInputMIDI
 - AAX_SInstrumentSetupInfo, 976
- mNeedsTransport
 - AAX_SInstrumentSetupInfo, 977
- mNumAdditionalInputMIDI nodes
 - AAX_SInstrumentSetupInfo, 977
- mNumAudioInputs
 - AAX_SHybridRenderInfo, 969
- mNumAudioOutputs
 - AAX_SHybridRenderInfo, 970
- mNumAuxOutputStems
 - AAX_SInstrumentSetupInfo, 978
- mNumChunkedParameters
 - AAX_CEffectParameters, 481
- mNumMeters
 - AAX_SInstrumentSetupInfo, 978
- mNumPlugInChanges
 - AAX_CEffectParameters, 480
- mNumSamples
 - AAX_SHybridRenderInfo, 970
 - AAX_SInstrumentRenderInfo, 972
- mNumSteps
 - AAX_CParameter< T >, 565
- Monolithic parameters, 112
- Monolithic VIs and Effects, 280

- mOrientation
 - AAX_CParameter< T >, 565
- mOutputStemFormat
 - AAX_SInstrumentSetupInfo, 980
- mPacketDispatcher
 - AAX_CEffectParameters, 481
- mParameterManager
 - AAX_CEffectParameters, 481
- mParameters
 - AAX_CParameterManager, 572
- mParametersMap
 - AAX_CParameterManager, 572
- mPlugInID
 - AAX_SPlugInIdentifierTriad, 988
- mPluginID
 - AAX_SInstrumentSetupInfo, 981
- mPrivateData
 - AAX_SInstrumentRenderInfo, 973
- mProductID
 - AAX_SInstrumentSetupInfo, 981
 - AAX_SPlugInIdentifierTriad, 987
- mRecordMode
 - AAX_TransportStateInfo_V1, 990
- mSampleLocation
 - AAX_CTempoBreakpoint, 659
- mString
 - AAX_CString, 641
- mStringMap
 - AAX_CStringDisplayDelegate< T >, 652
- mStringValue
 - AAX_CChunkDataParser::DataValue, 1092
- mTaperDelegate
 - AAX_CParameter< T >, 565
- mTimestamp
 - AAX_CMidiPacket, 507
- mTransportMIDINodeName
 - AAX_SInstrumentSetupInfo, 977
- mTransportNode
 - AAX_SInstrumentRenderInfo, 973
- mTransportState
 - AAX_TransportStateInfo_V1, 990
- mUnitString
 - AAX_CUnitDisplayDelegateDecorator< T >, 663
- mUseHostGeneratedGUI
 - AAX_SInstrumentSetupInfo, 980
- mValue
 - AAX_CParameter< T >, 566
 - AAX_CTempoBreakpoint, 659
- mValueString
 - AAX_CStatelessParameter, 625
- Name
 - AAX_CParameter< T >, 539
 - AAX_CStatelessParameter, 606
 - AAX_IParameter, 904
- NAME_NOT_FOUND
 - AAX_ChunkDataParserDefs, 401
- NewComponentDescriptor
 - AAX_IEffectDescriptor, 865
- AAX_VEffectDescriptor, 1034
- NewDescriptor
 - AAX_ICollection, 804
 - AAX_VCollection, 996
- NewPropertyMap
 - AAX_ICollection, 806
 - AAX_IComponentDescriptor, 818
 - AAX_IEffectDescriptor, 867
 - AAX_VCollection, 998
 - AAX_VComponentDescriptor, 1010
 - AAX_VEffectDescriptor, 1037
- None
 - Task agent interface, 106
- NormalizedToInt32
 - AAX_CEffectParameters.h, 1136
- NormalizedToReal
 - AAX_CBinaryTaperDelegate< T >, 422
 - AAX_CLinearTaperDelegate< T, RealPrecision >, 502
 - AAX_CLogTaperDelegate< T, RealPrecision >, 506
 - AAX_CPieceWiseLinearTaperDelegate< T, RealPrecision >, 587
 - AAX_CRangeTaperDelegate< T, RealPrecision >, 592
 - AAX_CStateTaperDelegate< T >, 628
 - AAX_ITaperDelegate< T >, 941
- NotificationReceived
 - AAX_CEffectDirectData, 436
 - AAX_CEffectGUI, 440
 - AAX_CEffectParameters, 454
 - AAX_CSessionDocumentClient, 595
 - AAX_IACFEEffectDirectData_V2, 706
 - AAX_IACFEEffectGUI, 709
 - AAX_IACFEEffectParameters, 717
 - AAX_IACFSessionDocumentClient, 776
- NumAttempted
 - AAX_AggregateResult, 406
- NumFailed
 - AAX_AggregateResult, 406
- NumParameters
 - AAX_CParameterManager, 568
- NumSucceeded
 - AAX_AggregateResult, 406
- Offline processing interface, 102
- operator AAX_Result
 - AAX_AggregateResult, 405
 - AAX_CheckedResult, 485
- operator!=
 - AAX_CString, 638, 639
 - AAX_GUITypes.h, 1222, 1223
 - AAX_TransportTypes.h, 1291
- operator<
 - AAX_CString, 639
 - AAX_GUITypes.h, 1222
- operator<<
 - AAX_CString, 640
- operator<=

- AAX_GUITypes.h, [1222](#)
- operator>
 - AAX_CString, [639](#)
 - AAX_GUITypes.h, [1222](#)
- operator>>
 - AAX_CString, [640](#)
- operator>=
 - AAX_GUITypes.h, [1222](#)
- operator+
 - AAX_CString.h, [1153](#)
- operator+=
 - AAX_CString, [639](#), [640](#)
- operator=
 - AAX::Exception::Any, [1090](#)
 - AAX_AggregateResult, [405](#)
 - AAX_CArrayDataBuffer< D >, [408](#)
 - AAX_CArrayDataBufferOfType< T, D >, [411](#)
 - AAX_CEffectParameters, [453](#)
 - AAX_CheckedResult, [485](#)
 - AAX_CRangeTaperDelegate< T, RealPrecision >, [590](#)
 - AAX_CString, [632](#), [633](#)
 - AAX_CStringDataBuffer, [644](#)
 - AAX_CStringDataBufferOfType< T >, [647](#)
 - AAX_IString, [939](#)
- operator==
 - AAX_CString, [638](#)
 - AAX_GUITypes.h, [1222](#), [1223](#)
 - AAX_TransportTypes.h, [1291](#)
- operator[]
 - AAX_CString, [639](#)
- operator |=
 - AAX_CheckedResult, [485](#)
- Other Extensions, [280](#)
 - AsStringMIDIStream_Debug, [280](#)
 - GetPathToPlugInBundle, [281](#)
- override
 - AAX_IEffectDirectData, [870](#)
 - AAX_IEffectGUI, [871](#)
 - AAX_IEffectParameters, [874](#)
 - AAX_IHostProcessor, [877](#)
 - AAX_ISessionDocumentClient, [937](#)
- Page Table Guide, [225](#)
- PageTableParameterMappingsAreEqual
 - AAX, [379](#)
- PageTableParameterNameVariationsAreEqual
 - AAX, [379](#)
- PageTablesAreEqual
 - AAX, [379](#)
- Parameter automation, [114](#)
- Parameter Manager, [106](#)
- Parameter update timing, [116](#)
- Parameter updates, [116](#)
- ParameterNameChanged
 - AAX_IAutomationDelegate, [802](#)
 - AAX_VAutomationDelegate, [994](#)
- ParameterUpdated
 - AAX_CEffectGUI, [443](#)
 - AAX_IACFEffEffectGUI, [711](#)
- Peek
 - AAX_CAtomicQueue< T, S >, [415](#)
 - AAX_IPointerQueue< T >, [927](#)
- Plug-in meters, [97](#)
- Plug-in type conversion, [140](#)
- PolyEval
 - AAX, [389](#)
- Pop
 - AAX_CAtomicQueue< T, S >, [415](#)
 - AAX_IPointerQueue< T >, [927](#)
- PostAnalyze
 - AAX_CHostProcessor, [492](#)
 - AAX_IACFHostProcessor, [746](#)
- PostCurrentValue
 - AAX_IACFAutomationDelegate, [671](#)
 - AAX_IAutomationDelegate, [801](#)
 - AAX_VAutomationDelegate, [993](#)
- PostMIDIPacket
 - AAX_IMIDINode, [887](#)
- PostPacket
 - AAX_IACFController, [691](#)
 - AAX_IController, [831](#)
 - AAX_VController, [1022](#)
- PostReleaseRequest
 - AAX_IACFAutomationDelegate, [671](#)
 - AAX_IAutomationDelegate, [801](#)
 - AAX_VAutomationDelegate, [994](#)
- PostRender
 - AAX_CHostProcessor, [491](#)
 - AAX_IACFHostProcessor, [744](#)
- PostRequest
 - AAX_IDma, [856](#)
- PostSetValueRequest
 - AAX_IACFAutomationDelegate, [670](#)
 - AAX_IAutomationDelegate, [800](#)
 - AAX_VAutomationDelegate, [993](#)
- PostTouchRequest
 - AAX_IACFAutomationDelegate, [671](#)
 - AAX_IAutomationDelegate, [801](#)
 - AAX_VAutomationDelegate, [993](#)
- PreAnalyze
 - AAX_CHostProcessor, [492](#)
 - AAX_IACFHostProcessor, [745](#)
- PreRender
 - AAX_CHostProcessor, [491](#)
 - AAX_IACFHostProcessor, [744](#)
- Primary
 - AAX_CStringAbbreviations, [642](#)
- Pro Tools Guide, [150](#)
- Properties File, [102](#)
- pt2Object
 - AAX_CPacketHandler< TWorker >, [529](#)
- Push
 - AAX_CAtomicQueue< T, S >, [414](#)
 - AAX_IPointerQueue< T >, [926](#)
- QueryInterface
 - IACFUnknown, [1095](#)

- ReadMe.doxygen, [1322](#)
- ReadPortDirect
 - AAX_IACFPrivateDataAccess, [768](#)
 - AAX_IPrivateDataAccess, [928](#)
 - AAX_VPrivateDataAccess, [1059](#)
- Real-time algorithm callback, [68](#)
- Real-time performance, [112](#)
- RealToNormalized
 - AAX_CBinaryTaperDelegate< T >, [423](#)
 - AAX_CLinearTaperDelegate< T, RealPrecision >, [502](#)
 - AAX_CLogTaperDelegate< T, RealPrecision >, [506](#)
 - AAX_CPieceWiseLinearTaperDelegate< T, RealPrecision >, [588](#)
 - AAX_CRangeTaperDelegate< T, RealPrecision >, [592](#)
 - AAX_CStateTaperDelegate< T >, [628](#)
 - AAX_ITaperDelegate< T >, [942](#)
- ReceiveTask
 - AAX_CTaskAgent, [658](#)
- RegisterPacket
 - AAX_CPacketDispatcher, [526](#)
- RegisterParameter
 - AAX_IACFAutomationDelegate, [669](#)
 - AAX_IAutomationDelegate, [799](#)
 - AAX_VAutomationDelegate, [992](#)
- Release
 - AAX_CParameter< T >, [552](#)
 - AAX_CStatelessParameter, [608](#)
 - AAX_IParameter, [905](#)
 - IACFUnknown, [1096](#)
- ReleaseParameter
 - AAX_CEffectParameters, [465](#)
 - AAX_IACFEffEffectParameters, [728](#)
- RemoveAllParameters
 - AAX_CParameterManager, [568](#)
- RemovePage
 - AAX_IACFPageTable, [755](#)
 - AAX_IPageTable, [892](#)
 - AAX_VPageTable, [1050](#)
- RemoveParameter
 - AAX_CParameterManager, [571](#)
- RemoveParameterByID
 - AAX_CParameterManager, [568](#)
- RemoveProperty
 - AAX_IACFPropertyMap, [771](#)
 - AAX_IPropertyMap, [933](#)
 - AAX_VPropertyMap, [1063](#)
- RenderAudio
 - AAX_CHostProcessor, [490](#)
 - AAX_CMonolithicParameters, [512](#)
 - AAX_IACFHostProcessor, [743](#)
- RenderAudio_Hybrid
 - AAX_CEffectParameters, [477](#)
 - Hybrid Processing architecture, [96](#)
- Replace
 - AAX_CString, [635](#), [636](#)
- ReplaceDouble
 - AAX_CChunkDataParser, [428](#)
- RequestTransportStart
 - AAX_IACFTransportControl, [792](#)
 - AAX_ITransport, [956](#)
 - AAX_VTransport, [1081](#)
- RequestTransportStop
 - AAX_IACFTransportControl, [792](#)
 - AAX_ITransport, [956](#)
 - AAX_VTransport, [1081](#)
- Reset
 - SAutoArray< T >, [1099](#)
- ResetAcceptedResults
 - AAX_CheckedResult, [484](#)
- ResetFieldData
 - AAX_CEffectParameters, [467](#)
 - AAX_CMonolithicParameters, [514](#)
 - AAX_IACFEffEffectParameters, [730](#)
- Result
 - AAX::Exception::ResultError, [1098](#)
- ResultError
 - AAX::Exception::ResultError, [1097](#)
- Round
 - AAX_CLinearTaperDelegate< T, RealPrecision >, [502](#)
 - AAX_CLogTaperDelegate< T, RealPrecision >, [507](#)
 - AAX_CPieceWiseLinearTaperDelegate< T, RealPrecision >, [588](#)
 - AAX_CRangeTaperDelegate< T, RealPrecision >, [592](#)
- SafeLog
 - AAX, [384](#)
- SafeLogf
 - AAX, [384](#)
- sampleRateInMask
 - AAX_Enums.h, [1204](#)
- SAutoArray
 - SAutoArray< T >, [1098](#)
- SAutoArray< T >, [1098](#)
 - ~SAutoArray, [1098](#)
 - Get, [1099](#)
 - Reset, [1099](#)
 - SAutoArray, [1098](#)
- SendNotification
 - AAX_IACFController_V2, [694](#)
 - AAX_IController, [832](#)
 - AAX_VController, [1023](#), [1024](#)
- SessionDocumentChanged
 - AAX_CSessionDocumentClient, [596](#)
- SessionDocumentWillChange
 - AAX_CSessionDocumentClient, [596](#)
- Set
 - AAX_CHostServices, [498](#)
 - AAX_CParameterValue< T >, [576](#)
 - AAX_CString, [632](#)
 - AAX_IString, [939](#)
- SetAutomationDelegate

- AAX_CParameter< T >, 551
- AAX_CStatelessParameter, 608
- AAX_IPParameter, 905
- SetBaseOffset
 - AAX_IDma, 863
- SetBurstLength
 - AAX_IDma, 859
- SetChunk
 - AAX_CEffectParameters, 470
 - AAX_IACFEEffectParameters, 733
- SetCoefficients
 - AAX_Map, 964
- SetControlHighlightInfo
 - AAX_CEffectGUI, 444
 - AAX_IACFEEffectGUI, 712
- SetCustomData
 - AAX_CEffectParameters, 475
 - AAX_IACFEEffectParameters, 734
- SetCycleCount
 - AAX_IACFController, 690
 - AAX_IController, 830
 - AAX_VController, 1022
- SetDefaultValue
 - AAX_CParameter< T >, 558
- SetDirty
 - AAX_CPacket, 524
 - AAX_CPacketDispatcher, 526
- SetDisplayDelegate
 - AAX_CEffectParameters, 478
 - AAX_CParameter< T >, 544
 - AAX_CStatelessParameter, 624
 - AAX_IPParameter, 920
- SetDmaState
 - AAX_IDma, 857
- SetDone
 - AAX_CTask, 655
 - AAX_IACFTask, 779
 - AAX_ITask, 946
 - AAX_VTask, 1071
- SetDst
 - AAX_IDma, 858
- SetFifoBuffer
 - AAX_IDma, 860
- SetFifoSize
 - AAX_IDma, 863
- SetLinearBuffer
 - AAX_IDma, 861
- SetLocation
 - AAX_CHostProcessor, 490
 - AAX_IACFHostProcessor, 743
- SetManufacturerName
 - AAX_IACFCollection, 673
 - AAX_ICollection, 804
 - AAX_VCollection, 997
- SetName
 - AAX_CParameter< T >, 538
 - AAX_CStatelessParameter, 606
 - AAX_IPParameter, 903
- SetNormalizedDefaultValue
 - AAX_CParameter< T >, 540
 - AAX_CStatelessParameter, 609
 - AAX_IPParameter, 906
- SetNormalizedValue
 - AAX_CParameter< T >, 540
 - AAX_CStatelessParameter, 609
 - AAX_IPParameter, 906
- SetNumberOfSteps
 - AAX_CParameter< T >, 541
 - AAX_CStatelessParameter, 610
 - AAX_IPParameter, 907
- SetNumBursts
 - AAX_IDma, 859
- SetNumOffsets
 - AAX_IDma, 862
- SetOffsetTable
 - AAX_IDma, 861
- SetOrientation
 - AAX_CParameter< T >, 543
 - AAX_CStatelessParameter, 623
 - AAX_IPParameter, 919
- SetPackageVersion
 - AAX_IACFCollection, 674
 - AAX_ICollection, 805
 - AAX_VCollection, 997
- SetParameterDefaultNormalizedValue
 - AAX_CEffectParameters, 457
 - AAX_IACFEEffectParameters, 721
- SetParameterNameVariation
 - AAX_IACFPageTable_V2, 763
 - AAX_IPageTable, 899
 - AAX_VPageTable, 1057
- SetParameterNormalizedRelative
 - AAX_CEffectParameters, 464
 - AAX_IACFEEffectParameters, 726
- SetParameterNormalizedValue
 - AAX_CEffectParameters, 463
 - AAX_IACFEEffectParameters, 726
- SetParameters
 - AAX_FastInterpolatedTableLookup< TFLOAT, DFLOAT >, 667
- SetPrimary
 - AAX_CStringAbbreviations, 642
- SetProgress
 - AAX_CTask, 654
 - AAX_IACFTask, 778
 - AAX_ITask, 945
 - AAX_VTask, 1070
- SetProperties
 - AAX_IACFCollection, 674
 - AAX_IACFEEffectDescriptor, 702
 - AAX_ICollection, 806
 - AAX_IEffectDescriptor, 867
 - AAX_VCollection, 998
 - AAX_VEffectDescriptor, 1037
- SetSessionDocument
 - AAX_CSessionDocumentClient, 595

- AAX_IACFSessionDocumentClient, 776
- SetSignalLatency
 - AAX_IACFController, 690
 - AAX_IController, 829
 - AAX_VController, 1021
- SetSrc
 - AAX_IDma, 858
- SetStepValue
 - AAX_CParameter< T >, 542
 - AAX_CStatelessParameter, 611
 - AAX_IParameter, 908
- SetTaperDelegate
 - AAX_CEffectParameters, 478
 - AAX_CParameter< T >, 544
 - AAX_CStatelessParameter, 623
 - AAX_IParameter, 920
- SetToDefaultValue
 - AAX_CParameter< T >, 540
 - AAX_CStatelessParameter, 610
 - AAX_IParameter, 907
- SetTransferSize
 - AAX_IDma, 860
- SetType
 - AAX_CParameter< T >, 543
 - AAX_CStatelessParameter, 622
 - AAX_IParameter, 919
- SetValue
 - AAX_CParameter< T >, 557
- SetValueFromString
 - AAX_CParameter< T >, 551
 - AAX_CStatelessParameter, 617
 - AAX_IParameter, 914
- SetValueWithBool
 - AAX_CParameter< T >, 555, 559
 - AAX_CStatelessParameter, 620
 - AAX_IParameter, 917
- SetValueWithDouble
 - AAX_CParameter< T >, 556, 560
 - AAX_CStatelessParameter, 621
 - AAX_IParameter, 918
- SetValueWithFloat
 - AAX_CParameter< T >, 555, 560
 - AAX_CStatelessParameter, 621
 - AAX_IParameter, 918
- SetValueWithInt32
 - AAX_CParameter< T >, 555, 559
 - AAX_CStatelessParameter, 620
 - AAX_IParameter, 917
- SetValueWithString
 - AAX_CParameter< T >, 556, 560
 - AAX_CStatelessParameter, 622
 - AAX_IParameter, 918
- SetViewContainer
 - AAX_CEffectGUI, 441
 - AAX_IACFEffectGUI, 710
- SetViewSize
 - AAX_IACFViewContainer, 794
 - AAX_IViewContainer, 960
- AAX_VViewContainer, 1084
- SHORT_STRING_IDENTIFIER
 - AAX_ChunkDataParserDefs, 400
- SHORT_TYPE
 - AAX_ChunkDataParserDefs, 400
- SHORT_TYPE_INCR
 - AAX_ChunkDataParserDefs, 400
- SHORT_TYPE_SIZE
 - AAX_ChunkDataParserDefs, 400
- ShortenedName
 - AAX_CParameter< T >, 539
 - AAX_CStatelessParameter, 607
 - AAX_IParameter, 904
- Sidechain Inputs, 103
- Sign
 - AAX, 389
- SinCosMix
 - AAX, 390
- Size
 - AAX_CArrayDataBuffer< D >, 409
 - AAX_CArrayDataBufferOfType< T, D >, 412
 - AAX_CStringDataBuffer, 645
 - AAX_CStringDataBufferOfType< T >, 648
 - AAX_IACFDataBuffer, 698
 - AAX_IDataBufferWrapper, 842
 - AAX_ISessionDocument::TempoMap, 1100
 - AAX_VDataBufferWrapper, 1030
 - AAX_VSessionDocument::VTempoMap, 1101
- SmartRound
 - AAX_CRangeTaperDelegate< T, RealPrecision >, 593
- StackTrace
 - AAX_CHostServices, 499
 - AAX_IACFHostServices_V2, 752
 - AAX_IHostServices, 882
 - AAX_VHostServices, 1045
- StaticDescribe
 - AAX_CMonolithicParameters, 515
- StaticRenderAudio
 - AAX_CMonolithicParameters, 516
- Status
 - AAX_CTask, 655
- StdString
 - AAX_CString, 633
- String2Binary
 - AAX, 381
- STRING_IDENTIFIER_SIZE
 - AAX_ChunkDataParserDefs, 401
- STRING_STRING_IDENTIFIER
 - AAX_ChunkDataParserDefs, 400
- STRING_TYPE
 - AAX_ChunkDataParserDefs, 400
- StringToValue
 - AAX_CBinaryDisplayDelegate< T >, 419
 - AAX_CDecibelDisplayDelegateDecorator< T >, 433
 - AAX_CNumberDisplayDelegate< T, Precision, SpaceAfter >, 520

- AAX_CPercentDisplayDelegateDecorator< T >, 583
- AAX_CStateDisplayDelegate< T >, 601
- AAX_CStringDisplayDelegate< T >, 651
- AAX_CUnitDisplayDelegateDecorator< T >, 662
- AAX_CUnitPrefixDisplayDelegateDecorator< T >, 666
- AAX_IDisplayDelegate< T >, 848
- AAX_IDisplayDelegateDecorator< T >, 853
- SubString
 - AAX_CString, 637
- Supplemental Information, 281
- Supported
 - AAX_VDescriptionHost, 1032
- SupportLevel
 - AAX_IACFFeatureInfo, 740
 - AAX_IFeatureInfo, 875
 - AAX_VFeatureInfo, 1040
- Taper delegates, 109
- TaperDelegate
 - AAX_CParameter< T >, 558
- Task agent interface, 104
 - AAX_TaskCompletionStatus, 105
 - Canceled, 106
 - Done, 106
 - Error, 106
 - None, 106
- template_size
 - AAX_CAtomicQueue< T, S >, 416
- template_type
 - AAX_CAtomicQueue< T, S >, 413
 - AAX_IPointerQueue< T >, 926
- The Avid Component Framework (ACF), 144
- ThirtyTwoBitDSPCoefToDouble
 - AAX_CommonConversions.h, 1143
- TI_VERSION
 - AAX.h, 1106
- TimerWakeup
 - AAX_CEffectDirectData, 436
 - AAX_CEffectGUI, 442
 - AAX_CEffectParameters, 471
 - AAX_CMonolithicParameters, 514
 - AAX_IACFEffEffectDirectData, 705
 - AAX_IACFEffEffectGUI, 711
 - AAX_IACFEffEffectParameters, 733
- TimerWakeup_PrivateDataAccess
 - AAX_CEffectDirectData, 437
- ToDouble
 - AAX_CString, 637
- ToHexadecimal
 - AAX::internal, 398
- ToInteger
 - AAX_CString, 637
- Token protocol, 123
- top
 - AAX_Rect, 968
- ToString
 - AAX_IMIDIMessageInfoDelegate, 885
 - AAX_TransportStateInfo_V1, 989
- ToString_AppendByteRange
 - AAX_IMIDIMessageInfoDelegate, 886
- ToString_AppendCStr
 - AAX_IMIDIMessageInfoDelegate, 886
- ToString_AppendNumber
 - AAX_IMIDIMessageInfoDelegate, 885
- ToString_AppendValid
 - AAX_IMIDIMessageInfoDelegate, 886
- Touch
 - AAX_CParameter< T >, 552
 - AAX_CStatelessParameter, 608
 - AAX_IPParameter, 905
- TouchParameter
 - AAX_CEffectParameters, 464
 - AAX_IACFEffEffectParameters, 727
- TParamValPair
 - AAX_CMonolithicParameters, 511
- Trace
 - AAX_CHostServices, 498
 - AAX_IACFHostServices, 751
 - AAX_IHostServices, 881
 - AAX_VHostServices, 1044
- TranslateOutputBounds
 - AAX_CHostProcessor, 495
- Transport
 - AAX_CEffectGUI, 446
 - AAX_CEffectParameters, 478
- Troubleshooting, 282
- Try_Lock
 - AAX_CMutex, 517
- Type
 - AAX_CArrayDataBuffer< D >, 408
 - AAX_CArrayDataBufferOfType< T, D >, 411
 - AAX_CParameter< T >, 534
 - AAX_CStringDataBuffer, 644
 - AAX_CStringDataBufferOfType< T >, 647
 - AAX_IACFDataBuffer, 698
 - AAX_IDataBufferWrapper, 842
 - AAX_VDataBufferWrapper, 1030
- Uninitialize
 - AAX_CEffectDirectData, 435
 - AAX_CEffectGUI, 440
 - AAX_CEffectParameters, 454
 - AAX_CHostProcessor, 488
 - AAX_CSessionDocumentClient, 595
 - AAX_CTaskAgent, 658
 - AAX_IACFEffEffectDirectData, 705
 - AAX_IACFEffEffectGUI, 709
 - AAX_IACFEffEffectParameters, 717
 - AAX_IACFHostProcessor, 742
 - AAX_IACFSessionDocumentClient, 775
 - AAX_IACFTaskAgent, 781
 - AAX_IHostTaskAgent, 883
 - AAX_VHostTaskAgent, 1046
- Unlock
 - AAX_CMutex, 517
- UnregisterParameter

- AAX_IACFAutomationDelegate, 670
 - AAX_IAutomationDelegate, 800
 - AAX_VAutomationDelegate, 992
- UpdateAllParameters
 - AAX_CEffectGUI, 445
- UpdateControlMIDINodes
 - AAX_CEffectParameters, 476
 - AAX_IACFEffEffectParameters_V2, 736
- UpdateMIDINodes
 - AAX_CEffectParameters, 476
 - AAX_IACFEffEffectParameters_V2, 736
- UpdateNormalizedValue
 - AAX_CParameter< T >, 557
 - AAX_CStatelessParameter, 624
 - AAX_IParameter, 921
- UpdatePageTable
 - AAX_CEffectParameters, 474, 479
 - AAX_IACFEffEffectParameters_V4, 738
- UpdateParameterNormalizedRelative
 - AAX_CEffectParameters, 466
 - AAX_IACFEffEffectParameters, 729
- UpdateParameterNormalizedValue
 - AAX_CEffectParameters, 466
 - AAX_CMonolithicParameters, 513
 - AAX_IACFEffEffectParameters, 729
- UpdateParameterTouch
 - AAX_CEffectParameters, 465
 - AAX_IACFEffEffectParameters, 728
- Valid
 - AAX_ISessionDocument, 935
 - AAX_VSessionDocument, 1066
- value_type
 - AAX_CAtomicQueue< T, S >, 414
 - AAX_IPointerQueue< T >, 926
- ValueToString
 - AAX_CBinaryDisplayDelegate< T >, 418, 419
 - AAX_CDecibelDisplayDelegateDecorator< T >, 432
 - AAX_CNumberDisplayDelegate< T, Precision, SpaceAfter >, 518, 520
 - AAX_CPercentDisplayDelegateDecorator< T >, 582, 583
 - AAX_CStateDisplayDelegate< T >, 600
 - AAX_CStringDisplayDelegate< T >, 650, 651
 - AAX_CUnitDisplayDelegateDecorator< T >, 661, 662
 - AAX_CUnitPrefixDisplayDelegateDecorator< T >, 665, 666
 - AAX_IDisplayDelegate< T >, 846, 847
 - AAX_IDisplayDelegateDecorator< T >, 851, 852
- VENUE Guide, 354
- VERSION_ID_1
 - AAX_ChunkDataParserDefs, 401
- vert
 - AAX_Point, 967
- VTempoMap
 - AAX_VSessionDocument::VTempoMap, 1101
- What
 - AAX::Exception::Any, 1090
- width
 - AAX_Rect, 968
- WordAlign
 - AAX_CChunkDataParser, 429
- WritePortDirect
 - AAX_IACFPrivateDataAccess, 769
 - AAX_IPrivateDataAccess, 929
 - AAX_VPrivateDataAccess, 1059
- ZeroMemoryDW
 - AAX, 387
- ZeroMemorySW
 - AAX, 387